Linux

K19G

2026-11-09

Table of contents

Introduction **146**

Linux **147**
Commands — scope . . . 147
Related books . . . 148
How to use . . . 148
Commands page template . . . 148
Commands — parts at a glance . . . 149
Editors (this book) . . . 149
Scripting → Bash (this book) . . . 150
Conventions . . . 150
Formats . . . 150

Commands **151**

Help and Docs **152**

Intro **153**

Intro **154**
Commands in this part . . . 154
Suggested starting points . . . 154
Related parts . . . 154

man **155**
Overview . . . 155
Syntax . . . 155
Sections (common) . . . 155
Common Options . . . 156
Examples with Explanations . . . 156
Open a page . . . 156
Disambiguate sections . . . 156
Search when you forget the name . . . 156
Locate the source file . . . 157
Readable width and plain text dump . . . 157
HTML / browser (if supported) . . . 157
Scripting existence checks . . . 157
Pager keys (less) . . . 157
Notes / Pitfalls . . . 158

2026-relevant notes . . . 158
Related Commands . . . 158
Additional Resources . . . 158

help (shell builtin) 159
Overview . . . 159
Syntax . . . 159
Common Options / discovery helpers . . . 159
Examples with Explanations . . . 160
Read builtin docs . . . 160
Builtin vs external . . . 160
List builtins (bash) . . . 160
Disable / re-enable a builtin (rare, debugging) . . . 160
Conditional help in interactive setups . . . 161
Job-control and shell language topics . . . 161
Notes / Pitfalls . . . 161
2026-relevant notes . . . 161
Related Commands . . . 162
Additional Resources . . . 162

info 163
Overview . . . 163
Syntax . . . 163
Common Options . . . 163
Navigation Keys (standalone info) . . . 163
Examples with Explanations . . . 164
Open a manual . . . 164
Jump to options / a node . . . 164
Search . . . 164
Where is the file? . . . 165
Non-interactive extract . . . 165
Prefer man for quick flags, info for depth . . . 165
Understanding the Info tree . . . 165
Notes / Pitfalls . . . 165
2026-relevant notes . . . 166
Related Commands . . . 166
Additional Resources . . . 166

apropos 167
Overview . . . 167
Syntax . . . 167
Common Options . . . 167
Examples with Explanations . . . 168
Basic keyword search . . . 168
AND multiple terms . . . 168
Limit by section . . . 168
Regex and wildcards . . . 168
Reduce noise to section-1 commands . . . 168

Rebuild empty database . . . 169
Pipeline into selection . . . 169
Compare with full-text search . . . 169
Notes / Pitfalls . . . 169
2026-relevant notes . . . 169
Related Commands . . . 170
Additional Resources . . . 170

whatis **171**
Overview . . . 171
Syntax . . . 171
Common Options . . . 171
Examples with Explanations . . . 171
Quick one-liners . . . 171
Disambiguate by section . . . 172
Multiple names in scripts . . . 172
Missing database . . . 172
Combine with type / path . . . 172
Wildcard / regex (when supported) . . . 172
Verify pages after package install . . . 173
Understanding Output . . . 173
Notes / Pitfalls . . . 173
2026-relevant notes . . . 173
Related Commands . . . 173
Additional Resources . . . 174

Files and Paths 175

Intro 176

Intro 177
Commands in this part . . . 177
Suggested starting points . . . 179
Related parts . . . 180

pwd **181**
Overview . . . 181
Syntax . . . 181
Common Options . . . 181
Key Use Cases . . . 181
Examples with Explanations . . . 182
Basics . . . 182
Capture in scripts . . . 182
Symlink illustration . . . 182
Prompt and logging recipes . . . 182
Resolve then act . . . 183
Compare with realpath . . . 183
Directory stack awareness . . . 183

Notes / Pitfalls . . . 183
2026-relevant notes . . . 183
Related Commands . . . 184
Additional Resources . . . 184

cd **185**
Overview . . . 185
Syntax . . . 185
Common Options / operands . . . 185
Key Use Cases . . . 185
Examples with Explanations . . . 186
Basic navigation . . . 186
Previous directory . . . 186
Symlink behavior . . . 186
Paths that start with a dash . . . 186
Script: run relative to script location . . . 186
Script: require a directory . . . 187
pushd / popd alternative workflow . . . 187
One-liners . . . 187
Understanding success and failure . . . 187
Notes / Pitfalls . . . 187
2026-relevant notes . . . 188
Related Commands . . . 188
Additional Resources . . . 188

ls **189**
Overview . . . 189
Syntax . . . 189
Common Options . . . 189
Examples with Explanations . . . 190
Everyday . . . 190
Hidden and directories . . . 190
Sorting and time . . . 190
Recursive (careful) . . . 190
Classification and inodes . . . 190
Globs and multiple paths . . . 191
Scripts: stable output . . . 191
Color and aliases . . . 191
Compare modern alternatives . . . 191
Understanding long format . . . 191
Notes / Pitfalls . . . 191
2026-relevant notes . . . 192
Related Commands . . . 192
Additional Resources . . . 192

tree **193**
Overview . . . 193
Syntax . . . 193

Common Options . . . 193
Examples with Explanations . . . 194
Shallow overview . . . 194
Hide dependency / VCS noise . . . 194
Hidden files and permissions . . . 194
Sizes . . . 194
Full paths and filtering . . . 194
HTML report . . . 194
JSON for tooling (when available) . . . 195
Compare with find / eza . . . 195
Document a project layout . . . 195
Notes / Pitfalls . . . 195
2026-relevant notes . . . 195
Related Commands . . . 195
Additional Resources . . . 196

mkdir **197**
Overview . . . 197
Syntax . . . 197
Common Options . . . 197
Key Use Cases . . . 197
Examples with Explanations . . . 198
Basics . . . 198
Parents . . . 198
Permissions . . . 198
Brace expansion scaffolds . . . 198
Script: ensure writable data dir . . . 198
SELinux (when relevant) . . . 199
Parallel with install . . . 199
Failure cases . . . 199
Notes / Pitfalls . . . 199
2026-relevant notes . . . 199
Related Commands . . . 199
Additional Resources . . . 200

rmdir **201**
Overview . . . 201
Syntax . . . 201
Common Options . . . 201
Key Use Cases . . . 201
Examples with Explanations . . . 202
Basic removal . . . 202
Non-empty fails . . . 202
Remove parent chain . . . 202
Verbose . . . 202
Ignore non-empty (continue batch) . . . 202
Test emptiness in scripts . . . 203
Combine with find (empty dirs only) . . . 203

Brace / multi targets . . . 203
Notes / Pitfalls . . . 203
2026-relevant notes . . . 203
Related Commands . . . 204
Additional Resources . . . 204

touch **205**
Overview . . . 205
Syntax . . . 205
Common Options . . . 205
Key Use Cases . . . 205
Examples with Explanations . . . 206
Create or refresh . . . 206
Do not create . . . 206
atime / mtime only . . . 206
Date strings . . . 206
Compact `-t` form . . . 206
Reference file . . . 207
Make and empty targets . . . 207
Scripts: ensure path exists as empty file . . . 207
Clear mtime into the past for cleanup tests . . . 207
Understanding timestamps . . . 207
Notes / Pitfalls . . . 208
2026-relevant notes . . . 208
Related Commands . . . 208
Additional Resources . . . 208

cp **209**
Overview . . . 209
Syntax . . . 209
Common Options . . . 209
Safety . . . 210
Examples with Explanations . . . 210
Files . . . 210
Directories . . . 210
Reflink (btrfs/xfs when available) . . . 210
Preserve attributes selectively . . . 210
Parents and target dir . . . 210
Symlinks . . . 211
One-liners . . . 211
Notes / Pitfalls . . . 211
2026-relevant notes . . . 211
Related Commands . . . 211
Additional Resources . . . 211

mv **212**
Overview . . . 212
Syntax . . . 212

Common Options . . . 212
Key Use Cases . . . 212
Examples with Explanations . . . 213
Rename and move . . . 213
Interactive / no-clobber . . . 213
Atomic replace pattern . . . 213
Verbose bulk . . . 213
Backup on conflict . . . 213
Directory edge cases . . . 213
Cross-device awareness . . . 214
Swap two files (bash) . . . 214
Notes / Pitfalls . . . 214
2026-relevant notes . . . 214
Related Commands . . . 214
Additional Resources . . . 215

rm **216**
Overview . . . 216
Syntax . . . 216
Common Options . . . 216
Safety . . . 216
Examples with Explanations . . . 217
Files . . . 217
Interactive . . . 217
Directory trees . . . 217
Empty directory only . . . 217
Dry-run then delete with find . . . 217
Stay on one filesystem . . . 218
Quarantine pattern . . . 218
Deleted but space held open . . . 218
Notes / Pitfalls . . . 218
2026-relevant notes . . . 218
Related Commands . . . 219
Additional Resources . . . 219

cat **220**
Overview . . . 220
Syntax . . . 220
Common Options . . . 220
Key Use Cases . . . 220
Examples with Explanations . . . 221
Print files . . . 221
Concatenate . . . 221
Heredocs . . . 221
Here-string and stdin . . . 221
Show non-printing / DOS endings . . . 222
Useless use of cat — avoid . . . 222
Binary caution . . . 222

Multiple files with separators (manual) . . . 222
Notes / Pitfalls . . . 222
2026-relevant notes . . . 222
Related Commands . . . 223
Additional Resources . . . 223

less **224**
Overview . . . 224
Syntax . . . 224
Common Options . . . 224
Essential keys . . . 224
Examples with Explanations . . . 225
Open files . . . 225
Pipe command output . . . 225
Follow a growing log . . . 225
Chop long lines (tables, wide logs) . . . 226
Case-insensitive search . . . 226
Multiple files . . . 226
Sensible defaults via environment . . . 226
Compare with more / bat . . . 226
Notes / Pitfalls . . . 226
2026-relevant notes . . . 227
Related Commands . . . 227
Additional Resources . . . 227

more **228**
Overview . . . 228
Syntax . . . 228
Common Options . . . 228
Common keys . . . 228
Examples with Explanations . . . 229
Basic . . . 229
Pipelines . . . 229
Prefer less when available . . . 229
Start position . . . 229
Tiny environments . . . 230
Compare pagers . . . 230
Notes / Pitfalls . . . 230
2026-relevant notes . . . 230
Related Commands . . . 230
Additional Resources . . . 230

chmod **231**
Overview . . . 231
Syntax . . . 231
Common Options . . . 231
Mode primer . . . 231
Safety . . . 232

Examples with Explanations . . . 232
Octal . . . 232
Symbolic . . . 232
Capital X (conditional execute) . . . 233
Recursive the safe way . . . 233
Reference mode . . . 233
Diagnose . . . 233
umask interaction (creation, not chmod) . . . 233
Notes / Pitfalls . . . 233
2026-relevant notes . . . 234
Related Commands . . . 234
Additional Resources . . . 234

chown **235**
Overview . . . 235
Syntax . . . 235
Common Options . . . 235
OWNER:GROUP forms . . . 235
Examples with Explanations . . . 236
Basic . . . 236
Recursive web tree . . . 236
Reference file . . . 236
Conditional change . . . 236
Symlinks . . . 236
Deploy pattern . . . 237
Audit before change . . . 237
Notes / Pitfalls . . . 237
2026-relevant notes . . . 237
Related Commands . . . 237
Additional Resources . . . 238

stat **239**
Overview . . . 239
Syntax . . . 239
Common Options . . . 239
Useful format sequences (GNU file) . . . 239
Examples with Explanations . . . 240
Default view . . . 240
Script-friendly fields . . . 240
Compare identities (same file?) . . . 240
Timestamps . . . 240
Filesystem info . . . 241
Batch . . . 241
Permissions debugging . . . 241
Terse / portable caution . . . 241
Notes / Pitfalls . . . 241
2026-relevant notes . . . 241
Related Commands . . . 242

Additional Resources . . . 242

file **243**
Overview . . . 243
Syntax . . . 243
Common Options . . . 243
Examples with Explanations . . . 244
Basic classification . . . 244
Brief and MIME . . . 244
Inside compressed data . . . 244
Symlinks . . . 244
Scripts and interpreters . . . 244
Stdin . . . 244
Batch unknown directory . . . 245
Guard uploads / pipelines . . . 245
ELF deep dive (after file) . . . 245
Notes / Pitfalls . . . 245
2026-relevant notes . . . 245
Related Commands . . . 245
Additional Resources . . . 246

find **247**
Overview . . . 247
Syntax . . . 247
Common Options / Primaries . . . 247
Key Use Cases . . . 248
Safety . . . 248
Examples with Explanations . . . 248
Find by name (case-insensitive) . . . 248
Limit depth . . . 248
Large files on one filesystem . . . 248
Modified in the last day . . . 249
Exclude a directory . . . 249
Safe batch action . . . 249
Exec with batching . . . 249
Permission audit (setuid) . . . 249
Empty directories . . . 249
Understanding Output . . . 249
Common Usage Patterns . . . 250
Newest N files under a tree . . . 250
Delete with interactive dry-run . . . 250
Notes & Pitfalls . . . 250
Related Commands . . . 250
Additional Resources . . . 250

fd **251**
Overview . . . 251
Syntax . . . 251

Common Options . . . 251
Key Use Cases . . . 252
Examples with Explanations . . . 252
By extension and pattern . . . 252
Types and hidden . . . 252
Exec . . . 252
Ignore rules . . . 252
One-liner recipes . . . 253
Notes & Pitfalls . . . 253
2026-relevant notes . . . 253
Comparison to alternatives . . . 253
Related Commands . . . 253
Additional Resources . . . 254

locate **255**
Overview . . . 255
Syntax . . . 255
Common Options . . . 255
Database updates . . . 256
Examples with Explanations . . . 256
Basic . . . 256
Limit and count . . . 256
Regex . . . 256
Basename-only . . . 256
Existence filter . . . 257
NUL-safe handoff . . . 257
When locate is empty . . . 257
locate vs find . . . 257
Notes / Pitfalls . . . 257
2026-relevant notes . . . 258
Related Commands . . . 258
Additional Resources . . . 258

ripgrep (rg) **259**
Overview . . . 259
Syntax . . . 259
Common Options . . . 259
Key Use Cases . . . 260
Examples with Explanations . . . 260
Basics . . . 260
Types and globs . . . 260
Context and files . . . 260
Unrestricted search . . . 260
One-liner recipes . . . 261
Notes & Pitfalls . . . 261
2026-relevant notes . . . 261
Comparison to alternatives . . . 261
Related Commands . . . 261

Additional Resources . . . 262

ln **263**
Overview . . . 263
Syntax . . . 263
Common Options . . . 263
Hard vs symbolic . . . 263
Examples with Explanations . . . 264
Symbolic links . . . 264
Relative symlink (GNU) . . . 264
Hard links . . . 264
Bulk into a directory . . . 264
Inspect . . . 264
Replace symlink safely . . . 265
Common failure: relative target confusion . . . 265
Notes / Pitfalls . . . 265
2026-relevant notes . . . 265
Related Commands . . . 265
Additional Resources . . . 266

readlink **267**
Overview . . . 267
Syntax . . . 267
Common Options (GNU) . . . 267
Key Use Cases . . . 267
Examples with Explanations . . . 268
Immediate target . . . 268
Canonical forms . . . 268
Multiple links . . . 268
Script directory pattern . . . 268
BusyBox / portability note . . . 268
Inspect vs ls . . . 269
NUL-safe with find . . . 269
Notes / Pitfalls . . . 269
2026-relevant notes . . . 269
Related Commands . . . 269
Additional Resources . . . 269

realpath **270**
Overview . . . 270
Syntax . . . 270
Common Options . . . 270
Key Use Cases . . . 270
Examples with Explanations . . . 271
Absolute paths . . . 271
Relative formatting . . . 271
Script root . . . 271
Same-file check . . . 271

No symlink expand . . . 272
Batch NUL-safe . . . 272
Combine with find . . . 272
Notes / Pitfalls . . . 272
2026-relevant notes . . . 272
Related Commands . . . 272
Additional Resources . . . 273

basename **274**
Overview . . . 274
Syntax . . . 274
Common Options . . . 274
Key Use Cases . . . 274
Examples with Explanations . . . 275
Basics . . . 275
Multiple names . . . 275
Script name . . . 275
Strip extension patterns . . . 275
Loop rename . . . 275
Combine with dirname . . . 276
NUL-safe pipeline . . . 276
Parameter expansion alternative . . . 276
Notes / Pitfalls . . . 276
2026-relevant notes . . . 276
Related Commands . . . 277
Additional Resources . . . 277

dirname **278**
Overview . . . 278
Syntax . . . 278
Common Options . . . 278
Key Use Cases . . . 278
Examples with Explanations . . . 279
Basics . . . 279
Multiple arguments . . . 279
Script directory pattern . . . 279
Ensure parent exists . . . 279
Sibling paths . . . 279
Combine with realpath . . . 280
Parameter expansion alternative (bash) . . . 280
NUL-safe . . . 280
Notes / Pitfalls . . . 280
2026-relevant notes . . . 280
Related Commands . . . 280
Additional Resources . . . 281

which **282**
Overview . . . 282

Syntax . . . 282
Common Options (GNU / common) . . . 282
Key Use Cases . . . 282
Examples with Explanations . . . 283
Basic . . . 283
All matches . . . 283
Prefer command -v in scripts . . . 283
Compare resolution tools . . . 283
Debug PATH order . . . 284
Silent existence (when -s exists) . . . 284
Notes / Pitfalls . . . 284
2026-relevant notes . . . 284
Related Commands . . . 284
Additional Resources . . . 285

alias / unalias **286**
Overview . . . 286
Syntax . . . 286
Common Options / related . . . 286
Examples with Explanations . . . 287
Everyday shortcuts . . . 287
List and inspect . . . 287
Persist (bash) . . . 287
Remove . . . 287
Bypass aliases . . . 288
sudo and alias expansion (bash) . . . 288
Better as functions . . . 288
Safety alias caveats . . . 288
Notes / Pitfalls . . . 288
2026-relevant notes . . . 289
Related Commands . . . 289
Additional Resources . . . 289

install **290**
Overview . . . 290
Syntax . . . 290
Common Options . . . 290
Key Use Cases . . . 291
Examples with Explanations . . . 291
Install a script or binary . . . 291
Create directories . . . 291
Config with ownership . . . 291
Leading path creation (systemd unit) . . . 291
Multiple sources into a directory . . . 291
Compare before replace (GNU) . . . 292
Makefile fragment . . . 292
Versus cp . . . 292
Notes / Pitfalls . . . 292

2026-relevant notes . . . 292
Related Commands . . . 293
Additional Resources . . . 293

df **294**
Overview . . . 294
Syntax . . . 294
Common Options . . . 294
Examples with Explanations . . . 294
Everyday overview . . . 294
Inodes exhausted? . . . 295
Specific path . . . 295
Portable scripting . . . 295
JSON-ish via findmnt (often clearer) . . . 295
Correlate devices . . . 295
Root reserved space . . . 295
Understanding output . . . 295
Notes / Pitfalls . . . 296
2026-relevant notes . . . 296
Related Commands . . . 296
Additional Resources . . . 296

du **297**
Overview . . . 297
Syntax . . . 297
Common Options . . . 297
Examples with Explanations . . . 298
Top-level breakdown . . . 298
Summaries . . . 298
One filesystem (skip mounts) . . . 298
Apparent vs allocated . . . 298
Exclude noise . . . 298
Sort biggest . . . 298
Scripts (bytes) . . . 299
Find large files (pair with find) . . . 299
Notes / Pitfalls . . . 299
2026-relevant notes . . . 299
Related Commands . . . 299
Additional Resources . . . 299

bat **300**
Overview . . . 300
Syntax . . . 300
Common Options . . . 300
Examples with Explanations . . . 301
View files . . . 301
Debian binary name . . . 301
Plain / script-friendly . . . 301

Ranges and highlights . . . 301
Force language . . . 301
Multiple files and stdin . . . 301
Git-aware . . . 302
Themes and style . . . 302
As man / pager helper . . . 302
Compare with cat / less . . . 302
Notes / Pitfalls . . . 302
2026-relevant notes . . . 302
Related Commands . . . 303
Additional Resources . . . 303

eza **304**
Overview . . . 304
Syntax . . . 304
Common Options . . . 304
Examples with Explanations . . . 305
Everyday aliases . . . 305
Sorting . . . 305
Tree view . . . 305
Git-aware listing . . . 305
Headers and metadata . . . 305
Compare with ls / tree . . . 306
Scripts: prefer classic ls . . . 306
Notes / Pitfalls . . . 306
2026-relevant notes . . . 306
Related Commands . . . 306
Additional Resources . . . 307

sha256sum **308**
Overview . . . 308
Syntax . . . 308
Common Options . . . 308
Examples with Explanations . . . 309
Hash a file . . . 309
Hash several files . . . 309
Stdin . . . 309
Write and verify a checksum file . . . 309
Quiet / script-friendly check . . . 309
Verify published ISO pattern . . . 310
Directory of artifacts . . . 310
Compare two local copies without a list . . . 310
Notes . . . 310
Related Commands . . . 310
Additional Resources . . . 311

base64 **312**
Overview . . . 312

Syntax . . . 312
Common Options . . . 312
Examples with Explanations . . . 312
Encode a file . . . 312
Decode to a file . . . 313
Encode/decode stdin pipelines . . . 313
Small binary in a shell variable (careful) . . . 313
Kubernetes-style secret (illustrative) . . . 313
PEM / cert peek . . . 313
Ignore whitespace garbage when decoding . . . 313
Notes . . . 314
Related Commands . . . 314
Additional Resources . . . 314

cmp **315**
Overview . . . 315
Syntax . . . 315
Common Options . . . 315
Examples with Explanations . . . 315
Basic equal / not equal . . . 315
Where they first differ . . . 316
Quiet for scripts . . . 316
List every difference (small files) . . . 316
Compare after skipping headers . . . 316
Limit length . . . 316
Against a device (careful read-only) . . . 317
Notes . . . 317
Related Commands . . . 317
Additional Resources . . . 317

getfacl / setfacl **318**
Overview . . . 318
Syntax . . . 318
Common Options . . . 318
getfacl . . . 318
setfacl . . . 319
Safety . . . 319
Examples with Explanations . . . 319
View ACLs . . . 319
Grant a user access . . . 320
Grant a group and adjust mask . . . 320
Default ACL on a directory (inherit for new files) . . . 320
Recursive grant . . . 320
Remove entries / clear ACLs . . . 320
Copy ACL from one file to another . . . 321
Dry-run . . . 321
Backup and restore ACL tree . . . 321
Notes . . . 321

Related Commands . . . 321
Additional Resources . . . 322

chattr / lsattr **323**
Overview . . . 323
Syntax . . . 323
Common Options . . . 323
lsattr . . . 323
chattr . . . 323
Important attribute letters . . . 324
Safety . . . 324
Examples with Explanations . . . 324
List attributes . . . 324
Make a file immutable . . . 325
Append-only log . . . 325
Clear / set exactly . . . 325
Protect a directory entry (directory itself) . . . 325
No-dump flag . . . 325
Combine with permissions . . . 326
Notes . . . 326
Related Commands . . . 326
Additional Resources . . . 326

xxd **327**
Overview . . . 327
Syntax . . . 327
Common Options . . . 327
Examples with Explanations . . . 328
Basic dump . . . 328
Limit and offset . . . 328
Plain hex (scripts / compare) . . . 328
C array for embedding . . . 328
Reverse plain hex to binary . . . 328
Edit a few bytes (workflow) . . . 329
Inspect file magic . . . 329
Grouping and columns for readability . . . 329
Bits view . . . 329
Notes . . . 329
Related Commands . . . 330
Additional Resources . . . 330

strings **331**
Overview . . . 331
Syntax . . . 331
Common Options . . . 331
Examples with Explanations . . . 332
Basic scan . . . 332
Longer strings only (less noise) . . . 332

Offsets for later xxd . . . 332
UTF-16 little-endian (Windows-ish binaries) . . . 332
Multiple files . . . 332
Full file scan . . . 332
Pipe from another command . . . 333
Hunt secrets carefully . . . 333
Notes . . . 333
Related Commands . . . 333
Additional Resources . . . 333

namei **334**
Overview . . . 334
Syntax . . . 334
Common Options . . . 334
Examples with Explanations . . . 334
Diagnose a path . . . 334
Follow ownership along the way . . . 335
Mount boundaries . . . 335
Notes & Pitfalls . . . 335
Related Commands . . . 335
Additional Resources . . . 335

umask **336**
Overview . . . 336
Syntax . . . 336
Common Options . . . 336
Key Use Cases . . . 336
Examples with Explanations . . . 337
Show current mask . . . 337
How mask applies . . . 337
Stricter group isolation . . . 337
Secrets in a script . . . 337
Persist for login shells . . . 337
Notes & Pitfalls . . . 338
Related Commands . . . 338
Additional Resources . . . 338

Watching the filesystem: inotify vs fanotify **339**
Overview . . . 339
Mental model . . . 340
Syntax . . . 341
Safety . . . 342
Examples with Explanations . . . 343
1. One directory watch, everyday operations . . . 343
2. The editor trap: a watch on the file dies after the first save . . . 345
3. Overflow: what you get when you read too slowly . . . 347
4. Who is using up the watches? . . . 348
5. fanotify without root: what the 5.13 unprivileged mode allows . . . 349

6. What privileged fanotify adds (from the docs, not run) . . . 352
The trap . . . 352
Notes & Pitfalls . . . 353
The boring rule . . . 353
Related Commands . . . 354
Try this . . . 354
Verification . . . 354
Sources . . . 355

Archives and Compression **356**

Intro **357**

Intro **358**
Commands in this part . . . 358
Suggested starting points . . . 358
Related parts . . . 359

tar **360**
Overview . . . 360
Syntax . . . 360
Common Options . . . 360
Key Use Cases . . . 360
Safety . . . 361
Examples with Explanations . . . 361
Create gzip archive . . . 361
Create zstd archive . . . 361
List . . . 361
Extract . . . 361
Strip top-level directory . . . 361
Exclude patterns . . . 362
Stream over ssh . . . 362
Append is special . . . 362
Common Usage Patterns . . . 362
Quick directory move . . . 362
Notes & Pitfalls . . . 362
Related Commands . . . 362
Additional Resources . . . 363

gzip **364**
Overview . . . 364
Syntax . . . 364
Common Options . . . 364
Examples with Explanations . . . 365
Compress / decompress . . . 365
stdout pipelines . . . 365
Level tradeoffs . . . 365
Test and list . . . 365

Recursive logs . . . 365
View without extracting . . . 365
Integrity in scripts . . . 366
Notes / Pitfalls . . . 366
2026-relevant notes . . . 366
Related Commands . . . 366
Additional Resources . . . 366

xz **367**
Overview . . . 367
Syntax . . . 367
Common Options . . . 367
Examples with Explanations . . . 368
Basic . . . 368
Pipelines with tar . . . 368
Presets and threads . . . 368
Test and list . . . 368
Keep and stdout . . . 368
Memory safety on small hosts . . . 368
Convert from gzip (recompress) . . . 369
Notes / Pitfalls . . . 369
2026-relevant notes . . . 369
Related Commands . . . 369
Additional Resources . . . 369

bzip2 **370**
Overview . . . 370
Syntax . . . 370
Common Options . . . 370
Examples with Explanations . . . 371
Compress / decompress . . . 371
Pipelines . . . 371
Levels . . . 371
Test . . . 371
Search compressed . . . 371
Prefer modern tools when choosing . . . 371
Notes / Pitfalls . . . 372
2026-relevant notes . . . 372
Related Commands . . . 372
Additional Resources . . . 372

zstd **373**
Overview . . . 373
Syntax . . . 373
Common Options . . . 373
Examples with Explanations . . . 374
Basic . . . 374
Levels and threads . . . 374

Pipelines with tar . . . 374
Integrity . . . 374
Remove source when desired . . . 374
Dictionaries (small similar files) . . . 375
Notes / Pitfalls . . . 375
2026-relevant notes . . . 375
Related Commands . . . 375
Additional Resources . . . 375

zip **376**
Overview . . . 376
Syntax . . . 376
Common Options . . . 376
Examples with Explanations . . . 377
Create archives . . . 377
Update and list . . . 377
Flatten paths . . . 377
Store without compression . . . 377
Encryption (know the limits) . . . 377
Split archives . . . 377
Exclude junk . . . 377
Date-stamped backup . . . 378
Notes / Pitfalls . . . 378
2026-relevant notes . . . 378
Related Commands . . . 378
Additional Resources . . . 378

unzip **379**
Overview . . . 379
Syntax . . . 379
Common Options . . . 379
Examples with Explanations . . . 380
List and test . . . 380
Extract . . . 380
Selective extract . . . 380
Overwrite policies . . . 380
Flatten paths . . . 380
Pipe / quiet automation . . . 380
Untrusted archives — path safety . . . 381
Password . . . 381
Notes / Pitfalls . . . 381
2026-relevant notes . . . 381
Related Commands . . . 381
Additional Resources . . . 382

zcat **383**
Overview . . . 383
Syntax . . . 383

Examples with Explanations . . . 383
Read a gzip log . . . 383
Concatenate multiple . . . 383
Portable explicit form . . . 383
Notes & Pitfalls . . . 384
Related Commands . . . 384
Additional Resources . . . 384

zgrep **385**
Overview . . . 385
Syntax . . . 385
Examples with Explanations . . . 385
Search compressed logs . . . 385
Recursive pattern over mixed trees . . . 385
Count matches . . . 385
Notes & Pitfalls . . . 386
Related Commands . . . 386
Additional Resources . . . 386

System Information 387

Intro **388**

Intro **389**
Commands in this part . . . 389
Suggested starting points . . . 390
Related parts . . . 390

uname **392**
Overview . . . 392
Syntax . . . 392
Common Options . . . 392
Examples with Explanations . . . 392
Everyday . . . 392
Architecture branching . . . 393
Kernel version checks . . . 393
vs os-release . . . 393
Container note . . . 393
Compact inventory line . . . 393
Notes / Pitfalls . . . 393
2026-relevant notes . . . 394
Related Commands . . . 394
Additional Resources . . . 394

hostname **395**
Overview . . . 395
Syntax . . . 395
Common Options . . . 395

Examples with Explanations . . . 396
Read . . . 396
Prefer hostnamectl on systemd . . . 396
Temporary classic set (legacy) . . . 396
Scripts . . . 396
Correlate with DNS / addresses . . . 396
Notes / Pitfalls . . . 397
2026-relevant notes . . . 397
Related Commands . . . 397
Additional Resources . . . 397

hostnamectl **398**
Overview . . . 398
Syntax . . . 398
Common Commands / Options . . . 398
Examples with Explanations . . . 399
Status . . . 399
Set hostname . . . 399
Pretty name . . . 399
Transient (temporary) . . . 399
Chassis / location metadata . . . 399
Remote / container . . . 399
Verify files . . . 400
Notes / Pitfalls . . . 400
2026-relevant notes . . . 400
Related Commands . . . 400
Additional Resources . . . 400

lsb_release **401**
Overview . . . 401
Syntax . . . 401
Common Options . . . 401
Examples with Explanations . . . 401
Everyday . . . 401
Short for scripts . . . 402
Prefer os-release (portable modern) . . . 402
Compare sources . . . 402
Install if missing . . . 402
Notes / Pitfalls . . . 402
2026-relevant notes . . . 403
Related Commands . . . 403
Additional Resources . . . 403

uptime **404**
Overview . . . 404
Syntax . . . 404
Common Options . . . 404

Examples with Explanations . . . 404
Default . . . 404
Pretty / since . . . 405
Interpret load vs CPUs . . . 405
Watch over time . . . 405
Scripts . . . 405
Correlate with other tools . . . 405
Containers . . . 405
Understanding load average . . . 405
Notes / Pitfalls . . . 406
2026-relevant notes . . . 406
Related Commands . . . 406
Additional Resources . . . 406

date **407**
Overview . . . 407
Syntax . . . 407
Common Options . . . 407
Useful format sequences . . . 407
Examples with Explanations . . . 408
Now . . . 408
Convert / relative . . . 408
Filenames and logs . . . 408
File mtime . . . 408
Timezone one-off . . . 408
Set clock (prefer NTP) . . . 409
GNU vs scripts . . . 409
Notes / Pitfalls . . . 409
2026-relevant notes . . . 409
Related Commands . . . 409
Additional Resources . . . 409

timedatectl **410**
Overview . . . 410
Syntax . . . 410
Common Commands / Options . . . 410
Key Use Cases . . . 410
Examples with Explanations . . . 411
Status . . . 411
Machine-readable . . . 411
List and set timezone . . . 411
Enable NTP (systemd-timesyncd path) . . . 411
Chrony hosts . . . 411
Manual time (maintenance / air-gap) . . . 412
RTC in UTC (recommended on Linux) . . . 412
Verify sync health . . . 412
date interaction . . . 412
Understanding Output . . . 413

Notes & Pitfalls . . . 413
Related Commands . . . 413
Additional Resources . . . 413

cal **414**
Overview . . . 414
Syntax . . . 414
Common Options (util-linux / common) . . . 414
Examples with Explanations . . . 415
Current month . . . 415
Specific month / year . . . 415
Week numbers for planning . . . 415
Surrounding months . . . 415
Historical curiosity . . . 415
Scripts: parse carefully . . . 415
ncal alternative . . . 416
Notes / Pitfalls . . . 416
2026-relevant notes . . . 416
Related Commands . . . 416
Additional Resources . . . 416

whoami **417**
Overview . . . 417
Syntax . . . 417
Common Options . . . 417
Key Use Cases . . . 417
Examples with Explanations . . . 418
Basics . . . 418
Script guards . . . 418
Contrast with id / logname . . . 418
Logging . . . 418
Containers . . . 418
Notes / Pitfalls . . . 419
2026-relevant notes . . . 419
Related Commands . . . 419
Additional Resources . . . 419

id **420**
Overview . . . 420
Syntax . . . 420
Common Options . . . 420
Examples with Explanations . . . 420
Default . . . 420
Specific fields . . . 421
Other users . . . 421
Root check in scripts . . . 421
Real vs effective (setuid context) . . . 421
SELinux . . . 421

Format for logs . . . 421
Permission debugging pattern . . . 422
Notes / Pitfalls . . . 422
2026-relevant notes . . . 422
Related Commands . . . 422
Additional Resources . . . 422

who **423**
Overview . . . 423
Syntax . . . 423
Common Options . . . 423
Examples with Explanations . . . 424
Who is on the system . . . 424
Boot time . . . 424
Yourself . . . 424
Compare with w . . . 424
Historical . . . 424
Scripting inventory . . . 424
Notes / Pitfalls . . . 425
2026-relevant notes . . . 425
Related Commands . . . 425
Additional Resources . . . 425

w **426**
Overview . . . 426
Syntax . . . 426
Common Options . . . 426
Examples with Explanations . . . 426
Default . . . 426
Filter one user . . . 427
Short / no header . . . 427
IP addresses . . . 427
Correlate with load tools . . . 427
When you need history . . . 427
Understanding columns . . . 427
Notes / Pitfalls . . . 428
2026-relevant notes . . . 428
Related Commands . . . 428
Additional Resources . . . 428

last **429**
Overview . . . 429
Syntax . . . 429
Common Options . . . 429
Examples with Explanations . . . 429
Recent logins . . . 429
One user . . . 430
Reboots and shutdowns . . . 430

Bad logins . . . 430
Rotate awareness . . . 430
SSH audit starter . . . 430
Notes / Pitfalls . . . 431
2026-relevant notes . . . 431
Related Commands . . . 431
Additional Resources . . . 431

env **432**
Overview . . . 432
Syntax . . . 432
Common Options . . . 432
Examples with Explanations . . . 432
Print environment . . . 432
Run with extra variables . . . 433
Clear environment . . . 433
Unset for one command . . . 433
Shebang portability . . . 433
Chdir then run (when supported) . . . 433
NUL-safe dump . . . 433
Notes / Pitfalls . . . 434
2026-relevant notes . . . 434
Related Commands . . . 434
Additional Resources . . . 434

free **435**
Overview . . . 435
Syntax . . . 435
Common Options . . . 435
Examples with Explanations . . . 435
Everyday . . . 435
Watch live . . . 436
Scripts . . . 436
Interpret with pressure tools . . . 436
Containers / cgroups . . . 436
Understanding the columns . . . 436
Notes / Pitfalls . . . 437
2026-relevant notes . . . 437
Related Commands . . . 437
Additional Resources . . . 437

lscpu **438**
Overview . . . 438
Syntax . . . 438
Common Options . . . 438
Examples with Explanations . . . 438
Overview . . . 438
Parseable / JSON . . . 439

Core counts for scripts . . . 439
Virtualization and flags . . . 439
NUMA . . . 439
Security mitigations summary . . . 439
Containers . . . 439
Notes / Pitfalls . . . 440
2026-relevant notes . . . 440
Related Commands . . . 440
Additional Resources . . . 440

lsmem **441**
Overview . . . 441
Syntax . . . 441
Common Options . . . 441
Output Columns . . . 441
Key Use Cases . . . 442
Examples with Explanations . . . 442
Example 1: Basic Memory Information . . . 442
Example 2: Human-Readable Sizes . . . 442
Example 3: Summary Only . . . 442
Understanding Memory States . . . 442
Memory Block Management . . . 442
Common Usage Patterns . . . 443
NUMA Memory Information . . . 443
Memory Hotplug Operations . . . 443
Performance Analysis . . . 443
Related Commands . . . 443
Additional Resources . . . 444
Best Practices . . . 444
Scripting Examples . . . 444
Memory Zones . . . 444
System Integration . . . 444
Troubleshooting . . . 445
Advanced Usage . . . 445
Memory Block Operations . . . 445
Integration Examples . . . 446
Output Formatting . . . 446

lsusb **447**
Overview . . . 447
Syntax . . . 447
Common Options . . . 447
Examples with Explanations . . . 447
List devices . . . 447
Filter . . . 448
Verbose probe . . . 448
Correlate with kernel messages . . . 448
Find device nodes . . . 448

Tree + drivers . . . 448
Notes / Pitfalls . . . 448
2026-relevant notes . . . 449
Related Commands . . . 449
Additional Resources . . . 449

lsmod **450**
Overview . . . 450
Syntax . . . 450
Examples with Explanations . . . 450
List modules . . . 450
Search for a driver . . . 450
Dependency / use count . . . 450
Correlate with hardware . . . 451
Module details . . . 451
Unload safety check . . . 451
Blacklist context . . . 451
Understanding columns . . . 451
Notes / Pitfalls . . . 452
2026-relevant notes . . . 452
Related Commands . . . 452
Additional Resources . . . 452

hwinfo **453**
Overview . . . 453
Syntax . . . 453
Common Options . . . 453
Key Use Cases . . . 453
Examples with Explanations . . . 454
Example 1: Brief Summary . . . 454
Example 2: CPU Info . . . 454
Example 3: Storage Info . . . 454
Understanding Output . . . 454
Common Usage Patterns . . . 454
Performance Analysis . . . 455
Related Commands . . . 455
Additional Resources . . . 455
Hardware Categories . . . 455
Best Practices . . . 455

Four Ways to Talk to the Kernel **456**

Four Ways to Talk to the Kernel **457**
Thesis . . . 457
Mental model . . . 457
Worked examples . . . 457
Trap . . . 457
Boring rule . . . 457

Try this . . . 457
Sources . . . 458

CPU masks and lists: reading the kernel's bitmaps **459**
Overview . . . 459
Mental model . . . 459
Case 1 — The same bits, two files . . . 460
Case 2 — Sysfs: lists and masks side by side . . . 461
Case 3 — The arithmetic, in user space . . . 462
Case 4 — Converting by hand and in a script . . . 465
Where you will meet each form . . . 465
Gotchas . . . 466
Quick reference . . . 466
Sources . . . 467

Processes and Jobs **468**

Intro **469**

Intro **470**
Commands in this part . . . 470
Suggested starting points . . . 472
Related parts . . . 472

ps **473**
Overview . . . 473
Syntax . . . 473
Common Invocations . . . 473
Key Use Cases . . . 473
Examples with Explanations . . . 474
Classic full lists . . . 474
Top memory and CPU . . . 474
Custom columns . . . 474
By user / name . . . 474
Tree view . . . 474
One PID in detail . . . 475
Threads . . . 475
Full command line (no truncation) . . . 475
Zombies and D-state . . . 475
Script: fail if process missing . . . 475
Understanding Output . . . 475
Notes & Pitfalls . . . 476
Related Commands . . . 476
Additional Resources . . . 476

pstree **477**
Overview . . . 477
Syntax . . . 477

Common Options . . . 477
Examples with Explanations . . . 478
Basics . . . 478
One user or PID . . . 478
Show ancestry of a process . . . 478
Avoid compaction . . . 478
Threads vs processes . . . 478
SELinux . . . 478
Compare with ps . . . 479
Notes / Pitfalls . . . 479
2026-relevant notes . . . 479
Related Commands . . . 479
Additional Resources . . . 479

top **480**
Overview . . . 480
Syntax . . . 480
Common Options . . . 480
Interactive keys (procps-ng) . . . 480
Key Use Cases . . . 481
Examples with Explanations . . . 481
Example: interactive default . . . 481
Example: refresh every two seconds . . . 481
Example: single-user focus . . . 481
Example: watch specific PIDs . . . 482
Example: batch sample for logs / tickets . . . 482
Example: sort by memory from the CLI . . . 482
Example: threads view . . . 482
Example: kill from inside top . . . 482
Example: one-shot CPU leaders without staying in top . . . 482
Understanding Output . . . 483
Notes & Pitfalls . . . 483
Related Commands . . . 483
Additional Resources . . . 484

htop **485**
Overview . . . 485
Syntax . . . 485
Common Options . . . 485
Useful keys . . . 485
Key Use Cases . . . 486
Examples with Explanations . . . 486
Example: start htop . . . 486
Example: only your processes . . . 486
Example: tree view for process storms . . . 487
Example: attach to known PIDs . . . 487
Example: slower refresh on overloaded boxes . . . 487
Example: kill workflow inside htop . . . 487

Example: nice a batch job interactively . . . 487
Example: setup persistence . . . 487
Understanding Output . . . 488
Notes & Pitfalls . . . 488
Related Commands . . . 488
Additional Resources . . . 488

procs **489**
Overview . . . 489
Syntax . . . 489
Common Options . . . 489
Examples with Explanations . . . 490
Everyday . . . 490
Sorting . . . 490
Columns . . . 490
JSON for tooling . . . 490
Compare with ps . . . 490
Watch like top (lightweight) . . . 490
Notes / Pitfalls . . . 491
2026-relevant notes . . . 491
Related Commands . . . 491
Additional Resources . . . 491

pgrep **492**
Overview . . . 492
Syntax . . . 492
Common Options . . . 492
Safety . . . 493
Key Use Cases . . . 493
Examples with Explanations . . . 493
Example: list matching PIDs . . . 493
Example: exact name match . . . 493
Example: full command line . . . 494
Example: filter by user . . . 494
Example: count for monitoring . . . 494
Example: newest / oldest only . . . 494
Example: children of a parent . . . 494
Example: delimiter for other tools . . . 494
Example: pkill graceful then force . . . 495
Example: pkill with full-line match . . . 495
Example: avoid ps | grep . . . 495
Understanding Output . . . 495
Notes & Pitfalls . . . 495
Related Commands . . . 496
Additional Resources . . . 496

pidof **497**
Overview . . . 497

Syntax . . . 497
Common Options . . . 497
Examples with Explanations . . . 497
Basics . . . 497
Single PID . . . 498
Scripts . . . 498
Existence gate . . . 498
Omit PIDs . . . 498
Feed to kill . . . 498
Compare tools . . . 498
Notes / Pitfalls . . . 499
2026-relevant notes . . . 499
Related Commands . . . 499
Additional Resources . . . 499

kill **500**
Overview . . . 500
Syntax . . . 500
Common Options . . . 500
Safety . . . 501
Key Use Cases . . . 501
Examples with Explanations . . . 501
Example: graceful stop (default TERM) . . . 501
Example: TERM, wait, then KILL . . . 501
Example: force kill only when necessary . . . 502
Example: reload via HUP . . . 502
Example: signal by name via pgrep . . . 502
Example: stop / continue (debug freeze) . . . 502
Example: list signals . . . 503
Example: multiple PIDs . . . 503
Example: permission check without killing . . . 503
Understanding Output . . . 503
Notes & Pitfalls . . . 503
Related Commands . . . 504
Additional Resources . . . 504

killall **505**
Overview . . . 505
Syntax . . . 505
Common Options . . . 505
Safety . . . 505
Key Use Cases . . . 506
Examples with Explanations . . . 506
Example: graceful stop by name . . . 506
Example: TERM then KILL . . . 506
Example: only your processes . . . 506
Example: interactive confirm . . . 506
Example: verbose + wait . . . 507

Example: HUP reload pattern . . . 507
Example: case-insensitive / regex . . . 507
Example: list signals . . . 507
Example: dry reconnaissance first . . . 507
Notes & Pitfalls . . . 508
Related Commands . . . 508
Additional Resources . . . 508

jobs **509**
Overview . . . 509
Syntax . . . 509
Common Options (bash) . . . 509
Job states . . . 509
Examples with Explanations . . . 510
List jobs . . . 510
Act on a job . . . 510
Status changes . . . 510
wait integration . . . 510
Enable job control in scripts (rare) . . . 510
disown removes from table . . . 511
Notes / Pitfalls . . . 511
2026-relevant notes . . . 511
Related Commands . . . 511
Additional Resources . . . 511

fg **512**
Overview . . . 512
Syntax . . . 512
Job specification . . . 512
Examples with Explanations . . . 512
After Ctrl-Z . . . 512
Pull background job forward . . . 513
Switch between jobs . . . 513
With pipelines . . . 513
Script note . . . 513
Notes / Pitfalls . . . 513
2026-relevant notes . . . 514
Related Commands . . . 514
Additional Resources . . . 514

bg **515**
Overview . . . 515
Syntax . . . 515
Job specification . . . 515
Common Options . . . 515
Examples with Explanations . . . 516
Suspend then background . . . 516
Start in background directly . . . 516

Specific job . . . 516
Multiple jobs . . . 516
Typical workflow . . . 516
Disown / survive hangup . . . 517
Not for scripts by default . . . 517
Notes / Pitfalls . . . 517
2026-relevant notes . . . 517
Related Commands . . . 517
Additional Resources . . . 518

nohup **519**
Overview . . . 519
Syntax . . . 519
Common Options . . . 519
Behavior (what you actually care about) . . . 519
Key Use Cases . . . 520
Examples with Explanations . . . 520
Example: basic background job . . . 520
Example: explicit log file . . . 520
Example: find and stop later . . . 520
Example: disown alternative (bash) . . . 520
Example: combine with nice / ionice . . . 521
Example: remote one-liner over SSH . . . 521
Example: check nohup.out . . . 521
Safety . . . 521
Notes & Pitfalls . . . 521
Related Commands . . . 522
Additional Resources . . . 522

timeout **523**
Overview . . . 523
Syntax . . . 523
Duration format . . . 523
Common Options . . . 523
Exit status (GNU coreutils) . . . 524
Safety . . . 524
Key Use Cases . . . 524
Examples with Explanations . . . 524
Example: basic time limit . . . 524
Example: TERM then KILL grace period . . . 525
Example: choose the first signal . . . 525
Example: capture timeout in scripts . . . 525
Example: verbose diagnosis . . . 525
Example: network tool guardrails . . . 525
Example: command that starts with a dash . . . 526
Example: foreground mode . . . 526
Notes & Pitfalls . . . 526
Related Commands . . . 526

Additional Resources . . . 526

watch **527**
Overview . . . 527
Syntax . . . 527
Common Options . . . 527
Key Use Cases . . . 527
Examples with Explanations . . . 528
Example: basic interval monitor . . . 528
Example: highlight changes . . . 528
Example: multi-command pipeline . . . 528
Example: systemd / journal peek . . . 528
Example: exit when output changes . . . 528
Example: stop on error . . . 529
Example: colors from modern CLIs . . . 529
Example: precise interval . . . 529
Example: exec mode (no shell) . . . 529
Notes & Pitfalls . . . 529
Related Commands . . . 530
Additional Resources . . . 530

sleep **531**
Overview . . . 531
Syntax . . . 531
Time suffixes (GNU) . . . 531
Examples with Explanations . . . 531
Basics . . . 531
Simple retry loop . . . 532
Exponential backoff (bash) . . . 532
Rate limiting . . . 532
Background heartbeat . . . 532
Interruptible . . . 532
Portable note . . . 533
Notes / Pitfalls . . . 533
2026-relevant notes . . . 533
Related Commands . . . 533
Additional Resources . . . 533

nice **534**
Overview . . . 534
Syntax . . . 534
Common Options . . . 534
Safety . . . 535
Key Use Cases . . . 535
Examples with Explanations . . . 535
Example: start a polite batch job . . . 535
Example: background + log . . . 535
Example: renice a running PID . . . 535

Example: renice all of a user's jobs . . . 536
Example: observe nice in ps / top . . . 536
Example: nice + ionice + nohup combo . . . 536
Example: systemd equivalent knobs . . . 536
Example: print current nice (when supported) . . . 536
Understanding Output . . . 536
Notes & Pitfalls . . . 537
Related Commands . . . 537
Additional Resources . . . 537

renice **538**
Overview . . . 538
Syntax . . . 538
Common Options . . . 538
Nice Values . . . 538
Key Use Cases . . . 539
Examples with Explanations . . . 539
Example 1: Basic Usage . . . 539
Example 2: User Processes . . . 539
Example 3: Process Group . . . 539
Understanding Output . . . 539
Common Usage Patterns . . . 539
Performance Analysis . . . 540
Related Commands . . . 540
Additional Resources . . . 540
Best Practices . . . 540
Use Cases . . . 541

ionice **542**
Overview . . . 542
Syntax . . . 542
I/O classes . . . 542
Common Options . . . 542
Safety . . . 543
Key Use Cases . . . 543
Examples with Explanations . . . 543
Example: run command as idle I/O . . . 543
Example: best-effort low priority . . . 543
Example: change a running process . . . 544
Example: query current setting . . . 544
Example: nice + ionice together . . . 544
Example: nohup overnight job . . . 544
Example: try without failing scripts . . . 544
Example: avoid realtime unless justified . . . 545
Notes & Pitfalls . . . 545
Related Commands . . . 545
Additional Resources . . . 545

fuser **546**
Overview . . . 546
Syntax . . . 546
Common Options . . . 546
Key Use Cases . . . 546
Examples with Explanations . . . 547
Mount busy . . . 547
Ports . . . 547
Files . . . 547
One-liner recipes . . . 547
Notes & Pitfalls . . . 547
2026-relevant notes . . . 548
Related Commands . . . 548
Additional Resources . . . 548

ulimit **549**
Overview . . . 549
Syntax . . . 549
Common Options . . . 549
Key Use Cases . . . 550
Examples with Explanations . . . 550
Show limits . . . 550
Raise open files (session) . . . 550
Cores and stack . . . 550
One-liner recipes . . . 550
Persistent configuration (context) . . . 551
Notes & Pitfalls . . . 551
2026-relevant notes . . . 551
Related Commands . . . 551
Additional Resources . . . 551

ipcs **552**
Overview . . . 552
Syntax . . . 552
Common Options . . . 552
Examples with Explanations . . . 553
Overview . . . 553
Limits and usage . . . 553
Creator and PIDs . . . 553
Find leftovers by user . . . 553
Cleanup companion . . . 553
Kernel params (context) . . . 554
Notes / Pitfalls . . . 554
2026-relevant notes . . . 554
Related Commands . . . 554
Additional Resources . . . 554

ipcrm **555**
Overview . . . 555
Syntax . . . 555
Common Options . . . 555
Examples with Explanations . . . 556
Inspect then remove one segment . . . 556
Remove by key . . . 556
Message queues / semaphores . . . 556
Scripted cleanup of your own leftovers . . . 556
After crash lab example . . . 556
Notes / Pitfalls . . . 557
2026-relevant notes . . . 557
Related Commands . . . 557
Additional Resources . . . 557

flock **558**
Overview . . . 558
Mental model . . . 558
Syntax . . . 559
Safety . . . 559
Examples with Explanations . . . 560
1. Wait, fail fast, or wait a bounded time . . . 560
2. Tell "skipped" apart from "failed" with `-E` . . . 560
3. cron: no overlapping runs . . . 561
4. Lock a whole script from the inside (file-descriptor form) . . . 561
5. Self-locking script boilerplate . . . 562
6. Many readers, one writer . . . 562
7. Who is holding the lock? . . . 563
8. `-F`, directories, and `-c` . . . 563
9. `--fcntl` locks are a different lock . . . 564
The trap . . . 564
Notes & Pitfalls . . . 565
The boring rule . . . 566
Related Commands . . . 566
Try this . . . 566
Sources . . . 566

taskset **567**
Overview . . . 567
Mental model . . . 567
Syntax . . . 568
Safety . . . 568
Examples with Explanations . . . 569
1. Read and set: masks, lists, strides . . . 569
2. Change a running process . . . 569
3. See contention instead of guessing . . . 569
4. Pin a whole build or benchmark . . . 570
5. Make the program see fewer CPUs . . . 571

The trap . . . 571
Notes & Pitfalls . . . 572
The boring rule . . . 573
Related Commands . . . 573
Try this . . . 573
Sources . . . 574

sched_ext sub-schedulers: lending CPUs from a BPF root **575**
Overview . . . 575
Mental model . . . 575
How this differs from `taskset` . . . 576
Worked examples (docs-faithful) . . . 576
Case 1: Is sched_ext alive? . . . 576
Case 2: Lending CPUs (capability grant story) . . . 577
Case 3: Partial switch vs whole-machine takeover . . . 577
The trap . . . 578
The boring rule . . . 578
Try this . . . 578
Sources . . . 578

choom and the OOM killer score **579**
Overview . . . 579
Mental model . . . 579
Syntax . . . 581
Safety . . . 582
Examples with Explanations . . . 582
1. Read the three numbers for one process . . . 582
2. Watch the score follow resident memory, and check the formula . . . 583
3. What an unprivileged user may change . . . 585
4. Rank the candidates before the kernel has to . . . 586
5. The same `oom_score_adj` means different things in different domains . . . 587
6. Read the cgroup's OOM counters and group setting . . . 588
7. Make the policy stick: systemd units and Kubernetes (not run here) . . . 588
The trap . . . 589
Notes & Pitfalls . . . 590
The boring rule . . . 590
Related Commands . . . 590
Try this . . . 590
Verification . . . 591
Sources . . . 591

pidfd: process handles that survive PID reuse **592**
Overview . . . 592
Mental model . . . 594
Syntax . . . 594
Safety . . . 595
Examples with Explanations . . . 596
1. One child, one pidfd: inspect, poll, signal, reap, use again . . . 596

2. A pidfd has its own identity (pidfs, Linux 6.9+) . . . 597
3. Wait for a process that is not your child . . . 598
4. Get the pidfd at birth: `clone3(CLONE_PIDFD)` and `pidfd_spawnp()` . . . 599
5. Opening is free, using it is permission-checked . . . 600
6. From a pidfile to a pidfd you can trust . . . 601
7. A supervisor: deadlines, escalation and process groups in one `epoll` loop . . . 602
8. How fast PIDs come around . . . 604
9. Who is on the other end of this socket? . . . 605
The trap . . . 606
Notes & Pitfalls . . . 606
The boring rule . . . 607
Related Commands . . . 607
Try this . . . 607
Verification . . . 608
Sources . . . 608

Focused strace with -e trace Filters **609**

Focused strace with -e trace Filters **610**
Thesis . . . 610
Mental model . . . 610
Worked example . . . 610
Trap . . . 611
Boring rule . . . 611
Try this . . . 611
Sources . . . 611

Everything is an fd: signalfd, timerfd and eventfd in one epoll loop **612**

Everything is an fd: signalfd, timerfd and eventfd in one epoll loop **613**
Overview . . . 613
Mental model . . . 613
Syntax . . . 614
Safety . . . 615
Examples with Explanations . . . 615
1. One loop, four kinds of event . . . 615
2. The kernel's view: `/proc/self/fdinfo`, and `EFD_SEMAPHORE` . . . 618
3. The trap: the blocked mask leaks into children . . . 619
The trap . . . 619
Notes & Pitfalls . . . 620
The boring rule . . . 620
Related Commands . . . 620
Try this . . . 620
Verification . . . 621
Sources . . . 621

How programs get run: execve, binfmt_elf, PT_INTERP and the auxiliary vector **622**

How programs get run: execve, binfmt_elf, PT_INTERP and the auxiliary vector **623**
Overview . . . 623
Mental model . . . 623
Syntax . . . 624
Safety . . . 625
Examples with Explanations . . . 625
1. A PIE, a static binary, and their program headers . . . 625
2. Everything the kernel told the loader: `LD_SHOW_AUXV` . . . 627
3. The loader is a program . . . 628
4. "No such file or directory" for a file that exists . . . 629
The trap: `ldd` runs code . . . 630
Notes and pitfalls . . . 630
The boring rule . . . 630
Related Commands . . . 630
Try this . . . 630
Verification . . . 631
Sources . . . 631

Threads Are Just clone() Flags: What CLONE_VM, FILES, FS and SIGHAND Share **632**

Threads Are Just clone() Flags: What CLONE_VM, FILES, FS and SIGHAND Share **633**
Learning goals . . . 633
The flags in this lab . . . 633
Lab 1: change it in the child, look in the parent . . . 634
Lab 2: what CLONE_THREAD adds . . . 636
Where else these flags show up . . . 637
Try this . . . 637
Sources . . . 638

System Monitoring **639**

Intro **640**

Intro **641**
Commands in this part . . . 641
Suggested starting points . . . 642
Related parts . . . 642

vmstat **643**
Overview . . . 643
Syntax . . . 643
Common Options . . . 643
Key Use Cases . . . 643
Examples with Explanations . . . 644
Live sample (default triage) . . . 644
Timestamped capture for logs . . . 644
Memory summary counters . . . 644
Disk stats mode . . . 644

Active / inactive memory view . . . 644
Correlate with siblings . . . 645
Understanding Output . . . 645
Notes & Pitfalls . . . 646
Related Commands . . . 646
Additional Resources . . . 646

iostat **647**
Overview . . . 647
Syntax . . . 647
Common Options . . . 647
Key Use Cases . . . 647
Examples with Explanations . . . 648
Extended live samples (default incident command) . . . 648
Skip since-boot line explicitly . . . 648
One device (NVMe / disk) . . . 648
CPU only vs devices only . . . 648
Partitions and mapper names . . . 648
Timestamped capture . . . 648
Pair with process-level I/O . . . 649
Understanding Output . . . 649
Notes & Pitfalls . . . 649
Related Commands . . . 650
Additional Resources . . . 650

mpstat **651**
Overview . . . 651
Syntax . . . 651
Common Options . . . 651
Key Use Cases . . . 651
Examples with Explanations . . . 652
Average CPU samples . . . 652
Per-CPU breakdown (primary triage) . . . 652
Specific cores . . . 652
Interrupts . . . 652
JSON for tooling . . . 652
Pair with process CPU . . . 653
Understanding Output . . . 653
Notes & Pitfalls . . . 653
Related Commands . . . 654
Additional Resources . . . 654

sar **655**
Overview . . . 655
Syntax . . . 655
Common Options . . . 655
Key Use Cases . . . 655

Examples with Explanations . . . 656
Live CPU samples . . . 656
Memory and load . . . 656
Disk and I/O rates . . . 656
Network devices . . . 656
Per-CPU history of “now” . . . 656
Today’s history (collection must be enabled) . . . 656
Slice a time window . . . 657
Read another day’s file . . . 657
Enable collection on Ubuntu (sketch) . . . 657
Incident bundle . . . 657
Understanding Output . . . 657
Notes & Pitfalls . . . 658
Related Commands . . . 658
Additional Resources . . . 658

lsof **659**
Overview . . . 659
Syntax . . . 659
Common Options . . . 659
Key Use Cases . . . 659
Examples with Explanations . . . 660
Who listens on port 22? . . . 660
Open files for a PID . . . 660
Deleted files still open (disk full mystery) . . . 660
Files under a mount (before umount) . . . 660
Network connections for a user . . . 661
Command name filter . . . 661
PIDs only for scripting . . . 661
UNIX sockets / pipes (advanced) . . . 661
Understanding Output . . . 661
Notes & Pitfalls . . . 662
Related Commands . . . 662
Additional Resources . . . 662

strace **663**
Overview . . . 663
Syntax . . . 663
Common Options . . . 663
Safety . . . 664
Key Use Cases . . . 664
Examples with Explanations . . . 664
Why did it fail? (file open) . . . 664
Network path of a client . . . 664
Summary profile . . . 664
Attach to a running PID . . . 665
Write a full log . . . 665
Longer strings and FD paths . . . 665

Child processes / build systems . . . 665
Hang diagnosis sketch . . . 665
Understanding Output . . . 665
Notes & Pitfalls . . . 666
Related Commands . . . 666
Additional Resources . . . 666

perf **667**
Overview . . . 667
Syntax . . . 667
Common Subcommands . . . 667
Common Options (record / stat) . . . 667
Key Use Cases . . . 668
Examples with Explanations . . . 668
High-level counters for one command . . . 668
Live profiling . . . 668
Record + report workflow . . . 668
System-wide snapshot . . . 669
One-liner recipes . . . 669
Paranoid settings (understand before changing) . . . 669
Notes & Pitfalls . . . 669
2026-relevant notes . . . 669
Comparison to alternatives . . . 670
Related Commands . . . 670
Additional Resources . . . 670

btop **671**
Overview . . . 671
Syntax . . . 671
Common Options . . . 671
Interactive Keybindings (typical) . . . 671
Key Use Cases . . . 672
Examples with Explanations . . . 672
Start defaults . . . 672
Low-color SSH / serial console . . . 672
Faster or slower refresh . . . 672
One-liner recipes . . . 673
Notes & Pitfalls . . . 673
2026-relevant notes . . . 673
Comparison to alternatives . . . 673
Related Commands . . . 673
Additional Resources . . . 674

ltrace **675**
Overview . . . 675
Syntax . . . 675
Common Options . . . 675
Safety . . . 675

Examples with Explanations . . . 676
Trace a command . . . 676
Summary counts . . . 676
Attach . . . 676
Notes & Pitfalls . . . 676
Related Commands . . . 676
Additional Resources . . . 676

pmap **677**

pmap **678**
Overview . . . 678
Mental model . . . 678
Syntax . . . 679
Safety . . . 679
Examples with Explanations . . . 680
1. Symptom: RSS climbs, the heap does not . . . 682
2. Where: watch the biggest mappings with `pmap -x` . . . 683
3. Why an `LD_PRELOAD` hook sees nothing . . . 684
4. Who: count and attribute syscalls at the kernel boundary . . . 685
5. Who, in production: `bpftrace` on the syscall tracepoints . . . 686
6. Prove it: scripted `gdb` that stops only on the guilty call . . . 687
7. Fix and verify . . . 689
The trap . . . 690
Notes & Pitfalls . . . 690
The boring rule . . . 691
Related Commands . . . 691
Try this . . . 692
Sources . . . 692

Transparent Huge Pages: /sys/kernel/mm/transparent_hugepage, AnonHugePages and khugepaged **693**
Overview . . . 693
Mental model . . . 694
Syntax . . . 695
Safety . . . 696
Examples with Explanations . . . 697
1. Read the whole policy, not just `enabled` . . . 697
2. What `MADV_HUGEPAGE` changes on first touch . . . 698
3. "Touch 1 byte, get 2 MiB": the bloat effect . . . 701
4. Collapse now, collapse later, or never . . . 701
5. `fork()` and copy-on-write: the Redis case . . . 703
6. Which processes hold huge pages, and does it help? . . . 705
7. Make the policy stick (root; not run here) . . . 708
The trap . . . 708
Notes & Pitfalls . . . 709
The boring rule . . . 709
Related Commands . . . 709

Try this . . . 709
Verification . . . 710
Sources . . . 710

Pressure Stall Information: /proc/pressure and cgroup *.pressure 711
Overview . . . 711
Mental model . . . 712
Syntax . . . 713
Safety . . . 713
Examples with Explanations . . . 714
1. Read the three system files as a table . . . 714
2. Make CPU `some` move (processes, not threads) . . . 714
3. Triggers with `poll()` (not run here — write denied) . . . 716
4. What `systemd-oomd` does with memory pressure (docs) . . . 716
The trap . . . 717
Notes & Pitfalls . . . 717
The boring rule . . . 717
Related Commands . . . 717
Try this . . . 718
Verification . . . 718
Sources . . . 718

The vDSO: why clock_gettime() usually never enters the kernel 719
Overview . . . 719
Mental model . . . 720
Syntax . . . 721
Safety . . . 722
Examples with Explanations . . . 722
1. Find the vDSO in a process . . . 722
2. Copy the vDSO out and read it like any shared library . . . 723
3. What the fast path is worth . . . 724
4. Proof that the kernel is not involved: seccomp cannot see it . . . 725
5. Is this host on the fast path? . . . 727
6. Time namespaces: the vDSO answers differently per namespace . . . 728
7. The `LD_PRELOAD` trap: who calls the vDSO directly? . . . 729
The trap . . . 730
Notes & Pitfalls . . . 730
The boring rule . . . 730
Related Commands . . . 731
Try this . . . 731
Verification . . . 731
Sources . . . 732

/proc Is the API: Rebuilding ps Columns from stat, status and smaps_rollup 733

/proc Is the API: Rebuilding ps Columns from stat, status and smaps_rollup 734
Learning goals . . . 734
The files . . . 734

Lab: a mini-ps . . . 735
Reading the numbers . . . 736
Practical rules . . . 737
Try this . . . 737
Sources . . . 737

/proc/PID/pagemap: Reading Your Own Page Tables **738**

/proc/PID/pagemap: Reading Your Own Page Tables **739**
Architecture and Paging Context . . . 739
The `/proc/PID/pagemap` File Format . . . 739
Security and Rowhammer Mitigation . . . 741
Related Interfaces . . . 741
Inspecting Virtual Memory Ranges . . . 742
Decoding `/proc/PID/pagemap` in Python . . . 742
Lab: Observing Demand Paging . . . 744
Lab: Tracking Writes with Soft-Dirty Bits . . . 746
Lab: Fork and Copy-on-Write Exclusivity . . . 747
Pitfalls and Common Mistakes . . . 749
Summary . . . 749
Sources . . . 750

Users and Groups **751**

Intro **752**

Intro **753**
Commands in this part . . . 753
Suggested starting points . . . 754
Related parts . . . 754

sudo **755**
Overview . . . 755
Syntax . . . 755
Common Options . . . 755
Key Use Cases . . . 756
Safety . . . 756
Examples with Explanations . . . 756
Run one admin command . . . 756
Edit a system file safely . . . 756
Root login shell when you need many steps . . . 757
Run as another service user . . . 757
See your privileges . . . 757
Pass env vars explicitly (preferred over `-E`) . . . 757
Credential cache control . . . 757
Non-interactive / scripts . . . 757
Dangerous patterns to avoid . . . 758
Read logs of sudo use . . . 758

Understanding Output . . . 758
Notes & Pitfalls . . . 758
Related Commands . . . 759
Additional Resources . . . 759

visudo **760**
Overview . . . 760
Syntax . . . 760
Common Options . . . 760
Examples with Explanations . . . 760
Edit main sudoers . . . 760
Edit a drop-in (preferred) . . . 761
Check syntax . . . 761
Common policy snippets (examples) . . . 761
Recover from broken sudoers (prevention) . . . 761
Test as user . . . 761
Notes / Pitfalls . . . 761
2026-relevant notes . . . 762
Related Commands . . . 762
Additional Resources . . . 762

useradd **763**
Overview . . . 763
Syntax . . . 763
Common Options . . . 763
Defaults and files . . . 764
Safety . . . 764
Key Use Cases . . . 764
Examples with Explanations . . . 764
Example: human user with home and bash . . . 764
Example: Ubuntu interactive alternative . . . 765
Example: supplementary groups at creation . . . 765
Example: service / system account . . . 765
Example: fixed UID/GID for NFS . . . 765
Example: account expiry . . . 765
Example: custom home location . . . 765
Example: do not create user-private group . . . 766
Example: verify skeleton contents . . . 766
Notes & Pitfalls . . . 766
Related Commands . . . 766
Additional Resources . . . 766

usermod **767**
Overview . . . 767
Syntax . . . 767
Common Options . . . 767
Safety . . . 768
Key Use Cases . . . 768

Examples with Explanations . . . 768
Example: append supplementary groups (`-aG`) . . . 768
Example: wrong vs right group edit . . . 768
Example: change shell . . . 769
Example: lock and unlock . . . 769
Example: move home directory . . . 769
Example: rename login . . . 769
Example: change primary group . . . 769
Example: change UID carefully . . . 770
Example: expiry . . . 770
Example: service account hardening . . . 770
Notes & Pitfalls . . . 770
Related Commands . . . 770
Additional Resources . . . 771

userdel **772**
Overview . . . 772
Syntax . . . 772
Common Options . . . 772
Examples with Explanations . . . 772
Delete account, keep home . . . 772
Delete account and home . . . 773
Refuse when logged in . . . 773
Processes still running . . . 773
Orphan files outside home . . . 773
System users . . . 773
Notes / Pitfalls . . . 773
2026-relevant notes . . . 774
Related Commands . . . 774
Additional Resources . . . 774

passwd **775**
Overview . . . 775
Syntax . . . 775
Common Options . . . 775
Examples with Explanations . . . 775
Change passwords . . . 775
Status . . . 776
Lock / unlock . . . 776
Force change next login . . . 776
Scripted set (controlled environments) . . . 776
Service accounts . . . 776
Notes / Pitfalls . . . 777
2026-relevant notes . . . 777
Related Commands . . . 777
Additional Resources . . . 777

chage **778**
Overview . . . 778
Syntax . . . 778
Common Options . . . 778
Examples with Explanations . . . 778
View policy . . . 778
Force change on next login . . . 779
Set max age 90 days, warn 14 . . . 779
Disable aging . . . 779
Account expiry date . . . 779
Batch service accounts . . . 779
Combine with passwd . . . 779
Notes / Pitfalls . . . 779
2026-relevant notes . . . 780
Related Commands . . . 780
Additional Resources . . . 780

groupadd **781**
Overview . . . 781
Syntax . . . 781
Common Options . . . 781
Related membership tools . . . 781
Safety . . . 782
Key Use Cases . . . 782
Examples with Explanations . . . 782
Example: simple project group . . . 782
Example: fixed GID for NFS fleet . . . 782
Example: system group for a daemon . . . 782
Example: create group then add members (`-aG`) . . . 783
Example: Ubuntu interactive alternative . . . 783
Example: idempotent-ish create in scripts . . . 783
Example: shared directory ownership pattern . . . 783
Example: list and inspect . . . 783
Notes & Pitfalls . . . 784
Related Commands . . . 784
Additional Resources . . . 784

groupmod **785**
Overview . . . 785
Syntax . . . 785
Common Options . . . 785
Examples with Explanations . . . 785
Rename a group . . . 785
Change GID . . . 786
Migrate file ownership after GID change . . . 786
Non-unique GID (rare) . . . 786
Verify . . . 786
Notes / Pitfalls . . . 786

2026-relevant notes . . . 787
Related Commands . . . 787
Additional Resources . . . 787

groupdel **788**
Overview . . . 788
Syntax . . . 788
Common Options . . . 788
Examples with Explanations . . . 788
Delete a group . . . 788
Refuse if primary group . . . 789
Check membership first . . . 789
Automation cleanup . . . 789
LDAP / remote groups . . . 789
Notes / Pitfalls . . . 789
2026-relevant notes . . . 790
Related Commands . . . 790
Additional Resources . . . 790

getent **791**
Overview . . . 791
Syntax . . . 791
Common Databases . . . 791
Common Options . . . 791
Key Use Cases . . . 792
Examples with Explanations . . . 792
Users and groups . . . 792
Hosts . . . 792
Services . . . 792
One-liner recipes . . . 792
Notes & Pitfalls . . . 793
2026-relevant notes . . . 793
Related Commands . . . 793
Additional Resources . . . 793

su **794**
Overview . . . 794
Syntax . . . 794
Common Options . . . 794
Safety . . . 794
Examples with Explanations . . . 795
Become root (login) . . . 795
Run one command as root . . . 795
Switch to a service user . . . 795
Preserve selected environment . . . 795
Notes & Pitfalls . . . 795
Related Commands . . . 795
Additional Resources . . . 796

sudo-rs **797**
Overview . . . 797
Mental model . . . 797
Syntax . . . 798
Safety . . . 798
Examples with Explanations . . . 799
1. Which sudo answers, and why . . . 799
2. Password feedback: what changed and how to turn it off . . . 799
3. The same file, two verdicts . . . 800
4. Rewrite the rejected rules . . . 802
5. Preflight: check files with the target release's visudo, without installing it . . . 803
6. Switch implementation (and back) . . . 805
7. Know which security fixes you have . . . 805
The trap . . . 805
The boring rule . . . 806
Try this . . . 806
Sources . . . 806

Networking 807

Intro 808

Intro 809
Commands in this part . . . 809
Suggested starting points . . . 811
Related parts . . . 812

ip **813**
Overview . . . 813
Syntax . . . 813
Common Options . . . 813
Key Use Cases . . . 813
Safety . . . 814
Examples with Explanations . . . 814
Brief overview . . . 814
Full address list . . . 814
Bring link up/down . . . 814
Add/remove an address . . . 814
Default route . . . 815
Static route . . . 815
Neighbor table (ARP) . . . 815
Statistics . . . 815
Network namespace (quick) . . . 815
Understanding Output . . . 815
Common Usage Patterns . . . 816
Live IP on a server you did not configure . . . 816
Flush all IPv4 addresses on a NIC (careful) . . . 816

Notes & Pitfalls . . . 816
Related Commands . . . 816
Additional Resources . . . 816

ss **817**
Overview . . . 817
Syntax . . . 817
Common Options . . . 817
Key Use Cases . . . 817
Examples with Explanations . . . 818
Listening TCP with processes . . . 818
Established connections . . . 818
Filter by port . . . 818
Filter by address . . . 818
UDP listeners . . . 818
Summary and states . . . 819
UNIX domain sockets . . . 819
"What owns this port?" . . . 819
IPv6 only . . . 819
Understanding Output . . . 819
Notes & Pitfalls . . . 819
Related Commands . . . 820
Additional Resources . . . 820

ping **821**
Overview . . . 821
Syntax . . . 821
Common Options . . . 821
Safety . . . 821
Examples with Explanations . . . 822
Basic connectivity (4 probes) . . . 822
Numeric only (faster, no rDNS noise) . . . 822
IPv6 . . . 822
Quiet summary for scripts . . . 822
Deadline: fail fast in automation . . . 822
Larger payload (rough MTU / fragmentation check) . . . 823
Source from a specific interface . . . 823
Faster sampling (still polite) . . . 823
Compare loss to a hop vs destination . . . 823
Understanding Output . . . 823
Notes & Pitfalls . . . 824
Related Commands . . . 824
Additional Resources . . . 824

traceroute **825**
Overview . . . 825
Syntax . . . 825
Common Options . . . 825

Safety . . . 825
Examples with Explanations . . . 826
Basic path . . . 826
ICMP mode (when UDP probes are blocked) . . . 826
TCP traceroute to HTTPS (firewall-friendly) . . . 826
Limit hop count and speed up failure . . . 826
IPv6 path . . . 826
Compare UDP vs TCP vs ICMP . . . 827
Start mid-path (skip nearby hops) . . . 827
Use a specific source address (multi-homed) . . . 827
Understanding Output . . . 827
Notes & Pitfalls . . . 827
Related Commands . . . 828
Additional Resources . . . 828

mtr **829**
Overview . . . 829
Syntax . . . 829
Common Options . . . 829
Key Use Cases . . . 830
Examples with Explanations . . . 830
Interactive session . . . 830
Non-interactive report (scripts & tickets) . . . 830
TCP path to HTTPS . . . 830
IPv6 and dual-stack . . . 830
One-liner recipes . . . 831
Interpreting loss (important) . . . 831
Understanding Output . . . 831
Notes & Pitfalls . . . 831
2026-relevant notes . . . 832
Comparison to alternatives . . . 832
Related Commands . . . 832
Additional Resources . . . 832

dig **833**
Overview . . . 833
Syntax . . . 833
Common Options . . . 833
Examples with Explanations . . . 834
Basic A record . . . 834
Short output for scripts . . . 834
Query a specific resolver . . . 834
MX and TXT . . . 834
NS and SOA (delegation / zone authority) . . . 834
Reverse DNS (PTR) . . . 834
Answer section only (readable) . . . 835
Trace from the root (find where delegation breaks) . . . 835
Follow CNAME manually . . . 835

TCP query (firewall / large answers) . . . 835
Non-standard port (lab DNS) . . . 835
Batch mode . . . 835
Check mail-related records quickly . . . 835
Understanding Output . . . 836
Notes & Pitfalls . . . 836
Related Commands . . . 836
Additional Resources . . . 836

host **837**
Overview . . . 837
Syntax . . . 837
Common Options . . . 837
Key Use Cases . . . 837
Examples with Explanations . . . 838
Forward lookup . . . 838
Mail and text records . . . 838
Reverse DNS . . . 838
Query a specific server . . . 838
Verbose and script-friendly . . . 838
One-liner recipes . . . 839
Notes & Pitfalls . . . 839
2026-relevant notes . . . 839
Comparison to alternatives . . . 839
Related Commands . . . 840
Additional Resources . . . 840

nslookup **841**
Overview . . . 841
Syntax . . . 841
Common Options / Directives . . . 841
Key Use Cases . . . 841
Examples with Explanations . . . 842
Non-interactive . . . 842
Interactive . . . 842
Prefer modern equivalents . . . 842
One-liner recipes . . . 842
Notes & Pitfalls . . . 842
2026-relevant notes . . . 843
Related Commands . . . 843
Additional Resources . . . 843

doggo **844**
Overview . . . 844
Syntax . . . 844
Common Options (typical) . . . 844
Examples with Explanations . . . 845
Basic queries . . . 845

Choose resolver . . . 845
Reverse / other types . . . 845
Compare with dig . . . 845
Scripting . . . 845
Encrypted DNS (when built-in) . . . 846
Notes / Pitfalls . . . 846
2026-relevant notes . . . 846
Related Commands . . . 846
Additional Resources . . . 846

resolvectl **847**
Overview . . . 847
Syntax . . . 847
Common Commands . . . 847
Key Use Cases . . . 847
Examples with Explanations . . . 848
Status and query . . . 848
Cache . . . 848
Per-link DNS . . . 848
One-liner recipes . . . 848
Notes & Pitfalls . . . 848
2026-relevant notes . . . 849
Related Commands . . . 849
Additional Resources . . . 849

curl **850**
Overview . . . 850
Syntax . . . 850
Common Options . . . 850
Key Use Cases . . . 851
Safety . . . 851
Examples with Explanations . . . 851
Download . . . 851
API GET with pretty JSON . . . 851
POST JSON . . . 851
Headers only / method checks . . . 852
Follow redirects, treat 404 as failure . . . 852
Timing metrics . . . 852
Auth . . . 852
Upload file (multipart) . . . 852
Client certificate (mTLS) . . . 852
Proxy and retries . . . 852
Show response headers + body separately . . . 853
Understanding Output . . . 853
Notes & Pitfalls . . . 853
Related Commands . . . 853
Additional Resources . . . 853

wget **854**
Overview . . . 854
Syntax . . . 854
Common Options . . . 854
Safety . . . 855
Examples with Explanations . . . 855
Simple download (keeps remote name) . . . 855
Save under a specific name . . . 855
Resume interrupted download . . . 855
Quiet download into a directory . . . 855
Rate-limited fetch with retries . . . 856
Multiple URLs from a file . . . 856
Spider: check if URL is reachable . . . 856
Authenticated download (prefer netrc) . . . 856
Custom header (e.g. token) without logging token in shell history carefully . . . 856
One-page offline copy (HTML + assets) . . . 856
Bounded recursive mirror of a docs tree . . . 857
Timestamped re-sync of a single artifact . . . 857
Understanding Output . . . 857
Notes & Pitfalls . . . 857
Related Commands . . . 857
Additional Resources . . . 858

ssh **859**
Overview . . . 859
Syntax . . . 859
Common Options . . . 859
Key Use Cases . . . 859
Safety . . . 860
Examples with Explanations . . . 860
Login . . . 860
Remote command . . . 860
Custom port / options . . . 860
ProxyJump via bastion . . . 860
Local port forward (remote DB to local) . . . 860
Dynamic SOCKS proxy . . . 861
Config file (~/.ssh/config) . . . 861
Verbose auth debug . . . 861
Common Usage Patterns . . . 861
Copy key (once) . . . 861
Escape sequence (client) . . . 861
Notes & Pitfalls . . . 861
Related Commands . . . 862
Additional Resources . . . 862

scp **863**
Overview . . . 863
Syntax . . . 863

Common Options . . . 863
Safety . . . 863
Examples with Explanations . . . 864
Local → remote file . . . 864
Remote → local . . . 864
Directory tree . . . 864
Custom port and key . . . 864
Preserve times/modes . . . 864
Via jump host (bastion) . . . 864
Bandwidth-limited copy . . . 865
Multiple sources to a remote directory . . . 865
Remote-to-remote through your laptop . . . 865
Verbose debug when auth fails . . . 865
Understanding Output . . . 865
Notes & Pitfalls . . . 865
Related Commands . . . 866
Additional Resources . . . 866

rsync **867**
Overview . . . 867
Syntax . . . 867
Common Options . . . 867
Safety . . . 868
Examples with Explanations . . . 868
Dry-run first (always for delete or production) . . . 868
Local archive backup with progress . . . 868
Push project over SSH . . . 868
Pull logs from a server . . . 869
Mirror with excludes . . . 869
Throttled backup (shared link) . . . 869
One filesystem only (skip other mounts) . . . 869
Checksum mode when clocks or touch times lie . . . 869
Show only what would change (itemize) . . . 869
Understanding Output . . . 869
Notes & Pitfalls . . . 870
Related Commands . . . 870
Additional Resources . . . 870

nc **871**
Overview . . . 871
Syntax . . . 871
Common Options (OpenBSD nc on Ubuntu) . . . 871
Safety . . . 871
Examples with Explanations . . . 872
Test if a TCP port is open . . . 872
HTTP GET by hand . . . 872
TLS is not built into classic nc . . . 872
Simple file transfer (lab / trusted LAN) . . . 872

Listen and show inbound banner/data . . . 873
UDP probe . . . 873
Port range scan (light) . . . 873
Pipe program output to a remote listener . . . 873
Bind to a specific local address . . . 873
Chatty timeout for scripts . . . 873
Understanding Output . . . 874
Notes & Pitfalls . . . 874
Related Commands . . . 874
Additional Resources . . . 874

tcpdump **875**
Overview . . . 875
Syntax . . . 875
Common Options . . . 875
Safety . . . 876
Filter expressions (BPF) essentials . . . 876
Examples with Explanations . . . 876
See anything on default route interface . . . 876
HTTPS-related traffic for one host . . . 876
DNS queries and responses . . . 876
Only SYN packets (connection attempts) . . . 877
Capture full packets to file for Wireshark . . . 877
Read a pcap back in the terminal . . . 877
HTTP cleartext headers (lab only) . . . 877
ICMP (ping) traffic . . . 877
Traffic involving a local port (e.g. reverse proxy) . . . 877
Limit and timestamp deltas . . . 877
Exclude SSH so your session does not flood the capture . . . 878
Understanding Output . . . 878
Notes & Pitfalls . . . 878
Related Commands . . . 878
Additional Resources . . . 879

nmap **880**
Command Overview . . . 880
Syntax . . . 880
Common Options . . . 880
Key Use Cases . . . 881
Examples with Explanations . . . 881
1. Basic Scan . . . 881
2. Intensive Scan . . . 881
3. Stealth Scan . . . 882
4. Service Version Detection . . . 882
5. OS Detection . . . 882
Understanding Nmap . . . 882
Scan Types . . . 882
Port Selection . . . 883

Host Discovery . . . 883
Version Detection . . . 883
Script Scanning . . . 884
Output Formats . . . 884
Performance Tuning . . . 884
Advanced Techniques . . . 885
Firewall Evasion . . . 885
NSE Scripts Examples . . . 885
Best Practices . . . 886
Related Commands . . . 886
Additional Resources . . . 886

openssl **887**
Overview . . . 887
Syntax . . . 887
Common Subcommands & Options . . . 887
s_client (frequent flags) . . . 887
x509 (frequent flags) . . . 888
Safety . . . 888
Examples with Explanations . . . 888
Version and providers . . . 888
Probe a website's certificate . . . 889
Full chain and leaf details . . . 889
SHA-256 fingerprint . . . 889
Inspect a local PEM cert . . . 889
STARTTLS (mail example) . . . 889
Generate lab key + self-signed cert . . . 889
CSR for an external CA . . . 890
PEM DER . . . 890
Digests (alternate to *sum tools) . . . 890
Random secrets . . . 890
Quick HTTPS port check (banner/handshake only) . . . 890
Notes . . . 890
Related Commands . . . 891
Additional Resources . . . 891

nmcli **892**
Overview . . . 892
Syntax . . . 892
Common Commands . . . 892
Key Use Cases . . . 892
Examples with Explanations . . . 893
Overview . . . 893
Bounce a connection . . . 893
Static Ethernet profile . . . 893
DHCP Ethernet . . . 893
Wi-Fi . . . 893
Reload and DNS insight . . . 893

One-liner recipes . . . 894
Notes & Pitfalls . . . 894
2026-relevant notes . . . 894
Related Commands . . . 894
Additional Resources . . . 894

ufw **895**
Overview . . . 895
Syntax . . . 895
Common Commands . . . 895
Key Use Cases . . . 895
Safety . . . 896
Examples with Explanations . . . 896
Safe first enable . . . 896
Web and custom ports . . . 896
From management network only . . . 896
App profiles . . . 896
Numbered delete / limit . . . 896
One-liner recipes . . . 897
Notes & Pitfalls . . . 897
2026-relevant notes . . . 897
Related Commands . . . 897
Additional Resources . . . 897

nft **898**
Overview . . . 898
Syntax . . . 898
Common Commands . . . 898
Safety . . . 899
Examples with Explanations . . . 899
See what is active . . . 899
Minimal inet filter sketch (lab / understanding) . . . 899
Sets for many ports/IPs . . . 900
Delete a rule by handle . . . 900
Save and restore . . . 900
Persist on Ubuntu (high level) . . . 900
Coexistence checks . . . 900
JSON for scripts . . . 901
Notes . . . 901
Related Commands . . . 901
Additional Resources . . . 901

iptables **902**
Overview . . . 902
Syntax . . . 902
Tables & common chains . . . 902
Common Options . . . 902
Safety . . . 903

Examples with Explanations . . . 903
List . . . 903
Allow SSH, HTTP; default drop (sketch) . . . 903
NAT masquerade (router) . . . 903
Save (Debian/Ubuntu example) . . . 903
Prefer UFW on Ubuntu . . . 904
Notes . . . 904
Related Commands . . . 904

ifconfig **905**
Overview . . . 905
Syntax . . . 905
Common Options / forms . . . 905
Examples with Explanations . . . 906
Display . . . 906
Preferred modern equivalents . . . 906
Bring up/down . . . 906
Set address (legacy) . . . 906
MTU . . . 906
Install on Debian/Ubuntu if missing . . . 907
Notes / Pitfalls . . . 907
2026-relevant notes . . . 907
Related Commands . . . 907
Additional Resources . . . 907

netstat **908**
Overview . . . 908
Syntax . . . 908
Common Options . . . 908
Examples with Explanations . . . 909
Listening services . . . 909
Established connections . . . 909
Routing . . . 909
Interfaces / stats . . . 909
Continuous . . . 909
Translate old → new . . . 909
Notes / Pitfalls . . . 910
2026-relevant notes . . . 910
Related Commands . . . 910
Additional Resources . . . 910

ftp **911**
Overview . . . 911
Syntax . . . 911
Common Options / commands . . . 911
Examples with Explanations . . . 912
Interactive session . . . 912
Prefer SFTP instead . . . 912

lftp (better modern client) . . . 912
Scripted (discouraged for secrets) . . . 912
Firewall note . . . 912
Notes / Pitfalls . . . 913
2026-relevant notes . . . 913
Related Commands . . . 913
Additional Resources . . . 913

telnet **914**
Overview . . . 914
Syntax . . . 914
Common Options . . . 914
Examples with Explanations . . . 914
Port open check . . . 914
Speak HTTP manually . . . 915
SMTP banner grab (authorized testing only) . . . 915
Escape and quit . . . 915
Prefer TLS-aware tools for HTTPS/SMTPS . . . 915
Notes / Pitfalls . . . 915
2026-relevant notes . . . 916
Related Commands . . . 916
Additional Resources . . . 916

ssh-keygen **917**
Overview . . . 917
Syntax . . . 917
Common Options . . . 917
Safety . . . 917
Examples with Explanations . . . 918
Create an Ed25519 key . . . 918
RSA for legacy only . . . 918
Fingerprint . . . 918
Fix / inspect known_hosts . . . 918
Public key from private . . . 918
Related Commands . . . 918
Additional Resources . . . 919

ssh-copy-id **920**
Overview . . . 920
Syntax . . . 920
Common Options . . . 920
Safety . . . 920
Examples with Explanations . . . 921
Default identity . . . 921
Explicit key and port . . . 921
Via ProxyJump . . . 921
Manual equivalent (when tool missing) . . . 921
Notes & Pitfalls . . . 921

Related Commands . . . 921
Additional Resources . . . 922

sftp **923**
Overview . . . 923
Syntax . . . 923
Common Options . . . 923
Examples with Explanations . . . 923
Interactive session . . . 923
Batch download . . . 924
Jump host . . . 924
Non-interactive one-shot via ssh . . . 924
Notes & Pitfalls . . . 924
Related Commands . . . 924
Additional Resources . . . 925

ethtool **926**
Overview . . . 926
Syntax . . . 926
Common Options . . . 926
Safety . . . 926
Examples with Explanations . . . 927
Link status . . . 927
Driver info . . . 927
Statistics . . . 927
Identify physical port . . . 927
Offloads . . . 927
Persist settings . . . 927
Related Commands . . . 927
Additional Resources . . . 928

firewall-cmd **929**
Overview . . . 929
Syntax . . . 929
Common Options / Patterns . . . 929
Safety . . . 929
Examples with Explanations . . . 930
Inventory . . . 930
Allow HTTP permanently . . . 930
Open a custom port . . . 930
Rich rule example . . . 930
Runtime vs permanent . . . 930
Notes & Pitfalls . . . 930
Related Commands . . . 931
Additional Resources . . . 931

ssh-agent / ssh-add **932**
Overview . . . 932

Syntax . . . 932
Common Options . . . 932
ssh-agent . . . 932
ssh-add . . . 933
Client config that cooperates with the agent . . . 933
Safety . . . 934
Key Use Cases . . . 934
Examples with Explanations . . . 934
Example: start an agent and add a key . . . 934
Example: confirm the environment is wired . . . 935
Example: time-limited identity for a change window . . . 935
Example: list, remove one key, or clear all . . . 935
Example: AddKeysToAgent on first use . . . 935
Example: prefer ProxyJump over agent forwarding . . . 936
Example: destination-constrained key (OpenSSH 8.9+) . . . 936
Example: lock the agent while you step away . . . 936
Example: kill a manually started agent . . . 937
Example: systemd user unit sketch (persistent per login) . . . 937
Understanding Output . . . 937
Notes & Pitfalls . . . 937
Common Usage Patterns . . . 938
Related Commands . . . 938
Additional Resources . . . 938

Certificate renewal audit (64-day and 45-day certificates) 939

Certificate renewal audit (64-day and 45-day certificates) 940
Overview . . . 940
Mental model . . . 940
Syntax . . . 941
Safety . . . 942
Examples with Explanations . . . 942
1. What does your CA offer, and what do certificates live today? . . . 942
2. Compute an ARI certificate ID and check it against the RFC . . . 943
3. Ask the CA when to renew . . . 944
4. What each renewal policy does as lifetimes shrink . . . 945
5. Monitor by fraction used, not days left . . . 946
6. Audit a host for hard-coded lifetimes . . . 948
The trap . . . 950
Notes & Pitfalls . . . 951
The boring rule . . . 952
Related Commands . . . 952
Try this . . . 952
Sources . . . 953

poll vs epoll vs io_uring 954

poll vs epoll vs io_uring **955**
Overview . . . 955
Mental model . . . 955
Syntax . . . 956
Worked examples . . . 956
Case 1: `poll` with a timeout (portable readiness) . . . 956
Case 2: `epoll` edge-triggered on a pipe . . . 957
Case 3: `io_uring` completion with a NOP . . . 958
Case 4: When to stop climbing the ladder . . . 959
The trap . . . 959
The boring rule . . . 960
Try this . . . 960
Sources . . . 960

Scheduling 961

Intro 962

Intro 963
Commands in this part . . . 963
Suggested starting points . . . 963
Related parts . . . 963

cron 964
Overview . . . 964
Syntax . . . 964
Common schedule patterns . . . 964
Key Use Cases . . . 965
Safety . . . 965
Examples with Explanations . . . 965
User crontab (via crontab tool) . . . 965
Example job lines . . . 965
System-wide locations (Ubuntu) . . . 966
Environment inside cron . . . 966
Debugging “it didn’t run” . . . 966
Prefer systemd timers when… . . . 967
Understanding Output . . . 967
Notes & Pitfalls . . . 967
Related Commands . . . 967
Additional Resources . . . 968

crontab 969
Overview . . . 969
Syntax . . . 969
Common Options . . . 969
Key Use Cases . . . 969
Safety . . . 970

Examples with Explanations . . . 970
List and edit . . . 970
Install from a file (idempotent in config mgmt) . . . 970
Append a line carefully . . . 970
Remove with confirmation . . . 971
Root manages another user . . . 971
Validate schedule fragments . . . 971
Common job examples . . . 971
Understanding Output . . . 971
Notes & Pitfalls . . . 972
Related Commands . . . 972
Additional Resources . . . 972

anacron **973**
Overview . . . 973
Syntax . . . 973
Common Options . . . 973
Configuration . . . 973
Examples with Explanations . . . 974
Status / integration . . . 974
Force a run (careful) . . . 974
Update timestamps only . . . 974
Put a job in daily.d style . . . 974
Prefer systemd timer for new work . . . 974
Notes / Pitfalls . . . 975
2026-relevant notes . . . 975
Related Commands . . . 975
Additional Resources . . . 975

at **976**
Overview . . . 976
Syntax . . . 976
Common Options . . . 976
Time expressions . . . 977
Examples with Explanations . . . 977
Interactive job . . . 977
One-liner from pipe . . . 977
From file . . . 977
Queue management . . . 977
batch . . . 978
Ensure service . . . 978
Notes / Pitfalls . . . 978
2026-relevant notes . . . 978
Related Commands . . . 978
Additional Resources . . . 978

Logging **979**

Intro **980**

Intro **981**
Commands in this part . . . 981
Suggested starting points . . . 981
Related parts . . . 981

dmesg **982**
Overview . . . 982
Syntax . . . 982
Common Options . . . 982
Examples with Explanations . . . 983
Recent kernel messages . . . 983
Follow live (plug hardware) . . . 983
Errors and warnings . . . 983
OOM investigation . . . 983
USB / disk events . . . 983
Clear buffer (careful) . . . 983
Permissions . . . 984
Notes / Pitfalls . . . 984
2026-relevant notes . . . 984
Related Commands . . . 984
Additional Resources . . . 984

logger **985**
Overview . . . 985
Syntax . . . 985
Common Options . . . 985
Examples with Explanations . . . 986
Simple messages . . . 986
From scripts . . . 986
Pipe multi-line output . . . 986
Priority levels . . . 986
View on systemd . . . 986
stderr mirror . . . 987
Structured-ish with journald (when supported) . . . 987
Notes / Pitfalls . . . 987
2026-relevant notes . . . 987
Related Commands . . . 987
Additional Resources . . . 987

logrotate **988**
Overview . . . 988
Syntax . . . 988
Common Options . . . 988
Common config directives . . . 988
Key Use Cases . . . 989

Examples with Explanations . . . 989
Dry-run entire system config . . . 989
Force one drop-in (verbose) . . . 989
Inspect state . . . 989
Example application drop-in . . . 990
copytruncate when the app won't reopen . . . 990
size-based emergency pattern . . . 990
When does it run? (Ubuntu) . . . 991
Disk pressure triage with logs . . . 991
Understanding Output . . . 991
Notes & Pitfalls . . . 991
Related Commands . . . 992
Additional Resources . . . 992

Hardware **993**

Intro **994**

Intro **995**
Commands in this part . . . 995
Suggested starting points . . . 995
Related parts . . . 995

lshw **997**
Overview . . . 997
Syntax . . . 997
Common Options . . . 997
Key Use Cases . . . 997
Examples with Explanations . . . 998
Quick overview . . . 998
Network and disks . . . 998
Memory and CPU . . . 998
Export . . . 998
One-liner recipes . . . 998
Notes & Pitfalls . . . 999
2026-relevant notes . . . 999
Related Commands . . . 999
Additional Resources . . . 999

lspci **1000**
Overview . . . 1000
Syntax . . . 1000
Common Options . . . 1000
Key Use Cases . . . 1000
Examples with Explanations . . . 1001
Everyday list . . . 1001
Drivers and modules . . . 1001
Network / GPU filters . . . 1001

One device, deep detail . . . 1001
Tree view . . . 1001
Machine-readable . . . 1002
ID database refresh . . . 1002
Correlate with sysfs / ethtool . . . 1002
Understanding Output . . . 1002
Notes & Pitfalls . . . 1003
Related Commands . . . 1003
Additional Resources . . . 1003

dmidecode **1004**
Overview . . . 1004
Syntax . . . 1004
Common Options . . . 1004
Key Use Cases . . . 1004
Examples with Explanations . . . 1005
System identity . . . 1005
BIOS / firmware . . . 1005
Memory population . . . 1005
Processor summary . . . 1005
Baseboard and chassis . . . 1005
Inventory one-liners for scripts . . . 1006
Compare with sysfs (no full dump) . . . 1006
Understanding Output . . . 1006
Notes & Pitfalls . . . 1006
Related Commands . . . 1007
Additional Resources . . . 1007

hdparm **1008**
Overview . . . 1008
Syntax . . . 1008
Common Options . . . 1008
Examples with Explanations . . . 1009
Identify . . . 1009
Power status . . . 1009
Read-only info queries (safer) . . . 1009
Benchmarks (I/O heavy; maintenance window) . . . 1009
Write cache (know the risk) . . . 1009
Reread partitions . . . 1009
Notes / Pitfalls . . . 1010
2026-relevant notes . . . 1010
Related Commands . . . 1010
Additional Resources . . . 1010

Reading NUMA Topology without numactl **1011**

Reading NUMA Topology without numactl **1012**
Thesis . . . 1012

Mental model . . . 1012
Worked examples . . . 1012
Trap . . . 1013
Boring rule . . . 1013
Try this . . . 1013
Sources . . . 1013

Storage and Filesystems **1014**

Intro **1015**

Intro **1016**
Commands in this part . . . 1016
Suggested starting points . . . 1018
Related parts . . . 1018

lsblk **1019**
Overview . . . 1019
Syntax . . . 1019
Common Options . . . 1019
Key Use Cases . . . 1020
Examples with Explanations . . . 1020
Everyday inventory . . . 1020
UUID-ready view for fstab . . . 1020
Hide loop noise (snaps) . . . 1020
JSON for scripts . . . 1021
Topology / rotational / transport . . . 1021
One device subtree . . . 1021
Align with blkid and mounts . . . 1021
Empty or new disk check . . . 1021
Understanding Output . . . 1021
Notes & Pitfalls . . . 1022
Related Commands . . . 1022
Additional Resources . . . 1022

blkid **1023**
Overview . . . 1023
Syntax . . . 1023
Common Options . . . 1023
Key Use Cases . . . 1023
Examples with Explanations . . . 1024
List all probed devices . . . 1024
One device . . . 1024
Export form (shell-friendly) . . . 1024
UUID only (fstab snippets) . . . 1024
Lookup by UUID or LABEL . . . 1024
Compare with lsblk . . . 1025
After mkfs verification . . . 1025

udev property style . . . 1025
Understanding Output . . . 1025
Notes & Pitfalls . . . 1026
Related Commands . . . 1026
Additional Resources . . . 1026

findmnt **1027**
Overview . . . 1027
Syntax . . . 1027
Common Options . . . 1027
Key Use Cases . . . 1028
Examples with Explanations . . . 1028
Full tree . . . 1028
df-like sizes . . . 1028
What holds this path? . . . 1028
By source or target . . . 1028
Script-friendly . . . 1029
Busy mount debugging . . . 1029
Verify fstab . . . 1029
NFS / bind focus . . . 1029
Understanding Output . . . 1029
Notes & Pitfalls . . . 1030
Related Commands . . . 1030
Additional Resources . . . 1030

fdisk **1031**
Overview . . . 1031
Syntax . . . 1031
Common Options . . . 1031
Safety . . . 1032
Key Use Cases . . . 1032
Examples with Explanations . . . 1032
List all disks (read-only) . . . 1032
Confirm target before editing . . . 1032
Interactive GPT data disk (sketch) . . . 1033
Kernel re-read after write . . . 1033
Next steps after partitioning . . . 1033
Scripted dump / restore (sfdisk) . . . 1033
EFI system partition type (boot disk work) . . . 1034
Understanding Output . . . 1034
Notes & Pitfalls . . . 1034
Related Commands . . . 1034
Additional Resources . . . 1034

mkfs **1035**
Overview . . . 1035
Syntax . . . 1035
Common Options . . . 1035

Safety . . . 1036
Key Use Cases . . . 1036
Examples with Explanations . . . 1036
Verify target, then ext4 data disk . . . 1036
XFS data volume . . . 1036
FAT32 for USB interoperability . . . 1037
Reduce ext* reserved space on large data disks . . . 1037
LVM logical volume . . . 1037
Capture UUID for fstab . . . 1037
Whole-disk vs partition . . . 1037
Understanding Output . . . 1038
Notes & Pitfalls . . . 1038
Related Commands . . . 1038
Additional Resources . . . 1038

fsck **1039**
Overview . . . 1039
Syntax . . . 1039
Common Options . . . 1039
Safety . . . 1039
Key Use Cases . . . 1040
Examples with Explanations . . . 1040
Report-only on unmounted ext4 . . . 1040
Interactive repair . . . 1040
Auto-yes repair (after backup / when you accept loss) . . . 1040
Dry-run which helpers would run . . . 1041
Force ext4 check with progress . . . 1041
Root filesystem . . . 1041
XFS path . . . 1041
Mount and verify after clean check . . . 1041
Understanding Output / Exit codes . . . 1042
Notes & Pitfalls . . . 1042
Related Commands . . . 1042
Additional Resources . . . 1043

mount **1044**
Overview . . . 1044
Syntax . . . 1044
Common Options . . . 1044
Key Use Cases . . . 1044
Safety . . . 1045
Examples with Explanations . . . 1045
Inventory . . . 1045
Mount a partition by UUID . . . 1045
Read-only mount (forensics / recovery) . . . 1045
NFS . . . 1046
CIFS / SMB . . . 1046
Bind mounts . . . 1046

tmpfs . . . 1046
Remount . . . 1046
fstab-driven . . . 1046
Overlay (container-style) . . . 1047
Understanding Output . . . 1047
Notes & Pitfalls . . . 1047
Related Commands . . . 1047
Additional Resources . . . 1048

umount **1049**
Overview . . . 1049
Syntax . . . 1049
Common Options . . . 1049
Key Use Cases . . . 1049
Safety . . . 1050
Examples with Explanations . . . 1050
Unmount by mountpoint (preferred) . . . 1050
Unmount by device . . . 1050
Confirm before and after . . . 1050
Busy mount diagnosis . . . 1050
Lazy unmount . . . 1051
Force NFS (last resort) . . . 1051
Remount read-only fallback . . . 1051
Multiple mounts / bind trees . . . 1051
Before fsck . . . 1051
Understanding Output . . . 1051
Notes & Pitfalls . . . 1052
Related Commands . . . 1052
Additional Resources . . . 1052

dd **1053**
Overview . . . 1053
Syntax . . . 1053
Common Options . . . 1053
Safety . . . 1054
Examples with Explanations . . . 1054
List devices before any write . . . 1054
Write an ISO to a USB stick (destructive to the stick) . . . 1054
Create a raw disk image (backup a USB) . . . 1055
Restore an image to a USB . . . 1055
Sparse empty file (test disk image) . . . 1055
Sample first megabyte (forensics / header peek) . . . 1055
Damaged media (best-effort copy) . . . 1056
Progress with pv (when installed) . . . 1056
Wipe a USB free-space / reuse (destructive) . . . 1056
Notes . . . 1056
Related Commands . . . 1056
Additional Resources . . . 1057

sync **1058**
Overview . . . 1058
Syntax . . . 1058
Common Options . . . 1058
Safety . . . 1058
Examples with Explanations . . . 1059
Flush everything (classic) . . . 1059
After writing a USB with dd . . . 1059
Sync specific file (GNU) . . . 1059
Safe removable-media workflow . . . 1059
In scripts before reboot/power events . . . 1059
Drop caches (admin debug — not for production routine) . . . 1060
Notes . . . 1060
Related Commands . . . 1060
Additional Resources . . . 1060

losetup **1061**
Overview . . . 1061
Syntax . . . 1061
Common Options . . . 1061
Key Use Cases . . . 1061
Examples with Explanations . . . 1062
Attach and list . . . 1062
Mount ISO . . . 1062
Partitions inside image . . . 1062
Detach . . . 1062
One-liner recipes . . . 1062
Notes & Pitfalls . . . 1063
2026-relevant notes . . . 1063
Related Commands . . . 1063
Additional Resources . . . 1063

duf **1064**
Overview . . . 1064
Syntax . . . 1064
Common Options . . . 1064
Examples with Explanations . . . 1065
Everyday . . . 1065
Inodes . . . 1065
JSON . . . 1065
Hide noise . . . 1065
Compare with df . . . 1065
Threshold awareness . . . 1065
Notes / Pitfalls . . . 1066
2026-relevant notes . . . 1066
Related Commands . . . 1066
Additional Resources . . . 1066

dust **1067**
Overview . . . 1067
Syntax . . . 1067
Common Options . . . 1067
Examples with Explanations . . . 1068
Find big directories . . . 1068
Apparent size . . . 1068
Ignore noise . . . 1068
File types . . . 1068
Compare with du / ncdu . . . 1068
Min size filter . . . 1068
Scripts: stick to du . . . 1068
Notes / Pitfalls . . . 1069
2026-relevant notes . . . 1069
Related Commands . . . 1069
Additional Resources . . . 1069

parted **1070**
Overview . . . 1070
Syntax . . . 1070
Common Commands (interactive or CLI) . . . 1070
Safety . . . 1070
Examples with Explanations . . . 1071
Print table . . . 1071
GPT with one partition (scripted) . . . 1071
Interactive . . . 1071
Notes & Pitfalls . . . 1071
Related Commands . . . 1071
Additional Resources . . . 1072

wipefs **1073**
Overview . . . 1073
Syntax . . . 1073
Common Options . . . 1073
Safety . . . 1073
Examples with Explanations . . . 1074
Probe signatures . . . 1074
Erase all signatures . . . 1074
After wipe — new filesystem . . . 1074
Related Commands . . . 1074
Additional Resources . . . 1074

smartctl **1075**
Overview . . . 1075
Syntax . . . 1075
Common Options . . . 1075
Safety . . . 1075

Examples with Explanations . . . 1076
Health summary . . . 1076
Full report . . . 1076
Short self-test . . . 1076
Watch failure predictors . . . 1076
Scan devices . . . 1076
Related Commands . . . 1076
Additional Resources . . . 1077

pvs **1078**
Overview . . . 1078
Syntax . . . 1078
Common Options . . . 1078
Examples with Explanations . . . 1078
List physical volumes . . . 1078
Initialize a new PV (related workflow) . . . 1079
Free space across PVs . . . 1079
Notes & Pitfalls . . . 1079
Related Commands . . . 1079
Additional Resources . . . 1079

vgs **1080**
Overview . . . 1080
Syntax . . . 1080
Common Options . . . 1080
Examples with Explanations . . . 1080
List volume groups . . . 1080
Create VG from PVs (workflow) . . . 1080
Free space before creating an LV . . . 1081
Notes & Pitfalls . . . 1081
Related Commands . . . 1081
Additional Resources . . . 1081

lvs **1082**
Overview . . . 1082
Syntax . . . 1082
Common Options . . . 1082
Examples with Explanations . . . 1082
List LVs . . . 1082
Create and format an LV (workflow) . . . 1082
Grow an LV (then filesystem) . . . 1083
Snapshots (classic thick) . . . 1083
Understanding Output . . . 1083
Related Commands . . . 1083
Additional Resources . . . 1083

cryptsetup **1084**
Overview . . . 1084

Syntax . . . 1084
Common Commands . . . 1084
Safety . . . 1084
Examples with Explanations . . . 1085
Format and open (data disk) . . . 1085
Status and close . . . 1085
Add a second passphrase . . . 1085
Header backup . . . 1085
fstab / crypttab sketch . . . 1085
Notes & Pitfalls . . . 1085
Related Commands . . . 1086
Additional Resources . . . 1086

resize2fs **1087**
Overview . . . 1087
Syntax . . . 1087
Common Options . . . 1087
Safety . . . 1087
Examples with Explanations . . . 1088
Grow to fill LV after lvextend . . . 1088
Explicit size . . . 1088
Offline shrink (dangerous — backup first) . . . 1088
Notes & Pitfalls . . . 1088
Related Commands . . . 1088
Additional Resources . . . 1089

tune2fs **1090**
Overview . . . 1090
Syntax . . . 1090
Common Options . . . 1090
Safety . . . 1090
Examples with Explanations . . . 1091
Inspect . . . 1091
Label and UUID . . . 1091
Reserved space (big data disks) . . . 1091
Disable frequent forced fsck by mount count . . . 1091
Related Commands . . . 1091
Additional Resources . . . 1091

ncdu **1092**
Overview . . . 1092
Syntax . . . 1092
Common Options . . . 1092
Examples with Explanations . . . 1092
Scan root (one FS) . . . 1092
Home directory . . . 1093
Export for later / remote analysis . . . 1093
Notes & Pitfalls . . . 1093

Related Commands . . . 1093
Additional Resources . . . 1093

RWF_DONTCACHE (uncached buffered I/O) 1094

RWF_DONTCACHE (uncached buffered I/O) 1095
Overview . . . 1095
Mental model . . . 1096
Syntax . . . 1096
Safety . . . 1097
Examples with Explanations . . . 1097
1. Probe: does this kernel and filesystem accept the flag? . . . 1097
2. Streaming read with a correct fallback . . . 1099
3. Streaming write: bound dirty memory . . . 1101
4. `fio`: fails rather than falls back . . . 1103
The trap . . . 1103
Notes & Pitfalls . . . 1104
The boring rule . . . 1105
Related Commands . . . 1105
Try this . . . 1106
Sources . . . 1106

io_uring (batched asynchronous I/O) 1107

io_uring (batched asynchronous I/O) 1108
Overview . . . 1108
Mental model . . . 1108
Syntax . . . 1109
Safety . . . 1109
Examples with Explanations . . . 1110
1. Three reads, one system call . . . 1110
2. Is io_uring allowed here? . . . 1111
3. A fallback that keeps the program working . . . 1112
4. Benchmarks with fio . . . 1113
The trap . . . 1113
Notes & Pitfalls . . . 1114
The boring rule . . . 1114
Related Commands . . . 1114
Try this . . . 1115
Sources . . . 1115

I/O Schedulers: none, mq-deadline, kyber, bfq 1116

I/O Schedulers: none, mq-deadline, kyber, bfq 1117
Thesis . . . 1117
Mental model . . . 1117
Worked examples . . . 1117
Case 1: list devices and the bracketed active scheduler . . . 1117
Trap . . . 1118

Boring rule . . . 1118
Try this . . . 1118
Sources . . . 1118

tmpfs as Scratch Space (/dev/shm) 1119

tmpfs as Scratch Space (/dev/shm) 1120
Thesis . . . 1120
Mental model . . . 1120
Worked examples . . . 1120
Case 1: findmnt, write, df . . . 1120
Trap . . . 1121
Boring rule . . . 1121
Try this . . . 1121
Sources . . . 1121

io_uring Basics without the Ceremony 1122

io_uring Basics without the Ceremony 1123
Thesis . . . 1123
Mental model . . . 1123
Worked example . . . 1123
Trap . . . 1124
Boring rule . . . 1124
Try this . . . 1124
Sources . . . 1124

Bind mounts and mount propagation: shared, slave, private, unbindable 1125

Bind mounts and mount propagation: shared, slave, private, unbindable 1126
Overview . . . 1126
Mental model . . . 1126
Syntax . . . 1127
Safety . . . 1127
Examples with Explanations . . . 1128
1. Shared: two places, one peer group . . . 1128
2. Slave: receive, never send . . . 1128
3. Private: a frozen copy . . . 1129
4. Unbindable: no copies at all . . . 1129
5. `--bind` vs `--rbind` . . . 1130
6. Read-only binds are not recursive by default . . . 1130
The trap: a new mount namespace that is still shared . . . 1130
Notes and pitfalls . . . 1131
The boring rule . . . 1131
Related Commands . . . 1131
Try this . . . 1132
Verification . . . 1132
Sources . . . 1132

Terminals and Multiplexers **1133**

Intro **1134**

Intro **1135**
Commands in this part . . . 1135
Suggested starting points . . . 1135
Related parts . . . 1135

tmux **1137**
Overview . . . 1137
Syntax . . . 1137
Install (Ubuntu) . . . 1137
Prefix key . . . 1137
Common CLI commands . . . 1138
Key Use Cases . . . 1138
Examples with Explanations . . . 1139
Example: create, detach, reattach . . . 1139
Example: start detached job session . . . 1139
Example: nested SSH-safe habit . . . 1139
Example: splits for ops . . . 1139
Example: scripted window layout . . . 1139
Example: send keys to a session (automation) . . . 1140
Example: shared socket / pair-ish access . . . 1140
Example: minimal `~/.tmux.conf` . . . 1140
Example: copy mode (scrollback) . . . 1140
Safety . . . 1141
Notes & Pitfalls . . . 1141
Related Commands . . . 1141
Additional Resources . . . 1141

screen **1142**
Overview . . . 1142
Syntax . . . 1142
Install (Ubuntu) . . . 1142
Prefix key . . . 1142
Common CLI options . . . 1143
Key Use Cases . . . 1143
Examples with Explanations . . . 1143
Example: create, detach, resume . . . 1143
Example: create-or-attach . . . 1144
Example: start detached command . . . 1144
Example: list and wipe zombies . . . 1144
Example: multi-attach . . . 1144
Example: send commands from outside . . . 1144
Example: log session output . . . 1144
Example: minimal `~/.screenrc` . . . 1145
Example: vertical split note . . . 1145

Safety . . . 1145
Notes & Pitfalls . . . 1145
Related Commands . . . 1145
Additional Resources . . . 1146

byobu **1147**
Overview . . . 1147
Syntax . . . 1147
Common Options . . . 1147
Function Keys . . . 1147
Key Use Cases . . . 1148
Examples with Explanations . . . 1148
Example 1: Start Session . . . 1148
Example 2: Named Session . . . 1148
Example 3: List Sessions . . . 1148
Common Usage Patterns . . . 1149
Security Considerations . . . 1149
Related Commands . . . 1149
Additional Resources . . . 1149
Best Practices . . . 1149
Configuration . . . 1150
Troubleshooting . . . 1150

tmate **1151**
Overview . . . 1151
Syntax . . . 1151
Common usage patterns . . . 1151
Examples with Explanations . . . 1151
Start a session . . . 1151
Capture SSH URL for a colleague . . . 1152
Read-only vs read-write links . . . 1152
Config sketch . . . 1152
End session . . . 1152
Prefer local tmux when not sharing . . . 1152
Notes / Pitfalls . . . 1152
2026-relevant notes . . . 1153
Related Commands . . . 1153
Additional Resources . . . 1153

abduco **1154**
Overview . . . 1154
Syntax . . . 1154
Common Options . . . 1154
Examples with Explanations . . . 1155
Create / attach . . . 1155
Run a specific command . . . 1155
List and reattach . . . 1155
Detach . . . 1155

Read-only observe . . . 1155
Pair with dvtm . . . 1155
vs tmux . . . 1155
Notes / Pitfalls . . . 1156
2026-relevant notes . . . 1156
Related Commands . . . 1156
Additional Resources . . . 1156

Text and Pipes **1157**

Intro **1158**

Intro **1159**
Commands in this part . . . 1159
Suggested starting points . . . 1160
Related parts . . . 1160

grep **1161**
Overview . . . 1161
Syntax . . . 1161
Common Options . . . 1161
Key Use Cases . . . 1162
Examples with Explanations . . . 1162
Basic . . . 1162
Extended regex . . . 1162
Fixed string (fast, safe) . . . 1162
Recursive code search . . . 1162
Context . . . 1162
Files that match . . . 1162
Exit status in scripts . . . 1163
Only matching part . . . 1163
Invert . . . 1163
Understanding Output . . . 1163
Common Usage Patterns . . . 1163
Pipe from journalctl . . . 1163
Count HTTP status codes . . . 1163
Notes & Pitfalls . . . 1163
Related Commands . . . 1164
Additional Resources . . . 1164

sed **1165**
Overview . . . 1165
Syntax . . . 1165
Common Options . . . 1165
Common script commands . . . 1165
Safety . . . 1166
Key Use Cases . . . 1166

Examples with Explanations . . . 1166
Substitute first match per line . . . 1166
Global substitute . . . 1166
Extended regex . . . 1167
Delete matching lines . . . 1167
Print only rewritten matches . . . 1167
In-place with backup . . . 1167
Line addresses and ranges . . . 1167
Multiple expressions . . . 1167
Use match in replacement . . . 1168
Change delimiter when paths have slashes . . . 1168
Hold space: join two lines (advanced) . . . 1168
Dry-run pattern for risky edits . . . 1168
Understanding Output . . . 1168
Notes & Pitfalls . . . 1168
Related Commands . . . 1169
Additional Resources . . . 1169

awk **1170**
Overview . . . 1170
Syntax . . . 1170
Common Options . . . 1170
Key Use Cases . . . 1170
Examples with Explanations . . . 1171
Print columns . . . 1171
Filter rows . . . 1171
Sum and average a column . . . 1171
Change separators (CSV → TSV) . . . 1171
Last field and field count . . . 1171
Multiple patterns / counters . . . 1172
Build a frequency report . . . 1172
Extract IP from mixed lines . . . 1172
Skip header, process the rest . . . 1172
Script-friendly exit status . . . 1172
Field splitting on multi-character FS (gawk) . . . 1172
Understanding Output . . . 1173
Notes & Pitfalls . . . 1173
Related Commands . . . 1173
Additional Resources . . . 1173

cut **1174**
Overview . . . 1174
Syntax . . . 1174
Common Options . . . 1174
Field/Character Lists . . . 1174
Key Use Cases . . . 1175
Examples with Explanations . . . 1175
Example 1: Extract Fields . . . 1175

Example 2: Extract Characters . . . 1175
Example 3: Custom Delimiter . . . 1175
Working with Different Delimiters . . . 1175
Common Usage Patterns . . . 1175
Advanced Operations . . . 1176
Character vs Field Extraction . . . 1176
Performance Analysis . . . 1176
Related Commands . . . 1176
Additional Resources . . . 1177
Best Practices . . . 1177
Common Patterns . . . 1177
Integration Examples . . . 1177
Troubleshooting . . . 1178

sort **1179**
Overview . . . 1179
Syntax . . . 1179
Common Options . . . 1179
Key syntax (`-k`) . . . 1180
Examples with Explanations . . . 1180
Lexicographic sort . . . 1180
Numeric sort . . . 1180
Human-readable sizes . . . 1180
Version numbers . . . 1180
Sort by a column (whitespace) . . . 1180
CSV-ish second column (simple cases) . . . 1181
Unique lines . . . 1181
Frequency report (sort | uniq -c | sort) . . . 1181
Check if already sorted . . . 1181
Safe "sort in place" . . . 1181
Merge pre-sorted shards . . . 1181
Stable sort by key, preserve secondary order . . . 1181
NUL-safe with find . . . 1182
Randomize lines . . . 1182
Understanding Output . . . 1182
Notes & Pitfalls . . . 1182
Related Commands . . . 1182
Additional Resources . . . 1183

uniq **1184**
Overview . . . 1184
Syntax . . . 1184
Common Options . . . 1184
Examples with Explanations . . . 1184
Remove adjacent duplicates . . . 1184
The usual "unique lines in file" pattern . . . 1185
Count occurrences . . . 1185
Top IPs from a log (field extract → sort → uniq -c → sort) . . . 1185

Only lines that appear more than once . . . 1185
Only lines that appear exactly once . . . 1185
Case-insensitive uniqueness . . . 1185
Ignore leading fields when comparing . . . 1186
Compare only a prefix of each line . . . 1186
Show every line that is part of a duplicate group . . . 1186
NUL-separated path lists . . . 1186
Count total unique lines (quick) . . . 1186
Understanding Output . . . 1186
Notes & Pitfalls . . . 1187
Related Commands . . . 1187
Additional Resources . . . 1187

tr **1188**
Overview . . . 1188
Syntax . . . 1188
Common Options . . . 1188
Character Sets . . . 1188
Key Use Cases . . . 1189
Examples with Explanations . . . 1189
Example 1: Uppercase Conversion . . . 1189
Example 2: Delete Characters . . . 1189
Example 3: Replace Characters . . . 1189
Example 4: Squeeze Repeated Characters . . . 1189
Common Usage Patterns . . . 1190
Character Ranges . . . 1190
Advanced Usage . . . 1190
Text Processing . . . 1191
Performance Analysis . . . 1191
Related Commands . . . 1191
Best Practices . . . 1191
Data Cleaning . . . 1191
File Processing . . . 1192
Integration Examples . . . 1192
Scripting Applications . . . 1192
Special Characters . . . 1193
Troubleshooting . . . 1193
Security Applications . . . 1193
Performance Optimization . . . 1194
Real-world Examples . . . 1194

head **1195**
Overview . . . 1195
Syntax . . . 1195
Common Options . . . 1195
Key Use Cases . . . 1195
Examples with Explanations . . . 1196
Example 1: Default Usage . . . 1196

Example 2: Specific Line Count . . . 1196
Example 3: Multiple Files . . . 1196
Example 4: Byte Count . . . 1196
Common Usage Patterns . . . 1196
Advanced Usage . . . 1197
Performance Analysis . . . 1197
Related Commands . . . 1197
Best Practices . . . 1197
Integration Examples . . . 1197
Scripting Applications . . . 1198
Additional Examples . . . 1198

tail **1199**
Overview . . . 1199
Syntax . . . 1199
Common Options . . . 1199
Key Use Cases . . . 1199
Examples with Explanations . . . 1200
Example 1: Default Usage . . . 1200
Example 2: Follow Log File . . . 1200
Example 3: Specific Line Count . . . 1200
Example 4: Multiple Files . . . 1200
Follow Mode Options . . . 1200
Common Usage Patterns . . . 1201
Advanced Usage . . . 1201
Performance Analysis . . . 1201
Related Commands . . . 1201
Best Practices . . . 1202
Integration Examples . . . 1202
Log Rotation Handling . . . 1202
Scripting Applications . . . 1202
System Monitoring . . . 1203
Additional Examples . . . 1203

wc **1204**
Overview . . . 1204
Syntax . . . 1204
Common Options . . . 1204
Default Output Format . . . 1204
Key Use Cases . . . 1205
Examples with Explanations . . . 1205
Example 1: Basic Count . . . 1205
Example 2: Lines Only . . . 1205
Example 3: Multiple Files . . . 1205
Understanding Counts . . . 1205
Common Usage Patterns . . . 1206
Advanced Usage . . . 1206
Pipeline Integration . . . 1206

Performance Analysis . . . 1207
Related Commands . . . 1207
Additional Resources . . . 1207
Best Practices . . . 1207
Scripting Examples . . . 1207
Character Encoding . . . 1208
Common Patterns . . . 1208
Integration Examples . . . 1208
Troubleshooting . . . 1209
Real-world Applications . . . 1209

tee **1210**
Overview . . . 1210
Syntax . . . 1210
Common Options . . . 1210
Safety . . . 1210
Examples with Explanations . . . 1211
Save output and still see it . . . 1211
Append to a running log . . . 1211
Split to multiple files . . . 1211
Continue the pipeline after saving . . . 1211
Write files that need root (classic sudo tee) . . . 1211
Same pattern: append as root . . . 1211
Suppress terminal output but keep the file . . . 1212
Capture and process with jq later . . . 1212
Timestamped session log with a second process . . . 1212
Combine with process substitution (bash) for dual processing . . . 1212
Preserve exit status of the left side (bash pipefail) . . . 1212
Understanding Output . . . 1212
Notes & Pitfalls . . . 1213
Related Commands . . . 1213
Additional Resources . . . 1213

xargs **1214**
Overview . . . 1214
Syntax . . . 1214
Common Options . . . 1214
Safety . . . 1215
Examples with Explanations . . . 1215
Basic: echo words as args . . . 1215
Delete files listed by find (NUL-safe) . . . 1215
One invocation per item with placeholder . . . 1215
Limit batch size . . . 1215
Parallel compression . . . 1216
Show commands (`-t`) while learning . . . 1216
Skip empty input (GNU) . . . 1216
Lines as items (GNU `-d`) . . . 1216
Placeholders with multiple uses . . . 1216

From a file directly . . . 1216
Combine with git . . . 1217
Understanding Output . . . 1217
Notes & Pitfalls . . . 1217
Related Commands . . . 1217
Additional Resources . . . 1217

diff **1218**
Overview . . . 1218
Syntax . . . 1218
Common Options . . . 1218
Output Formats . . . 1218
Key Use Cases . . . 1219
Examples with Explanations . . . 1219
Example 1: Basic Comparison . . . 1219
Example 2: Unified Format . . . 1219
Example 3: Directory Comparison . . . 1219
Understanding Output . . . 1219
Common Usage Patterns . . . 1220
Advanced Options . . . 1220
Patch Creation . . . 1220
Performance Analysis . . . 1220
Related Commands . . . 1221
Additional Resources . . . 1221
Best Practices . . . 1221
Directory Comparison . . . 1221
Integration Examples . . . 1221
Scripting Applications . . . 1222
Special Cases . . . 1222
Troubleshooting . . . 1222
Output Redirection . . . 1223
Color Output . . . 1223

comm **1224**
Overview . . . 1224
Syntax . . . 1224
Common Options . . . 1224
Examples with Explanations . . . 1224
Basic three columns . . . 1224
Only in first / second / both . . . 1225
Users differences . . . 1225
Package set ops . . . 1225
Check order . . . 1225
Process substitution . . . 1225
Empty markers . . . 1225
Notes / Pitfalls . . . 1226
2026-relevant notes . . . 1226
Related Commands . . . 1226

Additional Resources 1226

paste **1227**
Overview 1227
Syntax 1227
Common Options 1227
Examples with Explanations 1227
Side by side 1227
Custom delimiter 1228
Serialize lines of one file 1228
Combine with cut/seq 1228
Join data columns from commands 1228
NUL-safe 1228
Classic: make CSV from columns 1228
Notes / Pitfalls 1228
2026-relevant notes 1229
Related Commands 1229
Additional Resources 1229

column **1230**
Overview 1230
Syntax 1230
Common Options 1230
Examples with Explanations 1230
Table-ize whitespace 1230
Custom separators 1231
Headers (newer) 1231
Right-align numbers 1231
JSON (when available) 1231
Pretty fstab view 1231
Pipelines 1231
Notes / Pitfalls 1231
2026-relevant notes 1232
Related Commands 1232
Additional Resources 1232

jq **1233**
Overview 1233
Syntax 1233
Common Options 1233
Key Use Cases 1233
Examples with Explanations 1234
Pretty-print stdin 1234
Extract a field 1234
Map a list of objects to TSV 1234
Filter array elements 1234
Nested paths and optional keys 1235
Update / reshape objects 1235

Flatten and walk arrays . . . 1235
Build JSON from shell (safe) . . . 1235
Merge two JSON files . . . 1235
Exit status for CI . . . 1236
Compact logs / NDJSON . . . 1236
Understanding Output . . . 1236
Notes & Pitfalls . . . 1236
Related Commands . . . 1236
Additional Resources . . . 1237

pv **1238**
Overview . . . 1238
Syntax . . . 1238
Common Options . . . 1238
Examples with Explanations . . . 1239
Copy with progress . . . 1239
dd-style disk image . . . 1239
Rate-limit a transfer . . . 1239
Multi-stage labeled bars . . . 1239
Related Commands . . . 1239
Additional Resources . . . 1239

Packages 1240

Intro 1241

Intro 1242
Commands in this part . . . 1242
Suggested starting points . . . 1242
Related parts . . . 1243

apt **1244**
Overview . . . 1244
Syntax . . . 1244
Common Commands . . . 1244
Key Use Cases . . . 1244
Safety . . . 1245
Examples with Explanations . . . 1245
Refresh and upgrade . . . 1245
Install / remove . . . 1245
Search and inspect . . . 1245
Version pinning at install . . . 1246
Fix broken dependencies . . . 1246
Clean caches . . . 1246
Noninteractive (scripts/CI) . . . 1246
Simulate / dry-run . . . 1246
Which package owns a file? . . . 1247
Notes & Pitfalls . . . 1247

Related Commands . . . 1247
Additional Resources . . . 1247

dpkg **1248**
Overview . . . 1248
Syntax . . . 1248
Common Actions . . . 1248
Common Options . . . 1248
Key Use Cases . . . 1249
Examples with Explanations . . . 1249
Example 1: Install Package . . . 1249
Example 2: Remove Package . . . 1249
Example 3: List Files . . . 1249
Common Usage Patterns . . . 1250
Security Considerations . . . 1250
Related Commands . . . 1250
Additional Resources . . . 1250
Best Practices . . . 1250
Package States . . . 1251
Troubleshooting . . . 1251

dnf **1252**
Overview . . . 1252
Syntax . . . 1252
Common commands . . . 1252
Common options . . . 1253
Safety . . . 1253
Key Use Cases . . . 1253
Examples with Explanations . . . 1254
Example: refresh and update . . . 1254
Example: install and remove . . . 1254
Example: search and inspect . . . 1254
Example: which package owns a file . . . 1254
Example: repository visibility . . . 1254
Example: transaction history . . . 1255
Example: download only . . . 1255
Example: groups and modules . . . 1255
Example: security-focused updates . . . 1255
Example: clean caches . . . 1255
Example: scripted noninteractive . . . 1256
Notes & Pitfalls . . . 1256
Related Commands . . . 1256
Additional Resources . . . 1256

rpm **1257**
Overview . . . 1257
Syntax . . . 1257
Common Options . . . 1257

Query Options . . . 1257
Key Use Cases . . . 1258
Examples with Explanations . . . 1258
Example 1: Install Package . . . 1258
Example 2: Query Package . . . 1258
Example 3: Verify Package . . . 1258
Common Usage Patterns . . . 1259
Security Considerations . . . 1259
Related Commands . . . 1259
Additional Resources . . . 1259
Best Practices . . . 1259
Package Information . . . 1260
Troubleshooting . . . 1260

yum **1261**
Overview . . . 1261
Syntax . . . 1261
Common Commands . . . 1261
Common Options . . . 1261
Examples with Explanations . . . 1262
Everyday . . . 1262
Search and info . . . 1262
Repos . . . 1262
History / undo . . . 1262
Clean cache . . . 1263
Prefer dnf when present . . . 1263
Groups . . . 1263
Notes / Pitfalls . . . 1263
2026-relevant notes . . . 1263
Related Commands . . . 1263
Additional Resources . . . 1264

snap **1265**
Overview . . . 1265
Syntax . . . 1265
Common Commands . . . 1265
Safety . . . 1266
Key Use Cases . . . 1266
Examples with Explanations . . . 1266
Inventory and search . . . 1266
Install / remove (server-oriented) . . . 1266
Channels / tracks . . . 1267
Services and logs . . . 1267
Hold refreshes (ops) . . . 1267
Revisions / disk use . . . 1267
Notes & Pitfalls . . . 1267
Related Commands . . . 1268
Additional Resources . . . 1268

Services and Runtime **1269**

Intro **1270**

Intro **1271**
Commands in this part . . . 1271
Suggested starting points . . . 1272
Related parts . . . 1272

systemctl **1273**
Overview . . . 1273
Syntax . . . 1273
Common Commands . . . 1273
Key Use Cases . . . 1273
Safety . . . 1274
Examples with Explanations . . . 1274
Status and logs peek . . . 1274
Start / enable . . . 1274
Restart vs reload . . . 1274
Failed units . . . 1275
Script checks . . . 1275
After editing unit files . . . 1275
Drop-in override (preferred over editing vendor files) . . . 1275
Timers (cron alternative) . . . 1275
Dependencies and reverse deps . . . 1276
User services . . . 1276
Isolate targets (careful) . . . 1276
Understanding Output . . . 1276
Notes & Pitfalls . . . 1276
Related Commands . . . 1276
Additional Resources . . . 1277

journalctl **1278**
Overview . . . 1278
Syntax . . . 1278
Common Options . . . 1278
Key Use Cases . . . 1279
Examples with Explanations . . . 1279
Unit logs . . . 1279
Follow . . . 1279
Previous boot / crash triage . . . 1279
Kernel / dmesg-like . . . 1279
Time window . . . 1279
Priority . . . 1280
JSON for jq . . . 1280
Field matches . . . 1280
Disk usage and vacuum . . . 1280
Catalog / explain . . . 1280

Understanding Output . . . 1280
Notes & Pitfalls . . . 1281
Related Commands . . . 1281
Additional Resources . . . 1281

sysctl **1282**
Overview . . . 1282
Syntax . . . 1282
Common Options . . . 1282
Key Use Cases . . . 1282
Examples with Explanations . . . 1283
Read . . . 1283
Write (runtime) . . . 1283
Persist . . . 1283
One-liner recipes . . . 1283
Notes & Pitfalls . . . 1283
2026-relevant notes . . . 1284
Related Commands . . . 1284
Additional Resources . . . 1284

shutdown **1285**
Overview . . . 1285
Syntax . . . 1285
Time forms . . . 1285
Common options (`shutdown`) . . . 1285
systemd / loginctl cousins . . . 1286
Safety . . . 1286
Key Use Cases . . . 1286
Examples with Explanations . . . 1287
Example: power off now . . . 1287
Example: reboot now . . . 1287
Example: delayed reboot with message . . . 1287
Example: wall-clock schedule . . . 1287
Example: cancel . . . 1287
Example: message only (drill) . . . 1287
Example: systemd reboot with wall message . . . 1288
Example: check last reboots . . . 1288
Example: inhibit shutdown (awareness) . . . 1288
Example: forced paths (last resort) . . . 1288
Understanding Output . . . 1288
Notes & Pitfalls . . . 1288
Related Commands . . . 1289
Additional Resources . . . 1289

reboot **1290**
Overview . . . 1290
Syntax . . . 1290
Common Options . . . 1290

Examples with Explanations . . . 1290
Normal reboot . . . 1290
Scheduled . . . 1291
With broadcast message . . . 1291
Inhibitors . . . 1291
Firmware setup (UEFI) . . . 1291
After kernel install . . . 1291
Containers / hosts . . . 1291
Notes / Pitfalls . . . 1291
2026-relevant notes . . . 1292
Related Commands . . . 1292
Additional Resources . . . 1292

podman **1293**
Overview . . . 1293
Syntax . . . 1293
Common Commands . . . 1293
Key Use Cases . . . 1293
Examples with Explanations . . . 1294
Run . . . 1294
Build . . . 1294
Rootless tips . . . 1294
systemd . . . 1294
One-liner recipes . . . 1295
Notes & Pitfalls . . . 1295
2026-relevant notes . . . 1295
Related Commands . . . 1295
Additional Resources . . . 1295

systemd-run **1296**
Overview . . . 1296
Syntax . . . 1296
Common Options . . . 1296
Safety . . . 1296
Examples with Explanations . . . 1297
One-shot with wait . . . 1297
CPU/memory limits . . . 1297
Transient timer . . . 1297
User scope . . . 1297
Interactive shell in a scope . . . 1297
Related Commands . . . 1297
Additional Resources . . . 1298

systemd-analyze **1299**
Overview . . . 1299
Syntax . . . 1299
Common Subcommands . . . 1299

Examples with Explanations . . . 1299
Boot timings . . . 1299
Plot . . . 1300
Verify units after editing . . . 1300
Calendar expressions (timers) . . . 1300
Security profile (where available) . . . 1300
Notes & Pitfalls . . . 1300
Related Commands . . . 1300
Additional Resources . . . 1301

loginctl **1302**
Overview . . . 1302
Syntax . . . 1302
Common Commands . . . 1302
Examples with Explanations . . . 1302
Who is logged in . . . 1302
Session details . . . 1303
Enable lingering for user services . . . 1303
Terminate a stale session . . . 1303
Notes & Pitfalls . . . 1303
Related Commands . . . 1303
Additional Resources . . . 1303

modprobe **1304**
Overview . . . 1304
Syntax . . . 1304
Common Options . . . 1304
Safety . . . 1304
Examples with Explanations . . . 1305
Load / unload . . . 1305
Dependencies . . . 1305
Module parameters . . . 1305
Blacklist (persist) . . . 1305
List available modules . . . 1305
Related Commands . . . 1305
Additional Resources . . . 1306

systemd-creds **1307**
Overview . . . 1307
Syntax . . . 1307
Safety . . . 1308
Examples with Explanations . . . 1308
Encrypt a file and load it in a transient service . . . 1308
Emit `SetCredentialEncrypted=` for a unit file . . . 1308
Relative names and credstore search paths . . . 1308
List / show in context . . . 1309
Application read pattern . . . 1309
TPM / host key notes . . . 1309

Related Commands . . . 1309
Try this . . . 1310
Sources . . . 1310

unshare / nsenter **1311**
Overview . . . 1311
Syntax . . . 1311
Safety . . . 1313
Examples with Explanations . . . 1313
1. Rootless "root": a user namespace . . . 1313
2. Run a command with no network . . . 1314
3. Private hostname (UTS) . . . 1314
4. New PID namespace: be PID 1 . . . 1315
5. Private mounts: a throwaway tmpfs . . . 1315
6. Time namespace: fake uptime . . . 1316
7. Enter a running sandbox with `nsenter` . . . 1316
8. Debug a container or sandboxed service with host tools . . . 1317
9. Persistent namespaces and `ip netns` . . . 1318
10. List namespaces . . . 1319
Notes & Pitfalls . . . 1319
Related Commands . . . 1319
Try this . . . 1320
Sources . . . 1320

systemd-sysext **1321**
Overview . . . 1321
Mental model . . . 1322
Syntax . . . 1322
Safety . . . 1323
Examples with Explanations . . . 1323
1. Inventory and status . . . 1323
2. Build a directory-based extension . . . 1324
3. `extension-release` matching . . . 1324
4. Mutability (systemd 256+) . . . 1325
5. OCI / image delivery pattern (openSUSE-style) . . . 1325
6. confext vs sysext . . . 1326
The trap . . . 1326
The boring rule . . . 1326
Try this . . . 1327
Sources . . . 1327

systemd early-boot units **1328**
Overview . . . 1328
Mental model . . . 1328
Syntax . . . 1329
Safety . . . 1330
Examples with Explanations . . . 1330
1. Inventory: who opts out of default dependencies? . . . 1330

2. Confirm sysinit sits before basic . . . 1330
3. `systemd-analyze verify` on a draft unit (no install) . . . 1331
4. Reading a suspicious unit . . . 1331
5. What early boot is *not* . . . 1331
The trap . . . 1332
The boring rule . . . 1332
Try this . . . 1332
Sources . . . 1332

Reading cgroup v2 CPU and Memory Limits **1333**

Reading cgroup v2 CPU and Memory Limits **1334**
Thesis . . . 1334
Mental model . . . 1334
Worked example . . . 1334
Trap . . . 1335
Boring rule . . . 1335
Try this . . . 1335
Sources . . . 1335

Containers from Scratch: unshare, mount and pivot_root **1336**

Containers from Scratch: unshare, mount and pivot_root **1337**
Thesis . . . 1337
Environment verification . . . 1337
The plan . . . 1337
The container script . . . 1338
Running the container . . . 1339
What the host sees . . . 1340
Why pivot_root instead of chroot? . . . 1341
Requirements for pivot_root(2) . . . 1341
The PATH gotcha . . . 1341
What a real container runtime adds . . . 1342
Common mistakes . . . 1342
Exercises . . . 1343
Summary . . . 1343
Sources . . . 1343

Shell Commands **1344**

Intro **1345**

Intro **1346**
Commands in this part . . . 1346
Suggested starting points . . . 1346
Related parts . . . 1346

echo **1348**
Overview . . . 1348
Syntax . . . 1348
Common Options . . . 1348
Examples with Explanations . . . 1348
Print text . . . 1348
No newline . . . 1349
Escapes (bash) . . . 1349
Redirect / append . . . 1349
Pipe into other commands . . . 1349
Show a path or command you are about to run . . . 1349
Notes . . . 1349
Related Commands . . . 1350
Additional Resources . . . 1350

printf **1351**
Overview . . . 1351
Syntax . . . 1351
Common Format Pieces . . . 1351
Examples with Explanations . . . 1351
Safe newline print . . . 1351
No trailing newline . . . 1352
Columns . . . 1352
Zero-pad numbers . . . 1352
Interpret escapes in data . . . 1352
Hex / bytes (light) . . . 1352
Build a file . . . 1352
Arbitrary strings that look like options . . . 1353
Notes . . . 1353
Related Commands . . . 1353
Additional Resources . . . 1353

history **1354**
Overview . . . 1354
Syntax . . . 1354
Common Options . . . 1354
Examples with Explanations . . . 1355
Show recent commands . . . 1355
Search history (interactive bash) . . . 1355
Re-run by number / last command . . . 1355
Append now (multi-session) . . . 1355
Clear session history . . . 1355
Delete one awkward line . . . 1355
Where history is stored . . . 1356
Notes . . . 1356
Related Commands . . . 1356
Additional Resources . . . 1356

type / command **1357**
Overview . . . 1357
Syntax . . . 1357
Common Options . . . 1357
type (bash) . . . 1357
command . . . 1357
Examples with Explanations . . . 1358
What is this name? . . . 1358
Machine-readable kind . . . 1358
Path only . . . 1358
Run without alias/function . . . 1358
Compare to which / command -v . . . 1358
Debug “wrong binary” . . . 1359
Notes . . . 1359
Related Commands . . . 1359
Additional Resources . . . 1359

export **1360**
Overview . . . 1360
Syntax . . . 1360
Common Options . . . 1360
Examples with Explanations . . . 1360
Set for children . . . 1360
One-shot without permanent export . . . 1361
Show exports . . . 1361
Unexport / unset . . . 1361
Common operator variables . . . 1361
Persist for your user (pointer) . . . 1361
Notes . . . 1361
Related Commands . . . 1362
Additional Resources . . . 1362

time **1363**
Overview . . . 1363
Syntax . . . 1363
Common Options . . . 1363
Bash builtin . . . 1363
GNU `/usr/bin/time` (package `time`) . . . 1363
Examples with Explanations . . . 1364
Basic timing . . . 1364
Time a pipeline (bash builtin) . . . 1364
GNU verbose . . . 1364
Portable three lines . . . 1364
Custom format . . . 1364
Compare two approaches . . . 1364
Bash TIMEFORMAT . . . 1365
Notes . . . 1365
Related Commands . . . 1365

Additional Resources . . . 1365

pushd / popd / dirs **1366**
Overview . . . 1366
Syntax . . . 1366
Common Options . . . 1366
dirs . . . 1366
pushd / popd . . . 1366
Examples with Explanations . . . 1367
Jump and return . . . 1367
Nested hops . . . 1367
List the stack . . . 1367
Swap two project roots . . . 1367
Numbered rotation . . . 1368
Clear . . . 1368
Notes . . . 1368
Related Commands . . . 1368
Additional Resources . . . 1368

Security in Shell Scripts **1369**

Intro **1370**

Intro **1371**
Commands in this part . . . 1371
Suggested starting points . . . 1371
Related parts . . . 1372

getenforce **1373**
Overview . . . 1373
Syntax . . . 1373
Examples with Explanations . . . 1373
Check mode . . . 1373
Related status . . . 1373
Notes & Pitfalls . . . 1373
Related Commands . . . 1374
Additional Resources . . . 1374

setenforce **1375**
Overview . . . 1375
Syntax . . . 1375
Safety . . . 1375
Examples with Explanations . . . 1375
Temporary permissive for debugging . . . 1375
Persist mode (config — not setenforce) . . . 1376
Related Commands . . . 1376
Additional Resources . . . 1376

restorecon **1377**
Overview . . . 1377
Syntax . . . 1377
Common Options . . . 1377
Examples with Explanations . . . 1377
Fix a web tree after rsync/mv . . . 1377
Dry-run . . . 1377
Single file after manual copy . . . 1378
Notes & Pitfalls . . . 1378
Related Commands . . . 1378
Additional Resources . . . 1378

getcap / setcap **1379**
Overview . . . 1379
Syntax . . . 1379
Examples with Explanations . . . 1379
Inspect . . . 1379
Allow binding privileged ports (example) . . . 1379
Remove capabilities . . . 1380
Safety . . . 1380
Notes & Pitfalls . . . 1380
Related Commands . . . 1380
Additional Resources . . . 1380

gpg **1381**
Overview . . . 1381
Syntax . . . 1381
Common Options . . . 1381
Safety . . . 1382
Key Use Cases . . . 1382
Examples with Explanations . . . 1383
Example: install and confirm the toolchain . . . 1383
Example: generate a key pair (interactive) . . . 1383
Example: list keys and show fingerprints . . . 1383
Example: export and import a public key . . . 1383
Example: symmetric encrypt and decrypt a file . . . 1384
Example: public-key encrypt and decrypt . . . 1384
Example: detached sign and verify (ASCII) . . . 1384
Example: verify a signed checksum list (download integrity) . . . 1385
Example: armor / dearmor a keyring blob for apt-style keyrings . . . 1385
Understanding Output . . . 1385
Notes & Pitfalls . . . 1386
Related Commands . . . 1386
Additional Resources . . . 1386

age **1387**
Overview . . . 1387
Syntax . . . 1387

Common Options . . . 1387
Safety . . . 1388
Key Use Cases . . . 1388
Examples with Explanations . . . 1389
Example: install and confirm the toolchain . . . 1389
Example: generate a native identity . . . 1389
Example: identity → recipient string . . . 1389
Example: encrypt and decrypt a file (native recipient) . . . 1390
Example: multiple recipients and a recipients file . . . 1390
Example: passphrase encrypt (no key file) . . . 1390
Example: passphrase-protected identity file . . . 1391
Example: encrypt to an SSH public key . . . 1391
Example: compose with tar (directory archive) . . . 1391
Example: encrypt using an identity file's matching recipient . . . 1391
Understanding Output . . . 1392
Notes & Pitfalls . . . 1392
Related Commands . . . 1392
Additional Resources . . . 1393

io_uring and seccomp: filters per opcode 1394

io_uring and seccomp: filters per opcode 1395
Overview . . . 1395
Mental model . . . 1395
Syntax . . . 1396
Coarse allowlist (widely available): restrictions . . . 1396
Fine-grained (Linux 7.0+ / liburing helpers) . . . 1396
Safety . . . 1397
Examples with Explanations . . . 1397
1. Opcode allowlist with `IORING_REGISTER_RESTRICTIONS` (runs here) . . . 1397
2. Allow only `AF_INET` sockets (docs listing — needs Linux 7.0+) . . . 1399
3. Allowlist pattern: allow NOP, deny everything else . . . 1400
The trap . . . 1400
The boring rule . . . 1401
Try this . . . 1401
Verification . . . 1401
Sources . . . 1401

Landlock from the command line: setpriv 1402

Landlock from the command line: setpriv 1403
Overview . . . 1403
Mental model . . . 1404
Syntax . . . 1405
Safety . . . 1406
Examples with Explanations . . . 1406
1. Probe the ABI (runs here) . . . 1406
2. Read-only `/usr`, writable scratch (runs here) . . . 1407

3. Landlock does not outrank DAC (runs here) . . . 1409
4. Block TCP connect and bind (runs here) . . . 1409
5. Allow TCP connect to 443 only (runs here) . . . 1411
6. `EPERM`: missing `no_new_privs`, and scoped signals (runs here) . . . 1413
The trap . . . 1414
The boring rule . . . 1415
Try this . . . 1415
Verification . . . 1415
Sources . . . 1415

Unprivileged User Namespaces: What "root" Inside One Can Reach 1416

Unprivileged User Namespaces: What "root" Inside One Can Reach 1417
Overview . . . 1417
Mental model . . . 1418
Examples with Explanations . . . 1418
1. What namespace-root can and cannot do (runs here) . . . 1418
2. The network stack is now yours to configure (runs here) . . . 1420
3. Limits are per namespace, and a sandbox can deny nesting (runs here) . . . 1421
Choosing a policy (not run here) . . . 1421
The trap . . . 1422
The boring rule . . . 1422
Try this . . . 1423
Sources . . . 1423

OpenSSH on musl (sshole): detect, mitigate, upgrade 1424

OpenSSH on musl (sshole): detect, mitigate, upgrade 1425
Overview . . . 1425
Vulnerability Mechanism . . . 1426
musl `crypt(3)` behavior . . . 1426
Shadow record locking conventions . . . 1426
OpenSSH comparison flaw . . . 1426
Mental model & Scope . . . 1427
Safety . . . 1428
Auditing and Detection . . . 1428
Checking C library and OpenSSH version . . . 1428
Auditing SSH daemon configuration . . . 1428
Auditing locked shadow accounts . . . 1429
Mitigation and Remediation . . . 1429
Immediate Mitigation: Disabling Password Authentication . . . 1429
Remediation: Upgrading OpenSSH on Alpine Linux . . . 1430
Automated Auditing Script . . . 1430
Common Mistakes . . . 1431
Summary . . . 1432
Sources . . . 1432

Appendices **1433**

Intro **1434**

Intro **1435**
Commands in this part . . . 1435
Suggested starting points . . . 1435
Related parts . . . 1435

Ubuntu Cheatsheet **1436**

Ubuntu Cheatsheet **1437**
System . . . 1437
System Information . . . 1437
System Monitoring and Management . . . 1437
Running Commands . . . 1437
Service Management . . . 1437
Cron Jobs and Scheduling . . . 1437
Files . . . 1438
File Management . . . 1438
Directory Navigation . . . 1438
File Permissions and Ownership . . . 1438
Searching and Finding . . . 1438
Archiving and Compression . . . 1438
Text Editing and Processing . . . 1438
Packages . . . 1439
Package Management (APT) . . . 1439
Package Management (Snap) . . . 1439
Users & Groups . . . 1439
User Management . . . 1439
Group Management . . . 1439
Networking . . . 1440
Network Configuration . . . 1440
Netplan Configuration . . . 1440
Firewall Management . . . 1440
SSH and Remote Access . . . 1440
LXD . . . 1440
Basic Setup . . . 1440
Creating Instances . . . 1441
Managing Instances . . . 1441
Accessing Instances . . . 1441
Using Projects . . . 1441
Ubuntu Pro . . . 1441
Activating Ubuntu Pro . . . 1441
Managing Services . . . 1442
Extended Security Maintenance (ESM) . . . 1442
Livepatch Service . . . 1442
FIPS Mode . . . 1442

Updating Configuration 1442

Editors **1443**

Intro **1444**

Overview **1445**

Overview: Editing on Linux **1446**
The three layers 1446
Why terminal editors still matter 1446
Editor families (quick map) 1447
Workspace tools (quick map) 1447
A sane default stack (2026) 1447
Learning order inside this book 1448
Practice goals 1448
What "deepen" means here 1448

Choosing an Editor **1449**

Choosing an Editor **1450**
Decision matrix 1450
Time investment vs power 1450
Modal vs modeless 1451
GUI vs TUI vs remote 1451
Compatibility with the rest of your stack 1451
Recommendation cheat sheet 1452
Anti-patterns 1452
Pick one for thirty days 1452

Terminal Workspaces **1453**

Terminal Workspaces **1454**
Problems workspaces solve 1454
Core concepts (shared vocabulary) 1454
Session 1454
Window / tab 1454
Pane 1455
Prefix key 1455
Detach / attach 1455
Choosing among multiplexers 1455
When not to nest 1456
Editor integration patterns 1456
Pattern A — editor full window 1456
Pattern B — editor + shell split 1457
Pattern C — agent herd (Herdr) 1457
Pattern D — remote box 1457

Copy mode and scrollback . . . 1457
Persistence expectations . . . 1457
Minimal practice lab . . . 1458
Keybindings to keep conflict-free . . . 1458
Next chapters . . . 1459

Vi **1460**

Vi: The Original Visual Editor **1461**
Why Learn Vi? . . . 1461
Vi Modes . . . 1461
1. Command Mode (Normal Mode) . . . 1461
2. Insert Mode . . . 1461
3. Command-Line Mode . . . 1461
Basic Operations . . . 1462
Starting Vi . . . 1462
Exiting Vi . . . 1462
Navigation Commands . . . 1463
Basic Movement . . . 1463
Advanced Navigation . . . 1463
Text Editing Commands . . . 1464
Inserting Text . . . 1464
Deleting Text . . . 1464
Changing Text . . . 1464
Copying and Pasting . . . 1465
Working with Multiple Files . . . 1465
Search and Replace . . . 1465
Basic Search . . . 1465
Search and Replace . . . 1466
Advanced Features . . . 1466
Marks and Bookmarks . . . 1466
Registers . . . 1466
Macros . . . 1467
Configuration . . . 1467
Vi Configuration File (.exrc) . . . 1467
Environment Variables . . . 1467
Real-World Scenarios . . . 1468
Scenario 1: Emergency Linux System Recovery . . . 1468
Scenario 2: Linux Log File Analysis . . . 1468
Scenario 3: Linux System Configuration . . . 1469
Vi vs Modern Editors . . . 1470
Advantages of Vi . . . 1470
Limitations . . . 1470
Tips and Best Practices . . . 1470
Essential Tips . . . 1470
Common Mistakes . . . 1471
Productivity Shortcuts . . . 1471

Troubleshooting 1471
Common Issues 1471
Recovery Options 1472

Vim **1473**

Vim: Vi Improved **1474**
Why Choose Vim? 1474
Installation 1474
Linux Distributions 1474
Building from Source 1475
Vim Modes 1475
1. Normal Mode 1475
2. Insert Mode 1475
3. Visual Mode 1475
4. Command-Line Mode 1475
5. Replace Mode 1476
Enhanced Navigation 1476
Advanced Movement 1476
Window and Tab Management 1476
Advanced Text Editing 1477
Visual Mode Operations 1477
Advanced Copy/Paste 1478
Text Objects 1478
Configuration and Customization 1479
Vimrc Configuration 1479
Advanced Vimrc Features 1481
Plugin Management 1482
Using vim-plug 1482
Installing Plugins 1483
Programming with Vim 1484
Code Navigation 1484
Code Completion 1484
Debugging Integration 1484
Real-World Development Scenarios 1485
Scenario 1: Python Development Setup 1485
Scenario 2: Web Development Workflow 1485
Scenario 3: Linux System Administration 1486
Advanced Vim Techniques 1486
Macros and Automation 1486
Regular Expressions 1487
Custom Commands 1487
Performance Optimization 1487
Vim Performance Tips 1487
Large File Handling 1488
Troubleshooting Common Issues 1488
Performance Problems 1488

Plugin Conflicts . . . 1488
Recovery and Backup . . . 1489

Neovim **1490**

Neovim: The Future of Vim **1491**
Why Choose Neovim? . . . 1491
Installation . . . 1491
Package Managers . . . 1491
From Source . . . 1492
AppImage (Portable) . . . 1492
Configuration Structure . . . 1492
Basic Lua Configuration . . . 1493
init.lua Structure . . . 1493
Plugin Management with Packer . . . 1494
Installing Packer . . . 1494
Packer Configuration . . . 1494
Loading Plugins in init.lua . . . 1497
Language Server Protocol (LSP) Setup . . . 1497
LSP Configuration . . . 1497
Installing Language Servers . . . 1499
Tree-sitter Configuration . . . 1499
Tree-sitter Setup . . . 1499
Advanced Neovim Features . . . 1500
Telescope Configuration . . . 1500
Terminal Integration . . . 1501
Real-World Development Setups . . . 1502
Python Development . . . 1502
JavaScript/TypeScript Development . . . 1503
Go Development . . . 1504
Debugging with DAP . . . 1504
DAP Configuration . . . 1504
Performance Optimization . . . 1505
Startup Optimization . . . 1505
Migration from Vim . . . 1506
Compatibility Layer . . . 1506
Troubleshooting . . . 1507
Common Issues . . . 1507

Nano **1508**

Nano: The User-Friendly Terminal Editor **1509**
Why Choose Nano? . . . 1509
Installation . . . 1509
Most Linux Distributions (Pre-installed) . . . 1509
Additional Linux Distributions . . . 1510

Basic Usage . . . 1510
Starting Nano . . . 1510
Interface Overview . . . 1510
Essential Commands . . . 1511
File Operations . . . 1511
Navigation . . . 1511
Text Editing . . . 1512
Search and Replace . . . 1512
Configuration . . . 1513
Global Configuration . . . 1513
Sample .nanorc Configuration . . . 1513
Syntax Highlighting . . . 1515
Custom Syntax Highlighting . . . 1516
Advanced Features . . . 1516
Multiple Buffers . . . 1516
Spell Checking . . . 1516
Backup Files . . . 1517
Mouse Support . . . 1517
Real-World Usage Scenarios . . . 1517
Scenario 1: Quick Configuration File Edit . . . 1517
Scenario 2: System Log Analysis . . . 1518
Scenario 3: Script Development . . . 1518
Scenario 4: Crontab Editing . . . 1519
Command Line Options . . . 1519
Useful Nano Options . . . 1519
Environment Variables . . . 1520
Tips and Best Practices . . . 1520
Productivity Tips . . . 1520
Common Shortcuts Summary . . . 1520
Customization for Different Use Cases . . . 1521
Troubleshooting . . . 1522
Common Issues . . . 1522
Recovery Options . . . 1523

Emacs **1524**

Emacs: The Extensible Editor **1525**
Why Choose Emacs? . . . 1525
Installation . . . 1525
Linux Distributions . . . 1525
Additional Linux Distributions . . . 1526
Building from Source . . . 1526
Basic Concepts . . . 1526
Key Notation . . . 1526
Buffers, Windows, and Frames . . . 1526
Major and Minor Modes . . . 1527

Essential Commands 1527
Starting and Exiting 1527
File Operations 1527
Buffer Management 1528
Window Management 1528
Navigation and Editing 1529
Basic Movement 1529
Advanced Navigation 1529
Text Editing 1530
Selection (Regions) 1530
Search and Replace 1531
Basic Search 1531
Replace Operations 1531
Advanced Search 1531
Configuration 1532
Init File 1532
Package Management 1534
Popular Packages 1534
Programming with Emacs 1535
Language-Specific Modes 1535
Code Navigation 1536
Debugging 1537
Org Mode 1537
Basic Org Syntax 1537
Org Mode Commands 1538
Org Configuration 1538
Real-World Development Scenarios 1539
Scenario 1: Python Development Setup 1539
Scenario 2: Web Development Workflow 1540
Scenario 3: System Administration 1540
Advanced Emacs Features 1541
Macros 1541
Registers 1541
Dired (Directory Editor) 1542
Shell Integration 1542
Customization and Themes 1542
Custom Themes 1542
Custom Functions 1543
Performance Optimization 1544
Startup Optimization 1544
Large File Handling 1544
Troubleshooting 1544
Common Issues 1544
Recovery 1545

LazyVim **1546**

LazyVim: Modern Neovim Configuration **1547**
What is LazyVim? . . . 1547
Why Choose LazyVim? . . . 1547
Installation . . . 1547
Prerequisites . . . 1547
Required Dependencies . . . 1548
Installation Steps . . . 1548
Initial Setup and Configuration . . . 1549
First Launch . . . 1549
Basic Configuration Structure . . . 1549
Basic Options Configuration . . . 1549
Custom Keymaps . . . 1551
Auto Commands . . . 1552
Plugin Configuration . . . 1554
Adding Custom Plugins . . . 1554
Language-Specific Configurations . . . 1555
Essential LazyVim Features . . . 1557
File Explorer (Neo-tree) . . . 1557
Fuzzy Finding (Telescope) . . . 1557
LSP Features . . . 1558
Git Integration (Lazygit) . . . 1559
Terminal Integration . . . 1559
Real-World Development Setups . . . 1560
Python Development . . . 1560
Web Development . . . 1562
Go Development . . . 1564
Customization Examples . . . 1566
Custom Colorscheme . . . 1566
Custom Dashboard . . . 1567
Performance Optimization . . . 1569
Startup Optimization . . . 1569
Memory Optimization . . . 1570
Troubleshooting . . . 1572
Common Issues . . . 1572
Reset Configuration . . . 1572
Performance Issues . . . 1573

Helix **1574**

Helix: A Modern Modal Editor **1575**
What is Helix? . . . 1575
Why Choose Helix? . . . 1575
Installation . . . 1575
Package Managers . . . 1575
Building from Source . . . 1576

Post-Installation Setup . . . 1576
Basic Concepts . . . 1577
Selection-First Editing . . . 1577
Multiple Selections . . . 1577
Modes . . . 1577
Essential Commands . . . 1578
Starting Helix . . . 1578
Basic Navigation . . . 1578
Text Selection . . . 1579
Editing Commands . . . 1579
Search and Replace . . . 1580
File and Buffer Management . . . 1581
File Operations . . . 1581
File Picker and Fuzzy Finding . . . 1581
Configuration . . . 1581
Configuration File . . . 1581
Language Configuration . . . 1584
Themes . . . 1585
Custom Theme . . . 1586
Real-World Usage Scenarios . . . 1587
Scenario 1: Python Development . . . 1587
Scenario 2: Web Development . . . 1587
Scenario 3: Configuration File Editing . . . 1588
Scenario 4: Log File Analysis . . . 1588
Advanced Features . . . 1589
Multiple Selections Mastery . . . 1589
Text Objects . . . 1589
Tree-sitter Navigation . . . 1590
LSP Features . . . 1590
Customization and Scripting . . . 1591
Custom Commands . . . 1591
Integration with External Tools . . . 1591
Shell Integration . . . 1591
Performance and Optimization . . . 1592
Large File Handling . . . 1592
Memory Usage . . . 1592
Troubleshooting . . . 1593
Common Issues . . . 1593
Debug Mode . . . 1593
Reset Configuration . . . 1593
Migration from Other Editors . . . 1593
From Vim/Neovim . . . 1593
From VSCode . . . 1594
From Kakoune . . . 1594

Micro **1595**

Micro: A Modern Terminal Editor **1596**
What is Micro? . . . 1596
Why Choose Micro? . . . 1596
Installation . . . 1596
Package Managers . . . 1596
Direct Download . . . 1597
From Source . . . 1597
Basic Usage . . . 1598
Starting Micro . . . 1598
Interface Overview . . . 1598
Essential Keybindings . . . 1599
File Operations . . . 1599
Navigation . . . 1599
Text Editing . . . 1599
Multiple Cursors . . . 1600
Search and Replace . . . 1600
Configuration . . . 1600
Settings File . . . 1600
Common Configuration Changes . . . 1602
Keybinding Customization . . . 1602
Color Schemes . . . 1603
Custom Color Scheme . . . 1603
Plugin System . . . 1604
Plugin Management . . . 1604
Popular Plugins . . . 1604
Plugin Configuration . . . 1605
Real-World Usage Scenarios . . . 1605
Scenario 1: Linux System Configuration . . . 1605
Scenario 2: Python Development . . . 1606
Scenario 3: Log File Analysis . . . 1607
Scenario 4: Web Development . . . 1607
Advanced Features . . . 1608
Command Mode . . . 1608
Macros . . . 1609
Bookmarks . . . 1609
Splits and Tabs . . . 1609
Integration with Development Tools . . . 1610
Git Integration . . . 1610
Language Server Integration . . . 1610
Build System Integration . . . 1610
Customization Examples . . . 1611
Custom Status Line . . . 1611
Custom Key Bindings for Productivity . . . 1611
File Type Specific Settings . . . 1611
Performance and Optimization . . . 1612
Large File Handling . . . 1612

Memory Usage . . . 1612
Troubleshooting . . . 1613
Common Issues . . . 1613
Debug Mode . . . 1613
Reset Configuration . . . 1613
Migration from Other Editors . . . 1613
From Nano . . . 1613
From GUI Editors . . . 1614
From Vim . . . 1614

Kakoune **1615**

Kakoune: The Selection-Based Editor **1616**
What is Kakoune? . . . 1616
Why Choose Kakoune? . . . 1616
Installation . . . 1616
Package Managers . . . 1616
Building from Source . . . 1617
Development Version . . . 1617
Basic Concepts . . . 1618
Selection-First Philosophy . . . 1618
Multiple Selections . . . 1618
Modes . . . 1618
Essential Commands . . . 1618
Starting Kakoune . . . 1618
Basic Navigation . . . 1619
Text Selection . . . 1620
Multiple Selections . . . 1620
Editing Commands . . . 1621
Search and Replace . . . 1621
File and Buffer Management . . . 1622
Buffer Operations . . . 1622
File Operations . . . 1622
Window Management . . . 1623
Configuration . . . 1623
Configuration File . . . 1623
Plugin Management . . . 1624
Popular Plugins . . . 1625
Real-World Usage Scenarios . . . 1626
Scenario 1: Refactoring Variable Names . . . 1626
Scenario 2: Multiple Line Editing . . . 1627
Scenario 3: Complex Text Manipulation . . . 1627
Scenario 4: Log File Analysis . . . 1628
Advanced Features . . . 1628
Hooks and Automation . . . 1628
Custom Commands . . . 1629
Text Objects . . . 1629

Macros and Repetition . . . 1630
Language-Specific Configurations . . . 1630
Python Development . . . 1630
Web Development . . . 1631
Go Development . . . 1632
Integration with External Tools . . . 1632
Git Integration . . . 1632
Build System Integration . . . 1633
Terminal Integration . . . 1634
Performance and Optimization . . . 1634
Large File Handling . . . 1634
Memory Optimization . . . 1635
Troubleshooting . . . 1635
Common Issues . . . 1635
Session Management . . . 1635
Configuration Issues . . . 1636

Modern Tools **1637**

Modern Tools **1638**

Modern Linux Development Tools **1639**
Why Modern Tools on Linux? . . . 1639
Suggested “editor kit” install . . . 1639
Terminal Multiplexers (map) . . . 1639
GNU screen (legacy multiplexer) . . . 1640
Terminal-Based Development Tools . . . 1640
Zellij: Modern Terminal Multiplexer . . . 1640
Starship: Cross-Shell Prompt . . . 1641
Zoxide: Smart Directory Navigation . . . 1644
Eza: Modern ls Replacement . . . 1645
Bat: Better Cat . . . 1646
Ripgrep: Fast Text Search . . . 1646
Fzf: Fuzzy Finder . . . 1647
Lazygit: Terminal Git UI . . . 1648
Yazi: Terminal File Manager (modern) . . . 1649
Fd: Better Find . . . 1649
Delta: Better Git Diff . . . 1650
File Management Tools . . . 1651
Broot: Interactive Directory Navigation . . . 1651
Nnn: Terminal File Manager . . . 1652
Development Environment Tools . . . 1653
Direnv: Environment Management . . . 1653
Mise: Runtime Version Manager . . . 1654
Just: Command Runner . . . 1656
Text Processing Tools . . . 1658
Jq: JSON Processor . . . 1658

Yq: YAML Processor . . . 1658
Sd: Sed Alternative . . . 1659
Monitoring and System Tools . . . 1660
Htop: Process Viewer . . . 1660
Bottom: System Monitor . . . 1661
Procs: Process Information . . . 1661
Linux-Specific Integration . . . 1662
Systemd Integration . . . 1662
Linux Distribution Package Management . . . 1663
Linux Desktop Integration . . . 1664
Integration Examples . . . 1665
Complete Development Setup . . . 1665
Project Template with Modern Tools . . . 1666
Linux-Specific Troubleshooting . . . 1668
Permission Issues . . . 1668
Package Manager Conflicts . . . 1668
Performance Optimization for Linux . . . 1669
Container Integration . . . 1669
Putting it together with editors . . . 1670

Tmux **1671**

tmux **1672**

tmux: Terminal Multiplexer **1673**
Why tmux . . . 1673
Installation . . . 1673
Mental model . . . 1674
Quick start lab . . . 1674
Essential keybindings . . . 1675
Sessions . . . 1675
Windows . . . 1675
Panes . . . 1675
Copy mode (scrollback) . . . 1676
Command-line interface . . . 1676
Send keys and automation . . . 1677
Configuration (`~/.tmux.conf`) . . . 1677
Sensible starter config . . . 1677
Nested tmux (SSH jump hosts) . . . 1678
Nested editors and key conflicts . . . 1679
SSH workflows . . . 1679
Keepalive . . . 1679
Plugins (optional) . . . 1679
Pairing with editors . . . 1680
Neovim / Vim . . . 1680
Helix . . . 1680
Emacs . . . 1680

Nano / Micro . . . 1681
Comparison: tmux vs screen vs Zellij vs Herdr . . . 1681
Troubleshooting . . . 1681
Cheatsheet card . . . 1681
Practice checklist . . . 1682

Herdr **1683**

Herdr **1684**

Herdr: Agent-Native Terminal Workspaces **1685**
Why Herdr exists . . . 1685
Install . . . 1686
Quick install (Linux / macOS) . . . 1686
Package managers . . . 1686
Manual binary . . . 1686
Channels . . . 1686
Concepts . . . 1687
Quick start . . . 1687
Keyboard and mouse . . . 1688
CLI automation . . . 1688
Working with editors . . . 1689
`$EDITOR` still matters . . . 1690
Herdr vs tmux vs Zellij . . . 1690
Agents and detection . . . 1690
Configuration sketch . . . 1691
Plugins . . . 1691
Session state and persistence . . . 1691
Security notes . . . 1692
Troubleshooting . . . 1692
Practice lab . . . 1692
When to use what . . . 1692
Further reading . . . 1693

Zed **1694**

Zed **1695**

Zed: Fast Native Code Editor **1696**
Why Zed on Linux . . . 1696
Install . . . 1696
Wait flag for `$EDITOR` . . . 1697
Core UI map . . . 1697
Settings and config . . . 1697
Themes . . . 1698
Vim mode . . . 1698
Language servers and extensions . . . 1698

Projects and workspaces . . . 1699
Pairing with tmux and Herdr . . . 1699
A. Local Zed + terminal multiplexer for agents . . . 1699
B. All in terminal (no Zed remote) . . . 1699
C. Remote code . . . 1699
Collaboration . . . 1700
Git workflow . . . 1700
Comparison: Zed vs neighbors . . . 1700
Migration tips . . . 1700
From VS Code . . . 1700
From Neovim . . . 1701
From Helix . . . 1701
Troubleshooting . . . 1701
Practice checklist . . . 1701
When to choose Zed . . . 1701

Workflows **1703**

Workflows **1704**

Workflows: Editors + Workspaces + Tools **1705**
Stack templates . . . 1705
1) Server / sysadmin (minimal) . . . 1705
2) Terminal IDE (Neovim) . . . 1705
3) Selection-first modal (Helix) . . . 1706
4) Friendly modeless (Micro) . . . 1706
5) Desktop GUI + agents (Zed + Herdr) . . . 1706
6) Multi-agent feature work (Herdr-centric) . . . 1706
7) Emacs as environment . . . 1707
Cross-cutting recipes . . . 1707
Open search hits in your editor . . . 1707
Git commit with correct blocking editor . . . 1707
kubectl / sudoedit . . . 1708
One-shot session script (tmux) . . . 1708
Agent + editor discipline . . . 1708
Keybinding peace treaty . . . 1708
Dotfiles layout (suggested) . . . 1709
Choosing under pressure . . . 1709
Practice: build your “default” . . . 1709
See also . . . 1709

Scripting **1711**

Bash **1712**

Intro to Shells **1713**

Shell vs Terminal: Understanding the Difference **1714**

Shell vs Terminal: Understanding the Difference **1715**

Shell vs Terminal: Understanding the Difference **1716**
What is a Terminal? . . . 1716
Common Terminal Emulators . . . 1716
What is a Shell? . . . 1717
The Relationship . . . 1717
Practical Examples . . . 1717
Example 1: Multiple Shells in One Terminal . . . 1717
Example 2: Same Shell in Different Terminals . . . 1718
Example 3: Remote Shells . . . 1718
Terminal Features vs Shell Features . . . 1718
Terminal Features: . . . 1718
Shell Features: . . . 1718
Configuration Files . . . 1718
Terminal Configuration: . . . 1718
Shell Configuration: . . . 1719
Choosing Terminal and Shell Combinations . . . 1719
For Development: . . . 1719
For System Administration: . . . 1719
For Beginners: . . . 1719
Common Misconceptions . . . 1719
Misconception 1: “I’m using Terminal” . . . 1719
Misconception 2: “Bash is a terminal” . . . 1719
Misconception 3: “All terminals are the same” . . . 1720
Practical Tips . . . 1720
Check what terminal you’re using: . . . 1720
Check what shell you’re using: . . . 1720
Launch different shells: . . . 1720
Summary . . . 1720

Types of Shells **1722**

Types of Shells **1723**

Types of Shells **1724**
Common Shell Types . . . 1724
1. Bourne Shell (sh) . . . 1724
2. Bash (Bourne Again Shell) . . . 1724
3. Zsh (Z Shell) . . . 1725
4. Fish (Friendly Interactive Shell) . . . 1725
5. Dash (Debian Almquist Shell) . . . 1725
6. Tcsh (TENEX C Shell) . . . 1725
7. Ksh (Korn Shell) . . . 1726

Checking Available Shells . . . 1726
View all available shells: . . . 1726
Check your current shell: . . . 1726
Check shell being used by a script: . . . 1726
Shell Comparison . . . 1726
Choosing the Right Shell . . . 1727
For Scripting: . . . 1727
For Interactive Use: . . . 1727
For System Administration: . . . 1727
Switching Between Shells . . . 1727
Temporarily switch: . . . 1727
Change default shell: . . . 1728
Shell Compatibility . . . 1728

What is a Shell? 1729

What is a Shell? 1730

What is a Shell? 1731
The Role of a Shell . . . 1731
How Shells Work . . . 1731
Interactive vs Non-Interactive Shells . . . 1731
Interactive Shell . . . 1731
Non-Interactive Shell . . . 1732
Login vs Non-Login Shells . . . 1732
Login Shell . . . 1732
Non-Login Shell . . . 1732
Shell Capabilities . . . 1732
Example: Basic Shell Interaction . . . 1732

Scripting Fundamentals 1734

Your First Shell Script 1735

Your First Shell Script 1736

Your First Shell Script 1737
Step 1: Create Your First Script . . . 1737
Using a Text Editor . . . 1737
Write the Script . . . 1737
Save the File . . . 1737
Step 2: Understanding the Script . . . 1738
Shebang Line . . . 1738
Comments . . . 1738
Commands . . . 1738
Command Substitution . . . 1738
Step 3: Make the Script Executable . . . 1738
Understanding Permissions . . . 1739

Step 4: Run Your Script . . . 1739
Method 1: Direct Execution . . . 1739
Method 2: Using bash Command . . . 1739
Method 3: Using sh Command . . . 1740
Method 4: Full Path . . . 1740
Step 5: Enhance Your Script . . . 1740
Step 6: Adding Error Handling . . . 1741
Step 7: Script with Functions . . . 1742
Common Beginner Mistakes . . . 1743
1. Forgetting the Shebang . . . 1743
2. Not Making Script Executable . . . 1743
3. Wrong Path in Shebang . . . 1743
4. Spaces in Variable Assignment . . . 1743
Best Practices for Beginners . . . 1744
1. Always Use Shebang . . . 1744
2. Add Comments . . . 1744
3. Use Meaningful Names . . . 1744
4. Quote Variables . . . 1744
5. Check for Errors . . . 1745
Testing Your Scripts . . . 1745
1. Syntax Check . . . 1745
2. Debug Mode . . . 1745
3. Use shellcheck . . . 1745
Next Steps . . . 1745

Script Execution Methods **1746**

Script Execution Methods **1747**

Script Execution Methods **1748**
Execution Methods Overview . . . 1748
Method 1: Direct Execution . . . 1748
Prerequisites . . . 1748
Execution . . . 1748
Example Script . . . 1749
Test It . . . 1749
Method 2: Using Shell Interpreter . . . 1749
Bash Interpreter . . . 1749
Other Interpreters . . . 1749
Advantages . . . 1750
Example . . . 1750
Method 3: Source (Dot) Command . . . 1750
Syntax . . . 1750
Key Characteristics . . . 1750
Example Script . . . 1751
Test Sourcing . . . 1751
Method 4: Absolute Path Execution . . . 1751
Examples . . . 1752

Use Cases . . . 1752
Method 5: Command Substitution . . . 1752
Syntax . . . 1752
Example . . . 1752
Execution Context and Environment . . . 1753
Process Creation . . . 1753
Environment Inheritance . . . 1753
Test Environment . . . 1753
Script Arguments and Parameters . . . 1754
Passing Arguments . . . 1754
Accessing Arguments . . . 1754
Test Arguments . . . 1754
Exit Status and Error Handling . . . 1754
Exit Status . . . 1754
Check Exit Status . . . 1755
Advanced Execution Techniques . . . 1755
Background Execution . . . 1755
Conditional Execution . . . 1755
Piping Script Output . . . 1755
Debugging Script Execution . . . 1756
Debug Mode . . . 1756
Verbose Mode . . . 1756
Syntax Check . . . 1756
Combined Options . . . 1756
Security Considerations . . . 1757
File Permissions . . . 1757
PATH Security . . . 1757
Input Validation . . . 1757
Best Practices . . . 1757
1. Always Use Shebang . . . 1757
2. Set Error Handling . . . 1758
3. Use Appropriate Execution Method . . . 1758
4. Handle Arguments Properly . . . 1758
5. Provide Clear Output . . . 1758

What is Shell Scripting? 1759

What is Shell Scripting? 1760

What is Shell Scripting? 1761
Definition and Purpose . . . 1761
Key Benefits: . . . 1761
When to Use Shell Scripts . . . 1761
Perfect for: . . . 1761
Not Ideal for: . . . 1761
Types of Shell Scripts . . . 1762
1. Simple Command Sequences . . . 1762
2. Interactive Scripts . . . 1762

3. System Administration Scripts . . . 1762
4. Data Processing Scripts . . . 1762
Shell Script Components . . . 1763
1. Shebang Line . . . 1763
2. Comments . . . 1763
3. Variables . . . 1763
4. Commands . . . 1763
5. Control Structures . . . 1763
Script Execution Methods . . . 1764
1. Make Executable and Run . . . 1764
2. Run with Shell Interpreter . . . 1764
3. Source the Script . . . 1764
Script Structure Best Practices . . . 1764
Basic Template: . . . 1764
Common Shell Scripting Patterns . . . 1765
1. Error Handling . . . 1765
2. Argument Processing . . . 1765
3. Configuration Files . . . 1766
Shell Scripting vs Other Languages . . . 1766
Shell Scripts Excel At: . . . 1766
Other Languages Better For: . . . 1766
Development Environment . . . 1766
Essential Tools: . . . 1766
Useful Commands: . . . 1767
Real-World Example . . . 1767

Bash Features **1768**

Bash Environment **1769**

Bash Environment **1770**

Bash Environment **1771**
Environment Variables . . . 1771
Viewing Environment Variables . . . 1771
Common Environment Variables . . . 1772
Setting Environment Variables . . . 1773
Unsetting Variables . . . 1773
Shell Variables vs Environment Variables . . . 1773
Shell Variables . . . 1773
Environment Variables . . . 1774
Special Variables . . . 1774
Positional Parameters . . . 1774
Process Variables . . . 1775
Test Special Variables . . . 1775
Configuration Files . . . 1775
Login Shell Files (in order) . . . 1775

Non-Login Interactive Shell Files . . . 1775
Non-Interactive Shell Files . . . 1775
Logout Files . . . 1775
Configuration File Examples . . . 1776
Prompt Customization . . . 1777
Basic Prompt Variables . . . 1777
Colorized Prompt . . . 1777
Advanced Prompt with Git Status . . . 1778
PATH Management . . . 1778
Viewing PATH . . . 1778
Modifying PATH . . . 1778
PATH Best Practices . . . 1779
Shell Options and Settings . . . 1779
Set Options . . . 1779
Shopt Options . . . 1779
Environment Management Scripts . . . 1780
Environment Setup Script . . . 1780
Environment Backup and Restore . . . 1781
Debugging Environment Issues . . . 1781
Common Environment Problems . . . 1781
Environment Diagnostic Script . . . 1781

Bash Features **1783**

Bash Features **1784**

Bash Features **1785**
Interactive Features . . . 1785
Command History . . . 1785
History Configuration . . . 1785
Tab Completion . . . 1786
Custom Completion . . . 1786
Command Line Editing . . . 1786
Job Control . . . 1787
Background Jobs . . . 1787
Job Management . . . 1787
Process Control . . . 1788
Aliases and Functions . . . 1788
Aliases . . . 1788
Functions . . . 1789
Variable Features . . . 1790
Variable Types . . . 1790
Parameter Expansion . . . 1790
Default Values . . . 1791
Pattern Matching and Globbing . . . 1791
Basic Wildcards . . . 1791
Extended Globbing . . . 1791
Globstar (Bash 4+) . . . 1792

Brace Expansion . . . 1792
Sequence Generation . . . 1792
String Expansion . . . 1793
Process Substitution . . . 1793
Input Substitution . . . 1793
Output Substitution . . . 1793
Command Substitution . . . 1794
Modern Syntax . . . 1794
Legacy Syntax . . . 1794
Nested Substitution . . . 1794
Arithmetic Operations . . . 1794
Arithmetic Expansion . . . 1794
Arithmetic Commands . . . 1795
Advanced Arithmetic . . . 1795
Conditional Expressions . . . 1795
Test Command . . . 1795
Extended Test . . . 1796
I/O Redirection . . . 1796
Basic Redirection . . . 1796
Advanced Redirection . . . 1796
Shell Options . . . 1797
Set Options . . . 1797
Shopt Options . . . 1797

What is Bash? 1798

What is Bash? 1799

What is Bash? 1800
History and Background . . . 1800
The Evolution . . . 1800
Why “Bourne Again”? . . . 1800
Bash Characteristics . . . 1800
POSIX Compliance . . . 1800
GNU Project . . . 1801
Bash Versions . . . 1801
Check Your Bash Version . . . 1801
Major Version Differences . . . 1801
Version-Specific Features . . . 1802
Bash vs Other Shells . . . 1802
Bash vs Bourne Shell (sh) . . . 1802
Bash vs Zsh . . . 1802
Bash vs Fish . . . 1802
Where Bash is Used . . . 1803
Default Shell . . . 1803
System Scripts . . . 1803
Development and DevOps . . . 1803

Bash Capabilities . . . 1804
Interactive Features . . . 1804
Programming Features . . . 1804
Advanced Features . . . 1805
Bash Configuration . . . 1805
Configuration Files . . . 1805
Customization Example . . . 1806
Bash Built-in Commands . . . 1806
Common Built-ins . . . 1806
Bash Scripting Advantages . . . 1807
1. Ubiquity . . . 1807
2. Integration . . . 1807
3. Learning Curve . . . 1807
4. Power and Flexibility . . . 1807
When to Use Bash . . . 1808
Perfect For: . . . 1808
Consider Alternatives For: . . . 1808
Getting Started with Bash . . . 1808
Check if Bash is Available . . . 1808
Start Using Bash . . . 1808
First Bash Script . . . 1808

The Bash Execution Model **1810**

How Bash Reads and Parses Input **1811**

How Bash Reads and Parses Input **1812**
Builtin vs. Keyword vs. External Binary . . . 1812

The Expansion Order: A Critical Sequence **1813**

The Expansion Order: A Critical Sequence **1814**
Why Order Matters: The Variable in Brace Pitfall . . . 1814

Word Splitting and Pathname Expansion **1815**

Word Splitting and Pathname Expansion **1816**
The Dangers of Word Splitting . . . 1816
Controlling Globbing (Pathname Expansion) . . . 1816

Subshells and Process Forking **1817**

Subshells and Process Forking **1818**
When Subshells Are Created . . . 1818
The Pipeline Subshell Trap . . . 1818

Interactive vs Non-Interactive Shells **1820**

Interactive vs Non-Interactive Shells **1821**

Variables, Quoting, and Safe Syntax **1822**

The Three Types of Quotes **1823**

The Three Types of Quotes **1824**
1. Single Quotes (`'...'`) . . . 1824
2. Double Quotes (`"..."`) . . . 1824
3. ANSI-C Quoting (`$'...'`) . . . 1824

Parameter Expansion and Default Values **1826**

Parameter Expansion and Default Values **1827**
Default Values and Fallbacks . . . 1827
String Slicing and Length . . . 1827
Pattern Stripping and Substitution . . . 1828

Special Parameters: Positional, IFS, and PATH **1829**

Special Parameters: Positional, IFS, and PATH **1830**
Positional Arguments and `$@` vs `$*` . . . 1830
Process & Runtime Parameters . . . 1830

Common Quoting Pitfalls and Safe Syntax **1831**

Common Quoting Pitfalls and Safe Syntax **1832**
Pitfall 1: Unquoted Variables in Commands . . . 1832
Pitfall 2: Unquoted Test Conditions in `[ ]` . . . 1832
Pitfall 3: Storing Commands in Variables . . . 1832

Variable Basics **1834**

Variable Basics **1835**

Variable Basics **1836**
What are Variables? . . . 1836
Variable Declaration and Assignment . . . 1836
Basic Syntax . . . 1836
Important Rules . . . 1836
Common Mistakes . . . 1837
Variable Naming Conventions . . . 1837
Valid Variable Names . . . 1837
Invalid Variable Names . . . 1837
Best Practices . . . 1837
Accessing Variable Values . . . 1838
Basic Access . . . 1838
When to Use Braces . . . 1838
Variable Types and Behavior . . . 1839
String Variables . . . 1839
Numeric Variables . . . 1839
Boolean-like Variables . . . 1839

Variable Scope . . . 1840
Global Variables . . . 1840
Local Variables . . . 1840
Reading User Input . . . 1840
Basic Input . . . 1840
Input with Prompt . . . 1841
Multiple Variables . . . 1841
Reading into Array . . . 1841
Silent Input (Passwords) . . . 1841
Variable Assignment Methods . . . 1841
Direct Assignment . . . 1841
Command Substitution . . . 1842
Reading from Files . . . 1842
Arithmetic Assignment . . . 1842
Variable Manipulation . . . 1843
String Length . . . 1843
Substring Extraction . . . 1843
Case Conversion . . . 1843
Pattern Replacement . . . 1843
Default Values and Error Handling . . . 1844
Default Values . . . 1844
Error on Unset Variables . . . 1844
Practical Example . . . 1844
Variable Arrays (Preview) . . . 1845
Simple Array . . . 1845
Best Practices . . . 1845
1. Always Quote Variables . . . 1845
2. Use Meaningful Names . . . 1845
3. Initialize Variables . . . 1846
4. Use readonly for Constants . . . 1846
5. Validate Input . . . 1846
Common Variable Patterns . . . 1847
Configuration Variables . . . 1847
Temporary Variables . . . 1847

Control Flow and Logic **1848**

Test Commands: [ ], [[ ]], and test **1849**

Test Commands: [ ], [[ ]], and test **1850**
Single Bracket `[ ]` (POSIX standard) . . . 1850
Double Bracket `[[ ]]` (Bash Keyword) . . . 1850

Pattern Matching with case Statements **1851**

Pattern Matching with case Statements **1852**
Advanced Terminators in Bash . . . 1852

Break, Continue, and Early Exit Patterns **1853**

Break, Continue, and Early Exit Patterns **1854**

Loops in Depth (C-Style, Nested, and Range Loops) **1855**

Loops in Depth (C-Style, Nested, and Range Loops) **1856**

Loop from 1 to 5 **1857**

Loop from 1 to 10, incrementing by 2 **1858**

Loop over a list of strings **1859**

Define an array **1860**

Loop over the array **1861**

Loop from 1 to 5 **1862**

Loop through an array and break when a specific value is found **1863**

Loop through a range of numbers and skip multiples of 3 **1864**

Conditional Statements **1865**

Conditional Statements **1866**

Conditional Statements **1867**

The if Statement . . . 1867
Basic Syntax . . . 1867
Simple Example . . . 1867
if-else Statement . . . 1867
if-elif-else Statement . . . 1868
Test Conditions . . . 1868
Numeric Comparisons . . . 1868
String Comparisons . . . 1869
File Tests . . . 1870
Advanced Test Constructs . . . 1871
Double Brackets [[ ]] . . . 1871
Arithmetic Evaluation (( )) . . . 1871
Logical Operators . . . 1872
AND Operator (&&) . . . 1872
OR Operator (||) . . . 1872
NOT Operator (!) . . . 1873
Practical Examples . . . 1873
File Backup Script . . . 1873
System Health Check . . . 1874
User Input Validation . . . 1875
Nested Conditionals . . . 1876

Short-Circuit Evaluation . . . 1876
Common Pitfalls and Best Practices . . . 1877
1. Always Quote Variables . . . 1877
2. Use [[ ]] for String Operations . . . 1877
3. Handle Empty Variables . . . 1878
4. Use Meaningful Conditions . . . 1878
5. Exit Codes for Error Handling . . . 1878

Loops (for, while, until) 1879

Loops (for, while, until) 1880

Loops 1881
for Loops . . . 1881
Basic for Loop Syntax . . . 1881
Simple Examples . . . 1881
Looping Through Files . . . 1881
C-style for Loop . . . 1882
Using Brace Expansion . . . 1882
Looping Through Arrays . . . 1883
while Loops . . . 1883
Basic while Loop . . . 1883
Reading File Line by Line . . . 1884
Menu System with while Loop . . . 1884
Monitoring with while Loop . . . 1885
until Loops . . . 1885
Basic until Loop . . . 1886
Waiting for a Service . . . 1886
Waiting for File . . . 1886
Loop Control Statements . . . 1887
break Statement . . . 1887
continue Statement . . . 1887
Nested Loops with break and continue . . . 1887
Practical Loop Examples . . . 1888
Batch File Processing . . . 1888
Log Analysis . . . 1889
System Backup Script . . . 1889
User Management Script . . . 1890
Loop Performance Tips . . . 1891
Avoid Unnecessary Command Substitution . . . 1891
Use Built-in Commands When Possible . . . 1892
Process Files Efficiently . . . 1892
Common Loop Patterns . . . 1892
Retry Pattern . . . 1892
Progress Indicator . . . 1893

Functions and Modular Design **1894**

Local Variables and Scope **1895**

Local Variables and Scope **1896**

Positional Parameters in Functions **1897**

Positional Parameters in Functions **1898**

Return Values and Exit Codes **1899**

Return Values and Exit Codes **1900**
Returning Data Strategies 1900

Script Libraries and Sourcing **1901**

Script Libraries and Sourcing **1902**

Function Basics **1903**

Function Basics **1904**

Function Basics **1905**
What are Functions? 1905
Function Syntax 1905
Method 1: Using the `function` keyword 1905
Method 2: POSIX-compatible syntax (preferred) 1905
Basic Function Examples 1906
Simple Function 1906
Function with Commands 1906
Function Parameters 1906
Accessing Parameters 1906
Multiple Parameters 1907
Parameter Variables 1907
Return Values 1907
Return Exit Status 1908
Return Values via Echo 1908
Local Variables 1908
Without Local Variables (Global) 1909
With Local Variables 1909
Best Practice Example 1909
Practical Function Examples 1910
File Operations Function 1910
System Information Function 1911
String Processing Function 1911
Function Libraries 1912
Create a Library File (utils.sh) 1912
Use the Library in Your Script 1913

Function Best Practices 1914
1. Use Descriptive Names 1914
2. Validate Input Parameters 1914
3. Use Local Variables 1915
4. Provide Usage Information 1915
5. Handle Errors Gracefully 1916
Common Function Patterns 1916
Configuration Function 1916
Cleanup Function 1916
Menu Function 1917

Arrays and Associative Arrays **1918**

Indexed Arrays: Creation and Manipulation **1919**

Indexed Arrays: Creation and Manipulation **1920**

Iterating Over Arrays Safely **1921**

Iterating Over Arrays Safely **1922**

Array Operations: Slicing, Searching, Deleting **1923**

Array Operations: Slicing, Searching, Deleting **1924**

Associative Arrays for Key-Value Data **1925**

Associative Arrays for Key-Value Data **1926**

Portability Considerations for Arrays **1927**

Portability Considerations for Arrays **1928**

Input/Output, Redirection, and File Descriptors **1929**

Standard Streams and File Descriptors **1930**

Standard Streams and File Descriptors **1931**

Redirection Operators Explained **1932**

Redirection Operators Explained **1933**

Pipelines and Command Chaining **1934**

Pipelines and Command Chaining **1935**
Conditional Execution: `&&` and `||` 1935

Here-Documents and Here-Strings **1936**

Here-Documents and Here-Strings 1937

Process Substitution and Named Pipes 1938

Process Substitution and Named Pipes 1939

Process and Job Management 1940

Background Jobs and the & Operator 1941

Background Jobs and the & Operator 1942

Waiting for Processes to Complete 1943

Waiting for Processes to Complete 1944

Killing and Managing Child Processes 1945

Killing and Managing Child Processes 1946

Process Groups and Sessions 1947

Process Groups and Sessions 1948

Monitoring Long-Running Operations 1949

Monitoring Long-Running Operations 1950

Signals, Traps, and Cleanup 1951

Understanding Unix Signals 1952

Understanding Unix Signals 1953

The trap Command 1954

The trap Command 1955

Handling Common Signals in Scripts 1956

Handling Common Signals in Scripts 1957

Ensuring Cleanup Runs on Exit 1958

Ensuring Cleanup Runs on Exit 1959

Signal Handling in Pipelines and Subshells 1960

Signal Handling in Pipelines and Subshells 1961

Error Handling and Defensive Scripting 1962

Exit Codes and Their Meaning 1963

Exit Codes and Their Meaning 1964

The set Command: -e, -u, -o pipefail 1965

The set Command: -e, -u, -o pipefail 1966

Validating Input and Preconditions 1967

Validating Input and Preconditions 1968

Graceful Failure and Partial Success 1969

Graceful Failure and Partial Success 1970

Logging Errors Effectively 1971

Logging Errors Effectively 1972

Debugging and Testing Scripts 1973

Bash Built-In Debugging Options 1974

Bash Built-In Debugging Options 1975

Using set -x Wisely 1976

Using set -x Wisely 1977

Static Analysis Tools for Shell Scripts 1978

Static Analysis Tools for Shell Scripts 1979

Writing Tests for Shell Scripts 1980

Writing Tests for Shell Scripts 1981

Continuous Integration for Shell Scripts 1982

Continuous Integration for Shell Scripts 1983

Text Processing Essentials in Scripts 1984

grep and Extended Pattern Matching in Scripts 1985

grep and Extended Pattern Matching in Scripts 1986

sed for Stream Editing in Scripts 1987

sed for Stream Editing in Scripts 1988

awk for Field-Based Processing 1989

awk for Field-Based Processing 1990

Safe File Discovery with find and xargs 1991

Safe File Discovery with find and xargs 1992

Combining Tools in Complex Pipelines 1993

Combining Tools in Complex Pipelines 1994

POSIX Compatibility and Portability 1995

The POSIX Shell Standard 1996

The POSIX Shell Standard 1997

Bashisms vs POSIX Shell Features 1998

Bashisms vs POSIX Shell Features 1999

Detecting the Running Shell 2000

Detecting the Running Shell 2001

Writing Portable Scripts with Strict Options 2002

Writing Portable Scripts with Strict Options 2003

When to Use #!/bin/sh vs #!/usr/bin/env bash 2004

When to Use #!/bin/sh vs #!/usr/bin/env bash 2005

Linux Distribution Differences 2006

Default Shell Versions Across Distributions 2007

Default Shell Versions Across Distributions 2008

Path and Tooling Variations 2009

Path and Tooling Variations 2010

Systemd vs SysVinit vs OpenRC in Scripts 2011

Systemd vs SysVinit vs OpenRC in Scripts **2012**

Package Managers in Scripts **2013**

Package Managers in Scripts **2014**

Alpine Linux and BusyBox Considerations **2015**

Alpine Linux and BusyBox Considerations **2016**

Security in Shell Scripts **2017**

Command Injection Vulnerabilities **2018**

Command Injection Vulnerabilities **2019**

PATH Manipulation and Environment Attacks **2020**

PATH Manipulation and Environment Attacks **2021**

Handling User Input Safely **2022**

Handling User Input Safely **2023**

Secrets and Credentials in Scripts **2024**

Secrets and Credentials in Scripts **2025**

File Permissions, SUID, and Safe Temp Files **2026**

File Permissions, SUID, and Safe Temp Files **2027**

Performance Optimization and Concurrency **2028**

The Cost of Forking External Commands **2029**

The Cost of Forking External Commands **2030**

Using Builtins Over External Tools **2031**

Using Builtins Over External Tools **2032**

Minimizing Subshells and Efficient I/O **2033**

Minimizing Subshells and Efficient I/O **2034**

Parallel Execution with Background Jobs and Limits **2035**

Parallel Execution with Background Jobs and Limits **2036**

Synchronization, Locking, and Atomic Operations 2037

Synchronization, Locking, and Atomic Operations 2038

Automation, DevOps, and CI/CD 2039

Shell Scripts in CI/CD Pipelines 2040

Shell Scripts in CI/CD Pipelines 2041

Infrastructure as Code and Shell Automation 2042

Infrastructure as Code and Shell Automation 2043

Container Build Scripts and Entrypoints 2044

Container Build Scripts and Entrypoints 2045

Configuration Management Integration 2046

Configuration Management Integration 2047

Monitoring and Alerting Scripts 2048

Monitoring and Alerting Scripts 2049

System Administration Patterns 2050

User and Group Management Scripts 2051

User and Group Management Scripts 2052

Log Processing and Rotation 2053

Log Processing and Rotation 2054

Backup and Restore Procedures 2055

Backup and Restore Procedures 2056

Service Health Checks with Retries 2057

Service Health Checks with Retries 2058

Disk Space and Resource Management 2059

Disk Space and Resource Management 2060

Workbook **2061**

Basic Bash Examples **2062**

Basic Bash Examples **2063**

Basic Bash Examples **2064**
Hello World Variations . . . 2064
Simple Hello World . . . 2064
Hello World with Variables . . . 2064
Interactive Hello World . . . 2064
Hello World with Date . . . 2064
Variable Examples . . . 2065
Basic Variable Operations . . . 2065
Environment Variables . . . 2065
Variable Default Values . . . 2065
Variable Length and Manipulation . . . 2066
Command Line Arguments . . . 2066
Basic Argument Handling . . . 2066
Argument Validation . . . 2066
Processing Multiple Arguments . . . 2067
Simple Calculations . . . 2067
Basic Arithmetic . . . 2067
Calculator Script . . . 2067
Temperature Converter . . . 2068
File Operations . . . 2068
File Information . . . 2068
File Backup . . . 2069
Directory Listing . . . 2069
Text Processing . . . 2070
Word Count . . . 2070
Text Search . . . 2070
Line Numbering . . . 2071
System Information . . . 2071
System Status . . . 2071
Process Monitor . . . 2072
Network Information . . . 2072
User Input Examples . . . 2073
Menu System . . . 2073
User Registration . . . 2073
Quiz Game . . . 2074
String Manipulation . . . 2075
String Operations . . . 2075
Palindrome Checker . . . 2076
Password Generator . . . 2076
Array Examples . . . 2077
Basic Array Operations . . . 2077
Shopping List . . . 2077

Grade Calculator 2079

Intermediate Bash Examples **2081**

Intermediate Bash Examples **2082**

Intermediate Bash Examples **2083**
Function Examples 2083
Library of Utility Functions 2083
Configuration Manager 2085
Database Backup Script 2088
File Processing Examples 2092
Log Analyzer 2092
File Organizer 2094
CSV Processor 2098
System Administration Examples 2104
System Monitor 2104

Advanced Bash Examples **2113**

Advanced Bash Examples **2114**

Advanced Bash Examples **2115**
Process Management and IPC 2115
Process Pool Manager 2115
Network Programming 2119
HTTP Server in Bash 2119
Network Scanner 2123
Data Processing and Analysis 2127
JSON Processor 2127
Data Aggregator 2130
Security and Encryption 2134
Password Manager 2134

Automation Examples **2141**

Automation Examples **2142**

Automation Examples **2143**
System Administration Automation 2143
Automated Backup System 2143
System Health Monitor 2151
Environment Provisioning Script 2170
Web Scraping and API Automation 2179
Web Scraper 2179

Utility Scripts **2190**

Utility Scripts **2191**

Utility Scripts **2192**
File and Directory Utilities 2192
Duplicate File Finder 2192
Directory Synchronizer 2200
System Maintenance Utilities 2210
Log Rotator 2210

Appendices 2220

Appendix A: Bash Syntax Cheat Sheet **2221**

Appendix A: Bash Syntax Cheat Sheet **2222**

Appendix B: Common Production Patterns Library **2223**

Appendix B: Common Production Patterns Library **2224**

Appendix C: POSIX vs. Bash Feature Comparison Matrix **2225**

Appendix C: POSIX vs. Bash Feature Comparison Matrix **2226**

Introduction

Linux

A practical library for **Linux on the command line**, organized as three parts under one book:

```
Linux
   Commands          → topics (files, processes, networking, …) → individual commands
   Editors           → vi, Neovim, tmux, Herdr, … → chapter pages
   Scripting
       Bash          → shells, scripting, variables, control, workbook
```

This landing page restores the **Commands** orientation (scope, page template, parts map) from the former standalone *Linux-Commands* book, and points you into Editors and Bash as well.

Commands — scope

A practical, **Ubuntu-first** encyclopedia of **general Linux CLI** and **server-side** commands for administration, scripting, and troubleshooting on headless and SSH-managed systems.

Commands stays on the terminal and the OS. It does **not** cover desktop/GUI applications, graphical package stores, or printer/desktop workflows.

Each Commands chapter covers **one command** (or a tight family): syntax, important options, realistic examples, output notes, safety tips where it matters, and related tools.

In scope	Out of scope
Core userland (`ls`, `find`, pipes, …)	Desktop apps and GUI installers
Networking, SSH, firewalls, capture	Display servers (X11/Wayland), window managers
systemd, journals, packages (`apt`/`dnf`)	Flatpak / GUI app stores
Storage, LVM, LUKS, disk health	CUPS client printing queues
Processes, monitoring, scheduling	Desktop session “app” management
Terminal multiplexers (`tmux`, …)	Browser-based or GUI-only tools
SELinux/capabilities as server concerns	End-user desktop customization

Editors and Bash live in the other parts of **this** book (they used to be sibling books in the monorepo).

Related books

Book	Overlap
Networking	Deeper TCP/IP, routing, and lab topologies beyond host CLI
VCS	Git workflows used on the servers you administer
NixOS	Declarative hosts after you know classic Linux ops
Go / C	Systems programming that assumes this CLI fluency

How to use

1. Expand **Commands** → open a topic → open a command page.
2. Each Commands topic starts with an **Intro** page: overview, command table, and suggested starting points — then one page per command.
3. Expand **Editors** → pick an editor or workspace tool (vi, Neovim, tmux, Herdr, …).
4. Expand **Scripting → Bash** for shell programming and automation beyond one-liners.
5. **Search** for a command or editor name when you already know what you need.
6. Prefer **worked examples** over option dumps — copy, adapt, verify on a server or VM.

Commands page template

Most command pages follow this shape:

1. Overview
2. Syntax
3. Common options
4. Safety (when destructive or privileged)
5. Key use cases (Core pages)
6. Examples with **named** explanations
7. Understanding output (when useful)
8. Notes & pitfalls

9. Related commands

10. Further reading (optional)

Short utilities may use a lighter form. See `content/_planning/command-page-template.md` for author depth tiers (Core / Standard / Light).

Commands — parts at a glance

Section	Focus
Help and docs	`man`, `info`, `help`, `apropos`, `whatis`
Files and paths	Navigation, copy/move, permissions, ACLs, attributes, search
Archives and compression	`tar`, `gzip`, `xz`, `zstd`, zip, `zcat`/`zgrep`
System information	Host identity, memory, sessions, hardware summaries
Processes and jobs	`ps`, `top`, signals, job control, priority
System monitoring	`iostat`, `vmstat`, `sar`, `strace`, `ltrace`, `perf`
Users and groups	Accounts, passwords, aging, `su`/`sudo`
Networking	`ip`, `ss`, DNS, SSH, transfer, firewalls, capture, `ethtool`
Scheduling	`cron`, `crontab`, `at`, `anacron`
Logging	`logger`, `logrotate` (see also `journalctl`)
Hardware	`lshw`, `lspci`, `dmidecode`, `hdparm`
Storage and filesystems	`lsblk`, LVM, LUKS, `parted`, SMART, grow/tune ext*
Terminals and multiplexers	`tmux`, `screen`, and related terminal tools
Text and pipes	`grep`, `sed`, `awk`, `jq`, `pv`, sort/uniq pipelines
Packages	`apt`/`dpkg`, `dnf`/`rpm`, `yum`, server-oriented `snap`
Services and runtime	`systemctl`, `journalctl`, timers, modules, `podman`
Shell commands	`echo`/`printf`, history, `type`, `export`, `time`, directory stack
Security	SELinux mode/labels, file capabilities (RHEL-family focused)
Appendices	Cheatsheets and longer extras

Editors (this book)

Terminal-native editing — classic (`vi`/`vim`/`nano`), modern modal (Neovim, LazyVim, Helix, Kakoune), GUI-speed **Zed**, plus workspace tools (**tmux**, **Herdr**). Start under **Editors → Intro** for paths (sysadmin emergency, Vim family, AI agents).

Scripting → Bash (this book)

Shell *programming* beyond one-liners: shells vs terminals, first scripts, variables, control flow, functions, and a workbook. Use **Commands** for individual utilities; use **Bash** when you are writing scripts.

Conventions

- Examples assume a modern GNU/Linux **server** userland (Ubuntu 22.04/24.04 Server compatible unless noted).
- Prefer `ip` / `ss` / `systemctl` over legacy `ifconfig` / `netstat` / SysV-only workflows.
- Ubuntu-first package names where they matter; note Fedora/RHEL variants when tools differ.
- Worked examples over option dumps.
- Safety notes on destructive or privileged operations.

Formats

HTML, PDF, and EPUB when built via the monorepo portal.

Commands

Help and Docs

Intro

Intro

This part covers how to find answers **on the system itself** before searching the web. Offline docs match the versions you actually installed — critical when flags change between releases.

Commands in this part

Command	Role
`man`	man displays the system manual.
help (shell builtin)	help shows documentation for shell builtins — commands implemented inside the shell itself (cd, export, read, [[,…
`info`	info reads documentation in Info (Texinfo) format.
`apropos`	apropos searches man page names and short descriptions for keywords.
`whatis`	whatis prints single-line descriptions from the man-db whatis index.

Suggested starting points

1. Quick hint: `whatis cmd` or `cmd --help`.
2. Full story: `man cmd` (try section 5 for configs, 8 for admin tools).
3. Forgot the name: `apropos keyword`, then open the best hit with `man`.
4. Shell builtin: `type cmd`, then `help cmd` if it is a builtin.
5. GNU deep dive: `info` for coreutils, bash, and related tools.

Related parts

- `type` / `command -v` (Shell commands) — resolve names
- Book introduction — how pages are structured

Continue with the individual command pages in this part.

man

Overview

`man` displays the system manual. Pages are organized into **sections** covering user commands, system calls, libraries, file formats, and admin tools. It remains the authoritative offline reference on Linux — faster and more version-accurate than random web snippets for the packages you actually have installed.

Most pages are viewed through a pager (usually `less`). Press `q` to quit, `/` to search.

Syntax

```
man [section] name
man [options] name
man -k keyword          # same as apropos
man -f name             # same as whatis
```

Sections (common)

Sec	Content	Example
1	User commands	`man 1 printf`
2	System calls	`man 2 open`
3	Library functions	`man 3 printf`
4	Special files / devices	`man 4 null`
5	File formats / configs	`man 5 passwd`
6	Games	—
7	Overviews / conventions	`man 7 signal`
8	Admin commands	`man 8 mount`
9	Kernel routines (some systems)	—

When the same name exists in multiple sections (e.g. `printf` in 1 and 3, `passwd` in 1 and 5), specify the section number.

Common Options

Option	Description
`-k` / `--apropos`	Search short descriptions
`-f` / `--whatis`	One-line summary for a known name
`-a`	Show all matching sections in turn
`-w` / `--path`	Print path to the man file, don't open it
`-W`	List all matching paths
`-K`	Full-text search of page bodies (slow)
`-P pager`	Override pager (`less`, `cat`, …)
`-L locale`	Force locale for translated pages
`MANPAGER` / `PAGER`	Environment override for pager
`MANWIDTH`	Force formatting width (useful in scripts)

Examples with Explanations

Open a page

```
man ls
man 5 sshd_config
man 8 systemctl
man man
```

`sshd_config` lives in section 5 (file formats), not as a user command.

Disambiguate sections

```
man -a printf          # cycle through every section that has printf
man 3 printf           # C library specifically
whatis printf          # see which sections exist
man -w -a passwd       # paths for all passwd pages
```

Search when you forget the name

```
man -k resize
man -k 'disk usage'
apropos network interface
man -K 'AddressFamily'   # full-text; can take a while
```

Locate the source file

```
man -w ls
# e.g. /usr/share/man/man1/ls.1.gz
zcat "$(man -w ls)" | head
```

Useful when debugging missing or outdated pages after package installs.

Readable width and plain text dump

```
MANWIDTH=80 man ls
man ls | col -b > /tmp/ls.txt    # strip backspaces/overstrike
man -P cat ls | head -40
```

HTML / browser (if supported)

```
man -H ls                 # uses browser from $BROWSER or similar
man -Tutf8 ls | less -R
```

Scripting existence checks

```
if man -w journalctl >/dev/null 2>&1; then
  echo "journalctl man page present"
fi
```

Pager keys (less)

Key	Action
`q`	Quit
`/pattern`	Search forward
`?pattern`	Search backward
`n` / `N`	Next / previous match
`g` / `G`	Top / bottom
`Space` / `b`	Page down / up
`h`	Help

Notes / Pitfalls

- Empty or stale results after installing packages: rebuild the index with `sudo mandb` (or `sudo makewhatis` on some BSDs / older systems).
- Container images often ship **without** man pages to save space. Install `man-db` and the relevant `*-doc` / man packages, or read docs on the host.
- Section order and default "first match" can surprise you — prefer explicit `man 5 name` for configs.
- Locale: if a translated page is incomplete, force English with `LANG=C man …` or `man -L C …`.
- `man` is not for shell builtins (`cd`, `[[`, `declare`) — use `help` (bash) instead.

2026-relevant notes

- Systemd, iproute2, and Podman pages are high quality; prefer `man systemctl`, `man ip-link`, `man podman-run` over outdated blog posts for flag accuracy.
- Many modern tools also ship `--help` that is good enough for flags; use `man` when you need semantics, exit codes, and file formats.
- Container and snap sandboxes may not see host man pages; read docs in the environment where the tool is installed.

Related Commands

- `info` — Texinfo hypertext manuals (GNU tools)
- `help` — shell builtins
- `apropos` / `whatis` — search and one-liners
- `tldr` / `cheat` — community example sheets (if installed)
- `mandb` — rebuild the whatis database

Additional Resources

- `man man`, `man man-pages`, `man 7 man-pages`
- Linux man-pages project

help (shell builtin)

Overview

`help` shows documentation for **shell builtins** — commands implemented inside the shell itself (`cd`, `export`, `read`, `[[`, `declare`, job control, etc.), not as separate binaries under `/usr/bin`. External programs use `--help` or `man` instead.

If `man cd` looks wrong or minimal, you are probably looking at a stub; the real reference is `help cd` (bash) or the shell's own manual.

Syntax

```
help [-dms] [pattern ...]
```

Bash-oriented options (check `help help` on your shell):

Option	Description
`-d`	Short description only
`-m`	Manpage-like layout
`-s`	Short usage synopsis
pattern	Builtin name or pattern to match

Common Options / discovery helpers

Command	Description
`help`	List topics / usage overview
`help name`	Full builtin help
`type name`	How the shell resolves `name`
`type -a name`	All candidates (builtin + PATH)
`command -v name`	Path or builtin indicator
`compgen -b`	List bash builtins
`enable -a`	Enabled/disabled builtins

Examples with Explanations

Read builtin docs

```
help cd
help [[
help declare
help read
help -m printf          # man-like layout for the builtin printf
help | less             # browse all topics
```

Builtin vs external

```
type cd                 # builtin
type ls                 # usually /usr/bin/ls (hashed)
type -a echo            # may show builtin and /usr/bin/echo
type -a kill            # often both: shell kill + /bin/kill
help cd                 # works for builtin
ls --help               # external GNU flag
man ls                  # full external manual
```

Prefer `type` / `command -v` in scripts over `which` for resolving names.

List builtins (bash)

```
compgen -b | column
compgen -b | grep -E '^(cd|pushd|popd|dirs)$'
enable -a | head
```

Disable / re-enable a builtin (rare, debugging)

```
enable -n echo          # force external echo next
type echo
enable echo             # restore builtin
```

Useful when testing whether a weird `echo` behavior comes from the builtin or a shadowed binary.

Conditional help in interactive setups

```
# only define an alias if the real command exists
if type -P rg >/dev/null; then
  alias grep='rg'
fi
```

`type -P` searches PATH only (ignores builtins/aliases/functions).

Job-control and shell language topics

```
help jobs
help fg
help bg
help wait
help trap
help set
help shopt
```

These are not separate man pages in the same way; learn them via `help` + `man bash`.

Notes / Pitfalls

- `help` is primarily for **interactive** humans. Non-interactive scripts still *run* builtins; they just don't need `help`.
- **zsh** uses `run-help` (often bound to `Esc-h`) and different docs; **fish** integrates web/local docs under `help`.
- `help printf` is the **shell** printf; `man 1 printf` / `man 3 printf` are different tools/APIs.
- Aliases and functions shadow names: `type foo` before assuming you are reading the right help.
- BusyBox / dash environments have smaller builtin sets; don't assume full bash `help` text.

2026-relevant notes

- Bash 5.x remains the default interactive shell on many servers; `help` text tracks your installed bash, not a website.
- For deep language reference (arrays, coprocs, `mapfile`, loadable builtins), combine `help` with `man bash` / `info bash`.
- In containers, shells are often `bash` or `sh` → `dash`; verify with `echo $0` and `help` availability.

Related Commands

- `man bash` / `info bash` — full shell language reference
- `type` / `command -V` — resolve names
- `whatis` / `man` — external commands
- `compgen` / `complete` — completion and name generation
- `set` / `shopt` — shell options (see `help set`)

Additional Resources

- `help help`, `man bash`
- Bash Reference Manual

info

Overview

`info` reads documentation in **Info** (Texinfo) format. GNU projects often put their deepest manuals here — structured as a tree of **nodes** with menus and cross-references — while `man` pages stay as flatter reference sheets.

For coreutils, bash, and many GNU tools, `info` can be more complete than the man page. Navigation is keyboard-driven and different from `less`; invest a few minutes in the keys once.

Syntax

```
info [options] [menu-item ...]
info [options] -f file [node]
info [options] --index-search=string
```

Common Options

Option	Description
`-f file`	Open a specific Info file
`-n node`	Start at named node
`-a`	Use all matching manuals
`-k string` / `--apropos`	Search indices for string
`-w`	Show location of Info file
`--index-search=str`	Go to index entry
`--show-options`	Jump to command-line options node (when present)
`--subnodes`	Dump node and subnodes (non-interactive)
`-o file`	Write output to file
`--vi-keys`	Vi-like key bindings

Navigation Keys (standalone info)

Key	Action
`?`	List commands
`h`	Tutorial
`q`	Quit
`n` / `p`	Next / previous node at this level
`u`	Up to parent node
`l`	Last node (history back)
`]` / `[`	Forward / back in document order
`m`	Menu item by name
`f`	Follow cross-reference
`s` / `/`	Search
`Space` / `Del`	Scroll
`d`	Directory node (top-level catalogue)

Emacs users often prefer `M-x info` inside Emacs; keys differ slightly.

Examples with Explanations

Open a manual

```
info ls
info coreutils
info bash
info libc
```

Tab completion or the directory node (`info` with no args, then browse) helps discovery.

Jump to options / a node

```
info --show-options ls
info -n 'Invoking ls' coreutils
info -f coreutils -n 'Directory listing'
```

Search

```
info --apropos=printf
info -k symlink
info --index-search=regex
```

Where is the file?

```
info -w coreutils
# e.g. /usr/share/info/coreutils.info.gz
```

Non-interactive extract

```
info --subnodes -o /tmp/ls-info.txt ls
info -o - --show-options ls | head -50
```

Useful for grepping offline or pasting into notes.

Prefer man for quick flags, info for depth

```
man ls                  # quick reference
info ls                 # full GNU narrative + examples
info coreutils          # whole suite manual
```

Understanding the Info tree

1. **Directory node** — catalogue of installed manuals (`d`).
2. **Top node** — start of one manual.
3. **Menus** — child nodes (chapters/sections).
4. **Cross-references** — `*note …` links between topics.
5. **Indices** — topic/command/option indices for search.

Think of it as a hypertext book, not a single scrollable man page.

Notes / Pitfalls

- Missing docs: install packages like `info`, `bash-doc`, `coreutils` info files (`*-doc` on some distros).
- People open `info` once, hate the UI, and never return — spend five minutes on `h` / `?`.
- Not every tool has an Info manual; many only ship man pages.
- `pinfo` is a friendlier alternative viewer if you install it.
- Dumping with `--subnodes` can be huge (`libc`); narrow to a node first.

2026-relevant notes

- Man pages improved a lot for systemd/iproute2; Info still wins for **GNU coreutils, findutils, bash, gdb, gcc** deep dives.
- In CI/containers, Info files are often omitted; generate or fetch docs on a full workstation.
- Some projects moved primary docs to HTML/web; Info remains the classic offline GNU format.

Related Commands

- `man` — traditional manual pages
- `pinfo` — alternative Info browser
- `apropos` / `whatis` — man-db search / one-liners
- `help` — shell builtins
- `info --help` — viewer options

Additional Resources

- `info info` — the Info reader manual itself
- GNU Texinfo / Info documentation

apropos

Overview

`apropos` searches man page **names and short descriptions** for keywords. Use it when you remember *what you want to do* but not the command name. It is equivalent to `man -k`.

It queries the **whatis database** built by `mandb`. If that DB is missing or stale, results will be empty or outdated — rebuild it rather than assuming the command does not exist.

Syntax

```
apropos [options] keyword...
man -k [options] keyword...
```

Common Options

Option	Description
`-a`	Require **every** keyword to match (AND)
`-r`	Treat keywords as regular expressions
`-w`	Shell-style wildcards
`-s list`	Limit to sections (e.g. `1,8`)
`-e`	Exact match on page name/description token
`-l`	Do not truncate long descriptions
`-C file`	Alternate man config file

Exact flags can vary slightly by man-db version; check `man apropos` on your host.

Examples with Explanations

Basic keyword search

```
apropos password
apropos socket
apropos 'disk usage'
man -k resize
```

Quotes help when the phrase contains spaces or shell metacharacters.

AND multiple terms

```
apropos -a network interface
apropos -a list directory
```

Without `-a`, multiple words are often OR'd (implementation-dependent noise). Prefer `-a` to narrow results.

Limit by section

```
apropos -s 8 mount          # admin tools about mount
apropos -s 1,8 kill         # user + admin kill-related pages
apropos -s 5 passwd         # file format, not the passwd(1) command
```

Regex and wildcards

```
apropos -r 'zip$'           # names/descriptions ending in zip
apropos -r '^ip\b'
apropos -w 'sched*'
```

Reduce noise to section-1 commands

```
apropos copy | grep -E '^\S+\s+\(1\)' | head -20
apropos -s 1 copy | head
```

Rebuild empty database

```
sudo mandb
# older / alternate systems:
# sudo makewhatis
apropos printf
whatis ls
```

Containers and minimal images often need `man-db` installed **and** `mandb` run once.

Pipeline into selection

```
cmd=$(apropos -s 1 disk | fzf | awk '{print $1}')
man "$cmd"
```

Handy on interactive workstations with `fzf`.

Compare with full-text search

```
apropos AddressFamily       # only names + short desc
man -K AddressFamily        # body text; much slower
```

Use `-K` when you know a config keyword but not which man page owns it.

Notes / Pitfalls

- "nothing appropriate" usually means **stale/missing mandb**, not that Linux lacks the tool.
- Descriptions are short; false positives are common (`time` matches many pages). Narrow with `-s` and `-a`.
- Localized whatis DBs may hide English-only phrasing; try `LANG=C apropos …`.
- `apropos` does **not** search shell builtins; use `help` / `compgen -b` for those.
- Package docs not installed → no man page → not in the index.

2026-relevant notes

- Prefer modern names in queries: `apropos systemd`, `apropos nft`, `apropos iproute` rather than only legacy `ifconfig/iptables` wording if you want current tooling.
- On immutable / ostree systems, man pages may live in different layers; ensure the layer with docs is available before blaming `apropos`.
- AI chat is fine for discovery, but `apropos` + `man` match **your installed version's** flags.

Related Commands

- `whatis` / `man -f` — one-line summary for a known name
- `man -K` — full-text search of page bodies
- `man` — open the full page
- `info --apropos` — search Texinfo manuals
- `mandb` — rebuild the index

Additional Resources

- `man apropos`, `man man`, `man mandb`

whatis

Overview

`whatis` prints **single-line descriptions** from the man-db whatis index. Use it when you want a one-sentence reminder of what a command or page is without opening the full manual. Same idea as `man -f`.

It answers "what is this name?" — not "what command does X?" (that's `apropos` / `man -k`).

Syntax

```
whatis [options] name...
man -f name...
```

Common Options

Option	Description
`-s list`	Only these sections (e.g. `1,8` or `5`)
`-l`	Long / multi-line output when available
`-r`	Interpret name as regex (implementation-dependent)
`-w`	Wildcard match on names
`-v`	Verbose diagnostics
`-d`	Debug (man-db)

Examples with Explanations

Quick one-liners

```
whatis ls
whatis passwd
whatis passwd chmod mount
man -f ls                    # equivalent
```

Disambiguate by section

```
whatis passwd
whatis -s 1 passwd          # the command
whatis -s 5 passwd          # /etc/passwd format
whatis -s 2,3 open          # syscall / libc
```

Multiple names in scripts

```
for c in curl wget aria2c; do
  whatis "$c" 2>/dev/null || echo "$c: no man summary"
done
```

Missing database

```
# typical message: whatis: nothing appropriate
sudo mandb
whatis ls
whatis systemctl
```

Minimal Docker images: install `man-db` and relevant man packages first.

Combine with type / path

```
type mount
command -v mount
whatis mount
man 8 mount
```

`type` shows *how the shell runs it*; `whatis` shows *what the man index claims it is*.

Wildcard / regex (when supported)

```
whatis -w 'system*'
whatis -r '^ip$'
```

Verify pages after package install

```
sudo apt install --reinstall manpages manpages-dev   # Debian/Ubuntu example
sudo mandb
whatis open openat
```

Understanding Output

Typical line shape:

```
ls (1)               - list directory contents
passwd (1)           - change user password
passwd (5)           - the password file
```

Columns: **name**, **(section)**, **short description**. Multiple lines appear when the name exists in several sections.

Exit status is non-zero when no entry matches (handy in scripts).

Notes / Pitfalls

- No output / "nothing appropriate" → run `sudo mandb`, install docs packages, or wrong name.
- Builtins like `cd` may have thin or missing whatis entries; use `help cd`.
- Descriptions can be outdated relative to a newer binary if packages diverged; open `man` when accuracy matters.
- `whatis` does not search descriptions for keywords — use `apropos`.
- Localized systems may show translated one-liners; use `LANG=C whatis …` for English.

2026-relevant notes

- Still the fastest "sanity check" that a tool is installed *with documentation* on the box.
- Pair with `command -v` in onboarding scripts: binary present + whatis present = usable offline reference.
- Some distros split man pages into `*-doc` packages (especially libraries); `whatis` missing does not always mean the binary is missing.

Related Commands

- `apropos` / `man -k` — keyword search across descriptions
- `man` — full page
- `info` — Texinfo manuals
- `command -v` / `type` — how the shell resolves a name

- `mandb` — rebuild the whatis database

Additional Resources

- `man whatis`, `man man`

Files and Paths

Intro

Intro

Navigate the filesystem, create and move data, inspect metadata, search by name or content, and manage permissions (mode bits, ACLs, and attributes). This is the daily CLI surface for almost every task.

Commands in this part

Command	Role
`pwd`	pwd (print working directory) writes the absolute path of the current directory.
`cd`	cd (change directory) sets the shell's current working directory.
`ls`	ls lists directory contents.
`tree`	tree prints a directory hierarchy as an indented tree.
`mkdir`	mkdir (make directory) creates new directories.
`rmdir`	rmdir removes empty directories only.
`touch`	touch updates file timestamps and, by default, creates empty files that do not exist.
`cp`	cp copies files and directories.
`mv`	mv renames or moves files and directories.
`rm`	rm removes directory entries (unlinks files).
`cat`	cat (concatenate) reads files sequentially and writes them to standard output.
`less`	less is a pager: it displays text one screen at a time with searching, scrolling both ways, and optional follow mode.
`more`	more is a simple historical pager that displays text one screen at a time.
`chmod`	chmod changes file mode bits (permissions) and special bits (setuid, setgid, sticky).

Command	Role
`chown`	chown (change owner) sets the user and/or group ownership of files and directories.
`stat`	stat displays detailed inode metadata: size, ownership, mode, timestamps, device/inode numbers, links, and…
`file`	file classifies files by inspecting content magic (and optionally the filesystem type), not just the name extension.
`find`	find walks a directory tree and selects files by name, type, time, size, permissions, and more.
`fd`	fd is a fast, user-friendly alternative to find.
`locate`	locate finds files by name using a prebuilt database (historically mlocate; on modern Ubuntu often plocate).
ripgrep (rg)	ripgrep (rg) recursively searches the filesystem for a regex pattern.
`ln`	ln creates links: hard links (default) or symbolic links (-s).
`readlink`	readlink prints the target of a symbolic link, or with GNU flags like -f/-e/-m, canonicalizes paths by resolving…
`realpath`	realpath prints the resolved absolute path of each argument, expanding .
`basename`	basename strips the directory portion (and optionally a suffix) from a path, leaving the final component.
`dirname`	dirname removes the last path component, returning the directory portion of a path.
`which`	which locates executables by searching the PATH environment variable and printing the path of the first match.
alias / unalias	alias is a shell builtin that creates command shortcuts in the current shell session.
`install`	install copies files with explicit mode (and optional owner/group) and can create destination directories.
`df`	df reports filesystem disk space usage for mounted filesystems.

Command	Role
`du`	du estimates file and directory space usage by walking the tree.
`bat`	bat is a modern cat replacement with syntax highlighting, git integration, line numbers, and automatic paging.
`eza`	eza is a modern replacement for ls (community successor to exa) with colors, git status, icons (optional), tree…
`sha256sum`	sha256sum computes or verifies SHA-256 message digests.
`base64`	base64 encodes binary data to ASCII text, or decodes it back.
`cmp`	cmp compares two files byte by byte.
getfacl / setfacl	POSIX ACLs extend the classic owner/group/other mode bits with named user and group entries.
chattr / lsattr	chattr sets ext file attributes (immutable, append-only, no-dump, etc.); lsattr lists them.
`xxd`	xxd creates a hex dump of a file (or stdin) and can reverse a dump back to binary.
`strings`	strings extracts printable character sequences from binary files.
`namei`	namei walks a path component by component, showing whether each element is a file, directory, or symlink — and…
`umask`	umask sets the file mode creation mask for the current shell: bits that are turned *off* when new files and…
inotify / fanotify	inotify and fanotify deliver filesystem change events without polling: watches, overflow, limits, and the editor-save trap.

Suggested starting points

1. Orientation: `pwd`, `ls`, `cd`, `tree` / `eza`.
2. Mutate carefully: `cp`, `mv`, `rm`, `mkdir` (prefer dry-run / backups when unsure).
3. Permissions: `chmod`, `chown`, then ACLs (`getfacl`) or attributes (`chattr`/`lsattr`) when modes are not enough.
4. Find things: `find` or `fd` for live trees; `locate` for indexed names; `rg` for content.

5. Space: `df`, `du` (and later Storage tools like `ncdu`).
6. React to changes: inotify / fanotify (`tail -F`, watchers, `max_user_watches`) instead of polling loops.

Related parts

- Storage and filesystems — block devices, mounts, LVM
- Text and pipes — process file contents
- Security — SELinux labels when DAC is not the issue

Continue with the individual command pages in this part.

pwd

Overview

`pwd` (print working directory) writes the absolute path of the current directory. Shells often implement `pwd` as a **builtin** that can show a **logical** path (may include symlink components via `$PWD`) or a **physical** path (symlinks resolved).

Use it after deep navigation, in scripts that log context, and whenever logical vs physical paths matter (bind mounts, symlinked project roots).

Syntax

```
pwd [options]
```

Common Options

Option	Description
`-L`	Logical path (`$PWD`; may retain symlink names) — often default for the builtin
`-P`	Physical path (resolve all symlinks)
`--help`	Help (external `/bin/pwd`)

`help pwd` documents the **bash builtin**; `man pwd` documents the external binary. Behavior can differ slightly.

Key Use Cases

1. Confirm location after many `cd`s
2. Capture cwd in scripts and logs
3. Compare logical vs physical paths through symlink trees
4. Build absolute paths for configs and tooling
5. Debug "wrong directory" issues in containers and chroots

Examples with Explanations

Basics

```
pwd
pwd -P
pwd -L
type pwd                # builtin or external?
/bin/pwd -P             # force external binary
```

Capture in scripts

```
current_dir=$(pwd)
current_dir=$PWD          # logical; no subprocess
phys=$(pwd -P)
echo "running in $phys"
```

Prefer `"$PWD"` when you want the logical path without spawning a process; call `pwd -P` when you need the real path.

Symlink illustration

```
mkdir -p /tmp/realdir
ln -sfn /tmp/realdir /tmp/linkdir
cd /tmp/linkdir
pwd -L    # often /tmp/linkdir
pwd -P    # /tmp/realdir
ls -ld .  # may still show link path depending on tools
```

Prompt and logging recipes

```
# Show physical path in a custom prompt experiment
PS1='\u@\h $(pwd -P)$ '

# Log absolute cwd with a command
echo "$(date -Is) cwd=$(pwd -P) cmd=$*" >> ~/cmd.log
```

Resolve then act

```
# Ensure absolute before relative ops
cd "$(pwd)/subdir"

# Physical home resolution
cd -P ~
pwd -P

# Fail if cwd vanished (deleted out from under the shell)
pwd -P || echo "cwd invalid"
```

Compare with realpath

```
pwd -P
realpath .
realpath -e .
```

`realpath .` is often equivalent to physical cwd; `realpath` also works on arbitrary paths, not just cwd.

Directory stack awareness

```
pushd /var/log >/dev/null
pwd
popd >/dev/null
pwd
dirs -v
```

Notes / Pitfalls

- Builtin vs `/bin/pwd` can disagree on default `-L`/`-P` semantics; check `type pwd` and `help pwd`.
- If the directory was deleted while you were inside it, `pwd` may error or show a stale `$PWD`.
- Don't mix logical and physical paths blindly in scripts (e.g. `cd` by logical path, then `pwd -P` for logs).
- `cd` following symlinks leaves a logical path unless you use `cd -P` or `set -o physical` (bash).
- In subshells, cwd changes don't affect the parent: `(cd /tmp && pwd)` vs later `pwd`.

2026-relevant notes

- Containers: `WORKDIR` in the image defines initial cwd; `pwd` inside the container reflects that namespace's mounts.

- Network filesystems and bind mounts: physical paths help correlate with host mounts (`findmnt`, `df`).
- Pair with `realpath` for **file** paths; reserve `pwd` for **directory** context.

Related Commands

- `cd` — change directory
- `realpath` — resolve arbitrary paths
- `readlink -f` — canonical path (GNU)
- `dirname` / `basename` — split path components
- `ls` — list contents
- `findmnt` — what is mounted where

Additional Resources

- `help pwd` (bash), `man pwd`

cd

Overview

`cd` (change directory) sets the shell's current working directory. It is a **shell builtin**, not an external program — each shell (bash, zsh, dash) implements it. Directory changes never affect other already-running processes; only the current shell (and its children) see the new cwd.

Syntax

```
cd [-L|[-P [-e]] [-@]] [directory]
cd -
```

Common Options / operands

Option / form	Description
`-L`	Follow logical path (default); retain symlink components
`-P`	Physical structure; resolve symlinks when determining path
`-e`	With `-P`, exit non-zero if physical path cannot be determined
`-`	Switch to previous directory (`$OLDPWD`)
`~` or empty	Home directory (`$HOME`)
`~user`	Home of `user`
`..` / `.`	Parent / current
`--`	End of options (path starts with `-`)

Key Use Cases

1. Navigate the filesystem in interactive shells
2. Anchor scripts to a known directory
3. Toggle between two locations with `cd -`
4. Control symlink resolution (`-L` vs `-P`)
5. Combine with `pushd`/`popd` for a directory stack

Examples with Explanations

Basic navigation

```
cd /usr/local/bin
cd /var/log
cd ~
cd                          # also home
cd ..
cd ../..
```

Previous directory

```
cd /etc
cd /var/log
cd -                        # back to /etc
cd -                        # toggle again
```

`$OLDPWD` holds the previous directory; `cd -` prints the new path in many shells.

Symlink behavior

```
mkdir -p /tmp/real/sub
ln -sfn /tmp/real /tmp/link
cd /tmp/link/sub
pwd -L                      # /tmp/link/sub
pwd -P                      # /tmp/real/sub
cd -P /tmp/link/sub
pwd                         # physical
```

Paths that start with a dash

```
mkdir -- -weird
cd -- -weird
```

Script: run relative to script location

```
#!/usr/bin/env bash
set -euo pipefail
cd "$(dirname "$(readlink -f "$0")")"
pwd
```

```
# now relative paths are stable regardless of caller cwd
```

Script: require a directory

```
cd /srv/app || { echo "missing /srv/app" >&2; exit 1; }
```

pushd / popd alternative workflow

```
pushd /var/log
pushd /etc/systemd/system
dirs -v
popd
popd
```

`cd` alone does not maintain a stack; use `pushd`/`popd` when you need one.

One-liners

```
cd /usr/local/bin && pwd
cd ~alice
cd "$(dirname "$(realpath ./config.yml)")"
cd "$(mktemp -d)"          # disposable workspace
```

Understanding success and failure

- Success: no output (except `cd -` often prints the path).
- Common errors: permission denied, no such file, not a directory, deleted cwd.
- Exit status is non-zero on failure — always check in scripts (`cd dir || exit 1`).

Notes / Pitfalls

- `cd` in a subshell does not change the parent: `(cd /tmp); pwd` still shows the old path.
- Aliases like `alias cd='cd -P'` change defaults for interactive use only.
- `CDPATH` (if set) makes `cd some` search a list of base directories — surprising in scripts; unset or avoid relying on it in portable scripts.
- Broken symlinks and removed directories: physical `cd -P` fails more honestly.
- `zsh` auto-`cd` (typing a directory name) is a shell feature, not POSIX `cd`.

2026-relevant notes

- Containers and distroboxes: know whether you are on the host or in a named mount namespace before bulk relative operations.
- Network homes (`$HOME` on NFS) make frequent `cd` across mounts latency-sensitive; stay local when compiling.
- Prefer explicit absolute anchors in automation (`cd /opt/app`) over long chains of `cd ...`

Related Commands

- `pwd` — print working directory
- `pushd` / `popd` / `dirs` — directory stack
- `realpath` / `readlink` — resolve paths
- `ls` — list destination
- `findmnt` — mount context

Additional Resources

- `help cd` (bash)
- Bash manual — Bourne Shell Builtins

ls

Overview

`ls` lists directory contents. With long format, sorting, and classification options it is the primary interactive tool for inspecting the filesystem tree. Prefer `eza`/`lsd` if installed for richer defaults; every admin still needs classic `ls` for scripts and minimal systems.

Syntax

```
ls [options] [file...]
```

Common Options

Option	Description
`-l`	Long format (mode, links, owner, group, size, mtime, name)
`-a` / `-A`	All (include `./..`) / almost all (hide `./..` only)
`-h`	Human-readable sizes (with `-l`)
`-r`	Reverse sort
`-t` / `-S` / `-u` / `-c`	Sort by mtime / size / atime / ctime
`-R`	Recursive
`-1`	One entry per line
`-d`	List directories themselves, not their contents
`-F`	Classify (`/` dir, `*` exec, `@` symlink, …)
`-i`	Inode numbers
`-s`	Size in blocks
`--color=auto`	Colorize when stdout is a tty
`-X`	Sort by extension
`-U`	Do not sort (directory order)
`--group-directories-first`	Dirs before files (GNU)
`-N`	Show raw names (less escaping)
`-L`	Follow symlinks for file info

Examples with Explanations

Everyday

```
ls
ls -la
ls -lah
ls -lt                  # newest first
ls -lS                  # largest first
ls -1
```

Hidden and directories

```
ls -A
ls -ld /var/log         # info about the dir itself
ls -ld .??*             # hidden entries without . and ..
```

Sorting and time

```
ls -lt --time-style=long-iso
ls -ltu                 # sort by atime
ls -lc                  # sort by ctime
```

Recursive (careful)

```
ls -R /etc/ssh
# for deep trees prefer tree/find
tree -L 2
find /etc/ssh -maxdepth 2
```

Classification and inodes

```
ls -F
ls -li
ls -lH                  # follow link for size/mtime display variants
```

Globs and multiple paths

```
ls -l *.md
ls -ld /etc /var /usr
ls /usr/bin/python*
```

Scripts: stable output

```
ls -1
ls -1 --quoting-style=shell-escape
# parse carefully; prefer find -print0 for machine use
```

Color and aliases

```
ls --color=auto
alias ll='ls -lah --group-directories-first'
```

Compare modern alternatives

```
ls -lah
eza -lah --git          # if installed
```

Understanding long format

```
-rw-r--r-- 1 alice alice  1234 2026-01-15 10:02 file.txt
\ type+mode  links owner group size mtime       name
```

Leading type char: `-` file, `d` directory, `l` symlink, `b`/`c` devices, `p` fifo, `s` socket.

Notes / Pitfalls

- Output columns are for humans; parsing `ls` in scripts is fragile — use `find`, `stat`, or shell globs.
- Locale affects sort order and date formats; set `LC_ALL=C` for stable ASCII sort.
- `ls` of a symlink without `-l` may show the name only; details need `-l` / `-L`.
- Huge directories: `ls` sorts by default and can be slow; try `-U` or `find`.
- SELinux systems: `-Z` shows security context (GNU ls).

2026-relevant notes

- Many desktops alias `ls` to colorized forms; use `command ls` when you need vanilla behavior.
- `eza`/`lsd`/`ls++` improve UX; still teach classic flags for SSH to minimal servers.
- Combine with `bat`/`less -R` when piping colorized output.

Related Commands

- `eza` / `lsd` — modern listing
- `tree` — hierarchical view
- `find` / `fd` — programmable selection
- `stat` — precise metadata
- `dir` / `vdir` — alternate coreutils front-ends
- `namei` — path component walk

Additional Resources

- `man ls`

tree

Overview

`tree` prints a directory hierarchy as an indented tree. It is excellent for quick structural overviews, documentation screenshots, and seeing project layout without a full recursive `ls`. Often an optional package (`sudo apt install tree` / `dnf install tree`).

For huge trees, limit depth (`-L`) and prune noisy dirs (`-I`). For size-oriented exploration, prefer `du`, `dust`, `duf`, or `ncdu`.

Syntax

```
tree [options] [directory ...]
```

Common Options

Option	Description
`-L n`	Max depth
`-a`	Include hidden files
`-d`	Directories only
`-f`	Print full path prefix for each entry
`-i`	No indentation lines (plain list with nesting)
`-h`	Human-readable sizes (with size options)
`-s`	Raw sizes
`--du`	Report directory sizes (when supported)
`-P pattern`	Include only matching files
`-I pattern`	Exclude pattern (often |-separated)
`-C` / `-n`	Color on / off
`-H url`	HTML output with base URL
`-J` / `-X`	JSON / XML (newer tree)
`--gitignore`	Honor `.gitignore` (newer tree)
`-p` / `-u` / `-g`	Permissions / user / group
`-D`	Show modified time
`--dirsfirst`	List directories before files

Examples with Explanations

Shallow overview

```
tree -L 2
tree -L 3 /etc
tree -d -L 3 /etc/systemd
```

Hide dependency / VCS noise

```
tree -I 'node_modules|.git|dist|target|.venv|__pycache__'
tree -L 2 -I 'node_modules|.git'
```

Hidden files and permissions

```
tree -a -L 1 ~
tree -pug -L 2 /var/www
```

Sizes

```
tree -h -L 2 --du        # if your tree supports --du
tree -h -L 1 /var
```

Full paths and filtering

```
tree -f -L 2
tree -P '*.service' -L 3 /etc/systemd
tree -P '*.md' -I node_modules
```

HTML report

```
tree -H '.' -L 2 -o /tmp/tree.html
# open /tmp/tree.html in a browser
```

JSON for tooling (when available)

```
tree -J -L 2 > tree.json
```

Compare with find / eza

```
tree -L 2
find . -maxdepth 2
eza -T -L 2              # modern ls alternative with tree mode
```

Document a project layout

```
tree -L 2 -I '.git|node_modules' > STRUCTURE.txt
```

Notes / Pitfalls

- Not installed by default on many minimal servers — package name is usually `tree`.
- Unbounded `tree` on `/` or large NFS shares is slow and noisy; always use `-L` / `-I`.
- Patterns for `-P`/`-I` are **not** full shell globs in all versions; check `man tree`.
- Color codes can break logs; use `-n` or redirect carefully.
- Do not treat `tree` as a security audit tool; it follows normal user permissions and may hide unreadable dirs.

2026-relevant notes

- Prefer `eza -T` or `lsd --tree` if you already use modern `ls` replacements; keep classic `tree` for portable docs.
- `--gitignore` support varies; upgrade `tree` or combine with `fd` for git-aware listing.
- In monorepos, generate shallow trees for READMEs rather than committing giant dumps.

Related Commands

- `ls -R` — recursive list
- `find` — programmable walk and filters
- `eza -T` / `lsd --tree` — modern tree views
- `du` / `dust` / `ncdu` — size-oriented
- `fd` — fast file find with ignores

Additional Resources

- `man tree`

mkdir

Overview

`mkdir` (make directory) creates new directories. With `-p` it creates parent paths as needed and does not error if the target already exists — the workhorse flag for scripts. Optional `-m` sets the mode for the created directory (subject to umask unless you understand the interaction).

Syntax

```
mkdir [options] directory...
```

Common Options

Option	Description
`-p`, `--parents`	Create parents; no error if existing
`-m MODE`, `--mode=MODE`	Set permission mode (e.g. `750`, `u+rwx`)
`-v`, `--verbose`	Print a line for each created directory
`-Z`	Set SELinux context (when SELinux enabled)
`--context=CTX`	Set complete SELinux context

Key Use Cases

1. Create project and deploy directory trees
2. Idempotent path ensure in scripts (`-p`)
3. Create with restrictive permissions (`-m 700`)
4. Batch-create sibling dirs with braces
5. Scaffold log/cache layouts

Examples with Explanations

Basics

```
mkdir project
mkdir dir1 dir2 dir3
mkdir -v newdir
```

Parents

```
mkdir -p parent/child/grandchild
mkdir -p /var/lib/myapp/{data,cache,logs}
```

`mkdir -p` is safe to re-run: existing directories are left alone.

Permissions

```
mkdir -m 755 public_dir
mkdir -m 700 private_dir
mkdir -m 750 secure_dir
# mode applies to the leaf created; with -p, behavior for parents is version-specific -
# set explicitly if parents need special modes:
mkdir -p -m 755 /srv/app
chmod 750 /srv/app
```

Brace expansion scaffolds

```
mkdir -p logs/{app,nginx,db}
mkdir -p src/{cmd,internal,pkg} docs scripts
```

Script: ensure writable data dir

```
#!/usr/bin/env bash
set -euo pipefail
DATA_DIR=${DATA_DIR:-/var/lib/myapp}
mkdir -p "$DATA_DIR"
chmod 750 "$DATA_DIR"
```

SELinux (when relevant)

```
mkdir -Z /srv/webdata
# or restorecon after create on labeled systems
```

Parallel with install

```
install -d -m 755 /opt/myapp/bin
install -d -m 700 /opt/myapp/secrets
```

`install -d` is often preferred in packaging for explicit modes.

Failure cases

```
mkdir /proc/foo            # typically fails (pseudo-fs / perms)
mkdir /existing/file/sub   # fails if component is a file
```

Notes / Pitfalls

- Without `-p`, existing directory → error; missing parent → error.
- **umask** affects final mode when you don't fully specify bits; verify with `stat -c %a dir`.
- Race in concurrent scripts: two `mkdir` without `-p` can fail; prefer `-p` for idempotency.
- Creating under sticky dirs (`/tmp`) is fine; deleting others' dirs there is not.
- NFS root_squash: creating as root may result in `nobody` ownership.

2026-relevant notes

- In containers, create runtime dirs in entrypoints with `-p` rather than baking empty layers unless needed for ownership.
- systemd `RuntimeDirectory=` / `StateDirectory=` can replace hand-rolled `mkdir` in unit files — prefer unit directives for services.
- Immutable systems may only allow mkdir on writable mounts (`/var`, `/home`, `/tmp`).

Related Commands

- `rmdir` — remove empty directories
- `rm -r` — remove trees
- `install -d` — create dirs with mode/owner
- `chmod` / `chown` — adjust after create
- `mktemp -d` — secure temporary directories

- `ln -s` — link instead of nesting copies

Additional Resources

- `man mkdir`
- GNU coreutils — mkdir

rmdir

Overview

`rmdir` removes **empty** directories only. It is safer than `rm -r` when you want the kernel to refuse deletion if anything is still inside. For recursive removal of non-empty trees, use `rm -r` (carefully) or empty the tree first with `find`/`rm`.

Syntax

```
rmdir [options] directory...
```

Common Options

Option	Description
`-p`, `--parents`	Remove directory **and** empty ancestors
`--ignore-fail-on-non-empty`	Do not error when a directory is non-empty
`-v`, `--verbose`	Print a line for each directory removed
`--help`	Help
`--version`	Version

There is **no** `-f` force that deletes contents — by design.

Key Use Cases

1. Remove empty directories safely
2. Tear down a path of empty parents (`-p`)
3. Verify a directory is empty (failure means "not empty" or other error)
4. Clean scaffolding left after build/install scripts
5. Safer automation than blanket `rm -rf`

Examples with Explanations

Basic removal

```
mkdir /tmp/empty_dir
rmdir /tmp/empty_dir
```

Non-empty fails

```
mkdir -p /tmp/d
echo x > /tmp/d/file
rmdir /tmp/d
# rmdir: failed to remove '/tmp/d': Directory not empty
rm /tmp/d/file
rmdir /tmp/d
```

Remove parent chain

```
mkdir -p /tmp/a/b/c
rmdir -p /tmp/a/b/c
# removes c, then b, then a if each is empty
```

Equivalent idea: `rmdir /tmp/a/b/c /tmp/a/b /tmp/a` from deepest up.

Verbose

```
rmdir -v empty1 empty2
```

Ignore non-empty (continue batch)

```
rmdir --ignore-fail-on-non-empty dir1 dir2 dir3
echo $?
```

Useful when cleaning many dirs and some still have files.

Test emptiness in scripts

```
if rmdir "$dir" 2>/dev/null; then
  echo "removed empty $dir"
else
  echo "not empty or missing: $dir"
fi
```

Combine with find (empty dirs only)

```
# list empty directories
find /tmp/project -type d -empty

# delete empty directories depth-first
find /tmp/project -depth -type d -empty -delete
# or
find /tmp/project -depth -type d -empty -exec rmdir {} +
```

Brace / multi targets

```
mkdir -p logs/{app,nginx,db}
# ... later, if empty:
rmdir logs/app logs/nginx logs/db logs 2>/dev/null
```

Notes / Pitfalls

- Hidden files (including `.gitkeep`, `.DS_Store`) make a directory non-empty.
- You need write permission on the **parent** directory to remove an entry.
- `rmdir -p` stops when it hits a non-empty ancestor; earlier empties are already gone.
- Never confuse with `rm -r`; `rmdir` will not delete files.
- Mount points and sticky directories (e.g. `/tmp`) follow special rules; you may not remove others' dirs there.

2026-relevant notes

- Build systems and containers often leave empty layers of dirs; `find -type d -empty` is the scalable cleanup.
- Prefer explicit empty-dir removal in deploy scripts over recursive force deletes.
- On immutable OS trees, empty-dir removal may be blocked by filesystem policy — expect EROFS/EPERM.

Related Commands

- `rm` — remove files and (with `-r`) trees
- `mkdir` — create directories
- `find -type d -empty` — locate empty dirs
- `rm -r` — recursive delete (dangerous)
- `unlink` — remove a single filesystem entry

Additional Resources

- `man rmdir`
- GNU coreutils — rmdir

touch

Overview

`touch` updates file timestamps and, by default, **creates** empty files that do not exist. Operators use it to create placeholders, nudge `make` rebuilds, normalize mtimes in tests, and clone timestamps from a reference file.

Linux files typically track at least **atime** (access), **mtime** (content modification), and **ctime** (metadata change). `touch` adjusts atime/mtime; ctime updates as a side effect of the metadata change.

Syntax

```
touch [options] file...
```

Common Options

Option	Description
`-a`	Change access time only
`-m`	Change modification time only
`-c`, `--no-create`	Do not create missing files
`-d STRING`, `--date=STRING`	Parse human-friendly time string
`-t [[CC]YY]MMDDhhmm[.ss]`	Set time in compact form
`-r REF`, `--reference=REF`	Copy timestamps from REF
`--time=WORD`	`access`/`atime`/`use` or `modify`/`mtime`
`-h`	Affect symlink itself (when supported)

Key Use Cases

1. Create empty files / placeholders
2. Update mtime to trigger build systems
3. Set explicit timestamps for tests and packages
4. Copy times from a reference file
5. Batch-create names (`touch file{1..5}`)

Examples with Explanations

Create or refresh

```
touch newfile
touch file1 file2 file3
touch file{1..5}
touch logs/.gitkeep
```

Do not create

```
touch -c missing_file     # no-op if absent; exit 0 typically
ls missing_file
```

atime / mtime only

```
touch -a file             # access time
touch -m file             # modification time
stat file
```

Date strings

```
touch -d '2020-01-01' file
touch -d '2 days ago' file
touch -d '2024-06-15 14:30:00' file
touch -d 'next Monday' file
```

GNU `touch` accepts many `date`-style strings.

Compact `-t` form

```
touch -t 202312201200 file
touch -t 202312201200.30 file
# [[CC]YY]MMDDhhmm[.ss]
```

Reference file

```
touch -r /etc/hosts mycopy
stat -c '%y %n' /etc/hosts mycopy
```

Make and empty targets

```
touch src/main.c          # force rebuild consumers of main.c
make
```

Scripts: ensure path exists as empty file

```
mkdir -p "$(dirname "$out")"
touch "$out"
```

Clear mtime into the past for cleanup tests

```
touch -d '30 days ago' /tmp/old.log
find /tmp -name 'old.log' -mtime +7
```

Understanding timestamps

```
stat file
stat -c 'mtime=%y atime=%x ctime=%z' file
```

Field	Meaning
atime	Last access (often lazy/relatime on modern mounts)
mtime	Last content modification
ctime	Last inode/metadata change (not “creation”)
birth	Creation time on some filesystems (`stat %w`)

relatime / **noatime** mount options mean `touch -a` may not behave as you expect for real reads; the explicit `touch -a` still sets atime.

Notes / Pitfalls

- Creating files requires write permission on the parent directory.
- `touch` on a directory updates the directory's timestamps, not children.
- Timestamp precision and timezone depend on filesystem and locale settings.
- NFS root_squash and permission quirks can make `touch` fail with EACCES.
- `touch` is not a substitute for writing content — use redirection or editors.

2026-relevant notes

- Build caches (Bazel, Nix, container layer caches) care deeply about mtime vs content hashes; know which model your toolchain uses.
- `touch -r` is handy when replaying reproducible package layouts.
- For pure "ensure file exists" in bash, `: >>file` or `umask` + redirect may be clearer than `touch` when you also write content.

Related Commands

- `stat` — display timestamps and inode data
- `date` — print/set system time; format strings
- `find -mtime/-mmin` — select by age
- `mkdir` — create directories
- `install` — install files with mode/owner
- `make` — rebuild based on mtimes

Additional Resources

- `man touch`
- GNU coreutils — touch

cp

Overview

`cp` copies files and directories. For large trees with incremental updates, prefer `rsync`. Mind directory semantics: trailing slashes and whether the destination already exists change results. Default is not recursive — use `-r`/`-R` or `-a` for trees.

Syntax

```
cp [options] source dest
cp [options] source... directory
cp [options] -t directory source...
```

Common Options

Option	Description
`-a`, `--archive`	Same as `-dR --preserve=all` (typical full tree copy)
`-r`, `-R`, `--recursive`	Copy directories recursively
`-i`, `--interactive`	Prompt before overwrite
`-n`, `--no-clobber`	Never overwrite
`-u`, `--update`	Copy only when source is newer or dest missing
`-v`, `--verbose`	Show files copied
`-p`	Preserve mode, ownership, timestamps (subset of archive)
`--preserve=ATTR_LIST`	Fine-grained preserve (`mode,ownership,timestamps,links,xattr,context`)
`-L` / `-P` / `-H`	Always follow / never follow / follow command-line symlinks
`-l` / `-s`	Hardlink / symlink instead of copying data
`--reflink=auto`	CoW lightweight clone when filesystem supports it
`-T`	Treat dest as a normal file (do not put source *into* dest dir)
`-t DIR`	Destination directory first
`--parents`	Create full parent path under dest

Safety

- `cp -a src dest` vs `cp -a src/ dest/` vs `cp -a src/. dest/` behave differently when `dest` exists.
- Dry-run mentally or use `rsync -n` for complex trees.
- Overwriting live configs can break services — copy to `.new` then `mv`.

Examples with Explanations

Files

```
cp file.txt file.bak
cp -i *.conf /etc/myapp/
cp -n important.dat /backup/     # do not overwrite
cp -u -v *.o build/
```

Directories

```
cp -a project/ project-backup/
cp -a src/. dest/                # copy *contents* into dest
cp -r --preserve=timestamps logs/ logs-copy/
```

Reflink (btrfs/xfs when available)

```
cp --reflink=auto big.img big-clone.img
```

Falls back to full copy if CoW clone is unsupported.

Preserve attributes selectively

```
cp --preserve=mode,timestamps src dest
cp -a --no-preserve=ownership src dest   # useful as non-root
```

Parents and target dir

```
cp --parents etc/ssh/sshd_config /backup/
# creates /backup/etc/ssh/sshd_config
cp -t /backup/ a.txt b.txt
```

Symlinks

```
cp -P link dest_link          # copy symlink itself
cp -L link dest_file          # copy the referent's data
```

One-liners

```
cp -a /var/www/html/. /var/www/html.bak/
cp -ai src/* dest/            # interactive archive-ish for files
install -D -m 644 app.conf /etc/app/app.conf   # alternative for install paths
```

Notes / Pitfalls

- Cross-filesystem copies rewrite data; hard links cannot span devices.
- Sparse files: GNU `cp` has `--sparse=always|auto|never` — important for VM images.
- Permissions: non-root cannot always preserve ownership; archive mode still copies content.
- Trailing slash myths: prefer explicit `src/.` into existing `dest/` when you mean "contents".
- `cp` does not delete extraneous files in dest (mirroring needs `rsync --delete`).

2026-relevant notes

- For backups and sync, `rsync -aHAX --info=progress2` is usually better than raw `cp -a`.
- Reflink copies shine on btrfs/xfs with CoW; verify with `stat`/`filefrag` when testing.
- In containers, watch UID maps — preserved numeric owners may not match host users.

Related Commands

- `rsync` — incremental and remote-capable copy
- `install` — copy with explicit mode/owner
- `mv` — rename/move
- `scp` / `sftp` — remote copy over SSH
- `tar` — stream trees with ownership
- `dd` — block-level copy

Additional Resources

- `man cp`
- GNU coreutils — cp

mv

Overview

`mv` renames or moves files and directories. On the **same filesystem**, rename is typically atomic and instant (directory entry update). **Across filesystems**, `mv` copies then deletes, which is slower, non-atomic, and can change performance and link behavior.

Syntax

```
mv [options] source dest
mv [options] source... directory
mv [options] -t directory source...
```

Common Options

Option	Description
`-i`, `--interactive`	Prompt before overwrite
`-n`, `--no-clobber`	Do not overwrite
`-f`, `--force`	Do not prompt (default in many cases)
`-u`, `--update`	Move only when source is newer or dest missing
`-v`, `--verbose`	Print each move
`-b`, `--backup`	Backup existing destinations
`-S SUF`	Backup suffix
`-t DIR`	Target directory first
`-T`	Treat dest as a normal file (not a directory)
`--strip-trailing-slashes`	Remove trailing slashes from sources

Key Use Cases

1. Rename files and directories
2. Move into another directory
3. Atomic replace of configs (`mv new current`)
4. Bulk relocate with shell globs
5. Safer overwrite policies (`-i` / `-n`)

Examples with Explanations

Rename and move

```
mv old.txt new.txt
mv file.txt /tmp/
mv chapter.md docs/
mv -t /backup/ *.log
```

Interactive / no-clobber

```
mv -i *.md docs/
mv -n src dest              # leave dest if it exists
```

Atomic replace pattern

```
cp app.conf app.conf.bak
install -m 644 app.conf.new /etc/myapp/app.conf.new
mv /etc/myapp/app.conf.new /etc/myapp/app.conf
```

Readers opening the path see old or new content, not a partial write (when same filesystem).

Verbose bulk

```
mv -v *.png ~/Pictures/
```

Backup on conflict

```
mv -b file.txt /dest/file.txt
# creates file.txt~ or numbered backups depending on options
```

Directory edge cases

```
mv dir existing_dir/        # moves dir *into* existing_dir
mv -T dir existing_file     # refuse treating dest as directory
```

Cross-device awareness

```
df -P file.txt /mnt/usb
mv file.txt /mnt/usb/       # copy+delete if different devices
```

Swap two files (bash)

```
mv a.tmp a.bak && mv a a.tmp && mv a.bak a
# or use a third temporary name carefully
```

Notes / Pitfalls

- Running processes that already hold an open file descriptor keep the **old inode** after you `mv` a replacement into place — they won't see new content until reopen.
- Cross-device moves break hard links and expand sparse files unless tools special-case them.
- Globs that accidentally match the destination directory can error or nest oddly — quote and order carefully.
- `mv` does not update application-internal paths stored inside files.
- Permissions: need write access on source's parent and dest's parent.

2026-relevant notes

- Deploy pattern: write `foo.new` → fsync → `mv foo.new foo` remains the standard atomic publish on POSIX local filesystems.
- For directory trees across mounts, prefer `rsync` then delete, or `tar` pipelines, for progress and resumability.
- On overlay/container layers, "rename atomicity" still holds within a layer but publishing images is a different model.

Related Commands

- `cp` — copy without removing source
- `rm` — delete
- `rename` — bulk rename (Perl and util-linux variants differ!)
- `rsync --remove-source-files` — move-like with rsync features
- `install` — install to final path with mode
- `ln` — links instead of moving

Additional Resources

- `man mv`

rm

Overview

`rm` removes directory entries (unlinks files). By default it does **not** remove directories unless `-r`/`-R` is given. There is no undelete in coreutils — recovery requires backups, snapshots, or filesystem forensics.

Treat `rm -rf` as a loaded tool: verify the path, prefer dry-runs, and avoid root experiments on broad globs.

Syntax

```
rm [options] file...
```

Common Options

Option	Description
`-r`, `-R`, `--recursive`	Remove directories and their contents
`-f`, `--force`	Ignore missing files; never prompt
`-i`	Prompt before every removal
`-I`	Prompt once before removing many / recursive
`-v`, `--verbose`	Print each path as it is removed
`-d`	Remove empty directories (like `rmdir`)
`--one-file-system`	Do not recurse into different mount points
`--preserve-root`	Refuse to recursively remove `/` (default on GNU)
`--no-preserve-root`	Dangerous override — do not use casually
`--interactive=WHEN`	`never`, `once`, `always`

Safety

Never experiment with `rm -rf /`, `rm -rf /*`, or unquoted root globs as root.

Prefer:

- Dry-run with `find … -print` before `-delete`

- Move to a quarantine directory first
- `trash-cli` / desktop trash when available
- Snapshots (btrfs/zfs/LVM) for real recovery options

Do not rely on `alias rm='rm -i'` alone — **scripts and other shells bypass aliases**.

Examples with Explanations

Files

```
rm file.txt
rm -v file1 file2
rm -f stale.lock          # no error if missing
```

Interactive

```
rm -i *.log
rm -I -r build/           # single confirmation for recursive
```

Directory trees

```
rm -r build/
rm -rf build/             # no prompts; double-check path first
```

Empty directory only

```
rm -d empty_dir           # fails if not empty
rmdir empty_dir           # clearer intent
```

Dry-run then delete with find

```
find ./tmpdir -type f -name '*.tmp' -print
find ./tmpdir -type f -name '*.tmp' -delete
# or interactive:
find ./tmpdir -type f -name '*.tmp' -ok rm {} \;
```

Stay on one filesystem

```
sudo rm -r --one-file-system /mnt/usb/data
```

Avoids descending into unexpected bind mounts.

Quarantine pattern

```
mkdir -p ~/QUARANTINE
mv suspect_dir ~/QUARANTINE/
# later, after review:
rm -rf ~/QUARANTINE/suspect_dir
```

Deleted but space held open

```
rm huge.log
# space may not free until writers close the fd
sudo lsof +L1 | head
sudo lsof | grep '(deleted)'
```

Notes / Pitfalls

- Globs expand in the **shell** before `rm` runs — `rm *` is only as safe as your cwd.
- `rm *` skips dotfiles; `rm -r dir` removes dotfiles **inside** `dir`.
- Write permission on the **parent directory** is required to unlink a name (sticky bit on `/tmp` restricts deleting others' files).
- On CoW/SSD, `shred` is unreliable for sanitization; use disk encryption and secure erase procedures instead.
- Root can remove almost anything — accidental paths under `/var` or `/home` are common outages.

2026-relevant notes

- Prefer immutable infrastructure and snapshots over hoping `rm` is reversible.
- Container layers: deleting in a running container does not shrink the image; rebuild layers properly.
- For selective cleanup, `find`/`fd` + review beats recursive force from the wrong cwd.

Related Commands

- `unlink` — remove a single name
- `rmdir` — empty directories only
- `find -delete` — selective tree deletes
- `trash-put` — soft delete if installed
- `shred` — overwrite content (limited effectiveness)
- `lsof` — find deleted-but-open files

Additional Resources

- `man rm`

cat

Overview

`cat` (concatenate) reads files sequentially and writes them to standard output. Use it to print small files, join files, and write heredocs. For large files, prefer a pager (`less`). For syntax-highlighted viewing, consider `bat`. For following growth, use `tail -f` / `journalctl -f`.

Classic Unix joke still holds: `cat file | something` is often better as `something < file` or `something file`.

Syntax

```
cat [options] [file ...]
```

With no file (or `-`), reads standard input.

Common Options

Option	Description
`-n`	Number all output lines
`-b`	Number non-blank lines
`-s`	Squeeze multiple blank lines
`-A`	Show all non-printing (`-vET`)
`-E`	Show `$` at end of lines
`-T`	Show tabs as `^I`
`-v`	Show non-printing characters
`-u`	Unbuffered (historical; often default)
`--`	End of options

Key Use Cases

1. Print small config/text files
2. Concatenate parts into one file
3. Create files via heredoc
4. Reveal invisible characters (`-A`)

5. Quick copy via redirect

Examples with Explanations

Print files

```
cat file.txt
cat /etc/os-release
cat -n file.txt              # number lines
cat -A file.txt              # show tabs/line endings
```

Concatenate

```
cat part1 part2 part3 > whole
cat part1 part2 >> whole     # append
cat header.json body.json > combined.json
```

Heredocs

```
cat <<'EOF' > greeting.txt
hello
world
EOF

cat <<EOF >> /etc/hosts
# added by bootstrap
10.0.0.5 app.local
EOF
```

Quoted `'EOF'` disables expansion; unquoted `EOF` expands `$vars` and command substitutions.

Here-string and stdin

```
cat <<< 'single line'
cat - <<'EOF' | ssh host 'cat > /tmp/x'
content
EOF
```

Show non-printing / DOS endings

```
cat -A dosfile.txt
# CRLF shows as ^M$
```

Useless use of cat — avoid

```
# avoid
cat file | grep pattern
# prefer
grep pattern file
grep pattern < file
```

Binary caution

```
cat binary.dat                # can mess up terminal
cat binary.dat | xxd | less   # better
```

Multiple files with separators (manual)

```
for f in *.conf; do
  echo "===== $f ====="
  cat "$f"
done | less
```

Notes / Pitfalls

- Large files: use `less`, `tail`, or streaming tools; don't dump multi-GB logs with `cat`.
- `cat file1 file2 > file1` **truncates file1 first** — data loss. Write to a new name.
- Terminal corruption from binary output: `reset` or `tput reset`.
- Order matters for concatenation; globs are sorted by shell locale.
- BusyBox `cat` may support fewer flags.

2026-relevant notes

- Prefer `bat` for interactive reading; keep `cat` for scripts and POSIX pipelines.
- `systemd-cat` sends stdin to the journal — different tool for logging.
- For cloud-init / config blobs, heredoc with `cat <<'EOF'` remains standard in shell provisioning.

Related Commands

- `less` / `more` — pagers
- `bat` — enhanced viewing
- `tac` — reverse line order
- `head` / `tail` — portions of files
- `tee` — copy stdin to file and stdout
- `paste` / `join` — columnar combine

Additional Resources

- `man cat`

less

Overview

`less` is a **pager**: it displays text one screen at a time with searching, scrolling both ways, and optional follow mode. It is the default pager for `man` on most systems and a better interactive viewer than `more` for large logs and command output.

Unlike editors, `less` does not modify files (unless you explicitly open an editor from it).

Syntax

```
less [options] [file ...]
command | less
```

Common Options

Option	Description
`-N`	Show line numbers
`-i` / `-I`	Ignore case in searches (smart / full)
`-S`	Chop long lines (no wrap); horizontal scroll
`-R`	Raw ANSI colors (for colored logs/`rg`/`ls`)
`-F`	Quit if content fits on one screen
`-X`	Don't clear screen on quit (behavior depends on term)
`-n`	Suppress line numbers (default)
`+F`	Start in follow mode (like `tail -f`)
`+G`	Start at end of file
`+/pattern`	Start at first match
`-M` / `-m`	Long / short prompt with position
`-J`	Status column for marks
`--incsearch`	Incremental search (newer less)

Essential keys

Key	Action
`q`	Quit
`Space` / `b`	Page down / up
`d` / `u`	Half page down / up
`j` / `k` or arrows	Line down / up
`g` / `G`	Top / bottom
`/pattern`	Search forward
`?pattern`	Search backward
`n` / `N`	Next / previous match
`&pattern`	Show only matching lines
`F`	Follow (tail -f style); `Ctrl-C` to stop
`ma`	Mark position `a`
`'a`	Go to mark `a`
`s`	Save input to a file (when from pipe, if supported)
`h`	Help
`v`	Open in `$VISUAL` / `$EDITOR`

Examples with Explanations

Open files

```
less /var/log/syslog
less -N /etc/ssh/sshd_config
less +G /var/log/kern.log          # jump to end
less +/Listen /etc/ssh/sshd_config # jump to pattern
```

Pipe command output

```
journalctl -u nginx -n 500 | less
dmesg --color=always | less -R
ls --color=always | less -R
ps auxf | less
```

Follow a growing log

```
less +F /var/log/syslog
# or open normally, then press F
# Ctrl-C leaves follow mode but keeps less open
```

Chop long lines (tables, wide logs)

```
less -S /var/log/app.log
# use arrow keys / horizontal scroll
```

Case-insensitive search

```
less -i messages.log
# then /error matches Error ERROR error
```

Multiple files

```
less *.conf
# :n next file, :p previous file
```

Sensible defaults via environment

```
export LESS='-R -i -M'
export LESSHISTFILE=-          # disable ~/.lesshst if desired
export PAGER=less
export MANPAGER='less -R'
```

Compare with more / bat

```
more file.txt           # limited backward motion
less file.txt           # full interactive pager
bat file.txt            # syntax highlight; still may use less as pager
```

Notes / Pitfalls

- Without `-R`, ANSI color sequences show as junk.
- Binary files: less may warn; use `less -a` / `strings` / `xxd` instead of paging binaries casually.
- Very large files: less is efficient but searching huge unindexed logs still costs I/O.
- `LESSOPEN` preprocessor can auto-decompress `.gz` on some setups — know your distro defaults.
- When used as `PAGER`, interactive keys must work; avoid forcing `cat` as pager except in scripts.

2026-relevant notes

- `journalctl` integrates with pagers; `SYSTEMD_PAGER=less` / `PAGER=less` still apply.
- Colored tooling (`rg`, `fd`, `eza`, `ip -c`) should be paired with `less -R` or `unbuffer`-style hacks when piping.
- `bat` / `delta` improve reading UX but often shell out to `less` underneath.

Related Commands

- `more` — simpler historical pager
- `most` — alternate pager (if installed)
- `bat` — cat clone with highlighting
- `tail -f` / `journalctl -f` — follow only
- `view` / `vim -R` — read-only editor as pager
- `man` — uses a pager by default

Additional Resources

- `man less`, press `h` inside less

more

Overview

`more` is a simple historical **pager** that displays text one screen at a time. On modern Linux, **`less` is almost always the better interactive choice** (bi-directional scroll, better search). `more` remains useful for POSIX familiarity, tiny environments, and scripts that expect the classic pager name.

Syntax

```
more [options] [file ...]
command | more
```

Common Options

Option	Description
`-d`	Prompt with more helpful “press space” text
`-f`	Count logical lines (don’t fold long lines as multiple)
`-l`	Do not treat form-feed specially
`-c` / `-p`	Paint from top / clear then show (terminal-dependent)
`-s`	Squeeze multiple blank lines
`-u`	Suppress underlining
`+num`	Start at line number
`+/pattern`	Start at pattern
`-n num`	Screen size (lines)

Feature sets differ between implementations (util-linux vs BSD heritage).

Common keys

Key	Action
`Space`	Next page
`Enter`	Next line

Key	Action
`q`	Quit
`/pattern`	Search (implementation-dependent)
`h`	Help (if supported)
`b`	Back (often **not** available in classic more)

Limited backward movement is the classic reason people switch to `less`.

Examples with Explanations

Basic

```
more /etc/os-release
more +20 /var/log/dmesg
more +/failed /var/log/syslog
```

Pipelines

```
dmesg | more
seq 1 200 | more
ls -l /usr/bin | more
```

Prefer less when available

```
# interactive systems
less /var/log/syslog

# force more only if required
PAGER=more man ls
```

Start position

```
more +100 /var/log/kern.log
more +/error app.log
```

Tiny environments

```
# busybox / recovery shells may only have more
more /etc/passwd
```

Compare pagers

```
printf '%s\n' {1..200} | more
printf '%s\n' {1..200} | less
```

Notice bidirectional movement and search comfort in `less`.

Notes / Pitfalls

- Default `PAGER` / `MANPAGER` on desktop distros is usually `less`, not `more`.
- Behavior of flags is **less portable** than many coreutils tools.
- Not ideal for huge logs; still reads sequentially.
- Some `more` versions exit automatically at EOF when not a tty — good for scripts, surprising interactively.
- Teaching material still mentions `more`; production muscle memory should favor `less`.

2026-relevant notes

- Recovery initramfs and busybox environments may ship `more` only — know both.
- `systemctl` / `journalctl` respect `$SYSTEMD_PAGER` / `$PAGER`; set to `less -R` for color.
- Don't write new tooling that requires `more`-specific keybindings.

Related Commands

- `less` — preferred full-featured pager
- `most` — alternate pager
- `bat` — highlighting viewer
- `pg` — historical pager (rare)
- `cat` — dump entire file

Additional Resources

- `man more`

chmod

Overview

`chmod` changes file mode bits (permissions) and special bits (setuid, setgid, sticky). Modes can be **symbolic** (`u+x`, `go-rwx`) or **octal** (`755`, `640`). Ownership is separate — see `chown`. ACLs (`setfacl`) can grant additional access beyond the classic rwx triad.

Syntax

```
chmod [options] mode[,mode]... file...
chmod [options] --reference=RFILE file...
```

Common Options

Option	Description
`-R`, `--recursive`	Recurse into directories
`-c`	Report only when a change is made
`-v`	Verbose every file
`-f`	Suppress most errors
`--reference=RFILE`	Copy mode from reference file
`-h`	Affect symlinks themselves where supported

Mode primer

Octal digit	rwx meaning
7	rwx
6	rw-
5	r-x
4	r–
0	—

Common whole modes:

Mode	Typical use
755	Directories / public executables
644	Normal files
600	Private keys / secrets
700	Private directories
640	Group-readable configs
4755	setuid binary (rare; high risk)
1777	Sticky world-writable dir (/tmp style)

Symbolic form: who + op + perms

- who: u (user), g (group), o (others), a (all)
- op: + add, - remove, = set exactly
- perms: r, w, x, X, s, t

Safety

- Recursive chmod -R 777 is almost always wrong on multi-user systems.
- Avoid setuid/setgid unless you fully understand the trust model.
- Directories need the execute bit to be entered (x on dirs = traverse).
- Prefer find -type f/d with separate modes over blind recursive octal.

Examples with Explanations

Octal

```
chmod 644 README.md
chmod 755 scripts/run.sh
chmod 600 ~/.ssh/id_ed25519
chmod 700 ~/.ssh
chmod 640 /etc/myapp/app.conf
```

Symbolic

```
chmod u+x tool.sh
chmod go-rwx secret.txt
chmod a+r file.txt
chmod u=rwx,g=rx,o= file
chmod g+s shared_dir          # setgid directory
chmod +t /shared/uploads      # sticky
```

Capital X (conditional execute)

```
chmod -R a+X project/          # add x to dirs and already-executable files
```

Useful to make trees traversable without making every file executable.

Recursive the safe way

```
find ./public -type d -exec chmod 755 {} +
find ./public -type f -exec chmod 644 {} +
```

Reference mode

```
chmod --reference=good.sh new.sh
```

Diagnose

```
ls -l file
stat -c '%a %A %n' file
namei -l /path/to/file
```

umask interaction (creation, not chmod)

```
umask
umask 022
touch newfile                  # typically 644
mkdir newdir                   # typically 755
```

`chmod` sets absolute/symbolic targets; `umask` affects default creation masks.

Notes / Pitfalls

- Execute on directories: without `x`, you cannot `cd` into them even if `r` is set.
- `noexec` mount option blocks execution regardless of mode.
- ACLs may allow access that `ls -l` mode bits alone don't explain — check `getfacl`.
- Copying with `cp` without `-p`/`-a` may drop modes; `install -m` is explicit.
- Root can chmod almost anything; wrong recursive mode on `/var/lib` causes outages.

2026-relevant notes

- Secrets: prefer `600`/`640` and dedicated system users; consider systemd `LoadCredential=` instead of world-readable files.
- Containers: mode bits still matter for processes sharing volumes; UIDs may be remapped.
- Git tracks executable bit only (`100755` vs `100644`), not full Unix modes — deploy with `install`/`chmod` in packaging.

Related Commands

- `chown` / `chgrp` — ownership
- `umask` — default create mask
- `stat` — numeric mode
- `getfacl` / `setfacl` — ACLs
- `install -m` — copy with mode
- `ls -l` — human view
- `namei -l` — path component modes

Additional Resources

- `man chmod`
- `info coreutils 'File permissions'`

chown

Overview

`chown` (change owner) sets the **user** and/or **group** ownership of files and directories. Only root (or a process with `CAP_CHOWN`) can change the user owner on typical Linux systems. Changing the group may be allowed if you are a member of the target group (rules vary; root always can).

Ownership is central to discretionary access control together with `chmod` modes and ACLs.

Syntax

```
chown [options] OWNER[:GROUP] file...
chown [options] :GROUP file...
chown [options] --reference=RFILE file...
```

Common Options

Option	Description
`-R`, `--recursive`	Operate on directory trees
`-H` / `-L` / `-P`	Symlink traversal policy with `-R`
`-h`	Affect symlink itself, not target (when supported)
`-c`	Report only when a change is made
`-v`	Verbose
`-f`	Suppress most error messages
`--from=CURRENT`	Change only if currently owned by CURRENT
`--reference=RFILE`	Copy ownership from RFILE
`--preserve-root`	Do not recurse on `/` (default on GNU)

OWNER:GROUP forms

Form	Meaning
`user`	Set owner; group unchanged
`user:group`	Set owner and group

Form	Meaning
`user:`	Set owner; group to user's primary group (GNU)
`:group`	Set group only (like `chgrp`)
numeric `0:0`	UID:GID by number

Examples with Explanations

Basic

```
sudo chown alice file.txt
sudo chown alice:alice file.txt
sudo chown alice:devs project/
sudo chown :devs project/          # group only
sudo chown 1000:1000 file.txt      # numeric
```

Recursive web tree

```
sudo chown -R www-data:www-data /var/www/html
sudo chown -R alice:alice /home/alice/proj
```

Reference file

```
sudo chown --reference=/var/www/html/index.html /var/www/html/app.php
```

Conditional change

```
sudo chown --from=root:root alice:alice file.txt
```

Useful in scripts that should not clobber unexpected owners.

Symlinks

```
sudo chown -h alice linkname        # change the link's owner if supported
sudo chown -R -H alice dir_with_links
```

Know whether you mean the symlink or its target; mistakes reassign the wrong object.

Deploy pattern

```
sudo install -o root -g root -m 644 app.conf /etc/myapp/app.conf
sudo chown -R appuser:appuser /var/lib/myapp
```

Often `install`/`rsync --chown` is cleaner than copy + chown.

Audit before change

```
find /srv/data -not -user app -o -not -group app
sudo find /srv/data -not -user app -exec chown app:app {} +
```

Notes / Pitfalls

- Recursive `chown -R` on the wrong path is a classic outage (`/` or `/usr`). Prefer absolute paths and double-check.
- NFS root_squash maps root to `nobody` — remote `chown` may fail or not stick as expected.
- Containers: numeric UIDs inside user namespaces map differently on the host.
- Changing ownership does not change mode bits; pair with `chmod`/`setfacl` as needed.
- Sticky directories and special bits behave independently of owner changes.

2026-relevant notes

- systemd services: prefer `User=` / `Group=` in units and `StateDirectory=` ownership over ad-hoc chown in scripts.
- Rootless Podman/Docker: host UID remapping means `chown` inside may show as high UIDs on the host.
- For bulk sync with ownership, `rsync -a --chown=user:group` is often safer and more visible.

Related Commands

- `chmod` — change mode bits
- `chgrp` — change group only
- `ls -l` / `stat` — inspect ownership
- `id` — your uid/gids
- `install` — copy with owner/mode
- `setfacl` / `getfacl` — ACLs
- `namei -l` — path component owners

Additional Resources

- `man chown`

stat

Overview

`stat` displays detailed **inode metadata**: size, ownership, mode, timestamps, device/inode numbers, links, and filesystem identity. It is more precise than `ls -l` for scripting and debugging ("why can't I write this?", "is this the same file?", "when was mtime?").

GNU `stat` supports custom output formats with `-c` (files) and `-f` (filesystems).

Syntax

```
stat [options] file...
stat -c FORMAT file...
stat -f -c FORMAT file...     # filesystem (GNU)
```

Common Options

Option	Description
`-c FORMAT`, `--format=FORMAT`	Custom file format string
`-f`	Filesystem status instead of file
`-L`	Follow symlinks
`-t`	Terse form
`--printf=FORMAT`	Like `-c` with escape sequences; no forced newline
`-x`	BSD-style verbose (some systems)

Useful format sequences (GNU file)

Seq	Meaning
`%n`	File name
`%s`	Size in bytes
`%a` / `%A`	Octal / human access rights
`%u` / `%U`	UID / user name
`%g` / `%G`	GID / group name

Seq	Meaning
%i	Inode number
%d / %D	Device (decimal / hex)
%h	Hard link count
%F	File type string
%x / %X	atime human / epoch
%y / %Y	mtime human / epoch
%z / %Z	ctime human / epoch
%w / %W	birth time human / epoch (if supported)
%N	Quoted name with symlink target

Examples with Explanations

Default view

```
stat /etc/hosts
stat -L /usr/bin/python3
```

Script-friendly fields

```
stat -c '%a %n' file
stat -c '%U:%G %a %n' file
stat -c '%s bytes  mtime=%y  %n' file
stat -c '%d:%i %n' file          # device:inode identity
```

Compare identities (same file?)

```
stat -c '%d:%i' a b
# equal device+inode → hard-linked or same path
```

Timestamps

```
stat -c 'atime=%x' file
stat -c 'mtime=%y' file
stat -c 'ctime=%z' file
stat -c 'birth=%w' file          # may show '-' if unsupported
stat -c '%Y' file                # mtime epoch for sorting
```

Filesystem info

```
stat -f /var
stat -f -c '%T %b %f %S' /       # type, blocks, free, block size (see man)
df -T /var
```

Batch

```
stat -c '%08a %s %n' /etc/*.conf
find . -type f -printf '' -exec stat -c '%Y %n' {} + | sort -n | tail
```

Permissions debugging

```
namei -l /var/lib/myapp/data.db
stat -c '%A %U %G %n' /var/lib/myapp /var/lib/myapp/data.db
```

Terse / portable caution

```
stat -t file                     # compact; format not for portable scripts
```

Prefer explicit `-c` formats in automation.

Notes / Pitfalls

- **ctime** is metadata change time, not "creation time". Birth/creation is `%w` where supported (ext4, btrfs, …).
- atime may be unreliable under `relatime`/`noatime` mounts.
- Following symlinks (`-L`) vs not changes which inode you inspect.
- BSD `stat` format flags differ completely — GNU examples are Linux-oriented.
- Parsing default multi-line output is fragile; always use `-c`/`--printf` in scripts.

2026-relevant notes

- Still the best quick metadata tool before reaching for Python `os.stat`.
- Network FS may fake or delay some timestamps; verify on the server.
- Combine with `find -printf` for bulk metadata without one `stat` exec per file when possible.

Related Commands

- `ls -l` — human listing
- `file` — content type guess
- `stat` filesystem vs `df` / `findmnt`
- `readlink` / `realpath` — path resolution
- `getfacl` — ACLs beyond mode bits
- `lsattr` / `chattr` — ext attributes

Additional Resources

- `man stat`
- GNU coreutils — stat

file

Overview

`file` classifies files by inspecting **content magic** (and optionally the filesystem type), not just the name extension. Use it on unknown downloads, mystery binaries, compressed blobs, and scripts without a reliable extension.

It reads magic patterns from a database (e.g. `/usr/share/misc/magic`). Classification is heuristic — good for ops triage, not a security sandbox.

Syntax

```
file [options] file...
file -               # read stdin
```

Common Options

Option	Description
`-b`, `--brief`	Don't prefix filename
`-i`, `--mime`	MIME type output
`-I`	MIME with encoding (some versions)
`-z`	Look inside compressed files
`-L`	Follow symlinks
`-s`	Allow special/block files
`-k`	Keep going; don't stop at first match
`-f listfile`	Read filenames from listfile
`-e test`	Exclude a test (e.g. `soft`, `tokens`)
`-h`	Don't follow symlinks (default often)

Examples with Explanations

Basic classification

```
file /bin/ls
file /etc/hosts
file mystery.bin
file photo.jpg note.txt archive.tar.gz
```

Brief and MIME

```
file -b mystery.bin
file -i photo.jpg
# photo.jpg: image/jpeg; charset=binary
file -bi photo.jpg
```

Inside compressed data

```
file -z backup.tar.gz
file -z something.xz
```

Symlinks

```
file /usr/bin/python3
file -L /usr/bin/python3
```

Scripts and interpreters

```
file bootstrap.sh
# may report: Bourne-Again shell script, ASCII text executable
head -1 bootstrap.sh      # shebang check
```

Stdin

```
head -c 256 blob | file -
curl -fsSL https://example.com/file | file -
```

Batch unknown directory

```
file *
find . -type f -print0 | xargs -0 file | grep -i 'executable\|ELF'
```

Guard uploads / pipelines

```
mime=$(file -bi "$upload" | cut -d';' -f1)
case $mime in
  image/jpeg|image/png) echo ok ;;
  *) echo "rejected: $mime" >&2; exit 1 ;;
esac
```

ELF deep dive (after file)

```
file /usr/local/bin/app
readelf -h /usr/local/bin/app
ldd /usr/local/bin/app
```

Notes / Pitfalls

- Extensions lie; `file` can too — polyglots and crafted headers fool magic.
- Text encodings may be guessed; don't treat as authoritative Unicode detection.
- Very large files: `file` only needs the beginning — still be careful with special devices.
- Without `-s`, behavior on device nodes is limited.
- Magic DB age depends on package `libmagic` / `file` updates.

2026-relevant notes

- Container distroless images may omit `file`; copy samples to a debug image.
- For security scanning, use dedicated malware/YARA tooling beyond `file`.
- MIME output pairs well with web and S3 content-type checks in shell tooling.

Related Commands

- `stat` — metadata, not content type
- `hexdump` / `xxd` — raw bytes
- `strings` — printable spans in binaries
- `readelf` / `objdump` — ELF analysis

- `identify` (ImageMagick) — image details
- `ls -l` — names and modes only

Additional Resources

- `man file`
- libmagic / file(1) docs

find

Overview

`find` walks a directory tree and selects files by name, type, time, size, permissions, and more. It can also **act** on matches (`-exec`, `-delete`). Prefer it for precise searches; use `locate` only for pre-indexed name lookups.

Syntax

```
find [path...] [expression]
```

Expression primaries are AND by default; use `-o`, `\!`, and escaped parentheses `\( \)` for complex logic.

Common Options / Primaries

Primary / option	Description
`-name PAT` / `-iname PAT`	Basename glob (case-insensitive with `-iname`)
`-path PAT` / `-ipath PAT`	Full path glob
`-regex PAT`	Path regex (see `-regextype`)
`-type f|d|l|b|c|p|s`	File type
`-size n[cwbkMGT]`	Size (`+n` greater, `-n` less)
`-mtime n` / `-mmin n`	Modified n days / minutes ago
`-atime` / `-ctime`	Access / inode change time
`-user` / `-group`	Owner
`-perm mode`	Permissions (`-perm -u+x`, `-perm /u=x`)
`-empty`	Empty file or directory
`-maxdepth n` / `-mindepth n`	Limit descent
`-xdev`	Do not cross filesystems
`-prune`	Do not descend into match
`-print0`	NUL-delimited paths (safe for weird names)
`-exec cmd {} \;`	Run per file
`-exec cmd {} +`	Batched args (like xargs)
`-delete`	Delete match (implies depth-first)

Key Use Cases

1. Locate files by name, age, or size across trees
2. Clean temp/log artifacts safely
3. Feed paths into greps, compressors, or fixers
4. Audit permissions and setuid files
5. Stay on one mount with `-xdev` during disk hunts

Safety

`-delete` and `-exec rm` are irreversible. Test with `-print` first. Quote globs so the shell does not expand them (`-name '*.log'`). Prefer `-exec … {} +` or `-print0 | xargs -0` for speed and safety with spaces.

Examples with Explanations

Find by name (case-insensitive)

```
find /var/log -iname '*.log' -type f
```

Basename match only; still recurses into subdirectories.

Limit depth

```
find /etc -maxdepth 2 -type f -name '*.conf'
```

Avoids walking huge trees when you know the shape.

Large files on one filesystem

```
find / -xdev -type f -size +500M -printf '%s\t%p\n' 2>/dev/null | sort -nr | head
```

`-xdev` skips other mounts (NFS, USB). `2>/dev/null` hides permission noise.

Modified in the last day

```
find ~/projects -type f -mtime -1
```

`-mtime -1` = less than 24 hours ago (integer day semantics; see also `-mmin`).

Exclude a directory

```
find . -path './.git' -prune -o -path './node_modules' -prune -o -type f -name '*.js' -print
```

`-prune` skips those trees; structure is: prune OR (criteria AND print).

Safe batch action

```
find /var/tmp -type f -mtime +30 -print0 | xargs -0 -r rm -v
```

NUL-delimited pipeline handles spaces; `-r` skips `rm` if empty.

Exec with batching

```
find . -type f -name '*.png' -exec gzip -9 {} +
```

One `gzip` with many args — faster than `\;`.

Permission audit (setuid)

```
find /usr -xdev -perm -4000 -type f -printf '%m %u %p\n' 2>/dev/null
```

Classic security inventory.

Empty directories

```
find . -type d -empty
```

Understanding Output

Default is one path per line (`-print`). Use `-printf` for custom columns (size, mtime, mode). Exit status is non-zero if errors occurred during traversal (e.g. permission denied), even if some matches printed.

Common Usage Patterns

Newest N files under a tree

```
find . -type f -printf '%T@ %p\n' | sort -nr | head | cut -d' ' -f2-
```

Delete with interactive dry-run

```
find /tmp -type f -mtime +14 -print   # review
find /tmp -type f -mtime +14 -delete  # then delete
```

Notes & Pitfalls

- Day filters (`-mtime`) truncate; for "last 2 hours" use `-mmin -120`.
- Globs in `-name` are **not** regex; use `-regex` for regular expressions.
- `find -delete` deletes directories only when empty (after contents removed in depth-first order).
- Prefer `locate` for "I only know a filename and the DB is fresh"; prefer `find` for live criteria.

Related Commands

- `locate` / `updatedb` — indexed name search
- `fd` — modern friendlier finder (if installed)
- `grep -R` — search *contents*
- `du` / `ncdu` — size-oriented exploration
- `xargs -0` — build argument lists from `-print0`

Additional Resources

- `man find`
- GNU Findutils manual

fd

Overview

`fd` is a fast, user-friendly alternative to `find`. Defaults are developer-friendly: colorized output, sensible ignore of hidden files and `.gitignore` entries, parallel traversal, and simple regex patterns. Keep `find` for full POSIX power and exotic predicates; use `fd` for everyday "find me files named …" work.

```
sudo apt install fd-find   # Debian/Ubuntu binary often named fdfind
# symlink tip on Debian:
# mkdir -p ~/.local/bin && ln -s $(command -v fdfind) ~/.local/bin/fd
sudo dnf install fd-find
```

Syntax

```
fd [options] [pattern] [path...]
```

Common Options

Option	Description
`-e EXT`	Extension filter (repeatable)
`-t f/d/l/x/e`	Type: file, dir, symlink, executable, empty
`-H`	Include hidden
`-I`	Don't respect ignore files
`-u` / `-uu`	Unrestricted (like rg's `-u`)
`-x cmd`	Exec per result (`{}` placeholder)
`-X cmd`	Batch exec with all results
`-d N`	Max depth
`-g glob`	Glob mode instead of regex
`-s`	Case sensitive (default may be smart case)
`-p`	Match full path
`-a`	Absolute paths
`-0`	NUL-delimited
`-S size`	Size filter (`+1m`, `-100k`)
`--changed-within`	Time filters (version-dependent)

Key Use Cases

1. Find files by name/extension quickly
2. Run a command on each match (`-x`)
3. Search including gitignored build dirs (`-I` / `-u`)
4. Generate file lists for pipelines (`-0` + `xargs -0`)

Examples with Explanations

By extension and pattern

```
fd -e md
fd -e go -e rs
fd 'config.*\.ya?ml$'
fd -g '*.toml'
```

Types and hidden

```
fd -t d node_modules
fd -t f -e log /var/log
fd -H '^\.env'
```

Exec

```
fd -e log -x rm
fd -e jpg -x convert {} {.}_thumb.jpg   # if ImageMagick present
fd -e md -X wc -l
```

`-x` runs once per file; `-X` batches arguments.

Ignore rules

```
fd pattern           # respects .gitignore
fd -I pattern        # include ignored
fd -u secret         # more unrestricted
```

One-liner recipes

```
# Recent-ish markdown under repo (if --changed-within available)
fd -e md --changed-within 7d

# Safe delete of __pycache__
fd -t d -g __pycache__ -x rm -rf

# Feed ripgrep
fd -e py -0 | xargs -0 rg 'TODO'
```

Notes & Pitfalls

- On Debian/Ubuntu the binary is often **`fdfind`** to avoid clashing with `fdclone`.
- Regex is **not** shell glob by default—use `-g` for globs.
- `-x rm` without care deletes widely; preview with plain `fd` first.
- Color can break scripts—use `fd --color never` or redirect.

2026-relevant notes

- `fd` + `rg` is the modern find/grep pair for source trees; `find` + `grep` remain the portable baseline.
- Large monorepos: prefer `fd`'s parallelism; for network filesystems test both tools.
- Combine with `fzf` for interactive picking: `fd -t f | fzf`.

Comparison to alternatives

Tool	Notes
`fd`	Fast UX defaults
`find`	Full predicates, portable
`locate/plocate`	DB-backed name search
`rg --files`	List files rg would search

Related Commands

- `find` — classic recursive search
- `rg` — content search
- `locate` — indexed names
- `tree` / `eza -T` — visual trees

Additional Resources

- Upstream `fd` README
- `man fd` / `man fdfind`

locate

Overview

`locate` finds files by **name** using a prebuilt database (historically mlocate; on modern Ubuntu often **plocate**). It is much faster than walking the live filesystem with `find` for simple name queries, but results can be **stale** until the database is updated.

Use `find` when you need live data or predicates (size, mtime, permissions, content).

Syntax

```
locate [options] pattern...
```

Common Options

Option	Description
`-i`	Case-insensitive
`-l N` / `-n N`	Limit to N results
`-b` / `--basename`	Match basename only
`-r` / `--regexp`	Extended regex
`-c`	Count matches
`-e`	Only print paths that still exist
`-0`	NUL-delimited output
`-d DB` / `--database=DB`	Alternate database path
`-A`	Match all patterns (AND; when supported)
`-w`	Match whole path as a word (implementation-dependent)

Check `man locate` / `man plocate` for your provider.

Database updates

```
sudo updatedb
# plocate often uses a systemd unit/timer:
sudo systemctl start plocate-updatedb.service
systemctl status plocate-updatedb.timer
```

Daily timers/cron usually refresh automatically. Freshly created files won't appear until the next update.

Examples with Explanations

Basic

```
locate sshd_config
locate -i readme
locate -b '\.bashrc'
locate nginx.conf
```

Limit and count

```
locate -l 20 nginx
locate -n 50 '*.service'
locate -c '*.png'
```

Regex

```
locate -r '/etc/.*\.conf$'
locate -r 'systemd.*\.service$'
```

Basename-only

```
locate -b 'passwd'
# vs full path match noise:
locate passwd | head
```

Existence filter

```
locate -e mydeletedfile     # suppress already-removed paths still in DB
```

NUL-safe handoff

```
locate -0 -b 'TODO.md' | xargs -0 ls -l
```

When locate is empty

```
command -v locate
dpkg -l plocate mlocate 2>/dev/null
sudo apt install plocate     # Debian/Ubuntu example
sudo systemctl start plocate-updatedb.service
```

locate vs find

Need	Tool
Fast name search system-wide	`locate`
Just-created files	`find` / `fd`
Size / mtime / mode filters	`find`
Content search	`rg` / `grep -R`
PATH executables only	`command -v` / `type`

Notes / Pitfalls

- Security/privacy: databases may omit unreadable paths or require permissions to query — don't assume world visibility of private files.
- Patterns are often substring matches against full paths — expect noise without `-b` or regex anchors.
- Network filesystems may be excluded from `updatedb` for performance (see `PRUNEPATHS` / plocate config).
- Different implementations (mlocate vs plocate) change defaults and speed; prefer distro docs.
- `locate` is not a real-time security audit of "what exists now".

2026-relevant notes

- **plocate** is the common modern default — faster compressed DB.
- Immutable/ostree systems may index differently; containers often lack a DB entirely.
- For developer trees, `fd`/`find` inside the project beats system `locate`.

Related Commands

- `find` — live criteria search
- `fd` — modern friendly finder
- `updatedb` / `plocate-updatedb` — refresh DB
- `which` / `type` — executables on PATH
- `dpkg -S` / `rpm -qf` — which package owns a path

Additional Resources

- `man locate`, `man plocate`, `man updatedb`

ripgrep (rg)

Overview

`ripgrep` (`rg`) recursively searches the filesystem for a regex pattern. It respects `.gitignore` by default, skips hidden/binary files sensibly, and is typically much faster than `grep -r` on large codebases.

```
sudo apt install ripgrep
sudo dnf install ripgrep
```

Syntax

```
rg [options] PATTERN [PATH...]
```

Common Options

Option	Description
`-i`	Case insensitive
`-w`	Whole word
`-t TYPE`	File type (`py`, `rust`, `go`, `md`, …)
`-T TYPE`	Exclude type
`-g GLOB`	Include/exclude globs (`-g '!**/vendor/**'`)
`-C N` / `-A` / `-B`	Context lines
`-n` / `-N`	Line numbers on/off
`-l`	Files with matches
`-c`	Count per file
`-o`	Only matching part
`-v`	Invert match
`-u` / `-uu` / `-uuu`	Unrestricted (ignore ignore rules / hidden / binary)
`--hidden`	Search hidden files
`--no-ignore`	Don't respect ignore files
`-F`	Fixed string (no regex)
`-P`	PCRE2 (if built with it)
`-z`	Search compressed files
`--json`	JSON lines output

Option	Description
`-0` with `-l`	NUL file names
`--stats`	Search statistics

Key Use Cases

1. Codebase search ignoring build artifacts
2. Language-scoped searches (`-t go`)
3. Incident hunting in logs (with `-uu` when needed)
4. Pre-commit / review "where is this symbol?"

Examples with Explanations

Basics

```
rg 'process_data'
rg -i 'todo|fixme'
rg -w 'main'
```

Types and globs

```
rg -t python 'import os'
rg -t go -t rust 'TODO'
rg -g '*.qmd' 'include::'
rg -g '!**/node_modules/**' 'left-pad'
```

Context and files

```
rg -C 3 'panic\('
rg -l 'DEPRECATED'
rg -c 'error' /var/log 2>/dev/null
```

Unrestricted search

```
rg -uu 'SECRET_TOKEN'
rg --hidden --no-ignore 'password'
```

One-liner recipes

```
# Who calls this function?
rg -n 'funcName\(' -t go

# JSON for tools
rg --json 'pattern' | head

# Replace workflow: list then edit
rg -l 'OldName' | xargs -r sed -i 's/OldName/NewName/g'   # review first!

# Fixed string with special chars
rg -F 'C++' -t md
```

Notes & Pitfalls

- Default ignore can **hide** hits in `build/`, `dist/`, or gitignored configs—use `-u` deliberately.
- Binary matches are skipped unless unrestricted; good for speed, surprising for firmware blobs.
- Multiline requires `-U` / `--multiline` (see man).
- `grep` is still required for POSIX scripts and tiny systems without rg.

2026-relevant notes

- `rg` + `fd` is the standard modern search pair; IDEs shell out to similar engines.
- For huge monorepos, prefer type/glob filters over unconstrained `-uuu`.
- PCRE2 (`-P`) is optional at build time—don't assume it on every distro build.

Comparison to alternatives

Tool	Role
`rg`	Fast recursive search
`grep -R`	Portable
`ag` / `ack`	Older dev search tools
`git grep`	Only tracked files

Related Commands

- `grep` — standard search
- `fd` — find files by name
- `git grep` — repo-tracked search

- `find … -exec grep` — classic combo

Additional Resources

- `man rg`
- BurntSushi/ripgrep user guide

ln

Overview

`ln` creates **links**: hard links (default) or symbolic links (`-s`). Symlinks are path pointers and can cross filesystems and point at directories. Hard links are additional directory entries for the **same inode**, must stay on one filesystem, and generally cannot link directories on Linux.

Syntax

```
ln [options] target link_name
ln [options] target... directory
ln [options] -t directory target...
```

Common Options

Option	Description
`-s`, `--symbolic`	Create a symbolic link
`-f`, `--force`	Remove existing destination files
`-i`, `--interactive`	Prompt before removing destination
`-n`, `--no-dereference`	Treat link_name as a normal file if it is a symlink to a dir
`-v`, `--verbose`	Verbose
`-r`, `--relative`	Create symlink with relative target path (GNU)
`-T`	Always treat link_name as a normal file
`-t DIR`	Directory to place links into
`-b`	Backup destination before overwrite

Hard vs symbolic

Property	Hard link	Symlink
Cross filesystem	No	Yes
Point to directory	No (normal Linux)	Yes
Broken if target removed	Still has data if any link remains	Becomes dangling

Property	Hard link	Symlink
Relative semantics	N/A (inode)	Stored path can be relative or absolute
`ls -l`	Same inode, no `->`	Shows `-> target`

Examples with Explanations

Symbolic links

```
ln -s /usr/bin/python3 /tmp/py
ln -s ../config/app.yml .
ln -sfn /opt/app/releases/1.2 /opt/app/current   # atomic-ish switch
```

`-f` replaces; with symlinks to dirs, combine carefully with `-n`/`-T` to avoid nesting into the old dir.

Relative symlink (GNU)

```
ln -sr /opt/app/shared/.env /opt/app/current/.env
```

Stores a relative path from link location to target — friendlier when trees move together.

Hard links

```
ln file.txt file-hard.txt
ls -li file.txt file-hard.txt     # same inode
```

Bulk into a directory

```
ln -s /usr/bin/python3 /usr/bin/pip3 ~/bin/
ln -t ~/bin -s /usr/local/bin/app
```

Inspect

```
ls -l link
readlink link
readlink -f link
stat link
namei -l /path/to/link
```

Replace symlink safely

```
ln -sfn /opt/app/releases/1.3 /opt/app/current
# or
ln -s /opt/app/releases/1.3 /opt/app/current.new
mv -Tf /opt/app/current.new /opt/app/current
```

Common failure: relative target confusion

```
cd /tmp
ln -s file.txt link          # target is string "file.txt" relative to link's directory
```

If you create the link elsewhere, a relative target resolves from the **link's** directory, not your cwd at creation time (you stored a string).

Notes / Pitfalls

- `ln -s existing_dir name` when `name` exists as directory may create `name/existing_dir` — use `-T`/`-n` and check.
- Hard links to shared libraries/configs can confuse package upgrades; prefer symlinks for admin sugar.
- Backup tools and containers may treat hard links specially (`rsync -H`).
- Dangling symlinks are valid entries; tools may or may not follow them.
- Permissions on a symlink are largely ignored on Linux; the **target's** mode matters for access through the link.

2026-relevant notes

- Blue/green deploys still use `current` symlink switches extensively.
- Special filesystems (overlay, FUSE) may restrict hard links.
- Prefer `ln -sfn` patterns carefully; test once in a throwaway path.

Related Commands

- `readlink` / `realpath` — resolve links
- `ls -l` / `stat` — inspect
- `cp -a` / `cp -P` — copy preserving links
- `namei` — walk path components
- `find -type l` — find symlinks
- `rm` — remove a link (does not delete hard-linked data until last link)

Additional Resources

- `man ln`

readlink

Overview

`readlink` prints the target of a symbolic link, or with GNU flags like `-f`/`-e`/`-m`, canonicalizes paths by resolving symlinks (similar territory to `realpath`). Essential for debugging link chains, deploy symlinks, and locating the real path of `$0` in scripts.

Syntax

```
readlink [options] file...
```

Common Options (GNU)

Option	Description
`-f`, `--canonicalize`	Canonical path; all but last component must exist
`-e`, `--canonicalize-existing`	All components must exist
`-m`, `--canonicalize-missing`	No component need exist; still normalize
`-n`, `--no-newline`	Do not output trailing newline
`-q`, `-s`	Suppress most errors
`-v`	Verbose errors
`-z`	NUL-terminate output
`--`	End of options

Without flags, `readlink` prints the **raw** symlink contents (may be relative) and fails if the path is not a symlink.

Key Use Cases

1. Show immediate symlink target
2. Resolve full canonical path (`-f`/`-e`)
3. Script location detection
4. Verify deploy `current` links
5. Safe path comparisons

Examples with Explanations

Immediate target

```
ln -s ../etc/hosts /tmp/h
readlink /tmp/h
# ../etc/hosts
```

Canonical forms

```
readlink -f /tmp/h
readlink -e /usr/bin/python3
readlink -m /not/yet/created
```

Flag	Missing path behavior
`-e`	Fail if any component missing
`-f`	Last component may be missing (GNU)
`-m`	Tolerate missing components

Multiple links

```
readlink -f /usr/bin/python /usr/bin/java
```

Script directory pattern

```
#!/usr/bin/env bash
SELF=$(readlink -f "$0")
ROOT=$(dirname "$SELF")
cd "$ROOT"
```

BusyBox / portability note

```
# portable-ish alternative when readlink -f missing:
realpath "$(dirname "$0")"
# or
python3 -c 'import os,sys; print(os.path.realpath(sys.argv[1]))' "$0"
```

Inspect vs ls

```
ls -l /usr/bin/python3
readlink /usr/bin/python3
readlink -f /usr/bin/python3
```

NUL-safe with find

```
find /opt -type l -print0 | while IFS= read -r -d '' L; do
  printf '%s -> %s\n' "$L" "$(readlink "$L")"
done
```

Notes / Pitfalls

- Bare `readlink` **errors** on non-symlinks; `readlink -f` works on regular files too (returns canonical path).
- Relative symlink text is relative to the **link's directory**, not your cwd.
- `readlink -f` behavior differs slightly across BSD/GNU; check man pages on non-Linux.
- Prefer `realpath` when you always want canonicalize semantics by name clarity.
- Race: target can change between resolve and use — TOCTOU in security-sensitive code.

2026-relevant notes

- Deploy tooling still relies on `readlink -f` for resolving release symlinks.
- In containers, resolved paths reflect container mounts, not host paths.
- Combine with `namei -l` when permissions on intermediate components fail.

Related Commands

- `realpath` — canonicalize paths
- `ln -s` — create symlinks
- `ls -l` — show link arrow
- `stat` — inode metadata
- `namei` — stepwise path resolution
- `dirname` / `basename` — split paths

Additional Resources

- `man readlink`

realpath

Overview

`realpath` prints the resolved absolute path of each argument, expanding `.` / `..` and optionally resolving symbolic links. Use it for canonical comparisons, stable script roots, and turning relative paths into absolute ones without reinventing path logic in bash.

GNU `realpath` overlaps with `readlink -f/-e/-m` but is clearer when "canonicalize this path" is the intent.

Syntax

```
realpath [options] file...
```

Common Options

Option	Description
`-e`, `--canonicalize-existing`	All components must exist
`-m`, `--canonicalize-missing`	No component need exist
`-s`, `--no-symlinks`	Do not expand symlinks
`-q`, `--quiet`	Suppress most error messages
`-z`, `--zero`	NUL-terminated output
`--relative-to=DIR`	Print path relative to DIR
`--relative-base=DIR`	Absolute unless under base
`-L` / `-P`	Logical / physical variants (see man)

Default (no `-e`/`-m`) is similar to canonicalizing with last component requirements depending on version — prefer explicit `-e` or `-m` in scripts.

Key Use Cases

1. Absolute canonical paths
2. Relative path between two locations
3. Script root detection

4. Compare whether two path strings name the same place
5. Normalize user input paths

Examples with Explanations

Absolute paths

```
realpath ../file.txt
realpath -e /etc/hosts
realpath -m /not/yet/created
realpath -m ./build/out/binary
```

Relative formatting

```
realpath --relative-to=/home/user /home/user/docs/file.txt
# docs/file.txt
realpath --relative-to="$PWD" /etc/hosts
realpath --relative-base=/srv /srv/app/bin/tool
```

Script root

```
#!/usr/bin/env bash
set -euo pipefail
ROOT=$(realpath "$(dirname "$0")")
# or resolve the script itself first if it may be a symlink:
ROOT=$(dirname "$(realpath -e "$0")")
cd "$ROOT"
```

Same-file check

```
p1=$(realpath -e "$1")
p2=$(realpath -e "$2")
[[ $p1 == "$p2" ]] && echo 'same path'
# stronger: compare device+inode
stat -c '%d:%i' "$p1" "$p2"
```

No symlink expand

```
realpath -s /tmp/linkdir
```

Batch NUL-safe

```
printf '%s\0' ./* | xargs -0 realpath -e
```

Combine with find

```
find . -name '*.so' -print0 | xargs -0 realpath
```

Notes / Pitfalls

- Without `-m`, missing paths error — good for validation, bad for pre-creating destinations.
- Canonicalization depends on current mounts and namespaces (containers differ from hosts).
- Race conditions apply: resolved path may change before use.
- BusyBox may offer a smaller `realpath`; test flags.
- `realpath .` vs `pwd -P` — similar idea for cwd.

2026-relevant notes

- Prefer `realpath` in new scripts over ad-hoc bash string chopping.
- Build systems and monorepos use relative-from-root paths extensively — `--relative-to` helps generate them.
- Network FS: resolving through automount points can trigger mounts as a side effect.

Related Commands

- `readlink -f` — GNU canonicalize via readlink
- `pwd -P` — physical cwd
- `dirname` / `basename` — split components
- `cd -P` — physical change directory
- `stat` — identity via inode
- `findmnt` — mount context

Additional Resources

- `man realpath`

basename

Overview

`basename` strips the directory portion (and optionally a suffix) from a path, leaving the final component. It is the complement of `dirname` for simple path surgery in shell scripts — naming logs, deriving output files, and parsing `$0`.

Syntax

```
basename NAME [SUFFIX]
basename OPTION... NAME...
```

Common Options

Option	Description
`-a`, `--multiple`	Treat all arguments as NAMEs
`-s SUF`, `--suffix=SUF`	Strip suffix (implies multiple mode when combined properly)
`-z`, `--zero`	NUL-terminated output

Traditional form: `basename /path/to/file.txt .txt` → `file`.

Key Use Cases

1. Extract filename from a path
2. Strip extensions
3. Derive names from `$0` or CLI args
4. Build sibling output paths in loops
5. Clean display names in logs

Examples with Explanations

Basics

```
basename /path/to/file.txt
# file.txt
basename /path/to/file.txt .txt
# file
basename /path/to/dir/
# dir
basename /
# /   (or implementation-defined root behavior)
```

Multiple names

```
basename -a /a/b /c/d
basename -a -s .conf /etc/ssh/sshd_config /etc/systemd/logind.conf
```

Script name

```
SCRIPT_NAME=$(basename "$0")
echo "usage: $SCRIPT_NAME [options]"
```

Strip extension patterns

```
f=/data/report.2024.csv
base=$(basename "$f")           # report.2024.csv
stem=$(basename "$f" .csv)      # report.2024
# careful: only strips exact suffix match at end
```

Loop rename

```
for f in /var/log/app/*.log; do
  b=$(basename "$f")
  gzip -c "$f" > "/backup/${b}.gz"
done
```

Combine with dirname

```
path=/opt/app/bin/tool
echo "dir=$(dirname "$path") name=$(basename "$path")"
```

NUL-safe pipeline

```
find . -type f -name '*.md' -print0 |
  while IFS= read -r -d '' f; do
    basename -z "$f"
  done
```

Parameter expansion alternative

```
# bash-only equivalents often used instead of basename
f=/path/to/file.txt
echo "${f##*/}"           # file.txt
echo "${f##*/}" | sed 's/\.txt$//'
# or
b=${f##*/}; echo "${b%.txt}"
```

Builtins avoid spawning `basename` in hot loops.

Notes / Pitfalls

- Suffix strip requires an **exact** trailing match; `.tar.gz` must be stripped carefully (often twice or with special logic).
- Trailing slashes: GNU `basename` handles them; still quote variables.
- Don't use for security canonicalization — use `realpath` / proper validation.
- BusyBox `basename` may lack `-a`/`-z`.
- Filenames with newlines need `-z` / careful reading.

2026-relevant notes

- In hot path bash, prefer `${var##*/}` over external `basename`.
- Still excellent for readability in admin scripts and one-liners.
- Pair with `realpath` when the path may contain `..` or symlinks before basenaming for display only.

Related Commands

- `dirname` — parent path
- `realpath` — canonicalize
- `readlink` — resolve links
- bash parameter expansion — `${f##*/}`, `${f%.*}`
- `cut` / `sed` — ad-hoc parsing

Additional Resources

- `man basename`

dirname

Overview

`dirname` removes the last path component, returning the directory portion of a path. Together with `basename` it covers most shell path-splitting needs: locating a script's directory, ensuring parent dirs exist, and computing sibling paths.

Syntax

```
dirname [OPTION] NAME...
```

Common Options

Option	Description
`-z`, `--zero`	NUL-terminated output

GNU coreutils accepts multiple NAME arguments and prints one line each.

Key Use Cases

1. Find a script's directory
2. `mkdir -p` parents before writing a file
3. Compute sibling paths
4. Normalize path structure in scripts
5. Logging / display of parent locations

Examples with Explanations

Basics

```
dirname /path/to/file.txt
# /path/to
dirname file.txt
# .
dirname /usr/local/bin/
# /usr/local
dirname /usr
# /
dirname /
# /
```

Multiple arguments

```
dirname /a/b /c/d/e
```

Script directory pattern

```
#!/usr/bin/env bash
set -euo pipefail
SCRIPT_DIR=$(cd "$(dirname "$0")" && pwd)
# better when $0 is a symlink:
SCRIPT_DIR=$(cd "$(dirname "$(readlink -f "$0")")" && pwd)
cd "$SCRIPT_DIR"
```

Ensure parent exists

```
out=/var/lib/myapp/data/file.db
mkdir -p "$(dirname "$out")"
touch "$out"
```

Sibling paths

```
conf=/etc/myapp/app.conf
dir=$(dirname "$conf")
cp "$conf" "$dir/app.conf.bak"
```

Combine with realpath

```
abs=$(realpath -m "$1")
parent=$(dirname "$abs")
```

Parameter expansion alternative (bash)

```
f=/path/to/file.txt
echo "${f%/*}"            # /path/to  (careful with no-slash cases)
# dirname is safer for edge cases like no slash → "."
```

NUL-safe

```
find /etc -name '*.conf' -print0 |
  xargs -0 -n1 dirname |
  sort -u
```

Notes / Pitfalls

- `dirname file` (no slash) → `.` not empty string — important for `cd`.
- Trailing slashes are normalized by GNU dirname.
- Don't implement security checks with string dirname alone; canonicalize first.
- Hot loops: bash `${f%/*}` is faster but handle "no slash" yourself.
- `dirname` does not require the path to exist.

2026-relevant notes

- Still the readable choice in installer and devops shell scripts.
- For complex path logic, consider Python/`pathlib` in larger tools.
- Pair with `realpath -m` when creating outputs under computed absolute parents.

Related Commands

- `basename` — final component
- `realpath` — canonicalize
- `readlink -f` — resolve script path
- `mkdir -p` — create parents
- `cd` / `pwd` — navigate and display
- bash expansions — `${f%/*}`

Additional Resources

- `man dirname`

which

Overview

`which` locates executables by searching the `PATH` environment variable and printing the path of the first match. It is a traditional convenience for interactive use. For scripts, prefer **`command -v`** (POSIX) or **`type -P`** (bash) — they integrate better with shell functions, aliases, and builtins.

Syntax

```
which [options] command...
```

Implementations differ (GNU `which`, Debian shell script `which`, BusyBox). Options are **not** fully portable.

Common Options (GNU / common)

Option	Description
`-a`	Show all matches on `PATH`, not just the first
`-s`	Silent; exit status only (some versions)
`--version` / `--help`	Version / help (GNU)

Key Use Cases

1. Interactive "where is this binary?"
2. See which of several installs comes first (`-a`)
3. Quick sanity check after modifying `PATH`
4. Compare with shell resolution (`type`)

Examples with Explanations

Basic

```
which python3
which ls
which docker kubectl
```

All matches

```
which -a python3
which -a java
```

Shows shadowing (e.g. `/usr/local/bin` before `/usr/bin`).

Prefer command -v in scripts

```
# good for scripts
if command -v jq >/dev/null 2>&1; then
  jq --version
fi

# bash: PATH only, ignore functions/aliases
if type -P jq >/dev/null; then
  echo "jq binary found"
fi
```

Compare resolution tools

```
type python3
type -a python3
command -v python3
which python3
which -a python3
```

Tool	Sees aliases	Sees functions	Sees builtins	PATH binaries
`type`	yes	yes	yes	yes
`command -v`	yes*	yes*	yes	yes
`which`	often no	no	no	yes

*shell-dependent; still better than `which` for "what will run".

Debug PATH order

```
echo "$PATH" | tr ':' '\n' | nl
which -a node
```

Silent existence (when -s exists)

```
which -s docker && echo present
# portable:
command -v docker >/dev/null && echo present
```

Notes / Pitfalls

- Debian/Ubuntu historically shipped `which` as a shell script with different flags; don't rely on GNU-only options in portable scripts.
- `which` may not know about shell aliases/functions you use interactively — misleading for "what runs when I type this".
- Hashed commands in bash (`hash -r` to clear) affect speed of lookup, not usually `which`'s external search.
- Relative PATH entries (`.` or empty) are a security footgun — `which` will show what they resolve to.
- In login vs non-login shells, `PATH` differs; results differ.

2026-relevant notes

- Version managers (asdf, nvm, pyenv, mise) put shims early on `PATH` — `which -a` is useful to see real binaries behind shims.
- Containers often have a minimal PATH; `command -v` in entrypoints is the standard check.
- Prefer documenting `command -v` in runbooks and automation.

Related Commands

- `command -v` / `command -V` — POSIX resolution
- `type` / `type -a` — bash detailed resolution
- `whereis` — binary/source/man search (rough)
- `ls -l` / `readlink -f` — inspect the binary found
- `enable` — manage bash builtins

Additional Resources

- `man which` (if present), `help type`, `help command`

alias / unalias

Overview

`alias` is a **shell builtin** that creates command shortcuts in the current shell session. Aliases are primarily for interactive convenience. They are **not** automatically available in non-interactive scripts (unless you explicitly enable/source them). For anything with arguments or logic, prefer a **shell function**.

Persist aliases in `~/.bashrc`, `~/.zshrc`, or a dedicated `~/.bash_aliases` sourced from your rc file.

Syntax

```
alias
alias name='command string'
alias name=$'command with specials'
unalias name
unalias -a
```

Common Options / related

Form	Description
`alias`	List defined aliases
`alias name=value`	Define alias
`unalias name`	Remove one
`unalias -a`	Remove all
`type name`	Show how name resolves
`command name`	Bypass alias/function for external
`\name`	Bypass alias

Examples with Explanations

Everyday shortcuts

```
alias ll='ls -lah'
alias la='ls -A'
alias gs='git status'
alias ..='cd ..'
alias ...='cd ../..'
alias grep='grep --color=auto'
alias ip='ip -c'
alias please='sudo'
```

List and inspect

```
alias
alias ll
type ll
type -a ls
```

Persist (bash)

```
# ~/.bashrc
if [ -f ~/.bash_aliases ]; then
  . ~/.bash_aliases
fi
```

```
# ~/.bash_aliases
alias ll='ls -lah'
alias dc='docker compose'
```

Then `source ~/.bashrc` or open a new shell.

Remove

```
unalias ll
unalias -a              # nuclear; current shell only
```

Bypass aliases

```
command ls
\ls
/bin/ls
```

Essential when an alias breaks a scripted expectation interactively.

sudo and alias expansion (bash)

```
alias sudo='sudo '
# trailing space makes bash expand the next word as an alias
sudo ll                 # may work if ll is alias
```

Better as functions

```
mkcd() { mkdir -p -- "$1" && cd -- "$1"; }
backup() { cp -a -- "$1" "$1.bak.$(date +%Y%m%d%H%M%S)"; }
```

Aliases do not take parameters cleanly; functions do.

Safety alias caveats

```
alias rm='rm -i'
# feels safe; non-interactive scripts still use raw rm
# other users/shells unaffected
```

Notes / Pitfalls

- Non-interactive bash does not expand aliases by default (`shopt expand_aliases` can enable, but functions are clearer).
- Quote carefully: `alias e=echo hi` is wrong; use `alias e='echo hi'`.
- Recursive aliasing and shadowing builtins can confuse (`alias cd=...` — avoid unless intentional).
- zsh/fish have richer abbreviation systems; syntax differs.
- Exporting aliases to child shells is not a thing — source rc files instead.

2026-relevant notes

- Dotfile managers (chezmoi, yadm) commonly template alias collections per machine.
- Prefer shared functions in a small `~/.config/shell/functions` for portability across bash/zsh when possible.
- Tool-provided completions > heavy aliases for complex CLIs (`kubectl`, `docker`).

Related Commands

- Shell functions — parameterized shortcuts
- `type` / `command -v` — resolution
- `ln -s` — filesystem shortcuts
- `hash` — bash command hashing
- `bind` / keymaps — interactive productivity beyond aliases

Additional Resources

- `help alias`, `help unalias`, `man bash`

install

Overview

`install` copies files with **explicit mode** (and optional owner/group) and can create destination directories. It is preferred in Makefiles, packaging, and deploy scripts over ad-hoc `cp` + `chmod` + `mkdir -p` sequences because intent is clear and modes are set in one step.

Despite the name, it is a userland coreutils program — not a package manager.

Syntax

```
install [options] SOURCE DEST
install [options] SOURCE... DIRECTORY
install [options] -t DIRECTORY SOURCE...
install -d [options] DIRECTORY...
```

Common Options

Option	Description
`-d`, `--directory`	Create directories (like `mkdir -p` with mode)
`-m MODE`, `--mode=MODE`	Permission mode (default often `755` for files historically — set explicitly)
`-o OWNER`, `-g GROUP`	Owner/group (requires privileges)
`-t DIR`, `--target-directory=DIR`	Destination directory
`-D`	Create leading components of DEST path
`-b`	Backup existing destination
`-S SUF`	Backup suffix
`-v`	Verbose
`-p`	Preserve timestamps (when supported)
`-C`	Install only if contents differ (compare; GNU)
`--strip` / `-s`	Strip symbol tables from binaries (toolchain)

Key Use Cases

1. Install binaries/scripts to `/usr/local/bin`
2. Drop configs with correct mode/owner
3. Create system directory trees with known permissions
4. Reproducible Makefile `make install` steps
5. Install unit files and server config snippets with correct modes

Examples with Explanations

Install a script or binary

```
sudo install -m 755 bin/tool /usr/local/bin/tool
sudo install -m 755 target/release/app /usr/local/bin/app
```

Create directories

```
sudo install -d -m 755 /etc/myapp
sudo install -d -m 750 -o myapp -g myapp /var/lib/myapp
```

Config with ownership

```
sudo install -m 640 -o root -g adm myapp.conf /etc/myapp/myapp.conf
```

Leading path creation (systemd unit)

```
sudo install -D -m 644 myapp.service \
  /etc/systemd/system/myapp.service
sudo install -D -m 644 myapp.conf \
  /etc/myapp/myapp.conf
```

`-D` creates parent directories of the final DEST file.

Multiple sources into a directory

```
sudo install -m 644 -t /etc/myapp/ conf.d/*.conf
```

Compare before replace (GNU)

```
sudo install -C -m 644 app.conf /etc/myapp/app.conf
```

Avoids unnecessary writes/timestamp bumps when content is identical.

Makefile fragment

```
PREFIX ?= /usr/local
bindir ?= $(PREFIX)/bin

install:
    install -d $(DESTDIR)$(bindir)
    install -m 755 tool $(DESTDIR)$(bindir)/tool
```

Versus cp

```
# verbose multi-step
mkdir -p /opt/app/bin
cp tool /opt/app/bin/tool
chmod 755 /opt/app/bin/tool

# one tool
install -D -m 755 tool /opt/app/bin/tool
```

Notes / Pitfalls

- Default mode if you forget `-m` may not be what you want — **always set `-m`**.
- `-o`/`-g` need privileges; packaging often installs as root in `DESTDIR` then packages own metadata.
- `install` is not atomic across filesystems in a fancy way — still a copy into place; for live configs consider write-temp + `mv`.
- Stripping (`-s`) needs strip tools and is for binaries, not scripts.
- SELinux systems may need `restorecon` after install into labeled paths.

2026-relevant notes

- systemd unit installs: `install -D -m 644 unit /etc/systemd/system/` then `systemctl daemon-reload`.
- Prefer `%install`/`install` in packaging (RPM/deb helpers) with explicit modes for supply-chain clarity.

- User-level installs under `~/.local` pair well with XDG paths and no root.

Related Commands

- `cp` — general copy
- `mkdir -p` — directories only
- `chmod` / `chown` — adjust after the fact
- `rsync` — sync trees
- `install` target in autotools/Meson/CMake — ecosystem convention
- `ln -s` — link instead of copy

Additional Resources

- `man install`

df

Overview

`df` reports **filesystem disk space usage** for mounted filesystems. First stop for "is the disk full?" before descending into per-directory `du`/`dust`/`ncdu`. It answers capacity questions at mount granularity, not "which folder is large?".

Syntax

```
df [options] [file...]
```

When given a path, reports the filesystem containing that path.

Common Options

Option	Description
`-h`, `--human-readable`	Sizes in powers of 1024
`-H`	Powers of 1000
`-T`	Print filesystem type
`-i`	Inode usage instead of blocks
`-l`	Local filesystems only
`-P`	POSIX output format (stable columns)
`--output=FIELD_LIST`	Select columns (GNU)
`-t type` / `-x type`	Include / exclude filesystem type
`-a`	Include pseudo, duplicate, inaccessible

Examples with Explanations

Everyday overview

```
df -h
df -hT
df -h -x tmpfs -x devtmpfs
```

Inodes exhausted?

```
df -ih
```

Millions of tiny files can exhaust inodes while `df -h` still shows free blocks — common on mail spool or container image layers.

Specific path

```
df -h /var
df -h .
df -h /var/lib/docker
```

Shows the mount that holds that path (critical with bind mounts).

Portable scripting

```
df -P / | tail -1
df -P --output=target,pcent /var
```

JSON-ish via findmnt (often clearer)

```
findmnt -D
findmnt -no TARGET,FSTYPE,USE% /
```

Correlate devices

```
df -hT
lsblk -f
findmnt
```

Root reserved space

On ext*, ~5% may be reserved for root. `Use%` at 100% can still allow root writes; non-root fails earlier.

Understanding output

Field	Meaning
Filesystem	Device or source
Size / Used / Avail	Block capacity
Use%	Used / size (rounding applies)
Mounted on	Mountpoint
Type (`-T`)	ext4, xfs, btrfs, nfs, …

Notes / Pitfalls

- tmpfs/overlay/snap noise hides real disks — filter with `-x` / `-t` / `duf -only local`.
- NFS free space can reflect server quotas, not your expectation.
- Btrfs/ZFS free space may need native tools for accurate "allocatable" views.
- Deleted-but-open files still consume space until close (`lsof + (deleted)`).
- Don't parse human `-h` in scripts — use `-P` or `--output` bytes fields.

2026-relevant notes

- Prefer `duf` interactively; keep `df` in scripts and alerts.
- Container hosts: full `/var/lib/containerd` or docker roots are common; `df` the right mount.
- Metrics: node exporters scrape the same ideas — alert on both block and inode %.

Related Commands

- `du` / `dust` / `ncdu` — directory usage
- `duf` — modern df UI
- `lsblk` / `findmnt` — topology and mounts
- `lsof` — open deleted files holding space
- `btrfs filesystem usage` / `zpool list` — advanced FS free space

Additional Resources

- `man df`

du

Overview

`du` estimates **file and directory space usage** by walking the tree. Use it after `df` shows a full mount to find which directories consume space. For interactive exploration, prefer `ncdu`/`dust`; for scripts, `du -sb` or `du -sh`.

Syntax

```
du [options] [file...]
```

Common Options

Option	Description
`-h`	Human-readable
`-s`	Summarize (total only)
`-a`	All files, not just dirs
`-c`	Grand total
`-d N`, `--max-depth=N`	Limit depth
`-x`	One filesystem only
`-b` / `-k` / `-m`	Bytes / KiB / MiB
`--apparent-size`	Logical size vs allocated blocks
`-L`	Follow symlinks
`--exclude=PATTERN`	Skip matching paths
`-t SIZE`	Threshold (GNU)
`--inodes`	Count inodes (GNU)
`-0`	NUL-terminated output

Examples with Explanations

Top-level breakdown

```
du -h -d 1 /var 2>/dev/null | sort -h
du -sh /var/*
```

Summaries

```
du -sh .
du -sh /home/*
du -c -sh /var/log /var/cache
```

One filesystem (skip mounts)

```
du -x -h -d 1 /
```

Avoids descending into `/mnt`, network mounts, etc.

Apparent vs allocated

```
du -sh file.img
du -sh --apparent-size file.img
# sparse files differ a lot
```

Exclude noise

```
du -sh --exclude='.git' --exclude='node_modules' *
```

Sort biggest

```
du -h -d 1 | sort -h
du -h -d 2 /var | sort -h | tail
```

Scripts (bytes)

```
du -sb /var/lib/myapp | awk '{print $1}'
```

Find large files (pair with find)

```
find /var -xdev -type f -size +1G -exec ls -lh {} \;
du -ah /var 2>/dev/null | sort -h | tail -20
```

Notes / Pitfalls

- Permission denied dirs undercount silently unless stderr watched.
- Hard links: GNU `du` counts once per tree walk by default when possible.
- Different mounts under a path inflate totals unless `-x`.
- Snapshots (btrfs/zfs) make "usage" subtle — use native tools for pool accounting.
- `sort -h` needs human units from `du -h`.

2026-relevant notes

- Interactive: `dust`, `ncdu`, `dua`. Scripts: stick to `du`.
- Container graph drivers: measure the host path carefully; overlay layers confuse naive `du`.
- Combine `df` (full?) + `du` (where?) + `lsof` (deleted open files).

Related Commands

- `df` — filesystem free space
- `dust` / `ncdu` — interactive usage
- `find -size` — size predicates
- `ls -lh` — single file sizes
- `btrfs filesystem du` — special accounting

Additional Resources

- `man du`

bat

Overview

`bat` is a modern `cat` replacement with **syntax highlighting**, git integration, line numbers, and automatic paging. Optional package on most distros (`apt install bat` — binary may be `batcat` on Debian/Ubuntu). Excellent for reading code and configs interactively; keep `cat` for portable scripts.

Syntax

```
bat [options] [file...]
batcat [options] [file...]     # Debian/Ubuntu package name collision
```

Common Options

Option	Description
`-n`, `--number`	Show line numbers (often default in some themes)
`-p`, `--plain`	Plain mode (no decorations; closer to cat)
`-A`, `--show-all`	Show non-printable characters
`-l`, `--language`	Force language for highlighting
`-r`, `--line-range`	Only print a range (`N:M`)
`-H`, `--highlight-line`	Highlight a line
`-C`, `--context`	Context lines around highlights (with grep integration)
`--style`	Configure decorations (`auto,full,plain,numbers,…`)
`--theme`	Color theme
`-f`, `--force-colorization`	Color even when piping (careful)
`--paging=never`	Disable pager
`-d`, `--diff`	Show git diff modifications
`-m`, `--map-syntax`	Map glob → language

Examples with Explanations

View files

```
bat README.md
bat /etc/ssh/sshd_config
bat src/main.rs
```

Debian binary name

```
command -v bat batcat
alias bat=batcat          # common convenience on Ubuntu
batcat -p file.txt
```

Plain / script-friendly

```
bat -p file.txt
bat --paging=never -p file.txt | wc -l
```

Ranges and highlights

```
bat -r 20:40 file.go
bat -H 42 config.yaml
bat -r :50 -H 10 app.py
```

Force language

```
bat -l json mystery.conf
bat -l yaml /etc/netplan/01-netcfg.yaml
```

Multiple files and stdin

```
bat *.md
command | bat -l log
curl -fsSL https://example.com/file.yaml | bat -l yaml
```

Git-aware

```
bat -d src/main.rs          # highlight modifications when in a git repo
```

Themes and style

```
bat --list-themes
bat --theme=TwoDark file.rs
bat --style=numbers,changes file.rs
```

As man / pager helper

```
export MANPAGER="sh -c 'col -bx | bat -l man -p'"
# or use bat's own man integration if configured
```

Compare with cat / less

```
cat file.md          # raw dump
bat file.md          # highlight + pager
less -R file.md      # pager only
```

Notes / Pitfalls

- Not installed by default on servers; scripts should not require `bat`.
- Color in pipes can break consumers — default is usually no color when non-tty; use `-p`/`--paging=never` when unsure.
- Debian renames to `batcat` because of another `bat` package.
- Huge minified JSON/logs: disable paging or use `less`/`jq` for structure.
- Theme readability depends on terminal truecolor support.

2026-relevant notes

- Pairs well with `eza`, `fd`, `rg`, `delta` as a modern CLI toolkit.
- `batcache` / automatic syntax detection covers most languages; force `-l` for unusual extensions.
- In CI, stick to `cat` for deterministic plain logs.

Related Commands

- `cat` — portable concatenate/print
- `less` — pager
- `highlight` / `pygmentize` — other highlighters
- `delta` — git diff highlighter
- `jq` — structured JSON view
- `man` — manuals (can pipe through bat)

Additional Resources

- `man bat` / `bat --help`
- sharkdp/bat

eza

Overview

`eza` is a modern replacement for `ls` (community successor to `exa`) with colors, git status, icons (optional), tree view, and friendlier defaults. Install separately (`apt install eza`, `cargo install eza`, etc.). Keep knowing classic `ls` for minimal systems and scripts.

Syntax

```
eza [options] [path...]
```

Common Options

Option	Description
`-l`, `--long`	Long format
`-a`, `--all`	Show hidden (not `.` / `..`)
`-A`	Show hidden including `.` / `..` variants per version
`-h`, `--header`	Header row in long mode
`-g`, `--group`	Show group
`-b`, `--binary`	Binary (power-of-two) sizes
`-B`, `--bytes`	Exact bytes
`--git`	Git status column
`--git-ignore`	Ignore gitignored entries
`-T`, `--tree`	Tree view
`-L`, `--level`	Tree depth
`-s`, `--sort`	Sort field (`name`, `size`, `modified`, …)
`-r`, `--reverse`	Reverse sort
`-t`, `--time` / `--time-style`	Time field / format
`-i`, `--inode`	Inodes
`-u` / `-U`	User / don't show user
`--icons`	Icon glyphs (needs font)
`-1`	One per line
`-d`	List directories as dirs, not contents
`-R`	Recurse without tree graphics

Flags evolve quickly — `eza --help` is authoritative for your version.

Examples with Explanations

Everyday aliases

```
eza
eza -la
eza -lah --git
eza -lah --group --header
```

Suggested aliases:

```
alias ls='eza'
alias ll='eza -lah --git'
alias lt='eza -T -L 2'
```

Sorting

```
eza -l --sort=modified
eza -l --sort=size -r
eza -l --sort=ext
```

Tree view

```
eza -T -L 2
eza -T -L 3 --git-ignore
eza -T -L 2 -a
```

Git-aware listing

```
eza -l --git
eza -l --git --git-ignore
```

Shows modified/untracked markers when inside a repository.

Headers and metadata

```
eza -lbh --header --group
eza -l --inode --links
```

Compare with ls / tree

```
ls -lah
eza -lah --git
tree -L 2
eza -T -L 2
```

Scripts: prefer classic ls

```
# portable scripts
ls -1
# interactive human shell
eza -lah --git
```

Notes / Pitfalls

- Not installed everywhere; don't hard-depend in production automation.
- Icons need a Nerd Font / patched font or they look like tofu boxes.
- Color/git features add cost on huge directories and network FS — fall back to `ls -U`/`find` when needed.
- Option names differ from `ls`; muscle memory transfer is incomplete.
- Output is still human-oriented; don't parse it in scripts.

2026-relevant notes

- `eza` is the maintained path for many who used `exa`.
- Fits a modern toolkit with `bat`, `fd`, `rg`, `delta`, `zoxide`.
- Remote SSH to Alpine/BusyBox hosts: expect plain `ls` only.

Related Commands

- `ls` — portable listing
- `tree` — classic tree
- `lsd` — another modern ls
- `fd` / `find` — selection
- `stat` — precise metadata
- `git status` — full git detail

Additional Resources

- `eza --help`, `man eza` (if packaged)
- eza-community/eza

sha256sum

Overview

`sha256sum` computes or verifies **SHA-256** message digests. Operators use it to confirm ISO/downloads, detect bit-rot or accidental corruption, and check published checksums. It is not encryption and does not prove a file is free of malware — only that content matches an expected hash (or another copy).

Related tools: `sha1sum`, `md5sum` (weaker; avoid for new security checks), `b2sum`, `sha512sum`.

Syntax

```
sha256sum [options] [file...]
sha256sum -c [options] checksums.txt
```

With no files (or `-`), reads stdin. Default output: `<hash>  <filename>` (two spaces = text mode).

Common Options

Option	Description
`-b`, `--binary`	Binary mode marker (`*filename`)
`-t`, `--text`	Text mode (default on GNU/Linux)
`-c`, `--check`	Read checksums from files and verify
`--ignore-missing`	Skip missing files when checking
`--quiet`	When checking, only print failures
`--status`	Silent; exit status only
`--strict`	Non-zero exit on malformed lines
`-w`, `--warn`	Warn about improperly formatted lines
`--tag`	BSD-style output

Examples with Explanations

Hash a file

```
sha256sum ubuntu-24.04.1-live-server-amd64.iso
# e3b0c442…  ubuntu-24.04.1-live-server-amd64.iso
```

Compare the printed digest to the publisher's checksum page (preferably over HTTPS / signed).

Hash several files

```
sha256sum *.iso
sha256sum file1 file2 | tee SHA256SUMS
```

Stdin

```
echo -n 'hello' | sha256sum
cat archive.tar | sha256sum
```

Note: `echo` adds a newline unless `-n`; that changes the hash.

Write and verify a checksum file

```
sha256sum image.iso config.yaml > SHA256SUMS
sha256sum -c SHA256SUMS
# image.iso: OK
# config.yaml: OK
```

Keep `SHA256SUMS` next to the files; run `-c` after transfer or storage.

Quiet / script-friendly check

```
sha256sum -c --status SHA256SUMS && echo all-ok || echo FAIL
sha256sum -c --quiet SHA256SUMS
```

`--status` is ideal for CI: no stdout noise, rely on exit code.

Verify published ISO pattern

```
# After downloading both the ISO and SHA256SUMS (or .sha256) from a trusted source:
sha256sum -c SHA256SUMS 2>&1 | grep iso
# or single line:
echo 'EXPECTEDHASH  ubuntu.iso' | sha256sum -c -
```

Always obtain the expected hash from a **trustworthy** channel (vendor site, signed list), not an untrusted mirror alone.

Directory of artifacts

```
find dist -type f -print0 | sort -z | xargs -0 sha256sum > dist/SHA256SUMS
(cd dist && sha256sum -c SHA256SUMS)
```

Stable sort keeps the checksum file reproducible across machines.

Compare two local copies without a list

```
sha256sum a/big.img b/big.img
# same hash → same content (collision risk is negligible for accidental errors)
```

For byte-identical compare of two paths, `cmp` is enough; hashes shine when one side is remote or listed.

Notes

- On GNU/Linux the "text vs binary" distinction rarely changes results for normal files; Windows tools may differ in line-ending handling.
- Renaming a file does not change its hash; only content does.
- Truncation/corruption almost always changes the digest — a matching hash is strong integrity evidence against random errors.
- SHA-256 is a one-way fingerprint, not a MAC. For authenticity against an attacker, use **signed** checksums (GPG/cosign) or TLS from a trusted origin.
- Huge files: hashing is CPU + full sequential read; expect time proportional to size.

Related Commands

- `md5sum` / `sha1sum` / `sha512sum` / `b2sum` — other digests
- `cmp` — direct byte compare of two files
- `diff` — line-oriented text compare
- `openssl dgst -sha256` — OpenSSL digest interface
- `gpg --verify` — signature verification for published artifacts

Additional Resources

- `man sha256sum`
- Ubuntu installation checksum documentation (per-release)

base64

Overview

`base64` encodes binary data to ASCII text, or decodes it back. Common uses: paste small secrets into YAML/JSON, embed short binaries in scripts, and read tokens from cloud/K8s manifests. It is **encoding, not encryption** — anyone can decode it.

Syntax

```
base64 [options] [file]
base64 --decode [options] [file]
```

Default reads stdin or a single file and writes encoded text to stdout (GNU coreutils).

Common Options

Option	Description
`-d`, `-D`, `--decode`	Decode
`-w COLS`, `--wrap=COLS`	Wrap encoded lines at `COLS` (0 = no wrap; GNU default often 76)
`-i`, `--ignore-garbage`	When decoding, ignore non-alphabet characters
(file)	Input file; omit or `-` for stdin

macOS `base64` flags differ slightly (`-D` vs `-d`); this page targets GNU/Ubuntu.

Examples with Explanations

Encode a file

```
base64 secret.key
base64 -w0 secret.key > secret.b64     # single line (scripts/YAML)
```

`-w0` avoids line breaks that break some config parsers.

Decode to a file

```
base64 -d secret.b64 > secret.key
base64 --decode -i pasted.txt > out.bin
```

Encode/decode stdin pipelines

```
echo -n 'hello' | base64
echo 'aGVsbG8=' | base64 -d
printf 'hello' | base64 -w0
```

Prefer `printf` when you must control trailing newlines precisely.

Small binary in a shell variable (careful)

```
B64=$(base64 -w0 icon.png)
echo "$B64" | base64 -d > icon-copy.png
cmp icon.png icon-copy.png && echo identical
```

Large blobs belong in files, not shell variables.

Kubernetes-style secret (illustrative)

```
echo -n 's3cr3t' | base64 -w0
# put result in Secret data: fields - still not encryption at rest without cluster config
```

Anyone with API read access can base64-decode Secret data unless encryption-at-rest is configured.

PEM / cert peek

```
# Intermediate: binary DER to base64 lines is what PEM body is
base64 -w64 cert.der
openssl x509 -in cert.pem -noout -subject   # prefer openssl for real cert work
```

Ignore whitespace garbage when decoding

```
base64 -d -i email-attachment.b64 > file.bin
```

Helpful when copy-paste introduced spaces or headers.

Notes

- Encoded size 4/3 of input (plus newlines if wrapping).
- Base64 is reversible and **not** a confidentiality control.
- Wrong alphabet/variants (URL-safe base64) may need `tr -- '+/' '-_'` transforms or other tools.
- Always use `-w0` for single-line config values unless the format requires classic MIME wrap.
- Decoding invalid input may produce partial garbage — verify with length checks or hashes when it matters.

Related Commands

- `xxd` / `hexdump` — hex views of binary
- `openssl base64` — alternate CLI
- `sha256sum` — integrity after transfer
- `jq -r` — often used with JSON that embeds base64 fields

Additional Resources

- `man base64`

cmp

Overview

`cmp` compares two files **byte by byte**. It reports the first difference (byte number and line number) or stays silent if they match. Use it to verify copies, compare binaries, or confirm a write succeeded. Prefer `diff` for human-readable text changes; prefer `sha256sum` when comparing against a published digest or many files via a list.

Syntax

```
cmp [options] file1 file2
```

Either file may be `-` for stdin (not both in all cases — see man page).

Common Options

Option	Description
`-l`, `--verbose`	List all differing byte positions and values
`-s`, `--quiet`, `--silent`	No output; exit status only
`-n BYTES`, `--bytes=BYTES`	Compare at most `BYTES`
`-i BYTES`, `--ignore-initial=BYTES`	Skip first `BYTES` of both files
`-i BYTES1:BYTES2`	Skip different prefixes per file
`-b`, `--print-bytes`	Show differing bytes (with `-l`)

Examples with Explanations

Basic equal / not equal

```
cmp a.bin b.bin && echo same || echo differ
cp a.bin a.copy
cmp a.bin a.copy && echo copy-ok
```

Silent success + exit 0 means identical through EOF of the shorter comparison path.

Where they first differ

```
cmp report-v1.dat report-v2.dat
# report-v1.dat report-v2.dat differ: byte 1024, line 1
```

Byte index is 1-based in GNU cmp messages. Line number counts newlines seen — less meaningful for pure binary.

Quiet for scripts

```
if cmp -s "$src" "$dst"; then
  echo 'unchanged'
else
  echo 'needs update'
fi
```

List every difference (small files)

```
cmp -l file1 file2 | head
# byte_num  octal_val1  octal_val2
```

Noisy for large divergent files — stop with `head` or use `diff`/`sha256sum` instead.

Compare after skipping headers

```
cmp -i 512 disk.img partition.img
cmp -i 0:2048 full.img payload.bin
```

Useful when one side has a preamble (MBR, archive header) the other lacks.

Limit length

```
cmp -n 1M image.iso /dev/sdX
```

Compares only the first 1 MiB — e.g. quick check that a written ISO prefix matches without reading the whole medium.

Against a device (careful read-only)

```
sudo cmp -n "$(stat -c%s ubuntu.iso)" ubuntu.iso /dev/sdX && echo usb-matches-iso-prefix
```

Read-only verification after `dd`. Still confirm device identity with `lsblk` first.

Notes

- Exit status: `0` same, `1` different, `2` error (missing file, etc.).
- `cmp` stops at the first difference unless `-l`.
- For large "are these the same?" checks over the network, hashing or `rsync -nc` may be more practical.
- Text normalization (CRLF vs LF) is **not** ignored — bytes must match.
- Sparse files and special files follow normal read semantics; comparing devices needs privileges and caution.

Related Commands

- `diff` — line-oriented text
- `sha256sum` — digest compare / published checksums
- `md5sum` — weaker legacy digest
- `comm` — compare sorted line sets
- `rsync -nc` — dry-run content check via rsync

Additional Resources

- `man cmp`

getfacl / setfacl

Overview

POSIX **ACLs** extend the classic owner/group/other mode bits with named user and group entries. On Ubuntu, `getfacl` displays ACLs; `setfacl` changes them. Use ACLs when several users/groups need different rights on shared directories without resorting to `777` or endless group reshuffles. Classic `chmod` bits remain the base; ACL masks interact with them — always verify with `getfacl` after changes.

Package: `acl` (`sudo apt install acl` if missing). Filesystem must be mounted with ACL support (default for ext4/xfs on modern Ubuntu).

Syntax

```
getfacl [options] file...
setfacl [options] file...
setfacl -b|-k file...          # remove ACLs / default ACLs
```

Common Options

getfacl

Option	Description
`-a`, `--access`	Access ACL only
`-d`, `--default`	Default ACL only (directories)
`-c`, `--omit-header`	Skip file name header
`-R`, `--recursive`	Recurse
`-p`, `--absolute-names`	Don't strip leading `/`
`--all-effective`	Comment effective rights (mask)

Option	Description

setfacl

Option	Description
`-m`, `--modify=acl`	Modify ACL entries
`-M file`	Modify from ACL file
`-x`, `--remove=acl`	Remove entries
`-b`, `--remove-all`	Remove all extended ACL entries
`-k`, `--remove-default`	Remove default ACL
`-R`, `--recursive`	Recurse
`-d`, `--default`	Apply to default ACL (dirs)
`--set=acl`	Replace entire ACL
`-n`, `--no-mask`	Don't recalculate mask (advanced)
`--test`	Dry-run: show resulting ACL

ACL entry forms: `u:user:rwx`, `g:group:rx`, `m::rx`, `o::r`, `u::rwx` (owner), `g::rx` (owning group).

Safety

- Recursive `setfacl -R` can lock out services if you omit needed identities — test on a small tree first (`--test` / non-prod).
- The **mask** entry caps effective rights for named users/groups; surprising "I granted rwx but it's only r-x" is usually the mask.
- Removing ACLs (`-b`) falls back to ordinary mode bits — confirm those bits with `ls -l` afterward.
- Network shares (NFS/SMB) and containers may map IDs differently; ACL usernames must resolve on the host that enforces them.

Examples with Explanations

View ACLs

```
getfacl /srv/shared
getfacl -p /srv/shared/file.txt
ls -l /srv/shared
# files with ACLs show a trailing '+' on the mode, e.g. -rw-rwxr--+
```

Grant a user access

```
sudo setfacl -m u:alice:rwx /srv/shared
sudo setfacl -m u:bob:r-x /srv/shared
getfacl /srv/shared
```

Grant a group and adjust mask

```
sudo setfacl -m g:devs:rwx /srv/shared
getfacl /srv/shared
# check "effective:" comments if mask limits rights
```

Default ACL on a directory (inherit for new files)

```
sudo setfacl -d -m u:alice:rwx /srv/shared
sudo setfacl -d -m g:devs:rwx /srv/shared
sudo setfacl -d -m o::--- /srv/shared
touch /srv/shared/newfile
getfacl /srv/shared/newfile
```

Default ACLs apply to **new** children; existing files need a separate recursive pass if required.

Recursive grant

```
sudo setfacl -R -m g:devs:rwX /srv/shared
sudo setfacl -R -d -m g:devs:rwX /srv/shared
```

Capital `X` in ACL text means execute only if the file is a directory or already executable — same idea as `chmod`'s `X`.

Remove entries / clear ACLs

```
sudo setfacl -x u:bob /srv/shared
sudo setfacl -b /srv/shared/file.txt     # strip extended ACL
sudo setfacl -k /srv/shared              # strip default ACL only
getfacl /srv/shared
```

Copy ACL from one file to another

```
getfacl file1 | sudo setfacl --set-file=- file2
# or
sudo getfacl --access file1 | sudo setfacl --set-file=- file2
```

Dry-run

```
setfacl --test -m u:alice:rw /srv/shared/file.txt
```

Shows the ACL that would be applied without writing it.

Backup and restore ACL tree

```
getfacl -R /srv/shared > shared-acls.backup
# later:
sudo setfacl --restore=shared-acls.backup
```

Paths in the backup must still make sense relative to how `getfacl` recorded them.

Notes

- Install: `sudo apt install acl`.
- `ls -l` `+` flag means "has ACL (or other security xattr)" — inspect with `getfacl`.
- umask and default ACLs both affect new file modes; defaults win for ACL-aware creates when a default ACL exists.
- `chmod` can change the ACL mask as a side effect when ACLs are present — re-check with `getfacl` after chmod.
- Not every backup tool preserves ACLs; use `getfacl --restore`, `rsync -A`, or `tar --acls` consciously.

Related Commands

- `chmod` / `chown` — base mode and ownership
- `ls -l` — trailing `+` hint
- `namei -l` — path component permissions
- `rsync -A` — copy ACLs
- `chattr` — ext file attributes (different mechanism)

Additional Resources

- `man getfacl`, `man setfacl`, `man acl`
- `man 5 acl`

chattr / lsattr

Overview

`chattr` sets **ext** file attributes (immutable, append-only, no-dump, etc.); `lsattr` lists them. These are separate from mode bits (`chmod`) and ACLs (`setfacl`). Classic operator uses: make a file immutable against accidental deletion, mark logs append-only, or exclude paths from `dump`. Attributes are filesystem-specific — full support is strongest on **ext2/3/4**; other FS may ignore or partially support flags.

Syntax

```
lsattr [options] [file...]
chattr [options] mode file...
```

`mode` is `+-=` followed by attribute letters, e.g. `+i`, `-a`, `=i`.

Common Options

lsattr

Option	Description
`-a`	Include dotfiles
`-d`	List directories themselves, not contents
`-R`	Recursive
`-v`	Version/generation if available
`-l`	Long option names for flags (newer e2fsprogs)

chattr

Option	Description
`+X` / `-X` / `=X`	Add / remove / set exactly attribute `X`
`-R`	Recursive
`-V`	Verbose
`-f`	Suppress most errors

Option	Description
`-v version`	Set file version/generation

Important attribute letters

Flag	Meaning (typical)
`i`	**Immutable** — cannot modify, delete, rename, link (root must clear first)
`a`	**Append-only** — open only in append mode for writes
`A`	No atime updates
`c`	Compressed (FS-dependent; often no-op on plain ext4)
`d`	No dump
`e`	Extent format (usually set by FS; don't clear casually)
`j`	Data journaling (ext3/4 specifics)
`s`	Secure deletion (often not honored as expected)
`S`	Synchronous updates
`u`	Undeletable (legacy; rarely useful)

Most operators only need `i` and `a`.

Safety

- **`chattr +i`** as root can lock out application writes until you remember to `-i` — document it.
- Recursive `+i` on a tree can break package upgrades and log rotation.
- Attributes are **not a substitute for backups or proper permissions**.
- On non-ext filesystems, commands may succeed partially or reject flags — verify with `lsattr` and test writes.
- Immutable system files can block `apt`/`dpkg` — clear before upgrades.

Examples with Explanations

List attributes

```
lsattr file.txt
lsattr -d /etc/hostname
lsattr -R /var/log/myapp 2>/dev/null | head
```

A line like `----i---------e-------` `file` means immutable (`i`) plus extent (`e`).

Make a file immutable

```
sudo chattr +i /etc/myapp/critical.conf
lsattr /etc/myapp/critical.conf
# even root cannot overwrite until:
sudo chattr -i /etc/myapp/critical.conf
```

Useful against accidental `rm`/overwrite; not a strong security boundary against a malicious root.

Append-only log

```
sudo chattr +a /var/log/myapp/audit.log
# app can append; cannot truncate/overwrite in place
# to rotate: clear a, rotate, restore a
sudo chattr -a /var/log/myapp/audit.log
```

Coordinate with logrotate (`endscript` hooks) if you use `+a`.

Clear / set exactly

```
sudo chattr -i -a file
sudo chattr = file          # clear all settable attrs (careful)
```

Protect a directory entry (directory itself)

```
sudo chattr +i /important-dir
# prevents adding/removing entries in some cases; know the semantics before using
sudo chattr -i /important-dir
```

Test on non-production paths — immutable directories surprise people.

No-dump flag

```
sudo chattr +d /var/cache/bulk
lsattr -d /var/cache/bulk
```

Hints to `dump(8)` to skip; modern backup tools may ignore this flag unless coded for it.

Combine with permissions

```
sudo chown root:root /etc/myapp/critical.conf
sudo chmod 644 /etc/myapp/critical.conf
sudo chattr +i /etc/myapp/critical.conf
```

Mode bits control who can read; `i` resists modification/deletion.

Notes

- Requires appropriate privileges (usually root) for security-sensitive flags like `i` and `a`.
- `e` (extents) is normal on modern ext4 files — don't treat it as a problem.
- XFS has `chattr`/`lsattr` compatibility for some flags but also its own `xfs_io`/`xfs_admin` tooling.
- Backups: confirm your backup agent preserves attributes if you rely on them after restore.
- Containers: attributes on bind-mounted host ext volumes work; the container root may still need CAP_LINUX_IMMUTABLE-related capabilities for `+i`.

Related Commands

- `chmod` / `chown` — mode and ownership
- `getfacl` / `setfacl` — ACLs
- `ls -l` — basic metadata
- `stat` — inode details
- `chattr` man page attribute list for full alphabet

Additional Resources

- `man chattr`, `man lsattr`
- `man 1 chattr` attribute table

xxd

Overview

`xxd` creates a **hex dump** of a file (or stdin) and can reverse a dump back to binary. Operators use it to inspect magic numbers, protocol payloads, firmware blobs, and corrupted headers. It ships with Vim on many Ubuntu installs (`vim-common`). Alternatives: `hexdump`, `od`, `hexyl` (nicer TUI).

Syntax

```
xxd [options] [file]
xxd -r [options] [file]
```

Default: hex + ASCII side panel. No file means stdin.

Common Options

Option	Description
`-p`, `-ps`, `-postscript`, `-plain`	Continuous plain hex (no addresses/ASCII)
`-i`	C include-style array
`-r`, `-revert`	Reverse: hex dump → binary
`-r -p`	Reverse plain hex
`-g N`, `-groupsize N`	Bytes per group (default 2)
`-c N`, `-cols N`	Bytes per line
`-l LEN`, `-len LEN`	Stop after `LEN` bytes
`-s OFF`, `-seek OFF`	Start at offset (supports `+`/`-` relative forms)
`-u`	Uppercase hex
`-b`	Bits (binary) dump
`-e`	Little-endian style grouping (when supported)
`-a`	Autoskip nul lines (like `*`)

Examples with Explanations

Basic dump

```
xxd /bin/ls | head
xxd -l 64 file.bin
```

First lines show file offset, hex bytes, and ASCII.

Limit and offset

```
xxd -s 0x100 -l 32 firmware.bin
xxd -s -16 file.bin          # last 16 bytes (GNU/common form)
```

Jump to headers, ELF sections, or trailing magic without dumping the whole file.

Plain hex (scripts / compare)

```
xxd -p -l 16 file.bin
# 7f454c4602010100...
xxd -p file.bin | tr -d '\n' | head -c 32; echo
```

C array for embedding

```
xxd -i blob.bin > blob.h
# unsigned char blob_bin[] = { 0x... };
```

Handy for tiny firmware embeds in C/Go build pipelines (prefer proper resource packing for large assets).

Reverse plain hex to binary

```
echo '68656c6c6f' | xxd -r -p > hello.bin
xxd -p hello.bin
# 68656c6c6f
```

Also works with classic `xxd` output via `xxd -r`.

Edit a few bytes (workflow)

```
xxd -l 256 config.bin > config.hex
# edit carefully in editor
xxd -r config.hex > config-new.bin
cmp -l config.bin config-new.bin | head
```

For serious binary patching use proper tools; `xxd` is fine for small fixes.

Inspect file magic

```
xxd -l 4 photo.jpg
# ... ff d8 ff e0 ...  JPEG SOI
xxd -l 8 /bin/ls
# ... 7f 45 4c 46 ...  ELF
file photo.jpg /bin/ls
```

Pair with `file` for classification; `xxd` shows the raw bytes.

Grouping and columns for readability

```
xxd -g 1 -c 16 packet.bin | head
xxd -g 4 -c 16 -u header.bin | head
```

Bits view

```
xxd -b -l 8 flags.bin
```

Shows bit patterns — occasional use for flag bytes.

Notes

- Large dumps flood the terminal — always prefer `-l` / `head` for first look.
- Plain hex reverse (`-r -p`) ignores whitespace; classic reverse expects `xxd`'s address format.
- Package: often `vim-common` on Ubuntu (`dpkg -S $(which xxd)`).
- Offsets in the left column are hexadecimal by default.
- Not a disassembler — use `objdump`/`llvm-objdump` for instructions.

Related Commands

- `file` — classify content
- `strings` — printable spans
- `od` / `hexdump` — other dump formats
- `cmp` — compare binaries
- `sha256sum` — integrity after edits
- `readelf` / `objdump` — ELF structure

Additional Resources

- `man xxd`

strings

Overview

`strings` extracts **printable character sequences** from binary files. Operators use it to skim unknown binaries for version strings, URLs, error messages, file paths, and config keys without a full reverse-engineering setup. It is a triage tool — noisy, incomplete, and easy to miss UTF-16 or compressed data.

Syntax

```
strings [options] file...
```

Reads each file and prints sequences of printable characters longer than a minimum length (default often 4).

Common Options

Option	Description
`-n N`, `--bytes=N`	Minimum sequence length (default 4)
`-a`, `--all`	Scan entire file (not only initialized data sections)
`-t o|d|x`	Print offset in octal/decimal/hex before each string
`-e enc`	Encoding: `s` (7-bit), `S` (8-bit), `b/l` (16-bit big/little), `B/L` (32-bit)
`-w`, `--include-all-whitespace`	Include wider whitespace in sequences
`-f`, `--print-file-name`	Prefix each line with filename
`-d`	Only data sections (object files; opposite of scanning all on some systems)

GNU binutils `strings` is the Ubuntu default (`binutils` package).

Examples with Explanations

Basic scan

```
strings /usr/local/bin/app | less
strings /usr/local/bin/app | grep -i version
```

Longer strings only (less noise)

```
strings -n 10 mystery.bin | head -50
strings -n 8 firmware.img | grep -E 'https?://|\.so|error'
```

Raising `-n` drops short garbage sequences.

Offsets for later xxd

```
strings -t x -n 8 blob | head
# 00004a0 some_version_string
xxd -s 0x4a0 -l 64 blob
```

Hex offsets pair well with `xxd -s`.

UTF-16 little-endian (Windows-ish binaries)

```
strings -e l -n 6 app.exe | head
strings -el -n 6 app.exe | grep -i http
```

Default 7/8-bit scan misses many wide-char strings.

Multiple files

```
strings -f -n 8 /lib/x86_64-linux-gnu/*.so.6 2>/dev/null | grep -i 'OpenSSL\|GNU'
```

`-f` labels which file each hit came from.

Full file scan

```
strings -a -n 8 packed.bin | less
```

Some binaries hide strings outside default sections; `-a` scans everything.

Pipe from another command

```
dd if=/dev/sdX bs=1M count=2 status=none | strings -n 8 | head
curl -fsSL https://example.com/file | strings -n 6 | head
```

Useful for quick content smell tests on images or downloads.

Hunt secrets carefully

```
strings -n 12 config.dump | grep -iE 'password|token|secret|api_key'
```

May surface accidental leaks in binaries or core dumps — handle output as sensitive.

Notes

- Compressed or encrypted regions will not yield meaningful plaintext.
- Minimum length trade-off: low `-n` → noise; high `-n` → missed short tokens.
- Not a substitute for proper malware analysis sandboxes.
- Output can be huge; always filter with `grep`/`head`/`less`.
- License strings and compiler markers often identify build toolchain and versions.

Related Commands

- `file` — type classification
- `xxd` / `hexdump` — raw bytes
- `readelf` / `objdump` — structured ELF
- `grep -a` — treat binary as text for patterns
- `binwalk` — firmware carving (separate install)
- `nm` / `objdump -T` — symbol tables when not stripped

Additional Resources

- `man strings`

namei

Overview

`namei` walks a path component by component, showing whether each element is a file, directory, or symlink — and optionally the permission checks a process would face. Excellent for debugging "permission denied" on deep paths and symlink loops.

Package: usually `util-linux` (preinstalled on Ubuntu).

Syntax

```
namei [options] pathname...
```

Common Options

Option	Description
`-l`	Long listing (mode, owner, group)
`-m`	Mode bits only style (see man)
`-o`	Show owner
`-v`	Vertical layout
`-x`	Show mount point crossings
`-n`	Don't follow symlinks (show link itself)

Examples with Explanations

Diagnose a path

```
namei -l /var/www/html/index.html
```

Reveals which directory lacks `x` for your user or which symlink target is wrong.

Follow ownership along the way

```
namei -l /home/alice/.ssh/authorized_keys
```

Classic SSH "modes are too open" / wrong ownership investigations.

Mount boundaries

```
namei -x /mnt/data/subdir/file
```

Shows when a path crosses into another mount.

Notes & Pitfalls

- Interprets the path as the **calling user** would for access checks when using modes.
- Symlink cycles are reported instead of hanging forever.
- Still verify with `ls -ld` on each component for ACLs (`getfacl`).

Related Commands

- `ls -ld` — single component metadata
- `realpath` / `readlink -f` — resolve final path
- `stat` — inode details
- `getfacl` — ACL beyond mode bits

Additional Resources

- `man namei`

umask

Overview

`umask` sets the **file mode creation mask** for the current shell: bits that are turned *off* when new files and directories are created. It is a shell built-in (and a system call); this page covers the interactive/operator usage. Default Ubuntu login shells often use `0002` or `0022`.

Syntax

```
umask [-p] [-S] [mode]
```

Common Options

Option	Description
(none)	Print current mask (octal)
`-S`	Symbolic print (`u=rwx,g=rx,o=rx`)
`-p`	Print as a reusable `umask …` command
`mode`	Set mask (octal like `027` or symbolic)

Key Use Cases

1. Tighten default permissions for shared servers
2. Understand why new files are `644` vs `664`
3. Script sections that create secrets with stricter modes

Examples with Explanations

Show current mask

```
umask
umask -S
```

How mask applies

```
umask 022
touch /tmp/a; mkdir /tmp/d
ls -l /tmp/a /tmp/d
# file 666 & ~022 → 644; dir 777 & ~022 → 755
```

Base modes before umask: files `666`, directories `777` (exec bits matter for dirs).

Stricter group isolation

```
umask 027
# new files 640, dirs 750 - others get nothing
```

Secrets in a script

```
old=$(umask)
umask 077
install -m 600 /dev/null "$HOME/.config/myapp/token"
umask "$old"
```

Prefer `install -m` / explicit `chmod` for critical files; umask is a safety net.

Persist for login shells

```
# ~/.profile or /etc/profile.d/umask.sh
umask 027
```

systemd user services and cron jobs may **not** read your interactive profile — set umask there too if needed.

Notes & Pitfalls

- umask does not change existing files (`chmod` does).
- ACLs default masks on directories can further alter effective rights.
- Numeric umask is **not** the resulting mode; it is the bits stripped from the base.
- BusyBox/dash vs bash symbolic forms can differ slightly.

Related Commands

- `chmod` / `chown` — explicit modes/ownership
- `install` — create files with mode in one step
- `getfacl` / `setfacl` — ACL defaults for shared dirs
- `stat` — inspect resulting mode

Additional Resources

- `help umask` (bash)
- `man 2 umask`

Watching the filesystem: inotify vs fanotify

Overview

Thesis: Linux can tell you when files change, so you do not have to poll. Two kernel APIs do this. **inotify** (Linux 2.6.13) watches individual inodes. It needs no privilege and is what `tail -f`, editors, file managers, build watchers and systemd `.path` units use. **fanotify** (2.6.37) can mark a whole mount or filesystem and can even *block* an open until a scanner answers. Those powers need `CAP_SYS_ADMIN`. Since Linux 5.13 there is also a restricted unprivileged mode. Both APIs have sharp edges that show up in production: watches are **not recursive**, the queue **overflows** and drops events, editors **replace** files instead of writing them, and every watch counts against **per-user limits**. When a watcher "randomly misses changes", one of those four is almost always the reason.

Typical operator and programmer jobs:

- Explain why a dev server or log shipper stopped reacting after an editor save, a `git checkout` or a log rotation.
- Find which process uses up the user's inotify watches when tools fail with "no space left on device" or "too many open files".
- Write a watcher that survives event bursts (handle `IN_Q_OVERFLOW`) and new subdirectories (the add-then-scan race).
- Decide when fanotify fits better (whole-filesystem audit, on-access scanning, pre-content hooks) and what it needs.

```
uname -r
python3 --version
tail --version | head -1
```

```
6.12.94+
Python 3.13.5
tail (GNU coreutils) 9.7
```

All output below comes from a Debian 13 host, kernel 6.12 (an LTS series), x86_64, as an unprivileged user (`uid=1000`, no capabilities). Every listing ran there as shown. The examples call the syscalls through Python's standard `ctypes` module, so they need no extra packages. `inotifywait`/`inotifywatch` from **inotify-tools** wrap the same API on the command line. They were not installed on this box, so their use is described from their manual and not run.

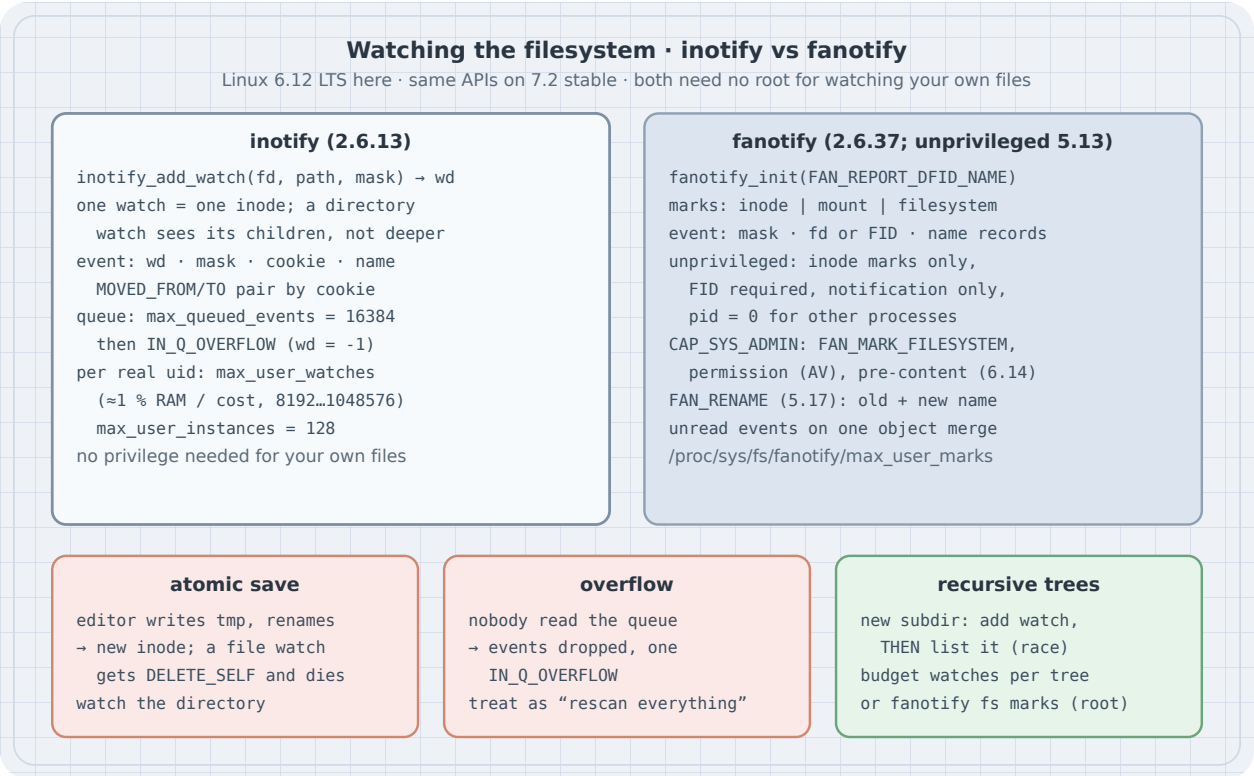


Figure 1: inotify vs fanotify

Mental model

inotify works like this:

- `inotify_init1()` returns a file descriptor: an **instance** with its own event queue.
- `inotify_add_watch(fd, path, mask)` resolves `path` once, attaches a **watch** to that *inode* and returns a **watch descriptor** (`wd`, a small integer).
- From then on the kernel appends `struct inotify_event { wd, mask, cookie, len, name[] }` records to the queue, and you `read()` them. The fd works with `poll`/`epoll` like any other.

Three consequences follow, and most surprises come from them:

1. **A watch follows the inode, not the name.** If someone renames a new file over the watched one, your watch stays attached to the old inode. That inode is unlinked, you get `IN_DELETE_SELF` and `IN_IGNORED`, and the watch is gone.
2. **A directory watch sees its direct children only.** A change in `dir/sub/file` produces nothing on the watch for `dir`. Recursive watching means one watch per directory, added by you, with a race each time a directory appears.
3. **The queue is bounded.** After `max_queued_events` (16384 by default) unread events, further events are dropped and a single `IN_Q_OVERFLOW` event with `wd = -1` is queued. You learn that you lost something, but not what.

fanotify inverts the design:

- `fanotify_init()` creates a **group**, and `fanotify_mark()` attaches **marks** to an inode, a *mount* or a whole *filesystem*.
- Classic fanotify hands you an open **file descriptor** of the object in every event. That is why it needs `CAP_SYS_ADMIN`: a listener could otherwise gain access to files it could not open.
- In modern "FID" mode (`FAN_REPORT_FID`, `FAN_REPORT_DFID_NAME`, Linux 5.1 and 5.9) events carry a **file handle** plus the entry name instead.
- With FID reporting, an **unprivileged** user may use inode marks for notification-only events (Linux 5.13).
- A mount or filesystem mark removes the recursion problem entirely: everything below it is covered. Only privileged listeners get them.

Syntax

```
#include <sys/inotify.h>
int fd = inotify_init1(IN_NONBLOCK | IN_CLOEXEC);
int wd = inotify_add_watch(fd, "/srv/app/conf.d", IN_CLOSE_WRITE | IN_MOVED_TO | IN_DELETE);
inotify_rm_watch(fd, wd);
/* read(fd, buf, len) returns one or more struct inotify_event */

#include <sys/fanotify.h>
int g = fanotify_init(FAN_CLASS_NOTIF | FAN_REPORT_DFID_NAME, O_RDONLY);
fanotify_mark(g, FAN_MARK_ADD, FAN_CREATE | FAN_DELETE | FAN_MOVE | FAN_ONDIR, AT_FDCWD, "/s
/* privileged only: FAN_MARK_ADD | FAN_MARK_FILESYSTEM, FAN_OPEN_PERM, FAN_CLASS_CONTENT ...
```

Knob	Where	Default here	Meaning
`max_user_watches`	`/proc/sys/fs/inotify/`	127859	inotify watches per **real user ID**, across all their processes. Since 5.11 the default is 1 % of RAM divided by the cost of one watch, clamped to 8192…1048576.
`max_user_instances`	`/proc/sys/fs/inotify/`	128	inotify instances (fds) per real user ID
`max_queued_events`	`/proc/sys/fs/inotify/`	16384	queue length per instance before `IN_Q_OVERFLOW`
`max_user_marks`	`/proc/sys/fs/fanotify/`	136162	fanotify marks per user (scaled to RAM, like inotify)
`max_user_groups`	`/proc/sys/fs/fanotify/`	128	fanotify groups per user

Knob	Where	Default here	Meaning
`max_queued_events`	`/proc/sys/fs/fanotify/`	16384	per-group queue; `FAN_UNLIMITED_QUEUE` lifts it (privileged)

```
/proc/sys/fs/inotify/max_queued_events:16384
/proc/sys/fs/inotify/max_user_instances:128
/proc/sys/fs/inotify/max_user_watches:127859
/proc/sys/fs/fanotify/max_queued_events:16384
/proc/sys/fs/fanotify/max_user_groups:128
/proc/sys/fs/fanotify/max_user_marks:136162
MemTotal:       16386368 kB
```

(16 GiB of RAM: 1 % of 16386368 kB / 127859 1.3 KiB “cost” per watch. The constant `INOTIFY_WATCH_COST` in `fs/notify/inotify/inotify_user.c` counts the mark plus two inodes it may pin.)

inotify mask	Fires when
`IN_CREATE` / `IN_DELETE`	an entry appeared / vanished in a watched directory
`IN_MODIFY`	data was written (once per `write()`, merged if unread)
`IN_CLOSE_WRITE`	a file opened for writing was closed. **The “file is complete” signal.**
`IN_MOVED_FROM` / `IN_MOVED_TO`	rename out of / into a watched directory; the pair shares a `cookie`
`IN_ATTRIB`	metadata: mode, owner, timestamps, **link count**
`IN_DELETE_SELF` / `IN_MOVE_SELF`	the watched object itself was deleted / moved
`IN_IGNORED`	the watch was removed (explicitly, or because its inode died or the fs was unmounted)
`IN_Q_OVERFLOW`	events were dropped; `wd` is `-1`
`IN_ISDIR`	flag: the subject is a directory

Safety

- Watching is read-only. None of the examples change anything outside a fresh `mktemp -d` directory under `/tmp`.
- Raising the sysctls (`sysctl fs.inotify.max_user_watches=…`) needs root and affects every user on the host. It was **not run** here. Persist such changes in `/etc/sysctl.d/*.conf` only after you have found the process using them up (Example 4).

- fanotify **permission events** (`FAN_OPEN_PERM`, `FAN_ACCESS_PERM`, pre-content events) make other processes **wait** for your answer. A buggy listener that stops reading can hang every `open()` on the filesystem, including the shell you would use to kill it. Develop those on a throwaway VM, never on the host you are logged into.
- Do not deliberately exhaust `max_user_watches` or `max_user_instances` on a shared machine to "see the error". Every other process of your user ID (editors, desktop daemons, language servers) loses the ability to watch at the same moment. That is why this chapter describes `ENOSPC`/`EMFILE` from the documentation instead of triggering them.

Examples with Explanations

1. One directory watch, everyday operations

A tiny wrapper, used by the next examples:

```
# inotify.py - a tiny ctypes wrapper around inotify(7); no third-party modules
import ctypes, ctypes.util, os, struct

libc = ctypes.CDLL(ctypes.util.find_library("c"), use_errno=True)
libc.inotify_add_watch.argtypes = [ctypes.c_int, ctypes.c_char_p, ctypes.c_uint32]

MASKS = {  # <sys/inotify.h>
    "ACCESS": 0x1, "MODIFY": 0x2, "ATTRIB": 0x4, "CLOSE_WRITE": 0x8, "CLOSE_NOWRITE": 0x10,
    "OPEN": 0x20, "MOVED_FROM": 0x40, "MOVED_TO": 0x80, "CREATE": 0x100, "DELETE": 0x200,
    "DELETE_SELF": 0x400, "MOVE_SELF": 0x800, "UNMOUNT": 0x2000, "Q_OVERFLOW": 0x4000,
    "IGNORED": 0x8000, "ISDIR": 0x40000000,
}
IN_ALL_EVENTS = 0xFFF
IN_NONBLOCK, IN_CLOEXEC = os.O_NONBLOCK, os.O_CLOEXEC
EVENT = struct.Struct("iIII")            # wd, mask, cookie, len  (+ name[len])

def init(flags=IN_CLOEXEC):
    fd = libc.inotify_init1(flags)
    if fd < 0:
        raise OSError(ctypes.get_errno(), os.strerror(ctypes.get_errno()))
    return fd

def add_watch(fd, path, mask=IN_ALL_EVENTS):
    wd = libc.inotify_add_watch(fd, os.fsencode(path), mask)
    if wd < 0:
        e = ctypes.get_errno()
        raise OSError(e, os.strerror(e), path)
    return wd

def names(mask):
    return "|".join(n for n, b in MASKS.items() if mask & b)
```

```
def read_events(fd, bufsize=65536):
    """Yield (wd, mask, cookie, name) for everything currently queued."""
    try:
        buf = os.read(fd, bufsize)
    except BlockingIOError:
        return
    off = 0
    while off < len(buf):
        wd, mask, cookie, n = EVENT.unpack_from(buf, off)
        name = buf[off + EVENT.size: off + EVENT.size + n].rstrip(b"\0").decode()
        off += EVENT.size + n
        yield wd, mask, cookie, name
```

```
# watch_dir.py - what one directory watch reports for everyday file operations
import os, shutil, tempfile, inotify

d = tempfile.mkdtemp(prefix="ino-")
fd = inotify.init(inotify.IN_NONBLOCK | inotify.IN_CLOEXEC)
wd = inotify.add_watch(fd, d)

def step(label, action):
    action()
    print(f"-- {label}")
    for wd_, mask, cookie, name in inotify.read_events(fd):
        extra = f"  cookie={cookie}" if cookie else ""
        print(f"   wd={wd_} {inotify.names(mask):<22} {name or '(the watched dir)'}{extra}")

p = os.path.join(d, "notes.txt")
step("echo hi > notes.txt", lambda: open(p, "w").write("hi\n"))
step("cat notes.txt", lambda: open(p).read())
step("chmod 600 notes.txt", lambda: os.chmod(p, 0o600))
step("mv notes.txt todo.txt", lambda: os.rename(p, os.path.join(d, "todo.txt")))
step("mkdir sub && touch sub/deep.txt", lambda: (os.mkdir(os.path.join(d, "sub")),
                                                 open(os.path.join(d, "sub", "deep.txt"),
step("rm -r the directory itself", lambda: shutil.rmtree(d))
```

```
$ python3 watch_dir.py
-- echo hi > notes.txt
   wd=1 CREATE                 notes.txt
   wd=1 OPEN                   notes.txt
   wd=1 MODIFY                 notes.txt
   wd=1 CLOSE_WRITE            notes.txt
-- cat notes.txt
   wd=1 OPEN                   notes.txt
   wd=1 ACCESS                 notes.txt
```

```
   wd=1 CLOSE_NOWRITE          notes.txt
-- chmod 600 notes.txt
   wd=1 ATTRIB                 notes.txt
-- mv notes.txt todo.txt
   wd=1 MOVED_FROM             notes.txt  cookie=808168
   wd=1 MOVED_TO               todo.txt  cookie=808168
-- mkdir sub && touch sub/deep.txt
   wd=1 CREATE|ISDIR           sub
-- rm -r the directory itself
   wd=1 OPEN|ISDIR             (the watched dir)
   wd=1 ACCESS|ISDIR           (the watched dir)
   wd=1 DELETE                 todo.txt
   wd=1 OPEN|ISDIR             sub
   wd=1 ACCESS|ISDIR           sub
   wd=1 CLOSE_NOWRITE|ISDIR    sub
   wd=1 DELETE|ISDIR           sub
   wd=1 CLOSE_NOWRITE|ISDIR    (the watched dir)
   wd=1 DELETE_SELF            (the watched dir)
   wd=1 IGNORED                (the watched dir)
```

What to read from it:

- **Writing a file** gives `CREATE`, `OPEN`, `MODIFY`, `CLOSE_WRITE`. If you act on `MODIFY` you may read a half-written file. Act on `CLOSE_WRITE` (or on `MOVED_TO`, see Example 2).
- **Reading** produces events too (`OPEN`, `ACCESS`, `CLOSE_NOWRITE`). A watcher that subscribes to `IN_ALL_EVENTS` and reacts to everything will wake itself up by reading the files it was told about. Ask for the events you need.
- **Rename** is two events with the same `cookie`. If only one side is inside your watched set, you get only one half: a `MOVED_FROM` without a partner means "gone", a lone `MOVED_TO` means "appeared".
- **`mkdir sub && touch sub/deep.txt`**: only `CREATE|ISDIR sub`. `deep.txt` is invisible, because the watch on the parent does not cover grandchildren.
- **Removing the watched directory** ends with `DELETE_SELF` and `IGNORED`. The kernel dropped the watch, and the `wd` number may later be reused.

2. The editor trap: a watch on the file dies after the first save

Many editors and config tools (and `sed -i`, `install`, most deploy tools) save **atomically**: they write a temporary file and `rename()` it over the target. Readers then never see a half-written file, but the target is a *new inode*:

```
# atomic_save.py - watch a FILE, then save it the way editors do (write temp, rename over)
import os, tempfile, inotify

d = tempfile.mkdtemp(prefix="ino-")
cfg = os.path.join(d, "app.conf")
```

```
open(cfg, "w").write("v=1\n")

fd = inotify.init(inotify.IN_NONBLOCK)
wd_file = inotify.add_watch(fd, cfg)            # watch the file itself
wd_dir = inotify.add_watch(fd, d, inotify.MASKS["CLOSE_WRITE"] | inotify.MASKS["MOVED_TO"])
who = {wd_file: "file-watch", wd_dir: "dir-watch "}

def save(text):                                 # atomic replace: new inode, same name
    tmp = cfg + ".tmp"
    open(tmp, "w").write(text)
    os.rename(tmp, cfg)

for n in (2, 3):
    save(f"v={n}\n")
    print(f"-- save #{n - 1} (v={n})")
    for wd, mask, cookie, name in inotify.read_events(fd):
        print(f"   {who.get(wd, wd)} {inotify.names(mask):<12} {name}")
```

```
$ python3 atomic_save.py
-- save #1 (v=2)
   dir-watch  CLOSE_WRITE  app.conf.tmp
   dir-watch  MOVED_TO     app.conf
   file-watch ATTRIB
   file-watch DELETE_SELF
   file-watch IGNORED
-- save #2 (v=3)
   dir-watch  CLOSE_WRITE  app.conf.tmp
   dir-watch  MOVED_TO     app.conf
```

The file watch sees the old inode lose its name (`ATTRIB`: the link count dropped), get freed (`DELETE_SELF`) and lose the watch (`IGNORED`). The second save is **silent** for it, and a program that watches the file stops reloading after the first edit. The directory watch keeps working: `MOVED_TO app.conf` is the one reliable "the new version is in place" signal.

GNU `tail` shows both strategies. `tail -f` follows the *descriptor* (one watch on the file, `IN_MODIFY` only). `tail -F` follows the *name*: it also watches the directory for creates, deletes and renames, so it can reopen a rotated or replaced log:

```
# tailf.sh - GNU tail -f / -F use inotify; see their watches in /proc
show() {   # print every inotify watch held by PID $1
  for fd in /proc/$1/fd/*; do
    [ "$(readlink "$fd")" = anon_inode:inotify ] && grep '^inotify' /proc/$1/fdinfo/${fd##*/
  done
}
f=$(mktemp)
printf 'file %s: ino %x   dir /tmp: ino %x\n' "$f" "$(stat -c %i "$f")" "$(stat -c %i /tmp)"
tail -f "$f" > /dev/null &  p1=$!
```

```
tail -F "$f" > /dev/null &  p2=$!
sleep 0.3
echo "tail -f:"; show $p1
echo "tail -F:"; show $p2
kill $p1 $p2; rm -f "$f"
```

```
$ bash tailf.sh
file /tmp/tmp.NNcmyC9GoV: ino 17c178   dir /tmp: ino bd
tail -f:
inotify wd:1 ino:17c178 sdev:fe00010 mask:2
tail -F:
inotify wd:2 ino:17c178 sdev:fe00010 mask:c06
inotify wd:1 ino:bd sdev:fe00010 mask:784
```

The `fdinfo` lines name the watched inode in hex. Mask `2` is `IN_MODIFY`. Mask `c06` adds `IN_ATTRIB|IN_DELETE_SELF|IN_MOVE_SELF` on the file, and `784` on `/tmp` (inode `bd`) is `IN_ATTRIB|IN_MOVED_TO|IN_CREATE|IN_DELETE|IN_DELETE_SELF`. Use `tail -F` for logs that get rotated.

3. Overflow: what you get when you read too slowly

```
# overflow.py - what happens when nobody reads the queue: IN_Q_OVERFLOW
import os, tempfile, inotify

limit = int(open("/proc/sys/fs/inotify/max_queued_events").read())
d = tempfile.mkdtemp(prefix="ino-")
fd = inotify.init(inotify.IN_NONBLOCK)
inotify.add_watch(fd, d, inotify.MASKS["CREATE"] | inotify.MASKS["CLOSE_WRITE"])

files = 10_000                          # 2 events each: CREATE + CLOSE_WRITE = 20,000 events
for i in range(files):
    open(os.path.join(d, f"f{i:05}"), "w").close()

got, overflow, last = 0, 0, None
while True:
    batch = list(inotify.read_events(fd))
    if not batch:
        break
    for wd, mask, cookie, name in batch:
        if mask & inotify.MASKS["Q_OVERFLOW"]:
            overflow += 1
            print(f"Q_OVERFLOW: wd={wd} name={name!r} after {got} events (last name {last})"
        else:
            got, last = got + 1, name
print(f"max_queued_events={limit}  generated={2 * files}  delivered={got}  lost={2 * files -
```

```
$ python3 overflow.py
Q_OVERFLOW: wd=-1 name='' after 16384 events (last name f08191)
max_queued_events=16384  generated=20000  delivered=16384  lost=3616
```

10,000 new files produced 20,000 events. The queue held exactly `max_queued_events` (16384 events, files `f00000` to `f08191`). The remaining 3616 events were dropped, and a single `IN_Q_OVERFLOW` with `wd=-1` marks the gap. One `git checkout`, `npm install` or `tar x` easily produces such a burst. A correct watcher therefore treats `IN_Q_OVERFLOW` as "my picture is stale": it rescans the tree, or closes the instance and rebuilds its watches. Reading faster (a dedicated reader thread, large `read()` buffers) makes it rarer but never impossible.

The inotify man page also notes that **identical consecutive unread events are coalesced** (same `wd`, mask, cookie and name). Ten quick `write()`s to one file can arrive as one `IN_MODIFY`, so you cannot count writes with inotify.

4. Who is using up the watches?

When a tool fails to add a watch, `inotify_add_watch()` returns `ENOSPC` ("No space left on device", a misleading message: the user's `max_user_watches` is exhausted, not the disk). When it cannot create an instance, `inotify_init1()` returns `EMFILE` ("Too many open files": `max_user_instances` reached). Both limits are **per real user ID**, so the culprit is often a *different* program of the same user, typically an IDE or language server that watches a whole `node_modules`. Every inotify fd lists its watches in `/proc/PID/fdinfo/FD`, one `inotify wd:` line per watch:

```
# who_watches.py - count inotify instances and watches per process (only processes you may i
import os, re

rows = []
for pid in filter(str.isdigit, os.listdir("/proc")):
    try:
        comm = open(f"/proc/{pid}/comm").read().strip()
        inst = watches = 0
        for fd in os.listdir(f"/proc/{pid}/fd"):
            if os.readlink(f"/proc/{pid}/fd/{fd}") == "anon_inode:inotify":
                inst += 1
                watches += sum(1 for l in open(f"/proc/{pid}/fdinfo/{fd}") if l.startswith("
        if inst:
            rows.append((watches, inst, int(pid), comm))
    except (PermissionError, FileNotFoundError, ProcessLookupError):
        continue
lim = {k: int(open(f"/proc/sys/fs/inotify/{k}").read()) for k in ("max_user_watches", "max_u
print(f"limits: max_user_watches={lim['max_user_watches']} max_user_instances={lim['max_user
print(f"{'PID':>7} {'COMM':<16} {'INSTANCES':>9} {'WATCHES':>8}")
for w, i, pid, comm in sorted(rows, reverse=True)[:8]:
    print(f"{pid:>7} {comm:<16} {i:>9} {w:>8}")
print(f"total: {sum(r[1] for r in rows)} instances, {sum(r[0] for r in rows)} watches")
```

```
$ python3 who_watches.py
limits: max_user_watches=127859 max_user_instances=128
    PID COMM             INSTANCES  WATCHES
 184516 node                     1       75
 189580 bamfdaemon               1        5
  18492 bamfdaemon               1        5
 189262 plank                    1        4
  25625 bamfdaemon               1        4
  25400 plank                    1        4
  18317 plank                    1        4
  14946 bamfdaemon               1        4
total: 17 instances, 128 watches
```

Run as yourself, it sees your own processes (`/proc/PID/fd` of other users needs root). On a busy developer machine the top entry is usually the editor or a file-watching dev server with tens of thousands of watches. Exclude build output and dependency directories in its settings before you raise the limit.

5. fanotify without root: what the 5.13 unprivileged mode allows

```
# fan.py - unprivileged fanotify (Linux 5.13+): directory-entry events with FID + name recor
import ctypes, ctypes.util, os, struct, tempfile

libc = ctypes.CDLL(ctypes.util.find_library("c"), use_errno=True)
libc.fanotify_mark.argtypes = [ctypes.c_int, ctypes.c_uint, ctypes.c_uint64, ctypes.c_int, c

# <linux/fanotify.h>
FAN_CLASS_NOTIF, FAN_REPORT_FID, FAN_REPORT_DIR_FID, FAN_REPORT_NAME = 0x0, 0x200, 0x400, 0x
FAN_REPORT_DFID_NAME = FAN_REPORT_DIR_FID | FAN_REPORT_NAME
FAN_MARK_ADD, FAN_MARK_MOUNT, FAN_MARK_FILESYSTEM = 0x1, 0x10, 0x100
FAN_MOVED_FROM, FAN_MOVED_TO, FAN_CREATE, FAN_DELETE = 0x40, 0x80, 0x100, 0x200
FAN_ONDIR = 0x40000000
AT_FDCWD = -100
NAMES = {FAN_CREATE: "CREATE", FAN_MOVED_FROM: "MOVED_FROM", FAN_MOVED_TO: "MOVED_TO",
         FAN_DELETE: "DELETE", FAN_ONDIR: "ONDIR"}

def call(fn, *a):
    r = fn(*a)
    if r < 0:
        e = ctypes.get_errno()
        raise OSError(e, os.strerror(e))
    return r

FAN_CLASS_CONTENT, FAN_UNLIMITED_QUEUE = 0x4, 0x10
print("uid", os.getuid(), "CapEff", [l.split()[1] for l in open("/proc/self/status") if l.st
for label, flags in [("FAN_CLASS_NOTIF (no FID)", FAN_CLASS_NOTIF),
```

```
                      ("FAN_CLASS_CONTENT|FID", FAN_CLASS_CONTENT | FAN_REPORT_FID),
                      ("FAN_UNLIMITED_QUEUE|FID", FAN_UNLIMITED_QUEUE | FAN_REPORT_FID)]:
    try:
        os.close(call(libc.fanotify_init, flags, os.O_RDONLY))
        print(f"fanotify_init({label:<26}) -> ok")
    except OSError as e:
        print(f"fanotify_init({label:<26}) -> {e.strerror}")
fd = call(libc.fanotify_init, FAN_CLASS_NOTIF | FAN_REPORT_DFID_NAME, os.O_RDONLY)
print(f"fanotify_init({'FAN_REPORT_DFID_NAME':<26}) -> fd {fd}")

d = tempfile.mkdtemp(prefix="fan-")
mask = FAN_CREATE | FAN_DELETE | FAN_MOVED_FROM | FAN_MOVED_TO | FAN_ONDIR
call(libc.fanotify_mark, fd, FAN_MARK_ADD, mask, AT_FDCWD, d.encode())
print(f"fanotify_mark({'inode mark (directory)':<26}) -> ok")

for label, flags, m in [("FAN_MARK_FILESYSTEM", FAN_MARK_ADD | FAN_MARK_FILESYSTEM, FAN_CREA
                        ("FAN_MARK_MOUNT", FAN_MARK_ADD | FAN_MARK_MOUNT, FAN_CREATE)]:
    try:
        call(libc.fanotify_mark, fd, flags, m, AT_FDCWD, d.encode())
        print(f"fanotify_mark({label:<26}) -> ok")
    except OSError as e:
        print(f"fanotify_mark({label:<26}) -> {e.strerror}")

os.mkdir(f"{d}/sub")
open(f"{d}/a.txt", "w").close()
os.rename(f"{d}/a.txt", f"{d}/b.txt")
os.unlink(f"{d}/b.txt")
os.spawnlp(os.P_WAIT, "touch", "touch", f"{d}/by-child")     # an event from another process

META = struct.Struct("IBBHQii")        # event_len, vers, reserved, metadata_len, mask, fd,
buf, off = os.read(fd, 65536), 0
while off < len(buf):
    ev_len, vers, _, meta_len, emask, efd, pid = META.unpack_from(buf, off)
    info, name = off + meta_len, None
    while info < off + ev_len:                                    # info records follow the m
        itype, _, ilen = struct.unpack_from("BBH", buf, info)
        if itype == 2:                                            # FAN_EVENT_INFO_TYPE_DFID_
            hbytes, = struct.unpack_from("I", buf, info + 4 + 8)  # skip header + fsid
            name = buf[info + 4 + 8 + 8 + hbytes: info + ilen].split(b"\0")[0].decode()
        info += ilen
    who = "self" if pid == os.getpid() else pid
    print(f"  {'|'.join(v for k, v in NAMES.items() if emask & k):<18} name={name:<9} fd={ef
    off += ev_len
os.unlink(f"{d}/by-child"); os.rmdir(f"{d}/sub"); os.rmdir(d)
```

```
$ python3 fan.py
uid 1000 CapEff 0000000000000000
fanotify_init(FAN_CLASS_NOTIF (no FID)  ) -> Operation not permitted
fanotify_init(FAN_CLASS_CONTENT|FID     ) -> Operation not permitted
fanotify_init(FAN_UNLIMITED_QUEUE|FID   ) -> Operation not permitted
fanotify_init(FAN_REPORT_DFID_NAME      ) -> fd 5
fanotify_mark(inode mark (directory)    ) -> ok
fanotify_mark(FAN_MARK_FILESYSTEM       ) -> Operation not permitted
fanotify_mark(FAN_MARK_MOUNT            ) -> Operation not permitted
  CREATE|ONDIR       name=sub       fd=-1 pid=self
  CREATE|MOVED_FROM  name=a.txt     fd=-1 pid=self
  MOVED_TO|DELETE    name=b.txt     fd=-1 pid=self
  CREATE             name=by-child  fd=-1 pid=0
```

Line by line:

- **Classic mode (event carries an fd) is refused** to an unprivileged user (`EPERM`). So are content/permission classes and the unlimited queue.
- **FID mode with names works** (`FAN_REPORT_DFID_NAME`): each event identifies the *parent directory* by file handle and carries the entry name. `fd=-1` because no fd is opened for you.
- **Only inode marks.** `FAN_MARK_FILESYSTEM` and `FAN_MARK_MOUNT` fail with `EPERM`. Without root, fanotify is not recursive either.
- **Events merge.** `CREATE|MOVED_FROM a.txt` and `MOVED_TO|DELETE b.txt` are each *one* event: unread events for the same directory entry from the same process collapse into one record with several mask bits. The final state is clear (a.txt came and left, b.txt came and left), but the order is not. inotify kept the order (Example 1).
- **No pid for other processes.** The `touch` child shows `pid=0`. An unprivileged listener only learns the pid when it caused the event itself.

The man page `fanotify_init(2)` lists exactly these restrictions for unprivileged callers. fanotify renames are better than inotify's cookie pairs since **Linux 5.17** (backported to 5.15.154 and 5.10.220): `FAN_RENAME` delivers *one* event with both the old and the new directory+name records, even across two marked directories:

```
# fan_rename.py - FAN_RENAME (Linux 5.17+): one event carrying both the old and the new name
import ctypes, ctypes.util, os, struct, tempfile

libc = ctypes.CDLL(ctypes.util.find_library("c"), use_errno=True)
libc.fanotify_mark.argtypes = [ctypes.c_int, ctypes.c_uint, ctypes.c_uint64, ctypes.c_int, c
FAN_REPORT_DFID_NAME, FAN_MARK_ADD, FAN_RENAME, AT_FDCWD = 0xC00, 0x1, 0x10000000, -100
INFO = {2: "DFID_NAME", 10: "OLD_DFID_NAME", 12: "NEW_DFID_NAME"}   # FAN_EVENT_INFO_TYPE_*

fd = libc.fanotify_init(FAN_REPORT_DFID_NAME, os.O_RDONLY)
d = tempfile.mkdtemp(prefix="fan-")
os.mkdir(f"{d}/inbox"); os.mkdir(f"{d}/done")
for sub in ("inbox", "done"):
    assert libc.fanotify_mark(fd, FAN_MARK_ADD, FAN_RENAME, AT_FDCWD, f"{d}/{sub}".encode())
```

```
open(f"{d}/inbox/job-42", "w").close()
os.rename(f"{d}/inbox/job-42", f"{d}/done/job-42.ok")

buf = os.read(fd, 4096)
ev_len, _, _, meta_len, mask, _, _ = struct.unpack_from("IBBHQii", buf, 0)
print(f"one event, mask=0x{mask:x} (FAN_RENAME), {ev_len} bytes")
off = meta_len
while off < ev_len:
    itype, _, ilen = struct.unpack_from("BBH", buf, off)
    hbytes, = struct.unpack_from("I", buf, off + 12)
    name = buf[off + 20 + hbytes: off + ilen].split(b"\0")[0].decode()
    print(f"  {INFO.get(itype, itype):<14} name={name}")
    off += ilen
```

```
$ python3 fan_rename.py
one event, mask=0x10000000 (FAN_RENAME), 100 bytes
  OLD_DFID_NAME  name=job-42
  NEW_DFID_NAME  name=job-42.ok
```

6. What privileged fanotify adds (from the docs, not run)

With `CAP_SYS_ADMIN` (not available here, so none of this was run):

- `FAN_MARK_FILESYSTEM` covers every object on a filesystem, and `FAN_MARK_MOUNT` covers a mount. That is the scalable answer to "watch this huge tree": one mark instead of a watch per directory, no add-then-scan race. Mount marks cannot report directory-entry events (`FAN_CREATE` and friends); use a filesystem mark for those.
- **Permission events** (`FAN_OPEN_PERM`, `FAN_OPEN_EXEC_PERM`, `FAN_ACCESS_PERM`, class `FAN_CLASS_CONTENT`) make the opener wait until the listener writes `FAN_ALLOW` or `FAN_DENY`. This is how on-access virus scanners and some DLP tools work.
- **Pre-content events** (`FAN_PRE_ACCESS`, class `FAN_CLASS_PRE_CONTENT`, Linux 6.14) fire before a range of a file is read and come with the offset and length. A hierarchical-storage manager can fill in the data on demand.
- **Mount-namespace marks** (`FAN_MARK_MNTNS` with `FAN_REPORT_MNT`, Linux 6.14) report `FAN_MNT_ATTACH`/`FAN_MNT_DETACH` when mounts appear or disappear in a namespace.
- `FAN_REPORT_PIDFD` (5.15) returns a pidfd for the event's process instead of a pid that could be recycled, and `FAN_UNLIMITED_QUEUE` removes the 16384-event bound.

The trap

Trap: "I watch the config file, so I will see every change." You see changes until the first atomic save replaces the inode (Example 2). Then `IN_IGNORED`, and silence. Watch the **directory**, filter by name, and react to `CLOSE_WRITE` and `MOVED_TO`.

Trap: "inotify is recursive if I pass a directory." It is not (Example 1). Recursive watchers add one watch per directory. When `IN_CREATE|IN_ISDIR` arrives, files may already exist inside the new directory before your watch is in place. The man page's advice: add the watch **first**, then list the directory and treat what you find as created.

Trap: "No space left on device — the disk is full." From `inotify_add_watch()` it means `max_user_watches` is exhausted for your user ID. `df` is irrelevant. Count watches per process (Example 4) before raising the limit.

Trap: "Events tell me exactly what happened." They can be dropped (`IN_Q_OVERFLOW`, Example 3), coalesced (identical inotify events) or merged (fanotify, Example 5). Use events as **hints to look**, then `stat`/scan to learn the real state.

Notes & Pitfalls

- **Not every filesystem reports.** Changes made by *other* machines on NFS, SMB, FUSE or 9p (including many VM and container bind/shared folders) do not generate local events, because the kernel never sees them. Pseudo-filesystems such as `/proc`, `/sys` and `/dev/pts` cannot be monitored with inotify either (inotify(7)). Watchers on these need polling; most watcher libraries have a polling fallback setting.
- **Bind mounts and overlays:** inotify watches inodes, so a change through any path to the same inode is reported. On overlayfs, a watch on a merged file sees changes made through the overlay, not direct edits to a lower layer.
- **`IN_MODIFY` on `mmap` writes is not reported.** Writes through a shared memory mapping (`mmap`, `msync`, `munmap`) do not produce events (inotify(7), "Limitations and caveats").
- **Watch descriptors are reused** after `IN_IGNORED`. Drop the `wd → path` entry from your map when you see it, or late events will be attributed to the wrong path.
- **Renames across watched directories** need the cookie to pair `MOVED_FROM`/`MOVED_TO`. The two events are usually adjacent, but not guaranteed to arrive in the same `read()`.
- **systemd `.path` units** (`PathChanged=`, `PathModified=`, `DirectoryNotEmpty=`) are inotify watches held by PID 1 and the user manager, with the same non-recursive and atomic-save behaviour. `PathChanged=` fires on `CLOSE_WRITE` and moves, `PathModified=` also on every write.
- Containers share the host kernel. Processes that run as the same host UID draw on the same `max_user_watches` budget, even when they are in different containers.

The boring rule

Watch **directories**, not files. Act on `IN_CLOSE_WRITE` and `IN_MOVED_TO`, never on `IN_MODIFY`. Treat `IN_Q_OVERFLOW` as "rescan". When you see a new subdirectory, add the watch before you list it. When a watcher says "no space left on device", count watches per process before you touch the sysctl. Reach for fanotify only when you need whole-filesystem coverage or a say in `open()`, and test permission events on a machine you can afford to hang.

Related Commands

- `tail -F` follows a log by name across rotation (inotify on the directory); `tail -f` follows the descriptor.
- `inotifywait` / `inotifywatch` (inotify-tools) are inotify on the command line: `inotifywait -m -e close_write,moved_to dir/`. Not installed here; see their manual.
- `fatrace` and `fsnotifywait` (inotify-tools 4.x, fanotify-based) give filesystem-wide views and need root.
- `systemctl list-units --type=path` lists inotify-driven systemd path units.
- `find /proc/*/fd -lname anon_inode:inotify 2>/dev/null` finds every inotify fd you may inspect.
- `stat -c %i FILE` gives the inode number to match against `fdinfo ino:` (hex).

Try this

1. In `watch_dir.py`, add a watch for `sub` as soon as `CREATE|ISDIR sub` arrives, then `touch sub/deep.txt`. Now create `sub2/a/b/c` with one `os.makedirs()` call and see how many levels you catch without the "scan after adding" step.
2. Change `atomic_save.py` to write the file in place (`open(cfg, "w")`). Which watch now reports the save, and with which events?
3. Lower the burst in `overflow.py` until no overflow happens, then read the queue from a thread while generating events. How much further can you push it?
4. In `fan.py`, add `FAN_REPORT_TARGET_FID` (Linux 5.17) to `fanotify_init()` and print how many info records each event carries.
5. Run `who_watches.py` while your editor opens a large project, and again after excluding its dependency directory from file watching.

Verification

- Run unprivileged on this box: Debian 13 (trixie), Linux **6.12.94+** (x86_64), Python **3.13.5** (standard library `ctypes` only), GNU coreutils **9.7**. `uid=1000`, `CapEff 0000000000000000`. All output above is pasted from one run of the listed scripts. Cookies, inode numbers, PIDs and watch counts change between runs.
- Kernel limits read from `/proc/sys/fs/inotify` and `/proc/sys/fs/fanotify`. The default formula for `max_user_watches` was checked in `fs/notify/inotify/inotify_user.c` (v6.12), and the fanotify defaults in `fs/notify/fanotify/fanotify_user.c`.
- Checked against kernel.org on 2026-10-19: stable **7.2.11**, mainline **7.3-rc7**, longterm **6.18.57** and **6.12.113**. The APIs used here are unchanged on those kernels. The main fanotify additions since 6.12 are pre-content events (`FAN_PRE_ACCESS`) and mount-namespace marks, both in 6.14.
- **Not run:** raising the sysctls (root), exhausting `max_user_watches`/`max_user_instances` to trigger `ENOSPC`/`EMFILE` (would disturb other processes on a shared host), privileged fanotify (filesystem/mount marks, permission and pre-content events, `FAN_REPORT_PIDFD`), `inotifywait`, `fatrace` and `fsnotifywait` (not installed; nothing was installed for this chapter). Those parts are written from the man pages and kernel sources below.

- No packages installed, no root, no `sudo`.

Sources

- man7: `inotify(7)` (events, limits, coalescing, the recursive-watch race, overflow advice, BUGS), `inotify_add_watch(2)` (`ENOSPC`), `inotify_init(2)` (`EMFILE`), `fanotify(7)` (event merging, info records), `fanotify_init(2)` (unprivileged restrictions, `FAN_REPORT_*`), `fanotify_mark(2)` (`FAN_RENAME`, `FAN_PRE_ACCESS`, mark types), `proc_pid_fdinfo(5)`
- Kernel docs: inotify — a powerful yet simple file change notification system; Documentation for /proc/sys/fs/
- Linux sources: `fs/notify/inotify/inotify_user.c` (`INOTIFY_WATCH_COST`, default limits), `fs/notify/fanotify/fanotify_user.c` (unprivileged checks, defaults)
- Kernel merge "fsnotify_hsm_for_v6.14-rc1" (pre-content events): https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=8883957b3c9de2087fb6cf9691c1188cccf1ac9c
- GNU coreutils manual, `tail` invocation (`--follow=name` vs `descriptor`, `-F`)
- systemd: `systemd.path(5)`
- kernel.org release list (versions checked 2026-10-19): https://www.kernel.org/

Archives and Compression

Intro

Intro

Pack, compress, and inspect archives for backup, distribution, and log history. Prefer `tar` with modern compressors (`zstd`, `xz`) for Linux trees; use zip when exchanging with non-Unix tools.

Commands in this part

Command	Role
`tar`	tar packs multiple files into one archive (and extracts them).
`gzip`	gzip compresses or decompresses files using the DEFLATE algorithm (.gz).
`xz`	xz compresses and decompresses files in the XZ format (LZMA2), typically yielding better ratios than gzip/bzip2 at…
`bzip2`	bzip2 compresses files with the Burrows–Wheeler algorithm (.bz2).
`zstd`	zstd (Zstandard) is a modern compression tool offering an excellent speed vs ratio curve, levels from very fast to…
`zip`	zip creates ZIP archives widely used for cross-platform interchange (Windows, macOS, Linux).
`unzip`	unzip lists, tests, and extracts ZIP archives.
`zcat`	zcat writes gzip-compressed files to stdout as if they were uncompressed (equivalent to gzip -dc).
`zgrep`	zgrep runs grep over gzip-compressed files without manual decompression.

Suggested starting points

1. Create/extract trees: `tar` (+ `zstd`/`gzip`/`xz`).
2. Single-file compressors: `gzip`, `xz`, `bzip2`, `zstd`.
3. Interchange with Windows/macOS users: `zip` / `unzip`.

4. Search rotated logs without unpacking: `zgrep`, `zcat`.

Related parts

- Files and paths — copy and verify before archiving
- Text and pipes — process extracted streams
- Networking — `rsync`/`scp` for transfer after packing

Continue with the individual command pages in this part.

tar

Overview

`tar` packs multiple files into one archive (and extracts them). Compression is added via flags or pipes (`gzip`, `xz`, `zstd`). GNU tar is the Linux default.

Syntax

```
tar [OPTIONS] -f ARCHIVE [FILES...]
```

Mnemonic ops: **c**reate, e**x**tract, **t**list, plus compression letters.

Common Options

Option	Description
`-c`	Create
`-x`	Extract
`-t`	List
`-f file`	Archive path (required in practice)
`-v`	Verbose
`-z`	gzip
`-J`	xz
`-I zstd` / `--zstd`	zstd (GNU tar 1.31+)
`-C dir`	Change directory
`-p`	Preserve permissions (extract)
`--exclude=PAT`	Exclude glob
`--strip-components=n`	Drop n path prefix levels on extract
`-a`	Auto-compress from archive suffix

Key Use Cases

1. Source/release tarballs

2. Backup directories
3. Move trees between hosts (with `ssh`)
4. Inspect packages without unpacking fully

Safety

Extracting as root can overwrite system paths if the archive contains absolute or `../` paths. Prefer extracting as unprivileged user into a clean directory; inspect with `-t` first. Avoid piping untrusted tar to disk as root.

Examples with Explanations

Create gzip archive

```
tar -czvf project.tar.gz project/
```

Create zstd archive

```
tar --zstd -cf project.tar.zst project/
# or: tar -c project | zstd -T0 -o project.tar.zst
```

List

```
tar -tzf project.tar.gz | head
tar -tvf project.tar.gz | head
```

Extract

```
tar -xzf project.tar.gz
tar -xzf project.tar.gz -C /tmp/unpack
```

Strip top-level directory

```
tar -xzf project.tar.gz --strip-components=1 -C /opt/app
```

Exclude patterns

```
tar -czf src.tar.gz --exclude='.git' --exclude='node_modules' .
```

Stream over ssh

```
tar -C /var/www -czf - site | ssh backup 'cat > /backups/site.tgz'
ssh backup 'cat /backups/site.tgz' | tar -C /restore -xzf -
```

Append is special

GNU tar can update some archives, but **prefer rebuilding** compressed archives rather than `--append`.

Common Usage Patterns

Quick directory move

```
tar -C /src -cf - mydir | tar -C /dst -xf -
```

Notes & Pitfalls

- Order of options is flexible with GNU tar but put `-f` next to the filename.
- Binary integrity: pair with `sha256sum` for releases.
- Sparse files: `--sparse` when needed.

Related Commands

- `gzip` / `xz` / `zstd` — compressors
- `zip` / `unzip` — PKZip format
- `rsync` — sync without full archive
- `cpio` / `pax` — alternate archivers

Additional Resources

- `man tar`

gzip

Overview

`gzip` compresses or decompresses files using the DEFLATE algorithm (`.gz`). It is the most universal compression tool on Unix-like systems — logs, tarballs (`tar.gz` / `.tgz`), and HTTP content-encoding all speak gzip. Default behavior **replaces** the input file with `file.gz` (use `-k` / `--keep` to retain the original on modern gzip).

For better speed/ratio tradeoffs on new pipelines, consider `zstd`. For maximum ratio on cold archives, consider `xz`. Keep `gzip` for compatibility.

Syntax

```
gzip [options] [file...]
gunzip [options] [file...]
gzcat / zcat [file...]          # decompress to stdout
```

Common Options

Option	Description
`-c`, `--stdout`	Write to stdout; keep inputs
`-d`, `--decompress`	Decompress (same as `gunzip`)
`-k`, `--keep`	Keep input files
`-f`, `--force`	Force overwrite / compress links
`-n` / `-N`	Don't save / do save original name & time
`-q`	Quiet
`-r`	Recursive directories
`-t`	Test integrity
`-v`	Verbose
`-1` … `-9`	Fastest … best compression (default 6)
`-l`	List compressed size / ratio for `.gz` files

Examples with Explanations

Compress / decompress

```
gzip file.txt                # creates file.txt.gz, removes file.txt
gzip -k file.txt             # keep original
gunzip file.txt.gz
gzip -d file.txt.gz          # same
```

stdout pipelines

```
gzip -c file.txt > file.txt.gz
gunzip -c file.txt.gz > file.txt
tar cf - dir | gzip -9 > dir.tar.gz
gzip -dc dir.tar.gz | tar xf -
```

Level tradeoffs

```
gzip -1 fast.log             # CPU cheap
gzip -9 archive.dat          # smaller, slower
```

Test and list

```
gzip -t file.txt.gz
gzip -l *.gz
```

Recursive logs

```
gzip -r /var/log/old/        # careful: compresses in place
find /var/log -name '*.log' -mtime +7 -exec gzip -9 {} +
```

View without extracting

```
zcat file.txt.gz | less
zgrep 'ERROR' app.log.gz
zless app.log.gz
```

Integrity in scripts

```
if gzip -t backup.gz 2>/dev/null; then
  echo ok
fi
```

Notes / Pitfalls

- Without `-k`, the source file is removed after successful compression.
- Compressing already-compressed data (jpeg, mp4, `.gz`) wastes CPU and may grow size.
- `gzip -r` is easy to over-apply on system directories — scope with `find`.
- Concatenated `.gz` members are valid; some tools only read the first.
- Sparse files and special files: stick to regular files.

2026-relevant notes

- Still the interchange default for tarballs and many APIs.
- Prefer `zstd` for local high-volume logs when all consumers support it.
- `pigz` provides parallel gzip when you need multi-core compression of large streams.

Related Commands

- `gunzip` / `zcat` / `zgrep` / `zless` — decompress / search / page
- `pigz` — parallel gzip
- `xz` / `zstd` / `bzip2` — alternate compressors
- `tar` — archive + compress combo
- `gzip -l` vs `stat` — size inspection

Additional Resources

- `man gzip`

xz

Overview

`xz` compresses and decompresses files in the **XZ** format (LZMA2), typically yielding better ratios than gzip/bzip2 at the cost of more CPU and memory — especially at high presets. Common for source tarballs (`*.tar.xz`), package payloads, and cold archives.

Companion commands: `unxz`, `xzcat`, `xzgrep`.

Syntax

```
xz [options] [file...]
unxz [options] [file...]
xzcat [file...]
```

Common Options

Option	Description
`-z` / `-d`	Compress (default) / decompress
`-c`	Write to stdout
`-k`, `--keep`	Keep input files
`-f`	Force
`-t`	Test integrity
`-l`	List information about `.xz` files
`-0` … `-9`	Compression preset (default 6); higher = smaller/slower/more RAM
`-e`, `--extreme`	Try harder within preset (more CPU)
`-T N`, `--threads=N`	Multithreaded compression (when built with support)
`-v`	Verbose
`-q`	Quiet
`--memlimit=SIZE`	Cap memory usage

Examples with Explanations

Basic

```
xz file.tar
xz -k file.tar
unxz file.tar.xz
xz -d file.tar.xz
```

Pipelines with tar

```
tar -cJf archive.tar.xz dir/          # tar invokes xz
tar -caf archive.tar.xz dir/          # auto by extension (GNU tar)
xz -dc archive.tar.xz | tar xf -
tar -xJf archive.tar.xz
```

Presets and threads

```
xz -0 -T0 fast.dat          # fast, multi-thread (T0 = all cores when supported)
xz -9e cold-archive.img     # max effort; watch RAM
xz -T4 -6 backup.sql
```

Test and list

```
xz -t archive.tar.xz
xz -l *.xz
xz -lv package.tar.xz
```

Keep and stdout

```
xz -kc file > file.xz
xzcat file.xz | less
```

Memory safety on small hosts

```
xz --memlimit=50% -T2 big.img
```

High `-9` on a 1 GB VPS can invoke the OOM killer — lower preset or set a memlimit.

Convert from gzip (recompress)

```
gzip -dc old.tar.gz | xz -T0 -c > new.tar.xz
```

Notes / Pitfalls

- Decompression is faster than compression but still heavier than gzip for weak CPUs.
- Multi-threaded mode increases memory roughly with threads × preset needs.
- Not as ubiquitous as `.gz` on ancient appliances — confirm consumer support.
- Default may delete inputs without `-k` (like gzip).
- Don't use extreme presets for live log shipping; use zstd/gzip.

2026-relevant notes

- Many distros ship packages and kernel sources as `.tar.xz`.
- For new internal pipelines, **zstd** often wins on speed with comparable ratio at mid levels; xz remains fine for cold distribution archives.
- `pixz` / threaded xz variants exist for parallelize-friendly workflows.

Related Commands

- `unxz` / `xzcat` / `xzgrep` — decompress helpers
- `gzip` / `zstd` / `bzip2` — alternatives
- `tar -J` / `tar --xz` — archive integration
- `lzma` — legacy LZMA utils (related family)
- `7z` — multi-format archiver

Additional Resources

- `man xz`

bzip2

Overview

`bzip2` compresses files with the Burrows–Wheeler algorithm (`.bz2`). It typically achieves better ratios than gzip but is slower and has largely been superseded by **xz** (better ratio) and **zstd** (better speed/ratio balance) for new work. You will still encounter `.bz2` and `.tar.bz2` in older archives and some software distributions.

Syntax

```
bzip2 [options] [file...]
bunzip2 [options] [file...]
bzcat [file...]
```

Common Options

Option	Description
`-c`	Write to stdout
`-d`	Decompress
`-z`	Compress (default)
`-k`	Keep input files
`-f`	Force
`-t`	Test integrity
`-v`	Verbose
`-q`	Quiet
`-s`	Small memory mode
`-1` … `-9`	Block size / effort (100k–900k blocks)

Examples with Explanations

Compress / decompress

```
bzip2 file.txt
bzip2 -k file.txt
bunzip2 file.txt.bz2
bzip2 -d file.txt.bz2
```

Pipelines

```
bzip2 -c file.txt > file.txt.bz2
bzcat file.txt.bz2 | less
tar cjf archive.tar.bz2 dir/
tar xjf archive.tar.bz2
```

Levels

```
bzip2 -1 quick.dat
bzip2 -9 small.dat
```

Test

```
bzip2 -t file.txt.bz2
```

Search compressed

```
bzgrep 'ERROR' app.log.bz2
bzless app.log.bz2
```

Prefer modern tools when choosing

```
# old style
tar cjf backup.tar.bz2 data/
# modern defaults for new archives
tar --zstd -cf backup.tar.zst data/
# or
tar cJf backup.tar.xz data/
```

Notes / Pitfalls

- Deletes inputs by default without `-k`.
- CPU-heavy relative to gzip/zstd for similar tasks.
- Memory scales with block size (`-s` helps constrained systems).
- Less common in new HTTP APIs than gzip/br/zstd.
- Parallel variant: `pbzip2` / `lbzip2` if installed.

2026-relevant notes

- Maintain decompression support for legacy artifacts; avoid bzip2 for new internal pipelines unless required.
- GNU tar still supports `-j` / `--bzip2`.
- If an upstream only publishes `.tar.bz2`, trust `tar xjf` / `bzip2 -d` rather than recompressing without need.

Related Commands

- `bunzip2` / `bzcat` / `bzgrep` / `bzless`
- `gzip` / `xz` / `zstd`
- `tar -j`
- `pbzip2` — parallel bzip2

Additional Resources

- `man bzip2`

zstd

Overview

`zstd` (Zstandard) is a modern compression tool offering an excellent **speed vs ratio** curve, levels from very fast to extremely dense, optional multithreading, and framing with checksums. It is the preferred default for many new local pipelines, package systems, and container layers. Extension: `.zst`.

Syntax

```
zstd [options] [file...]
zstd -d [options] [file...]
zstdcat [file...]
unzstd [file...]
```

Common Options

Option	Description
`-d`, `--decompress`	Decompress
`-c`, `--stdout`	Write to stdout
`-f`, `--force`	Overwrite / compress links
`-o file`	Output file name
`-k` / `--rm`	Keep sources (default keep) / remove sources after success
`-t`	Test integrity
`-T#`	Threads (`-T0` = all cores)
`-#`	Level 1–19 (default 3); ultra with `--ultra -20…-22`
`-v`	Verbose
`-q`	Quiet
`-l` / `--list`	Show frame info
`--train`	Build a dictionary (advanced)
`-D dict`	Compress/decompress with dictionary
`--long`	Long distance matching mode

Unlike classic gzip defaults, **zstd keeps the original file** unless you pass `--rm`.

Examples with Explanations

Basic

```
zstd file.tar
zstd -d file.tar.zst
zstd -o out.zst file
unzstd file.tar.zst
```

Levels and threads

```
zstd -1 -T0 fast.log
zstd -19 -T0 cold.img
zstd --ultra -22 -T0 deepest.img   # very slow/heavy
for l in 1 3 6 9 15 19; do
  zstd -$l -f -o /tmp/t.zst sample.bin && ls -lh /tmp/t.zst
done
```

Pipelines with tar

```
tar -I 'zstd -T0' -cf backup.tar.zst dir/
tar --zstd -cf backup.tar.zst dir/          # GNU tar 1.31+
zstd -dc backup.tar.zst | tar xf -
tar -I zstd -xf backup.tar.zst
```

Integrity

```
zstd -t backup.tar.zst
zstd -l backup.tar.zst
```

Remove source when desired

```
zstd --rm big.img
```

Dictionaries (small similar files)

```
zstd --train -o dict samples/*
zstd -D dict small1 small2
zstd -d -D dict small1.zst
```

Useful for massive numbers of similar tiny JSON/logs.

Notes / Pitfalls

- Confirm consumers understand `.zst` before replacing gzip in public APIs.
- Ultra levels need large memory windows — watch small VMs.
- Default **keeps** inputs; scripts ported from gzip may leave duplicates if they assume deletion.
- `tar --zstd` requires sufficiently new GNU tar; `-I 'zstd -T0'` is explicit and flexible.
- Older enterprise appliances may still demand `.gz`.

2026-relevant notes

- Default choice for many new internal artifacts, caches, and log archives.
- Kernel squashfs, package managers, and container tooling increasingly support zstd natively.
- Compare: **gzip** (universal), **xz** (max ratio, slow), **zstd** (best general default).

Related Commands

- `gzip` / `xz` / `bzip2` — alternatives
- `tar` — archive integration
- `zstdcat` / `zstdgrep` (if packaged)
- `pzstd` — parallel client in some packages
- `lz4` — even faster, lower ratio

Additional Resources

- `man zstd`
- facebook/zstd

zip

Overview

`zip` creates **ZIP** archives widely used for cross-platform interchange (Windows, macOS, Linux). Unlike `tar` + compressor pipelines, zip typically **compresses each member** and stores a central directory. Prefer `tar` + `gzip`/`zstd` for Unix-native backups with permissions/owners; use `zip` when recipients expect `.zip` or you need simple selective compression of mixed files.

Syntax

```
zip [options] archive.zip file...
zip [options] archive.zip -r dir/
```

Common Options

Option	Description
`-r`	Recurse into directories
`-q`	Quiet
`-v`	Verbose
`-u`	Update existing entries
`-m`	Move files into zip (delete after)
`-e`	Encrypt (traditional; weak by modern standards)
`-P pass`	Password on CLI (visible in process list — avoid)
`-x pattern`	Exclude patterns
`-i pattern`	Include patterns
`-y`	Store symbolic links as links (Unix)
`-j`	Junk (store) path names — flatten
`-0 … -9`	Compression level (0 = store)
`-s size`	Split archive into segments
`-F / -FF`	Fix archive

Examples with Explanations

Create archives

```
zip notes.zip *.txt
zip -r project.zip project/
zip -r project.zip project/ -x 'project/node_modules/*' -x '*.git*'
```

Update and list

```
zip -u project.zip project/README.md
zip -sf project.zip                 # show files (zipinfo-like)
unzip -l project.zip                # often clearer listing
```

Flatten paths

```
zip -j flat.zip path/to/a.txt path/to/b.txt
```

Store without compression

```
zip -0 -r media.zip photos/         # already-compressed JPEGs
```

Encryption (know the limits)

```
zip -e secret.zip secrets.txt
# traditional zip crypto is weak; prefer 7z AES or age/gpg for real secrets
```

Split archives

```
zip -r -s 100m backup.zip data/
```

Exclude junk

```
zip -r src.zip src/ -x '*/__pycache__/*' -x '*.pyc' -x '*/.DS_Store'
```

Date-stamped backup

```
zip -r "backup-$(date +%Y%m%d).zip" documents/
```

Notes / Pitfalls

- Unix permissions, owners, and special files are a poor fit; `tar` preserves more faithfully.
- Password on the command line (`-P`) leaks via `ps` — interactive `-e` is better but still weak crypto.
- Large file ZIP64 support depends on zip version; very old unzip may fail.
- Default may follow or store links differently than you expect — test with `-y`.
- Windows/macOS clients may mishandle exotic UTF-8 names depending on flags/tools.

2026-relevant notes

- For secure sharing, prefer `age`, `gpg`, or encrypted 7z over zip's traditional encryption.
- CI artifacts often use zip for multi-OS consumers — keep compression level modest for speed.
- `unzip` / `bsdunzip` / Explorer all need to open your result — avoid exotic extensions.

Related Commands

- `unzip` — extract / list
- `zipinfo` — detailed listing
- `7z` — stronger formats/crypto
- `tar` — Unix-native archives
- `zstd` / `gzip` — stream compressors

Additional Resources

- `man zip`

unzip

Overview

`unzip` lists, tests, and extracts **ZIP** archives. It is the usual counterpart to `zip` on Linux. Options differ from `tar`; path traversal and overwrite behavior deserve care when extracting untrusted archives.

Syntax

```
unzip [options] archive.zip [file...] [-x xfile...] [-d exdir]
```

Common Options

Option	Description
`-l`	List contents
`-v`	Verbose listing / version
`-t`	Test integrity
`-o`	Overwrite files without prompting
`-n`	Never overwrite
`-d dir`	Extract into directory
`-j`	Junk paths (flatten)
`-q`	Quiet
`-f`	Freshen existing files only
`-u`	Update (freshen + extract new)
`-P pass`	Password (visible in process list — avoid)
`-x pattern`	Exclude files
`-Z`	Zipinfo mode (if compiled)

Examples with Explanations

List and test

```
unzip -l archive.zip
unzip -v archive.zip
unzip -t archive.zip
```

Extract

```
unzip archive.zip
unzip archive.zip -d /tmp/out/
mkdir -p /tmp/out && unzip -d /tmp/out archive.zip
```

Selective extract

```
unzip archive.zip readme.txt
unzip archive.zip 'src/*.c' -d /tmp/src
unzip archive.zip -x '*.o' -d /tmp/out
```

Overwrite policies

```
unzip -o archive.zip            # always overwrite
unzip -n archive.zip            # never overwrite
```

Flatten paths

```
unzip -j archive.zip -d /tmp/flat
```

Pipe / quiet automation

```
unzip -q -o artifact.zip -d "$BUILD_DIR"
```

Untrusted archives — path safety

```
# inspect before extract
unzip -l untrusted.zip | less
# extract into empty sandbox only
mkdir -p /tmp/sandbox && unzip -d /tmp/sandbox untrusted.zip
# watch for entries with .. or absolute paths
unzip -l untrusted.zip | awk '{print $NF}' | grep -E '(^\.\.|^\|/)'
```

Modern `unzip` versions try to block some unsafe paths; still extract untrusted input in a sandbox.

Password

```
unzip -P 'secret' encrypted.zip     # leaks via ps
unzip encrypted.zip                 # prompts
```

Notes / Pitfalls

- Default extract path is the **current directory** — use `-d` deliberately.
- Filename encoding (CP437 vs UTF-8) can garble non-ASCII names from Windows zips.
- ZIP64 needed for huge archives; old unzip fails.
- Don't use `-P` in shared environments.
- `busybox unzip` supports a subset of flags.

2026-relevant notes

- CI jobs should pin `-o`/`-n` explicitly for non-interactive reliability.
- Prefer artifact scanning before unzip on multi-tenant builders.
- For tar-based Linux artifacts, stick to `tar` rather than converting everything to zip.

Related Commands

- `zip` — create archives
- `zipinfo` — detailed metadata
- `7z x` — alternative extractor
- `bsdtar` / `tar` — can read some zips depending on build
- `funzip` — pipe single-member extract

Additional Resources

- `man unzip`

zcat

Overview

`zcat` writes **gzip-compressed** files to stdout as if they were uncompressed (equivalent to `gzip -dc`). Handy in pipelines without creating temporary files. For `.xz`/`.zst` use `xzcat`/`zstdcat` (or `zcat` only where your distro aliases them carefully — do not assume).

Syntax

```
zcat [options] [file...]
gzip -dc file.gz
```

Examples with Explanations

Read a gzip log

```
zcat /var/log/syslog.1.gz | less
zcat app.log.gz | grep ERROR | tail
```

Concatenate multiple

```
zcat part-*.gz > combined.log
```

Portable explicit form

```
gzip -dc file.gz | jq .
```

Notes & Pitfalls

- `zcat` on some systems is a symlink to `gzip -dc` and only handles gzip.
- Compressed sparse/binary data still expands fully — watch disk when redirecting.
- Prefer `zgrep` when you only need matches.

Related Commands

- `gzip` / `gunzip`
- `zgrep` / `zless`
- `xzcat` / `bzcat` / `zstdcat`
- `tar -xzOf` — extract one member to stdout

Additional Resources

- `man zcat` / `man gzip`

zgrep

Overview

`zgrep` runs `grep` over **gzip-compressed** files without manual decompression. Useful for rotated logs (`*.gz`). Variants exist for other compressors (`xzgrep`, `bzgrep`, `zstdgrep`).

Syntax

```
zgrep [grep-options] pattern [file...]
```

Examples with Explanations

Search compressed logs

```
zgrep -H 'error' /var/log/nginx/access.log.*.gz
zgrep -E 'HTTP/1\.[01]" 5[0-9]{2}' access.log.2.gz
```

Recursive pattern over mixed trees

```
find /var/log -name '*.gz' -print0 | xargs -0 zgrep -H 'OutOfMemory'
```

Count matches

```
zgrep -c 'ERROR' app.log.gz
```

Notes & Pitfalls

- Performance is worse than grepping already-decompressed data — decompress hot logs if you query them constantly.
- Binary matches: use `grep` flags carefully (`-a`, `-I`).
- Confirm whether your `zgrep` shells out to `gzip -dc` + `grep`.

Related Commands

- `grep` / `rg`
- `zcat` / `zless`
- `journalctl` — live system logs
- `xzgrep` / `zstdgrep`

Additional Resources

- `man zgrep`

System Information

Intro

Intro

Identify the host, kernel, time zone, logged-in users, environment, and high-level hardware inventory. Start here when answering “what machine is this?” and “who is on it?”.

Commands in this part

Command	Role
`uname`	uname prints system information: kernel name, hostname/nodename, kernel release/version, machine hardware name, and…
`hostname`	hostname prints or sets the system hostname.
`hostnamectl`	hostnamectl queries and changes the system hostname and related machine identity fields on systemd systems.
`lsb_release`	lsb_release prints distribution identification claimed via the Linux Standard Base (LSB) interfaces and…
`uptime`	uptime shows how long the system has been running, how many users are logged in, and the load average (1, 5, and 15…
`date`	date prints or sets the system date and time.
`timedatectl`	timedatectl views and changes the system clock, timezone, RTC settings, and NTP synchronization on systemd systems.
`cal`	cal displays a calendar in the terminal — current month by default, or a specific month/year.
`whoami`	whoami prints the effective username of the current user.
`id`	id prints the real and effective user and group IDs for a process (by default, yours), plus supplementary groups.

Command	Role
`who`	who shows who is logged in, with terminal, login time, and sometimes host/remote info from utmp/wtmp accounting.
`w`	w shows who is logged in and what they are doing, plus system uptime and load averages.
`last`	last shows a history of logins, reboots, and runlevel changes from the wtmp accounting file (typically /var/log/wtmp).
`env`	env runs a program in a modified environment, or prints the current environment variables when given no command.
`free`	free summarizes system memory usage: physical RAM and swap.
`lscpu`	lscpu displays CPU architecture information from sysfs and /proc/cpuinfo: sockets, cores, threads, model name,…
`lsmem`	The lsmem command displays information about memory ranges and their online/offline status.
`lsusb`	lsusb lists USB devices connected to the system (buses, IDs, vendor/product strings).
`lsmod`	lsmod lists loaded kernel modules (name, size, use count, dependents).
`hwinfo`	The hwinfo command provides comprehensive hardware information.

Suggested starting points

1. Kernel/OS: `uname`, `lsb_release` / os-release, `hostnamectl`.
2. Time: `date`, `timedatectl`.
3. Identity: `whoami`, `id`, `who`, `w`, `last`.
4. Resources snapshot: `free`, `lscpu`, `lsmem`, `uptime`.
5. Hardware peek: `lsusb`, `lsmod`, `hwinfo` (deeper inventory is in Hardware).

Related parts

- Hardware — PCI/DMI/disk identity tools

- Processes and jobs — live load after identity checks
- Services and runtime — other systemd `*ctl` tools

Continue with the individual command pages in this part.

uname

Overview

`uname` prints system information: kernel name, hostname/nodename, kernel release/version, machine hardware name, and sometimes processor/OS fields. Scripts use it to branch on architecture (`x86_64` vs `aarch64`) and kernel release.

Syntax

```
uname [options]
```

Common Options

Option	Description
`-a`, `--all`	All information in standard order
`-s`, `--kernel-name`	Kernel name (default if no options) e.g. `Linux`
`-n`, `--nodename`	Network node hostname
`-r`, `--kernel-release`	Kernel release e.g. `6.8.0-xx`
`-v`, `--kernel-version`	Kernel build version string
`-m`, `--machine`	Machine hardware name e.g. `x86_64`, `aarch64`
`-p`, `--processor`	Processor type (often `unknown` on Linux)
`-i`, `--hardware-platform`	Hardware platform (often `unknown`)
`-o`, `--operating-system`	OS name e.g. `GNU/Linux`

Examples with Explanations

Everyday

```
uname
uname -a
uname -r
uname -m
```

Architecture branching

```
case $(uname -m) in
  x86_64)  arch=amd64 ;;
  aarch64) arch=arm64 ;;
  armv7l)  arch=armhf ;;
  *) echo "unsupported: $(uname -m)" >&2; exit 1 ;;
esac
echo "download package for $arch"
```

Kernel version checks

```
uname -r
# compare carefully - string sort   version sort
printf '%s\n' "$(uname -r)" | sort -V
```

vs os-release

```
uname -a
cat /etc/os-release
hostnamectl
```

`uname` is kernel-focused; distro identity lives in `/etc/os-release`.

Container note

```
uname -r
# often shows the *host* kernel - containers share it
cat /etc/os-release   # container userspace
```

Compact inventory line

```
printf '%s %s %s\n' "$(uname -s)" "$(uname -r)" "$(uname -m)"
```

Notes / Pitfalls

- `-p`/`-i` frequently print `unknown` on Linux — use `-m`.
- Don't parse `uname -a` as a stable API; request specific flags.
- Version compares need `sort -V` or proper version libraries.
- Different libc/coreutils may format fields slightly differently.
- WSL and custom kernels still report as Linux with distinctive release strings.

2026-relevant notes

- Multi-arch containers and ARM laptops make `uname -m` essential in install scripts.
- Long-term kernels vs mainline: check `uname -r` against CVE guidance for your distro package, not upstream only.
- Pair with `lscpu` for CPU features (`flags`) when deciding SIMD builds.

Related Commands

- `hostname` / `hostnamectl` — host identity
- `cat /etc/os-release` — distro identity
- `lscpu` — CPU details
- `arch` — often alias-like to `uname -m`
- `hostnamectl` — pretty OS string on systemd

Additional Resources

- `man uname`

hostname

Overview

hostname prints or sets the system hostname. On modern systemd systems, prefer **hostnamectl** for persistent hostname changes and clearer status. The classic **hostname** command remains useful for quick reads and scripts.

Related names: static hostname, pretty hostname, transient hostname (DHCP/cloud).

Syntax

```
hostname [options] [name]
```

Common Options

Option	Description
(none)	Print hostname
`name`	Set hostname (needs root; often transient without systemd integration)
`-f`, `--fqdn`	Fully qualified domain name (best effort)
`-s`, `--short`	Short name (before first dot)
`-i`, `--ip-address`	Addresses for host (unreliable; prefer `ip`)
`-I`	All IP addresses on interfaces (not DNS)
`-d`	DNS domain name (best effort)
`-A`	All FQDNs
`-y`	NIS domain
`--help`	Help

Behavior of DNS-related flags depends on resolver configuration and `/etc/hosts`.

Examples with Explanations

Read

```
hostname
hostname -s
hostname -f
hostname -I
```

Prefer hostnamectl on systemd

```
hostnamectl
hostnamectl status
sudo hostnamectl set-hostname app-01
```

This updates the persistent hostname properly on systemd hosts.

Temporary classic set (legacy)

```
sudo hostname app-temp
# may not survive reboot without writing config / hostnamectl
```

Scripts

```
h=$(hostname -s)
echo "running on $h"
```

Correlate with DNS / addresses

```
hostname -f
getent hosts "$(hostname -f)"
ip -br addr
```

`-i`/`-f` can lie if `/etc/hosts` and DNS disagree — verify with `ip` and `resolvectl`.

Notes / Pitfalls

- Setting via bare `hostname` without `hostnamectl`/config is a common "lost after reboot" mistake.
- Cloud images may regenerate hostnames via cloud-init.
- FQDN detection is best-effort, not a pure DNS authoritative query.
- Containers have their own hostname (pod name, docker `--hostname`).
- Valid hostname rules: prefer short DNS labels (`a-z`, `0-9`, `-`).

2026-relevant notes

- systemd hosts: **`hostnamectl`** is the supported admin interface.
- Kubernetes sets pod hostnames independently of node hostname.
- Inventory tools (Ansible) often use inventory names that differ from `hostname` — don't assume equality.

Related Commands

- `hostnamectl` — systemd hostname management
- `resolvectl` / `getent hosts` — name resolution
- `ip` — addresses
- `uname -n` — nodename (usually same as hostname)
- `nmcli` — NetworkManager host/domain bits

Additional Resources

- `man hostname`, `man hostnamectl`

hostnamectl

Overview

`hostnamectl` queries and changes the system hostname and related machine identity fields on **systemd** systems. It is the preferred way to set a **persistent** hostname (vs classic `hostname` alone, which may only change the transient name until reboot).

Syntax

```
hostnamectl [options] [command]
```

Common Commands / Options

Command / option	Description
`status`	Show current hostnames and OS info (default)
`set-hostname NAME`	Set hostname
`set-hostname NAME --static`	Static hostname
`set-hostname NAME --transient`	Transient only
`set-hostname NAME --pretty`	Pretty (presentation) name
`set-icon-name` / `set-chassis` / `set-deployment` / `set-location`	Machine metadata
`set-hostname "" --pretty`	Clear pretty name
`-H host` / `--machine=`	Operate on remote/container via systemd
`--no-ask-password`	Non-interactive privilege

Examples with Explanations

Status

```
hostnamectl
hostnamectl status
```

Shows static/transient/pretty hostnames, machine ID icons, chassis, OS pretty name, kernel, architecture.

Set hostname

```
sudo hostnamectl set-hostname app-01
hostnamectl
hostname
```

Pretty name

```
sudo hostnamectl set-hostname "App Server 01" --pretty
sudo hostnamectl set-hostname app-01 --static
```

Transient (temporary)

```
sudo hostnamectl set-hostname temp-probe --transient
```

Chassis / location metadata

```
sudo hostnamectl set-chassis server
sudo hostnamectl set-location "rack-3-u12"
```

Remote / container

```
hostnamectl -H alice@host.example
hostnamectl --machine=mycontainer
```

Verify files

```
cat /etc/hostname
ls -l /etc/machine-id
hostname
```

Notes / Pitfalls

- Requires systemd; classic SysV-only systems won't have it.
- cloud-init may overwrite hostnames on next boot if configured to manage them.
- DNS is separate: setting hostname does not create DNS records.
- Valid hostname labels: prefer RFC-ish DNS labels for static names; pretty names can be freer.
- Containers may restrict hostname changes.

2026-relevant notes

- Standard admin path on almost all mainstream server distros.
- Inventory tools should read `hostnamectl` status or `/etc/hostname` + `/etc/os-release`.
- Pair with `timedatectl` for complete first-boot identity setup.

Related Commands

- `hostname` — classic get/set
- `timedatectl` — time/timezone
- `localectl` — locale/keymap
- `resolvectl` — DNS
- `cat /etc/os-release` — distro identity

Additional Resources

- `man hostnamectl`

lsb_release

Overview

`lsb_release` prints distribution identification claimed via the Linux Standard Base (LSB) interfaces and distro-specific release files. Still useful for quick "what am I on?" checks, but many modern tools prefer reading **`/etc/os-release`**, which is the freedesktop standard and always present on current systemd distros.

On some minimal images, `lsb_release` is not installed (`lsb-release` package).

Syntax

```
lsb_release [options]
```

Common Options

Option	Description
`-v`	LSB modules / version info
`-i`	Distributor ID
`-d`	Description
`-r`	Release number
`-c`	Codename
`-a`	All of the above
`-s`	Short output (no headers)

Examples with Explanations

Everyday

```
lsb_release -a
lsb_release -is
lsb_release -cs
lsb_release -rs
```

Short for scripts

```
distro=$(lsb_release -is 2>/dev/null || true)
codename=$(lsb_release -cs 2>/dev/null || true)
```

Prefer os-release (portable modern)

```
. /etc/os-release
echo "$ID $VERSION_ID $PRETTY_NAME"
# or
grep -E '^(ID|VERSION_ID|PRETTY_NAME)=' /etc/os-release
```

Compare sources

```
lsb_release -a 2>/dev/null
cat /etc/os-release
hostnamectl | grep -i 'operating system'
```

Install if missing

```
# Debian/Ubuntu
sudo apt install lsb-release
# Fedora
sudo dnf install redhat-lsb-core   # may be transitional / optional
```

Often unnecessary if you only need `os-release`.

Notes / Pitfalls

- Not installed everywhere; scripts should fall back to `/etc/os-release`.
- Derived images and containers can customize IDs — trust but verify package availability separately.
- Codename strings differ (`jammy`, `bookworm`, `Trixie`) — don't assume Ubuntu-only.
- LSB compliance fields (`-v`) are historical and less operationally useful today.
- Rolling releases may show unusual version strings.

2026-relevant notes

- **Canonical for automation:** `/etc/os-release` fields `ID`, `ID_LIKE`, `VERSION_ID`.
- Cloud images still include `lsb_release` sometimes for legacy install scripts.
- Prefer feature detection (`command -v`, `/etc/os-release`) over brittle distro name switches when possible.

Related Commands

- `cat /etc/os-release` — standard identity
- `hostnamectl` — pretty OS string
- `uname` — kernel identity
- `cat /etc/debian_version` / `redhat-release` — legacy files
- package managers — `apt`/`dnf` confirm edition

Additional Resources

- `man lsb_release`
- freedesktop os-release specification

uptime

Overview

`uptime` shows how long the system has been running, how many users are logged in, and the **load average** (1, 5, and 15 minutes). It is a quick health snapshot before diving into `top`/`htop`/`vmstat`.

Load average is the average number of runnable **plus** uninterruptible (disk sleep) tasks — not a pure "CPU %" meter.

Syntax

```
uptime [options]
```

Common Options

Option	Description
`-p`, `--pretty`	Pretty duration only
`-s`, `--since`	System up since timestamp
`-h`, `--help`	Help
`-V`	Version

Examples with Explanations

Default

```
uptime
# 10:15:02 up 21 days,  4:03,  3 users,  load average: 0.08, 0.12, 0.09
```

Pretty / since

```
uptime -p
uptime -s
```

Interpret load vs CPUs

```
uptime
nproc
lscpu | grep '^CPU(s):'
# rough rule of thumb: load near nproc   busy; much higher may mean saturation
```

Watch over time

```
watch -n1 uptime
```

Scripts

```
load=$(awk '{print $1}' /proc/loadavg)
echo "1-minute load: $load"
# /proc/loadavg is the source of truth
cat /proc/loadavg
```

Correlate with other tools

```
uptime
who
w
top -bn1 | head
```

Containers

```
uptime
# may reflect host uptime depending on visibility of /proc
```

Understanding load average

Observation	Possible meaning
Low load, high latency	Network/app issues, not CPU
Load nproc, CPU idle% high	I/O wait / uninterruptible tasks
Load runnable tasks	CPU contention
Sudden load spikes	Cron storms, backups, build jobs

Use `vmstat 1`, `iostat -xz 1`, `ps`, `pidstat` for root cause.

Notes / Pitfalls

- Load is not "percent CPU"; a 64-core host can show load 20 and still be fine.
- Suspend-to-RAM / cloud live migration can make "up" time less intuitive.
- User count is from utmp; containers/ssh multiplexing may look odd.
- Don't alert only on load without CPU, memory, and I/O context.

2026-relevant notes

- For long-term trends use Prometheus node exporter / `sar` rather than one-off `uptime`.
- Latency-sensitive systems care more about p99 and steal time (VMs) than raw load.
- `systemd-analyze` helps boot time; `uptime -s` shows when the current boot began.

Related Commands

- `w` / `who` — who is logged in
- `top` / `htop` / `btop` — live process view
- `vmstat` / `iostat` — run queue vs block I/O
- `nproc` / `lscpu` — CPU count
- `cat /proc/loadavg` — raw values
- `sar` — historical load

Additional Resources

- `man uptime`, `man 5 proc` (`/proc/loadavg`)

date

Overview

`date` prints or sets the system date and time. Daily use is formatting timestamps for logs, filenames, and scripts. Setting the clock requires privileges and is usually better left to **NTP/`timedatectl`** on modern systems.

Syntax

```
date [options] [+FORMAT]
date [-u|--utc|--universal] [MMDDhhmm[[CC]YY][.ss]]   # set (legacy)
```

Common Options

Option	Description
`+FORMAT`	Output format string
`-u`, `--utc`	UTC
`-d STRING`, `--date=`	Display time described by STRING (not set)
`-f FILE`	Like `-d` for each line in FILE
`-s STRING`, `--set=`	Set time (privileged)
`-R`	RFC 5322 format
`-I[TIMESPEC]`	ISO 8601 (`date -Iseconds`)
`-r FILE`	Use file mtime

Useful format sequences

Seq	Meaning
`%Y-%m-%d`	2026-08-04
`%H:%M:%S`	14:05:09
`%F`	Same as `%Y-%m-%d`
`%T`	Same as `%H:%M:%S`
`%s`	Epoch seconds
`%z` / `%Z`	Offset / timezone name

Seq	Meaning
%V	ISO week
%a / %A	Weekday

Examples with Explanations

Now

```
date
date -u
date -Iseconds
date +%F
date +%Y%m%d-%H%M%S
date +%s
```

Convert / relative

```
date -d '2026-01-15 10:00'
date -d '2 days ago' +%F
date -d '@1700000000'
date -d 'next Monday'
```

Filenames and logs

```
logfile="app-$(date +%Y%m%d).log"
echo "$(date -Is) started" >> "$logfile"
```

File mtime

```
date -r /etc/hosts
stat -c %y /etc/hosts
```

Timezone one-off

```
TZ=UTC date
TZ=America/New_York date
timedatectl
```

Set clock (prefer NTP)

```
# last resort; prefer chrony/systemd-timesyncd
sudo date -s '2026-08-04 12:00:00'
sudo timedatectl set-time '2026-08-04 12:00:00'
sudo timedatectl set-ntp true
```

GNU vs scripts

```
# portable-ish epoch:
date +%s
# ISO local:
date -Iseconds
```

Notes / Pitfalls

- Setting time on production without NTP understanding breaks certs, logs, and Kerberos.
- Locale affects names (`%B`); use `LC_ALL=C` for stable parsing.
- macOS `date` flags differ; this book targets GNU/Linux.
- Leap seconds / monotonic clocks: for durations prefer `EPOCHREALTIME` or dedicated monotonic APIs in apps.
- Containers may use host time namespace.

2026-relevant notes

- Always enable NTP via `timedatectl` / chrony on servers.
- Log pipelines prefer UTC (`date -u -Iseconds`).
- For scheduling, pair with `systemd-run` / cron and explicit TZ.

Related Commands

- `timedatectl` — timezone and NTP status
- `hwclock` — hardware clock
- `cal` — calendar
- `touch -d` — set file times
- `stat` — file timestamps

Additional Resources

- `man date`, `man timedatectl`

timedatectl

Overview

`timedatectl` views and changes the system clock, timezone, RTC settings, and NTP synchronization on **systemd** systems. Prefer it over ad-hoc `date -s` for persistent timezone and NTP policy. On Ubuntu Server, default NTP is often **systemd-timesyncd**; some hosts use **chrony** instead — enable one cleanly, not both fighting.

Syntax

```
timedatectl [options] [command]
```

Common Commands / Options

Command / option	Description
`status`	Show time, TZ, NTP state (default)
`show`	Machine-readable properties
`list-timezones`	All zone names
`set-timezone Zone`	Set timezone (e.g. `UTC`, `America/New_York`)
`set-time 'YYYY-MM-DD HH:MM:SS'`	Set clock manually (disable NTP first if required)
`set-ntp true|false`	Enable/disable NTP (systemd unit)
`set-local-rtc 0|1`	RTC in UTC (0, recommended for Linux) or local
`-H host / --machine=`	Remote/container via systemd
`--no-ask-password`	Non-interactive privilege

Key Use Cases

1. First-boot timezone + NTP setup
2. Verify clock sync before Kerberos/TLS-sensitive work
3. Switch TZ for multi-region servers (prefer UTC on servers)
4. Diagnose “TLS not yet valid” / log timestamp skew

Examples with Explanations

Status

```
timedatectl
timedatectl status
```

Shows local time, universal time, RTC, timezone, and whether NTP is active/synchronized.

Machine-readable

```
timedatectl show
timedatectl show -p Timezone -p NTP -p NTPSynchronized --value
```

Good for scripts and monitoring checks.

List and set timezone

```
timedatectl list-timezones | grep -i america
timedatectl list-timezones | grep -i '^Asia/'
sudo timedatectl set-timezone UTC
sudo timedatectl set-timezone America/New_York
timedatectl
```

Server fleets usually standardize on **UTC**; set human TZ only when local wall time is required.

Enable NTP (systemd-timesyncd path)

```
sudo timedatectl set-ntp true
timedatectl
systemctl status systemd-timesyncd --no-pager
# Ubuntu may use:
systemctl status systemd-timesyncd.service
```

`set-ntp true` lets systemd enable the appropriate time-sync unit.

Chrony hosts

```
# If chrony is the chosen stack:
sudo apt install chrony
sudo systemctl enable --now chrony
# timedatectl should still report NTP synchronized when chrony steps/slews correctly
chronyc tracking
```

```
chronyc sources -v
timedatectl
```

Don't leave timesyncd and chrony both active and conflicting — pick one source of truth.

Manual time (maintenance / air-gap)

```
sudo timedatectl set-ntp false
sudo timedatectl set-time '2026-08-09 12:00:00'
# re-enable NTP when network is available:
sudo timedatectl set-ntp true
```

Manual set is a last resort; large steps can confuse databases and cert validation.

RTC in UTC (recommended on Linux)

```
timedatectl | grep -i rtc
sudo timedatectl set-local-rtc 0
```

Dual-boot with Windows may fight over local vs UTC RTC — document the choice on shared hardware.

Verify sync health

```
timedatectl | grep -iE 'ntp|time zone|synchronized'
# timesyncd:
timedatectl timesync-status 2>/dev/null || true
journalctl -u systemd-timesyncd -n 20 --no-pager
# chrony:
chronyc tracking 2>/dev/null || true
```

"System clock synchronized: yes" is the headline for TLS and distributed systems.

date interaction

```
date
date -u
date --iso-8601=seconds
timedatectl
```

`date` formats the current clock; `timedatectl` manages policy (TZ/NTP/RTC).

Understanding Output

Typical `timedatectl` status fields:

Field	Meaning
Local time	Wall time in current TZ
Universal time	UTC
RTC time	Hardware clock reading
Time zone	IANA zone name + offset
System clock synchronized	Whether systemd believes sync is good
NTP service	active/inactive
RTC in local TZ	n = UTC (preferred on Linux-only)

`timedatectl show` exposes properties like `Timezone=`, `NTP=`, `NTPSynchronized=`, `TimeUSec=`.

Notes & Pitfalls

- Prefer NTP over permanent manual `set-time` on networked hosts.
- Containers often inherit host time and **cannot** change the host clock; TZ can still be set in-process via `TZ` env.
- Cloud images may already enforce UTC + timesync — cloud-init can reapply settings.
- Large backward clock jumps break `make`, systemd units, and DB leases — prefer slewing via NTP.
- Kerberos and TLS are intolerant of multi-minute skew.
- `set-timezone` does not rewrite historical log timestamps already written.
- Virtual hardware RTC quality varies; NTP remains mandatory for accuracy.

Related Commands

- `date` — print/format time
- `hwclock` — read/set hardware clock directly
- `chronyc` / `chronyd` — chrony stack
- `ntpdate` — legacy one-shot (avoid on modern systemd; use chrony/timesyncd)
- `hostnamectl` — host identity companion on first boot
- `timedatectl timesync-status` — timesyncd detail when available

Additional Resources

- `man timedatectl`
- `man systemd-timesyncd.service`
- `man chrony.conf` (when using chrony)

cal

Overview

`cal` displays a calendar in the terminal — current month by default, or a specific month/year. Handy for quick date context, scripting reminders, and verifying week boundaries. Some systems ship `ncal` with alternate layouts.

Syntax

```
cal [options] [[[day] month] year]
cal [options] [[month] year]
cal [options] -y [year]
```

Exact syntax variants exist between util-linux `cal` and BSD-style `cal`/`ncal`.

Common Options (util-linux / common)

Option	Description
`-1` / `-3`	One month / three months
`-y`	Whole year
`-m`	Monday as first day of week
`-s`	Sunday first (when supported)
`-w`	Week numbers
`-j`	Julian day numbers
`-A n` / `-B n`	Months after / before
`-h`	Highlight off (or help, depending on version)
`--color`	Colorize today (newer)

Check `cal --help` on your host.

Examples with Explanations

Current month

```
cal
cal -3
cal -m
cal -w
```

Specific month / year

```
cal 12 2026
cal 2026
cal -y 2026
```

Week numbers for planning

```
cal -w
cal -mw 9 2026
```

Surrounding months

```
cal -A 1 -B 1
cal -3
```

Historical curiosity

```
cal 9 1752                # UK calendar reform month (implementation-dependent)
```

Scripts: parse carefully

```
# better for machine dates:
date +%F
date +%V                  # ISO week
# use cal for human display only
```

ncal alternative

```
ncal
ncal -b
ncal -w
```

Notes / Pitfalls

- Highlighting “today” depends on terminal and build options.
- Week-start Monday/Sunday differs by locale and flags — be explicit with `-m` when it matters.
- Do not parse `cal` output for business logic; use `date` and proper libraries.
- Year range limits differ; very ancient/future years may error.
- BusyBox `cal` is minimal.

2026-relevant notes

- ISO week numbers (`date +%V`, `cal -w`) still matter for release trains and on-call rotations.
- Prefer `date` for automation; keep `cal` as a human aid over SSH.
- Terminal Unicode width issues can misalign rare locales.

Related Commands

- `date` — current date/time formatting
- `ncal` — alternate calendar view
- `timedatectl` — timezone/NTP status
- `calendar` — reminder file tool (if installed)
- `hwclock` — hardware clock

Additional Resources

- `man cal`, `man ncal`

whoami

Overview

`whoami` prints the **effective** username of the current user. It is a tiny identity check for scripts and for noticing when `sudo`/`su` changed who you are. For full identity (uid, gid, groups) use `id`. For login name from the controlling terminal, see `logname` (different edge cases).

Syntax

```
whoami [OPTION]
# usually no options needed
```

Common Options

Option	Description
`--help`	Help
`--version`	Version

Key Use Cases

1. Confirm effective user in a shell
2. Guard scripts that must (or must not) run as root
3. Annotate logs with actor name
4. Verify `sudo -u` targets

Examples with Explanations

Basics

```
whoami
sudo whoami                  # usually root
sudo -u nobody whoami
su - alice -c whoami
```

Script guards

```
if [ "$(whoami)" != root ]; then
  echo "run as root" >&2
  exit 1
fi

# prefer numeric for reliability:
if [ "$(id -u)" -ne 0 ]; then
  echo "run as root" >&2
  exit 1
fi
```

Contrast with id / logname

```
whoami
id -un                       # equivalent effective username
id
logname                      # user logged on the tty (may differ)
```

After `sudo -s`, `whoami` is `root` while `logname` may still show the original login user.

Logging

```
echo "$(date -Is) user=$(whoami) action=deploy" >> deploy.log
```

Containers

```
whoami
id
# USER instruction in Dockerfile determines default
```

Notes / Pitfalls

- Reflects **effective** ids (setuid binaries can change this).
- Prefer `id -u` for root checks — numeric, locale-proof.
- Not a substitute for authentication/authorization in applications.
- May print `I have no name!` style issues only via related tools when passwd entry missing; `whoami` uses libc name lookup.

2026-relevant notes

- In Kubernetes/pods, check whether the container runs as non-root (`whoami` / `id`).
- Rootless Podman maps users — names inside may differ from host.
- Automation should key off uid numbers stored in configs, not display names alone.

Related Commands

- `id` — full identity
- `logname` — login name from utmp
- `who` / `w` — logged-in sessions
- `sudo` / `su` — change user
- `getent passwd` — account database

Additional Resources

- `man whoami`

id

Overview

`id` prints the real and effective user and group IDs for a process (by default, yours), plus supplementary groups. It is the authoritative quick identity tool for permissions debugging — better than `whoami` alone when groups and numeric IDs matter.

Syntax

```
id [options] [user]
```

Common Options

Option	Description
`-u`, `--user`	Print only user ID
`-g`, `--group`	Print only primary group ID
`-G`, `--groups`	Print all group IDs
`-n`, `--name`	Print names instead of numbers (with `-u`/`-g`/`-G`)
`-r`, `--real`	Real ID instead of effective
`-z`, `--zero`	NUL-terminate (with `-G` etc.)
`-A`	Security context extras on some systems
`-Z`	SELinux context (when available)

Examples with Explanations

Default

```
id
# uid=1000(alice) gid=1000(alice) groups=1000(alice),27(sudo),100(users)
```

Specific fields

```
id -u
id -un
id -g
id -gn
id -G
id -Gn
```

Other users

```
id www-data
id -un root
getent passwd www-data
```

Root check in scripts

```
if [ "$(id -u)" -ne 0 ]; then
  echo "need root" >&2
  exit 1
fi
```

Real vs effective (setuid context)

```
id -u
id -ru
id -u -r
# if they differ, effective privileges differ from real user
```

SELinux

```
id -Z
```

Format for logs

```
printf 'User: %s (UID: %d)\n' "$(id -un)" "$(id -u)"
echo "groups=$(id -Gn)"
```

Permission debugging pattern

```
id
namei -l /path/to/resource
ls -l /path/to/resource
getfacl /path/to/resource 2>/dev/null
```

Notes / Pitfalls

- Names come from NSS (`passwd`/`group`/sssd/ldap) — offline LDAP can make lookups hang.
- Prefer numeric IDs in scripts for comparisons; names can rename.
- Supplementary groups affect file access; primary gid alone is incomplete.
- Containers may show only a subset of host groups or remapped UIDs.
- `id user` looks up that account; default is the calling process credentials.

2026-relevant notes

- Systemd user sessions and linger affect available groups for some services — debug with `id` inside the unit's environment (`systemd-run --user`).
- Rootless containers: high UIDs on the host map to low UIDs inside.
- For policy ("must be in group docker"), check `id -nG | tr ' ' '\n' | grep -x docker`.

Related Commands

- `whoami` — effective username only
- `groups` — group names shorthand
- `getent` — account database lookup
- `newgrp` / `sg` — run with alternate group
- `sudo` / `su` — change identity
- `getfacl` — ACL permissions

Additional Resources

- `man id`

who

Overview

`who` shows who is logged in, with terminal, login time, and sometimes host/remote info from utmp/wtmp accounting. Related tools: `w` (richer: load + what they run), `users` (names only), `last` (historical logins).

Syntax

```
who [options] [file]
who am i
whoami                  # different command: effective user
```

Common Options

Option	Description
`-a`	All information (combo flags)
`-b`	Boot time
`-d`	Dead processes
`-H`	Header
`-l`	Login processes
`-m`	Same as `who am i` (hostname and user for stdin)
`-q`	Count and names only
`-r`	Runlevel (legacy)
`-s`	Short format
`-t`	Last system clock change
`-u`	Idle time / PID
`-T`	Message status of mesg (+/−)

Optional `file` defaults to utmp (often `/var/run/utmp` or `/run/utmp`).

Examples with Explanations

Who is on the system

```
who
who -H
who -u
who -q
```

Boot time

```
who -b
uptime -s
```

Yourself

```
who am i
who -m
logname
```

Compare with w

```
who
w
users
```

`w` adds load averages and the current command line of each session.

Historical

```
last
last -a
who /var/log/wtmp          # if supported / accessible
```

Scripting inventory

```
who | awk '{print $1}' | sort -u
```

Notes / Pitfalls

- utmp accuracy varies: display managers, wayland, lingering sessions, and containers may not appear as classic ttys.
- SSH multiplexing and tmux can make "one human, many lines".
- Empty output on some minimal containers is normal — no utmp traffic.
- Privacy: listing logins is normal for admins; still treat usernames carefully in logs.
- `whoami` ≈ `who am i`.

2026-relevant notes

- Prefer `loginctl` on systemd for modern session management views.
- Audit pipelines use `last`/`journalctl` more than live `who`.
- Remote access gateways may hide real client IPs behind proxies — check sshd logs.

Related Commands

- `w` — richer logged-in view
- `users` — names only
- `last` / `lastlog` — history
- `logname` — login name
- `loginctl` — systemd sessions
- `whoami` / `id` — identity of current process

Additional Resources

- `man who`

w

Overview

`w` shows who is logged in **and** what they are doing, plus system uptime and load averages. It combines aspects of `uptime`, `who`, and a glimpse of each session's current process — a classic first glance after SSH'ing into a busy server.

Syntax

```
w [options] [user]
```

Common Options

Option	Description
`-h`	No header
`-s`	Short format (less detail)
`-f`	Toggle from field (remote host)
`-i`	Show IP instead of hostname when possible
`-o`	Old-style output order
`-u`	Ignore username for current process / idle calc variants
`user`	Show only this user

Examples with Explanations

Default

```
w
# 10:22:01 up 5 days,  3:01,  4 users,  load average: 0.42, 0.35, 0.30
# USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
# alice    pts/0    203.0.113.10     09:01    0.00s  0.12s  0.01s w
```

Filter one user

```
w alice
```

Short / no header

```
w -h
w -s
```

IP addresses

```
w -i
```

Correlate with load tools

```
w
uptime
ps -u alice
top -bn1 | head
```

When you need history

```
w
last -a | head
journalctl _COMM=sshd -n 20
```

Understanding columns

Column	Meaning
USER	Login name
TTY	Terminal device
FROM	Remote host (if any)
LOGIN@	Login time
IDLE	Idle time on tty
JCPU	Time of all processes on tty
PCPU	Time of current process
WHAT	Current command line

Exact columns vary slightly by implementation (procps-ng).

Notes / Pitfalls

- `WHAT` is a snapshot — can be misleading for short-lived commands.
- Idle time does not mean CPU idle for the whole system.
- Containers and GUI sessions may not appear as classic pts entries.
- Do not rely on `w` alone for security auditing; use auth logs/journal.
- High load with few users often means services/cron, not interactive humans.

2026-relevant notes

- Still excellent for quick SSH triage; pair with `systemd-cgtop` / `btop` for service-heavy hosts.
- `loginctl list-sessions` gives systemd's session view.
- Cloud bastions: many `FROM` IPs may be the bastion, not the end user.

Related Commands

- `who` — simpler login list
- `uptime` — load + uptime only
- `ps` / `top` / `htop` — process detail
- `last` — login history
- `loginctl` — systemd sessions
- `finger` — legacy user info (if installed)

Additional Resources

- `man w`

last

Overview

`last` shows a history of logins, reboots, and runlevel changes from the `wtmp` accounting file (typically `/var/log/wtmp`). Use it after security incidents, to see when a machine rebooted, or who logged in from where. Companion: `lastb` (bad logins from `btmp`), `lastlog` (per-user last login).

Syntax

```
last [options] [username...] [tty...]
```

Common Options

Option	Description
`-a`	Display hostname in last column
`-d`	Translate IP to hostname
`-F`	Full timestamps
`-n N`, `-N`	Show N lines
`-R`	Suppress hostname field
`-i`	Show IP as numbers
`-x`	Include shutdown/runlevel entries
`-w`	Widen fields for long names
`-f file`	Alternate wtmp file
`-s` / `-t`	Since / until times (when supported)

Examples with Explanations

Recent logins

```
last
last -n 20
last -a
last -i
```

```
last -F
```

One user

```
last alice
last root
```

Reboots and shutdowns

```
last reboot
last -x | head
who -b
uptime -s
```

Bad logins

```
sudo lastb | head
sudo lastb -a
```

Requires permission to read `btmp`.

Rotate awareness

```
last -f /var/log/wtmp.1
ls /var/log/wtmp*
```

After logrotate, older history lives in rotated files.

SSH audit starter

```
last -ai | head -50
journalctl -u ssh -n 50
journalctl _COMM=sshd -n 50
```

Notes / Pitfalls

- Missing/empty wtmp means no history (containers, stripped images, misconfigured logging).
- Hostnames from `-d` depend on DNS and can mislead; prefer `-i` for forensics.
- Entries can be incomplete if the machine crashed without clean logout (shows `crash` / `down` / still logged in).
- Not a full security log — pair with `journalctl`, auditd, cloud provider logs.
- Timezones: interpret with system TZ settings.

2026-relevant notes

- Many fleets centralize auth logs; local `last` is still valuable on a single host incident.
- systemd / logind sessions: also check `loginctl` and journal.
- Immutable hosts may reset wtmp on reboot depending on design — know your image.

Related Commands

- `lastb` — failed logins
- `lastlog` — per-user last login table
- `who` / `w` — currently logged in
- `journalctl` — structured logs
- `utmpdump` — raw utmp/wtmp decode
- `aureport` — audit framework reports (if auditd)

Additional Resources

- `man last`, `man wtmp`

env

Overview

`env` runs a program in a modified environment, or prints the current environment variables when given no command. It is essential for shebangs (`#!/usr/bin/env bash`), clearing the environment (`-i`), and injecting variables for a single command without exporting them permanently in the shell.

Syntax

```
env [options] [NAME=VALUE]... [command [args...]]
env                          # print environment
```

Common Options

Option	Description
`-i`, `--ignore-environment`	Start with an empty environment
`-u NAME`, `--unset=NAME`	Remove variable
`-0`, `--null`	NUL-delimit output
`-C dir`, `--chdir=dir`	Change directory before exec (newer coreutils)
`-S string`	Split string on shebang lines (see man)
`-v`	Verbose
`--`	End of options

Examples with Explanations

Print environment

```
env
env | sort
env | grep -E '^(PATH|HOME|USER)='
printenv PATH                # single var alternative
```

Run with extra variables

```
env FOO=bar BAZ=1 ./script.sh
env NODE_ENV=production node app.js
```

Equivalent to shell `FOO=bar ./script.sh` for simple cases; `env` shines with cleared environments and shebangs.

Clear environment

```
env -i PATH=/usr/bin:/bin HOME="$HOME" bash --noprofile --norc
env -i $(cat allowed.env | xargs) command
```

Useful for clean builds and debugging "works in my shell" issues.

Unset for one command

```
env -u http_proxy -u https_proxy curl -I https://example.com
```

Shebang portability

```
#!/usr/bin/env bash
#!/usr/bin/env python3
```

`env` locates the interpreter via `PATH`, avoiding hard-coded `/usr/bin/python3` when layouts differ. (Security-sensitive contexts may prefer absolute paths.)

Chdir then run (when supported)

```
env -C /var/www -i PATH=/usr/bin:/bin ./deploy.sh
```

NUL-safe dump

```
env -0 | tr '\0' '\n' | sort
```

Notes / Pitfalls

- `env -i` is easy to over-strip — many programs need `PATH`, `HOME`, `TERM`, `LANG`.
- Shebang line length limits exist; complex `env -S` usage has edge cases.
- Printing env may leak secrets (`AWS_*`, tokens) — avoid pasting full dumps.
- Order of assignment and `-u` matters; see man page for processing order.
- Shell builtins are not found by `env` — it execs real programs from PATH.

2026-relevant notes

- Container entrypoints often use `env` or exec forms to inject config.
- Prefer explicit `.env` loading tools or systemd `Environment=` for services.
- Security: `LD_PRELOAD` and similar can be injected via env — hardened systems scrub them for setuid.

Related Commands

- `printenv` — print variables
- `export` / `set` — shell environment
- `envdir` / `dotenv` — external env loaders
- `systemd-run` / unit `Environment=` — service env
- `sudo -E` / `sudo --preserve-env` — elevated env policy

Additional Resources

- `man env`

free

Overview

`free` summarizes system memory usage: physical RAM and swap. Output comes from `/proc/meminfo`. Modern `free` includes an **available** estimate (memory reclaimable without swapping) that is more useful than the old "free only" column for deciding if a host is under memory pressure.

Syntax

```
free [options]
```

Common Options

Option	Description
`-h`, `--human`	Human-readable units
`-b` / `-k` / `-m` / `-g`	Bytes / KiB / MiB / GiB
`-t`	Total line
`-s N`	Refresh every N seconds
`-c N`	Repeat N times (with `-s`)
`-w`	Wide output (separate buffers/cache on older styles)
`-v`	Version
`--si`	Powers of 1000 instead of 1024

Examples with Explanations

Everyday

```
free -h
free -m
free -h -t
```

Watch live

```
free -h -s 2
watch -n1 free -h
```

Scripts

```
free -b | awk '/^Mem:/ {print $2, $7}'   # total, available (column positions can shift - ve
# safer parse of meminfo:
awk '/MemAvailable:/ {print $2}' /proc/meminfo
```

Interpret with pressure tools

```
free -h
cat /proc/pressure/memory 2>/dev/null    # PSI on modern kernels (full chapter: [Pressure St
vmstat 1 5
swapon --show
```

Containers / cgroups

```
free -h
# may show host memory unless constrained views; check cgroup limits:
cat /sys/fs/cgroup/memory.max 2>/dev/null
cat /sys/fs/cgroup/memory.current 2>/dev/null
```

Understanding the columns

Typical modern output:

```
               total        used        free      shared  buff/cache   available
Mem:            ...
Swap:           ...
```

Field	Meaning
total	Installed / visible memory
used	In use (implementation-defined accounting)
free	Completely unused
shared	tmpfs / shared memory estimate
buff/cache	Kernel buffers and page cache
available	Estimate of memory available for new workloads

Field	Meaning

Linux uses free RAM for cache — low "free" with high "available" is often healthy.

Notes / Pitfalls

- Do not panic solely because `free` is low; watch **available**, reclaim, PSI, and OOM logs.
- Swap usage can be normal for infrequently touched pages; heavy swap I/O is the problem.
- Column layout changed over the years — verify before parsing.
- Huge pages, zswap, and compressed RAM affect interpretation.
- VMs: balloon drivers and host overcommit complicate the picture.

2026-relevant notes

- Prefer **PSI** (`/proc/pressure`) and metrics backends for production memory SLOs.
- `systemd-cgtop` and cgroup v2 memory stats matter more than host `free` inside dense multi-service hosts.
- OOM: `journalctl -k | grep -i oom` / `dmesg` for killer events.

Related Commands

- `vmstat` — memory + runqueue + io
- `top` / `htop` / `btop` — per-process memory
- `swapon` / `swapoff` — swap devices
- `cat /proc/meminfo` — raw counters
- `smem` — proportional set size (if installed)
- `slabtop` — kernel slab usage

Additional Resources

- `man free`, `man proc`

lscpu

Overview

`lscpu` displays CPU architecture information from sysfs and `/proc/cpuinfo`: sockets, cores, threads, model name, caches, virtualization flags, NUMA layout, and vulnerabilities mitigations summary. Use it before tuning parallelism, interpreting load averages, or choosing binary architectures.

Syntax

```
lscpu [options]
```

Common Options

Option	Description
`-a` / `-b` / `-c`	Online/offline CPU filtering variants
`-e`	Extended parsable table of CPUs
`-p`	Parseable (key,value or CSV-like)
`-J`	JSON output (newer util-linux)
`-x`	Hex masks
`-y`	Physical instead of logical in some fields
`-s`	Sysroot for inspection
`-B`	Byte sizes raw

Examples with Explanations

Overview

```
lscpu
```

Parseable / JSON

```
lscpu -p
lscpu -J | jq .
lscpu -e=cpu,node,socket,core,online
```

Core counts for scripts

```
nproc
lscpu | awk -F: '/^CPU\(s\):/ {gsub(/ /,"",$2); print $2}'
# prefer:
nproc --all
getconf _NPROCESSORS_ONLN
```

Virtualization and flags

```
lscpu | grep -E 'Virtualization|Hypervisor|Flags|Model name|Thread|Core|Socket'
grep -m1 flags /proc/cpuinfo
```

Look for `vmx` (Intel) / `svm` (AMD) for hardware virtualization; `ht` for hyperthreading.

NUMA

```
lscpu | grep -i numa
numactl -H 2>/dev/null
ls /sys/devices/system/node
```

Security mitigations summary

```
lscpu | sed -n '/Vulnerabilities:/,$p'
# detailed:
grep . /sys/devices/system/cpu/vulnerabilities/*
```

Containers

```
lscpu
nproc
# CPU affinity may be constrained by cpuset cgroup
cat /sys/fs/cgroup/cpuset.cpus.effective 2>/dev/null
```

Notes / Pitfalls

- `CPU(s)` counts logical CPUs (hardware threads), not physical cores.
- Hypervisors can present fake topology; verify when licensing by socket/core.
- Offline CPUs and hotplug change counts — re-run after changes.
- Flags strings are long; pipe to `grep` for specific features (`avx2`, `aes`, …).
- Don't use load average without knowing logical CPU count.

2026-relevant notes

- ARM (`aarch64`) and RISC-V hosts are common; scripts must not assume `x86_64`.
- JSON output helps inventory agents.
- Mitigations still affect performance; compare with workload benchmarks, not only flag presence.

Related Commands

- `nproc` — online processor count
- `lshw -class processor` — alternate inventory
- `cat /proc/cpuinfo` — per-cpu raw
- `numactl` — NUMA policy
- `taskset` — CPU affinity
- `tuned` / power profiles — CPU governors

Additional Resources

- `man lscpu`

lsmem

Overview

The `lsmem` command displays information about memory ranges and their online/offline status. It's particularly useful for systems with memory hotplug capabilities and NUMA architectures.

Syntax

```
lsmem [options]
```

Common Options

Option	Description
`-a`	List all memory ranges
`-b`	Show output in bytes
`-h`	Human-readable sizes
`-o columns`	Specify output columns
`-r`	Raw output format
`-S`	Split by node
`-s`	Show summary only
`--sysroot=dir`	Use alternative sysfs root

Output Columns

Column	Description
RANGE	Memory address range
SIZE	Size of memory range
STATE	Online/Offline status
REMOVABLE	Whether memory can be removed
BLOCK	Memory block number
NODE	NUMA node number
ZONES	Memory zones

Key Use Cases

1. Memory inventory
2. NUMA topology analysis
3. Memory hotplug management
4. System capacity planning
5. Memory troubleshooting

Examples with Explanations

Example 1: Basic Memory Information

```
lsmem
```

Shows memory ranges and their status

Example 2: Human-Readable Sizes

```
lsmem -h
```

Displays memory sizes in human-readable format (KB, MB, GB)

Example 3: Summary Only

```
lsmem -s
```

Shows only summary information

Understanding Memory States

State	Description
online	Memory is available for use
offline	Memory is not available
going-offline	Memory is being taken offline
going-online	Memory is being brought online

Memory Block Management

Memory is managed in blocks: - Block size typically 128MB or 256MB - Blocks can be individually onlined/offlined - Useful for memory hotplug operations

Common Usage Patterns

1. Check total memory:

```
lsmem -s | grep "Total online memory"
```

2. List offline memory:

```
lsmem | grep offline
```

3. Show NUMA distribution:

```
lsmem -S
```

NUMA Memory Information

For NUMA systems: - Shows memory distribution across nodes - Helps with memory locality optimization - Useful for performance tuning

Memory Hotplug Operations

1. Check removable memory:

```
lsmem | grep "yes" | grep "REMOVABLE"
```

2. Find offline blocks:

```
lsmem -a | awk '$3=="offline" {print $4}'
```

Performance Analysis

Memory information useful for: - Memory bandwidth optimization - NUMA-aware application tuning - Memory pressure analysis - Capacity planning

Related Commands

- `free` - Memory usage statistics
- `cat /proc/meminfo` - Detailed memory information
- `numactl --hardware` - NUMA topology
- `dmidecode -t memory` - Physical memory information
- `lshw -C memory` - Hardware memory details

Additional Resources

- lsmem Manual
- Memory Management Guide

Best Practices

1. Monitor memory hotplug status
2. Understand NUMA topology
3. Check removable memory before maintenance
4. Use with other memory analysis tools
5. Consider memory block alignment

Scripting Examples

1. Count online memory blocks:

   ```
   lsmem | grep -c "online"
   ```

2. Get total memory size:

   ```
   lsmem -s | grep "Total online memory" | awk '{print $4}'
   ```

3. Check NUMA nodes:

   ```
   lsmem -o NODE | grep -v NODE | sort -u
   ```

Memory Zones

Common memory zones: - **DMA**: Direct Memory Access zone - **DMA32**: 32-bit DMA zone - **Normal**: Regular memory zone - **HighMem**: High memory zone (32-bit systems) - **Movable**: Memory that can be migrated

System Integration

1. Memory monitoring:

   ```
   watch -n 5 'lsmem -s'
   ```

2. NUMA optimization:

```
lsmem -S | grep "node 0"
```

3. Capacity reporting:

```
echo "Total Memory: $(lsmem -s | grep 'Total online' | awk '{print $4}')"
```

Troubleshooting

1. Memory not showing up
2. Offline memory blocks
3. NUMA node misalignment
4. Memory hotplug failures
5. Inconsistent memory reporting

Advanced Usage

1. Custom column output:

```
lsmem -o RANGE,SIZE,STATE,NODE
```

2. Raw format for parsing:

```
lsmem -r
```

3. Alternative sysfs root:

```
lsmem --sysroot=/alternative/path
```

Memory Block Operations

To manage memory blocks (requires root):

```
# Online a memory block
echo online > /sys/devices/system/memory/memory64/state

# Offline a memory block
echo offline > /sys/devices/system/memory/memory64/state
```

Integration Examples

1. With NUMA tools:

   ```
   lsmem -S && numactl --hardware
   ```

2. Memory pressure monitoring:

   ```
   lsmem -s && free -h
   ```

3. System inventory:

   ```
   echo "Memory Layout:" && lsmem -h
   ```

Output Formatting

1. Specific columns:

   ```
   lsmem -o SIZE,STATE | column -t
   ```

2. Summary with details:

   ```
   lsmem -s && echo "---" && lsmem
   ```

3. Node-specific information:

   ```
   lsmem | awk '$6==0 {print}'  # Node 0 only
   ```

lsusb

Overview

`lsusb` lists USB devices connected to the system (buses, IDs, vendor/product strings). Essential for debugging docks, keys, serial adapters, and whether a device is even enumerated before loading drivers.

Package: often `usbutils`.

Syntax

```
lsusb [options]
```

Common Options

Option	Description
`-v`, `--verbose`	Detailed descriptors
`-vv`	Even more verbose
`-t`	Tree view
`-s [[bus]:][devnum]`	Show only specific device
`-d vendor:product`	Filter by ID
`-D device`	Query device node under `/dev/bus/usb`
`-k`	Kernel driver binding info (when supported)

Examples with Explanations

List devices

```
lsusb
lsusb -t
```

Filter

```
lsusb -d 0781:5567
lsusb -s 001:005
```

Verbose probe

```
sudo lsusb -v | less
sudo lsusb -v -d 1d6b:0002 | less
```

Correlate with kernel messages

```
lsusb
dmesg -w
# plug device, watch enumeration
journalctl -k -f
```

Find device nodes

```
ls -l /dev/bus/usb/*/*
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

Tree + drivers

```
lsusb -t
# shows hubs and nesting; helpful for power/hub issues
```

Notes / Pitfalls

- Permission: some verbose details need root.
- Unauthorized USB policy (USBGuard) may block devices that still appear partially.
- VMs need USB passthrough configured; host `lsusb` guest.
- IDs are hex vendor:product from the USB-IF database; strings can be generic.
- Unstable enumeration often means cable/power/hub issues, not software.

2026-relevant notes

- USB-C docks create deep hub trees — use `-t`.
- `usbguard`, `udev` rules, and secure boot policies affect usability more than `lsusb` itself.
- For serial adapters, also check `dmesg` for `ttyUSB`/`ttyACM` assignment.

Related Commands

- `lspci` — PCI devices
- `lshw` — hardware inventory
- `dmesg` / `journalctl -k` — kernel enumeration logs
- `udevadm info` — device metadata
- `usb-devices` — alternate text dump
- `modprobe` — load drivers

Additional Resources

- `man lsusb`

lsmod

Overview

`lsmod` lists loaded **kernel modules** (name, size, use count, dependents). It is a thin, readable view of `/proc/modules`. Use it when debugging drivers, missing hardware support, or before `modprobe`/`rmmod` operations.

Syntax

```
lsmod
# no significant options - see modprobe/rmmod for management
```

Examples with Explanations

List modules

```
lsmod
lsmod | sort
lsmod | head
```

Search for a driver

```
lsmod | grep -i usb
lsmod | grep -i nvidia
lsmod | grep -E '^(nfs|overlay|br_netfilter)'
```

Dependency / use count

```
lsmod | awk 'NR==1 || $3>0 {print}'
# third column: used by count; Used by column shows dependents
```

Correlate with hardware

```
lsusb
lspci -k
lsmod | grep -i xhci
```

`lspci -k` shows which kernel driver is bound to a PCI device.

Module details

```
modinfo e1000e
modprobe -n -v e1000e          # dry-run
cat /proc/modules | grep e1000e
```

Unload safety check

```
lsmod | grep module_name
sudo modprobe -r module_name   # preferred over raw rmmod
# fails if in use - check dependents in lsmod
```

Blacklist context

```
lsmod | grep nouveau
# if you blacklisted a module, it should not appear after reboot
grep -r nouveau /etc/modprobe.d/ 2>/dev/null
```

Understanding columns

Column	Meaning
Module	Module name
Size	Memory size in bytes
Used by	Reference count and dependent module names

A non-zero use count means unload will fail until dependents release it.

Notes / Pitfalls

- Built-in kernel features are **not** modules — absence from `lsmod` doesn't mean "unsupported".
- Out-of-tree modules (ZFS, some NVIDIA) still appear when loaded.
- Containers usually cannot load modules; operations need host privileges.
- Secure Boot may restrict unsigned modules.
- Don't randomly `rmmod` storage/network drivers on remote hosts.

2026-relevant notes

- Initramfs and UKI images embed critical modules — `lsmod` after boot is not the whole story.
- Prefer `modprobe` over `insmod` for dependency resolution.
- For policy, use `/etc/modules-load.d/` and `/etc/modprobe.d/` rather than ad-hoc loads.

Related Commands

- `modprobe` / `rmmod` / `insmod` — load/unload
- `modinfo` — module metadata
- `lspci -k` / `lsusb` — hardware binding
- `dmesg` — load errors
- `depmod` — rebuild module deps

Additional Resources

- `man lsmod`, `man 5 modules`

hwinfo

Overview

The `hwinfo` command provides comprehensive hardware information. It's a powerful tool that probes for hardware and displays detailed information about various hardware components.

Syntax

```
hwinfo [options]
```

Common Options

Option	Description
`--short`	Brief hardware list
`--arch`	Show architecture info
`--bios`	Show BIOS info
`--block`	Show block devices
`--cpu`	Show CPU info
`--disk`	Show disk devices
`--memory`	Show memory info
`--network`	Show network devices
`--pci`	Show PCI devices
`--usb`	Show USB devices
`--all`	Show all hardware
`--save-config`	Save hardware config
`--log file`	Write log to file

Key Use Cases

1. Hardware inventory
2. System diagnostics
3. Driver verification
4. System documentation
5. Troubleshooting

Examples with Explanations

Example 1: Brief Summary

```
hwinfo --short
```

Show brief hardware list

Example 2: CPU Info

```
hwinfo --cpu
```

Show detailed CPU information

Example 3: Storage Info

```
hwinfo --disk --short
```

Show brief disk information

Understanding Output

Categories of information: - System overview - CPU details - Memory configuration - Storage devices - Network interfaces - Peripheral devices

Common Usage Patterns

1. Full system scan:

   ```
   hwinfo --all
   ```

2. Network check:

   ```
   hwinfo --network
   ```

3. Save hardware info:

   ```
   hwinfo --all --log hardware.log
   ```

Performance Analysis

- Comprehensive scanning
- Resource intensive
- Database lookups
- Device probing time
- Log file generation

Related Commands

- `lshw` - List hardware
- `lspci` - List PCI devices
- `lsusb` - List USB devices
- `dmidecode` - DMI table info
- `inxi` - System information

Additional Resources

- Hardware Info Documentation
- Linux Hardware Guide
- System Information Tools

Hardware Categories

1. Processors
2. Memory modules
3. Storage devices
4. Network adapters
5. Peripheral devices

Best Practices

1. Regular hardware audits
2. Document configurations
3. Monitor changes
4. Keep logs
5. Update hardware database

Four Ways to Talk to the Kernel

Four Ways to Talk to the Kernel

Thesis

User space reaches the kernel through a few stable doors: **syscalls**, **/proc & /sys**, **netlink**, and **ioctl/eBPF**.

Mental model

```
syscall | proc/sys | netlink | ioctl/eBPF
```

Worked examples

```
python3 -c 'import os; print(os.getpid())'
cat /proc/sys/net/ipv4/tcp_fastopen
cat /sys/block/vda/queue/scheduler
```

`ip`/`ss` speak netlink; eBPF is for programmable hooks when the other doors are not enough.

Trap

Parsing `ps` instead of `/proc`. Columns change; `/proc/<pid>/status` is stabler.

Boring rule

Prefer syscalls and documented proc/sys. Escalate carefully.

Try this

1. `man 7 netlink`, `man 2 bpf`.
2. Compare `sysctl` vs `/proc/sys` writes on a machine you own.
3. Trace `ss` with strace where available.

Sources

- syscalls(2), proc(5), netlink(7), bpf(2)

CPU masks and lists: reading the kernel's bitmaps

Overview

Thesis: every "which CPUs" or "which NUMA nodes" answer the kernel gives you is a **bitmap**: an array of `unsigned long` where bit n means CPU n (or node n). The kernel prints that one structure in two text forms. One is a **list** (`0-1,3,8`). The other is a **mask** of comma-separated 32-bit hex chunks (`81,00000000,00000d0b`). Know the two forms and the arithmetic behind them, and `Cpus_allowed`, `taskset -p`, `smp_affinity`, `cpu/online` and `core_cpus` all read the same way.

Typical jobs this chapter covers:

- Turn `Cpus_allowed: 3a` into "CPUs 1, 3, 4, 5" in your head, and back again.
- Know why the same file says `ff` on one box and `00000000,000000ff` on another.
- Choose between the `*_list` file and the mask file in a script, and parse either one correctly.
- Read kernel code that uses `DECLARE_BITMAP`, `set_bit` and `for_each_set_bit` without guessing.

```
uname -r
gcc --version | head -1
nproc
id -u
```

```
6.12.94+
gcc (Debian 14.2.0-19) 14.2.0
8
1000
```

Every listing below ran on a Debian 13 host with kernel 6.12, x86_64, 8 vCPUs and one NUMA node, as an unprivileged user (`uid=1000`). Nothing needs root.

Mental model

```
bitmap of N bits  =  unsigned long word[BITS_TO_LONGS(N)]
bit nr lives in   word[nr / BITS_PER_LONG]   at   1UL << (nr % BITS_PER_LONG)

printed as a list :  ranges, ascending, comma separated        0-1,3,8,10-11
```

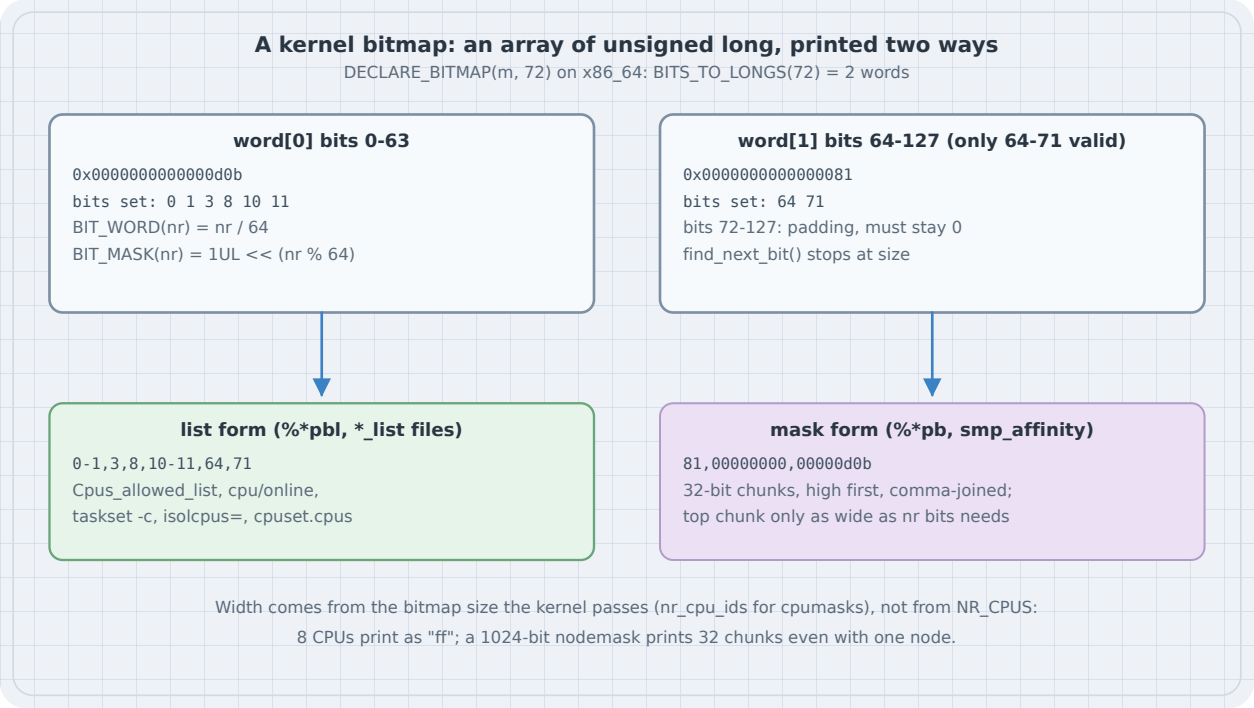


Figure 1: A kernel bitmap and its two text forms

```
printed as a mask :  hex, 32 bits per chunk, high chunk first  81,00000000,00000d0b
                     the highest chunk is only as wide as N needs
```

Three facts carry the chapter:

1. **Bit number = CPU number.** There is no other mapping. CPU 5 is bit 5, which is `0x20` in the lowest chunk.
2. **The width of a mask follows the bitmap's size, not the number of set bits.** For cpumasks that size is `nr_cpu_ids`, the number of possible CPU ids on this boot. It is not `NR_CPUS`, the compile-time maximum.
3. **Lists are for humans and for writing; masks are compact but width-dependent.** For scripts, prefer the `*_list` file when one exists.

Case 1 — The same bits, two files

The kernel shows the calling task's allowed CPUs and memory nodes in `/proc/self/status`, once as a mask and once as a list:

```
grep -E 'Cpus_allowed|Mems_allowed' /proc/self/status
```

```
Cpus_allowed:   ff
Cpus_allowed_list:  0-7
```

```
Mems_allowed:   00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000,0000
Mems_allowed_list:  0
```

`Cpus_allowed` is two hex digits because this boot has `nr_cpu_ids = 8`, so the bitmap is 8 bits wide. `Mems_allowed` is a nodemask sized by the kernel's configured maximum number of nodes, here 1024 bits, which is 32 chunks. Only the lowest bit is set: node 0. The `_list` lines say the same thing in one token each.

Narrow the affinity with `taskset` and both forms follow:

```
taskset -p $$
taskset -c 0,2 grep Cpus_allowed /proc/self/status
taskset -c 0-7:2 sh -c 'grep Cpus_allowed_list /proc/self/status'
```

```
pid 1241299's current affinity mask: ff
Cpus_allowed:   05
Cpus_allowed_list:  0,2
Cpus_allowed_list:  0,2,4,6
```

`05` is binary `0000 0101`: bits 0 and 2. The last line shows the stride syntax `first-last:step` that util-linux accepts in `taskset -c`. The kernel accepts a related list syntax for boot parameters such as `isolcpus=` and `nohz_full=`. Its form is `first-last:used/group`; for example, `0-7:2/4` selects CPUs 0, 1, 4 and 5. It is described under "cpu lists" in the kernel parameters documentation.

Case 2 — Sysfs: lists and masks side by side

The CPU topology directory has both forms for many attributes:

```
cd /sys/devices/system/cpu
cat possible present online
for f in cpu0/topology/core_cpus cpu0/topology/core_cpus_list \
         cpu0/topology/package_cpus_list kernel_max; do
  printf '%-34s %s\n' "$f" "$(cat $f)"
done
cat /sys/devices/system/node/node0/cpumap /sys/devices/system/node/node0/cpulist
cat /proc/irq/default_smp_affinity
cat /proc/irq/*/smp_affinity_list | sort | uniq -c
```

```
0-7
0-7
0-7
cpu0/topology/core_cpus            01
cpu0/topology/core_cpus_list       0
cpu0/topology/package_cpus_list    0-7
kernel_max                         63
```

```
ff
0-7
ff
     21 0-7
```

`kernel_max` is `NR_CPUS - 1`. This kernel was built for up to 64 CPUs, but the masks are two hex digits wide because the boot found 8 possible CPUs. Copy a mask from this box to a host with 96 possible CPUs and it has a different width and comma layout. Code that parses masks with a fixed width breaks there; code that reads the `_list` file does not.

`core_cpus` is `01` because each vCPU here is its own core, with no SMT sibling. On a bare-metal host with hyper-threading, `core_cpus_list` for CPU 0 typically reads something like `0,48`.

Case 3 — The arithmetic, in user space

The kernel's bitmap API lives in `include/linux/bitmap.h` and `include/linux/bitops.h`. The core idioms are short enough to copy into a user-space program. The point is to watch the words and the printed forms agree. Kernel code calls the real functions; it does not open-code them.

```c
/* bitmap.c: a user-space copy of the kernel bitmap idioms (include/linux/bitmap.h) */
#define _GNU_SOURCE
#include <limits.h>
#include <sched.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BITS_PER_LONG     (sizeof(long) * CHAR_BIT)
#define BITS_TO_LONGS(n)  (((n) + BITS_PER_LONG - 1) / BITS_PER_LONG)
#define DECLARE_BITMAP(name, bits) unsigned long name[BITS_TO_LONGS(bits)]
#define BIT_WORD(nr)      ((nr) / BITS_PER_LONG)
#define BIT_MASK(nr)      (1UL << ((nr) % BITS_PER_LONG))

static void set_bit(unsigned nr, unsigned long *a)   { a[BIT_WORD(nr)] |=  BIT_MASK(nr); }
static void clear_bit(unsigned nr, unsigned long *a) { a[BIT_WORD(nr)] &= ~BIT_MASK(nr); }
static int  test_bit(unsigned nr, const unsigned long *a) { return !!(a[BIT_WORD(nr)] & BIT_

static unsigned find_next_bit(const unsigned long *a, unsigned size, unsigned off)
{
    while (off < size) {
        unsigned long w = a[BIT_WORD(off)] & (~0UL << (off % BITS_PER_LONG));
        if (w) {
            unsigned r = BIT_WORD(off) * BITS_PER_LONG + __builtin_ctzl(w);
            return r < size ? r : size;
        }
        off = (BIT_WORD(off) + 1) * BITS_PER_LONG;
```

```
    }
    return size;
}
#define for_each_set_bit(b, a, size) \
    for ((b) = find_next_bit((a), (size), 0); (b) < (size); (b) = find_next_bit((a), (size),

static unsigned bitmap_weight(const unsigned long *a, unsigned size)
{
    unsigned w = 0;
    for (unsigned i = 0; i < BITS_TO_LONGS(size); i++)
        w += __builtin_popcountl(a[i]);
    return w;
}

/* "%*pbl" style: 0-3,8,10-11 */
static void print_list(const unsigned long *a, unsigned size)
{
    unsigned b, first = 1;
    for (b = find_next_bit(a, size, 0); b < size; ) {
        unsigned e = b;
        while (e + 1 < size && test_bit(e + 1, a)) e++;
        printf(first ? "%u" : ",%u", b);
        if (e > b) printf("-%u", e);
        first = 0;
        b = find_next_bit(a, size, e + 1);
    }
    printf("\n");
}

/* "%*pb" style: 32-bit hex words, most significant first, comma separated */
static void print_mask(const unsigned long *a, unsigned size)
{
    int words = (size + 31) / 32;
    int top = size % 32 ? (size % 32 + 3) / 4 : 8;   /* first chunk is only as wide as neede
    for (int i = words - 1; i >= 0; i--) {
        unsigned long v = a[(i * 32) / BITS_PER_LONG] >> ((i * 32) % BITS_PER_LONG);
        if (i == words - 1) printf("%0*lx", top, v & 0xffffffffUL);
        else                printf(",%08lx", v & 0xffffffffUL);
    }
    printf("\n");
}

int main(void)
{
    enum { NBITS = 72 };            /* deliberately not a multiple of 64 */
    DECLARE_BITMAP(m, NBITS) = {0};
```

```
    unsigned b;

    printf("BITS_PER_LONG=%zu  BITS_TO_LONGS(%d)=%zu  sizeof(m)=%zu bytes\n",
           BITS_PER_LONG, NBITS, BITS_TO_LONGS(NBITS), sizeof(m));

    for (b = 0; b < 4; b++) set_bit(b, m);
    set_bit(8, m); set_bit(10, m); set_bit(11, m);
    set_bit(64, m); set_bit(71, m);
    clear_bit(2, m);

    printf("weight=%u  test_bit(3)=%d test_bit(2)=%d\n", bitmap_weight(m, NBITS),
           test_bit(3, m), test_bit(2, m));
    printf("for_each_set_bit:");
    for_each_set_bit(b, m, NBITS) printf(" %u", b);
    printf("\nlist: "); print_list(m, NBITS);
    printf("mask: "); print_mask(m, NBITS);
    printf("word[0]=%#018lx word[1]=%#018lx\n", m[0], m[1]);

    /* The same structure the scheduler hands back */
    cpu_set_t cs;
    CPU_ZERO(&cs);
    if (sched_getaffinity(0, sizeof(cs), &cs) != 0) { perror("sched_getaffinity"); return 1;
    printf("cpu_set_t: %zu bytes (%zu bits), CPU_COUNT=%d\n",
           sizeof(cs), sizeof(cs) * CHAR_BIT, CPU_COUNT(&cs));
    printf("affinity list: "); print_list((unsigned long *)&cs, 8);
    printf("affinity mask: "); print_mask((unsigned long *)&cs, 8);
    return 0;
}
```

```
gcc -O2 -Wall -Wextra -o bitmap bitmap.c && ./bitmap
echo ---
taskset -c 1,3-5 ./bitmap | tail -2
```

```
BITS_PER_LONG=64  BITS_TO_LONGS(72)=2  sizeof(m)=16 bytes
weight=8  test_bit(3)=1 test_bit(2)=0
for_each_set_bit: 0 1 3 8 10 11 64 71
list: 0-1,3,8,10-11,64,71
mask: 81,00000000,00000d0b
word[0]=0x0000000000000d0b word[1]=0x0000000000000081
cpu_set_t: 128 bytes (1024 bits), CPU_COUNT=8
affinity list: 0-7
affinity mask: ff
---
affinity list: 1,3-5
affinity mask: 3a
```

Read the output against the diagram:

- **72 bits need two longs.** `BITS_TO_LONGS` rounds up, so bits 72 to 127 of word 1 are padding. The kernel's helpers mask the last word with `BITMAP_LAST_WORD_MASK(nbits)`, and every search takes a `size`, so padding bits never show up as CPUs.
- `word[0] = 0xd0b` is bits 0, 1, 3, 8, 10, 11 (`1101 0000 1011`). Bit 2 was set and then cleared.
- **The mask form splits the bitmap into 32-bit chunks, high chunk first.** With 72 bits the top chunk holds 8 bits, so it prints as two digits (`81`). That matches the kernel's `%*pb` output and `Cpus_allowed`.
- **`for_each_set_bit` is a `find_next_bit` loop.** The search is word-at-a-time with count-trailing-zeros, so walking a sparse 4096-CPU mask costs about 64 word loads, not 4096 bit tests.
- **glibc's `cpu_set_t` is a 1024-bit bitmap** laid out the same way. That is why the cast to `unsigned long *` works on Linux. Portable code should use the `CPU_*` macros instead. `taskset -c 1,3-5` gives mask `3a` = `0011 1010` = CPUs 1, 3, 4, 5.

Case 4 — Converting by hand and in a script

Mask to list, one chunk at a time (here `3a`):

```
  hex  3    a
  bin  0011 1010
  bit  7654 3210   → bits 1,3,4,5  → list 1,3-5
```

For more than 32 CPUs, strip the commas, read the whole string as one hex number, and test bits from 0 upward. In bash, arithmetic is 64-bit, so a mask over 64 bits must be handled chunk by chunk. Usually the simplest correct script reads the list file instead:

```
# list → individual CPU ids, no width assumptions
expand_list() {
  local IFS=, part
  for part in $1; do
    if [[ $part == *-* ]]; then seq "${part%-*}" "${part#*-}"; else echo "$part"; fi
  done
}
expand_list "$(cat /sys/devices/system/cpu/online)" | paste -sd' '
expand_list "1,3-5" | paste -sd' '
```

```
0 1 2 3 4 5 6 7
1 3 4 5
```

This helper handles the plain `a-b` and `a,b` forms that sysfs prints. It does not handle the `:used/group` stride syntax, which appears only on input, in boot parameters and `taskset -c`.

Where you will meet each form

You see	Form	Typical files and tools
`0-3,8`	list	`/sys/devices/system/cpu/{possible,prese` `*_list` topology files, `Cpus_allowed_list`, cgroup v2 `cpuset.cpus`, `taskset -c`, `isolcpus=`, `nohz_full=`
`ff`, `00000000,000000ff`	mask	`Cpus_allowed`, `Mems_allowed`, `taskset -p`, `/proc/irq/*/smp_affinity`, `/proc/irq/default_smp_affinity`, topology `core_cpus`, node `cpumap`
`[0, 1, 3]`	JSON array	`lscpu -J`, not a kernel format

When writing, IRQ affinity accepts both (`smp_affinity` takes a mask, `smp_affinity_list` takes a list). Writing either one needs root. The list form is harder to get wrong.

Gotchas

- **Width differs between hosts.** A mask is only meaningful together with its bitmap size. Never compare mask strings across machines; compare lists or sets of CPU ids.
- **Empty is still a valid answer.** `isolated` and `offline` print an empty line when no CPU is in the set. A parser must accept an empty string.
- **Possible is not online.** Bitmaps are sized by possible CPUs. Hot-pluggable or offline CPUs keep their bit positions; `online` tells you which are usable now.
- **`cpu_set_t` has a ceiling.** It is fixed at 1024 bits. On hosts with more possible CPUs, use `CPU_ALLOC` and `CPU_ALLOC_SIZE`, or `sched_getaffinity` fails with `EINVAL`, as `sched_setaffinity(2)` documents.
- **Atomic vs non-atomic in kernel code.** `set_bit` and `clear_bit` are atomic read-modify-write operations. `__set_bit` and `__clear_bit` are the non-atomic variants for bitmaps that a lock already protects. The user-space copy above is non-atomic.

Quick reference

Operation	Kernel API	Notes
Declare	`DECLARE_BITMAP(name, nbits)`	`unsigned long name[BITS_TO_LONGS(nbits)]`
Set / clear / test	`set_bit`, `clear_bit`, `test_bit`	atomic; `__set_bit` is the non-atomic form
Count	`bitmap_weight(src, nbits)`	popcount over words
Iterate	`for_each_set_bit(bit, addr, size)`	`find_next_bit` loop

Operation	Kernel API	Notes
Combine	`bitmap_and`, `bitmap_or`, `bitmap_andnot`	word-wise
Print	`%*pb` (mask), `%*pbl` (list)	width = number of bits
Parse	`bitmap_parselist`, `bitmap_parse`	list with `:used/group`, hex mask
CPU wrappers	`cpumask_*`, `for_each_cpu`, `cpumask_pr_args()`	sized by `nr_cpu_ids`

Sources

- Linux kernel documentation, "How to get printk format specifiers right", bitmap section (`%*pb`, `%*pbl`): https://docs.kernel.org/core-api/printk-formats.html
- Linux kernel documentation, "Basic C Library Functions": bitmap operations (`bitmap_parselist`, `bitmap_weight`, …): https://docs.kernel.org/core-api/kernel-api.html
- Linux kernel documentation, kernel parameters, "cpu lists" syntax: https://docs.kernel.org/admin-guide/kernel-parameters.html
- Linux kernel ABI, `/sys/devices/system/cpu/` (possible, present, online, topology): https://docs.kernel.org/admin-guide/abi-stable.html and https://docs.kernel.org/admin-guide/cputopology.html
- Kernel source, `include/linux/bitmap.h`, `include/linux/find.h`, `lib/bitmap-str.c`: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/linux/bitmap.h
- `proc_pid_status(5)` (`Cpus_allowed`, `Mems_allowed`): https://man7.org/linux/man-pages/man5/proc_pid_status.5.html
- `sched_setaffinity(2)` and `CPU_SET(3)`: https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html, https://man7.org/linux/man-pages/man3/CPU_SET.3.html
- `taskset(1)`, util-linux: https://man7.org/linux/man-pages/man1/taskset.1.html

Processes and Jobs

Intro

Intro

List, prioritize, signal, and background work. Covers classic job control in the shell as well as system-wide process tools (`ps`, `top` family, `pgrep`/`pkill`).

Commands in this part

Command	Role
`ps`	ps snapshots processes.
`pstree`	pstree shows running processes as a tree, making parent/child relationships obvious — ideal for seeing what a…
`top`	top is the classic interactive process monitor: CPU, memory, and a live-sorted process table.
`htop`	htop is an interactive process viewer with a clearer UI than classic top: mouse support, tree view, easy…
`procs`	procs is a modern replacement for ps written in Rust, with colorized output, tree views, keyword highlighting, and…
`pgrep`	pgrep lists process IDs matching selection criteria (name, full command line, user, terminal, etc.).
`pidof`	pidof prints the PIDs of running programs matched by name.
`kill`	kill sends a signal to one or more processes by PID.
`killall`	killall sends a signal to all processes matching a command name.
`jobs`	jobs lists active jobs of the current shell: running background tasks, stopped jobs, and their job IDs.
`fg`	fg resumes a job in the foreground, giving it the terminal again (stdin/stdout/signals).

Command	Role
`bg`	bg resumes a stopped job in the background so it continues running without occupying the terminal.
`nohup`	nohup runs a command immune to SIGHUP, so it keeps running after you log out of a terminal session.
`timeout`	timeout runs a command and sends a signal if it still runs after a duration.
`watch`	watch runs a command repeatedly, showing fullscreen output so you can see changes over time.
`sleep`	sleep suspends execution for a specified time.
`nice`	nice starts a command with a modified CPU niceness — a soft priority bias for the scheduler.
`renice`	The renice command alters the scheduling priority of running processes.
`ionice`	ionice sets or queries the I/O scheduling class and priority of a process (Linux CFQ/BFQ-oriented interface;…
`taskset`	taskset reads and sets CPU affinity: which CPUs a process, or every one of its threads, may run on.
`fuser`	fuser lists PIDs using a file, mount point, or network port.
`ulimit`	ulimit is a shell builtin (bash/zsh/etc.) that shows or sets resource limits for the current shell and its child…
`ipcs`	ipcs lists System V IPC objects: message queues, shared memory segments, and semaphore arrays.
`ipcrm`	ipcrm removes System V IPC objects: shared memory segments, semaphore arrays, and message queues.
`flock`	flock wraps a command or script section in a kernel file lock so two copies never run at once (cron overlap, deploy/backup mutual exclusion).
`choom`	choom shows and sets a process's OOM-killer bias (`oom_score_adj`); the chapter explains how the kernel scores and picks an OOM victim, globally and per memory cgroup.

Command	Role
`pidfd`	pidfds are file descriptors that name one process and survive PID reuse: signal, wait, poll for exit (even of non-children), copy fds and identify socket peers without racing a recycled PID.
`signalfd` / `timerfd` / `eventfd`	Signals, timers and cross-thread wake-ups as file descriptors in one `epoll` loop: blocking first, read-to-clear, coalesced counts, and the blocked mask that leaks into children.

Suggested starting points

1. Inventory: `ps`, `pstree`, then interactive `top`/`htop`/`btop`.
2. Select by name: `pgrep` / `pidof` before `kill`/`killall`.
3. Job control in one shell: `jobs`, `fg`, `bg`, `nohup`.
4. Priority, I/O class, and CPU placement: `nice`, `renice`, `ionice`, `taskset`.
5. Limits and IPC: `ulimit`, `ipcs`/`ipcrm`, `fuser`.
6. Mutual exclusion for jobs and scripts: `flock`.
7. Out-of-memory victims: `choom -p PID`, `/proc/PID/oom_score`, and cgroup `memory.events`.
8. Race-free process handles: pidfds (`pidfd_open`, `pidfd_send_signal`, `waitid(P_PIDFD)`) instead of saved PIDs.
9. One event loop for signals, timers and wake-ups: `signalfd`, `timerfd`, `eventfd` with `epoll`.

Related parts

- System monitoring — deeper CPU/I/O tracing
- Services and runtime — prefer `systemctl` for daemons
- Shell commands — `time` while iterating

Continue with the individual command pages in this part.

ps

Overview

`ps` snapshots processes. Linux `ps` accepts both **BSD-style** (`ps aux`) and **UNIX-style** (`ps -ef`) option sets — mixing styles works but confuses readers; pick one family per invocation. For a live view use `top`/`htop`; for PID selection by name prefer `pgrep`.

Syntax

```
ps [options]
```

Common Invocations

Form	Description
`ps aux`	BSD: all users, CPU/MEM columns
`ps -ef`	UNIX: full listing
`ps -u USER`	One user's processes
`ps -p PID`	Specific PID(s)
`ps -C name`	By command name
`ps --forest` / `f`	Tree view
`ps -eo …`	Custom columns

Useful selectors: `-o pid,ppid,user,pcpu,pmem,stat,etime,cmd`, `--sort=-%mem`.

Key Use Cases

1. Identify heavy CPU/memory consumers
2. Find PIDs before signaling
3. Inspect command lines and parentage
4. Script process inventory / monitoring checks

Examples with Explanations

Classic full lists

```
ps aux | head
ps -ef | head
```

`aux` is the muscle-memory form on Linux; `-ef` is common in SysV docs.

Top memory and CPU

```
ps aux --sort=-%mem | head -n 15
ps aux --sort=-%cpu | head -n 15
```

Snapshot only — a process can spike between samples.

Custom columns

```
ps -eo pid,ppid,user,%cpu,%mem,stat,etime,cmd --sort=-%cpu | head
```

`etime` is elapsed runtime; handy for spotting stuck long jobs.

By user / name

```
ps -u www-data -o pid,cmd
ps -C nginx -o pid,cmd
pgrep -a nginx          # often clearer for name → PID
```

Tree view

```
ps auxf | less
ps -ejH | less
pstree -ap | less
```

Shows parent/child relationships (workers under a master, shells under sshd).

One PID in detail

```
ps -p 1 -o pid,user,cmd,lstart,etime
tr '\0' ' ' < /proc/1/cmdline; echo
```

`/proc/PID` is the source of truth when `ps` columns are not enough.

Threads

```
ps -eLf | head
ps -p "$(pgrep -n nginx)" -L -o pid,tid,pcpu,cmd
```

`-L` / `lwp` shows threads; high thread counts can matter for ulimits.

Full command line (no truncation)

```
ps -eo pid,args | less
ps auxww | less          # wide; ww avoids early wrap on BSD style
```

Default width may truncate `CMD`; widen the terminal or use `args`/`ww`.

Zombies and D-state

```
ps aux | awk '$8 ~ /Z/ {print}'     # zombies (STAT has Z)
ps aux | awk '$8 ~ /D/ {print}'     # uninterruptible sleep (often I/O)
```

Zombies need parent reaping; `D` state often means storage/NFS pain.

Script: fail if process missing

```
ps -C mydaemon >/dev/null || { echo "mydaemon not running" >&2; exit 1; }
pgrep -x mydaemon >/dev/null
```

Understanding Output

- **STAT**: `R` running, `S` interruptible sleep, `D` uninterruptible, `Z` zombie, `T` stopped; extras like `+` (foreground), `l` (multi-threaded), `<` (high priority).
- **%CPU**: not identical to `top`'s averaged view.

- **VSZ vs RSS**: virtual size vs resident RAM.
- **USER**: effective user; real UID can differ for setuid binaries.

Notes & Pitfalls

- `ps` is a **point in time**; loops need `watch` or a real monitor.
- Prefer `pgrep`/`pkill` over `ps \| grep` (avoids matching the grep itself).
- Kernel threads appear as `[kthreadd]`-style names.
- Container views may only show processes in the same PID namespace.

Related Commands

- `top` / `htop` / `btop` — interactive
- `pgrep` / `pkill` — select/signal by pattern
- `pstree` — hierarchy
- `pidof` — simple name lookup
- `/proc/PID` — kernel process API

Additional Resources

- `man ps`
- `man proc`

pstree

Overview

`pstree` shows running processes as a **tree**, making parent/child relationships obvious — ideal for seeing what a supervisor spawned, which shell owns a pipeline, or how threads hang under a process.

Syntax

```
pstree [options] [pid|user]
```

Common Options

Option	Description
`-a`	Show command-line args
`-p`	Show PIDs
`-c`	Don't compact identical subtrees
`-h` / `-H pid`	Highlight current / specified
`-n`	Sort by PID
`-u`	Show uid transitions
`-g`	Show PGID
`-s`	Show parents of specified PID
`-l`	Long lines (no truncation)
`-T`	Hide threads
`-A` / `-U`	ASCII / UTF-8 lines
`-Z`	SELinux context
`-t`	Show full thread names when available

Examples with Explanations

Basics

```
pstree
pstree -p
pstree -ap
```

One user or PID

```
pstree alice
pstree -p 1
pstree -ap $(pidof -s nginx)
```

Show ancestry of a process

```
pstree -s -p 12345
ps -o pid,ppid,cmd -p 12345
```

Avoid compaction

```
pstree -c -p
```

Identical child names are otherwise merged with counts.

Threads vs processes

```
pstree -p 12345
pstree -T -p 12345
```

SELinux

```
pstree -Z
```

Compare with ps

```
ps auxf
pstree -ap
systemd-cgls
```

`ps f` / `ps --forest` is another forest view; `systemd-cgls` shows cgroup trees.

Notes / Pitfalls

- Large trees on busy hosts are noisy — filter by user/PID.
- Compact mode can hide multiplicity — use `-c` when counting workers.
- Permission: some args may be invisible for other users' processes.
- PID namespaces in containers show a nested world (PID 1 is container init).
- Not a real-time monitor — re-run or use `watch -n1 pstree -p`.

2026-relevant notes

- systemd-heavy systems: also learn `systemctl status` and `systemd-cgtop`.
- For containers: `podman top` / `docker top` + host `pstree -p` with care.
- Kernel threads appear under `kthreadd` (PID 2) on the host.

Related Commands

- `ps auxf` — forest view
- `pgrep` / `pidof` — find PIDs
- `systemd-cgls` — cgroup tree
- `top` / `htop` — live views with tree modes
- `pstree` package often from `psmisc`

Additional Resources

- `man pstree`

top

Overview

`top` is the classic interactive process monitor: CPU, memory, and a live-sorted process table. It is installed almost everywhere, so it is the default triage tool when `htop`/`btop` are not present. For scripts and snapshots, prefer `ps`; for friendlier interactive use, install `htop`.

Syntax

```
top [options]
```

Common Options

Option	Description
`-d SECS`	Delay between refreshes (fractional OK)
`-n COUNT`	Exit after COUNT iterations (batch-friendly)
`-b`	Batch mode (non-interactive; good for logs)
`-p PID`, `-p PID1,PID2`	Watch only these PIDs
`-u USER` / `-U USER`	Filter by effective / any user id
`-H`	Show threads
`-o FIELD`	Sort by field (e.g. `%CPU`, `%MEM`)
`-c`	Toggle command line vs name (often available as interactive `c`)
`-w`	Wide output (batch)

Interactive keys (procps-ng)

Key	Action
`h` / `?`	Help
`q`	Quit
`Space` / `Enter`	Refresh now
`d` / `s`	Change delay
`P`	Sort by CPU

Key	Action
`M`	Sort by memory
`T`	Sort by time
`N`	Sort by PID
`k`	Kill (prompt for PID and signal — **TERM default**)
`r`	Renice
`u`	Filter user
`c`	Command line vs basename
`V`	Forest / tree view (when supported)
`H`	Toggle threads
`1`	Expand / collapse per-CPU lines
`f` / `F`	Field management
`W`	Write config (`~/.config/procps/toprc` or `~/.toprc`)

Key Use Cases

1. Live "who is burning CPU/RAM right now"
2. Quick renice or kill during an incident
3. Capture a few samples in batch for a ticket
4. Per-CPU and load-average glance during overload

Examples with Explanations

Example: interactive default

```
top
```

Header shows uptime, load average, task counts, CPU breakdown, and memory. Process list is typically sorted by `%CPU`.

Example: refresh every two seconds

```
top -d 2
```

Slower refresh reduces overhead on already-loaded systems.

Example: single-user focus

```
top -u www-data
```

Limits noise on multi-tenant or app servers.

Example: watch specific PIDs

```
top -p 1,1234,$(pgrep -d, -x nginx)
```

Useful when you already know the suspects from `ps`/`pgrep`.

Example: batch sample for logs / tickets

```
top -b -n 3 -d 1 > /tmp/top-snapshot.txt
top -b -n 1 -o %MEM | head -n 30
```

`-b` makes output non-interactive and redirectable. Great for attaching evidence to an incident note.

Example: sort by memory from the CLI

```
top -o %MEM
```

Start already sorted by resident memory pressure.

Example: threads view

```
top -H -p "$(pgrep -d, -x java)"
```

Each thread appears as a row — helpful for multi-threaded runtimes.

Example: kill from inside top

Press `k`, enter PID, accept default signal **15 (TERM)** unless you have already tried graceful stop. Escalate to 9 only if needed.

Example: one-shot CPU leaders without staying in top

```
ps aux --sort=-%cpu | head -n 15
```

When you only need a snapshot, `ps` is lighter and script-friendlier.

Understanding Output

Header

- **load average** — 1/5/15 minute runnable+uninterruptible averages (not "percent CPU")
- **Cpu(s)** — `%us` user, `%sy` system, `%id` idle, `%wa` I/O wait, `%st` steal (VMs)
- **MiB Mem / Swap** — total, free, used, buff/cache (wording varies by procps version)

Process columns (typical)

Column	Meaning
`PID`	Process ID
`USER`	Effective user
`PR` / `NI`	Priority / nice
`VIRT` / `RES` / `SHR`	Virtual / resident / shared memory
`S`	State (`R` run, `S` sleep, `D` disk sleep, `Z` zombie, `T` stopped)
`%CPU` / `%MEM`	Share of CPU / physical memory
`TIME+`	Cumulative CPU time
`COMMAND`	Name or full command

On multi-core hosts, `%CPU` can exceed 100% for multi-threaded processes.

Notes & Pitfalls

- `top` is **live**; do not parse interactive mode in scripts — use `-b` or `ps`.
- High `%wa` points to storage/I/O, not always "need more CPU".
- High `%st` (steal) means the hypervisor is scheduling other VMs — look outside the guest.
- Memory: `buff/cache` is usually reclaimable; "free" alone is a poor metric — check available if shown.
- Killing from `top` still needs the same discipline as `kill`: TERM before KILL.
- Config saved with `W` can surprise you later if sort/fields change.

Related Commands

- `htop` / `btop` — modern interactive monitors
- `ps` — snapshots and scripting
- `uptime` / `vmstat` / `mpstat` — load and CPU breakdown
- `pidstat` — per-process rates over time
- `free` — memory summary

Additional Resources

- `man top`
- `man procps`

htop

Overview

`htop` is an interactive process viewer with a clearer UI than classic `top`: mouse support, tree view, easy search/filter, color meters, and safer multi-select kill. It is **not** always installed by default on Ubuntu servers — install with `apt`. For automation, keep using `ps`/`pgrep`; `htop` is for humans at a terminal.

Syntax

```
htop [options]
```

Common Options

Option	Description
`-d DELAY`	Update delay in **tenths** of a second (`-d 10` 1s)
`-u USER`	Show only this user
`-p PID[,PID…]`	Restrict to these PIDs
`-t`, `--tree`	Start in tree view
`-C`, `--no-color`	Monochrome
`-s COLUMN`	Sort by column name
`-H DEPTH`	Hide threads beyond depth (version-dependent)

Install on Ubuntu:

```
sudo apt update
sudo apt install htop
```

Useful keys

Key	Action
`F1` / `h`	Help

Key	Action
`F2` / `S`	Setup (meters, columns, display options)
`F3` / `/`	Search
`F4` / `\`	Filter (hide non-matches)
`F5` / `t`	Tree view
`F6` / `>`	Choose sort column
`F7` / `F8`	Decrease / increase nice
`F9` / `k`	Kill — pick signal from a list (**prefer TERM**)
`Space`	Tag process
`c`	Tag process **and** its children (with tree)
`U`	Untag all
`u`	User filter
`H`	Hide user threads
`K`	Hide kernel threads
`M` / `P` / `T`	Sort memory / CPU / time
`p`	Show full paths
`I`	Invert sort
`l`	List open files (`lsof`) if available
`s`	`strace` selected process (if permitted)
`q`	Quit

Key Use Cases

1. Interactive incident triage with filters and trees
2. Finding the parent of a worker storm
3. Niceness tweaks and careful multi-process kill
4. Per-user resource glance on shared hosts

Examples with Explanations

Example: start htop

```
htop
```

Meters at the top, process table below. Use mouse or keys to select rows.

Example: only your processes

```
htop -u "$USER"
```

Cuts system noise during personal debugging.

Example: tree view for process storms

```
htop -t
# or press F5 inside htop
```

Parent/child relationships show which master spawned runaway workers.

Example: attach to known PIDs

```
htop -p "$(pgrep -d, nginx)"
```

Focuses the display on nginx-related PIDs only.

Example: slower refresh on overloaded boxes

```
htop -d 20
```

Two-second delay (20 tenths) reduces sampling cost while you think.

Example: kill workflow inside htop

1. Filter with `F4` or search with `F3`
2. Tag targets with `Space` (optional)
3. `F9` / `k` → choose `SIGTERM` first
4. Wait; only then send `SIGKILL` to leftovers

Never default to SIGKILL out of habit.

Example: nice a batch job interactively

Select the process → `F8` a few times to make it friendlier (higher nice), or `F7` if you have privilege to make it more aggressive.

Example: setup persistence

`F2` → adjust CPU meter style, add columns (`IO_RATE`, `STARTTIME`, …) → leave setup; htop writes `~/.config/htop/htoprc` automatically on exit in modern versions.

Understanding Output

- **CPU bars** — per-core usage; colors distinguish user/system/irq/nice/io-wait/steal depending on version and setup.
- **Memory bar** — used vs buffers/cache; green/blue conventions differ from `free` wording but convey pressure.
- **Load average** — same kernel figures as `uptime`/`top`.
- **Process states** — same single-letter codes as `ps` (`R`, `S`, `D`, `Z`, `T`, …).

Notes & Pitfalls

- Not for scripts: output is interactive and not a stable API.
- Kill and renice still obey **permissions** — root for others' processes and for negative nice.
- Kernel threads clutter the view; toggle `K`/`H` for clarity.
- Container hosts: PIDs may be host-namespaced depending on where you run `htop`.
- Alternatives: `btop` (pretty), `atop` (historical accounting), classic `top` (always there).

Related Commands

- `top` — ubiquitous interactive monitor
- `btop` — modern TUI alternative
- `ps` / `pgrep` — snapshots and selection
- `vmstat` / `iostat` / `pidstat` — subsystem rates
- `nice` / `renice` / `ionice` — priority controls

Additional Resources

- `man htop`
- htop.dev

procs

Overview

`procs` is a modern replacement for `ps` written in Rust, with colorized output, tree views, keyword highlighting, and friendlier defaults. Optional install (`cargo install procs`, distro packages). Keep `ps` for portability and scripts.

Syntax

```
procs [options] [query...]
procs --help
```

Common Options

Option	Description
`-a`, `--and`	All keywords must match
`-o`, `--or`	Any keyword (default-ish behavior depends version)
`-d`, `--tree`	Tree view
`-w`, `--watch`	Watch mode refresh
`-p`, `--pager`	Use pager
`--sortd KEY` / `--sorta KEY`	Sort descending/ascending
`--only COL`	Show specific columns
`--insert COL`	Add columns
`--json`	JSON output
`--color`	Color mode
`query`	Filter by keyword (command, user, …)

Flags evolve quickly — check `procs --help` for your version.

Examples with Explanations

Everyday

```
procs
procs nginx
procs alice
procs --tree
procs -w
```

Sorting

```
procs --sortd cpu
procs --sortd mem
procs --sorta pid
```

Columns

```
procs --only pid,user,cmd
procs --insert tcp,udp
```

JSON for tooling

```
procs --json nginx | jq .
```

Compare with ps

```
ps aux | head
procs
ps -fp $(pgrep -d, nginx)
procs nginx
```

Watch like top (lightweight)

```
procs -w
# or
watch -n1 procs --sortd cpu
```

Notes / Pitfalls

- Not installed by default on servers — don't require it in production scripts.
- Keyword filtering is convenience, not a security boundary.
- Column names differ from `ps` — learn the map once.
- JSON schema may change across versions; pin version in automation.
- Permissions still limit visibility of other users' full command lines.

2026-relevant notes

- Part of the modern CLI set with `bat`, `eza`, `fd`, `rg`, `btop`.
- For portable ops docs, show `ps` equivalents alongside `procs`.
- Prefer `pgrep`/`systemctl` for scripting; use `procs` interactively.

Related Commands

- `ps` — portable process status
- `pgrep` / `pkill` — match / signal
- `top` / `htop` / `btop` — interactive monitors
- `pstree` — tree view
- `pidof` — PIDs by name

Additional Resources

- `procs --help`
- dalance/procs

pgrep

Overview

`pgrep` lists process IDs matching selection criteria (name, full command line, user, terminal, etc.). `pkill` uses the **same selectors** but sends a signal instead of printing PIDs. Together they replace brittle `ps | grep` pipelines for operations and scripts. On Ubuntu both come from **procps**.

Syntax

```
pgrep [options] pattern
pkill [options] pattern
```

Common Options

Shared selection flags (both tools):

Option	Description
`-u USER`, `--uid`	Match effective user (name or UID)
`-U USER`, `--euid`	Match real user (see man for distinction)
`-g PGID` / `-G GID`	Process group / group membership
`-t TERM`	Controlling terminal (e.g. `pts/0`)
`-P PPID`	Children of PPID
`-f`, `--full`	Match against full command line, not only comm
`-x`, `--exact`	Exact match of the name (or full line with `-f`)
`-n` / `-o`	Newest / oldest match only
`-v`	Inverse match
`-c`	Count matches (pgrep)
`-d DELIM`	PID delimiter (default newline; `,` useful for `htop -p`)
`-a`	Show PID and full command (pgrep)
`-l`	Show PID and process name (pgrep)
`-i`	Case-insensitive
`-r`	Ignore descendants of pgrep/pkill itself (where supported)

`pkill`-specific:

Option	Description
`-SIGNAL` / `-s SIGNAL`	Signal name or number (default **TERM**)
`-e`, `--echo`	Print what will be signaled (newer procps)
`-q`	Quiet

Safety

- Broad patterns (`pkill python`, `pkill java`) are classic outage sources — prefer **`-x`**, **`-f` with a specific string**, and **`-u`**.
- Escalate signals: **TERM → wait → KILL**; `pkill -9` should be rare.
- Managed services: prefer `systemctl stop/reload` over `pkill nginx`.
- Always inventory with `pgrep -a` before `pkill` on production.

Key Use Cases

1. Resolve PIDs for scripts without parsing `ps`
2. Signal a homogeneous worker pool
3. Count processes for simple health checks
4. Feed PID lists to `top`/`htop`/`gdb`

Examples with Explanations

Example: list matching PIDs

```
pgrep sshd
pgrep -a nginx
pgrep -l python3
```

`-a` shows full command lines — best pre-kill inspection.

Example: exact name match

```
pgrep -x sshd
pgrep -x nginx
```

Avoids partial matches against longer comm names when you want precision.

Example: full command line

```
pgrep -af 'gunicorn.*myapp'
pgrep -f 'python3 /opt/app/worker.py'
```

`-f` is essential when every process is `python3` or `java`.

Example: filter by user

```
pgrep -u www-data -a php-fpm
pgrep -u "$USER" -f train.py
```

Reduces collateral damage on shared systems.

Example: count for monitoring

```
pgrep -c -x sidekiq
pgrep -c -u redis redis-server
```

Exit status is still 0/1 for found/not found; combine carefully in scripts.

Example: newest / oldest only

```
pgrep -n -x myapp     # newest
pgrep -o -x myapp     # oldest (often the master)
```

Useful when a master/worker model embeds the same name.

Example: children of a parent

```
pgrep -P "$(pgrep -xo master-process)"
```

List direct children by parent PID.

Example: delimiter for other tools

```
htop -p "$(pgrep -d, nginx)"
top -p "$(pgrep -d, -x myapp)"
```

Comma-separated PID lists fit `htop -p` / `top -p`.

Example: pkill graceful then force

```
pkill -TERM -x myapp
sleep 3
pkill -KILL -x myapp 2>/dev/null
```

Same discipline as `kill`: TERM first, KILL only for stragglers.

Example: pkill with full-line match

```
pkill -u deploy -f 'uvicorn app.main:app'
```

Targets one service invocation pattern under one user.

Example: avoid ps | grep

```
# fragile:
ps aux | grep '[n]ginx'
# better:
pgrep -a nginx
```

Character-class `grep` tricks are obsolete for this job.

Understanding Output

- Default: one PID per line.
- Exit status **0** if any match, **1** if none, **2** on syntax/usage error, **3** fatal error (procps conventions).
- `pkill` exit status follows the same "matched or not" idea — check man page when scripting strict failure modes.

Notes & Pitfalls

- Pattern is an **extended regex** by default — dots and brackets matter; quote patterns.
- Without `-f`, matching uses the process name (comm), often truncated to 15 characters historically — use `-f` for long names.
- Race: process can exit between `pgrep` and `kill`; handle missing PIDs.
- `pkill`/`pgrep` may see processes in other sessions; still restricted by permissions for signaling.
- In containers, you only see PIDs in your PID namespace.

Related Commands

- `pkill` — signal instead of list (same selectors)
- `kill` / `killall` — by PID or simple name
- `pidof` — simpler name → PID (fewer filters)
- `ps` — rich table and custom columns
- `systemctl` — prefer for supervised units

Additional Resources

- `man pgrep`
- `man pkill`

pidof

Overview

`pidof` prints the PIDs of running programs matched by name. It is a quick alternative to `pgrep -x` for simple lookups and remains common in init scripts and one-liners. For flexible pattern matching, prefer `pgrep`.

Syntax

```
pidof [options] program [program...]
```

Common Options

Option	Description
`-s`	Single PID only (first/one)
`-c`	Only processes with same root (chroot)
`-x`	Also match scripts (by script name)
`-o omitpid`	Omit PID(s); often `-o %PPID` or self
`-z`	Skip zombies (when supported)

Options vary slightly between implementations (sysvinit-utils vs others).

Examples with Explanations

Basics

```
pidof nginx
pidof sshd
pidof bash
```

Single PID

```
pidof -s nginx
kill -HUP "$(pidof -s nginx)"
```

Scripts

```
pidof -x backup.sh
pgrep -f backup.sh          # often clearer for scripts
```

Existence gate

```
if pidof dockerd >/dev/null; then
  echo docker running
fi
```

Omit PIDs

```
pidof -o $$ bash            # other bashes, not this script's shell if named bash
pidof -o %PPID nginx
```

Feed to kill

```
kill $(pidof myapp)
# safer with pgrep:
pgrep -x myapp | xargs -r kill
```

Compare tools

```
pidof nginx
pgrep -x nginx
pgrep -a nginx
systemctl status nginx
```

Notes / Pitfalls

- Name matching is not always exact across threads/workers — verify with `ps`.
- Multiple PIDs print space-separated — quote carefully in scripts.
- Race: process may exit between `pidof` and `kill`.
- Containers: only sees processes in the same PID namespace.
- Prefer `systemctl` for supervised services instead of raw PID kills.

2026-relevant notes

- systemd units: `systemctl show -p MainPID nginx`.
- `pgrep`/`pkill` from procps are more expressive for modern scripts.
- Rootless stacks may have multiple same-named processes per user — be specific.

Related Commands

- `pgrep` / `pkill` — pattern match / signal
- `kill` / `killall` — signal by PID / name
- `ps` — detailed process list
- `systemctl` — service PIDs
- `pidwait` / `pwait` — wait for PID exit (if installed)

Additional Resources

- `man pidof`

kill

Overview

`kill` sends a signal to one or more processes by **PID**. Despite the name, the default signal is `TERM` **(15)** — a polite request to exit. Only escalate to `KILL` **(9)** after a graceful stop fails. Prefer `systemctl stop` / `systemctl reload` for units managed by systemd.

Syntax

```
kill [options] PID...
kill -SIGNAL PID...
kill -s SIGNAL PID...
kill -l [signal]
```

Common Options

Option	Description
`-s SIGNAL`, `-SIGNAL`	Signal name or number (`TERM`, `HUP`, `9`, …)
`-l`	List signal names (or translate a number name)
`-n SIGNAL`	Same idea as `-s` on some implementations
`-p` (bash builtin only)	Print PID instead of signaling (shell-specific)

Signals you will use constantly:

Signal	Number	Typical meaning
`TERM`	15	Graceful stop (**default**)
`HUP`	1	Hangup / often “reload config”
`INT`	2	Interrupt (like Ctrl-C)
`QUIT`	3	Quit with core (if allowed)
`KILL`	9	Force kill — **uncatchable**
`STOP`	19	Pause (uncatchable)
`CONT`	18	Resume after STOP
`USR1` / `USR2`	10 / 12	Application-defined

Safety

1. **Always verify the PID** before signaling: `ps -p PID -o pid,user,cmd` or `ps -fp PID`.
2. **TERM first, then wait, then KILL** — never jump to `-9` on production workers unless the process is wedged.
3. Wrong PID on a multi-tenant host can take down databases, SSH, or the wrong customer job.
4. You may only signal processes you own unless you are root.
5. Prefer **service managers** for supervised daemons: `systemctl stop UNIT`, not ad-hoc `kill` on random PIDs from `ps`.

Key Use Cases

1. Stop a runaway user process after verifying the PID
2. Reload a daemon that documents `SIGHUP` handling
3. Force-kill a process stuck in userspace after TERM fails
4. Test whether a PID still exists (`kill -0`)

Examples with Explanations

Example: graceful stop (default TERM)

```
ps -fp 1234
kill 1234
# same as:
kill -TERM 1234
kill -15 1234
kill -s TERM 1234
```

Default signal is TERM. Confirm the process is the one you intend with `ps` first.

Example: TERM, wait, then KILL

```
PID=1234
kill -TERM "$PID"
for i in 1 2 3 4 5; do
  kill -0 "$PID" 2>/dev/null || break
  sleep 1
done
if kill -0 "$PID" 2>/dev/null; then
  echo "still alive; forcing" >&2
  kill -KILL "$PID"
fi
```

`kill -0 PID` checks existence and permission **without** delivering a real signal. Use this pattern instead of blind `kill -9`.

Example: force kill only when necessary

```
kill -KILL 1234
# or
kill -9 1234
```

KILL cannot be caught or cleaned up. File locks, temp files, and child reaping may be messy. Prefer TERM whenever the process still responds.

Example: reload via HUP

```
kill -HUP "$(pidof nginx)"
# better for packaged services:
sudo systemctl reload nginx
```

Many classic daemons reload config on HUP. Prefer `systemctl reload` so systemd tracks state and uses the unit's configured reload command.

Example: signal by name via pgrep

```
kill $(pgrep -x myapp)
# often clearer:
pkill -x myapp
pkill -TERM -x myapp
```

Shell word-splitting on `$(pgrep …)` is fine for PIDs; still prefer exact match (`-x`) so you do not hit similarly named processes.

Example: stop / continue (debug freeze)

```
kill -STOP 1234    # freeze
# inspect, attach debugger, etc.
kill -CONT 1234    # resume
```

STOP/CONT are useful for debugging or temporarily freezing a CPU hog. A STOP'd process will not exit until CONT (or KILL).

Example: list signals

```
kill -l
kill -l 15          # → TERM (often)
kill -l TERM        # → 15
```

Handy when docs show only numbers or only names.

Example: multiple PIDs

```
kill -TERM 1001 1002 1003
kill -TERM $(pgrep -u "$USER" -f 'worker.py')
```

One invocation can target many PIDs. Combine with careful `pgrep` filters (`-u`, `-f`, `-x`).

Example: permission check without killing

```
if kill -0 1234 2>/dev/null; then
  echo "process exists and is signalable"
else
  echo "gone or not permitted"
fi
```

Exit status of `kill -0` is ideal for scripts that poll for death after TERM.

Understanding Output

- Success is silent; failures print to stderr (`No such process`, `Operation not permitted`).
- Exit status non-zero if **any** PID failed (implementation details vary slightly for multi-PID).
- Bash has a `kill` **builtin** that may differ slightly from `/bin/kill` (util-linux); both understand standard signals.

Notes & Pitfalls

- **Zombies** (`Z` state) ignore signals; kill the **parent** or fix whatever is not `wait()`ing.
- Processes in uninterruptible sleep (`D` state, often I/O) may not die until the wait ends — KILL does not magically fix stuck NFS or bad block devices.
- Killing PID **1** is a terrible idea; on systemd hosts it will refuse nonsense, but do not experiment.
- Job-control shells also understand `%1` job specs with the **builtin** `kill` — different from numeric PIDs.
- Containers: killing PID 1 inside a container usually stops the container; know your runtime's restart policy.

Related Commands

- `pkill` / `pgrep` — select and signal by name/pattern
- `killall` — signal by process name (GNU vs BSD differences)
- `systemctl` — stop/reload supervised services
- `timeout` — auto-signal a command after a duration
- `ps` / `pidof` — identify targets
- `fuser` — signal processes holding files or ports

Additional Resources

- `man 1 kill`
- `man 7 signal`

killall

Overview

`killall` sends a signal to all processes matching a **command name**. On Ubuntu it is the **psmisc** implementation (name match), not the ancient SysV "kill everything" binary. Prefer `pkill` when you need regex/full-command-line filters; use `killall` for simple exact-name mass signals. Prefer `systemctl` for managed services.

Syntax

```
killall [options] name...
```

Common Options

Option	Description
`-s SIGNAL`, `-SIGNAL`	Signal to send (default `TERM`)
`-e`, `--exact`	Require exact name match (long names)
`-I`, `--ignore-case`	Case-insensitive name
`-i`, `--interactive`	Prompt before each kill
`-l`, `--list`	List known signal names
`-u USER`, `--user`	Only this user's processes
`-v`, `--verbose`	Report what was signaled
`-w`, `--wait`	Wait until processes die
`-r`, `--regexp`	Interpret name as extended regex (psmisc)
`-n`, `--ns PID`	Match processes in same namespaces as PID
`-Z`, `--context`	SELinux context match (when relevant)
`-q`, `--quiet`	Suppress "no process found" complaint

Safety

- Name matching can still hit **more instances** than you expect (every `python3`, every `java`).
- Prefer **`TERM` first**; only use `-9` after a wait.
- Always narrow with **`-u`** on shared hosts.
- Interactive confirm (`-i`) is cheap insurance on production bastions.

- Do not confuse with Solaris/AIX/older docs where `killall` meant something more aggressive.

Key Use Cases

1. Stop all instances of a user-space tool by binary name
2. Reload or terminate workers that are not under systemd
3. Interactive cleanup of leftover processes after a failed deploy
4. Wait until a named process set has exited before continuing a script

Examples with Explanations

Example: graceful stop by name

```
killall myapp
killall -TERM myapp
```

Sends TERM to every process whose comm/name matches `myapp`. Equivalent intent to `pkill -x myapp` in many cases.

Example: TERM then KILL

```
killall -TERM myapp
sleep 3
killall -KILL myapp 2>/dev/null
```

Give the app time to flush and exit; force only stragglers. Redirect stderr if absence after TERM is expected success.

Example: only your processes

```
killall -u "$USER" -TERM chrome
killall -u deploy -TERM 'node'
```

Critical on multi-user machines so you do not signal another user's same-named binary.

Example: interactive confirm

```
killall -i -TERM python3
```

Prompts per process — use when several interpreters are running and only some should die.

Example: verbose + wait

```
killall -v -w myapp
echo "all myapp processes gone"
```

`-w` blocks until matching processes exit (or cannot be killed). Useful in stop scripts before releasing a lock or port.

Example: HUP reload pattern

```
killall -HUP nginx
# preferred on Ubuntu for packaged nginx:
sudo systemctl reload nginx
```

HUP is conventional for reload; systemd still owns the proper lifecycle for units.

Example: case-insensitive / regex

```
killall -I MyApp
killall -r 'worker-[0-9]+'
```

`-r` uses extended regular expressions (psmisc). Test with `pgrep` first when unsure.

Example: list signals

```
killall -l
```

Same idea as `kill -l`.

Example: dry reconnaissance first

```
pgrep -a myapp
ps -C myapp -o pid,user,cmd
killall -i -TERM myapp
```

Always inventory before mass signaling when the blast radius is unclear.

Notes & Pitfalls

- **GNU/psmisc vs BSD**: FreeBSD `killall` options differ — scripts should stick to portable flags or use `pkill`.
- Matching is by process **name** (comm), not necessarily full path; long names may need `-e`.
- `killall python` will not match `python3` unless you name it correctly (or use regex).
- Exit status is non-zero when **no** process matched (unless quieted) — check that in scripts.
- Kernel threads and some system processes cannot be killed; errors are expected.
- Prefer `systemctl stop foo` over `killall foo` for daemons shipped as units.

Related Commands

- `kill` — signal by PID
- `pkill` / `pgrep` — pattern-based select/signal
- `pidof` — simple name → PID list
- `systemctl` — supervised stop/reload
- `kill` / `pkill` — more controlled signaling on servers

Additional Resources

- `man killall` (psmisc)
- `man 7 signal`

jobs

Overview

`jobs` lists **active jobs** of the current shell: running background tasks, stopped jobs, and their job IDs. Essential for job control with `fg`, `bg`, `kill %n`, and `wait`.

Jobs are not a system-wide process table — use `ps`/`pgrep` for that.

Syntax

```
jobs [options] [job_spec ...]
```

Common Options (bash)

Option	Description
`-l`	Include PIDs
`-p`	PIDs only
`-n`	Only jobs with status changes since last notification
`-r`	Running only
`-s`	Stopped only
`-x command`	Replace job specs in command with PIDs and run

Job states

State	Meaning
Running	Executing in background
Stopped	Suspended (Ctrl-Z / SIGSTOP)
Done	Completed successfully
Exit N	Exited with status N
Terminated	Killed by signal

Examples with Explanations

List jobs

```
sleep 300 &
sleep 400 &
jobs
jobs -l
jobs -p
jobs -r
jobs -s
```

Act on a job

```
jobs -l
fg %1
bg %2
kill %1
kill -9 %2
```

Status changes

```
jobs -n
```

Useful after many completions to see what finished.

wait integration

```
longtask &
pid=$!
jobs -l
wait "$pid"
```

Enable job control in scripts (rare)

```
set -m
sleep 5 &
jobs -l
wait
```

disown removes from table

```
sleep 1000 &
jobs
disown
jobs          # empty regarding that job
# process may still run - check ps
```

Notes / Pitfalls

- Empty `jobs` does not mean no processes exist — only no **shell jobs**.
- Job IDs are not PIDs; use `jobs -l` or `$!` for PIDs.
- Subshells have their own job tables: `( sleep 10 & jobs )`.
- Non-interactive shells often disable job control until `set -m`.
- Don't use `jobs` for service supervision — use systemd.

2026-relevant notes

- Interactive terminal multiplexers reduced reliance on many concurrent jobs, but `%` job specs remain daily muscle memory.
- CI shells may not support job control the same way — use PIDs + `wait`.
- For parallel shell work, GNU `parallel` or `xargs -P` may be clearer than raw jobs.

Related Commands

- `fg` / `bg` — resume jobs
- `kill %n` — signal jobs
- `wait` — wait for completion
- `disown` — remove from job table
- `ps` / `pgrep` — system-wide process view
- `set -m` — monitor mode

Additional Resources

- `help jobs`, `man bash`

fg

Overview

`fg` resumes a job in the **foreground**, giving it the terminal again (stdin/stdout/signals). Use it to continue a stopped job interactively or to pull a background job back for interaction.

Syntax

```
fg [job_spec]
```

Default is the current job (`%+`).

Job specification

Spec	Meaning
`%n`	Job number n
`%name`	Command begins with name
`%?str`	Command contains str
`%%` / `%+`	Current
`%-`	Previous

Examples with Explanations

After Ctrl-Z

```
vim bigfile.txt
# Ctrl-Z
jobs
fg
# or
fg %vim
```

Classic suspend-and-resume editor workflow.

Pull background job forward

```
./server &
jobs -l
fg %1
# Ctrl-C now kills the server (foreground signals)
```

Switch between jobs

```
jobs
fg %1
# Ctrl-Z
fg %2
```

With pipelines

```
tar czf - dir | ssh host 'cat > backup.tgz' &
jobs
fg %1
```

Script note

```
set -m
sleep 5 &
fg %1
```

Usually unnecessary — scripts should `wait` on PIDs instead.

Notes / Pitfalls

- Only jobs of the current shell.
- Foreground job receives terminal-generated signals (`Ctrl-C` → SIGINT, `Ctrl-Z` → SIGTSTP).
- If job is already foreground, `fg` errors.
- Background jobs needing stdin will stop with SIGTTIN — `fg` to provide input.
- Job numbers change as jobs complete — verify with `jobs` first.

2026-relevant notes

- Interactive recovery still common; long-running prod work belongs in systemd/tmux.
- SSH drop kills foreground jobs; use multiplexers for remote interactive work.
- Know difference: `fg` (shell job) vs `docker attach` / `kubectl attach` (container).

Related Commands

- `bg` — resume in background
- `jobs` — list
- `kill %1` — signal a job
- `Ctrl-Z` — stop foreground
- `tmux` — better multi-tasking than many suspended jobs

Additional Resources

- `help fg`, `man bash`

bg

Overview

`bg` resumes a **stopped** job in the **background** so it continues running without occupying the terminal. It is part of interactive shell **job control** (bash/zsh), together with `fg`, `jobs`, `Ctrl-Z`, and `&`.

Job control is per-shell: background jobs are not the same as systemd services or `nohup` daemons.

Syntax

```
bg [job_spec ...]
```

With no argument, resumes the current job (`%+`).

Job specification

Spec	Meaning
`%n`	Job number n
`%string`	Job whose command begins with string
`%?string`	Job whose command contains string
`%%` or `%+`	Current job
`%-`	Previous job

Common Options

None significant — `bg` is a shell builtin. See `help bg`.

Examples with Explanations

Suspend then background

```
# start a long command in foreground
find / -xdev -name '*.log' 2>/dev/null
# press Ctrl-Z  → Stopped
bg
# continues in background
jobs -l
```

Start in background directly

```
make -j$(nproc) &
jobs
```

`&` starts background immediately; `bg` is for jobs already stopped.

Specific job

```
jobs
bg %1
bg %find
```

Multiple jobs

```
bg %1 %2
```

Typical workflow

```
gzip -k huge.img
# Ctrl-Z
bg
# continue using the shell
jobs -l
fg %1          # bring back if needed
```

Disown / survive hangup

```
# after bg:
disown %1
# or start with:
nohup longcmd &
# or use tmux/systemd for real persistence
```

Not for scripts by default

```
# non-interactive shells often have job control off
set -m              # monitor mode (enable job control) if needed
sleep 30 &
jobs
```

Prefer explicit background with `&` and `wait` in scripts rather than `bg`.

Notes / Pitfalls

- Only works with **job control enabled** and jobs owned by this shell.
- Background jobs writing to the terminal can interleave messily — redirect stdout/stderr.
- `bg` on an already running background job is a no-op / error depending on shell.
- Closing the terminal may SIGHUP jobs unless `disown`/`nohup`/tmux/systemd.
- Don't confuse with `systemd bg` — there is none; use units for services.

2026-relevant notes

- For anything important, prefer `tmux`/`systemd-run`/`nohup` over bare `bg`.
- Desktop environments may use their own session management; servers still rely on classic job control daily.
- Pipeline job control can be surprising — know whether the whole pipeline is one job.

Related Commands

- `fg` — resume in foreground
- `jobs` — list jobs
- `Ctrl-Z` / `kill -STOP` — stop
- `disown` — detach from shell job table
- `nohup` — ignore hangup
- `tmux` / `screen` — persistent sessions
- `wait` — wait for background PIDs/jobs

Additional Resources

- `help bg`, `man bash` (JOB CONTROL)

nohup

Overview

`nohup` runs a command immune to **`SIGHUP`**, so it keeps running after you log out of a terminal session. It is a minimal "survive hangup" wrapper — not a full service manager. For interactive long-lived SSH work prefer **`tmux`**/**`screen`**; for production daemons prefer **`systemd`** (user or system units).

Syntax

```
nohup COMMAND [ARGS]...
nohup COMMAND [ARGS]... &
```

Common Options

Option	Description
`--help`	Usage
`--version`	Version

`nohup` itself has almost no flags; behavior is about signals, redirection, and the shell background operator `&`.

Behavior (what you actually care about)

- Ignores `SIGHUP` for the started command.
- If **stdout is a terminal**, output is appended to `./nohup.out` (or `$HOME/nohup.out` if the cwd is not writable).
- If you already redirect stdout/stderr, `nohup` does not create `nohup.out`.
- Exit status is that of `COMMAND`, or 127 if the command cannot be invoked.

Key Use Cases

1. Kick off a one-off long job before disconnecting SSH (when you did not start tmux)
2. Quick background batch with a simple log file
3. Legacy scripts that expect `nohup` semantics

Examples with Explanations

Example: basic background job

```
nohup ./long-job.sh &
```

Starts the job, ignores hangup, backgrounds it. Output typically lands in `nohup.out` in the current directory.

Example: explicit log file

```
nohup ./long-job.sh > /var/tmp/long-job.log 2>&1 &
echo $!
```

Always redirect in real use so logs do not scatter into surprise `nohup.out` files. `$!` is the background PID.

Example: find and stop later

```
nohup python3 train.py --epochs 100 > train.log 2>&1 &
echo $! > train.pid

# later:
kill -TERM "$(cat train.pid)"
# wait, then kill -KILL if needed
```

Record the PID; do not rely on grepping vaguely for `python3`.

Example: disown alternative (bash)

```
./long-job.sh > job.log 2>&1 &
disown
```

`disown` removes the job from the shell's job table so logout does not send HUP to it. Different mechanism than `nohup`, similar operator goal.

Example: combine with nice / ionice

```
nohup nice -n 19 ionice -c3 ./batch-compress.sh > batch.log 2>&1 &
```

Low CPU and idle I/O class — polite neighbor on shared hosts.

Example: remote one-liner over SSH

```
ssh host 'nohup /opt/app/rebuild.sh > /tmp/rebuild.log 2>&1 &'
```

SSH can still race with job startup; for reliability prefer `tmux new -d` or a systemd unit.

Example: check nohup.out

```
nohup seq 1 1000000 &
tail -f nohup.out
```

Demonstrates default output capture when no redirection is given.

Safety

- `nohup` does **not** restart on crash, reboot, or OOM kill — not a supervisor.
- Writing `nohup.out` into a shared or production cwd can fill disks or leak sensitive prints.
- Background jobs may still receive other signals; only HUP is specially handled by `nohup`.
- Root + nohup + destructive scripts is still destructive after you disconnect.

Notes & Pitfalls

- Prefer `tmux new -s name` for anything you might need to reattach to interactively.
- Prefer `systemd-run --user` or a real unit for recurring or critical jobs.
- Relative paths in the command resolve at start time; if you `cd` later in another shell, the job's cwd is unchanged.
- `nohup.out` is **append** mode — old runs accumulate; rotate or redirect explicitly.
- Some environments use `setsid` or `ssh -f` patterns instead; know which layer owns the process.

Related Commands

- `tmux` / `screen` — reattachable sessions
- `systemd-run` / `systemctl` — supervised execution
- `disown` — bash job-table detach
- `setsid` — new session
- `timeout` — bound runtime
- `nice` / `ionice` — resource politeness

Additional Resources

- `man nohup`
- `man systemd-run`

timeout

Overview

`timeout` runs a command and sends a signal if it still runs after a duration. Use it to bound hung network tools, flaky tests, or "should never take this long" maintenance scripts. Default kill signal is `TERM`; you can escalate to `KILL` after a grace period.

Syntax

```
timeout [options] DURATION COMMAND [ARG]...
```

Duration format

Form	Meaning
`10`	10 seconds
`10s`	10 seconds
`5m`	5 minutes
`2h`	2 hours
`1d`	1 day
`0` / unset special	See man page — `0` disables the timeout on GNU coreutils

Floating point seconds are accepted (e.g. `0.5`).

Common Options

Option	Description
`-s SIGNAL`, `--signal=SIGNAL`	Signal on timeout (default `TERM`)
`-k DURATION`, `--kill-after=DURATION`	After first signal, wait then send `KILL`
`-f`, `--foreground`	Do not run command in a separate group (foreground quirks)
`-p`, `--preserve-status`	Exit with the command's status even on timeout

Option	Description
`-v`, `--verbose`	Diagnose signal sends
`--`	End of options (when command looks like a flag)

Exit status (GNU coreutils)

Status	Meaning
`124`	Command timed out (default)
`125`	`timeout` itself failed
`126`	Command found but not invocable
`127`	Command not found
`137`	Often command killed by SIGKILL (128+9) depending on path
other	Exit status of `COMMAND` if it finished in time

With `--preserve-status`, a timed-out command may return the status as if it received the signal (e.g. 128+signal) rather than 124 — check `man timeout` for your coreutils version.

Safety

- Bound commands that talk to **production** carefully — abrupt TERM can leave partial migrations; prefer app-level deadlines when available.
- Always consider `-k` so wedged processes that ignore TERM do not hang your automation forever.
- `timeout` is not a substitute for proper service health checks and systemd `TimeoutStopSec=`.

Key Use Cases

1. Cap flaky CI steps and network probes
2. Prevent hung `fsck`, `scp`, or package hooks from blocking unattended jobs
3. Force-fail long unit tests
4. Demo/debug with short lifetimes

Examples with Explanations

Example: basic time limit

```
timeout 10s ping -c 1000 1.1.1.1
```

Stops ping after 10 seconds even if count is not finished.

Example: TERM then KILL grace period

```
timeout -k 5s 30s ./migrate.sh
```

At 30s send **TERM**; if still alive 5s later, send **KILL**. Default operator pattern for stubborn processes.

Example: choose the first signal

```
timeout -s INT 20s ./app
timeout --signal=TERM -k 10s 2m ./app
```

INT mimics Ctrl-C; TERM is the usual graceful stop.

Example: capture timeout in scripts

```
if timeout 1m ./backup.sh; then
  echo "backup ok"
else
  rc=$?
  if [ "$rc" -eq 124 ]; then
    echo "backup timed out" >&2
  else
    echo "backup failed rc=$rc" >&2
  fi
  exit "$rc"
fi
```

Treat 124 as a distinct failure mode in monitoring and retries.

Example: verbose diagnosis

```
timeout -v -k 2s 5s sleep 60
```

Shows which signals were sent when.

Example: network tool guardrails

```
timeout 15s curl -fsS https://example.com/health
timeout 5s dig +time=2 +tries=1 example.com
```

Keeps cron probes from stacking up when DNS or HTTP hangs.

Example: command that starts with a dash

```
timeout 10s -- ./mytool --dangerous-flag
```

`--` prevents option eating by `timeout`.

Example: foreground mode

```
timeout --foreground 30s vim file.txt
```

Sometimes needed for programs that expect to be session leaders or read the TTY specially; default separate process group is usually correct for daemons/scripts.

Notes & Pitfalls

- Children that **double-fork** or reparent may outlive the timed command group depending on how they detach — design jobs not to daemonize if you need hard bounds.
- Pipeline gotcha: `timeout 5s cmd1 | cmd2` only times out `cmd1` unless you structure differently (`timeout 5s bash -c 'cmd1 | cmd2'`).
- systemd units have their own start/stop timeouts; combine thoughtfully.
- BusyBox `timeout` option sets are smaller — stick to common flags in portable scripts.

Related Commands

- `kill` — manual signaling
- `timelimit` — alternative on some systems
- `systemd-run -p RuntimeMaxSec=` — cgroup-enforced limits
- `nice` / `ionice` — resource politeness without a deadline
- `nohup` — survive hangup (orthogonal concern)

Additional Resources

- `man timeout`

watch

Overview

`watch` runs a command repeatedly, showing fullscreen output so you can see changes over time. Ideal for ad-hoc monitoring (`df`, `kubectl get`, `systemctl status`) without writing a loop. Not a metrics system — for long-term graphs use Prometheus/node_exporter or similar.

Syntax

```
watch [options] COMMAND
watch [options] -- COMMAND
```

`COMMAND` is passed to `sh -c` by default, so shell features work inside the watched string.

Common Options

Option	Description
`-n SECS`, `--interval SECS`	Seconds between runs (default 2; fractions OK on modern procps)
`-d`, `--differences`	Highlight differences between runs
`-d=permanent`	Keep highlighting changes persistently
`-t`, `--no-title`	Hide header (interval, host, command)
`-b`, `--beep`	Beep if command exits non-zero
`-e`, `--errexit`	Freeze screen on non-zero exit
`-g`, `--chgexit`	Exit when output changes
`-c`, `--color`	Interpret ANSI color sequences
`-x`, `--exec`	Pass command to `exec` instead of `sh -c`
`-p`, `--precise`	Attempt to run every interval accounting for command runtime

Key Use Cases

1. Watch disk, memory, or service state during an incident
2. Wait until a condition changes (`-g`) then continue a scripted workflow

3. Highlight diffs while configs or replica counts converge
4. Lightweight “dashboard” over SSH without extra tools

Examples with Explanations

Example: basic interval monitor

```
watch -n 1 df -h
```

Refresh disk usage every second.

Example: highlight changes

```
watch -d free -h
watch -d 'ip -s link show eth0'
```

Changed characters flash between iterations — great for counters and free memory.

Example: multi-command pipeline

```
watch -n 2 'ps aux --sort=-%cpu | head -n 15'
```

Quote the whole pipeline so the shell inside `watch` owns the pipe.

Example: systemd / journal peek

```
watch -n 2 'systemctl --failed --no-pager'
watch -n 1 'systemctl is-active nginx; systemctl status nginx --no-pager | head -n 15'
```

Handy while restarting services.

Example: exit when output changes

```
watch -g -n 1 'cat /var/run/myapp/ready'
echo "ready flag changed"
```

Blocks until the command’s output differs from the first run, then exits. Useful as a crude barrier in shell workflows.

Example: stop on error

```
watch -e -n 5 './healthcheck.sh'
```

Freezes on first non-zero exit so you can read the failure output.

Example: colors from modern CLIs

```
watch -c -n 2 'ls --color=always -l /srv/data'
```

Without `-c`, ANSI escapes may show as garbage.

Example: precise interval

```
watch -p -n 5 'date +%T; ./poll.sh'
```

Tries to keep wall-clock cadence even when `poll.sh` takes a second or two (not a hard realtime guarantee).

Example: exec mode (no shell)

```
watch -x -n 1 ping -c 1 1.1.1.1
```

Avoids `sh -c` — safer when you do not want shell expansion, but pipelines/globs will not work.

Notes & Pitfalls

- Default shell is `sh -c`; quote carefully and avoid unescaped `$` that expand **before** `watch` runs (use single quotes for the outer command string).
- Long-running watched commands that exceed the interval will stack poorly without `-p` discipline; keep the inner command fast.
- `watch` clears the screen — redirect-friendly logging needs a different approach (`while sleep; do …; done >> log`).
- Exit with `Ctrl-C`.
- On minimal systems ensure `procps`/`procps-ng` is installed (Ubuntu has `watch` from that package).

Related Commands

- `while sleep N; do …; done` — scriptable loop with logging
- `vmstat 1` / `iostat -xz 1` — specialized continuous metrics
- `journalctl -f` / `tail -f` — follow streams
- `systemd-run` / timers — scheduled work
- `timeout` — bound a single run

Additional Resources

- `man watch`

sleep

Overview

`sleep` suspends execution for a specified time. Used in shell scripts for pacing, simple retries, and demos. For precise timers, event-driven waits, or jittered backoff, higher-level tools may be better — but `sleep` remains the universal delay primitive.

Syntax

```
sleep NUMBER[SUFFIX]...
```

GNU sleep accepts floating point and multiple arguments (sums them).

Time suffixes (GNU)

Suffix	Unit
`s`	Seconds (default)
`m`	Minutes
`h`	Hours
`d`	Days

Examples with Explanations

Basics

```
sleep 5
sleep 0.5
sleep 2m
sleep 1h
sleep 1m 30s            # GNU: 90 seconds total
```

Simple retry loop

```
until ping -c1 -W1 gateway >/dev/null; do
  sleep 2
done
```

Exponential backoff (bash)

```
delay=1
for i in 1 2 3 4 5; do
  curl -fsS https://example.com/ && break
  sleep "$delay"
  delay=$((delay * 2))
done
```

Rate limiting

```
while read -r host; do
  ssh "$host" 'uname -r'
  sleep 0.2
done < hosts.txt
```

Background heartbeat

```
while true; do
  date -Is >> /tmp/heartbeat
  sleep 60
done &
```

Prefer systemd timers for production heartbeats.

Interruptible

```
sleep 300
# Ctrl-C sends SIGINT to foreground sleep
```

Portable note

```
# POSIX: integer seconds only on some platforms
sleep 5
# fractional may require GNU coreutils
```

Notes / Pitfalls

- Busy loops without sleep burn CPU — always throttle polls.
- `sleep infinity` works on GNU; not portable — use large numbers or `tail -f /dev/null`.
- Sleep drift: not a real-time scheduler; long loops accumulate error.
- Don't use sleep as the only readiness check for services — probe health endpoints.
- In pipelines, know which process sleeps.

2026-relevant notes

- Prefer `systemd-run` timers / cron / `watch` for periodic work.
- Kubernetes readiness/liveness already handle waits — avoid crude sleep in hot paths.
- `timeout` pairs with commands that must not hang forever.

Related Commands

- `timeout` — bound a command's runtime
- `watch` — run a command repeatedly
- `usleep` — legacy microseconds (often gone; use `sleep 0.001`)
- `at` / `systemd-run` — schedule future work
- `wait` — wait for jobs, not wall clock alone

Additional Resources

- `man sleep`

nice

Overview

`nice` starts a command with a modified **CPU niceness** — a soft priority bias for the scheduler. Higher nice means **more polite** (less CPU when others compete); lower (negative) nice means more aggressive. Range is typically **-20..19**, default **0**. Unprivileged users can only **increase** niceness. Use `renice` to change a running process; use `ionice` for disk I/O class.

Syntax

```
nice [OPTION] [COMMAND [ARG]...]
nice -n ADJUSTMENT COMMAND [ARG]...
```

With no command, some implementations print the current niceness.

Related:

```
renice [-n] PRIORITY -p PID...
renice [-n] PRIORITY -u USER...
```

Common Options

Option	Description
`-n ADJ`, `--adjustment=ADJ`	Add ADJ to default nice (often written as absolute intent: `nice -n 10`)
`--help` / `--version`	Meta

`renice` selection:

Option	Description
`-p PID`	Process ID(s)
`-g PGID`	Process group
`-u USER`	All of user's processes
`-n PRIO`	Absolute nice value to set (util-linux style)

Safety

- Negative nice values need **root** (or `CAP_SYS_NICE`) — do not renice critical daemons downward without understanding latency impact.
- Nice is **not** a hard cap or cgroup limit; a niced process can still use 100% CPU when the machine is idle.
- For multi-tenant fairness prefer **cgroups / systemd resource controls** (`CPUWeight=`, `CPUQuota=`) over global renice.

Key Use Cases

1. Run backups/compiles politely during business hours
2. Deprioritize a runaway batch without killing it
3. Slightly boost an interactive build only when policy allows
4. Inspect niceness during incident triage

Examples with Explanations

Example: start a polite batch job

```
nice -n 19 ./batch-compress.sh
nice -n 10 tar -czf /backup/home.tgz /home
```

`19` is maximum politeness on Linux. Good default for "finish eventually, don't fight production".

Example: background + log

```
nice -n 15 ./reindex.sh > /var/tmp/reindex.log 2>&1 &
```

Combine with shell backgrounding; still not a supervisor.

Example: renice a running PID

```
ps -o pid,ni,cmd -p 1234
renice -n 15 -p 1234
sudo renice -n -5 -p 1234
```

Ordinary users can raise nice (e.g. 0 → 15) on their own processes; lowering requires privilege.

Example: renice all of a user's jobs

```
sudo renice -n 10 -u builduser
```

Useful when a CI account is starving interactive sessions.

Example: observe nice in ps / top

```
ps -eo pid,user,ni,pri,cmd --sort=ni | head
ps -o pid,ni,comm -p 1234
# top/htop: NI column; htop F7/F8 adjust interactively
```

`NI` is niceness; `PRI` is the kernel priority derived from scheduling policy and nice.

Example: nice + ionice + nohup combo

```
nohup nice -n 19 ionice -c3 ./nightly-rsync.sh > /var/log/nightly-rsync.log 2>&1 &
```

CPU polite + idle I/O class + survive hangup — classic overnight job pattern.

Example: systemd equivalent knobs

```
systemd-run --user --nice=15 ./batch.sh
# in a unit:
# Nice=15
# CPUWeight=50
```

Prefer unit settings for services you own long-term.

Example: print current nice (when supported)

```
nice
```

Some systems print the shell's current niceness if no command is given.

Understanding Output

- `nice` itself is silent on success when launching a command.
- Failed renice prints permission or ESRCH errors.
- Inherited niceness: children typically inherit the parent's nice value.

Notes & Pitfalls

- Syntax confusion: old forms like `nice -10 cmd` mean adjustment **-10** on some Unixes; on GNU prefer **`nice -n 10 cmd`** explicitly.
- Realtime policies (`SCHED_FIFO` via `chrt`) override ordinary nice semantics — different tool.
- Containers may restrict `CAP_SYS_NICE`; negative renice fails even as "root" in a locked-down pod.
- Autogroup / CFS details can make nice effects less dramatic than expected on desktop kernels — still useful under contention.

Related Commands

- `renice` — change running processes
- `ionice` — I/O scheduling class
- `chrt` — realtime/other scheduling policies
- `timeout` — bound wall time
- `systemctl` / cgroups — proper resource control
- `top` / `htop` / `ps` — observe `NI`

Additional Resources

- `man nice`
- `man renice`
- `man sched(7)`

renice

Overview

The `renice` command alters the scheduling priority of running processes. It allows you to change the nice value of processes that are already running.

Syntax

```
renice [-n] priority [-g|-p|-u] identifier...
```

Common Options

Option	Description
`-n`	Specify nice increment
`-g`	Interpret as process groups
`-p`	Interpret as process IDs
`-u`	Interpret as usernames
`--help`	Display help
`--version`	Show version

Nice Values

- Range: -20 to 19
- Lower values = higher priority
- Higher values = lower priority
- Only root can set negative values
- Default: current nice value

Key Use Cases

1. Adjust process priority
2. Resource management
3. Performance tuning
4. System optimization
5. Process control

Examples with Explanations

Example 1: Basic Usage

```
renice +5 -p 1234
```

Change priority of PID 1234

Example 2: User Processes

```
renice 10 -u username
```

Change priority of user's processes

Example 3: Process Group

```
renice -5 -g 100
```

Change priority of process group

Understanding Output

Format:

```
1234: old priority 0, new priority 5
```

Error messages for: - Permission denied - Invalid priority - Process not found - User not found

Common Usage Patterns

1. Lower process priority:

```
renice +10 -p $(pgrep firefox)
```

2. Increase user priority:

```
sudo renice -5 -u apache
```

3. Multiple processes:

```
renice 5 -p 1234 5678
```

Performance Analysis

- Priority impact
- System load effect
- Resource allocation
- Process behavior
- Scheduling changes

Related Commands

- `nice` - Start with priority
- `top` - Process monitoring
- `ps` - Process status
- `ionice` - I/O scheduling
- `chrt` - Real-time scheduling

Additional Resources

- Renice Manual
- Process Priority Guide
- CPU Scheduling

Best Practices

1. Monitor system impact
2. Document changes
3. Consider dependencies
4. Regular review
5. Use appropriate values

Use Cases

1. Performance tuning
2. Resource allocation
3. System maintenance
4. Service optimization
5. Troubleshooting

ionice

Overview

`ionice` sets or queries the **I/O scheduling class and priority** of a process (Linux CFQ/BFQ-oriented interface; behavior depends on the current I/O scheduler and kernel). Use it so bulk disk work (backups, `updatedb`, large `rsync`) yields to latency-sensitive services. Orthogonal to CPU `nice`.

Syntax

```
ionice [options] -p PID...
ionice [options] COMMAND [ARG]...
ionice -p PID
```

I/O classes

Class	`-c`	Meaning
None	`0`	No class set / request default handling
Realtime	`1`	Highest access (requires privilege; can starve others)
Best-effort	`2`	Default class; priority 0–7 within class
Idle	`3`	Only when disk is idle — safest for bulk jobs

Best-effort priority: **0** = highest within class, **7** = lowest (opposite intuition from CPU nice numbers — read twice).

Common Options

Option	Description
`-c CLASS`	Class number or name (`idle`, `best-effort`, `realtime`) on newer util-linux

Option	Description
`-n PRIO`	Priority 0–7 within best-effort or realtime
`-p PID`	Act on existing PID (repeatable)
`-t`	Ignore failures (e.g. unsupported) — "try"
`-P PGID`	Process group (where supported)
`-u UID`	As user (where supported)

Safety

- **Realtime I/O (`-c1`)** can starve the system — avoid on production unless you know the storage stack and blast radius.
- Prefer **idle (`-c3`)** for overnight compaction, virus scans, and catalog updates.
- Capability/`CAP_SYS_ADMIN` or root may be required for realtime class; idle/best-effort for own processes is usually fine.
- On modern multi-queue SSDs with certain schedulers, ionice effects can be **weaker** than on classic CFQ HDDs — still good hygiene for HDDs and shared spindles.

Key Use Cases

1. Start backups in the idle I/O class
2. Soften a live `rsync`/`tar` hurting databases
3. Query current I/O class of a suspect PID
4. Combine with `nice` for "be a good neighbor" batch windows

Examples with Explanations

Example: run command as idle I/O

```
ionice -c3 ./backup.sh
ionice -c idle ./backup.sh
```

Only issues disk I/O when the device is not busy with others — best default for bulk work.

Example: best-effort low priority

```
ionice -c2 -n7 tar -czf /backup/data.tgz /data
```

Class best-effort, lowest priority level. Less extreme than idle; still polite.

Example: change a running process

```
ionice -c3 -p 1234
ionice -c2 -n7 -p 1234
```

Apply without restarting. Verify with a query afterward.

Example: query current setting

```
ionice -p 1234
# example output: none: prio 0
# or: idle
# or: best-effort: prio 4
```

Silent context for incident notes: “PID 1234 was idle class during the spike”.

Example: nice + ionice together

```
nice -n 19 ionice -c3 rsync -a /data/ /mnt/backup/data/
```

CPU and disk both deprioritized. Standard pattern for shared hosts.

Example: nohup overnight job

```
nohup nice -n 19 ionice -c3 \
  rsync -a --delete /srv/ /mnt/cold/srv/ \
  > /var/log/rsync-cold.log 2>&1 &
```

Survives logout, polite CPU/I/O, logged.

Example: try without failing scripts

```
ionice -t -c3 ./portable-job.sh
```

`-t` ignores failures when ionice cannot set class (containers, odd kernels) so the job still runs.

Example: avoid realtime unless justified

```
# dangerous on shared storage - shown for completeness only
# sudo ionice -c1 -n0 -p "$LATENCY_CRITICAL_PID"
```

Prefer storage QoS, cgroup `io.latency`/`IOWeight=`, and proper service isolation instead.

Notes & Pitfalls

- **Not portable** to non-Linux; no-op or absent on macOS/BSD.
- Effect depends on **scheduler** (`mq-deadline`, `bfq`, `none` for NVMe, etc.). Check `/sys/block/*/queue/scheduler`.
- Children inherit I/O priority at fork time; later `ionice -p` on the parent does not always retarget already-spawned children — apply to the whole tree if needed.
- systemd: `IOSchedulingClass=` / `IOSchedulingPriority=` in units; often cleaner than wrappers.
- Do not confuse with `nice` (CPU) or `chrt` (CPU policy).

Related Commands

- `nice` / `renice` — CPU niceness
- `chrt` — CPU scheduling policy
- `ionice` + `nohup` / `tmux` — long batch sessions
- `iostat` / `iotop` — observe disk pressure
- `systemctl` — unit-level I/O settings

Additional Resources

- `man ionice`
- `man ioprio_set` (syscall background)

fuser

Overview

`fuser` lists PIDs using a file, mount point, or network port. It is a concise complement to `lsof`: faster for "who is blocking my umount?" and optional signaling of those processes.

Syntax

```
fuser [options] name...
```

Common Options

Option	Description
`-v`	Verbose (USER, PID, ACCESS, COMMAND)
`-m`	Argument is a mount / filesystem
`-k`	Kill processes (default SIGKILL with `-k` alone on many builds—**confirm man**)
`-i`	Interactive confirm before kill
`-n tcp|udp`	Namespace for ports
`-u`	Append user
`-4` / `-6`	Address family
`-M`	Check is a mountpoint
`signal`	e.g. `-TERM` before `-k` on some implementations

Access codes (verbose): `c` current dir, `e` executable, `f` open file, `r` root dir, `m` mmap, etc.

Key Use Cases

1. Find processes preventing `umount`
2. See who holds a TCP/UDP port
3. Signal processes holding a lock file

Examples with Explanations

Mount busy

```
sudo fuser -vm /mnt/data
sudo fuser -vm /mnt/data  # review
# last resort:
sudo fuser -kim /mnt/data  # interactive kill on mount
```

Ports

```
sudo fuser -v 22/tcp
sudo fuser -v 8080/tcp
sudo fuser -n tcp 5432
```

Prefer also: `ss -ltnp | grep 8080`.

Files

```
fuser -v /var/log/app.log
fuser -ki file.lock
```

One-liner recipes

```
# Before umount
sudo fuser -vm /mnt && sudo umount /mnt

# Who owns this listening port?
sudo ss -ltnp | grep ':8080'
sudo fuser -v 8080/tcp
```

Notes & Pitfalls

- **`-k` is destructive**—always `-v` first; prefer `-i`.
- Default signal behavior differs slightly by util-linux version—read `man fuser` on the host.
- Network namespace: ports need `tcp`/`udp` suffix or `-n`.
- Containers: fuser inside vs on host sees different process IDs/namespaces.

2026-relevant notes

- For listeners, **`ss -lntp`** is usually clearer than fuser alone.
- Combine with `lsof -t` for scripts that need PID lists.
- Lazy umount (`umount -l`) hides problems—fix the holders when possible.

Related Commands

- `lsof` — detailed open files
- `ss` — sockets
- `umount` — detach mounts
- `fuser -k` vs `kill` — scope differs

Additional Resources

- `man fuser`

ulimit

Overview

`ulimit` is a **shell builtin** (bash/zsh/etc.) that shows or sets **resource limits** for the current shell and its child processes: open files, processes, stack size, core dumps, and more. It does not permanently reconfigure the whole system—use `/etc/security/limits.conf`, systemd unit directives, or cgroups for persistence and service-wide policy.

Syntax

```
ulimit [-SHacdefilmnpqrstuvx] [limit]
```

Common Options

Option	Resource
`-a`	Show all limits
`-n`	Max open file descriptors
`-u`	Max user processes
`-v`	Address space (KB) where supported
`-f`	Max file size (blocks)
`-c`	Core file size
`-s`	Stack size
`-l`	Locked memory
`-m`	Resident set (often ignored on Linux)
`-t`	CPU seconds
`-H`	Hard limit
`-S`	Soft limit

Soft limits can be raised up to the hard limit by the user; hard limits need root (or were set that way at login).

Key Use Cases

1. Raise `nofile` for databases, proxies, load generators
2. Allow core dumps (`-c unlimited`) while debugging
3. Diagnose "Too many open files" / "cannot fork"
4. Compare shell limits vs systemd service limits

Examples with Explanations

Show limits

```
ulimit -a
ulimit -n
ulimit -u
ulimit -H -n
ulimit -S -n
```

Raise open files (session)

```
ulimit -n 65536
ulimit -n
```

Fails if hard limit is lower—check `ulimit -H -n` and raise via limits.d or systemd.

Cores and stack

```
ulimit -c unlimited
ulimit -s 8192
```

One-liner recipes

```
# Reproduce app under higher nofile
ulimit -n 100000 && ./my-server

# Show soft/hard nofile pair
printf 'soft=%s hard=%s\n' "$(ulimit -Sn)" "$(ulimit -Hn)"

# Compare with PID 1 or a service (systemd)
systemctl show nginx -p LimitNOFILE --value
cat /proc/$(pgrep -n nginx)/limits
```

Persistent configuration (context)

```
# /etc/security/limits.d/99-nofile.conf  (pam_limits - login sessions)
# myuser soft nofile 65536
# myuser hard nofile 1048576

# systemd service drop-in
# [Service]
# LimitNOFILE=65536
```

`ulimit` alone in `~/.bashrc` does **not** affect systemd services.

Notes & Pitfalls

- **Builtin**: not `/usr/bin/ulimit`; dash/busybox shells differ.
- **Non-interactive SSH** may skip bashrc—put service limits in systemd/unit files.
- Containers inherit cgroup and docker `--ulimit` settings; shell ulimit cannot exceed them.
- `unlimited` is not always valid for every resource.
- Changing `-n` in one terminal doesn't affect already-running daemons.

2026-relevant notes

- Prefer **systemd** `LimitNOFILE=`, `TasksMax=`, and cgroup v2 pressure metrics over only pam limits on pure server images.
- Kubernetes sets pod resource limits separately; `ulimit` inside a pod is still constrained by the container runtime.
- Kernel `fs.file-max` and per-user limits both matter for "too many open files".

Related Commands

- `prlimit` — get/set limits of a running PID
- `systemctl show` — unit Limit* properties
- `cat /proc/PID/limits` — effective process limits
- `sysctl fs.file-max` — system-wide file ceiling

Additional Resources

- `help ulimit` (bash)
- `man limits.conf`, `man systemd.exec`

ipcs

Overview

`ipcs` lists **System V IPC** objects: message queues, shared memory segments, and semaphore arrays. Useful when debugging legacy applications, databases, or middleware that use SysV IPC, and when cleaning up leftover segments after crashes.

POSIX shared memory and mutexes use different interfaces (`/dev/shm`, pthreads) — not shown by `ipcs`.

Syntax

```
ipcs [options]
```

Common Options

Option	Description
`-a`	All resources (default summary style)
`-q`	Message queues
`-m`	Shared memory
`-s`	Semaphores
`-t`	Time info
`-p`	PIDs (creator/last operator)
`-c`	Creator user/group
`-l`	Resource limits
`-u`	Summary usage
`-i id`	Details for specific ID
`-b`	Brief bytes columns where applicable

Examples with Explanations

Overview

```
ipcs
ipcs -a
ipcs -m
ipcs -q
ipcs -s
```

Limits and usage

```
ipcs -l
ipcs -u
```

Creator and PIDs

```
ipcs -m -p
ipcs -m -c
ipcs -i 123456 -m
```

Find leftovers by user

```
ipcs -m | awk 'NR>3 && $3=="alice" {print}'
```

Cleanup companion

```
ipcs -m
sudo ipcrm -m <shmid>
# or by key:
sudo ipcrm -M 0x00001234
```

See `ipcrm` page for deletion safety.

Kernel params (context)

```
sysctl kernel.shmmax kernel.shmall kernel.shmmni
sysctl kernel.msgmax kernel.msgmni
sysctl kernel.sem
```

Notes / Pitfalls

- Deleting live shared memory of a running DB will wreck it — identify owners first.
- IDs vs keys: `ipcrm` flags differ (`-m` id vs `-M` key).
- Docker/IPC namespaces hide or isolate objects — empty `ipcs` may be namespace-related.
- Modern apps often prefer POSIX shm (`/dev/shm`) and won't appear here.
- Permissions: you may only see objects you can access.

2026-relevant notes

- Still relevant for Oracle, older SAP, and legacy C apps; less so for greenfield Go/Node services.
- Prefer container IPC isolation over host-global SysV segments when designing new systems.
- Monitor with app-native tools first; use `ipcs` for orphan cleanup after hard crashes.

Related Commands

- `ipcrm` — remove IPC objects
- `ls -l /dev/shm` — POSIX shared memory files
- `sysctl` — kernel IPC limits
- `fuser` / `lsof` — who uses files (not always SysV)
- `pmap` — process memory maps

Additional Resources

- `man ipcs`, `man ipc`

ipcrm

Overview

`ipcrm` removes **System V IPC** objects: shared memory segments, semaphore arrays, and message queues. Use after confirming with `ipcs` that objects are orphaned — never delete resources still owned by live production processes.

Syntax

```
ipcrm [options]
ipcrm -m id | -M key
ipcrm -q id | -Q key
ipcrm -s id | -S key
```

Common Options

Option	Description
`-m shmid`	Remove shared memory by **ID**
`-M key`	Remove shared memory by **key**
`-q msqid`	Remove message queue by ID
`-Q key`	Remove message queue by key
`-s semid`	Remove semaphore by ID
`-S key`	Remove semaphore by key
`-a`	Remove all (dangerous; some versions / with filters)
`-v`	Verbose

Exact `-a` behavior varies — read `man ipcrm` before bulk deletes.

Examples with Explanations

Inspect then remove one segment

```
ipcs -m
sudo ipcrm -m 98305
ipcs -m
```

Remove by key

```
ipcs -m
# note the key column (hex)
sudo ipcrm -M 0x00000000
```

Message queues / semaphores

```
ipcs -q
sudo ipcrm -q 12
ipcs -s
sudo ipcrm -s 34
```

Scripted cleanup of your own leftovers

```
# example: remove shared memory owned by current user carefully
id -u
ipcs -m -c
# manually select ids, then:
# ipcrm -m ID
```

Automating bulk `ipcrm` without filters is risky on multi-app hosts.

After crash lab example

```
# app crashed without IPC_RMID
ipcs -m -p
# verify no live PID uses it
ps -p <cpid>,<lpid>
sudo ipcrm -m <shmid>
```

Notes / Pitfalls

- Wrong ID → impact on another application (databases!).
- Need permission; root can remove almost anything.
- Race with process still attaching — prefer graceful app shutdown.
- Keys of `0x00000000` may mean `IPC_PRIVATE` — identify carefully.
- Does not remove POSIX shm files in `/dev/shm` — use `rm` there if appropriate.

2026-relevant notes

- Prefer designs that set `IPC_RMID` or use POSIX shm with clear lifecycle.
- In containers, IPC namespace teardown often cleans objects automatically on exit.
- Document any required SysV objects for legacy apps so ops doesn't "clean" them.

Related Commands

- `ipcs` — list IPC
- `sysctl` — kernel limits
- `ls /dev/shm` — POSIX shm
- `rm` — remove POSIX shm files
- `ipcmk` — create IPC (rarely needed manually)

Additional Resources

- `man ipcrm`

flock

Overview

Thesis: if two copies of a job must never run at once, take a lock. Do not rely on timing, PID files, or "it usually finishes in time". `flock` (util-linux) is the shell's interface to the kernel's `flock(2)` advisory locks. It wraps a command, or a block of a script, in a lock that the kernel releases **automatically** when the holder exits, even after `kill -9`, a crash, or a reboot. Nothing stale is left behind to clean up.

Typical operator jobs:

- Stop a cron job from piling up when a run takes longer than the interval.
- Make a deploy, backup, or sync script refuse to run twice.
- Serialize jobs that touch the same resource, such as a backup and a prune of the same repository.
- Let many readers proceed together while a writer waits for exclusive access.

```
# Debian/Ubuntu/Fedora: part of util-linux, installed by default
flock --version
```

```
flock from util-linux 2.41.5
```

Mental model

```
process A                          kernel                        process B

open("job.lock")     open file description
flock(fd, LOCK_EX)   lock on that description   flock(fd, LOCK_EX|LOCK_NB) → EWOULDBLOCK
                                                     (flock -n exits 1)
fork() child    inherits fd   same lock
exit / crash     last fd closed   lock released    B's blocking flock() returns
```

Fact	Consequence
The lock belongs to an **open file description**, not to a PID or a file name	Every child that inherits the fd also "holds" it. The lock ends when the **last** copy of the fd closes
Locks are **advisory**	Only programs that also call `flock` are excluded. `cat`, `echo >>`, and editors can still write the file

Fact	Consequence
The file's **contents are irrelevant**	An empty file is fine. The lock lives on the inode
Release is automatic on close or exit	No stale-lock cleanup, unlike PID files and `mkdir` locks
`flock(2)` and `fcntl(2)` locks are separate	`flock` and `flock --fcntl` (or Python `fcntl.lockf`, SQLite, …) do **not** exclude each other

Syntax

```
flock [options] FILE|DIR COMMAND [ARG...]    # lock, run COMMAND, unlock
flock [options] FILE|DIR -c 'SHELL STRING'   # same, through $SHELL -c (default /bin/sh)
flock [options] FD                           # lock an fd the calling shell already opened
```

Option	Meaning
`-x`, `-e`, `--exclusive`	Exclusive (write) lock. **Default**
`-s`, `--shared`	Shared (read) lock. Many holders, but no exclusive holder at the same time
`-n`, `--nonblock`	Do not wait: exit at once if the lock is busy
`-w SECS`, `--timeout SECS`	Wait at most SECS (fractions allowed; `-w 0` = `-n`)
`-E N`, `--conflict-exit-code N`	Exit status for "busy" with `-n` or `-w` (default `1`)
`-o`, `--close`	Close the lock fd before running COMMAND, so children do **not** inherit the lock
`-F`, `--no-fork`	`exec` COMMAND instead of forking; COMMAND itself holds the lock (incompatible with `-o`)
`-u`, `--unlock`	Drop the lock on FD early (the third form)
`--verbose`	Report how long acquiring took, or why it failed
`--fcntl`	Use an OFD `fcntl(2)` lock instead of `flock(2)` (util-linux 2.41+, kernel 3.15+)
`--start N` / `--length N`	Lock only a byte range (implies `--fcntl`, util-linux 2.42+)

Exit status: in the wrapping forms, the **command's** status. With `-n`/`-w`, a busy lock gives the `-E` value (default `1`). flock's own errors use `<sysexits.h>` codes (for example `64` for bad usage).

Safety

- **Put lock files where other users cannot pre-create them.** Use `/run/lock/<name>.lock` for root jobs and `$XDG_RUNTIME_DIR/<name>.lock` for user jobs. In world-writable `/tmp` on a shared host, another user can create your lock file first and hold it forever (a denial of service).

- **Never delete the lock file "to clean up".** That breaks mutual exclusion (see Notes & Pitfalls). Leave it in place. It costs one inode.
- **Do not lock on NFS or CIFS** unless you have verified that it works there. `flock` may always fail on those filesystems, or lock only locally, depending on mount options.
- **A lock is not a timeout.** A hung holder blocks everyone forever. Pair `flock` with `-w` on the waiting side, and with `timeout` on the holding side.

Examples with Explanations

All examples run as an ordinary user in a scratch directory:

```
mkdir -p /tmp/fl && cd /tmp/fl
```

1. Wait, fail fast, or wait a bounded time

```
L=/tmp/fl/demo.lock
(flock "$L" sleep 3 &) ; sleep 0.3                # holder: keeps the lock for 3 s

flock -n "$L" echo "got it";            echo "exit=$?"
flock --verbose -n "$L" true;           echo "exit=$?"
flock --verbose -w 1.5 "$L" true;       echo "exit=$?"
flock --verbose "$L" echo "waited, then ran"; echo "exit=$?"
```

```
exit=1
flock: failed to get lock
exit=1
flock: timeout while waiting to get lock
exit=1
waited, then ran
flock: getting lock took 1.195380 seconds
flock: executing echo
exit=0
```

By default `flock` blocks, which is what you want for "queue behind the other run". Use `-n` for "skip this run". Use `-w` for "wait a little, then give up". `--verbose` writes to stderr, so it is safe to leave on in cron jobs that mail you their output.

2. Tell "skipped" apart from "failed" with `-E`

```
(flock "$L" sleep 2 &) ; sleep 0.3
flock -n -E 75 "$L" true;      echo "exit=$?"      # busy → 75 (EX_TEMPFAIL)
flock "$L" sh -c 'exit 3';     echo "exit=$?"      # ran → command's own status
```

```
exit=75
exit=3
```

Without `-E`, a busy lock and a command that failed with status `1` look the same to your monitoring. Pick a code the job never uses itself. `75` (`EX_TEMPFAIL`) is the conventional choice, or `0` if a skipped run should count as success.

3. cron: no overlapping runs

```
# /etc/cron.d/poll - every 5 minutes; skip quietly if the last run is still going
*/5 * * * * root flock -n -E 0 /run/lock/poll.lock /usr/local/bin/poll.sh >>/var/log/poll.lo
```

`-n -E 0` means a skipped run is not an error, so cron sends no mail. Drop `-n` if runs should **queue** instead. Watch out, though: a job that is always slower than its interval then builds an ever-growing queue of waiting `flock` processes.

Under a **systemd timer** you do not need `flock` to stop a job overlapping *itself*. systemd will not start a unit that is still active. You still need it for **cross-job** exclusion, such as a backup and a prune of the same repository, or a timer run and a manual run.

4. Lock a whole script from the inside (file-descriptor form)

```
#!/usr/bin/env bash
# nightly-sync.sh - refuses to run twice; lock lives as long as this shell
set -euo pipefail
LOCK="${XDG_RUNTIME_DIR:-/tmp}/nightly-sync.lock"

exec 9>"$LOCK"                          # open (create) the lock file on fd 9
if ! flock -n 9; then
  echo "nightly-sync: another run holds $LOCK, skipping" >&2
  exit 0                                # a skipped run is not a failure
fi

echo "nightly-sync: started as PID $$"
sleep "${1:-5}"                         # stand-in for the real work
echo "nightly-sync: done"
```

```
chmod +x nightly-sync.sh
./nightly-sync.sh 4 & sleep 0.5
./nightly-sync.sh 1; echo "exit=$?"
wait
```

```
nightly-sync: started as PID 343505
nightly-sync: another run holds /tmp/nightly-sync.lock, skipping
```

```
exit=0
nightly-sync: done
```

`exec 9>FILE` opens the file in the current shell. `flock -n 9` locks that open description and exits, but the **shell** keeps fd 9 open, so the lock lasts until the script ends. Open the file with `>` or `>>` so it is created if missing (that needs write permission), or with `<` when it already exists and you only have read access. Release early with `flock -u 9` if the tail of the script does not need the lock.

To lock only a critical section, use a subshell, and the lock ends when the subshell ends:

```
(
  flock -w 10 9 || { echo "busy" >&2; exit 1; }
  # ... commands that must not run concurrently ...
) 9>/run/lock/critical.lock
```

5. Self-locking script boilerplate

```
#!/usr/bin/env bash
# selflock.sh - locks itself on first run (boilerplate from flock(1))
[ "${FLOCKER:-}" != "$0" ] && exec env FLOCKER="$0" flock -en "$0" "$0" "$@" || :
echo "selflock: working as PID $$"
sleep 2
```

```
chmod +x selflock.sh
./selflock.sh & sleep 0.3; ./selflock.sh; echo "second exit=$?"; wait
```

```
selflock: working as PID 345323
second exit=1
```

On the first run, `FLOCKER` is unset, so the script re-executes itself under `flock -en`, using **its own file** as the lock. The second copy finds the lock busy and exits `1` silently. This form needs no lock path, but anyone who can read the script can also hold its lock. Keep it for scripts you own.

6. Many readers, one writer

```
flock -s rw.lock sh -c 'echo "reader 1 in"; sleep 2' &
flock -s rw.lock sh -c 'echo "reader 2 in"; sleep 2' &
sleep 0.3
flock -n -x rw.lock echo "writer in"; echo "writer -n exit=$?"
flock --verbose -x rw.lock echo "writer in"
wait
```

```
reader 2 in
reader 1 in
writer -n exit=1
writer in
flock: getting lock took 1.699388 seconds
flock: executing echo
```

Both shared holders run together. The exclusive writer waits until the last reader leaves. A typical use: report jobs take `-s` on a data directory, and the nightly rebuild takes `-x` on the same lock file.

7. Who is holding the lock?

In the wrapping form, the long-lived `flock` process owns the lock, and `lslocks` shows it:

```
flock /tmp/fl/demo.lock sleep 3 & sleep 0.3
lslocks -o COMMAND,PID,TYPE,MODE,PATH | awk 'NR==1 || /demo.lock/'
```

```
COMMAND            PID  TYPE MODE  PATH
flock           344271 FLOCK WRITE /tmp/fl/demo.lock
```

In the fd form (example 4), the `flock` process that took the lock has already exited. Ask instead which processes have the **file open**. That is every holder, including inherited children:

```
./nightly-sync.sh 3 >/dev/null & sleep 0.3
fuser -v /tmp/nightly-sync.lock
```

```
                     USER        PID ACCESS COMMAND
/tmp/nightly-sync.lock:
                     box       344275 F.... bash
                     box       344278 F.... sleep
```

The `sleep` child appears too: it inherited fd 9. That matters for the trap below.

8. `-F`, directories, and `-c`

```
mkdir -p state && flock -n state echo "locked a directory"
flock -F -n f.lock sh -c 'echo "PID $$ holds the lock itself"; ls -l /proc/$$/fd | grep f.lo
```

```
locked a directory
PID 345416 holds the lock itself
/tmp/fl/f.lock
```

A directory works as a lock target, because `flock` opens it read-only. That is handy for "lock the data dir" conventions. `-F` replaces `flock` with the command, so no extra process sits in the tree, and signals go straight to the command. That suits supervisors that track the main PID.

9. `--fcntl` locks are a different lock

```
flock x.lock sleep 2 & sleep 0.2
flock -n --fcntl x.lock echo "fcntl lock acquired despite flock(2) holder"; echo "exit=$?"
flock -n x.lock echo nope; echo "exit=$?"
wait
```

```
fcntl lock acquired despite flock(2) holder
exit=0
exit=1
```

Use `--fcntl` only when you must interoperate with programs that use `fcntl`/`lockf` locks on the same file, such as a Python or C daemon. Every participant must use the **same** lock family. util-linux 2.42 adds `--start`/`--length` for byte-range locks of this kind.

The trap

A background child inherits the lock, and keeps it after `flock` has exited.

```
#!/bin/sh
# start-worker.sh - launches a long-lived background worker, then returns
sleep 30 &                      # stand-in for a daemon that outlives this script
echo "worker started as PID $!"
```

```
chmod +x start-worker.sh
flock -n app.lock ./start-worker.sh; echo "exit=$?"
flock -n app.lock echo "second run"; echo "exit=$?"
fuser -v app.lock
```

```
worker started as PID 344748
exit=0
exit=1
                     USER        PID ACCESS COMMAND
/tmp/fl/app.lock:    box       344748 f.... sleep
```

The wrapped script returned, and `flock` exited `0`, yet every later run is "busy" for as long as the worker lives. The worker inherited the lock fd. In a cron job this looks like "the job silently stopped running". With `-o`, the fd is closed before the command runs:

```
fuser -k app.lock >/dev/null 2>&1           # kill the stray worker from the first run
flock -n -o app.lock ./start-worker.sh; echo "exit=$?"
flock -n app.lock echo "second run";    echo "exit=$?"
```

```
worker started as PID 344757
exit=0
second run
exit=0
```

Choose deliberately:

- `-o`: the lock covers only the **launch**. The worker runs unprotected.
- No `-o`: the lock lasts as long as the **worker**. The worker should be the one that takes the lock (`flock -F`, or the fd form inside it).
- In the fd form, start background helpers with the fd closed: `helper 9>&- &`.

Notes & Pitfalls

Pitfall	Symptom / fix
Deleting the lock file while it is held	A new run creates a **new inode** and gets its own "exclusive" lock alongside the old holder. Demo: holder A runs, `rm job.lock`, then `flock -n job.lock …` succeeds. Never `rm` lock files; use `/run/lock`, which is cleared at boot
Expecting the lock to stop writes	Locks are advisory. `echo x >> job.lock` succeeds while it is held. Every cooperating program must call `flock`
Lock file in shared `/tmp`	Another user can pre-create and hold it. Use `/run/lock` or `$XDG_RUNTIME_DIR`
`flock -n 9` but forgot `exec 9>file`	`flock: 9: Bad file descriptor` (exit 65). Open the fd first, in the **same** shell (not a pipeline subshell)
Busy and failed both exit `1`	Add `-E 75` (or `-E 0` for silent skips)
Blocking `flock` in a fast cron interval	Waiting `flock` processes pile up behind a slow run. Use `-n` or `-w`
NFS/CIFS home directories	Locks fail or are local-only. Keep lock files on local disk or tmpfs
Mixing `flock` with `--fcntl`/`lockf` users	No mutual exclusion. Standardize on one family per lock file
`-o` with `-F`	`flock: the --no-fork and --close options are incompatible` (exit 64): nothing would be left holding the lock
Stuck holder	Find it with `lslocks` (wrapper form) or `fuser -v FILE` (fd form), then decide whether to kill it

The boring rule

> One job, one lock file under `/run/lock` (or `$XDG_RUNTIME_DIR`), never deleted. Cron jobs use `flock -n -E 0` (or `-E 75` if skips should page). Scripts lock themselves with `exec 9>…; flock -n 9`. Anything that backgrounds a child either uses `-o` or makes the child the lock holder. Diagnose with `lslocks` and `fuser -v`, never by guessing.

Related Commands

Command	Role
`lslocks`	List file locks held on the system, with the holder's command and PID
`fuser -v FILE`	Show every process with the lock file open, including inherited fds
`timeout`	Bound how long a lock holder may run (`flock L timeout 30m job`)
`systemd-run --on-calendar` / timers	Scheduled units that never overlap themselves
`cron`, `crontab`	Classic scheduler; pair with `flock -n`
`lsof FILE`	Alternative to `fuser` for open-file inspection

Try this

1. Add `flock -n -E 0 /run/lock/<job>.lock` in front of your longest-running cron job. Run it twice by hand at the same moment and confirm the second run exits `0` without doing any work.
2. Convert a script that writes a PID file into the fd form (`exec 9>…; flock -n 9`). Kill it with `kill -9` and confirm the next run starts immediately, with no stale-file cleanup.
3. Reproduce the trap with `start-worker.sh`, then fix it twice: once with `-o`, and once by making the worker take the lock itself with `flock -F`.
4. Build a reader/writer pair: two `flock -s` report loops and one `flock -x` rebuild with `-w 30`. Watch `lslocks` while they contend.
5. Show yourself why deleting lock files is unsafe: start a holder, `rm` the lock file, take the lock again, and list both holders with `lslocks`.

Sources

- flock(1): util-linux manual (options, exit status, `FLOCKER` boilerplate, NFS/CIFS notes)
- flock(2): kernel semantics (open file descriptions, inheritance across `fork`, advisory locking)
- util-linux v2.41 release notes (`--fcntl`) and v2.42 release notes (byte-range `--start`/`--length`)
- lslocks(8), fuser(1)

taskset

Overview

Thesis: the scheduler is usually better at placing work than you are. `taskset` is for the cases where it is not: keeping a noisy batch job off the CPUs a latency-sensitive service needs, reproducing a benchmark without migrations, or proving that a "slow" program is really just sharing a CPU. `taskset` (util-linux) reads and sets a task's **CPU affinity**, the set of CPUs the kernel is *allowed* to run it on, through `sched_getaffinity(2)` / `sched_setaffinity(2)`.

Typical operator jobs:

- Start a benchmark or build on fixed CPUs so the results can be repeated.
- Move a running batch process off the CPUs a database or packet path uses.
- Check *where* a process may run when it is slower than expected.
- Shrink a process's view of the machine so it sizes its thread pool smaller.

```
# Debian/Ubuntu/Fedora: part of util-linux, installed by default
taskset --version
nproc --all
```

```
taskset from util-linux 2.41.5
8
```

All outputs below come from an 8-vCPU Debian 13 VM (kernel 6.12, no SMT siblings). Your CPU numbers and timings will differ.

Mental model

```
cgroup cpuset (AllowedCPUs=)    hard ceiling, set by the admin/systemd/container runtime

task affinity mask               per THREAD, set by sched_setaffinity / taskset


scheduler picks a CPU from the intersection; inherited by fork() and new threads
```

Fact	Consequence
Affinity is a **per-thread** attribute	`taskset -p PID` changes only the thread whose TID equals PID (the main thread). Use `-a` for all threads

Fact	Consequence
Children and new threads **inherit** the mask	`taskset -c 3 make -j` keeps the whole build on CPU 3, which is probably not what you meant
It is a **permission**, not a reservation	Pinning A to CPU 2 does not keep B off CPU 2. Exclusivity needs cpusets or `isolcpus=`
Any process may **widen** its own mask again	`taskset` is a placement tool, not a security or quota boundary. Use cgroups for that
The effective set is the mask **cpuset**	Inside a container or a systemd unit with `AllowedCPUs=`, CPUs outside the cpuset are not reachable

Syntax

```
taskset [options] MASK COMMAND [ARG...]     # launch with a hex mask
taskset [options] -c LIST COMMAND [ARG...]  # launch with a CPU list
taskset [options] -p [MASK] PID             # read (no MASK) or set an existing PID
taskset [options] -cp [LIST] PID            # same, with a list; -p and -c grouped
```

Option	Meaning
`-c`, `--cpu-list`	Read/write a **list** (`0,5,8-11`, stride `0-10:2`) instead of a hex mask
`-p`, `--pid`	Operate on an existing PID instead of launching a command
`-a`, `--all-tasks`	With `-p`: apply to **every thread** of PID (all of `/proc/PID/task/*`)

Mask form: `0x1` = CPU 0, `0x5` = CPUs 0 and 2, `ff` = CPUs 0–7. On large machines the kernel prints masks as comma-separated 32-bit groups (`ffffffff,ffffffff`). Prefer `-c` lists in anything a human will read.

Exit status: `0` on success, `1` on failure (bad PID, invalid mask, no permission). When launching, `taskset` `exec`s the command, so the status you see is the command's own.

Safety

- **Changing someone else's process needs `CAP_SYS_NICE`.** You can always change your own processes and read anyone's. Do not "fix" a system daemon with `sudo taskset -p`. Set `CPUAffinity=` in its unit so the change survives a restart and is visible in config review.
- **Pinning a multi-threaded service to fewer CPUs than it has busy threads** turns a capacity problem into a latency problem. Measure before and after.
- **Never pin a production process to CPU 0 by habit.** CPU 0 often takes more housekeeping (timers, some IRQs) than the others.

Examples with Explanations

1. Read and set: masks, lists, strides

```
taskset -p $$                    # mask form
taskset -cp $$                   # list form
taskset -c 0-7:2 grep Cpus_allowed_list /proc/self/status
taskset 0x5     grep Cpus_allowed_list /proc/self/status
```

```
pid 257922's current affinity mask: ff
pid 257922's current affinity list: 0-7
Cpus_allowed_list:  0,2,4,6
Cpus_allowed_list:  0,2
```

`/proc/PID/status` (`Cpus_allowed_list`) is the kernel's own view and needs no tool. With util-linux 2.41, `taskset -p 0` is **bad usage** (pid 0 is accepted from 2.42 on). Use `$$` in scripts that must run on both.

2. Change a running process

```
sleep 30 & P=$!
taskset -cp 1,3 $P
taskset -cp $P
ps -o pid,psr,comm -p $P        # PSR = CPU it last ran on
```

```
pid 258147's current affinity list: 0-7
pid 258147's new affinity list: 1,3
pid 258147's current affinity list: 1,3
    PID PSR COMMAND
 258147   1 sleep
```

The man page guarantees that when `taskset` returns, the task has been moved to a legal CPU. You do not need to wait for it to drift there.

3. See contention instead of guessing

```
# file: burn.py - a fixed amount of pure-CPU work; prints wall time and the CPU it finished
import os, time

t0 = time.perf_counter()
n = 0
for i in range(30_000_000):
    n += i & 7
```

```
with open("/proc/self/stat") as f:
    cpu = f.read().rsplit(")", 1)[1].split()[36]   # field 39: "processor", CPU last run on
print(f"pid {os.getpid()} cpu {cpu} wall {time.perf_counter() - t0:.2f}s")
```

```
#!/usr/bin/env bash
# file: contention.sh - same work, three placements
set -u
echo "--- one copy, pinned to CPU 0"
taskset -c 0 python3 burn.py
echo "--- two copies, both pinned to CPU 0"
taskset -c 0 python3 burn.py & taskset -c 0 python3 burn.py & wait
echo "--- two copies, CPU 0 and CPU 1"
taskset -c 0 python3 burn.py & taskset -c 1 python3 burn.py & wait
```

```
chmod +x contention.sh && ./contention.sh
```

```
--- one copy, pinned to CPU 0
pid 259467 cpu 0 wall 1.85s
--- two copies, both pinned to CPU 0
pid 259475 cpu 0 wall 3.87s
pid 259474 cpu 0 wall 3.92s
--- two copies, CPU 0 and CPU 1
pid 259494 cpu 1 wall 1.77s
pid 259493 cpu 0 wall 1.90s
```

Two CPU-bound tasks on one CPU each take about **twice** as long, because the fair scheduler splits the CPU between them. Spread across two CPUs, each runs at full speed. This is the cheapest way to prove, or rule out, the theory that "it's slow because something else is on that core". On a VM, expect the occasional slow run when the hypervisor shares the physical core. Repeat a run before believing it.

4. Pin a whole build or benchmark

```
# file: Makefile - each target reports which CPUs its recipe shell may use
all: a b c d
a b c d:
    @echo "$@: $$(grep Cpus_allowed_list /proc/self/status | cut -f2)"
```

```
taskset -c 4-7 make -j4
```

```
b: 4-7
a: 4-7
d: 4-7
c: 4-7
```

Inheritance is the feature here. One `taskset` at the top covers `make` and every shell and compiler it forks. The same pattern (`taskset -c 2 ./bench …`) keeps a benchmark from migrating between runs, so you compare like with like.

5. Make the program see fewer CPUs

```
taskset -c 0-3 nproc
taskset -c 0-3 nproc --all
taskset -c 0-1 python3 -c "import os; print('cpu_count', os.cpu_count(), '| process_cpu_coun
```

```
4
8
cpu_count 8 | process_cpu_count 2
```

`nproc` honours affinity, and `nproc --all` does not. Runtimes differ. Python 3.13's `os.process_cpu_count()` follows the mask (and is what `concurrent.futures` now uses by default), while `os.cpu_count()` still reports the whole machine. A program that sizes its pool from the machine total will start 8 workers on 2 CPUs. Check which call your software uses before trusting `taskset` to shrink it.

The trap

`taskset -p` changes **one thread**. Most real services are multi-threaded, and their worker threads already exist when you run it.

```
# file: threads.py - a process with a main thread and three worker threads that just wait
import os, threading, time

stop = threading.Event()
for i in range(3):
    threading.Thread(target=stop.wait, name=f"worker-{i}").start()
print(os.getpid(), flush=True)
time.sleep(60)
stop.set()
```

```
#!/usr/bin/env bash
# file: show-tasks.sh - print every thread of PID $1 with its allowed-CPU list
for t in /proc/"$1"/task/*; do
  printf '%-8s %-10s %s\n' "${t##*/}" "$(cat "$t"/comm)" "$(awk '/Cpus_allowed_list/ {print
done
```

```
chmod +x show-tasks.sh
python3 threads.py > pid.txt & sleep 0.5; P=$(cat pid.txt)
```

```
taskset -cp 2 $P;  ./show-tasks.sh $P
taskset -acp 2 $P; ./show-tasks.sh $P
```

```
pid 258915's current affinity list: 0-7
pid 258915's new affinity list: 2
258915   python3    2
258917   python3    0-7
258918   python3    0-7
258919   python3    0-7
pid 258915's current affinity list: 2
pid 258915's new affinity list: 2
pid 258917's current affinity list: 0-7
pid 258917's new affinity list: 2
pid 258918's current affinity list: 0-7
pid 258918's new affinity list: 2
pid 258919's current affinity list: 0-7
pid 258919's new affinity list: 2
258915   python3    2
258917   python3    2
258918   python3    2
258919   python3    2
```

After plain `-p`, `taskset -cp $P` reports "2", and the three workers doing the actual work are still free to run anywhere. **Fix:** use `-a` for running processes. Better still, set affinity *before* the process starts (`taskset -c … cmd`, or `CPUAffinity=` in a systemd unit), so every thread inherits it. `-a` only covers threads that exist now. A thread pool that spawns new threads later inherits from whichever thread created it, and that is usually fine once all threads are pinned.

Notes & Pitfalls

Pitfall	Symptom / fix
Partly-invalid list	`taskset -c 0,99 cmd` succeeds and quietly runs on CPU 0 only. Only an **entirely** invalid set fails: `taskset -c 99 true` → `failed to set pid …'s affinity: Invalid argument` (exit `1`). Check the result with `taskset -cp`
`-c` after `-p`	Write `-cp LIST PID` (or `-p -c`). The list must come right before the PID
Pinning isolation	Other tasks and IRQs still use "your" CPU. For exclusivity use cpusets (`AllowedCPUs=` on the other slices) or `isolcpus=`/`nohz_full=` on the kernel command line
A process widens its own mask	`taskset -c 0 taskset -c 0-7 grep Cpus_allowed_list /proc/self/status` prints `0-7`. Enforce limits with cgroups, not affinity

Pitfall	Symptom / fix
Container/systemd cpuset	Lists outside the cpuset fail with `Invalid argument`. Read `/sys/fs/cgroup/cpuset.cpus.effective` to see what you actually have
CPU numbering vs topology	On SMT machines, CPUs n and $n{+}k$ may be siblings sharing one core. Check `lscpu -e` or `/sys/devices/system/cpu/cpu*/topology/thread_siblings_list` before "spreading" work
NUMA	Affinity says nothing about where memory comes from. Use `numactl --cpunodebind --membind` when locality matters
`taskset -p 0`	Bad usage on util-linux 2.41; accepted from 2.42. Use `$$`

The boring rule

> Don't pin by default. When you do, pin **at launch** (`taskset -c LIST cmd`, or `CPUAffinity=` in the unit), pin **all threads** (`-a`) if you must change a running process, check the result in `/proc/PID/task/*/status`, and use cpusets, not affinity, when the goal is to keep other work *off* those CPUs.

Related Commands

Command	Role
`nice`, `renice`	Weight within the normal scheduling class, without restricting CPUs
`chrt`	Scheduling policy (`SCHED_OTHER`/`BATCH`/`IDLE`/`FIFO`/`RR`/`DEADLINE`)
`lscpu -e`	CPU, core, socket and NUMA-node numbering, needed before choosing a list
`numactl`	CPU **and memory** placement on NUMA machines
`systemd-run -p CPUAffinity=… -p AllowedCPUs=…`	Affinity or a cpuset for a transient unit
`ps -o pid,psr,comm`, `top` (field `P`)	Which CPU a task last ran on
`mpstat -P ALL 1`	Per-CPU utilisation, to confirm the placement

Try this

1. Run `contention.sh`, then add a third `burn.py` pinned to CPU 0 and predict the wall time before you look.

2. Start `threads.py`, pin it with plain `-p`, and inspect it with `show-tasks.sh`. Then re-pin it with `-a` and compare.
3. Launch `taskset -c 0-1 python3 -c 'import concurrent.futures as f; print(f.ProcessPoolExecu` and explain the number.
4. Write a systemd drop-in with `CPUAffinity=2-3` for a test service, restart it, and confirm every thread's `Cpus_allowed_list`.
5. Prove that pinning is not isolation: pin one `burn.py` to CPU 3, run an unpinned `burn.py` loop alongside it, and watch `mpstat -P 3 1`.

Sources

- taskset(1): util-linux manual (mask and list syntax, strides, `-a`, PERMISSIONS, the "scheduled to a legal CPU" guarantee)
- sched_setaffinity(2): per-thread semantics, inheritance across `fork`, `EINVAL` when no CPU in the mask is permitted, `CAP_SYS_NICE`
- cpuset(7) and the kernel's cgroup v2 cpuset documentation: how cpusets bound affinity
- systemd.exec(5) `CPUAffinity=` and systemd.resource-control(5) `AllowedCPUs=`
- util-linux v2.42 release notes (taskset accepts pid 0)
- Python `os.process_cpu_count()` (3.13+)

sched_ext sub-schedulers: lending CPUs from a BPF root

Overview

Thesis: `taskset` and cpusets tell the *fair* scheduler where a thread may run. **sched_ext** lets a BPF program *be* the scheduler. On the Linux **7.3**-era sched_ext work, a **root** BPF scheduler can hand a cgroup subtree to a nested **sub-scheduler** and **lend CPUs** with revocable, capability-based grants — so the child fully owns enqueue, dispatch, and preemption on those CPUs until the parent takes them back. This chapter is the operator/mental-model view: what shows up in sysfs, how grants differ from affinity masks, and what to read in `Documentation/scheduler/sched-ext.rst` and the for-7.3 pull summary. No BPF programs are loaded here (that needs privileges and a configured kernel); we stay on primary docs and observable interfaces.

Typical reasons to care:

- Run a latency-tuned BPF policy for one service cgroup without replacing the whole machine's scheduler.
- Understand why a nested scx scheduler starts with **zero** CPUs until the parent grants some.
- Debug "tasks stuck / rescue execution" when grants and affinity disagree.

```
# Read-only probes - safe on any modern host; values vary
uname -r
test -d /sys/kernel/sched_ext && echo 'sched_ext sysfs present' || echo 'sched_ext sysfs abs
```

On a kernel built without `CONFIG_SCHED_CLASS_EXT`, the directory is simply missing. That is expected.

Mental model

```
                root BPF scheduler (owns all caps on all CPUs)

          scx_bpf_sub_grant(cgroup, cmask, caps)


             sub-scheduler on a cgroup subtree
        (enqueue / preempt / cpufreq caps only where granted)

          scx_bpf_sub_revoke(...)  → tasks requeued with SCX_TASK_REENQ_CAP
```

Idea	Meaning
Root	Attached BPF scheduler that starts with every capability on every CPU id (cid)
Sub-scheduler	Another `scx_sched` attached under a cgroup; initially **no** CPU access
Grant / revoke	Parent hands a **subset** of its caps on selected CPUs; child never exceeds parent
Caps (per CPU)	Roughly: place on idle CPU, enqueue into that CPU's DSQ, preempt others, influence frequency — enforced on every path that touches a CPU
Dispatch vs enqueue	Earlier sub-sched support could recurse dispatch with `scx_bpf_sub_dispatch()`; 7.3-era work makes **enqueue** capability-based state too, so parents can partition CPUs

Affinity (`taskset`) still intersects: a task pinned off every granted CPU has nowhere legal to run. The 7.3 series adds **rescue execution** — a small bandwidth budget so such tasks are not left runnable-forever until the watchdog kills the whole BPF scheduler.

How this differs from `taskset`

Tool	What it constrains
`taskset` / `sched_setaffinity`	Which CPUs the *fair* (or current) class may pick for a thread
cgroup cpuset	Hard ceiling for a group
sched_ext sub-grants	Which CPUs a *nested BPF scheduler* may use for its tasks' enqueue/preempt paths

You can combine them; grants do not replace cpusets.

Worked examples (docs-faithful)

Case 1: Is sched_ext alive?

From `Documentation/scheduler/sched-ext.rst`:

```
cat /sys/kernel/sched_ext/state          # enabled | disabled | …
cat /sys/kernel/sched_ext/root/ops       # name of loaded root ops, if any
cat /sys/kernel/sched_ext/enable_seq     # monotonically increasing load count
```

Illustrative shape when a simple example scheduler is running (from the kernel doc):

```
# cat /sys/kernel/sched_ext/state
enabled
# cat /sys/kernel/sched_ext/root/ops
simple
```

Event counters live under the scheduler kobject, e.g. `/sys/kernel/sched_ext/root/events` (`SCX_EV_*` lines). Sub-scheduler bypass events (such as `SCX_EV_SUB_BYPASS_DISPATCH`) are part of the diagnostic surface when hierarchy misbehaves.

Case 2: Lending CPUs (capability grant story)

From the sched_ext/for-7.3 patchset and LWN coverage of sub-scheduler enqueue support:

1. Sub-scheduler attaches under a cgroup → **no** CPUs yet.
2. Root (or parent) calls **`scx_bpf_sub_grant`** with the child's cgroup id and a CPU mask (and capability bits).
3. Child learns via ops such as **`sub_caps_updated` / `sub_ecaps_updated`**; idle CPUs may get an **`update_idle`** re-notification so the child can use them immediately.
4. Parent calls **`scx_bpf_sub_revoke`** → tasks on those CPUs are pulled and re-enqueued to the child with **`SCX_TASK_REENQ_CAP`** so it must pick a still-allowed CPU.

```
Example (conceptual, not a runnable bpftrace script):

  root R: grant cids 8-15 to child A  →  A may enqueue/preempt only there
  A:      grant cids 8-11 to child B  →  B's set   A's set   R's set
  R:      revoke cid 11 from A        →  cascades; tasks requeued
```

The example scheduler **`scx_qmap`** in the 7.3 pull is cited as demonstrating full hierarchical sub-scheduling once the series lands.

Case 3: Partial switch vs whole-machine takeover

Kernel doc flags:

`ops->flags`	Effect
default (no partial)	`SCHED_NORMAL` / `BATCH` / `IDLE` / `EXT` tasks go through sched_ext when loaded
`SCX_OPS_SWITCH_PARTIAL`	Only `SCHED_EXT` policy tasks use the BPF scheduler; fair class keeps the rest

Abort paths: terminate the scheduler binary, `SysRq-S`, or internal errors (including stalled runnable tasks) → revert to fair-class.

The trap

- Assuming a nested scheduler can "see" all CPUs because the machine has them — **grants start empty**.
- Using only dispatch-time recursion and expecting enqueue isolation — without capability state, children could still insert into local DSQs they should not own (the problem the 7.3 enqueue work fixes).
- Pinning tasks with `taskset` onto CPUs their sub-scheduler was never granted — you need rescue/fallback behavior or you look like a hung BPF scheduler.
- Loading random out-of-tree BPF schedulers on production without a revert plan (`SysRq-S`, fair-class fallback).

The boring rule

Treat the root sched_ext program as the **owner of CPU capabilities**; lend with grant/revoke; keep cgroup boundaries and affinity consistent with those grants; watch `/sys/kernel/sched_ext/` before you blame the fair scheduler.

Try this

1. On a lab kernel with `CONFIG_SCHED_CLASS_EXT=y`, read `Documentation/scheduler/sched-ext.rst` and the `tools/sched_ext` examples (`scx_simple`, `scx_qmap`).
2. Compare `cat /sys/kernel/sched_ext/state` before and after starting/stopping an example scheduler (requires privileges — do this only on machines you administer).
3. Read the sched_ext-for-7.3 pull summary (lore.kernel.org / kernel.org git) for grant/revoke and rescue execution.
4. Related userspace: revisit `taskset` and cpusets in this part — they constrain *tasks*; sub-grants constrain *which BPF scheduler may place them.*

Sources

- Linux kernel: `Documentation/scheduler/sched-ext.rst` (Extensible Scheduler Class)
- `[GIT PULL] sched_ext: Changes for v7.3` — hierarchical sub-scheduling / CPU delegation summary
- Patchset: *sched_ext: Capability-based CPU delegation for sub-schedulers* (sched_ext/for-7.3)
- LWN: *Sched-ext: enqueue() for sub-schedulers and proxy-execution support* (https://lwn.net/Articles/1082717/)

choom and the OOM killer score

Overview

Thesis: when a Linux host or a memory cgroup runs out of memory and reclaim cannot help, the kernel kills one process, and it picks the one with the highest **badness**. Badness is not a mystery heuristic. It is **resident memory + swap + page tables**, plus a per-process bias called `oom_score_adj`, measured in thousandths of the memory the OOM domain has. `choom` (util-linux) reads and sets that bias; `/proc/PID/oom_score` shows the result. If you know the formula, you can predict which process dies, steer the choice away from the database and toward the batch job, and read an `OOMKilled` without guessing.

Typical operator jobs:

- Explain why a particular process was killed (it was the biggest in its domain, or someone gave it a large `oom_score_adj`).
- Start a disposable job so it dies first: `choom -n 500 -- ./batch`.
- Protect a critical daemon, with the right tool (a unit setting or a privileged write, not a shell trick).
- Decide whether a container should lose one process or all of them (`memory.oom.group`).

```
# Debian/Ubuntu/Fedora: part of util-linux, installed by default
choom --version
uname -r
```

```
choom from util-linux 2.41.5
6.12.94+
```

All output below comes from a Debian 13 host with 16 GiB of RAM, **no swap**, kernel 6.12, as an unprivileged user (`uid=1000`, no capabilities). The kernel code quoted is `mm/oom_kill.c` and `fs/proc/base.c` from Linux 6.12. Nothing in this chapter triggers a real OOM kill: on a shared machine that would kill someone else's work. The kill path is described from the kernel source.

Mental model

```
allocation fails, reclaim and compaction did not help


OOM domain    global:        totalpages = RAM pages + swap pages
              memory cgroup: totalpages = memory.max (+ swap.max) in pages
```

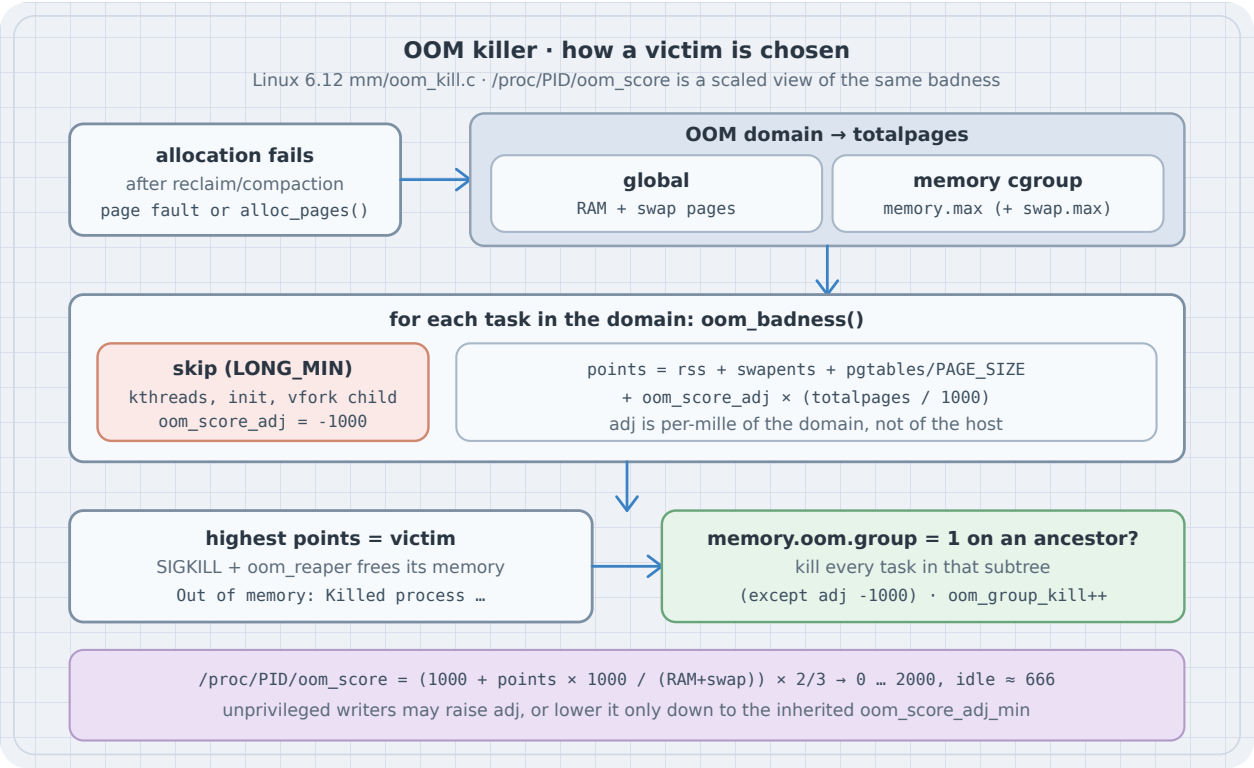


Figure 1: How the OOM killer chooses a victim

```
          for every task in the domain   (oom_badness, mm/oom_kill.c)
   skip  kernel threads, global init, a vfork child, oom_score_adj == -1000
   points = rss + swap entries + page-table pages
          + oom_score_adj × (totalpages / 1000)


highest points wins   SIGKILL, oom_reaper frees its anonymous memory

          an ancestor cgroup (up to the OOM domain) has memory.oom.group = 1?
             → kill every task in that cgroup subtree too (except -1000)
```

Three facts carry the whole topic:

1. **The base score is size.** Only memory the kernel can take back counts: RSS (anon + file + shmem), swap entries, and page-table pages. CPU time, age, nice value, and root-ness are not in the formula (they were, long ago; the heuristic was replaced in Linux 2.6.36).
2. **`oom_score_adj` is per-mille of the OOM domain.** `+100` means "pretend I use 10 % more of *this domain's* memory." On a 16 GiB host that is about 1.6 GiB. Inside a 512 MiB container it is 51 MiB.
3. **`/proc/PID/oom_score` is a display scale, not the raw number.** `fs/proc/base.c` maps badness into 0–2000 with `(1000 + points × 1000 / totalpages) × 2 / 3`. A tiny process reads **666**, not 0. `-1000` reads **0** and means "never chosen."

Values of `oom_score_adj` and what they mean:

`oom_score_adj`	Effect
`-1000`	`OOM_SCORE_ADJ_MIN`: the task is never chosen and is spared by `memory.oom.group` kills
`-999` … `-1`	Subtract up to 99.9 % of the domain from the score: strong protection, still killable
`0`	Default. Pure size.
`1` … `999`	Add a handicap: chosen earlier than a bigger neighbour
`1000`	`OOM_SCORE_ADJ_MAX`: chosen first in practice, whatever its size

The value lives in the process's `signal_struct`, so every thread of a process shares one value. It is copied to children on `fork()` and survives `execve()`. That is why `choom -n 300 -- cmd` works: `choom` sets its own value, then execs `cmd`.

Syntax

```
choom -p PID                    # show score and adjust value
choom -n ADJ -p PID             # set oom_score_adj of a running process
choom -n ADJ [--] COMMAND ARGS  # run COMMAND with oom_score_adj = ADJ
```

Interface	Meaning
`/proc/PID/oom_score`	Read-only, 0–2000 display of current badness. Recomputed on every read.
`/proc/PID/oom_score_adj`	Read/write, `-1000` … `1000`. Out-of-range writes fail with `EINVAL`.
`/proc/PID/oom_adj`	Legacy, `-17` … `15`, a scaled view of the same value (`-17` = disable). The kernel logs a deprecation warning on write. Do not use it.
`memory.oom.group` (cgroup v2)	`1` = kill the cgroup's whole subtree together when one of its tasks is the victim.
`memory.events` (cgroup v2)	Counters: `oom` (limit hit), `oom_kill` (processes killed), `oom_group_kill` (group kills).
`vm.panic_on_oom`, `vm.oom_kill_allocating_task`, `vm.oom_dump_tasks`	Global policy: panic instead of kill, kill the allocator instead of scanning, print the task table.

Who may write `oom_score_adj` (`__set_oom_adj()` in `fs/proc/base.c`):

- Anyone allowed to write the `/proc` file (the process owner, or root) may **raise** it.

- Lowering is allowed only down to the task's **`oom_score_adj_min`**. Going lower needs `CAP_SYS_RESOURCE` and fails with `EACCES` otherwise.
- `oom_score_adj_min` is set whenever a writer **with** `CAP_SYS_RESOURCE` sets the value, and it is **inherited** on `fork()`. It is not visible in `/proc`.

Safety

- Do not "test" the OOM killer on a shared host. **`stress-ng --vm`** or an allocation loop with no limit kills whatever is biggest, which is rarely your test. Use a memory cgroup with a limit (**`systemd-run --user -p MemoryMax=…`** on a host with a user manager), a VM, or a disposable container.
- `-1000` on a process that leaks memory moves the kill to its neighbours, and in the worst case leaves a host with nothing killable: **`Out of memory and no killable processes...`** followed by **`panic("System is deadlocked on memory")`**.
- Setting `oom_score_adj` on a running process affects only that process (and other processes sharing its address space through `CLONE_VM`, which the kernel keeps in sync). Children started later inherit it. Children started earlier do not change.
- Container runtimes and the kubelet rewrite `oom_score_adj` for container processes. A manual change inside a container can be overwritten on restart.

Examples with Explanations

1. Read the three numbers for one process

```
#!/bin/sh
# oom_inspect.sh - the three OOM files for one process (default: this shell)
pid=${1:-$$}
printf 'pid            %s (%s)\n' "$pid" "$(cat /proc/$pid/comm)"
printf 'oom_score      %s\n' "$(cat /proc/$pid/oom_score)"
printf 'oom_score_adj  %s\n' "$(cat /proc/$pid/oom_score_adj)"
printf 'oom_adj        %s   (legacy, -17..15)\n' "$(cat /proc/$pid/oom_adj)"
grep -E '^(VmRSS|RssAnon|RssFile|RssShmem|VmSwap|VmPTE):' /proc/$pid/status
```

```
chmod +x oom_inspect.sh && ./oom_inspect.sh
```

```
pid            32554 (oom_inspect.sh)
oom_score      666
oom_score_adj  0
oom_adj        0   (legacy, -17..15)
VmRSS:      1884 kB
RssAnon:         120 kB
RssFile:        1764 kB
RssShmem:          0 kB
```

```
VmPTE:        48 kB
VmSwap:        0 kB
```

A 2 MiB shell reads **666**. That is the scale's zero point: `(1000 + 0) × 2 / 3`. The memory lines are what the kernel adds up: `VmRSS` (the sum of the three `Rss*` lines), `VmSwap`, and `VmPTE` (page tables).

2. Watch the score follow resident memory, and check the formula

```python
# oom_grow.py - watch /proc/self/oom_score follow RSS, and check the kernel formula
import mmap, os

PAGE = os.sysconf("SC_PAGE_SIZE")

def meminfo_kb(key):
    with open("/proc/meminfo") as f:
        for line in f:
            if line.startswith(key + ":"):
                return int(line.split()[1])

# proc_oom_score(): totalpages = totalram_pages() + total_swap_pages
totalpages = (meminfo_kb("MemTotal") + meminfo_kb("SwapTotal")) * 1024 // PAGE

def status_kb(key):
    with open("/proc/self/status") as f:
        for line in f:
            if line.startswith(key + ":"):
                return int(line.split()[1])

def predicted(adj):
    if adj == -1000:
        return 0                                    # OOM_SCORE_ADJ_MIN: never a candidate
    # oom_badness(): rss + swapents + page-table pages, plus adj * (totalpages/1000)
    pts = (status_kb("VmRSS") + status_kb("VmSwap") + status_kb("VmPTE")) * 1024 // PAGE
    badness = pts + adj * (totalpages // 1000)
    # proc_oom_score(): scale into 0..2000 with C integer division
    q = abs(badness * 1000) // totalpages           # C '/' truncates toward zero
    return (1000 + (q if badness >= 0 else -q)) * 2 // 3

def show(label):
    adj = int(open("/proc/self/oom_score_adj").read())
    score = int(open("/proc/self/oom_score").read())
    print(f"{label:<16} VmRSS={status_kb('VmRSS')//1024:>5} MiB  adj={adj:>5}  "
          f"oom_score={score:>4}  predicted={predicted(adj):>4}")
```

```
print(f"totalpages={totalpages} ({totalpages*PAGE>>20} MiB RAM+swap)")
show("start")
chunks = []
for gib in (1, 2, 3):
    m = mmap.mmap(-1, 1 << 30)            # anonymous, private
    for off in range(0, 1 << 30, PAGE):   # touch every page so it counts in RSS
        m[off] = 1
    chunks.append(m)
    show(f"+{gib} GiB touched")
for adj in (500, 1000, -500, -1000):
    with open("/proc/self/oom_score_adj", "w") as f:
        f.write(str(adj))
    show(f"adj -> {adj}")
```

```
python3 oom_grow.py
```

```
totalpages=4096592 (16002 MiB RAM+swap)
start            VmRSS=    9 MiB  adj=    0  oom_score= 666  predicted= 666
+1 GiB touched   VmRSS= 1033 MiB  adj=    0  oom_score= 709  predicted= 709
+2 GiB touched   VmRSS= 2057 MiB  adj=    0  oom_score= 752  predicted= 752
+3 GiB touched   VmRSS= 3081 MiB  adj=    0  oom_score= 794  predicted= 794
adj -> 500       VmRSS= 3081 MiB  adj=  500  oom_score=1128  predicted=1128
adj -> 1000      VmRSS= 3081 MiB  adj= 1000  oom_score=1461  predicted=1461
adj -> -500      VmRSS= 3081 MiB  adj= -500  oom_score= 462  predicted= 462
adj -> -1000     VmRSS= 3081 MiB  adj=-1000  oom_score=   0  predicted=   0
```

Read it as arithmetic, not as magic:

- Each touched GiB is 6.4 % of 16 GiB, so it adds `64 × 2/3  43` display points (666 → 709 → 752 → 794). An `mmap` that was never touched would add nothing: virtual size is not in the formula.
- `+500` adds `500 × 2/3  333`, the same as owning half the machine. A 3 GiB process with `+500` now outranks a 9 GiB process with `0`.
- `-500` subtracts the same amount. The process is still a candidate. Only `-1000` takes it out (`0`).
- The prediction matches the kernel exactly once you use C integer division (truncate toward zero). With floats the first line reads 667.

The `-500` and `-1000` writes succeeded here without privileges. Example 3 explains why, and why that is not true everywhere.

3. What an unprivileged user may change

```
#!/bin/sh
# oom_adj_perms.sh - what this (unprivileged) user may do with oom_score_adj
set -u
echo "== who am I"
id -u; grep CapEff /proc/self/status
echo "== choom starts a child with an adjusted score (inherited across fork+exec)"
choom -n 300 -- sh -c 'echo "child adj=$(cat /proc/self/oom_score_adj) score=$(cat /proc/sel

echo "== raise, then lower back to 0"
sh -c '
  echo 800 > /proc/self/oom_score_adj && echo "raised:  $(cat /proc/self/oom_score_adj)"
  echo 0   > /proc/self/oom_score_adj && echo "lowered: $(cat /proc/self/oom_score_adj)"
'

echo "== go below 0 without CAP_SYS_RESOURCE"
choom -n -500 -- sh -c 'echo "child adj=$(cat /proc/self/oom_score_adj) score=$(cat /proc/se
echo "choom exit=$?"
echo "== why: the floor (oom_score_adj_min) is inherited from an ancestor"
choom -p 1
cat /proc/1/comm

echo "== out of range"
python3 -c '
import os
fd = os.open("/proc/self/oom_score_adj", os.O_WRONLY)
try:
    os.write(fd, b"1001")
except OSError as e:
    print("write 1001:", e.errno, e.strerror)
'
```

```
chmod +x oom_adj_perms.sh && ./oom_adj_perms.sh
```

```
== who am I
1000
CapEff: 0000000000000000
== choom starts a child with an adjusted score (inherited across fork+exec)
child adj=300 score=866 oom_adj=5
== raise, then lower back to 0
raised:  800
lowered: 0
== go below 0 without CAP_SYS_RESOURCE
child adj=-500 score=334
choom exit=0
```

```
== why: the floor (oom_score_adj_min) is inherited from an ancestor
pid 1's current OOM score: 0
pid 1's current OOM score adjust value: -1000
tini
== out of range
write 1001: 22 Invalid argument
```

The surprise is the `-500` line. An unprivileged user with no capabilities protected a process. The kernel did not skip a check. This machine is a container whose init (`tini`, PID 1) was set to `-1000` by a privileged runtime. That write also set `oom_score_adj_min = -1000`, and `copy_signal()` / `copy_oom_score_adj()` in `kernel/fork.c` copy both the value and the floor to every descendant. Later processes raised their own value back to 0, but the floor stayed at `-1000`, so any of them may lower it again.

On a normal host, where systemd starts your session with `oom_score_adj_min = 0`, the same `choom -n -500` fails:

```
choom: failed to set score adjust value: Permission denied
```

(That line is the `EACCES` path in `__set_oom_adj()`. It was **not** reproduced on this host: no process here has a floor of 0, and setting one needs `CAP_SYS_RESOURCE`.)

`oom_adj=5` for `adj=300` is the legacy scale: `300 × 17 / 1000 = 5` (truncated). `-1000` maps to `-17` (`OOM_DISABLE`) and anything that would exceed 15 is clamped to 15.

4. Rank the candidates before the kernel has to

```
#!/bin/sh
# oom_rank.sh - who would the global OOM killer pick first? (highest oom_score)
printf '%7s %6s %5s %9s  %s\n' PID SCORE ADJ RSS_MiB COMM
for d in /proc/[0-9]*; do
  pid=${d#/proc/}
  s=$(cat "$d/oom_score" 2>/dev/null) || continue
  a=$(cat "$d/oom_score_adj" 2>/dev/null) || continue
  r=$(awk '/^VmRSS:/ {print int($2/1024)}' "$d/status" 2>/dev/null)
  c=$(cat "$d/comm" 2>/dev/null)
  printf '%7s %6s %5s %9s  %s\n' "$pid" "$s" "$a" "${r:-0}" "$c"
done | sort -k2,2nr | head -n "${1:-8}"
```

```
chmod +x oom_rank.sh && ./oom_rank.sh 8
./oom_rank.sh 400 | awk '$2==0' | head -5
```

```
    PID  SCORE   ADJ   RSS_MiB  COMM
  24938    671     0       125  node
    237    670     0        84  node
  11824    669     0        67  Xvfb
```

```
 14393    669     0        68  node
 17838    669     0        77  node
 25026    669     0        76  xfwm4
 25400    669     0        76  plank
   221    668     0        34  node
```

```
     1      0 -1000         0  tini
     7      0 -1000         8  pod-daemon
   305      0 -1000         4  bash
   350      0 -1000         3  bash
   356      0 -1000        82  node
```

The first table is the kill order for a global OOM right now: the largest resident process (`node`, 125 MiB) is first. The scores are close because every process here is small against 16 GiB; a single 1 GiB process would jump to about 709. The second table shows processes that will never be picked. An 82 MiB `node` sits at 0 because it inherited `-1000` from its parents. That is the usual way production hosts end up with a large unkillable process nobody chose to protect.

5. The same `oom_score_adj` means different things in different domains

```
# oom_adj_weight.py - oom_score_adj is per-mille of the OOM *domain*, not of the host
MiB = 256  # 4 KiB pages per MiB

def badness(rss_mib, adj, total_mib):
    totalpages = total_mib * MiB
    return rss_mib * MiB + adj * (totalpages // 1000)   # oom_badness()

for name, total in (("host, 16 GiB RAM+swap", 16384), ("container, memory.max=512M", 512)):
    print(f"{name}: 1 adj point = {total*MiB//1000} pages = {total*MiB//1000/MiB:.2f} MiB")
    a = badness(300, 0, total)    # big, untouched process
    b = badness(100, 300, total)  # small helper marked "kill me first"
    pick = "A (300 MiB, adj 0)" if a > b else "B (100 MiB, adj 300)"
    print(f"  A={a:>8} pages  B={b:>8} pages  -> victim {pick}")
```

```
python3 oom_adj_weight.py
```

```
host, 16 GiB RAM+swap: 1 adj point = 4194 pages = 16.38 MiB
  A=   76800 pages  B= 1283800 pages  -> victim B (100 MiB, adj 300)
container, memory.max=512M: 1 adj point = 131 pages = 0.51 MiB
  A=   76800 pages  B=   64900 pages  -> victim A (300 MiB, adj 0)
```

For a cgroup OOM, `oom_kill.c` sets `totalpages = mem_cgroup_get_max(memcg)`, the cgroup's limit (plus swap allowance), not the host's RAM. The helper's `+300` is worth about 4.8 GiB on the host but only 154 MiB inside a 512 MiB limit. Same processes, same adjust values, different victim. `/proc/PID/oom_score` always uses the **host** total, so it cannot tell you the ranking inside

a container's own OOM.

6. Read the cgroup's OOM counters and group setting

```
cat /proc/self/cgroup
cat /sys/fs/cgroup/agent/memory.max /sys/fs/cgroup/agent/memory.oom.group
cat /sys/fs/cgroup/agent/memory.events
```

```
0::/agent
max
0
low 0
high 0
max 0
oom 0
oom_kill 0
oom_group_kill 0
```

`memory.events` is where a cgroup OOM leaves evidence even after the logs rotate. `oom` counts how often the limit was hit with an allocation about to fail. `oom_kill` counts processes killed. `oom_group_kill` counts kills of the whole group. They are hierarchical: a kill in a child cgroup shows up in its parents too (`memory.events.local` has the cgroup's own numbers). Here the cgroup has no limit and nothing was killed. The files are owned by root, so `memory.oom.group` could not be changed in this run.

When `memory.oom.group` is `1`, `mem_cgroup_get_oom_group()` walks from the victim's cgroup up to the OOM domain and picks the **highest** cgroup that has it set. Every task in that subtree is killed, except tasks at `-1000` and init. The kernel log then contains:

```
Tasks in /system.slice/app.service are going to be killed due to memory.oom.group set
```

(Format from `mem_cgroup_print_oom_group()`; not produced on this host.)

7. Make the policy stick: systemd units and Kubernetes (not run here)

This host has no systemd (PID 1 is `tini`), so these are written from the `systemd.exec(5)`, `systemd.service(5)`, and Kubernetes documentation:

```
# /etc/systemd/system/batch-report.service.d/oom.conf
[Service]
OOMScoreAdjust=500     # this unit's processes are preferred victims
OOMPolicy=kill         # one OOM kill => systemd sets memory.oom.group=1, whole unit dies
```

```
# /etc/systemd/system/postgresql.service.d/oom.conf
[Service]
OOMScoreAdjust=-900    # strong protection, still killable as a last resort
```

`OOMScoreAdjust=` takes `-1000` … `1000` and is applied by the service manager, which has the capability to set a negative value (and therefore a negative floor for the unit's children). `OOMPolicy=` is `continue`, `stop`, or `kill`. `kill` sets `memory.oom.group=1` on the unit's cgroup; `stop` lets systemd stop the remaining processes cleanly. `stop` and `kill` both leave the unit in the `oom-kill` failed state, where `Restart=` can apply.

The kubelet sets `oom_score_adj` from the pod's QoS class:

QoS class	`oom_score_adj`
Guaranteed (and `system-node-critical` pods)	`-997`
BestEffort	`1000`
Burstable	`min(max(2, 1000 - 1000 × memoryRequest / nodeMemoryCapacity), 999)`

On cgroup v2 nodes the kubelet also sets `memory.oom.group=1` on each **container** cgroup unless `singleProcessOOMKill: true` is set. So a multi-process container loses all its processes on one OOM kill, which the pod sees as `OOMKilled` and restarts per `restartPolicy`. Inside the container, the limit is the domain (Example 5), so the biggest process *in that container* is the victim, not the biggest on the node.

The trap

Trap: "only root can lower `oom_score_adj`." The check is against an inherited, invisible floor (`oom_score_adj_min`), not against your uid. Example 3 lowered a process to `-500` with `CapEff: 0` because PID 1 had been set to `-1000` by a privileged writer. If a container runtime or a supervisor gives its init `-1000`, every process in that tree can protect itself. Look at the ancestors (`choom -p 1`, the supervisor PID) before you trust that "nobody can opt out."

Trap: "`oom_score` 666 means the process is a big target." 666 is the bottom of the scale for a tiny process with `adj 0`. The range is 0–2000: 0 is "never," 666 is "uses nothing," 1333 is "uses everything" (or `+1000` with nothing). Compare scores with each other, not with an absolute threshold.

Trap: "the score in `/proc` is what the cgroup OOM uses." `/proc/PID/oom_score` is scaled by host RAM + swap. A memcg OOM scales `oom_score_adj` by the cgroup's limit. Example 5 shows the same pair of processes giving different victims.

Trap: "set the database to `-1000` and the problem is solved." If the database is the process growing without bound, `-1000` moves the kill to everything else, including `sshd` and the agent you would use to fix it. Prefer a strong negative value (`-900`) and a memory limit on the noisy neighbours.

Trap: "`VmSize` / virtual memory decides it." The formula uses resident pages, swap entries, and page tables. A process that reserved 64 GiB and touched 100 MiB scores as 100 MiB.

Notes & Pitfalls

- `vm.panic_on_oom=1` panics instead of killing (`2` panics even for cgroup-constrained OOMs). `vm.oom_kill_allocating_task=1` kills the task that triggered the OOM instead of scanning, which is cheaper on huge process tables but chooses badly. Both are `0` here; `vm.oom_dump_tasks=1` prints the task table (`[ pid ] uid tgid total_vm rss … oom_score_adj name`) before the kill.
- The victim line in the kernel log is `Out of memory: Killed process PID (comm) total-vm:…kB, anon-rss:…kB, file-rss:…kB, shmem-rss:…kB, UID:… pgtables:…kB oom_score_adj:…`. For a cgroup OOM the prefix is `Memory cgroup out of memory`. `journalctl -k -g 'Killed process'` or `dmesg -T | grep -i 'killed process'` find it.
- The kernel OOM killer acts only when allocation would otherwise fail. Long stalls under memory pressure come first. `systemd-oomd` acts earlier, on pressure (PSI) or swap use, and kills whole cgroups; the kubelet evicts whole pods at its eviction thresholds. Neither uses `oom_score_adj` to make that choice.
- `/proc/PID/oom_adj` writes log `oom_adj is deprecated, please use /proc/PID/oom_score_adj instead.` once per boot.

The boring rule

Size decides; `oom_score_adj` only tilts, and it tilts in per-mille of the domain that ran out. Set the tilt where the process is started (a unit's `OOMScoreAdjust=`, the kubelet's QoS, `choom -n` for ad-hoc jobs). Give disposable work a positive value, give critical services a strong negative value instead of `-1000`, and put a memory limit on anything that can grow.

Related Commands

- `ulimit` / `prlimit` — per-process limits; `RLIMIT_AS` makes `malloc` fail, which is a different failure from an OOM kill.
- `nice` / `renice` — CPU priority. It does not affect OOM choice.
- `ps -o pid,rss,comm` / `top` — RSS, the main input to the score.
- `free`, `/proc/meminfo` — `MemTotal` + `SwapTotal` is the global domain size.
- `systemd-run --user -p MemoryMax=… --scope cmd` — run a command in a limited cgroup, the safe way to watch a real OOM kill.

Try this

1. Run `oom_grow.py` with 1, 2, 3 GiB on your machine and predict each `oom_score` before it prints. Then add swap (or test on a host that has it) and see the numbers fall: `totalpages` grows, the same RSS is a smaller share.
2. Start two copies of a 1 GiB allocator, one under `choom -n 100`. Which `oom_score` is higher, and by how much? Check against `100 × 2/3`.

3. On a host with a user systemd manager: `systemd-run --user -p MemoryMax=200M -p MemorySwapMax=0 --wait python3 -c 'b = bytearray(400 << 20)'`. Read the unit cgroup's `memory.events` and `journalctl -k -g 'Memory cgroup out of memory'`.
4. In a test cgroup, start `sh -c 'sleep 600 & python3 alloc.py'`, set `memory.oom.group` to `0` and then `1`, and see whether `sleep` survives the kill.
5. Run `./oom_rank.sh 400 | awk '$3 < 0'` on a production host. For every negative value, find who set it (unit file, kubelet, runtime) and whether it was intended.

Verification

- Run unprivileged on this box: Debian 13 (trixie), Linux **6.12.94+**, 16 GiB RAM, no swap, util-linux **choom 2.41.5**, Python **3.13.5**, `uid=1000`, `CapEff` 0, cgroup v2 (`0::/agent`). All output above is pasted from those runs.
- `oom_grow.py` touched up to 3 GiB of anonymous memory (about 19 % of RAM, well inside `MemAvailable`) and compared `/proc/self/oom_score` with `oom_badness()` + `proc_oom_score()` from Linux 6.12: all eight lines match exactly.
- **Not run:** a real OOM kill (shared host), writes to `memory.oom.group` / `memory.max` (cgroup files are root-owned), the `EACCES` path for lowering below the floor (no process here has a floor of 0), systemd `OOMScoreAdjust=` / `OOMPolicy=` (no systemd on this host), and the kubelet. These are written from the kernel source and the documentation listed below.
- No packages installed, no root, no `sudo`.

Sources

- Linux 6.12 `mm/oom_kill.c` — `oom_badness()`, `constrained_alloc()` (`totalpages` for global vs memcg), `oom_kill_process()`, `__oom_kill_process()` log line: https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/mm/oom_kill.c?h=linux-6.12.y
- Linux 6.12 `fs/proc/base.c` — `proc_oom_score()` (0–2000 scaling), `__set_oom_adj()` (`EACCES`, `oom_score_adj_min`), legacy `oom_adj` mapping: https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/fs/proc/base.c?h=linux-6.12.y
- Linux 6.12 `kernel/fork.c` — `copy_signal()` / `copy_oom_score_adj()` (value and floor inherited)
- Linux 6.12 `mm/memcontrol.c` — `mem_cgroup_get_oom_group()`, `mem_cgroup_print_oom_group()`
- Kernel docs — Control Group v2: `memory.oom.group`, `memory.events`
- Kernel docs — The /proc filesystem: `oom_adj` & `oom_score_adj`, `oom_score`
- Kernel docs — `/proc/sys/vm`: `panic_on_oom`, `oom_kill_allocating_task`, `oom_dump_tasks`
- man7 — `proc_pid_oom_score_adj(5)`, `proc_pid_oom_score(5)`, `choom(1)`
- systemd — `systemd.exec(5)` `OOMScoreAdjust=`, `systemd.service(5)` `OOMPolicy=`
- Kubernetes — Node-pressure eviction: node out of memory behavior, KubeletConfiguration `singleProcessOOMKill`

pidfd: process handles that survive PID reuse

Overview

Thesis: a PID is a number, and the kernel recycles numbers. Between "I saved PID 4242" and "`kill(4242, SIGTERM)`" the process can exit, be reaped, and its number can be handed to a stranger, so the signal lands on the wrong process. A **pidfd** is a file descriptor that refers to *one specific process*. It cannot be retargeted: once that process has gone, every operation on the descriptor fails with `ESRCH` instead of touching whoever now owns the number. With a pidfd you can signal a process, wait for it, poll for its exit (even when it is not your child), copy a descriptor out of it, and learn which process is on the other end of a Unix socket. None of this needs root.

Typical operator and programmer jobs:

- A supervisor or job runner that kills jobs past a deadline without ever hitting a recycled PID.
- Waiting for a process that is **not** your child (a daemon from a pidfile, a peer) without a `kill(pid, 0)` polling loop.
- Turning a PID from a pidfile or `ps` into a handle you can trust, by checking identity *after* opening.
- Recognising pidfd-based code in systemd, D-Bus, polkit, container runtimes and language runtimes, and reading it in `/proc/PID/fdinfo`.

```
uname -r
python3 --version
ldd --version | head -1
gcc --version | head -1
```

```
6.12.94+
Python 3.13.5
ldd (Debian GLIBC 2.41-12+deb13u4) 2.41
gcc (Debian 14.2.0-19) 14.2.0
```

All output below comes from a Debian 13 host, kernel 6.12 (an LTS series), as an unprivileged user (`uid=1000`, `CapEff: 0`), in a container whose PID 1 is a root-owned `tini`. Every listing ran there as shown. Kernel features newer than 6.12 are marked as such and were not run.

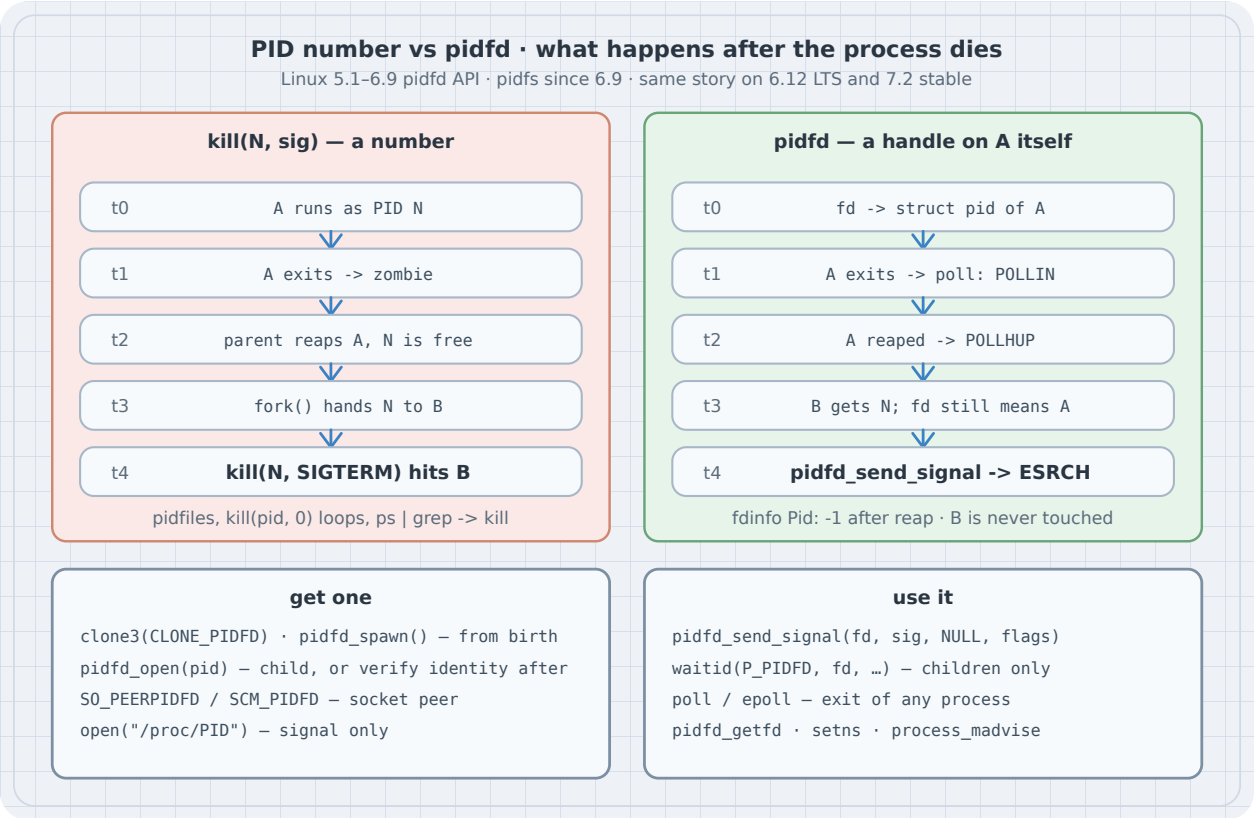


Figure 1: PID number vs pidfd

Mental model

```
PID number                          pidfd

an int in a table of pid_max        an fd → the kernel's struct pid for ONE process
freed when the zombie is reaped     holds a reference: the struct pid outlives the
then reissued to the next fork()    process, but never moves to another process

kill(N, sig)    whoever is N now   pidfd_send_signal(fd)   that process, or ESRCH
waitpid(N)      children only      waitid(P_PIDFD, fd)     children only
kill(N, 0) loop   "is N alive?"    poll/epoll(fd)          POLLIN on exit (any process)
                                                            POLLHUP once it is reaped
```

Four facts carry the whole topic:

1. **The race is between "reaped" and "reused".** A PID cannot be reused while it is a zombie. It is freed when the parent (or a subreaper, or init) calls `wait*()`. After that the next `fork()` that reaches that number gets it. Your own unreaped child is therefore safe to `pidfd_open()`; a PID you read from a file or from `ps` is not, until you check.
2. **A pidfd pins the identity, not the process.** It does not keep the process alive or stop the reap. It keeps the kernel's `struct pid` alive, so the descriptor can always answer "is *that* process still around?" honestly.
3. **Exit is an fd event.** The descriptor becomes readable when the process exits (for a process, when its *last* thread exits) and reports hang-up once it has been reaped. That puts process exits into the same `poll/epoll/io_uring` loop as sockets and timers.
4. **Opening is free, acting is checked.** Anyone may open a pidfd for any visible process. Signalling it needs the same permission as `kill(2)`; `pidfd_getfd(2)` needs ptrace-attach permission.

Syntax

The calls, the kernel that introduced them, and the libc wrappers (glibc 2.36 added `pidfd_open`, `pidfd_send_signal`, `pidfd_getfd`; 2.39 added `pidfd_spawn`, `pidfd_spawnp` and `pidfd_getpid`; Python has `os.pidfd_open`, `signal.pidfd_send_signal` and `os.waitid(os.P_PIDFD, …)` since 3.9):

Call	Since	What it does
`pidfd_send_signal(pidfd, sig, info, flags)`	5.1	Signal the process the fd refers to. `ESRCH` once it has been reaped.
`clone(CLONE_PIDFD)` / `clone3(.flags = CLONE_PIDFD)`	5.2 / 5.3	Create a child and get its pidfd atomically, before anyone can reap it.
`pidfd_open(pid, flags)`	5.3	Open a pidfd for an existing PID. Close-on-exec is always set.

Call	Since	What it does
`waitid(P_PIDFD, pidfd, &info, options)`	5.4	Wait for / reap a **child** through its pidfd. `ECHILD` for non-children.
`pidfd_getfd(pidfd, targetfd, 0)`	5.6	Duplicate another process's fd into yours (needs `PTRACE_MODE_ATTACH_REALCREDS`).
`SO_PEERPIDFD` / `SCM_PIDFD`	6.5	Get the pidfd of a Unix-socket peer / of a message's sender.
`PIDFD_NONBLOCK` (open flag)	5.10	`waitid()` returns `EAGAIN` instead of blocking.
pidfs, `PIDFD_THREAD`, `PIDFD_SIGNAL_{THREAD,THREAD_GROUP,PROCESS_GROUP}`	6.9	pidfds get their own filesystem and inode numbers; thread pidfds; signal scope flags.
`ioctl(pidfd, PIDFD_GET_INFO, &info)`	6.13	pid/tgid/ppid, credentials, cgroup id without `/proc` (not on this 6.12 box).
`PIDFD_INFO_EXIT`	6.15	Exit status after the process has been reaped (not on this 6.12 box).

Things that are also pidfds, or close: a descriptor from `open("/proc/PID", O_DIRECTORY)` works with `pidfd_send_signal()` but cannot be polled or waited on. A pidfd also works as the target of `setns(2)` (join that process's namespaces) and `process_madvise(2)`.

What you see from outside:

Interface	Shows
`readlink /proc/PID/fd/N`	`anon_inode:[pidfd]` (kept for compatibility, even on pidfs)
`/proc/PID/fdinfo/N`	`Pid:` the process's PID in your namespace, `-1` after it was reaped; `NSpid:` the PID in each nested namespace
`fstat()` on 6.9+	`st_ino` identifies the process (pidfs); two pidfds for the same process have the same inode

Safety

- Every example signals only its own children. Example 5 aims at PID 1 with **signal 0** only, which checks permission and delivers nothing. Do not "test" with a real signal on someone else's process.
- A pidfd held open keeps a small kernel object (`struct pid`) alive, not the process. Close them: a supervisor that leaks one per job leaks file descriptors until `EMFILE`.
- `pidfd_getfd()` gives you a *duplicate*: the file offset and status flags are shared with the target. Reading from it moves the target's offset too.

- Some programs set `SIGCHLD` to `SIG_IGN` (or use `SA_NOCLDWAIT`). Their children are reaped automatically, so `pidfd_open(child)` after `fork()` can already race. Use `CLONE_PIDFD` or `pidfd_spawn()` there.

Examples with Explanations

1. One child, one pidfd: inspect, poll, signal, reap, use again

```
# pidfd_basics.py - one child, one pidfd: inspect, poll, signal, reap, then use it again
import os, select, signal, subprocess

child = subprocess.Popen(["sleep", "60"])
fd = os.pidfd_open(child.pid)                 # a file descriptor that *is* this process

def fdinfo(fd):
    with open(f"/proc/self/fdinfo/{fd}") as f:
        return {k: v.strip() for k, v in (l.split(":", 1) for l in f)}

print("child pid      ", child.pid)
print("pidfd          ", fd, "->", os.readlink(f"/proc/self/fd/{fd}"))
print("fdinfo Pid     ", fdinfo(fd)["Pid"], "| NSpid", fdinfo(fd)["NSpid"])

p = select.poll()
p.register(fd, select.POLLIN)
print("poll (running) ", p.poll(0))           # [] : not readable while alive

signal.pidfd_send_signal(fd, signal.SIGTERM)  # signal the process, not a number
print("poll (exited)  ", p.poll(1000))        # readable once it has exited

info = os.waitid(os.P_PIDFD, fd, os.WEXITED)  # reap through the same handle
print("waitid         ", f"si_pid={info.si_pid} si_code={info.si_code}"
      f" (CLD_KILLED={os.CLD_KILLED}) si_status={info.si_status}"
      f" ({signal.Signals(info.si_status).name})")
print("fdinfo Pid     ", fdinfo(fd)["Pid"], "(after reap)")
ev = p.poll(0)[0][1]
print("poll (reaped)  ", f"revents=0x{ev:x}", "POLLIN|POLLHUP" if ev == select.POLLIN | sele

try:
    signal.pidfd_send_signal(fd, signal.SIGKILL)
except ProcessLookupError as e:
    print("signal again   ", e.__class__.__name__, f"errno={e.errno}")
child.returncode = -15                        # tell Popen we already reaped it
os.close(fd)
```

```
python3 pidfd_basics.py
```

```
child pid       111127
pidfd           5 -> anon_inode:[pidfd]
fdinfo Pid      111127 | NSpid 111127
poll (running)  []
poll (exited)   [(5, 1)]
waitid          si_pid=111127 si_code=2 (CLD_KILLED=2) si_status=15 (SIGTERM)
fdinfo Pid      -1 (after reap)
poll (reaped)   revents=0x11 POLLIN|POLLHUP
signal again    ProcessLookupError errno=3
```

The life of a pidfd in nine lines. While the child runs, `poll` returns nothing. After `SIGTERM` it becomes readable (`POLLIN`, event `1`). `waitid(P_PIDFD)` reaps through the same handle and reports `CLD_KILLED` with `SIGTERM`. After the reap, `fdinfo` shows `Pid: -1`, `poll` reports `POLLIN|POLLHUP` (`0x11`), and a second signal fails with `ESRCH` (errno 3). The PID number is now free for anyone; the descriptor still means the dead process and nothing else.

2. A pidfd has its own identity (pidfs, Linux 6.9+)

```
# pidfs_identity.py - on Linux 6.9+ a pidfd lives on pidfs; its inode number names the proce
import ctypes, os, subprocess

class Statfs(ctypes.Structure):
    _fields_ = [("f_type", ctypes.c_long), ("rest", ctypes.c_byte * 112)]
libc = ctypes.CDLL(None, use_errno=True)

a = subprocess.Popen(["sleep", "30"])
fd1 = os.pidfd_open(a.pid)
fd2 = os.pidfd_open(a.pid)          # a second, independent open of the same process
b = subprocess.Popen(["sleep", "30"])
fd3 = os.pidfd_open(b.pid)

sfs = Statfs()
libc.fstatfs(fd1, ctypes.byref(sfs))
print(f"f_type        0x{sfs.f_type & 0xffffffff:08x}  (PID_FS_MAGIC is 0x50494446, 'PIDF')"
for name, fd in (("a, open #1", fd1), ("a, open #2", fd2), ("b", fd3)):
    st = os.fstat(fd)
    print(f"{name:12}  st_dev={st.st_dev} st_ino={st.st_ino}")
for p in (a, b):
    p.kill(); p.wait()
```

```
python3 pidfs_identity.py
```

```
f_type        0x50494446  (PID_FS_MAGIC is 0x50494446, 'PIDF')
a, open #1    st_dev=5 st_ino=114794
a, open #2    st_dev=5 st_ino=114794
```

```
b              st_dev=5 st_ino=114795
```

Since Linux 6.9 pidfds live on **pidfs**, a tiny internal filesystem (`PID_FS_MAGIC`). Two separate opens of the same process have the **same inode**; a different process has a different one. On 64-bit kernels pidfs takes inode numbers from a 64-bit counter that is not reset during a boot, so the inode is a stable process identity you can store and compare, which the PID never was. On older kernels the fd is an anonymous inode and all pidfds share one inode, so do not rely on this below 6.9.

3. Wait for a process that is not your child

```
# watch_nonchild.py - wait for a process that is NOT our child, without polling kill(pid, 0)
import os, select, subprocess, sys, time

# A double fork: the shell exits at once, so `sleep` is re-parented away from us.
out = subprocess.run(["sh", "-c", "sleep 2 >/dev/null 2>&1 & echo $!"],
                     capture_output=True, text=True, check=True)
pid = int(out.stdout)
with open(f"/proc/{pid}/stat") as f:
    ppid = int(f.read().rsplit(")", 1)[1].split()[1])
print(f"target pid {pid}, its parent is {ppid}, we are {os.getpid()}")

fd = os.pidfd_open(pid)
try:
    os.waitid(os.P_PIDFD, fd, os.WEXITED | os.WNOHANG)
except ChildProcessError as e:
    print("waitid on a non-child:", e.__class__.__name__, f"errno={e.errno}")

t0 = time.monotonic()
ready = select.select([fd], [], [], 10)[0]   # sleeps in the kernel, no busy loop
print(f"readable after {time.monotonic() - t0:.2f}s: {ready == [fd]}")
os.close(fd)
```

```
python3 watch_nonchild.py
```

```
target pid 111133, its parent is 52, we are 111131
waitid on a non-child: ChildProcessError errno=10
readable after 2.00s: True
```

The shell exits at once, so `sleep` is re-parented to the nearest subreaper (PID 52 in this container, init on a plain host). We cannot reap it, and `waitid(P_PIDFD)` says so with `ECHILD` (errno 10). We can still *watch* it: `select` blocks in the kernel and wakes when the process exits, after exactly its 2 s. Before pidfds the options were a `kill(pid, 0)` polling loop (busy and racy: a reused PID looks "alive") or `ptrace`. Here we opened the pidfd while the process was certainly alive and was certainly the one we started; for a PID from somewhere else, see Example 6.

4. Get the pidfd at birth: `clone3(CLONE_PIDFD)` and `pidfd_spawnp()`

```c
// born_with_pidfd.c - get the pidfd at creation time: clone3(CLONE_PIDFD) and pidfd_spawnp(
#define _GNU_SOURCE
#include <errno.h>
#include <linux/sched.h>    // struct clone_args, CLONE_PIDFD
#include <signal.h>
#include <spawn.h>          // pidfd_spawnp (glibc 2.39+)
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/pidfd.h>      // pidfd_getpid, pidfd_send_signal (glibc 2.36+)
#include <sys/syscall.h>
#include <sys/wait.h>
#include <unistd.h>

extern char **environ;

static void reap(const char *who, int pidfd) {
    siginfo_t si = {0};
    if (waitid(P_PIDFD, pidfd, &si, WEXITED) == -1) { perror("waitid"); exit(1); }
    printf("%-14s reaped pid %d: %s %d\n", who, si.si_pid,
           si.si_code == CLD_EXITED ? "exit" : "signal", si.si_status);
}

int main(void) {
    // 1. clone3: the kernel writes the pidfd into args.pidfd before either side runs on.
    int pidfd = -1;
    struct clone_args args = {
        .flags = CLONE_PIDFD,
        .pidfd = (uint64_t)(uintptr_t)&pidfd,
        .exit_signal = SIGCHLD,
    };
    long pid = syscall(SYS_clone3, &args, sizeof args);
    if (pid == -1) { perror("clone3"); return 1; }
    if (pid == 0) _exit(7);                          // child
    printf("clone3         pid %ld, pidfd %d, pidfd_getpid -> %d\n",
           pid, pidfd, pidfd_getpid(pidfd));
    reap("clone3", pidfd);
    close(pidfd);

    // 2. pidfd_spawnp: posix_spawn that hands back a pidfd instead of a PID.
    char *argv[] = {"sleep", "30", NULL};
    int err = pidfd_spawnp(&pidfd, "sleep", NULL, NULL, argv, environ);
    if (err) { errno = err; perror("pidfd_spawnp"); return 1; }
```

```
    printf("pidfd_spawnp   pidfd %d, pidfd_getpid -> %d\n", pidfd, pidfd_getpid(pidfd));
    if (pidfd_send_signal(pidfd, SIGTERM, NULL, 0) == -1) { perror("pidfd_send_signal"); ret
    reap("pidfd_spawnp", pidfd);

    // 3. After the reap the handle is dead, but it can never point at someone else.
    pid_t gone = pidfd_getpid(pidfd);
    printf("pidfd_getpid   after reap -> %d (%s)\n", gone, strerror(errno));
    close(pidfd);
    return 0;
}
```

```
gcc -Wall -Wextra -O2 -o born_with_pidfd born_with_pidfd.c && ./born_with_pidfd
```

```
clone3         pid 113166, pidfd 5, pidfd_getpid -> 113166
clone3         reaped pid 113166: exit 7
pidfd_spawnp   pidfd 5, pidfd_getpid -> 113167
pidfd_spawnp   reaped pid 113167: signal 15
pidfd_getpid   after reap -> -1 (No such process)
```

With `CLONE_PIDFD` the kernel writes the new child's pidfd into `args.pidfd` as part of the same system call that creates the child, so there is no window in which a signal handler or another thread could reap it first. `pidfd_spawnp()` (glibc 2.39) is the same idea with the `posix_spawn` interface: you never even see a PID unless you ask `pidfd_getpid()`. After the reap, `pidfd_getpid()` returns `-1` with `ESRCH`. This is the pattern for libraries and runtimes that do not control the process-wide `SIGCHLD` disposition.

5. Opening is free, using it is permission-checked

```
# pidfd_perms.py - opening a pidfd is not permission to use it: signal 0 and pidfd_getfd
import ctypes, os, signal, subprocess, time

libc = ctypes.CDLL(None, use_errno=True)
def getfd(pidfd, targetfd):                    # glibc 2.36+: pidfd_getfd(2)
    r = libc.pidfd_getfd(pidfd, targetfd, 0)
    if r < 0:
        e = ctypes.get_errno(); raise OSError(e, os.strerror(e))
    return r

# Our own child keeps a file open on fd 3; we copy that fd out of it.
with open("/tmp/pidfd-demo.txt", "w") as f:
    f.write("held open by the child\n")
child = subprocess.Popen(["sh", "-c", "exec 3</tmp/pidfd-demo.txt; exec sleep 30"])
time.sleep(0.2)                                # let the shell open fd 3
fd = os.pidfd_open(child.pid)
mine = getfd(fd, 3)
```

```
print("child fd 3 ->", os.readlink(f"/proc/self/fd/{mine}"), "|", os.read(mine, 64).decode()
signal.pidfd_send_signal(fd, 0)                # signal 0: permission check only
print("signal 0 to child: ok")
child.kill(); child.wait()

# PID 1 belongs to root. Anyone may open a pidfd for it (watching is harmless) ...
one = os.pidfd_open(1)
print("pidfd_open(1): ok, fd", one)
for what, call in (("pidfd_send_signal(sig 0)", lambda: signal.pidfd_send_signal(one, 0)),
                   ("pidfd_getfd(fd 0)", lambda: getfd(one, 0))):
    try:
        call(); print(what, "ok")
    except OSError as e:
        print(f"{what}: {e.__class__.__name__} errno={e.errno} ({os.strerror(e.errno)})")
```

```
python3 pidfd_perms.py
```

```
child fd 3 -> /tmp/pidfd-demo.txt | held open by the child
signal 0 to child: ok
pidfd_open(1): ok, fd 7
pidfd_send_signal(sig 0): PermissionError errno=1 (Operation not permitted)
pidfd_getfd(fd 0): PermissionError errno=1 (Operation not permitted)
```

`pidfd_getfd()` copied file descriptor 3 out of our own child: same user, so ptrace-attach permission holds (no Yama LSM on this host; with `kernel.yama.ptrace_scope=1` only a parent/ancestor may do it, and at `2` or `3` you need `CAP_SYS_PTRACE` or nobody can). For PID 1, owned by root, `pidfd_open()` succeeds (watching the exit of PID 1 is harmless) but signal 0 and `pidfd_getfd()` both fail with `EPERM`. The pidfd is a name, not a capability: the checks are the same ones `kill(2)` and `ptrace(2)` make.

6. From a pidfile to a pidfd you can trust

```
# pidfile_open.py - turn a PID from a pidfile into a pidfd you can trust
import os, subprocess, sys

def starttime(pid):
    """Field 22 of /proc/PID/stat: start time in clock ticks since boot."""
    with open(f"/proc/{pid}/stat") as f:
        return int(f.read().rsplit(")", 1)[1].split()[19])

def open_verified(pid, expected_start):
    """pidfd_open first, *then* check identity: if the PID was recycled before
    the open, the start time read afterwards is the newcomer's and won't match."""
    try:
        fd = os.pidfd_open(pid)
```

```
    except ProcessLookupError:
        return None, "gone (ESRCH)"
    try:
        if starttime(pid) != expected_start:
            os.close(fd); return None, "PID now belongs to a different process"
    except FileNotFoundError:
        os.close(fd); return None, "exited while checking"
    return fd, "verified"

daemon = subprocess.Popen(["sleep", "30"])
record = (daemon.pid, starttime(daemon.pid))      # what a careful pidfile stores
print("pidfile record       ", record)

fd, why = open_verified(*record);           print("same process         ", why)
fd2, why = open_verified(record[0], record[1] + 1)
print("start time mismatch  ", why, "  (simulated reuse)")
daemon.kill(); daemon.wait()
fd3, why = open_verified(*record);          print("after it exited      ", why)
```

```
python3 pidfile_open.py
```

```
pidfile record        (113156, 842250)
same process          verified
start time mismatch   PID now belongs to a different process   (simulated reuse)
after it exited       gone (ESRCH)
```

The order matters: **open first, then verify**. If the PID had been recycled before `pidfd_open()`, the start time read afterwards belongs to the newcomer and does not match the record, so we close and refuse. If it matches, the pidfd was opened on the right process, and from then on the descriptor cannot drift. Checking first and opening second leaves the race open. The middle line simulates reuse with a wrong start time: forcing a real PID wrap on a shared host means burning through the whole PID space (Example 8). On 6.9+ you can also store the pidfs inode from Example 2 instead of the start time.

7. A supervisor: deadlines, escalation and process groups in one `epoll` loop

```
# supervise.py - run jobs with deadlines: one epoll set of pidfds, no SIGCHLD handler, no PI
import os, select, signal, subprocess, time

PIDFD_SIGNAL_PROCESS_GROUP = 4                  # <linux/pidfd.h>, Linux 6.9+

JOBS = {"fast": ["sleep", "0.3"], "fails": ["sh", "-c", "exit 3"],
        "hangs": ["sleep", "60"], "stubborn": ["sh", "-c", "trap '' TERM; sleep 60"]}
DEADLINE, GRACE = 1.0, 0.5                      # seconds
```

```
ep = select.epoll()
jobs = {}                                       # pidfd -> dict
t0 = time.monotonic()
for name, argv in JOBS.items():
    p = subprocess.Popen(argv, start_new_session=True)   # job = its own process group
    fd = os.pidfd_open(p.pid)                   # safe: our child, not yet reaped
    ep.register(fd, select.EPOLLIN)
    jobs[fd] = dict(name=name, popen=p, due=t0 + DEADLINE, next_sig=signal.SIGTERM)

def log(job, msg):
    print(f"{time.monotonic() - t0:5.2f}s  {job['name']:8} pid {job['popen'].pid}  {msg}")

while jobs:
    timeout = max(0.0, min(j["due"] for j in jobs.values()) - time.monotonic())
    for fd, _ in ep.poll(timeout):              # exited: reap through the same handle
        job = jobs.pop(fd)
        info = os.waitid(os.P_PIDFD, fd, os.WEXITED)
        if info.si_code == os.CLD_EXITED:
            job["popen"].returncode = info.si_status
            log(job, f"exit {info.si_status}")
        else:
            job["popen"].returncode = -info.si_status
            log(job, f"killed by {signal.Signals(info.si_status).name}")
        ep.unregister(fd); os.close(fd)
    for fd, job in jobs.items():
        if time.monotonic() >= job["due"]:
            log(job, f"deadline, {job['next_sig'].name} to its process group via pidfd")
            signal.pidfd_send_signal(fd, job["next_sig"], None,   # cannot hit a recycled PI
                                     PIDFD_SIGNAL_PROCESS_GROUP)
            job["next_sig"], job["due"] = signal.SIGKILL, time.monotonic() + GRACE

leftovers = subprocess.run(["pgrep", "-f", "sleep 60"], capture_output=True, text=True).stdo
print("left behind:", leftovers or "nothing")
```

```
python3 supervise.py
```

```
 0.00s  fails    pid 113159  exit 3
 0.30s  fast     pid 113158  exit 0
 1.00s  hangs    pid 113160  deadline, SIGTERM to its process group via pidfd
 1.00s  stubborn pid 113161  deadline, SIGTERM to its process group via pidfd
 1.00s  hangs    pid 113160  killed by SIGTERM
 1.50s  stubborn pid 113161  deadline, SIGKILL to its process group via pidfd
 1.50s  stubborn pid 113161  killed by SIGKILL
left behind: nothing
```

No `SIGCHLD` handler, no `waitpid(-1)` loop, no PID bookkeeping: each job is one pidfd in one `epoll`

set, and the loop's timeout is the nearest deadline. `fails` and `fast` are reaped as they exit. At 1 s `hangs` gets `SIGTERM` and dies; `stubborn` ignores it, so 0.5 s later it gets `SIGKILL`. Every signal goes through the pidfd, so a job that exited a microsecond earlier yields `ESRCH`, never a stranger.

The first draft of this script signalled only the job's own process. Its `sh -c "trap '' TERM; sleep 60"` died to `SIGKILL`, but the `sleep 60` child (which inherited the ignored `SIGTERM`) was left running as an orphan. Starting each job in its own session (`start_new_session=True`, so the job's PID is its process-group ID) and passing `PIDFD_SIGNAL_PROCESS_GROUP` (Linux 6.9+) signals the whole group through the pidfd, and `pgrep` confirms nothing is left behind. On older kernels, or for jobs that create their own sessions, put each job in a cgroup and use `cgroup.kill` instead.

8. How fast PIDs come around

```
# pid_odometer.py - how fast does this host burn through PIDs, and when does the counter wra
import os, time

def fork_and_reap():
    pid = os.fork()
    if pid == 0:
        os._exit(0)                    # child: exit at once
    os.waitpid(pid, 0)                 # parent: reap, which frees the PID number
    return pid

pid_max = int(open("/proc/sys/kernel/pid_max").read())
N = 2000
first = fork_and_reap()
t0 = time.monotonic()
for _ in range(N):
    last = fork_and_reap()
rate = N / (time.monotonic() - t0)
print(f"pid_max          {pid_max}")
print(f"{N} forks       pid {first} -> {last}  (+{last - first}; other processes took the re
print(f"fork+reap rate   {rate:,.0f}/s from one Python loop")
print(f"full wrap        {pid_max / rate / 60:.1f} min at that rate  (with pid_max 32768: {3
```

```
python3 pid_odometer.py
```

```
pid_max          4194304
2000 forks       pid 111144 -> 113154  (+2010; other processes took the rest)
fork+reap rate   2,278/s from one Python loop
full wrap        30.7 min at that rate  (with pid_max 32768: 14.4 s)
```

One single-threaded Python loop advances the PID counter by about 2,300 per second; other processes on the host took a few numbers in between. This host's `pid_max` is 4194304, the value systemd's `50-pid-max.conf` sets on 64-bit systems; at that rate a wrap takes about half an hour. The kernel's own default is 32768 or 1024 per possible CPU, whichever is larger, so hosts without

that sysctl (minimal images, many containers, small machines) wrap in about **14 seconds** at the same rate. A busy CI runner or a fork-heavy shell script does better than one Python loop. “PIDs are effectively unique” is a statement about load, not a guarantee.

9. Who is on the other end of this socket?

```c
// peer_pidfd.c - who is on the other end of this Unix socket? SO_PEERCRED vs SO_PEERPIDFD
#define _GNU_SOURCE
#include <poll.h>
#include <stdio.h>
#include <sys/pidfd.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <unistd.h>

#ifndef SO_PEERPIDFD
#define SO_PEERPIDFD 77              // <asm-generic/socket.h>, Linux 6.5+
#endif

int main(void) {
    struct sockaddr_un a = {.sun_family = AF_UNIX};
    snprintf(a.sun_path, sizeof a.sun_path, "/tmp/peer-pidfd.%d", getpid());
    int srv = socket(AF_UNIX, SOCK_STREAM, 0);
    unlink(a.sun_path);
    if (bind(srv, (void *)&a, sizeof a) || listen(srv, 1)) { perror("bind/listen"); return 1

    pid_t client = fork();
    if (client == 0) {                          // client: connect, linger, exit
        int c = socket(AF_UNIX, SOCK_STREAM, 0);
        connect(c, (void *)&a, sizeof a);
        sleep(1);
        _exit(0);
    }
    int conn = accept(srv, NULL, NULL);

    struct ucred cr; socklen_t len = sizeof cr;
    getsockopt(conn, SOL_SOCKET, SO_PEERCRED, &cr, &len);
    int pfd; len = sizeof pfd;
    if (getsockopt(conn, SOL_SOCKET, SO_PEERPIDFD, &pfd, &len)) { perror("SO_PEERPIDFD"); re
    printf("SO_PEERCRED   pid=%d uid=%d (a number, valid at connect time)\n", cr.pid, cr.uid
    printf("SO_PEERPIDFD  fd=%d -> pid %d (a handle on that exact process)\n", pfd, pidfd_ge

    struct pollfd p = {.fd = pfd, .events = POLLIN};
    int n = poll(&p, 1, 3000);
    printf("peer exit seen by poll(pidfd): %s\n", n == 1 ? "yes" : "no");
```

```
    waitpid(client, NULL, 0);
    unlink(a.sun_path);
    return 0;
}
```

```
gcc -Wall -Wextra -O2 -o peer_pidfd peer_pidfd.c && ./peer_pidfd
```

```
SO_PEERCRED   pid=113169 uid=1000 (a number, valid at connect time)
SO_PEERPIDFD  fd=7 -> pid 113169 (a handle on that exact process)
peer exit seen by poll(pidfd): yes
```

`SO_PEERCRED` returns the peer's PID as a number recorded at `connect()`. By the time a service acts on it (looks up `/proc/PID/exe`, asks polkit whether *that* process may do something), the process may have exited and the number may be someone else's. That is the class of bug behind several polkit/D-Bus authorisation issues. `SO_PEERPIDFD` (Linux 6.5) returns a pidfd for the peer instead; `SCM_PIDFD` does the same per message. Here the server also learns from `poll()` when the client process exits. D-Bus brokers and systemd use these for exactly this reason.

The trap

Trap: "check, then act." `if os.path.exists(f"/proc/{pid}")` followed by `os.kill(pid, sig)` is the race, not a fix for it. So is reading `/proc/PID/comm` and then calling `kill`. The only safe order is to get a handle first and then check that the handle is the right process (Example 6).

Trap: "`pidfd_open()` makes any PID safe." It makes the *descriptor* stable from the moment it is opened. If the PID was already recycled before the call, you now hold a perfectly stable handle on the wrong process. For children: open before you reap, or get the fd at creation. For anything else: verify after opening.

Trap: "a readable pidfd means the process was reaped." Readable (`POLLIN`) means it exited and is a zombie. `POLLHUP` means it was reaped. For a non-child you will usually see both together, because its parent reaps it immediately. For your own child, it stays a zombie until you call `waitid()`.

Trap: "signalling the pidfd stops the job." It signals one process (or one thread). Shell pipelines, `sh -c` wrappers and daemons have children. Use `PIDFD_SIGNAL_PROCESS_GROUP` with one process group per job, or a cgroup and `cgroup.kill` (Example 7).

Trap: "a pidfd is a capability." It is a reference. Holding one does not give you the right to signal or inspect the process; every operation is permission-checked when you make it (Example 5). Passing a pidfd to another process over a socket gives it the same name, not more power.

Notes & Pitfalls

- `waitid(P_PIDFD)` only reaps your own children, exactly like `waitpid()`. For other processes, a pidfd tells you *when* they exit but not their exit status. Linux 6.15 added `PIDFD_INFO_EXIT` to

PIDFD_GET_INFO, which returns the exit status after the process has been reaped (by anyone), as long as you held the pidfd.

- PIDFD_NONBLOCK affects waitid() only (EAGAIN while the process runs). poll is always non-blocking with a zero timeout.
- A process pidfd becomes readable when the **last thread** exits. If the thread-group leader exits early while other threads run, the pidfd stays quiet. PIDFD_THREAD (6.9+) gives a per-thread pidfd that fires when that thread exits.
- The pidfd follows the process across execve(): the process is the same, only its program changed. If you need "the program I started", verify /proc/PID/exe or comm through /proc after opening, as in Example 6.
- systemd, D-Bus brokers, polkit and container runtimes track processes and socket peers by pidfd where the kernel supports it (the 6.13 PIDFD_GET_INFO merge message names systemd, dbus, dbus-daemon and polkit as users). The procps kill on this host (procps-ng 4.0.4) still takes plain numbers, so the shell route remains racy.
- Kernels before 5.3 have none of this except pidfd_send_signal() on a /proc/PID directory fd (5.1). Linux 7.2 is the current stable series; everything in this chapter's tables up to 6.15 is in it.

The boring rule

Treat a PID as a hint, never as an address. Get the pidfd at creation (CLONE_PIDFD, pidfd_spawn), or open it while the process is your unreaped child, or open first and verify identity after. Then do everything through the descriptor: signal with pidfd_send_signal, watch with poll/epoll, reap with waitid(P_PIDFD), and close it when you are done.

Related Commands

- kill, pkill, killall — signal by PID or by name; all number-based from the shell.
- ps -o pid,lstart,cmd, /proc/PID/stat field 22 — the start time used to verify identity.
- timeout — runs a command with a deadline; supervise.py is the multi-job version.
- nsenter — joins a process's namespaces; setns(2) accepts a pidfd for the same job.
- ss -xp — Unix-socket peers by PID; SO_PEERPIDFD is the race-free version for programs.

Try this

1. Run pidfd_basics.py under strace -f -e trace=pidfd_open,pidfd_send_signal,waitid (where strace is available) and match each line of output to a system call.
2. Change watch_nonchild.py to watch a process from pgrep -n sshd (or any long-lived daemon), then restart that daemon. Does the pidfd fire? Does the new instance have a different pidfs inode (6.9+)?
3. In supervise.py, remove PIDFD_SIGNAL_PROCESS_GROUP and start_new_session=True, run it, and find the orphan with pgrep -a -f 'sleep 60'. Then kill it with SIGKILL (it ignores SIGTERM).

4. Write a pidfile format that stores `pid starttime` (or `pid pidfs-inode` on 6.9+) and change a shell `kill $(cat app.pid)` script to call a small helper built from `pidfile_open.py`.
5. On a 6.13+ kernel, call `ioctl(pidfd, PIDFD_GET_INFO, &info)` and compare `info.ppid` and `info.ruid` with `/proc/PID/status`. On 6.15+, request `PIDFD_INFO_EXIT` after a non-child exits and read its exit code.

Verification

- Run unprivileged on this box: Debian 13 (trixie), Linux **6.12.94+**, glibc **2.41**, gcc **14.2.0**, Python **3.13.5**, `uid=1000`, `CapEff` 0. All output above is pasted from those runs; PIDs differ between runs.
- Checked against kernel.org on 2026-10-18: stable **7.2.11**, mainline **7.3-rc7**, longterm **6.18.57** and **6.12.113**. Everything this host runs exists in all of them.
- **Not run:** a real PID wrap (it would mean hundreds of thousands of forks on a shared host; Example 6 simulates the mismatch), `PIDFD_GET_INFO` and `PIDFD_INFO_EXIT` (need 6.13 / 6.15), Yama `ptrace_scope` levels (Yama is not enabled here), and `strace` (not installed; nothing was installed for this chapter). These are written from the man pages and kernel sources below.
- No packages installed, no root, no `sudo`.

Sources

- man7 — `pidfd_open(2)` (zombie/reap conditions, poll semantics, `PIDFD_THREAD`), `pidfd_send_signal(2)` (`ESRCH`, scope flags), `pidfd_getfd(2)`, `clone(2)` `CLONE_PIDFD`, `waitid(2)` `P_PIDFD`, `proc_pid_fdinfo(5)`
- Linux `include/uapi/linux/pidfd.h` — `PIDFD_*` flags, `struct pidfd_info`, `PIDFD_GET_INFO`: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/linux/pidfd.h
- Linux `fs/pidfs.c` (pidfs, poll, fdinfo) and `kernel/pid.c` (`pidfd_open`, `pidfd_getfd`): https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/fs/pidfs.c
- Kernel merge for 6.13 `PIDFD_GET_INFO` ("vfs-6.13.pidfs") and LWN, Two pidfd tweaks: PIDFD_GET_INFO and PIDFD_SELF
- Kernel Newbies — Linux 6.5: `SO_PEERPIDFD`, `SCM_PIDFD`, Linux 6.9: pidfs, thread pidfds, Linux 6.15: `PIDFD_INFO_EXIT`
- glibc NEWS — 2.36 (`pidfd_open`, `pidfd_getfd`, `pidfd_send_signal`) and 2.39 (`pidfd_spawn`, `pidfd_spawnp`, `pidfd_getpid`): https://sourceware.org/git/?p=glibc.git;a=blob_plain;f=NEWS;hb=HEAD
- Python — `os.pidfd_open`, `signal.pidfd_send_signal`, `os.waitid`
- kernel.org release list (versions checked 2026-10-18): https://www.kernel.org/

Focused strace with -e trace Filters

Focused strace with -e trace Filters

Thesis

`strace` is only useful when the noise is gone. `-e trace=` (and friends) cut the stream to the syscalls you care about — file opens, network, signals — so a short reproduction stays readable.

Mental model

```
strace -e trace=openat,close       # only those syscalls
strace -e trace=%file              # class: file ops
strace -e trace=%network           # class: socket ops
strace -f                          # follow children
strace -o out.txt                  # don't mix with program stdout
```

Classes (`%file`, `%network`, `%process`, `%signal`, …) are listed in `man strace`.

Worked example

`strace` is **not installed** on this writing box (policy: no package installs). The patterns below are from the strace man page and everyday ops practice; run them on a lab host that has `strace`.

```
# What files does `ls /tmp` touch?
strace -e trace=openat,close -o /tmp/ls.strace ls /tmp >/dev/null
head /tmp/ls.strace

# Network-only view of a curl
strace -f -e trace=%network -o /tmp/curl.strace curl -s -o /dev/null https://example.com

# Failed opens only (path bugs)
strace -e trace=openat -e fault=openat:error=ENOENT:when=1  # advanced; see man
strace -e trace=openat -Y true  # decode paths; version-dependent
```

Typical `openat` lines look like:

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/tmp", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
```

Trap

`strace -f` on a production service under load. Every syscall traps into the tracer; latency explodes. Prefer a short reproduction, `perf`, or eBPF for live systems. Also: tracing secrets-handling processes can spill passwords into the log — redirect to a protected file.

Boring rule

Start with `-e trace=` for one class or a short syscall list, always `-o` a file, add `-f` only when children matter, and stop as soon as you have the failing path or errno.

Try this

1. Compare `-e trace=%file` vs listing `openat,stat,statx` explicitly.
2. Use `-e write=all -e read=all` carefully to dump buffer contents for a small repro only.
3. When `strace` is unavailable, read `/proc/<pid>/fd` and `/proc/<pid>/stack` as a lighter first pass.

Sources

- `man 1 strace`
- `man 2 openat`

Everything is an fd: signalfd, timerfd and eventfd in one epoll loop

Everything is an fd: signalfd, timerfd and eventfd in one epoll loop

Overview

Thesis: Linux can turn signals, timers, cross-thread wake-ups and process exits into **file descriptors**. Once they are fds, a single `epoll_wait()` loop handles all of them, in order, on one thread. You get no async-signal-safety rules, no `EINTR` storms from handlers, and no self-pipe trick. The price is a small contract you must know: you have to **read** each fd to clear it, and what you read is a **count or a record, not one event per wake-up**.

Descriptor	Since	Replaces	`read()` returns
`signalfd`	2.6.22	signal handlers, the self-pipe trick	one `struct signalfd_siginfo` (128 bytes) per pending signal
`timerfd`	2.6.25	`setitimer`/`SIGALRM`, sleep-based ticks	`uint64_t` expirations since the last read
`eventfd`	2.6.22	a pipe used only to wake another thread	`uint64_t` counter, then reset to 0 (or 1 at a time with `EFD_SEMAPHORE`)
`pidfd`	5.3 (`waitid(P_PIDFD)` 5.4)	`SIGCHLD` + `waitpid`	not read: it becomes *readable* when the process exits

Where you meet these: systemd's `sd-event` loop is built on all four. libuv (`uv_async_send`) and Go's netpoller use an eventfd on Linux to wake a sleeping loop. Seeing `anon_inode:[eventfd]` in `ls -l /proc/PID/fd` of a stuck service is far less mysterious once you have written this loop yourself. pidfds get their own chapter in this part; here they are just one more fd in the loop.

Mental model

```
sigprocmask(BLOCK)   signalfd
timerfd_settime      timerfd
write(efd, 1)        eventfd     epoll_wait   read()/waitid() the ready fd
fork / pidfd_open    pidfd
```

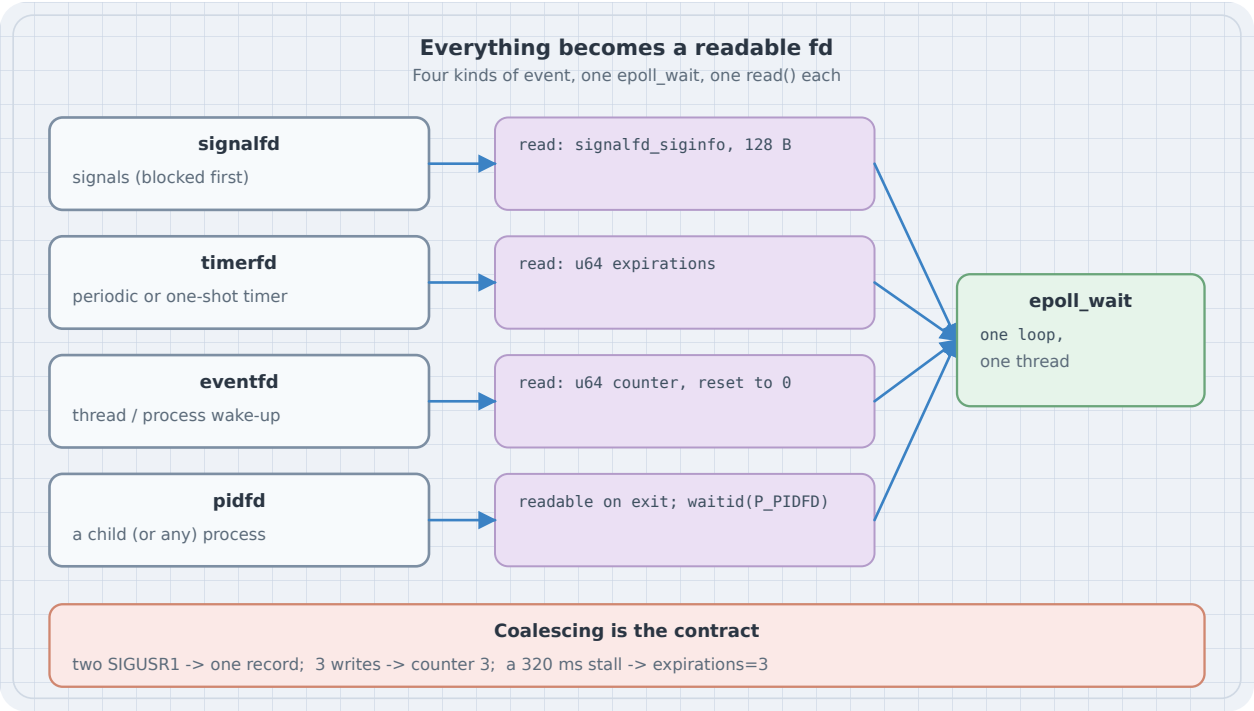


Figure 1: Four event sources become four fds, all registered with one epoll instance

Three rules carry the chapter:

1. **Block before you signalfd.** A signal that is not blocked is still delivered the old way (default action or handler). The signalfd only sees signals that stay *pending*.
2. **Read to clear.** epoll is level-triggered by default. An fd you don't drain is reported again on every `epoll_wait`, and the loop spins.
3. **Counts, not queues.** Standard signals of the same number merge while pending. eventfd adds up writes. timerfd counts expirations you missed. Design handlers for "at least one happened".

Syntax

```
#include <sys/signalfd.h>   // int signalfd(int fd, const sigset_t *mask, int flags);  SFD_N
#include <sys/timerfd.h>    // int timerfd_create(int clockid, int flags);             TFD_
                            // int timerfd_settime(int fd, int flags, const struct itimerspe
#include <sys/eventfd.h>    // int eventfd(unsigned int initval, int flags);          EFD_N
#include <sys/epoll.h>      // epoll_create1, epoll_ctl(EPOLL_CTL_ADD), epoll_wait
// pidfd_open(pid, 0) via syscall(SYS_pidfd_open, ...) or glibc 2.36+ <sys/pidfd.h>
```

Python exposes `os.eventfd` (3.10+) and `os.timerfd_create`/`os.timerfd_settime` (3.13+). It has no `signalfd`; Python code uses `signal.set_wakeup_fd` for the same job.

Safety

- Everything here is unprivileged and only touches your own process and its children.
- Always pass the `*_CLOEXEC` flags. Otherwise every program you `exec` inherits your timers and counters.
- Use `*_NONBLOCK` and read until `EAGAIN` when several records may be queued, as with signalfd.

Examples with Explanations

Run on Linux 6.12, glibc, GCC and Python 3.13. All outputs are genuine.

1. One loop, four kinds of event

```c
// fdloop.c: one epoll loop where signals, a timer, a thread's wake-ups and a
// child's exit all arrive as readable file descriptors.
// build: cc -O2 -Wall -pthread -o fdloop fdloop.c
#define _GNU_SOURCE
#include <errno.h>
#include <pthread.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/epoll.h>
#include <sys/eventfd.h>
#include <sys/signalfd.h>
#include <sys/syscall.h>
#include <sys/timerfd.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

static struct timespec t0;
static double ms(void) {
    struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t);
    return (t.tv_sec - t0.tv_sec) * 1e3 + (t.tv_nsec - t0.tv_nsec) / 1e6;
}
static void die(const char *m) { perror(m); exit(1); }

static void *worker(void *arg) {          // posts three "jobs done" in a row
    int efd = *(int *)arg; uint64_t one = 1;
    usleep(50 * 1000);
    for (int i = 0; i < 3; i++)
```

```
        if (write(efd, &one, sizeof one) != sizeof one) die("eventfd write");
    return NULL;
}

static void show(const char *name, int fd) {
    char path[64], target[128];
    snprintf(path, sizeof path, "/proc/self/fd/%d", fd);
    ssize_t n = readlink(path, target, sizeof target - 1);
    target[n < 0 ? 0 : n] = 0;
    printf("  fd %d %-9s -> %s\n", fd, name, target);
}

int main(void) {
    clock_gettime(CLOCK_MONOTONIC, &t0);

    // 1. Signals: block them first, so they queue for the signalfd instead of
    //    running a handler. Blocking before creating threads makes every thread inherit it.
    sigset_t mask; sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1); sigaddset(&mask, SIGTERM);
    if (sigprocmask(SIG_BLOCK, &mask, NULL)) die("sigprocmask");
    int sfd = signalfd(-1, &mask, SFD_CLOEXEC | SFD_NONBLOCK);

    // 2. A periodic timer: first expiry after 100 ms, then every 100 ms.
    int tfd = timerfd_create(CLOCK_MONOTONIC, TFD_CLOEXEC | TFD_NONBLOCK);
    struct itimerspec its = { .it_interval = {0, 100000000}, .it_value = {0, 100000000} };
    if (timerfd_settime(tfd, 0, &its, NULL)) die("timerfd_settime");

    // 3. A counter another thread can bump.
    int efd = eventfd(0, EFD_CLOEXEC | EFD_NONBLOCK);

    // 4. A child that exits with status 7 after 450 ms, watched through a pidfd.
    pid_t pid = fork();
    if (pid == 0) { usleep(450 * 1000); _exit(7); }
    int pfd = syscall(SYS_pidfd_open, pid, 0);
    if (sfd < 0 || tfd < 0 || efd < 0 || pfd < 0) die("creating fds");

    puts("what the descriptors are:");
    show("signalfd", sfd); show("timerfd", tfd); show("eventfd", efd); show("pidfd", pfd);

    int ep = epoll_create1(EPOLL_CLOEXEC);
    int fds[] = { sfd, tfd, efd, pfd };
    for (int i = 0; i < 4; i++) {
        struct epoll_event ev = { .events = EPOLLIN, .data.fd = fds[i] };
        if (epoll_ctl(ep, EPOLL_CTL_ADD, fds[i], &ev)) die("epoll_ctl");
    }

    pthread_t th; pthread_create(&th, NULL, worker, &efd);
```

```
    kill(getpid(), SIGUSR1); kill(getpid(), SIGUSR1);   // two of the same standard signal

    puts("event loop:");
    int ticks = 0;
    for (;;) {
        struct epoll_event evs[4];
        int n = epoll_wait(ep, evs, 4, -1);
        if (n < 0) { if (errno == EINTR) continue; die("epoll_wait"); }
        for (int i = 0; i < n; i++) {
            int fd = evs[i].data.fd;
            if (fd == tfd) {
                uint64_t exp; if (read(tfd, &exp, sizeof exp) != sizeof exp) die("read timer
                printf("%7.1f ms  timerfd  expirations=%llu\n", ms(), (unsigned long long)ex
                if (++ticks == 2) { usleep(320 * 1000); printf("%7.1f ms  (loop was busy for
            } else if (fd == efd) {
                uint64_t c; if (read(efd, &c, sizeof c) != sizeof c) die("read eventfd");
                printf("%7.1f ms  eventfd  counter=%llu (then reset to 0)\n", ms(), (unsigne
            } else if (fd == pfd) {
                siginfo_t si = {0};
                if (waitid(P_PIDFD, pfd, &si, WEXITED)) die("waitid");
                printf("%7.1f ms  pidfd    child %d exited, status %d\n", ms(), si.si_pid, s
                epoll_ctl(ep, EPOLL_CTL_DEL, pfd, NULL); close(pfd);
                kill(getpid(), SIGTERM);                        // ask ourselves to stop
            } else if (fd == sfd) {
                struct signalfd_siginfo si;
                while (read(sfd, &si, sizeof si) == sizeof si) {
                    printf("%7.1f ms  signalfd %s from pid %u\n", ms(), strsignal(si.ssi_sig
                    if (si.ssi_signo == SIGTERM) { pthread_join(th, NULL); puts("clean exit"
                }
            }
        }
    }
}
```

```
$ cc -O2 -Wall -pthread -o fdloop fdloop.c && ./fdloop
what the descriptors are:
  fd 5 signalfd  -> anon_inode:[signalfd]
  fd 6 timerfd   -> anon_inode:[timerfd]
  fd 7 eventfd   -> anon_inode:[eventfd]
  fd 8 pidfd     -> anon_inode:[pidfd]
event loop:
    0.2 ms  signalfd User defined signal 1 from pid 1582492
   50.6 ms  eventfd  counter=3 (then reset to 0)
  100.1 ms  timerfd  expirations=1
  200.2 ms  timerfd  expirations=1
  520.3 ms  (loop was busy for 320 ms)
```

```
  520.3 ms  timerfd  expirations=3
  520.3 ms  pidfd    child 1582493 exited, status 7
  520.3 ms  signalfd Terminated from pid 1582492
clean exit
```

Line by line:

- **`/proc/self/fd`** shows every object as an anonymous inode named after its type. That is how you spot these in someone else's process with `ls -l /proc/PID/fd`. The exact link name for pidfds varies with kernel version and config.
- **`signalfd … from pid`**: two `kill(SIGUSR1)` calls produced **one** record. Standard signals are a pending *bit*, not a queue (real-time signals, `SIGRTMIN+n`, do queue). The record also carries the sender's PID (`ssi_pid`), which a classic handler only gets through `SA_SIGINFO`.
- **`eventfd counter=3`**: the worker wrote `1` three times before the loop woke. One wake-up, one read, total 3.
- **`timerfd expirations=3`**: the loop stalled for 320 ms after the second tick. The timer did not queue three events; it counted them, and one read returned `3`. A periodic job can use that number to catch up, or decide to skip.
- **`pidfd`**: the child's exit made the pidfd readable. `waitid(P_PIDFD)` reaped it and returned status 7. No `SIGCHLD` handler was involved.
- The process then sent itself `SIGTERM`, which arrived as just another fd event. The loop shut down cleanly, joining the worker thread, instead of being killed half-way through.

2. The kernel's view: `/proc/self/fdinfo`, and `EFD_SEMAPHORE`

```python
# fdinfo.py: the kernel's view of an eventfd and a timerfd, and EFD_SEMAPHORE (Python 3.13+)
import os, time

def fdinfo(fd):
    lines = open(f"/proc/self/fdinfo/{fd}").read().splitlines()
    return [l for l in lines if not l.startswith(("pos:", "flags:", "mnt_id:", "ino:"))]

e = os.eventfd(0, os.EFD_CLOEXEC | os.EFD_NONBLOCK)
os.eventfd_write(e, 5)
print("eventfd after write(5):", fdinfo(e))
print("read ->", os.eventfd_read(e), "| read again ->", end=" ")
try: os.eventfd_read(e)
except BlockingIOError: print("EAGAIN (counter is 0)")

s = os.eventfd(3, os.EFD_SEMAPHORE | os.EFD_NONBLOCK)
got = []
while True:
    try: got.append(os.eventfd_read(s))
    except BlockingIOError: break
print("EFD_SEMAPHORE, initial 3: reads ->", got)
```

```
t = os.timerfd_create(time.CLOCK_MONOTONIC, flags=os.TFD_NONBLOCK)
os.timerfd_settime(t, initial=2.0, interval=0.5)
print("timerfd armed:", fdinfo(t))
```

```
$ python3 fdinfo.py
eventfd after write(5): ['eventfd-count:                5', 'eventfd-id: 117', 'eventfd-sema
read -> 5 | read again -> EAGAIN (counter is 0)
EFD_SEMAPHORE, initial 3: reads -> [1, 1, 1]
timerfd armed: ['clockid: 1', 'ticks: 0', 'settime flags: 00', 'it_value: (1, 999977754)', '
```

`fdinfo` is the debugging view for someone else's process too (`/proc/PID/fdinfo/FD`, readable by the same user). It shows an eventfd's pending count and a timerfd's clock (`1` = `CLOCK_MONOTONIC`), time to next expiry and interval. A plain eventfd hands the whole sum to one reader. With `EFD_SEMAPHORE`, each read takes **1**, so N waiting workers can each claim one unit.

3. The trap: the blocked mask leaks into children

The mask you set for the signalfd is inherited across `fork` **and** `execve`. Anything you start from the loop begins life with those signals blocked:

```
# maskleak.py: a blocked signal mask survives fork+exec, so a child can't be stopped with SI
import signal, subprocess
signal.pthread_sigmask(signal.SIG_BLOCK, {signal.SIGTERM, signal.SIGUSR1})
out = subprocess.run(["grep", "SigBlk", "/proc/self/status"], capture_output=True, text=True
print("child", out.strip(), "(signal N is bit N-1: 0x200 = SIGUSR1, 0x4000 = SIGTERM)")
p = subprocess.Popen(["sleep", "30"]); p.terminate()
try: p.wait(timeout=1); print("sleep stopped by SIGTERM")
except subprocess.TimeoutExpired: print("sleep ignored SIGTERM for 1 s (blocked); killing");
```

```
$ python3 maskleak.py
child SigBlk:   0000000000004200 (signal N is bit N-1: 0x200 = SIGUSR1, 0x4000 = SIGTERM)
sleep ignored SIGTERM for 1 s (blocked); killing
```

`SigBlk` in `/proc/PID/status` is a bitmask (signal N is bit N−1), and the child's shows both signals still blocked. `sleep` never sees `SIGTERM`; only `SIGKILL`, which cannot be blocked, gets rid of it. This is a classic cause of "my service manager can't stop the worker", because the supervisor's signalfd setup leaked into the workers. The fix is to unblock in the child between `fork` and `exec` (`sigprocmask(SIG_SETMASK, &old, NULL)`), or to use `posix_spawn` with `posix_spawnattr_setsigmask`.

The trap

Forgetting that threads have their own masks. Block the signals in `main` **before** creating any thread, as `fdloop.c` does, so every thread inherits the block. If one thread still has `SIGTERM` unblocked,

the kernel may deliver the signal to that thread with its default action, killing the process, and the signalfd never sees it.

Notes & Pitfalls

- **Reads must be the right size.** eventfd and timerfd reads need an 8-byte buffer (`EINVAL` otherwise). signalfd needs room for at least one 128-byte `signalfd_siginfo`.
- **eventfd writes of `0xffffffffffffffff` fail** with `EINVAL`. A write that would overflow the counter blocks, or returns `EAGAIN` with `EFD_NONBLOCK`.
- **`CLOCK_MONOTONIC` vs `CLOCK_REALTIME`.** Use monotonic for intervals. For wall-clock alarms use realtime with `TFD_TIMER_ABSTIME`, and add `TFD_TIMER_CANCEL_ON_SET` to hear about clock jumps (the read fails with `ECANCELED`).
- `CLOCK_BOOTTIME` keeps counting through suspend. The `*_ALARM` clocks can wake the system, but they need `CAP_WAKE_ALARM`.
- **A signalfd sees only the signals in its mask** that are pending for the process or for the reading thread. Signals aimed at another thread (`pthread_kill`) stay with that thread.
- **epoll and fork.** A child shares the parent's epoll instance and its registrations. Create event loops after forking, not before.

The boring rule

Block signals first, before threads. Create every fd with `CLOEXEC|NONBLOCK`. Drain what epoll reports, and treat every read as "N things happened". Restore the signal mask in children before `exec`.

Related Commands

- `pidfd` (this part): process handles, including the identity checks this chapter skips
- `strace -e trace=signalfd4,timerfd_create,timerfd_settime,eventfd2,epoll_wait`: watch a real event loop's setup
- `ls -l /proc/PID/fd`, `cat /proc/PID/fdinfo/FD`, `grep Sig /proc/PID/status`

Try this

1. Change `fdloop.c` to send `SIGRTMIN` twice instead of `SIGUSR1` (add it to the mask). How many signalfd records do you get now?
2. Comment out the `read()` in the timerfd branch and run it under `timeout 1`. Count how often the loop wakes. That is level triggering.
3. Create the eventfd with `EFD_SEMAPHORE` and see what the loop prints for the worker's three writes.
4. Find a long-running process on your machine (`systemd --user`, a browser, an editor) and list which anonymous-inode fds it holds: `ls -l /proc/PID/fd | grep anon_inode`.

Verification

- `fdloop.c` was built with GCC (`-Wall`, no warnings) and run on Linux 6.12. `fdinfo.py` and `maskleak.py` were run on Python 3.13.5 on the same machine. All outputs above are unedited.
- Timings (50, 100, 200, 520 ms) are wall-clock and will vary slightly; the counts (1 SIGUSR1 record, counter 3, expirations 3, status 7) are deterministic.
- Kernel version numbers in the table come from the man pages listed below.

Sources

- signalfd(2): https://man7.org/linux/man-pages/man2/signalfd.2.html
- timerfd_create(2): https://man7.org/linux/man-pages/man2/timerfd_create.2.html
- eventfd(2): https://man7.org/linux/man-pages/man2/eventfd.2.html
- epoll(7): https://man7.org/linux/man-pages/man7/epoll.7.html
- pidfd_open(2) and waitid(2): https://man7.org/linux/man-pages/man2/pidfd_open.2.html, https://man7.org/linux/man-pages/man2/waitid.2.html
- signal(7), standard vs real-time signal queueing: https://man7.org/linux/man-pages/man7/signal.7.html
- proc_pid_fdinfo(5): https://man7.org/linux/man-pages/man5/proc_pid_fdinfo.5.html
- Python `os` module, eventfd and timerfd: https://docs.python.org/3/library/os.html#os.eventfd

How programs get run: execve, binfmt_elf, PT_INTERP and the auxiliary vector

How programs get run: execve, binfmt_elf, PT_INTERP and the auxiliary vector

Overview

Thesis: when you type `./hello`, the kernel does not "run main". `execve(2)` asks a **binary format handler** to recognise the file, maps its `PT_LOAD` segments, and, for a dynamically linked program, also maps the **interpreter** named in `PT_INTERP` (the dynamic loader, `ld-linux-x86-64.so.2` on x86-64 glibc). The kernel then jumps to the *loader's* entry point and hands it a small table of facts, the **auxiliary vector** (auxv). The loader uses auxv to find the program, loads its libraries, runs constructors, and only then calls `main`.

Step	Who	What you can inspect
Recognise the file	kernel: `binfmt_script` (`#!`), `binfmt_elf`, `binfmt_misc`	`head -c4 FILE`, `/proc/sys/fs/binfmt_misc/`
Map the program	kernel (`binfmt_elf`)	`readelf -lW FILE` (`LOAD` rows)
Map the interpreter	kernel, from `PT_INTERP`	`readelf -lW FILE | grep interpreter`
Build the stack: argv, envp, **auxv**	kernel	`LD_SHOW_AUXV=1 PROG`, `/proc/PID/auxv`, `getauxval(3)`
Load libraries, relocate, run init	user space: `ld.so`	`/proc/PID/maps`, `LD_DEBUG=libs`
`_start` → `__libc_start_main` → `main`	libc	your program

Why it matters day to day: the famous "**No such file or directory**" for a file that plainly exists (a binary copied from another distro, an Alpine binary on glibc, a Nix-built binary outside Nix, a script with CRLF in its `#!` line) is the kernel failing to open the *interpreter*, not your file.

Mental model

```
execve(path)   binfmt_script?  "#!interp arg"  → execve(interp, [interp, arg, path, ...])
               binfmt_elf?     "\x7fELF"        → map PT_LOAD, map PT_INTERP, push argv/envp
               binfmt_misc?    registered magic → e.g. qemu-user, wine, java
```

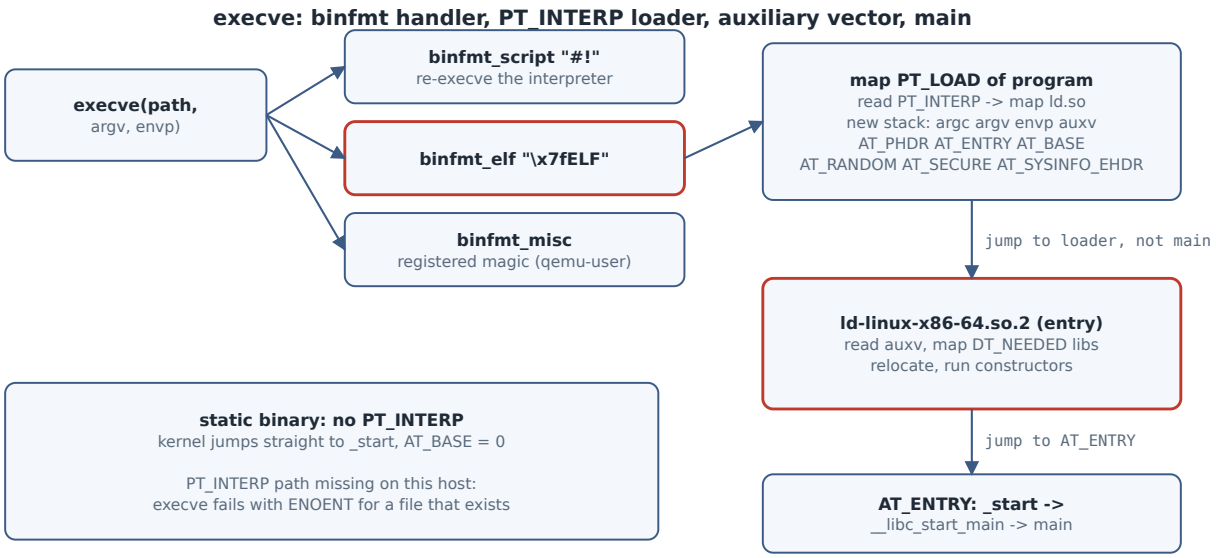


Figure 1: execve hands the file to a binfmt handler; binfmt_elf maps the program and its PT_INTERP loader, builds argv, envp and auxv, and starts the loader, which loads libraries and calls main

```
                jump to ld.so entry (or to _start if static)

        ld.so: read auxv → map DT_NEEDED libs → relocate → init → jump AT_ENTRY → main
```

Three facts carry the chapter:

1. **`PT_INTERP` is a path, baked into the binary.** If it does not exist on this machine, `execve` fails with `ENOENT`, even though the file you named exists.
2. **The loader is just an ELF file you can run directly.** `/lib64/ld-linux-x86-64.so.2 ./prog` loads `prog` itself, which is how `ldd` and Nix's `patchelf`-free wrappers work.
3. **auxv is the kernel→user-space handshake.** `AT_PHDR`/`AT_ENTRY` tell the loader where the program is, `AT_BASE` where the loader is, `AT_RANDOM` seeds the stack canary, `AT_SECURE` says "setuid, ignore `LD_*` variables", and `AT_SYSINFO_EHDR` is the vDSO.

Syntax

```c
#include <unistd.h>
int execve(const char *path, char *const argv[], char *const envp[]);
int execveat(int dirfd, const char *path, char *const argv[], char *const envp[], int flags)

#include <sys/auxv.h>
unsigned long getauxval(unsigned long type);   // AT_PHDR, AT_ENTRY, AT_BASE, AT_RANDOM, AT_
```

```
readelf -hW PROG            # ELF header: Type (EXEC vs DYN = PIE) and entry point
readelf -lW PROG            # program headers: INTERP, LOAD, DYNAMIC, GNU_STACK, GNU_RELRO
LD_SHOW_AUXV=1 PROG         # glibc's loader prints the auxv it received, then runs PROG
od -An -tx8 /proc/PID/auxv  # the raw (type, value) pairs of a running process
```

Safety

- Everything here is read-only inspection of your own binaries and processes. No root is needed.
- The "broken interpreter" demo patches a **copy** of a binary in `/tmp`. Never patch system binaries in place.
- `LD_SHOW_AUXV`, `LD_DEBUG` and `LD_LIBRARY_PATH` are ignored for setuid programs (`AT_SECURE=1`). That is a security feature, not a bug.

Examples with Explanations

1. A PIE, a static binary, and their program headers

A tiny C program prints a few auxv entries with `getauxval(3)`:

```c
#include <stdio.h>
#include <sys/auxv.h>
#include <elf.h>
#include <link.h>
int main(int argc, char **argv) {
    printf("main() reached, argv[0]=%s\n", argv[0]);
    printf("AT_PHDR   = %#lx (program headers of the executable)\n", getauxval(AT_PHDR));
    printf("AT_ENTRY  = %#lx (entry point of the executable)\n", getauxval(AT_ENTRY));
    printf("AT_BASE   = %#lx (load address of the interpreter)\n", getauxval(AT_BASE));
    printf("AT_PAGESZ = %lu\n", getauxval(AT_PAGESZ));
    printf("AT_SECURE = %lu\n", getauxval(AT_SECURE));
    printf("AT_EXECFN = %s\n", (char *)getauxval(AT_EXECFN));
    printf("AT_RANDOM = %#lx (16 random bytes for stack canary / pointer guard)\n", getauxva
    return 0;
}
```

```
$ gcc -O2 -Wall -o hello hello.c
$ gcc -O2 -Wall -static -o hello-static hello.c
$ readelf -hW hello | grep -E 'Type|Entry'
  Type:                              DYN (Position-Independent Executable file)
  Entry point address:               0x1140
$ readelf -lW hello | sed -n '/Program Headers/,/GNU_RELRO/p'
Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
```

```
  PHDR           0x000040 0x0000000000000040 0x0000000000000040 0x000310 0x000310 R   0x8
  INTERP         0x000394 0x0000000000000394 0x0000000000000394 0x00001c 0x00001c R   0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x000000 0x0000000000000000 0x0000000000000000 0x000680 0x000680 R   0x1000
  LOAD           0x001000 0x0000000000001000 0x0000000000001000 0x000235 0x000235 R E 0x1000
  LOAD           0x002000 0x0000000000002000 0x0000000000002000 0x00023c 0x00023c R   0x1000
  LOAD           0x002dd0 0x0000000000003dd0 0x0000000000003dd0 0x000250 0x000258 RW  0x1000
  DYNAMIC        0x002de0 0x0000000000003de0 0x0000000000003de0 0x0001e0 0x0001e0 RW  0x8
  NOTE           0x000350 0x0000000000000350 0x0000000000000350 0x000020 0x000020 R   0x8
  NOTE           0x000370 0x0000000000000370 0x0000000000000370 0x000024 0x000024 R   0x4
  NOTE           0x00221c 0x000000000000221c 0x000000000000221c 0x000020 0x000020 R   0x4
  GNU_PROPERTY   0x000350 0x0000000000000350 0x0000000000000350 0x000020 0x000020 R   0x8
  GNU_EH_FRAME   0x002148 0x0000000000002148 0x0000000000002148 0x00002c 0x00002c R   0x4
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW  0x10
  GNU_RELRO      0x002dd0 0x0000000000003dd0 0x0000000000003dd0 0x000230 0x000230 R   0x1
```

Read it top to bottom:

- **`Type: DYN`** is a position-independent executable. Its addresses start at 0 and the kernel picks a random base (ASLR).
- **`INTERP`** is 0x1c = 28 bytes: the 27-character path plus a NUL. That one string decides which loader runs.
- Four **`LOAD`** segments with flags `R`, `R E`, `R`, `RW`: headers, code, read-only data, writable data. The kernel `mmap`s each with those permissions.
- **`GNU_STACK RW`** (no `E`) asks for a non-executable stack. **`GNU_RELRO`** is a range the loader makes read-only after relocation.

Now run both:

```
$ ./hello
main() reached, argv[0]=./hello
AT_PHDR   = 0x55f3a870c040 (program headers of the executable)
AT_ENTRY  = 0x55f3a870d140 (entry point of the executable)
AT_BASE   = 0x7f42f153d000 (load address of the interpreter)
AT_PAGESZ = 4096
AT_SECURE = 0
AT_EXECFN = ./hello
AT_RANDOM = 0x7ffe54264479 (16 random bytes for stack canary / pointer guard)

$ ./hello-static
main() reached, argv[0]=./hello-static
AT_PHDR   = 0x400040 (program headers of the executable)
AT_ENTRY  = 0x4016a0 (entry point of the executable)
AT_BASE   = 0 (load address of the interpreter)
AT_PAGESZ = 4096
AT_SECURE = 0
AT_EXECFN = ./hello-static
```

```
AT_RANDOM = 0x7ffd0c796b29 (16 random bytes for stack canary / pointer guard)
$ readelf -hW hello-static | grep Type; readelf -lW hello-static | grep -c INTERP
  Type:                              EXEC (Executable file)
0
```

The PIE's `AT_BASE` is where the kernel mapped `ld-linux-x86-64.so.2`. The static binary is `EXEC` at the fixed address 0x400000, has **no** `INTERP` header, and gets `AT_BASE = 0`: the kernel jumped straight to its `_start`. The addresses change on every run (ASLR); the `0x...040` offset of `AT_PHDR` and the `...140` offset of `AT_ENTRY` match `readelf`'s `PHDR` offset (0x40) and entry point (0x1140).

2. Everything the kernel told the loader: `LD_SHOW_AUXV`

```
$ LD_SHOW_AUXV=1 ./hello | head -22
AT_SYSINFO_EHDR:      0x7fa1b8fec000
AT_MINSIGSTKSZ:       11952
AT_HWCAP:             1fabfbff
AT_PAGESZ:            4096
AT_CLKTCK:            100
AT_PHDR:              0x55d1fe9a0040
AT_PHENT:             56
AT_PHNUM:             14
AT_BASE:              0x7fa1b8fee000
AT_FLAGS:             0x0
AT_ENTRY:             0x55d1fe9a1140
AT_UID:               1000
AT_EUID:              1000
AT_GID:               1000
AT_EGID:              1000
AT_SECURE:            0
AT_RANDOM:            0x7ffc82d7b3e9
AT_HWCAP2:            0x2
AT_EXECFN:            ./hello
AT_PLATFORM:          x86_64
AT_RSEQ_FEATURE_SIZE: 28
AT_RSEQ_ALIGN:        32
```

The entries worth knowing:

Entry	Meaning
`AT_SYSINFO_EHDR`	address of the vDSO's ELF header (how `clock_gettime` avoids a syscall)
`AT_HWCAP`, `AT_HWCAP2`, `AT_PLATFORM`	CPU feature bits; glibc uses them to pick optimised `memcpy` and `glibc-hwcaps` library directories
`AT_PHDR`, `AT_PHENT`, `AT_PHNUM`	where the program's headers are, their size (56) and count (14, as `readelf` said)

Entry	Meaning
`AT_BASE`, `AT_ENTRY`	the loader's base and the program's entry point
`AT_UID`..`AT_EGID`, `AT_SECURE`	real and effective IDs; `AT_SECURE=1` for setuid/setgid/file-capability programs
`AT_RANDOM`	16 kernel-random bytes on the stack, used for the stack protector canary and pointer mangling
`AT_EXECFN`	the path passed to `execve`
`AT_MINSIGSTKSZ`, `AT_RSEQ_*`	newer entries: minimum signal stack for this CPU (AVX-512 state is big), restartable-sequences layout

3. The loader is a program

```
$ ls -l /lib64/ld-linux-x86-64.so.2 | awk '{print $1, $(NF-2), $(NF-1), $NF}'
lrwxrwxrwx /lib64/ld-linux-x86-64.so.2 -> ../lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
$ /lib64/ld-linux-x86-64.so.2 ./hello
main() reached, argv[0]=./hello
AT_PHDR   = 0x7fdc18bf1040 (program headers of the executable)
AT_ENTRY  = 0x7fdc18bf2140 (entry point of the executable)
AT_BASE   = 0 (load address of the interpreter)
AT_PAGESZ = 4096
AT_SECURE = 0
AT_EXECFN = ./hello
AT_RANDOM = 0x7ffd68b77d09 (16 random bytes for stack canary / pointer guard)
```

Run that way, the *kernel* executed `ld-linux-x86-64.so.2` (so from the kernel's point of view there is no interpreter: `AT_BASE = 0`), and the loader mapped `./hello` itself. This is the escape hatch when `PT_INTERP` points somewhere that does not exist. It is not a way around `noexec` mounts for ELF files: the loader has to `mmap` the program with `PROT_EXEC`, and the kernel refuses that on a `noexec` mount. Script interpreters are different (`python3 /mnt/noexec/x.py` only reads the file), which is why `noexec` is a speed bump, not a sandbox.

Here is the result in a running process, filtered to the interesting mappings of a small program that prints its own `/proc/self/maps`:

```
$ ./maps | grep -E 'ld-linux|libc|vdso|stack|/maps' | awk '{print $1, $2, $6}'
560196c94000-560196c95000 r--p /tmp/execlab/maps
560196c95000-560196c96000 r-xp /tmp/execlab/maps
560196c96000-560196c97000 r--p /tmp/execlab/maps
560196c97000-560196c98000 r--p /tmp/execlab/maps
560196c98000-560196c99000 rw-p /tmp/execlab/maps
7f7411605000-7f741162d000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6
7f741162d000-7f7411790000 r-xp /usr/lib/x86_64-linux-gnu/libc.so.6
7f7411790000-7f74117e6000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6
```

```
7f74117e6000-7f74117ea000 r--p /usr/lib/x86_64-linux-gnu/libc.so.6
7f74117ea000-7f74117ec000 rw-p /usr/lib/x86_64-linux-gnu/libc.so.6
7f741180a000-7f741180c000 r-xp [vdso]
7f741180c000-7f741180d000 r--p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f741180d000-7f7411835000 r-xp /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f7411835000-7f7411840000 r--p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f7411840000-7f7411842000 r--p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7f7411842000-7f7411843000 rw-p /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7fff9b777000-7fff9b799000 rw-p [stack]
```

The program's five mappings sit at a random `0x55...` base, libc and the loader near the top of the address space, the `[vdso]` from `AT_SYSINFO_EHDR`, and `[stack]`, where argv, envp and auxv were written.

4. "No such file or directory" for a file that exists

Copy the binary and change one byte-for-byte equal-length string in `PT_INTERP`, as if it came from a system with a different loader path:

```
$ readelf -lW hello-badinterp | grep interpreter
      [Requesting program interpreter: /lib64/ld-gonex-x86-64.so.2]
$ ls -l hello-badinterp
-rwxr-xr-x 16008 hello-badinterp
$ ./hello-badinterp
run.sh: line 4: ./hello-badinterp: cannot execute: required file not found
exit=127
$ python3 -c "import os; os.execv(\"./hello-badinterp\", [\"x\"])"
FileNotFoundError: [Errno 2] No such file or directory
$ head -1 s.sh; ./s.sh
#!/no/such/shell
run.sh: line 6: ./s.sh: cannot execute: required file not found
exit=127
```

The file is there and executable, yet `execve` returns `ENOENT` (bash 5.2 now words this as "required file not found"; older shells print the misleading "No such file or directory"). The missing file is `/lib64/ld-gonex-x86-64.so.2`. A `#!` line naming a missing interpreter fails the same way, because `binfmt_script` turns it into a second `execve` of the interpreter. A Windows-edited script whose first line is `#!/bin/sh\r` fails for the same reason: the interpreter it asks for is `/bin/sh\r`.

Fixes, in order of preference: install a build for this platform; run it through the right loader (`/path/to/ld.so ./prog`); or rewrite the header with `patchelf --set-interpreter` (on a copy).

The trap: `ldd` runs code

`ldd` is a shell script that runs the program's loader with `LD_TRACE_LOADED_OBJECTS=1`. With a hostile or simply foreign binary, a custom `PT_INTERP` means "running `ldd` on it" executes an attacker-chosen loader. The ldd(1) man page says so. For untrusted files, read the headers instead: `readelf -dW FILE | grep NEEDED` and `readelf -lW FILE | grep interpreter` never execute anything.

Notes and pitfalls

- **Order of handlers.** The kernel tries the registered formats in turn. `binfmt_misc` entries (`/proc/sys/fs/binfmt_misc/`, usually set up by systemd-binfmt) are how `qemu-user` lets an arm64 binary "just run" on x86-64. Inside containers that directory is often not mounted.
- **`#!` limits.** The kernel reads only the first line, up to 255 bytes (`BINPRM_BUF_SIZE` 256), and passes **at most one** argument: `#!/usr/bin/env python3 -u` passes `"python3 -u"` as one word. `env -S` exists to split it.
- **Static-PIE** binaries (`-static-pie`) are `DYN` with no `INTERP`: they relocate themselves.
- **Executing from an fd.** `execveat(fd, "", ..., AT_EMPTY_PATH)` and `fexecve(3)` run a file you already opened, which is how `memfd_create` + exec works. `/proc/PID/exe` then points at `/memfd:...`.
- **What `argv[0]` is** is entirely up to the caller of `execve`; `AT_EXECFN` and `/proc/self/exe` are the honest answers.
- **Setuid binaries** get `AT_SECURE=1`; the loader then ignores `LD_PRELOAD` paths with slashes, `LD_LIBRARY_PATH` and `LD_SHOW_AUXV`.

The boring rule

When a binary "isn't found", run `readelf -lW FILE | grep interpreter` and check that path. Read auxv with `LD_SHOW_AUXV=1` or `getauxval`, never by walking past `envp` yourself. Use `readelf`, not `ldd`, on files you do not trust.

Related Commands

- `readelf -hW`, `readelf -lW`, `readelf -dW` (binutils)
- `LD_SHOW_AUXV=1`, `LD_DEBUG=libs`, `/lib64/ld-linux-x86-64.so.2 --list PROG`
- `cat /proc/PID/maps`, `od -An -tx8 /proc/PID/auxv`, `ls -l /proc/PID/exe`
- `ls /proc/sys/fs/binfmt_misc/`, `systemd-binfmt`

Try this

1. Run `LD_SHOW_AUXV=1 /bin/true` and `LD_SHOW_AUXV=1 ./hello-static`. Which one prints nothing, and why?

2. Run `LD_DEBUG=libs ./hello 2>&1 | head` and find where the loader looks for `libc.so.6`.
3. Compare `readelf -lW` for a binary built with `-z execstack` and without. What changes in `GNU_STACK`?
4. Write a script whose first line is `#!/usr/bin/env python3 -u`. What error do you get, and how does `#!/usr/bin/env -S python3 -u` fix it?

Verification

- `hello.c` and `maps.c` were built with GCC (`-O2 -Wall`, no warnings) against glibc 2.41 (Debian 13) and run on Linux 6.12 x86-64. The bad-interpreter demo ran under bash 5.2.37 and Python 3.13.
- All outputs above are unedited. Addresses differ from run to run because of ASLR; offsets, sizes, counts and error codes are stable.

Sources

- execve(2): https://man7.org/linux/man-pages/man2/execve.2.html
- getauxval(3): https://man7.org/linux/man-pages/man3/getauxval.3.html
- ld.so(8) (`LD_SHOW_AUXV`, `AT_SECURE`, running the loader directly): https://man7.org/linux/man-pages/man8/ld.so.8.html
- ldd(1), the security note: https://man7.org/linux/man-pages/man1/ldd.1.html
- elf(5): https://man7.org/linux/man-pages/man5/elf.5.html
- Kernel docs, binfmt_misc: https://docs.kernel.org/admin-guide/binfmt-misc.html
- Kernel source, `fs/binfmt_elf.c` (`load_elf_binary`, `create_elf_tables`): https://github.com/torvalds/linux/blob/master/fs/binfmt_elf.c
- LWN, "How programs get run" and "How programs get run: ELF binaries" (David Drysdale): https://lwn.net/Articles/630727/, https://lwn.net/Articles/631631/

Threads Are Just clone() Flags: What CLONE_VM, FILES, FS and SIGHAND Share

Threads Are Just clone() Flags: What CLONE_VM, FILES, FS and SIGHAND Share

Learning goals

- See that Linux has no separate "thread" object. `fork()`, `pthread_create()` and container runtimes are all `clone()` with different **sharing flags**.
- Prove what each of `CLONE_VM`, `CLONE_FILES`, `CLONE_FS` and `CLONE_SIGHAND` shares, by changing state in a child and inspecting it in the parent.
- Call `clone3()` directly with `struct clone_args`.
- Read the exact flag set glibc passes for a pthread, and the kernel rules that stop nonsensical combinations.

Version note: run on Linux **6.12** with gcc 14.2 and glibc 2.41 (Debian 13), as an ordinary user. `strace` is not installed on the box, so the glibc flag set is quoted from source instead of traced.

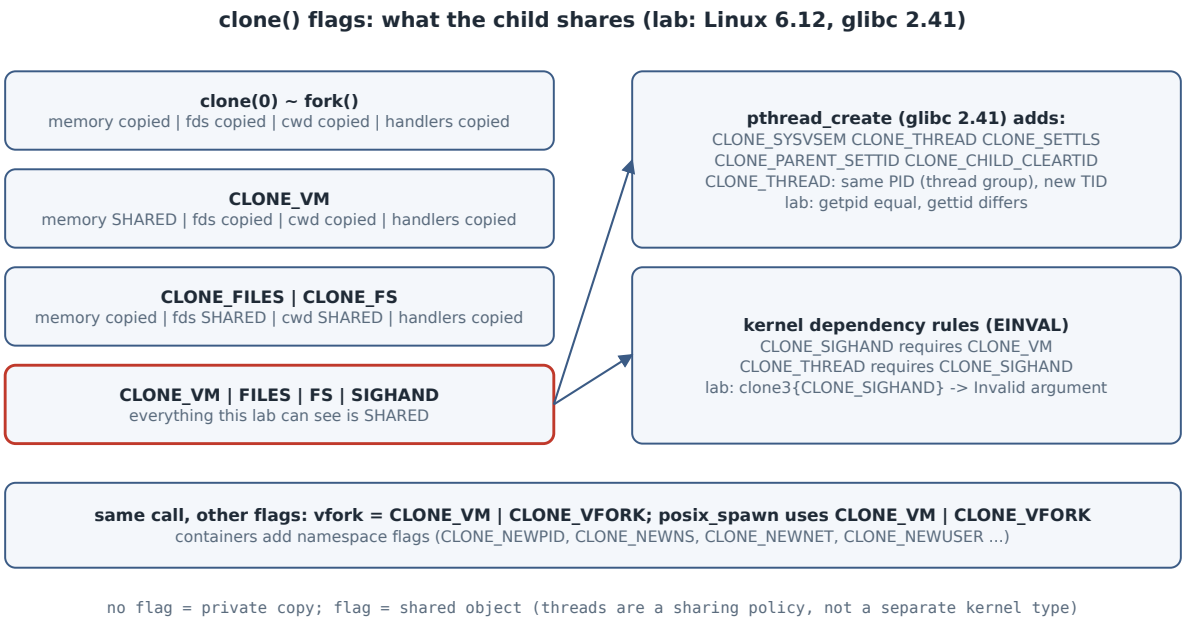


Figure 1: clone() flags decide what a child shares with its parent.

The flags in this lab

From clone(2):

Flag	If set, the child shares the parent's…
`CLONE_VM`	memory (same address space; writes are visible both ways)
`CLONE_FILES`	file-descriptor table (open/close in one affects the other)
`CLONE_FS`	filesystem info: root, cwd, umask
`CLONE_SIGHAND`	table of signal handlers (requires `CLONE_VM`)
`CLONE_THREAD`	thread group, so the same PID with a different TID (requires `CLONE_SIGHAND`)

Without a flag, the child gets a **copy** of that resource, which is exactly what `fork()` gives you.

Lab 1: change it in the child, look in the parent

The child sets a global, closes an fd, `chdir`s to `/tmp`, and installs a `SIGUSR1` handler. The parent then checks which of those changes it can see.

```c
// What does a clone() child share with its parent? Try four flag sets.
#define _GNU_SOURCE
#include <fcntl.h>
#include <linux/sched.h>    /* struct clone_args */
#include <sched.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/syscall.h>
#include <sys/wait.h>
#include <unistd.h>
#include <errno.h>

static int counter = 0;
static int fd = -1;
static void on_usr1(int s) { (void)s; }

static int child(void *arg) {
    (void)arg;
    counter = 42;                   /* memory   (CLONE_VM)      */
    close(fd);                      /* fd table (CLONE_FILES)   */
    chdir("/tmp");                  /* cwd      (CLONE_FS)      */
    signal(SIGUSR1, on_usr1);       /* handlers (CLONE_SIGHAND) */
    return 0;
}
```

```
static void report(const char *name, int flags) {
    counter = 0;
    fd = open("/dev/null", O_RDONLY);
    chdir("/");
    signal(SIGUSR1, SIG_DFL);

    size_t sz = 1 << 16;
    char *stack = malloc(sz);
    pid_t pid = clone(child, stack + sz, flags | SIGCHLD, NULL);
    waitpid(pid, NULL, 0);
    free(stack);

    char cwd[64]; getcwd(cwd, sizeof cwd);
    struct sigaction sa; sigaction(SIGUSR1, NULL, &sa);
    printf("%-34s memory:%-7s fds:%-7s cwd:%-7s sighand:%s\n", name,
           counter == 42 ? "shared" : "copied",
           fcntl(fd, F_GETFD) == -1 ? "shared" : "copied",
           strcmp(cwd, "/tmp") == 0 ? "shared" : "copied",
           sa.sa_handler == on_usr1 ? "shared" : "copied");
    if (fcntl(fd, F_GETFD) != -1) close(fd);
}

int main(void) {
    report("0 (like fork)", 0);
    report("CLONE_VM", CLONE_VM);
    report("CLONE_FILES|CLONE_FS", CLONE_FILES | CLONE_FS);
    report("CLONE_VM|FILES|FS|SIGHAND", CLONE_VM | CLONE_FILES | CLONE_FS | CLONE_SIGHAND);

    /* clone3: same idea, arguments in a struct; returns like fork(). */
    struct clone_args ca = { .flags = CLONE_FILES, .exit_signal = SIGCHLD };
    fd = open("/dev/null", O_RDONLY);
    long r = syscall(SYS_clone3, &ca, sizeof ca);
    if (r == 0) { close(fd); _exit(0); }
    waitpid(r, NULL, 0);
    printf("%-34s fds:%s (child closed fd %d)\n", "clone3 {.flags=CLONE_FILES}",
           fcntl(fd, F_GETFD) == -1 ? "shared" : "copied", fd);

    /* What pthread_create actually asks for: */
    printf("\nthread-like set rejected without CLONE_VM? ");
    struct clone_args bad = { .flags = CLONE_SIGHAND, .exit_signal = SIGCHLD };
    r = syscall(SYS_clone3, &bad, sizeof bad);
    if (r == 0) _exit(0);
    if (r > 0) { waitpid(r, NULL, 0); puts("no (accepted)"); }
    else printf("yes: %s\n", strerror(errno));
    return 0;
}
```

```
$ gcc -O0 -Wall -o shares shares.c && ./shares
0 (like fork)                       memory:copied  fds:copied  cwd:copied  sighand:copied
CLONE_VM                            memory:shared  fds:copied  cwd:copied  sighand:copied
CLONE_FILES|CLONE_FS                memory:copied  fds:shared  cwd:shared  sighand:copied
CLONE_VM|FILES|FS|SIGHAND           memory:shared  fds:shared  cwd:shared  sighand:shared
clone3 {.flags=CLONE_FILES}         fds:shared (child closed fd 5)

thread-like set rejected without CLONE_VM? yes: Invalid argument
```

Each row matches the table. The four "thread-ish" flags together share everything this lab can see, which is what makes the child feel like a thread. `clone3()` takes the same flags in a `struct clone_args`, and the kernel enforces the dependency rules: `CLONE_SIGHAND` without `CLONE_VM` gives `EINVAL` ("Since Linux 2.6.0", per clone(2)). A shared handler table over separate memory would point at handler addresses that mean nothing in the other address space.

Note that the glibc `clone()` wrapper needs a caller-allocated stack, here `malloc`ed, passed as its *top* because the stack grows down on x86-64. Raw `clone3()` with no stack behaves like `fork()` and uses a copy-on-write duplicate of the current stack. That is safe only because this example does not pass `CLONE_VM`.

Lab 2: what CLONE_THREAD adds

glibc 2.41, nptl/pthread_create.c:

```
const int clone_flags = (CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SYSVSEM
                         | CLONE_SIGHAND | CLONE_THREAD
                         | CLONE_SETTLS | CLONE_PARENT_SETTID
                         | CLONE_CHILD_CLEARTID
                         | 0);
```

Beyond Lab 1's four flags:

- `CLONE_SYSVSEM` shares System V semaphore undo lists.
- `CLONE_SETTLS` gives the thread its own thread-local storage.
- `CLONE_PARENT_SETTID`/`CLONE_CHILD_CLEARTID` let `pthread_join` find out when the thread exits, through a futex on the TID word.
- `CLONE_THREAD` makes the child part of the same thread group:

```
// A pthread is a clone with CLONE_THREAD: same PID (thread group), new TID.
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

static void *worker(void *arg) {
```

```
    (void)arg;
    printf("thread: getpid=%d gettid=%d\n", getpid(), gettid());
    return NULL;
}

int main(void) {
    printf("main:   getpid=%d gettid=%d\n", getpid(), gettid());
    pthread_t t;
    pthread_create(&t, NULL, worker, NULL);
    pthread_join(t, NULL);
    fflush(stdout);                 /* don't duplicate buffered output */
    pid_t c = fork();
    if (c == 0) { printf("fork:   getpid=%d gettid=%d\n", getpid(), gettid()); fflush(stdout
    waitpid(c, NULL, 0);
    return 0;
}
```

```
$ gcc -Wall -o thread thread.c && ./thread
main:   getpid=1685152 gettid=1685152
thread: getpid=1685152 gettid=1685154
fork:   getpid=1685155 gettid=1685155
```

`getpid()` returns the **thread-group ID**, so it is the same for the main thread and the pthread, while `gettid()` differs. The forked child has a new PID that equals its TID. This is why `ps` shows one process and `ps -L` or `/proc/PID/task/` shows the threads.

Where else these flags show up

- **Containers** use the same call with *namespace* flags (`CLONE_NEWPID`, `CLONE_NEWNS`, `CLONE_NEWNET`, `CLONE_NEWUSER`, …) instead of, or alongside, sharing flags. That is the subject of the namespace chapters.
- **`vfork()`** is `CLONE_VM | CLONE_VFORK`: the child borrows the parent's memory until it `execve`s.
- **`posix_spawn`** in glibc uses `CLONE_VM | CLONE_VFORK` internally for speed.

Try this

1. Add `CLONE_THREAD` to the last `report()` row. Why does `waitpid()` now fail, and what does clone(2) say about the exit signal of a thread?
2. Pass `CLONE_THREAD` without `CLONE_SIGHAND` to `clone3()` and confirm the second `EINVAL` rule.
3. Run `./thread` with `sleep(30)` in the worker and compare `ls /proc/<pid>/task` against `ps -o nlwp <pid>`.

Sources

- clone(2), flags and `EINVAL` conditions: https://man7.org/linux/man-pages/man2/clone.2.html
- glibc 2.41 `nptl/pthread_create.c` (`clone_flags`): https://sourceware.org/git/?p=glibc.git;a=blob;f=nptl/pthread_create.c;hb=refs/heads/release/2.41/master
- gettid(2) and getpid(2): https://man7.org/linux/man-pages/man2/gettid.2.html
- vfork(2): https://man7.org/linux/man-pages/man2/vfork.2.html
- glibc 2.41 `sysdeps/unix/sysv/linux/spawni.c` (posix_spawn uses `CLONE_VM | CLONE_VFORK`): https://sourceware.org/git/?p=glibc.git;a=blob;f=sysdeps/unix/sysv/linux/spawni.c;hb=refs/heads/release/2.41/master
- Linux `include/uapi/linux/sched.h` (`struct clone_args`, flag values): https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/linux/sched.h?h=v6.12

System Monitoring

Intro

Intro

Measure subsystem health over time and debug *why* a host is slow or stuck: virtual memory, disks, CPUs, open files, and syscall/library traces.

Commands in this part

Command	Role
`vmstat`	vmstat reports virtual memory, process run/block queues, CPU, and I/O counters in a compact table.
`iostat`	iostat (from the sysstat package) reports CPU utilization and per-device I/O rates, queue depth, await, and utilization.
`mpstat`	mpstat (sysstat) reports CPU utilization, optionally per logical CPU, so you can see single-thread bottlenecks,…
`sar`	sar (System Activity Reporter, sysstat) prints live and historical metrics: CPU, memory, load, disk, network, and more.
`lsof`	lsof lists open files.
`strace`	strace traces system calls and signals for a process.
`perf`	perf is Linux's primary performance analysis suite.
`btop`	btop is a modern, interactive resource monitor with a polished TUI: CPU (often per-core with clocks/temps when…
`ltrace`	ltrace intercepts and prints dynamic library calls of a process (similar to how strace prints syscalls).
`pmap`	pmap prints a process's memory map (size, RSS, dirty per mapping); the first step in hunting native leaks that heap profilers miss.

Command	Role
THP (`/sys/kernel/mm/transparent_hugepage`)	Transparent Huge Pages: read the policy, find who holds 2 MiB pages (`AnonHugePages`), and opt in or out with `madvise`/`prctl`.
PSI (`/proc/pressure`, cgroup `*.pressure`)	Pressure Stall Information: share of time tasks are stalled on CPU, memory or I/O — `some` vs `full`, averages and triggers.
vDSO (`[vdso]`, `[vvar]`)	The kernel-provided shared object that answers `clock_gettime`, `gettimeofday`, `time`, `getcpu` and `getrandom` without a syscall — why `strace` and seccomp never see them, and when they fall back.

Suggested starting points

1. Load vs I/O vs CPU: `vmstat`, `iostat`, `mpstat`, `sar`.
2. Who holds a file or port: `lsof`.
3. Why a process fails: `strace` (syscalls), `ltrace` (library calls).
4. Hotspots: `perf` (and interactive `btop` for a dashboard).
5. RSS keeps growing: `pmap` (which mapping?), then `strace`/`bpftrace` and `gdb` (who maps it?).
6. RSS jumps or latency spikes after a THP policy change: Transparent Huge Pages (`AnonHugePages`, `/proc/vmstat` `thp_*`, `compact_stall`).
7. Latency without obvious saturation: `/proc/pressure/*` and the service cgroup's `*.pressure` (`some` vs `full`).
8. Timestamps look slow, or `strace` shows no clock reads: the vDSO (`[vdso]` in `/proc/PID/maps`, `current_clocksource`).

Related parts

- Processes and jobs — PIDs and signals
- Storage and filesystems — disk health and space
- Networking — `ss`/`tcpdump` for network-bound issues

Continue with the individual command pages in this part.

vmstat

Overview

`vmstat` reports virtual memory, process run/block queues, CPU, and I/O counters in a compact table. Ideal for a quick terminal loop that answers: **CPU-bound, RAM pressure, or disk wait?** Prefer sample mode (`delay count`) over a single line — the first line is often averages since boot.

Syntax

```
vmstat [options] [delay [count]]
```

Common Options

Option	Description
`delay count`	Sample every *delay* seconds, *count* times
`-w`	Wide columns (avoid truncated fields)
`-S k|K|m|M`	Unit scaling for memory fields
`-a`	Active/inactive memory instead of buff/cache split detail
`-s`	One-shot event counter summary
`-d`	Disk statistics
`-D`	Disk summary totals
`-t`	Timestamp each sample line
`-n`	Header once (not every screenful)
`-f`	Fork count since boot

Install note: `vmstat` is from **procps** (preinstalled on Ubuntu). Disk/historical depth may still push you to **sysstat** (`iostat`, `sar`).

Key Use Cases

1. Live "why is it slow?" triage loop
2. Detect swap thrash (`si`/`so`)

3. Spot I/O wait vs CPU user/system
4. VM steal time (`st`) on cloud guests

Examples with Explanations

Live sample (default triage)

```
vmstat 1 10
vmstat -w 1 5
```

One-second samples, ten lines (or five wide). **Ignore the first data line** when using a delay — it is often since-boot averages; subsequent lines are interval deltas.

Timestamped capture for logs

```
vmstat -t 2 30 | tee /tmp/vmstat.out
```

Useful during an incident window you will review later.

Memory summary counters

```
vmstat -s
vmstat -s -S M
```

Cumulative counters (forks, pages paged in/out, etc.) — good for a snapshot, not a live rate loop.

Disk stats mode

```
vmstat -d 1 5
```

Per-disk reads/writes. For modern extended fields (`await`, `%util`), prefer `iostat -xz`.

Active / inactive memory view

```
vmstat -a 1 5
```

Highlights reclaimable cache vs active pages when diagnosing memory pressure.

Correlate with siblings

```
vmstat 1 5
iostat -xz 1 5
mpstat -P ALL 1 5
free -h
```

vmstat for system pulse; iostat for which disk; mpstat for per-CPU imbalance; free for human memory totals.

Understanding Output

Classic columns (names vary slightly by version/width):

Area	Fields	Operator hints
Procs	`r`, `b`	`r` > CPU count → runnable contention; `b` blocked (often I/O)
Memory	`swpd`, `free`, `buff`, `cache`	Rising `swpd` + low free → pressure (also check `free -h`)
Swap	`si`, `so`	Sustained non-zero swap in/out → RAM shortage / thrash
IO	`bi`, `bo`	Blocks in/out (not full disk latency picture)
System	`in`, `cs`	Interrupts, context switches (high `cs` can mean chatty scheduling)
CPU	`us`, `sy`, `id`, `wa`, `st`	`%` of time: user, kernel, idle, **I/O wait**, **steal** (hypervisor)

Pattern cheat sheet:

Pattern	Likely story	Next tool
High `r`, high `us`/`sy`, low `id`	CPU bound	`mpstat -P ALL`, `pidstat`/`top`
High `b` or `wa`	I/O bound	`iostat -xz`, `iotop`
Continuous `si`/`so`	Memory pressure	`free -h`, `ps` RSS, reduce cache pressure
High `st`	Hypervisor contention / undersized VM	host metrics, resize instance
High `cs` with low work	Possible scheduling noise	app profiling, `perf`

Units: without `-S`, memory fields are often KiB pages-related scaling depending on build — use `-w` and `-S M` when comparing to `free -h`.

Notes & Pitfalls

- **First line with delay** is historically since-boot — always take interval lines for "now".
- `wa` means CPUs idle waiting on I/O, not "disk % busy"; a single slow device can elevate `wa`.
- `free` memory near zero can still be healthy if cache is reclaimable — watch `si`/`so` and application latency.
- Containers/cgroups: host `vmstat` sees the host; inside a container you still often see host-wide stats unless using cgroup-aware tools.
- Very short intervals (sub-second) add noise; 1s is a good default.

Related Commands

- `iostat` — per-device I/O rates and `%util`
- `mpstat` — per-CPU breakdown
- `free -h` — memory snapshot
- `top` / `htop` / `pidstat` — per-process
- `sar` — historical sysstat data
- `uptime` — load averages only

Additional Resources

- `man vmstat`
- `man free`

iostat

Overview

`iostat` (from the **sysstat** package) reports CPU utilization and **per-device** I/O rates, queue depth, await, and utilization. Use it to find busy disks, confirm I/O-wait hypotheses from `vmstat`/`top`, and compare devices under load. Install on Ubuntu with `sudo apt install sysstat`.

Syntax

```
iostat [options] [interval [count]]
iostat [options] [interval [count]] [device...]
```

Common Options

Option	Description
`-x`	Extended device stats (`%util`, `await`, `aqu-sz`, …)
`-z`	Omit idle devices
`-y`	Omit the first since-boot report when using intervals
`-d` / `-c`	Devices only / CPU only
`-h`	Human-readable where supported
`-k` / `-m`	KB/s or MB/s units
`-p`	Include partitions
`-t`	Timestamps
`-N`	Map device-mapper names to prettier LV names
`-j ID UUID …`	Display persistent device identifiers (version-dependent)

Key Use Cases

1. Live disk saturation checks (`%util`, `await`)
2. Confirm which device backs high `wa` in vmstat/top
3. Partition-level I/O (`-p`) during noisy-neighbor hunts
4. Scripted samples during load tests

Examples with Explanations

Extended live samples (default incident command)

```
iostat -xz 1 5
iostat -xyz 1 10
```

`-x` extended fields, `-z` hide idle noise, interval 1s. Prefer `-y` (or ignore first report) so you are not reading since-boot averages.

Skip since-boot line explicitly

```
iostat -xyz 1 5
# first sample after -y is the first real interval
```

One device (NVMe / disk)

```
iostat -xz 1 5 nvme0n1
iostat -xz 1 5 sda
```

Focus when you already know the hot device from `lsblk`.

CPU only vs devices only

```
iostat -c 1 5
iostat -d -xz 1 5
```

Partitions and mapper names

```
iostat -xzp 1 3
iostat -xzN 1 3
```

`-p` splits partitions; `-N` helps on LVM hosts.

Timestamped capture

```
iostat -txz 1 30 | tee /tmp/iostat.out
```

Pair with process-level I/O

```
iostat -xz 1 5
# if installed:
sudo iotop -oPa
pidstat -d 1 5
```

iostat shows **devices**; iotop/pidstat show **who** is writing.

Understanding Output

CPU section resembles other sysstat tools (`%user`, `%system`, `%iowait`, `%idle`, `%steal`).

Extended device fields (names can vary slightly by sysstat version):

Field	Meaning
`r/s w/s`	Read/write requests completed per second
`rkB/s wkB/s` (or `rMB/s`)	Throughput
`rrqm/s wrqm/s`	Merged requests (elevator/scheduler merging)
`await`	Average time (ms) for requests (queue + service)
`aqu-sz`	Average queue length
`rareq-sz wareq-sz`	Average request size
`%util`	Share of time the device had outstanding I/O (~saturation signal for simple devices)

How to read it under fire:

Signal	Interpretation
`%util` ~ 100% and high `await`	Device saturated or slow
High `await`, modest `%util`	Latency issue (storage tier, queueing elsewhere, NFS)
High `w/s` + rising `aqu-sz`	Write burst / flush pressure
Only one partition hot	Noisy tenant or log volume on that part
`%iowait` high on CPU block	CPUs waiting on I/O — align with device rows

Caveats: on RAID, multipath, and NVMe with deep queues, `%util` is not a perfect "percent busy" for modern parallel devices — still useful, but combine with `await` and application latency.

Notes & Pitfalls

- Package: `sudo apt install sysstat` if command missing.
- Without interval, or the first line of interval mode, stats are often **since boot** — use intervals + `-y`.
- Device names from `lsblk`; cloud reattach can rename `sdX`.

- NFS latency does not show as a local block device — check client `mount` stats / server side.
- Containers: host-level iostat unless you instrument the host.

Related Commands

- `vmstat` — system-level bi/bo and `wa`
- `iotop` — per-process I/O (if installed)
- `pidstat -d` — per-process disk stats (sysstat)
- `lsblk` — map names to mounts
- `sar -d` — historical disk from sysstat logs
- `nfsiostat` — NFS-specific (nfs-common)

Additional Resources

- `man iostat`
- `man pidstat`

mpstat

Overview

`mpstat` (sysstat) reports **CPU utilization**, optionally **per logical CPU**, so you can see single-thread bottlenecks, softirq load, and steal time on individual cores. Aggregate tools (`vmstat`, `uptime`) hide imbalance; `mpstat -P ALL` reveals it. Install: `sudo apt install sysstat`.

Syntax

```
mpstat [options] [interval [count]]
```

Common Options

Option	Description
`-P ALL`	Each CPU plus `all` average row
`-P 0,2`	Specific CPU indices
`-u`	Utilization report (default)
`-I SUM|CPU|SCPU|ALL`	Interrupt statistics
`-A`	Wide set of reports
`-o JSON`	JSON output (newer sysstat)
`-n`	Numeric only where applicable
`-T`	Topology-aware presentation (version-dependent)

Key Use Cases

1. Detect single-core saturation (one CPU 100%, others idle)
2. Softirq / network stack pressure (`%soft`)
3. Hypervisor steal (`%steal` / `%st`) per CPU
4. Interrupt distribution imbalances

Examples with Explanations

Average CPU samples

```
mpstat 1 5
```

Whole-system average every second — similar role to the CPU part of `vmstat`, with clearer labels.

Per-CPU breakdown (primary triage)

```
mpstat -P ALL 1 5
mpstat -P ALL -u 1 3
```

Look for one or few CPUs pinned at high `%usr` while others idle → single-threaded or poorly parallelized work.

Specific cores

```
mpstat -P 0,1 1 5
```

Useful when pinning (taskset/cgroups) or NUMA-aware debugging.

Interrupts

```
mpstat -I SUM 1 3
mpstat -I CPU 1 3
mpstat -I SCPU 1 3
```

`SUM` totals; `CPU` / `SCPU` help when hunting IRQ imbalance or softirq storms (pair with `grep` in `/proc/interrupts`).

JSON for tooling

```
mpstat -P ALL -o JSON 1 1 | jq .
```

Pair with process CPU

```
mpstat -P ALL 1 5
pidstat -u 1 5
# top, then press 1 for per-CPU
```

mpstat shows **where** CPUs burn; pidstat/top show **who**.

Understanding Output

Common utilization fields:

Field	Meaning
`%usr`	User-space applications
`%nice`	User-space nice'd
`%sys`	Kernel
`%iowait`	Idle waiting for I/O
`%irq` / `%soft`	Hard / soft interrupts
`%steal`	Time stolen by hypervisor
`%guest`	Running guest VMs (host)
`%idle`	Idle not waiting on I/O

Patterns:

Observation	Likely story
One CPU `%usr` 100, others idle	Single-thread bottleneck / GIL / hot lock
All CPUs high `%usr`	Healthy parallel load or capacity shortfall
High `%soft` on few CPUs	Network/softirq concentration — RSS/RPS/IRQ affinity
High `%steal`	Noisy neighbor or oversubscribed VM
High `%iowait`	Storage latency (confirm with `iostat -xz`)
High `%sys`	Kernel-heavy workload, syscalls, networking, virtualization

First sample without care about "since boot" applies less than iostat, but interval mode is still the right way to watch live change.

Notes & Pitfalls

- Install sysstat if missing: `sudo apt install sysstat`.
- CPU numbering is logical CPUs (includes SMT/hyperthreads).
- Inside containers you may still see host CPUs depending on visibility; cgroup limits need `cpuset`/`cpu` controllers and specialized tools.

- Balancing IRQs is a separate operational topic (`/proc/irq`, `irqbalance`).
- `%iowait` can look "high" on mostly-idle systems with occasional slow I/O — correlate with latency SLOs, not only percentages.

Related Commands

- `vmstat` — compact aggregate loop
- `sar -P ALL` — historical per-CPU
- `top` (key `1`) / `htop` — interactive per-CPU
- `pidstat -u` — per-process CPU
- `perf top` — see hot kernel/user symbols
- `nproc` / `lscpu` — how many CPUs exist

Additional Resources

- `man mpstat`
- `man pidstat`

sar

Overview

`sar` (System Activity Reporter, **sysstat**) prints **live and historical** metrics: CPU, memory, load, disk, network, and more. Live mode is like specialized siblings (`mpstat`, `iostat`); historical mode reads daily binary logs written by `sadc` via cron or systemd timers. On Ubuntu, collection is **off until enabled**.

Syntax

```
sar [options] [interval [count]]
sar [options] -f /var/log/sysstat/saDD
sar [options] -s HH:MM:SS -e HH:MM:SS
```

Common Options

Option	Description
`-u`	CPU utilization (default)
`-r`	Memory utilization
`-q`	Load average / run queue
`-b`	I/O transfer rates
`-d`	Per-block device
`-n DEV|EDEV|TCP|SOCK|…`	Network statistics
`-P ALL`	Per-CPU
`-s` / `-e`	Start / end time for historical
`-f file`	Read a specific saDD data file
`-A`	Nearly all reports
`-o file`	Write binary samples (advanced)
`-f` + date	Files under `/var/log/sysstat/` (`sa07` = day 07)

Key Use Cases

1. Live sampling during an incident (same family as mpstat/iostat)
2. "What happened at 09:15 yesterday?" from sa logs

3. Capacity trends when full Prometheus stack is overkill
4. Correlate CPU/mem/net windows after outages

Examples with Explanations

Live CPU samples

```
sar -u 1 5
sar -u ALL 1 5
```

Memory and load

```
sar -r 1 5
sar -q 1 5
sar -r -q 1 5
```

Disk and I/O rates

```
sar -b 1 5
sar -d 1 5
```

Network devices

```
sar -n DEV 1 5
sar -n EDEV 1 5    # errors/drops
sar -n TCP,ETCP 1 5
```

Per-CPU history of "now"

```
sar -P ALL 1 3
```

Today's history (collection must be enabled)

```
sar -u
sar -r
sar -q
ls -la /var/log/sysstat/
```

Without data files, sar only works in live interval mode or errors on historical queries.

Slice a time window

```
sar -u -s 09:00:00 -e 10:00:00
sar -r -s 09:00:00 -e 10:00:00
sar -n DEV -s 09:00:00 -e 10:00:00
```

Read another day's file

```
# Day-of-month file name (locale/sysstat config can vary)
sudo ls /var/log/sysstat/
sar -u -f /var/log/sysstat/sa08
sar -r -f /var/log/sysstat/sa08 -s 14:00:00 -e 15:00:00
```

Enable collection on Ubuntu (sketch)

```
sudo apt install sysstat
# Debian/Ubuntu: enable data collector
sudo sed -i 's/^ENABLED=.*/ENABLED="true"/' /etc/default/sysstat
sudo systemctl enable --now sysstat
# timers: sysstat-collect.timer / sysstat-summary.timer (names may vary by release)
systemctl list-timers | grep -i sysstat
ls /var/log/sysstat/
```

Confirm files appear under `/var/log/sysstat/` after the next collect interval (often ~10 minutes).

Incident bundle

```
sar -u 1 5
sar -r 1 5
sar -q 1 5
sar -n DEV 1 5
# if history enabled:
sar -u -s 11:00:00 -e 12:00:00
```

Understanding Output

Output is tabular with a timestamp column in historical mode. Field meanings align with other sysstat tools:

Report	Useful fields
`-u`	`%user`, `%system`, `%iowait`, `%steal`, `%idle`

Report	Useful fields
`-r`	`kbmemfree`, `kbavail`, `%memused`, `kbcached`, `%swpused`
`-q`	`runq-sz`, `plist-sz`, `ldavg-1/5/15`
`-d`	`tps`, `rkB/s`, `wkB/s`, `await`, `%util` (version-dependent)
`-n DEV`	`rxkB/s`, `txkB/s`, `rxpck/s`, `txpck/s`

Reading historical rows: averages between sample points (e.g. every 10 minutes) smooth spikes — a 30-second outage may barely move a 10-minute bucket. For micro-incidents, live 1s tools win; sar history wins for longer windows.

Notes & Pitfalls

- **Ubuntu default:** package installed collection enabled — set `ENABLED="true"` and start sysstat timers.
- Data retention/rotation is configured under `/etc/sysstat/` and logrotate — don't assume months of history.
- Timezone of timestamps follows system clock — wrong TZ confuses windows (`timedatectl`).
- Binary sa files are not plain text; always use `sar -f`.
- Modern long-term monitoring is often Prometheus/node_exporter/Grafana; sar remains excellent for local/air-gapped hosts.
- Permissions: reading some logs may need root depending on directory mode.

Related Commands

- `vmstat` / `iostat` / `mpstat` — live siblings
- `pidstat` — per-process over intervals
- `journalctl` — logs, not metrics
- `prometheus` / `node_exporter` — long-term modern stacks
- `dstat` / `atop` — alternative live dashboards (if installed)

Additional Resources

- `man sar`
- `man sadc`
- `/usr/share/doc/sysstat/` on package install

lsof

Overview

`lsof` lists **open files**. On Linux, "files" include regular files, directories, block/char devices, pipes, and **sockets** — so it answers both "what files does this process hold?" and "who has this port?". Essential for busy `umount`, deleted-but-open disk space mysteries, and classic port ownership checks (modern alternative for sockets alone: `ss -lntp`).

Syntax

```
lsof [options]
lsof -p PID
lsof -i :port
```

Common Options

Option	Description
`-p PID`	Files for process (comma-separated PIDs)
`-i`	Internet sockets; optional address/port filter
`-iTCP:443 -sTCP:LISTEN`	Listening TCP on 443
`-u user`	Files for user
`-c name`	Command name prefix match
`+D dir`	Recurse directory (expensive)
`+f -- path`	Path as filesystem (busy mount checks)
`-n`	No host DNS resolution (faster)
`-P`	No port name resolution (show numbers)
`-a`	AND selections (default is often OR — easy footgun)
`-t`	PIDs only (for scripts/kill pipelines)
`-R`	Show parent PID

Key Use Cases

1. Who is listening on a TCP/UDP port?
2. Why is umount busy?

3. Deleted files still holding disk space
4. Audit files/sockets for a PID or user

Examples with Explanations

Who listens on port 22?

```
sudo lsof -nP -iTCP:22 -sTCP:LISTEN
# modern alternative focused on sockets:
ss -lntp | grep ':22'
```

`-nP` skips DNS and service names for speed and stable output.

Open files for a PID

```
sudo lsof -p "$(pgrep -x nginx | head -1)"
sudo lsof -p 1234 -nP
```

Configs, logs, `.so` mappings, and sockets held by that process.

Deleted files still open (disk full mystery)

```
sudo lsof +L1
sudo lsof -nP | grep '(deleted)'
# focus large ones:
sudo lsof +L1 | awk 'NR==1 || $7 ~ /[0-9]/ {print}' | head
```

A process can keep a deleted log open — `df` shows full, `du` does not account for it until restart/close.

Files under a mount (before umount)

```
sudo lsof +f -- /mnt/data
sudo lsof /var
sudo fuser -vm /mnt/data
```

Identify shells (cwd), daemons, and lockers preventing `umount`.

Network connections for a user

```
sudo lsof -u deploy -a -i
sudo lsof -a -u deploy -iTCP -sTCP:ESTABLISHED -nP
```

`-a` ANDs the filters — without it, `-u` and `-i` can OR and surprise you.

Command name filter

```
sudo lsof -c python -nP
sudo lsof -c java -i -nP
```

Prefix match on the command column.

PIDs only for scripting

```
sudo lsof -t -iTCP:8080 -sTCP:LISTEN
# careful:
# sudo kill "$(sudo lsof -t -iTCP:8080 -sTCP:LISTEN)"
```

UNIX sockets / pipes (advanced)

```
sudo lsof -U | head
sudo lsof -p PID -a -U
```

Useful for local agent debugging (docker.sock, etc.).

Understanding Output

Typical columns:

Column	Meaning
`COMMAND`	Process name
`PID`	Process ID
`USER`	Owner
`FD`	File descriptor (`cwd`, `txt`, `mem`, `0u`, `1w`, `2u`, `DEL`, …)
`TYPE`	`REG`, `DIR`, `CHR`, `FIFO`, `IPv4`, `IPv6`, `unix`, …
`DEVICE`	Device numbers
`SIZE/OFF`	Size or offset
`NODE`	Inode / protocol node
`NAME`	Path, socket endpoint, or `(deleted)`

FD suffixes: `r` read, `w` write, `u` read/write. `cwd` is current working directory — a common umount blocker when your shell sits on the mount.

Notes & Pitfalls

- Root is required for other users' processes and many sockets.
- Full-system `lsof` is expensive on busy hosts — narrow with `-p`, `-i`, `-u`, paths.
- Default combination of tests can be **OR**; use `-a` when you mean AND.
- Prefer `ss -lntp` for "what listens where?" when you only care about sockets.
- NFS and FUSE paths can hang lsof if the remote is stuck — try scoped queries first.
- Containers: run lsof in the same PID/mount namespace you care about (host vs container).

Related Commands

- `ss` — modern socket statistics
- `fuser` — PIDs using a file, mount, or port
- `strace` — syscall-level open/connect trail
- `lsof` vs `netstat` — prefer `ss` for sockets-only
- `fuser -vm` / `findmnt` — umount busy workflow
- `ps` / `pgrep` — find PIDs before `lsof -p`

Additional Resources

- `man lsof`
- `man ss`
- `man fuser`

strace

Overview

`strace` traces **system calls** and signals for a process. When a program fails with a vague error ("permission denied", "connection refused", silent hang), strace often shows the failing `openat`, `connect`, `execve`, or `access`. It is a diagnostic scalpel — powerful, noisy, and can slow production processes. Prefer filtering (`-e`) and short attaches.

Syntax

```
strace [options] command [args...]
strace [options] -p PID
```

Common Options

Option	Description
`-e trace=file|network|process|desc|…`	Syscall sets
`-e openat,connect,execve`	Specific calls
`-e errno=ENOENT`	Only failures of a type (version-dependent usage)
`-o file`	Write trace to file (keep stderr clean)
`-f`	Follow forks/threads (usually want this)
`-p PID`	Attach to running process
`-c`	Summary counts and time per syscall
`-C`	Summary plus regular trace
`-s N`	Max string length printed (default often 32)
`-y`	Resolve FD paths
`-yy`	Extra protocol details for sockets (newer)
`-tt` / `-ttt`	Timestamps (wall / epoch)
`-T`	Time spent in each syscall
`-q`	Quieter attach/detach messages

Safety

- Attaching to live production processes **adds overhead** — can stall latency-sensitive apps.
- Needs privileges for other users' PIDs (`CAP_SYS_PTRACE` / root); Yama `ptrace_scope` may block attaches.
- Do not casually attach to security-sensitive processes (agents, secret managers) on shared notes/screens.
- Detach with Ctrl-C carefully; prefer `-o` logs over flooding terminals in incidents.

Key Use Cases

1. Why did a command fail to open a file or config?
2. Where is a hang — blocking `read`/`poll`/`futex`/`connect`?
3. Which addresses/ports does a binary hit?
4. Rough syscall cost profile (`-c`) before deeper `perf`

Examples with Explanations

Why did it fail? (file open)

```
strace -f -e openat,access ls /no/such 2>&1 | tail
strace -f -e openat,stat,access cat /etc/shadow
```

Look for `= -1 ENOENT` or `EACCES` on the path you expected.

Network path of a client

```
strace -f -e connect,sendto,recvfrom,poll,select \
  curl -sS https://example.com/ -o /dev/null
```

See DNS vs connect failures (pair with `curl -v` / `dig`).

Summary profile

```
strace -c ls -R /etc >/dev/null
strace -c -f make -j"$(nproc)" >/dev/null
```

Counts and rough time per syscall — good for "too many small reads" stories.

Attach to a running PID

```
pidof myapp
sudo strace -p "$(pidof -s myapp)" -f -e file -s 200
# or:
sudo strace -p PID -f -e trace=network -tt
```

`-f` follows worker threads. Scope with `-e` or the terminal becomes unreadable.

Write a full log

```
strace -f -tt -T -o /tmp/trace.txt ./app --flag
less /tmp/trace.txt
grep -E 'ENOENT|EACCES|ECONNREFUSED' /tmp/trace.txt | tail
```

Always useful: keep app stdout free; search the log for errnos.

Longer strings and FD paths

```
strace -f -s 200 -y -e openat,read,write ./app
```

`-s` avoids truncated paths; `-y` shows what FD 7 actually is.

Child processes / build systems

```
strace -f -e execve,openat make package 2>&1 | grep execve
```

See which helpers are actually invoked.

Hang diagnosis sketch

```
sudo strace -p PID -f -tt -T
# stuck in futex → locking; poll/select → waiting I/O; read on socket → peer silent
```

Understanding Output

Typical lines:

```
openat(AT_FDCWD, "/etc/passwd", O_RDONLY|O_CLOEXEC) = 3
connect(4, {sa_family=AF_INET, sin_port=htons(443), ...}, 16) = -1 EINPROGRESS (Operation no
```

```
openat(AT_FDCWD, "/etc/foo.conf", O_RDONLY) = -1 ENOENT (No such file or directory)
```

Piece	Meaning
Name	Syscall
Args	Paths, FDs, flags, addresses
`= N`	Success return (FD number, byte count, …)
`= -1 ERRNO (text)`	Failure — usually the smoking gun
Duration (`-T`)	Time inside the call (blocking shows up here)

Common errnos: `ENOENT` missing path, `EACCES`/`EPERM` rights, `ECONNREFUSED` nothing listening, `ETIMEDOUT` network path, `EAGAIN` nonblocking empty.

Notes & Pitfalls

- Without `-e`, output volume explodes — always filter first when possible.
- Multi-threaded apps need `-f` or you miss workers.
- Go/Java/Node runtimes are syscall-noisy; start with `-e network` or `-e file`.
- `seccomp`/sandboxes may alter available calls; container seccomp profiles matter.
- Kernel `yama/ptrace_scope` (`/proc/sys/kernel/yama/ptrace_scope`) can deny non-root attaches.
- For performance at scale prefer `perf`, eBPF (`bpftrace`), or app metrics — strace is not a profiler for production load tests.

Related Commands

- `ltrace` — library calls (not syscalls)
- `perf` — CPU performance counters / flame graphs
- `gdb` — breakpoints and state inspection
- `lsof` — open files/sockets snapshot without tracing
- `journalctl` / app logs — higher-level errors first
- `ss` / `tcpdump` — network without process tracing

Additional Resources

- `man strace`
- `man ptrace`

perf

Overview

`perf` is Linux's primary performance analysis suite. It samples hardware performance counters, software events, and kernel tracepoints to show *where* CPU time goes—functions, shared libraries, kernel paths—not just which process is "busy" in `top`.

```
# typical packages
sudo apt install linux-perf        # Debian/Ubuntu (name varies by kernel)
sudo dnf install perf
# ensure matching kernel headers/debug symbols for readable stacks when possible
```

Syntax

```
perf [subcommand] [options] [--] [command]
```

Common Subcommands

Subcommand	Description
`stat`	Aggregate counters for a command or system-wide
`record`	Sample events → `perf.data`
`report`	Interactive/text report from `perf.data`
`top`	Live function-level top
`script`	Dump raw samples (for flame graphs, custom tools)
`list`	List available events
`annotate`	Source/asm annotation
`probe`	Dynamic probes (advanced)

Common Options (record / stat)

Option	Description
`-e EVENT`	Event(s): `cycles`, `instructions`, `cache-misses`, …

Option	Description
`-g` / `--call-graph`	Call graphs (`fp`, `dwarf`, `lbr` depending on arch)
`-p PID` / `-t TID`	Attach to process/thread
`-a`	System-wide
`-F freq`	Sample frequency
`-o file`	Output file (default `perf.data`)

Key Use Cases

1. Find hot functions in user or kernel space
2. Measure IPC, cache misses, branch misses
3. System-wide profiling under load
4. Generate data for flame graphs

Examples with Explanations

High-level counters for one command

```
perf stat ./my_program
perf stat -e cycles,instructions,cache-misses,branch-misses ./my_program
```

`instructions` / `cycles` → rough IPC. High cache-miss rates often explain "CPU busy but slow" workloads.

Live profiling

```
sudo perf top
sudo perf top -p $(pgrep -n myapp)
```

Like `top`, but for symbols. Needs root or `kernel.perf_event_paranoid` relaxed appropriately.

Record + report workflow

```
perf record -g ./my_program
perf report
# or
perf record -F 99 -g -p $(pgrep -n myapp) -- sleep 30
perf report --stdio | less
```

`-g` collects call graphs so you see callers, not only leaf functions.

System-wide snapshot

```
sudo perf record -a -g -- sleep 10
sudo perf report
```

One-liner recipes

```
# Why is this box busy?
sudo perf top -g

# Count context switches / migrations during a run
perf stat -e context-switches,cpu-migrations,page-faults ./server

# Dump samples for external flamegraph tooling
perf script > out.perf
```

Paranoid settings (understand before changing)

```
sysctl kernel.perf_event_paranoid
# -1: least restricted; higher values limit unprivileged use
# Prefer temporary, documented changes over permanent wide-open settings
```

Notes & Pitfalls

- **Symbols**: stripped binaries show hex addresses. Install debuginfo / build with frame pointers or use DWARF call graphs (`--call-graph dwarf`, higher overhead).
- **Overhead**: high-frequency sampling and DWARF unwind cost CPU and disk.
- **Containers**: host `perf` can profile containers with care; cgroup modes and symbol paths complicate analysis.
- **Permissions**: modern kernels gate unprivileged `perf`; don't casually set paranoid to `-1` on multi-tenant hosts.
- `perf.data` is bulky—delete or compress after analysis.

2026-relevant notes

- Prefer **frame pointers** (`-fno-omit-frame-pointer` / modern defaults on some stacks) or **ORC/DWARF** unwind as appropriate for your distro/arch.
- Pair with `bpftrace`, `bcc`, and `pidstat`/`perf stat` for layered diagnosis: counters → samples → targeted traces.
- For continuous profiling in production, evaluate eBPF-based agents; use classic `perf` for deep dives and offline `perf report`.

Comparison to alternatives

Tool	Role
`perf`	Sampling + counters, deep Linux integration
`strace`	Syscall trace (different question)
`top`/`htop`/`btop`	Process-level overview
`bpftrace`	Ad-hoc kernel/user probes

Related Commands

- `strace` — syscalls
- `top` / `htop` / `btop` — process monitors
- `vmstat` / `iostat` / `mpstat` — subsystem stats
- `sysctl` — kernel knobs including perf policy

Additional Resources

- `man perf` and `man perf-record`
- Kernel `tools/perf` documentation
- Brendan Gregg's Linux performance materials (flame graphs, methodology)

btop

Overview

`btop` is a modern, interactive resource monitor with a polished TUI: CPU (often per-core with clocks/temps when available), memory, disks, network, and a process list. It sits in the same family as `htop` and `bashtop`/`bpytop`, with a denser visual layout.

```
sudo apt install btop    # Debian/Ubuntu when packaged
sudo dnf install btop
# or install from upstream releases if your distro lags
```

Syntax

```
btop [options]
```

Common Options

Option	Description
`-lc`, `--low-color`	16-color / limited terminals
`-t`, `--tty`	Force tty-friendly mode (check `--help` on your build)
`--utf-force`	Force UTF-8 drawing
`-p`, `--preset N`	Start with layout preset
`-u`, `--update MS`	Refresh rate in milliseconds
`-h`, `--help`	Help

Exact flags vary slightly by version—run `btop -h`.

Interactive Keybindings (typical)

Key	Action
`↑ ↓` / `j k`	Move process selection
`Enter`	Process details / options

Key	Action
`f`	Filter processes
`m`	Memory-related layout toggle
`p`	Process sort / panel focus (version-dependent)
`k` or signal menu	Kill / send signal to selected process
`t`	Tree view (if enabled in version)
`F2` / preset keys	Cycle presets
`Esc` / `q`	Back / quit
`?` or `h`	Help overlay

If a key does nothing, open the in-app help—bindings are configurable.

Key Use Cases

1. At-a-glance host health on a jump box
2. Spot CPU steal, memory pressure, or disk saturation visually
3. Find and signal runaway processes without remembering `ps` pipelines
4. Live network throughput per interface

Examples with Explanations

Start defaults

```
btop
```

Low-color SSH / serial console

```
btop --low-color
# or
btop -lc
```

Faster or slower refresh

```
btop -u 500
btop -u 2000
```

One-liner recipes

```
# Prefer btop when available, else htop, else top
command -v btop >/dev/null && btop || command -v htop >/dev/null && htop || top

# Record a note of load while viewing (separate terminal)
while true; do printf '%s %s\n' "$(date -Iseconds)" "$(cat /proc/loadavg)"; sleep 5; done
```

Notes & Pitfalls

- **Remote SSH**: needs a reasonable terminal (xterm-256color, width). Tiny terminals break layout—use `-lc` or resize.
- **Sensors/temps**: may need `lm-sensors` configured; containers often lack host sensors.
- **Permissions**: process details and some disk stats are richer as root but daily use as normal user is fine.
- **Not a logger**: for historical metrics use Prometheus node exporter, Netdata, or `sar`.
- Config usually lives under `~/.config/btop/`—back it up if you customize themes heavily.

2026-relevant notes

- `btop` is a strong default **interactive** monitor; keep `htop` for muscle memory and `top` for universal availability on rescue images.
- On servers, pair TUI tools with `journalctl -f`, `ss -lntp`, and `dmesg -T` rather than expecting one dashboard to explain outages.
- In Kubernetes/node shells, metrics may reflect the container cgroup, not the whole node—interpret carefully.

Comparison to alternatives

Tool	Notes
`btop`	Modern TUI, visual meters
`htop`	Ubiquitous interactive process focus
`top`	Always there, minimal
`glances`	Cross-platform, optional web UI
`btm` / `zenith`	Other modern Rust/Go monitors

Related Commands

- `top` / `htop` — process monitors
- `vmstat` / `iostat` / `mpstat` — batch subsystem stats

- `nproc` / `free` / `df` — quick non-interactive checks
- `perf top` — function-level CPU profiling

Additional Resources

- Upstream `btop` README for install and themes
- `man btop` when packaged

ltrace

Overview

`ltrace` intercepts and prints **dynamic library calls** of a process (similar to how `strace` prints syscalls). Useful when you need to see `malloc`, `strcmp`, `fopen`, or library-level behavior without a full debugger.

```
sudo apt install ltrace
```

Syntax

```
ltrace [options] command [args...]
ltrace -p PID
```

Common Options

Option	Description
`-e filter`	Include/exclude symbols
`-p PID`	Attach to process
`-c`	Counts / summary
`-T`	Show time in calls
`-f`	Follow forks
`-o file`	Output file
`-s strsize`	String capture length

Safety

- Attaching can slow or perturb production processes.
- Needs appropriate `ptrace` permissions (`kernel.yama.ptrace_scope`).
- Do not leave high-volume tracing on busy services.

Examples with Explanations

Trace a command

```
ltrace -e fopen+fclose+read ls /tmp 2>&1 | head
```

Summary counts

```
ltrace -c true
ltrace -c ./myapp --flag 2>&1 | tail
```

Attach

```
ltrace -p "$(pidof myapp)" -e 'malloc+free' -c
```

Notes & Pitfalls

- Statically linked binaries show little.
- Prefer `strace` for syscall/ENOENT path issues; `ltrace` for library logic.
- Go/Rust binaries may be less informative depending on linkage.

Related Commands

- `strace` — syscalls
- `gdb` — full debugger
- `perf` — profiling
- `nm` / `ldd` — symbols and libraries

Additional Resources

- `man ltrace`

pmap

pmap

Overview

Thesis: when a process's RSS keeps climbing but every heap profiler says "no leak", believe the kernel, not the allocator. `pmap` (procps-ng) prints a process's memory map from `/proc/PID/smaps`. It is the quickest way to see *which mapping* is growing: heap, stack, a library, or an anonymous region that `malloc` never knew about. This page uses it as the first step of a three-tool hunt for a native leak:

1. **`pmap`** shows *where* the memory is (an anonymous region that grows and moves).
2. **A syscall tracer** shows *who* creates it. Use `bpftrace` where the kernel allows it, and `strace -k` where it does not.
3. **`gdb`, scripted** stops only on the guilty call and prints the full stack and the returned address, which you can match against step 1.

Typical operator jobs:

- Tell a heap leak (`[heap]` / `malloc` arenas grow) from a mapping leak (`[ anon ]` regions grow).
- Find out what a 2 GB RSS is actually made of: libraries, thread stacks, arenas, or shared memory.
- Confirm a fix by watching the guilty region stop growing.

```
# Debian/Ubuntu/Fedora: part of procps(-ng), installed by default
pmap --version
```

```
pmap from procps-ng 4.0.4
```

Mental model

```
 virtual address space of one process            what each tool can see

 [stack]                                          pmap / smaps:  every mapping, its size and
 ...                                              heap profilers: only malloc/free (and new/
 [ anon ]     mmap(MAP_ANONYMOUS) by an          LD_PRELOAD mmap hook: only calls through th
 [ anon ]       allocator, a pool, a JIT, a NIC      mmap() *symbol*, not syscall(SYS_mmap,
libfoo.so     driver library...                   strace / bpftrace: every syscall, however
 [heap]       brk(): small malloc() blocks       gdb: anything you can put a breakpoint on
 binary
```

Fact	Consequence
`malloc` serves small blocks from `[heap]`/arenas and large ones from fresh anonymous mappings	Heap profilers account for both, but only for memory obtained **through `malloc`**
Libraries may call `mmap` directly, or bypass even that with the raw `syscall(SYS_mmap, …)`	That memory is invisible to heap profilers *and* to `LD_PRELOAD` hooks on `mmap`
The kernel **merges adjacent anonymous mappings** with the same permissions	Many leaked 256 KiB blocks show up as **one `[ anon ]`** region that grows downwards: same end address, smaller start address
`Kbytes` is reserved address space, `RSS` is what is in RAM	A pool that maps 256 KiB but touches 64 KiB grows `Kbytes` 4× faster than `RSS`. Watch both
`pmap` reads `/proc/PID/smaps` and does not stop the process	It is safe to run against production every second

Syntax

```
pmap [options] PID [PID ...]
```

Option	Meaning
(none)	Address, size, permissions, mapping name
`-x`, `--extended`	Add `RSS` and `Dirty` columns (KiB) and a `total kB` line
`-X`	Every common `smaps` field (`Pss`, `Anonymous`, `Swap`, `THPeligible`, …). The format follows the kernel's `smaps`, so do not parse columns by position across kernels
`-XX`	Everything the kernel provides
`-q`, `--quiet`	No header or footer, which is handy for `sort`
`-p`, `--show-path`	Full path of file mappings
`-A low[,high]`	Only mappings in an address range
`-d`, `--device`	Device format (offset and device per mapping)

For one number instead of a table, `/proc/PID/smaps_rollup` gives totals (`Rss`, `Pss_Anon`, `Anonymous`, …) without walking every mapping.

Safety

- `pmap` needs the same access as reading `/proc/PID/smaps`. That means your own processes, or root (or `CAP_SYS_PTRACE`) for others. It does **not** pause the target.

- `strace -p` and `gdb -p` **attach** with `ptrace`. They stop or slow every thread of the target. Prefer launching the program under the tool, as below, or use `bpftrace`, which does not stop the process.
- Attaching is also restricted by `kernel.yama.ptrace_scope` (Debian/Ubuntu default `1`: only descendants). Do not loosen it on production hosts just to debug.
- A conditional breakpoint still traps on **every** call to the function, and gdb then evaluates the condition. On a hot function that is a large slowdown. Keep such sessions short.

Examples with Explanations

All examples use a small, self-contained leak, so you can reproduce every step as an ordinary user. `leaky.c` simulates a request loop. Each request uses a 64 KiB `malloc` buffer, which it frees correctly. It also takes a 256 KiB "pinned" buffer from a tiny pool, which gets memory with a **raw `syscall(SYS_mmap, …)`** and, on release, parks the buffer on a deferred-release list. The list is drained only when it grows past `POOL_MAX_UNRELEASED`, and the default is unlimited. This is the same shape as a real-world leak in a registration-cache library (see the case note after example 7).

```
/* leaky.c - a request loop with a native leak that malloc never sees.
 *
 * Each "request" uses a 64 KiB heap buffer (freed, healthy) and a 256 KiB
 * "pinned" buffer from a tiny pool allocator. The pool gets memory with a raw
 * syscall(SYS_mmap) and, on release, parks buffers on a deferred-release
 * list that is drained only when it exceeds POOL_MAX_UNRELEASED.
 * Default: unlimited, so nothing is ever unmapped.
 *
 *   ./leaky [requests]          POOL_MAX_UNRELEASED=16 ./leaky
 */
#define _GNU_SOURCE
#include <malloc.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/syscall.h>
#include <unistd.h>

#define PIN_SIZE (256 * 1024)

struct parked { void *addr; struct parked *next; };
static struct parked *unreleased;
static long n_unreleased, max_unreleased = -1;   /* -1 = unlimited */

static void *pool_get(void)
{
    /* raw syscall: bypasses glibc's mmap() symbol and any hook on it */
    void *p = (void *)syscall(SYS_mmap, NULL, PIN_SIZE,
```

```
                               PROT_READ | PROT_WRITE,
                               MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (p == MAP_FAILED) { perror("mmap"); exit(1); }
    return p;
}

static void pool_put(void *p)
{
    struct parked *e = malloc(sizeof *e);   /* tiny: 16 bytes on the heap */
    e->addr = p;
    e->next = unreleased;
    unreleased = e;
    if (max_unreleased >= 0 && ++n_unreleased > max_unreleased) {
        while (unreleased) {                /* drain the whole list */
            struct parked *x = unreleased;
            unreleased = x->next;
            syscall(SYS_munmap, x->addr, PIN_SIZE);
            free(x);
        }
        n_unreleased = 0;
    }
}

static long rss_kib(void)
{
    char line[256];
    long kib = -1;
    FILE *f = fopen("/proc/self/status", "r");
    while (f && fgets(line, sizeof line, f))
        if (sscanf(line, "VmRSS: %ld kB", &kib) == 1) break;
    if (f) fclose(f);
    return kib;
}

static void handle_request(int i)
{
    char *scratch = malloc(64 * 1024);      /* normal heap use */
    memset(scratch, i & 0xff, 64 * 1024);
    char *pinned = pool_get();              /* "registered" buffer */
    memcpy(pinned, scratch, 64 * 1024);     /* touch it: now resident */
    free(scratch);
    pool_put(pinned);                       /* "released" ... */
}

int main(int argc, char **argv)
{
```

```
    int requests = argc > 1 ? atoi(argv[1]) : 2000;
    const char *cap = getenv("POOL_MAX_UNRELEASED");
    if (cap) max_unreleased = atol(cap);
    printf("pid %d, %d requests, max_unreleased=%ld\n",
           getpid(), requests, max_unreleased);
    fflush(stdout);
    int every = requests >= 4 ? requests / 4 : 1;
    for (int i = 1; i <= requests; i++) {
        handle_request(i);
        if (i % every == 0) {
            struct mallinfo2 mi = mallinfo2();
            printf("req %5d  VmRSS %7ld KiB  malloc in-use %5zu KiB\n",
                   i, rss_kib(), mi.uordblks / 1024);
            fflush(stdout);
        }
        usleep(1000);
    }
    return 0;
}
```

```
mkdir -p /tmp/leak && cd /tmp/leak        # save leaky.c here
gcc -O0 -g -fno-omit-frame-pointer -o leaky leaky.c
```

`-g` and `-fno-omit-frame-pointer` keep stacks readable for the later steps. `-O0` keeps `pool_get` as a real function. At `-O1` and above, GCC inlines the whole request path into `main`, and the stacks below collapse to a single frame.

Tested on Debian 13 (kernel 6.12, x86-64) with gcc 14.2, glibc 2.41, procps-ng 4.0.4, strace 6.13, gdb 16.3 and bpftrace 0.23.2. PIDs, addresses and paths in the outputs are from that box.

1. Symptom: RSS climbs, the heap does not

```
./leaky 2000
```

```
pid 473958, 2000 requests, max_unreleased=-1
req   500  VmRSS   33844 KiB  malloc in-use    20 KiB
req  1000  VmRSS   65920 KiB  malloc in-use    37 KiB
req  1500  VmRSS   97936 KiB  malloc in-use    53 KiB
req  2000  VmRSS  129952 KiB  malloc in-use    68 KiB
```

RSS grows by about 64 KiB per request: the part of each pinned buffer that was touched. `malloc` reports 68 KiB in use, and that is only the 16-byte list nodes. Any tool that counts `malloc`/`free` (heaptrack, Memray, Valgrind's massif in heap mode, `mallinfo2`) would truthfully report "no leak". **Heaps lie by omission**: they only know about memory they handed out.

2. Where: watch the biggest mappings with `pmap -x`

`top-maps.sh` prints the header, the total, and the top N mappings by RSS:

```
#!/bin/sh
# top-maps.sh PID [N] - largest mappings by RSS, with header and total
pmap -x "$1" | awk 'NR == 2 || /^total/'
pmap -x "$1" | sed '1,2d;$d' | sort -k3 -nr | head -n "${2:-3}"
```

```
chmod +x top-maps.sh
./leaky 8000 > leaky.out & sleep 0.5
P=$(awk '{print $2; exit}' leaky.out | tr -d ,)   # the PID leaky printed, not $!
echo "leaky pid $P"; sleep 1.5
echo "### t=2s"; ./top-maps.sh "$P"; sleep 3
echo "### t=5s"; ./top-maps.sh "$P"
wait
```

```
leaky pid 474858
### t=2s
Address           Kbytes     RSS   Dirty Mode  Mapping
total kB          459004  115876  114340
00007ff333c02000  457228  114312  114312 rw---   [ anon ]
00007ff34faad000    1420    1024       0 r-x-- libc.so.6
00007ff34fc8e000     160     160       0 r-x-- ld-linux-x86-64.so.2
### t=5s
Address           Kbytes     RSS   Dirty Mode  Mapping
total kB         1143684  287096  285560
00007ff309f82000 1141772  285448  285448 rw---   [ anon ]
00007ff34faad000    1420    1024       0 r-x-- libc.so.6
000055bbfe2e4000     268     212     212 rw---   [ anon ]
```

Read it like this:

- **One anonymous region holds almost all of the RSS** (114 MB, then 285 MB), and it is not `[heap]`. This is a mapping leak, not a `malloc` leak.
- **Its start address moves down** (`…7ff333c02000` → `…7ff309f82000`) while its size grows. New mappings are placed just below the previous ones, and the kernel merges them into one region. "Growing and moving" is the signature of repeated `mmap` with no matching `munmap`. A single region that grows in place points to `mremap` instead.
- **`Kbytes` grows 4× faster than `RSS`** (each 256 KiB mapping has 64 KiB touched). On a long-running service, address-space growth like this ends in `ENOMEM` from `mmap` or an overcommit OOM, even if RSS looks tolerable.

To follow it live, use `watch -n 1 ./top-maps.sh "$P"`. For a single number, read `/proc/PID/smaps_rollup`. Here it is one second into a fresh run:

```
./leaky 1500 > leaky.out & sleep 1
P=$(awk '{print $2; exit}' leaky.out | tr -d ,)
grep -E 'Rss|Pss_Anon|Anonymous' /proc/$P/smaps_rollup; wait
```

```
Rss:                58664 kB
Pss_Anon:           56904 kB
Anonymous:          56904 kB
```

Nearly all of RSS (56.9 of 58.7 MB) is anonymous memory. Libraries and the binary account for the rest. That rules out file-backed growth (a mapped file or a cache) before you look at a single mapping.

3. Why an `LD_PRELOAD` hook sees nothing

The usual next move is to interpose `mmap`/`munmap` and count calls:

```
/* mmaphook.c - LD_PRELOAD shim that counts calls to the mmap()/munmap() symbols */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <stdio.h>
#include <sys/mman.h>
#include <sys/types.h>

static long n_mmap, n_munmap;

void *mmap(void *addr, size_t len, int prot, int flags, int fd, off_t off)
{
    static void *(*real)(void *, size_t, int, int, int, off_t);
    if (!real) real = dlsym(RTLD_NEXT, "mmap");
    n_mmap++;
    return real(addr, len, prot, flags, fd, off);
}

int munmap(void *addr, size_t len)
{
    static int (*real)(void *, size_t);
    if (!real) real = dlsym(RTLD_NEXT, "munmap");
    n_munmap++;
    return real(addr, len);
}

__attribute__((destructor)) static void report(void)
{
    fprintf(stderr, "[mmaphook] mmap() calls: %ld, munmap() calls: %ld\n",
            n_mmap, n_munmap);
}
```

```
gcc -shared -fPIC -O2 -o mmaphook.so mmaphook.c -ldl
LD_PRELOAD=./mmaphook.so ./leaky 400
```

```
pid 475280, 400 requests, max_unreleased=-1
req   100  VmRSS    8256 KiB  malloc in-use     7 KiB
req   200  VmRSS   14724 KiB  malloc in-use    12 KiB
req   300  VmRSS   21124 KiB  malloc in-use    15 KiB
req   400  VmRSS   27528 KiB  malloc in-use    18 KiB
[mmaphook] mmap() calls: 0, munmap() calls: 0
```

Zero calls, while RSS grew by 19 MB. `LD_PRELOAD` interposes **symbols**. `syscall(SYS_mmap, …)` never calls the `mmap` symbol, and neither does a library that patches its own GOT entries or issues the `syscall` instruction itself. An empty hook log does not mean "no `mmap`". It means "no `mmap` through this door". You need a tracer that sits at the **kernel boundary**.

4. Who: count and attribute syscalls at the kernel boundary

`strace` sees every system call, however it was issued. Launch the program under it rather than attaching:

```
strace -f -c -e trace=mmap,munmap ./leaky 400 2>&1 | tail -n 8
```

```
req   300  VmRSS   21012 KiB  malloc in-use    15 KiB
req   400  VmRSS   27416 KiB  malloc in-use    18 KiB
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00    0.001799           4       408           mmap
  0.00    0.000000           0         1           munmap
------ ----------- ----------- --------- --------- ----------------
100.00    0.001799           4       409           total
```

408 `mmap` (400 requests plus 8 from the dynamic loader) against 1 `munmap`. That is the leak, counted. `-k` adds a user-space stack to each call:

```
strace -k -e trace=mmap ./leaky 2 2>&1 | grep -A 6 'mmap(NULL, 262144' | head -n 7
```

```
mmap(NULL, 262144, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fef2bb1b000
 > /usr/lib/x86_64-linux-gnu/libc.so.6(syscall+0x19) [0x10e4f9]
 > /workspace/scratch/leak/leaky(pool_get+0x38) [0x1291]
 > /workspace/scratch/leak/leaky(handle_request+0x37) [0x146c]
 > /workspace/scratch/leak/leaky(main+0xd9) [0x157c]
 > /usr/lib/x86_64-linux-gnu/libc.so.6(__libc_start_call_main+0x78) [0x29ca8]
 > /usr/lib/x86_64-linux-gnu/libc.so.6(__libc_start_main_alias_2+0x85) [0x29d65]
```

The top frame, **`syscall+0x19`**, is the tell. The call came through glibc's generic `syscall()` wrapper, not through `mmap()`, which is exactly why the hook missed it. The next frame names the culprit,

`pool_get`.

`strace` is the right tool for a reproducer. On a busy production process it is not: every traced syscall costs two `ptrace` stops, and `-p` attaches to (and slows) every thread. That is where `bpftrace` comes in.

5. Who, in production: `bpftrace` on the syscall tracepoints

```
// mmapwatch.bt - who maps anonymous memory, and who gives it back?
tracepoint:syscalls:sys_enter_mmap /pid == cpid && args.fd == -1/
{
    @mmap_bytes[ustack(perf)] = sum(args.len);
    @mmap_calls = count();
}
tracepoint:syscalls:sys_enter_munmap /pid == cpid/
{
    @munmap_bytes = sum(args.len);
    @munmap_calls = count();
}
```

```
sudo bpftrace mmapwatch.bt -c './leaky 400'       # launch under bpftrace (cpid = child)
# or, for a running process: replace "cpid" with "$1" and run
#   sudo bpftrace mmapwatch.bt PID
```

The script sums anonymous `mmap` bytes **per user stack** and totals `munmap`, all in the kernel, with no per-call stop of the target. When the program exits (or on Ctrl-C) bpftrace prints the maps: the stack that ends in `pool_get` would carry about 100 MiB with 400 calls, against a near-zero `munmap` total.

Not run on this book's test box. The test VM's kernel exposes no tracefs and no kprobe/uprobe PMUs (`/sys/bus/event_source/devices` lists only `breakpoint msr power software`), so the run fails before attaching:

```
mmapwatch.bt:1-2: ERROR: tracepoint not found: syscalls:sys_enter_mmap
```

Kprobe and uprobe variants fail on the same box with `ERROR: Unknown BPF object load failure`. This is common in sandboxes, some containers and minimal cloud kernels. Check before an incident with `sudo bpftrace -l 'tracepoint:syscalls:sys_enter_mmap'`. If it prints nothing, use `strace -k` (step 4) on a reproducer, or the gdb approach in step 6.

Two field notes for when it does run:

- `ustack()` depends on frame pointers (or DWARF unwinding, which bpftrace does not do in-kernel). A library built without frame pointers shows **one** frame (`syscall+N`) and then nothing. That single frame is still enough to drive step 6.
- Add `tracepoint:syscalls:sys_enter_mremap` if `pmap` shows a region growing **in place** rather than moving.

6. Prove it: scripted `gdb` that stops only on the guilty call

`gdb` gives the complete stack *and* the address returned, which you can match against the `pmap` region. Put a breakpoint on glibc's `syscall()` with a condition on the syscall number. On x86-64 the first argument, `$rdi`, is the number (`9` = `mmap`) and the third, `$rdx`, is the length. A second breakpoint just after the `syscall` instruction reads `$rax`, the kernel's return value.

First find that offset in *your* glibc:

```
gdb -q -batch -ex 'disassemble syscall' /lib/x86_64-linux-gnu/libc.so.6 | head -n 12
```

```
Dump of assembler code for function syscall:
   0x000000000010e4e0 <+0>: mov    %rdi,%rax
   0x000000000010e4e3 <+3>: mov    %rsi,%rdi
   0x000000000010e4e6 <+6>: mov    %rdx,%rsi
   0x000000000010e4e9 <+9>: mov    %rcx,%rdx
   0x000000000010e4ec <+12>:    mov    %r8,%r10
   0x000000000010e4ef <+15>:    mov    %r9,%r8
   0x000000000010e4f2 <+18>:    mov    0x8(%rsp),%r9
   0x000000000010e4f7 <+23>:    syscall
   0x000000000010e4f9 <+25>:    cmp    $0xfffffffffffff001,%rax
   0x000000000010e4ff <+31>:    jae    0x10e502 <syscall+34>
   0x000000000010e501 <+33>:    ret
```

`+25` is the instruction after `syscall`. It matches the `syscall+0x19` frame from `strace -k`. Other glibc builds differ (the vLLM investigation saw `syscall+29`), so always check.

```
# mmap-catch.gdb - stop only on raw syscall(SYS_mmap, ...), print who and what
set pagination off
set $want = 0

# entry: rdi = syscall number (9 = mmap on x86-64), rdx = length
break syscall if $rdi == 9
commands
  silent
  set $want = 1
  printf "syscall(SYS_mmap, len=%lu) from:\n", $rdx
  bt 4
  continue
end

# right after the `syscall` instruction: rax = the kernel's return value
break *syscall+25 if $want == 1
commands
  silent
  set $want = 0
  printf "  -> mapped at 0x%lx\n\n", $rax
```

```
  continue
end

# just before the process exits, show where those mappings ended up
catch syscall exit_group
commands
  silent
  info proc mappings
  continue
end

run 3
```

```
gdb -q -batch -x mmap-catch.gdb ./leaky 2>&1 | grep -v '^\['
```

```
Breakpoint 1 at 0x10c0
Breakpoint 2 at 0x10d9
Catchpoint 3 (syscall 'exit_group' [231])
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
pid 475570, 3 requests, max_unreleased=-1
syscall(SYS_mmap, len=262144) from:
#0  syscall () at ../sysdeps/unix/sysv/linux/x86_64/syscall.S:30
#1  0x0000555555555291 in pool_get () at leaky.c:29
#2  0x000055555555546c in handle_request (i=1) at leaky.c:68
#3  0x000055555555557c in main (argc=2, argv=0x7fffffffde68) at leaky.c:84
  -> mapped at 0x7ffff7d7c000

req     1  VmRSS    1820 KiB  malloc in-use     4 KiB
syscall(SYS_mmap, len=262144) from:
#0  syscall () at ../sysdeps/unix/sysv/linux/x86_64/syscall.S:30
#1  0x0000555555555291 in pool_get () at leaky.c:29
#2  0x000055555555546c in handle_request (i=2) at leaky.c:68
#3  0x000055555555557c in main (argc=2, argv=0x7fffffffde68) at leaky.c:84
  -> mapped at 0x7ffff7d3c000

req     2  VmRSS    1948 KiB  malloc in-use     6 KiB
syscall(SYS_mmap, len=262144) from:
#0  syscall () at ../sysdeps/unix/sysv/linux/x86_64/syscall.S:30
#1  0x0000555555555291 in pool_get () at leaky.c:29
#2  0x000055555555546c in handle_request (i=3) at leaky.c:68
#3  0x000055555555557c in main (argc=2, argv=0x7fffffffde68) at leaky.c:84
  -> mapped at 0x7ffff7cfc000

req     3  VmRSS    2012 KiB  malloc in-use     6 KiB
process 475570
Mapped address spaces:
```

```
Start Addr         End Addr           Size               Offset             Perms File
0x0000555555554000 0x0000555555555000 0x1000             0x0                r--p  /workspace
...
0x0000555555559000 0x000055555557a000 0x21000            0x0                rw-p  [heap]
0x00007ffff7cfc000 0x00007ffff7dbf000 0xc3000            0x0                rw-p
0x00007ffff7dbf000 0x00007ffff7de7000 0x28000            0x0                r--p  /usr/lib/x
...
```

(The `...` lines are elided from the full `info proc mappings` table: the remaining library, `[vvar]`, `[vdso]`, `[stack]` and `[vsyscall]` mappings.)

Each hit prints the full stack to `leaky.c:29`, and the returned address. The three returns step down by `0x40000` (256 KiB), and the final map shows them merged into one anonymous region, `0x7ffff7cfc000-0x7ffff7dbf000`, that starts at the last address returned. That is the same "growing and moving down" region that `pmap` flagged in step 2, now tied to a line of source.

The "Breakpoint 1 at 0x10c0" line is gdb resolving `syscall` to the program's PLT stub before libc is loaded. After `run`, it re-resolves both breakpoints into libc, as the `#0 syscall () at …syscall.S` frames show.

Two practical notes:

- `finish` cannot be used inside breakpoint `commands` (it resumes and loses the remaining commands). Hence the `$want` flag and the second breakpoint at a fixed offset.
- In production the same script would start with `attach PID` instead of `run`. That stops **every thread** while gdb sets up and on every breakpoint hit. Do it on a canary instance, for seconds, never on the only replica.

7. Fix and verify

Here the fix is to bound the deferred-release list:

```
POOL_MAX_UNRELEASED=16 ./leaky 2000
```

```
pid 473970, 2000 requests, max_unreleased=16
req   500  VmRSS    2256 KiB  malloc in-use     4 KiB
req  1000  VmRSS    2704 KiB  malloc in-use     6 KiB
req  1500  VmRSS    2064 KiB  malloc in-use     6 KiB
req  2000  VmRSS    2512 KiB  malloc in-use     6 KiB
```

RSS stays flat at about 2–3 MB, against 130 MB for the same 2,000 requests in step 1. Verify a fix in production the same way you found the leak: `top-maps.sh` should show no anonymous region that keeps growing.

Real-world case. The design of `leaky.c` mirrors a 2025 leak in vLLM's prefill/decode-disaggregated serving. The UCX communication library (used through NIXL) intercepts `mmap`/`munmap` for its registration cache. It allocated with raw `syscall(SYS_mmap)`, and it parked "released" regions in an invalidation queue whose default limit (`UCX_RCACHE_MAX_UNRELEASED`) was

unbounded. Heap profilers saw nothing, `LD_PRELOAD` hooks saw nothing, `pmap` showed growing, moving anonymous regions, `bpftrace` found `syscall+29`, and conditional gdb breakpoints named `ucm_mmap`. The workaround merged into vLLM sets `UCX_MEM_MMAP_HOOK_MODE=none`. Capping `UCX_RCACHE_MAX_UNRELEASED` is the alternative knob.

The trap

Watching the wrong PID, or the wrong half of a pipe.

The program you started is often not the process holding the memory. Depending on the shell and how the job line is written, `$!` can be a forked shell rather than the program, and `uv run`, `npm`, `sudo` and container shims all sit in front of the real process. In step 2 the script used the PID that `leaky` printed. Here is what `$!` was in the test box's non-interactive bash, for the same job line:

```
./leaky 8000 > /tmp/leaky.out & P=$!; sleep 2; ./top-maps.sh "$P"
```

```
Address           Kbytes     RSS   Dirty Mode  Mapping
total kB            4764    2112     544
00007f3c2fe7f000    1420     700       0 r-x-- libc.so.6
000055d9eaf9b000     804     652       0 r-x-- bash
000055da0a97b000     404     340     340 rw---   [ anon ]
```

That is a 4.7 MB `bash`, not the 450 MB leaker. It looks healthy, and it stays healthy, so you conclude "no leak". Check with `ps -o pid,ppid,comm --ppid "$P" -p "$P"` before watching anything.

A second, quieter version of the same trap is the "keep the header" idiom that appears in many write-ups:

```
pmap -x "$P" | (head -n 2; tail -n +3 | sort -k3 -nr | head -n 3)
```

```
475171:   ./leaky 3000
Address           Kbytes     RSS   Dirty Mode  Mapping
```

`head` reads a whole buffer from the pipe, not just two lines, so `tail` gets what is left, which here is nothing. On a large process `tail` gets the remainder of the buffer, so the first few kilobytes of mappings silently **disappear from the sort**. Run `pmap` twice (as `top-maps.sh` does) or use `pmap -q`.

Notes & Pitfalls

Pitfall	Symptom / fix
Trusting heap profilers for RSS	"No leak" while RSS climbs. Compare `mallinfo2`/profiler totals with `VmRSS` and `smaps_rollup` first
Looking only at RSS	Reserved address space (`Kbytes`) can grow much faster. Watch both columns

Pitfall	Symptom / fix
One big `[ anon ]` line looks like one allocation	Adjacent anonymous mappings merge. Use the moving start address, or `strace`/gdb, to see individual calls
`LD_PRELOAD` hook shows nothing	Raw `syscall()`, GOT patching and static binaries bypass symbol interposition. Trace at the kernel boundary
`bpftrace` "tracepoint not found"	No tracefs, or a locked-down kernel. `sudo bpftrace -l 'tracepoint:syscalls:*mmap*'` tells you in advance
One-frame `ustack()`	The library was built without frame pointers. Use the top frame (`syscall+N`) as a gdb breakpoint target
Inlined code in stacks	Production builds inline aggressively. Use `-g` plus `addr2line -i` or gdb's inline frames, or reproduce at `-O0`
`gdb`/`strace -p` on production	Stops or slows all threads. Prefer launching under the tool, a canary, or bpftrace
`pmap -X` column positions	Columns follow the kernel's `smaps` fields and change between kernels. Select fields by name

The boring rule

When RSS grows: `smaps_rollup` first (anon or file?), then `pmap -x` twice, a few seconds apart (which mapping, and does it move?), then count `mmap` vs `munmap` at the kernel boundary (`bpftrace` in production, `strace -c`/`-k` on a reproducer), then a conditional gdb breakpoint on the one call site you found, on a canary. Fix by bounding the cache or pool, and verify with the same `pmap` loop.

Related Commands

Command	Role
`strace -c` / `strace -k`	Syscall counts and per-call user stacks (launch, don't attach, when you can)
`bpftrace`	In-kernel syscall aggregation with stacks, without stopping the target
`gdb`	Conditional breakpoints, full stacks, register values
`ltrace`	Library-call tracing (symbol-level, so it shares `LD_PRELOAD`'s blind spot for raw syscalls)
`perf`	Sampling profiles; `perf trace` for syscalls where tracepoints exist
`/proc/PID/smaps_rollup`	One-shot totals: `Rss`, `Pss_Anon`, `Anonymous`, `Swap`

Try this

1. Run `leaky` with `POOL_MAX_UNRELEASED=0`, `16` and unset, and watch `top-maps.sh` each time. At what list size does the anonymous region stop growing?
2. Change `pool_get` to call `mmap()` instead of `syscall(SYS_mmap, …)`, rebuild, and rerun step 3. What does `mmaphook.so` report now, and which `strace -k` frame changes?
3. Rebuild `leaky` with `-O2` and repeat steps 4 and 6. Which frames disappear, and can you still identify the call site?
4. On a host where `bpftrace -l 'tracepoint:syscalls:sys_enter_mmap'` prints a line, run `mmapwatch.bt` against `leaky` and compare its byte totals with `strace -c`.
5. Add `tracepoint:syscalls:sys_enter_mremap` to `mmapwatch.bt` and write a variant of `leaky.c` that grows one buffer with `mremap`. How does it look in `pmap` compared with the `mmap` leak?

Sources

- pmap(1), procps-ng manual
- proc_pid_smaps(5) and proc_pid_maps(5): mapping fields; `smaps_rollup`
- mmap(2), syscall(2)
- strace(1) (`-c`, `-k`)
- bpftrace manual: tracepoints, `args`, `ustack`, `cpid`
- GDB manual: Break Commands and Conditions
- vLLM issue #24264 (CPU memory leak in P/D disaggregation) and PR #32181 (`UCX_MEM_MMAP_HOOK_MODE=none`)

Transparent Huge Pages: /sys/kernel/mm/transparent_hugepage, AnonHugePages and khugepaged

Overview

Thesis: Transparent Huge Pages (THP) let the kernel back anonymous memory with 2 MiB pages instead of 4 KiB ones, without the application asking. One 2 MiB page is mapped by a single page-middle-directory (PMD) entry instead of 512 PTEs, so it costs one page fault instead of 512, and one TLB entry instead of 512. The price is paid elsewhere: the kernel may stall a page fault to **compact** memory into a free 2 MiB block, a sparse write can make a process resident for 2 MiB instead of 4 KiB, and `fork()` plus writes splits huge mappings back into small ones. Which side wins depends on the workload, so the kernel offers a policy (`always`, `madvise`, `never`), and every serious database (Redis, Valkey, MongoDB, many JVM guides) tells you what it wants. This page shows how to read the policy, see which processes really have huge pages, measure the fault, bloat and copy-on-write effects, and turn THP on or off for one process or one range.

Typical operator jobs:

- Answer "is THP on?" correctly. It is not one switch: there is a global policy, a defrag policy, per-size policies (mTHP), a per-process flag, and per-range advice.
- Find out which processes actually hold huge pages (`AnonHugePages`), and whether faults are falling back or stalling in compaction (`/proc/vmstat`).
- Follow a database's advice ("disable THP") without breaking the JVM next to it that wants THP, using `madvise` and per-process controls.
- Explain a latency spike or an RSS jump after enabling `always`.

```
uname -r
cat /sys/kernel/mm/transparent_hugepage/enabled
cat /sys/kernel/mm/transparent_hugepage/hpage_pmd_size
```

```
6.12.94+
always [madvise] never
2097152
```

All output below comes from a Debian 13 host (a virtual machine) with 16 GiB of RAM, **no swap**, kernel 6.12, x86-64, run as an unprivileged user (`uid=1000`, no capabilities). Python 3.13.5 and GCC 14.2 come with the box; nothing was installed. Every `/sys/kernel/mm/transparent_hugepage` file is root-writable only, so all policy *changes* in this page are described from the kernel documentation

and not run here. Everything a normal user can do (read the policy, `madvise`, `prctl`, read `smaps`) was run.

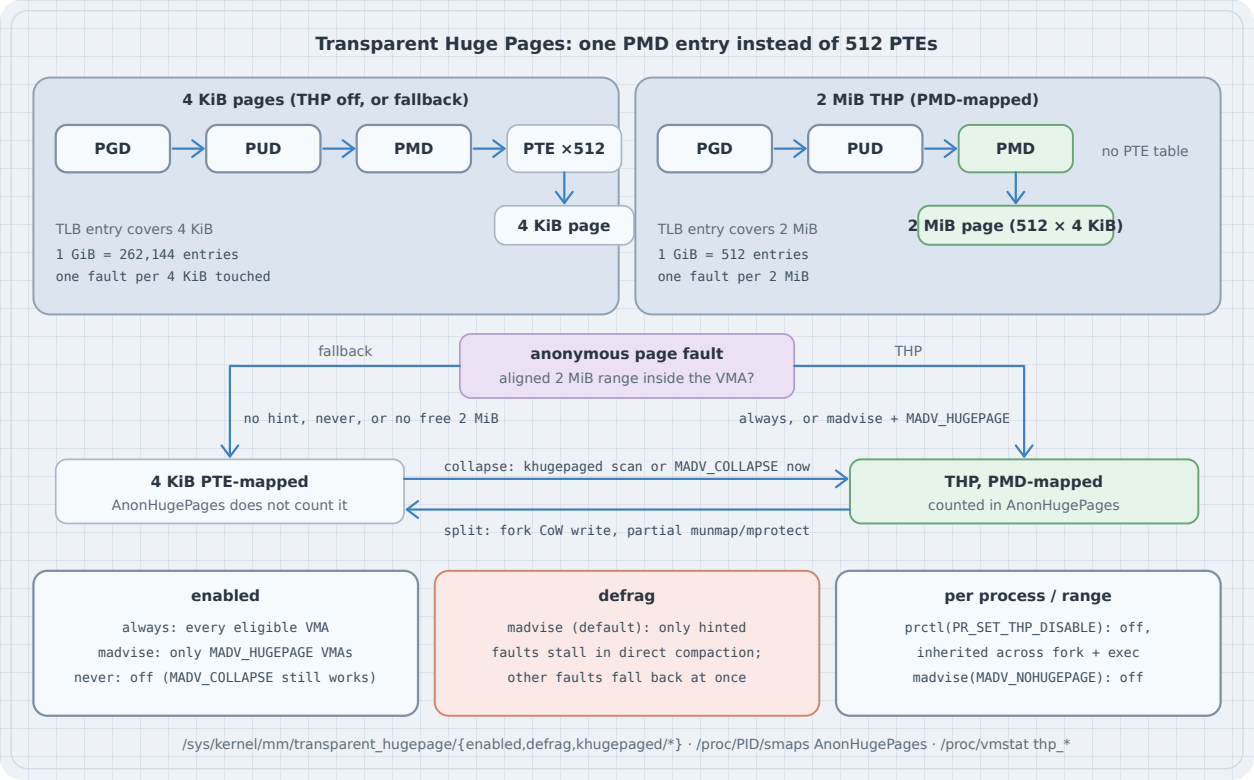


Figure 1: Transparent Huge Pages: page tables, the fault and collapse/split lifecycle, and the controls

Mental model

```
anonymous write fault at address A

        is the aligned 2 MiB range around A inside one VMA,
        and is THP allowed here?
            enabled=always                          → yes
            enabled=madvise and VMA has MADV_HUGEPAGE → yes
            enabled=never, MADV_NOHUGEPAGE, PR_SET_THP_DISABLE → no

try to allocate 2 MiB of physically contiguous memory
        free block?           → map it with one PMD entry   (thp_fault_alloc)
        none: defrag policy says stall? → direct compaction  (compact_stall)
        still none / no stall → fall back to a 4 KiB page    (thp_fault_fallback)

later, in the background or on request
      khugepaged / MADV_COLLAPSE: 512 small pages → one THP   (thp_collapse_alloc)
```

```
        CoW after fork, partial munmap/mprotect: PMD → 512 PTEs (thp_split_pmd)
```

Four facts carry the topic:

1. **THP is about anonymous memory** (heap, `mmap(MAP_ANONYMOUS)`) and tmpfs/shmem. Page-cache THPs for regular files exist on newer kernels but are a separate story. `AnonHugePages` counts only PMD-mapped anonymous THPs.
2. **The win is mostly the TLB, not the fault.** The kernel documentation says the 512× fewer faults are "almost completely irrelevant" next to the TLB effect: fewer misses, and a shorter page walk per miss. It also says that inside a VM the TLB only gets 2 MiB entries if *both* the guest and the hypervisor use huge pages.
3. **The cost is contiguity.** A 2 MiB page needs 512 physically contiguous, aligned 4 KiB frames. On a long-running host they are often not free, so the kernel must either compact memory (a stall, inside your page fault) or give up (fallback).
4. **Policies only affect future faults.** Changing `enabled` does not split or collapse existing pages. Restart the process to get the new behaviour everywhere.

Syntax

There is no `thp` command. The interface is files and two system calls:

```
cat  /sys/kernel/mm/transparent_hugepage/enabled           # global policy, brackets = curre
echo madvise > /sys/kernel/mm/transparent_hugepage/enabled  # root only; lost on reboot
grep AnonHugePages /proc/meminfo                            # system-wide PMD-mapped anon TH
grep AnonHugePages /proc/PID/smaps_rollup                   # one process
grep -E '^thp_|^compact_stall' /proc/vmstat                 # success, fallback, split count
```

```
madvise(addr, len, MADV_HUGEPAGE);    // allow THP for this range (needed when enabled=madvi
madvise(addr, len, MADV_NOHUGEPAGE);  // forbid THP for this range
madvise(addr, len, MADV_COLLAPSE);    // collapse this range to THPs now (Linux 6.1+)
prctl(PR_SET_THP_DISABLE, 1, 0, 0, 0); // no THP for this process; inherited by fork and exe
```

File under `/sys/kernel/mm/transparent_hugepage/`	Meaning
`enabled`	`always`: any eligible anonymous VMA. `madvise`: only `MADV_HUGEPAGE` ranges. `never`: off (but see `MADV_COLLAPSE`).
`defrag`	What a fault does when no 2 MiB block is free. `always`: stall in direct reclaim/compaction. `defer`: wake `kswapd`/`kcompactd`, fall back now. `defer+madvise`: stall for `MADV_HUGEPAGE` ranges, defer for others. `madvise` (default): stall only for `MADV_HUGEPAGE` ranges. `never`: never stall.

File under /sys/kernel/mm/transparent_hugepage/	Meaning
hugepages-<size>kB/enabled	Per-size policy for multi-size THP (mTHP, Linux 6.8+): always, inherit (use top-level enabled), madvise, never. By default 2048 kB is inherit and the rest never.
shmem_enabled	Policy for tmpfs/shmem (always, within_size, advise, never, deny, force).
use_zero_page	1: a read fault maps the shared huge zero page.
shrink_underused	1: under memory pressure, split THPs whose zero-filled subpages exceed khugepaged/max_ptes_none.
khugepaged/pages_to_scan, scan_sleep_millisecs	Background collapse rate: pages scanned per pass, and sleep between passes.
khugepaged/max_ptes_none	How many of the 512 small pages may be *unmapped* and still be collapsed (filled with zeros). 511 = collapse even a range with one page in it.
khugepaged/max_ptes_swap, max_ptes_shared	How many swapped-out or shared pages may be in a range khugepaged collapses.
khugepaged/pages_collapsed, full_scans	Progress counters.

Boot-time default: transparent_hugepage=always|madvise|never on the kernel command line, and thp_anon= for per-size policy. Distributions choose differently; this Debian kernel defaults to madvise, and some distributions ship always.

Safety

- Changing enabled or defrag is root-only and affects every process on the host, including ones you do not own. Change it in a maintenance window. It does not undo existing huge pages; restart the services that matter.
- Setting khugepaged/scan_sleep_millisecs to 0 makes khugepaged spin on one CPU. Setting max_ptes_none high (the default 511) lets khugepaged *grow* RSS: a range with one resident 4 KiB page becomes 2 MiB.
- enabled=always with defrag=always puts compaction directly into page faults for every process. That is a known source of multi-millisecond latency spikes on fragmented hosts.
- Writes to /sys do not survive a reboot. Persist with the kernel command line, or a boot-time unit such as a tmpfiles.d line w /sys/kernel/mm/transparent_hugepage/enabled - - - - madvise.

Examples with Explanations

1. Read the whole policy, not just `enabled`

```
#!/bin/sh
# thp_settings.sh - read-only THP report: policy, khugepaged, mTHP sizes, counters
T=/sys/kernel/mm/transparent_hugepage
echo "kernel $(uname -r), PMD huge page $(( $(cat $T/hpage_pmd_size) / 1024 )) KiB"
for f in enabled defrag shmem_enabled use_zero_page shrink_underused; do
    [ -r "$T/$f" ] && printf '  %-16s %s\n' "$f" "$(cat $T/$f)"
done
echo "khugepaged:"
for f in pages_to_scan scan_sleep_millisecs alloc_sleep_millisecs defrag \
         max_ptes_none max_ptes_swap max_ptes_shared full_scans pages_collapsed; do
    printf '  %-22s %s\n' "$f" "$(cat $T/khugepaged/$f)"
done
echo "per-size anon policy (mTHP):"
for d in $T/hugepages-*kB; do
    if [ -r "$d/enabled" ]; then pol=$(cat "$d/enabled"); else pol="(no anon control: shmem-
    printf '  %-10s %s\n' "${d##*/hugepages-}" "$pol"
done | sort -n
echo "counters (/proc/vmstat):"
grep -E '^(thp_fault_alloc|thp_fault_fallback|thp_collapse_alloc|thp_split_pmd|compact_stall
grep AnonHugePages /proc/meminfo | sed 's/^/  /'
```

```
./thp_settings.sh
```

```
kernel 6.12.94+, PMD huge page 2048 KiB
  enabled          always [madvise] never
  defrag           always defer defer+madvise [madvise] never
  shmem_enabled    always within_size advise [never] deny force
  use_zero_page    1
  shrink_underused 1
khugepaged:
  pages_to_scan          4096
  scan_sleep_millisecs   10000
  alloc_sleep_millisecs  60000
  defrag                 1
  max_ptes_none          511
  max_ptes_swap          64
  max_ptes_shared        256
  full_scans             32
  pages_collapsed        22
per-size anon policy (mTHP):
  8kB        (no anon control: shmem-only size)
```

```
  16kB       always inherit madvise [never]
  32kB       always inherit madvise [never]
  64kB       always inherit madvise [never]
  128kB      always inherit madvise [never]
  256kB      always inherit madvise [never]
  512kB      always inherit madvise [never]
  1024kB     always inherit madvise [never]
  2048kB     always [inherit] madvise never
counters (/proc/vmstat):
  compact_stall 2631
  thp_fault_alloc 3006
  thp_fault_fallback 706
  thp_collapse_alloc 118
  thp_split_pmd 204
  AnonHugePages:         0 kB
```

How to read it:

- `enabled [madvise]` with `hugepages-2048kB [inherit]`: 2 MiB THPs only where a program asked with `MADV_HUGEPAGE`. All smaller mTHP sizes are `never`. `8kB` has no anonymous control because anonymous mTHP starts at order 2 (16 kB); order 1 exists only for shmem/file memory.
- `defrag [madvise]`: a hinted fault may stall in compaction, an unhinted one never does.
- `max_ptes_none 511`: khugepaged would collapse a range with a single resident page.
- The counters are cumulative since boot and system-wide. `thp_fault_fallback` 706 next to `thp_fault_alloc` 3006 says about one in five huge-page faults on this host could not get a 2 MiB block. `compact_stall` 2631 says processes have stalled in direct compaction thousands of times. Read them as deltas around an event, as the next examples do.
- `AnonHugePages: 0 kB`: right now nobody on this host holds a PMD-mapped anonymous THP.

2. What `MADV_HUGEPAGE` changes on first touch

A small helper module is shared by the Python listings below. It uses only the standard library: `ctypes` calls `madvise(2)` and `prctl(2)` directly, because Python's `mmap` module has `MADV_HUGEPAGE` but not `MADV_COLLAPSE`.

```
# thp_util.py - small helpers shared by the THP listings (Linux, Python 3 stdlib only)
import ctypes, mmap, os, re, resource

MB = 1 << 20
MADV_COLLAPSE = 25          # <asm-generic/mman-common.h>, Linux 6.1+; Python's mmap module
PR_SET_THP_DISABLE = 41     # <linux/prctl.h>, Linux 3.15+
PR_GET_THP_DISABLE = 42
libc = ctypes.CDLL(None, use_errno=True)
libc.madvise.argtypes = [ctypes.c_void_p, ctypes.c_size_t, ctypes.c_int]
libc.prctl.argtypes = [ctypes.c_int, ctypes.c_ulong, ctypes.c_ulong, ctypes.c_ulong, ctypes.
```

```
def anon_map(size):
    """Private anonymous mapping; returns (mmap object, start address)."""
    m = mmap.mmap(-1, size, flags=mmap.MAP_PRIVATE | mmap.MAP_ANONYMOUS)
    return m, ctypes.addressof(ctypes.c_char.from_buffer(m))

def madvise(addr, size, advice):
    """Raw madvise(2) so we can pass advice values the mmap module does not know."""
    if libc.madvise(addr, size, advice) != 0:
        e = ctypes.get_errno()
        return f"{os.strerror(e)} ({e})"
    return "ok"

def touch(m, size, step=4096):
    """Write one byte every `step` bytes; returns minor page faults taken."""
    before = resource.getrusage(resource.RUSAGE_SELF).ru_minflt
    for off in range(0, size, step):
        m[off] = 1
    return resource.getrusage(resource.RUSAGE_SELF).ru_minflt - before

def smaps(addr, pid="self"):
    """The /proc/PID/smaps fields of the VMA that starts at addr (sizes in kB)."""
    fields, mine = {}, False
    with open(f"/proc/{pid}/smaps") as f:
        for line in f:
            head = line.split()
            if re.match(r"^[0-9a-f]+-[0-9a-f]+$", head[0]):        # a new VMA header line
                if mine:
                    break
                mine = head[0].startswith(f"{addr:x}-")
            elif mine:
                fields[head[0].rstrip(":")] = int(head[1]) if head[1].isdigit() else head[1]
    return fields

def vmstat(*names):
    with open("/proc/vmstat") as f:
        d = dict(line.split() for line in f)
    return {n: int(d.get(n, 0)) for n in names}
```

```
# thp_fault.py - what MADV_HUGEPAGE changes on first touch (enabled=madvise host)
import mmap, time
from thp_util import MB, anon_map, madvise, touch, smaps, vmstat

SIZE = 256 * MB
print(open("/sys/kernel/mm/transparent_hugepage/enabled").read().strip(), "<- enabled")
print(f"{'mapping':<15}{'2M-aligned':>11}{'minor faults':>14}{'touch ms':>10}{'Rss kB':>9}"
      f"{'AnonHuge kB':>13}{'fault_alloc +':>15}{'fallback +':>12}{'compact_stall +':>17}")
```

```
C = ("thp_fault_alloc", "thp_fault_fallback", "compact_stall")
for label, advice in (("plain mmap", None), ("MADV_HUGEPAGE", mmap.MADV_HUGEPAGE)):
    m, addr = anon_map(SIZE)
    if advice is not None:
        assert madvise(addr, SIZE, advice) == "ok"
    v0 = vmstat(*C)
    t0 = time.perf_counter()
    faults = touch(m, SIZE)
    ms = (time.perf_counter() - t0) * 1000
    v1 = vmstat(*C)
    d = [v1[k] - v0[k] for k in C]
    s = smaps(addr)
    print(f"{label:<15}{str(addr % (2 * MB) == 0):>11}{faults:>14}{ms:>10.0f}{s['Rss']:>9}"
          f"{s['AnonHugePages']:>13}{d[0]:>15}{d[1]:>12}{d[2]:>17}")
    m.close()
```

```
python3 thp_fault.py; sleep 5; python3 thp_fault.py
```

```
always [madvise] never <- enabled
mapping         2M-aligned  minor faults  touch ms   Rss kB  AnonHuge kB  fault_alloc +  fal
plain mmap            True         65536       198   262144            0              0
MADV_HUGEPAGE         True          4216       267   262144       245760            120
always [madvise] never <- enabled
mapping         2M-aligned  minor faults  touch ms   Rss kB  AnonHuge kB  fault_alloc +  fal
plain mmap            True         65536       208   262144            0              0
MADV_HUGEPAGE         True           128       194   262144       262144            128
```

Read it row by row:

- **Plain mapping:** 65,536 faults for 256 MiB, one per 4 KiB page, and `AnonHugePages` 0. With `enabled=madvise` an unhinted range never gets THPs.
- **`MADV_HUGEPAGE`, second run:** 128 faults, one per 2 MiB, the whole 256 MiB is huge, and `thp_fault_alloc` went up by 128. Still, `compact_stall` rose by 17: even a fully successful run can stall in direct compaction first.
- **`MADV_HUGEPAGE`, first run:** only 120 of 128 blocks got a THP. 8 fell back to small pages (hence 4,216 faults), and there were 68 compaction stalls. The same program on the same host gets a different answer depending on how fragmented physical memory is at that moment.
- **The time column is not a THP benchmark.** It is dominated by the Python loop writing 65,536 bytes either way. It shows that the hinted run is not reliably faster on first touch: it trades 512× fewer faults for zeroing 2 MiB at a time and sometimes compacting.

The mapping is 2 MiB-aligned without asking: Linux 6.12 places anonymous mappings whose length is a multiple of 2 MiB on a 2 MiB boundary (`__get_unmapped_area()` in `mm/mmap.c`; older kernels may not). A THP can only map an aligned 2 MiB range that lies wholly inside one VMA. With a misaligned or smaller mapping, the ragged ends always use 4 KiB pages.

3. "Touch 1 byte, get 2 MiB": the bloat effect

```
# thp_bloat.py - "touch 1 byte, get 2 MiB": sparse writes with and without THP
import mmap
from thp_util import MB, anon_map, madvise, touch, smaps

SIZE = 64 * MB
print(f"{'mapping':<16}{'bytes written':>14}{'minor faults':>14}{'Rss kB':>9}{'AnonHugePages
for label, advice in (("plain mmap", None), ("MADV_HUGEPAGE", mmap.MADV_HUGEPAGE)):
    m, addr = anon_map(SIZE)
    if advice is not None:
        madvise(addr, SIZE, advice)
    faults = touch(m, SIZE, step=2 * MB)        # one byte in each 2 MiB block
    s = smaps(addr)
    print(f"{label:<16}{SIZE // (2 * MB):>14}{faults:>14}{s['Rss']:>9}{s['AnonHugePages']:>1
    m.close()
```

```
python3 thp_bloat.py
```

```
mapping          bytes written  minor faults   Rss kB  AnonHugePages kB
plain mmap                  32            32      128                 0
MADV_HUGEPAGE               32            32    65536             65536
```

The same 32 one-byte writes make the process resident for 128 KiB with small pages and for **64 MiB** with THP: 512 times more. This is the case the kernel documentation warns about ("an application may mmap a large region but only touch 1 byte of it"). It is why `enabled=always` can raise RSS for allocators that leave holes in their arenas, and why `madvise` is the conservative default. `shrink_underused=1` lets the kernel split such mostly-zero THPs again, but only under memory pressure.

4. Collapse now, collapse later, or never

`MADV_HUGEPAGE` affects *future* faults. Memory already faulted with 4 KiB pages stays small until something collapses it: khugepaged in the background, or `MADV_COLLAPSE` synchronously.

```
# thp_collapse.py - MADV_COLLAPSE (synchronous collapse) and PR_SET_THP_DISABLE
import sys
from thp_util import (MB, MADV_COLLAPSE, PR_SET_THP_DISABLE, PR_GET_THP_DISABLE,
                      libc, anon_map, madvise, touch, smaps)
import mmap

SIZE = 64 * MB

def show(step, addr):
    s = smaps(addr)
```

```
    print(f"  {step:<44} AnonHugePages {s['AnonHugePages']:>6} kB   THPeligible {s['THPeligi

mode = sys.argv[1] if len(sys.argv) > 1 else "hint"       # hint | collapse-only | disabled
if mode == "disabled":
    print("prctl(PR_SET_THP_DISABLE, 1) ->", libc.prctl(PR_SET_THP_DISABLE, 1, 0, 0, 0),
          "| PR_GET_THP_DISABLE ->", libc.prctl(PR_GET_THP_DISABLE, 0, 0, 0, 0))
print(f"mode {mode}:")
m, addr = anon_map(SIZE)
touch(m, SIZE)
show("touched with 4 KiB pages, no advice", addr)
if mode != "collapse-only":
    print("  madvise(MADV_HUGEPAGE)  ->", madvise(addr, SIZE, mmap.MADV_HUGEPAGE))
    show("after MADV_HUGEPAGE (nothing re-faults)", addr)
print("  madvise(MADV_COLLAPSE)  ->", madvise(addr, SIZE, MADV_COLLAPSE))
show("after MADV_COLLAPSE", addr)
```

```
for m in hint collapse-only disabled; do python3 thp_collapse.py $m; done
```

```
mode hint:
  touched with 4 KiB pages, no advice          AnonHugePages      0 kB   THPeligible 0
  madvise(MADV_HUGEPAGE)  -> ok
  after MADV_HUGEPAGE (nothing re-faults)      AnonHugePages      0 kB   THPeligible 1
  madvise(MADV_COLLAPSE)  -> ok
  after MADV_COLLAPSE                          AnonHugePages  65536 kB   THPeligible 1
mode collapse-only:
  touched with 4 KiB pages, no advice          AnonHugePages      0 kB   THPeligible 0
  madvise(MADV_COLLAPSE)  -> ok
  after MADV_COLLAPSE                          AnonHugePages  65536 kB   THPeligible 0
prctl(PR_SET_THP_DISABLE, 1) -> 0 | PR_GET_THP_DISABLE -> 1
mode disabled:
  touched with 4 KiB pages, no advice          AnonHugePages      0 kB   THPeligible 0
  madvise(MADV_HUGEPAGE)  -> ok
  after MADV_HUGEPAGE (nothing re-faults)      AnonHugePages      0 kB   THPeligible 0
  madvise(MADV_COLLAPSE)  -> Invalid argument (22)
  after MADV_COLLAPSE                          AnonHugePages      0 kB   THPeligible 0
```

What it shows:

- **hint:** `MADV_HUGEPAGE` alone changes nothing in memory that is already resident (it only flips `THPeligible` to 1). `MADV_COLLAPSE` turns the 64 MiB into THPs immediately, in the calling thread, paying the copy cost there.
- **collapse-only:** `MADV_COLLAPSE` works without `MADV_HUGEPAGE`, even with `enabled=madvise`. The kernel documentation says it ignores the sysfs `enabled` setting, even `never`. The range is collapsed although it is still not eligible for *fault-time* THPs (`THPeligible 0`).
- **disabled:** after `prctl(PR_SET_THP_DISABLE, 1)` both the hint and the collapse are refused. `MADV_COLLAPSE` fails with `EINVAL` because `thp_vma_allowable_order()` sees the process's `MMF_DISABLE_THP` flag (`mm/khugepaged.c`, `include/linux/huge_mm.h` in 6.12). This is the

only control that beats `MADV_COLLAPSE`, apart from `MADV_NOHUGEPAGE` on the range.

Without `MADV_COLLAPSE`, khugepaged gets there eventually:

```
# thp_khugepaged.py - let khugepaged collapse an MADV_HUGEPAGE region in the background
import mmap, time
from thp_util import MB, anon_map, madvise, touch, smaps

KH = "/sys/kernel/mm/transparent_hugepage/khugepaged/"
def kh(name):
    return int(open(KH + name).read())

SIZE = 16 * MB
m, addr = anon_map(SIZE)
touch(m, SIZE)                                  # 4096 small pages, faulted before the hint
madvise(addr, SIZE, mmap.MADV_HUGEPAGE)         # registers this mm with khugepaged
print(f"scan_sleep_millisecs={kh('scan_sleep_millisecs')} pages_to_scan={kh('pages_to_scan')} "
      f"max_ptes_none={kh('max_ptes_none')}")
c0, f0, t0 = kh("pages_collapsed"), kh("full_scans"), time.time()
last = -1
while time.time() - t0 < 180:
    huge = smaps(addr)["AnonHugePages"]
    if huge != last:
        print(f"t={time.time() - t0:5.1f}s  AnonHugePages {huge:>6} kB   "
              f"pages_collapsed +{kh('pages_collapsed') - c0}   full_scans +{kh('full_scans'
        last = huge
    if huge == SIZE // 1024:
        break
    time.sleep(0.5)
```

```
python3 thp_khugepaged.py
```

```
scan_sleep_millisecs=10000 pages_to_scan=4096 max_ptes_none=511
t=  0.0s  AnonHugePages      0 kB   pages_collapsed +0   full_scans +0
t= 10.0s  AnonHugePages  16384 kB   pages_collapsed +8   full_scans +0
```

The `MADV_HUGEPAGE` call registered this process with khugepaged. One pass later (`scan_sleep_millisecs` = 10 s), the 16 MiB had been collapsed into 8 THPs. On a busy host with many registered processes, khugepaged's 4096 pages per 10 s can take minutes to reach your range. That is why "THP coverage slowly creeps up after start" is normal, and why `MADV_COLLAPSE` exists.

5. `fork()` and copy-on-write: the Redis case

Redis, Valkey and similar servers `fork()` to write a snapshot while the parent keeps serving writes. Every write after the fork is a copy-on-write fault on a page shared with the child.

```
# thp_fork_cow.py - copy-on-write of a THP after fork(): what does one write cost?
import mmap, os
from thp_util import MB, anon_map, madvise, touch, smaps, vmstat

SIZE = 64 * MB
m, addr = anon_map(SIZE)
madvise(addr, SIZE, mmap.MADV_HUGEPAGE)
touch(m, SIZE)
s = smaps(addr)
print(f"parent before fork: Rss {s['Rss']} kB, AnonHugePages {s['AnonHugePages']} kB")

v0 = vmstat("thp_split_pmd", "thp_split_page", "thp_deferred_split_page")
r, w = os.pipe()
pid = os.fork()
if pid == 0:                                    # child: one byte into each 2 MiB block
    os.close(r)
    faults = touch(m, SIZE, step=2 * MB)
    s = smaps(addr)
    os.write(w, f"{faults} {s['Private_Dirty']} {s['Shared_Dirty']} {s['AnonHugePages']}".en
    os._exit(0)
os.close(w)
faults, priv, shared, huge = map(int, os.read(r, 200).split())
os.waitpid(pid, 0)
v1 = vmstat(*v0)
print(f"child wrote 32 bytes: {faults} minor faults, Private_Dirty {priv} kB, "
      f"Shared_Dirty {shared} kB, AnonHugePages {huge} kB")
print("vmstat deltas:", {k: v1[k] - v0[k] for k in v0})
s = smaps(addr)
print(f"parent after child exit: Rss {s['Rss']} kB, AnonHugePages {s['AnonHugePages']} kB")
```

```
python3 thp_fork_cow.py
```

```
parent before fork: Rss 65536 kB, AnonHugePages 65536 kB
child wrote 32 bytes: 45 minor faults, Private_Dirty 128 kB, Shared_Dirty 65408 kB, AnonHuge
vmstat deltas: {'thp_split_pmd': 32, 'thp_split_page': 0, 'thp_deferred_split_page': 0}
parent after child exit: Rss 65536 kB, AnonHugePages 65536 kB
```

32 one-byte writes in the child cost 32 × 4 KiB = **128 KiB** of copied memory, not 32 × 2 MiB. Each write split the writer's PMD (`thp_split_pmd` +32) and copied one 4 KiB page; the rest stayed shared (`Shared_Dirty` 65,408 kB). The writer's mapping is no longer PMD-mapped (`AnonHugePages 0`), while the parent kept its THPs.

This is Linux 5.8+ behaviour (`do_huge_pmd_wp_page()` in `mm/huge_memory.c` ends in `__split_huge_pmd(...)` and `VM_FAULT_FALLBACK`). The commit that introduced it ("thp: change CoW semantics for anon-THP") names Redis as the workload that made THP unusable before. Older kernels allocated and copied a whole new 2 MiB page on the first write, so a snapshot under write load could double memory and stall. That is where the classic "disable THP for Redis"

advice comes from. On current kernels the remaining costs are the split itself, the loss of huge pages in the writer until khugepaged re-collapses them, the bloat from Example 3, and compaction stalls at fault time from Example 2. That is enough for Redis to still refuse THP, but as a latency choice, not because a fork doubles memory.

What Redis and Valkey actually do (source: `redis.conf`/`valkey.conf`, `src/server.c`, `src/syscheck.c`): at start-up they read `/sys/kernel/mm/transparent_hugepage/enabled`. Only if it says `[always]` do they call `prctl(PR_SET_THP_DISABLE, 1, 0, 0, 0)` for themselves (config `disable-thp yes`, the default). They log the warning "You have Transparent Huge Pages (THP) support enabled in your kernel" only if that call fails. With `madvise` or `never` they do nothing: they never ask for THP, so they never get it.

6. Which processes hold huge pages, and does it help?

```
#!/bin/sh
# thp_top.sh - which processes hold PMD-sized THPs? (smaps_rollup; readable PIDs only)
printf '%8s %8s %12s %12s  %s\n' PID THP_en "Rss kB" "AnonHuge kB" COMMAND
for d in /proc/[0-9]*; do
    r=$(grep -E '^(Rss|AnonHugePages):' "$d/smaps_rollup" 2>/dev/null | awk '{printf "%s ",
    [ -n "$r" ] || continue                    # not ours (or a kernel thread): skip
    en=$(awk '/^THP_enabled:/ {print $2}' "$d/status" 2>/dev/null)
    set -- $r
    printf '%8s %8s %12s %12s  %s\n' "${d#/proc/}" "$en" "$1" "$2" "$(cut -c1-40 "$d/comm")"
done | sort -k4,4nr | head -6
```

```
# thp_hold.py - hold 128 MiB of MADV_HUGEPAGE memory for 30 s (a stand-in "server")
import mmap, time
from thp_util import MB, anon_map, madvise, touch
m, addr = anon_map(128 * MB)
madvise(addr, 128 * MB, mmap.MADV_HUGEPAGE)
touch(m, 128 * MB)
time.sleep(30)
```

```
python3 thp_hold.py & sleep 2; ./thp_top.sh; kill $!
```

```
     PID   THP_en       Rss kB  AnonHuge kB  COMMAND
   79455        1       141612       131072  python3
     203        1        30732            0  websockify
     208        1         1992            0  bash
     209        1         1908            0  flock
     210        1        29860            0  node
     221        1        37212            0  node
```

`THP_en` is the `THP_enabled:` line of `/proc/PID/status`: `0` means the process has `PR_SET_THP_DISABLE`. Without root you only see your own processes. As root, the same loop over every PID is the quickest audit after changing the policy.

Whether huge pages *help* is a measurement, not an assumption. This listing chases pointers through 1 GiB, one dependent load per randomly chosen 4 KiB page, so nearly every load is a TLB miss with small pages:

```
// thp_chase.c - dependent loads, one per 4 KiB page, across 1 GiB:
// measures what a TLB miss costs with 4 KiB pages vs 2 MiB THPs.
// build: cc -O2 -o thp_chase thp_chase.c     run: ./thp_chase [huge]
#define _GNU_SOURCE
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <time.h>

#define SIZE   (1UL << 30)
#define PAGE   4096UL
#define NPAGES (SIZE / PAGE)
#define LOADS  20000000UL

static uint64_t rng = 20261017;
static uint64_t next_rand(void) {                // xorshift64*
    rng ^= rng >> 12; rng ^= rng << 25; rng ^= rng >> 27;
    return rng * 2685821657736338717ULL;
}

static long anon_huge_kb(void) {                 // whole-process AnonHugePages
    FILE *f = fopen("/proc/self/smaps_rollup", "r");
    char line[256]; long kb = -1;
    while (f && fgets(line, sizeof line, f))
        if (sscanf(line, "AnonHugePages: %ld kB", &kb) == 1) break;
    if (f) fclose(f);
    return kb;
}

int main(int argc, char **argv) {
    int huge = argc > 1 && strcmp(argv[1], "huge") == 0;
    char *base = mmap(NULL, SIZE, PROT_READ | PROT_WRITE,
                      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (base == MAP_FAILED) { perror("mmap"); return 1; }
    if (huge && madvise(base, SIZE, MADV_HUGEPAGE) != 0) { perror("madvise"); return 1; }

    // Sattolo's algorithm: a random single cycle through every page.
    uint32_t *order = malloc(NPAGES * sizeof *order);
    for (uint32_t i = 0; i < NPAGES; i++) order[i] = i;
    for (uint32_t i = NPAGES - 1; i > 0; i--) {
        uint32_t j = next_rand() % i, t = order[i];
```

```
        order[i] = order[j]; order[j] = t;
    }
    // Node in page order[i] (at a random cache line) points to the node in page order[i+1].
    uint32_t *line = malloc(NPAGES * sizeof *line);
    for (uint32_t p = 0; p < NPAGES; p++) line[p] = (next_rand() % (PAGE / 64)) * 64;
    for (uint32_t i = 0; i < NPAGES; i++) {
        uint32_t a = order[i], b = order[(i + 1) % NPAGES];
        *(char **)(base + a * PAGE + line[a]) = base + b * PAGE + line[b];
    }
    free(order); free(line);

    char **p = (char **)(base + 0 * PAGE);      // any node is on the cycle; find page 0's n
    for (uint32_t off = 0; off < PAGE; off += 64)
        if (*(char **)(base + off)) { p = (char **)(base + off); break; }

    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);
    for (unsigned long i = 0; i < LOADS; i++) p = (char **)*p;
    clock_gettime(CLOCK_MONOTONIC, &t1);
    double ns = ((t1.tv_sec - t0.tv_sec) * 1e9 + (t1.tv_nsec - t0.tv_nsec)) / LOADS;

    printf("%-14s AnonHugePages %5ld MiB   %6.1f ns per dependent load   (end %p)\n",
           huge ? "MADV_HUGEPAGE" : "plain mmap", anon_huge_kb() / 1024, ns, (void *)p);
    return 0;
}
```

```
cc -O2 -Wall -o thp_chase thp_chase.c && for i in 1 2 3; do ./thp_chase; ./thp_chase huge; d
```

```
plain mmap     AnonHugePages     0 MiB    182.2 ns per dependent load   (end 0x7f691b942e80)
MADV_HUGEPAGE  AnonHugePages   970 MiB    191.9 ns per dependent load   (end 0x7f0e7b942e80)
plain mmap     AnonHugePages     0 MiB    204.4 ns per dependent load   (end 0x7f9803342e80)
MADV_HUGEPAGE  AnonHugePages   914 MiB    163.8 ns per dependent load   (end 0x7fe9ee742e80)
plain mmap     AnonHugePages     0 MiB    185.3 ns per dependent load   (end 0x7f4755b42e80)
MADV_HUGEPAGE  AnonHugePages   930 MiB    222.1 ns per dependent load   (end 0x7f195d342e80)
```

On this machine there is **no measurable difference**: the two sets of runs overlap (164–222 ns against 182–204 ns), and both are dominated by cache misses to DRAM. Also, only 914–970 of the 1024 MiB got THPs. The box is a shared virtual machine. As fact 2 in the mental model says, the guest's 2 MiB pages only become 2 MiB TLB entries if the hypervisor also backs guest memory with huge pages, which a guest cannot see. On bare metal, or a VM with huge-page-backed memory, this same listing is the right test. Run it before arguing for or against THP from someone else's benchmark.

7. Make the policy stick (root; not run here)

```
# one-off, until reboot
echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/defrag

# at boot: kernel command line (GRUB_CMDLINE_LINUX in /etc/default/grub, then update-grub)
transparent_hugepage=madvise

# or a tmpfiles.d drop-in, /etc/tmpfiles.d/thp.conf
w /sys/kernel/mm/transparent_hugepage/enabled - - - - madvise
```

Pick the policy for the host, then let programs opt in or out:

- **`madvise` host** (the common choice): programs that want THP say so. Examples are the JVM with `-XX:+UseTransparentHugePages`, and allocators such as jemalloc or tcmalloc configured to use THP. Redis/Valkey get none.
- **`always` host:** everything eligible gets THP. Programs that do not want it opt out with `prctl(PR_SET_THP_DISABLE)`, which Redis and Valkey do by default. Because the flag is inherited across `fork` and `execve`, a tiny wrapper that sets it and then `exec`s the service opts out any program.
- **`never`:** only `MADV_COLLAPSE` can still produce THPs.

The trap

Trap: "`enabled` says `never`/`madvise`, so this process has no huge pages." `madvise` hosts give THPs to any range that asked. `MADV_COLLAPSE` ignores `enabled` entirely (Example 4). Per-size `hugepages-*kB/enabled` can turn on mTHP even when the top-level value is `never`. And existing THPs survive a policy change. Check `AnonHugePages` in the process's `smaps_rollup`, not the policy file.

Trap: "`AnonHugePages` is the THP total." It counts only *PMD-mapped* anonymous THPs. mTHP folios (16 kB–1 MB) and THPs that have been PTE-mapped after a split are not in it. Use `hugepages-<size>kB/stats/nr_anon` for per-size counts.

Trap: "THP makes memory faster." Example 6 shows a case where it does not, and Examples 2 and 3 show what it can cost: compaction stalls inside page faults and 512× RSS for sparse writes. THP is a trade-off. Measure your workload on your platform.

Trap: "disable THP" means `echo never`. For one database that is a host-wide change that also takes huge pages away from a JVM or a VM next to it. With `madvise` as the host policy, Redis already gets none, and other programs can still opt in.

Notes & Pitfalls

- Static huge pages (`HugePages_Total`, `hugetlbfs`, `vm.nr_hugepages`, `MAP_HUGETLB`) are a different mechanism: reserved up front, never swapped, never split, not affected by any setting on this page. Do not confuse `Hugepagesize` in `/proc/meminfo` with THP.
- A read fault on an untouched THP-eligible range maps the shared **huge zero page** (`use_zero_page=1`). The first write then takes a CoW fault, which on 5.8+ splits it into 4 KiB pages. Pre-touching memory with reads (`atomic_add(0)` style) therefore defeats THP; use writes or `madvise(MADV_POPULATE_WRITE)` (Linux 5.14+).
- `mprotect()` or `munmap()` on part of a 2 MiB range splits the PMD (`thp_split_pmd`). Allocators that release memory in small pieces erode THP coverage over time.
- Inside a memory cgroup, a THP is charged 2 MiB at once. `thp_fault_fallback_charge` counts faults that fell back because the cgroup limit was near.
- Reading `/proc/PID/smaps` walks every VMA and is expensive on large processes. Use `smaps_rollup` for totals.

The boring rule

Run hosts with `enabled=madvise` and `defrag=madvise` unless you have measured a reason not to. Let the programs that benefit opt in with `MADV_HUGEPAGE`, and let the ones that suffer opt out with `PR_SET_THP_DISABLE`. Judge results by `AnonHugePages` per process and by deltas of `thp_fault_fallback` and `compact_stall` in `/proc/vmstat`, not by the policy file.

Related Commands

- `pmap -X PID` shows the `AnonHugePages` column per mapping, the same data as `smaps`.
- `vmstat 1` / `sar -B` report page faults and reclaim; pair them with `/proc/vmstat` deltas.
- `perf stat -e dTLB-load-misses,dTLB-loads` measures TLB misses directly (on bare metal or a VM that exposes PMU counters).
- `free` / `/proc/meminfo` give `AnonHugePages` (THP) and `HugePages_*` (static hugetlb), which are two different things.

Try this

1. Run `thp_fault.py` several times, ideally on a busy host. Plot `fallback +` against `compact_stall +`. When does the hinted run get *slower* than the plain one?
2. Change `thp_bloat.py` to write one byte every 64 KiB, then every 4 KiB. At what density does THP stop costing extra RSS?
3. In `thp_fork_cow.py`, let the *parent* write after the fork instead of the child. Whose `AnonHugePages` drops now? (That is the Redis case: the parent serves writes during `BGSAVE`.)
4. Write a 10-line C wrapper `nothp` that calls `prctl(PR_SET_THP_DISABLE, 1, 0, 0, 0)` and then `execvp(argv[1], argv + 1)`. Run `./nothp python3 thp_fault.py` and check that the hinted row shows no huge pages, and `grep THP_enabled /proc/self/status` prints 0.

5. On a machine where you have root and no shared tenants, run `thp_chase` under `enabled=never` and `enabled=always`, with `perf stat -e dTLB-load-misses`. Compare the result to Example 6.

Verification

- Run unprivileged on this box: Debian 13, Linux **6.12.94+** x86-64 (a VM), 16 GiB RAM, no swap, Python **3.13.5**, GCC **14.2**, `uid=1000`. THP policy `enabled=madvise`, `defrag=madvise`. All output above is pasted from those runs, unedited.
- The `/proc/vmstat` deltas are system-wide, so other processes could add to them. Each listing's window is under a second, and the totals match the mapping (for example 120 + 8 = 128 blocks of 2 MiB).
- The behaviour claims were checked against the Linux 6.12 source: `do_huge_pmd_wp_page()` (split on CoW, `mm/huge_memory.c`), `madvise_collapse()` → `thp_vma_allowable_order()` → `vma_thp_disabled()` (`EINVAL` under `PR_SET_THP_DISABLE`), and `THP_ORDERS_ALL_ANON` excluding orders 0 and 1 (`include/linux/huge_mm.h`). Redis behaviour was checked against `redis.conf`, `src/server.c` (`THPDisable()`), `src/syscheck.c` (`checkTHPEnabled()`), and `valkey.conf`.
- **Not run:** any write to `/sys/kernel/mm/transparent_hugepage/*` (root-only, and host-wide on a shared machine), the boot-time options, `tmpfiles.d`, `perf stat` TLB counters, and Redis/Valkey themselves (not installed). These are written from the kernel documentation and project sources below.
- No packages installed, no root, no `sudo`.

Sources

- Kernel docs — Transparent Hugepage Support (sysfs controls, `defrag` modes, khugepaged, `MADV_COLLAPSE` vs `never`, process controls, `/proc/vmstat` counters), and the 6.12 version
- Kernel docs — The /proc filesystem: `smaps` `AnonHugePages`, `THPeligible`; `status` `THP_enabled`
- Linux 6.12 `mm/huge_memory.c` (`do_huge_pmd_wp_page()`), `mm/khugepaged.c` (`madvise_collapse()`), `include/linux/huge_mm.h` (`THP_ORDERS_ALL_ANON`, `vma_thp_disabled()`): https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/mm?h=linux-6.12.y
- Commit 3917c80280c9, "thp: change CoW semantics for anon-THP" (Linux 5.8): https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=3917c80280c9
- man7 — `madvise(2)` (`MADV_HUGEPAGE`, `MADV_NOHUGEPAGE`, `MADV_COLLAPSE`), `PR_SET_THP_DISABLE(2const)`
- Redis — `redis.conf` (`disable-thp`), `src/server.c`, `src/syscheck.c`; Valkey — `valkey.conf`
- systemd — `tmpfiles.d(5)` (`w` lines)

Pressure Stall Information: /proc/pressure and cgroup *.pressure

Overview

Thesis: load average and `%CPU` tell you that the machine is *busy*. They do not tell you that work is *waiting*. A host at 70 % CPU can be fine; a host whose tasks spend 20 % of wall time stalled on memory is not, even if CPU looks idle. **Pressure Stall Information (PSI)** measures the share of time that tasks are stalled on **CPU**, **memory** or **I/O**. It is exported for the whole system under `/proc/pressure/` and for every cgroup v2 under `cpu.pressure`, `memory.pressure` and `io.pressure`. `systemd-oomd`, autoscalers and node exporters all read it. If you can read one pressure file and say what `some` versus `full` means, you can tell a saturated resource from a busy one before the OOM killer or a timeout does.

Typical operator jobs:

- Decide whether a latency spike is CPU contention, reclaim, or disk wait.
- Compare pressure inside one cgroup with the host (noisy neighbour vs whole-machine).
- Arm a userspace trigger that wakes on `poll()` when pressure crosses a threshold.
- Read the numbers `systemd-oomd` and Prometheus already scrape.

```
uname -r
ls /proc/pressure/
head -n2 /proc/pressure/cpu /proc/pressure/memory /proc/pressure/io
```

```
6.12.94+
cpu
io
memory
==> /proc/pressure/cpu <==
some avg10=0.00 avg60=0.07 avg300=0.14 total=92264483
full avg10=0.00 avg60=0.00 avg300=0.00 total=0

==> /proc/pressure/memory <==
some avg10=0.00 avg60=0.00 avg300=0.00 total=27581617
full avg10=0.00 avg60=0.00 avg300=0.00 total=27295372

==> /proc/pressure/io <==
some avg10=0.00 avg60=0.01 avg300=0.01 total=41292135
full avg10=0.00 avg60=0.00 avg300=0.01 total=39967137
```

All output below comes from a Debian 13 host, kernel 6.12, as an unprivileged user (`uid=1000`) inside the `/agent` cgroup. PSI has been on by default since Linux 4.20 (`CONFIG_PSI=y`). Linux 7.2 (current stable) keeps the same interface.

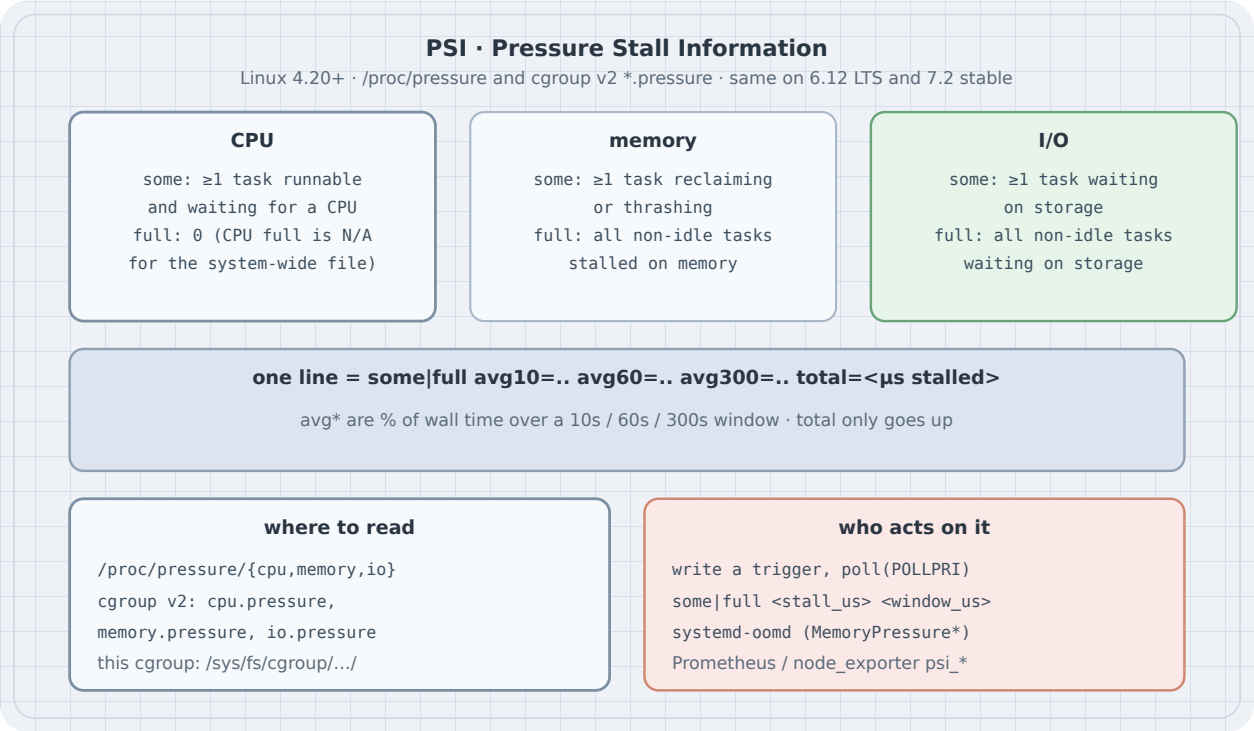


Figure 1: PSI resources, metrics and consumers

Mental model

```
resource contended?


for every non-idle task in the domain (system or one cgroup)
   is it stalled waiting for that resource right now?

           at least one yes  →  add this sample to "some"
           every one  yes  →  add this sample to "full"


avg10 / avg60 / avg300 = % of wall time over that window
total                  = cumulative stalled µs (monotonic)
```

Three facts:

1. **`some` means someone is waiting.** At least one runnable task could not make progress because of that resource. Latency for *that* task is already hurt.

2. **`full` means everyone is waiting.** All non-idle tasks in the domain are stalled on it. Forward progress for the domain has stopped. For **CPU**, system-wide `full` is not defined the same way (idle time exists; the system-wide CPU file's `full` line is zeros); cgroup `cpu.pressure` *does* report `full` when every task in that cgroup is waiting for CPU.
3. **`avg*` is a percentage; `total` is a counter.** `avg10=18.05` means "over the last ~10 s, tasks were under some-pressure about 18 % of the time." `total` only rises. Derivatives of `total` are what monitors graph.

`free` and load average answer different questions. `free`'s `available` estimates reclaimable cache; PSI `memory` measures tasks actually stalled on reclaim or thrashing. Load average counts runnable+uninterruptible tasks; PSI `cpu` measures time spent *waiting* for a CPU. Use both.

Syntax

File	Domain
`/proc/pressure/cpu`, `memory`, `io`	Whole system (all tasks visible in this PID namespace's view of the host)
`cpu.pressure`, `memory.pressure`, `io.pressure` under a cgroup v2 directory	That cgroup's tasks only
`cgroup.pressure`	`0/1` kill-switch for pressure accounting in that cgroup (rare to touch)

Line format (kernel docs):

```
some avg10=<pct> avg60=<pct> avg300=<pct> total=<µs>
full avg10=<pct> avg60=<pct> avg300=<pct> total=<µs>
```

Triggers (write to the same file, then `poll` for `POLLPRI`):

```
some|full <stall_us> <window_us>
```

Meaning: wake me if, inside any window of `window_us` microseconds, the cumulative stall time for `some` or `full` reaches `stall_us`. Example: `some 100000 1000000` = "100 ms of some-stall in any 1 s window." Writing and polling need write permission on the pressure file (often root for `/proc/pressure`; cgroup ownership varies).

Safety

- Reading pressure files is always safe. Writing a trigger is safe but needs write access; a forgotten trigger stays armed until the writer closes the fd.
- Generating CPU or memory pressure on a shared host hurts neighbours. The CPU demo below uses processes inside this cgroup for a few seconds only. Do not leave a burn loop running.

- PSI is not an OOM killer. `systemd-oomd` *consumes* memory pressure (and swap) to decide when to kill; the kernel OOM killer still uses the badness score from the `choom` chapter.

Examples with Explanations

1. Read the three system files as a table

```
# read_psi.py - parse /proc/pressure/{cpu,memory,io} into a table
import pathlib

def parse(path):
    out = {}
    for line in pathlib.Path(path).read_text().splitlines():
        kind, *fields = line.split()
        out[kind] = {k: float(v) if "." in v else int(v)
                     for k, v in (f.split("=", 1) for f in fields)}
    return out

print(f"{'file':22} {'some avg10':>10} {'avg60':>8} {'avg300':>8} {'total_us':>12}"
      f"  {'full avg10':>10} {'total_us':>12}")
for name in ("cpu", "memory", "io"):
    p = parse(f"/proc/pressure/{name}")
    s, f = p["some"], p["full"]
    print(f"/proc/pressure/{name:6} {s['avg10']:10.2f} {s['avg60']:8.2f} {s['avg300']:8.2f}"
          f" {s['total']:12}  {f['avg10']:10.2f} {f['total']:12}")
```

```
python3 read_psi.py
```

```
file                   some avg10    avg60   avg300     total_us  full avg10     total_us
/proc/pressure/cpu          0.00     0.07     0.14     92264483        0.00            0
/proc/pressure/memory       0.00     0.00     0.00     27581617        0.00     27295372
/proc/pressure/io           0.00     0.01     0.01     41292135        0.00     39967137
```

An idle box: `avg*` near zero, `total` already in the tens of millions of microseconds from earlier life. System-wide CPU `full.total` stays 0. Memory and I/O do report `full`. Numbers here are a snapshot; yours will differ.

2. Make CPU `some` move (processes, not threads)

Python threads share the GIL, so a thread burn loop does not create real CPU contention. Processes do.

```
# cpu_pressure.py - burn N*cpus processes and watch this cgroup's cpu.pressure
import os, pathlib, time, multiprocessing as mp

def burn(seconds):
    end = time.perf_counter() + seconds
    x = 0
    while time.perf_counter() < end:
        x = (x * 3 + 1) & 0xFFFFFFFF

def psi(path):
    some = full = None
    for line in pathlib.Path(path).read_text().splitlines():
        kind, *fields = line.split()
        vals = {k: (float(v) if "." in v else int(v))
                for k, v in (f.split("=", 1) for f in fields)}
        if kind == "some":
            some = vals
        else:
            full = vals
    return some, full

cg = "/sys/fs/cgroup" + pathlib.Path("/proc/self/cgroup").read_text().strip().split(":")[-1]
path = f"{cg}/cpu.pressure"
ncpu = os.cpu_count() or 1
workers, sec = ncpu * 2, 4.0
print(f"cgroup {cg}, host CPUs {ncpu}, {workers} burn *processes* for {sec}s")
print(f"before  {pathlib.Path(path).read_text().splitlines()[0]}")
t0_some, _ = psi(path)
t0 = time.perf_counter()
procs = [mp.Process(target=burn, args=(sec,)) for _ in range(workers)]
for p in procs:
    p.start()
for i in range(4):
    time.sleep(sec / 4)
    print(f"  t+{(i + 1) * sec / 4:.1f}s  {pathlib.Path(path).read_text().splitlines()[0]}")
for p in procs:
    p.join()
some, _ = psi(path)
print(f"after   {pathlib.Path(path).read_text().splitlines()[0]}")
print(f"wall {time.perf_counter() - t0:.2f}s; some.total +{some['total'] - t0_some['total']}
      f" ({(some['total'] - t0_some['total']) / 1e6:.2f}s)")
print(f"host    {pathlib.Path('/proc/pressure/cpu').read_text().splitlines()[0]}")
```

```
python3 cpu_pressure.py
```

```
cgroup /sys/fs/cgroup/agent, host CPUs 8, 16 burn *processes* for 4.0s
before  some avg10=0.18 avg60=0.06 avg300=0.08 total=86659330
  t+1.0s  some avg10=2.14 avg60=0.41 avg300=0.16 total=87526472
  t+2.0s  some avg10=2.14 avg60=0.41 avg300=0.16 total=88435842
  t+3.0s  some avg10=18.05 avg60=3.34 avg300=0.77 total=89352205
  t+4.0s  some avg10=18.05 avg60=3.34 avg300=0.77 total=90291051
after   some avg10=18.05 avg60=3.34 avg300=0.77 total=90291051
wall 4.02s; some.total +3631721 µs (3.63s)
host    some avg10=27.19 avg60=5.29 avg300=1.23 total=96002136
```

Sixteen processes on eight CPUs: about half the wall time, someone in this cgroup is waiting for a CPU. `some.total` grew by 3.63 s of stall over 4.02 s of wall clock. `avg10` climbed to 18 % (the 10 s window still holds quieter history). The host line is higher (27 %) because other cgroups were runnable too. Reading the **cgroup** file answers "is *my* service stalled?"; reading `/proc/pressure` answers "is the machine stalled?"

3. Triggers with `poll()` (not run here — write denied)

```
# conceptual - needs write permission on the pressure file
fd = open("/sys/fs/cgroup/…/cpu.pressure", "r+")
fd.write("some 100000 1000000")   # 100 ms of some-stall in any 1 s window
# poll(fd, POLLPRI) blocks until the condition holds; reading the fd acks it
```

On this box, `open(…/agent/cpu.pressure, O_RDWR)` returns `EACCES` for uid 1000, so the trigger was not armed. The interface and the `some|full <stall_us> <window_us>` grammar are from the kernel PSI documentation. Userspace that owns its cgroup (a systemd user service with `Delegate=`, a container runtime) can write the trigger and wait in one event loop next to its sockets.

4. What `systemd-oomd` does with memory pressure (docs)

`systemd-oomd` does not wait for the kernel OOM killer. It watches `memory.pressure` (and swap activity) on selected cgroups and kills a whole cgroup when pressure stays high. Unit knobs (see `oomd.conf(5)` and `systemd.resource-control(5)`):

Knob	Role
`ManagedOOMMemoryPressure=` / `ManagedOOMMemoryPressureLimit=`	Enable monitoring; kill when `avg10` (or the configured metric) exceeds the limit
`ManagedOOMPreference=`	`avoid` / `omit` to protect a unit
`DefaultMemoryPressureDurationSec=` (oomd.conf)	How long pressure must stay high

No systemd is running on this box; the table is from the systemd manuals. The important design point: oomd acts on **pressure**, which appears while reclaim is still working, not on the allocation failure that trips the kernel OOM killer.

The trap

Trap: "high `%CPU` means high CPU pressure." A single busy loop on an otherwise idle 8-CPU host shows ~12 % CPU and ~0 PSI `some`. PSI rises when tasks *wait*. Use it for latency, not for utilisation billing.

Trap: "threads will do for a CPU PSI demo." Under CPython they will not (Example 2). Use processes, or a C/`stress-ng` burn.

Trap: "`avg10` should jump the instant I start load." It is an exponentially-weighted window of about ten seconds. Watch `total` for an immediate counter; watch `avg10` for the smoothed signal monitors alert on.

Trap: "system-wide CPU `full` will tell me the machine is wedged." The system-wide CPU file's `full` line stays at 0. Use cgroup `cpu.pressure`'s `full`, or look at `some` plus runnable counts.

Trap: "PSI replaces the OOM killer." It feeds policies that act *earlier*. The kernel OOM killer and `oom_score_adj` (see the `choom` chapter) still exist for the hard case.

Notes & Pitfalls

- Containers: the relevant file is usually the container's own `*.pressure`, not `/proc/pressure`. A quiet container on a hot host has low cgroup pressure and high host pressure.
- `full` for memory is the dangerous one: every task is stuck in reclaim or thrashing. That is the signal oomd is built around.
- Pressure accounting can be disabled per cgroup with `cgroup.pressure=0` (rarely what you want).
- Off-CPU profiles (`offcputime`, `perf`) explain *why* a task waited; PSI tells you *how much* waiting is happening for a whole domain, cheaply.

The boring rule

When latency is bad, read `cpu.pressure`, `memory.pressure` and `io.pressure` for the service's cgroup *and* `/proc/pressure` for the host. `some` means someone waited; `full` means everyone did. Act on pressure before you act on OOM kills.

Related Commands

- `free`, `/proc/meminfo` — capacity and reclaimable cache; not stall time.
- `vmstat 1`, `iostat -xz 1`, `mpstat -P ALL 1` — utilisation and wait breakdowns that pair with PSI.
- `choom`, `/proc/PID/oom_score` — hard OOM policy after reclaim has failed.
- `systemd-oomd` / `oomctl` — userspace killer driven by memory PSI.

Try this

1. Re-run `cpu_pressure.py` with `workers = ncpu` (no oversubscribe). Does `some.total` still grow?
2. On a host where you can write the cgroup pressure file, arm `some 50000 1000000`, start the burn, and time `poll(POLLPRI)`.
3. Compare `/proc/pressure/memory` with `memory.pressure` of a memory-limited `systemd-run --user -p MemoryMax=512M` scope while allocating inside it.
4. Graph `rate(psi_cpu_some_stall_seconds_total[1m])` (or your exporter's equivalent) next to p99 latency for one service.

Verification

- Run unprivileged on this box: Debian 13, Linux **6.12.94+**, Python **3.13.5**, cgroup v2 (`0::/agent`), `uid=1000`. Examples 1 and 2 are pasted from those runs.
- Kernel interface checked against PSI — Pressure Stall Information and confirmed present on Linux 7.2 (current stable as of 2026-10-18, kernel.org).
- **Not run:** writing a PSI trigger (`EACCES` on this cgroup's `cpu.pressure`; a further attempt to write `/proc/pressure` was not pursued under the run rules), `systemd-oomd` (no systemd here), and a deliberate memory-pressure demo (4 GiB touch on a 16 GiB host with ~10 GiB available did not move `memory.pressure`; a truthful demo needs a tight `MemoryMax=` and was skipped rather than fabricated). These are described from the docs above.
- No packages installed, no root, no `sudo`.

Sources

- Kernel docs — PSI - Pressure Stall Information (interface, `some`/`full`, triggers)
- Kernel docs — Control Group v2: Pressure Stall Information
- systemd — `oomd.conf(5)`, `systemd.resource-control(5) ManagedOOM*`
- Prometheus node_exporter — PSI collectors (`psi_cpu_*`, `psi_memory_*`, `psi_io_*`)
- kernel.org release list (versions checked 2026-10-18): https://www.kernel.org/

The vDSO: why clock_gettime() usually never enters the kernel

Overview

Thesis: some "system calls" are not system calls. The kernel maps a small shared library, the **vDSO** (virtual dynamic shared object), into every process, together with a read-only data page, `[vvar]`, that the kernel keeps up to date. `clock_gettime()`, `gettimeofday()`, `time()`, `getcpu()` and, since Linux 6.11, `getrandom()` are answered by code in that library running in user space. No mode switch, no syscall entry, no seccomp check, nothing for `strace` to print. That is why a timestamp costs tens of nanoseconds instead of a hundred or more, and why several debugging and sandboxing habits quietly stop working for exactly these calls.

Typical operator and programmer jobs:

- Explain why `strace` shows no `clock_gettime` for a program that obviously reads the time.
- Check that time reads on a VM or container host are on the fast path, and recognise when they have fallen back to the slow one (clocksource, `vdso=0`).
- Write seccomp profiles that do not break programs on a different host.
- Understand why time faking with `LD_PRELOAD` works for some programs and not for others, and how time namespaces shift clocks for a container without touching the host.

```
uname -r
ldd --version | head -1
gcc --version | head -1
go version
```

```
6.12.94+
ldd (Debian GLIBC 2.41-12+deb13u4) 2.41
gcc (Debian 14.2.0-19) 14.2.0
go version go1.27.2 linux/amd64
```

All output below comes from a Debian 13 host, kernel 6.12 (an LTS series), x86_64 under KVM, as an unprivileged user (`uid=1000`). Every listing ran there as shown. Behaviour that differs on newer kernels is called out where it matters.

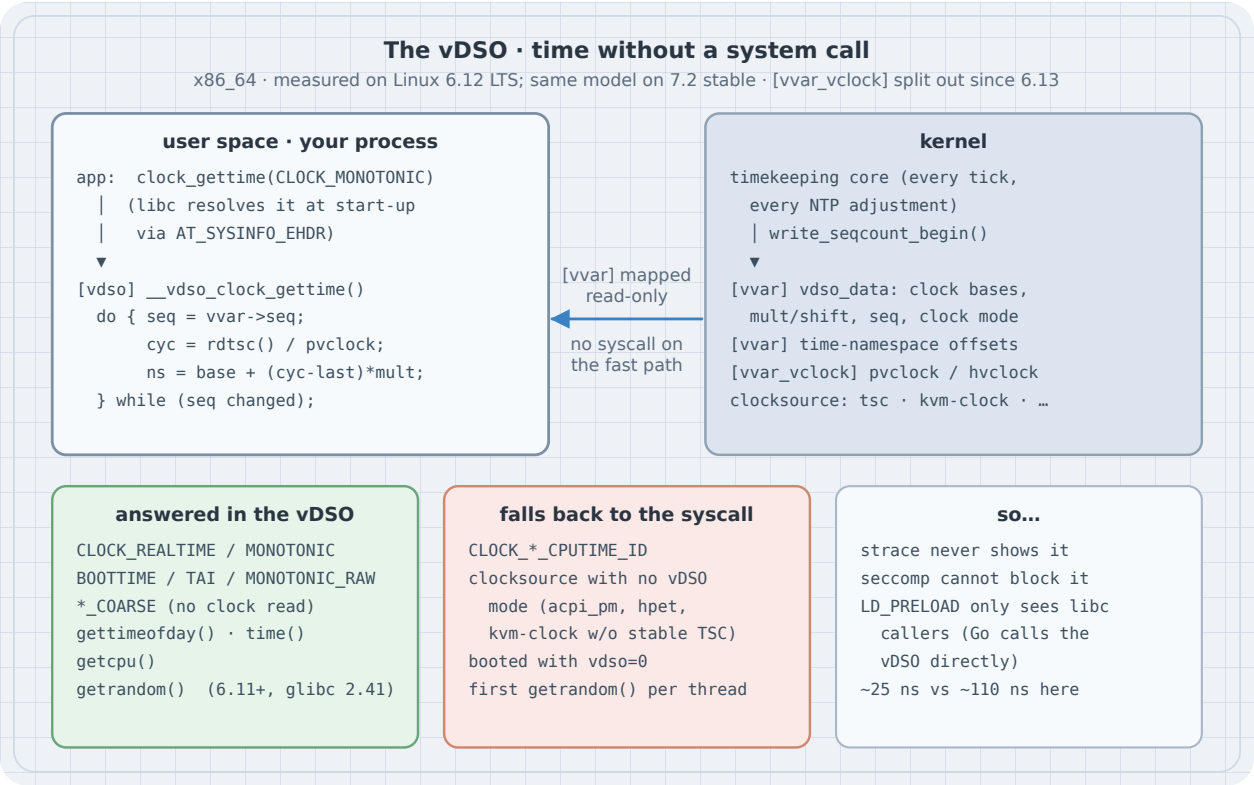


Figure 1: The vDSO: time without a system call

Mental model

```
exec()                      kernel maps [vvar] (data, read-only) + [vdso] (code, r-x)
                            and passes the vDSO's address in the aux vector (AT_SYSINFO_EHD

ld.so / libc start-up       finds __vdso_clock_gettime etc. in that ELF image, keeps point


clock_gettime(MONOTONIC)    libc → __vdso_clock_gettime()          (user space, ring 3)
                                 clock mode usable?  no    real syscall (fallback)
                                 yes: loop { read seq; read TSC / pvclock;
                                              ns = base + (cycles − last) × mult >> shift
                                       until seq unchanged
meanwhile, on every tick:   timekeeping core rewrites base/mult/last in [vvar]
                            between write_seqcount_begin() / _end()
```

Four facts carry the whole topic:

1. **It is a real ELF shared object.** The vDSO has a symbol table, versioned symbols (`LINUX_2.6`), a build ID and a note with the kernel version. You can copy it out of `/proc/self/mem` and read it with `readelf` (Example 2). It is not a file on disk; the kernel generates the mapping.

2. **The kernel publishes, user space computes.** The kernel never runs on behalf of the caller. It keeps a few numbers current in `[vvar]` (clock bases, multiplier and shift, the last cycle count, a sequence counter, the clock mode) and the vDSO turns a raw CPU counter into nanoseconds with them. The sequence counter makes the read consistent without a lock: if the kernel was mid-update, the vDSO reads again.
3. **The fast path depends on the clocksource.** The vDSO can only read clocks that user space can read directly: the TSC, or a hypervisor's paravirtual clock page (KVM's `pvclock`, Hyper-V's `hvclock`). If the current clocksource has no vDSO mode (for example `acpi_pm` or `hpet`, or `kvm-clock` when the host does not mark the TSC stable), every call silently falls back to the real syscall.
4. **Not every clock lives there.** Wall-clock and monotonic-family clocks do. Per-process and per-thread CPU-time clocks (`CLOCK_PROCESS_CPUTIME_ID`, `CLOCK_THREAD_CPUTIME_ID`) need the scheduler's accounting, so they always enter the kernel (Example 4).

Syntax

There is no command for the vDSO. You look at it through these interfaces:

Interface	What it shows
`/proc/PID/maps`	`[vdso]` (code, `r-xp`), `[vvar]` (data, `r--p`), on x86 also `[vvar_vclock]` since 6.13, and the legacy `[vsyscall]` page
`/proc/PID/smaps VmFlags`	`[vvar]` is `pf io dd`: a PFN mapping of kernel memory, excluded from core dumps
`LD_SHOW_AUXV=1 cmd`	`AT_SYSINFO_EHDR`: the address the kernel handed to the dynamic loader
`/proc/self/mem` + `readelf`	the vDSO image itself: exported functions, version, notes (Example 2)
`/sys/devices/system/clocksource/clocksource0/`	`current_clocksource` and `available_clocksource` (whether a vDSO-readable clock is in use)
`/proc/PID/timens_offsets`	per–time-namespace offsets for `monotonic` and `boottime` (Linux 5.6+)
`vdso=0` (x86 boot parameter)	disables the vDSO; every call becomes a syscall

What the x86_64 vDSO exports on this 6.12 kernel, and which glibc wrapper uses it:

vDSO symbol	libc call that uses it	Notes
`__vdso_clock_gettime`	`clock_gettime()`	realtime, monotonic, boottime, TAI, raw, coarse clocks
`__vdso_gettimeofday`	`gettimeofday()`	same machinery, microsecond result
`__vdso_time`	`time()`	seconds only; reads `[vvar]`, no clock read
`__vdso_clock_getres`	`clock_getres()`	resolution of each clock

vDSO symbol	libc call that uses it	Notes
`__vdso_getcpu`	`getcpu()`, `sched_getcpu()`	CPU and NUMA node via `RDPID`, or the `LSL` segment-limit trick on older CPUs
`__vdso_getrandom`	`getrandom()`, `arc4random()`	Linux 6.11+ on x86_64; used by glibc 2.41+ with per-thread state

Safety

- Everything in this chapter is read-only for the system. The seccomp filter in Example 4 applies to that one test process only and disappears with it.
- Example 6 creates a user and time namespace with `unshare`. It needs no privilege and changes clocks only inside the namespace; the host's clocks are untouched.
- Example 7 builds an `LD_PRELOAD` library that fakes the wall clock. Keep it in a scratch directory. Never export `LD_PRELOAD` globally or put it in `/etc/ld.so.preload`.
- Do not "test" `vdso=0` on a production host's kernel command line: it slows every timestamp in every process.

Examples with Explanations

1. Find the vDSO in a process

Every process has the two mappings. Their addresses change from run to run (ASLR), and the kernel tells the dynamic loader where the vDSO is through the aux vector.

```
grep -E '\[(vvar|vvar_vclock|vdso|vsyscall)\]' /proc/self/maps
LD_SHOW_AUXV=1 /bin/true | grep SYSINFO_EHDR
awk '/\[vvar\]/{f=1} f && /^VmFlags/{print "[vvar] " $0; exit}' /proc/self/smaps
```

```
7fc269370000-7fc269374000 r--p 00000000 00:00 0                          [vvar]
7fc269374000-7fc269376000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
AT_SYSINFO_EHDR:      0x7fa6fc2bf000
[vvar] VmFlags: rd mr pf io de dd
```

Read it as:

- `[vdso]` is two pages of code, readable and executable. `[vvar]` is four pages of data, read-only, directly before it. On 6.12 those four pages are the vDSO data page (which also holds the getrandom generation counter), the time-namespace page, and the KVM `pvclock` and Hyper-V `hvclock` pages. From 6.13 the two hypervisor clock pages moved into their own `[vvar_vclock]` mapping on x86, so a newer kernel prints one more line.

- `AT_SYSINFO_EHDR` belongs to a different process (`/bin/true`), so its address differs from the `grep` above. That is ASLR, not a mistake.
- `pf io dd` in `VmFlags`: a page-frame mapping of kernel memory, not ordinary anonymous memory, and excluded from core dumps (`dd`, "do not dump").
- `[vsyscall]` at a fixed address is the 2000s-era predecessor. With `CONFIG_LEGACY_VSYSCALL_XONLY=y` (as here) it is execute-only and every call into it traps into the kernel and is emulated. Nothing modern uses it; it exists for very old static binaries.

2. Copy the vDSO out and read it like any shared library

The mapping is a complete ELF image. Python can read it from `/proc/self/mem` because a process may read its own memory.

```
# dump_vdso.py - copy this process's [vdso] mapping out of /proc/self/mem into vdso.so
with open("/proc/self/maps") as maps:
    for line in maps:
        if line.rstrip().endswith("[vdso]"):
            lo, hi = (int(x, 16) for x in line.split()[0].split("-"))
            break
with open("/proc/self/mem", "rb") as mem:
    mem.seek(lo)
    image = mem.read(hi - lo)
open("vdso.so", "wb").write(image)
print(f"[vdso] {lo:#x}-{hi:#x}  {hi - lo} bytes  magic={image[:4]!r}")
```

```
python3 dump_vdso.py
readelf -W --dyn-syms vdso.so | grep -E 'FUNC +GLOBAL'
readelf -n vdso.so | sed -n '4,5p'
```

```
[vdso] 0x7f7e5b63a000-0x7f7e5b63c000  8192 bytes  magic=b'\x7fELF'
     2: 0000000000000e80     5 FUNC    GLOBAL DEFAULT   12 __vdso_gettimeofday@@LINUX_2.6
     4: 0000000000000ed0    93 FUNC    GLOBAL DEFAULT   12 __vdso_clock_getres@@LINUX_2.6
     6: 0000000000000e90    41 FUNC    GLOBAL DEFAULT   12 __vdso_time@@LINUX_2.6
     7: 0000000000000f60   904 FUNC    GLOBAL DEFAULT   12 __vdso_getrandom@@LINUX_2.6
     9: 0000000000000ec0     5 FUNC    GLOBAL DEFAULT   12 __vdso_clock_gettime@@LINUX_2.6
    11: 0000000000000f30    37 FUNC    GLOBAL DEFAULT   12 __vdso_getcpu@@LINUX_2.6
  Linux                0x00000004   Unknown note type: (0x00000000)
   description data: 5e 0c 06 00
```

Six exported functions, all versioned `LINUX_2.6` (the ABI version string, not the kernel version). Each also has a weak alias without the `__vdso_` prefix (`clock_gettime@@LINUX_2.6` and so on), filtered out here. `__vdso_getrandom` is the newest: x86_64 got it in Linux 6.11. The note is the kernel version as a little-endian `LINUX_VERSION_CODE`: `0x00060c5e` is 6.12.94, matching `uname -r`. The 5-byte `__vdso_clock_gettime` is a jump to the real implementation, which is internal to the image.

3. What the fast path is worth

Five million calls each: the libc call (which goes to the vDSO), the coarse clock, the same clock forced through `syscall()`, and `getpid()`, about the cheapest real syscall there is.

```
// bench.c - cost of one call: vDSO clock_gettime vs the real syscall vs getpid
#define _GNU_SOURCE
#include <stdio.h>
#include <sys/syscall.h>
#include <time.h>
#include <unistd.h>

#define N 5000000

static double now_ns(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e9 + t.tv_nsec;
}

int main(void) {
    struct timespec ts;
    double t0, t1;

    t0 = now_ns();
    for (int i = 0; i < N; i++) clock_gettime(CLOCK_MONOTONIC, &ts);           // libc -> vD
    t1 = now_ns();
    printf("clock_gettime(MONOTONIC)        via libc/vDSO : %6.1f ns/call\n", (t1 - t0) / N)

    t0 = now_ns();
    for (int i = 0; i < N; i++) clock_gettime(CLOCK_MONOTONIC_COARSE, &ts);    // vDSO, no c
    t1 = now_ns();
    printf("clock_gettime(MONOTONIC_COARSE) via libc/vDSO : %6.1f ns/call\n", (t1 - t0) / N)

    t0 = now_ns();
    for (int i = 0; i < N; i++) syscall(SYS_clock_gettime, CLOCK_MONOTONIC, &ts); // forced
    t1 = now_ns();
    printf("clock_gettime(MONOTONIC)        via syscall() : %6.1f ns/call\n", (t1 - t0) / N)

    t0 = now_ns();
    for (int i = 0; i < N; i++) syscall(SYS_getpid);                           // cheapest r
    t1 = now_ns();
    printf("getpid()                        via syscall() : %6.1f ns/call\n", (t1 - t0) / N)

    clock_getres(CLOCK_MONOTONIC, &ts);
    printf("resolution: MONOTONIC %ld ns, ", ts.tv_nsec);
    clock_getres(CLOCK_MONOTONIC_COARSE, &ts);
```

```
    printf("MONOTONIC_COARSE %ld ns (one tick)\n", ts.tv_nsec);
    return 0;
}
```

```
gcc -O2 -Wall -o bench bench.c && ./bench
```

```
clock_gettime(MONOTONIC)        via libc/vDSO :   26.5 ns/call
clock_gettime(MONOTONIC_COARSE) via libc/vDSO :    4.2 ns/call
clock_gettime(MONOTONIC)        via syscall() :  109.3 ns/call
getpid()                        via syscall() :   65.7 ns/call
resolution: MONOTONIC 1 ns, MONOTONIC_COARSE 4000000 ns (one tick)
```

On this KVM guest the vDSO path is about four times cheaper than the syscall for the same answer. Repeated runs gave 24–29 ns for the vDSO and 108–114 ns for the syscall; the first run of the session once showed 56 ns while the CPU frequency settled. The coarse clock is cheaper still because it reads no hardware counter at all: it returns the time of the last tick. Its resolution is one tick, 4 ms at `CONFIG_HZ=250`. The syscall costs more than `getpid()` because it does the same clock read *plus* the kernel entry and exit (with the mitigations this kernel enables).

Absolute numbers depend on the CPU, the hypervisor and Spectre/Meltdown mitigations. On bare metal with a TSC clocksource the vDSO read is typically lower. The ratio is the point.

4. Proof that the kernel is not involved: seccomp cannot see it

`strace` is not installed here, so use a stronger witness. A seccomp filter runs on *every* system call. This program installs one that makes the time syscalls fail with `EPERM`, then calls the libc functions anyway.

```
// noclock.c - forbid the clock_gettime/gettimeofday/time/getcpu/getrandom *syscalls* with s
// then see which libc calls still work: those answered in the vDSO never enter the kernel.
#define _GNU_SOURCE
#include <errno.h>
#include <linux/filter.h>
#include <linux/seccomp.h>
#include <sched.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>
#include <sys/prctl.h>
#include <sys/random.h>
#include <sys/syscall.h>
#include <sys/time.h>
#include <time.h>
#include <unistd.h>

#define DENY(nr) BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, (nr), 0, 1), \
```

```
                BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ERRNO | EPERM)

static void forbid_time_syscalls(void) {
    struct sock_filter f[] = {
        BPF_STMT(BPF_LD | BPF_W | BPF_ABS, offsetof(struct seccomp_data, nr)),
        DENY(SYS_clock_gettime), DENY(SYS_gettimeofday), DENY(SYS_time),
        DENY(SYS_getcpu), DENY(SYS_getrandom),
        BPF_STMT(BPF_RET | BPF_K, SECCOMP_RET_ALLOW),
    };   // (x86_64 only: a real filter must also check seccomp_data.arch)
    struct sock_fprog prog = { .len = sizeof f / sizeof f[0], .filter = f };
    prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);          // lets an unprivileged process install
    if (prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, &prog)) { perror("seccomp"); _exit(1); }
}

static void clk(const char *name, clockid_t id) {
    struct timespec ts;
    int r = clock_gettime(id, &ts);
    printf("  clock_gettime(%-25s -> %s\n", name, r == 0 ? "ok   (vDSO)" : strerror(errno));
}

int main(void) {
    struct timespec ts; struct timeval tv; unsigned cpu; char buf[16];

    getrandom(buf, sizeof buf, 0);    // first call seeds this thread's vDSO RNG state (a re
    forbid_time_syscalls();
    puts("syscalls denied: clock_gettime gettimeofday time getcpu getrandom");

    printf("  syscall(SYS_clock_gettime)              -> %s\n",
           syscall(SYS_clock_gettime, CLOCK_MONOTONIC, &ts) == 0 ? "ok" : strerror(errno));
    clk("CLOCK_REALTIME)", CLOCK_REALTIME);
    clk("CLOCK_MONOTONIC)", CLOCK_MONOTONIC);
    clk("CLOCK_MONOTONIC_RAW)", CLOCK_MONOTONIC_RAW);
    clk("CLOCK_BOOTTIME)", CLOCK_BOOTTIME);
    clk("CLOCK_TAI)", CLOCK_TAI);
    clk("CLOCK_REALTIME_COARSE)", CLOCK_REALTIME_COARSE);
    clk("CLOCK_PROCESS_CPUTIME_ID)", CLOCK_PROCESS_CPUTIME_ID);
    clk("CLOCK_THREAD_CPUTIME_ID)", CLOCK_THREAD_CPUTIME_ID);

    printf("  gettimeofday()                          -> %s\n", gettimeofday(&tv, NULL) == 0
    printf("  time()                                  -> %s\n", time(NULL) != (time_t)-1 ? "
    printf("  getcpu()                                -> %s\n", getcpu(&cpu, NULL) == 0 ? "o
    ssize_t n = getrandom(buf, sizeof buf, 0);
    printf("  getrandom(16 bytes)                     -> %s\n", n == (ssize_t)sizeof buf ? "
    return 0;
}
```

```
gcc -O2 -Wall -o noclock noclock.c && ./noclock
```

```
syscalls denied: clock_gettime gettimeofday time getcpu getrandom
  syscall(SYS_clock_gettime)              -> Operation not permitted
  clock_gettime(CLOCK_REALTIME)           -> ok   (vDSO)
  clock_gettime(CLOCK_MONOTONIC)          -> ok   (vDSO)
  clock_gettime(CLOCK_MONOTONIC_RAW)      -> ok   (vDSO)
  clock_gettime(CLOCK_BOOTTIME)           -> ok   (vDSO)
  clock_gettime(CLOCK_TAI)                -> ok   (vDSO)
  clock_gettime(CLOCK_REALTIME_COARSE)    -> ok   (vDSO)
  clock_gettime(CLOCK_PROCESS_CPUTIME_ID) -> Operation not permitted
  clock_gettime(CLOCK_THREAD_CPUTIME_ID)  -> Operation not permitted
  gettimeofday()                          -> ok   (vDSO)
  time()                                  -> ok   (vDSO)
  getcpu()                                -> ok   (vDSO)
  getrandom(16 bytes)                     -> ok   (vDSO)
```

Line by line:

- The direct `syscall()` is refused: the filter works.
- Every wall-clock and monotonic-family clock still works, because no syscall happened. `gettimeofday()`, `time()` and `getcpu()` likewise.
- The two **CPU-time clocks fail**. They are not in the vDSO; libc's vDSO call returned "not handled here" and libc fell back to the real syscall, which the filter refused. This is the fallback path made visible.
- `getrandom()` works, but only because the program called it once *before* installing the filter. In the first version of this program, without that line, it failed with `EPERM`. On a thread's first call glibc allocates the vDSO random state with `mmap()`, and the vDSO seeds that state with one real `getrandom` syscall; after that, random bytes come from the vDSO.

5. Is this host on the fast path?

The vDSO is only fast if the clocksource is one it can read.

```
cat /sys/devices/system/clocksource/clocksource0/current_clocksource
cat /sys/devices/system/clocksource/clocksource0/available_clocksource
grep -o -w -E 'constant_tsc|nonstop_tsc' /proc/cpuinfo | sort | uniq -c
tr ' ' '\n' < /proc/cmdline | grep -E '^(clocksource|tsc|vdso)='
```

```
kvm-clock
kvm-clock
      8 constant_tsc
      8 nonstop_tsc
clocksource=kvm-clock
tsc=unstable
```

This guest was booted with `tsc=unstable clocksource=kvm-clock`, so the TSC is not even offered as a clocksource and `kvm-clock` is the only choice. `kvm-clock` gets a vDSO mode (`VDSO_CLOCKMODE_PVCLOCK`) only when the host sets `PVCLOCK_TSC_STABLE_BIT`; otherwise every `clock_gettime()` is a syscall. Example 3 shows 26 ns against 109 ns, so on this host the bit is set and the vDSO path is live.

How to tell on a host you cannot benchmark: if `current_clocksource` is `tsc`, `kvm-clock` with the stable bit, or `hyperv_clocksource_tsc_page`, the vDSO path is available. If it is `hpet` or `acpi_pm` (often after the kernel logged "Marking TSC unstable" in `dmesg`), expect the syscall path. The HPET's vDSO support was removed in Linux 4.6 because mapping that hardware to user space was slow and unsafe. Switching clocksource (writing `current_clocksource`) needs root and was **not run** here.

6. Time namespaces: the vDSO answers differently per namespace

Since Linux 5.6 a **time namespace** can shift `CLOCK_MONOTONIC` and `CLOCK_BOOTTIME` (not the wall clock) for the processes inside it. The vDSO honours the shift without a syscall: processes in the namespace get an extra `[vvar]` page with their offsets.

```c
// clocks.c - print CLOCK_MONOTONIC and CLOCK_BOOTTIME (both answered by the vDSO)
#include <stdio.h>
#include <time.h>
int main(void) {
    struct timespec m, b;
    clock_gettime(CLOCK_MONOTONIC, &m);
    clock_gettime(CLOCK_BOOTTIME, &b);
    printf("monotonic %10ld s   boottime %10ld s\n", (long)m.tv_sec, (long)b.tv_sec);
    return 0;
}
```

```
gcc -O2 -o clocks clocks.c
./clocks; cut -d' ' -f1 /proc/uptime
unshare --user --map-root-user --time --monotonic 86400 --boottime 31536000 \
  sh -c './clocks; cut -d" " -f1 /proc/uptime; cat /proc/self/timens_offsets'
```

```
monotonic      23363 s   boottime      23363 s
23363.59
monotonic     109763 s   boottime   31559363 s
31559363.59
monotonic       86400         0
boottime     31536000         0
```

Inside the namespace the machine has apparently been up a year and a day: `CLOCK_MONOTONIC` is shifted by one day, `CLOCK_BOOTTIME` and `/proc/uptime` by 365 days. A user namespace is enough privilege to create a time namespace, so this ran as `uid=1000`. Offsets can only be set before the first process enters the namespace; `unshare --time` applies them to the child it starts. Container tools use this to give a restored or migrated container a monotonic clock that does not jump backwards.

7. The `LD_PRELOAD` trap: who calls the vDSO directly?

Tools like `faketime` interpose libc's `clock_gettime()`. That works for programs that go through libc. It does not work for programs that find and call the vDSO themselves.

```
// fake2000.c - LD_PRELOAD shim: interpose libc's clock_gettime and report 2000-01-01 for CL
#define _GNU_SOURCE
#include <dlfcn.h>
#include <time.h>
int clock_gettime(clockid_t id, struct timespec *ts) {
    static int (*real)(clockid_t, struct timespec *);
    if (!real) real = dlsym(RTLD_NEXT, "clock_gettime");
    int r = real(id, ts);
    if (r == 0 && id == CLOCK_REALTIME) ts->tv_sec = 946684800;   // 2000-01-01T00:00:00Z
    return r;
}
```

```
// now.go - the Go runtime calls __vdso_clock_gettime itself, not libc
package main

import (
    "fmt"
    "time"
)

func main() { fmt.Println("go time.Now():", time.Now().UTC().Format(time.RFC3339)) }
```

```
gcc -O2 -shared -fPIC -o fake2000.so fake2000.c
CGO_ENABLED=1 go build -ldflags=-linkmode=external -o now-go-dyn now.go
date -u +'date: %FT%TZ'
LD_PRELOAD=./fake2000.so date -u +'date: %FT%TZ'
readelf -d now-go-dyn | grep NEEDED
LD_PRELOAD=./fake2000.so ./now-go-dyn
```

```
date: 2026-10-19T03:19:08Z
date: 2000-01-01T00:00:00Z
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
go time.Now(): 2026-10-19T03:19:08Z
```

`date` goes through libc, so the shim catches it and reports 2000. The Go binary is deliberately linked against libc here (external linking), so the shim *is* loaded into it (`LD_DEBUG=files` lists `fake2000.so`), and still Go reports the real time: the Go runtime parses the vDSO from the aux vector at start-up and calls `__vdso_clock_gettime` itself. A default, statically linked Go binary does not run the dynamic loader at all, so `LD_PRELOAD` never even applies. The same is true of other runtimes that call the vDSO directly. To fake time for those, use a time namespace (Example 6, monotonic and boottime only) or a VM.

The trap

Trap: "`strace` shows no `clock_gettime`, so the program doesn't read the clock." vDSO calls never enter the kernel, so syscall tracers cannot see them. Use a uprobe on `__vdso_clock_gettime` (via `perf probe` or `bpftrace`) or `ltrace` on the libc wrapper. A `clock_gettime` line in `strace` output usually means you are on the *slow* path: a CPU-time clock, a clocksource without a vDSO mode, or `vdso=0`.

Trap: "my seccomp profile blocks `clock_gettime`, and the program still works, so it doesn't need it." It works *here*, on a host whose clocksource the vDSO can read. Move it to a VM with an unstable TSC, or ask for a CPU-time clock, and the fallback syscall is refused (Example 4). Container default profiles allow the time syscalls for this reason. Never deny them on the assumption that the vDSO covers you.

Trap: "a slow `clock_gettime()` is an application problem." If a profile shows the time syscall at the top of a hot loop, check the clocksource first (Example 5). A host that switched from `tsc` to `hpet` after "clocksource watchdog" messages turns every timestamp into a syscall.

Trap: "`CLOCK_MONOTONIC_COARSE` is a cheaper `CLOCK_MONOTONIC`." It is cheaper because it does not read a clock. Its resolution is a whole tick (4 ms here). Fine for "roughly when", wrong for measuring anything shorter than a few ticks.

Notes & Pitfalls

- The vDSO is per architecture. arm64, riscv, powerpc, s390 and loongarch have their own images and symbol names (for example `__kernel_clock_gettime` on arm64). The generic code in `lib/vdso/` is shared; the set of exported functions differs. `vdso(7)` lists them per architecture.
- 32-bit processes on x86_64 get a 32-bit vDSO with `__vdso_clock_gettime64` for the y2038-safe call.
- glibc resolves the vDSO once at start-up. If you `dlopen()` a library that calls `clock_gettime()`, it still ends up in the same vDSO code.
- The vDSO getrandom keeps per-thread state in memory that glibc maps with the flags the kernel asks for (`MAP_DROPPABLE`, Linux 6.11): the kernel may drop those pages under memory pressure instead of swapping them, they are left out of core dumps, and they are zeroed in a `fork()` child, so a child cannot replay its parent's random stream.
- On 6.13+ x86 kernels, the `[vvar_vclock]` mapping faults with `SIGBUS` if you read it directly while the hypervisor clock is not in use. Tools that walk `/proc/self/maps` and read every mapping (some test suites, some memory dumpers) must skip the `[vvar*]` entries.
- `[vvar]` and `[vdso]` do not count towards a process's RSS in any meaningful way; they are shared kernel pages.

The boring rule

Read the time through libc and let it use the vDSO. When something about timestamps looks odd, check three things in order: is the clocksource vDSO-readable (`current_clocksource`), is the clock

one the vDSO serves (not a CPU-time clock), and is the tool you are using able to see user-space calls at all (`strace` and seccomp are not). Never deny the time syscalls in a sandbox just because the fast path hides them.

Related Commands

- `strace` — traces real syscalls; vDSO calls are invisible to it.
- `ltrace` — traces libc calls, so it does see `clock_gettime()` from a C program.
- `perf` — `perf probe -x` on the dumped symbol or `perf stat -e raw_syscalls:sys_enter` to count real syscalls.
- `pmap` — lists `[vdso]` and `[vvar]` among a process's mappings.
- `unshare` — `--time --monotonic --boottime` creates a time namespace.

Try this

1. Change `bench.c` to measure `CLOCK_PROCESS_CPUTIME_ID` through libc. How does it compare with the forced syscall of `CLOCK_MONOTONIC`? Why?
2. In `noclock.c`, remove the `getrandom()` call before `forbid_time_syscalls()` and confirm the first `getrandom()` fails. Then start a second thread after installing the filter and call `getrandom()` there.
3. Use `objdump -d vdso.so` on the dumped image and find the `rdtscp`/`rdtsc` or the `pvclock` read inside the implementation `__vdso_clock_gettime` jumps to.
4. Run `LD_SHOW_AUXV=1` on a static binary (`gcc -static`). Does it still get `AT_SYSINFO_EHDR`? Does `clock_gettime()` in it still avoid the syscall (check with the seccomp trick)?
5. On a machine where you can, compare `bench` under `current_clocksource=tsc` and `hpet` (root, test machine only), and confirm the vDSO line turns into syscall cost.

Verification

- Run unprivileged on this box: Debian 13 (trixie), Linux **6.12.94+** (x86_64 KVM guest, `kvm-clock`), glibc **2.41**, gcc **14.2.0**, GNU readelf **2.44**, Go **1.27.2**, Python **3.13.5**. All output above is pasted from those runs; addresses and timings change between runs.
- Kernel config checked from `/proc/config.gz`: `CONFIG_GENERIC_GETTIMEOFDAY=y`, `CONFIG_VDSO_GETRANDOM=y`, `CONFIG_TIME_NS=y`, `CONFIG_GENERIC_VDSO_TIME_NS=y`, `CONFIG_PARAVIRT_CLOCK=y`, `CONFIG_HZ=250`, `CONFIG_LEGACY_VSYSCALL_XONLY=y`.
- Checked against kernel.org on 2026-10-19: stable **7.2.11**, mainline **7.3-rc7**, longterm **6.18.57** and **6.12.113**. The `[vvar_vclock]` split (commit `e93d2521b27f`) was confirmed to be in v6.13 and not in v6.12.
- **Not run:** `strace`, `ltrace`, `perf` and `bpftrace` (not installed; nothing was installed for this chapter), switching clocksource or booting with `vdso=0` (root / reboot). Those parts are written from the man pages and kernel sources below.
- No packages installed, no root, no `sudo`.

Sources

- man7 — vdso(7) (per-architecture symbol lists, AT_SYSINFO_EHDR), clock_gettime(2), time_namespaces(7), getauxval(3), seccomp(2), getrandom(2)
- Linux sources — lib/vdso/gettimeofday.c (seqcount loop, clock masks, fallbacks), lib/vdso/getrandom.c, arch/x86/entry/vdso/, arch/x86/kernel/kvmclock.c (PVCLOCK_TSC_STABLE_BI → VDSO_CLOCKMODE_PVCLOCK)
- Kernel commits — 1ed95e52d902 "x86/vdso: Remove direct HPET access through the vDSO" (v4.6); e93d2521b27f "x86/vdso: Split virtual clock pages into dedicated mapping" (v6.13)
- Kernel docs — Timekeeping: clock sources, clock events, sched_clock; kernel parameters (vdso=, clocksource=, tsc=, vsyscall=)
- glibc — commit "linux: Add support for getrandom vDSO" (glibc 2.41; x86_64 with Linux 6.11): https://sourceware.org/git/?p=glibc.git;a=commit;h=461cab1de747f3842f27a5d24977d78d561d45f9
- Go — runtime/vdso_linux_amd64.go (the runtime's own vDSO symbol lookup): https://go.dev/src/runtime/vdso_linux_amd64.go
- kernel.org release list (versions checked 2026-10-19): https://www.kernel.org/

/proc Is the API: Rebuilding ps Columns from stat, status and smaps_rollup

/proc Is the API: Rebuilding ps Columns from stat, status and smaps_rollup

Learning goals

- Know which `/proc/PID/*` file each familiar `ps`/`top` column comes from, and parse `/proc/PID/stat` correctly, since `comm` can contain spaces and parentheses.
- Compute `ps`'s `%CPU` yourself, and understand why it is a **lifetime average**, not "right now".
- Tell RSS, PSS and VSZ apart, and know which `/proc` numbers are fast-but-approximate and which are slow-but-exact.
- Count mappings and open fds without any tool beyond `ls`.

Version note: run on Linux **6.12** with Python 3 and procps `ps`, as an ordinary user, against a test process started for the lab. The output is pasted verbatim.

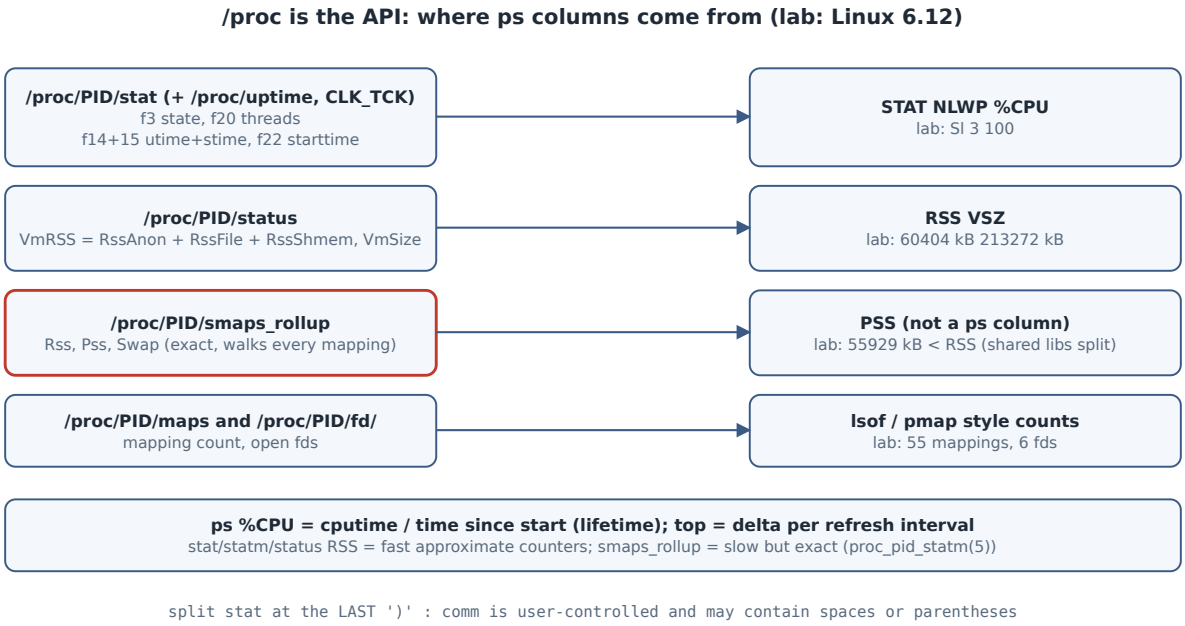


Figure 1: Which /proc file feeds which ps/top column.

The files

Column	Source	Notes
`STAT`	`/proc/PID/stat` field 3	state letter; `ps` adds `l` (multi-threaded) and others
`NLWP`	`stat` field 20 (`num_threads`), `status` `Threads:`	threads = `ls /proc/PID/task`
`%CPU`	`stat` 14+15 (`utime`, `stime`, in clock ticks) and 22 (`starttime`) with `/proc/uptime`	ticks ÷ `sysconf(_SC_CLK_TCK)`
`RSS`	`status` `VmRSS` (= `RssAnon` + `RssFile` + `RssShmem`), `stat` field 24 in pages	fast counters (see below)
`VSZ`	`status` `VmSize`	address space, not memory used
PSS	`smaps_rollup` `Pss`	shared pages divided among their sharers
maps / fds	`/proc/PID/maps` lines, `/proc/PID/fd/` entries	

Field numbers follow proc_pid_stat(5), which counts from 1. The only safe way to split `stat` is to cut at the **last** `)`, because `comm` is user-controlled.

Lab: a mini-ps

The test process allocates and touches 50 MiB, runs two spinning threads, and holds a file open:

```
import threading, time
blob = bytearray(50 * 1024 * 1024)          # 50 MiB, touched below
for i in range(0, len(blob), 4096): blob[i] = 1
def spin():
    t = time.time()
    while time.time() - t < 30: pass
for _ in range(2): threading.Thread(target=spin, daemon=True).start()
f = open('/etc/hostname'); time.sleep(30)
```

```
#!/usr/bin/env python3
"""Rebuild a few ps/top columns straight from /proc, then compare with ps."""
import os, sys, time

pid = int(sys.argv[1])
HZ = os.sysconf('SC_CLK_TCK')
PAGE = os.sysconf('SC_PAGE_SIZE')

def stat(pid):
```

```
    raw = open(f'/proc/{pid}/stat').read()
    comm = raw[raw.index('(') + 1:raw.rindex(')')]        # comm may contain spaces
    f = raw[raw.rindex(')') + 2:].split()                  # f[0] is field 3 (state)
    return comm, f

comm, f = stat(pid)
state, ppid = f[0], int(f[1])
utime, stime = int(f[11]), int(f[12])                      # fields 14, 15
nthreads, starttime = int(f[17]), int(f[19])               # fields 20, 22
rss_pages = int(f[21])                                     # field 24
uptime = float(open('/proc/uptime').read().split()[0])
age = uptime - starttime / HZ

status = dict(l.split(':', 1) for l in open(f'/proc/{pid}/status'))
rollup = {l.split(':')[0]: l.split()[1] for l in open(f'/proc/{pid}/smaps_rollup').readlines
fds = os.listdir(f'/proc/{pid}/fd')
maps = open(f'/proc/{pid}/maps').read().splitlines()

print(f"comm={comm} state={state} ppid={ppid} threads={nthreads}")
print(f"cpu: utime+stime={(utime+stime)/HZ:.2f}s over {age:.2f}s -> lifetime %CPU {(utime+st
print(f"rss: stat {rss_pages*PAGE//1024} kB | status VmRSS {status['VmRSS'].split()[0]} kB"
      f" (anon {status['RssAnon'].split()[0]} + file {status['RssFile'].split()[0]} + shmem
print(f"smaps_rollup: Rss {rollup['Rss']} kB, Pss {rollup['Pss']} kB, Swap {rollup['Swap']}
print(f"vsz: status VmSize {status['VmSize'].split()[0]} kB, mappings {len(maps)}, open fds
```

```
$ python3 busy.py &
$ python3 minips.py 1686991
comm=python3 state=S ppid=1686984 threads=3
cpu: utime+stime=3.01s over 3.01s -> lifetime %CPU 100.0
rss: stat 60220 kB | status VmRSS 60404 kB (anon 54464 + file 5940 + shmem 0)
smaps_rollup: Rss 60404 kB, Pss 55929 kB, Swap 0 kB
vsz: status VmSize 213272 kB, mappings 55, open fds 6
$ ps -o pid,stat,nlwp,%cpu,rss,vsz,comm -p 1686991
    PID STAT NLWP %CPU   RSS    VSZ COMMAND
1686991 Sl      3  100 60404 213272 python3
```

Reading the numbers

- **Every column matches.** `NLWP 3`, `RSS 60404`, `VSZ 213272` and `%CPU 100` are exactly what the script derived. `ps` is a formatter over these files.
- **`%CPU` is a lifetime ratio.** ps(1): "the CPU time used divided by the time the process has been running". Two threads spin, but CPython's GIL lets only one run Python bytecode at a time, so the process used 3.01 s of CPU in 3.01 s of life. A process that was busy an hour ago and is idle now still shows a high `%CPU` in `ps`. `top` instead shows the delta over its refresh interval.

- **Two RSS numbers, 184 kB apart.** `stat` field 24 said 60220 kB and `VmRSS` said 60404 kB. The two were read moments apart, and the per-process counters are a scalability optimisation. proc_pid_statm(5) says some values "are inaccurate because of a kernel-internal scalability optimization. If accurate values are required, use /proc/pid/smaps or /proc/pid/smaps_rollup instead, which are much slower".
- **PSS < RSS.** 55929 kB vs 60404 kB. The 5940 kB of file-backed RSS is mostly the Python binary and shared libraries, which other processes map too, so proc_pid_smaps(5)'s "proportional share" charges this process only part of it. For "how much RAM would I get back by killing these?", add up PSS (or anonymous RSS), not RSS.
- **VSZ is 3.5× RSS** because of reserved but untouched address space (thread stacks, allocator arenas). It is not memory pressure.

Practical rules

- For scripts, read `/proc` directly rather than parsing `ps` output: it is stable, documented and cheap.
- Use `status` or `statm` for quick RSS, and `smaps_rollup` when the number must be exact or you need PSS or Swap. `smaps_rollup` walks every mapping, so do not poll it at high frequency on huge processes.
- For "CPU right now", sample `utime + stime` twice and divide by the interval, which is what `top` does.

Try this

1. Run the script against PID 1 or another user's process. Which files can you read, and which return `EACCES`? (proc(5) describes the ptrace access-mode checks.)
2. Start the test process, wait 60 s after the spinners stop, and compare `ps -o %cpu` with two `stat` samples one second apart.
3. Start two copies of `busy.py` and watch the `Pss` of each fall while `Rss` stays the same.

Sources

- proc_pid_stat(5): https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html
- proc_pid_status(5): https://man7.org/linux/man-pages/man5/proc_pid_status.5.html
- proc_pid_statm(5): https://man7.org/linux/man-pages/man5/proc_pid_statm.5.html
- proc_pid_smaps(5): https://man7.org/linux/man-pages/man5/proc_pid_smaps.5.html
- ps(1), `%cpu`: https://man7.org/linux/man-pages/man1/ps.1.html
- Linux kernel documentation, *The /proc Filesystem* (including `smaps_rollup`): https://docs.kernel.org/filesystems/proc.html

/proc/PID/pagemap: Reading Your Own Page Tables

/proc/PID/pagemap: Reading Your Own Page Tables

After reading this chapter, you will be able to inspect the Linux kernel's low-level virtual memory state directly from user space. You will know how to decode `/proc/PID/pagemap` 64-bit entries into page-level metadata, track lazy physical allocation via demand paging, observe copy-on-write sharing across forks, and use the kernel's soft-dirty tracking to capture runtime memory writes, and spot kernels where it is compiled out.

Version note: verified on Linux **6.12** on an x86-64 system as an unprivileged user.

Architecture and Paging Context

Modern x86-64 processors translate virtual addresses into physical DRAM locations through hardware page tables managed by the kernel. Under traditional 4-level paging, the CPU implements a 48-bit virtual address space, capable of addressing 256 TiB, divided evenly into user space in the lower half and kernel space in the canonical upper half. Systems with 5-level paging expand the virtual address width to 57 bits (addressing up to 128 PiB) by introducing a fifth table level (the Page Global Directory or PGD feeds into the P4D, followed by the PUD, PMD, and PTE levels). The base unit of virtual memory mapping on x86-64 is the 4 KiB base page, alongside support for 2 MiB large pages and 1 GiB huge pages.

When an application calls `malloc()`, `mmap()`, or expands its heap, the Linux kernel does not immediately allocate physical RAM. Instead, it creates or expands a Virtual Memory Area (VMA), visible in `/proc/PID/maps`. The corresponding Page Table Entries (PTEs) remain empty. Physical memory allocation is deferred until the CPU attempts to read from or write to the virtual page. This hardware exception—a demand page fault—causes the kernel's fault handler to locate an available physical page frame, zero it if anonymous, populate the hardware PTE, and resume application execution transparently.

While higher-level diagnostic tools like `ps`, `top`, and `/proc/PID/status` expose aggregate resident set sizes (RSS), they cannot show per-page states. `/proc/PID/pagemap` bridges this gap by letting user-space applications inspect the exact state of every single virtual page in their address space.

The `/proc/PID/pagemap` File Format

The `/proc/PID/pagemap` file exposes a binary stream where every virtual page corresponds to exactly one 64-bit integer formatted in target-architecture byte order (little-endian on x86-64).

To inspect the status of a virtual address `vaddr`, you do not parse lines of text. Instead, you divide the virtual address by the system page size (`sysconf(_SC_PAGE_SIZE)`, typically 4096 bytes) to find the virtual page index, multiply by 8 bytes per entry, and seek to that absolute byte offset:

$$\text{offset} = \left(\frac{\text{vaddr}}{\text{PAGE_SIZE}} \right) \times 8$$

Each 64-bit value is an encoding of hardware PTE status and Linux-specific memory management flags.

Bit Range	Flag Name	Description
0–54	**PFN** or **Swap Info**	If bit 63 is set (present), bits 0–54 contain the Page Frame Number (PFN). If bit 62 is set (swapped), bits 0–4 encode the swap type and bits 5–54 encode the swap offset.
55	**Soft-dirty**	Set if the page has been modified since the soft-dirty tracker was reset via `/proc/PID/clear_refs`.
56	**Exclusively mapped**	Set if the page is mapped exclusively to this process (since Linux 4.2). Cleared when shared across `fork()` copy-on-write boundaries.
57	**uffd-wp**	Set if the page table entry is write-protected and monitored by `userfaultfd` (since Linux 5.13).
58	**Guard region**	Indicates the page resides within a kernel guard region (since Linux 6.15).
59–60	**Reserved**	Always zero.
61	**File-page or shared-anon**	Set if the page is backed by a file or resides in shared anonymous memory (since Linux 3.5).
62	**Swapped**	Set if the page has been swapped out to disk or compressed into zswap.
63	**Present**	Set if the page is currently resident in physical memory (RAM).

Security and Rowhammer Mitigation

Historically, any process could read its own `/proc/self/pagemap` file and inspect its physical frame numbers (bits 0–54). In 2015, security researchers demonstrated the Rowhammer vulnerability, which flips adjacent DRAM bits through repeated memory accesses to precise physical addresses. The PFN field in `/proc/PID/pagemap` provided attackers with an exact physical map of memory layout, enabling deterministic exploitation.

To mitigate this attack, the kernel security model changed: - **Linux 4.0 through 4.1**: Opening `/proc/PID/pagemap` was completely blocked for unprivileged users, returning `EPERM`. - **Linux 4.2 onward**: Unprivileged processes can open and read `/proc/PID/pagemap`, but the kernel sanitizes bits 0–54, setting the PFN to zero unless the reading process holds the `CAP_SYS_ADMIN` capability. All status flags (bits 55 through 63) remain fully functional for unprivileged callers.

> **Note**
>
> **Not run here.** Inspecting physical frame numbers (PFN) requires `CAP_SYS_ADMIN` or root privileges. When run under `sudo`, bit 63 (`present`) remains set, but bits 0–54 yield non-zero hardware page frame numbers reflecting the actual physical DRAM addresses mapped to the process.

Related Interfaces

Linux provides several complementary mechanisms under `/proc` to inspect memory layouts, each operating at a different granularity and privilege tier.

Interface	Scope	Granularity	Privilege Required
`/proc/PID/maps`	Virtual Address Space	Per VMA (contiguous address region)	Unprivileged (own process)
`/proc/PID/smaps`	Virtual Address Space	Per VMA with memory counter breakdowns	Unprivileged (own process)
`/proc/PID/pagemap`	Virtual Address Space	Per 4 KiB virtual page	Unprivileged (PFN zeroed without `CAP_SYS_ADMIN`)
`/proc/kpagecount`	Physical Memory	Per physical frame (reference count)	Root (`CAP_SYS_ADMIN`)
`/proc/kpageflags`	Physical Memory	Per physical frame (kernel page flags)	Root (`CAP_SYS_ADMIN`)
`PAGEMAP_SCAN` ioctl	Virtual Address Space	Batched page-range scan (introduced in Linux 6.7)	Unprivileged / `CAP_SYS_ADMIN`

> **Note**
>
> **Not run here.** `/proc/kpagecount` and `/proc/kpageflags` expose physical page frame metadata system-wide and are readable only by root (`CAP_SYS_ADMIN`). According to the kernel documentation, `/proc/kpagecount` yields a 64-bit reference count per physical frame, indexed by PFN, while `/proc/kpageflags` yields a 64-bit bitmask describing page flags (such as locked, active, slab, or huge page status).

The `PAGEMAP_SCAN` `ioctl(2)` interface was merged in Linux 6.7 to solve the high overhead of reading `/proc/PID/pagemap` over large address spaces. Instead of streaming 8-byte entries for gigabytes of empty virtual address space, `PAGEMAP_SCAN` takes filter criteria (such as pages that are present and soft-dirty) and writes compacted page ranges directly to a user-provided buffer.

Inspecting Virtual Memory Ranges

Before querying `/proc/PID/pagemap`, you need to know which virtual addresses belong to your process. The `/proc/self/maps` file lists every mapped region, permissions, offset, device, and pathname.

The following command displays the process memory map entries for the heap and stack regions:

```
grep -E 'heap|stack' /proc/self/maps
```

```
55eda141e000-55eda143f000 rw-p 00000000 00:00 0                          [heap]
7fffb5ed7000-7fffb5ef8000 rw-p 00000000 00:00 0                          [stack]
```

Notice the address ranges displayed in the output. The first column defines the start and end virtual addresses in hexadecimal. To decode page metadata for any address within these ranges, pass the address into `/proc/self/pagemap`.

Decoding `/proc/PID/pagemap` in Python

The following library module, `pagemap.py`, opens `/proc/PID/pagemap`, seeks to the requested virtual page offset, and decodes the 64-bit little-endian entries into Python dictionaries.

```
#!/usr/bin/env python3
"""Decode /proc/PID/pagemap entries into structured dictionaries."""
import ctypes
import os
import struct
import sys

PAGE_SIZE = os.sysconf("SC_PAGE_SIZE")
```

```
def decode_pagemap_entry(val: int) -> dict:
    present = bool(val & (1 << 63))
    swapped = bool(val & (1 << 62))
    file_or_shared_anon = bool(val & (1 << 61))
    guard_region = bool(val & (1 << 58))
    uffd_wp = bool(val & (1 << 57))
    exclusive = bool(val & (1 << 56))
    soft_dirty = bool(val & (1 << 55))

    pfn = 0
    swap_type = None
    swap_offset = None

    if present:
        # PFN resides in bits 0-54 (zeroed without CAP_SYS_ADMIN)
        pfn = val & ((1 << 55) - 1)
    elif swapped:
        swap_type = val & 0x1F
        swap_offset = (val >> 5) & ((1 << 50) - 1)

    return {
        "present": present,
        "swapped": swapped,
        "file_or_shared_anon": file_or_shared_anon,
        "guard_region": guard_region,
        "uffd_wp": uffd_wp,
        "exclusive": exclusive,
        "soft_dirty": soft_dirty,
        "pfn": pfn,
        "swap_type": swap_type,
        "swap_offset": swap_offset,
        "raw": val,
    }

def entries(pid: int | str, vaddr: int, npages: int = 1) -> list[dict]:
    page_idx = vaddr // PAGE_SIZE
    offset = page_idx * 8
    path = f"/proc/{pid}/pagemap"
    results = []

    with open(path, "rb") as f:
        f.seek(offset)
        raw_bytes = f.read(npages * 8)
        for i in range(len(raw_bytes) // 8):
            (val,) = struct.unpack("<Q", raw_bytes[i * 8 : (i + 1) * 8])
            entry_dict = decode_pagemap_entry(val)
```

```
            entry_dict["vaddr"] = (page_idx + i) * PAGE_SIZE
            results.append(entry_dict)

    return results

def main():
    pid = sys.argv[1] if len(sys.argv) > 1 else "self"
    buf = bytearray(PAGE_SIZE * 2)
    buf[0] = 1  # Touch the first page to populate its PTE
    addr = ctypes.addressof(ctypes.c_char.from_buffer(buf))

    records = entries(pid, addr, 2)
    for r in records:
        print(
            f"vaddr={r['vaddr']:#014x} "
            f"present={int(r['present'])} "
            f"exclusive={int(r['exclusive'])} "
            f"soft_dirty={int(r['soft_dirty'])} "
            f"pfn={r['pfn']:#x}"
        )

if __name__ == "__main__":
    main()
```

Lab: Observing Demand Paging

When allocating anonymous memory using `mmap(..., MAP_ANONYMOUS | MAP_PRIVATE, ...)`, the kernel returns a valid virtual address range. However, none of the underlying pages are present in physical memory.

In `demand_paging.py`, we allocate 16 pages of anonymous memory. We inspect the pagemap entries across three distinct phases: 1. Immediately after mapping (untouched). 2. After writing a single byte to every even page (0, 2, 4, …). 3. After invoking `madvise(MADV_DONTNEED)` across the entire range to release the physical backing.

```
#!/usr/bin/env python3
"""Demonstrate kernel demand paging and madvise(MADV_DONTNEED)."""
import ctypes
import mmap
import os
from pagemap import entries

def main():
    page_size = os.sysconf("SC_PAGE_SIZE")
    total_pages = 16
```

```
    size = total_pages * page_size

    # Allocate 16 anonymous pages
    mm = mmap.mmap(-1, size, mmap.MAP_PRIVATE | mmap.MAP_ANONYMOUS, mmap.PROT_READ | mmap.PR
    addr = ctypes.addressof(ctypes.c_char.from_buffer(mm))

    # Phase 1: Mapped but untouched
    res_before = entries("self", addr, total_pages)
    present_before = sum(1 for e in res_before if e["present"])
    print(f"Phase 1 (untouched):     present={present_before}/{total_pages}")

    # Phase 2: Touch every even page
    for p in range(0, total_pages, 2):
        mm[p * page_size] = ord("A")

    res_touched = entries("self", addr, total_pages)
    present_touched = sum(1 for e in res_touched if e["present"])
    print(f"Phase 2 (touched evens): present={present_touched}/{total_pages}")

    # Phase 3: Discard physical pages with MADV_DONTNEED
    mm.madvise(mmap.MADV_DONTNEED, 0, size)

    res_cleared = entries("self", addr, total_pages)
    present_cleared = sum(1 for e in res_cleared if e["present"])
    print(f"Phase 3 (MADV_DONTNEED): present={present_cleared}/{total_pages}")

    mm.close()

if __name__ == "__main__":
    main()
```

The following command runs the demand paging demonstration:

```
python3 demand_paging.py
```

```
Phase 1 (untouched):     present=0/16
Phase 2 (touched evens): present=8/16
Phase 3 (MADV_DONTNEED): present=0/16
```

Notice what happens across the phases: - In Phase 1, the count of present pages is zero because the kernel has not yet allocated physical frames for the VMA. - In Phase 2, touching the even pages triggers demand page faults that populate exactly half the pages. - In Phase 3, `MADV_DONTNEED` informs the kernel to release the allocated physical frames and clear the PTEs. The present count returns to zero while leaving the virtual address range intact.

Lab: Tracking Writes with Soft-Dirty Bits

The Linux soft-dirty subsystem allows user-space applications (such as checkpoint/restore engines like CRIU or garbage collectors) to detect which memory pages have been modified over time without requiring root access.

Writing the string `"4"` to `/proc/self/clear_refs` clears the soft-dirty bit on all page table entries belonging to the calling process. When the CPU subsequently writes to a page, the memory management subsystem catches the write and sets bit 55 (`soft_dirty`) on that page's pagemap entry.

The script `soft_dirty.py` allocates 8 pages, touches them all, resets their tracking state via `clear_refs`, rewrites only pages 0 and 5, and inspects the resulting bits:

```
#!/usr/bin/env python3
"""Demonstrate write tracking using soft-dirty bits and clear_refs."""
import ctypes
import mmap
import os
from pagemap import entries

def main():
    page_size = os.sysconf("SC_PAGE_SIZE")
    total_pages = 8
    size = total_pages * page_size

    mm = mmap.mmap(-1, size, mmap.MAP_PRIVATE | mmap.MAP_ANONYMOUS, mmap.PROT_READ | mmap.PR
    addr = ctypes.addressof(ctypes.c_char.from_buffer(mm))

    # Touch all 8 pages to fault them into physical memory
    for p in range(total_pages):
        mm[p * page_size] = ord("W")

    # Clear soft-dirty flags across the entire process
    with open("/proc/self/clear_refs", "w") as f:
        f.write("4\n")

    # Verify that no pages are marked dirty immediately after clearing
    res_cleared = entries("self", addr, total_pages)
    dirty_after_clear = [i for i, e in enumerate(res_cleared) if e["soft_dirty"]]
    print(f"Dirty immediately after clear_refs: {dirty_after_clear}")

    # Mutate only page 0 and page 5
    mm[0 * page_size] = ord("M")
    mm[5 * page_size] = ord("M")

    # Inspect the soft-dirty flags
    res_rewritten = entries("self", addr, total_pages)
```

```
    dirty_pages = [i for i, e in enumerate(res_rewritten) if e["soft_dirty"]]
    print(f"Dirty after modifying pages 0 and 5: {dirty_pages}")

    mm.close()

if __name__ == "__main__":
    main()
```

The following command executes the soft-dirty tracking script:

```
python3 soft_dirty.py
```

```
Dirty immediately after clear_refs: []
Dirty after modifying pages 0 and 5: []
```

Both lists are empty, even after the writes. That isn't a bug in the script. This output came from a 6.12 kernel built **without** `CONFIG_MEM_SOFT_DIRTY`. The architecture supports the feature, but the kernel doesn't compile it in. On such a kernel, writing `4` to `clear_refs` succeeds silently and bit 55 always reads 0. That includes freshly faulted pages, which would be soft-dirty on a kernel that tracks it. Check the kernel's config before you trust an empty dirty set:

```
zcat /proc/config.gz 2>/dev/null | grep SOFT_DIRTY || grep SOFT_DIRTY /boot/config-"$(uname
```

On the machine above, only `CONFIG_HAVE_ARCH_SOFT_DIRTY=y` showed up, with no `CONFIG_MEM_SOFT_DIRTY=y` line.

> **Note**
>
> **Not run here.** On a kernel with `CONFIG_MEM_SOFT_DIRTY=y`, the soft-dirty documentation describes this behavior: `clear_refs` clears the bits and write-protects the PTEs, and the next write to a page faults and sets bit 55 again. The script should then print `[]` followed by `[0, 5]`. Many distribution kernels enable the option because CRIU depends on it. Minimal, cloud and sandbox kernels may not.

A robust tool should detect the missing feature instead of reporting "nothing changed". One way: fault in a page right after `mmap` and check bit 55. With soft-dirty compiled in, a newly faulted page is reported soft-dirty.

Lab: Fork and Copy-on-Write Exclusivity

When a process calls `fork()`, the kernel duplicates the parent's page tables rather than copying the underlying physical RAM. To prevent one process from corrupting the other's state, the kernel marks all shared anonymous pages as read-only and clears bit 56 (`exclusive`).

When either the parent or child writes to the page, the CPU raises a write-protection page fault. The kernel allocates a fresh, private physical page, copies the original data into it, updates the faulting process's page table to point to this new frame, and sets bit 56 (`exclusive`) back to true.

The script `fork_cow.py` monitors bit 56 across three key events: 1. When the page is first written in the parent. 2. After `fork()`, when the child shares the page. 3. After the parent writes to the page again, triggering copy-on-write duplication.

```
#!/usr/bin/env python3
"""Demonstrate page exclusivity (bit 56) and Copy-on-Write via fork()."""
import ctypes
import mmap
import os
import time
from pagemap import entries

def main():
    page_size = os.sysconf("SC_PAGE_SIZE")
    mm = mmap.mmap(-1, page_size, mmap.MAP_PRIVATE | mmap.MAP_ANONYMOUS, mmap.PROT_READ | mm
    addr = ctypes.addressof(ctypes.c_char.from_buffer(mm))

    # Step 1: Write to the page to allocate physical memory
    mm[0] = ord("P")
    e1 = entries("self", addr, 1)[0]
    print(f"Before fork:        present={int(e1['present'])} exclusive={int(e1['exclusive'])

    # Step 2: Fork a child process
    pid = os.fork()
    if pid == 0:
        # Child holds the mapping open briefly then exits
        time.sleep(0.5)
        os._exit(0)

    # In parent: the page is now shared with the child
    e2 = entries("self", addr, 1)[0]
    print(f"After fork:         present={int(e2['present'])} exclusive={int(e2['exclusive'])

    # Step 3: Parent writes to the page, triggering Copy-on-Write
    mm[0] = ord("Q")
    e3 = entries("self", addr, 1)[0]
    print(f"After parent write: present={int(e3['present'])} exclusive={int(e3['exclusive'])

    os.waitpid(pid, 0)
    mm.close()

if __name__ == "__main__":
    main()
```

The following command executes the Copy-on-Write demonstration:

```
python3 fork_cow.py
```

```
Before fork:        present=1 exclusive=1
After fork:         present=1 exclusive=0
After parent write: present=1 exclusive=1
```

Notice the state transitions in the output: - Before the fork, the page is both present and exclusive (mapped only into the parent process). - Immediately after `fork()`, the page remains present, but the exclusive bit drops to false because the page frame is shared by both parent and child. - After the parent performs a write, the kernel triggers copy-on-write and allocates a separate private frame, restoring the exclusive bit to true.

Pitfalls and Common Mistakes

- **Seeking by byte address instead of entry offset.** Seeking to `vaddr` directly in `/proc/PID/pagemap` is incorrect. The file contains 8-byte entries indexed by page number. The seek target must be `(vaddr // PAGE_SIZE) * 8`. Seeking to the unadjusted `vaddr` reads unrelated metadata far past the intended mapping.
- **Assuming PFN availability for unprivileged processes.** Since Linux 4.2, reading PFNs (bits 0–54) requires `CAP_SYS_ADMIN`. Code that assumes non-zero PFNs for present pages will silently misbehave unless executed as root.
- **Ignoring unaligned base addresses.** If inspecting arbitrary pointers, always mask or divide by `PAGE_SIZE`. Passing a pointer into the middle of a page without integer division results in an off-by-one or non-8-byte-aligned file seek.
- **Missing kernel config for soft-dirty.** Soft-dirty tracking requires the kernel to be compiled with `CONFIG_MEM_SOFT_DIRTY=y`. If this option is omitted, writing `4` to `/proc/PID/clear_refs` still succeeds, but bit 55 is permanently 0. The empty result above shows exactly that, and it is easy to misread as "no writes happened".
- **Sequential pagemap parsing at high frequency.** Reading `/proc/PID/pagemap` sequentially across sparse multi-gigabyte heaps causes significant overhead because the kernel must traverse page tables for every 8-byte slice. For bulk dirty-page detection on Linux 6.7 and newer, use the `PAGEMAP_SCAN` ioctl instead.

Summary

- `/proc/PID/pagemap` provides a binary representation of user-space page table entries, formatted as 64-bit little-endian words.
- Seek offsets are computed by `(vaddr // PAGE_SIZE) * 8`, allowing direct indexing into any virtual address range.
- Bit 63 indicates physical memory residency (`present`), bit 62 indicates swap residency, bit 56 flags whether a page is exclusively mapped, and bit 55 tracks modifications (`soft_dirty`).
- Physical Page Frame Numbers (bits 0–54) are sanitized to zero for unprivileged processes to mitigate Rowhammer attacks.
- Demand paging defers physical RAM allocation until first access, while `madvise(MADV_DONTNEED)` unbinds physical frames and clears present bits without destroying the virtual mapping.

- Writing `"4"` to `/proc/self/clear_refs` clears the soft-dirty bit across the calling process's address space, turning bit 55 into an incremental write log, but only on kernels built with `CONFIG_MEM_SOFT_DIRTY=y`.

Sources

- Linux pagemap documentation
- Linux soft-dirty documentation
- Linux kernel page table management
- proc_pid_pagemap(5) man page

Users and Groups

Intro

Intro

Create and modify accounts, password aging, groups, and privilege elevation. Prefer `sudo` for policy-controlled admin work; use account tools carefully on production identity stores.

Commands in this part

Command	Role
`sudo`	sudo runs a command as another user (default: root) according to policy in /etc/sudoers and /etc/sudoers.d/*.
`visudo`	visudo safely edits the sudoers configuration (/etc/sudoers and files under /etc/sudoers.d/).
`useradd`	useradd creates a new user account and related system entries (/etc/passwd, /etc/shadow, /etc/group as needed).
`usermod`	usermod modifies an existing user account: shell, home, groups, UID, lock state, expiry, and more.
`userdel`	userdel deletes a user account from the system.
`passwd`	passwd sets or changes user passwords and can lock/unlock accounts or expire passwords.
`chage`	chage (change age) views and modifies password aging policy for a user: last change date, minimum/maximum days…
`groupadd`	groupadd creates a new group entry in the local group database (/etc/group, and /etc/gshadow when used).
`groupmod`	groupmod modifies an existing group: rename it, change its GID, or adjust related attributes depending on the…
`groupdel`	groupdel deletes a group from the system group database (typically /etc/group and /etc/gshadow).

Command	Role
`getent`	getent queries Name Service Switch (NSS) databases—the same view applications get via glibc: local files, LDAP/SSSD,…
`su`	su (“substitute user”) starts a shell or runs a command as another user.
`sudo-rs`	sudo-rs is the Rust sudo that is `/usr/bin/sudo` on Ubuntu 25.10+; what differs from C sudo, the password-feedback default, and a sudoers preflight before upgrading.

Suggested starting points

1. Elevation: `sudo`, `visudo`, `su`. On Ubuntu 25.10 and later, read `sudo-rs` before relying on C-sudo-only sudoers syntax.
2. Accounts: `useradd`/`usermod`/`userdel`, `passwd`, `chage`.
3. Groups: `groupadd`/`groupmod`/`groupdel`.
4. Lookups: `getent` (works with NSS/LDAP backends, not only local files).

Related parts

- Security — SELinux contexts when access still fails
- Files and paths — ownership and modes
- Services and runtime — run daemons as dedicated users

Continue with the individual command pages in this part.

sudo

Overview

`sudo` runs a command as another user (default: **root**) according to policy in `/etc/sudoers` and `/etc/sudoers.d/*`. Prefer one-shot elevation over lingering in a root shell. On Ubuntu, members of the `sudo` group typically get full rights via the distro sudoers snippet.

Syntax

```
sudo [options] [command]
sudo -i
sudo -u user command
sudoedit file          # same as sudo -e
```

Common Options

Option	Description
`-u user`	Run as user (not root)
`-g group`	Set primary group for the command
`-i`	Login shell as target user (loads target profile)
`-s`	Shell as target user (minimal env change)
`-E`	Preserve environment (policy permitting — often restricted)
`-k`	Invalidate cached credentials
`-K`	Remove timestamp entirely
`-l` / `-l -U user`	List allowed commands
`-n`	Non-interactive (fail if a password would be needed)
`-v`	Validate / extend timestamp without running a command
`-e` / `sudoedit`	Edit file safely as root
`-b`	Run command in background
`-g group`	Primary group for the command

Key Use Cases

1. Administrative package and service changes
2. Least-privilege elevation per command
3. Audited privileged actions (auth log / journal)
4. Safe file edits as root (`sudoedit`)
5. Run maintenance as a service account (`-u`)

Safety

- **Never** `chmod` sudoers to world-writable. Always edit with `visudo`.
- Pipelines: `sudo cat file | grep x` only elevates `cat`. Use `sudo sh -c '…'` or `sudo grep x file` when the whole pipeline needs root.
- Avoid `sudo su` / `sudo -i` as a daily habit — harder to audit, easy to leave open.
- Be careful with `sudo -E` and `env_keep` — leaking `LD_PRELOAD`/`PATH` into root is a classic privilege path.
- `sudoedit` uses your editor on a temp copy; do not point `$EDITOR` at untrusted wrappers.

Examples with Explanations

Run one admin command

```
sudo apt update
sudo systemctl restart nginx
```

Elevates only those processes; your shell remains an unprivileged user.

Edit a system file safely

```
sudoedit /etc/hosts
sudo -e /etc/ssh/sshd_config
```

Writes to a temporary file and installs only if the editor exits successfully — safer than some `sudo vim` permission edge cases.

Root login shell when you need many steps

```
sudo -i
# … work …
exit
```

Loads root's login environment (`HOME=/root`, profile scripts). Exit promptly when finished.

Run as another service user

```
sudo -u www-data ls -la /var/www
sudo -u postgres psql -c '\conninfo'
```

Debug permissions and DB access as the account that actually runs the service.

See your privileges

```
sudo -l
```

Shows what policy allows for your account (and matching host/command rules).

Pass env vars explicitly (preferred over `-E`)

```
sudo ENV=prod /usr/local/bin/deploy
sudo --preserve-env=http_proxy,https_proxy apt update   # when policy allows
```

Prefer naming variables over blanket environment preservation.

Credential cache control

```
sudo -v          # refresh timestamp
sudo -k          # drop cache for this tty (next sudo asks again)
```

Default timeout is often ~15 minutes per tty — do not assume it on shared jump hosts.

Non-interactive / scripts

```
sudo -n true || { echo "sudo needs a password; cannot continue" >&2; exit 1; }
sudo -n systemctl is-active nginx
```

`-n` fails instead of prompting — essential for unattended scripts.

Dangerous patterns to avoid

```
# BAD: elevates only the left side
sudo cat /etc/shadow | grep root

# GOOD:
sudo grep root /etc/shadow
sudo sh -c 'cat /etc/shadow | grep root'
```

Read logs of sudo use

```
journalctl -u sudo -n 50 --no-pager          # when unit exists
grep -i sudo /var/log/auth.log | tail        # Debian/Ubuntu rsyslog path
journalctl _COMM=sudo -n 50 --no-pager
```

Understanding Output

Successful commands look like the underlying tool's output. Failures include password rejects, policy denials (`Sorry, user … is not allowed…`), and missing tty for password prompts. Exit status is usually that of the invoked command; policy denials return non-zero without running it.

Notes & Pitfalls

- Ubuntu's default is group-based admin (`%sudo ALL=(ALL:ALL) ALL`) — still prefer explicit command lists on hardened hosts.
- Lecture/insults and password prompts are configured in sudoers (`Defaults`).
- On Ubuntu 25.10 and later, `/usr/bin/sudo` is **sudo-rs** (C sudo stays installed as `sudo.ws`). It accepts a subset of sudoers syntax and shows `*` while you type the password. See the `sudo-rs` page in this part before upgrading hosts with custom sudoers rules.
- `sudo !!` re-runs the previous shell command with sudo — review `!!` expansion before Enter.
- Root-equivalent via misconfigured `NOPASSWD: ALL` is convenient and dangerous; scope it tightly.
- Containers: passwordless sudo inside images is common for convenience — treat those images as root-equivalent.

Related Commands

- `visudo` — edit sudoers safely
- `su` / `runuser` — switch user (different auth model)
- `passwd` — change passwords
- `pkexec` — polkit elevation on desktops
- `loginctl` — session / lingering control

Additional Resources

- `man sudo` / `man sudoers`
- `visudo(8)`

visudo

Overview

`visudo` safely edits the **sudoers** configuration (`/etc/sudoers` and files under `/etc/sudoers.d/`). It locks the file and **syntax-checks** before installing changes — preventing a bad edit from locking out all sudo access. Always use `visudo` instead of editing sudoers directly with a random editor.

Syntax

```
visudo [options]
visudo -f file
```

Common Options

Option	Description
`-f file`, `--file=file`	Edit a specific file (e.g. drop-in)
`-c`, `--check`	Check syntax only
`-s`, `--strict`	Stricter parse
`-q`, `--quiet`	Quiet
`-x file`	Export as JSON (newer) / legacy dump options vary
`-N`	No default includes (see man)

Editor from `$EDITOR` / `$VISUAL` / compile-time default (`vi`/`nano`).

Examples with Explanations

Edit main sudoers

```
sudo visudo
```

Edit a drop-in (preferred)

```
sudo visudo -f /etc/sudoers.d/90-alice
```

Drop-ins should be numeric-prefixed and avoid `.` or `~` in names (sudo ignores many invalid names).

Check syntax

```
sudo visudo -c
sudo visudo -cf /etc/sudoers.d/90-alice
```

Common policy snippets (examples)

```
# /etc/sudoers.d/90-alice  (via visudo -f)
alice ALL=(ALL) ALL
# passwordless for a single command (still risky):
alice ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx
# group based:
%sudo ALL=(ALL:ALL) ALL
```

Understand tag implications (`NOPASSWD`, `SETENV`, `NOEXEC`) before deploying.

Recover from broken sudoers (prevention)

If you never skip `visudo`, you avoid the classic lockout. Recovery usually needs:

- console/root login
- or break-glass cloud serial console
- then `visudo` fix

Test as user

```
sudo -l -U alice
sudo -u alice sudo -l
```

Notes / Pitfalls

- Never `nano /etc/sudoers` without visudo — a typo can deny all sudo.
- Last matching rule wins in many cases — order matters.
- `#includedir /etc/sudoers.d` is standard; file names must meet sudo's filters.
- `NOPASSWD: ALL` is almost always too broad for humans.
- Keep a root console session open while testing policy changes on remote hosts.

2026-relevant notes

- Prefer small drop-ins per role over one monolithic sudoers file.
- Many orgs move to SSH certs + policy engines; sudoers still ubiquitous on servers.
- Audit with `sudo -l` and central config management (Ansible `template` + validate).

Related Commands

- `sudo` / `sudo -l`
- `sudoedit` — edit files as another user safely
- `pkexec` — polkit privilege (desktop/admin)
- `su` — switch user
- editor env: `EDITOR=vim visudo`

Additional Resources

- `man visudo`, `man sudoers`

useradd

Overview

`useradd` creates a new user account and related system entries (`/etc/passwd`, `/etc/shadow`, `/etc/group` as needed). On **Ubuntu/Debian**, the friendlier interactive wrapper is **`adduser`**; `useradd` is the low-level, script-friendly tool common in configuration management. Creating a login-ready human usually means: `useradd -m` → `passwd` → optional groups → verify with `getent`/`id`.

Syntax

```
sudo useradd [options] LOGIN
```

Common Options

Option	Description
`-m`, `--create-home`	Create home directory from skeleton
`-M`	Do **not** create home (override defaults)
`-d HOME`, `--home-dir`	Home path (default under `/home`)
`-s SHELL`, `--shell`	Login shell
`-g GROUP`, `--gid`	Primary group (name or GID)
`-G GROUPS`, `--groups`	Comma-separated **supplementary** groups
`-u UID`, `--uid`	Numeric UID
`-c COMMENT`, `--comment`	GECOS (full name, room, …)
`-r`, `--system`	System account (UID from system range; often no aging)
`-e EXPIRE`, `--expiredate`	Account expiry (`YYYY-MM-DD`)
`-f INACTIVE`	Days after password expiry until disable
`-p PASSWORD`	**Encrypted** password string (prefer `passwd`/`chpasswd`)
`-k SKEL`	Skeleton directory (default `/etc/skel`)
`-N`	Do not create a group with the same name as the user
`-U`	Create a same-named user-private group (common default)
`-D`	Display (or set, with flags) default `useradd` values

Defaults and files

Path	Role
`/etc/login.defs`	UID/GID ranges, mail dir, umask-related defaults
`/etc/default/useradd`	Default shell, home base, expire, skeleton
`/etc/skel/`	Files copied into new homes with `-m`
`/etc/passwd` / `/etc/shadow` / `/etc/group`	Account databases

View effective defaults:

```
sudo useradd -D
```

Safety

- Do not pass cleartext secrets on the command line via `-p` — they land in shell history and process listings; use `passwd` or carefully controlled `chpasswd`.
- Choose UIDs deliberately when syncing NFS/LDAP identities — collisions break ownership.
- System accounts (`-r`) should usually get `/usr/sbin/nologin` or `/bin/false`.
- Creating users with `sudo`/admin groups is a privilege decision — prefer documented role groups.

Key Use Cases

1. Provision local human accounts on Ubuntu servers
2. Create service accounts for daemons
3. Align UID/GID with networked filesystems
4. Script reproducible account creation in labs/CI images

Examples with Explanations

Example: human user with home and bash

```
sudo useradd -m -s /bin/bash -c 'Alice Admin' alice
sudo passwd alice
id alice
getent passwd alice
ls -la /home/alice
```

`-m` is easy to forget — without it, no home directory is created on many defaults.

Example: Ubuntu interactive alternative

```
sudo adduser charlie
```

Walks through password and GECOS prompts; preferred for one-off humans on Debian/Ubuntu.

Example: supplementary groups at creation

```
sudo useradd -m -s /bin/bash -G sudo,docker bob
id bob
```

`-G` sets supplementary groups. On Ubuntu, membership in `sudo` grants sudo via the default sudoers rule (still verify policy).

Example: service / system account

```
sudo useradd -r -s /usr/sbin/nologin -d /nonexistent -c 'App service' appsvc
getent passwd appsvc
```

No interactive shell; often no home. Pair with systemd `User=` directives.

Example: fixed UID/GID for NFS

```
sudo groupadd -g 12000 appdata
sudo useradd -m -u 12000 -g appdata -s /bin/bash appuser
id appuser
```

Match numeric IDs across servers that share filesystems without centralized identity.

Example: account expiry

```
sudo useradd -m -e 2026-12-31 -s /bin/bash contractor
sudo chage -l contractor
```

Expiry disables login after the date; refine aging with `chage`.

Example: custom home location

```
sudo useradd -m -d /srv/homes/alice -s /bin/bash alice
```

Ensure parent directories exist and permissions make sense before login.

Example: do not create user-private group

```
sudo useradd -m -N -g users -s /bin/bash dave
```

`-N` avoids a per-user group when site policy uses a shared primary group.

Example: verify skeleton contents

```
ls -la /etc/skel
sudo useradd -m -k /etc/skel -s /bin/bash skeltest
ls -la /home/skeltest
sudo userdel -r skeltest
```

Custom skeletons help place mandatory `.bashrc` snippets or ssh config templates.

Notes & Pitfalls

- **`useradd` vs `adduser`**: different tools; scripts should call `useradd` for stable flags; humans on Ubuntu often use `adduser`.
- Without `-m`, applications expecting `$HOME` fail in confusing ways.
- Primary group (`-g`) vs supplementary (`-G`): both matter for file access.
- LDAP/FreeIPA/sssd environments: create users in the identity system, not only locally — local-only accounts confuse operators.
- Removing accounts is `userdel` (see that page); home cleanup needs `-r` carefully.
- Default shell paths differ: `/bin/bash` vs `/usr/bin/bash` — use paths present on the image (`getent passwd` after creation).

Related Commands

- `adduser` — Debian/Ubuntu interactive helper
- `usermod` — modify existing accounts
- `userdel` — delete accounts
- `groupadd` / `gpasswd` — groups
- `passwd` / `chage` — passwords and aging
- `getent` / `id` — verify lookups

Additional Resources

- `man useradd`
- `man login.defs`
- `man adduser` (Debian/Ubuntu)

usermod

Overview

`usermod` modifies an existing user account: shell, home, groups, UID, lock state, expiry, and more. Prefer targeted flags over hand-editing `/etc/passwd`. For group membership, know the difference between **replace** (`-G`) and **append** (`-aG`) — forgetting `-a` is a classic outage that drops sudo rights.

Syntax

```
sudo usermod [options] LOGIN
```

Common Options

Option	Description
`-l NEWNAME`, `--login`	Rename login
`-c COMMENT`, `--comment`	GECOS field
`-d HOME`, `--home`	New home path
`-m`, `--move-home`	Move home contents when used with `-d`
`-s SHELL`, `--shell`	Login shell
`-u UID`, `--uid`	Change UID (files not auto-chowned everywhere)
`-g GROUP`, `--gid`	Primary group
`-G GROUPS`, `--groups`	**Set** supplementary groups to this list (replaces!)
`-a`, `--append`	With `-G`, **append** groups instead of replacing
`-L`, `--lock`	Lock password
`-U`, `--unlock`	Unlock password
`-e DATE`, `--expiredate`	Account expiry (`YYYY-MM-DD` or empty to clear)
`-f DAYS`, `--inactive`	Inactivity after password expiry
`-p PASSWORD`	Set encrypted password (prefer `passwd`)
`-r`	When changing UID, experimental options vary — read man page carefully

Critical group flags: always use `-aG` to add supplementary groups without wiping existing ones.

Safety

- **`usermod -G group user` without `-a` replaces all supplementary groups.** Operators have locked themselves out of `sudo` this way. Prefer:

  ```
  sudo usermod -aG docker alice
  ```

- Changing UID/home on a live logged-in user confuses running sessions; schedule maintenance.
- Locking (`-L`) blocks password auth; SSH keys may still work — know your access paths.
- Rename (`-l`) does not rewrite crontabs, systemd units, mail spools, or ACLs automatically.

Key Use Cases

1. Add a user to `sudo`, `docker`, or app groups (`-aG`)
2. Change login shell or GECOS
3. Move/rename home during account cleanup
4. Lock contractors without deleting data immediately

Examples with Explanations

Example: append supplementary groups (`-aG`)

```
id alice
sudo usermod -aG docker,sudo alice
id alice
# alice must re-login (or newgrp) for group membership to apply fully
```

`-aG` is the safe daily form. Verify with `id` after a fresh login.

Example: wrong vs right group edit

```
# DANGEROUS - replaces supplementary groups with only "docker"
sudo usermod -G docker alice

# CORRECT - append docker, keep sudo and others
sudo usermod -aG docker alice
```

If you truly intend to set the definitive list, pass the full list explicitly:

```
sudo usermod -G sudo,docker,adm alice
```

Example: change shell

```
sudo usermod -s /bin/bash alice
getent passwd alice
```

Common fix when an account was created with `nologin` by mistake (for human users only).

Example: lock and unlock

```
sudo usermod -L alice
passwd -S alice
sudo usermod -U alice
```

Related: `passwd -l`/`-u`. Locking is not the same as setting shell to nologin — consider both for service accounts.

Example: move home directory

```
sudo usermod -d /srv/homes/alice -m alice
ls -la /srv/homes/alice
getent passwd alice
```

`-m` moves files from the old home. Ensure destination parent exists; watch disk space and SELinux/AppArmor contexts on hardened systems.

Example: rename login

```
sudo usermod -l newalice alice
# often also:
sudo usermod -d /home/newalice -m newalice
```

Update references in sudoers, cron, CI credentials, and docs.

Example: change primary group

```
sudo groupadd appteam
sudo usermod -g appteam alice
id alice
```

Primary group affects default group of newly created files (with usual umask/dir sticky behaviors).

Example: change UID carefully

```
sudo usermod -u 12000 alice
# fix ownership of home (minimum):
sudo chown -R alice:alice /home/alice
# search for old UID leftovers if needed:
sudo find / -xdev -uid 1001 2>/dev/null
```

UID changes do not rewrite the whole filesystem for you.

Example: expiry

```
sudo usermod -e 2026-12-31 alice
sudo usermod -e '' alice          # clear expiry (syntax may vary; confirm man)
sudo chage -l alice
```

Example: service account hardening

```
sudo usermod -s /usr/sbin/nologin -L appsvc
```

No interactive shell + locked password for non-human accounts (SSH keys should also be absent).

Notes & Pitfalls

- Group changes apply to **new** sessions; existing SSH sessions keep old group sets until re-login.
- `newgrp` / `sg` can temporarily switch for testing without full logout.
- Directory services (sssd/LDAP): modify the identity source of truth, not only the local cache.
- `-a` without `-G` is an error; they are a pair for append mode.
- Prefer `gpasswd -a user group` as an alternative for single-group adds.

Related Commands

- `useradd` / `userdel` — create/delete
- `groupadd` / `gpasswd` / `groupmod` — group side
- `passwd` / `chage` — password and aging
- `id` / `getent` — verify
- `vipw` — last-resort file edit (avoid)

Additional Resources

- `man usermod`
- `man group(5)`, `man passwd(5)`

userdel

Overview

`userdel` deletes a user account from the system. By default it removes the account entry but may leave the home directory and mail spool unless you pass `-r`. Always confirm you have backups and that no critical processes still run as that user.

Syntax

```
userdel [options] LOGIN
```

Common Options

Option	Description
`-r`, `--remove`	Remove home directory and mail spool
`-f`, `--force`	Force removal even if logged in / files busy (dangerous)
`-Z`	Remove SELinux user mapping (when applicable)
`-R CHROOT`	Chroot
`-P PREFIX`	Prefix

Examples with Explanations

Delete account, keep home

```
id bob
sudo userdel bob
ls /home/bob                # may still exist
```

Delete account and home

```
sudo userdel -r bob
```

Refuse when logged in

```
who | grep bob
loginctl user-status bob
sudo userdel bob             # may fail if logged in
# terminate sessions first:
sudo loginctl terminate-user bob
sudo userdel -r bob
```

Processes still running

```
ps -u bob
sudo pkill -u bob
sudo userdel -r bob
```

Orphan files outside home

```
# after delete, numeric UID may remain on files:
sudo find /var /srv -nouser 2>/dev/null | head
```

Reassign or delete intentionally.

System users

```
# prefer package uninstall / systemd DynamicUser cleanup
# only manually remove custom system accounts you created
sudo userdel -r appuser
```

Notes / Pitfalls

- Without `-r`, homes accumulate on disk — disk bloat and privacy residue.
- Force-deleting logged-in users can confuse running jobs; terminate cleanly first.
- Crontabs, systemd user units, and mail spools may need manual cleanup.
- UID reuse: deleting then creating a new user with same UID exposes old files — wipe or reown.
- LDAP users: use directory tools, not local `userdel`, for network accounts.

2026-relevant notes

- Prefer lifecycle via configuration management and IdP offboarding playbooks.
- Containers: removing users in a container image layer doesn't rewrite earlier layers' files.
- Check `loginctl`, `crontab -u`, and `/var/lib/systemd/` leftovers for interactive users.

Related Commands

- `useradd` / `usermod`
- `passwd` / `chage`
- `groupdel`
- `find -nouser`
- `loginctl`
- `vipw` — manual editor (last resort)

Additional Resources

- `man userdel`

passwd

Overview

`passwd` sets or changes user passwords and can lock/unlock accounts or expire passwords. Ordinary users change their own password; root can set any user's password and manage lock state. Aging policy is refined with `chage`.

Syntax

```
passwd [options] [LOGIN]
```

Common Options

Option	Description
`-S`, `--status`	Account password status
`-l`, `--lock`	Lock password (prefix `!` in shadow)
`-u`, `--unlock`	Unlock password
`-d`, `--delete`	Delete password (careful: may allow passwordless login depending on PAM)
`-e`, `--expire`	Expire password; force change on next login
`-n MIN` / `-x MAX` / `-w WARN` / `-i INACT`	Aging shortcuts (often prefer `chage`)
`-a`, `--all`	Status for all users (root)
`--stdin`	Read password from stdin (root; scripts — prefer chpasswd carefully)

Examples with Explanations

Change passwords

```
passwd                    # own password
sudo passwd alice         # set alice's password as root
```

Status

```
passwd -S alice
sudo passwd -Sa | head
```

Typical status letters: `P` (usable password), `L` (locked), `NP` (no password) — confirm with `man passwd` on your distro.

Lock / unlock

```
sudo passwd -l alice
sudo passwd -u alice
sudo usermod -L alice      # related
sudo usermod -U alice
```

Locking disables password auth; SSH keys may still work depending on config.

Force change next login

```
sudo passwd -e alice
# or
sudo chage -d 0 alice
```

Scripted set (controlled environments)

```
echo 'alice:NewSecurePass' | sudo chpasswd
# passwd --stdin is not portable across all distros
```

Prefer configuration management vaults over shell history for secrets.

Service accounts

```
sudo passwd -l svc_backup
# or lock shell:
sudo usermod -s /usr/sbin/nologin svc_backup
```

Notes / Pitfalls

- PAM policy enforces complexity/retries — errors may be vague.
- Root bypasses most complexity checks — still use strong secrets.
- Deleting passwords (`-d`) can be dangerous with certain PAM stacks.
- LDAP/sssd users: password changes may need `passwd` via SSSD or IdP self-service.
- Shadow file integrity: never edit `/etc/shadow` by hand if tools exist.

2026-relevant notes

- Prefer SSO/IdP for humans; local `passwd` for break-glass and system accounts.
- SSH certificate / key-only logins reduce password surface.
- Pair with `chage` for compliance aging on local accounts.

Related Commands

- `chage` — aging policy
- `chpasswd` — batch updates
- `usermod -L/-U` — lock flags
- `vipw -s` — edit shadow carefully
- `pam-auth-update` — PAM stacks (Debian family)

Additional Resources

- `man passwd`, `man shadow`, `man chage`

chage

Overview

`chage` (change age) views and modifies **password aging** policy for a user: last change date, minimum/maximum days between changes, warn days, and account inactivity/expiry. Complements `passwd` (set password) and is useful for compliance-driven rotation policies.

Syntax

```
chage [options] LOGIN
chage -l LOGIN                  # list aging info
```

Common Options

Option	Description
`-l`, `--list`	Show aging info
`-d DATE`, `--lastday`	Last password change day
`-E DATE`, `--expiredate`	Account expiration date (`-1` never)
`-I DAYS`, `--inactive`	Days after password expiry until account inactive (`-1` never)
`-m DAYS`, `--mindays`	Minimum days between password changes
`-M DAYS`, `--maxdays`	Maximum password age
`-W DAYS`, `--warndays`	Warn days before password expiry
`-R CHROOT`	Chroot

Dates are often `YYYY-MM-DD` or days since 1970-01-01 depending on option forms — see `man chage`.

Examples with Explanations

View policy

```
sudo chage -l alice
```

Force change on next login

```
sudo chage -d 0 alice
# last day set so password is immediately expired → user must change
```

Set max age 90 days, warn 14

```
sudo chage -M 90 -W 14 alice
```

Disable aging

```
sudo chage -M -1 -E -1 alice
```

Account expiry date

```
sudo chage -E 2026-12-31 contractor
sudo chage -E -1 contractor        # clear expiry
```

Batch service accounts

```
# service accounts often should not expire interactively
sudo chage -M -1 -I -1 -E -1 svc_backup
```

Combine with passwd

```
sudo passwd alice
sudo chage -l alice
```

Notes / Pitfalls

- Requires shadow suite and proper privileges.
- Policies in `/etc/login.defs` supply defaults for new users; `chage` adjusts per-user.
- LDAP/sssd environments may store aging in the directory — local `chage` won't apply.
- Forcing `-d 0` on service accounts can break automation that cannot interactively change passwords.
- Date formats and epoch-day forms are easy to mis-set — always `-l` to verify.

2026-relevant notes

- Prefer centralized IdP password policies when users live in SSO; use `chage` for local/system accounts.
- Compliance scanners still look at shadow aging fields on classic Linux servers.
- Pair with `passwd -S` and account locking (`usermod -L`) for full lifecycle control.

Related Commands

- `passwd` — set/lock passwords
- `usermod` — account flags
- `chage -l` vs `passwd -S`
- `vipw` / `vipw -s` — edit passwd/shadow carefully
- `pam_unix` / `login.defs` — defaults

Additional Resources

- `man chage`, `man shadow`

groupadd

Overview

`groupadd` creates a new group entry in the local group database (`/etc/group`, and `/etc/gshadow` when used). Groups grant shared file access and sudo/role membership without sharing login accounts. On Ubuntu, interactive admins sometimes use `addgroup`; automation should prefer `groupadd` for stable flags.

Syntax

```
sudo groupadd [options] GROUP
```

Common Options

Option	Description
`-g GID`, `--gid`	Numeric GID
`-r`, `--system`	System group (GID from system range in `login.defs`)
`-f`, `--force`	Exit successfully if group already exists; still fails on GID conflict unless handled
`-K KEY=VAL`	Override `/etc/login.defs` defaults for this invocation
`-o`, `--non-unique`	Allow non-unique GID (usually avoid)
`-p PASSWORD`	Encrypted group password (rare; prefer no group passwords)
`-U USERS`, `--users`	Comma-separated user list to add (util-linux; version-dependent)

Related membership tools

Creating a group is only half the job — membership is managed with:

```
sudo usermod -aG GROUP USER
sudo gpasswd -a USER GROUP
sudo gpasswd -d USER GROUP
getent group GROUP
```

```
id USER
```

Safety

- Pick GIDs deliberately when using NFS or shared storage; mismatched GIDs mean "same name, different access".
- Do not reuse GIDs of deleted groups until you understand leftover file ownership on disk.
- Group passwords are an old pattern; modern sites use explicit membership instead.
- Naming collisions with system groups (`sudo`, `adm`, `disk`, `docker`) are high risk — list first.

Key Use Cases

1. Create application/shared-data groups (`appdata`, `deploy`)
2. Create system groups for daemons
3. Align GIDs across a fleet for shared filesystems
4. Prepare groups before `useradd -G` / `usermod -aG`

Examples with Explanations

Example: simple project group

```
sudo groupadd appdata
getent group appdata
```

Creates the group with the next available GID from the user range.

Example: fixed GID for NFS fleet

```
sudo groupadd -g 12000 appdata
getent group appdata
```

Use the same GID on every server that mounts the same export.

Example: system group for a daemon

```
sudo groupadd -r appsvc
getent group appsvc
```

System GID range is defined in `/etc/login.defs` (`SYS_GID_MIN` / `SYS_GID_MAX`).

Example: create group then add members (`-aG`)

```
sudo groupadd deploy
sudo usermod -aG deploy alice
sudo usermod -aG deploy bob
getent group deploy
id alice
```

Always **append** with `usermod -aG` so existing supplementary groups remain intact.

Example: Ubuntu interactive alternative

```
sudo addgroup designers
```

Debian/Ubuntu helper; fine for humans, less ideal for exact GID control in scripts.

Example: idempotent-ish create in scripts

```
if ! getent group appdata >/dev/null; then
  sudo groupadd -g 12000 appdata
fi
```

Or experiment with `groupadd -f` carefully — still verify GID when identity matters.

Example: shared directory ownership pattern

```
sudo groupadd appdata
sudo mkdir -p /srv/appdata
sudo chown root:appdata /srv/appdata
sudo chmod 2775 /srv/appdata
# setgid bit (2) → new files inherit group appdata
```

Classic multi-user write directory pattern; pair with sensible umask or ACL if needed.

Example: list and inspect

```
getent group | tail
getent group sudo
grep '^docker:' /etc/group
```

Prefer `getent` so NSS sources (LDAP) are included.

Notes & Pitfalls

- **Name vs number**: tools and NFS care about GID; humans care about names — keep them consistent across systems.
- Deleting groups (`groupdel`) does not rewrite file ownership on disk — orphaned GIDs appear in `ls -ln`.
- Membership changes need re-login to appear in a user's active session.
- Directory services: create groups in FreeIPA/LDAP when that is the source of truth.
- `docker`/socket groups grant near-root power — treat membership as privileged.

Related Commands

- `groupdel` / `groupmod` — delete/rename/change GID
- `gpasswd` — members and administrators of a group
- `usermod -aG` — append user to group
- `useradd -G` — supplementary groups at creation time
- `getent group` / `id` — verify
- `addgroup` — Debian/Ubuntu helper

Additional Resources

- `man groupadd`
- `man group(5)`
- `man login.defs`

groupmod

Overview

`groupmod` modifies an existing group: rename it, change its GID, or adjust related attributes depending on the implementation. Membership lists are often managed via `usermod`/`gpasswd` rather than `groupmod`.

Requires root privileges.

Syntax

```
groupmod [options] GROUP
```

Common Options

Option	Description
`-n NEWNAME`, `--new-name`	Rename group
`-g GID`, `--gid`	Change GID
`-o`, `--non-unique`	Allow non-unique GID
`-p PASSWORD`	Set encrypted password (rare; prefer gpasswd)
`-R CHROOT`	Chroot directory
`-P PREFIX`	Prefix directory

Examples with Explanations

Rename a group

```
getent group devs
sudo groupmod -n developers devs
getent group developers
```

Change GID

```
getent group developers
sudo groupmod -g 1200 developers
```

Warning: files owned by the old numeric GID are **not** rewritten automatically. Plan a `find`/`chgrp` migration.

Migrate file ownership after GID change

```
old=1004
new=1200
sudo find /srv/data -group $old -exec chgrp $new {} +
```

Scope carefully; avoid full-filesystem finds without need.

Non-unique GID (rare)

```
sudo groupmod -o -g 1000 shared
```

Usually a bad idea — collisions confuse tools.

Verify

```
getent group developers
grep developers /etc/group /etc/gshadow
```

Notes / Pitfalls

- Renaming may break configs that hard-code the old group name (sudoers, ACLs, unit files).
- Changing GID without file migration leaves orphaned ownership.
- Directory services: local `groupmod` won't change LDAP groups.
- Running processes keep old credentials until re-login / re-spawn.
- Avoid editing `/etc/group` by hand when `groupmod` exists.

2026-relevant notes

- Infrastructure as code should declare desired group names/GIDs immutably; avoid frequent renames.
- Container images: GID consistency across images matters for shared volumes — set explicitly.
- Prefer supplementary group membership management with `usermod -aG` over renaming shared groups.

Related Commands

- `groupadd` / `groupdel`
- `usermod` / `gpasswd`
- `getent group`
- `chgrp`
- `newgidmap` — user namespace gid maps (advanced)

Additional Resources

- `man groupmod`

groupdel

Overview

`groupdel` deletes a group from the system group database (typically `/etc/group` and `/etc/gshadow`). It does **not** delete users who had that group as supplementary — it removes the group entry. You cannot delete a group that is still any user's **primary** group.

Requires root (or equivalent CAP_DAC / account tools).

Syntax

```
groupdel [options] GROUP
```

Common Options

Option	Description
`-f`, `--force`	Force deletion in some situations (implementation-dependent; use carefully)
`-h`, `--help`	Help
`-R`, `--root CHROOT`	Apply in chroot
`-P`, `--prefix PREFIX`	Alternate passwd/group prefix tree

Examples with Explanations

Delete a group

```
getent group oldteam
sudo groupdel oldteam
getent group oldteam || echo gone
```

Refuse if primary group

```
sudo groupdel alice
# groupdel: cannot remove the primary group of user 'alice'
```

Change users' primary group first:

```
sudo usermod -g users alice
sudo groupdel alice
```

Check membership first

```
getent group devs
# see members field
grep -E '^devs:' /etc/group
sudo libuser-groupdel 2>/dev/null   # alternate tools exist
# list users with primary gid:
gid=$(getent group devs | cut -d: -f3)
awk -F: -v g="$gid" '$4==g {print $1}' /etc/passwd
```

Automation cleanup

```
if getent group tempci >/dev/null; then
  sudo groupdel tempci
fi
```

LDAP / remote groups

```
getent group eng
# if from sssd/ldap, groupdel on local files won't remove directory groups
```

Use your directory admin tools for networked accounts.

Notes / Pitfalls

- Files previously group-owned keep the **numeric GID** — they become "orphan" GIDs in `ls -l` until reowned.
- Does not modify ACLs that referenced the group name (may become unresolved).
- Always inventory with `getent group` and filesystem owners before deleting shared groups.
- Busy production groups (e.g. shared data) need migration of ownership first.
- Containers: deleting groups inside a container doesn't affect the host.

2026-relevant notes

- Prefer configuration management (Ansible users modules) over one-off groupdel on fleets.
- System groups for systemd DynamicUser units are managed differently — don't manually delete unknown system groups.
- Audit file ownership: `find / -group oldgid` can be expensive; scope paths.

Related Commands

- `groupadd` / `groupmod` — create / modify
- `useradd` / `usermod` — user membership
- `getent group` — lookup
- `gpasswd` — administer group members
- `chgrp` / `chown` — fix file ownership

Additional Resources

- `man groupdel`

getent

Overview

`getent` queries **Name Service Switch (NSS)** databases—the same view applications get via glibc: local files, LDAP/SSSD, mDNS, DNS for hosts, and more. Prefer it over `cat /etc/passwd` when directory services or nsswitch plugins are in play.

Syntax

```
getent [options] database [key ...]
```

Common Databases

Database	Purpose
`passwd`	Users
`group`	Groups
`shadow`	Shadow passwords (privileged)
`hosts`	Host lookup
`ahosts` / `ahostsv4` / `ahostsv6`	getaddrinfo-style
`services`	Service name port
`protocols`	Protocol names
`networks`	Network names
`aliases`	Mail aliases (if used)
`ethers`	Hardware addresses (rare)

Common Options

Option	Description
`-s SERVICE`	Force NSS service
`-i`	Case insensitive where supported

Key Use Cases

1. Confirm a user exists in LDAP/SSSD, not only locally
2. Script portable account lookups
3. Debug host resolution order (files/dns)
4. Map service names to ports

Examples with Explanations

Users and groups

```
getent passwd alice
getent passwd 1000
getent group sudo
getent group 27
```

Hosts

```
getent hosts example.com
getent ahosts example.com
getent hosts 1.1.1.1
```

Services

```
getent services ssh
getent services 443/tcp
```

One-liner recipes

```
# Does this UID resolve?
getent passwd 1001 || echo 'unknown uid'

# Members visible via NSS
getent group developers

# Compare file vs NSS
getent passwd alice; grep '^alice:' /etc/passwd || true

# nsswitch insight
cat /etc/nsswitch.conf
```

Notes & Pitfalls

- Empty result means "not found in any configured source," not necessarily "query error."
- `shadow` access is restricted; applications should use proper auth APIs.
- Host lookups follow `nsswitch.conf`—may never touch DNS if `files` hits first.
- Enumerating all LDAP users with bare `getent passwd` can be huge or disabled—don't DoS your directory.

2026-relevant notes

- SSSD/FreeIPA/Azure AD-ish integrations still show up through NSS; `getent` is the smoke test.
- For DNS-only questions use `dig`/`resolvectl`; `getent hosts` includes non-DNS sources.
- Containers often have minimal passwd; UIDs from the host may not resolve by name inside.

Related Commands

- `id` — UIDs/GIDs for a user
- `passwd` / `usermod` — local account changes
- `dig` / `resolvectl` — pure DNS
- `getent` vs `cat /etc/passwd` — NSS is authoritative for apps

Additional Resources

- `man getent`, `man nsswitch.conf`

su

Overview

`su` ("substitute user") starts a shell or runs a command as another user. On modern Ubuntu, prefer `sudo` for policy-controlled elevation; `su` still matters for switching to service accounts, recovering systems without sudo, and older procedures. `runuser` is a non-setuid alternative used in scripts from root.

Syntax

```
su [options] [-] [user [argument...]]
su [options] -c command [user]
```

Common Options

Option	Description
`-` / `-l` / `--login`	Login shell (loads target profile, resets env)
`-c cmd`	Run command then exit
`-s shell`	Use alternate shell
`-w` / `--whitelist-environment`	Preserve listed vars (util-linux)
`-p` / `-m`	Preserve environment (careful)

Safety

- `su -` vs `su` differs wildly in environment (`HOME`, `PATH`). Prefer login form for interactive admin.
- From an unprivileged user, `su` asks for the **target** user's password (root password on many systems) — not your sudo password.
- Avoid `su -c` with unquoted complex scripts; quoting bugs become root bugs.

Examples with Explanations

Become root (login)

```
su -
# or
su - root
```

Run one command as root

```
su -c 'systemctl status ssh' root
```

Switch to a service user

```
sudo su - postgres
sudo -u postgres -i          # usually better when sudo is available
```

Preserve selected environment

```
su -w LANG,TERM - alice
```

Notes & Pitfalls

- Many Ubuntu systems (including Server) have **no root password** by default — `su` to root fails; use `sudo -i`.
- PAM modules may restrict `su` even if the password is known.
- `sudo su` works but obscures audit trails compared to `sudo -i` / direct `sudo cmd`.

Related Commands

- `sudo` — policy-based elevation
- `runuser` — switch user from root without setuid su
- `login` — full login path
- `machinectl shell` — containers/machines

Additional Resources

- `man su`
- `man runuser`

sudo-rs

Overview

Thesis: on Ubuntu 25.10, 26.04 LTS and 26.10, the `sudo` you type is **sudo-rs**, a memory-safe reimplementation in Rust. C sudo is still installed alongside it, as `sudo.ws`. For daily use nothing changes. The two places where it does change are:

- **The password prompt shows `*` per keystroke.** `pwfeedback` is on by default from sudo-rs 0.2.13, which is the version in 26.04.
- **Your sudoers files may no longer parse.** sudo-rs supports a deliberate subset of the sudoers language: no wildcards inside argument lists, no regexes, no `logfile`, no `INTERCEPT`, no `MAIL` tag, and no IPv4 addresses as hosts, among others. C sudo accepts every one of these.

The second point decides whether an upgrade is boring. If a rule that only C sudo understands sits in `/etc/sudoers.d/`, the deploy user that depends on it is the first to find out. So: check every sudoers file with sudo-rs's own `visudo -c` *before* the upgrade. The `.deb` can do that without being installed (Example 5).

```
Ubuntu series          /usr/bin/sudo is   rust-sudo-rs published (Launchpad, 2026-10-15)
24.04 LTS noble        C sudo             0.2.2-1 (optional, not the default)
25.10 questing         sudo-rs            0.2.8-1ubuntu5.3 (updates; series now obsolete)
26.04 LTS resolute     sudo-rs            0.2.13-0ubuntu1.2 (updates, security)
26.10 stonking         sudo-rs            0.2.14-1ubuntu4 (release pocket, pre-release)
upstream latest                           0.2.15 (2026-08-31)
```

Package outputs below come from the Ubuntu 26.04 LTS archive (`ubuntu-base-26.04` root filesystem plus the `sudo-rs`, `sudo` and `sudo-common` packages, fetched 2026-10-15). sudo-rs's `visudo` was run on Debian 13 (glibc 2.41, kernel 6.12). C sudo's `visudo.ws` needs glibc 2.42, so it was run inside the unpacked 26.04 root filesystem. Neither package was installed on the lab host. The interactive password prompt was therefore **not** reproduced here; its behaviour is taken from the sudo-rs man page, changelog and Ubuntu release notes cited below.

Mental model

```
/usr/bin/sudo    /etc/alternatives/sudo     /usr/lib/cargo/bin/sudo   sudo-rs  priority 50  (
                                              /usr/bin/sudo.ws          C sudo   priority 40
  slaves follow the master link:  visudo, sudoedit, and the sudo(8)/sudoers(5)/visudo(8) man
```

```
/etc/sudoers, /etc/sudoers.d/*   one shared policy (shipped by sudo-common), read by whichev
/etc/pam.d/sudo, sudo-i          sudo-rs always authenticates through PAM
```

Fact	Consequence
Both implementations are installed, and `ubuntu-minimal` depends on both	Switching is an `update-alternatives` choice, not a package removal. Removing either package takes the metapackage with it
The policy files are shared	A file valid for one implementation can be invalid for the other. Validate with the one that will run
`man sudoers` follows the alternative	With sudo-rs active you read `sudoers-rs(5)`, which describes the subset. Read the page that matches the binary
sudo-rs rejects unsupported syntax with an error	You get a clear error at check time, not a silently different meaning. Run the check before you depend on it
Some options are fixed	`env_reset` always on, `visiblepw` always off, `use_pty` on by default, `sudo -E` unsupported, logging only through syslog

Syntax

```
sudo --version | head -1                 # which implementation answers to "sudo"
update-alternatives --query sudo         # Value: and the two candidates
sudo update-alternatives --config sudo   # interactive switch between sudo-rs and sudo.ws
sudo visudo -c                           # check /etc/sudoers and its includes with the ACT
visudo -c -f FILE                        # check one file (no root needed for a copy you ca
```

Safety

- Keep a root shell (or console access) open while you change sudoers or switch implementations. A file that one implementation cannot parse can lock you out of sudo. The sudo-rs man page says so explicitly.
- Never edit `/etc/sudoers` or `/etc/sudoers.d/*` directly. `visudo` checks with the implementation it belongs to, so `visudo` and `visudo.ws` can disagree on the same file.
- Treat `pwfeedback` as a policy choice for shared screens (screen sharing, pairing, recorded demos). It reveals password length, nothing more.
- Track sudo-rs security fixes per release. The sudoedit symlink race GHSA-f42v-x7gq-phc8 affects upstream 0.2.8 up to 0.2.14 and is fixed in 0.2.15. Ubuntu 26.04 carries the fix as a patch in `0.2.13-0ubuntu1.2` (Example 7).

Examples with Explanations

1. Which sudo answers, and why

The priorities come from the packages' own maintainer scripts. You can read them without installing anything:

```
# alternatives.sh - show how each package registers /usr/bin/sudo
apt-get download sudo-rs sudo            # or fetch the .debs from the archive pool
for d in sudo-rs_*.deb sudo_*.deb; do
  mkdir -p ctl-$d && dpkg-deb -e "$d" ctl-$d
  grep -A1 'update-alternatives --install /usr/bin/sudo' ctl-$d/postinst
done
```

```
        update-alternatives --install /usr/bin/sudo sudo /usr/lib/cargo/bin/sudo 50 \
            --slave /usr/sbin/visudo visudo /usr/lib/cargo/bin/visudo \
        update-alternatives --install /usr/bin/sudo sudo /usr/bin/sudo.ws 40 \
            --slave /usr/sbin/visudo visudo /usr/sbin/visudo.ws \
```

In auto mode the higher priority wins, so sudo-rs is `/usr/bin/sudo` whenever both are installed. On 26.04 both always are:

```
apt-cache depends ubuntu-minimal | grep -E '^ubuntu|sudo'
apt-cache policy sudo-rs sudo | grep -E '^[a-z]|Candidate'
```

```
ubuntu-minimal
  Depends: sudo
  Depends: sudo-rs

sudo-rs:
  Candidate: 0.2.13-0ubuntu1.2
sudo:
  Candidate: 1.9.17p2-1ubuntu3.2
```

The binary identifies itself:

```
$ sudo --version
sudo-rs 0.2.13-0ubuntu1.2
```

On a real host, `readlink -f "$(command -v sudo)"` resolves to `/usr/lib/cargo/bin/sudo`. `update-alternatives --query sudo` shows `Value:` and both candidates with their priorities.

2. Password feedback: what changed and how to turn it off

sudo-rs 0.2.13 (2026-03-10) changed the default to `Defaults pwfeedback`. Ubuntu 26.04's release notes call it out ("Password feedback is now enabled by default in order to improve the user expe-

rience of sudo") and give the opt-out. Launchpad bug 2142721, which objected to the change, was closed as Won't Fix. Two controls exist:

- **Per prompt:** press **TAB** while typing. Feedback stops for that prompt (sudo-rs 0.2.13+, with Ubuntu's TAB fix patch).
- **Per host:** add the negation to sudoers through `visudo`.

```
sudo visudo -f /etc/sudoers.d/10-no-pwfeedback
```

```
# /etc/sudoers.d/10-no-pwfeedback
Defaults !pwfeedback
```

```
sudo visudo -c
```

Both implementations accept `!pwfeedback`. In the lab, a file containing `Defaults lecture=always`, `Defaults !pwfeedback` and `Defaults timestamp_timeout=5` parsed OK under both checkers (case 06 below).

3. The same file, two verdicts

Twelve small sudoers files, each showing one construct, checked with both implementations from the 26.04 archive:

```
# 01-wildcard-mid.sudoers
deploy ALL=(root) /usr/bin/systemctl restart app-*.service
# 02-wildcard-tail.sudoers
deploy ALL=(root) /usr/bin/journalctl -u app.service *
# 03-noexec-tag.sudoers
deploy ALL=(root) NOEXEC: /usr/bin/less /var/log/syslog
# 04-regex.sudoers
deploy ALL=(root) /usr/bin/systemctl ^(start|stop) app\.service$
# 05-ipv4-host.sudoers
deploy 10.0.0.5=(root) /usr/bin/systemctl restart app.service
# 06-defaults.sudoers
Defaults lecture=always
Defaults !pwfeedback
Defaults timestamp_timeout=5
deploy ALL=(root) /usr/bin/systemctl restart app.service
# 07-mail-tag.sudoers
deploy ALL=(root) MAIL: /usr/bin/systemctl restart app.service
# 08-env-keep.sudoers
Defaults env_keep += "HTTP_PROXY HTTPS_PROXY"
Defaults secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin"
deploy ALL=(root) SETENV: /usr/bin/apt-get update
# 09-logfile.sudoers
Defaults logfile=/var/log/sudo.log
```

```
deploy ALL=(root) /usr/bin/systemctl restart app.service
# 10-intercept.sudoers
deploy ALL=(root) INTERCEPT: /usr/bin/make -C /srv/app deploy
# 11-timestamp-global.sudoers
Defaults timestamp_type=global
# 12-cmd-glob.sudoers
%ops ALL=(root) /usr/sbin/fsck*
```

```
for f in cases/*.sudoers; do
  echo "== ${f##*/}"
  printf '  C sudo : '; visudo.ws -c -f "$f" 2>&1 | tr '\n' ' '; echo
  printf '  sudo-rs: '; /usr/lib/cargo/bin/visudo -c -f "$f" 2>&1 | head -1; echo
done
```

Case	C sudo 1.9.17p2 `visudo.ws`	sudo-rs 0.2.13 `visudo`
01 wildcard inside arguments	parsed OK	`syntax error: wildcards are not allowed in command arguments`
02 trailing `*` (any further args)	parsed OK	parsed OK
03 `NOEXEC:` tag	parsed OK	parsed OK
04 regex arguments (`^…$`)	parsed OK	`syntax error: illegal escape sequence`
05 IPv4 address as host	parsed OK	`syntax error: hosts cannot be specified using an IPv4 address`
06 `lecture`, `!pwfeedback`, `timestamp_timeout`	parsed OK	parsed OK
07 `MAIL:` tag	parsed OK	`syntax error: this tag is ignored by sudo-rs`
08 `env_keep`, `secure_path`, `SETENV:`	parsed OK	parsed OK

Case	C sudo 1.9.17p2 `visudo -c`	sudo-rs 0.2.13 `visudo`
09 `Defaults logfile=`	parsed OK	`syntax error: unknown setting: 'logfile'`
10 `INTERCEPT:` tag	parsed OK	`syntax error: INTERCEPT is not supported by sudo-rs`
11 `timestamp_type=global`	parsed OK	`syntax error: unknown setting: 'timestamp_type'`
12 wildcard in the command name	parsed OK	parsed OK

Ubuntu's stock `/etc/sudoers` (from `sudo-common`) parses OK under both. It uses `env_reset`, `mail_badpass`, `secure_path`, `use_pty`, group rules and `@includedir`. Breakage comes from local additions.

Two of these deserve a note:

- **Case 01 is the important one.** In C sudo, `restart app-*.service` also matches `restart app-x.service other-unit.service`, because `*` in an argument can match spaces. That is a classic over-grant. sudo-rs refuses the construct instead of reproducing the trap. A standalone `*` as the *last* argument means "zero or more further arguments" (case 02, 0.2.13+). Wildcards in the command path still work (case 12).
- **Case 11 depends on the version.** sudo-rs 0.2.15 adds `timestamp_type` with the values `tty` and `ppid`. `global` is not supported in any version. Check with the sudo-rs version that will actually run on the host.

4. Rewrite the rejected rules

Spell out the commands. A `Cmnd_Alias` keeps the rule readable:

```
# 01-wildcard-mid.fixed.sudoers
Cmnd_Alias APP_RESTART = /usr/bin/systemctl restart app-web.service, \
                         /usr/bin/systemctl restart app-worker.service
deploy ALL=(root) APP_RESTART
```

```
# 04-regex.fixed.sudoers
deploy ALL=(root) /usr/bin/systemctl start app.service, \
                  /usr/bin/systemctl stop app.service
```

For the others: drop `logfile` (sudo-rs always logs through syslog(3), so on a systemd host the events are in the journal). Replace `INTERCEPT` with a narrower command list or a wrapper script owned by root. Remove `MAIL:`. Use a hostname or `ALL` instead of an IPv4 address.

5. Preflight: check files with the target release's visudo, without installing it

sudo-rs's `visudo` is a normal dynamically linked program. The package declares `libc6 (>= 2.39)`, and `-c -f` needs no privileges, so you can run it from an extracted `.deb` on a CI runner or on the host before the upgrade:

```
#!/bin/sh
# sudo-rs-preflight.sh FILE... - check sudoers files with the visudo that
# ships in Ubuntu 26.04's sudo-rs package, without installing anything.
# Needs: curl, dpkg-deb, glibc >= 2.39 (the package's own Depends).
set -eu
DEB=sudo-rs_0.2.13-0ubuntu1.2_amd64.deb
URL=http://archive.ubuntu.com/ubuntu/pool/main/r/rust-sudo-rs/$DEB
WORK=${TMPDIR:-/tmp}/sudo-rs-preflight
mkdir -p "$WORK"
[ -f "$WORK/$DEB" ] || curl -sSfo "$WORK/$DEB" "$URL"
dpkg-deb -x "$WORK/$DEB" "$WORK/root"
VISUDO=$WORK/root/usr/lib/cargo/bin/visudo
echo "checker: $("$VISUDO" --version)"
bad=0
for f in "$@"; do
  if out=$("$VISUDO" -c -f "$f" 2>&1); then
    echo "ok    $f"
  else
    bad=$((bad + 1))
    echo "FAIL  $f"
    echo "$out" | sed -n '1,3s/^/      /p'
  fi
done
echo "$bad file(s) need changes before switching to sudo-rs"
[ "$bad" -eq 0 ]
```

```
./sudo-rs-preflight.sh *.sudoers; echo "exit=$?"
```

```
checker: visudo-rs 0.2.13
FAIL  01-wildcard-mid.sudoers
      01-wildcard-mid.sudoers:2:19: syntax error: wildcards are not allowed in command argum
      deploy ALL=(root) /usr/bin/systemctl restart app-*.service
                        ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
ok    02-wildcard-tail.sudoers
ok    03-noexec-tag.sudoers
```

```
FAIL  04-regex.sudoers
      04-regex.sudoers:2:56: syntax error: illegal escape sequence
      deploy ALL=(root) /usr/bin/systemctl ^(start|stop) app\.service$
                                                           ^
FAIL  05-ipv4-host.sudoers
      05-ipv4-host.sudoers:2:8: syntax error: hosts cannot be specified using an IPv4 addres
      deploy 10.0.0.5=(root) /usr/bin/systemctl restart app.service
             ^~~~~~~~
ok    06-defaults.sudoers
FAIL  07-mail-tag.sudoers
      07-mail-tag.sudoers:2:19: syntax error: this tag is ignored by sudo-rs
      deploy ALL=(root) MAIL: /usr/bin/systemctl restart app.service
                        ^~~~
ok    08-env-keep.sudoers
FAIL  09-logfile.sudoers
      09-logfile.sudoers:2:10: syntax error: unknown setting: 'logfile'
      Defaults logfile=/var/log/sudo.log
               ^~~~~~~
FAIL  10-intercept.sudoers
      10-intercept.sudoers:2:19: syntax error: INTERCEPT is not supported by sudo-rs
      deploy ALL=(root) INTERCEPT: /usr/bin/make -C /srv/app deploy
                        ^~~~~~~~~
FAIL  11-timestamp-global.sudoers
      11-timestamp-global.sudoers:2:10: syntax error: unknown setting: 'timestamp_type'
      Defaults timestamp_type=global
               ^~~~~~~~~~~~~~
ok    12-cmd-glob.sudoers
7 file(s) need changes before switching to sudo-rs
exit=1
```

After the rewrites in Example 4:

```
./sudo-rs-preflight.sh *.fixed.sudoers; echo "exit=$?"
```

```
checker: visudo-rs 0.2.13
ok    01-wildcard-mid.fixed.sudoers
ok    04-regex.fixed.sudoers
0 file(s) need changes before switching to sudo-rs
exit=0
```

On a host, point it at `/etc/sudoers /etc/sudoers.d/*` (as root, since the files are mode 0440). The exit status works as a CI gate for configuration-management repositories that template sudoers. For 26.10, change `DEB=` to the 0.2.14 package. The checker must match the release you are moving to.

6. Switch implementation (and back)

```
sudo update-alternatives --config sudo                  # pick interactively
sudo update-alternatives --set sudo /usr/bin/sudo.ws    # C sudo, scripted
sudo update-alternatives --auto sudo                    # back to priority order (sudo-rs)
sudo --version | head -1
```

The `visudo`, `sudoedit` and man-page slaves move with the master link (Example 1), so after `--set sudo /usr/bin/sudo.ws`, plain `visudo` is C sudo's again. Run `sudo visudo -c` straight after any switch, from a second, already-root shell.

7. Know which security fixes you have

```
apt changelog sudo-rs | head -6
```

```
rust-sudo-rs (0.2.13-0ubuntu1.2) resolute-security; urgency=medium

  * d/patches/fix-sudoedit.patch: Fix privilege escalation race in sudoedit
    (LP: #2165142)

 -- Simon Johnsson <simon.johnsson@canonical.com>  Thu, 27 Aug 2026 10:15:26 +0200
```

(Lab output from `zcat usr/share/doc/sudo-rs/changelog.Debian.gz` inside the extracted package. `apt changelog` shows the same text on an installed host.) Upstream advisory GHSA-f42v-x7gq-phc8 ("Time-of-check vs time-of-use issue in sudoedit allows local privilege escalation") covers sudo-rs `>= 0.2.8, < 0.2.15`. A Ubuntu version string like `0.2.13` does not tell you whether the fix is in. The package changelog does.

The trap

1. **Upgrading first, validating second.** The first person to discover that `/etc/sudoers.d/50-deploy` uses a mid-argument wildcard is the deploy pipeline, after the release upgrade. Run the preflight against the target release's checker before you upgrade.
2. **Trusting `visudo -c` from the wrong implementation.** On a 24.04 host, `visudo` is C sudo's and passes everything above. On 26.04, `visudo` is sudo-rs's. `visudo.ws` is still there and still says "parsed OK".
3. **Reading the wrong man page.** `man sudoers` follows the alternative. Docs and blog posts about sudoers usually describe C sudo, which supports regexes, `logfile`, `INTERCEPT`, LDAP and `cvtsudoers`. sudo-rs supports none of these.
4. **Relying on `sudo -E` in scripts.** sudo-rs does not support it. Use `--preserve-env=VAR,...` with matching `env_keep`, or pass `VAR=value` explicitly under `SETENV:`.
5. **Removing the `sudo-rs` package to "go back".** `ubuntu-minimal` depends on it. Use `update-alternatives`, not `apt remove`.

The boring rule

On Ubuntu 25.10 and later, assume `sudo` is sudo-rs. Keep sudoers inside its subset: explicit command lists, a trailing `*` only where any further arguments are really acceptable, no regexes, hosts by name or `ALL`, logging through the journal. Gate every sudoers change in CI with the checker from the release you run. If you need a C-sudo-only feature, switch with `update-alternatives` on that host deliberately and write down why. Decide `pwfeedback` per host class, once, with a drop-in file, not per user.

Try this

1. Run `sudo-rs-preflight.sh /etc/sudoers /etc/sudoers.d/*` (as root) on a 24.04 host you plan to upgrade. How many files fail?
2. On a 26.04 VM, type a sudo password, press TAB halfway through, and watch the feedback stop. Then add `Defaults !pwfeedback` with `visudo -f /etc/sudoers.d/10-no-pwfeedback` and confirm the prompt is silent.
3. On a throwaway VM running C sudo, install `deploy ALL=(root) /usr/bin/systemctl restart app-*.service` and run `sudo -l -U deploy /usr/bin/systemctl restart app-x.service ssh.service`. If it prints the command, the rule allows it. That is the over-grant sudo-rs rejects.
4. Swap the `DEB=` line to the 26.10 package (`sudo-rs_0.2.14-1ubuntu4_amd64.deb`) and re-run case 11 with `timestamp_type=tty`. Compare with 0.2.13.

Sources

- sudo-rs README, "Differences from original sudo": https://github.com/trifectatechfoundation/sudo-rs#differences-from-original-sudo
- sudo-rs CHANGELOG (0.2.13 `pwfeedback` default, TAB toggle, final-`*` wildcard; 0.2.14; 0.2.15 `timestamp_type=ppid`): https://github.com/trifectatechfoundation/sudo-rs/blob/main/CHANGELOG.md
- `sudoers-rs(5)`, `sudo-rs(8)`, `visudo-rs(8)`: man pages shipped in `sudo-rs 0.2.13-0ubuntu1.2` (also rendered in the repository's `docs/`)
- Advisory GHSA-f42v-x7gq-phc8 (sudoedit TOCTOU, fixed in 0.2.15): https://github.com/trifectatechfoundation/sudo-rs/security/advisories/GHSA-f42v-x7gq-phc8
- Ubuntu 26.04 LTS release notes, "sudo-rs" (password feedback default and `Defaults !pwfeedback`): https://documentation.ubuntu.com/release-notes/26.04/changes-since-previous-interim/
- Launchpad, `rust-sudo-rs` publishing history (24.04 to 26.10 versions): https://launchpad.net/ubuntu/+source/rust-sudo-rs
- Launchpad bug 2142721 (asterisks report, "Won't Fix", the `!pwfeedback` workaround): https://bugs.launchpad.net/ubuntu/+source/rust-sudo-rs/+bug/2142721
- Package maintainer scripts and metadata: `sudo-rs_0.2.13-0ubuntu1.2`, `sudo_1.9.17p2-1ubuntu3.2`, `sudo-common_1.2ubuntu`, `ubuntu-minimal 1.570` (Ubuntu 26.04 archive)
- C sudo `sudoers(5)` for the constructs sudo-rs omits: https://www.sudo.ws/docs/man/sudoers.man/

Networking

Intro

Intro

Configure and diagnose connectivity: addresses and routes (`ip`), sockets (`ss`), DNS, HTTP clients, SSH/file transfer, firewalls, and packet capture. Prefer modern tools (`ip`/`ss`/`nft`) over legacy ones.

Commands in this part

Command	Role
`ip`	ip (from iproute2) configures and displays interfaces, addresses, routes, neighbors, rules, and tunnels.
`ss`	ss (socket statistics) inspects TCP/UDP/UNIX sockets.
`ping`	ping sends ICMP Echo Request packets and reports whether a host answers with Echo Reply, plus round-trip time (RTT).
`traceroute`	traceroute maps the path packets take toward a destination by sending probes with increasing TTL and reading ICMP…
`mtr`	mtr (My Traceroute) combines continuous ping and traceroute: it probes each hop on the path and updates loss and…
`dig`	dig (Domain Information Groper) queries DNS servers and prints detailed answers.
`host`	host is a simple DNS lookup utility from BIND tools.
`nslookup`	nslookup queries DNS name servers interactively or non-interactively.
`doggo`	doggo is a modern command-line DNS client with colorful, human-friendly output.

Command	Role
`resolvectl`	resolvectl manages and queries systemd-resolved: DNS servers, search/route domains, LLMNR/mDNS, DNSSEC/DoT status,…
`curl`	curl transfers data to/from URLs.
`wget`	wget is a non-interactive downloader for HTTP(S), FTP, and related protocols.
`ssh`	ssh (OpenSSH client) creates encrypted sessions to a remote host for shell access, command execution, port…
`scp`	scp (secure copy) copies files to or from a remote host over SSH.
`rsync`	rsync synchronizes files and directories locally or over a remote shell (almost always SSH).
`nc`	nc (netcat) is a Swiss-army TCP/UDP tool: open connections, listen for inbound sockets, scan ports simply, and pipe…
`tcpdump`	tcpdump captures and displays packets on a network interface.
`nmap`	The nmap (Network Mapper) is a powerful open-source tool for network exploration, security scanning, and auditing.
`openssl`	openssl is a Swiss-army toolkit for TLS/SSL, X.509 certificates, keys, digests, and simple encrypted files.
`nmcli`	nmcli is the CLI for NetworkManager.
`ufw`	ufw (Uncomplicated Firewall) is Ubuntu's friendly frontend to netfilter.
`nft`	nft is the userspace CLI for nftables, the modern Linux packet-filtering framework that replaces legacy iptables…
`iptables`	iptables configures IPv4 packet filtering/NAT via netfilter.
`ifconfig`	ifconfig configures and displays network interfaces from the legacy net-tools package.
`netstat`	netstat displays network connections, listening sockets, routing tables, and interface statistics from the legacy…

Command	Role
`ftp`	ftp is the classic interactive client for the File Transfer Protocol.
`telnet`	telnet is a historic remote terminal protocol client.
`ssh-keygen`	ssh-keygen creates and manages SSH key pairs, fingerprints host keys, and can update known_hosts.
`ssh-copy-id`	ssh-copy-id installs your public key on a remote account's ~/.ssh/authorized_keys with safer permissions than…
`sftp`	sftp is the interactive SSH file-transfer client (SFTP subsystem).
`ethtool`	ethtool queries and controls Ethernet NIC settings: link speed/duplex, offloads, ring parameters, and driver info.
`firewall-cmd`	firewall-cmd is the client for firewalld, the dynamic firewall manager on Fedora, RHEL, CentOS Stream, and many…
ssh-agent / ssh-add	ssh-agent holds decrypted private keys in memory; ssh-add loads, lists, locks, and removes those identities.
Certificate renewal audit	Read real certificate lifetimes, ask the CA when to renew (ACME ARI), and find renewal schedules and alerts that hard-code a 90-day lifetime before 64-day (2027) and 45-day (2028) certificates arrive.

Suggested starting points

1. Local stack: `ip`, `ss`, `nmcli`/`resolvectl` as appropriate.
2. Path checks: `ping`, `traceroute`/`mtr`, then `dig`/`host` for DNS.
3. HTTP and APIs: `curl` (and `wget` for simple downloads).
4. Remote admin: `ssh`, keys via `ssh-keygen`/`ssh-copy-id`/`ssh-agent`, files via `scp`/`sftp`/`rsync`.
5. Firewalls: `ufw` (Ubuntu), `firewall-cmd` (firewalld), or `nft`/`iptables`.
6. Deep debug: `tcpdump`, `nmap`, `openssl`, `ethtool`.
7. TLS certificates that renew themselves: the certificate renewal audit (ARI, fraction-of-lifetime alerts, deploy hooks).

Related parts

- System monitoring — correlate with host load
- Security — host hardening beyond the firewall
- Services and runtime — network targets and sockets

Continue with the individual command pages in this part.

ip

Overview

`ip` (from **iproute2**) configures and displays interfaces, addresses, routes, neighbors, rules, and tunnels. It replaces legacy `ifconfig`/`route`/`arp` for modern Linux networking.

Syntax

```
ip [OPTIONS] OBJECT COMMAND
ip -h                    # help
ip OBJECT help           # object-specific help
```

Common objects: `link`, `address` (`addr`), `route`, `neigh`, `rule`, `netns`, `xfrm`.

Common Options

Option	Description
`-4` / `-6`	IPv4 / IPv6 only
`-s`	Statistics (can repeat for more detail)
`-d`	Extra detail
`-br`	Brief columnar output
`-c`	Color
`-o`	One-line output
`-j` / `-p`	JSON / pretty JSON (newer iproute2)

Key Use Cases

1. Inspect and set addresses
2. Bring links up/down, set MTU/MAC
3. Manage default and static routes

4. Debug neighbor (ARP/ND) tables

5. Work with network namespaces

Safety

Address and route changes can drop your SSH session. Prefer a **console/IPMI** session or a scripted rollback when changing the only default route. On Ubuntu servers, prefer **netplan** or **NetworkManager** for persistent config; use `ip` for live debug.

Examples with Explanations

Brief overview

```
ip -br link
ip -br addr
ip route
```

Fast "what interfaces, what IPs, what default route?".

Full address list

```
ip addr show
ip -4 addr show dev eth0
```

Bring link up/down

```
sudo ip link set dev eth0 up
sudo ip link set dev eth0 down
```

Add/remove an address

```
sudo ip addr add 192.168.1.50/24 dev eth0
sudo ip addr del 192.168.1.50/24 dev eth0
```

Prefix length is required. Secondary addresses are normal on multi-homed hosts.

Default route

```
sudo ip route add default via 192.168.1.1 dev eth0
sudo ip route replace default via 192.168.1.1 dev eth0
ip route get 1.1.1.1
```

`route get` shows which path the kernel would use.

Static route

```
sudo ip route add 10.0.0.0/8 via 192.168.1.254
sudo ip route del 10.0.0.0/8
```

Neighbor table (ARP)

```
ip neigh show
sudo ip neigh flush dev eth0
```

Statistics

```
ip -s link show eth0
ip -s -s link show eth0   # more error detail
```

Network namespace (quick)

```
sudo ip netns add lab
sudo ip netns exec lab ip link
sudo ip netns delete lab
```

Understanding Output

- `UP`/`LOWER_UP` on links: admin up vs carrier present
- `inet` / `inet6` lines: addresses and scopes (`global`, `link`)
- `proto kernel` vs `proto static` / `dhcp` on routes: origin of the route

Common Usage Patterns

Live IP on a server you did not configure

```
ip -br a; ip r; resolvectl status 2>/dev/null || cat /etc/resolv.conf
```

Flush all IPv4 addresses on a NIC (careful)

```
sudo ip -4 addr flush dev eth0
```

Notes & Pitfalls

- Changes with `ip` are **not durable** across reboot unless persisted by netplan/NM/scripts.
- Interface names may be predictable (`enp3s0`) not `eth0`.
- Policy routing uses `ip rule` + multiple tables — advanced; see `man ip-rule`.

Related Commands

- `nmcli` — NetworkManager CLI
- `resolvectl` — DNS via systemd-resolved
- `ss` — sockets
- `ping` / `traceroute` — path checks
- `ifconfig` — legacy; prefer `ip`

Additional Resources

- `man ip`, `man ip-link`, `man ip-route`
- iproute2 documentation

ss

Overview

`ss` (socket statistics) inspects TCP/UDP/UNIX sockets. It replaces most `netstat` workflows and is faster on busy systems because it talks to the kernel more efficiently. First stop for "what is listening?" and "who is connected?".

Syntax

```
ss [options] [filter]
```

Common Options

Option	Description
`-t` / `-u` / `-x`	TCP / UDP / UNIX
`-l`	Listening only
`-a`	All (listen + connected)
`-n`	Numeric hosts/ports (no DNS)
`-p`	Owning process (often needs root)
`-e`	Extended detail
`-m`	Socket memory info
`-s`	Summary counters
`-4` / `-6`	Address family
`-o`	Timer info (retransmit, keepalive)
`-K`	Kill socket (privileged; careful)
`-H`	Suppress header (newer ss)

Key Use Cases

1. See listeners and bound ports
2. Map sockets to PIDs/processes

3. Find connections to/from a host or port
4. Spot TIME-WAIT piles and backlog issues

Examples with Explanations

Listening TCP with processes

```
sudo ss -lntp
```

`-n` avoids reverse DNS delays; `-p` shows `users:(("nginx",pid=…))`.

Established connections

```
ss -tnp
ss -tnp state established
```

Filter by port

```
sudo ss -lntp '( sport = :22 )'
sudo ss -lntp '( sport = :8080 )'
ss -tnp '( dport = :443 or sport = :443 )'
```

Quote filters so the shell does not eat parentheses.

Filter by address

```
ss -tn dst 1.1.1.1
ss -tn src 10.0.0.0/8
```

UDP listeners

```
sudo ss -lunp
```

DNS, NTP, QUIC-related services often show here.

Summary and states

```
ss -s
ss -ant | awk 'NR>1 {c[$1]++} END {for (s in c) print c[s], s}' | sort -nr
```

High `TIME-WAIT` can be normal after load; runaway `SYN-RECV` may mean flood or backlog misconfig.

UNIX domain sockets

```
sudo ss -xlp | head
ss -x src /run/systemd/private
```

Local IPC for docker/podman/systemd often appears as UNIX sockets.

"What owns this port?"

```
sudo ss -lntp | grep -E ':8080\b'
# or filter expression:
sudo ss -lntp '( sport = :8080 )'
sudo lsof -iTCP:8080 -sTCP:LISTEN
```

IPv6 only

```
sudo ss -lntp -6
```

Understanding Output

Typical columns: state (`LISTEN`, `ESTAB`, `TIME-WAIT`, …), receive/send queues, local address:port, peer address:port, process. Queue columns matter when diagnosing overload (apps not `accept()`ing fast enough).

Notes & Pitfalls

- Without `sudo`, `-p` may omit other users' processes.
- `netstat` still appears in old docs; install `iproute2` (`ss`) as the default.
- Filters use a small language — see `man ss` "FILTER".
- For payload inspection use `tcpdump`/`wireshark`, not `ss`.

Related Commands

- `lsof -i` — open-file oriented view
- `ip` — addresses and routes
- `nmap` — external reachability scans
- `netstat` — legacy
- `fuser` — who uses a port/file

Additional Resources

- `man ss`

ping

Overview

`ping` sends ICMP Echo Request packets and reports whether a host answers with Echo Reply, plus round-trip time (RTT). It is the first tool for "is this address reachable?" and a quick signal for loss and latency. On Ubuntu, `ping` comes from `iputils-ping` and needs appropriate capabilities (usually installed setuid or with `cap_net_raw`) to open raw ICMP sockets.

Syntax

```
ping [OPTIONS] DESTINATION
ping -c COUNT [OPTIONS] DESTINATION
```

Common Options

Option	Description
`-c count`	Stop after `count` replies (script-friendly)
`-i interval`	Seconds between packets (default 1)
`-W timeout`	Seconds to wait for a reply
`-w deadline`	Exit after this many seconds regardless
`-s size`	Payload size in bytes
`-M do`	Path MTU discovery: prohibit fragmentation
`-4` / `-6`	Force IPv4 / IPv6
`-n`	Numeric output (skip reverse DNS)
`-q`	Quiet (summary only)
`-v`	Verbose
`-I device`	Bind to interface or source address
`-t ttl`	Set IP TTL

Safety

- Flooding (`-f`, if permitted) or very low intervals can look like abuse and may be rate-limited by networks. Prefer `-c` and sensible `-i` on production paths.

- “Ping works” “application works.” ICMP may be filtered while TCP 443 is fine, and the reverse can also happen.
- Do not rely on ping alone for SLOs; use service health checks.

Examples with Explanations

Basic connectivity (4 probes)

```
ping -c 4 1.1.1.1
ping -c 4 example.com
```

Numeric only (faster, no rDNS noise)

```
ping -n -c 4 8.8.8.8
```

IPv6

```
ping -6 -c 4 2606:4700:4700::1111
# or:
ping6 -c 4 example.com
```

Quiet summary for scripts

```
if ping -c 3 -W 2 -q gateway.local >/dev/null; then
  echo reachable
else
  echo unreachable
fi
```

Deadline: fail fast in automation

```
ping -c 5 -w 6 -q 203.0.113.1
echo exit:$?
```

`-w` caps total runtime so a black hole does not hang a script.

Larger payload (rough MTU / fragmentation check)

```
ping -c 3 -s 1472 -M do 1.1.1.1
```

1472 payload + 28 bytes IP/ICMP headers 1500-byte Ethernet frame. If this fails with "message too long" but small pings work, look at MTU/PMTUD issues (VPNs, tunnels).

Source from a specific interface

```
ping -c 4 -I eth0 10.0.0.1
ping -c 4 -I 192.0.2.10 10.0.0.1
```

Useful on multi-homed hosts and VPN boxes.

Faster sampling (still polite)

```
ping -c 20 -i 0.2 10.0.0.1
```

Requires privileges for intervals < 0.2s on many systems; stay reasonable on shared networks.

Compare loss to a hop vs destination

```
ping -c 50 -q 10.0.0.1   # gateway
ping -c 50 -q 1.1.1.1    # internet
```

High loss only to the far end suggests a path problem beyond the LAN.

Understanding Output

Typical line:

```
64 bytes from 1.1.1.1: icmp_seq=1 ttl=57 time=12.3 ms
```

- **icmp_seq** — sequence number; gaps imply loss (or reordering).
- **ttl** — remaining hop budget; lower often means more hops (not a strict hop count).
- **time** — RTT.

Summary:

```
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 11.2/12.5/14.1/1.1 ms
```

- **mdev** — mean deviation (jitter-ish signal).
- Exit status: `0` if enough replies received (see man page for exact rules with `-c`/`-w`), non-zero on total failure.

Notes & Pitfalls

- Many cloud security groups and enterprise firewalls **drop ICMP**. Unreachable ping does not prove the host is down — try `curl -I`, `nc -vz host 443`, or `mtr`.
- Name resolution failures look like ping failures; test with a raw IP too.
- Container and network-namespace environments may lack permission to ping; error messages about "Operation not permitted" point at capabilities, not routing.
- Do not confuse `ping` host reachability with DNS: always note whether you used a name or an IP.
- For continuous path quality, `mtr` or `smokeping`-style tools beat a single short ping.

Related Commands

- `mtr` / `mtr --report` — live traceroute + loss per hop
- `traceroute` / `tracepath` — path discovery
- `ss` / `ip route` — local sockets and routes
- `curl -I` / `nc -vz` — application-layer reachability
- `resolvectl query` / `dig` — name resolution checks

Additional Resources

- `man ping`
- `man ip-route` (when replies fail but the host seems local)

traceroute

Overview

`traceroute` maps the path packets take toward a destination by sending probes with increasing TTL and reading ICMP "Time Exceeded" messages from intermediate hops. Use it to find where latency jumps or where the path dies. On Ubuntu, install with `sudo apt install traceroute`. Related tools: `tracepath` (often installed by default), and `mtr` (continuous combined ping+trace).

Syntax

```
traceroute [options] host
traceroute -n -m 20 host
```

Common Options

Option	Description
`-n`	Numeric IPs only (no reverse DNS — faster, cleaner)
`-m max_ttl`	Max hops (default often 30)
`-q nqueries`	Probes per hop (default 3)
`-w wait`	Wait seconds per probe
`-I`	ICMP echo probes (may work when UDP is filtered)
`-T`	TCP SYN probes (often best through firewalls)
`-U`	UDP probes (classic default on many Linux builds)
`-p port`	Base port (UDP) or destination port (TCP with `-T`)
`-f first_ttl`	Start at this TTL
`-4` / `-6`	IPv4 / IPv6
`-i iface`	Outgoing interface
`-s src`	Source address

Safety

- Traceroute is generally benign but generates many probes; do not hammer production paths in tight loops.

- Some providers rate-limit or deprioritize traceroute-related ICMP; interpret stars (`* * *`) carefully.
- TCP traceroute to sensitive ports may look like a scan — prefer known service ports you own (e.g. 443) and authorized targets.

Examples with Explanations

Basic path

```
traceroute example.com
traceroute -n 1.1.1.1
```

ICMP mode (when UDP probes are blocked)

```
sudo traceroute -I -n example.com
```

May require root for raw ICMP.

TCP traceroute to HTTPS (firewall-friendly)

```
sudo traceroute -T -p 443 -n example.com
```

Probes look more like real client traffic to port 443; excellent for "UDP traceroute dies but the site loads" cases.

Limit hop count and speed up failure

```
traceroute -n -m 15 -w 2 -q 1 203.0.113.50
```

`-q 1` one probe per hop; `-w 2` shorter wait — noisier stats but faster.

IPv6 path

```
traceroute -6 -n example.com
```

Compare UDP vs TCP vs ICMP

```
traceroute -n example.com
sudo traceroute -I -n example.com
sudo traceroute -T -p 443 -n example.com
```

Different probe types can take different paths or be filtered differently — note which method matches user traffic.

Start mid-path (skip nearby hops)

```
traceroute -n -f 5 example.com
```

Use a specific source address (multi-homed)

```
sudo traceroute -n -s 192.0.2.10 example.com
```

Understanding Output

Classic lines:

```
 1  192.168.1.1  1.2 ms  1.1 ms  1.3 ms
 2  10.0.0.1     5.0 ms  4.8 ms  5.1 ms
 3  * * *
 4  203.0.113.1  20.2 ms  19.8 ms  20.5 ms
```

- Column 1 — hop number (TTL).
- Address — router that returned ICMP time exceeded (or final destination).
- Three times — RTTs for each probe (`-q`).
- `* * *` — no reply within timeout (filtering, loss, or rate-limit) — **not always a failure of the path**.

Notes on interpretation: - Latency often rises gradually; a sudden jump may mark a long-haul or congested link. - Asymmetric routing means the return path may differ; RTT includes both directions. - The last hop should be the destination answering the probe type; some hosts never respond to traceroute probes yet still serve TCP apps.

Notes & Pitfalls

- **Stars in the middle do not prove breakage** if later hops respond. Many routers simply do not generate TTL-exceeded for your probe type.

- Stars at the **end** with a working website often mean the destination ignores traceroute probes; test with `curl`/`nc -vz` too.
- Corporate firewalls may allow TCP/443 but drop UDP traceroute — use `-T -p 443`.
- Reverse DNS (`-n` off) slows output and can mislead if PTR records are wrong; prefer `-n` for ops work.
- `tracepath` is a lighter alternative that does not always need root and focuses on PMTU; `mtr --report` is better for ongoing loss per hop.
- ECMP / anycast can make successive runs show different hop addresses — capture multiple runs before concluding.

Related Commands

- `mtr` / `mtr --report` — continuous per-hop loss and latency
- `tracepath` — simple path / PMTU discovery
- `ping` — end-to-end reachability and RTT
- `ip route get ADDR` — which local route/next-hop would be used
- `dig` + `curl` — confirm name and application path independently

Additional Resources

- `man traceroute`
- `man mtr`
- `man tracepath`

mtr

Overview

`mtr` (My Traceroute) combines continuous **ping** and **traceroute**: it probes each hop on the path and updates loss and latency statistics in real time. Use it when intermittent packet loss or a "bad hop" is suspected—not just for a one-shot path dump.

```
# Debian/Ubuntu / RHEL-family (package name is usually mtr or mtr-tiny)
sudo apt install mtr-tiny   # or: mtr
sudo dnf install mtr
```

Syntax

```
mtr [options] HOSTNAME|IP
```

Common Options

Option	Description
`-r`, `--report`	Non-interactive report then exit
`-c N`, `--report-cycles N`	Probes per hop in report mode
`-n`, `--no-dns`	Skip reverse DNS (faster, clearer)
`-w`, `--report-wide`	Wide hostnames in reports
`-4` / `-6`	Force IPv4 or IPv6
`-T`, `--tcp`	TCP SYN probes instead of ICMP
`-u`, `--udp`	UDP probes
`-P PORT`, `--port PORT`	Target port for TCP/UDP
`-i SECS`	Interval between probes
`-m MAX`	Max hops (TTL)
`-b`	Show both hostname and IP
`-z`	Show AS numbers (if available)
`-s SIZE`	Packet size
`-e`	Show MPLS labels when present

Key Use Cases

1. Locate which hop introduces latency or loss
2. Generate shareable path reports for NOC tickets
3. Test path to a specific TCP port (HTTPS 443, etc.)
4. Compare IPv4 vs IPv6 paths

Examples with Explanations

Interactive session

```
mtr example.com
mtr -n example.com
```

Interactive UI refreshes hop stats until you quit (`q`). `-n` avoids slow or broken PTR lookups.

Non-interactive report (scripts & tickets)

```
mtr --report -c 20 -n example.com
mtr -rwc 50 example.com
```

`--report` runs N cycles and prints a table—ideal for pasting into chat or logs.

TCP path to HTTPS

```
mtr -T -P 443 -n example.com
mtr -T -P 22 -n bastion.example.com
```

ICMP is often rate-limited or deprioritized; TCP probes better match real application traffic when firewalls allow them.

IPv6 and dual-stack

```
mtr -6 -n example.com
mtr -4 -n example.com
```

Different address families can take completely different transit.

One-liner recipes

```
# Quick 10-cycle report to clipboard-friendly text
mtr -rwc 10 -n 1.1.1.1

# Compare two destinations
for h in 1.1.1.1 8.8.8.8; do echo "=== $h ==="; mtr -rwc 15 -n "$h"; done

# Watch path while changing VPN/Wi-Fi (interactive)
mtr -n vpn-endpoint.example.com
```

Interpreting loss (important)

- **Loss only on intermediate hops** but 0% at the destination often means the hop rate-limits ICMP replies—not necessarily real path loss.
- **Loss that grows and stays high through later hops including destination** usually indicates a real problem on or before that hop.
- Spike latency on one hop that recovers downstream can be ICMP deprioritization.

Understanding Output

Typical columns:

Field	Meaning
Host	Hop address / name
Loss%	Probe loss to that hop
Snt	Probes sent
Last / Avg / Best / Wrst	RTT samples (ms)
StDev	Jitter-ish spread

Notes & Pitfalls

- **Root/capabilities**: some probe types need elevated privileges; distro packages may install a setcap binary.
- **Firewalls**: corporate networks may drop traceroute-style probes; try `-T -P 443`.
- **Containers / cloud**: path may terminate at hypervisor or LB; don't expect full Internet-style hops inside VPC only.
- Prefer **`mtr` over pure `traceroute` + `ping`** for intermittent issues.

2026-relevant notes

- Cloud anycast and CGNAT make hop lists noisier; combine with `curl -w '%{time_connect}\n'`, `ss`, and provider path tools.
- For continuous SLOs, export `mtr --report` from a cron/systemd timer rather than relying on ad-hoc interactive sessions.
- `mtr-tiny` on Debian omits some extras but is fine for servers.

Comparison to alternatives

Tool	Role
`mtr`	Continuous hop stats
`traceroute` / `tracepath`	One-shot path
`ping`	Single-host RTT/loss
`npiping` / `smokeping`	Long-term historical

Related Commands

- `ping` — single-host reachability
- `traceroute` / `tracepath` — path discovery
- `ss` — local sockets
- `dig` / `host` — name resolution before tracing

Additional Resources

- `man mtr`
- Path diagnosis guides from major cloud and CDN providers

dig

Overview

`dig` (Domain Information Groper) queries DNS servers and prints detailed answers. It is the preferred CLI for debugging resolution on Ubuntu/Debian (package `dnsutils` / `bind9-dnsutils`). Use it to inspect A/AAAA/MX/TXT/NS/CNAME records, compare resolvers, and validate changes after DNS edits.

Syntax

```
dig [@server] [name] [type] [options]
dig [options] name type
```

Default type is `A`. If `@server` is omitted, dig uses the system resolver configuration (often `systemd-resolved` stub on Ubuntu).

Common Options

Option	Description
`@server`	Query this nameserver (IP or hostname)
`-t type` / positional `type`	Record type: `A`, `AAAA`, `MX`, `TXT`, `NS`, `CNAME`, `SOA`, `ANY` (limited), …
`+short`	Minimal answer only
`+noall +answer`	Show answer section only (structured)
`+trace`	Iterative resolve from root hints
`+dnssec`	Request DNSSEC records
`+tcp`	Use TCP instead of UDP
`+time=T`	Timeout seconds
`+tries=N`	Number of tries
`-p port`	Nameserver port (default 53)
`-x addr`	Reverse lookup (PTR) convenience
`-4` / `-6`	Transport IPv4 / IPv6 only
`+subnet=…`	EDNS client subnet (advanced)

Examples with Explanations

Basic A record

```
dig example.com
dig example.com A
```

Short output for scripts

```
dig +short example.com
dig +short example.com AAAA
```

Query a specific resolver

```
dig @1.1.1.1 example.com
dig @8.8.8.8 example.com AAAA
dig @127.0.0.53 example.com   # systemd-resolved stub on many Ubuntu hosts
```

Comparing `@1.1.1.1` vs system resolver isolates "DNS server cache/policy" vs "local stub" issues.

MX and TXT

```
dig example.com MX +short
dig example.com TXT +short
```

NS and SOA (delegation / zone authority)

```
dig example.com NS +short
dig example.com SOA
```

Reverse DNS (PTR)

```
dig -x 1.1.1.1 +short
# equivalent idea:
dig 1.1.1.1.in-addr.arpa PTR +short
```

Answer section only (readable)

```
dig +noall +answer example.com A
dig +noall +answer www.example.com
```

Trace from the root (find where delegation breaks)

```
dig +trace example.com
```

Shows root → TLD → authoritative path. Heavy but excellent when "some resolvers see X, others Y."

Follow CNAME manually

```
dig www.example.com +short
dig +noall +answer www.example.com
```

If you get a CNAME, dig again on the target name for the final A/AAAA.

TCP query (firewall / large answers)

```
dig +tcp example.com TXT
```

Non-standard port (lab DNS)

```
dig @127.0.0.1 -p 5353 test.lab A
```

Batch mode

```
dig -f names.txt +short
```

`names.txt` lists one name per line.

Check mail-related records quickly

```
dig example.com MX +short
dig example.com TXT +short | grep -i spf
dig _dmarc.example.com TXT +short
```

Understanding Output

Default output sections: - **HEADER** — opcode, status (`NOERROR`, `NXDOMAIN`, `SERVFAIL`, `REFUSED`), flags (`aa` authoritative, `rd` recursion desired, `ra` recursion available). - **QUESTION** — what you asked. - **ANSWER** — records for the name. - **AUTHORITY** — NS/SOA when relevant. - **ADDITIONAL** — glue / extras. - **Query time / SERVER / WHEN / MSG SIZE** — which server answered and how fast.

Status guide: | Status | Meaning | |———|———| | `NOERROR` | Query ok (answer may still be empty for that type) | | `NXDOMAIN` | Name does not exist | | `SERVFAIL` | Resolver failure (upstream, DNSSEC, timeout) | | `REFUSED` | Server refused the query |

Notes & Pitfalls

- Ubuntu's `127.0.0.53` is often a **stub** — it forwards to real resolvers. Debugging "wrong answer" may need `@` the upstream (`resolvectl status`) or a public resolver.
- `ANY` is deprecated/restricted on many public resolvers; query specific types instead.
- TTL in answers is **remaining** TTL from that resolver's cache, not necessarily the zone's configured TTL.
- Split-horizon DNS returns different answers by client network; VPN on/off can change results.
- `dig` success does not mean your application uses the same resolver path — check `resolvectl`, `/etc/nsswitch.conf`, and app-specific DNS settings.
- For quick human checks, `host` is shorter; for operators, `dig` is the right default.

Related Commands

- `host` — shorter DNS lookups
- `nslookup` — interactive/legacy style lookups
- `resolvectl query` — systemd-resolved view on Ubuntu
- `doggo` / `drill` — alternative dig-like tools if installed
- `whois` — registration data (not live DNS answers)

Additional Resources

- `man dig`
- `man resolvectl` (Ubuntu resolver integration)

host

Overview

`host` is a simple DNS lookup utility from BIND tools. It is faster to type than `dig` for everyday A/AAAA/MX checks. Use `dig` (or `drill`) when you need full message sections, DNSSEC detail, or uncommon query flags; use `resolvectl query` when you care what *this machine's* stub resolver returns.

Syntax

```
host [options] name [server]
host [options] IP
```

Common Options

Option	Description
`-t type`	Record type: `A`, `AAAA`, `MX`, `TXT`, `NS`, `CNAME`, `SOA`, `SRV`, …
`-a`	Equivalent to `-v -t ANY` (ANY often blocked)
`-v`	Verbose
`-W secs`	Wait timeout
`-R n`	Number of retries
`-C`	Query SOA records for zone name server check style
`-4` / `-6`	Force IPv4/IPv6 transport to the nameserver
`-r`	Non-recursive query

Key Use Cases

1. Quick "does this name resolve?"
2. MX / TXT / NS checks without dig verbosity
3. Reverse lookup of an IP
4. Point queries at a specific nameserver

Examples with Explanations

Forward lookup

```
host example.com
host -t A example.com
host -t AAAA example.com
```

Default output lists addresses and sometimes other associated data depending on version and response.

Mail and text records

```
host -t MX example.com
host -t TXT example.com
host -t NS example.com
host -t SOA example.com
```

Reverse DNS

```
host 1.1.1.1
host 8.8.8.8
```

PTR for the reverse zone; empty/no PTR is common for ephemeral cloud IPs.

Query a specific server

```
host example.com 1.1.1.1
host example.com 8.8.8.8
host example.com 127.0.0.53
```

Compare public DNS vs local systemd-resolved stub.

Verbose and script-friendly

```
host -v example.com
if host -W 2 example.com >/dev/null 2>&1; then echo ok; else echo fail; fi
```

One-liner recipes

```
# Extract first IPv4-looking answer (fragile but handy)
host -t A example.com | awk '/has address/ {print $4; exit}'

# MX hosts only
host -t MX example.com | awk '{print $NF}'

# Spot split-horizon DNS
diff <(host example.com 1.1.1.1) <(host example.com 127.0.0.53) || true
```

Notes & Pitfalls

- **ANY** (`-a`) is widely restricted on public resolvers—query explicit types.
- Output format is **not as stable as dig + short** for parsing; prefer `dig +short` or `doggo --json` in automation.
- **Search domains** from resolv.conf can make short names resolve differently than FQDNs.
- `host` may not be installed on minimal images (`bind-utils` / `dnsutils` / `bind9-host` packages).

2026-relevant notes

- Prefer `dig +short`, `doggo`, or `resolvectl query` for modern workflows; keep `host` for muscle memory and quick checks.
- Encrypted DNS is usually configured under resolved/NetworkManager—not via `host` flags. To test DoH, use `doggo` or a DoH-aware client.
- `drill` (ldns) is a good dig-like alternative on some distros.

Comparison to alternatives

Tool	When to use
`host`	Short interactive answers
`dig`	Full protocol control
`doggo`	UX + DoH/DoT
`nslookup`	Legacy interactive only
`getent hosts`	NSS (files + DNS + others)

Related Commands

- `dig` — detailed DNS
- `nslookup` — legacy
- `resolvectl` — systemd-resolved
- `doggo` — modern CLI
- `getent hosts` — NSS

Additional Resources

- `man host`
- BIND documentation for the host utility

nslookup

Overview

`nslookup` queries DNS name servers interactively or non-interactively. It remains common in legacy docs and Windows cross-training, but on modern Linux **`dig`**, **`host`**, **`doggo`**, and **`resolvectl query`** are usually better choices for clarity and scripting.

Syntax

```
nslookup [options] [name | -] [server]
```

Common Options / Directives

Option / interactive	Description
`-type=a` / `type=a`	A record
`-type=aaaa`	AAAA
`-type=mx`	MX
`-type=ns`	NS
`-type=soa`	SOA
`-type=txt`	TXT
`-type=ptr`	PTR
`-type=cname`	CNAME
`-debug`	Debug
`-port=N`	Server port
`-timeout=N`	Timeout
`server ADDR`	Interactive: change server
`set type=mx`	Interactive type

Key Use Cases

1. Quick interactive DNS checks
2. Following older runbooks
3. Comparing answers from a specific server

Examples with Explanations

Non-interactive

```
nslookup example.com
nslookup -type=mx example.com
nslookup -type=ns example.com
nslookup example.com 1.1.1.1
nslookup 1.1.1.1
```

Interactive

```
nslookup
> server 8.8.8.8
> set type=txt
> example.com
> exit
```

Prefer modern equivalents

```
dig +short example.com
dig example.com MX
host -t MX example.com
resolvectl query example.com
doggo example.com MX
```

One-liner recipes

```
# Script-friendly? Prefer dig:
dig +short example.com @1.1.1.1

# nslookup parsing is brittle; avoid in automation
```

Notes & Pitfalls

- Output format varies; **don't parse nslookup in scripts**.
- Interactive defaults can confuse (search lists, types).
- Package: `dnsutils` / `bind-utils` / `bind9-dnsutils`.
- ANY queries may fail on public resolvers.

2026-relevant notes

Goal	Prefer
Detail	`dig`
Short human	`host` / `doggo`
Host stub	`resolvectl`
DoH/DoT	`doggo`
Legacy interactive	`nslookup`

Related Commands

- `dig` — preferred detailed client
- `host` — simple lookups
- `doggo` — modern UX
- `resolvectl` — systemd-resolved

Additional Resources

- `man nslookup`

doggo

Overview

`doggo` is a modern command-line **DNS client** with colorful, human-friendly output. It is an optional alternative to `dig`/`host`/`nslookup` for interactive queries. Install separately (Go binary / distro package where available). Keep `dig` for portable scripts and ubiquitous server availability.

Syntax

```
doggo [query options] NAME [TYPE]
doggo [NAME] @SERVER
```

Exact flags evolve — always check `doggo --help` for your version.

Common Options (typical)

Option	Description
`NAME`	Domain name to query
`TYPE`	Record type (`A`, `AAAA`, `MX`, `TXT`, `NS`, …)
`@server`	Nameserver to query
`-n` / `--nameserver`	Nameserver
`--strategy`	e.g. random/round-robin among servers
`--time`	Show query time
`--json`	JSON output (if supported)
`--short`	Short answers
`-4` / `-6`	Transport family
`--tls` / `--https` / `--quic`	DoT/DoH/DoQ style transports when supported

Examples with Explanations

Basic queries

```
doggo example.com
doggo example.com A
doggo example.com AAAA
doggo example.com MX
doggo example.com TXT
```

Choose resolver

```
doggo example.com @1.1.1.1
doggo example.com @8.8.8.8
doggo example.com @127.0.0.53          # systemd-resolved stub
```

Reverse / other types

```
doggo 1.1.1.1 PTR
doggo example.com NS
doggo example.com SOA
doggo _http._tcp.example.com SRV
```

Compare with dig

```
dig +short example.com A
doggo example.com A
resolvectl query example.com
```

Scripting

```
# prefer dig for stable scripting
dig +short example.com A
# doggo if JSON mode available for humans/tools
doggo example.com --json 2>/dev/null | jq .
```

Encrypted DNS (when built-in)

```
doggo example.com --tls @1.1.1.1
doggo example.com --https @https://cloudflare-dns.com/dns-query
```

Feature support depends on build.

Notes / Pitfalls

- Not preinstalled on servers — document `dig` equivalents in runbooks.
- Pretty colors can break pipes — use short/JSON/plain modes.
- System stub resolvers (`127.0.0.53`) may differ from public resolvers — test both when debugging.
- DNSSEC/validation visibility differs by tool; confirm with `dig +dnssec` when needed.
- DoH/DoT may be blocked on corporate networks.

2026-relevant notes

- Encrypted DNS and modern transports are the interesting differentiator vs classic `dig`.
- For fleet automation, stick to `dig`/`resolvectl`/`getent hosts`.
- Pair with `mtr`/`curl -v` when debugging full user-visible path issues.

Related Commands

- `dig` — standard Swiss-army DNS tool
- `host` / `nslookup` — simpler classic clients
- `resolvectl` — systemd-resolved
- `getent hosts` — NSS lookup path
- `drill` — another dig-like client (ldns)

Additional Resources

- `doggo --help`
- project docs (mr-karan/doggo)

resolvectl

Overview

`resolvectl` manages and queries **systemd-resolved**: DNS servers, search/route domains, LLMNR/mDNS, DNSSEC/DoT status, and the DNS cache. It replaces the older `systemd-resolve` command name on current systems. Use it to answer "what DNS is *this host* actually using?"—which can differ from public `dig @8.8.8.8` results.

Syntax

```
resolvectl [options] command [argument...]
```

Common Commands

Command	Description
`status`	Global + per-link resolver status
`query NAME`	Resolve name via resolved
`query -t TYPE NAME`	Specific record type
`flush-caches`	Drop DNS cache
`reset-server-features`	Reset learned server features
`dns [LINK [SERVER…]]`	Show/set DNS servers for a link
`domain [LINK [DOMAIN…]]`	Search/routing domains
`default-route [LINK BOOL]`	Whether link is default DNS route
`statistics`	Cache statistics
`log-level [LEVEL]`	Debug resolved
`monitor`	Stream resolution events (if available)

Key Use Cases

1. Debug DNS on NetworkManager / netplan / systemd-networkd hosts
2. See split-DNS (VPN vs LAN) per interface
3. Flush stale cache after record changes
4. Confirm stub listener `127.0.0.53`

Examples with Explanations

Status and query

```
resolvectl status
resolvectl query example.com
resolvectl query -t MX example.com
resolvectl query -t AAAA example.com
```

Cache

```
sudo resolvectl flush-caches
resolvectl statistics
```

Per-link DNS

```
resolvectl dns
resolvectl dns eth0
resolvectl domain
ip -br link    # correlate interface names
```

One-liner recipes

```
# Compare stub vs public
resolvectl query example.com
dig +short example.com @1.1.1.1

# After VPN connect
resolvectl status | sed -n '1,80p'
resolvectl flush-caches

# Show current DNS from resolv.conf angle
cat /etc/resolv.conf
# often points to 127.0.0.53 when resolved is active
```

Notes & Pitfalls

- If **resolved is disabled**, `resolvectl` fails—use `dig` against real nameservers in `/etc/resolv.conf`.
- Setting DNS with `resolvectl dns LINK …` may be **transient**; persistent config belongs in netplan, NetworkManager, or `.network` files.

- Ubuntu netplan frequently uses resolved's stub; editing `/etc/resolv.conf` by hand is the wrong long-term fix when it's a symlink.
- LLMNR/mDNS noise on home LANs can surprise; status shows whether enabled.

2026-relevant notes

- **DNS-over-TLS** may be configured via resolved (`DNSOverTLS=`)—check status for "DNS-over-TLS: yes/no".
- Prefer `resolvectl` + `nmcli` over legacy `nm-tool`.
- Containers may not run resolved; DNS is often injected by the runtime (`/etc/resolv.conf` with cluster DNS).

Related Commands

- `dig` / `host` / `doggo` — query arbitrary servers
- `nmcli` — NetworkManager DNS
- `networkctl` — systemd-networkd
- `systemd-resolve` — old name (compat)

Additional Resources

- `man resolvectl`, `man systemd-resolved.service`
- freedesktop systemd resolved docs

curl

Overview

`curl` transfers data to/from URLs. It supports HTTP(S), FTP, SFTP, and many other protocols — the default tool for APIs, downloads, health checks, and quick TLS debugging. Prefer `curl` in scripts for predictable flags; use `wget` when you want recursive site mirrors.

Syntax

```
curl [options] [URL...]
```

Common Options

Option	Description
`-O`	Save using remote file name
`-o file`	Save to file
`-L`	Follow redirects
`-f`	Fail on HTTP 400 (exit non-zero)
`-sS`	Silent progress, but show errors
`-v` / `-vv`	Verbose (headers, TLS)
`-I`	HEAD request only
`-X METHOD`	Custom method (`POST`, `PUT`, …)
`-H "K: V"`	Request header
`-d data`	Body (form-encoded by default)
`--json data`	JSON body + content-type (curl 7.82+)
`-u user:pass`	HTTP auth
`-A agent`	User-Agent
`-m secs` / `--max-time`	Overall time limit
`--connect-timeout secs`	Connect phase only
`-w fmt`	Write-out variables after transfer
`-k`	Insecure TLS (lab only)
`-C -`	Resume transfer
`-x proxy`	HTTP/SOCKS proxy
`-b` / `-c`	Cookie read / write jar
`--retry N`	Retry transient failures

Key Use Cases

1. Download artifacts and releases
2. Call REST APIs
3. Debug HTTP headers and TLS
4. Scripted health checks and deploys

Safety

- Prefer env vars / netrc over secrets on the command line (`ps` can see argv).
- `-k` disables certificate verification — never in production automation.
- Recursive fetchers and huge mirrors belong to `wget`/`rclone` with bandwidth limits.

Examples with Explanations

Download

```
curl -fL -O https://example.com/file.tgz
curl -fL -o app.tgz https://example.com/file.tgz
```

`-f` fails on HTTP errors; `-L` follows redirects common on CDN links.

API GET with pretty JSON

```
curl -sS https://api.github.com/repos/jqlang/jq | jq .
```

`-sS` keeps scripts clean while still reporting failures.

POST JSON

```
curl -sS -X POST https://httpbin.org/post \
  -H 'Content-Type: application/json' \
  --data-binary '{"name":"ada","ok":true}' | jq .

# newer curl:
curl -sS --json '{"name":"ada"}' https://httpbin.org/post | jq .
```

Headers only / method checks

```
curl -sSI https://example.com
curl -sS -o /dev/null -w '%{http_code}\n' https://example.com
```

Follow redirects, treat 404 as failure

```
curl -fsSL -o out.html https://example.com/missing || echo "failed:$?"
```

`-f` makes 4xx/5xx non-zero — essential in CI.

Timing metrics

```
curl -o /dev/null -sS -w \
  'dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttr
  https://example.com
```

Useful for "is it DNS, TCP, TLS, or the app?"

Auth

```
curl -sS -H "Authorization: Bearer $TOKEN" https://api.example.com/v1/me
curl -sS -u "$USER:$PASS" https://example.com/private/
curl -sS --netrc-file ~/.netrc https://example.com/private/
```

Upload file (multipart)

```
curl -fL -F "file=@./report.pdf;type=application/pdf" https://httpbin.org/post
```

Client certificate (mTLS)

```
curl -sS --cert client.pem --key client.key --cacert ca.pem https://svc.internal/health
```

Proxy and retries

```
curl -fsSL --retry 3 --retry-delay 2 -x http://proxy:8080 -O https://example.com/file
```

Show response headers + body separately

```
curl -sS -D headers.txt -o body.json https://api.example.com/v1/x
```

Understanding Output

Default is the response body on stdout and a progress meter on stderr. HTTP status does **not** affect exit code unless `-f` (or you inspect `%{http_code}`). Write-out variables expand after the transfer completes.

Notes & Pitfalls

- Quote URLs containing `&`, `?`, or shell globs.
- HTTP/2 and HTTP/3 behavior depends on how curl was built.
- Multiple URLs on one command line are sequential.
- `curl | jq` fails closed if either side fails when `set -o pipefail` is on.

Related Commands

- `wget` — download-centric alternative
- `httpie` / `xh` — friendlier human HTTP CLIs
- `jq` — JSON post-processing
- `openssl s_client` — deep TLS debug
- `ss` / `dig` — connectivity / DNS triage

Additional Resources

- `man curl`
- Everything curl

wget

Overview

`wget` is a non-interactive downloader for HTTP(S), FTP, and related protocols. It shines at recursive site fetches, resume, rate limiting, and simple "get this URL into a file" scripts. For complex APIs and arbitrary methods, `curl` is usually better; for bulk downloads and mirroring, `wget` is often clearer.

Syntax

```
wget [OPTIONS] [URL...]
wget [OPTIONS] -i url-list.txt
```

Common Options

Option	Description
`-O file`	Write to this filename (`-` = stdout)
`-o logfile`	Write messages to log file
`-c`	Continue / resume partial download
`-q`	Quiet
`-v` / `-nv`	Verbose / non-verbose
`-b`	Go to background after start
`-P dir`	Directory prefix for saved files
`-N`	Timestamping (re-get only if newer)
`--limit-rate=RATE`	Throttle (e.g. `500k`)
`--timeout=SECS`	Network timeout
`--tries=N`	Retry count (`0` = infinite)
`--retry-connrefused`	Retry even on connection refused
`-U agent`	User-Agent string
`--header='K: V'`	Extra HTTP header
`--user=` / `--password=`	Auth (prefer netrc; see Safety)
`-r`	Recursive download
`-l N`	Recursion depth
`-np`	No parent (stay under URL path)
`-k`	Convert links for local viewing

Option	Description
`-p`	Page requisites (CSS/images for HTML)
`-m`	Mirror shorthand (`-r -N -l inf --no-remove-listing`)
`-A` / `-R`	Accept / reject filename patterns
`--spider`	Check existence without downloading
`--no-check-certificate`	Skip TLS verify (**avoid**)

Safety

- Do not put passwords on the command line (`ps` can see them). Prefer `~/.wgetrc` permissions `600` or `.netrc` with tight modes, or prompt-based tools.
- Recursive `-r` can pull far more than intended (entire sites). Use `-np`, `-l`, and `-A`/`-R` limits.
- `--no-check-certificate` disables TLS verification — only for lab debugging of broken certs, never production automation.
- Respect site `robots.txt` and terms; `-e robots=off` exists but is often rude or disallowed.

Examples with Explanations

Simple download (keeps remote name)

```
wget https://example.com/files/app-1.2.3.tgz
```

Save under a specific name

```
wget -O app.tgz https://example.com/files/app-latest.tgz
```

Resume interrupted download

```
wget -c https://example.com/big.iso
```

Quiet download into a directory

```
mkdir -p ~/downloads
wget -q -P ~/downloads https://example.com/tool.deb
```

Rate-limited fetch with retries

```
wget --limit-rate=1m --tries=5 --timeout=30 \
  https://example.com/dataset.tar.gz
```

Multiple URLs from a file

```
wget -i urls.txt -P ./artifacts/
```

One URL per line in `urls.txt`.

Spider: check if URL is reachable

```
wget --spider -q https://example.com/health && echo up || echo down
```

Authenticated download (prefer netrc)

```
# ~/.netrc (chmod 600):
# machine example.com login USER password SECRET
wget https://example.com/private/build.zip
```

Custom header (e.g. token) without logging token in shell history carefully

```
wget --header="Authorization: Bearer ${TOKEN}" \
  -O release.json https://api.example.com/v1/release
```

Still prefer env vars over pasting secrets into shell history.

One-page offline copy (HTML + assets)

```
wget -p -k -P ./mirror https://example.com/docs/intro.html
```

`-p` gets page requisites; `-k` rewrites links for local browsing.

Bounded recursive mirror of a docs tree

```
wget -r -np -nH -l 3 -P ./docs \
  https://example.com/manual/
```

- `-np` — do not ascend to parent
- `-nH` — no host-prefixed directory
- `-l 3` — depth limit

Timestamped re-sync of a single artifact

```
wget -N https://example.com/app.tgz
```

Skips download if local file is already up to date (server must send proper timestamps).

Understanding Output

Default progress shows percent, bar, speed, and ETA. `-q` silences progress (errors still appear unless fully quieted). HTTP error codes produce non-zero exit status in modern wget — check `$?` in scripts. Partial files remain when interrupted unless you clean them; `-c` reuses them.

Notes & Pitfalls

- `-O` with multiple URLs concatenates into one file — usually wrong; use `-P` or separate invocations.
- Recursive mode creates directory trees; know where `-P` points before filling the disk.
- Some CDNs need a browser-like User-Agent (`-U`) or extra headers; prefer fixing the URL/API over spoofing when you control the service.
- For REST APIs with POST/JSON, use `curl`; `wget` can POST (`--post-data`) but is less ergonomic.
- On Ubuntu, package is typically `wget` from `apt`; GNU wget 1.x is the common default (`wget --version`).

Related Commands

- `curl` — flexible transfers and APIs
- `aria2c` — multi-connection downloads (if installed)
- `apt` / `apt-get` — packages, not arbitrary URLs
- `rsync` — sync trees you already host over SSH
- `sha256sum` — verify downloads after fetch

Additional Resources

- `man wget`
- `/usr/share/doc/wget/` on Debian/Ubuntu packages

ssh

Overview

`ssh` (OpenSSH client) creates encrypted sessions to a remote host for shell access, command execution, port forwarding, and tunneling. Pair with `scp`/`sftp`/`rsync` for files.

Syntax

```
ssh [options] [user@]host [command]
ssh -J jump user@host
```

Common Options

Option	Description
`-i key`	Private key identity file
`-p port`	Remote port (default 22)
`-L` / `-R` / `-D`	Local / remote / dynamic port forwards
`-N`	No remote command (forward only)
`-f`	Background after auth
`-A`	Agent forwarding (use sparingly)
`-J host`	ProxyJump
`-o Opt=Val`	Config override
`-v` / `-vv`	Debug auth/connection
`-t`	Force TTY
`-C`	Compression

Key Use Cases

1. Remote administration
2. Run one-shot remote commands
3. Port forwards to private services

4. Jump-host (bastion) workflows

Safety

- Prefer keys over passwords; disable agent forwarding except when required.
- Verify host keys on first connect; investigate changed key warnings (possible MITM).
- Do not forward agent (`-A`) to untrusted hosts.

Examples with Explanations

Login

```
ssh alice@server.example.com
ssh -i ~/.ssh/id_ed25519 alice@server.example.com
```

Remote command

```
ssh alice@server 'hostname; uptime'
```

Custom port / options

```
ssh -p 2222 -o ConnectTimeout=5 alice@server
```

ProxyJump via bastion

```
ssh -J bastion.example.com alice@internal.example.com
# config equivalent: ProxyJump bastion.example.com
```

Local port forward (remote DB to local)

```
ssh -N -L 5432:127.0.0.1:5432 alice@server
# then connect local clients to localhost:5432
```

Dynamic SOCKS proxy

```
ssh -N -D 1080 alice@server
```

Config file (~/.ssh/config)

```
Host prod
  HostName 203.0.113.10
  User deploy
  IdentityFile ~/.ssh/prod_ed25519
  ForwardAgent no
```

```
ssh prod
```

Verbose auth debug

```
ssh -vv alice@server
```

Common Usage Patterns

Copy key (once)

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@server
```

Escape sequence (client)

`Enter ~ .` disconnects a wedged session; `Enter ~ ?` lists escapes.

Notes & Pitfalls

- `user@host` defaults user to local username if omitted.
- Global known hosts: `~/.ssh/known_hosts`.
- Multiplexing: `ControlMaster` / `ControlPath` speeds repeated connections.

Related Commands

- `scp` / `sftp` — file copy
- `rsync -e ssh` — efficient sync
- `ssh-keygen` — key pairs
- `sshd` / `sshd_config` — server side

Additional Resources

- `man ssh`, `man ssh_config`

scp

Overview

`scp` (secure copy) copies files to or from a remote host over SSH. It is simple and available almost everywhere, but it is **not** a sync tool: every run re-transfers whole files. For large trees, resumes, and mirrors, prefer `rsync` over SSH. On modern OpenSSH, `scp` uses the SFTP protocol under the hood by default.

Syntax

```
scp [OPTIONS] SOURCE... DEST
# SOURCE or DEST can be: local path  or  [user@]host:[path]
```

Common Options

Option	Description
`-r`	Recursive (directories)
`-P port`	SSH port (**capital** `-P`; not `-p`)
`-i identity`	Private key file
`-C`	Compression
`-p`	Preserve mtime and modes
`-q`	Quiet (less progress noise)
`-v`	Verbose (debug SSH/auth)
`-l limit`	Bandwidth limit in **kbit/s**
`-o Opt=Val`	Pass SSH option (e.g. `ProxyJump`)
`-3`	Copy between two remotes via local host

Safety

- `scp` overwrites destination files without an interactive "are you sure?" for normal paths — double-check remote paths.
- Do not embed passwords in scripts; use keys and `ssh-agent`.
- Recursively copying into the wrong remote directory can fill disks; test with a single file first.

- Prefer explicit paths (`user@host:/var/tmp/file`) over relying on remote home defaults when automating.

Examples with Explanations

Local → remote file

```
scp ./report.pdf alice@server.example.com:~/uploads/
```

Remote → local

```
scp alice@server:/var/log/nginx/access.log ./access.log
```

Directory tree

```
scp -r ./site alice@server:/var/www/
```

Copies the `site` directory as `/var/www/site` (unless you structure paths carefully).

Custom port and key

```
scp -P 2222 -i ~/.ssh/deploy_ed25519 \
  ./app.tgz deploy@server:/opt/releases/
```

Remember: **`-P`** for port on `scp`; **`ssh`/`rsync` use `-p`**.

Preserve times/modes

```
scp -p config.yml alice@server:/etc/myapp/config.yml
```

Via jump host (bastion)

```
scp -o ProxyJump=bastion.example.com \
  ./secret.env alice@internal:/tmp/
```

Or with OpenSSH `ProxyJump` in `~/.ssh/config` under a `Host` alias, then:

```
scp ./secret.env internal:/tmp/
```

Bandwidth-limited copy

```
scp -l 8000 large.iso alice@server:/data/
```

`8000` 8 Mbit/s (~1 MB/s). Units are kilobits per second.

Multiple sources to a remote directory

```
scp a.conf b.conf c.conf alice@server:/etc/myapp/
```

Remote-to-remote through your laptop

```
scp -3 alice@host-a:/data/file bob@host-b:/data/file
```

Useful when hosts cannot talk to each other directly.

Verbose debug when auth fails

```
scp -v ./file alice@server:~/
```

Understanding Output

By default `scp` shows a progress meter per file (name, percent, speed, ETA). `-q` suppresses most of that. Failures print to stderr; non-zero exit means the copy did not fully succeed — check partial remote files.

Notes & Pitfalls

- **Port flag case:** `scp -P 2222` vs `ssh -p 2222`. Mixing them is a common ops mistake.
- Spaces in paths need quoting on both local shell and careful remote escaping:

  ```
  scp './My File.txt' 'alice@server:./My File.txt'
  ```

- `scp` is poor at resume: interrupt a multi-GB transfer and you usually start over. Use `rsync -P` instead.
- Recursive `scp` does not delete extraneous remote files (not a mirror).
- Older docs mention `scp`'s legacy RCP-based protocol; current Ubuntu OpenSSH prefers SFTP mode. If a weird old device fails, check `man scp` for `-O` (legacy mode) on your version.

- Globs expand **locally** unless quoted for the remote shell. `scp server:*.log .` may not do what you want; often better: `ssh server 'tar czf - /var/log/*.log' | tar xzf -`.

Related Commands

- `rsync -e ssh` — delta sync, dry-run, delete, resume-friendly
- `sftp` — interactive SSH file transfer
- `ssh` — remote shell / one-shot commands
- `install` / `install -D` — local install with mode control
- `tar` + `ssh` — stream many files efficiently

Additional Resources

- `man scp`
- `man ssh_config` (ProxyJump, IdentityFile, Host aliases)

rsync

Overview

`rsync` synchronizes files and directories locally or over a remote shell (almost always SSH). It copies only changed blocks when possible, can preserve permissions/ownership/times, and is the standard tool for backups, deploys, and large tree moves on Ubuntu servers and workstations.

Trailing slashes matter: `src/` copies the *contents* of `src` into the destination; `src` (no slash) copies the directory itself as a child of the destination.

Syntax

```
rsync [OPTIONS] SOURCE... DEST
rsync [OPTIONS] SOURCE... [USER@]HOST:DEST
rsync [OPTIONS] [USER@]HOST:SOURCE DEST
```

Common Options

Option	Description
`-a`	Archive mode: recursive + preserve perms, times, symlinks, devices (`-rlptgoD`)
`-v`	Verbose (list transferred files)
`-z`	Compress data in flight (helps over slow links; less useful on LAN/SSD)
`-P`	`--partial --progress` — keep partial files and show progress
`-n` / `--dry-run`	Show what would happen; transfer nothing
`-e ssh`	Use SSH as remote shell (often default nowadays)
`--delete`	Delete files on dest that are not on source (mirror)
`--exclude=PAT`	Skip matching paths
`--include=PAT`	Include pattern (order with exclude matters)
`-x`	Don't cross filesystem boundaries
`-H`	Preserve hard links
`-A`	Preserve ACLs
`-X`	Preserve extended attributes
`--bwlimit=RATE`	Throttle (e.g. `10m` = 10 MB/s)

Option	Description
`--info=progress2`	Overall progress bar (GNU rsync)
`--checksum` / `-c`	Compare by checksum, not size+mtime
`-u`	Skip files newer on destination

Safety

- **`--delete` can remove large parts of the destination.** Always run with `-n` first and read the file list.
- Confirm trailing `/` intent before a mirror. `rsync -a --delete src/ dest/` is not the same as `src dest/`.
- Remote paths use `host:path` or `host:/abs/path`. A missing colon can make rsync create a local directory named like the remote host.
- Prefer SSH keys and a dedicated deploy user; avoid running rsync as root over the network unless you need ownership preservation on system trees.

Examples with Explanations

Dry-run first (always for delete or production)

```
rsync -anv --delete \
  --exclude '.git' \
  ./site/ user@server:/var/www/site/
# review the list, then drop -n:
rsync -av --delete \
  --exclude '.git' \
  ./site/ user@server:/var/www/site/
```

Local archive backup with progress

```
rsync -aHAX --info=progress2 /home/alice/ /mnt/backup/alice/
```

`-aHAX` is a solid "full metadata" local copy on Linux when ACLs/xattrs matter (e.g. home dirs, containers).

Push project over SSH

```
rsync -azP -e 'ssh -p 2222' \
  ./build/ deploy@app.example.com:/srv/app/current/
```

`-z` compresses; `-P` helps on long transfers that may interrupt.

Pull logs from a server

```
rsync -azP user@server:/var/log/nginx/ ./nginx-logs/
```

Mirror with excludes

```
rsync -a --delete \
  --exclude 'node_modules' \
  --exclude '.cache' \
  --exclude '*.tmp' \
  ./project/ /backup/project/
```

Throttled backup (shared link)

```
rsync -azP --bwlimit=5m /data/ user@offsite:/backups/data/
```

One filesystem only (skip other mounts)

```
rsync -ax / /mnt/root-backup/
```

Useful when `/` has bind mounts or extra disks you do not want pulled into the backup.

Checksum mode when clocks or touch times lie

```
rsync -avc ./release/ user@server:/opt/app/
```

Slower; compares content hashes instead of relying on size and mtime.

Show only what would change (itemize)

```
rsync -ani --delete ./src/ ./dest/
```

`-i` itemizes reasons for each transfer (permission change, size, etc.).

Understanding Output

With `-v`, each transferred path is printed. `--info=progress2` shows aggregate bytes and rate. Dry-run (`-n`) still lists actions but does not write. Exit status `0` means success; non-zero usually means partial failure (permissions, vanished files, network).

Notes & Pitfalls

- **Trailing slash is the #1 footgun.** Practice with `-n` until muscle memory is solid.
- Archive mode does **not** include hard links (`-H`), ACLs (`-A`), or xattrs (`-X`) by default — add them when needed.
- `--delete` does not delete excluded files on the destination unless you also use options like `--delete-excluded` (be even more careful).
- Sparse files, open databases, and live VMs need application-consistent backups; rsync alone is not a snapshot tool.
- For very large trees, run from `tmux`/`screen` so a dropped SSH session does not kill the job; combine with `-P` to resume.

Related Commands

- `scp` — simple one-shot copy over SSH (no delta sync)
- `sftp` — interactive SSH file transfer
- `cp -a` — local recursive copy without remote/delta features
- `tar` over `ssh` — stream archives when rsync is unavailable
- `rclone` — cloud object storage sync (if installed)

Additional Resources

- `man rsync`
- `man rsyncd.conf` (daemon mode, less common for SSH workflows)

nc

Overview

`nc` (netcat) is a Swiss-army TCP/UDP tool: open connections, listen for inbound sockets, scan ports simply, and pipe data across the network. Ubuntu typically ships **OpenBSD netcat** as `nc` / `netcat` (`openbsd-netcat` package family; classic `netcat-traditional` may also be available). Flag details differ slightly between implementations — check `nc -h` on your host.

Syntax

```
nc [options] host port
nc -l [options] port
command | nc host port
nc host port < file
```

Common Options (OpenBSD nc on Ubuntu)

Option	Description
`-l`	Listen mode
`-p port`	Local port (with `-l`, often just `nc -l port`)
`-u`	UDP
`-v`	Verbose
`-z`	Scan mode: no data, just connect
`-w secs`	Connect/idle timeout
`-n`	Numeric only (no DNS)
`-k`	Keep listening after client disconnect (where supported)
`-s addr`	Source address
`-q secs`	Quit after EOF on stdin after delay
`-4` / `-6`	IPv4 / IPv6 only

Safety

- Listening services with `nc -l` bind whatever you tell them — do not expose on public interfaces unless intended; combine with firewall rules.

- "Shell over netcat" patterns are classic attack/persistence techniques. Use only on systems you own, with authorization, for legitimate recovery labs.
- `-z` port scans against machines you do not operate may violate policy.

Examples with Explanations

Test if a TCP port is open

```
nc -vz example.com 443
nc -vz -w 3 10.0.0.10 22
```

Verbose zero-I/O connect is the operator's quick "is something listening?" check.

HTTP GET by hand

```
printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' \
  | nc example.com 80
```

Useful when debugging virtual hosts or seeing raw headers without curl.

TLS is not built into classic nc

```
# Prefer:
openssl s_client -connect example.com:443 -servername example.com
# or curl -v https://...
```

Plain `nc` speaks TCP bytes; it does not negotiate TLS for you.

Simple file transfer (lab / trusted LAN)

Receiver:

```
nc -l 9000 > received.tgz
```

Sender:

```
nc -w 3 receiver.example.com 9000 < payload.tgz
```

Prefer `scp`/`rsync` for anything real (auth, encryption, integrity).

Listen and show inbound banner/data

```
nc -lvk 9000
```

Connect from another host: `nc host 9000` and type lines.

UDP probe

```
nc -u -vz -w 2 10.0.0.10 53
```

UDP "success" is weaker than TCP — lack of ICMP unreachable may still print as open/filtered depending on version.

Port range scan (light)

```
nc -vz 10.0.0.10 20-25
```

For serious inventory use `nmap`; nc is fine for a handful of ports.

Pipe program output to a remote listener

```
# listener:
nc -l 7000 | tar xzf -
# sender:
tar czf - ./project | nc -w 3 host 7000
```

Bind to a specific local address

```
nc -l -s 127.0.0.1 8080
```

Localhost-only listeners reduce accidental exposure.

Chatty timeout for scripts

```
if nc -z -w 2 db.internal 5432; then
  echo db-port-open
else
  echo db-port-closed
fi
```

Understanding Output

- Connect success (OpenBSD nc): often `Connection to host port [tcp/*] succeeded!` with `-v`/`-z`.
- Failure: connection refused (nothing listening / RST), timed out (filtered or blackholed), name resolution errors.
- In connected mode, stdin goes to the socket and socket data goes to stdout — making nc a network pipe.

Notes & Pitfalls

- **Implementation flags differ.** Scripts written for `netcat-traditional` (`-e` exec) will not match OpenBSD nc. On Ubuntu, `update-alternatives` or package choice can change behavior — verify with `nc -h` and `readlink -f $(command -v nc)`.
- Exit codes for `-z` are useful in bash `if nc -z ...; then` but confirm on your variant.
- Listening without `-k` usually exits after one client (OpenBSD). Plan for that in demos.
- IPv6 addresses need careful syntax; prefer `-6` and explicit addresses when debugging dual-stack.
- For HTTP(S) health checks, `curl -fsS -o /dev/null -w '%{http_code}\n'` is usually better than raw nc.

Related Commands

- `curl` / `wget` — HTTP(S) clients
- `openssl s_client` — TLS debugging
- `ss -tlnp` — local listeners
- `nmap` — structured port scanning
- `socat` — more powerful relay/socket toolkit (if installed)
- `telnet` — interactive TCP (legacy; nc often replaces it)

Additional Resources

- `man nc`
- `man ncat` (Nmap's netcat, if installed as separate tool)

tcpdump

Overview

`tcpdump` captures and displays packets on a network interface. It is the classic CLI packet sniffer for verifying traffic, debugging firewalls, and confirming "did this SYN leave the box?" On Ubuntu, install with `sudo apt install tcpdump`. Captures usually need root or `cap_net_raw` / `cap_net_admin`.

Syntax

```
sudo tcpdump [OPTIONS] [expression]
sudo tcpdump -i IFACE -nn -s0 -w file.pcap 'filter'
```

Common Options

Option	Description
`-i iface`	Interface (`any` for all, where supported)
`-n`	Don't resolve hostnames
`-nn`	Don't resolve hostnames or ports
`-v` / `-vv`	More packet detail
`-c N`	Exit after N packets
`-s snaplen`	Capture snap length (`0` or `65535` = full packet)
`-w file`	Write raw packets to pcap file
`-r file`	Read packets from pcap file
`-A`	ASCII payload
`-X` / `-XX`	Hex + ASCII
`-q`	Quieter output
`-e`	Link-level header (MAC)
`-ttt`	Delta time between packets
`-Z user`	Drop privileges to user after opening capture
`--immediate-mode`	Lower latency display (when supported)

Safety

- Packet captures can include **credentials, cookies, and PII**. Restrict pcap file permissions (`chmod 600`) and delete when done.
- Capturing on production can add load; prefer tight BPF filters and `-c` limits.
- Legal/policy: only capture networks you are authorized to monitor.
- Do not publish pcaps from customer environments.

Filter expressions (BPF) essentials

Filters use libpcap syntax (same family as Wireshark capture filters):

Expression	Matches
`host 1.2.3.4`	Traffic to/from host
`net 10.0.0.0/8`	Network
`port 443`	TCP or UDP port
`tcp port 22`	SSH
`src` / `dst`	Direction qualifier
`icmp`	ICMP
`tcp[tcpflags] & tcp-syn != 0`	SYN packets
`and` / `or` / `not`	Boolean

Examples with Explanations

See anything on default route interface

```
ip route show default
sudo tcpdump -i eth0 -nn -c 20
```

HTTPS-related traffic for one host

```
sudo tcpdump -i any -nn 'host 203.0.113.10 and port 443'
```

DNS queries and responses

```
sudo tcpdump -i any -nn port 53
```

Only SYN packets (connection attempts)

```
sudo tcpdump -i any -nn 'tcp[tcpflags] & (tcp-syn) != 0 and tcp[tcpflags] & (tcp-ack) == 0'
```

Capture full packets to file for Wireshark

```
sudo tcpdump -i eth0 -nn -s0 -w /tmp/trace.pcap 'port 5432'
# later, as yourself:
sudo chown "$USER":"$USER" /tmp/trace.pcap
chmod 600 /tmp/trace.pcap
wireshark /tmp/trace.pcap   # or tcpdump -r
```

Read a pcap back in the terminal

```
tcpdump -nn -r /tmp/trace.pcap | less
tcpdump -nn -A -r /tmp/trace.pcap 'port 80'
```

HTTP cleartext headers (lab only)

```
sudo tcpdump -i any -nn -A 'tcp port 80 and (((ip[2:2] - ((ip[0]&0xf)<<2)) - ((tcp[12]&0xf0)
```

Prefer `-w` + Wireshark for serious HTTP analysis. TLS on 443 will not show HTTP headers in cleartext.

ICMP (ping) traffic

```
sudo tcpdump -i any -nn icmp
```

Traffic involving a local port (e.g. reverse proxy)

```
sudo tcpdump -i any -nn 'port 8080'
```

Limit and timestamp deltas

```
sudo tcpdump -i eth0 -nn -c 50 -ttt host 10.0.0.5
```

Exclude SSH so your session does not flood the capture

```
sudo tcpdump -i eth0 -nn "not port 22"
```

When capturing over SSH, your interactive session traffic will otherwise dominate the output.

Understanding Output

Example TCP line:

```
12:01:02.123456 IP 10.0.0.5.54321 > 10.0.0.10.443: Flags [S], seq 123, win 64240, options [.
```

- Addresses are `ip.port` with `-nn`.
- **Flags**: `S` SYN, `F` FIN, `P` PSH, `R` RST, `.` ACK only.
- **length** — payload length (0 for pure handshake packets).

If you see SYNs with no SYN-ACK, the peer or a middlebox is dropping or blackholing the connection. If SYN-ACK returns but app fails, debug above TCP (TLS, HTTP, app).

Notes & Pitfalls

- Without `-s0`/`-s 65535`, snaplen may truncate payloads (headers usually still visible).
- Interface `any` is convenient but may not show link-layer MACs the same way as a real NIC; use a specific iface for L2 issues.
- Offloading/TSO can make on-wire sizes look odd in captures; remember NIC offload when diagnosing MSS/MTU.
- Filters are **capture** filters (BPF), not Wireshark display filters. Syntax differs from Wireshark GUI search.
- Containers: capture in the correct network namespace (`ip netns exec`, or from the host veth) or you will see nothing useful.
- Prefer writing to pcap (`-w`) for anything beyond a quick glance; live `-A` scrolls too fast and loses context.

Related Commands

- `wireshark` / `tshark` — GUI / CLI deeper analysis
- `ss -tnp` — which processes own sockets
- `iptables` / `nft` / `ufw` — firewall rules that drop traffic
- `nmap` — active port discovery (different from capture)
- `curl -v` / `openssl s_client` — application-level TLS/HTTP debug

Additional Resources

- `man tcpdump`
- `man pcap-filter` — full BPF expression reference

nmap

Command Overview

The `nmap` (Network Mapper) is a powerful open-source tool for network exploration, security scanning, and auditing. It can discover hosts and services on a network, detect operating systems, and identify potential vulnerabilities.

Syntax

```
nmap [Scan Type] [Options] {target specification}
```

Common Options

Option	Description
`-sS`	TCP SYN scan (default)
`-sT`	TCP connect scan
`-sU`	UDP scan
`-sN`	TCP NULL scan
`-sF`	TCP FIN scan
`-sX`	TCP XMAS scan
`-sA`	TCP ACK scan
`-sW`	TCP Window scan
`-sM`	TCP Maimon scan
`-sn`	Ping scan (disable port scan)
`-Pn`	Skip host discovery
`-p`	Port specification
`-F`	Fast scan (100 ports)
`-r`	Scan ports consecutively
`-T<0-5>`	Timing template
`-sV`	Version detection
`-O`	OS detection
`-A`	Enable OS detection, version detection, script scanning, and traceroute
`-oN`	Output normal format
`-oX`	Output XML format

Option	Description
`-oG`	Output grepable format
`-v`	Increase verbosity
`-d`	Increase debugging
`--script`	NSE script selection
`--script-args`	NSE script arguments

Key Use Cases

1. Network discovery
2. Port scanning
3. Service version detection
4. Operating system detection
5. Vulnerability assessment
6. Security auditing
7. Network inventory
8. Performance analysis

Examples with Explanations

1. Basic Scan

```
$ nmap 192.168.1.0/24
Starting Nmap 7.94 ( https://nmap.org )
Nmap scan report for 192.168.1.1
Host is up (0.0020s latency).
Not shown: 995 closed ports
PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
443/tcp open  https
```

Scan entire subnet for open ports

2. Intensive Scan

```
$ nmap -A -T4 example.com
```

Aggressive scan with OS and version detection

3. Stealth Scan

```
$ sudo nmap -sS -p- example.com
```

SYN scan of all ports

4. Service Version Detection

```
$ nmap -sV -p 22,80,443 example.com
```

Detect service versions on specific ports

5. OS Detection

```
$ sudo nmap -O example.com
```

Identify operating system

Understanding Nmap

Scan Types

```
# TCP SYN Scan (Stealth)
$ sudo nmap -sS target

# TCP Connect Scan
$ nmap -sT target

# UDP Scan
$ sudo nmap -sU target

# SCTP INIT Scan
$ sudo nmap -sY target

# FIN Scan
$ sudo nmap -sF target
```

Port Selection

```
# Specific ports
$ nmap -p 80,443 target

# Port ranges
$ nmap -p 1-1000 target

# All ports
$ nmap -p- target

# Top ports
$ nmap --top-ports 100 target

# Fast scan
$ nmap -F target
```

Host Discovery

```
# Ping scan only
$ nmap -sn 192.168.1.0/24

# Skip ping
$ nmap -Pn target

# TCP SYN ping
$ nmap -PS22,80,443 target

# TCP ACK ping
$ nmap -PA22,80,443 target

# UDP ping
$ nmap -PU53 target
```

Version Detection

```
# Light version detection
$ nmap -sV --version-intensity 5 target

# Aggressive version detection
$ nmap -sV --version-all target

# With script scanning
$ nmap -sV -sC target
```

Script Scanning

```
# Default scripts
$ nmap -sC target

# Specific script
$ nmap --script=http-title target

# Script category
$ nmap --script=vuln target

# Multiple scripts
$ nmap --script=http-*,ssl-* target

# Script with arguments
$ nmap --script http-brute --script-args http-brute.path=/login target
```

Output Formats

```
# Normal output
$ nmap -oN scan.txt target

# XML output
$ nmap -oX scan.xml target

# Grepable output
$ nmap -oG scan.grep target

# All formats
$ nmap -oA scan target
```

Performance Tuning

```
# Timing templates
$ nmap -T0 target  # Paranoid
$ nmap -T1 target  # Sneaky
$ nmap -T2 target  # Polite
$ nmap -T3 target  # Normal
$ nmap -T4 target  # Aggressive
$ nmap -T5 target  # Insane

# Custom timing
$ nmap --min-rate 100 --max-rate 500 target
```

Advanced Techniques

```
# Fragmented packets
$ sudo nmap -f target

# Custom MTU
$ sudo nmap --mtu 24 target

# Decoy scan
$ sudo nmap -D decoy1,decoy2,ME target

# Idle scan
$ sudo nmap -sI zombie_host target

# Source port manipulation
$ sudo nmap --source-port 53 target
```

Firewall Evasion

```
# Fragment packets
$ sudo nmap -f target

# Use decoy
$ sudo nmap -D RND:10 target

# Spoof MAC
$ sudo nmap --spoof-mac Dell target

# Data length
$ sudo nmap --data-length 25 target
```

NSE Scripts Examples

```
# SSL/TLS scanning
$ nmap --script ssl-enum-ciphers -p 443 target

# Vulnerability scanning
$ nmap --script vuln target

# Brute force
$ nmap --script brute target

# Default credential check
```

```
$ nmap --script http-default-accounts target

# DNS enumeration
$ nmap --script dns-brute target
```

Best Practices

```
# Network inventory
$ nmap -sn -oX inventory.xml 192.168.1.0/24

# Security audit
$ sudo nmap -A -v -oA audit target

# Regular monitoring
$ nmap -sS -sV --open -oG monitor target

# Vulnerability assessment
$ nmap --script vuln -sV -p- target
```

Related Commands

- `netcat` - Network utility
- `tcpdump` - Packet analyzer
- `wireshark` - Network protocol analyzer
- `hping3` - Network tool
- `masscan` - Mass IP port scanner

Additional Resources

- Man page: `man nmap`
- Nmap reference guide
- NSE script documentation
- Network scanning guide
- Security best practices
- Port scanning techniques

openssl

Overview

`openssl` is a Swiss-army toolkit for TLS/SSL, X.509 certificates, keys, digests, and simple encrypted files. Operators use it to inspect certificates, test TLS handshakes to a host:port, generate CSRs/self-signed certs for labs, and convert PEM/DER formats. It is not a full ACME client — prefer `certbot`/`acme.sh` for Let's Encrypt production issuance. Deep crypto design and application embedding belong elsewhere; this page is **operator triage**.

Syntax

```
openssl <subcommand> [options]
openssl help
openssl help s_client
```

Common subcommands: `s_client`, `x509`, `req`, `rsa`, `ec`, `pkey`, `dgst`, `rand`, `enc`, `version`.

Common Subcommands & Options

Area	Command sketch	Role
TLS probe	`openssl s_client -connect host:443`	Live handshake + cert chain
Show cert	`openssl x509 -in f.pem -noout -text`	Decode PEM certificate
CSR	`openssl req -new -…`	Certificate signing request
Digest	`openssl dgst -sha256 file`	Hash file
Random	`openssl rand -hex 16`	Random bytes
Connect SNI	`s_client -servername name -connect …`	Virtual-host TLS

s_client (frequent flags)

Option	Description
`-connect host:port`	Target

Option	Description
`-servername name`	SNI (required for most vhosts)
`-showcerts`	Print full chain
`-starttls smtp|imap|…`	STARTTLS protocols
`-alpn h2,http/1.1`	ALPN offer
`-tls1_2` / `-tls1_3`	Pin protocol (version-dependent)
`-status`	OCSP stapling request

x509 (frequent flags)

Option	Description
`-in file` / `-out file`	Input/output
`-noout`	Don't emit encoded cert
`-text`	Full decode
`-subject -issuer -dates`	Short fields
`-fingerprint -sha256`	Fingerprint
`-inform PEM|DER`	Encoding

Safety

- Never paste **private keys** into tickets/chat. Mode `600`, dedicated paths.
- Self-signed lab certs are fine for experiments; browsers/clients will warn — don't disable verification in production clients casually.
- `openssl enc` password-based file encryption is easy to misuse (KDF/parameters); prefer age/GPG for human file encryption workflows unless you know the flags.
- Old TLS versions may be disabled in modern OpenSSL builds — failures can mean policy, not "host down".

Examples with Explanations

Version and providers

```
openssl version -a
```

Confirms library version (3.x on current Ubuntu LTS).

Probe a website's certificate

```
echo | openssl s_client -servername example.com -connect example.com:443 2>/dev/null | opens
```

`echo |` completes the session so `s_client` exits. Always set **SNI** (`-servername`) for shared hosting.

Full chain and leaf details

```
echo | openssl s_client -showcerts -servername example.com -connect example.com:443 2>/dev/n
echo | openssl s_client -servername example.com -connect example.com:443 2>/dev/null \
  | openssl x509 -noout -text | less
```

SHA-256 fingerprint

```
echo | openssl s_client -servername example.com -connect example.com:443 2>/dev/null \
  | openssl x509 -noout -fingerprint -sha256
openssl x509 -in /etc/ssl/certs/ssl-cert-snakeoil.pem -noout -fingerprint -sha256
```

Inspect a local PEM cert

```
openssl x509 -in fullchain.pem -noout -subject -issuer -dates
openssl x509 -in fullchain.pem -noout -ext subjectAltName
```

STARTTLS (mail example)

```
openssl s_client -starttls smtp -connect mail.example.com:587 -servername mail.example.com
```

Generate lab key + self-signed cert

```
openssl req -x509 -newkey rsa:2048 -keyout lab.key -out lab.crt -days 365 -nodes \
  -subj '/CN=lab.local'
chmod 600 lab.key
```

For anything public-facing, use a real CA (ACME). `-nodes` leaves the key unencrypted on disk — protect the file.

CSR for an external CA

```
openssl req -new -newkey rsa:2048 -nodes -keyout app.key -out app.csr \
  -subj '/CN=app.example.com'
openssl req -in app.csr -noout -text | head
```

PEM DER

```
openssl x509 -in cert.pem -outform DER -out cert.der
openssl x509 -inform DER -in cert.der -out cert.pem
```

Digests (alternate to *sum tools)

```
openssl dgst -sha256 image.iso
openssl dgst -sha512 file.bin
```

Random secrets

```
openssl rand -base64 32
openssl rand -hex 16
```

Quick HTTPS port check (banner/handshake only)

```
echo | timeout 5 openssl s_client -connect 203.0.113.10:443 -servername app.example.com
```

Useful when `curl` fails and you need to see whether TLS negotiates at all.

Notes

- Ubuntu packages: `openssl` CLI; libraries via `libssl*`.
- Certificate stores: system CAs under `/etc/ssl/certs` (managed by `ca-certificates`).
- `s_client` is a debugger, not a browser — it may accept certs your app stack rejects depending on flags/defaults.
- OpenSSL 3 moved some algorithms to the "legacy" provider; rare old ciphers need extra config.
- Prefer `curl -vI --tlsv1.2` for HTTP-level checks; use `openssl s_client` for raw TLS/cert focus.

Related Commands

- `curl` — HTTPS client with cert options
- `certbot` / ACME clients — issuance/renewal
- `update-ca-certificates` — trust store
- `sha256sum` — simple file digests
- `nft` / `ufw` — not TLS; path/firewall side
- `dig` / `host` — DNS before TLS troubleshooting

Additional Resources

- `man openssl`, `man openssl-s_client`, `man openssl-x509`
- OpenSSL documentation

nmcli

Overview

`nmcli` is the CLI for **NetworkManager**. On many Ubuntu desktops/servers with NM enabled it is the supported way to manage connections, devices, and Wi-Fi—prefer connection profiles over ephemeral `ifconfig/ip addr add` for persistent config.

Syntax

```
nmcli [OPTIONS] OBJECT {COMMAND | help}
```

Objects include: `general`, `networking`, `device`, `connection`, `radio`, `monitor`, …

Common Commands

Command	Description
`general status`	NM state
`device status`	Interfaces + state
`device wifi list`	Scan SSIDs
`connection show`	Profiles
`connection up/down NAME`	Activate/deactivate
`connection add …`	Create profile
`connection modify …`	Change settings
`connection reload`	Reload from disk
`networking off/on`	Global NM networking

Useful global flags: `-t` terse, `-f` fields, `-g` values only, `-p` pretty.

Key Use Cases

1. Inspect live NM state without a GUI
2. Create Ethernet/Wi-Fi profiles
3. Bring connections up/down safely
4. Script NM on headless laptops/servers

Examples with Explanations

Overview

```
nmcli general status
nmcli device status
nmcli connection show
nmcli -f NAME,UUID,TYPE,DEVICE,STATE connection show --active
```

Bounce a connection

```
nmcli connection down "Wired connection 1"
nmcli connection up "Wired connection 1"
```

Static Ethernet profile

```
nmcli connection add type ethernet ifname eth0 con-name office-static \
  ipv4.method manual \
  ipv4.addresses 192.168.1.50/24 \
  ipv4.gateway 192.168.1.1 \
  ipv4.dns "1.1.1.1,8.8.8.8"
nmcli connection up office-static
```

DHCP Ethernet

```
nmcli connection add type ethernet ifname eth0 con-name office-dhcp ipv4.method auto
```

Wi-Fi

```
nmcli device wifi list
nmcli device wifi connect "SSID" password "secret"
nmcli connection modify SSID wifi-sec.psk "new-secret"
```

Reload and DNS insight

```
nmcli connection reload
nmcli connection up <name>
nmcli dev show eth0 | rg 'IP4|IP6|DNS'
resolvectl status
```

One-liner recipes

```
# Airplane-mode-ish
nmcli radio wifi off
nmcli radio wifi on

# Script-friendly active names
nmcli -t -f NAME c show --active

# Delete a profile
nmcli connection delete office-static
```

Notes & Pitfalls

- Minimal cloud images may use **systemd-networkd + netplan** without NM—`nmcli` won't exist.
- Ephemeral `ip addr add` can fight NM; put durable config in profiles or netplan.
- Wi-Fi secrets end up in profile files—protect permissions.
- VPN plugins vary; check `nmcli connection show` types available.

2026-relevant notes

- Prefer **netplan** YAML on Ubuntu servers when that is the distro's source of truth—it may render NM or networkd backends.
- DoT/DNS settings increasingly appear in NM/resolved integration—verify with `resolvectl`.
- `ifconfig` is not the management API for NM systems.

Related Commands

- `ip` — kernel addresses/routes
- `resolvectl` — systemd-resolved
- `journalctl -u NetworkManager` — logs
- `nmtui` — curses UI
- `networkctl` — networkd

Additional Resources

- `man nmcli`
- NetworkManager documentation

ufw

Overview

`ufw` (Uncomplicated Firewall) is Ubuntu's friendly frontend to netfilter. It suits host firewalls on workstations and small servers when you want readable allow/deny rules without writing raw `iptables`/`nft` by hand. Under the hood current Ubuntu generations use nftables via ufw—you still manage policy with `ufw` commands.

Syntax

```
ufw [--dry-run] [commands]
```

Common Commands

Command	Description
`enable` / `disable`	Start/stop firewall
`status [verbose|numbered]`	Show rules
`allow` / `deny` / `reject`	Add rules
`limit`	Rate-limit (anti brute-force)
`delete`	Remove by rule or number
`insert`	Insert at position
`reload`	Reload
`reset`	Wipe rules (**careful**)
`logging on|off|low|medium|high`	Logging
`app list` / `app info`	Application profiles

Key Use Cases

1. Lock down a VPS
2. Allow SSH before enabling
3. Open HTTP/HTTPS
4. Limit SSH brute-force noise

Safety

Remote sessions: always allow admin access **before** `ufw enable`. Prefer `--dry-run`. Know console/IPMI/serial recovery. Cloud security groups are a second layer outside the guest.

Examples with Explanations

Safe first enable

```
sudo ufw allow OpenSSH
# or: sudo ufw allow 22/tcp
sudo ufw --dry-run enable
sudo ufw enable
sudo ufw status verbose
```

Web and custom ports

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 8080/tcp comment 'app'
```

From management network only

```
sudo ufw allow from 10.0.0.0/8 to any port 22 proto tcp
sudo ufw allow from 203.0.113.10 to any port 5432 proto tcp
```

App profiles

```
sudo ufw app list
sudo ufw allow 'Nginx Full'
sudo ufw app info 'OpenSSH'
```

Numbered delete / limit

```
sudo ufw status numbered
sudo ufw delete 3
sudo ufw limit 22/tcp
```

One-liner recipes

```
# Default policies (review before enable)
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Show everything
sudo ufw status verbose

# Disable quickly
sudo ufw disable
```

Notes & Pitfalls

- Default after enable: usually **deny incoming**, allow outgoing.
- **Docker** and some CNI setups publish ports in ways that bypass or confuse UFW—verify with `ss -lntp` and cloud SG.
- IPv6 rules: ensure `IPV6=yes` in `/etc/default/ufw` when dual-stack.
- `ufw reset` clears policy—have out-of-band access.

2026-relevant notes

- Prefer **nftables**-aware mental model; `iptables-save` may show compatibility layers.
- On multi-host production, network policy often lives in cloud SG + service mesh; UFW is still fine per-VM hardening.
- RHEL-family often uses `firewalld` instead—different tool, same need to allow SSH first.

Related Commands

- `iptables` / `nft` — lower level
- `ss -lntp` — listeners before opening ports
- `journalctl` — firewall-related logs
- `fail2ban` — complementary brute-force mitigation

Additional Resources

- `man ufw`, `/usr/share/doc/ufw/`
- Ubuntu server guide firewall chapter

nft

Overview

`nft` is the userspace CLI for **nftables**, the modern Linux packet-filtering framework that replaces legacy `iptables` backend plumbing on current Ubuntu releases. Use it when you need structured rulesets, sets/maps, or to inspect what `ufw/iptables-nft` installed. For simple host firewalls on Ubuntu, **`ufw` remains the friendly default**; use `nft` for visibility, advanced policy, and routers/firewalls you own end-to-end.

Syntax

```
nft [options] commands
nft -f ruleset.nft
nft list ruleset
```

Common Commands

Command	Description
`list ruleset`	Dump whole configuration
`list tables`	Tables only
`list table inet filter`	One table
`list chain inet filter input`	One chain
`add table inet filter`	Create table (family `ip`/`ip6`/`inet`/`netdev`/…)
`add chain …`	Create chain (with hooks for base chains)
`add rule …`	Append rule
`insert rule …`	Insert at position
`delete rule … handle N`	Delete by handle
`flush ruleset`	**Wipe all** nft rules
`flush table …` / `flush chain …`	Narrow flush
`-a` / `-n`	Show handles / numeric
`-j`	JSON output (tooling)
`-c`	Check mode (with `-f`)
`-f file`	Load from file

Family **`inet`** covers IPv4+IPv6 in one table — preferred for new dual-stack host filters.

Safety

Remote lockout risk is identical to iptables. Always ensure SSH/admin access is allowed **before** dropping input. Prefer:

1. Console/IPMI/serial access when experimenting.
2. `nft -c -f ruleset.nft` to syntax-check.
3. Incremental changes; know how to `flush` or reboot recovery.
4. On Ubuntu desktops/servers already using **ufw**, don't fight it with ad-hoc `nft` rules unless you understand the interaction — inspect first with `nft list ruleset`.

`nft flush ruleset` clears **everything** nft owns — including rules installed by other tools.

Examples with Explanations

See what is active

```
sudo nft list ruleset
sudo nft list tables
sudo nft -a list chain inet filter input   # handles for deletes (names vary)
```

On UFW-enabled hosts you will typically see ufw-related tables/chains.

Minimal inet filter sketch (lab / understanding)

```
sudo nft add table inet filter
sudo nft add chain inet filter input  '{ type filter hook input priority 0; policy accept; }
sudo nft add chain inet filter forward '{ type filter hook forward priority 0; policy drop;
sudo nft add chain inet filter output '{ type filter hook output priority 0; policy accept;

sudo nft add rule inet filter input iif lo accept
sudo nft add rule inet filter input ct state established,related accept
sudo nft add rule inet filter input tcp dport 22 accept
# only after SSH works from a second session:
# sudo nft chain inet filter input '{ policy drop; }'
```

Treat as a **teaching sketch** — production policy needs ICMP, rate limits, logging, and persistence.

Sets for many ports/IPs

```
sudo nft add set inet filter admin_v4 '{ type ipv4_addr; }'
sudo nft add element inet filter admin_v4 '{ 203.0.113.10, 198.51.100.25 }'
sudo nft add rule inet filter input ip saddr @admin_v4 tcp dport 22 accept
```

Sets keep rulesets readable when allow-lists grow.

Delete a rule by handle

```
sudo nft -a list chain inet filter input
sudo nft delete rule inet filter input handle 12
```

Handles change when rules are reloaded — re-list before deleting.

Save and restore

```
sudo nft list ruleset > backup.nft
# edit carefully...
sudo nft -c -f backup.nft          # check
sudo nft -f backup.nft             # apply (does not auto-flush unless file says so)
```

To replace entirely, many operators `flush ruleset` inside the file or before load — **know what you wipe**.

Persist on Ubuntu (high level)

```
# Example approach: package nftables + enable service
sudo apt install nftables
# put rules in /etc/nftables.conf then:
sudo nft -c -f /etc/nftables.conf
sudo systemctl enable --now nftables
```

If **ufw** is enabled, prefer configuring ufw rather than parallel conflicting `nftables.service` policies.

Coexistence checks

```
sudo ufw status verbose
sudo nft list ruleset | head -100
sudo iptables -L -n | head          # may be iptables-nft compat
sudo ss -lntp
```

Confirm listeners and path: cloud security group → host nft/ufw → app.

JSON for scripts

```
sudo nft -j list ruleset | jq '.nftables | length'
```

Notes

- Prefer **one** management plane: ufw *or* raw nft *or* firewalld (RHEL) — mixing without care causes "ghost" blocks.
- Docker/Kubernetes install their own chains/tables; order and forward policy matter for published ports.
- Counter/log statements help prove hits: `counter`, `log prefix "nft: "`.
- IPv6: `inet` family reduces duplicated ip/ip6 tables.
- Naming: table/chain names are yours in custom rulesets; distro tools use their own.

Related Commands

- `ufw` — Ubuntu host firewall front-end
- `iptables` — legacy/compat syntax (`iptables-nft`)
- `ss -lntp` — listening sockets before opening ports
- `tcpdump` — packet proof beyond firewall counters
- `sysctl` — e.g. `net.ipv4.ip_forward` for routers
- `systemctl` — `nftables` service when used

Additional Resources

- `man nft`
- `man nftables`
- `/etc/nftables.conf` (package example)
- nftables wiki

iptables

Overview

`iptables` configures IPv4 packet filtering/NAT via **netfilter**. Many modern distros (Ubuntu 22.04+) use **nftables** underneath (`iptables-nft` compatibility). Prefer **`ufw`** or **`nft`** for new designs; still know `iptables` for legacy hosts and debugging.

Syntax

```
iptables [-t table] command chain [rule...]
iptables-save / iptables-restore
```

Tables & common chains

Table	Chains	Role
`filter` (default)	INPUT, FORWARD, OUTPUT	Allow/deny
`nat`	PREROUTING, POSTROUTING, OUTPUT	DNAT/SNAT/MASQUERADE
`mangle`	*	TTL/mark rewrite
`raw`	PREROUTING, OUTPUT	Conntrack exceptions

Targets: `ACCEPT`, `DROP`, `REJECT`, `LOG`, `RETURN`, …

Common Options

Option	Description
`-A/-I/-D`	Append / insert / delete
`-L -n -v`	List numeric verbose
`-F`	Flush chain
`-P chain target`	Default policy
`-s/-d`	Source/dest
`-p tcp/udp/icmp`	Protocol
`--dport/--sport`	Ports (with `-m tcp`)

Option	Description
`-i/-o`	In/out interface
`-m conntrack --ctstate`	State match
`-j`	Target

Safety

Remote lockout risk: open SSH before default DROP. Use console access when changing INPUT policy. Persist rules (`iptables-save`, distro-specific services) or they vanish on reboot.

Examples with Explanations

List

```
sudo iptables -L -n -v
sudo iptables -t nat -L -n -v
```

Allow SSH, HTTP; default drop (sketch)

```
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
# OUTPUT often left ACCEPT
```

NAT masquerade (router)

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Save (Debian/Ubuntu example)

```
sudo apt install iptables-persistent
sudo netfilter-persistent save
# or: sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Prefer UFW on Ubuntu

```
sudo ufw allow OpenSSH
sudo ufw allow 80,443/tcp
sudo ufw enable
```

Notes

- IPv6 uses `ip6tables`.
- Docker injects its own chains — order interactions carefully.
- `nft list ruleset` shows backend on nft systems.

Related Commands

- `ufw` — Ubuntu host firewall
- `nft` — nftables CLI
- `ss -lntp` — listeners
- `tcpdump` — packet proof

ifconfig

Overview

`ifconfig` configures and displays network interfaces from the legacy **net-tools** package. On modern Linux, prefer `ip` from iproute2 (`ip addr`, `ip link`, `ip route`). This page exists for reading old docs, recovery shells that still ship net-tools, and muscle-memory translation to `ip`.

Many minimal images no longer install `ifconfig` by default.

Syntax

```
ifconfig [interface]
ifconfig interface [address_family] options
```

Common Options / forms

Form	Description
`ifconfig`	List up interfaces (often only “up”)
`ifconfig -a`	All interfaces including down
`ifconfig eth0`	Show one interface
`ifconfig eth0 up` / `down`	Bring interface up/down
`ifconfig eth0 192.0.2.10 netmask 255.255.255.0`	Set IPv4
`ifconfig eth0 add ...`	Additional addresses (implementation-dependent)
`ifconfig eth0 hw ether AA:BB:...`	Set MAC (may need down first)
`ifconfig eth0 mtu 1400`	Set MTU
`ifconfig eth0:0 ...`	Alias interface style (legacy)

Examples with Explanations

Display

```
ifconfig
ifconfig -a
ifconfig eth0
```

Preferred modern equivalents

```
ip -br link
ip -br addr
ip addr show dev eth0
ip link set eth0 up
ip addr add 192.0.2.10/24 dev eth0
ip route
```

Bring up/down

```
sudo ifconfig eth0 up
sudo ifconfig eth0 down
# modern:
sudo ip link set eth0 up
```

Set address (legacy)

```
sudo ifconfig eth0 192.0.2.10 netmask 255.255.255.0
# modern CIDR:
sudo ip addr add 192.0.2.10/24 dev eth0
```

MTU

```
sudo ifconfig eth0 mtu 1400
sudo ip link set eth0 mtu 1400
```

Install on Debian/Ubuntu if missing

```
sudo apt install net-tools
```

Still prefer learning `ip`.

Notes / Pitfalls

- Output and available flags differ across Unixes; Linux net-tools is not identical to BSD.
- Changes are not persistent — use netplan/NetworkManager/systemd-networkd.
- Alias interfaces (`eth0:0`) are obsolete vs multiple addresses on one device.
- Scripts parsing `ifconfig` are fragile — use `ip -j` JSON when available.
- Wireless details: use `iw` / NetworkManager, not only ifconfig.

2026-relevant notes

- Teaching and certification materials may still show ifconfig; translate on sight to `ip`.
- Containers: `ip` is the standard; ifconfig often absent.
- For persistent config, never rely on either ifconfig or ip alone — use the distro network stack.

Related Commands

- `ip` — modern interface/address/route tool
- `ss` — sockets (replaces netstat)
- `nmcli` / `networkctl` — higher-level management
- `ethtool` — NIC hardware settings
- `ifup` / `ifdown` — distro helper scripts (legacy)

Additional Resources

- `man ifconfig` (if installed), `man ip`

netstat

Overview

`netstat` displays network connections, listening sockets, routing tables, and interface statistics from the legacy **net-tools** suite. On modern Linux, prefer **`ss`** (sockets) and **`ip`** (routes/links). `netstat` remains useful when reading older documentation or working on systems that still ship it.

Syntax

```
netstat [options]
```

Common Options

Option	Description
`-t` / `-u`	TCP / UDP
`-l`	Listening sockets
`-a`	All sockets
`-n`	Numeric addresses (no DNS)
`-p`	Process PID/name (may need root)
`-r`	Routing table
`-i`	Interfaces
`-s`	Statistics
`-c`	Continuous refresh
`-w`	RAW sockets
`-x`	Unix sockets
`-e`	Extended
`-4` / `-6`	Address family

Classic combo: `netstat -tulpn` for listening TCP/UDP with PIDs.

Examples with Explanations

Listening services

```
netstat -tulpn
netstat -tlnp
# modern:
ss -tulpn
```

Established connections

```
netstat -tn
netstat -tp
ss -tp
```

Routing

```
netstat -rn
ip route
```

Interfaces / stats

```
netstat -i
netstat -s | less
ip -s link
```

Continuous

```
netstat -ct
watch -n1 'ss -tnp'
```

Translate old → new

Old	New
`netstat -tulpn`	`ss -tulpn`
`netstat -tn`	`ss -tn`
`netstat -rn`	`ip route`
`netstat -i`	`ip -s link` / `ip -br link`
`netstat -s`	`nstat` / `ss -s` / `/proc/net`

Notes / Pitfalls

- Not installed by default on many minimal distros (`net-tools` package).
- DNS reverse lookups without `-n` make output slow — prefer `-n`.
- `-p` requires privileges for others' processes.
- Output format is for humans; parse carefully or use `ss -H`/`ip -j`.
- Namespace-aware debugging needs `ip netns exec` + `ss`.

2026-relevant notes

- Muscle memory migration to `ss`/`ip` is complete for most ops teams; learn netstat only for translation.
- eBPF-based tools (`bpftool`, Pixie, etc.) go deeper than either for advanced tracing.
- Containers: check sockets **inside** the correct network namespace.

Related Commands

- `ss` — modern socket statistics
- `ip` — routes and links
- `lsof -i` — files/sockets by process
- `nstat` — network counters
- `nmap` — external port scans (authorized)

Additional Resources

- `man netstat` (if installed), `man ss`, `man ip`

ftp

Overview

`ftp` is the classic interactive client for the **File Transfer Protocol**. On modern systems it is largely superseded by **`sftp`/`scp`/`rsync` over SSH** and HTTPS downloads (`curl`/`wget`). Prefer encrypted channels whenever possible — plain FTP sends credentials and data in cleartext.

Many distros ship `ftp` as `ftp` or `tnftp` / require installing `ftp` package; some only provide `lftp` or `sftp`.

Syntax

```
ftp [options] [host]
ftp ftp://user@host/path     # URL form on some clients
```

Common Options / commands

Client option flags vary. Once connected, interactive commands include:

Command	Description
`open host`	Connect
`user`	Authenticate
`ls` / `dir`	List remote
`cd` / `lcd`	Remote / local directory
`get` / `mget`	Download
`put` / `mput`	Upload
`binary` / `ascii`	Transfer mode
`passive` / `passive on`	Passive mode (usually required today)
`bye` / `quit`	Exit
`hash`	Progress hashes
`prompt`	Toggle mget/mput prompts

Examples with Explanations

Interactive session

```
ftp ftp.example.com
# Name: anonymous
# Password: user@email
binary
passive
ls
get README
bye
```

Prefer SFTP instead

```
sftp user@host
scp file user@host:/path/
rsync -avP file user@host:/path/
```

lftp (better modern client)

```
lftp ftp://user@host
lftp -e 'set ftp:passive-mode true; mirror -c remote local; bye' host
```

Scripted (discouraged for secrets)

```
ftp -n host <<'EOF'
user anonymous guest@
binary
get file.bin
bye
EOF
```

Credentials in scripts leak; use SSH keys or secret stores.

Firewall note

Active FTP fails behind NATs; **passive mode** is the norm. Still painful vs SSH.

Notes / Pitfalls

- Cleartext auth and data — avoid on untrusted networks.
- FTPS (FTP over TLS) is different from SFTP (SSH); know which you need.
- Corporate scanners still find open FTP servers — disable if unused.
- ASCII mode corrupts binaries — use `binary`.
- Many public mirrors moved to HTTPS only.

2026-relevant notes

- Treat FTP as legacy interoperability only.
- Default transfer should be `sftp`/`rsync`/`curl` https.
- If you must run a server, prefer SFTP chroots or HTTPS object storage.

Related Commands

- `sftp` / `scp` / `rsync` — SSH-based transfer
- `lftp` / `ncftp` — richer FTP clients
- `curl` / `wget` — URL downloads
- `ssh` — remote shell + tunnels

Additional Resources

- `man ftp`, `man sftp`

telnet

Overview

`telnet` is a historic remote terminal protocol client. **Do not use telnet for remote logins** — it is unencrypted. The remaining legitimate use is as a **crude TCP debugging tool**: check whether a port accepts connections and speak simple line-based protocols by hand.

Prefer `ssh` for logins and `nc`/`openssl s_client`/`curl -v` for protocol debugging.

Syntax

```
telnet [options] host port
telnet host               # default port 23 (avoid)
```

Common Options

Option	Description
`host port`	Connect to TCP port
`-4` / `-6`	Address family
`-e char`	Escape character
`-E`	Disable escape
`-l user`	Autologin user (legacy servers)

Interactive: escape default often `Ctrl-]`, then `quit`.

Examples with Explanations

Port open check

```
telnet 192.0.2.10 22
# Trying... Connected to ...  → TCP open
# Connection refused → closed
# Timeout → filtered/dropped
```

Prefer:

```
nc -vz 192.0.2.10 22
# or
timeout 3 bash -c 'echo >/dev/tcp/192.0.2.10/22' && echo open
```

Speak HTTP manually

```
telnet example.com 80
# then type:
GET / HTTP/1.0
Host: example.com

# blank line ends headers
```

SMTP banner grab (authorized testing only)

```
telnet mail.example.com 25
```

Escape and quit

```
Ctrl-]
telnet> quit
```

Prefer TLS-aware tools for HTTPS/SMTPS

```
openssl s_client -connect example.com:443 -servername example.com
curl -vI https://example.com
```

Notes / Pitfalls

- Remote shell over telnet is **obsolete and unsafe**.
- "Connected" only means TCP handshake succeeded — not that the app protocol is healthy.
- Line endings and locale can confuse interactive protocol tests.
- Some minimal images omit telnet clients — install `telnet` or use `nc`.
- Never put passwords into telnet sessions on shared screen recordings.

2026-relevant notes

- Keep `nc`/`nmap`/`curl` in the toolbox; telnet is optional nostalgia/debug.
- Cloud security groups vs app failures: telnet/nc distinguish network path from HTTP 500s.
- IPv6: explicitly test with `-6` when dual-stack.

Related Commands

- `nc` / `ncat` — flexible network swiss army knife
- `ssh` — secure remote login
- `openssl s_client` — TLS debugging
- `curl` -v — HTTP debugging
- `nmap` — broader port scans (authorized)

Additional Resources

- `man telnet`

ssh-keygen

Overview

`ssh-keygen` creates and manages SSH key pairs, fingerprints host keys, and can update `known_hosts`. Modern default is **Ed25519**; RSA remains for legacy peers. Key management is the foundation of safe automation and admin access.

Syntax

```
ssh-keygen [options]
ssh-keygen -t ed25519 -C 'comment' -f path
ssh-keygen -lf key.pub
```

Common Options

Option	Description
`-t type`	`ed25519`, `rsa`, `ecdsa`, …
`-b bits`	RSA bit length (e.g. 4096)
`-C comment`	Comment (often email)
`-f file`	Output path (private key)
`-N passphrase`	Passphrase (empty for none — automation only with care)
`-lf file`	Show fingerprint
`-y`	Derive public key from private
`-p`	Change passphrase
`-R host`	Remove host from known_hosts
`-F host`	Find host key in known_hosts
`-A`	Generate missing host keys (on servers)

Safety

- Private keys stay mode `600` and never leave your control.
- Prefer passphrases + `ssh-agent` over unprotected keys on laptops.

- Automation keys: scope with `authorized_keys` options (`from=`, `command=`, `restrict`).
- Rotate and revoke by removing the public line from `authorized_keys`.

Examples with Explanations

Create an Ed25519 key

```
ssh-keygen -t ed25519 -C 'you@example.com' -f ~/.ssh/id_ed25519
```

RSA for legacy only

```
ssh-keygen -t rsa -b 4096 -C 'legacy' -f ~/.ssh/id_rsa_legacy
```

Fingerprint

```
ssh-keygen -lf ~/.ssh/id_ed25519.pub
ssh-keygen -E md5 -lf ~/.ssh/id_ed25519.pub   # older tooling compares MD5
```

Fix / inspect known_hosts

```
ssh-keygen -F github.com
ssh-keygen -R old-server.example.com
```

Public key from private

```
ssh-keygen -y -f ~/.ssh/id_ed25519 > ~/.ssh/id_ed25519.pub
```

Related Commands

- `ssh` / `ssh-copy-id` — use and install keys
- `ssh-agent` / `ssh-add` — unlock keys in memory
- `chmod` — enforce `700` on `~/.ssh`, `600` on private keys

Additional Resources

- `man ssh-keygen`
- OpenSSH release notes on default algorithms

ssh-copy-id

Overview

`ssh-copy-id` installs your public key on a remote account's `~/.ssh/authorized_keys` with safer permissions than hand-copying. Prefer it over pasting keys via ad-hoc `echo >>` when the remote still allows password auth for bootstrap.

Syntax

```
ssh-copy-id [options] [user@]host
ssh-copy-id -i key.pub user@host
```

Common Options

Option	Description
`-i key`	Public key file (default: first available id_*.pub)
`-p port`	SSH port
`-o Opt=Val`	Pass through to ssh
`-f`	Force add without duplicate check (implementation-dependent)
`-n`	Dry-run (where supported)

Safety

- Still authenticates once with the remote password (or existing key).
- Review remote `authorized_keys` after install.
- Do not use world-writable home or `.ssh` directories — sshd will ignore keys.

Examples with Explanations

Default identity

```
ssh-copy-id alice@server.example.com
ssh alice@server.example.com
```

Explicit key and port

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub -p 2222 alice@server.example.com
```

Via ProxyJump

```
ssh-copy-id -o ProxyJump=bastion.example.com alice@internal
```

Manual equivalent (when tool missing)

```
cat ~/.ssh/id_ed25519.pub | ssh alice@server \
  'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys'
```

Notes & Pitfalls

- Multiple keys: specify `-i` so the wrong identity is not installed.
- Some hardened images disable passwords — need console/cloud-init to place the first key.
- SELinux contexts on `~/.ssh` can block access until restored.

Related Commands

- `ssh-keygen` — create keys
- `ssh` — connect
- `getent passwd` — confirm remote home path

Additional Resources

- `man ssh-copy-id`

sftp

Overview

`sftp` is the interactive SSH file-transfer client (SFTP subsystem). Prefer it over legacy `ftp`/`telnet` for remote file work; for scripted bulk sync, prefer `rsync` or `scp`/`sftp` batch modes. Uses the same auth and config as `ssh`.

Syntax

```
sftp [options] [user@]host
sftp [options] -b batchfile [user@]host
```

Common Options

Option	Description
`-i key`	Private key
`-P port`	Port (note capital **P**, unlike ssh's `-p`)
`-o Opt=Val`	ssh options (`ProxyJump`, …)
`-b file`	Batch commands from file
`-a`	Attempt to continue interrupted downloads (where supported)
`-r`	Recursive for `get`/`put` in modern OpenSSH

Examples with Explanations

Interactive session

```
sftp alice@server.example.com
# sftp> ls
# sftp> get remote.txt
# sftp> put local.txt /var/tmp/
# sftp> bye
```

Batch download

```
cat > /tmp/sftp.batch <<'EOF'
cd /var/backups
lcd /tmp
get -r weekly/
bye
EOF
sftp -b /tmp/sftp.batch alice@server
```

Jump host

```
sftp -o ProxyJump=bastion.example.com alice@internal
```

Non-interactive one-shot via ssh

```
ssh alice@server 'cat /etc/os-release' > os-release.txt
rsync -av alice@server:/var/www/ ./www-mirror/
```

Often clearer than SFTP batch for automation.

Notes & Pitfalls

- Port flag is `-P` for sftp/scp historical reasons.
- Recursive behavior and resume support vary by OpenSSH version.
- File modes/ownership on put depend on remote umask and SSH config.

Related Commands

- `scp` — simple remote copy
- `rsync` — efficient sync
- `ssh` — remote shell
- `rclone` — multi-cloud sync if installed

Additional Resources

- `man sftp`
- `man sftp-server`

ethtool

Overview

`ethtool` queries and controls **Ethernet NIC** settings: link speed/duplex, offloads, ring parameters, and driver info. Use it when `ip link` shows a device but performance or link negotiation looks wrong. Requires root for most changes.

```
sudo apt install ethtool
```

Syntax

```
ethtool [options] devname
ethtool -s devname …          # change settings
```

Common Options

Option	Description
(dev)	Show basic link settings
`-i`	Driver and firmware info
`-k`	Offload features
`-K`	Change offloads
`-S`	NIC/driver statistics
`-g` / `-G`	Ring buffer get/set
`-s`	Change speed/duplex/autoneg/wol
`-p`	Identify port (blink LED)

Safety

- Forcing speed/duplex incorrectly causes silent poor performance or link down. Prefer autoneg when the peer supports it.
- Offload changes can fix or break checksum issues — test with care on production.

Examples with Explanations

Link status

```
ethtool eth0
# Speed, Duplex, Auto-negotiation, Link detected
```

Driver info

```
ethtool -i eth0
```

Statistics

```
ethtool -S eth0 | egrep -i 'err|drop|crc|miss'
```

Identify physical port

```
sudo ethtool -p eth0 5          # blink ~5 seconds
```

Offloads

```
ethtool -k eth0 | head
sudo ethtool -K eth0 gro off    # example; know why you toggle
```

Persist settings

Use distro network config (`netplan`, NetworkManager, systemd-networkd) or `ethtool` udev/systemd oneshot units — bare `ethtool -s` does not survive reboot alone.

Related Commands

- `ip link` / `ip -s link` — state and counters
- `nmcli` — NetworkManager
- `lspci` — hardware identity
- `dmesg` — driver attach messages

Additional Resources

- `man ethtool`

firewall-cmd

Overview

`firewall-cmd` is the client for **firewalld**, the dynamic firewall manager on Fedora, RHEL, CentOS Stream, and many related systems. Zones and services abstract raw nftables/iptables rules. On Ubuntu, prefer `ufw` or direct `nft` unless you installed firewalld intentionally.

```
sudo dnf install firewalld
sudo systemctl enable --now firewalld
```

Syntax

```
firewall-cmd [options]
```

Common Options / Patterns

Action	Example
State	`firewall-cmd --state`
Active zones	`--get-active-zones`
List zone	`--list-all` / `--zone=public --list-all`
Allow service	`--add-service=http --permanent`
Allow port	`--add-port=8080/tcp --permanent`
Reload	`--reload`
Panic	`--panic-on` (blocks all — careful)

Safety

- Always pair `--permanent` with `--reload` (or use runtime-only for experiments).
- Locking yourself out over SSH: ensure `ssh` service is allowed in the active zone before disconnecting.
- Panic mode is a last resort.

Examples with Explanations

Inventory

```
sudo firewall-cmd --state
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all
```

Allow HTTP permanently

```
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload
sudo firewall-cmd --list-services
```

Open a custom port

```
sudo firewall-cmd --permanent --add-port=9090/tcp
sudo firewall-cmd --reload
```

Rich rule example

```
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="10.0.0.0/8
sudo firewall-cmd --reload
```

Runtime vs permanent

```
sudo firewall-cmd --add-service=http          # runtime only
sudo firewall-cmd --runtime-to-permanent      # promote
```

Notes & Pitfalls

- Interfaces bind to zones (`--zone=public --change-interface=eth0`).
- Conflicts with manual `nft`/`iptables` rules are common — pick one manager.
- Container ports still need host firewall + publish rules.

Related Commands

- `ufw` — Ubuntu's simple frontend
- `nft` / `iptables` — low-level
- `ss` — verify listeners
- `nmcli` — interface/zone integration on NM systems

Additional Resources

- `man firewall-cmd`
- firewalld documentation (zones and services)

ssh-agent / ssh-add

Overview

`ssh-agent` holds decrypted private keys in memory so you unlock a passphrase-protected identity once per session instead of on every `ssh`/`scp`/`rsync`/`git` connection. `ssh-add` loads, lists, locks, and removes those identities. Use the agent on workstations and jump hosts you control; do **not** treat agent forwarding (`ssh -A`) as a convenience default — it extends trust to every host that can reach the forwarded socket.

Prefer passphrase-protected keys in the agent over unencrypted private keys on disk. For automation on servers, scoped `authorized_keys` options usually beat long-lived agent sessions.

Syntax

```
ssh-agent [-c|-s] [-Dd] [-a bind_address] [-t life] [command [arg ...]]
ssh-agent [-c|-s] -k

ssh-add [-cDdLlXx] [-t life] [-h constraint] [file ...]
ssh-add -T pubkey...
```

Bourne shells start an agent and import its environment with:

```
eval "$(ssh-agent -s)"
```

Common Options

ssh-agent

Option	Description
`-s`	Print Bourne-shell `export` lines (default for bash/zsh)
`-c`	Print C-shell `setenv` lines
`-k`	Kill the agent named by `SSH_AGENT_PID`
`-a path`	Bind the UNIX-domain socket to an explicit path
`-t life`	Default max lifetime for identities (seconds or `sshd_config` time format, e.g. `8h`)
`-D`	Foreground (no fork) — useful under systemd

Option	Description
`-d`	Debug + no fork
`command …`	Run a subprocess; agent exits when that command exits

ssh-add

Option	Description
(no args)	Add default identities (`~/.ssh/id_ed25519`, `id_rsa`, …)
`file`	Add a specific private key (and matching `-cert.pub` if present)
`-l`	List fingerprints of loaded identities
`-L`	List public keys of loaded identities
`-d`	Delete listed identities (paths to keys/`.pub`, or defaults if no args)
`-D`	Delete **all** identities
`-t life`	Lifetime for this add (overrides agent default)
`-c`	Require confirmation (via `ssh-askpass`) before each use
`-x` / `-X`	Lock / unlock the agent with a password
`-h constraint`	Destination-constrain the key (OpenSSH 8.9+)
`-T pubkey…`	Sign/verify test that matching private keys are usable
`-q`	Quiet on success

Client config that cooperates with the agent

`~/.ssh/config`	Effect
`AddKeysToAgent yes`	First use of a key file loads it into a running agent
`AddKeysToAgent confirm`	Like `yes`, but each use needs confirmation
`IdentityAgent path`	Use this socket (overrides `SSH_AUTH_SOCK`; `none` disables)
`IdentitiesOnly yes`	Offer only configured/`-i` keys — avoids "too many authentication failures" when the agent holds many keys
`ForwardAgent no`	Safe default; never enable globally

Safety

- **Agent forwarding (`ssh -A` / `ForwardAgent yes`)** lets a compromised remote host request signatures from your local agent for as long as the session lasts. Prefer ProxyJump / `ProxyCommand` over forwarding. If you must forward, constrain keys with `ssh-add -h` and keep lifetimes short (`-t`).
- Anyone who can talk to `SSH_AUTH_SOCK` (same UID, or root) can use your loaded keys. Lock with `ssh-add -x` when stepping away; clear with `ssh-add -D` after sensitive work.
- Do not start nested agents blindly in every shell — orphan agents leave sockets and PIDs around. Prefer one agent per login session (desktop session, systemd user unit, or explicit `eval`).
- Passphrases never leave the machine that runs the agent; only signature requests travel over a forwarded channel. That still means a malicious remote can authenticate *as you* to other hosts while the forward is up.

Key Use Cases

1. Unlock a passphrase-protected Ed25519 key once, then SSH to many hosts.
2. Keep several identities loaded and select with `IdentitiesOnly` + `IdentityFile`.
3. Time-box keys for a maintenance window (`ssh-add -t 2h`).
4. Avoid agent forwarding by using `ProxyJump` for bastion workflows.
5. Lock the agent on a shared workstation between tasks.

Examples with Explanations

Example: start an agent and add a key

```
eval "$(ssh-agent -s)"
# Agent pid 18421

ssh-add ~/.ssh/id_ed25519
# Enter passphrase for /home/alice/.ssh/id_ed25519:
# Identity added: /home/alice/.ssh/id_ed25519 (alice@laptop)

ssh-add -l
# 256 SHA256:AbCdEf... /home/alice/.ssh/id_ed25519 (ED25519)
```

`eval` imports `SSH_AUTH_SOCK` and `SSH_AGENT_PID` into the current shell. Subsequent OpenSSH clients in that environment ask the agent instead of reading the private key file (and prompting) on every connect.

Example: confirm the environment is wired

```
echo "$SSH_AUTH_SOCK"
# /tmp/ssh-XXXXXX/agent.18420

ssh-add -l >/dev/null && echo 'agent reachable' || echo 'no agent (exit 2)'
```

`ssh-add` exits **2** when it cannot contact an agent — useful in scripts and shell prompts.

Example: time-limited identity for a change window

```
ssh-add -t 2h ~/.ssh/id_ed25519_ops
ssh-add -l
# … (expires in ~2 hours)
```

After the lifetime, the agent drops the key; the next use prompts again (or fails if you rely only on the agent). Pair with a matching short change window.

Example: list, remove one key, or clear all

```
ssh-add -L                          # full public keys (pasteable)
ssh-add -d ~/.ssh/id_ed25519.pub    # remove one identity
ssh-add -D                          # remove everything
```

`-d` accepts private key paths or `.pub` paths; if the path has no `.pub` sibling it retries with `.pub` appended.

Example: AddKeysToAgent on first use

```
# ~/.ssh/config
Host *.example.com
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
  AddKeysToAgent yes
  ForwardAgent no
```

```
eval "$(ssh-agent -s)"   # once per session
ssh bastion.example.com  # prompts for passphrase once; agent keeps the key
ssh app1.example.com     # reuses the agent - no second passphrase
```

`IdentitiesOnly yes` stops ssh from trying every agent key before the intended `IdentityFile`, which is a common cause of `Too many authentication failures`.

Example: prefer ProxyJump over agent forwarding

```
# Good: auth from the laptop to both hops; no remote sees your agent
ssh -J bastion.example.com alice@app.internal.example.com

# ~/.ssh/config equivalent
Host app-internal
  HostName app.internal.example.com
  User alice
  ProxyJump bastion.example.com
  IdentityFile ~/.ssh/id_ed25519
  IdentitiesOnly yes
  ForwardAgent no
```

```
# Avoid unless you understand the blast radius
ssh -A bastion.example.com   # remote can request signatures from your agent
```

With ProxyJump, OpenSSH authenticates each hop from your client. With `-A`, the bastion (and anything that can access the forwarded socket there) can use your loaded keys.

Example: destination-constrained key (OpenSSH 8.9+)

```
# Allow this key only when authenticating to git.example.com
ssh-add -h git.example.com ~/.ssh/id_ed25519_git

# Allow use via bastion → internal (both hops must match)
ssh-add -h 'bastion.example.com>alice@app.internal.example.com' \
  ~/.ssh/id_ed25519_ops
```

Constraints are enforced by a cooperating agent and client. They reduce damage from forwarding but are **not** a substitute for `ForwardAgent no` as the default.

Example: lock the agent while you step away

```
ssh-add -x          # set an agent lock password
# … leave the desk …
ssh-add -X          # unlock
```

While locked, the agent refuses signature requests until unlocked. This is orthogonal to key lifetimes.

Example: kill a manually started agent

```
ssh-agent -k
# unset SSH_AUTH_SOCK;
# unset SSH_AGENT_PID;
# echo Agent pid 18421 killed;
```

Only works when `SSH_AGENT_PID` points at that agent. Desktop/session agents managed by systemd or a display manager should be stopped via that service, not `-k` from a random shell.

Example: systemd user unit sketch (persistent per login)

On Ubuntu Server / workstations with systemd user sessions, a common pattern is a user service that runs `ssh-agent -D` and exposes the socket under `$XDG_RUNTIME_DIR`, then points clients with:

```
# ~/.ssh/config
Host *
  AddKeysToAgent yes
  IdentityAgent $XDG_RUNTIME_DIR/ssh-agent.socket
```

Wire the exact unit file to your distro's docs; the important operator contract is **one socket path** exported (or set via `IdentityAgent`) for all shells in the session.

Understanding Output

Check	Meaning
`ssh-add -l` → fingerprints	Identities loaded and usable
`ssh-add -l` → `The agent has no identities.`	Agent up, empty — run `ssh-add`
`ssh-add -l` exit 2 / "Could not open a connection…"	No reachable agent (`SSH_AUTH_SOCK` wrong or dead)
`ssh -v` lines mentioning `Offering public key` / `Server accepts key`	Client is using a key (from file or agent)

Fingerprints default to SHA256; use `ssh-add -E md5 -l` only when comparing against older inventory that still prints MD5.

Notes & Pitfalls

- **Nested `eval "$(ssh-agent -s)"` in every new terminal** creates orphan agents. Start once per graphical/login session, or use a systemd user socket, then inherit `SSH_AUTH_SOCK`.

- **tmux/screen** panes may not see an agent started in another pane unless you export/refresh `SSH_AUTH_SOCK` (or use a stable socket path under `$XDG_RUNTIME_DIR`).
- **Many keys in the agent** without `IdentitiesOnly` causes servers with `MaxAuthTries` to reject you before the right key is offered.
- **macOS** may integrate with the Keychain (`UseKeychain` / `--apple-use-keychain`); those flags are not portable to Linux — stick to OpenSSH options here.
- **Debian/Ubuntu** historically install `ssh-agent` setgid to harden against ptrace stealing key material; unusual `LD_*` / `TMPDIR` behavior can surprise custom wrappers.
- FIDO/hardware keys (`sk` types) and PKCS#11 tokens have extra `ssh-add` flags (`-K`, `-S`, `-s`); treat them as specialized paths after the basic file-based workflow works.

Common Usage Patterns

```
# Session bootstrap (interactive laptop)
eval "$(ssh-agent -s)"
ssh-add -t 8h ~/.ssh/id_ed25519

# Verify before a deploy burst
ssh-add -l
ssh -o BatchMode=yes alice@app1 true && echo ok

# End of sensitive work
ssh-add -D
```

Related Commands

- `ssh-keygen` — create and fingerprint key pairs
- `ssh-copy-id` — install a public key on a remote account
- `ssh` — client; `ProxyJump`, `ForwardAgent`, `IdentityAgent`, `AddKeysToAgent`
- `scp` / `sftp` / `rsync` — reuse the same agent over SSH transports

Additional Resources

- `man ssh-agent`, `man ssh-add`, `man ssh_config` (Ubuntu 24.04 / OpenSSH as shipped)
- OpenSSH release notes for destination constraints (8.9+) and agent defaults

Certificate renewal audit (64-day and 45-day certificates)

Certificate renewal audit (64-day and 45-day certificates)

Overview

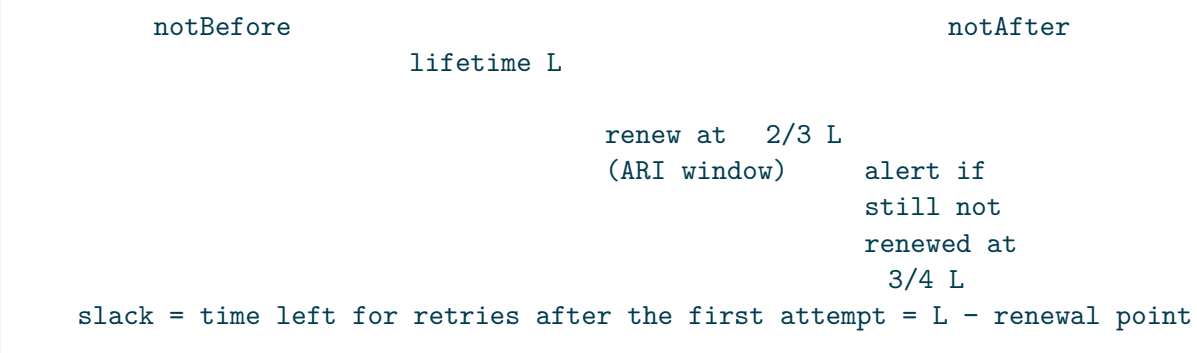
Thesis: for ten years a 90-day Let's Encrypt certificate forgave a lot. A cron job that renewed "every two months", an alert at "21 days left", a renewal that wrote a new file but never reloaded nginx: all of these worked because 90 days left plenty of room. That room is going away. Let's Encrypt switches its default profile to **64-day** certificates on **10 February 2027** (staging on **14 October 2026**), and to **45 days** in February 2028. The industry rules cap every public TLS certificate at 47 days from March 2029. Any renewal logic that counts in **fixed days** breaks somewhere on that path. Renewal logic that works in **fractions of the lifetime**, or asks the CA when to renew (ARI), keeps working.

This page is an audit you can run today:

- Read what lifetime your certificates actually have, and which ACME profiles your CA offers.
- Ask the CA when it wants each certificate renewed, with one unauthenticated `curl` (ACME Renewal Information, RFC 9773).
- Find the cron lines, renewal configs and monitoring thresholds that hard-code a lifetime.
- Check that the certificate you **serve** is the one you renewed.

Everything here uses `openssl`, `curl`, `jq`, `grep` and coreutils. No ACME client is needed to run the listings, and none of them requests a certificate.

Mental model



Lifetime	Who issues it	Renew at 2/3	Slack after first attempt
90 days	Let's Encrypt `classic` until 2027-02-10	day 60	30 days
64 days	`classic` from 2027-02-10	day 42.7	21.3 days
45 days	`tlsserver` since 2026-05-13, `classic` from 2028-02-16	day 30	15 days
160 hours	`shortlived` (opt-in)	day 4.4	2.2 days

Fact	Consequence
The lifetime is chosen by the **CA and profile**, not by you	Your scripts must read it from the certificate, never assume it
Let's Encrypt asks clients to renew at about **2/3** of the lifetime	A rule written as a fraction survives every change above
ARI lets the CA move the window (load spreading, mass revocation)	An ARI-aware client renews early when told to, and you do nothing
A renewed file is not a renewed service	Daemons keep the certificate they loaded until they reload
Authorization reuse drops from 30 days to 10 days on 2027-02-10, and to 7 hours in 2028	"Validate once a month, issue later" designs stop working

Syntax

The pieces of the audit:

```
openssl x509 -noout -startdate -enddate -in cert.pem        # lifetime from the certificate
openssl x509 -noout -ext authorityKeyIdentifier -serial      # the two halves of an ARI cert
openssl x509 -noout -checkend SECONDS                        # absolute "expires within N s"
curl -s https://acme-v02.api.letsencrypt.org/directory       # profiles and the renewalInfo
curl -s  <renewalInfo>/<AKI-keyid-b64url>.<serial-b64url>     # ARI: suggested renewal windo
```

Certbot behaviour you are auditing (from the upstream changelog and user guide):

Certbot	When `certbot renew` considers a certificate due
before 4.0	fixed: 30 days before expiry
4.0.0 (2025-04-07)	less than 1/3 of the lifetime left (1/2 for lifetimes of 10 days or less). Adds `--preferred-profile` / `--required-profile`
4.1.0 (2025-06-10)	as 4.0, plus ARI: an early ARI window overrides a later `renew_before_expiry`

Certbot	When `certbot renew` considers a certificate due
5.0.0 (2025-09-02)	stores ARI's `Retry-After` between runs. Current upstream is 5.8.0 (2026-09-01)

What distributions ship, checked against the Debian and Ubuntu archives in October 2026:

Distribution	Packaged certbot	Renewal rule	ARI
Debian 12 (bookworm)	2.1.0	fixed 30 days	no
Debian 13 (trixie)	4.0.0	1/3 left	no
Ubuntu 24.04 LTS	2.9.0	fixed 30 days	no
Ubuntu 26.04 LTS	4.0.0	1/3 left	no

No stable distribution package has ARI yet. The upstream snap, the Docker image and `pip` installs do.

Safety

Every listing on this page is read-only against the outside world. The ARI endpoint is a public, unauthenticated `GET` by design (RFC 9773 §6), and the TLS probes only complete a handshake. The test certificates are self-signed, live in a temporary directory, and are useless outside the lab. The audit script only reads files. Run it with `sudo` on a real host so it can see `/etc/letsencrypt`, and review what it prints before you change anything.

Examples with Explanations

Tested on Debian 13 with OpenSSL 3.5.7, curl, jq 1.7 and GNU coreutils 9.7, on 13 October 2026 (IST). The probes went to public hosts and the live Let's Encrypt API, so your dates and serials will differ.

1. What does your CA offer, and what do certificates live today?

The ACME directory is the canonical list of profiles:

```
curl -s https://acme-v02.api.letsencrypt.org/directory | jq .meta.profiles
```

```
{
  "classic": "https://letsencrypt.org/docs/profiles#classic",
  "shortlived": "https://letsencrypt.org/docs/profiles#shortlived",
  "tlsserver": "https://letsencrypt.org/docs/profiles#tlsserver"
}
```

Read the lifetime from the certificate itself:

```
#!/usr/bin/env bash
# cert-life.sh HOST... - issuer and total validity period of each host's leaf certificate
for h in "$@"; do
  pem=$(openssl s_client -connect "$h:443" -servername "$h" </dev/null 2>/dev/null | openssl
  iss=$(openssl x509 -noout -issuer <<<"$pem" | sed 's/.*CN=//')
  nb=$(date -d "$(openssl x509 -noout -startdate <<<"$pem" | cut -d= -f2)" +%s)
  na=$(date -d "$(openssl x509 -noout -enddate <<<"$pem" | cut -d= -f2)" +%s)
  printf '%-15s %-40s %3d days\n' "$h" "$iss" $(( (na - nb + 1) / 86400 ))
done
```

```
./cert-life.sh git.kernel.org lwn.net www.kernel.org
```

```
git.kernel.org  YE1                                        90 days
lwn.net         YE2                                        90 days
www.kernel.org  GlobalSign Atlas R46 DV TLS CA 2026 Q3    198 days
```

The `+ 1` is there because `notAfter` is inclusive. A 90-day certificate ends one second before `notBefore + 90 days`. The third line shows the other half of the story. Since 15 March 2026 the CA/Browser Forum Baseline Requirements cap public certificates at 200 days, so even a commercial CA now issues a 198-day certificate. The cap falls to 100 days on 15 March 2027 and to 47 days on 15 March 2029.

2. Compute an ARI certificate ID and check it against the RFC

ARI identifies a certificate by two values it already carries. The first is the `keyIdentifier` from the Authority Key Identifier extension. The second is the serial number as DER INTEGER content bytes. Both are base64url-encoded with the `=` padding stripped and joined by a dot:

```
#!/usr/bin/env bash
# ari-id.sh - print the RFC 9773 certificate ID (AKI keyIdentifier "." serial) of a PEM cert
set -euo pipefail
cert=$(openssl x509)
b64url() { tr -d ':\n' | basenc -d --base16 | basenc --base64url | tr -d '=\n'; }   # hex in
aki=$(openssl x509 -noout -ext authorityKeyIdentifier <<<"$cert" | grep -oE '([0-9A-F]{2}:){
ser=$(openssl x509 -noout -serial <<<"$cert" | cut -d= -f2)
(( ${#ser} % 2 )) && ser=0$ser            # whole bytes
[[ $ser == [89A-F]* ]] && ser=00$ser       # DER INTEGER: keep it positive
echo "$(b64url <<<"$aki").$(b64url <<<"$ser")"
```

RFC 9773 Appendix A prints an example certificate and its expected ID, `aYhba4dGQEHhs3uEe6CuLN4ByNQ.AIdlQyE`. Save the PEM from the RFC as `rfc9773-example.pem` and test the script against it:

```
openssl x509 -in rfc9773-example.pem -noout -serial -ext authorityKeyIdentifier
./ari-id.sh < rfc9773-example.pem
```

```
serial=87654321
X509v3 Authority Key Identifier:
    69:88:5B:6B:87:46:40:41:E1:B3:7B:84:7B:A0:AE:2C:DE:01:C8:D4
aYhba4dGQEHhs3uEe6CuLN4ByNQ.AIdlQyE
```

The serial `87…` has its top bit set, so DER needs a leading `00` byte to keep it positive. That is why the second half is `AIdlQyE` and not `h2VDIQ`. Get this wrong and the CA answers 404 for every certificate whose serial starts with `8`–`F`.

3. Ask the CA when to renew

```
#!/usr/bin/env bash
# ari-check.sh HOST - ask the issuing ACME CA when HOST's current certificate should be rene
set -euo pipefail
host=$1
dir=${ACME_DIR:-https://acme-v02.api.letsencrypt.org/directory}
here=$(dirname "$0")
cert=$(openssl s_client -connect "$host:443" -servername "$host" </dev/null 2>/dev/null | op
id=$("$here/ari-id.sh" <<<"$cert")

epoch() { date -d "$1" +%s; }
nb=$(epoch "$(openssl x509 -noout -startdate <<<"$cert" | cut -d= -f2)")
na=$(epoch "$(openssl x509 -noout -enddate <<<"$cert" | cut -d= -f2)")
life=$(( na - nb + 1 ))                    # notAfter is inclusive: 90 days = 7776000 s

hdr=$(mktemp); trap 'rm -f "$hdr"' EXIT
ari=$(curl -fsS "$dir" | jq -r .renewalInfo)
resp=$(curl -fsS -D "$hdr" "$ari/$id")
ws=$(epoch "$(jq -r .suggestedWindow.start <<<"$resp")")
we=$(epoch "$(jq -r .suggestedWindow.end <<<"$resp")")

echo "host:      $host"
echo "lifetime:  $(( life / 86400 )) days ($(date -ud @"$nb" +%F) .. $(date -ud @"$na" +%F))
echo "cert ID:   $id"
echo "ARI:       $(jq -c . <<<"$resp")"
echo "window:    $(( (ws - nb) * 100 / life ))% .. $(( (we - nb) * 100 / life ))% of the lif
echo "re-check:  in $(sed -n 's/^[Rr]etry-[Aa]fter: *\([0-9]*\).*/\1/p' "$hdr") s (Retry-Aft
```

```
./ari-check.sh git.kernel.org; echo; ./ari-check.sh lwn.net
```

```
host:      git.kernel.org
lifetime:  90 days (2026-08-14 .. 2026-11-12)
cert ID:   uyDKRwv-1-Wc-Y8JKqOMNOWxvNg.BaF78x8x9qWdG1GW5dVwbT4j
ARI:       {"suggestedWindow":{"start":"2026-10-12T23:32:36Z","end":"2026-10-14T18:43:26Z"}}
window:    65% .. 67% of the lifetime
```

```
re-check:  in 22184 s (Retry-After)

host:      lwn.net
lifetime:  90 days (2026-08-29 .. 2026-11-27)
cert ID:   uVnyjs8i8IbTN0j_dhQYuoLYVYc.Bdz9qYoc56M9jE8dBhwiBLEn
ARI:       {"suggestedWindow":{"start":"2026-10-27T03:29:03Z","end":"2026-10-28T22:39:53Z"}}
window:    65% .. 67% of the lifetime
re-check:  in 18632 s (Retry-After)
```

Read it like this:

- The window sits at **65–67 %** of the lifetime. That is the "about two-thirds" Let's Encrypt asks for, chosen per certificate so that renewals do not all arrive at once.
- The `git.kernel.org` window opened at 05:02 IST on the day of the run. A well-behaved client renews that certificate within the next day and a half.
- `Retry-After` (about five to seven hours here) is how often the CA wants you to ask again. RFC 9773 treats it as both the minimum and the maximum. A CA that needs certificates replaced before a mass revocation moves the window into the past, and clients that poll on schedule renew at once.
- The client picks a uniformly random moment inside the window. A cron-driven client that wakes up only once a day may need to wake up more often to honour it (RFC 9773 §4.2).

4. What each renewal policy does as lifetimes shrink

```
#!/usr/bin/env bash
# renew-margin.sh - when does each renewal policy fire, and how much slack is left, per cert
awk 'BEGIN {
  n = split("90 64 45 6.67", life, " ")
  split("90d 64d 45d 160h", name, " ")
  printf "%-26s", "policy \\ lifetime"
  for (i = 1; i <= n; i++) printf "%14s", name[i]
  print ""
  np = split("fixed:30-days-left|fraction:1/3-left|interval:every-60-days|fraction:2/3-elaps
  for (p = 1; p <= np; p++) {
    printf "%-26s", pol[p]
    for (i = 1; i <= n; i++) {
      L = life[i]
      if (pol[p] ~ /30-days-left/)    at = L - 30
      if (pol[p] ~ /1\/3-left/)       at = (L <= 10) ? L / 2 : L * 2 / 3
      if (pol[p] ~ /every-60-days/)   at = 60
      if (pol[p] ~ /2\/3-elapsed/)    at = L * 2 / 3
      if (at < 0) at = 0
      slack = L - at
      if (slack < 0)       cell = sprintf("EXPIRED %.0fd", -slack)
      else if (at == 0)    cell = "every run"
      else                 cell = sprintf("d%.1f +%.1fd", at, slack)
```

```
      printf "%14s", cell
    }
    print ""
  }
}'
```

```
policy \ lifetime                    90d           64d           45d          160h
fixed:30-days-left          d60.0 +30.0d  d34.0 +30.0d  d15.0 +30.0d     every run
fraction:1/3-left           d60.0 +30.0d  d42.7 +21.3d  d30.0 +15.0d    d3.3 +3.3d
interval:every-60-days      d60.0 +30.0d   d60.0 +4.0d   EXPIRED 15d   EXPIRED 53d
fraction:2/3-elapsed        d60.0 +30.0d  d42.7 +21.3d  d30.0 +15.0d    d4.4 +2.2d
```

`dN +Sd` means "first attempt on day N, S days of slack left for retries". The first column is why nobody noticed: every policy agrees on a 90-day certificate. The rows explain what Let's Encrypt is warning about:

- **Fixed 30 days left** (certbot before 4.0, still the packaged version on Debian 12 and Ubuntu 24.04) survives 64- and 45-day certificates. It just renews earlier than needed. On a 160-hour certificate it is due on every run, which means twice a day with the stock timer, forever.
- **Fixed interval** is the one that fails. "Renew every two months" leaves four days of slack on a 64-day certificate. On a 45-day certificate the site sits expired for 15 days of every cycle. This is the `60` in Let's Encrypt's "grep for 83, 80 or 60".
- **Fractions** (certbot 4.0+, ARI) keep the same *shape* at every lifetime. The slack shrinks with the lifetime, so your alerting has to shrink with it too.

5. Monitor by fraction used, not days left

```
#!/usr/bin/env bash
# make-test-certs.sh - self-signed certificates at chosen points in their lives: NAME LIFETI
set -euo pipefail
mkdir -p certs
ts() { date -u -d "@$1" +%Y%m%d%H%M%SZ; }
now=$(date +%s)
while read -r name life age; do
  nb=$(( now - age * 86400 )); na=$(( nb + life * 86400 - 1 ))
  openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout /dev/null \
    -subj "/CN=$name.example" -not_before "$(ts $nb)" -not_after "$(ts $na)" -out "certs/$na
done <<'LIST'
web-90d-healthy  90 50
api-45d-healthy  45 26
cdn-90d-stuck    90 68
mail-64d-stuck   64 55
LIST
ls certs
```

```
#!/usr/bin/env bash
# cert-due.sh CERT.pem... - judge renewal health by fraction of lifetime used, not by days l
# exit 0 = all fine, 1 = renewal overdue (>= 75% used), 2 = critical (>= 90% used or expired
set -uo pipefail
warn=${WARN_PCT:-75} crit=${CRIT_PCT:-90} now=$(date +%s) worst=0
for f in "$@"; do
  nb=$(date -d "$(openssl x509 -in "$f" -noout -startdate | cut -d= -f2)" +%s)
  na=$(date -d "$(openssl x509 -in "$f" -noout -enddate | cut -d= -f2)" +%s)
  life=$(( na - nb + 1 )) used=$(( (now - nb) * 100 / life )) left=$(( (na - now) / 86400 ))
  if   (( used >= crit )); then st=CRIT rc=2
  elif (( used >= warn )); then st=WARN rc=1
  else st=OK rc=0; fi
  (( rc > worst )) && worst=$rc
  printf '%-4s %-26s life=%3dd used=%3d%% left=%3dd\n' "$st" "$(basename "$f")" $(( life / 8
done
exit $worst
```

`openssl req -not_before/-not_after` (OpenSSL 3.4 and later) lets you back-date the test certificates, so you can test monitoring without waiting weeks:

```
./make-test-certs.sh
./cert-due.sh certs/*.pem; echo "exit=$?"
for f in certs/*.pem; do printf '%-22s ' "$(basename "$f")"; openssl x509 -in "$f" -noout -c
```

```
api-45d-healthy.pem
cdn-90d-stuck.pem
mail-64d-stuck.pem
web-90d-healthy.pem
OK   api-45d-healthy.pem        life= 45d used= 57% left= 18d
WARN cdn-90d-stuck.pem          life= 90d used= 75% left= 21d
WARN mail-64d-stuck.pem         life= 64d used= 85% left=  8d
OK   web-90d-healthy.pem        life= 90d used= 55% left= 39d
exit=1
api-45d-healthy.pem    Certificate will expire
cdn-90d-stuck.pem      Certificate will not expire
mail-64d-stuck.pem     Certificate will expire
web-90d-healthy.pem    Certificate will not expire
```

Compare the two verdicts. The classic "alert at 21 days left" check (`-checkend` with 21 days) gets two of four wrong. It pages for `api-45d-healthy`, which is only 57 % through its life and not even due for renewal. It stays silent for `cdn-90d-stuck`, whose renewal should have happened eight days ago. The fraction check gets both right with one threshold for every lifetime. 75 % is a choice, not a standard. It sits one-twelfth of the lifetime after the 2/3 renewal point, which is about five days on a 64-day certificate and four on a 45-day one: enough for a few failed attempts, early enough to fix by hand.

Point the same check at the **served** certificate, not only the file on disk (see the trap):

```
openssl s_client -connect shop.example:443 -servername shop.example </dev/null 2>/dev/null |
./cert-due.sh served.pem
```

6. Audit a host for hard-coded lifetimes

```
#!/usr/bin/env bash
# renewal-audit.sh [ROOT...] - find renewal logic that assumes a fixed certificate lifetime
set -uo pipefail
roots=("$@"); (( $# )) || roots=(/etc /usr/local/bin /opt)
hit() { grep -rnIE -e "$2" "${roots[@]}" 2>/dev/null | sed "s|^|[$1] |"; }

hit fixed-threshold 'renew_before_expiry *= *[0-9]+ *days?'
hit forced-interval '--force-renewal|--renew-by-default'
hit forced-interval '^[^#]*[0-9*]+ +[0-9*]+ +[0-9*]+ +\*/[0-9]+ +'     # cron: every N month
hit absolute-alert  '-checkend +[0-9$]|check_http.* -C *[0-9]+|check_ssl_cert.* -[wc] *[0-9]
hit magic-number    '\b(30|60|80|83|89|90) *days?\b|is within [0-9]+ days'

if command -v certbot >/dev/null; then
  v=$(certbot --version 2>&1 | awk '{print $2}')
  printf '[client] certbot %s: ' "$v"
  if   dpkg --compare-versions "$v" lt 4.0; then echo "fixed 30-day threshold, no ARI"
  elif dpkg --compare-versions "$v" lt 4.1; then echo "renews at 1/3 lifetime left, no ARI"
  else echo "ARI-aware"; fi
else
  echo "[client] certbot not on PATH here (check the hosts that renew)"
fi
```

To see it work without touching a real host, build a small fake `/etc` with the habits this page warns about. The `cron.d/certbot` and `certbot.timer` contents are taken (abridged) from Debian 13's own certbot 4.0.0 package:

```
#!/usr/bin/env bash
# make-fixture.sh - a fake /etc with the renewal habits this chapter warns about
set -euo pipefail
r=fixture/etc; rm -rf fixture; mkdir -p $r/cron.d $r/systemd/system $r/letsencrypt/renewal $

# shipped by Debian 13's certbot 4.0.0 package (abridged): note the stale comment
cat > $r/cron.d/certbot <<'X'
# Renewal will only occur if expiration
# is within 30 days.
0 */12 * * * root test -x /usr/bin/certbot -a \! -d /run/systemd/system && perl -e 'sleep in
X
cat > $r/systemd/system/certbot.timer <<'X'
[Timer]
OnCalendar=*-*-* 00,12:00:00
```

```
RandomizedDelaySec=43200
Persistent=true
X
# a lineage someone pinned years ago
cat > $r/letsencrypt/renewal/shop.example.conf <<'X'
version = 2.9.0
renew_before_expiry = 30 days
[renewalparams]
authenticator = webroot
server = https://acme-v02.api.letsencrypt.org/directory
X
# a hand-rolled "renew every two months" job
cat > $r/cron.d/legacy-tls <<'X'
15 3 1 */2 * root /usr/local/bin/renew-all.sh
X
cat > fixture/usr/local/bin/renew-all.sh <<'X'
#!/bin/sh
certbot renew --force-renewal --quiet && systemctl reload nginx
X
# a monitoring check with an absolute threshold
cat > $r/nagios/conf.d/tls.cfg <<'X'
command_line  /usr/lib/nagios/plugins/check_http -H $HOSTADDRESS$ -C 21,7
X
find fixture -type f | sort
```

```
./make-fixture.sh && ./renewal-audit.sh fixture
```

```
fixture/etc/cron.d/certbot
fixture/etc/cron.d/legacy-tls
fixture/etc/letsencrypt/renewal/shop.example.conf
fixture/etc/nagios/conf.d/tls.cfg
fixture/etc/systemd/system/certbot.timer
fixture/usr/local/bin/renew-all.sh
[fixed-threshold] fixture/etc/letsencrypt/renewal/shop.example.conf:2:renew_before_expiry =
[forced-interval] fixture/usr/local/bin/renew-all.sh:2:certbot renew --force-renewal --quiet
[forced-interval] fixture/etc/cron.d/legacy-tls:1:15 3 1 */2 * root /usr/local/bin/renew-all
[absolute-alert] fixture/etc/nagios/conf.d/tls.cfg:1:command_line  /usr/lib/nagios/plugins/c
[magic-number] fixture/etc/cron.d/certbot:2:# is within 30 days.
[magic-number] fixture/etc/letsencrypt/renewal/shop.example.conf:2:renew_before_expiry = 30
[client] certbot not on PATH here (check the hosts that renew)
```

How to act on each class of finding:

Finding	What to do
`fixed-threshold`	Delete `renew_before_expiry` from the lineage and let certbot 4.0+ use 1/3 (and ARI on 4.1+). Keep it only if you have a reason, written down next to it
`forced-interval`	Replace with plain `certbot renew` run at least twice a day. `renew` is idempotent: it does nothing until a certificate is due
`absolute-alert`	Rewrite as a fraction of lifetime (`cert-due.sh`), or derive the days from the actual lifetime
`magic-number`	Read each one. The Debian comment above is harmless but stale: the packaged 4.0.0 renews at 1/3, not 30 days. Stale runbooks cause bad 3 a.m. decisions
`[client]` older than 4.1	Plan the move to an ARI-aware client before 2027-02-10, through the upstream snap, a backport, or another ACME client that documents ARI support

The stock timer (`00,12:00` plus up to 12 hours of random delay, `Persistent=true`) is already right for every lifetime on this page. Leave it alone. Fix the things that override it.

The trap

The certificate was renewed. The service is still serving the old one.

`certbot renew` writes new files under `/etc/letsencrypt/live/`. nginx, HAProxy, Postfix and most other daemons read the certificate once at start or reload. Without a deploy hook nothing tells them, and the success shows only in the renewal log:

```
#!/usr/bin/env bash
# stale-serve.sh - the certificate on disk is renewed; the one being served is not
set -euo pipefail
d=$(mktemp -d); trap 'kill $srv 2>/dev/null; rm -rf "$d"' EXIT
issue() {  # issue SERIAL: write a fresh 45-day cert and key to $d/live/
  mkdir -p "$d/live"
  openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -days 45 -set_serial
    -subj /CN=shop.example -keyout "$d/live/privkey.pem" -out "$d/live/fullchain.pem" 2>/dev
}
served() { openssl s_client -connect 127.0.0.1:8443 </dev/null 2>/dev/null | openssl x509 -n

issue 0x1001
openssl s_server -quiet -accept 8443 -cert "$d/live/fullchain.pem" -key "$d/live/privkey.pem
sleep 1
echo "on disk: $(openssl x509 -in "$d/live/fullchain.pem" -noout -serial)   served: $(served
```

```
issue 0x1002                                        # "certbot renew" succeeded
echo "on disk: $(openssl x509 -in "$d/live/fullchain.pem" -noout -serial)   served: $(served

kill $srv; wait $srv 2>/dev/null || true       # the reload a deploy hook would have done
openssl s_server -quiet -accept 8443 -cert "$d/live/fullchain.pem" -key "$d/live/privkey.pem
sleep 1
echo "on disk: $(openssl x509 -in "$d/live/fullchain.pem" -noout -serial)   served: $(served
```

```
on disk: serial=1001   served: serial=1001
on disk: serial=1002   served: serial=1001
on disk: serial=1002   served: serial=1002
```

With 90-day certificates this bug hid behind routine work. Renewal at day 60 left 30 days, and some unrelated deploy, package upgrade or reboot almost always restarted the daemon in time. With a 45-day certificate renewed at day 30, the old certificate in memory has 15 days left. A host that reloads only on a monthly patch night serves an expired certificate in about half of all cycles, while every file-based check reports green.

The fix is a **deploy hook**, which runs only after a successful renewal. Put an executable in `/etc/letsencrypt/renewal-hooks/deploy/` (for example one that runs `systemctl reload nginx`), or pass `--deploy-hook` once so certbot stores it in the lineage. Then monitor the served certificate as in example 5. Probe from outside the host and against every name and port that terminates TLS, including the load balancer and the mail server on 465 and 993.

Notes & Pitfalls

Pitfall	Symptom / fix
Hard-coded `--days 60` style intervals	Fine on 90-day certificates. Four days of slack on 64-day ones, outages on 45-day ones. Use the client's due-check instead
Alert thresholds in absolute days	Pages for healthy short certificates, silent for stuck long ones. Alert on fraction used
Watching the file, not the socket	Green dashboards, browser errors. Probe the served certificate
`--required-profile` without a fallback plan	Renewal *fails* if the CA stops offering that profile. `--preferred-profile` falls back to the default. Choose deliberately
Relying on authorization reuse	From 2027-02-10 a validation is reusable for 10 days, not 30. Pre-validating once a month and issuing later no longer works
Expecting OCSP to tell you about revocation	Let's Encrypt turned off its OCSP responders on 2025-08-06. ARI is how the CA tells your client to replace a certificate early
ARI 404s on some certificates	Serial encoding: DER needs a leading `00` when the top bit is set (example 2)

Pitfall	Symptom / fix
Testing against production	Use the staging directory. It switches to 64-day certificates on 2026-10-14, four months before production

The boring rule

> Renew with a client that decides for itself when a certificate is due, preferably via ARI, and run it twice a day. Never schedule renewals by calendar or by days-before-expiry you typed yourself. Reload every consumer from a deploy hook. Alert on the **fraction of lifetime used by the certificate actually served**. Then the CA can shorten lifetimes again without you changing anything.

Related Commands

Command	Role
`openssl x509 -noout -dates -serial -ext authorityKeyIdentifier`	Lifetime and ARI identity of a certificate
`openssl s_client -connect host:443 -servername host`	Fetch the served certificate
`curl + jq`	ACME directory, profiles, ARI `renewalInfo`
`systemctl list-timers certbot.timer`	When renewal next runs
`journalctl -u certbot.service`	What the last renewal attempts did
`grep -rnE`	The audit itself

Try this

1. Run `ari-check.sh` against every public name you operate. Put the next `suggestedWindow.start` for each one in a calendar, and check that your client actually renews inside the window.
2. On a staging account after 2026-10-14, issue a certificate with your normal client and confirm with `cert-life.sh` (or `openssl x509 -dates`) that it is 64 days. Then run your monitoring against it.
3. Run `renewal-audit.sh` with `sudo` on one real host. For every hit, write down whether it breaks at 64 days, at 45 days, or never.
4. Add a deploy hook to one host, renew with `certbot renew --force-renewal --cert-name <name>` in a maintenance window, and use `openssl s_client` to prove the served serial changed without a manual restart.
5. Change `cert-due.sh` so it reads `host:port` arguments directly, and wire it into your monitoring with exit codes 1 and 2 as warning and critical.

Sources

- Let's Encrypt, 64-Day Certificate Lifetimes Coming Feb 2027 (2026-10-07): production date 2027-02-10, staging date 2026-10-14, renew at about 2/3, grep for 83, 80 or 60, authorization reuse from 30 to 10 days, last 90-day certificate expiring around 2027-05-11
- Let's Encrypt, Decreasing Certificate Lifetimes to 45 Days (2025-12-02): full timeline, `tlsserver` from 2026-05-13, `classic` 45 days from 2028-02-16
- Let's Encrypt, Profiles (updated 2026-10-04): `classic`, `tlsserver`, `shortlived` properties, Baseline Requirements limits
- CA/Browser Forum, Ballot SC-081v3: 200 days (2026-03-15), 100 days (2027-03-15), 47 days (2029-03-15), validation reuse 200, 100 and then 10 days
- RFC 9773, ACME Renewal Information (ARI): certificate ID construction (§4.1, Appendix A), `suggestedWindow`, `Retry-After` semantics, client algorithm
- RFC 8555, ACME
- Certbot, CHANGELOG (4.0.0 1/3 rule and profiles, 4.1.0 ARI, 5.0.0 stored `Retry-After`, 5.8.0 current) and User Guide (renewal threshold, `--deploy-hook`, `renewal-hooks/deploy`)
- Debian package `python-certbot` 4.0.0-2+deb13u1, `debian/certbot.timer` and `debian/certbot.cron.d`; package versions from sources.debian.org and Launchpad (`python-certbot` in noble and resolute)
- OpenSSL 3.5 `openssl-x509(1)` and `openssl-req(1)` (`-checkend`, `-not_before`, `-not_after`)

poll vs epoll vs io_uring

poll vs epoll vs io_uring

Overview

Thesis: Linux gives you three generations of "wait for I/O" APIs. `poll` (and older `select`) ask "which of these fds are ready?" by rescanning your whole interest list every call — fine for a handful of descriptors, painful for thousands. `epoll` keeps the interest list in the kernel and returns only the ready set — the workhorse of Linux network servers. `io_uring` flips the model from *readiness* to *completion*: you submit work (SQEs), the kernel posts results (CQEs), and you can batch syscalls. Pick the oldest API that fits; do not reach for io_uring to read one config file.

This page runs small C labs with **gcc**, **poll(2)**, **epoll(7)**, and **liburing** already present on the box. No packages were installed for these examples.

Mental model

```
readiness (poll / epoll):     "tell me when I MAY read/write"
     app                     kernel
      |-- wait ------------->|
      |<-- fd ready --------|
      |-- read(fd) -------->|   separate syscall for the I/O

completion (io_uring):        "do this I/O; tell me when DONE"
     app                     kernel
      |-- SQE: read(fd,buf)->|
      |<-- CQE: res=n -------|   I/O already finished (or failed)
```

API	Model	Cost scales with	Typical fit
`poll`	readiness; user passes fd array each call	**O(n)** fds watched per wait	small sets, portable code, quick prototypes
`epoll`	readiness; interest list lives in the kernel	**O(ready)** events returned	many sockets / pipes on Linux
`io_uring`	completion via SQ/CQ rings	submissions & completions (batchable)	high IOPS, syscall reduction, advanced storage/net

Fact	Consequence
`poll` copies and scans the fd list every wait	10k idle connections 10k checks even if nothing is ready
`epoll` interest list is stateful (`epoll_ctl`)	Add once; wait cheaply; mind level vs edge trigger
Edge trigger (`EPOLLET`) fires on state *change*	You must drain until `EAGAIN` or you miss data
io_uring posts completions, not readiness	One submission can include the read itself — fewer syscalls
Not every workload is "10k connections"	`poll` remains the right tool for two fds and a timeout

Syntax

```
poll(fds, nfds, timeout_ms);                    /* man 2 poll */
epoll_create1(flags);
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &event);     /* man 7 epoll */
epoll_wait(epfd, events, maxevents, timeout_ms);

io_uring_queue_init(entries, &ring, flags);     /* liburing / man 2 io_uring_setup */
io_uring_get_sqe(&ring);  io_uring_prep_*(sqe, ...);
io_uring_submit(&ring);
io_uring_wait_cqe(&ring, &cqe);  io_uring_cqe_seen(&ring, cqe);
```

Worked examples

Tested on Debian-like linux/amd64 with **GCC 14.2.0**, **strace 6.13**, and **liburing 2.9** headers/libs already on the system (October 2026). Exact `data.fd` numbers depend on prior opens; the control flow is what matters.

Case 1: `poll` with a timeout (portable readiness)

```
/* poll_stdin.c - readiness with poll(2) */
#include <poll.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    struct pollfd pfd = {.fd = STDIN_FILENO, .events = POLLIN};
    printf("polling stdin for 200ms...\n");
```

```
    int n = poll(&pfd, 1, 200);
    if (n < 0) { perror("poll"); return 1; }
    if (n == 0) {
        printf("timeout (no input)\n");
        return 0;
    }
    printf("revents=0x%x POLLIN=%d\n", pfd.revents, !!(pfd.revents & POLLIN));
    return 0;
}
```

```
gcc -O2 -o poll_stdin poll_stdin.c
./poll_stdin
```

```
polling stdin for 200ms...
timeout (no input)
```

With one fd this is trivial. With thousands of mostly idle fds, every `poll` still walks the whole array — that is the scaling cliff the epoll design removed.

Case 2: `epoll` edge-triggered on a pipe

```
/* epoll_pipe.c - edge-triggered readiness on a pipe */
#include <sys/epoll.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main(void)
{
    int fds[2];
    if (pipe(fds) < 0) { perror("pipe"); return 1; }

    int ep = epoll_create1(0);
    if (ep < 0) { perror("epoll_create1"); return 1; }

    struct epoll_event ev = {.events = EPOLLIN | EPOLLET, .data.fd = fds[0]};
    if (epoll_ctl(ep, EPOLL_CTL_ADD, fds[0], &ev) < 0) {
        perror("epoll_ctl"); return 1;
    }

    const char *msg = "hi";
    if (write(fds[1], msg, strlen(msg)) < 0) { perror("write"); return 1; }

    struct epoll_event out;
    int n = epoll_wait(ep, &out, 1, 500);
```

```
    printf("epoll_wait returned %d events=0x%x data.fd=%d\n",
           n, n > 0 ? out.events : 0, n > 0 ? out.data.fd : -1);

    char buf[8];
    ssize_t r = read(fds[0], buf, sizeof buf);
    printf("read %zd bytes\n", r);

    /* Edge-triggered + buffer empty  second wait times out */
    n = epoll_wait(ep, &out, 1, 100);
    printf("second epoll_wait returned %d (expect 0 timeout)\n", n);

    close(ep); close(fds[0]); close(fds[1]);
    return 0;
}
```

```
gcc -O2 -o epoll_pipe epoll_pipe.c
./epoll_pipe
```

```
epoll_wait returned 1 events=0x1 data.fd=5
read 2 bytes
second epoll_wait returned 0 (expect 0 timeout)
```

`EPOLLIN` is `0x1`. After the read drains the pipe, edge-triggered epoll stays quiet until new data arrives. Level-triggered (`EPOLLIN` without `EPOLLET`) would keep reporting readiness while data remains — easier to get right, slightly more wakeups.

Case 3: `io_uring` completion with a NOP

```
/* uring_nop.c - completion model: submit NOP, wait for CQE */
#include <liburing.h>
#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main(void)
{
    struct io_uring ring;
    int ret = io_uring_queue_init(8, &ring, 0);
    if (ret < 0) {
        fprintf(stderr, "io_uring_queue_init: %s\n", strerror(-ret));
        return 1;
    }

    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
    if (!sqe) { fprintf(stderr, "no SQE\n"); return 1; }
```

```
    io_uring_prep_nop(sqe);
    io_uring_sqe_set_data(sqe, (void *)(uintptr_t)42);

    ret = io_uring_submit(&ring);
    printf("submitted %d\n", ret);

    struct io_uring_cqe *cqe;
    ret = io_uring_wait_cqe(&ring, &cqe);
    if (ret < 0) {
        fprintf(stderr, "wait_cqe: %s\n", strerror(-ret));
        return 1;
    }
    printf("cqe res=%d user_data=%lu\n",
           cqe->res, (unsigned long)(uintptr_t)io_uring_cqe_get_data(cqe));
    io_uring_cqe_seen(&ring, cqe);
    io_uring_queue_exit(&ring);
    return 0;
}
```

```
gcc -O2 -o uring_nop uring_nop.c -luring
./uring_nop
```

```
submitted 1
cqe res=0 user_data=42
```

A NOP does no I/O; it proves the SQ→CQ path. Real programs replace `io_uring_prep_nop` with `prep_read` / `prep_write` / `prep_accept` / … and often submit batches before collecting CQEs. See Jens Axboe's io_uring.pdf for ring layout and ordering rules.

Case 4: When to stop climbing the ladder

Situation	Prefer
Two fds, script, teaching tool	`poll`
Linux service with many sockets	`epoll` (level trigger first)
Need portability to BSD/macOS	`poll` or `kqueue` (not epoll/io_uring)
Syscall overhead or storage IOPS dominates	`io_uring`
One blocking `read` in a simple CLI	plain `read` — skip all of the above

The trap

1. **Edge-triggered epoll without draining to `EAGAIN`.** You get one wakeup, leave bytes in the socket buffer, and hang waiting for an edge that never comes.

2. **Using io_uring because it is new.** Setup cost, CQ overflow handling, and restricted kernels (containers, older LTS) bite beginners; epoll still runs the internet.

3. **Confusing readiness with completion.** `epoll` saying "readable" does not mean your `read` finished — you still issue the read (unless you moved that work into an io_uring SQE).

The boring rule

Start with `poll` for small fd sets. Move to `epoll` when the fd count or wake rate hurts. Use `io_uring` when you have measured syscall or completion-batching benefits — and you are willing to handle ring lifecycle carefully. Always match trigger mode (level vs edge) to how thoroughly you drain.

Try this

1. Compile and run the three listings above; confirm timeout / one-shot / CQE behaviour.

2. `strace -e poll ./poll_stdin` and `strace -e epoll_wait,epoll_ctl ./epoll_pipe` — see the syscalls.

3. Change `epoll_pipe.c` to level trigger (drop `EPOLLET`) and call `epoll_wait` twice *before* reading; observe repeated readiness.

4. Replace the NOP in `uring_nop.c` with `io_uring_prep_read` on a pipe end and print `cqe->res`.

Sources

- `man 2 poll`, `man 7 epoll`, `man 2 io_uring_setup` / `io_uring_enter` / `io_uring_register`
- Jens Axboe, *io_uring*: https://kernel.dk/io_uring.pdf
- Linux kernel docs: `Documentation/io_uring/` (in-tree)
- Inspiration: https://x.com/straceX/status/2049810663409680487

Scheduling

Intro

Intro

Run work later or periodically. Classic `cron`/`crontab`/`anacron`/`at` remain everywhere; on systemd hosts also consider timers (`systemctl list-timers`, `systemd-run`).

Commands in this part

Command	Role
`cron`	cron is the classic daemon that runs scheduled jobs from per-user and system crontab files.
`crontab`	crontab installs, lists, and removes per-user crontab files that the cron daemon executes.
`anacron`	anacron runs periodic commands on machines that are not on 24/7 — laptops, desktops, and intermittent VMs.
`at`	at schedules a one-shot command to run once at a future time.

Suggested starting points

1. User jobs: `crontab -e` / `crontab -l`.
2. Daemon and system tables: `cron`, `/etc/cron.d`.
3. Machines that sleep: `anacron`.
4. One-shot future jobs: `at`.
5. Modern alternative: systemd timers (Services and runtime).

Related parts

- Services and runtime — `systemd-run`, timers
- Logging — capture job output with `logger`/`journalctl`
- Shell commands — reliable one-liners for jobs

Continue with the individual command pages in this part.

cron

Overview

`cron` is the classic daemon that runs scheduled jobs from per-user and system crontab files. On modern Ubuntu, **systemd timers** are often a better fit for service-integrated jobs, but cron remains everywhere: shared hosting, legacy scripts, and simple user schedules. This page covers how the daemon interprets schedules and where jobs live; day-to-day editing is usually via `crontab`.

Syntax

Jobs are **not** started as `cron …`. You install lines into a crontab; `cron` reads them.

Crontab schedule fields:

```
* * * * * command

         day of week (0-7, Sun=0 or 7; some versions allow names)
        month (1-12)
       day of month (1-31)
      hour (0-23)
     minute (0-59)
```

System drop-in files under `/etc/cron.d/` use an **extra username field** after the schedule:

```
* * * * * root /usr/local/bin/job.sh
```

Common schedule patterns

Expression	Meaning
`*/15 * * * *`	Every 15 minutes
`0 * * * *`	Top of every hour
`0 3 * * *`	Daily at 03:00
`0 9 * * 1-5`	Weekdays 09:00
`0 0 1 * *`	Monthly, 1st at midnight
`@reboot`	Once at cron startup (after boot)
`@daily` / `@weekly` / `@hourly`	Nicknames (implementation-specific paths)

Special characters: `*` any, `,` list, `-` range, `/` step. Extensions like `L`, `W`, `#` are **not** portable — stick to the five standard fields unless you know your cron flavor.

Key Use Cases

1. Periodic backups and reports
2. Log or temp cleanup
3. Polling scripts that are not worth a long-running daemon
4. User-level schedules without writing systemd units

Safety

- Jobs run with a **minimal environment** — always use absolute paths and set `PATH`/`SHELL` in the crontab when needed.
- Mail/`MAILTO` can flood or leak output; redirect stdout/stderr intentionally.
- Never put secrets on the command line in a world-readable system crontab; use root-owned scripts with `0600` or secret managers.
- Overlapping long jobs: guard with `flock` so two copies do not run at once.

Examples with Explanations

User crontab (via crontab tool)

```
crontab -l
crontab -e
```

`cron` executes the commands as that user. See the `crontab` chapter for flags.

Example job lines

```
# m h dom mon dow command
0 3 * * * /usr/local/bin/backup.sh >>/var/log/backup.log 2>&1
*/15 * * * * /usr/bin/flock -n /tmp/poll.lock /usr/local/bin/poll.sh
@reboot /usr/local/bin/start-tunnel.sh
```

Redirects capture output; `flock -n` skips a run if the previous still holds the lock.

System-wide locations (Ubuntu)

Path	Role
`/etc/crontab`	System crontab (has user field)
`/etc/cron.d/*`	Drop-in packages/admin jobs (user field)
`/etc/cron.daily/` etc.	Scripts run by `run-parts` on a schedule
`/var/spool/cron/crontabs/`	Per-user crontabs (do not edit by hand)

```
ls /etc/cron.d
ls /etc/cron.daily
```

Environment inside cron

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=you@example.com
0 4 * * * /usr/local/bin/nightly.sh
```

Variables at the top of a crontab apply to later lines. `HOME` is usually the user's home; do not assume interactive aliases.

Debugging "it didn't run"

```
# Is cron running?
systemctl status cron          # Debian/Ubuntu package name
systemctl status crond         # some RHEL-like systems

# Logs (paths vary)
journalctl -u cron -n 50 --no-pager
grep CRON /var/log/syslog | tail

# Syntax / permission issues
crontab -l
ls -l /usr/local/bin/nightly.sh   # must be executable; readable by the job user
```

Common failures: relative paths, missing `+x`, script expects a TTY, job runs as wrong user, anacron vs laptop sleep (see `anacron`).

Prefer systemd timers when…

```
systemctl list-timers --all
# unit pair: foo.service + foo.timer
```

Timers give dependency ordering, better logging via the journal, and clearer failure states. Use cron when you need a portable one-liner or cannot install units.

Understanding Output

Successful jobs are silent unless they print. By default many crons mail any stdout/stderr to the user (`MAILTO`). If mail is unconfigured, output may vanish into a local mailbox or be discarded — **always log explicitly** for important jobs.

Notes & Pitfalls

- Day-of-month **and** day-of-week both set: meaning is "either matches" on Vixie/cronie (easy to schedule more often than intended).
- Timezone is the system timezone (`timedatectl`); DST transitions can skip or double an hour.
- Laptops/desktops asleep at the scheduled minute **miss** the job (use anacron or timers with `Persistent=true`).
- Percent signs `%` in commands are special in some crons (newline to stdin) — escape as `\%`.
- SELinux/AppArmor may deny scripts that work interactively.

Related Commands

- `crontab` — install and list user crontabs
- `anacron` — catch up periodic jobs on machines that sleep
- `at` — one-shot future jobs
- `systemctl` / timers — modern alternative
- `flock` — mutual exclusion for long jobs
- `logger` — send job messages to syslog/journal

Additional Resources

- `man cron` / `man 5 crontab`
- crontab.guru — schedule explainer

crontab

Overview

`crontab` installs, lists, and removes **per-user** crontab files that the `cron` daemon executes. Prefer `crontab -e` over editing spool files by hand. System-wide jobs live under `/etc/cron.d` and friends — see the `cron` chapter.

Syntax

```
crontab [-u user] file
crontab [-u user] [-l | -r | -e] [-i]
```

Common Options

Option	Description
`-l`	List current crontab
`-e`	Edit current crontab in `$EDITOR` / `$VISUAL`
`-r`	Remove entire crontab
`-i`	Prompt before `-r`
`-u user`	Operate on another user's crontab (**root**)

Key Use Cases

1. Schedule periodic tasks as your own user
2. Install a crontab from a file in config management
3. Audit what a user has scheduled
4. Clean up obsolete jobs safely

Safety

- `crontab -r` deletes **all** of that user's jobs without a backup — use `-i` or list first.
- Installing from a file replaces the whole crontab; keep the source in git.
- Root using `-u` affects another account's automation — verify the user name.

Examples with Explanations

List and edit

```
crontab -l
EDITOR=vim crontab -e
```

Empty crontab may print `no crontab for user` (exit status can be non-zero depending on version).

Install from a file (idempotent in config mgmt)

```
cat > /tmp/my.cron <<'EOF'
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin
MAILTO=""
*/15 * * * * /usr/local/bin/poll.sh >>$HOME/logs/poll.log 2>&1
0 3 * * * /usr/local/bin/backup.sh >>$HOME/logs/backup.log 2>&1
EOF
crontab /tmp/my.cron
crontab -l
```

Redirects and absolute paths avoid "worked in SSH, failed in cron".

Append a line carefully

```
(crontab -l 2>/dev/null; echo '0 4 * * 0 /usr/local/bin/weekly.sh') | crontab -
```

Race-prone if two installers run at once — prefer full-file installs from CM.

Remove with confirmation

```
crontab -l > ~/crontab.backup.$(date +%F)
crontab -i -r
```

Always backup before delete.

Root manages another user

```
sudo crontab -u deploy -l
sudo crontab -u deploy -e
```

Validate schedule fragments

```
# After editing, re-list and reason about times:
crontab -l
# Cross-check system timers too:
systemctl list-timers --all
```

Common job examples

```
# every 15 minutes
*/15 * * * * /usr/bin/flock -n /tmp/poll.lock /usr/local/bin/poll.sh

# weekdays 09:00
0 9 * * 1-5 /usr/local/bin/report.sh

# after reboot
@reboot /usr/local/bin/start-tunnel.sh
```

See `cron` for field semantics, environment, and debugging.

Understanding Output

`crontab -l` prints the file body (comments and environment assignments included). Editors invoked by `-e` must exit zero for the new file to be installed; syntax is checked lightly — a bad command still "installs".

Notes & Pitfalls

- Per-user crontabs do **not** include a username field (unlike `/etc/crontab`).
- `%` is special in some cron implementations — escape as `\%` in commands.
- SELinux may block scripts that work in an interactive shell.
- Desktop users: session-dependent jobs often need systemd user timers + lingering, not cron.
- `crontab -e` uses `select-editor` on first run (Debian/Ubuntu).

Related Commands

- `cron` — daemon and system locations
- `anacron` — catch-up on machines that sleep
- `at` — one-shot schedules
- `systemctl list-timers` — systemd alternative
- `flock` — prevent overlapping runs

Additional Resources

- `man 1 crontab`
- `man 5 crontab`
- crontab.guru

anacron

Overview

`anacron` runs periodic commands on machines that are **not on 24/7** — laptops, desktops, and intermittent VMs. Unlike `cron`, which expects the system to be up at a specific minute, anacron ensures daily/weekly/monthly jobs run when the machine is powered on after a missed window.

On many modern distros, anacron is integrated with **systemd timers** and `/etc/cron.{daily,weekly,monthly}` via `cron`/`anacron` packages.

Syntax

```
anacron [options] [job]
anacron -T                    # test tab syntax (when supported)
```

Common Options

Option	Description
`-s`	Serialize jobs; run sequentially
`-f`	Force run ignoring timestamps
`-n`	Run jobs now, ignore delays
`-d`	Debug to stderr; don't background
`-q`	Quiet with `-d`
`-t file`	Use alternate anacrontab
`-S dir`	Spool directory for timestamps
`-u`	Update timestamps without running

Configuration

Primary table: `/etc/anacrontab`

Typical fields:

```
period  delay  job-id  command
1       5      cron.daily    run-parts --report /etc/cron.daily
7       10     cron.weekly   run-parts --report /etc/cron.weekly
@monthly 15    cron.monthly  run-parts --report /etc/cron.monthly
```

Timestamps live under `/var/spool/anacron/`.

Examples with Explanations

Status / integration

```
cat /etc/anacrontab
ls /var/spool/anacron/
systemctl status anacron.service anacron.timer 2>/dev/null
systemctl list-timers | grep -i cron
```

Force a run (careful)

```
sudo anacron -f -d
sudo anacron -n -d cron.daily
```

`-d` keeps output in the foreground for debugging.

Update timestamps only

```
sudo anacron -u
```

Marks jobs as done without executing — use sparingly.

Put a job in daily.d style

```
sudo install -m 755 /usr/local/sbin/backup-homes /etc/cron.daily/backup-homes
# anacron/cron.daily will pick it up on next daily pass
```

Prefer systemd timer for new work

```
# example unit pair
# /etc/systemd/system/mybackup.service
# /etc/systemd/system/mybackup.timer
sudo systemctl enable --now mybackup.timer
```

```
systemctl list-timers
```

Notes / Pitfalls

- Random **delay** field prevents thundering herds when many laptops resume together.
- Jobs must be safe to run after multi-day gaps (catch-up logic).
- Overlap: don't also schedule the same heavy job at a fixed cron minute without coordination.
- Spool timestamps require writable `/var`; read-only root systems need alternate design.
- Logging often via syslog/journal — check `journalctl -u anacron`.

2026-relevant notes

- systemd timers with `Persistent=true` cover many anacron use cases on modern hosts.
- Still relevant for classic `/etc/cron.daily` ecosystems on Debian/Ubuntu desktops.
- Cloud fleets that auto-scale should use durable queues/schedulers, not host-local anacron alone.

Related Commands

- `cron` / `crontab` — time-based schedules
- `systemd-run` / `.timer` units — modern scheduling
- `run-parts` — run scripts in a directory
- `journalctl` — logs
- `at` — one-shot future commands

Additional Resources

- `man anacron`, `man anacrontab`

at

Overview

`at` schedules a **one-shot** command to run once at a future time. Unlike `cron` (recurring) or systemd timers (rich units), `at` is ideal for "run this once tonight" admin tasks. Companion tools: `atq` (queue), `atrm` (remove), `batch` (run when load is low).

Requires `atd` service running and permission via `/etc/at.allow` / `/etc/at.deny`.

Syntax

```
at [-q queue] [-f file] [-m] TIME
atq
atrm JOB...
batch
```

Common Options

Option	Description
`-f file`	Read job from file
`-m`	Mail when done (even if no output)
`-M`	Never mail
`-q queue`	Queue letter (`a` default; higher letters later)
`-t time`	Time as `[[CC]YY]MMDDhhmm[.ss]`
`-v`	Show time job will run
`-c job`	Cat job contents
`-l`	Alias for `atq` on some systems
`-d`	Alias for `atrm` on some systems

Time expressions

```
now + 30 minutes
now + 2 hours
teatime          # 16:00
noon
midnight
23:30
10am tomorrow
4pm + 3 days
```

Examples with Explanations

Interactive job

```
at now + 1 hour
# then type commands, finish with Ctrl-D
echo "hello from at" >> /tmp/at.out
systemctl restart demo
^D
```

One-liner from pipe

```
echo 'systemctl restart nginx' | sudo at now + 5 minutes
echo 'wall "maintenance over"' | at 22:00
```

From file

```
cat > /tmp/job.sh <<'EOF'
#!/bin/bash
/usr/local/bin/backup.sh
EOF
at -f /tmp/job.sh 2am tomorrow
```

Queue management

```
atq
at -c 7                # show job 7 script
atrm 7
```

batch

```
echo '/usr/local/bin/heavy-index' | batch
```

Runs when system load allows (implementation-defined).

Ensure service

```
systemctl status atd
sudo systemctl enable --now atd
```

Notes / Pitfalls

- Environment is a snapshot — don't assume your interactive aliases; use absolute paths.
- Output mailed to user if local mail works; on servers mail may be undelivered — redirect to files.
- Permissions: not all users may use `at` (allow/deny files).
- Timezone is system timezone (`timedatectl`).
- Not a substitute for durable job queues in multi-host apps.

2026-relevant notes

- Prefer `systemd-run --on-calendar=` / `--on-active=` for modern one-shots with cgroups/logging.
- `at` remains handy on classic servers for quick deferred restarts.
- Containers often lack `atd` — use external schedulers.

Related Commands

- `atq` / `atrm` / `batch`
- `cron` / `crontab`
- `systemd-run` / timers
- `sleep` — simple delay in a live shell
- `wall` — broadcast messages

Additional Resources

- `man at`, `man atd`

Logging

Intro

Intro

Write and maintain traditional logs: kernel ring buffer, syslog-style messages, and rotation. On systemd systems, pair these with `journalctl` from Services and runtime.

Commands in this part

Command	Role
`dmesg`	dmesg prints the kernel ring buffer: hardware detection, driver messages, OOM kills, I/O errors, and other…
`logger`	logger writes messages to the system log from the shell or scripts.
`logrotate`	logrotate rotates, compresses, and retires log files so they do not fill disks.

Suggested starting points

1. Kernel messages: `dmesg` (and `journalctl -k`).
2. Emit a message: `logger`.
3. Manage growth: `logrotate`.

Related parts

- Services and runtime — `journalctl`
- Text and pipes — mine log files with `grep`/`jq`
- Archives and compression — `zgrep` on rotated `.gz` logs

Continue with the individual command pages in this part.

dmesg

Overview

`dmesg` prints the kernel ring buffer: hardware detection, driver messages, OOM kills, I/O errors, and other kernel-level events. First stop when a device doesn't appear, a disk fails, or the system OOMs. On systemd systems, the same stream is also available via `journalctl -k`.

Syntax

```
dmesg [options]
```

Common Options

Option	Description
`-H`	Human-readable timestamps
`-T`	Absolute human timestamps
`-k`	Kernel messages only
`-u`	Userspace messages (facility filter)
`-w` / `--follow`	Wait for new messages
`-W`	Follow after clearing wait logic variants
`-l level`	Filter by level
`-f facility`	Filter by facility
`-n level`	Set console log level
`-C` / `-c`	Clear / read-and-clear
`-x`	Decode facility/level prefixes
`-e`	Local time / reltime variants (see man)
`--color`	Colorize

Examples with Explanations

Recent kernel messages

```
dmesg | less
dmesg -T | less
dmesg -H
```

Follow live (plug hardware)

```
dmesg -w
# or
journalctl -kf
```

Errors and warnings

```
dmesg -T -l err,warn
dmesg -T | grep -iE 'error|fail|oom|i/o'
```

OOM investigation

```
dmesg -T | grep -i 'killed process'
journalctl -k | grep -i oom
```

USB / disk events

```
dmesg -T | tail -50
dmesg -T | grep -i usb
dmesg -T | grep -i sd
```

Clear buffer (careful)

```
sudo dmesg -C
```

Usually unnecessary; prefer filtering by time.

Permissions

```
# unprivileged may see restricted buffer on modern kernels
sudo dmesg
# sysctl kernel.dmesg_restrict
```

Notes / Pitfalls

- Ring buffer is finite — old messages drop; use journal persistence for history.
- Timestamps with `-T` depend on timekeeping across suspend/boot.
- Don't spam `dmesg -c` on production; you lose context for others.
- Userspace app logs are **not** here — use `journalctl` / syslog.
- Containers may not expose full host kernel messages.

2026-relevant notes

- `journalctl -k -b` for current boot kernel logs is often more convenient with persistent journal.
- Hardware still fails the same way — `dmesg` remains essential on bare metal.
- PSI and detailed MM logs complement OOM dmesg lines on modern kernels.

Related Commands

- `journalctl -k` — kernel via journal
- `udevadm` — device events
- `lspci` / `lsusb` — hardware lists
- `smartctl` — disk health
- `sysctl` — kernel parameters including log levels

Additional Resources

- `man dmesg`

logger

Overview

`logger` writes messages to the system log from the shell or scripts. It is the command-line interface to the syslog/journal stack — useful for marking deploy steps, cron output, and ad-hoc audit notes that should land next to service logs.

On systemd systems, messages typically appear in the **journal** (`journalctl`) and may also be forwarded to classic syslog daemons.

Syntax

```
logger [options] [message]
echo message | logger [options]
```

Common Options

Option	Description
`-p priority`	Facility.level e.g. `user.notice`, `local0.warning`
`-t tag`	Tag (defaults often to user name)
`-i`	Include PID in the tag
`-s`	Also write to stderr
`-f file`	Log each line of file
`-n host`	Remote syslog host (when supported)
`-P port`	Remote port
`-u socket`	Log socket path
`--journald`	Send via journal fields (newer util-linux)
`-e`	Skip empty lines

Examples with Explanations

Simple messages

```
logger "deploy started"
logger -t deploy -i "step 1 complete"
logger -p local0.info "backup ok"
```

From scripts

```
#!/bin/bash
tag=mybackup
log() { logger -t "$tag" -p user.notice "$*"; }
log "starting"
# ...
log "finished"
```

Pipe multi-line output

```
df -h | logger -t diskcheck
mycommand 2>&1 | logger -t mycommand -i
```

Priority levels

```
logger -p user.debug "debug detail"
logger -p user.warning "watch this"
logger -p user.err "failed to mount backup"
```

Facility.level pairs control routing in rsyslog/syslog-ng; journal stores priority as well.

View on systemd

```
logger -t demo "hello journal"
journalctl -t demo -n 5
journalctl -t demo -f
```

stderr mirror

```
logger -s -t demo "visible in console and log"
```

Structured-ish with journald (when supported)

```
logger --journald <<EOF
MESSAGE=deployment finished
PRIORITY=5
DEPLOY_ID=42
EOF
```

Notes / Pitfalls

- Rate limiting may drop floods of messages (rsyslog/journald).
- Remote logging needs network syslog config; `-n` alone may be insufficient on hardened hosts.
- Don't log secrets (tokens, passwords).
- Cron: prefer logging explicitly; cron mails can be unreliable.
- Priorities only help if collectors honor them.

2026-relevant notes

- Prefer `journalctl` for local investigation; ship logs with vector/fluent-bit/alloy for fleets.
- `systemd-cat` is an alternative that attaches stdout to the journal with unit-like metadata.
- For apps, native journal/syslog libraries beat wrapping everything in `logger`.

Related Commands

- `journalctl` — query journal
- `systemd-cat` — pipe to journal
- `rsyslogd` / `syslog-ng` — classic daemons
- `dmesg` — kernel ring buffer
- `wall` — broadcast to ttys (not syslog)

Additional Resources

- `man logger`

logrotate

Overview

`logrotate` rotates, compresses, and retires log files so they do not fill disks. On Ubuntu it is driven by a **systemd timer** or cron, reading `/etc/logrotate.conf` plus drop-ins in `/etc/logrotate.d/`. Application logs you add should get a file under `/etc/logrotate.d/`. Journald logs are **not** managed here — use `journalctl --vacuum-*` for the journal.

Syntax

```
logrotate [options] configfile
sudo logrotate -d /etc/logrotate.conf
sudo logrotate -f /etc/logrotate.d/rsyslog
```

Common Options

Option	Description
`-d`	Debug/dry-run — no changes
`-f`	Force rotation even if not due
`-v`	Verbose
`-s statefile`	Alternate state file (default `/var/lib/logrotate/status`)
`--skip-state-cleanup`	Keep status entries for vanished logs (version-dependent)

Common config directives

Directive	Meaning
`daily` / `weekly` / `monthly` / `hourly`	Time-based rotation
`size 100M` / `maxsize 100M`	Size-based (see man for size vs maxsize)
`rotate N`	Keep N old logs
`compress` / `delaycompress`	gzip (delay: skip most recent rotated)
`missingok`	Don't error if log absent
`notifempty`	Skip empty logs
`create mode owner group`	Create new empty log after rotate

Directive	Meaning
`copytruncate`	Copy then truncate in place (apps that never reopen)
`dateext` / `dateformat`	Date-based suffixes
`sharedscripts`	Run `postrotate` once for the whole block
`postrotate` … `endscript`	Reload/signal app after rotate
`su user group`	Run as user when directories are restricted
`olddir`	Move rotated files elsewhere
`maxage days`	Remove rotated logs older than N days

Key Use Cases

1. Keep `/var/log` from filling root
2. Rotate custom application logs with reload hooks
3. Force rotation during an incident when a log explodes
4. Dry-run config after edits before production force

Examples with Explanations

Dry-run entire system config

```
sudo logrotate -d /etc/logrotate.conf
```

Shows which logs **would** rotate and why (or why not). No files changed. Always do this after editing drop-ins.

Force one drop-in (verbose)

```
sudo logrotate -f -v /etc/logrotate.d/rsyslog
# or your app:
sudo logrotate -f -v /etc/logrotate.d/myapp
```

Use when testing a new stanza or recovering from a multi-GB log that should have rotated.

Inspect state

```
sudo cat /var/lib/logrotate/status | head -50
sudo grep myapp /var/lib/logrotate/status
```

State tracks last rotation times; corrupted/stale state can skip or confuse schedules — know where it lives.

Example application drop-in

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 14
    missingok
    notifempty
    compress
    delaycompress
    create 0640 myapp myapp
    sharedscripts
    postrotate
        systemctl reload myapp >/dev/null 2>&1 || true
    endscript
}
```

After install:

```
sudo logrotate -d /etc/logrotate.d/myapp
sudo logrotate -f -v /etc/logrotate.d/myapp
ls -la /var/log/myapp/
```

copytruncate when the app won't reopen

```
/var/log/legacy/app.log {
    size 100M
    rotate 7
    copytruncate
    missingok
    compress
    delaycompress
}
```

`copytruncate` avoids needing a signal, but can race and drop a few lines. Prefer apps that reopen on `SIGHUP`/`systemctl reload` with `create` + postrotate.

size-based emergency pattern

```
/var/log/myapp/debug.log {
    size 500M
    rotate 5
    compress
    missingok
```

```
    notifempty
    copytruncate
}
```

Helps when a debug flag left on would otherwise fill the disk overnight.

When does it run? (Ubuntu)

```
systemctl list-timers | grep -i logrotate
systemctl status logrotate.timer logrotate.service
# classic cron path may still exist on some releases:
ls /etc/cron.daily/logrotate
```

Timers can delay runs if the machine was off (`Persistent=` semantics vary).

Disk pressure triage with logs

```
df -h /var/log
sudo du -xh /var/log | sort -h | tail
sudo logrotate -f -v /etc/logrotate.conf
# journal is separate:
journalctl --disk-usage
sudo journalctl --vacuum-size=500M
```

Understanding Output

`-d` / `-v` lines explain decisions: "log does not need rotating", "log needs rotating", compression steps, and postrotate execution. Failures often come from permissions (`su` needed), missing directories, or postrotate scripts exiting non-zero.

Exit code non-zero should be treated as config or script failure — check unit logs:

```
journalctl -u logrotate.service -n 50
```

Notes & Pitfalls

- **Journald logrotate** — vacuum the journal separately.
- Prefer reload/reopen over `copytruncate` when the app supports it.
- After rotate, confirm the app still writes to the live path (not a deleted inode).
- `sharedscripts` + wildcards: postrotate runs once; without it, once per file.
- Permissions: log dirs owned by apps may need `su` in the stanza.
- Compressing huge logs can spike CPU/disk — `delaycompress` spreads pain.

- Don't rotate active database files or non-log data with logrotate recipes.
- Test with `-d` in CI or staging before shipping drop-ins.

Related Commands

- `logger` — write to syslog
- `journalctl` — systemd journal (vacuum, query)
- `systemctl reload` — reopen logs cleanly
- `df` / `du` — disk pressure
- `lsof` — deleted-but-open logs still holding space

Additional Resources

- `man logrotate`
- `man logrotate.conf`
- `/etc/logrotate.conf` and `/etc/logrotate.d/*` on the host

Hardware

Intro

Intro

Inventory and inspect physical devices: whole-system reports, PCI devices, DMI/SMBIOS data, and disk identity/timings. Use before opening tickets or swapping hardware.

Commands in this part

Command	Role
`lshw`	lshw (list hardware) probes the system and prints a hardware tree: CPU, memory, storage, network, firmware, and buses.
`lspci`	lspci lists PCI and PCI Express devices: NICs, GPUs, storage controllers, USB controllers, bridges, and more.
`dmidecode`	dmidecode dumps DMI/SMBIOS tables: manufacturer, product name, serial numbers, BIOS version, memory slot population,…
`hdparm`	hdparm gets and sets ATA/SATA disk parameters: identify data, power management, read-ahead, write-cache, and…

Suggested starting points

1. Broad inventory: `lshw`.
2. PCI devices: `lspci`.
3. Firmware/DMI: `dmidecode`.
4. Disk identity/timings: `hdparm` (SMART health is under Storage: `smartctl`).

Related parts

- System information — quick CPU/memory summaries
- Storage and filesystems — block devices and SMART

- Services and runtime — kernel modules via `modprobe`/`lsmod`

Continue with the individual command pages in this part.

lshw

Overview

`lshw` (list hardware) probes the system and prints a hardware tree: CPU, memory, storage, network, firmware, and buses. Run as **root** (or with capabilities) for full DMI/PCI detail; unprivileged runs omit sensitive or privileged data.

```
sudo apt install lshw
sudo dnf install lshw
```

Syntax

```
sudo lshw [options]
```

Common Options

Option	Description
`-short`	Compact table
`-class CLASS` / `-C`	Filter: `network`, `disk`, `memory`, `cpu`, `system`, `display`, `storage`, `power`, …
`-businfo`	Bus addresses / logical names
`-json` / `-xml` / `-html`	Export formats
`-sanitize`	Hide serials and UUIDs for sharing
`-numeric`	Numeric IDs
`-quiet`	Less noise
`-enable` / `-disable` tests	Control probe tests (see man)

Key Use Cases

1. Inventory machines for asset management
2. Identify NIC driver/logical name mismatches
3. Confirm RAM/slots and disk models
4. Generate sanitized HTML/JSON for tickets

Examples with Explanations

Quick overview

```
sudo lshw -short
sudo lshw -class system -short
```

Network and disks

```
sudo lshw -class network -short
sudo lshw -C network -businfo
sudo lshw -class disk
sudo lshw -class storage
```

Memory and CPU

```
sudo lshw -class memory
sudo lshw -class cpu -short
```

Export

```
sudo lshw -json > /tmp/hw.json
sudo lshw -sanitize -html > /tmp/hw.html
sudo lshw -json | jq '.[].class' 2>/dev/null | sort -u
```

One-liner recipes

```
# MAC and product for NICs
sudo lshw -class network | egrep 'description|product|serial|logical name|capacity'

# Disk models
sudo lshw -class disk | egrep 'description|product|serial|size|logical name'

# Compare businfo to ip link
sudo lshw -C network -businfo; ip -br link
```

Notes & Pitfalls

- Without root, expect incomplete PCI/DMI data.
- **Virtual machines** report virtual hardware; use cloud metadata APIs for instance type.
- Probing can be slow on large systems; class filters help.
- Serial numbers are sensitive—use `-sanitize` before pasting into public issues.
- Prefer `lscpu`, `lsblk`, `ip link` for single-purpose questions; `lshw` is the wide inventory tool.

2026-relevant notes

- On servers, **`dmidecode`**, **`lshw`**, and **`lscpu/lsblk`** remain the classic trio; some fleets use `inxi`, `hwinfo`, or vendor out-of-band (iDRAC/iLO) for authoritative inventory.
- NVMe and virtual NICs rename often (`ens`, `enp`, `eth`)—trust `businfo` + `ip link` together.
- JSON export helps config-management facts when agent-based inventory isn't installed.

Related Commands

- `dmidecode` — SMBIOS/DMI tables
- `lspci` / `lsusb` — buses
- `lscpu` / `lsblk` / `lsmem` — focused views
- `ip link` — live interfaces
- `hwinfo` — alternate prober (SUSE heritage)

Additional Resources

- `man lshw`
- Upstream ezix.org lshw documentation

lspci

Overview

`lspci` lists **PCI and PCI Express** devices: NICs, GPUs, storage controllers, USB controllers, bridges, and more. Part of the `pciutils` package (standard on Ubuntu servers). First stop for "is the NIC visible?", "which driver is bound?", and collecting `vendor:device` IDs for driver or firmware work.

Syntax

```
lspci [options]
```

Common Options

Option	Description
`-v` / `-vv` / `-vvv`	Increasing verbosity (capabilities, BARs, …)
`-k`	Kernel driver in use and modules that can bind
`-nn`	Numeric vendor:device (and class) codes
`-s slot`	Only this slot (`[[domain:]bus:]slot[.func]`)
`-d vendor:device`	Filter by IDs (`8086:`, `10de:`, …)
`-t`	Tree of buses/bridges
`-mm`	Machine-readable key-value
`-D`	Always show domain numbers
`-n`	Numeric IDs only (no name resolution)
`-x` / `-xxx`	Hex config space dump (needs root for full)

Key Use Cases

1. Confirm a device is enumerated after install/passthrough
2. See which kernel driver owns a NIC/GPU/storage HBA
3. Collect `vendor:device` for DKMS/out-of-tree modules
4. Compare before/after firmware or hardware changes

Examples with Explanations

Everyday list

```
lspci
lspci -nn
```

Human names plus, with `-nn`, stable `[vvvv:dddd]` IDs for docs and `modprobe` quirks.

Drivers and modules

```
lspci -k
lspci -nnk
```

`Kernel driver in use` is the bound driver; `Kernel modules` lists candidates. Empty driver often means missing firmware, blacklisted module, or unbound VFIO for passthrough.

Network / GPU filters

```
lspci | grep -i ethernet
lspci | grep -i network
lspci | grep -i nvidia
lspci | grep -i 'non-volatile\|nvme\|raid\|scsi'
lspci -nnk | grep -A3 -i ethernet
```

One device, deep detail

```
lspci -s 00:1f.2 -vv
sudo lspci -s 00:1f.2 -vvv
```

Slot address from the first column of `lspci`. Root unlocks more config space in deep dumps.

Tree view

```
lspci -t
lspci -tv
```

Shows how devices hang off bridges — useful for bandwidth/topology discussions and multi-GPU layouts.

Machine-readable

```
lspci -mmnn
lspci -mm | head
```

Easier for scripts than scraping the pretty format.

ID database refresh

```
sudo update-pciids
lspci -nn | head
```

Refreshes `/usr/share/misc/pci.ids` so new devices resolve to names (IDs still work offline).

Correlate with sysfs / ethtool

```
lspci -nnk | grep -A3 -i ethernet
# then for a netdev:
ethtool -i eth0
ls -l /sys/class/net/eth0/device/driver
```

lspci shows PCI binding; `ethtool -i` shows the netdev's driver/firmware strings.

Understanding Output

Default line shape:

```
00:1f.2 SATA controller: Intel Corporation ... [8086:…. ] (rev ..)

                         human name              vendor:device
bus:slot.func + class
```

Piece	Role
Domain/BDF	Address for `-s` and sysfs paths
Class	Ethernet, VGA, NVMe, bridge, …
Vendor/device IDs	Stable identity for drivers
`-k` lines	Bound driver vs available modules

Sysfs path pattern: `/sys/bus/pci/devices/0000:bus:slot.func/`.

Notes & Pitfalls

- Some extended reads need **root** (`-vvv`, full `-xxx`).
- Names depend on `pci.ids`; unknown devices still show IDs with `-nn`.
- SR-IOV VFs appear as extra devices — don't confuse PF vs VF when binding drivers.
- GPU "not found" can be power/BIOS/resizability (BAR) issues — dmesg matters.
- Hotplug/cloud passthrough: re-run lspci after attach; guest may need reboot for some devices.
- `update-pciids` needs network; air-gapped hosts keep stale names only.

Related Commands

- `lsusb` — USB bus inventory
- `lshw -class network -class display` — higher-level inventory
- `ethtool` — NIC driver/link settings
- `modinfo` / `lsmod` / `modprobe` — module details and load
- `dmesg` / `journalctl -b` — probe/firmware errors
- `setpci` — low-level PCI config (expert; easy to break systems)

Additional Resources

- `man lspci`
- `man update-pciids`
- `man pci.ids`

dmidecode

Overview

`dmidecode` dumps **DMI/SMBIOS** tables: manufacturer, product name, serial numbers, BIOS version, memory slot population, processor strings, chassis type, and more. It is the go-to CLI for "what hardware is this?" on bare metal without opening a ticket to the DC. Requires **root**. VMs expose synthetic or partial DMI data.

Syntax

```
sudo dmidecode [options]
```

Common Options

Option	Description
`-t type`	Restrict by type name or number: `system`, `bios`, `baseboard`, `memory`, `processor`, `chassis`, `slot`, …
`-s keyword`	Print a single string (`system-serial-number`, `system-product-name`, `bios-version`, …)
`-u`	Dump raw formatted entries
`-q`	Quieter (less decoration)
`-d file`	Read from dump file instead of `/sys/firmware/dmi/tables`
`--oem-string N`	OEM string index

Common `-t` names: `bios`, `system`, `baseboard`, `chassis`, `processor`, `memory`, `cache`, `connector`, `slot`.

Key Use Cases

1. Inventory serial / product / UUID for asset tracking
2. BIOS/UEFI version before firmware upgrades
3. Memory slot map (which DIMM empty/populated)
4. Confirm bare-metal vs obvious VM DMI strings

Examples with Explanations

System identity

```
sudo dmidecode -t system
sudo dmidecode -s system-manufacturer
sudo dmidecode -s system-product-name
sudo dmidecode -s system-serial-number
sudo dmidecode -s system-uuid
```

Serial and UUID are sensitive — scrub before pasting into public tickets.

BIOS / firmware

```
sudo dmidecode -t bios
sudo dmidecode -s bios-version
sudo dmidecode -s bios-release-date
sudo dmidecode -s bios-vendor
```

Capture before and after firmware updates.

Memory population

```
sudo dmidecode -t memory
sudo dmidecode -t memory | grep -E 'Size:|Locator:|Bank Locator:|Speed:|Manufacturer:|Part N
```

Shows per-slot size/speed; empty slots often report `No Module Installed` / size `No Module Installed`.

Processor summary

```
sudo dmidecode -t processor
sudo dmidecode -t processor | grep -E 'Version:|Core Count:|Core Enabled:|Thread Count:|Max
```

Complement with `lscpu` for the kernel's view of topology and flags.

Baseboard and chassis

```
sudo dmidecode -t baseboard
sudo dmidecode -t chassis
sudo dmidecode -s baseboard-serial-number
sudo dmidecode -s chassis-type
```

Useful for blade/rack asset fields and chassis form factor.

Inventory one-liners for scripts

```
sudo dmidecode -s system-manufacturer
sudo dmidecode -s system-product-name
sudo dmidecode -s system-serial-number
sudo dmidecode -s bios-version
```

`-s` fails closed if the keyword is invalid — check `man dmidecode` for the keyword list.

Compare with sysfs (no full dump)

```
# many fields also appear here (root not always required for all):
cat /sys/class/dmi/id/product_name
cat /sys/class/dmi/id/sys_vendor
cat /sys/class/dmi/id/product_serial   # may be root-only / permission-denied
```

sysfs is handy for a few fields; dmidecode is better for structured tables (memory slots).

Understanding Output

Output is grouped into **handles** (DMI record instances). Each handle has a type (e.g. Memory Device) and key/value fields. Multiple handles of the same type are normal (one per DIMM, per CPU socket, …).

Area	What operators extract
System	Vendor, model, serial, UUID
BIOS	Version, date, vendor
Memory Device	Locator, size, speed, part number
Processor	Version string, core/thread counts

Garbage or `Not Specified` / `To be filled by O.E.M.` means the vendor left SMBIOS incomplete — common on some whiteboxes and VMs.

Notes & Pitfalls

- **Root required** for full access; without it you get little or nothing.
- VMs (KVM, VMware, cloud) synthesize DMI — serials may be instance IDs or placeholders.
- Serial numbers are **PII/asset-sensitive** — redact in chat logs.
- DMI can be wrong if vendors mess up tables; cross-check labels and BMC/iDRAC/iLO when stakes are high.

- Not a substitute for `lshw` PCI detail or `lspci` driver binding.
- `system-uuid` may be used by software licensing — changing it (rare/firmware) has consequences.

Related Commands

- `lshw` — broader hardware tree (also often needs root)
- `lscpu` — CPU topology/flags from kernel
- `lspci` / `lsusb` — buses and peripherals
- `cat /sys/class/dmi/id/*` — quick sysfs fields
- `dmesg` — firmware/kernel hardware messages

Additional Resources

- `man dmidecode`
- SMBIOS field quality varies by OEM; trust but verify against physical tags for RMA.

hdparm

Overview

`hdparm` gets and sets **ATA/SATA disk** parameters: identify data, power management, read-ahead, write-cache, and controversial performance timings. Useful for bare-metal disk troubleshooting. Wrong settings can risk **data loss** — especially write-cache and dangerous benchmark flags.

Prefer vendor tools / smartmontools for health; use `hdparm` carefully.

Syntax

```
hdparm [options] device
```

Device is typically `/dev/sda` (whole disk), not a partition.

Common Options

Option	Description
`-I`	Detailed identity information
`-i`	Kernel's identify summary
`-C`	Power mode status
`-M`	AAM acoustic (legacy)
`-B`	APM level
`-S`	Standby timeout
`-W`	Write-cache on/off query/set
`-a` / `-A`	Read-ahead
`-t` / `-T`	Timing buffered disk / cache reads (**benchmark load**)
`--security-*`	ATA security (dangerous)
`-y` / `-Y`	Standby / sleep now
`-z`	Reread partition table

Examples with Explanations

Identify

```
sudo hdparm -I /dev/sda
sudo hdparm -i /dev/sda
sudo smartctl -a /dev/sda
```

Power status

```
sudo hdparm -C /dev/sda
```

Read-only info queries (safer)

```
sudo hdparm -W /dev/sda
sudo hdparm -a /dev/sda
```

Benchmarks (I/O heavy; maintenance window)

```
sudo hdparm -tT /dev/sda
```

Interprets sequential throughput roughly; not a substitute for `fio`.

Write cache (know the risk)

```
sudo hdparm -W /dev/sda
# enabling write cache without barriers/UPS can lose data on power loss
```

Prefer filesystem/mount and device defaults unless you know the storage stack.

Reread partitions

```
sudo hdparm -z /dev/sda
# often partprobe / blockdev --rereadpt also used
```

Notes / Pitfalls

- **Never** experiment with `--security-erase` / passwords on disks you care about without full understanding.
- USB bridges and NVMe devices may ignore or poorly implement ATA commands — NVMe has `nvme` CLI.
- Cloud block volumes often virtualize away hdparm usefulness.
- Timing tests stress disks — avoid on production peaks.
- Requires root.

2026-relevant notes

- NVMe dominates SSDs: learn `nvme list`, `nvme smart-log` alongside `hdparm`.
- For spinning rust power policies on laptops, distro power tools may be safer than manual `-B`/`-S`.
- Always have backups before storage experiments.

Related Commands

- `smartctl` — SMART health
- `nvme` — NVMe admin
- `lsblk` / `fdisk` / `blkid` — topology
- `fio` — proper benchmarks
- `blockdev` — low-level block ioctls

Additional Resources

- `man hdparm` (read warnings)

Reading NUMA Topology without numactl

Reading NUMA Topology without numactl

Thesis

NUMA (Non-Uniform Memory Access) means memory attached to one socket is faster for CPUs on that socket. Before you pin threads or blame "memory latency", read the topology. You do not need `numactl` for a first look: sysfs under `/sys/devices/system/node/` and `/proc/zoneinfo` are enough. This chapter inspects a single-node VM and shows what multi-node output would add.

Mental model

```
node0: CPUs 0-7, local DRAM
node1: CPUs 8-15, local DRAM
     cross-node load → higher latency / lower bandwidth
```

Worked examples

`numactl` is **not** installed here (and we will not install packages for a chapter). Read-only sysfs:

```
ls /sys/devices/system/node/
cat /sys/devices/system/node/node0/meminfo | head
cat /sys/devices/system/node/node0/cpulist
head -40 /proc/zoneinfo
```

Output (trimmed) from this run:

```
has_cpu
has_generic_initiator
has_memory
has_normal_memory
node0
online
possible
power
uevent
== node0 ==
Node 0 MemTotal:       16386368 kB
```

```
Node 0 MemFree:         5183212 kB
Node 0 MemUsed:        11203156 kB
Node 0 SwapCached:             0 kB
Node 0 Active:           1525412 kB
Node 0 Inactive:         5364312 kB
Node 0 Active(anon):        1372 kB
Node 0 Inactive(anon):   3180496 kB
cpus: 0-7
```

This guest is **one NUMA node** (`node0`, CPUs `0-7`, ~16 GiB). That is normal for small VMs — the skill is confirming it before applying multi-node tuning guides.

Trap

Copying `numactl --interleave=all` folklore onto a single-node machine. Interleave policies matter when remote memory is real. On one node they add complexity without benefit.

Boring rule

Always print `node*/cpulist` and `MemTotal` before changing CPU affinity or memory policy. Install `numactl` only on machines you own when you need `numactl --hardware` pretty-printing.

Try this

1. On a dual-socket host, compare `node0/distance` and `node1/distance`.
2. Use `lscpu` NUMA lines next to sysfs.
3. Read `numa(7)`.

Sources

- `numa(7)`, kernel Documentation/vm/numa*
- sysfs `devices/system/node`

Storage and Filesystems

Intro

Intro

Block devices, partitions, filesystems, mounts, LVM, encryption, integrity checks, and space usage. Destructive tools (`mkfs`, `dd`, `wipefs`, `cryptsetup luksFormat`) always need identity double-checks.

Commands in this part

Command	Role
`lsblk`	lsblk lists block devices (disks, partitions, LVM logical volumes, loop devices, ROM) as a tree.
`blkid`	blkid probes block devices for filesystem, RAID, and crypto signatures and prints tags such as TYPE, UUID, and LABEL.
`findmnt`	findmnt lists or searches mounted filesystems in tree or table form.
`fdisk`	fdisk creates and edits DOS/MBR and GPT partition tables on block devices.
`mkfs`	mkfs builds a filesystem on a block device (partition, LV, loop file).
`fsck`	fsck checks and optionally repairs filesystems.
`mount`	mount attaches a filesystem (block device, network share, bind path, tmpfs, …) onto a directory in the single Linux…
`umount`	umount detaches a mounted filesystem from the directory tree.
`dd`	dd copies and converts data at the block level.
`sync`	sync flushes filesystem buffers: dirty page cache and pending metadata are written to storage.
`losetup`	losetup associates regular files with loop devices (/dev/loopN) so the kernel can treat an image file like a block…

Command	Role
duf	duf is a modern, colorful disk usage / free space viewer — a friendlier alternative to df -h.
dust	dust is a modern du alternative that shows disk usage as a readable tree/bars, helping you find large directories…
parted	parted creates and modifies partition tables (GPT/MSDOS) on block devices.
wipefs	wipefs probes and can erase filesystem, RAID, or partition-table signatures from a device without zeroing the entire…
smartctl	smartctl (from smartmontools) queries disk S.M.A.R.T.
pvs	pvs reports LVM physical volumes (PVs) — disks or partitions initialized with pvcreate and used by volume groups.
vgs	vgs reports LVM volume groups (VGs): how physical volumes are pooled and how much space is free for logical volumes.
lvs	lvs reports LVM logical volumes (LVs) — the block devices you usually format and mount (/dev/VG/LV or…
cryptsetup	cryptsetup manages LUKS (and other) disk encryption: format, open, close, and resize encrypted block devices.
resize2fs	resize2fs grows or shrinks ext2/ext3/ext4 filesystems.
tune2fs	tune2fs adjusts tunable parameters on ext2/ext3/ext4 filesystems: labels, UUID, mount-count checks, reserved blocks,…
ncdu	ncdu (NCurses Disk Usage) is an interactive du for finding what consumes space.
RWF_DONTCACHE	RWF_DONTCACHE is the preadv2/pwritev2/io_uring flag for uncached buffered I/O: bulk reads and writes that do not flood the page cache (XFS 6.15+, ext4 6.17+, NFS 6.18+, block devices 7.3+).

Command	Role
`io_uring`	io_uring is the shared-ring asynchronous I/O interface: batch many reads and writes into one syscall, tag completions, and fall back when `kernel.io_uring_disabled` or seccomp blocks it.

Suggested starting points

1. See devices: `lsblk`, `blkid`, `findmnt`.
2. Partition: `parted`/`fdisk`, then `mkfs`, `mount`/`umount`.
3. Check/repair: `fsck` (unmounted); grow/tune ext*: `resize2fs`, `tune2fs`.
4. LVM trio: `pvs`, `vgs`, `lvs`.
5. Encryption: `cryptsetup`.
6. Space: `duf`/`dust`/`ncdu`; health: `smartctl`.
7. Bulk I/O without flushing the page cache: `RWF_DONTCACHE` (and its `fadvise` fallback).
8. Batched async I/O and its host policy: `io_uring` (with a synchronous fallback).

Related parts

- Files and paths — `df`/`du` at the path layer
- Hardware — disk controllers and DMI
- Security — labels on mounted data

Continue with the individual command pages in this part.

lsblk

Overview

`lsblk` lists block devices (disks, partitions, LVM logical volumes, loop devices, ROM) as a tree. First stop when you plug in a disk, write fstab entries, or answer "what is `/dev/sdb1`?". It shows topology and filesystem metadata; it does **not** report free space inside filesystems — use `df` / `duf` for capacity.

Syntax

```
lsblk [options] [device...]
```

Common Options

Option	Description
`-f`	Filesystem view: FSTYPE, LABEL, UUID, FSAVAIL, FSUSE%, mountpoints
`-o COLS`	Custom column list
`-p`	Full device paths (`/dev/sda` not `sda`)
`-a`	Include empty devices
`-d`	Devices only (no partition children)
`-e 7`	Exclude major number 7 (loop) — hide snap noise
`-J` / `-O`	JSON / all known columns
`-t`	Topology (alignment, scheduler, discard)
`-S`	SCSI devices only
`-n`	No header (script-friendly with `-o`)
`-r`	Raw (no tree indentation)

Useful columns: `NAME,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS,MODEL,TRAN,ROTA,SERIAL,PARTUUID`.

Key Use Cases

1. Inventory disks after attach (USB, cloud volume, NVMe)
2. Collect UUID/LABEL for durable fstab lines
3. Distinguish SSD/NVMe vs spinning (`ROTA`, `TRAN`)
4. Map LVM/mapper children under parent disks
5. Feed scripts with JSON or selected columns

Examples with Explanations

Everyday inventory

```
lsblk
lsblk -f
lsblk -p
```

Default tree shows NAME, MAJ:MIN, RM, SIZE, RO, TYPE, MOUNTPOINTS. `-f` adds FSTYPE/LABEL/UUID — the view you want before mount or fstab work. `-p` prints full paths for copy-paste into commands.

UUID-ready view for fstab

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,UUID,MOUNTPOINTS
lsblk -o NAME,PARTUUID,UUID,FSTYPE,MOUNTPOINTS -p
```

Prefer `UUID=` (or `PARTUUID=` for partitions without a FS yet) over `/dev/sdX` names that can reorder on reboot.

Hide loop noise (snaps)

```
lsblk -e 7
lsblk -e 7 -f
# or filter: lsblk | grep -v loop
```

Ubuntu systems with many snaps flood the tree with loop devices. Major 7 is loop; exclude it for human scanning.

JSON for scripts

```
lsblk -J -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS | jq .
lsblk -J -o NAME,SIZE,TYPE,MOUNTPOINTS | jq -r '.blockdevices[].name'
```

Stable structure for automation; combine with `jq` rather than parsing the pretty tree.

Topology / rotational / transport

```
lsblk -d -o NAME,SIZE,ROTA,TYPE,TRAN,MODEL,SERIAL
# ROTA=0 often SSD/NVMe; TRAN=nvme|sata|usb|sas
```

`-d` limits to whole disks. Use this when capacity planning or placing I/O-heavy workloads (prefer `TRAN=nvme` over USB).

One device subtree

```
lsblk -f /dev/nvme0n1
lsblk -f /dev/sdb
```

Focus on a single parent and its partitions/holders — less noise on multi-disk hosts.

Align with blkid and mounts

```
lsblk -f
sudo blkid
findmnt -D
```

Cross-check: `lsblk` topology, `blkid` superblock tags, `findmnt` what is actually mounted.

Empty or new disk check

```
lsblk -f /dev/sdb
# no FSTYPE / no children → raw or unpartitioned; confirm size matches expectation
```

Before `fdisk`/`mkfs`, verify size and that nothing is already mounted on that path.

Understanding Output

Field	Meaning
Tree indent	Parent disk → partitions → LVM/dm children
`TYPE`	`disk`, `part`, `lvm`, `crypt`, `loop`, `rom`, …
`FSTYPE`	Superblock type (`ext4`, `xfs`, `vfat`, `crypto_LUKS`, …); empty raw/unformatted
`UUID` / `LABEL`	Filesystem identity for fstab
`MOUNTPOINTS`	Where mounted (multiple possible with binds on recent util-linux)
`ROTA`	`1` rotational disk, `0` usually SSD/NVMe
`TRAN`	Transport: `nvme`, `sata`, `usb`, …
`RM`	Removable flag

Kernel device names (`sda` vs `sdb`, `nvme0n1p1`) are **not** stable across boots or controller probe order.

Notes & Pitfalls

- Kernel names change; always pin mounts with UUID/LABEL/PARTUUID.
- `lsblk` is not free-space accounting — use `df -h` / `findmnt -D` for capacity.
- Some details need root; listing usually works unprivileged.
- Multipath hosts may show both `/dev/sdX` and `/dev/mapper/mpath*` — mount the multipath node.
- Cloud "detach/attach" can renumber disks; re-run `lsblk` after attach before scripting.
- RAID/LVM: format/mount the logical device, not a single member disk.

Related Commands

- `blkid` — probe UUID/LABEL/TYPE from superblocks
- `findmnt` — mounts as tree/table/JSON
- `df` / `duf` — capacity of mounted filesystems
- `fdisk` / `parted` — partition tables
- `lvs` / `vgs` / `pvs` — LVM detail
- `ls -l /dev/disk/by-uuid` — udev stable symlinks

Additional Resources

- `man lsblk`
- `man 8 lsblk` column list (`--help` also lists columns)

blkid

Overview

`blkid` probes block devices for filesystem, RAID, and crypto signatures and prints tags such as **TYPE**, **UUID**, and **LABEL**. Use it when building durable `/etc/fstab` lines, verifying a format succeeded, or resolving `UUID=` / `LABEL=` to a device path. Complements `lsblk -f` (topology) with superblock-focused output.

Syntax

```
sudo blkid [options] [device...]
```

Without a device, lists known devices (may use cache; root improves completeness).

Common Options

Option	Description
`-p`	Low-level probe (bypass some cache behavior)
`-s TAG`	Show only tag(s): `UUID`, `LABEL`, `TYPE`, `PARTUUID`, …
`-o value|export|list|udev|full`	Output format
`-U UUID`	Look up device path by filesystem UUID
`-L LABEL`	Look up device path by LABEL
`-c /dev/null`	Ignore cache file (force fresh probe)

Key Use Cases

1. Copy UUID into fstab after `mkfs`
2. Resolve `UUID=` or `LABEL=` to `/dev/…` path
3. Confirm filesystem type before mount/fsck
4. Script-friendly single-value extraction (`-o value`)

Examples with Explanations

List all probed devices

```
sudo blkid
sudo blkid -c /dev/null
```

Shows TYPE/UUID/LABEL (and more) per device. `-c /dev/null` avoids stale cache after recent formats.

One device

```
sudo blkid /dev/sdb1
sudo blkid -p /dev/sdb1
```

Export form (shell-friendly)

```
sudo blkid -o export /dev/sdb1
# DEVNAME=/dev/sdb1
# UUID=…
# TYPE=ext4
```

Easy to `eval` carefully or parse line-by-line in scripts.

UUID only (fstab snippets)

```
sudo blkid -s UUID -o value /dev/sdb1
UUID=$(sudo blkid -s UUID -o value /dev/sdb1)
echo "UUID=$UUID  /mnt/data  ext4  defaults  0  2"
```

Prefer this over hard-coding `/dev/sdX`.

Lookup by UUID or LABEL

```
sudo blkid -U 123e4567-e89b-12d3-a456-426614174000
sudo blkid -L data
ls -l /dev/disk/by-uuid/
ls -l /dev/disk/by-label/
```

Returns the device node currently bound to that identity. udev symlinks under `/dev/disk/by-*` are the other stable interface.

Compare with lsblk

```
lsblk -f
sudo blkid
```

`lsblk -f` is tree-oriented; `blkid` is probe-oriented. Use both when something disagrees (cache, multipath aliases).

After mkfs verification

```
sudo mkfs.ext4 -L data /dev/sdb1
sudo blkid -c /dev/null /dev/sdb1
lsblk -f /dev/sdb1
```

Confirm new UUID/LABEL before writing fstab.

udev property style

```
sudo blkid -o udev /dev/sdb1
# ID_FS_UUID=…
# ID_FS_TYPE=…
```

Matches tags udev exposes; useful when correlating with `udevadm info`.

Understanding Output

Typical line:

```
/dev/sdb1: UUID="…" BLOCK_SIZE="4096" TYPE="ext4" PARTUUID="…"
```

Tag	Role
`TYPE`	Filesystem/crypto/RAID signature
`UUID`	Filesystem UUID (fstab `UUID=`)
`LABEL`	Human label (`LABEL=`)
`PARTUUID`	GPT partition UUID (stable even before FS)
`PARTLABEL`	GPT partition name

No output often means no recognized signature (raw, wiped, or inaccessible without root).

Notes & Pitfalls

- Root is often required for reliable probing of all devices.
- Cache can lag immediately after format — use `-c /dev/null` or `-p` when suspicious.
- Multipath/LVM: same UUID may appear under several paths; mount via mapper/multipath node consistently.
- LABEL collisions: two disks with `LABEL=data` make `LABEL=` fstab ambiguous — prefer UUID.
- LUKS: outer device shows `crypto_LUKS`; filesystem UUID appears on the opened mapper device.
- Swap shows `TYPE="swap"` with its own UUID (`/etc/crypttab` / fstab swap lines).

Related Commands

- `lsblk -f` — topology + FS columns
- `findmnt` — active mounts
- `udevadm info -q property -n /dev/sdb1` — udev properties
- `ls /dev/disk/by-uuid` — stable symlinks
- `wipefs -n` — list signatures without blkid cache

Additional Resources

- `man blkid`
- `man fstab`

findmnt

Overview

`findmnt` lists or searches mounted filesystems in tree or table form. It is clearer than parsing `mount` output and friendlier for scripts (`-J`, `-n`, `-o`, `-r`). First tool for "what is mounted where?", fstab verification, and busy-mount debugging before `umount`.

Syntax

```
findmnt [options] [device|mountpoint]
```

Common Options

Option	Description
(none)	Tree of all mounts
`-t types`	Filter by FS types (`ext4,xfs,nfs`)
`-S source`	Match source device/path
`-T path`	Filesystem that contains this path
`-D`	df-like size columns
`-o list`	Select columns
`-n`	No header
`-r`	Raw (no tree art)
`-J`	JSON
`-f`	First match only
`-R`	Sub-tree of a mountpoint
`--verify`	Check `/etc/fstab` against reality (util-linux)
`-A` / `-l`	All / list format

Useful columns: `TARGET,SOURCE,FSTYPE,OPTIONS,UUID,LABEL,SIZE,USE%`.

Key Use Cases

1. Human-readable mount inventory (better than raw `mount`)
2. Which mount owns a path (`-T`)
3. df-like capacity with mount context (`-D`)
4. Scripted JSON/table for monitoring
5. Verify fstab before reboot (`--verify`)

Examples with Explanations

Full tree

```
findmnt
findmnt -t ext4,xfs,btrfs,nfs
```

Tree shows mount hierarchy (binds and API filesystems included). Filter types to cut tmpfs/proc noise when scanning "real" disks.

df-like sizes

```
findmnt -D
findmnt -D -t ext4,xfs
```

Combines mount table with used/available — often nicer than juggling `df` + `mount`.

What holds this path?

```
findmnt -T /var/log
findmnt -T .
findmnt -T /var/lib/docker
```

Critical with bind mounts and nested volumes: `df /var/lib/docker` can surprise you; `-T` shows the owning mount.

By source or target

```
findmnt /
findmnt -S /dev/sdb1
findmnt /mnt/data
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS /mnt/data
```

Script-friendly

```
findmnt -n -o SOURCE /
findmnt -nro TARGET,FSTYPE
findmnt -J | jq -r '.filesystems[].target'
```

`-n` drops headers; `-r` raw fields; JSON for structured tools.

Busy mount debugging

```
findmnt /mnt/data
findmnt -R /mnt/data
sudo lsof +f -- /mnt/data | head
sudo fuser -vm /mnt/data
```

See exact mount options and nested mounts before `umount`.

Verify fstab

```
findmnt --verify
findmnt --verify --verbose
```

Catches missing devices, bad options, or stale UUIDs **before** the next boot fails.

NFS / bind focus

```
findmnt -t nfs,nfs4
findmnt -o TARGET,SOURCE,FSTYPE,OPTIONS -t nfs,nfs4
findmnt | grep bind   # binds often show same source path twice
```

Understanding Output

Column	Meaning
`TARGET`	Mountpoint
`SOURCE`	Device, UUID path, or bind source
`FSTYPE`	Filesystem type
`OPTIONS`	`rw`, `noexec`, `nosuid`, NFS opts, …
Size cols (`-D`)	Capacity like `df`

Tree indentation reflects mount nesting (e.g. bind under `/mnt`). Pseudo filesystems (`proc`, `sysfs`, `cgroup2`) dominate unfiltered views on modern systemd systems.

Notes & Pitfalls

- Unfiltered output is noisy — filter with `-t` or look at specific targets.
- Namespace matters: inside containers/`unshare` you see that mount NS only.
- `findmnt -T` follows the mount covering a path, not "directory exists".
- Options string is long; use `-o` to extract what you need.
- Prefer `findmnt` over scraping `/proc/mounts` in new scripts (still OK to read `/proc/self/mountinfo` for deep debugging).

Related Commands

- `mount` / `umount` — attach/detach
- `lsblk -f` / `blkid` — device identity
- `df -h` — classic capacity
- `lsof` / `fuser` — busy mounts
- `systemctl status *.mount` — systemd mount units

Additional Resources

- `man findmnt`
- `man mount`
- `man fstab`

fdisk

Overview

`fdisk` creates and edits DOS/MBR and GPT partition tables on block devices. Interactive by default; list-only with `-l`. For scripted GPT layouts many operators prefer `sfdisk` or `parted`; for pure GPT recovery, `gdisk` is another common tool. After partitioning you still need `mkfs` (or LVM/crypt setup) before data use.

Syntax

```
sudo fdisk [-l] [device...]
sudo fdisk device          # interactive editor
```

Common Options

Option	Description
`-l`	List partition tables (no write)
`-x`	Extra detail when listing
`-w always|never`	Wipe signatures when writing (newer util-linux)
`-t dos|gpt`	Restrict label type (context-dependent)

Interactive keys (common):

Key	Action
`m`	Help
`p`	Print table
`g`	New empty GPT
`o`	New empty DOS/MBR
`n`	New partition
`d`	Delete partition
`t`	Change type (Linux filesystem, LVM, EFI, …)
`w`	Write table to disk and exit
`q`	Quit **without** saving

Safety

Writing a partition table can destroy access to existing data. Treat `w` as irreversible without backups.

1. Identify the **correct** disk: `lsblk -f`, `lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN`, `sudo fdisk -l`.
2. Never guess `sdb` vs `sdc` on multi-disk hosts — match size/model/serial.
3. Do not rewrite tables on mounted system disks without a recovery plan and offline/maintenance window.
4. Prefer working on **data** disks first; practice on disposable VMs.
5. After write, re-read the table (`partprobe` / reboot if needed) before `mkfs`.

Key Use Cases

1. List all disks and partitions before storage work
2. Create GPT layout on a new data disk
3. Add a partition for LVM or a second filesystem
4. Dump/restore layouts via `sfdisk` for clones

Examples with Explanations

List all disks (read-only)

```
sudo fdisk -l
sudo fdisk -l /dev/sdb
sudo fdisk -l /dev/nvme0n1
```

Safe first step. Confirms capacity, label type (Disklabel type: gpt/dos), and existing partitions.

Confirm target before editing

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,MODEL,SERIAL,TRAN
lsblk -f /dev/sdb
# only proceed if size/model match the disk you intend to wipe
```

Wrong-device mistakes are the most common disaster with fdisk/mkfs.

Interactive GPT data disk (sketch)

```
sudo fdisk /dev/sdb
# g          → new GPT
# n          → new partition (accept defaults for whole disk, or +200G etc.)
# t          → type if needed (Linux filesystem is typical; LVM has its own type)
# p          → review
# w          → write (point of no return for the old table)
```

Quit with `q` instead of `w` if anything looks wrong. Partition numbers and paths: `/dev/sdb1`, NVMe uses `/dev/nvme0n1p1`.

Kernel re-read after write

```
sudo partprobe /dev/sdb
# or: sudo blockdev --rereadpt /dev/sdb
lsblk /dev/sdb
cat /proc/partitions | grep sdb
```

If nodes do not appear, a reboot or removing leftover holders may be required.

Next steps after partitioning

```
sudo mkfs.ext4 -L data /dev/sdb1
# or: sudo pvcreate /dev/sdb1 && … LVM …
lsblk -f /dev/sdb
sudo blkid /dev/sdb1
```

fdisk only lays out partitions; filesystems and fstab are separate.

Scripted dump / restore (sfdisk)

```
sudo sfdisk -d /dev/sdb > sdb.layout
# restore to same-size disk (DESTRUCTIVE - verify target):
# sudo sfdisk /dev/sdb < sdb.layout
```

Use for clone pipelines; still verify device identity before restore.

EFI system partition type (boot disk work)

```
# In fdisk: type often "EFI System" (GPT). Wrong type can break boot loaders.
# Prefer known-good installers for OS disks; manual ESP edits need care.
```

OS boot disks deserve extra caution — prefer installer tooling unless you know the boot chain.

Understanding Output

`fdisk -l` shows device path, size, sector size, disk identifier, and a table of partitions (number, start/end sectors, size, type). Modern util-linux fdisk aligns partitions automatically for 4K devices. GPT entries have PARTUUIDs visible via `lsblk -o NAME,PARTUUID` or `blkid`.

Notes & Pitfalls

- Device naming: SATA/SCSI `/dev/sdX`, NVMe `/dev/nvmeXnY` + `pN` partitions, virtio often `/dev/vdX`.
- Alignment is usually automatic; avoid manual cylinder math on modern disks.
- Changing a live root partition table is high risk — use rescue media.
- Residual filesystem signatures: wipe carefully (`wipefs`) only when intentional.
- Cloud images may already be GPT with growable partitions — don't blindly recreate.
- After type change to LVM/RAID, tools expect the matching stack (`pvcreate`, mdadm, …).

Related Commands

- `parted` / `gdisk` — alternative partition editors
- `sfdisk` — scriptable dump/restore
- `mkfs` — create filesystem on a partition
- `lsblk` / `blkid` — verify layout and UUIDs
- `partprobe` / `blockdev --rereadpt` — refresh kernel view
- `wipefs` — inspect/clear signatures (destructive)

Additional Resources

- `man fdisk`
- `man sfdisk`
- `man parted`

mkfs

Overview

`mkfs` builds a filesystem on a block device (partition, LV, loop file). On Ubuntu you usually call the type-specific tools: `mkfs.ext4`, `mkfs.xfs`, `mkfs.vfat`, `mkfs.btrfs`. **This destroys existing data** on the target. Pair with `lsblk`/`blkid` for identity, `fdisk` for partitions, and `mount`/`fstab` for use.

Syntax

```
sudo mkfs -t type [fs-options] device
sudo mkfs.ext4 [options] device
sudo mkfs.xfs [options] device
sudo mkfs.vfat [options] device
```

Common Options

Options vary by filesystem. Operators commonly use:

Tool / option	Description
`mkfs.ext4 -L label`	Volume label
`mkfs.ext4 -m percent`	Reserved blocks for root (default ~5%)
`mkfs.ext4 -N inodes`	Inode count (rare; plan for many tiny files)
`mkfs.xfs -L label`	Label
`mkfs.xfs -f`	Force overwrite existing FS
`mkfs.vfat -F 32 -n NAME`	FAT32 + short volume name
`mkfs -t type`	Generic front-end

Swap is **not** mkfs: use `mkswap`.

Safety

1. Confirm device with `lsblk -o NAME,SIZE,MODEL,SERIAL,FSTYPE,MOUNTPOINTS` and `sudo blkid`.
2. Ensure the target is **unmounted** and not in use (`findmnt`, `lsof`, `fuser`).
3. Never `mkfs` the wrong disk on multi-disk or cloud hosts — match size/serial.
4. Prefer operating on `/dev/disk/by-id/…` or by-uuid only **after** you know the new FS UUID (identity changes on format).
5. Have backups; there is no undo.

Key Use Cases

1. Format a new data partition after `fdisk`
2. Format an LVM logical volume
3. Prepare USB media (vfat/exfat) for interchange
4. Rebuild a filesystem after intentional wipe (with backups)

Examples with Explanations

Verify target, then ext4 data disk

```
lsblk -f
lsblk -o NAME,SIZE,MODEL,SERIAL,MOUNTPOINTS /dev/sdb
sudo findmnt /dev/sdb1   # should be empty / not mounted
sudo mkfs.ext4 -L data /dev/sdb1
sudo blkid /dev/sdb1
sudo mkdir -p /mnt/data
sudo mount /dev/sdb1 /mnt/data
df -h /mnt/data
```

Label `-L data` shows up in `lsblk -f` and can be used as `LABEL=data` in fstab (UUID is still more unique).

XFS data volume

```
sudo mkfs.xfs -f -L data /dev/sdb1
```

`-f` forces if a filesystem signature already exists. XFS is common for large sequential workloads; repair path is `xfs_repair`, not classic `fsck` semantics.

FAT32 for USB interoperability

```
sudo mkfs.vfat -F 32 -n KEY /dev/sdc1
```

Good for firmware, Windows interchange, EFI-related media. Volume name length limits apply (`-n`).

Reduce ext* reserved space on large data disks

```
sudo mkfs.ext4 -m 0.5 -L bulk /dev/sdb1
# or later: sudo tune2fs -m 1 /dev/sdb1
```

Default ~5% reserved for root wastes terabytes on multi-TB bulk disks. Root filesystem often keeps a higher reserve on purpose.

LVM logical volume

```
# after lvcreate …
sudo mkfs.ext4 -L pgdata /dev/mapper/vg-data
# or: /dev/vg/data symlink
lsblk -f /dev/mapper/vg-data
```

Format the LV (mapper node), not a raw PV member disk.

Capture UUID for fstab

```
UUID=$(sudo blkid -s UUID -o value /dev/sdb1)
echo "UUID=$UUID  /mnt/data  ext4  defaults,nofail  0  2"
# install into /etc/fstab after review, then:
sudo mount -a
```

`nofail` helps boot continue if an optional data disk is missing (tune to policy).

Whole-disk vs partition

```
# Prefer a partition table even for single-use data disks (clearer, ESP-friendly patterns).
# mkfs on whole /dev/sdb works but confuses some tools and dual-boot assumptions.
sudo fdisk /dev/sdb   # create one partition first
sudo mkfs.ext4 -L data /dev/sdb1
```

Understanding Output

Successful mkfs prints progress and superblock/UUID details (ext*). Afterward:

```
lsblk -f /dev/sdb1
sudo blkid /dev/sdb1
```

Expect new `FSTYPE`, `UUID`, and optional `LABEL`. Old UUIDs in fstab become stale — update them.

Notes & Pitfalls

- **Irreversible** for practical purposes; snapshots/backups first.
- XFS: use `xfs_repair` offline; `fsck.xfs` does little.
- btrfs/zfs have their own create tools and operational models.
- Cloud disks: detach/attach order can rename `/dev/sdX` — use UUID in fstab.
- Encryption: `cryptsetup luksFormat` then mkfs on `/dev/mapper/…`, not the raw LUKS device path alone without opening.
- Filesystem choice: ext4 default on Ubuntu root; XFS common for large data; vfat for portable USB.

Related Commands

- `fdisk` / `parted` — partitions before mkfs
- `blkid` / `lsblk -f` — verify identity after format
- `tune2fs` — tune ext* without reformat
- `mkswap` / `swapon` — swap
- `mount` / `umount` / fstab — use the new FS
- `wipefs` — inspect/clear old signatures

Additional Resources

- `man mkfs`
- `man mkfs.ext4`
- `man mkfs.xfs`

fsck

Overview

`fsck` checks and optionally repairs filesystems. It is a front-end that dispatches to type-specific helpers (`fsck.ext4` → `e2fsck`, etc.). **Never run a repairing check on a mounted read-write filesystem** — unmount first, or use recovery/single-user media for the root FS. XFS is special: use `xfs_repair` offline; `fsck.xfs` is effectively a no-op.

Syntax

```
fsck [options] [-t type] [device|mountpoint...]
fsck.ext4 [options] device
e2fsck [options] device
```

Common Options

Option	Description
`-t type`	Filesystem type (or from blkid/fstab)
`-A`	All filesystems from fstab
`-R`	Skip root (with `-A`)
`-M`	Skip mounted filesystems
`-N`	Dry-run — show what would run
`-y`	Assume "yes" to all repairs (careful)
`-n`	Assume "no" / report-only (ext* via e2fsck)
`-f`	Force check even if clean
`-C`	Progress bar (ext*)
`-V`	Verbose
`-T`	Hide title

Safety

1. Identify the device: `lsblk -f`, `sudo blkid`.
2. Unmount: `sudo umount /dev/sdXN` (or boot rescue for root).
3. Prefer **report-only** (`-n` / e2fsck `-n`) before auto-yes (`-y`).

4. **XFS:** `sudo xfs_repair -n /dev/…` then `xfs_repair` without `-n`; do not expect `fsck.xfs` to repair.
5. Repairs can delete corrupt inodes/data — backups matter.
6. Do not fsck the live RW root; use recovery, live USB, or vendor maintenance mode.

Key Use Cases

1. After unclean shutdown / journal errors
2. Before reusing a suspect data disk
3. Boot-time automatic checks (fstab pass field / systemd)
4. Forced ext* check after resize or corruption reports

Examples with Explanations

Report-only on unmounted ext4

```
lsblk -f /dev/sdb1
sudo umount /mnt/data 2>/dev/null || true
sudo findmnt /dev/sdb1 && { echo "still mounted - abort"; exit 1; }
sudo fsck -n /dev/sdb1
# or explicit:
sudo e2fsck -n /dev/sdb1
```

`-n` opens read-only and reports problems without writing fixes.

Interactive repair

```
sudo umount /mnt/data
sudo fsck -f /dev/sdb1
```

Answer prompts carefully. `-f` forces a full check even if the FS is marked clean.

Auto-yes repair (after backup / when you accept loss)

```
sudo e2fsck -f -y /dev/sdb1
```

Convenient for automation and clearly disposable data; risky on irreplaceable volumes without backups.

Dry-run which helpers would run

```
sudo fsck -N -A
sudo fsck -N /dev/sdb1
```

Shows the planned `fsck.<type>` invocations without executing checks.

Force ext4 check with progress

```
sudo e2fsck -f -C 0 /dev/sdb1
```

Progress on file descriptor 0's tty-style reporting; useful on large volumes.

Root filesystem

```
# Boot Ubuntu recovery / live USB, find root device:
lsblk -f
# ensure root is not mounted RW, then:
sudo e2fsck -f /dev/nvme0n1p2
# systemd may run fsck on boot when dirty flags are set - check:
journalctl -b | grep -i fsck
```

Plan downtime. For cloud VMs, attach the disk to a rescue instance and fsck there.

XFS path

```
sudo umount /mnt/data
sudo xfs_repair -n /dev/sdb1    # no-modify scan
sudo xfs_repair /dev/sdb1       # repair
# NOT: fsck.xfs  (does not repair)
```

Mount and verify after clean check

```
sudo fsck -n /dev/sdb1
sudo mount /dev/sdb1 /mnt/data
dmesg | tail
df -h /mnt/data
```

Understanding Output / Exit codes

Messages vary by helper. e2fsck prints phases (superblock, bitmaps, inodes, directory structure, …).

Exit code is a **bitmask** (bits can combine):

Bit	Meaning
0	No errors
1	File system errors corrected
2	System should be rebooted
4	File system errors left uncorrected
8	Operational error
16	Usage or syntax error
32	Fsck canceled by user
128	Shared library error

```
sudo fsck -n /dev/sdb1
echo $?   # interpret with the table above
```

Notes & Pitfalls

- Mounted RW checks are unsafe; some tools refuse, others can destroy data.
- fstab sixth field controls boot fsck order/pass; `0` skips.
- btrfs/zfs use native check/scrub tools, not classic fsck workflows.
- Hardware faults: pair with `smartctl` — repairing FS won't fix a dying disk.
- Journal-only recovery often happens automatically on mount for ext*; full fsck is for deeper issues.
- `-y` can remove damaged files to restore consistency — that is success for fsck, data loss for you.

Related Commands

- `umount` / `mount` — offline requirement
- `tune2fs -l` — ext* superblock / mount counts
- `xfs_repair` / `btrfs check` — type-specific
- `smartctl` — disk health (smartmontools)
- `journalctl -b` — boot-time fsck messages
- `blkid` / `lsblk -f` — identify devices

Additional Resources

- `man fsck`
- `man e2fsck`
- `man xfs_repair`

mount

Overview

`mount` attaches a filesystem (block device, network share, bind path, tmpfs, …) onto a directory in the single Linux directory tree. Persistent mounts belong in `/etc/fstab` or systemd `.mount` units; interactive `mount` is for live operations and troubleshooting.

Syntax

```
mount [-t type] [-o options] device dir
mount                        # list mounts
mount -a                     # all fstab entries due at boot
mount -o remount,opts dir
```

Common Options

Option	Description
`-t type`	Filesystem type (`ext4`, `xfs`, `nfs4`, `cifs`, `tmpfs`, `overlay`, …)
`-o opts`	Mount options (`ro`, `noexec`, `nosuid`, `nodev`, `defaults`, …)
`-a`	Mount all appropriate fstab entries
`-r` / `-w`	Read-only / read-write
`--bind` / `--rbind`	Bind mount (recursive)
`-v`	Verbose
`--source` / `--target`	Explicit long options

Key Use Cases

1. Attach disks and USB volumes
2. Mount NFS/CIFS shares
3. Bind mounts for chroots, containers, and rebuilds

4. Temporary tmpfs workspaces

5. Remount root or data with new options

Safety

- Wrong device → wrong data exposure. Verify with `lsblk -f` and `blkid` **before** mounting.
- Prefer `UUID=` / `LABEL=` in fstab over volatile `/dev/sdX` names.
- Network mounts can hang the shell on outage — consider `soft`/`timeo` options and separate automount units.
- Unmount cleanly (`umount`) before yanking USB or unplugging SAN LUNs.

Examples with Explanations

Inventory

```
findmnt
findmnt -t ext4,xfs
mount | column -t
findmnt -J | jq .          # if jq installed
```

Prefer `findmnt` for tree and JSON views.

Mount a partition by UUID

```
lsblk -f
sudo mkdir -p /mnt/data
sudo mount UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx /mnt/data
df -h /mnt/data
```

Read-only mount (forensics / recovery)

```
sudo mount -o ro /dev/sdb1 /mnt/data
```

NFS

```
sudo mount -t nfs4 -o rw,hard,timeo=600 server.example:/export/path /mnt/nfs
```

Options trade durability vs hang behavior — know your storage SLA.

CIFS / SMB

```
sudo mount -t cifs //server/share /mnt/share \
  -o credentials=/root/.smbcred,uid=1000,gid=1000,iocharset=utf8
```

Keep credentials files mode `600`.

Bind mounts

```
sudo mount --bind /var/www /mnt/www-view
sudo mount --rbind /home /mnt/chroot/home
```

Bind mounts share the same filesystem; `rbind` includes submounts.

tmpfs

```
sudo mount -t tmpfs -o size=1G,mode=0755 tmpfs /mnt/scratch
```

RAM-backed; contents vanish on unmount/reboot.

Remount

```
sudo mount -o remount,ro /
sudo mount -o remount,rw /
```

Common in recovery environments.

fstab-driven

```
# /etc/fstab example:
# UUID=…  /mnt/data  ext4  defaults,nofail  0  2
sudo mount -a
sudo mount /mnt/data
findmnt /mnt/data
```

`nofail` avoids boot hangs if the disk is absent (laptops/USB).

Overlay (container-style)

```
sudo mount -t overlay overlay -o lowerdir=/base,upperdir=/upper,workdir=/work /merged
```

Used heavily by container engines; easy to get wrong — read `overlay` docs before production use.

Understanding Output

With no arguments, `mount` lists source, target, type, and options. Exit status non-zero means the mount failed (busy target, bad superblock, missing helper binary for NFS/CIFS, permission denied).

Notes & Pitfalls

- Helpers live as `mount.nfs`, `mount.cifs` — missing packages cause cryptic errors.
- "Target is busy" on umount → `lsof +f -- /mnt/data`, `fuser -vm /mnt/data`.
- Systemd may own the mount: `systemctl status mnt-data.mount`.
- `user`/`users` fstab options allow non-root mount/umount with restrictions.
- Mount namespaces (containers) mean `findmnt` inside a pod is not the host's view.

Related Commands

- `umount` — detach
- `lsblk` / `blkid` — identify devices
- `findmnt` — rich listing
- `df` / `du` — space usage
- `losetup` — loop devices for images
- `fstab(5)` — persistence

Additional Resources

- `man mount`
- `man fstab`
- `man systemd.mount`

umount

Overview

`umount` detaches a mounted filesystem from the directory tree. Use it before safe device removal, before offline `fsck`/`mkfs`, or when tearing down bind/NFS mounts. Prefer the **mountpoint** path over the device node. If the target is busy, find holders with `lsof`/`fuser`/`findmnt` rather than forcing blindly.

Syntax

```
umount [options] <mountpoint|device>
umount -a [options]
```

Common Options

Option	Description
`-l`	Lazy unmount — detach now, cleanup when busy refs drop
`-f`	Force (mainly stale NFS); dangerous on local disks
`-r`	If unmount fails, try remount read-only
`-v`	Verbose
`-a`	Unmount all (respects filters; use carefully)
`-t type`	Limit by filesystem type (with `-a`)
`-R`	Recursive unmount of target and children (newer util-linux)

Key Use Cases

1. Clean detach of USB/data disks before unplug
2. Unmount before `fsck`, repartition, or `mkfs`
3. Clear busy or stale NFS mounts
4. Tear down bind mounts in chroots/containers/scripts

Safety

- Do **not** unmount `/` on a live system.
- Prefer stopping writers (services, shells with cwd on the mount) over `-f` on local media.
- Lazy (`-l`) hides problems; processes may still hold files until exit.
- Unmounting active databases or VMs can corrupt data — shut them down first.

Examples with Explanations

Unmount by mountpoint (preferred)

```
sudo umount /mnt/data
```

Matches how you mounted it; unambiguous when one device has multiple mounts.

Unmount by device

```
sudo umount /dev/sdb1
```

Works when you know the block device; fails if not mounted or still busy.

Confirm before and after

```
findmnt /mnt/data
lsblk -f /dev/sdb
sudo umount /mnt/data
findmnt /mnt/data || echo "unmounted"
```

Busy mount diagnosis

```
findmnt /mnt/data
sudo lsof +f -- /mnt/data
sudo fuser -vm /mnt/data
# common causes: shell cwd on mount, open logs, NFS locks, container bind
cd /   # if your shell is the culprit
sudo umount /mnt/data
```

“Target is busy” almost always means open files or a process cwd under the tree.

Lazy unmount

```
sudo umount -l /mnt/data
```

Detaches from the namespace immediately; actual teardown when references drop. Use for stuck mounts when you understand leftover processes may still write until they exit.

Force NFS (last resort)

```
sudo umount -f /mnt/nfs
# or lazy: sudo umount -l /mnt/nfs
```

For **stale** NFS when the server is gone. Avoid `-f` as a habit on local ext4/xfs.

Remount read-only fallback

```
sudo umount -r /mnt/data
```

If full unmount fails, try to freeze writes via remount-ro (util-linux behavior with `-r`).

Multiple mounts / bind trees

```
findmnt -R /mnt/data
sudo umount -R /mnt/data   # if supported: recursive
# else unmount children first, then parent
```

Binds and nested mounts must be torn down from the leaves upward if recursive umount is unavailable.

Before fsck

```
sudo umount /mnt/data
sudo fsck -n /dev/sdb1
```

Never run repairing fsck on a read-write mounted filesystem.

Understanding Output

Success is silent (exit 0). Failures print to stderr:

Message	Typical cause
`target is busy`	Open files / cwd / mounts beneath
`not mounted`	Wrong path or already unmounted
`must be superuser`	Need root for most system mounts

Notes & Pitfalls

- Systemd may manage mounts as `*.mount` units — `systemctl stop mnt-data.mount` can be cleaner than raw umount.
- USB: unmount **then** wait for LED idle before unplug; otherwise risk corruption.
- `udisksctl unmount -b /dev/sdb1` is the desktop-friendly path on Ubuntu workstations.
- Open-but-deleted files on other filesystems don't block this umount; busy means *this* mount's tree.
- In containers, mounts may be shared/slave — unmount semantics follow mount propagation.

Related Commands

- `mount` — attach filesystems
- `findmnt` — show what is mounted
- `lsblk` — device mountpoint map
- `lsof` / `fuser` — who is busy
- `fsck` — check offline filesystems
- `systemctl` — systemd mount units

Additional Resources

- `man umount`
- `man findmnt`

dd

Overview

`dd` copies and converts data at the block level. Operators use it to image disks/USBs, write ISO files to removable media, create sparse files, and dump raw device ranges. It is powerful and **easy to destroy the wrong disk** — always verify source and destination with `lsblk` first. For day-to-day file copies prefer `cp` or `rsync`; for cloning partitions with progress, many prefer `pv` + `dd` or dedicated tools (`Clonezilla`, `partclone`).

Syntax

```
dd if=INPUT of=OUTPUT [bs=BYTES] [count=N] [status=progress] [conv=OPTS] [oflag=OPTS] ...
```

Operands are `key=value` (GNU style). Order of operands generally does not matter.

Common Options

Option	Description
`if=FILE`	Input file/device (default: stdin)
`of=FILE`	Output file/device (default: stdout)
`bs=BYTES`	Read and write `BYTES` at a time (e.g. `4M`)
`ibs=` / `obs=`	Separate input/output block sizes
`count=N`	Copy only `N` input blocks
`skip=N`	Skip `N` input blocks
`seek=N`	Skip `N` output blocks
`status=progress`	Periodic transfer stats (GNU)
`conv=fsync`	fsync output file before finish
`conv=noerror,sync`	Continue on read errors; pad bad blocks (imaging damaged media)
`conv=sparse`	Try to seek over zero blocks on output
`iflag=fullblock`	Accumulate full input blocks (safer pipes)
`oflag=direct`	Attempt O_DIRECT writes (device-dependent)
`oflag=sync`	Synchronous writes each block (slow, durable)

Size suffixes (GNU): `K`, `M`, `G` (1024-based); `KB`, `MB`, `GB` (1000-based). Check `man dd` for your coreutils version.

Safety

`dd` will overwrite the destination without asking. Wrong `of=` is a classic way to wipe a system disk.

1. **Identify devices first** — never guess `/dev/sdX` vs `/dev/nvme…`:

   ```
   lsblk -o NAME,SIZE,TYPE,TRAN,MODEL,MOUNTPOINTS,SERIAL
   # unplug/replug USB and re-run to see which node appears
   ```

2. **Confirm the USB/target is unmounted** before writing a whole disk image.
3. Prefer **`status=progress`** so a hung or wrong device is obvious sooner.
4. After writing boot media, run **`sync`** (or `conv=fsync`) before unplugging.
5. Prefer **read-only checks** (`if=` only, or write to a file first) when learning.
6. On multi-disk hosts, label targets and double-check `SERIAL`/`MODEL`.
7. **Do not** copy from/to a mounted filesystem's block device for "live backup" unless you know the consistency implications.

If unsure of the device node: **stop**. Physical write-protect switches and working on a spare machine reduce risk.

Examples with Explanations

List devices before any write

```
lsblk -o NAME,SIZE,TYPE,TRAN,MODEL,MOUNTPOINTS
# Example: USB stick shows as sdb, not sda (system disk)
```

Always map physical media → `/dev/...` before setting `of=`.

Write an ISO to a USB stick (destructive to the stick)

```
# 1) Identify stick - here assumed /dev/sdX (REPLACE X)
lsblk -o NAME,SIZE,TRAN,MODEL,MOUNTPOINTS

# 2) Unmount any auto-mounted partitions
sudo umount /dev/sdX* 2>/dev/null || true

# 3) Write whole-disk image
sudo dd if=ubuntu-24.04-desktop-amd64.iso of=/dev/sdX bs=4M status=progress conv=fsync oflag

# 4) Ensure buffers flushed before yank
```

```
sync
```

`of=` is the **whole disk** (`/dev/sdX` / `/dev/nvme1n1`), not a partition, for hybrid ISOs. `bs=4M` is a practical throughput compromise; `conv=fsync` flushes on completion.

Create a raw disk image (backup a USB)

```
sudo umount /dev/sdX* 2>/dev/null || true
sudo dd if=/dev/sdX of=usb-backup-$(date +%F).img bs=4M status=progress conv=fsync
ls -lh usb-backup-*.img
```

Stores every byte of the device (including free space). Compress afterward if needed: `gzip -1 usb-backup.img`.

Restore an image to a USB

```
sudo umount /dev/sdX* 2>/dev/null || true
sudo dd if=usb-backup.img of=/dev/sdX bs=4M status=progress conv=fsync
sync
```

Destination size must be image size. Again: verify `of=` with `lsblk`.

Sparse empty file (test disk image)

```
dd if=/dev/zero of=disk.img bs=1M count=0 seek=1024
# or: truncate -s 1G disk.img
ls -lh disk.img
du -h disk.img          # sparse: little real space used
```

Useful for loop devices (`losetup`) and VM-style experiments without writing 1 GiB of zeros immediately.

Sample first megabyte (forensics / header peek)

```
sudo dd if=/dev/sdX bs=1M count=1 status=none | xxd | head
sudo dd if=/dev/sdX bs=512 count=1 of=mbr.bin
```

Read-only sampling of boot sectors or magic headers without imaging the whole device.

Damaged media (best-effort copy)

```
sudo dd if=/dev/sdX of=recovered.img bs=4M conv=noerror,sync status=progress
```

Continues past read errors and pads failed blocks. For serious recovery prefer `ddrescue` (`gddrescue` package) — better logging and retry logic.

Progress with pv (when installed)

```
sudo apt install pv    # if needed
sudo pv -tpreb ubuntu.iso | sudo dd of=/dev/sdX bs=4M conv=fsync
```

`pv` gives a clearer progress bar; still requires correct `of=`.

Wipe a USB free-space / reuse (destructive)

```
# Optional: overwrite with zeros (slow) or random
sudo dd if=/dev/zero of=/dev/sdX bs=4M status=progress conv=fsync
# Then re-partition / mkfs as needed
```

This destroys all partitions and data on the device. For secure erase of SSDs prefer vendor tools / `blkdiscard` where appropriate — sequential `dd` is not always the right model for flash.

Notes

- GNU `dd` on Ubuntu supports `status=progress`; some embedded `dd` builds do not.
- Block size affects speed more than correctness for full copies; very small `bs` is slow, huge `bs` can increase memory pressure.
- Writing an ISO with `cp`/`cat` to the device node also works on Linux; `dd` remains common in docs and scripts.
- USB path stability: prefer `/dev/disk/by-id/...` when scripting if nodes reorder across boots.
- Exit status is non-zero on incomplete/error copies — check it in scripts.
- `oflag=direct` can fail on some filesystems/devices; drop it if `dd` errors immediately.

Related Commands

- `lsblk` / `blkid` — identify devices safely
- `sync` — flush dirty pages after writes
- `umount` — unmount before imaging
- `losetup` — attach image files as loop devices
- `pv` — pipe progress

- `ddrescue` — resilient imaging of failing disks
- `wipefs` / `mkfs` — clear signatures / make filesystems after wipe
- `cp` / `rsync` — file-level copies (usually better for files)

Additional Resources

- `man dd`
- `info coreutils 'dd invocation'`

sync

Overview

`sync` flushes filesystem buffers: dirty page cache and pending metadata are written to storage. Use it before removing USB media, after bulk `dd`/`cp` to removable devices, or when you need a durable checkpoint before power loss. Modern kernels and `umount` already sync as part of a clean unmount; calling `sync` explicitly is still a useful operator habit for removable media and scripts.

Syntax

```
sync [options] [file...]
```

With no arguments, flushes all filesystems (global sync). With file arguments (GNU), flushes those files and the filesystems that hold them.

Common Options

Option	Description
(none)	Sync all (classic usage)
`FILE...`	Sync data for listed files (GNU coreutils)
`-d`, `--data`	Sync data only, not metadata (when supported)
`-f`, `--file-system`	Sync the filesystems that contain the files

BusyBox/`sync` on minimal systems may only support the no-argument form.

Safety

- `sync` is **not a substitute for unmounting**. Always `umount` (or eject) removable volumes when possible so the kernel finishes metadata updates cleanly.
- `sync` does **not** make a crash-consistent snapshot of multi-file databases by itself — use application flush/quiesce or LVM/ZFS snapshots for that class of problem.
- Calling `sync` under heavy write load can stall the system briefly while I/O drains — expected, not a hang in most cases.

Examples with Explanations

Flush everything (classic)

```
sync
```

Blocks until outstanding dirty data is queued to devices (and typically completed for the global case). Common after imaging USBs or large copies.

After writing a USB with dd

```
sudo dd if=image.iso of=/dev/sdX bs=4M status=progress conv=fsync
sync
# safely remove when activity LED is idle
```

`conv=fsync` already fsyncs the output; an extra `sync` is cheap insurance before unplug.

Sync specific file (GNU)

```
cp bigfile /media/usb/bigfile
sync /media/usb/bigfile
# or
sync -f /media/usb/bigfile
```

Narrows flush to the file/filesystem of interest instead of the whole machine.

Safe removable-media workflow

```
cp -a project/ /media/$USER/STICK/
sync
sudo umount /media/$USER/STICK
# or: udisksctl unmount -b /dev/sdX1
```

Copy → sync → unmount is the reliable order for thumb drives.

In scripts before reboot/power events

```
install -m 644 app.conf /etc/myapp/app.conf
sync
sudo systemctl restart myapp
```

Ensures config hit disk before a service that might race with a crash/reboot path.

Drop caches (admin debug — not for production routine)

```
sync
echo 3 | sudo tee /proc/sys/vm/drop_caches
```

`sync` first so drop_caches does not discard dirty data. Used for I/O benchmarks; not a general cleanup step.

Notes

- Journaling filesystems reduce risk of total corruption, but applications can still lose unflushed writes on power loss without `fsync`/`fdatasync`.
- `dd … conv=fsync` and `cp` with certain flags interact with durability differently than a bare write + delayed writeback.
- Desktop "eject" / file-manager safe-remove typically unmounts (which syncs); CLI operators should mirror that with `umount`.
- Network filesystems: `sync` semantics depend on the protocol and mount options (`nfs`, `cifs`); local flush may not equal server durability.

Related Commands

- `dd` — block copy; pair with sync/fsync
- `umount` / `findmnt` — clean detach
- `mount -o sync` — mount with synchronous I/O (slow)
- `fsync` (syscall) / language APIs — per-fd durability
- `hdparm` / device caches — hardware write-cache caveats

Additional Resources

- `man sync`
- `man 2 fsync`

losetup

Overview

`losetup` associates regular files with **loop devices** (`/dev/loopN`) so the kernel can treat an image file like a block device—mount ISOs, inspect disk images, or access partitions inside a raw `.img`.

Syntax

```
losetup [options] [loopdev] [file]
losetup [options] -f file
```

Common Options

Option	Description
`-a`, `--all`	List active loop devices
`-f`, `--find`	Find first unused loop device
`-f --show`	Find, attach, print device path
`-d`, `--detach`	Detach
`-D`, `--detach-all`	Detach all (careful)
`-j file`	Show loops for file
`-r`, `--read-only`	Read-only
`-P`, `--partscan`	Scan partition table → `/dev/loopNp1…`
`-b SIZE`	Logical block size
`-o OFFSET`	Data offset
`--sizelimit`	Limit size
`-L`, `--nooverlap`	Refuse overlapping

Key Use Cases

1. Mount disk/ISO images
2. Access partitions inside a raw disk image
3. Filesystem experiments without real disks
4. Detach leftover loops after failed scripts

Examples with Explanations

Attach and list

```
sudo losetup -fP --show disk.img
sudo losetup -a
lsblk
```

Mount ISO

```
sudo losetup -f --show -r image.iso
# or directly:
sudo mount -o loop,ro image.iso /mnt/iso
```

Modern `mount -o loop` calls losetup for you.

Partitions inside image

```
sudo losetup -fP --show disk.img
lsblk /dev/loop0
sudo mount /dev/loop0p1 /mnt/p1
```

Detach

```
sudo umount /mnt/p1
sudo losetup -d /dev/loop0
sudo losetup -j disk.img
```

One-liner recipes

```
# Attach, print, and lsblk
dev=$(sudo losetup -fP --show disk.img); echo "$dev"; lsblk "$dev"

# Clean orphan loops (review first!)
sudo losetup -a
# sudo losetup -D   # nuclear option
```

Notes & Pitfalls

- Always **umount before detach**.
- Sparse image files and sparse loop mappings can confuse free-space calculations.
- Permission: root required for setup.
- Snap/container environments may restrict loop devices.
- For qcow2 use `qemu-nbd` or `guestmount`, not raw losetup alone.

2026-relevant notes

- Cloud "custom image" workflows still use loop/nbd for injection of SSH keys and cloud-init.
- Prefer `udisksctl loop-setup` on desktops for user-friendly attach.
- `losetup -P` + `lsblk` is the standard recovery path for raw VM disks copied as files.

Related Commands

- `mount` / `umount` — filesystem attach
- `lsblk` / `blkid` — inspect
- `kpartx` — map partitions (alternative)
- `qemu-nbd` — qcow2 as block

Additional Resources

- `man losetup`
- util-linux documentation

duf

Overview

`duf` is a modern, colorful **disk usage / free space** viewer — a friendlier alternative to `df -h`. It groups local, network, special, and inaccessible filesystems with readable tables. Optional install (`apt install duf`, etc.). Keep `df` for scripts and minimal systems.

Syntax

```
duf [options] [device/path...]
```

Common Options

Option	Description
`-all`	Show all filesystems including pseudo
`-only local|network|fuse|special|loops|binds`	Filter types
`-only-mp` / `-only-device`	Filter by mount/device strings
`-hide`	Hide types
`-output`	Choose columns
`-sort`	Sort key (`size`, `used`, `avail`, `usage`, …)
`-json`	JSON output
`-style`	UI style
`-theme`	Color theme
`-width`	Width
`-inodes`	Inode view
`-avail-threshold` / `-usage-threshold`	Highlight thresholds

Confirm with `duf --help` for your version.

Examples with Explanations

Everyday

```
duf
duf -only local
duf /
duf -sort usage
```

Inodes

```
duf -inodes
df -ih
```

JSON

```
duf -json | jq .
```

Hide noise

```
duf -hide special,loops,binds
# or only local disks:
duf -only local
```

Compare with df

```
df -hT
duf
findmnt
```

Threshold awareness

```
duf -usage-threshold 0.8
```

Highlights nearly full mounts when supported.

Notes / Pitfalls

- Not portable to bare recovery images — know `df -hT`.
- Colors/unicode need a capable terminal.
- Bind mounts and snap loops can still clutter — filter them.
- JSON schema may change across versions.
- Network FS free space can be misleading (quotas, cloud buckets).

2026-relevant notes

- Excellent interactive upgrade alongside `dust`/`eza`/`bat`.
- Monitoring systems should still scrape `node_exporter`/`df` metrics, not duf.
- Btrfs/ZFS may need native tools for pool-level free space accuracy.

Related Commands

- `df` — portable free space
- `findmnt` — mount tree
- `du` / `dust` / `ncdu` — directory usage
- `lsblk` — block devices
- `btrfs filesystem usage` / `zpool list` — volume managers

Additional Resources

- `duf --help`
- muesli/duf

dust

Overview

`dust` is a modern **`du` alternative** that shows disk usage as a readable tree/bars, helping you find large directories quickly. Written in Rust; optional install. For portable scripts use `du -sh`; for interactive TUI browsing also consider `ncdu`.

Syntax

```
dust [options] [directory...]
```

Common Options

Option	Description
`-d DEPTH`, `--depth`	Max depth
`-n N`, `--number-of-lines`	Entries to show
`-r`, `--reverse`	Reverse natural order
`-p`, `--full-paths`	Full paths
`-s`, `--apparent-size`	Apparent size vs allocated
`-i`, `--ignore-directory`	Ignore path
`-X file`	Ignore from file
`-e`, `--filter`	Include only matching names
`-v`, `--invert-filter`	Invert filter
`-c`, `--no-colors`	Disable colors
`-b`, `--no-percent-bars`	No bars
`-z`, `--min-size`	Min size threshold
`-t`, `--file-types`	Group by filetype
`-w`, `--terminal-width`	Width

Check `dust --help` for current flags.

Examples with Explanations

Find big directories

```
dust
dust -d 2 /var
dust -n 30 /home
```

Apparent size

```
dust -s
du -sh --apparent-size *
```

Ignore noise

```
dust -i node_modules -i .git
dust -X .dustignore
```

File types

```
dust -t
```

Compare with du / ncdu

```
du -h -d 1 /var | sort -h
dust -d 1 /var
ncdu /var
```

Min size filter

```
dust -z 100M /
```

Scripts: stick to du

```
du -sb /var/lib/docker
# dust is for humans
```

Notes / Pitfalls

- Permission errors on other users' dirs skew totals — run with appropriate rights.
- Hard links / snapshots / sparse files: understand apparent vs allocated.
- Network filesystems: slow; prefer scoped paths.
- Not preinstalled on servers.
- Colors in pipes may need `-c`.

2026-relevant notes

- Great laptop/dev host tool; production forensics still document `du`/`find`.
- Container disk growth: dust the overlay/graph roots carefully with host tools.
- Pair with `duf` for free space vs `dust` for "what ate it".

Related Commands

- `du` — portable usage
- `ncdu` — interactive TUI
- `duf` / `df` — free space
- `find -size` — size predicates
- `btrfs filesystem du` — special FS accounting

Additional Resources

- `dust --help`
- bootandy/dust

parted

Overview

`parted` creates and modifies **partition tables** (GPT/MSDOS) on block devices. Prefer it for GPT disks and scripted partitioning; `fdisk` remains common for interactive MBR/GPT on small admin tasks. Mistakes destroy data — work on the correct device only.

Syntax

```
sudo parted [options] device [command...]
sudo parted device print
```

Common Commands (interactive or CLI)

Command	Description
`print`	Show partition table
`mklabel gpt|msdos`	Create new disklabel (**wipes**)
`mkpart`	Create partition
`rm N`	Delete partition number N
`resizepart`	Resize partition boundary
`unit GiB`	Display/input units
`align-check`	Alignment check

Safety

- **Double-check** `lsblk` / `by-id` paths before any `mklabel` or `rm`.
- `mklabel` destroys the existing table.
- After partitioning, create filesystems with `mkfs` and update `fstab` by UUID.
- On disks with LVM/RAID, coordinate with `pvcreate`/`mdadm` first.

Examples with Explanations

Print table

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,UUID
sudo parted /dev/sdb print
sudo parted -l
```

GPT with one partition (scripted)

```
sudo parted -s /dev/sdb mklabel gpt
sudo parted -s /dev/sdb mkpart primary ext4 1MiB 100%
sudo parted /dev/sdb align-check optimal 1
sudo mkfs.ext4 /dev/sdb1
```

Interactive

```
sudo parted /dev/sdb
# (parted) unit GiB
# (parted) print
# (parted) quit
```

Notes & Pitfalls

- Kernel may need `partprobe` or a re-plug to re-read tables.
- NVMe partitions look like `/dev/nvme0n1p1` (note the `p`).
- Cloud disks often already have GPT — don't re-label casually.

Related Commands

- `fdisk` / `gdisk` — alternate partitioners
- `lsblk` / `blkid` — inventory
- `mkfs` / `mount` — after partitioning
- `wipefs` — clear signatures

Additional Resources

- `man parted`
- `info parted`

wipefs

Overview

`wipefs` probes and can **erase filesystem, RAID, or partition-table signatures** from a device without zeroing the entire disk. Useful before re-initializing disks that still claim to be members of LVM/MD/crypto volumes.

From `util-linux`.

Syntax

```
wipefs [options] device
sudo wipefs -a device
```

Common Options

Option	Description
`-a`	Erase all magic strings
`-o offset`	Erase specific offset
`-n`	Dry-run / no probe write (see man for version)
`-p`	From partition table probe
`-t type`	Limit to type
`-f`	Force

Safety

- This makes existing data **hard to mount**; it is destructive even though it is not a full shred.
- Confirm the device with `lsblk -f` and `blkid`.
- Prefer undoing LVM/MD membership with proper tools when the array is still healthy.

Examples with Explanations

Probe signatures

```
sudo wipefs /dev/sdb
sudo wipefs -p /dev/sdb
```

Erase all signatures

```
sudo wipefs -a /dev/sdb
sudo blkid /dev/sdb          # should be empty
```

After wipe — new filesystem

```
sudo parted -s /dev/sdb mklabel gpt
sudo parted -s /dev/sdb mkpart primary 1MiB 100%
sudo mkfs.xfs /dev/sdb1
```

Related Commands

- `blkid` — list signatures
- `lsblk -f` — overview
- `dd` — full overwrite (different goal)
- `cryptsetup` / `pvremove` / `mdadm` — structured teardown

Additional Resources

- `man wipefs`

smartctl

Overview

`smartctl` (from **smartmontools**) queries disk **S.M.A.R.T.** health data: overall assessment, attributes, error logs, and self-tests. Use it for predictive failure monitoring on HDDs/SSDs/NVMe (device-dependent).

```
sudo apt install smartmontools
```

Syntax

```
sudo smartctl [options] device
```

Common Options

Option	Description
`-a` / `-x`	All / extended SMART info
`-H`	Overall health PASS/FAIL
`-i`	Identity
`-A`	Vendor attributes table
`-l error`	Error log
`-l selftest`	Self-test log
`-t short|long|conveyance`	Start self-test
`-s on`	Enable SMART (if disabled)
`-d type`	Device type (raid, nvme, …)

Safety

- Long self-tests stress disks — schedule during maintenance.
- On hardware RAID, you often need `-d megaraid,N` or controller tools; smartctl on `/dev/sdX` may talk to the virtual disk only.

- SMART “PASSED” is not a guarantee — watch reallocated sectors and media errors over time.

Examples with Explanations

Health summary

```
sudo smartctl -H /dev/sda
sudo smartctl -i /dev/sda
```

Full report

```
sudo smartctl -a /dev/sda | less
sudo smartctl -x /dev/nvme0 | less
```

Short self-test

```
sudo smartctl -t short /dev/sda
# wait, then:
sudo smartctl -l selftest /dev/sda
```

Watch failure predictors

```
sudo smartctl -A /dev/sda | egrep -i 'reall|pend|uncorrect|media|temperature'
```

Scan devices

```
sudo smartctl --scan
```

Related Commands

- `lsblk` / `nvme` — device inventory
- `dmesg` — I/O errors
- `hdparm` — identify / timings (older ATA focus)
- `smartd` — background monitoring daemon

Additional Resources

- `man smartctl`
- smartmontools FAQ

pvs

Overview

`pvs` reports **LVM physical volumes** (PVs) — disks or partitions initialized with `pvcreate` and used by volume groups. Part of the LVM reporting trio with `vgs` and `lvs`.

```
sudo apt install lvm2
```

Syntax

```
pvs [options] [PV...]
```

Common Options

Option	Description
`-o fields`	Custom columns
`-v`	Verbose
`--units h`	Human units
`-a`	All devices (incl. empty)
`--reportformat json`	JSON (newer)

Examples with Explanations

List physical volumes

```
sudo pvs
sudo pvs -o+pv_used,pv_free,dev_size
```

Initialize a new PV (related workflow)

```
sudo wipefs -a /dev/sdb1          # if reusing a disk - careful
sudo pvcreate /dev/sdb1
sudo pvs
```

Free space across PVs

```
sudo pvs --units g -o pv_name,vg_name,pv_size,pv_free
```

Notes & Pitfalls

- Whole-disk vs partition PVs: prefer a partition or carefully documented whole-disk use.
- Removing a PV requires migrating extents (`pvmove`) first if the VG has data there.

Related Commands

- `pvcreate` / `pvremove` / `pvresize`
- `vgs` / `lvs` — VG and LV reports
- `lsblk` — block layer view

Additional Resources

- `man pvs`
- `man lvm`

vgs

Overview

`vgs` reports **LVM volume groups** (VGs): how physical volumes are pooled and how much space is free for logical volumes. Use with `pvs` and `lvs` for a full LVM picture.

Syntax

```
vgs [options] [VG...]
```

Common Options

Option	Description
`-o fields`	Custom columns
`-v`	Verbose
`--units h`	Human-readable sizes
`-a`	Include hidden/internal where applicable

Examples with Explanations

List volume groups

```
sudo vgs
sudo vgs -o+vg_extent_size,vg_extent_count,vg_free_count
```

Create VG from PVs (workflow)

```
sudo vgcreate vg_data /dev/sdb1 /dev/sdc1
sudo vgs
```

Free space before creating an LV

```
sudo vgs --units g -o vg_name,vg_size,vg_free
```

Notes & Pitfalls

- Extent size is fixed at VG creation — plan large VGs carefully.
- Cluster/shared VGs need locking infrastructure (not covered here).

Related Commands

- `vgcreate` / `vgextend` / `vgreduce` / `vgremove`
- `pvs` / `lvs`
- `lvmdiskscan`

Additional Resources

- `man vgs`
- `man vgcreate`

lvs

Overview

`lvs` reports **LVM logical volumes** (LVs) — the block devices you usually format and mount (`/dev/VG/LV` or `/dev/mapper/VG-LV`). Check size, attributes (thin, cache, snapshot), and path before growing filesystems.

Syntax

```
lvs [options] [LV...]
```

Common Options

Option	Description
`-o fields`	Custom columns (`lv_path`, `lv_size`, `lv_attr`, …)
`-a`	All LVs including internal
`--units h`	Human sizes
`-v`	Verbose

Examples with Explanations

List LVs

```
sudo lvs
sudo lvs -o lv_name,vg_name,lv_size,lv_path,lv_attr
```

Create and format an LV (workflow)

```
sudo lvcreate -n data -L 100G vg_data
sudo mkfs.ext4 /dev/vg_data/data
sudo mkdir -p /mnt/data
sudo mount /dev/vg_data/data /mnt/data
```

Grow an LV (then filesystem)

```
sudo lvextend -L +50G /dev/vg_data/data
sudo resize2fs /dev/vg_data/data      # ext*
# sudo xfs_growfs /mnt/data          # xfs, grow while mounted
```

Snapshots (classic thick)

```
sudo lvcreate -s -n data_snap -L 10G /dev/vg_data/data
sudo lvs
```

Understanding Output

`lv_attr` is a multi-character state field (open/mounted, snapshot, thin, …). Paths under `/dev/mapper/` are commonly used in fstab via UUID of the **filesystem**, not the LV name alone.

Related Commands

- `lvcreate` / `lvextend` / `lvremove` / `lvrename`
- `pvs` / `vgs`
- `resize2fs` / `xfs_growfs`
- `findmnt`

Additional Resources

- `man lvs`
- `man lvcreate`

cryptsetup

Overview

`cryptsetup` manages **LUKS** (and other) disk encryption: format, open, close, and resize encrypted block devices. Standard tool for full-disk and data-partition encryption on Linux.

```
sudo apt install cryptsetup
```

Syntax

```
sudo cryptsetup [options] command device [args]
```

Common Commands

Command	Description
`luksFormat`	Initialize LUKS header (**destroys data**)
`open` / `luksOpen`	Map decrypted device under `/dev/mapper/`
`close` / `luksClose`	Remove mapping
`status`	Show mapper status
`luksAddKey` / `luksRemoveKey`	Manage passphrases/keys
`luksDump`	Show header metadata
`resize`	Resize LUKS mapping after LV growth

Safety

- `luksFormat` is irreversible without backups of the header and key material.
- Store recovery passphrases offline; header backups (`luksHeaderBackup`) matter.
- Suspend-to-disk with unlocked volumes has threat-model implications.
- Always confirm the target device with `lsblk` first.

Examples with Explanations

Format and open (data disk)

```
sudo cryptsetup luksFormat /dev/sdb1
sudo cryptsetup open /dev/sdb1 crypt_data
sudo mkfs.ext4 /dev/mapper/crypt_data
sudo mount /dev/mapper/crypt_data /mnt/data
```

Status and close

```
sudo cryptsetup status crypt_data
sudo umount /mnt/data
sudo cryptsetup close crypt_data
```

Add a second passphrase

```
sudo cryptsetup luksAddKey /dev/sdb1
sudo cryptsetup luksDump /dev/sdb1 | less
```

Header backup

```
sudo cryptsetup luksHeaderBackup /dev/sdb1 --header-backup-file ~/luks-sdb1.header
chmod 600 ~/luks-sdb1.header
```

fstab / crypttab sketch

```
# /etc/crypttab
# crypt_data UUID=… none luks,discard
# then fstab mounts /dev/mapper/crypt_data
```

Use UUIDs from `blkid` on the **LUKS** device for crypttab.

Notes & Pitfalls

- LUKS2 is the modern default on current distros.

- SSD discard/TRIM: `discard` option trades some security for wear-leveling — know your threat model.
- Nested LVM-on-LUKS or LUKS-on-LVM both exist; order affects unlock at boot.

Related Commands

- `lsblk` / `blkid` — find devices
- `dmsetup` — device-mapper layer
- `lvs` — often underneath or above LUKS
- `mount` / `umount`

Additional Resources

- `man cryptsetup`
- `crypttab(5)`

resize2fs

Overview

`resize2fs` grows or shrinks **ext2/ext3/ext4** filesystems. Growing online (mounted) is common after `lvextend` or partition expansion; shrinking requires an offline (unmounted) filesystem and extreme care.

Syntax

```
sudo resize2fs [options] device [size]
```

If `size` is omitted, the filesystem expands to the size of the underlying partition/LV.

Common Options

Option	Description
`-p`	Progress bars
`-f`	Force (rare)
`-M`	Shrink to minimum size (offline)
`-d flags`	Debug

Safety

- **Shrink** only when unmounted, after full backup and `e2fsck -f`.
- Grow the **block device first** (`lvextend`, `parted resizepart`), then `resize2fs`.
- XFS uses `xfs_growfs` (grow only) — not this tool.
- Btrfs/ZFS have their own grow tools.

Examples with Explanations

Grow to fill LV after lvextend

```
sudo lvextend -L +20G /dev/vg0/root
sudo resize2fs /dev/vg0/root
df -h /
```

Explicit size

```
sudo resize2fs /dev/vg0/data 50G
```

Offline shrink (dangerous — backup first)

```
sudo umount /mnt/data
sudo e2fsck -f /dev/vg0/data
sudo resize2fs /dev/vg0/data 30G
# then shrink LV if desired (lvreduce), carefully matching sizes
```

Notes & Pitfalls

- Kernel/online resize support is standard for grow on ext4; shrink is offline-only.
- Cloud "growpart" + resize2fs is a common first-boot pattern.
- Always match units with LVM tools (`g` vs `G` conventions).

Related Commands

- `lvextend` / `lvs`
- `e2fsck` / `tune2fs`
- `xfs_growfs` — XFS
- `findmnt` / `df`

Additional Resources

- `man resize2fs`

tune2fs

Overview

`tune2fs` adjusts tunable parameters on **ext2/ext3/ext4** filesystems: labels, UUID, mount-count checks, reserved blocks, feature flags, and more. Inspect with `tune2fs -l` before changing anything.

Syntax

```
sudo tune2fs [options] device
```

Common Options

Option	Description
`-l`	List current superblock contents
`-L label`	Set volume label
`-U UUID|random|time`	Set UUID
`-c max-mount-counts`	Max mounts between fsck (`-1` disables)
`-i interval`	Max time between fsck
`-m percent`	Reserved blocks % for root
`-r blocks`	Reserved block count
`-o mount-options`	Default mount options
`-O features`	Set/clear features (careful)

Safety

- Wrong device ruins the wrong filesystem — confirm with `blkid` / `lsblk -f`.
- Feature flag changes (`-O`) can make a volume unmountable on older kernels.
- Reserved block percentage affects usable free space for non-root users.

Examples with Explanations

Inspect

```
sudo tune2fs -l /dev/vg0/root | egrep -i 'label|uuid|block count|reserved|mount count|check'
```

Label and UUID

```
sudo tune2fs -L ROOT /dev/vg0/root
sudo tune2fs -U random /dev/sdb1
sudo blkid /dev/sdb1
# update fstab if UUID changed!
```

Reserved space (big data disks)

```
sudo tune2fs -m 1 /dev/vg0/data     # 1% reserved instead of default ~5%
```

Disable frequent forced fsck by mount count

```
sudo tune2fs -c -1 /dev/vg0/data
sudo tune2fs -i 0 /dev/vg0/data
```

Still run intentional checks after crashes.

Related Commands

- `blkid` / `e2label` — labels/UUIDs
- `resize2fs` — size changes
- `e2fsck` / `fsck` — checks
- `dumpe2fs` — alternate dump

Additional Resources

- `man tune2fs`

ncdu

Overview

`ncdu` (NCurses Disk Usage) is an interactive `du` for finding what consumes space. Faster feedback than reading raw `du` trees; great on full root filesystems.

```
sudo apt install ncdu
```

Syntax

```
ncdu [options] [path]
```

Common Options

Option	Description
`-x`	One filesystem only
`-r`	Read-only (no delete)
`-o file`	Export scan
`-f file`	Load export
`-e`	Extended info (more stat calls)
`--exclude pattern`	Skip paths

Examples with Explanations

Scan root (one FS)

```
sudo ncdu -x /
```

Navigate with arrows; `d` deletes (if not `-r`); `q` quits.

Home directory

```
ncdu -x "$HOME"
```

Export for later / remote analysis

```
sudo ncdu -x -o /tmp/root.ncdu /
ncdu -f /tmp/root.ncdu
```

Notes & Pitfalls

- Needs read permission; use `sudo` for system trees.
- Exclude bind mounts and huge network mounts with `-x` / `--exclude`.
- Deleting from ncdu is still delete — careful on production.

Related Commands

- `du` / `dust` / `duf` — other usage views
- `df` — filesystem free space
- `find` — age-based cleanup

Additional Resources

- `man ncdu`

RWF_DONTCACHE (uncached buffered I/O)

RWF_DONTCACHE (uncached buffered I/O)

Overview

Thesis: a backup, a log shipper or a one-pass scan reads or writes data it will never touch again, yet ordinary buffered I/O leaves every byte in the page cache. There it pushes out the hot data your database and services actually reuse. `O_DIRECT` avoids the cache, but it imposes alignment rules and gives up readahead and write-behind. `RWF_DONTCACHE` is the middle path. It is a per-call flag to `preadv2(2)`/`pwritev2(2)` (and io_uring) that keeps buffered I/O's convenience but drops the pages it brought in as soon as the I/O completes, and starts writeback immediately for writes. It is a **hint**, it is **filesystem-specific**, and on an unsupported kernel or filesystem it fails loudly with `EOPNOTSUPP`. So every caller needs a fallback, and that fallback has a trap of its own.

Typical operator jobs:

- Run backups, `rsync`-style copies and checksum scans without flushing the working set of the services on the same host.
- Write large exports or logs without building up gigabytes of dirty pages and then a writeback storm.
- Benchmark "streaming" I/O with `fio --uncached=1`.
- Decide whether a tool's `--no-cache` option will actually do anything on *this* kernel and filesystem.

Kernel support, checked against mainline source:

Kernel	What landed
6.14	`RWF_DONTCACHE` (`0x80`) in the UAPI and the VFS/page-cache plumbing. **No filesystem opts in yet**, so every call still returns `EOPNOTSUPP`
6.15	**XFS**, the first filesystem (`xfs: flag as supporting FOP_DONTCACHE`), plus iomap buffered-write support
6.17	**ext4** (`ext4: support uncached buffered I/O`)
6.18	**NFS client** (`NFS: Enable use of the RWF_DONTCACHE flag on the NFS client`)
7.2	Writeback improvements: targeted flusher kicks and per-bdi dirty tracking for dontcache pages
7.3 (in release candidates: **v7.3-rc6** at the time of writing)	**Raw block devices** (`block: enable RWF_DONTCACHE for block devices`)
mainline today	Still **not** supported on btrfs, f2fs, tmpfs/shmem, FUSE or overlayfs. DAX files are refused

Mental model

```
                    plain buffered          RWF_DONTCACHE                 O_DIRECT
read     page cache    user buf    page cache    user buf        device    user buf
          (pages stay; LRU decides)  (pages it added are dropped     (no cache, aligned I/O,
                                      when the copy completes)        no readahead)
write    dirty pages; writeback    dirty pages + writeback started  device directly
          later (dirty_expire, ...)  now; pages dropped when clean
```

Fact	Consequence
It is **buffered** I/O	No alignment rules, readahead still works, and concurrent readers of the same range still share pages
Only pages **this call instantiated** are dropped	Ranges that were already cached stay cached. It never evicts someone else's hot data
Writes **start** writeback but do not wait for it	It is not `fsync`. Durability is unchanged, but dirty memory does not pile up
It is a **hint**, "best effort"	No guarantee about cache state afterwards. Do not build correctness on it
Support is per filesystem (`FOP_DONTCACHE`)	The same binary works on XFS, gets `EOPNOTSUPP` on btrfs, and works on ext4 only from 6.17
An old kernel and an unsupported filesystem fail the same way	`EOPNOTSUPP`. The man page lists both causes. You cannot tell them apart from the error

Syntax

```c
#include <sys/uio.h>
#ifndef RWF_DONTCACHE              /* glibc 2.41 / linux-libc-dev 6.12 headers lack it */
# define RWF_DONTCACHE 0x00000080  /* include/uapi/linux/fs.h, Linux 6.14+ */
#endif

ssize_t preadv2 (int fd, const struct iovec *iov, int iovcnt, off_t offset, int flags);
ssize_t pwritev2(int fd, const struct iovec *iov, int iovcnt, off_t offset, int flags);
/* flags = RWF_DONTCACHE; on failure: -1, errno == EOPNOTSUPP */
```

Interface	How to ask for uncached I/O
`preadv2`/`pwritev2`	`flags = RWF_DONTCACHE` (offset `-1` means "use and update the file position")
io_uring `READ`/`WRITE` (and `_FIXED`)	`sqe->rw_flags = RWF_DONTCACHE`
`fio`	`--ioengine=pvsync2` or `io_uring` with `--uncached=1`

Interface	How to ask for uncached I/O
Python	`os.preadv(fd, bufs, off, 0x80)` / `os.pwritev(fd, bufs, off, 0x80)`. The `os` module has no `RWF_DONTCACHE` constant (checked on 3.13 and 3.15.0rc3)
`dd`	**No equivalent.** `iflag=nocache` / `oflag=nocache` use `posix_fadvise(DONTNEED)`, which is the fallback described below

Safety

- The flag itself is harmless. Data is written exactly as with buffered I/O, and only cache retention changes.
- **The fallback is not harmless.** `posix_fadvise(POSIX_FADV_DONTNEED)` drops **every** clean cached page in the range, including pages another process is actively using (see The trap).
- Do not use it for files the same process will reread soon. You pay for the device reads again.
- Measuring page-cache effects needs a quiet host. On a busy server, `Cached:` in `/proc/meminfo` moves by itself. Use a cgroup's `memory.stat` (`file`, `file_dirty`) to isolate one job.

Examples with Explanations

All examples run as an ordinary user. Build with `gcc -O2 -Wall -o NAME NAME.c`.

The test box for this page ran **Linux 6.12** with `/workspace` on **overlayfs**. That is two reasons for `EOPNOTSUPP`: the kernel predates the flag, and overlayfs does not support it even on new kernels. So the outputs below show the **probe and fallback paths running for real**, and the success path is described from the man page and kernel source, not measured. That is also the situation most fleets are in today, which is why the fallback matters.

```
uname -r; findmnt -no FSTYPE -T .
```

```
6.12.94+
overlay
```

1. Probe: does this kernel and filesystem accept the flag?

```
/* probe.c - ask the kernel whether RWF_DONTCACHE is accepted on this fd */
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <linux/fs.h>
#include <stdio.h>
#include <string.h>
#include <sys/uio.h>
```

```c
#include <unistd.h>

#ifndef RWF_DONTCACHE
# define RWF_DONTCACHE 0x00000080
#endif

static int try_flag(const char *path, int open_flags)
{
    int fd = open(path, open_flags, 0600);
    if (fd < 0) { perror(path); return 1; }
    char buf[4096];
    memset(buf, 'A', sizeof buf);
    struct iovec iov = { .iov_base = buf, .iov_len = sizeof buf };
    ssize_t n = pwritev2(fd, &iov, 1, 0, RWF_DONTCACHE);
    if (n < 0)
        printf("%s: pwritev2(RWF_DONTCACHE) -> %s (%d)\n",
               path, strerror(errno), errno);
    else
        printf("%s: pwritev2(RWF_DONTCACHE) wrote %zd bytes\n", path, n);
    n = preadv2(fd, &iov, 1, 0, RWF_DONTCACHE);
    if (n < 0)
        printf("%s: preadv2(RWF_DONTCACHE) -> %s (%d)\n",
               path, strerror(errno), errno);
    else
        printf("%s: preadv2(RWF_DONTCACHE) read %zd bytes\n", path, n);
    close(fd);
    return 0;
}

int main(void)
{
    printf("RWF_DONTCACHE = 0x%x\n", RWF_DONTCACHE);
    try_flag("/tmp/dc-overlay.bin", O_RDWR|O_CREAT|O_TRUNC);
    try_flag("/dev/shm/dc-tmpfs.bin", O_RDWR|O_CREAT|O_TRUNC);
    return 0;
}
```

```
./probe
```

```
RWF_DONTCACHE = 0x80
/tmp/dc-overlay.bin: pwritev2(RWF_DONTCACHE) -> Operation not supported (95)
/tmp/dc-overlay.bin: preadv2(RWF_DONTCACHE) -> Operation not supported (95)
/dev/shm/dc-tmpfs.bin: pwritev2(RWF_DONTCACHE) -> Operation not supported (95)
/dev/shm/dc-tmpfs.bin: preadv2(RWF_DONTCACHE) -> Operation not supported (95)
```

Explanation. The `#ifndef` is needed because current distribution headers (here glibc 2.41 with 6.12 UAPI headers) do not define the constant yet. Its value is fixed by the kernel ABI. The kernel

rejects the call before doing any I/O, so a failed probe has no side effects: the `pwritev2` wrote nothing. On a 6.17+ kernel, the same binary pointed at an ext4 or XFS file would print "wrote 4096 bytes", and tmpfs would still fail. **Probe per file, not per host.** Mount points on one machine differ.

2. Streaming read with a correct fallback

```
/* stream_read.c - read a file once, front to back, without leaving it in
 * the page cache. Uses RWF_DONTCACHE when the kernel and filesystem accept
 * it; otherwise falls back to POSIX_FADV_DONTNEED behind the read cursor.
 *
 *   ./stream_read FILE [plain|dontcache]
 */
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/uio.h>
#include <unistd.h>

#ifndef RWF_DONTCACHE
# define RWF_DONTCACHE 0x00000080   /* include/uapi/linux/fs.h, Linux 6.14+ */
#endif

#define CHUNK (1 << 20)             /* 1 MiB per call */

int main(int argc, char **argv)
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s FILE [plain|dontcache]\n", argv[0]);
        return 2;
    }
    int want_dontcache = !(argc > 2 && strcmp(argv[2], "plain") == 0);

    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }
    char *buf = malloc(CHUNK);
    struct iovec iov = { .iov_base = buf, .iov_len = CHUNK };

    int flags = want_dontcache ? RWF_DONTCACHE : 0;
    const char *how = want_dontcache ? "RWF_DONTCACHE" : "plain buffered";
    off_t off = 0;
    unsigned long long sum = 0;
```

```
    for (;;) {
        ssize_t n = preadv2(fd, &iov, 1, off, flags);
        if (n < 0 && errno == EOPNOTSUPP && (flags & RWF_DONTCACHE)) {
            /* Old kernel, or a filesystem without FOP_DONTCACHE. */
            flags &= ~RWF_DONTCACHE;
            how = "fallback: fadvise(DONTNEED)";
            continue;                       /* retry this chunk */
        }
        if (n < 0) { perror("preadv2"); return 1; }
        if (n == 0) break;
        for (ssize_t i = 0; i < n; i += 4096)
            sum += (unsigned char)buf[i];   /* touch the data */
        if (want_dontcache && !(flags & RWF_DONTCACHE))
            posix_fadvise(fd, off, n, POSIX_FADV_DONTNEED);
        off += n;
    }
    printf("read %lld MiB via %s (checksum %llu)\n",
           (long long)(off >> 20), how, sum);
    close(fd);
    free(buf);
    return 0;
}
```

A two-line helper shows the system-wide page cache:

```
#!/bin/sh
# pagecache.sh - print the system-wide Cached and Dirty lines from /proc/meminfo, in MiB
awk '/^(Cached|Dirty):/ {printf "%s %d MiB  ", $1, $2/1024} END {print ""}' /proc/meminfo
```

```
head -c 256M /dev/urandom > big.bin             # any 256 MiB file; your checksum will diffe
dd if=big.bin iflag=nocache count=0 status=none   # start with big.bin uncached
./pagecache.sh
./stream_read big.bin plain;     ./pagecache.sh
dd if=big.bin iflag=nocache count=0 status=none
./pagecache.sh
./stream_read big.bin dontcache; ./pagecache.sh
```

```
Cached: 4587 MiB  Dirty: 0 MiB
read 256 MiB via plain buffered (checksum 8349472)
Cached: 4843 MiB  Dirty: 0 MiB
Cached: 4587 MiB  Dirty: 0 MiB
read 256 MiB via fallback: fadvise(DONTNEED) (checksum 8349472)
Cached: 4588 MiB  Dirty: 0 MiB
```

Explanation. A plain read of a 256 MiB file grows the page cache by exactly 256 MiB, and those pages now compete with everything else on the host. The `dontcache` run got `EOPNOTSUPP` on its

first chunk, switched to the fallback, and left the cache where it started. The fallback drops each chunk **behind** the cursor, after it has been consumed, so readahead still works ahead of the cursor. On a kernel and filesystem that accept the flag, the program never enters the fallback and the kernel does the dropping itself. Only the "via" line changes.

`dd if=FILE iflag=nocache count=0` is a handy way to drop **one file** from the cache without root, unlike `echo 1 > /proc/sys/vm/drop_caches`.

3. Streaming write: bound dirty memory

```c
/* stream_write.c - write N MiB of output without leaving it in the page cache.
 * RWF_DONTCACHE when available; otherwise start writeback per chunk, wait for
 * the previous chunk, and drop it with POSIX_FADV_DONTNEED.
 *
 *   ./stream_write FILE MIB [plain|dontcache]
 */
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/uio.h>
#include <unistd.h>

#ifndef RWF_DONTCACHE
# define RWF_DONTCACHE 0x00000080
#endif

#define CHUNK (1 << 20)

int main(int argc, char **argv)
{
    if (argc < 3) {
        fprintf(stderr, "usage: %s FILE MIB [plain|dontcache]\n", argv[0]);
        return 2;
    }
    long mib = atol(argv[2]);
    int want_dontcache = !(argc > 3 && strcmp(argv[3], "plain") == 0);
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }

    char *buf = malloc(CHUNK);
    memset(buf, 0x5a, CHUNK);
    struct iovec iov = { .iov_base = buf, .iov_len = CHUNK };
    int flags = want_dontcache ? RWF_DONTCACHE : 0;
```

```
    const char *how = want_dontcache ? "RWF_DONTCACHE" : "plain buffered";

    for (long i = 0; i < mib; i++) {
        off_t off = (off_t)i * CHUNK;
        ssize_t n = pwritev2(fd, &iov, 1, off, flags);
        if (n < 0 && errno == EOPNOTSUPP && (flags & RWF_DONTCACHE)) {
            flags &= ~RWF_DONTCACHE;
            how = "fallback: sync_file_range + fadvise(DONTNEED)";
            i--;                                /* retry this chunk */
            continue;
        }
        if (n != CHUNK) { perror("pwritev2"); return 1; }
        if (want_dontcache && !(flags & RWF_DONTCACHE)) {
            /* kick off writeback for this chunk ... */
            sync_file_range(fd, off, CHUNK, SYNC_FILE_RANGE_WRITE);
            /* ... and drop the previous one once it is on disk */
            if (i > 0) {
                sync_file_range(fd, off - CHUNK, CHUNK,
                                SYNC_FILE_RANGE_WAIT_BEFORE |
                                SYNC_FILE_RANGE_WRITE |
                                SYNC_FILE_RANGE_WAIT_AFTER);
                posix_fadvise(fd, off - CHUNK, CHUNK, POSIX_FADV_DONTNEED);
            }
        }
    }
    if (want_dontcache && !(flags & RWF_DONTCACHE)) {
        fdatasync(fd);
        posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED);
    }
    printf("wrote %ld MiB via %s\n", mib, how);
    close(fd);
    free(buf);
    return 0;
}
```

```
./pagecache.sh
./stream_write out.bin 256 plain;     ./pagecache.sh
sync; dd if=out.bin iflag=nocache count=0 status=none; rm -f out.bin
./pagecache.sh
./stream_write out.bin 256 dontcache; ./pagecache.sh; rm -f out.bin
```

```
Cached: 4588 MiB  Dirty: 0 MiB
wrote 256 MiB via plain buffered
Cached: 4844 MiB  Dirty: 256 MiB
Cached: 4588 MiB  Dirty: 0 MiB
wrote 256 MiB via fallback: sync_file_range + fadvise(DONTNEED)
```

```
Cached: 4588 MiB  Dirty: 0 MiB
```

Explanation. The plain write returns with **256 MiB dirty**. The kernel will flush it later, in bulk, at a time of its choosing, and a few such jobs at once is how a host ends up stalled in `balance_dirty_pages`. The fallback keeps at most about two chunks in flight: start writeback on chunk i, wait for chunk $i-1$, drop it. That bounds both dirty and cached memory. `RWF_DONTCACHE` does the "start writeback" part inside the kernel on every call and drops the pages when writeback completes, with no extra syscalls.

The cost is visible with `time`:

```
/usr/bin/time -f "%e s elapsed" ./stream_write out.bin 256 plain
sync; dd if=out.bin iflag=nocache count=0 status=none; rm -f out.bin
/usr/bin/time -f "%e s elapsed" ./stream_write out.bin 256 dontcache; rm -f out.bin
```

```
wrote 256 MiB via plain buffered
0.13 s elapsed
wrote 256 MiB via fallback: sync_file_range + fadvise(DONTNEED)
0.44 s elapsed
```

This is not "uncached writes are 3× slower". The plain run returned **before** writing anything to storage and left the device work to the flusher. The uncached run paid for that work inside its own runtime. Judge by end-to-end throughput and by what happens to the *other* workloads, not by how fast the writer returns.

4. `fio`: fails rather than falls back

```
fio --version
fio --name=uc --filename=big.bin --rw=read --bs=1M --size=64M \
    --ioengine=pvsync2 --uncached=1 2>&1 | grep -iE 'err' | head -3
```

```
fio-3.39
fio: io_u error on file big.bin: Operation not supported: read offset=0, buflen=1048576
fio: pid=481977, err=95/file:io_u.c:1976, func=io_u error, error=Operation not supported
uc: (groupid=0, jobs=1): err=95 (file:io_u.c:1976, func=io_u error, error=Operation not supp
```

Explanation. `fio` passes the flag straight through, so on an unsupported kernel or filesystem the job errors out. That is the right behaviour for a benchmark, and a quick capability check for a target directory: if this command errors, so will your application's uncached I/O there.

The trap

The portable fallback evicts other people's cache.

`RWF_DONTCACHE` only drops pages that *its own* I/O instantiated. `posix_fadvise(DONTNEED)` drops **every clean page in the range**, whoever loaded it. If a service already had the file hot, a "polite" backup using the fallback makes it cold:

```
cat big.bin > /dev/null; ./pagecache.sh        # another process warms the file
./stream_read big.bin dontcache; ./pagecache.sh   # our "polite" one-pass read
```

```
Cached: 4846 MiB  Dirty: 0 MiB
read 256 MiB via fallback: fadvise(DONTNEED) (checksum 8349472)
Cached: 4590 MiB  Dirty: 0 MiB
```

256 MiB that someone else had cached is gone. With real `RWF_DONTCACHE` the pre-existing pages would have stayed: the man page says ranges "already in cache before this read or write … will not be pruned". The same applies to `dd iflag=nocache` and to every "nocache" option built on fadvise.

`dd` shows the same effect:

```
dd if=big.bin of=/dev/null bs=1M iflag=nocache status=none; ./pagecache.sh   # leaves nothin
cat big.bin >/dev/null; ./pagecache.sh                                       # warm it
dd if=big.bin iflag=nocache count=0 status=none; ./pagecache.sh             # and dd drops i
```

```
Cached: 4590 MiB  Dirty: 0 MiB
Cached: 4846 MiB  Dirty: 0 MiB
Cached: 4590 MiB  Dirty: 0 MiB
```

To use the fallback safely, check residency first (with `mincore(2)` on a mapping of the file, or `fincore` where it works) and skip `DONTNEED` for ranges that were resident before you started. Or accept the collateral and run such jobs where nothing else needs that file hot.

Notes & Pitfalls

Pitfall	Symptom / fix
Header lacks `RWF_DONTCACHE`	Compile error. Use the `#ifndef … 0x80` guard, since the value is ABI
Treating `EOPNOTSUPP` as fatal	Your tool breaks on every pre-6.15 kernel and on btrfs/tmpfs. Retry the same chunk without the flag and use the fallback
Expecting "since 6.14" to mean "works on 6.14"	6.14 has the flag but no filesystem accepts it. XFS from 6.15, ext4 from 6.17, NFS client from 6.18, block devices from 7.3
Probing once per host	Support is per filesystem. `/`, `/var/lib/…` and `/tmp` can differ. Probe the fd you will use

Pitfall	Symptom / fix
`fincore` says `0B` for a file you just read	Seen on overlayfs here: `fincore big.bin` printed `0B 0 256M` right after `cat big.bin` (a tmpfs copy of the same file reported `256M`). The overlay file's mapping is not where the cached pages live. Measure with `/proc/meminfo` or cgroup `memory.stat` instead
Confusing it with `O_DIRECT`	No alignment rules, and it still goes through the cache (briefly). Pages already cached are used
Expecting durability	Writes only **start** writeback. Use `fdatasync`/`fsync` for durability, as with any buffered write
`POSIX_FADV_NOREUSE` as an alternative	It was suggested in review of the original patches. It is advice about how the LRU should age the pages, not an instruction to drop them when the I/O completes, so it is not a drop-in replacement
Mixing with `mmap` of the same file	Pages that are mapped or otherwise in use may stay cached. The hint is best-effort, and the man page promises nothing about the cache state afterwards

The boring rule

> One-pass bulk I/O (backups, scans, exports) uses `RWF_DONTCACHE` per call, and on `EOPNOTSUPP` retries that call without the flag and switches to a bounded fallback (`sync_file_range` + `fadvise` for writes, `fadvise` behind the cursor for reads), but only on files nobody else keeps hot. Probe per file, never trust the kernel version alone, and judge the result by the *other* workloads' cache hit rate, not by the job's own speed.

Related Commands

Command	Role
`dd iflag=nocache` / `oflag=nocache`	fadvise-based cache dropping (the fallback, with its collateral)
`fio --uncached=1`	Benchmark real `RWF_DONTCACHE` with `pvsync2` or `io_uring`
`fincore`	Page-cache residency per file (unreliable on overlayfs)
`sync`, `fdatasync`	Durability, which `RWF_DONTCACHE` does not provide
`vmstat`, `/proc/meminfo`	`Cached`, `Dirty`, `Writeback` trends while a job runs
`findmnt -T PATH`	Which filesystem a path is actually on, before you trust a capability

Try this

1. Run `probe` against a file on every mount point of a host (`findmnt -rno TARGET -t ext4,xfs,btrfs,nfs4,tmpfs`) and build a table of which accept the flag on that kernel.
2. On a 6.17+ kernel with ext4 or XFS, rerun example 2 and check that the program reports `via RWF_DONTCACHE` and that `Cached:` stays flat. Then warm the file with `cat` first and confirm the cache is **not** dropped, unlike the trap.
3. Write 4 GiB with `stream_write … plain` while running `vmstat 1`, then again with the uncached path. Compare the `bo` column and how long `Dirty:` takes to return to zero.
4. Extend `stream_read.c` to call `mincore(2)` on an `mmap` of each chunk before reading, and skip `POSIX_FADV_DONTNEED` for chunks that were already resident. Verify with the trap's commands.
5. On a 7.3+ kernel, point `fio --uncached=1` at a scratch block device (never one holding data) and compare `Cached:` with and without `--uncached`.

Sources

- readv(2): `RWF_DONTCACHE` semantics (pre-existing ranges not pruned, writeback kicked off, best effort, `EOPNOTSUPP`)
- posix_fadvise(2), sync_file_range(2), mincore(2)
- Kernel commits (mainline): `fs: add RWF_DONTCACHE iocb and FOP_DONTCACHE file_operations flag` (6.14), `xfs: flag as supporting FOP_DONTCACHE` (6.15), `ext4: support uncached buffered I/O` (6.17), `NFS: Enable use of the RWF_DONTCACHE flag on the NFS client` (6.18), `mm: kick writeback flusher for IOCB_DONTCACHE with targeted dirty tracking` (7.2), `block: enable RWF_DONTCACHE for block devices` (7.3)
- LWN: The return of RWF_UNCACHED and Uncached buffered I/O
- fio documentation, `uncached`
- GNU coreutils, `dd` invocation (`nocache`)

io_uring (batched asynchronous I/O)

io_uring (batched asynchronous I/O)

Overview

Thesis: with `read(2)`, every read is a separate trip into the kernel: you ask, you wait, you ask again. **io_uring** replaces that conversation with two ring buffers shared between your process and the kernel. You write requests (**SQEs**) into the submission queue. The kernel writes results (**CQEs**) into the completion queue. One `io_uring_enter(2)` call can hand over a whole batch, and reaping completions that are already posted costs no system call at all. That makes it the fastest general-purpose I/O interface on Linux. It is also an interface that operators often switch **off**, so every program that uses it needs a fallback.

Typical operator jobs:

- Recognise io_uring in `strace` output and in tools that use it (`fio --ioengine=io_uring`, databases, QEMU, newer runtimes and imaging tools).
- Confirm whether io_uring is allowed on a host or in a container before you blame an application.
- Decide on the policy knob `kernel.io_uring_disabled` for a multi-tenant host.
- Read a small liburing program and know where the buffers, tags and errors live.

Mental model

```
your process                                  kernel

io_uring_get_sqe()     [ SQ ring: SQE SQE SQE ]      executes requests
io_uring_submit()     one io_uring_enter(2)         (in any order)
io_uring_wait_cqe()    [ CQ ring: CQE CQE CQE ]      posts results
                        (shared memory: reading a posted CQE is a load, not a syscall)
```

Fact	Consequence
Rings are memory mapped and shared	Submitting N requests can cost one syscall, and reaping can cost zero
Completions arrive **in any order**	Every SQE carries `user_data` (a 64-bit tag). You match results by tag, never by position
A CQE's `res` is the syscall's return value	Negative `res` is `-errno`. Positive `res` can be a **short** read, exactly like `read(2)`
The kernel uses your buffer until the CQE arrives	A stack buffer that goes out of scope before completion is a memory-corruption bug

Fact	Consequence
It can be disabled by sysctl, seccomp or an LSM	`io_uring_setup(2)` fails with `EPERM` (policy) or `ENOSYS` (filtered or very old kernel)

Syntax

The raw interface is three system calls (`io_uring_setup`, `io_uring_enter`, `io_uring_register`). Nearly everyone uses **liburing**, the library maintained alongside the kernel code:

```
struct io_uring ring;
io_uring_queue_init(entries, &ring, 0);            /* io_uring_setup + mmap the rings */
struct io_uring_sqe *sqe = io_uring_get_sqe(&ring); /* claim a slot */
io_uring_prep_read(sqe, fd, buf, len, offset);      /* fill it like a pread() call */
io_uring_sqe_set_data64(sqe, tag);                  /* your tag, echoed in the CQE */
io_uring_submit(&ring);                             /* io_uring_enter: hand over the batch *
io_uring_wait_cqe(&ring, &cqe);                     /* next completion (no syscall if posted
io_uring_cqe_seen(&ring, cqe);                      /* give the CQ slot back */
io_uring_queue_exit(&ring);
```

On Debian and Ubuntu the headers are in `liburing-dev`, on Fedora in `liburing-devel`. Link with `-luring`.

The host policy knob (Linux 6.6 and later):

`kernel.io_uring_disabled`	Effect
`0` (default)	Anyone may create rings
`1`	Only processes with `CAP_SYS_ADMIN`, or in the group named by `kernel.io_uring_group`, may create rings
`2`	Nobody may create rings. Existing rings keep working

Safety

Reading files with io_uring is no more dangerous than reading them with `read(2)`. The risk is on the **kernel** side: io_uring is a large attack surface and has a long CVE history. That is why Android, ChromeOS, Google's production fleet and Docker's default seccomp profile restrict it, and why the sysctl exists. Changing `kernel.io_uring_disabled` affects the whole host. Try it only on a machine you own, and set it back.

Examples with Explanations

Tested on Linux 6.12 (Debian 13) with GCC 14.2 and liburing 2.9. Every call used here has been in the kernel since 5.6 and in liburing since 2.2, so the listings work unchanged on current kernels.

1. Three reads, one system call

```c
/* batch3.c - read three files with one io_uring_submit() */
#include <fcntl.h>
#include <liburing.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    struct io_uring ring;
    char buf[3][64];
    int fds[3], n = argc - 1;

    if (n < 1 || n > 3) { fprintf(stderr, "usage: %s f1 [f2 [f3]]\n", argv[0]); return 2; }
    int rc = io_uring_queue_init(8, &ring, 0);
    if (rc < 0) { fprintf(stderr, "io_uring_queue_init: %s\n", strerror(-rc)); return 1; }

    for (int i = 0; i < n; i++) {
        fds[i] = open(argv[i + 1], O_RDONLY);
        if (fds[i] < 0) { perror(argv[i + 1]); return 1; }
        struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
        io_uring_prep_read(sqe, fds[i], buf[i], sizeof buf[i] - 1, 0);
        io_uring_sqe_set_data64(sqe, i);          /* tag: which file */
    }
    printf("submitted %d SQEs in one call\n", io_uring_submit_and_wait(&ring, n));

    for (int done = 0; done < n; done++) {
        struct io_uring_cqe *cqe;
        io_uring_wait_cqe(&ring, &cqe);           /* already complete: no syscall */
        int i = (int)io_uring_cqe_get_data64(cqe);
        if (cqe->res < 0)
            printf("%s: error %s\n", argv[i + 1], strerror(-cqe->res));
        else {
            buf[i][cqe->res] = '\0';
            buf[i][strcspn(buf[i], "\n")] = '\0';
            printf("%-20s %3d bytes  \"%s\"\n", argv[i + 1], cqe->res, buf[i]);
        }
        io_uring_cqe_seen(&ring, cqe);
    }
```

```
    io_uring_queue_exit(&ring);
    return 0;
}
```

```
gcc -O2 -Wall -o batch3 batch3.c -luring
./batch3 /etc/hostname /proc/version /etc/os-release
```

```
submitted 3 SQEs in one call
/proc/version         63 bytes  "Linux version 6.12.94+ (root@758c51f35d4c) (gcc (Debian 12.
/etc/hostname         22 bytes  "grok-bot-vm-399265740"
/etc/os-release       63 bytes  "PRETTY_NAME="Debian GNU/Linux 13 (trixie)""
```

Two things to notice:

- The files were submitted in the order hostname, version, os-release, but **`/proc/version` completed first**. That is why each SQE carries a tag (`io_uring_sqe_set_data64`) and why the loop looks the file up by tag.
- `io_uring_submit_and_wait(&ring, n)` submits and waits for `n` completions in the same `io_uring_enter` call. After that, `io_uring_wait_cqe` only reads shared memory.

Count the system calls:

```
strace -c -e trace=io_uring_enter,io_uring_setup,read,openat ./batch3 /etc/hostname /proc/ve
```

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
  0.00    0.000000           0         2           read
  0.00    0.000000           0         6           openat
  0.00    0.000000           0         1           io_uring_setup
  0.00    0.000000           0         1           io_uring_enter
------ ----------- ----------- --------- --------- ----------------
100.00    0.000000           0        10           total
```

One `io_uring_enter` did all three reads. The two `read` calls are the dynamic loader reading shared libraries, not our files.

2. Is io_uring allowed here?

```
cat /proc/sys/kernel/io_uring_disabled      # 0, 1 or 2
cat /proc/sys/kernel/io_uring_group         # -1 means no group is exempt
sudo sysctl -q kernel.io_uring_disabled=2   # lab only: turn it off for everyone
./batch3 /etc/hostname
sudo sysctl -q kernel.io_uring_disabled=1   # privileged only
./batch3 /etc/hostname
sudo ./batch3 /etc/hostname
```

```
sudo sysctl -q kernel.io_uring_disabled=0   # restore the default
```

```
0
-1
io_uring_queue_init: Operation not permitted
io_uring_queue_init: Operation not permitted
submitted 1 SQEs in one call
/etc/hostname         22 bytes  "grok-bot-vm-399265740"
```

With `1`, the unprivileged run is refused and the root run works. In a container the same `EPERM` (or `ENOSYS`) usually comes from the runtime's seccomp profile rather than the sysctl. Check `grep Seccomp /proc/self/status` inside the container and the runtime's profile before you change any host setting.

3. A fallback that keeps the program working

```
/* readall.c - io_uring when allowed, plain pread() when not */
#include <errno.h>
#include <fcntl.h>
#include <liburing.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

static long read_with_pread(int fd, char *buf, size_t len)
{
    return pread(fd, buf, len, 0);
}

int main(int argc, char **argv)
{
    char buf[256];
    struct io_uring ring;
    int fd = open(argc > 1 ? argv[1] : "/etc/hostname", O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }

    int rc = io_uring_queue_init(4, &ring, 0);
    if (rc == -EPERM || rc == -ENOSYS) {          /* sysctl, seccomp, or old kernel */
        fprintf(stderr, "io_uring unavailable (%s); using pread\n", strerror(-rc));
        long n = read_with_pread(fd, buf, sizeof buf);
        printf("pread: %ld bytes\n", n);
        return n < 0;
    }
    if (rc < 0) { fprintf(stderr, "io_uring_queue_init: %s\n", strerror(-rc)); return 1; }
```

```
    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
    io_uring_prep_read(sqe, fd, buf, sizeof buf, 0);
    io_uring_submit(&ring);
    struct io_uring_cqe *cqe;
    io_uring_wait_cqe(&ring, &cqe);
    printf("io_uring: %d bytes\n", cqe->res);
    io_uring_cqe_seen(&ring, cqe);
    io_uring_queue_exit(&ring);
    return 0;
}
```

```
gcc -O2 -Wall -o readall readall.c -luring
./readall
sudo sysctl -q kernel.io_uring_disabled=2; ./readall; sudo sysctl -q kernel.io_uring_disable
```

```
io_uring: 22 bytes
io_uring unavailable (Operation not permitted); using pread
pread: 22 bytes
```

The fallback is decided **once, at ring setup**, and treats `EPERM` and `ENOSYS` as "not here" rather than as fatal errors. Any other error is a real failure and is reported as one.

4. Benchmarks with fio

```
fio --name=ur --filename=testfile --size=256M --rw=randread --bs=4k \
    --ioengine=io_uring --iodepth=32 --direct=1 --runtime=10 --time_based
```

Run the same job with `--ioengine=psync` to compare against one syscall per read. If io_uring is disabled, fio fails at setup instead of quietly falling back, which is what you want from a benchmark.

The trap

The buffer must outlive the request. `io_uring_prep_read` only records a pointer. The kernel writes into that memory whenever the read completes, which may be after the function that prepared it has returned:

```
void start_read(struct io_uring *ring, int fd)
{
    char buf[4096];                         /* BUG: lives on this stack frame */
    struct io_uring_sqe *sqe = io_uring_get_sqe(ring);
    io_uring_prep_read(sqe, fd, buf, sizeof buf, 0);
    io_uring_submit(ring);
}                                           /* returns; the kernel may write here later */
```

On a fast local file this often "works", because the read completes before the frame is reused. On a pipe, socket or slow disk the kernel writes 4 KiB into whatever stack frame occupies that address later. ASan does not see kernel writes, so it will not catch this. The rule is the same as for POSIX AIO: allocate request buffers with the same lifetime as the request (heap, a per-request struct, or registered buffers) and free them only after you have seen the CQE.

Notes & Pitfalls

Pitfall	Symptom / fix
Matching results by order	Wrong data attributed to the wrong file. Tag every SQE with `user_data`
Ignoring short reads	`res` smaller than requested. Loop, like `read(2)`
Forgetting `io_uring_cqe_seen`	The CQ fills up and new completions are dropped or overflow. Always mark CQEs seen
Treating `EPERM` as a bug	It is policy (sysctl, seccomp, LSM). Fall back to synchronous I/O
`io_uring_get_sqe` returns `NULL`	The SQ is full. Submit what you have, then retry
Expecting `strace` to show the reads	It shows only `io_uring_enter`. Use `bpftrace` on `io_uring:*` tracepoints to see individual operations
Using it for three small files	It works, but it saves almost nothing. The gains appear with high queue depth, many small requests, or network servers

The boring rule

> Use io_uring through liburing, tag every request, keep its buffer alive until its CQE is seen, and decide at ring setup whether it is available, falling back to plain syscalls on `EPERM`/`ENOSYS`. On shared hosts, set `kernel.io_uring_disabled` deliberately (`1` plus `io_uring_group` for trusted services) rather than leaving it to defaults.

Related Commands

Command	Role
`strace -c`	Count `io_uring_enter` calls versus `read`/`write`
`sysctl kernel.io_uring_disabled` / `kernel.io_uring_group`	Host policy
`fio --ioengine=io_uring`	Benchmark io_uring against `psync` and `libaio`
`bpftrace -l 'tracepoint:io_uring:*'`	Per-request tracing
`grep Seccomp /proc/self/status`	Is a seccomp filter active in this process or container?

Try this

1. Change `batch3.c` to read the same three files 1,000 times with a ring of 64 entries, submitting in batches of 32, and compare `strace -c` totals with a `pread` loop.
2. Run `batch3` inside `docker run --rm` with the default profile, then with `--security-opt seccomp=unconfined`, and record the error each time.
3. Set `kernel.io_uring_disabled=1` and `kernel.io_uring_group` to a group your user belongs to. Confirm unprivileged rings work again. Restore both values.
4. Rewrite the trap so the buffer lives in a heap-allocated request struct that is freed after `io_uring_cqe_seen`.

Sources

- io_uring(7), io_uring_setup(2), io_uring_enter(2)
- liburing man pages and source: github.com/axboe/liburing (`io_uring_queue_init(3)`, `io_uring_prep_read(3)`, `io_uring_submit_and_wait(3)`)
- Jens Axboe, Efficient IO with io_uring
- Kernel documentation, `/proc/sys/kernel/` — io_uring_disabled, io_uring_group
- fio documentation, `ioengine=io_uring`

I/O Schedulers: none, mq-deadline, kyber, bfq

I/O Schedulers: none, mq-deadline, kyber, bfq

Thesis

Linux multi-queue block devices pick an **I/O scheduler** (elevator) that decides how requests are ordered and merged. NVMe often wants **`none`** (hardware queues are enough). Spindles and busy shared disks still care about **`mq-deadline`**, **`kyber`**, or **`bfq`**. This chapter shows how to *read* the active scheduler from sysfs on a live system — without changing it (that can need root and can surprise co-tenants).

Mental model

```
app write/read → page cache / O_DIRECT → block layer → scheduler → driver → device

                                /sys/block/<dev>/queue/scheduler
```

Name	Typical fit
`none`	Fast NVMe; let the device reorder
`mq-deadline`	General purpose; latency bounds
`kyber`	Latency-oriented mq scheduler
`bfq`	Desktop fairness / cgroup-aware weighting

Worked examples

Kernel on this box: **6.12.94+**. No scheduler was switched (read-only).

Case 1: list devices and the bracketed active scheduler

```
lsblk -d -o NAME,TYPE,SIZE,ROTA,SCHED
cat /sys/block/vda/queue/scheduler
cat /sys/block/vda/queue/rotational
cat /sys/block/vda/queue/nr_requests
cat /sys/block/vda/queue/read_ahead_kb
```

Output from this run:

```
=== block devices ===
NAME TYPE  SIZE ROTA SCHED
vda  disk  128G    1 mq-deadline
vdb  disk  128G    1 mq-deadline
=== vda scheduler ===
none [mq-deadline] kyber bfq
=== vda rotational / nr_requests / read_ahead ===
1
256
128
=== available schedulers per real disk ===
vda: none [mq-deadline] kyber bfq
vdb: none [mq-deadline] kyber bfq
6.12.94+
```

Both virtio disks advertise `none [mq-deadline] kyber bfq` with **mq-deadline** active. `ROTA=1` here is the virtio default signal — treat it as a lab artifact, not proof you have spinning rust.

Trap

Echoing a new scheduler in production without a rollback. Writing to `scheduler` is immediate and global for that device. On shared CI VMs it can starve neighbours. Prefer documenting the current value and changing it only in a controlled image or with root you own.

Boring rule

Read before you write. For NVMe hosts, start from `none` unless a latency SLO says otherwise; for mixed HDD, measure `mq-deadline` vs `bfq` with your real workload, not `dd` alone.

Try this

1. Compare `SCHED` column across bare metal NVMe vs virtio.
2. Plot p99 latency under load for two schedulers in a disposable VM you own.
3. Read `Documentation/block/bfq-iosched.rst` in the kernel tree.

Sources

- Linux kernel docs: `Documentation/block/` (mq-deadline, kyber, bfq)
- `lsblk(8)`, sysfs `queue/scheduler`

tmpfs as Scratch Space (/dev/shm)

tmpfs as Scratch Space (/dev/shm)

Thesis

tmpfs is a filesystem backed by page cache / RAM (and swap if the kernel must). `/dev/shm` is the usual shared-memory tmpfs for POSIX shm and a convenient place for **fast, disposable scratch files**. It is not free disk: fill it and you pressure memory. This chapter inspects mount options and writes a 64 MiB test file to watch capacity move.

Mental model

```
write(file in tmpfs) → RAM pages (file-backed)
unlink / reboot      → pages freed
under pressure       → may spill to swap (still not "durable disk")
```

Worked examples

Case 1: findmnt, write, df

```
findmnt -t tmpfs -o TARGET,SOURCE,FSTYPE,OPTIONS,SIZE,AVAIL
df -h /dev/shm
dd if=/dev/zero of=/dev/shm/books-tmpfs-test bs=1M count=64 status=none
ls -lh /dev/shm/books-tmpfs-test
df -h /dev/shm
rm -f /dev/shm/books-tmpfs-test
```

Output from this run:

```
TARGET     SOURCE FSTYPE OPTIONS                           SIZE AVAIL
/dev/shm   tmpfs  tmpfs  rw,nosuid,nodev                   7.8G  7.8G
 /dev/shm shm    tmpfs  rw,nosuid,nodev,noexec,relatime  7.8G  7.8G
---
Filesystem      Size  Used Avail Use% Mounted on
shm             7.9G     0  7.9G   0% /dev/shm
--- write test ---
-rw-r--r-- 1 box box 64M Oct 27 21:08 /dev/shm/books-tmpfs-test
Filesystem      Size  Used Avail Use% Mounted on
```

```
shm             7.9G   64M  7.8G   1% /dev/shm
--- mount options from /proc/mounts ---
tmpfs /dev/shm tmpfs rw,nosuid,nodev 0 0
shm /dev/shm tmpfs rw,nosuid,nodev,noexec,relatime 0 0
rw,nosuid,nodev
rw,nosuid,nodev,noexec,relatime
```

A 64 MiB write moved `/dev/shm` from empty to ~1% used on an ~7.9 G tmpfs.

Trap

Using tmpfs for data you cannot rebuild. Power loss or OOM kills the files. Also watch `noexec` on some mounts — this host shows a nested `shm` mount with `noexec`, which breaks running binaries from `/dev/shm`.

Boring rule

Scratch, locks, and IPC only. Cap size with `size=` in the mount unit if workloads can balloon.

Try this

1. Compare write bandwidth of `/dev/shm` vs a disk-backed `/tmp`.
2. Read `man tmpfs` and your distro's `/etc/fstab` or systemd mount unit.
3. Check whether CI runners mount `/dev/shm` smaller than you expect (often 64 MiB).

Sources

- `tmpfs(5)`, `shm_overview(7)`
- systemd `tmpfs` mount units

io_uring Basics without the Ceremony

io_uring Basics without the Ceremony

Thesis

io_uring is Linux's submission/completion ring for async I/O: you push SQEs (submission queue entries), the kernel does work, and you reap CQEs (completion queue entries) — amortising syscall overhead for chatty I/O. You do not need a framework to understand the shape.

Mental model

```
mmap two rings shared with the kernel
      SQ                 CQ
  SQE: op, fd, buf…   -->  CQE: res, user_data

io_uring_enter() wakes / waits when needed
```

Common ops (from `linux/io_uring.h`): `IORING_OP_READV`, `WRITEV`, `FSYNC`, `ACCEPT`, `CONNECT`, …

Worked example

This box has kernel UAPI headers but **no liburing** and we will not install packages. Header presence:

```
* If the ring is initialized with IORING_SETUP_SQE128, then
#define IORING_SETUP_IOPOLL (1U << 0)   /* io_context is polled */
#define IORING_SETUP_SQPOLL (1U << 1)   /* SQ poll thread */
#define IORING_SETUP_SQ_AFF (1U << 2)   /* sq_thread_cpu is valid */
#define IORING_SETUP_CQSIZE (1U << 3)   /* app defines CQ size */
#define IORING_SETUP_CLAMP  (1U << 4)   /* clamp SQ/CQ ring sizes */
#define IORING_SETUP_ATTACH_WQ  (1U << 5)   /* attach to existing wq */
#define IORING_SETUP_R_DISABLED (1U << 6)   /* start with ring disabled */
#define IORING_SETUP_SUBMIT_ALL (1U << 7)   /* continue submit on error */
#define IORING_SETUP_COOP_TASKRUN   (1U << 8)
#define IORING_SETUP_TASKRUN_FLAG   (1U << 9)
#define IORING_SETUP_SQE128     (1U << 10) /* SQEs are 128 byte */
#define IORING_SETUP_CQE32      (1U << 11) /* CQEs are 32 byte */
#define IORING_SETUP_SINGLE_ISSUER  (1U << 12)
#define IORING_SETUP_DEFER_TASKRUN  (1U << 13)
```

Minimal conceptual flow with liburing (run on a host that has it):

```
/* sketch - requires liburing */
#include <liburing.h>
/* io_uring_queue_init(entries, &ring, 0);
   sqe = io_uring_get_sqe(&ring);
   io_uring_prep_read(sqe, fd, buf, len, offset);
   io_uring_submit(&ring);
   io_uring_wait_cqe(&ring, &cqe);
   // cqe->res is bytes or -errno
   io_uring_cqe_seen(&ring, cqe); */
```

Without liburing, the same steps are `io_uring_setup`, mmap of SQ/CQ, fill SQEs, `io_uring_enter`, reap CQEs — see the kernel documentation.

For sequential bulk writes (imagers, installers), io_uring shines when you keep multiple SQEs in flight instead of write-syscall-per-block.

Trap

Ignoring `cqe->res < 0`. Completions report errors as negative errno. Also: mixing registered buffers / fixed files incorrectly; start with plain fds and anonymous buffers.

Boring rule

Learn the SQE→CQE loop with one `READV`/`WRITEV` before registering buffers or using `IORING_SETUP_SQPOLL`. Prefer a maintained helper (liburing) over hand-rolled mmap unless you are writing a runtime.

Try this

1. On a lab host: `sudo apt install liburing-dev` (there — not on this bot box) and run the liburing examples.
2. Compare `dd` of a large file with and without an io_uring-based copy tool.
3. Read `IORING_OP_*` in your kernel's `io_uring.h`.

Sources

- Kernel io_uring docs
- `linux/io_uring.h`
- liburing man pages (`io_uring_queue_init`, …)

Bind mounts and mount propagation: shared, slave, private, unbindable

Bind mounts and mount propagation: shared, slave, private, unbindable

Overview

Thesis: a bind mount makes the same directory tree visible at a second place. **Propagation** decides what happens *later*: when something is mounted or unmounted under one of those places, does the other place see it? Every mount has one propagation type, and the type answers that question. Containers, `systemd PrivateTmp=`, Kubernetes volume mounts (`mountPropagation:`), `chroot` build setups and "my USB stick appeared inside the container" all depend on it.

Type	Events from peers/master arrive here	Events here go out	`/proc/self/mountinfo` tag	`findmnt -o PROPAGATION`
shared	yes (from peers)	yes (to peers and slaves)	`shared:N`	`shared`
slave	yes (from its master)	no	`master:N`	`private,slave`
shared + slave	yes	yes, to its own peers	`shared:M` `master:N`	`shared,slave`
private	no	no	(none)	`private`
unbindable	no	no, and it cannot be bind-mounted	`unbindable`	`private,unbindable`

N is the **peer group** ID. Mounts with the same `shared:N` are peers. A mount with `master:N` receives from peer group N.

Mental model

```
      shared:98                 master:98
a  <==========>  b       a  ---------->  c        d (private)    d (unbindable)
mount a/sub  -> b/sub    mount a/two  -> c/two    a/four: not    mount --bind d e
                         mount c/three -/-> a     seen in d      -> EINVAL
```

Three rules:

1. **Propagation belongs to the mount, not the directory.** `mount --make-*` changes the mount at that path; `--make-r*` changes it and every mount below.

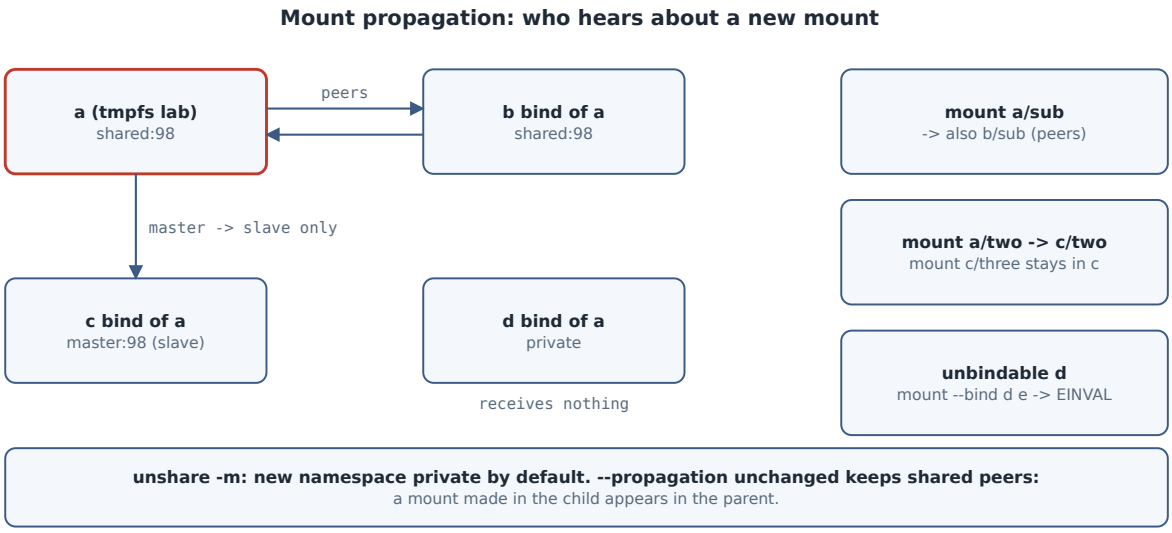


Figure 1: A shared mount at a and its bind mount b form peer group 98, so a mount under either appears under both. A slave bind at c has master 98: mounts under a arrive at c, but a mount made under c stays there. A private bind at d receives nothing. An unbindable mount refuses mount –bind.

2. **A bind mount copies the propagation of the source.** Bind a shared mount and the new mount joins the same peer group; that is why `b` saw `a/sub`.
3. **A new mount namespace starts as a copy.** Shared mounts stay peers *across* namespaces, so a mount made inside a "container" can show up on the host unless someone makes the tree private or slave first.

Syntax

```
mount --bind  OLD NEW            # one mount, no submounts
mount --rbind OLD NEW            # the mount and everything mounted below it
mount -o bind,ro OLD NEW         # read-only view of one mount
mount -o rbind,ro=recursive OLD NEW   # read-only for the whole tree (mount_setattr, Linux >
mount --make-shared   PATH       # also: --make-slave, --make-private, --make-unbindable
mount --make-rshared  PATH       # recursive variants: --make-rslave, --make-rprivate, --mak
findmnt -o TARGET,SOURCE,PROPAGATION [-R PATH]
grep PATH /proc/self/mountinfo   # field 7+: shared:N master:N propagate_from:N unbindable
unshare -m [--propagation private|slave|shared|unchanged] CMD   # default: private
```

Safety

- Every example below runs inside `unshare -rm`: a new user namespace (you are "root" there with no real privilege) plus a new mount namespace. Nothing touches the host's mount table,

and everything disappears when the shell exits.
- On a real host, `--make-rprivate /` or `--make-rshared /` changes behaviour for every service. Do it only inside a namespace you created.
- `umount -l` (lazy) on a shared mount propagates the unmount to every peer and slave.

Examples with Explanations

All examples come from one script run as `unshare -rm bash lab.sh` in `/tmp/proplab`. First, where we start:

```
$ findmnt -no PROPAGATION /
private
```

`unshare` makes the whole tree private by default (see Notes for what a normal systemd host looks like).

1. Shared: two places, one peer group

```
$ mount -t tmpfs lab a
$ mount --make-shared a
$ mount --bind a b
$ grep proplab /proc/self/mountinfo
260 242 0:38 / /tmp/proplab/a rw,relatime shared:98 - tmpfs lab rw,uid=1000,gid=1000
261 242 0:38 / /tmp/proplab/b rw,relatime shared:98 - tmpfs lab rw,uid=1000,gid=1000
$ mkdir a/sub && mount -t tmpfs one a/sub
$ findmnt -o TARGET,SOURCE,PROPAGATION -R a; findmnt -no TARGET,SOURCE,PROPAGATION -R b
TARGET               SOURCE PROPAGATION
/tmp/proplab/a       lab    shared
 /tmp/proplab/a/sub one    shared
/tmp/proplab/b       lab shared
 /tmp/proplab/b/sub one shared
```

`a` and `b` are the same tmpfs (device `0:38`) and both carry `shared:98`. The tmpfs mounted at `a/sub` afterwards appears at `b/sub` as well: a mount event under one peer is replayed under every peer.

2. Slave: receive, never send

```
$ mount --bind a c
$ mount --make-slave c
$ mkdir -p a/two c/three
$ mount -t tmpfs two a/two
$ mount -t tmpfs three c/three
$ grep -E "proplab/(a|c)(/two|/three)? " /proc/self/mountinfo
260 242 0:38 / /tmp/proplab/a rw,relatime shared:98 - tmpfs lab rw,uid=1000,gid=1000
```

```
264 242 0:38 / /tmp/proplab/c rw,relatime master:98 - tmpfs lab rw,uid=1000,gid=1000
265 260 0:45 / /tmp/proplab/a/two rw,relatime shared:126 - tmpfs two rw,uid=1000,gid=1000
267 264 0:45 / /tmp/proplab/c/two rw,relatime master:126 - tmpfs two rw,uid=1000,gid=1000
271 264 0:48 / /tmp/proplab/c/three rw,relatime - tmpfs three rw,uid=1000,gid=1000
$ findmnt -no TARGET,SOURCE,PROPAGATION -R c
/tmp/proplab/c         lab   private,slave
 /tmp/proplab/c/two   two   private,slave
 /tmp/proplab/c/three three private
$ findmnt -no TARGET -R a
/tmp/proplab/a
 /tmp/proplab/a/sub
 /tmp/proplab/a/two
```

`c` started as another peer of 98 (rule 2), and `--make-slave` turned `shared:98` into `master:98`. Now `a/two` arrives as `c/two` (also a slave, of the new group 126), but `c/three` stays local: it has no tag, and `a`'s tree has no `three`. This is the type you want for "a container should see new host mounts, but its own mounts must not leak out". `c` has no `sub` because `--bind` copies one mount, not the ones below it (example 5).

3. Private: a frozen copy

```
$ mount --bind a d
$ mount --make-private d
$ mkdir -p a/four && mount -t tmpfs four a/four
$ findmnt -no TARGET,PROPAGATION -R d
/tmp/proplab/d private
```

`d` was bound from `a` and made private. The later `a/four` never shows up in `d`.

4. Unbindable: no copies at all

```
$ mount --make-unbindable d
$ mount --bind d e; echo exit=$?
mount: /tmp/proplab/e: wrong fs type, bad option, bad superblock on /tmp/proplab/d, missing
       dmesg(1) may have more information after failed mount system call.
exit=32
```

The kernel refuses with `EINVAL`, which util-linux turns into its generic "wrong fs type, bad option…" message. Unbindable mounts exist to stop the exponential mount explosion you get when a tree is recursively bound into a directory inside itself.

5. `--bind` vs `--rbind`

```
$ mount --bind a e
$ mount --rbind a f
$ findmnt -no TARGET -R e | wc -l; findmnt -no TARGET -R f | wc -l
1
4
```

By now `a` has three submounts (`sub`, `two`, `four`). `--bind` copied only `a` itself; `--rbind` copied all four. For a chroot (`/dev`, `/proc`, `/sys` with their submounts) you almost always want `--rbind`, followed by `--make-rslave` so that unmounting the chroot does not unmount the host's `/dev/pts`.

6. Read-only binds are not recursive by default

A tmpfs `r` with a submount `r/in`, bound read-only two ways (`ro.sh`, also inside `unshare -rm`):

```
$ mount -o rbind,ro r s; findmnt -no TARGET,OPTIONS -R s | cut -d, -f1
/tmp/proplab/s      ro
 /tmp/proplab/s/in rw
$ mount -o rbind,ro=recursive r t; findmnt -no TARGET,OPTIONS -R t | cut -d, -f1
/tmp/proplab/t      ro
 /tmp/proplab/t/in ro
```

Plain `ro` with `rbind` made only the top mount read-only, and `s/in` is still writable. `ro=recursive` (util-linux with the new mount API, using `mount_setattr(2)` with `AT_RECURSIVE`) covers the whole tree. If you hand a container a "read-only" host tree, check with `findmnt -o TARGET,OPTIONS -R`.

The trap: a new mount namespace that is still shared

Same user namespace, a fresh shared mount `g`, and two child mount namespaces (`lab6.sh`):

```
$ mount -t tmpfs host g && mount --make-shared g
$ mkdir g/x g/y
# child namespace that keeps propagation as-is:
$ unshare -m --propagation unchanged sh -c 'mount -t tmpfs leak /tmp/proplab/g/x'
# child namespace with unshare default (--propagation private):
$ unshare -m sh -c 'mount -t tmpfs safe /tmp/proplab/g/y; findmnt -no TARGET,SOURCE /tmp/pro
/tmp/proplab/g/y safe
# back in the parent namespace:
$ findmnt -no TARGET,SOURCE,PROPAGATION -R g
/tmp/proplab/g     host shared
 /tmp/proplab/g/x leak shared
```

The first child kept propagation unchanged, so its `g/x` mount travelled back to the parent: a "private" namespace that was not private. The second used unshare's default and its `g/y` stayed inside. Hand-rolled container code that calls `unshare(CLONE_NEWNS)` directly gets the first behaviour, because the *syscall* does not change propagation; only the `unshare` command does, and only by default. Container runtimes call `mount("", "/", NULL, MS_REC|MS_SLAVE, NULL)` (or `MS_PRIVATE`) right after creating the namespace for exactly this reason.

One more cross-namespace rule from mount_namespaces(7): when the new mount namespace belongs to a *less privileged* user namespace, shared mounts copied from the parent become **slave** mounts, and they are locked, so the child cannot unmount them to reveal what is underneath.

Notes and pitfalls

- **systemd makes / shared.** On a normal systemd host, PID 1 runs the equivalent of `mount --make-rshared /` at boot, so `findmnt -no PROPAGATION /` prints `shared`. The kernel default for new mounts is private. The machine these examples ran on is itself a container, so its `/` is `private`.
- **`findmnt` wording.** Slaves print as `private,slave`, meaning "slave, and not shared"; `/proc/self/mountinfo` is the unambiguous source.
- **Kubernetes** exposes this as `mountPropagation: None | HostToContainer | Bidirectional`, which map to private, rslave and rshared. `Bidirectional` needs a privileged container for the leak reason above.
- **Docker** has `--mount type=bind,...,bind-propagation=rslave` (default `rprivate`).
- **`PrivateTmp=` and `ProtectSystem=`** in systemd units are built from a new mount namespace with slave propagation plus bind mounts.
- **Bind-mounting a file** works too (`mount --bind file1 file2`), but editors that save by rename replace the directory entry, and the bind keeps showing the old inode.
- **Recursive read-only** needs `ro=recursive` (example 6). It relies on `mount_setattr(2)` with `AT_RECURSIVE`, which arrived in Linux 5.12.

The boring rule

After creating a mount namespace, make it `rslave` (or `rprivate`) before mounting anything. Use `--rbind` plus `--make-rslave` for chroots. Read `/proc/self/mountinfo` when `findmnt`'s wording is unclear.

Related Commands

- `mount` and `umount` (this part): `--bind`, `--rbind`, lazy unmounts
- `unshare` (Services and runtime part): `--propagation`
- `findmnt -o TARGET,PROPAGATION -R`, `cat /proc/self/mountinfo`
- `nsenter -t PID -m findmnt`: look at another process's mount namespace

Try this

1. In `unshare -rm`, make `a` shared, bind it to `b`, then `umount a/sub`. Does `b/sub` disappear?
2. Build a shared+slave mount: bind a slave `c` to `h` and run `mount --make-shared c` first. What tags does `/proc/self/mountinfo` show for `c` and `h`?
3. Run `mount --rbind /tmp/proplab /tmp/proplab/a` twice, then count mounts with `findmnt -R /tmp/proplab | wc -l`. Repeat after `--make-unbindable`.
4. On a systemd machine, compare `findmnt -no PROPAGATION /` on the host and inside `systemd-run -p PrivateTmp=yes --pty findmnt -no PROPAGATION /tmp`.

Verification

- `lab.sh`, `lab6.sh` and `ro.sh` were run inside `unshare -rm` (util-linux 2.41.5) on Linux 6.12, without root or sudo. All listings are genuine and unedited. Peer group numbers (98, 126) and mount IDs change between runs.
- The systemd, Docker and Kubernetes behaviour in the Notes comes from their documentation (Sources). It was not run here, because the test machine has no systemd.

Sources

- Kernel docs, Shared Subtrees: https://docs.kernel.org/filesystems/sharedsubtree.html
- mount_namespaces(7): https://man7.org/linux/man-pages/man7/mount_namespaces.7.html
- mount(8), bind mounts and propagation flags: https://man7.org/linux/man-pages/man8/mount.8.html
- proc_pid_mountinfo(5): https://man7.org/linux/man-pages/man5/proc_pid_mountinfo.5.html
- unshare(1): https://man7.org/linux/man-pages/man1/unshare.1.html
- mount_setattr(2): https://man7.org/linux/man-pages/man2/mount_setattr.2.html
- Kubernetes, volumes, mount propagation: https://kubernetes.io/docs/concepts/storage/volumes/#mount-propagation
- Docker, bind mounts, configure bind propagation: https://docs.docker.com/engine/storage/bind-mounts/

Terminals and Multiplexers

Intro

Intro

Keep shells alive across disconnects, split panes, and share sessions. `tmux` is the usual default; `screen` remains common on older hosts.

Commands in this part

Command	Role
`tmux`	tmux is a terminal multiplexer: multiple windows and panes inside detachable sessions that survive SSH disconnects.
`screen`	screen is the classic terminal multiplexer: detachable sessions that survive hangups.
`byobu`	The byobu command is a text-based window manager and terminal multiplexer.
`tmate`	tmate is a fork of tmux focused on instant terminal sharing.
`abduco`	abduco is a minimal session attach/detach tool: it runs a process under a controlling terminal session you can…

Suggested starting points

1. Daily driver: `tmux` (sessions, windows, panes).
2. Legacy hosts: `screen`.
3. Convenience wrappers: `byobu`.
4. Remote pair/share: `tmate`; minimal attach tools: `abduco`.

Related parts

- Networking — `ssh` into the host first
- Processes and jobs — reattach instead of losing long jobs

- Shell commands — history inside each pane

Continue with the individual command pages in this part.

tmux

Overview

`tmux` is a **terminal multiplexer**: multiple windows and panes inside detachable sessions that survive SSH disconnects. Prefer it over `nohup` for interactive long-lived remote work, and over ad-hoc background jobs when you need to reattach and see output. For production services still prefer **systemd**.

Syntax

```
tmux [command] [options]
tmux [-L socket-name] [-S socket-path] [-f file]
```

Most daily use is either bare `tmux` or a short subcommand (`new`, `attach`, `ls`, `kill-session`).

Install (Ubuntu)

```
sudo apt update
sudo apt install tmux
tmux -V
```

Prefix key

Default prefix: **Ctrl-b**, then a command key. (Many people rebind to Ctrl-a.)

After prefix	Action
`c`	New window
`,`	Rename window
`n` / `p`	Next / previous window
`0`–`9`	Select window by index
`w`	Window list
`&`	Kill window
`%`	Split pane **left/right**
`"`	Split pane **top/bottom**

After prefix	Action
`o` / arrow keys	Cycle / move between panes
`;`	Last pane
`x`	Kill pane (confirm)
`z`	Zoom toggle (pane fullscreen)
`{` / `}`	Swap panes
`Space`	Cycle pane layouts
`d`	Detach session
`s`	Session list
`$`	Rename session
`[`	Copy / scroll mode
`]`	Paste buffer
`?`	List keys
`:`	Command prompt

Common CLI commands

Command	Description
`tmux`	New session (default name)
`tmux new -s NAME`	Named session
`tmux new -d -s NAME [cmd]`	Detached session (optional command)
`tmux ls`	List sessions
`tmux attach -t NAME`	Attach
`tmux attach -d -t NAME`	Attach and detach others
`tmux switch -t NAME`	Switch (from inside tmux)
`tmux kill-session -t NAME`	Kill session
`tmux kill-server`	Kill all sessions (careful)
`tmux list-windows -t NAME`	Windows in session

Key Use Cases

1. Survive laptop sleep / SSH drops during deploys
2. Pair multiple panes (logs + editor + shell)
3. Long builds or trainers you can reattach to
4. Named sessions per project (`tmux new -s api`)

Examples with Explanations

Example: create, detach, reattach

```
tmux new -s deploy
# ... work ...
# Ctrl-b d
tmux ls
tmux attach -t deploy
# short form:
tmux a -t deploy
```

Core workflow. Detach leaves processes running on the server.

Example: start detached job session

```
tmux new -d -s train 'python3 train.py 2>&1 | tee train.log'
tmux attach -t train
```

Session exists immediately; attach when you want the TTY.

Example: nested SSH-safe habit

```
ssh host
tmux new -s work || tmux attach -t work
```

Create-or-attach pattern for a daily session.

Example: splits for ops

Inside a session:

1. `Ctrl-b %` — vertical split (side by side)
2. `Ctrl-b "` — horizontal split
3. `Ctrl-b z` — zoom the pane you care about
4. Run `journalctl -fu nginx` in one pane, shell in another

Example: scripted window layout

```
tmux new -d -s app
tmux rename-window -t app:0 shell
tmux new-window -t app -n logs 'journalctl -f'
tmux new-window -t app -n top 'htop'
```

```
tmux split-window -h -t app:shell
tmux attach -t app
```

Good for standardizing a "ops dashboard" session.

Example: send keys to a session (automation)

```
tmux new -d -s patch
tmux send-keys -t patch 'sudo apt update && sudo apt upgrade' C-m
```

Use sparingly; prefer scripts over pseudo-interactive automation when possible.

Example: shared socket / pair-ish access

```
tmux -S /tmp/pair.sock new -s pair
# other user (permissions permitting):
tmux -S /tmp/pair.sock attach -t pair
```

For real pairing, tools like `tmate` or proper access control may be better.

Example: minimal `~/.tmux.conf`

```
set -g mouse on
set -g history-limit 50000
set -g base-index 1
setw -g pane-base-index 1
set -g renumber-windows on
# rebind prefix to Ctrl-a (optional):
# unbind C-b
# set -g prefix C-a
# bind C-a send-prefix
```

Reload from inside tmux: `Ctrl-b :` then `source-file ~/.tmux.conf`.

Example: copy mode (scrollback)

`Ctrl-b [` → move with arrows/PageUp → `Space` begin selection → `Enter` copy (bindings vary with mode-keys vi/emacs) → `Ctrl-b ]` paste.

With mouse on, scroll and select often work directly.

Safety

- `tmux kill-server` ends **all** sessions for that server socket — confirm `tmux ls` first.
- Detached sessions still consume resources; clean up dead project sessions.
- Do not run secrets printing in shared sessions.
- Nested tmux (local + remote) confuses prefixes — use a different remote prefix or SSH directly into remote tmux only.

Notes & Pitfalls

- Environment variables are captured at **session start**; re-login SSH agent sockets often need `tmux setenv` or plugin tooling after laptop sleep.
- `TERM`/`COLORTERM` mismatches cause weird colors — prefer `tmux -2` legacy force or correct `terminal-overrides` in conf for modern terminals.
- Clipboard integration needs `xclip`/`wl-clipboard` and conf bindings on desktop Linux.
- Version differences: Ubuntu LTS tmux may lag plugin ecosystem features — check `tmux -V`.
- Prefer one session per task rather than infinite windows without names.

Related Commands

- `screen` — older multiplexer
- `byobu` — profile/enhancement layer over tmux/screen
- `tmate` — easy remote sharing
- `abduco` / `dtach` — minimal detach only
- `nohup` / `systemd-run` — non-interactive longevity
- `ssh` — usual transport

Additional Resources

- `man tmux`
- `tmux list-keys` / `tmux list-commands`

screen

Overview

`screen` is the classic terminal multiplexer: detachable sessions that survive hangups. It is still available on Ubuntu and many rescue environments, but **new muscle memory should prefer `tmux`** unless you are on a host that only has `screen` or you maintain legacy automation. Concepts map closely: sessions, windows, detach/attach.

Syntax

```
screen [options] [command [args]]
screen -S name
screen -r [name]
screen -ls
```

Install (Ubuntu)

```
sudo apt update
sudo apt install screen
screen -v
```

Prefix key

Default prefix: **Ctrl-a**, then a command key. (Note: conflicts with readline "beginning of line" — press `Ctrl-a a` to send literal Ctrl-a.)

After prefix	Action
`c`	New window
`n` / `p`	Next / previous window
`"`	Window list
`A`	Rename window
`k`	Kill window
`S`	Split **horizontal** (regions)
`|`	Split **vertical** (modern screen; if unbound, use conf)

After prefix	Action
`Tab`	Focus next region
`X`	Remove region
`d`	Detach
`[` / `Esc`	Copy / scroll mode
`]`	Paste
`?`	Help
`:`	Command line

Common CLI options

Option	Description
`-S name`	Session name
`-dmS name cmd`	Start detached named session running `cmd`
`-ls` / `-list`	List sessions
`-r name`	Reattach
`-R`	Reattach if exists else create
`-d -r name`	Detach elsewhere then reattach here
`-x name`	Multi-display attach (share session)
`-X stuff '…'`	Send input to a session
`-wipe`	Remove dead session entries

Key Use Cases

1. Keep a shell alive across SSH drops on legacy hosts
2. Detached long runners where tmux is not installed
3. Sharing a session with `-x` for quick co-debugging
4. Rescue environments that ship `screen` only

Examples with Explanations

Example: create, detach, resume

```
screen -S deploy
# ... work ...
# Ctrl-a d
screen -ls
screen -r deploy
```

If only one detached session exists, `screen -r` is enough.

Example: create-or-attach

```
screen -D -RR work
```

Aggressive “get me a session named work” pattern used in older docs; know it may detach others.

Example: start detached command

```
screen -dmS train bash -lc 'python3 train.py 2>&1 | tee train.log'
screen -r train
```

`-dmS` = detached + create + name. Wrap with `bash -lc` when you need login profile or pipelines.

Example: list and wipe zombies

```
screen -ls
screen -wipe
```

Dead sessions after crashes leave “Dead ???” entries; wipe cleans the registry.

Example: multi-attach

```
screen -x deploy
```

Second terminal joins the same session (both see the same windows).

Example: send commands from outside

```
screen -S deploy -X stuff 'systemctl status nginx^M'
```

`^M` is Enter. Fragile compared to real scripts — use for glue only.

Example: log session output

Inside screen: `Ctrl-a H` toggles logging to `screenlog.n` in the cwd (default). Or start with:

```
screen -L -S logged
```

Example: minimal `~/.screenrc`

```
startup_message off
defscrollback 10000
shell -$SHELL
# harder status line:
hardstatus alwayslastline
hardstatus string '%{= kG}[ %{G}%H %{g}][%= %{= kw}%?%-Lw%?%{r}(%{W}%n*%f%t%?(%u)%?%{r})%{w}
```

Example: vertical split note

Classic `screen` was weak at tiling compared to tmux. Recent GNU screen supports vertical splits (`Ctrl-a |` when bound). If your Ubuntu version feels awkward for panes, that is a normal reason to switch to tmux.

Safety

- Killing the wrong session loses unsaved editor buffers in that session.
- `screen -X` can inject input into interactive tools — dangerous on shared sessions.
- Multi-attach (`-x`) shares keystrokes; coordinate before typing production commands.
- Session sockets live under hard-to-guess paths but are still local privilege boundaries — do not relax directory permissions carelessly.

Notes & Pitfalls

- **Prefix conflicts** with Emacs/readline Ctrl-a — muscle memory cost is the #1 complaint.
- Escape detachment: `Ctrl-a d` vs nested screen/ssh needs extra care (`Ctrl-a a d` patterns).
- Environment variable staleness after reattach (SSH_AUTH_SOCK) same class of problem as tmux.
- Prefer tmux for new documentation and training unless constrained.
- BusyBox or tiny systems may ship neither; fall back to `nohup`/`systemd`.

Related Commands

- `tmux` — modern default multiplexer
- `byobu` — friendly wrapper (can use screen backend)
- `tmate` — remote pairing
- `abduco` / `dtach` — detach without windows
- `nohup` — minimal HUP immunity

Additional Resources

- `man screen`
- `Ctrl-a ?` inside a session

byobu

Overview

The `byobu` command is a text-based window manager and terminal multiplexer. It's a wrapper for tmux/screen that provides enhanced features and an easier interface.

Syntax

```
byobu [options] [command]
```

Common Options

Option	Description
`-S`	Start in screen mode
`-T`	Start in tmux mode
`-v`	Version info
`-h`	Show help
`--help`	Detailed help
`-l`	List sessions
`-L`	Select backend
`-c command`	Run command
`-f`	Frontend
`-p profile`	Use profile

Function Keys

Key	Action
`F2`	Create window
`F3`	Previous window
`F4`	Next window
`F5`	Reload profile
`F6`	Detach session
`F7`	Enter scrollback

Key	Action
`F8`	Rename window
`F9`	Configuration menu
`F12`	Lock session
`Shift-F2`	Split vertical

Key Use Cases

1. Session management
2. System monitoring
3. Remote work
4. Project organization
5. Server administration

Examples with Explanations

Example 1: Start Session

```
byobu
```

Start new session

Example 2: Named Session

```
byobu new -s mysession
```

Create named session

Example 3: List Sessions

```
byobu list-sessions
```

Show running sessions

Common Usage Patterns

1. Basic start:

   ```
   byobu
   ```

2. Attach session:

   ```
   byobu attach
   ```

3. Kill session:

   ```
   byobu kill-session
   ```

Security Considerations

1. Session security
2. Remote access
3. Multi-user mode
4. Screen locking
5. SSH integration

Related Commands

- `tmux` - Terminal multiplexer
- `screen` - Terminal multiplexer
- `tmate` - Sharing tool
- `ssh` - Secure shell
- `dtach` - Session detachment

Additional Resources

- Byobu Documentation
- Usage Guide
- System Administration

Best Practices

1. Use status info
2. Configure profiles
3. Custom keybindings
4. Regular updates

5. Session naming

Configuration

1. Status notifications
2. Window layout
3. Color schemes
4. Key bindings
5. Profiles

Troubleshooting

1. Backend issues
2. Display problems
3. Key conflicts
4. Profile errors
5. Performance

tmate

Overview

`tmate` is a fork of tmux focused on **instant terminal sharing**. It creates a session and prints SSH/web URLs so collaborators can attach remotely. Great for pair debugging; treat access tokens as secrets. For local-only multiplexing without sharing, use stock `tmux`.

Syntax

```
tmate [tmux-compatible options]
tmate show-messages
tmate wait tmate-ready
```

Most session control mirrors **tmux** keybindings and commands.

Common usage patterns

Action	Example
Start shared session	`tmate`
Wait until ready	`tmate wait tmate-ready`
Print connection info	`tmate show-messages` / display-panes messages
Detach	`Ctrl-b d` (default prefix like tmux)
Local config	`~/.tmate.conf`

Examples with Explanations

Start a session

```
tmate
# note the ssh and web URLs printed
tmate wait tmate-ready
tmate show-messages
```

Capture SSH URL for a colleague

```
tmate -F show-messages | grep 'ssh session:'
# or scripted:
tmate -S /tmp/tmate.sock new-session -d
tmate -S /tmp/tmate.sock wait tmate-ready
tmate -S /tmp/tmate.sock display -p '#{tmate_ssh}'
```

Read-only vs read-write links

tmate provides different access strings (RO/RW). Share **read-only** by default; only give RW to trusted peers.

Config sketch

```
# ~/.tmate.conf
set -g tmate-server-host ssh.tmate.io
# org-hosted servers possible for privacy
```

End session

```
# exit shells or kill-session
tmate kill-server
```

Prefer local tmux when not sharing

```
tmux new -s solo
```

Notes / Pitfalls

- Default public relay means session metadata/paths may transit third parties — use a **self-hosted** tmate server for sensitive work.
- Anyone with the RW SSH string has full terminal control — treat like a root password.
- Corporate networks may block outbound SSH to tmate hosts.
- Still tmux-based: know prefix keys, panes, and copy-mode.
- Don't leave orphan shared sessions on bastion hosts.

2026-relevant notes

- Alternatives: VS Code Live Share, SSH with `tmux` + explicit user accounts, commercial pair tools.
- For regulated environments, mandate self-hosted relays or forbid tmate.
- Rotate by killing sessions after the pairing call ends.

Related Commands

- `tmux` — local multiplexer foundation
- `screen` — classic alternative
- `ssh` — direct remote access
- `script` / `asciinema` — record sessions without live share

Additional Resources

- `man tmate`, tmate.io documentation

abduco

Overview

`abduco` is a minimal **session attach/detach** tool: it runs a process under a controlling terminal session you can disconnect from and reattach to later. Lighter than full multiplexers (`tmux`, `screen`) — no window manager, just detachable sessions. Often paired with `dvtm` for tiling inside the session.

Optional package; not installed by default on most distros.

Syntax

```
abduco [options] [-e key] [-r] name
abduco -A name [command...]
abduco -l
```

Common Options

Option	Description
`-A name`	Attach or create session `name`
`-a name`	Attach to existing session
`-c name command`	Create session running command
`-e key`	Escape key (default `Ctrl-\`)
`-r`	Read-only attach
`-l`	List sessions
`-n`	Create but don't attach
`-f`	Force (see man for attach behavior)
`-q`	Quiet

Exact flags can vary slightly by version — check `abduco -h`.

Examples with Explanations

Create / attach

```
abduco -A work
# runs $SHELL by default in a new session if needed
```

Run a specific command

```
abduco -c build make -j$(nproc)
abduco -a build
```

List and reattach

```
abduco -l
abduco -a work
```

Detach

```
Ctrl-\   (default detach key - confirm with man/help)
```

Or close the SSH client carefully knowing the session survives.

Read-only observe

```
abduco -r -a work
```

Pair with dvtm

```
abduco -A dev dvtm
```

`dvtm` provides tiling; `abduco` provides detach.

vs tmux

```
# tmux: windows, panes, scripting, ecosystem
tmux new -s work
# abduco: tiny attach/detach
abduco -A work
```

Notes / Pitfalls

- Must be installed; rare on enterprise minimal images.
- Escape key conflicts — rebind with `-e` if needed.
- Not a full replacement for tmux scripting/automation.
- Session socket location depends on build (`ABDUCO_SOCKET_DIR` / runtime dir).
- Sharing sessions multi-user needs careful permissions (prefer tmux/socket policies or separate tools).

2026-relevant notes

- Nice for constrained environments where tmux feels heavy, or for teaching detach basics.
- For production admin work, **tmux** or **systemd** still dominate.
- Combine with mosh/SSH keepalive for flaky links.

Related Commands

- `tmux` / `screen` — full multiplexers
- `dvtm` — tiling wm for terminal
- `dtach` — similar detach tool
- `nohup` / `disown` — weaker persistence
- `systemd-run` — transient services

Additional Resources

- `abduco -h`, project docs (martanne/abduco)

Text and Pipes

Intro

Intro

Filter, reshape, and measure text streams — the core of shell pipelines. Line tools (`grep`/`sed`/`awk`) plus structured JSON (`jq`) and progress (`pv`).

Commands in this part

Command	Role
`grep`	grep searches input lines for a pattern.
`sed`	sed (stream editor) transforms text line by line — substitute, delete, print, and simple inserts — without opening…
`awk`	awk is a pattern–action language for column-oriented text.
`cut`	The cut command extracts specific columns or fields from lines of text.
`sort`	sort orders lines of text (or selected fields) and can merge pre-sorted files.
`uniq`	uniq reports or filters adjacent duplicate lines.
`tr`	The tr (translate) command translates or deletes characters from standard input.
`head`	The head command displays the first lines of files or input streams.
`tail`	The tail command displays the last lines of files or input streams.
`wc`	The wc (word count) command counts lines, words, characters, and bytes in files or input streams.
`tee`	tee reads stdin and writes it to both standard output and one or more files.
`xargs`	xargs builds and executes command lines from standard input.
`diff`	The diff command compares files line by line and displays the differences.
`comm`	comm compares two sorted files line by line and outputs three columns: lines only in file1, only in file2, and…

Command	Role
`paste`	paste merges lines of files side by side, writing corresponding lines separated by tabs (or another delimiter).
`column`	column formats text into aligned columns or a table.
`jq`	jq is a command-line JSON processor.
`pv`	pv (Pipe Viewer) shows a progress bar, throughput, and ETA for data moving through a pipeline.

Suggested starting points

1. Filter: `grep` (or `rg` in Files for recursive code search).
2. Rewrite: `sed`; fields/reports: `awk`.
3. Columns and sets: `cut`, `sort`, `uniq`, `comm`, `paste`, `column`.
4. Edges of streams: `head`, `tail`, `tee`, `xargs`.
5. Compare: `diff`; JSON: `jq`; progress: `pv`.

Related parts

- Files and paths — sources of text
- Shell commands — quoting and `printf`
- Scripting → Bash (this book) — programming beyond one-liners

Continue with the individual command pages in this part.

grep

Overview

`grep` searches input lines for a pattern. GNU grep supports basic/extended/Perl-like regexes, recursive directory search, and rich context controls. Still the default everywhere; `rg` (ripgrep) is a popular faster alternative when installed.

Syntax

```
grep [OPTIONS] PATTERN [FILE...]
grep [OPTIONS] -e PATTERN ... [FILE...]
command | grep [OPTIONS] PATTERN
```

Common Options

Option	Description
`-E`	Extended regex (ERE)
`-F`	Fixed strings (no regex)
`-P`	Perl-compatible regex (PCRE) if built-in
`-i`	Case-insensitive
`-v`	Invert match
`-n`	Line numbers
`-H` / `-h`	With / without filename
`-r` / `-R`	Recursive (follow / include symlinks differently)
`-I`	Skip binary
`-l` / `-L`	Files with / without matches
`-c`	Count matches per file
`-A`/`-B`/`-C` n	Context after/before/both
`-w`	Whole word
`-o`	Only the match
`-m n`	Stop after n matches per file
`--color=auto`	Highlight
`-q`	Silent (exit status only)

Key Use Cases

1. Filter logs and command output
2. Code/config search in trees
3. Scripting success/failure via exit status
4. Extract substrings with `-o`

Examples with Explanations

Basic

```
grep error /var/log/syslog
grep -i error /var/log/syslog
```

Extended regex

```
grep -E 'error|warn|fail' app.log
```

Fixed string (fast, safe)

```
grep -F 'https://example.com/path?x=1' urls.txt
```

Recursive code search

```
grep -RIn --exclude-dir=.git --exclude-dir=node_modules 'TODO' .
```

Context

```
grep -n -C3 'OutOfMemory' heap.log
```

Files that match

```
grep -rl 'Listen 80' /etc/apache2 2>/dev/null
```

Exit status in scripts

```
if grep -q 'ready' /tmp/status; then
  echo ok
fi
```

Only matching part

```
grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' access.log | sort -u
```

Invert

```
grep -v '^#' /etc/ssh/sshd_config | grep -v '^$'
```

Understanding Output

Default: matching lines. With multiple files, lines are prefixed by filename. Exit codes: `0` match, `1` no match, `2` error.

Common Usage Patterns

Pipe from journalctl

```
journalctl -u nginx --since today | grep -i error
```

Count HTTP status codes

```
grep -oE 'HTTP/[0-9.]+" [0-9]{3}' access.log | awk '{print $NF}' | sort | uniq -c
```

Notes & Pitfalls

- Quote patterns to stop shell globbing.
- Basic regex treats `+`/`|` differently than `-E`.
- Binary matches print “Binary file … matches”; use `-a` or `-I`.

- For huge repos, consider `rg` or `git grep`.

Related Commands

- `rg` — ripgrep (if installed)
- `ack` / `ag` — other code search tools
- `sed` / `awk` — transform, not just filter
- `find … -exec grep` — older recursive pattern
- `journalctl` / `dmesg` — log sources

Additional Resources

- `man grep`
- GNU grep manual

sed

Overview

`sed` (stream editor) transforms text **line by line** — substitute, delete, print, and simple inserts — without opening an interactive editor. GNU sed is standard on Ubuntu/Debian. Prefer `sed` for mechanical rewrites; use `awk` for field logic and an editor for multi-line structural edits.

Syntax

```
sed [options] 'script' [file...]
sed [options] -e 'script' -e 'script' [file...]
sed [options] -f script.sed [file...]
… | sed 'script'
```

Common Options

Option	Description
`-n`	Quiet; only explicit `p` prints
`-E` / `-r`	Extended regex (ERE)
`-i[SUFFIX]`	In-place edit (optional backup suffix)
`-e`	Add a script expression
`-f file`	Read script from file
`-z`	NUL-separated “lines” (GNU; useful with `find -print0`)

Common script commands

Cmd	Meaning
`s/pat/repl/flags`	Substitute (`g` global, `i` ignore case, `p` print, `2` Nth only)
`d`	Delete line
`p`	Print line
`a\` / `i\` / `c\`	Append / insert / change (multi-line form in scripts)
`q` / `Q`	Quit (GNU `Q` without printing pattern space)
`y/src/dst/`	Transliterate like `tr`

Cmd	Meaning
`=`	Print line number

Safety

- `sed -i` rewrites files — always keep backups (`-i.bak`) until the expression is trusted.
- Prefer **single quotes** around scripts so the shell does not expand `$` and `` ` ``.
- Never run `sed -i` on binary files or unknown encodings.
- On macOS, `-i` requires a mandatory suffix argument — scripts written for GNU sed may break there.

Key Use Cases

1. Bulk string replace in config/logs
2. Strip comments/blank lines
3. Extract a capture group (print-only substitute)
4. Normalize line endings / trailing whitespace
5. Quick in-place flips for feature flags

Examples with Explanations

Substitute first match per line

```
sed 's/foo/bar/' file.txt
```

Only the first `foo` on each line becomes `bar`.

Global substitute

```
sed 's/foo/bar/g' file.txt
```

Every non-overlapping match on the line is replaced.

Extended regex

```
sed -E 's/[0-9]{1,3}(\.[0-9]{1,3}){3}/IP/g' access.log
```

With `-E`, `{}` and `()` work without backslashes (GNU ERE).

Delete matching lines

```
sed '/^#/d; /^$/d' config.txt
```

Drops comment lines and empty lines — useful for "effective config" views.

Print only rewritten matches

```
sed -n 's/.*ERROR: //p' app.log
```

`-n` suppresses default output; `p` prints only successful substitutions — like a capturing `grep`.

In-place with backup

```
sed -i.bak 's/Enable=false/Enable=true/' app.conf
diff -u app.conf.bak app.conf
```

Inspect the diff before deleting the `.bak`.

Line addresses and ranges

```
sed -n '10,20p' file.txt          # print lines 10-20
sed '1,5d' file.txt               # delete first five
sed '/^## Start/,/^## End/d' doc.md
```

Addresses can be numbers, `$` (last line), or `/regex/`.

Multiple expressions

```
sed -e 's/\r$//' -e 's/[[:space:]]\+$//' file.txt
```

Strip CR from CRLF files and trailing whitespace.

Use match in replacement

```
sed -E 's/(user=)[^ ]+/\1REDACTED/g' app.log
```

Parentheses capture; `\1` reinserts the prefix. `&` is the whole match.

Change delimiter when paths have slashes

```
sed 's|/usr/local/bin|/opt/bin|g' env.sh
```

Any character after `s` can be the delimiter (`s|||`, `s###`).

Hold space: join two lines (advanced)

```
sed -n '1h;1!H;${x;s/\n/,/g;p}' list.txt
```

GNU sed can join lines via hold space; for heavy multi-line work prefer `awk` or `perl`.

Dry-run pattern for risky edits

```
sed 's/prod/staging/g' settings.yml | diff -u settings.yml -
# then:
sed -i.bak 's/prod/staging/g' settings.yml
```

Always preview against the original before `-i`.

Understanding Output

By default sed prints every line (after script processing) to stdout. Exit status is 0 on success; GNU sed can return non-zero for invalid options or I/O errors. In-place mode still writes a temporary file then renames — a full disk mid-edit can fail; keep backups.

Notes & Pitfalls

- Default regex is **BRE**; `+`, `?`, `|` need escapes unless you use `-E`.
- `s/a/b/` only changes the first match; forget `g` often.
- `&` in the replacement is the whole match; write `\&` for a literal ampersand.

- `sed` is line-oriented — multi-line HTML/XML edits are painful.
- Character classes: prefer `[[:space:]]` over `\s` for portable scripts.

Related Commands

- `awk` — field-oriented processing
- `grep` — filter only
- `tr` — character classes / deletion
- `perl -pe` — richer substitutions
- `diff` / `patch` — review and apply edits

Additional Resources

- `man sed`
- GNU sed manual

awk

Overview

`awk` is a pattern–action language for column-oriented text. It shines at field extraction, reports, and light transforms in pipelines — often replacing multi-step `cut`/`grep`/`sed` combinations. Prefer it when you think in **rows and fields**; use `sed` for pure stream edits and `jq` for JSON.

On Ubuntu you typically get `mawk` or `gawk` depending on package; install `gawk` when you need GNU extensions (gensub, multi-dimensional arrays, network redirections).

Syntax

```
awk [options] 'program' [file...]
awk -f program.awk [file...]
… | awk 'program'
```

Program form: `pattern { actions }`. Default action is `{ print }`. Built-in field vars: `$1`…`$NF`, `NF` (field count), `NR`/`FNR` (record numbers), `FS`/`OFS` (separators), `RS`/`ORS`, `FILENAME`.

Common Options

Option	Description
`-F fs`	Input field separator (same as `BEGIN { FS=… }`)
`-v var=val`	Set variable before program runs
`-f file`	Read program from file
`-W`	Implementation warnings (gawk)

Key Use Cases

1. Print or reorder selected columns
2. Filter rows by field values or regex
3. Sum / average / count groups

4. Reformat logs into CSV/TSV for spreadsheets

5. Quick reports from `/etc/passwd`, `ps`, or access logs

Examples with Explanations

Print columns

```
awk '{print $1, $3}' file.txt
awk -F: '{print $1, $7}' /etc/passwd
```

Space-separated print uses `OFS` (default space). For passwd, `-F:` is the standard delimiter.

Filter rows

```
ps aux | awk 'NR==1 || $3+0 > 5.0'          # header + high CPU
awk -F, '$3 > 100 {print $1}' data.csv
awk '/ERROR/ && !/retry/' app.log
```

`$3+0` forces a numeric comparison (avoids string "10" < "9" surprises).

Sum and average a column

```
awk '{s+=$1} END {print s}' numbers.txt
awk '{s+=$1; n++} END {if (n) print s/n}' numbers.txt
```

`END` runs once after all input. Guard division by zero when the file may be empty.

Change separators (CSV → TSV)

```
awk -F, 'BEGIN{OFS="\t"} {print $2,$1}' data.csv
```

Set `OFS` in `BEGIN` so every `print` uses tabs.

Last field and field count

```
awk '{print $NF}' file.txt          # last field
awk '{print NF, $0}' file.txt       # how many fields per line
```

`$NF` is "last field"; `$(NF-1)` is second-to-last.

Multiple patterns / counters

```
awk '/ERROR/ {err++} /WARN/ {warn++} END {print "err=" err, "warn=" warn}' app.log
```

Patterns without braces still run the default print; here braces only tally.

Build a frequency report

```
awk -F: '{uids[$3]++} END {for (u in uids) print u, uids[u]}' /etc/passwd | sort -n
```

Associative arrays are the usual way to group. Order of `for (u in …)` is undefined — pipe to `sort`.

Extract IP from mixed lines

```
awk 'match($0, /[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+/) {
  print substr($0, RSTART, RLENGTH)
}' access.log
```

`match` sets `RSTART`/`RLENGTH` for the first match on the line (works in mawk/gawk).

Skip header, process the rest

```
awk -F, 'NR==1 {next} {sum+=$4} END {print sum}' report.csv
```

`next` jumps to the following record without running later rules on this line.

Script-friendly exit status

```
awk 'BEGIN{exit 0}' </dev/null
# fail CI if any line exceeds threshold:
awk -v lim=90 '$1+0 > lim {bad=1; print} END {exit bad+0}' metrics.txt
```

`END { exit code }` is useful in pipelines and checks.

Field splitting on multi-character FS (gawk)

```
gawk -F'[[:space:]]*\|[[:space:]]*' '{print $2}' table.txt
```

When separators are messy, prefer `gawk` or preprocess with `tr`/`sed`.

Understanding Output

- Records are lines by default (`RS = "\n"`).
- Empty fields still count toward `NF` when `FS` is a single character (e.g. `a::b` → three fields with `-F:`).
- Uninitialized variables are `""` or `0` depending on context (string vs numeric).
- Exit status is 0 on success; non-zero if the program calls `exit n` or the implementation hits a fatal error.

Notes & Pitfalls

- Default `FS` is **whitespace** (runs of spaces/tabs), not a single space.
- Locale can affect regex and sorting of output you pipe to `sort`.
- Complex CSV with quoted commas needs a real CSV parser (`python`, `mlr`), not naive `-F,`.
- `print $1 $2` concatenates; `print $1, $2` inserts `OFS`.
- BusyBox awk is smaller — stick to portable constructs in scripts meant for containers/embed.

Related Commands

- `cut` — simple fixed-field slice
- `sed` — stream edits / substitutions
- `sort` / `uniq` — often after awk for reports
- `jq` — JSON
- `mlr` (Miller) — richer tabular CLI if installed

Additional Resources

- `man awk`
- GNU Awk User's Guide

cut

Overview

The `cut` command extracts specific columns or fields from lines of text. It's useful for processing structured data like CSV files, logs, and delimited text.

Syntax

```
cut [options] [file...]
```

Common Options

Option	Description
`-f list`	Select fields
`-d char`	Field delimiter
`-c list`	Select characters
`-b list`	Select bytes
`-s`	Suppress lines without delimiters
`--complement`	Invert selection
`--output-delimiter=string`	Output delimiter

Field/Character Lists

Format	Description
`1`	Field/character 1
`1,3,5`	Fields 1, 3, and 5
`1-5`	Fields 1 through 5
`1-`	Field 1 to end
`-5`	First 5 fields
`1,3-5,7`	Mixed selection

Key Use Cases

1. Extract CSV columns
2. Process log files
3. Parse structured text
4. Data extraction
5. Text manipulation

Examples with Explanations

Example 1: Extract Fields

```
cut -f 1,3 -d ',' data.csv
```

Extracts fields 1 and 3 from CSV file

Example 2: Extract Characters

```
cut -c 1-10 file.txt
```

Extracts first 10 characters from each line

Example 3: Custom Delimiter

```
cut -f 2 -d ':' /etc/passwd
```

Extracts usernames from passwd file

Working with Different Delimiters

Common delimiters: - `,` - Comma (CSV) - `:` - Colon (passwd, PATH) - `\t` - Tab (TSV) - ` ` - Space - `|` - Pipe

Common Usage Patterns

1. Extract usernames:

   ```
   cut -f 1 -d ':' /etc/passwd
   ```

2. Get file extensions:

```
ls | cut -d '.' -f 2-
```

3. Process CSV data:

```
cut -f 2,4,6 -d ',' data.csv
```

Advanced Operations

1. Suppress delimiter-less lines:

```
cut -f 1 -d ',' -s file.csv
```

2. Change output delimiter:

```
cut -f 1,2 -d ',' --output-delimiter='|' data.csv
```

3. Complement selection:

```
cut -f 1,3 --complement -d ',' data.csv
```

Character vs Field Extraction

Character extraction (`-c`): - Fixed position extraction - Useful for fixed-width data - Byte-based positioning

Field extraction (`-f`): - Delimiter-based extraction - Variable width fields - More flexible for structured data

Performance Analysis

- Very fast operation
- Minimal memory usage
- Streaming operation
- Efficient for large files
- Good pipeline performance

Related Commands

- `awk` - More powerful field processing
- `grep` - Pattern matching
- `sed` - Stream editing
- `sort` - Sort lines

- `uniq` - Remove duplicates

Additional Resources

- GNU cut manual
- Cut Command Examples

Best Practices

1. Specify delimiters explicitly
2. Test field numbers with sample data
3. Use character extraction for fixed-width data
4. Consider using awk for complex operations
5. Handle missing delimiters appropriately

Common Patterns

1. Extract IP addresses:

```
cut -f 1 -d ' ' access.log
```

2. Get file sizes:

```
ls -l | cut -c 30-40
```

3. Process PATH variable:

```
echo $PATH | cut -f 1 -d ':'
```

Integration Examples

1. With sort and uniq:

```
cut -f 1 -d ',' data.csv | sort | uniq -c
```

2. With grep:

```
grep "error" log.txt | cut -f 1 -d ' '
```

3. Pipeline processing:

```
cat data.txt | cut -f 2,4 -d '|' | sort
```

Troubleshooting

1. Wrong field numbers
2. Delimiter not found
3. Character encoding issues
4. Empty fields handling
5. Multi-character delimiters (use awk instead) ## Additional Examples

```
cut -d: -f1 /etc/passwd
cut -d, -f1,3 data.csv
echo 'a b c' | cut -d' ' -f2
cut -c1-10 file.txt
```

sort

Overview

`sort` orders lines of text (or selected fields) and can merge pre-sorted files. It is a core building block for log analytics, unique counts (`sort | uniq -c`), and deterministic output in scripts. GNU `sort` (Ubuntu default) supports huge files via temporary disk, multiple keys, numeric/version/human-size modes, and stable sort.

Syntax

```
sort [OPTIONS] [FILE...]
command | sort [OPTIONS]
```

With no files (or `-`), reads stdin. Multiple files are concatenated then sorted unless `-m` merges already-sorted inputs.

Common Options

Option	Description
`-n`	Numeric sort
`-h`	Human numeric (`2K`, `3M`, `1G`)
`-V`	Version sort (`v2.10` after `v2.9`)
`-r`	Reverse
`-u`	Unique (drop duplicate lines after sort)
`-f`	Fold case (ignore case)
`-b`	Ignore leading blanks
`-k POS[,POS]`	Sort by key/field (see below)
`-t CHAR`	Field separator
`-s`	Stable sort (preserve order of equal keys)
`-R`	Random shuffle (hash-based)
`-o file`	Write output to file (safe for in-place with care)
`-m`	Merge already-sorted files
`-c` / `-C`	Check whether sorted (`-C` quiet)
`-z`	NUL-terminated lines (with `find -print0` pipelines)
`--parallel=N`	Parallel sort threads (GNU)
`-S size`	Memory buffer size (GNU)

Key syntax (`-k`)

Fields are 1-based. Examples: - `-k2,2` — only field 2
- `-k2` — field 2 through end of line
- `-k2n` — field 2 as number
- `-k1,1 -k2,2n` — first by field 1, then numeric field 2

Default field separator is non-blank to blank transitions (whitespace runs).

Examples with Explanations

Lexicographic sort

```
sort names.txt
sort -r names.txt
```

Numeric sort

```
sort -n scores.txt
printf '10\n2\n1\n' | sort -n
```

Without `-n`, `10` sorts before `2` as text.

Human-readable sizes

```
du -h /var/* 2>/dev/null | sort -h
du -h /var/* 2>/dev/null | sort -hr | head
```

Version numbers

```
printf 'v1.9\nv1.10\nv2.0\n' | sort -V
```

Sort by a column (whitespace)

```
# passwd-like: sort by UID (field 3) numerically
sort -t: -k3,3n /etc/passwd
```

CSV-ish second column (simple cases)

```
sort -t, -k2,2 file.csv
```

Embedded commas/quotes need a real CSV tool (`mlr`, `python`); `sort` is field-dumb.

Unique lines

```
sort -u access-hosts.txt
# equivalent pipeline often written as:
sort access-hosts.txt | uniq
```

Frequency report (sort | uniq -c | sort)

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head
```

Classic "top talkers" pattern: count then re-sort by count.

Check if already sorted

```
sort -c names.txt && echo sorted
sort -C names.txt   # silent; use exit status
```

Safe "sort in place"

```
sort -o names.txt names.txt
```

GNU sort handles `-o` same-file correctly; `sort file > file` truncates first and destroys data.

Merge pre-sorted shards

```
sort -m part-a.txt part-b.txt part-c.txt > all-sorted.txt
```

Much faster than re-sorting the concatenation when inputs are already ordered.

Stable sort by key, preserve secondary order

```
sort -s -k1,1 events.log
```

NUL-safe with find

```
find . -type f -print0 | sort -z | xargs -0 ls -l
```

Randomize lines

```
sort -R items.txt | head -n 5
```

For cryptographic shuffles use `shuf` instead of relying on `sort -R` alone.

Understanding Output

Output is the full lines in sorted order. With `-u`, only the first of equal lines (per current key/order rules) remains. Exit status `0` on success; `sort -c` returns non-zero if out of order.

Locale (`LC_COLLATE` / `LANG`) affects byte vs dictionary order. For scripts that need byte-wise sort:

```
LC_ALL=C sort file
```

Notes & Pitfalls

- **Locale surprises:** `en_US.UTF-8` vs `C` can reorder punctuation and case. Pin `LC_ALL=C` for deterministic machine output.
- Numeric sort of non-numeric prefixes: `sort -n` parses a leading number; garbage may sort as zero.
- `-k2` vs `-k2,2` is a frequent bug — open-ended keys include the rest of the line.
- Large sorts use `/tmp` (or `$TMPDIR`); disk-full mid-sort fails — free space or set `-S`/`TMPDIR` on big data hosts.
- `sort -u` uniqueness respects the sort keys/options, not always the whole line if keys are restricted — know what you keyed.
- Prefer `shuf` for sampling random lines; prefer `mlr`/`csvkit` for real CSV.

Related Commands

- `uniq` — adjacent unique/count (usually after `sort`)
- `shuf` — random permutations
- `comm` — compare two sorted files
- `join` — relational join on sorted files
- `awk` / `cut` — extract fields before sorting
- `wc -l` — count lines after filtering

Additional Resources

- `man sort`
- GNU coreutils info: `info sort`

uniq

Overview

`uniq` reports or filters **adjacent** duplicate lines. It does not search the whole file for duplicates unless identical lines are already next to each other — which is why the standard pattern is `sort | uniq`. GNU `uniq` can count runs, show only duplicates, or skip comparison of leading fields/characters.

Syntax

```
uniq [OPTIONS] [INPUT [OUTPUT]]
command | uniq [OPTIONS]
```

Common Options

Option	Description
`-c`	Prefix each line with occurrence count
`-d`	Only print lines that are duplicated
`-D`	Print all duplicate lines (GNU; every copy in a run)
`-u`	Only print lines that are unique (appear once)
`-i`	Case-insensitive compare
`-f N`	Skip first N fields when comparing
`-s N`	Skip first N characters when comparing
`-w N`	Compare at most N characters
`-z`	NUL-terminated lines (GNU)

Examples with Explanations

Remove adjacent duplicates

```
printf 'a\na\nb\na\n' | uniq
# → a / b / a   (the second 'a' group is separate)
```

Without sorting, non-adjacent duplicates remain.

The usual "unique lines in file" pattern

```
sort names.txt | uniq
sort -u names.txt          # often faster one-process equivalent
```

Count occurrences

```
sort access-hosts.txt | uniq -c
sort access-hosts.txt | uniq -c | sort -nr | head
```

`uniq -c` counts **runs**; sorting first groups equal lines so the count is global.

Top IPs from a log (field extract → sort → uniq -c → sort)

```
awk '{print $1}' /var/log/nginx/access.log \
  | sort \
  | uniq -c \
  | sort -nr \
  | head -n 20
```

Only lines that appear more than once

```
sort ids.txt | uniq -d
```

Only lines that appear exactly once

```
sort ids.txt | uniq -u
```

Case-insensitive uniqueness

```
printf 'Foo\nfoo\nBAR\n' | sort -f | uniq -i
```

Sort with matching case rules (`-f`) so case variants become adjacent.

Ignore leading fields when comparing

```
# two columns: timestamp id - unique by id only among adjacent
sort -k2,2 events.txt | uniq -f 1
```

`-f 1` skips field 1 (the timestamp) during comparison; whitespace field rules match `sort`'s default.

Compare only a prefix of each line

```
uniq -w 8 hashes.txt
```

Useful when a fixed-width key leads the line.

Show every line that is part of a duplicate group

```
sort data.txt | uniq -D
```

NUL-separated path lists

```
find . -type f -print0 | sort -z | uniq -z | xargs -0 …
```

Rare but correct for exotic pipelines that stay NUL-safe end-to-end.

Count total unique lines (quick)

```
sort file.txt | uniq | wc -l
# or
sort -u file.txt | wc -l
```

Understanding Output

Default: one line per adjacent run of identical lines. With `-c`, output looks like:

```
     12 10.0.0.5
      7 10.0.0.8
      1 10.0.0.9
```

Counts are right-aligned in a fixed width (GNU). Leading spaces are normal — strip with `awk '{print $1}'` when you need the number alone.

Notes & Pitfalls

- **`uniq` without `sort` only collapses neighbors.** This is the #1 misconception.
- `sort -u` and `sort | uniq` are often equivalent for whole-line uniqueness; prefer one clear form in scripts.
- Locale can affect whether lines compare equal; use `LC_ALL=C` for byte-wise behavior in tooling.
- `uniq -c | sort -n` sorts by the whole line as text unless you use `sort -nr` and account for leading spaces — `sort -k1,1nr` is clearer on counted output.
- Blank lines are real lines; consecutive empties collapse under `uniq`.
- For multisets and complex keys, `awk` associative arrays or `datamash` may be clearer than `uniq -f` gymnastics.

Related Commands

- `sort` / `sort -u` — order and unique
- `comm` — compare two sorted unique streams
- `awk '!seen[$0]++'` — unique without sorting (memory-bound, order-preserving first wins)
- `grep -v` — filter patterns, not duplicate runs
- `wc -l` — count remaining lines

Additional Resources

- `man uniq`
- GNU coreutils info: `info uniq`

tr

Overview

The `tr` (translate) command translates or deletes characters from standard input. It's used for character substitution, deletion, and squeezing repeated characters.

Syntax

```
tr [options] set1 [set2]
```

Common Options

Option	Description
`-c`	Complement set1
`-d`	Delete characters in set1
`-s`	Squeeze repeated characters
`-t`	Truncate set1 to length of set2

Character Sets

Set	Description
`[:alnum:]`	Alphanumeric characters
`[:alpha:]`	Alphabetic characters
`[:digit:]`	Digits 0-9
`[:lower:]`	Lowercase letters
`[:upper:]`	Uppercase letters
`[:space:]`	Whitespace characters
`[:punct:]`	Punctuation characters
`[:print:]`	Printable characters
`[:cntrl:]`	Control characters

Key Use Cases

1. Case conversion
2. Character replacement
3. Delete unwanted characters
4. Format text data
5. Clean input data

Examples with Explanations

Example 1: Uppercase Conversion

```
echo "hello world" | tr '[:lower:]' '[:upper:]'
```

Output: `HELLO WORLD`

Example 2: Delete Characters

```
echo "hello123world" | tr -d '[:digit:]'
```

Output: `helloworld`

Example 3: Replace Characters

```
echo "hello world" | tr ' ' '_'
```

Output: `hello_world`

Example 4: Squeeze Repeated Characters

```
echo "hello    world" | tr -s ' '
```

Output: `hello world`

Common Usage Patterns

1. Convert to lowercase:

```
echo "HELLO" | tr '[:upper:]' '[:lower:]'
```

2. Remove newlines:

```
cat file.txt | tr -d '\n'
```

3. Replace multiple characters:

```
echo "a,b;c:d" | tr ',;:' '   '
```

Character Ranges

1. Letter ranges:

```
echo "hello" | tr 'a-z' 'A-Z'
```

2. Number ranges:

```
echo "123" | tr '1-3' 'abc'
```

3. Custom ranges:

```
echo "hello" | tr 'helo' '1234'
```

Advanced Usage

1. Complement sets:

```
echo "hello123" | tr -cd '[:alpha:]'  # Keep only letters
```

2. Multiple operations:

```
echo "Hello World" | tr '[:upper:]' '[:lower:]' | tr ' ' '_'
```

3. ROT13 encoding:

```
echo "hello" | tr 'a-zA-Z' 'n-za-mN-ZA-M'
```

Text Processing

1. Clean CSV data:

```
cat data.csv | tr -d '"' | tr ',' '\t'
```

2. Format phone numbers:

```
echo "1234567890" | tr '0-9' '(###) ###-####'
```

3. Remove control characters:

```
cat file.txt | tr -d '[:cntrl:]'
```

Performance Analysis

- Very fast character processing
- Stream-based operation
- Minimal memory usage
- Efficient for large files
- Good pipeline performance

Related Commands

- `sed` - Stream editor
- `awk` - Text processing
- `cut` - Extract fields
- `sort` - Sort lines
- `uniq` - Remove duplicates

Best Practices

1. Use character classes for portability
2. Test transformations on sample data
3. Combine with other text tools
4. Handle special characters carefully
5. Consider locale settings

Data Cleaning

1. Remove punctuation:

```
echo "Hello, World!" | tr -d '[:punct:]'
```

2. Normalize whitespace:

```
echo "hello    world" | tr -s '[:space:]' ' '
```

3. Extract numbers:

```
echo "abc123def456" | tr -cd '[:digit:]'
```

File Processing

1. Convert line endings:

```
tr -d '\r' < dos_file.txt > unix_file.txt
```

2. Create word list:

```
cat text.txt | tr '[:space:][:punct:]' '\n' | tr -s '\n'
```

3. Count characters:

```
cat file.txt | tr -cd '[:alpha:]' | wc -c
```

Integration Examples

1. With find for filename processing:

```
find . -name "*.txt" | tr '[:upper:]' '[:lower:]'
```

2. Log processing:

```
tail -f access.log | tr ',' '\t' | cut -f1
```

3. Data format conversion:

```
cat data.txt | tr ';' ',' > data.csv
```

Scripting Applications

1. Input validation:

```
validate_input() {
    echo "$1" | tr -cd '[:alnum:]' | grep -q . || return 1
}
```

2. Password generation:

```
generate_password() {
    tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 12
}
```

Special Characters

1. Handle tabs:

```
echo -e "hello\tworld" | tr '\t' ' '
```

2. Process escape sequences:

```
echo "hello\nworld" | tr '\\' '/'
```

3. Unicode handling:

```
echo "café" | tr 'é' 'e'
```

Troubleshooting

1. Character encoding issues
2. Locale-specific behavior
3. Special character handling
4. Set length mismatches
5. Unexpected transformations

Security Applications

1. Sanitize input:

```
echo "$user_input" | tr -cd '[:alnum:]._-'
```

2. Remove dangerous characters:

```
echo "$filename" | tr -d '/<>:|*?"\\'
```

Performance Optimization

1. Use character classes:

```
# Faster
tr '[:lower:]' '[:upper:]'
# Slower
tr 'abcdefghijklmnopqrstuvwxyz' 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

2. Combine operations:

```
# Single tr call is faster
echo "Hello World" | tr '[:upper:] ' '[:lower:]_'
```

Real-world Examples

1. Log analysis:

```
grep ERROR /var/log/app.log | tr '[:upper:]' '[:lower:]' | sort | uniq -c
```

2. Data migration:

```
cat old_format.txt | tr '|' ',' | tr -s ' ' > new_format.csv
```

3. Text normalization:

```
cat document.txt | tr -s '[:space:]' ' ' | tr '[:upper:]' '[:lower:]'
```

head

Overview

The `head` command displays the first lines of files or input streams. By default, it shows the first 10 lines, making it useful for previewing file contents.

Syntax

```
head [options] [file...]
```

Common Options

Option	Description
`-n num`	Show first num lines
`-c num`	Show first num bytes
`-q`	Suppress headers
`-v`	Always show headers
`-z`	Line delimiter is NUL
`--lines=num`	Same as -n
`--bytes=num`	Same as -c

Key Use Cases

1. Preview file contents
2. Extract file headers
3. Sample data examination
4. Log file monitoring
5. Quick file inspection

Examples with Explanations

Example 1: Default Usage

```
head file.txt
```

Shows first 10 lines of file

Example 2: Specific Line Count

```
head -n 5 file.txt
```

Shows first 5 lines

Example 3: Multiple Files

```
head -n 3 *.txt
```

Shows first 3 lines of each txt file

Example 4: Byte Count

```
head -c 100 file.txt
```

Shows first 100 bytes

Common Usage Patterns

1. Quick file preview:

   ```
   head -20 logfile.log
   ```

2. CSV header inspection:

   ```
   head -1 data.csv
   ```

3. Combined with tail:

   ```
   head -50 file.txt | tail -10
   ```

Advanced Usage

1. Suppress filename headers:

   ```
   head -q file1.txt file2.txt
   ```

2. Always show headers:

   ```
   head -v file.txt
   ```

3. Process substitution:

   ```
   head -5 <(command)
   ```

Performance Analysis

- Very fast for small line counts
- Efficient streaming operation
- Minimal memory usage
- Good for large files
- Stops reading after required lines

Related Commands

- `tail` - Show last lines
- `more` - Page through files
- `less` - Advanced pager
- `cat` - Display entire file
- `sed` - Stream editor

Best Practices

1. Use appropriate line counts
2. Combine with other text tools
3. Consider byte vs line counting
4. Use for quick file validation
5. Helpful for debugging scripts

Integration Examples

1. Log analysis:

```
head -100 /var/log/syslog | grep error
```

2. Data sampling:

```
head -1000 large_dataset.csv > sample.csv
```

3. Script debugging:

```
head -5 "$input_file" | while read line; do
    echo "Processing: $line"
done
```

Scripting Applications

1. File validation:

```
if head -1 "$file" | grep -q "^#"; then
    echo "File has header"
fi
```

2. Quick content check:

```
head -n 1 *.conf | grep -v "^#"
```

Additional Examples

```
head file.txt
head -n 50 file.txt
head -n -20 file.txt         # all but last 20 (GNU)
head -c 1K binary.dat
```

tail

Overview

The `tail` command displays the last lines of files or input streams. It's essential for monitoring log files and examining file endings.

Syntax

```
tail [options] [file...]
```

Common Options

Option	Description
`-n num`	Show last num lines
`-c num`	Show last num bytes
`-f`	Follow file changes
`-F`	Follow with retry
`-q`	Suppress headers
`-v`	Always show headers
`-s num`	Sleep seconds between checks
`--pid=pid`	Terminate after process dies
`--retry`	Keep trying to open file

Key Use Cases

1. Monitor log files
2. View file endings
3. Real-time file watching
4. Debug running processes
5. System monitoring

Examples with Explanations

Example 1: Default Usage

```
tail file.txt
```

Shows last 10 lines of file

Example 2: Follow Log File

```
tail -f /var/log/syslog
```

Continuously monitors log file for new entries

Example 3: Specific Line Count

```
tail -n 20 error.log
```

Shows last 20 lines

Example 4: Multiple Files

```
tail -f /var/log/*.log
```

Follows multiple log files simultaneously

Follow Mode Options

Option	Behavior
`-f`	Follow by file descriptor
`-F`	Follow by name (handles rotation)
`--retry`	Keep trying if file doesn't exist
`--pid=pid`	Stop when process dies

Common Usage Patterns

1. Monitor application logs:

```
tail -f /var/log/apache2/error.log
```

2. Watch system logs:

```
tail -f /var/log/messages
```

3. Debug scripts:

```
tail -f script.log &
./script.sh
```

Advanced Usage

1. Follow with retry:

```
tail -F /var/log/app.log
```

2. Stop after process ends:

```
tail -f --pid=$PID logfile.log
```

3. Custom sleep interval:

```
tail -f -s 0.1 fast_changing.log
```

Performance Analysis

- Efficient for file monitoring
- Low CPU usage in follow mode
- Handles log rotation well with -F
- Good for real-time monitoring
- Minimal memory footprint

Related Commands

- `head` - Show first lines
- `less` - Advanced pager
- `watch` - Execute commands periodically
- `journalctl` - Systemd log viewer

- `multitail` - Monitor multiple files

Best Practices

1. Use -F for log files that rotate
2. Combine with grep for filtering
3. Use –pid for temporary monitoring
4. Consider multitail for multiple files
5. Be aware of file descriptor limits

Integration Examples

1. Filtered monitoring:

```
tail -f /var/log/syslog | grep ERROR
```

2. Multiple log analysis:

```
tail -f /var/log/{messages,secure,maillog}
```

3. Application debugging:

```
tail -f app.log | while read line; do
    echo "$(date): $line"
done
```

Log Rotation Handling

1. Follow by name (handles rotation):

```
tail -F /var/log/app.log
```

2. Retry if file disappears:

```
tail -f --retry /var/log/app.log
```

Scripting Applications

1. Wait for log entry:

```
tail -f app.log | grep -q "Server started" && echo "Ready"
```

2. Monitor until condition:

```
timeout 60 tail -f deploy.log | grep -q "Deployment complete"
```

System Monitoring

1. Watch system load:

```
tail -f /proc/loadavg
```

2. Monitor memory:

```
watch -n 1 'tail -5 /proc/meminfo'
```

Additional Examples

```
tail file.txt
tail -n 100 file.txt
tail -f /var/log/syslog      # follow
tail -F app.log              # follow with rename reopen
journalctl -u nginx -f       # often better for units
```

wc

Overview

The `wc` (word count) command counts lines, words, characters, and bytes in files or input streams. It's essential for text analysis and file statistics.

Syntax

```
wc [options] [file...]
```

Common Options

Option	Description
`-l`	Count lines only
`-w`	Count words only
`-c`	Count bytes only
`-m`	Count characters only
`-L`	Length of longest line
`--files0-from=file`	Read null-separated filenames

Default Output Format

Without options, `wc` shows:

```
lines words bytes filename
```

Example output:

```
  42  156  892 file.txt
```

Key Use Cases

1. Count lines in files
2. Analyze text statistics
3. Monitor file growth
4. Validate data processing
5. Script automation

Examples with Explanations

Example 1: Basic Count

```
wc file.txt
```

Shows lines, words, and bytes count

Example 2: Lines Only

```
wc -l file.txt
```

Shows only line count

Example 3: Multiple Files

```
wc *.txt
```

Shows counts for all text files plus totals

Understanding Counts

- **Lines**: Number of newline characters
- **Words**: Sequences of non-whitespace characters
- **Characters**: Including multibyte characters
- **Bytes**: Raw byte count (may differ from characters)

Common Usage Patterns

1. Count log entries:

```
wc -l /var/log/syslog
```

2. Monitor file growth:

```
watch "wc -l growing_file.log"
```

3. Pipeline counting:

```
ps aux | wc -l
```

Advanced Usage

1. Longest line length:

```
wc -L file.txt
```

2. Character vs byte count:

```
wc -m file.txt  # characters
wc -c file.txt  # bytes
```

3. Multiple file totals:

```
wc -l *.log
```

Pipeline Integration

1. Count command output:

```
ls | wc -l
```

2. Count unique lines:

```
sort file.txt | uniq | wc -l
```

3. Count pattern matches:

```
grep "error" log.txt | wc -l
```

Performance Analysis

- Very fast operation
- Efficient for large files
- Minimal memory usage
- Good pipeline performance
- Streaming capability

Related Commands

- `nl` - Number lines
- `sort` - Sort lines
- `uniq` - Remove duplicates
- `cut` - Extract fields
- `awk` - Text processing

Additional Resources

- GNU wc manual
- Word Count Examples

Best Practices

1. Use specific options for clarity
2. Combine with other text tools
3. Consider character encoding
4. Use in scripts for validation
5. Monitor with watch for real-time updates

Scripting Examples

1. File size validation:

```
if [ $(wc -l < file.txt) -gt 1000 ]; then
    echo "File too large"
fi
```

2. Progress monitoring:

```
TOTAL=$(wc -l < input.txt)
echo "Processing $TOTAL lines"
```

3. Log rotation trigger:

```
[ $(wc -l < logfile) -gt 10000 ] && logrotate config
```

Character Encoding

Difference between `-c` and `-m`: - `-c` counts bytes - `-m` counts characters (important for UTF-8)

Example with Unicode:

```
echo "café" | wc -c  # 5 bytes
echo "café" | wc -m  # 4 characters
```

Common Patterns

1. Count non-empty lines:

```
grep -c "." file.txt
```

2. Count files in directory:

```
ls -1 | wc -l
```

3. Count unique users:

```
cut -d: -f1 /etc/passwd | wc -l
```

Integration Examples

1. With find:

```
find . -name "*.py" -exec wc -l {} + | tail -1
```

2. With xargs:

```
find . -name "*.txt" | xargs wc -l
```

3. Log analysis:

```
tail -f access.log | while read line; do
    echo "Total requests: $(wc -l < access.log)"
done
```

Troubleshooting

1. Binary files giving unexpected results
2. Character encoding issues
3. Very large files
4. Empty files
5. Permission problems

Real-world Applications

1. Code metrics:

```
find . -name "*.py" | xargs wc -l | tail -1
```

2. Data validation:

```
[ $(wc -l < data.csv) -eq $(wc -l < expected.csv) ]
```

3. Monitoring:

```
wc -l /var/log/messages | awk '{print $1}' > line_count.txt
```

tee

Overview

`tee` reads stdin and writes it to both standard output **and** one or more files. It is the standard way to keep a pipeline's live output while saving a copy for logs, audits, or later processing. Named after a T-splitter in plumbing.

Syntax

```
command | tee [OPTIONS] [FILE...]
command | tee [OPTIONS] FILE | other-command
```

With no files, `tee` only copies stdin to stdout (still useful with options like `-a` rarely alone).

Common Options

Option	Description
`-a` / `--append`	Append to files instead of overwriting
`-i` / `--ignore-interrupts`	Ignore SIGINT (less common in scripts)
`-p`	Diagnose errors writing to non-pipes (GNU; behavior with SIGPIPE)

Safety

- `tee file` **truncates** `file` by default when the pipeline starts — same class of footgun as `> file`. Use `-a` to append.
- Writing into system paths needs appropriate privileges (`sudo tee` pattern below).
- Do not `tee` secrets into world-readable locations; check umask and directory permissions.

Examples with Explanations

Save output and still see it

```
ping -c 4 1.1.1.1 | tee ping.log
```

Append to a running log

```
./deploy.sh 2>&1 | tee -a deploy.log
```

`2>&1` captures stderr too so the log is complete.

Split to multiple files

```
./build.sh 2>&1 | tee build.log build-copy.log
```

Continue the pipeline after saving

```
dmesg | tee dmesg.full | grep -i error
```

Full capture in `dmesg.full`; only errors scroll on the terminal.

Write files that need root (classic sudo tee)

```
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-forward.conf
```

Why not `sudo echo … > file`? The redirect is applied by the **local shell** before `sudo`, so the file open happens as you, not root. `sudo tee` opens the file as root.

Same pattern: append as root

```
echo 'export PATH="/opt/bin:$PATH"' | sudo tee -a /etc/environment
```

(Validate distro conventions before editing `/etc/environment`.)

Suppress terminal output but keep the file

```
make 2>&1 | tee build.log >/dev/null
# or simply:
make 2>&1 > build.log
```

`tee` is only needed when you still want a branch (multiple files, or later pipe stages).

Capture and process with jq later

```
curl -sS https://api.github.com/repos/jqlang/jq \
  | tee repo.json \
  | jq -r '.stargazers_count'
```

Timestamped session log with a second process

```
./long-job.sh 2>&1 | tee -a "job-$(date +%F).log"
```

Combine with process substitution (bash) for dual processing

```
./app 2>&1 | tee >(logger -t myapp) | grep -i error
```

Sends a copy to syslog via `logger` while grepping errors on the terminal (bash/zsh).

Preserve exit status of the left side (bash pipefail)

```
set -o pipefail
make 2>&1 | tee build.log
```

Without `pipefail`, the pipeline's exit status is `tee`'s (usually 0) even if `make` failed.

Understanding Output

Whatever arrives on stdin is duplicated to each listed file and to stdout, byte-for-byte (text or binary). Order is preserved. If a file cannot be opened, GNU tee reports an error and still attempts other outputs; check exit status in scripts.

Notes & Pitfalls

- **Truncation default:** `tee log` overwrites; `tee -a log` appends.
- **Exit codes:** with default bash pipes, failures on the left can be masked — enable `set -o pipefail` in scripts.
- Buffering: when stdout is a pipe, some programs fully buffer; you may see delayed terminal output. Tools like `stdbuf -oL` can force line buffering when needed.
- Binary data works; avoid viewing huge binary tees in the terminal.
- `tee` is not a remote multipath tool — for network fan-out use `socat`, message buses, or log shippers.
- Multiple writers appending to one log without coordination can interleave lines; for concurrent jobs prefer separate logs or a proper logger.

Related Commands

- shell redirects `>` / `>>` — single destination, no T-split
- `script` — record an entire interactive terminal session
- `logger` — send lines to syslog
- `sponge` (moreutils) — soak stdin then write (safe in-place pipeline edits)
- `pv` — progress meter in pipelines (if installed)

Additional Resources

- `man tee`
- GNU coreutils info: `info tee`

xargs

Overview

`xargs` builds and executes command lines from standard input. It bridges tools that emit lists (files, IDs, URLs) to commands that expect argv parameters. Essential in pipelines with `find`, `git`, and text filters — and easy to get wrong with spaces or special characters unless you use NUL-delimited mode.

Syntax

```
command | xargs [options] [utility [args...]]
xargs [options] -a file [utility [args...]]
```

If `utility` is omitted, `xargs` defaults to `/bin/echo`.

Common Options

Option	Description
`-0` / `--null`	Input items are NUL-separated (safe for arbitrary filenames)
`-n N`	At most N args per command invocation
`-n 1`	One arg per invocation
`-P N`	Run up to N processes in parallel
`-I {}`	Replace `{}` in the command template with each item
`-i`	Like `-I {}` (deprecated form on some systems)
`-t`	Print the command before running it
`-p`	Prompt before each command
`-r`	Do not run if stdin is empty (GNU)
`-d delim`	Input delimiter (GNU)
`-L N`	Use at most N lines per command
`-a file`	Read items from file instead of stdin
`-x`	Exit if a single item is too long for the command line

Safety

- **Never** pipe raw `find` output into bare `xargs` when names may contain spaces or newlines. Use `find … -print0 | xargs -0 …`.
- `xargs` splits on whitespace by default — a filename `my file.txt` becomes two arguments.
- Parallel `-P` can overwhelm disks, APIs, or remote hosts; start low.
- Interactive prompts (`-p`) are for humans; do not use in unattended scripts without care.

Examples with Explanations

Basic: echo words as args

```
echo 'one two three' | xargs
# runs: echo one two three
```

Delete files listed by find (NUL-safe)

```
find /tmp -name '*.tmp' -print0 | xargs -0 rm -f
```

Equivalent modern find:

```
find /tmp -name '*.tmp' -delete
```

One invocation per item with placeholder

```
find . -name '*.md' -print0 | xargs -0 -I {} cp {} /backup/docs/
```

`-I {}` forces one item at a time and allows the placeholder anywhere in the command.

Limit batch size

```
cat ids.txt | xargs -n 50 echo
# or feed a real tool:
cat ids.txt | xargs -n 100 my-bulk-api-delete
```

Avoids “argument list too long” and huge single CLI lines.

Parallel compression

```
find . -type f -name '*.log' -print0 \
  | xargs -0 -P 4 -n 1 gzip
```

Four `gzip` workers, one file each.

Show commands (`-t`) while learning

```
printf 'a\nb\nc\n' | xargs -n 1 -t rm -f
```

Skip empty input (GNU)

```
grep -rl 'obsolete' . | xargs -r rm
# better NUL-safe:
grep -rlZ 'obsolete' . | xargs -0 -r rm
```

Without `-r`, some xargs still run the command once with no args — dangerous for `rm`.

Lines as items (GNU `-d`)

```
xargs -d '\n' -n 1 echo < urls.txt
```

Still fails if lines contain embedded newlines (rare); for filenames prefer NUL.

Placeholders with multiple uses

```
cat hosts.txt | xargs -I% sh -c 'ssh % "hostname; uptime"'
```

Quote carefully; prefer a real loop for complex remote commands.

From a file directly

```
xargs -a packages.txt -n 1 sudo apt-get install -y
```

(Still review package lists before mass installs.)

Combine with git

```
git ls-files -z '*.png' | xargs -0 ls -l
git grep -z -l 'TODO' | xargs -0 "$EDITOR"
```

Understanding Output

`xargs` itself is quiet unless `-t`/`-p` is set; you see the **utility's** stdout/stderr. Exit status is non-zero if any invocation fails (details vary; with `-P`, failures can interleave). Use `-t` when a pipeline "does nothing" — often the built command is not what you expected.

Notes & Pitfalls

- **NUL safety is the professional default** for file paths: `-print0` / `-z` / `-Z` upstream + `xargs -0`.
- `-I {}` implies `-n 1` and disables some batching; good for clarity, slower for huge sets.
- Quoting inside `sh -c` with xargs is a common injection footgun — validate input or avoid `sh -c` when possible.
- "Argument list too long" (`E2BIG`) is exactly what batching with `-n` fixes.
- Busybox or non-GNU xargs may lack `-0`, `-r`, `-a`, `-d`, or `-P`. On Ubuntu GNU findutils xargs, these are available.
- For complex per-file logic, a `while IFS= read -r -d '' f; do …; done` loop is often clearer than nested xargs.

Related Commands

- `find -exec … {} +` — batch without a separate xargs
- `parallel` (GNU) — richer job control if installed
- `xargs -0` + `grep -Z` / `git -z` — NUL-safe ecosystems
- `printf` / `tr '\n' '\0'` — convert delimiters carefully
- shell `while read` loops — when you need full shell features

Additional Resources

- `man xargs`
- `man find` (especially `-print0`, `-exec`)

diff

Overview

The `diff` command compares files line by line and displays the differences. It's essential for version control, code review, and file comparison tasks.

Syntax

```
diff [options] file1 file2
diff [options] directory1 directory2
```

Common Options

Option	Description
`-u`	Unified diff format
`-c`	Context diff format
`-i`	Ignore case differences
`-w`	Ignore whitespace
`-b`	Ignore changes in whitespace
`-B`	Ignore blank lines
`-r`	Recursive directory comparison
`-q`	Brief output (only if files differ)
`-s`	Report identical files
`-y`	Side-by-side comparison
`--color`	Colorize output

Output Formats

Format	Description
Normal	Default format with line numbers
Unified (-u)	Git-style format
Context (-c)	Shows context around changes
Side-by-side (-y)	Two-column comparison

Format	Description

Key Use Cases

1. Compare file versions
2. Code review
3. Configuration changes
4. Backup verification
5. Patch creation

Examples with Explanations

Example 1: Basic Comparison

```
diff file1.txt file2.txt
```

Shows differences between two files

Example 2: Unified Format

```
diff -u original.txt modified.txt
```

Shows differences in unified format (like Git)

Example 3: Directory Comparison

```
diff -r dir1/ dir2/
```

Recursively compares two directories

Understanding Output

Normal format symbols: - `a` - Added lines - `d` - Deleted lines - `c` - Changed lines - `<` - Lines from first file - `>` - Lines from second file

Example:

```
2c2
< old line
---
> new line
```

Common Usage Patterns

1. Ignore whitespace:

```
diff -w file1.txt file2.txt
```

2. Side-by-side view:

```
diff -y file1.txt file2.txt
```

3. Quick check if files differ:

```
diff -q file1.txt file2.txt
```

Advanced Options

Option	Description
`--exclude=pattern`	Exclude files matching pattern
`--exclude-from=file`	Exclude patterns from file
`-x pattern`	Exclude files matching pattern
`-N`	Treat absent files as empty
`-a`	Treat all files as text
`--strip-trailing-cr`	Strip carriage returns

Patch Creation

Create patches for later application:

```
diff -u original.txt modified.txt > changes.patch
```

Apply patches:

```
patch original.txt < changes.patch
```

Performance Analysis

- Efficient for text files
- Memory usage scales with file size
- Good for moderate-sized files
- Consider alternatives for binary files
- Fast for small differences

Related Commands

- `cmp` - Compare files byte by byte
- `comm` - Compare sorted files
- `patch` - Apply diff patches
- `git diff` - Git version comparison
- `vimdiff` - Visual diff editor

Additional Resources

- GNU diff manual
- Diff Command Guide

Best Practices

1. Use unified format for patches
2. Ignore irrelevant whitespace
3. Use recursive mode for directories
4. Consider binary file handling
5. Use with version control systems

Directory Comparison

1. Compare structures:

   ```
   diff -r --brief dir1/ dir2/
   ```

2. Exclude files:

   ```
   diff -r --exclude="*.log" dir1/ dir2/
   ```

3. Show only differences:

   ```
   diff -r -q dir1/ dir2/
   ```

Integration Examples

1. With git:

   ```
   git diff > changes.patch
   ```

2. With find:

```
diff <(find dir1 -type f | sort) <(find dir2 -type f | sort)
```

3. Configuration management:

```
diff -u /etc/config.orig /etc/config
```

Scripting Applications

1. Backup verification:

```
if diff -q original.txt backup.txt > /dev/null; then
    echo "Backup is identical"
fi
```

2. Configuration monitoring:

```
diff /etc/passwd /etc/passwd.bak || echo "Password file changed"
```

3. Automated testing:

```
diff expected_output.txt actual_output.txt || exit 1
```

Special Cases

1. Binary files:

```
diff -q binary1 binary2
```

2. Large files:

```
diff --speed-large-files file1 file2
```

3. Case-insensitive:

```
diff -i file1.txt file2.txt
```

Troubleshooting

1. Binary file warnings
2. Memory issues with large files
3. Character encoding problems

4. Permission denied errors
5. Directory structure differences

Output Redirection

1. Save differences:

```
diff file1.txt file2.txt > differences.txt
```

2. Suppress output:

```
diff file1.txt file2.txt > /dev/null
```

3. Error handling:

```
diff file1.txt file2.txt 2>&1 | tee diff.log
```

Color Output

Modern diff versions support color:

```
diff --color=always file1.txt file2.txt
```

Environment variable:

```
export DIFF_COLORS="old=31:new=32:hunk=36"
```

comm

Overview

`comm` compares **two sorted files** line by line and outputs three columns: lines only in file1, only in file2, and common to both. Perfect for set operations on sorted word lists, user lists, and package inventories. Unsorted input produces garbage — sort first.

Syntax

```
comm [options] file1 file2
```

Common Options

Option	Description
`-1`	Suppress column 1 (unique to file1)
`-2`	Suppress column 2 (unique to file2)
`-3`	Suppress column 3 (common)
`-z`	NUL-terminated lines
`--check-order`	Check that inputs are sorted
`--nocheck-order`	Skip order check
`--output-delimiter=STR`	Column separator
`--total`	Output summary counts (newer)

Examples with Explanations

Basic three columns

```
sort a.txt -o a.s
sort b.txt -o b.s
comm a.s b.s
```

Only in first / second / both

```
comm -23 a.s b.s     # only in a
comm -13 a.s b.s     # only in b
comm -12 a.s b.s     # common
```

Mnemonic: suppress the columns you don't want (`-23` keeps col1).

Users differences

```
getent passwd | cut -d: -f1 | sort > host_users
# compare to expected list
comm -23 expected_users host_users    # expected but missing
comm -13 expected_users host_users    # unexpected present
```

Package set ops

```
rpm -qa | sort > now
comm -13 baseline now    # newly installed vs baseline
```

Check order

```
comm --check-order a.txt b.txt
```

Process substitution

```
comm -12 <(sort list1) <(sort list2)
```

Empty markers

Columns are tab-separated; empty leading fields mean a line isn't in earlier columns. Use `cat -A` to see tabs:

```
comm a.s b.s | cat -A
```

Notes / Pitfalls

- **Both files must be sorted** in the same collation (`LC_ALL=C sort` for stable byte order).
- Locale sort order can surprise — set `LC_ALL=C` for machine lists.
- Duplicate lines: `comm` is multiset-aware in the sense of sorted runs; unique with `sort -u` if needed.
- Binary files / CRLF: normalize first (`dos2unix`).
- For unsorted fuzzy joins, use `join`, `grep -F -f`, or databases.

2026-relevant notes

- Still excellent for offline set diffs without loading Python.
- Large inventories: ensure sort can use tmp space (`TMPDIR`).
- Combine with `rg --files` lists for workspace membership diffs.

Related Commands

- `sort` / `uniq` — prepare inputs
- `join` — join on fields
- `diff` / `git diff` — line-oriented diffs with context
- `grep -F -f` — filter by list
- `cmp` — binary compare

Additional Resources

- `man comm`

paste

Overview

`paste` merges lines of files **side by side**, writing corresponding lines separated by tabs (or another delimiter). The counterpart mindset to `cut` (split columns) — paste builds columns from separate files or serializes lines with `-s`.

Syntax

```
paste [options] [file...]
```

Common Options

Option	Description
`-d LIST`, `--delimiters=LIST`	Cycle delimiter characters instead of tab
`-s`, `--serial`	Paste one file at a time (serialize lines of each file)
`-z`, `--zero-terminated`	NUL-terminated lines

Use `-` for stdin.

Examples with Explanations

Side by side

```
paste names.txt ages.txt
paste <(echo -e 'a\nb') <(echo -e '1\n2')
```

Custom delimiter

```
paste -d, names.txt ages.txt
paste -d'|' a.txt b.txt c.txt
paste -d '\t|' a.txt b.txt     # cycle delimiters
```

Serialize lines of one file

```
# join all lines with comma
paste -sd, file.txt
# words to CSV row
paste -sd' ' words.txt
```

Combine with cut/seq

```
paste <(seq 1 3) <(seq 10 12)
cut -d: -f1 /etc/passwd | head | paste - - -    # 3 columns
```

Join data columns from commands

```
paste <(nproc; echo cores) <(free -h | awk '/Mem:/{print $2}')
```

NUL-safe

```
paste -z -d '' file1 file2
```

Classic: make CSV from columns

```
paste -d, col1.txt col2.txt col3.txt > out.csv
```

Notes / Pitfalls

- Unequal line counts: shorter files yield empty fields for missing lines.
- Default tab delimiter can be hard to see — use `-d,` or `cat -A`.
- `-s` changes semantics dramatically (horizontal merge of each file's own lines).
- Not a full CSV writer (no quoting of embedded commas) — use proper CSV tools when needed.
- Large files stream line-by-line; keep inputs aligned intentionally.

2026-relevant notes

- Still great for quick ops glue between two tool outputs.
- For structured data, prefer `jq -s` / `python` when quoting/escaping matters.
- Pair with `column -t` after paste for display.

Related Commands

- `cut` — extract columns
- `join` — merge sorted files on a key
- `awk` — general field processing
- `column` — pretty alignment
- `pr -m` — merge files in columns (different tool)

Additional Resources

- `man paste`

column

Overview

`column` formats text into aligned columns or a table. Ideal for making command output or colon-separated files readable. Commonly used with `mount`, `fstab`-like data, and custom script output. From `util-linux`.

Syntax

```
column [options] [file...]
```

Common Options

Option	Description
`-t`, `--table`	Create a table; determine columns automatically
`-s sep`, `--separator`	Input separator characters
`-o sep`, `--output-separator`	Output separator (default two spaces)
`-N names`	Column names header (newer)
`-R cols`	Right-align columns
`-J`	JSON output (newer util-linux)
`-x`	Fill columns before rows
`-c width`	Output width
`-n`	Disable “merge empty columns” behavior variants
`-l num`	Max columns keep (see man)

Examples with Explanations

Table-ize whitespace

```
mount | column -t
df -h | column -t
ps -eo pid,user,comm,%cpu --sort=-%cpu | head | column -t
```

Custom separators

```
column -t -s: /etc/passwd | less
echo -e 'a|b|c\n1|2|3' | column -t -s'|'
```

Headers (newer)

```
printf 'user:uid:shell\nalice:1000:/bin/bash\n' | column -t -s: -N USER,UID,SHELL
```

Right-align numbers

```
printf 'name val\na 10\nbb 3\n' | column -t -R 2
```

JSON (when available)

```
mount | column -t -J | jq .
```

Pretty fstab view

```
grep -vE '^\s*(#|$)' /etc/fstab | column -t
```

Pipelines

```
systemctl list-units --type=service --state=running | column -t
```

Notes / Pitfalls

- Input with inconsistent field counts produces jagged tables — clean data first.
- Wide Unicode can misalign in some terminals.
- Default without `-t` wraps differently (classic column mode) — for tables always use `-t`.
- Features like `-N`/`-J` need sufficiently new util-linux.
- Don't use as a CSV parser for complex quoting — use `csvkit`/`python` for real CSV.

2026-relevant notes

- Still the fastest path to readable admin tables over SSH.
- JSON mode helps bridge to `jq` tooling when available.
- For TUI tables, tools like `procs`/`eza` already format themselves.

Related Commands

- `awk` / `cut` / `paste` — field extraction/combine
- `rs` — reshape (BSD/some systems)
- `jq -C` / `column` — JSON vs text
- `tabs` / `expand` — tab handling
- `printf` — manual formatting

Additional Resources

- `man column`

jq

Overview

`jq` is a command-line JSON processor. It slices, filters, maps, and transforms structured data the way `sed`/`awk` do for text lines. Prefer it for APIs, `package.json`, cloud CLIs, and any tool that emits JSON.

```
sudo apt install jq
```

Syntax

```
jq [options] '<filter>' [file...]
… | jq [options] '<filter>'
```

Common Options

Option	Description
`.`	Identity — pretty-print JSON
`-r`	Raw string output (no JSON quotes)
`-c`	Compact (one value per line)
`-s`	Slurp: all inputs into one array
`-n`	Null input (build JSON from filter only)
`-e`	Exit non-zero if final result is `false` or `null`
`--arg name value`	Bind string variable `$name`
`--argjson name json`	Bind JSON-valued `$name`
`-S`	Sort object keys
`--stream`	Stream parse huge inputs (advanced)

Key Use Cases

1. Pretty-print API responses

2. Extract fields for shell scripts
3. Filter and map arrays of objects
4. Build JSON safely from shell variables
5. CI checks that required keys exist

Examples with Explanations

Pretty-print stdin

```
curl -sS https://api.github.com/repos/jqlang/jq | jq .
```

Readable indentation for exploration. `-sS` shows curl errors without progress meter.

Extract a field

```
jq -r .name package.json
jq -r '.networkSettings.IPAddress // empty' inspect.json
```

`-r` drops quotes so the value is shell-friendly. `// empty` suppresses nulls.

Map a list of objects to TSV

```
jq -r '.items[] | [.id, .name, .status] | @tsv' data.json
```

TSV pipes cleanly into `sort`, `column -t`, or spreadsheets.

Filter array elements

```
jq '.[] | select(.status=="active" and .role=="admin")' users.json
jq '[.[] | select(.cpu > 80)] | length' metrics.json
```

`select` keeps matching objects; wrap in `[]` to recollect into an array.

Nested paths and optional keys

```
jq -r '.spec.containers[0].image // "missing"' pod.json
jq 'has("error")' response.json
```

Use explicit paths; `has` is useful for boolean checks with `-e`.

Update / reshape objects

```
jq '.port = 8080 | .debug = false' config.json
jq '{name, version, scripts}' package.json
```

Filters are expressions: assignment returns the modified object; object construction picks fields.

Flatten and walk arrays

```
jq -r '.. | objects | select(has("email")) | .email' big.json
jq -r '.[] | .tags[]?' items.json
```

`?` avoids errors on missing paths; `..` is recursive descent (powerful, easy to overuse).

Build JSON from shell (safe)

```
jq -n \
  --arg user "$USER" \
  --arg host "$(hostname -f 2>/dev/null || hostname)" \
  --argjson n "$(nproc)" \
  '{user:$user, host:$host, cpus:$n, ts: now|todateiso8601}'
```

Always prefer `--arg` / `--argjson` over interpolating quotes by hand.

Merge two JSON files

```
jq -s '.[0] * .[1]' base.json overlay.json
```

`-s` slurps files into an array; `*` deep-merges objects (right wins on conflicts).

Exit status for CI

```
jq -e '.version != null' package.json >/dev/null
jq -e '.[] | select(.severity=="critical") | halt_error(1)' findings.json
```

`-e` fails on `false`/`null` final output; `halt_error` aborts mid-stream (jq 1.6+).

Compact logs / NDJSON

```
jq -c '.[]' array.json              # one object per line
jq -c -r 'select(.level=="error")' app.ndjson
```

Newline-delimited JSON is ideal for `tail -f` pipelines.

Understanding Output

`jq` prints the filter result as JSON by default. Multiple results print one after another. Errors go to stderr. Parse failures are non-zero. With `-r`, strings lose JSON quoting; numbers/bools still print in a shell-friendly form.

Notes & Pitfalls

- Filters are **programs** — learn `.`, `[]`, `|`, `select`, `map`, `keys`, `has`, `//`.
- Never build JSON with string concatenation when secrets or user input are involved.
- Huge single JSON documents may need `--stream` or preprocessing.
- YAML is not JSON; use `yq` (or convert) for YAML.
- Unicode and locale: raw bytes in invalid UTF-8 can make `jq` fail.

Related Commands

- `curl` — often the JSON source
- `grep` / `sed` / `awk` — line-oriented text
- `python3 -m json.tool` — pretty-print only
- `yq` — YAML-oriented cousin (if installed)

Additional Resources

- jq manual
- jq play

pv

Overview

`pv` (Pipe Viewer) shows a **progress bar, throughput, and ETA** for data moving through a pipeline. Perfect for long `dd`, compression, or network transfers where tools lack built-in progress.

```
sudo apt install pv
```

Syntax

```
pv [options] [file...]
… | pv | …
```

Common Options

Option	Description
`-s size`	Expected size (better ETA)
`-a`	Average rate
`-r`	Current rate
`-t`	Timer
`-b`	Bytes transferred
`-p`	Progress bar
`-N name`	Label
`-c`	Cursor progress for multi-pv
`-L rate`	Rate limit

Examples with Explanations

Copy with progress

```
pv big.iso > /mnt/usb/big.iso
pv big.iso | gzip > big.iso.gz
```

dd-style disk image

```
sudo dd if=/dev/sda bs=4M status=none | pv -s "$(blockdev --getsize64 /dev/sda)" | \
  sudo dd of=/dev/sdb bs=4M
```

(Or use `dd status=progress` on modern coreutils.)

Rate-limit a transfer

```
pv -L 10M archive.tar | ssh host 'cat > archive.tar'
```

Multi-stage labeled bars

```
pv -cN in file.tar | gzip | pv -cN gz > file.tar.gz
```

Related Commands

- `dd` — low-level copy
- `rsync --progress` — file sync progress
- `wget`/`curl` — HTTP progress
- `progress` / `cv` — attach to running coreutils (if installed)

Additional Resources

- `man pv`

Packages

Intro

Intro

Install and maintain software with distro package managers used on servers. Ubuntu/Debian: `apt`/`dpkg`; Fedora/RHEL: `dnf`/`rpm` (and legacy `yum`); optional Ubuntu `snap` for selected server tools. Desktop app stores (Flatpak, GUI software centers) are out of scope.

Commands in this part

Command	Role
`apt`	apt is the friendly package manager front-end on Debian and Ubuntu.
`dpkg`	The dpkg (Debian Package) command is a low-level package manager for Debian-based systems.
`dnf`	dnf (Dandified YUM) is the primary high-level package manager on Fedora, RHEL 8+, CentOS Stream, Rocky, AlmaLinux,…
`rpm`	The rpm (RPM Package Manager) command is a low-level package manager for RPM-based Linux distributions.
`yum`	yum (Yellowdog Updater Modified) is the classic RPM package manager front-end for older RHEL/CentOS/Scientific Linux…
`snap`	snap installs Snap packages from the Snap Store.

Suggested starting points

1. Debian/Ubuntu day-to-day: `apt` (low-level: `dpkg`).
2. Fedora/RHEL: `dnf` (low-level: `rpm`; legacy: `yum`).
3. Optional on Ubuntu: `snap` only when a server tool requires snapd.

Related parts

- Services and runtime — restart units after library upgrades
- Security — trust and third-party repos
- Files and paths — where binaries land (`which`, `dpkg -L`)

Continue with the individual command pages in this part.

apt

Overview

`apt` is the friendly package manager front-end on Debian and Ubuntu. It installs, updates, searches, and removes `.deb` packages with dependency resolution. Prefer `apt` interactively; keep `apt-get`/`apt-cache` for scripts and older documentation. Fedora/RHEL use `dnf` instead.

Syntax

```
apt [options] command
```

Common Commands

Command	Description
`update`	Refresh package indexes from mirrors
`upgrade`	Install newer versions (conservative)
`full-upgrade`	Upgrade allowing removals to satisfy deps
`install PKG…`	Install or upgrade packages
`reinstall PKG`	Reinstall current version
`remove` / `purge`	Remove (purge drops conffiles)
`autoremove`	Drop unused automatic deps
`search` / `show`	Discover packages
`list --installed` / `--upgradable`	Inventory
`policy PKG`	Candidate versions / pinning
`depends` / `rdepends`	Dependency graph peek
`edit-sources`	Open sources list
`history`	Recent transactions (newer apt)
`changelog PKG`	Show changelog

Key Use Cases

1. Install software on Ubuntu/Debian

2. Apply security updates
3. Inspect why a package is present
4. Clean leftover dependencies after removals

Safety

- Always `update` before install/upgrade when online.
- Read the transaction summary (packages removed!) before confirming.
- Random PPAs complicate upgrades — pin trust carefully.
- Prefer staged upgrades on production (`apt-get` with maintenance windows).
- Do not kill `dpkg`/`apt` mid-transaction without recovery plan.

Examples with Explanations

Refresh and upgrade

```
sudo apt update
sudo apt upgrade
sudo apt full-upgrade      # may remove packages to satisfy deps
apt list --upgradable
```

Install / remove

```
sudo apt install curl jq htop
sudo apt remove htop
sudo apt purge htop        # also remove /etc config files owned by the pkg
sudo apt autoremove --purge
```

Search and inspect

```
apt search '^nginx'
apt show nginx
apt policy nginx
apt depends nginx
apt rdepends openssl | head
```

Version pinning at install

```
apt policy nginx
sudo apt install nginx=1.24.*
apt-mark hold nginx
apt-mark unhold nginx
apt-mark showhold
```

Fix broken dependencies

```
sudo dpkg --configure -a
sudo apt --fix-broken install
```

Classic recovery after a failed upgrade or full disk mid-install.

Clean caches

```
sudo apt clean              # wipe package caches
sudo apt autoclean          # only obsolete debs
du -sh /var/cache/apt/archives
```

Noninteractive (scripts/CI)

```
export DEBIAN_FRONTEND=noninteractive
sudo apt-get -y update
sudo apt-get -y -o Dpkg::Options::="--force-confdef" \
  -o Dpkg::Options::="--force-confold" upgrade
sudo apt-get -y install package
```

`apt-get` flags are more stable for automation than interactive `apt`.

Simulate / dry-run

```
apt-get -s install heavy-metapackage
apt --dry-run full-upgrade
```

Which package owns a file?

```
dpkg -S /usr/bin/curl
apt-file search bin/curl     # needs apt-file package + update
```

Notes & Pitfalls

- `apt` CLI is **not** a stable machine API — many CI scripts stick to `apt-get`.
- Snaps (`snap`) are a separate delivery channel on some Ubuntu systems; prefer debs for core server packages.
- Held packages block upgrades silently until unheld.
- Multi-arch (`:i386` / foreign architectures) appears when enabled for cross-arch deps — uncommon on pure amd64 servers.
- Offline mirrors: `apt-offline` or local `file:` repositories.

Related Commands

- `dpkg` — low-level `.deb` operations
- `apt-get` / `apt-cache` — classic tooling
- `snap` — optional snapd packages on Ubuntu
- `dnf` / `rpm` — Fedora/RHEL family
- `needrestart` — restart services after library upgrades (if installed)

Additional Resources

- `man apt` / `man apt-get`
- Ubuntu Server Guide — package management

dpkg

Overview

The `dpkg` (Debian Package) command is a low-level package manager for Debian-based systems. It directly handles .deb package operations without managing dependencies.

Syntax

```
dpkg [options] action
```

Common Actions

Action	Description
`-i`	Install package
`-r`	Remove package
`-P`	Purge package
`-l`	List packages
`-s`	Package status
`-L`	List files
`-S`	Search file owner
`-C`	Check database
`--configure`	Configure package
`--unpack`	Unpack package
`--verify`	Verify package
`--audit`	Audit package

Common Options

Option	Description
`--force-all`	Force operations
`--ignore-depends`	Ignore dependencies
`--no-act`	Simulation mode
`--root=dir`	Alternative root

Option	Description
`--admindir=dir`	Alternative admin dir
`--log=file`	Alternative log file
`--status-fd n`	Send status to fd n
`--print-architecture`	Show architecture

Key Use Cases

1. Package installation
2. Package removal
3. Package queries
4. System verification
5. Package information

Examples with Explanations

Example 1: Install Package

```
dpkg -i package.deb
```

Install .deb package

Example 2: Remove Package

```
dpkg -r package_name
```

Remove package

Example 3: List Files

```
dpkg -L package_name
```

List package files

Common Usage Patterns

1. Package status:

```
dpkg -s package_name
```

2. Find owner:

```
dpkg -S /path/to/file
```

3. List installed:

```
dpkg -l | grep '^ii'
```

Security Considerations

1. Package verification
2. Root privileges
3. System integrity
4. Configuration files
5. Dependencies

Related Commands

- `apt` - High-level manager
- `apt-get` - Package management
- `apt-cache` - Package query
- `dselect` - Package selection
- `dpkg-query` - Package database

Additional Resources

- Dpkg Manual
- Package Management Guide
- System Administration

Best Practices

1. Verify packages
2. Backup configuration
3. Check dependencies
4. Document changes

5. Test installation

Package States

1. Not installed
2. Config-files
3. Half-installed
4. Unpacked
5. Half-configured

Troubleshooting

1. Broken packages
2. Dependencies
3. Configuration errors
4. Space issues
5. Database corruption ## Additional Examples

```
dpkg -l | less
dpkg -L bash                # files owned by package
dpkg -S /bin/ls             # which package owns file
sudo dpkg -i package.deb
sudo apt -f install         # fix deps after dpkg -i
```

dnf

Overview

`dnf` (Dandified YUM) is the primary high-level package manager on **Fedora, RHEL 8+, CentOS Stream, Rocky, AlmaLinux**, and related RPM distributions. This book is Ubuntu-first (`apt`), but operators routinely jump onto RHEL-family hosts — learn `dnf` as the counterpart to `apt`. It installs/removes RPM packages, resolves dependencies, and manages repositories/modules.

On older RHEL/CentOS 7 systems you will still see `yum`; on modern releases `yum` is often a compatibility symlink to `dnf`.

Syntax

```
dnf [options] <command> [<args>...]
```

Common commands

Command	Description
`install PKG…`	Install packages
`remove PKG…`	Remove packages
`reinstall PKG…`	Reinstall
`update` / `upgrade`	Apply updates (wording varies by version; both common)
`check-update`	List available updates without installing
`search KEYWORD`	Search names/descriptions
`info PKG`	Package metadata
`list`	List packages (`--installed`, `--available`, …)
`provides` / `whatprovides`	Which package owns a file/feature
`repoquery`	Advanced query (also `dnf repoquery`)
`repolist`	Enabled repositories
`makecache`	Refresh metadata
`clean all`	Clear caches
`history`	Transaction history / undo
`group install "Group Name"`	Environment/group packages
`module`	Modular streams (AppStream)
`distro-sync`	Sync to repo versions (powerful; careful)

Command	Description
`autoremove`	Remove unneeded deps (when supported)

Common options

Option	Description
`-y`, `--assumeyes`	Assume yes
`-q`	Quiet
`--refresh`	Force metadata refresh
`--enablerepo=ID` / `--disablerepo=ID`	Temporary repo toggles
`--setopt=`	Override config for this run
`--allowerasing`	Allow erasing conflicts to satisfy install
`--best` / `--nobest`	Strictness of version selection
`--security`	Prefer security updates where metadata allows
`--downloadonly`	Fetch without install
`--destdir=DIR`	With downloadonly, place RPMs here

Safety

- Read the **transaction summary** before accepting (package installs/removals).
- Avoid blind `dnf update -y` on production without a window and rollback plan.
- Third-party repos and modular stream switches can pin you into painful upgrades.
- `dnf history undo` is helpful but not magic after major version jumps or manual `rpm -Uvh` chaos.
- Prefer official AppStream modules over random COPR packages for critical runtimes.

Key Use Cases

1. Install and update software on RHEL-family hosts
2. Map a binary path back to an owning package
3. Apply security updates selectively
4. Review and reverse recent transactions

Examples with Explanations

Example: refresh and update

```
sudo dnf check-update
sudo dnf upgrade
# older muscle memory:
sudo dnf update
```

`check-update` is a dry look at pending work. Confirm changelogs for kernel and glibc bumps.

Example: install and remove

```
sudo dnf install htop jq curl
sudo dnf remove htop
```

Dependencies are resolved automatically. Removal may leave weak deps depending on config.

Example: search and inspect

```
dnf search nginx
dnf info nginx
dnf list installed 'nginx*'
```

Non-root search/info works for most queries.

Example: which package owns a file

```
dnf provides /usr/bin/ps
dnf provides '*/bin/htop'
```

Invaluable when a command is missing or two packages conflict.

Example: repository visibility

```
dnf repolist
dnf repolist all
sudo dnf config-manager --set-enabled crb   # name varies by distro
```

Rocky/Alma often need CodeReady/CRB/PowerTools-style repos for extra build deps.

Example: transaction history

```
dnf history
dnf history info 42
sudo dnf history undo 42
```

Undo a mistaken install when history still maps cleanly to reverse operations.

Example: download only

```
sudo dnf download --resolve nginx
# or
sudo dnf install --downloadonly --destdir=/var/tmp/rpms nginx
```

Useful for air-gapped copies and change tickets that require artifact review.

Example: groups and modules

```
dnf group list
sudo dnf group install "Development Tools"
dnf module list nodejs
sudo dnf module enable nodejs:18
sudo dnf module install nodejs:18/common
```

Modules select streams (versions) on RHEL-family AppStream — read stream notes before switching.

Example: security-focused updates

```
sudo dnf update --security
sudo dnf updateinfo list security
```

Depends on repository updateinfo metadata quality.

Example: clean caches

```
sudo dnf clean all
sudo dnf makecache
```

When metadata looks stale or disk under `/var/cache/dnf` is tight.

Example: scripted noninteractive

```
sudo dnf -y install package-name
```

Still log transactions and pin versions when builds must be reproducible.

Notes & Pitfalls

- This page is for **RPM-family** hosts; on Ubuntu use `apt` / `dpkg`.
- `yum` vs `dnf`: prefer `dnf` on modern systems; check `yum --version` if both exist.
- EPEL is common for extra packages — enable deliberately.
- Kernel updates need reboot; live patching may exist separately (kpatch, vendor tools).
- Mixing `rpm -e --nodeps` with dnf creates dependency debt — avoid.
- Proxy/SSL interception breaks metadata fetches; configure `/etc/dnf/dnf.conf` and repo files carefully.

Related Commands

- `rpm` — low-level RPM query/install
- `yum` — legacy / symlink front-end
- `apt` / `dpkg` — Debian/Ubuntu counterparts
- `subscription-manager` — RHEL entitlements
- `needs-restarting` — which services/pids need restart after updates (yum-utils/dnf-utils)

Additional Resources

- `man dnf`
- `man dnf.conf`
- Distribution docs: Fedora DNF guide, RHEL System Administrator's Guide (packaging chapters)

rpm

Overview

The `rpm` (RPM Package Manager) command is a low-level package manager for RPM-based Linux distributions. It handles individual package operations without managing dependencies.

Syntax

```
rpm [options] [package...]
```

Common Options

Option	Description
`-i`	Install package
`-U`	Upgrade package
`-e`	Erase package
`-q`	Query package
`-V`	Verify package
`-F`	Freshen package
`--nodeps`	Ignore dependencies
`--force`	Force operation
`--test`	Test only
`--rebuild`	Rebuild package
`--rebuilddb`	Rebuild database
`--checksig`	Check signature

Query Options

Option	Description
`-qa`	Query all
`-qi`	Package info
`-ql`	List files
`-qf`	File owner

Option	Description
`-qp`	Query package file
`-qR`	Requirements
`-qc`	Config files
`-qd`	Documentation

Key Use Cases

1. Package installation
2. Package queries
3. Package verification
4. Database maintenance
5. System verification

Examples with Explanations

Example 1: Install Package

```
rpm -ivh package.rpm
```

Install with verbose and hash progress

Example 2: Query Package

```
rpm -qi package_name
```

Show package information

Example 3: Verify Package

```
rpm -V package_name
```

Verify package files

Common Usage Patterns

1. List installed:

   ```
   rpm -qa
   ```

2. Find owner:

   ```
   rpm -qf /path/to/file
   ```

3. Show dependencies:

   ```
   rpm -qR package_name
   ```

Security Considerations

1. Package verification
2. GPG signatures
3. Root privileges
4. System integrity
5. Dependencies

Related Commands

- `yum` - High-level manager
- `dnf` - Next-gen manager
- `rpmbuild` - Build packages
- `rpm2cpio` - Convert package
- `rpmsign` - Sign package

Additional Resources

- RPM Manual
- Package Management Guide
- System Administration

Best Practices

1. Verify packages
2. Check signatures
3. Backup database
4. Document changes

5. Test installation

Package Information

1. Version
2. Release
3. Architecture
4. Dependencies
5. Changelog

Troubleshooting

1. Dependencies
2. Database issues
3. Conflicts
4. Space problems
5. Verification errors

yum

Overview

`yum` (Yellowdog Updater Modified) is the classic RPM package manager front-end for older RHEL/CentOS/Scientific Linux systems. On **RHEL 8+/Fedora/CentOS Stream**, the command is typically a compatibility wrapper around `dnf`. Prefer `dnf` on modern systems; learn `yum` for legacy fleets and muscle-memory commands that still work.

Syntax

```
yum [options] command
```

Common Commands

Command	Description
`install`	Install packages
`remove` / `erase`	Remove packages
`update` / `upgrade`	Update packages
`info`	Package details
`list`	List packages
`search`	Search metadata
`provides` / `whatprovides`	Which package owns a capability/file
`repolist`	Enabled repos
`clean all`	Clean caches
`history`	Transaction history
`groupinstall`	Group install
`check-update`	Check without installing

Common Options

Option	Description
`-y`	Assume yes
`-q`	Quiet

Option	Description
`--enablerepo=` / `--disablerepo=`	Toggle repos for one shot
`--setopt=`	Override config
`--downloadonly`	Download, don't install
`--security`	Security updates (plugin/platform dependent)
`-C`	Run from cache only

Examples with Explanations

Everyday

```
sudo yum check-update
sudo yum update -y
sudo yum install -y curl vim
sudo yum remove -y oldpkg
```

Search and info

```
yum search nginx
yum info nginx
yum list installed 'nginx*'
yum provides /usr/bin/ps
```

Repos

```
yum repolist
yum repolist all
sudo yum-config-manager --enable powertools   # platform specific
```

History / undo

```
sudo yum history
sudo yum history info 42
sudo yum history undo 42
```

Clean cache

```
sudo yum clean all
sudo yum makecache
```

Prefer dnf when present

```
command -v dnf && dnf --version
sudo dnf install -y htop      # modern path
# yum may redirect:
yum --version
```

Groups

```
yum group list
sudo yum groupinstall -y "Development Tools"
```

Notes / Pitfalls

- RHEL 8+: `yum` → `dnf` API; some old yum plugins differ.
- `-y` in unattended scripts needs good testing (major upgrades).
- Multiple transactions: watch disk space for `/var/cache`.
- Proxy and mirror issues show as metadata download failures.
- EPEL and third-party repos change availability — pin carefully.

2026-relevant notes

- New work: document **`dnf`** first; mention `yum` as alias/legacy.
- CentOS Linux classic is EOL — care with unmaintained mirrors.
- For containers, use minimal base images and explicit package lists rather than interactive yum sessions.

Related Commands

- `dnf` — modern replacement
- `rpm` — low-level RPM queries/installs
- `repoquery` — rich queries (dnf/yum tools)
- `subscription-manager` — RHEL entitlements
- `apt` — Debian family analog

Additional Resources

- `man yum`, `man dnf`
- RHEL system admin docs for your major version

snap

Overview

`snap` installs **Snap** packages from the Snap Store. On **Ubuntu Server** you mainly meet snaps as optional delivery for server tools (for example some Canonical products, confinement demos, or ops utilities) — not as a desktop app store. Prefer distro packages (`apt` / `dnf`) for core system software; use snaps when a vendor ships only that channel or you want the snap's isolation/refresh model.

Many minimal server images ship without `snapd`; install it only if you need it.

```
sudo apt update
sudo apt install snapd    # Debian/Ubuntu when required
```

Syntax

```
snap [options] command
```

Common Commands

Command	Description
`find query`	Search store
`info name`	Details / channels / confinement
`install name`	Install
`remove name`	Remove
`list`	Installed snaps
`refresh`	Update
`channels name`	Track/channel info
`services`	Services shipped inside snaps
`logs name`	Snap service logs
`connections`	Interface plugs/slots (confinement)

Safety

- Snaps **auto-refresh** by default — on production hosts, understand refresh timing or hold refreshes for critical tools.
- Prefer `apt`/`dnf` for libraries and daemons that integrate tightly with the OS.
- Classic confinement is less sandboxed — review `snap info` before install.
- Disk growth under `/var/lib/snapd` from old revisions — prune intentionally.

Key Use Cases

1. Vendor-distributed server tools published only as snaps
2. Optional Ubuntu server components that use snapd
3. Inspecting what snapd already installed on a host

Examples with Explanations

Inventory and search

```
snap version
snap list
snap find lxd
snap info lxd
```

Use `info` to see tracks, confinement, and whether the snap is maintained.

Install / remove (server-oriented)

```
# Example only - choose packages you actually need
sudo snap install hello-world
snap list
sudo snap remove hello-world
```

Prefer well-known server snaps you operate deliberately; avoid random desktop apps on headless hosts.

Channels / tracks

```
snap info kubectl
# sudo snap install kubectl --classic --channel=1.29/stable
```

`--classic` relaxes confinement (common for developer/ops CLIs). Pin channels on production.

Services and logs

```
snap services
# sudo snap restart <snap>.<service>
snap logs <snap-name>
```

Some snaps bundle systemd-like services managed through snapd.

Hold refreshes (ops)

```
sudo snap refresh --hold
# review: snap refresh --time
# later: sudo snap refresh --unhold
```

Useful in maintenance windows so unattended refreshes do not change binaries mid-incident.

Revisions / disk use

```
snap list --all
du -sh /var/lib/snapd
# remove disabled old revisions when appropriate:
# sudo snap remove <name> --revision=N
```

Notes & Pitfalls

- Headless servers: no GUI plugs; do not install GUI-oriented snaps.
- Mixing snap and apt packages of the same tool can put two binaries on `PATH` — check `which -a`.
- Network-restricted environments need store/proxy access for install and refresh.
- For Fedora/RHEL, this ecosystem is uncommon; use distro packages or containers instead.

Related Commands

- `apt` / `dpkg` — primary Ubuntu/Debian packaging
- `dnf` / `rpm` — Fedora/RHEL packaging
- `systemctl` / `journalctl` — host services and logs
- `podman` — containers as an alternative distribution model

Additional Resources

- `man snap`
- Ubuntu Server documentation on snapd (when using Ubuntu)

Services and Runtime

Intro

Intro

Control the running system under systemd: units, journals, kernel parameters, reboot/shutdown, transient jobs, sessions, modules, and container CLI (`podman`).

Commands in this part

Command	Role
`systemctl`	systemctl is the primary interface to systemd: start/stop units, enable on boot, inspect status, and list dependencies.
`journalctl`	journalctl queries the systemd journal — structured logs from the kernel, services, and many applications.
`sysctl`	sysctl reads and writes kernel parameters exposed under /proc/sys/.
`shutdown`	shutdown schedules a system power-off, halt, or reboot and notifies logged-in users.
`reboot`	reboot restarts the system.
`podman`	podman is a daemonless OCI container engine with a CLI largely compatible with Docker.
`systemd-run`	systemd-run creates transient units on the fly: one-shot commands, background services, or temporary timers without…
`systemd-analyze`	systemd-analyze diagnoses boot performance and unit graphs: how long boot took, which units delayed it, whether unit…
`loginctl`	loginctl manages systemd-logind seats, sessions, and users: who is logged in, idle hints, kill sessions, and…
`modprobe`	modprobe loads and unloads kernel modules, resolving dependencies via modules.dep — preferred over raw insmod/rmmod…

Command	Role
`systemd-creds`	systemd-creds encrypts, decrypts, and inspects service credentials for LoadCredentialEncrypted= / credstore.
`unshare` / `nsenter`	unshare runs a program in new namespaces (user, net, pid, mount, uts, time); nsenter joins the namespaces of a running process.

Suggested starting points

1. Services: `systemctl` status/start/enable; logs: `journalctl`.
2. Kernel tunables: `sysctl`; modules: `modprobe`.
3. Power: `shutdown`/`reboot` (prefer coordinated maintenance).
4. Ad-hoc units/timers: `systemd-run`; boot analysis: `systemd-analyze`.
5. User sessions / linger: `loginctl`.
6. Containers: `podman`.
7. Service secrets: `systemd-creds` + `LoadCredentialEncrypted=`.
8. Namespaces and sandboxes: `unshare` to create, `nsenter` to debug from the host.

Related parts

- Scheduling — cron vs timers
- Logging — classic log tools alongside the journal
- Processes and jobs — when a unit is really a process tree

Continue with the individual command pages in this part.

systemctl

Overview

`systemctl` is the primary interface to **systemd**: start/stop units, enable on boot, inspect status, and list dependencies. Ubuntu, Debian, Fedora, and RHEL-family systems all use it for service control.

Syntax

```
systemctl [options] command [unit...]
```

Units use suffixes: `.service`, `.socket`, `.timer`, `.mount`, `.target`, `.path`, …

Common Commands

Command	Description
`status UNIT`	State, PIDs, recent journal snippet
`start` / `stop` / `restart` / `reload`	Lifecycle
`enable` / `disable`	Boot linkage
`enable --now`	Enable and start
`is-active` / `is-enabled` / `is-failed`	Script-friendly checks
`mask` / `unmask`	Block start entirely / undo
`daemon-reload`	Reload unit files after edits
`list-units` / `list-unit-files`	Inventory
`list-timers` / `list-sockets`	Timers / sockets
`cat UNIT`	Show unit text (incl. drop-ins)
`edit UNIT`	Create drop-in override
`show UNIT`	Low-level properties
`reset-failed`	Clear failed state
`reboot` / `poweroff` / `suspend`	System actions

Key Use Cases

1. Operate application and infrastructure services

2. Enable services at boot
3. Debug failed units
4. Override vendor unit settings safely with drop-ins
5. Inspect timers as a cron alternative

Safety

- `mask` symlinks a unit to `/dev/null` — easy to forget; prefer `stop` + `disable` unless you must block activation.
- Do not `restart` remote access (`ssh.service`) on a fleet without console/recovery.
- `daemon-reload` after every unit file change — forgetting it is a common footgun.
- User vs system buses: `systemctl --user` is a separate world.

Examples with Explanations

Status and logs peek

```
systemctl status nginx --no-pager -l
journalctl -u nginx -n 50 --no-pager
```

Start / enable

```
sudo systemctl start nginx
sudo systemctl enable nginx
sudo systemctl enable --now nginx
```

Restart vs reload

```
sudo systemctl reload nginx          # graceful if ExecReload defined
sudo systemctl restart nginx
sudo systemctl try-restart nginx     # only if already active
```

Failed units

```
systemctl --failed
systemctl list-units --state=failed
sudo systemctl reset-failed
```

Script checks

```
systemctl is-active --quiet nginx && echo running
systemctl is-enabled nginx
systemctl is-failed nginx
```

After editing unit files

```
sudo systemctl daemon-reload
sudo systemctl restart myapp.service
```

Drop-in override (preferred over editing vendor files)

```
sudo systemctl edit nginx.service
# creates /etc/systemd/system/nginx.service.d/override.conf
sudo systemctl cat nginx.service
```

Example override snippet:

```
[Service]
Environment=GOMAXPROCS=4
LimitNOFILE=65535
```

Timers (cron alternative)

```
systemctl list-timers --all
systemctl status logrotate.timer
systemctl cat logrotate.timer
```

Dependencies and reverse deps

```
systemctl list-dependencies nginx.service
systemctl list-dependencies --reverse nginx.service
```

User services

```
systemctl --user status podman.socket
loginctl enable-linger "$USER"    # allow user services without login
```

Isolate targets (careful)

```
systemctl get-default
# sudo systemctl isolate multi-user.target   # stops graphical stack, etc.
```

Understanding Output

`status` shows load state (loaded/masked), active state (running/failed/dead), main PID, cgroup path, and a journal snippet. `is-active` / `is-enabled` exit codes are designed for shell scripts (0 = true for the asked condition).

Notes & Pitfalls

- Unit names can be abbreviated when unique (`systemctl status nginx`).
- `restart` is stop+start; open files/sockets may behave differently than `reload`.
- Template units: `systemctl start getty@tty2` instantiates `getty@.service`.
- Masked units look "disabled" but refuse start until `unmask`.
- Containers may run without systemd — commands then fail or talk to the host via bind mounts.

Related Commands

- `journalctl` — logs
- `loginctl` / `hostnamectl` / `timedatectl` — other systemd tools

- `systemd-analyze` — boot and unit blame
- `systemd-run` — ephemeral units / transient timers
- `service` — legacy wrapper

Additional Resources

- `man systemctl`
- `man systemd.service`
- `man systemd.timer`

journalctl

Overview

`journalctl` queries the **systemd journal** — structured logs from the kernel, services, and many applications. On journald systems it replaces a large share of traditional `/var/log/syslog` grepping while adding boot-aware filters and machine-readable output.

Syntax

```
journalctl [options] [matches...]
```

Matches look like `FIELD=value` (e.g. `_SYSTEMD_UNIT=nginx.service`, `_PID=1234`, `PRIORITY=3`).

Common Options

Option	Description
`-u UNIT`	Messages for unit (repeatable)
`-b` / `-b -1` / `-b ID`	Current / previous / specific boot
`-f`	Follow (like `tail -f`)
`-e`	Jump to end in pager
`-n N`	Last N entries
`-p PRIORITY` / `-p from..to`	Priority filter (`err`, `warning`, `0`–`7`)
`-S` / `-U` / `--since` / `--until`	Time window
`-o short-iso` / `json` / `json-pretty` / `cat`	Output format
`-k`	Kernel messages only
`-x`	Extra explanations when available
`-g REGEX`	Message regex filter (newer)
`--disk-usage`	On-disk journal size
`--vacuum-size=` / `--vacuum-time=` / `--vacuum-files=`	Reclaim space
`--no-pager`	stdout for scripts
`-r`	Reverse (newest first)

Key Use Cases

1. Debug a failed service
2. Follow live logs while reproducing a bug
3. Inspect the previous boot after a crash/reboot
4. Export structured logs for `jq` / SIEM

Examples with Explanations

Unit logs

```
journalctl -u ssh.service -n 50 --no-pager
journalctl -u nginx --since "1 hour ago"
journalctl -u nginx -u php8.3-fpm --since today
```

Follow

```
journalctl -u myapp -f
```

Previous boot / crash triage

```
journalctl --list-boots
journalctl -b -1 -p err..alert --no-pager
journalctl -b -1 -u kdump.service
```

Kernel / dmesg-like

```
journalctl -k -b
journalctl -k --since "10 min ago"
```

Time window

```
journalctl --since "2026-08-10 09:00:00" --until "2026-08-10 10:00:00"
journalctl --since -30m
```

Priority

```
journalctl -p err -b
journalctl -p warning..crit -u myapp
```

JSON for jq

```
journalctl -u nginx -o json -n 100 | jq -r '.MESSAGE'
journalctl -u nginx -o json-pretty -n 5
```

Field matches

```
journalctl _COMM=sudo -n 20
journalctl _UID=1000 --since today | head
journalctl CODE_FILE=/usr/bin/something   # when apps set structured fields
```

Disk usage and vacuum

```
journalctl --disk-usage
sudo journalctl --vacuum-size=500M
sudo journalctl --vacuum-time=14d
```

Persistent journals grow until vacuum or system policy limits apply.

Catalog / explain

```
journalctl -x -u NetworkManager -n 20
```

Understanding Output

Default opens a pager. Each line includes timestamp, hostname, syslog identifier, PID, and message. Use `--no-pager` in scripts. Privileges: members of `systemd-journal` or `adm` (distro-dependent) can read more than their own user journals.

Notes & Pitfalls

- Without persistent storage (`/var/log/journal`), logs may live only in volatile `/run/log/journal` and vanish on reboot.
- Vacuum carefully on hosts that need long forensic retention.
- Noisy units: rate-limit in the service or drop journal rate limits thoughtfully in `journald.conf`.
- `grep` on pager output is inferior to `-u`/`-g`/field matches for large journals.

Related Commands

- `systemctl` — service control and status
- `dmesg` — kernel ring buffer
- `logger` — write to journal/syslog
- `jq` — parse JSON output
- `last` / `lastb` — login history (not journal)

Additional Resources

- `man journalctl`
- `man journald.conf`
- `man systemd.journal-fields`

sysctl

Overview

`sysctl` reads and writes **kernel parameters** exposed under `/proc/sys/`. Changes with `-w` apply immediately to the running kernel; persistence requires files under `/etc/sysctl.conf` or `/etc/sysctl.d/*.conf`.

Syntax

```
sysctl [options] [variable[=value] ...]
sysctl -p [file]
```

Common Options

Option	Description
`-a`	Dump all
`-w var=val`	Write (some builds allow `var=val` alone)
`-p [file]`	Load from file (default `/etc/sysctl.conf`)
`-e`	Ignore unknown keys
`-n`	Values only
`-N`	Names only
`-r` pattern	Filter names by regex

Key Use Cases

1. Enable IP forwarding for routers/Kubernetes nodes
2. Tune vm.swappiness, inotify limits, file-max
3. Apply security-related knobs (rp_filter, kptr_restrict)
4. Inspect live vs on-disk config drift

Examples with Explanations

Read

```
sysctl net.ipv4.ip_forward
sysctl -a | rg 'ip_forward|swappiness|file-max'
sysctl -n vm.swappiness
```

Write (runtime)

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w vm.swappiness=10
sudo sysctl -w fs.inotify.max_user_watches=524288
```

Persist

```
# /etc/sysctl.d/99-local.conf
# net.ipv4.ip_forward = 1
# vm.swappiness = 10
sudo sysctl --system
# or
sudo sysctl -p /etc/sysctl.d/99-local.conf
```

One-liner recipes

```
# Forwarding check on a potential router
sysctl net.ipv4.ip_forward net.ipv6.conf.all.forwarding

# Common developer inotify fix
sudo sysctl -w fs.inotify.max_user_watches=524288

# Show non-default-ish network stack subset
sysctl net.core.somaxconn net.ipv4.tcp_tw_reuse 2>/dev/null
```

Notes & Pitfalls

- Typos with `-w` can break networking—prefer dry-run knowledge and staged files.
- Some keys are **read-only** or namespaced in containers (harder/impossible to change).
- Distribution defaults live in multiple `sysctl.d` snippets—use `sysctl --system` order carefully.
- Don't copy random "performance" sysctl pastebins without understanding.

2026-relevant notes

- Kubernetes/kube-proxy and CNI docs still require specific sysctls; prefer documented allowlists.
- Prefer **systemd-sysctl** (`sysctl --system`) over editing only `/etc/sysctl.conf`.
- Cgroup v2 and container runtimes own many resource controls that old sysctl guides misattribute.

Related Commands

- `cat /proc/sys/...` — direct access
- `systemctl` — services may set knobs
- `sysctl --system` — apply all conf
- `prlimit` / `ulimit` — process limits (different layer)

Additional Resources

- `man sysctl`, `man sysctl.d`
- Kernel `Documentation/admin-guide/sysctl/`

shutdown

Overview

`shutdown` schedules a system power-off, halt, or reboot and notifies logged-in users. On Ubuntu (systemd), it cooperates with **logind/PID 1** rather than implementing an old SysV path alone. Cousins: `reboot`, `poweroff`, `halt`, and `systemctl reboot|poweroff`. Use clear wall messages on multi-user hosts; cancel with `shutdown -c` when you abort a maintenance window.

Syntax

```
sudo shutdown [options] [time] [wall-message]
sudo reboot [options]
sudo poweroff [options]
sudo halt [options]
```

Time forms

Form	Meaning
`now`	Immediately
`+M`	M minutes from now (`+15`)
`hh:mm`	Clock time today (or tomorrow if past) on many implementations

Common options (`shutdown`)

Option	Description
`-r`	Reboot after shutdown sequence
`-h`	Halt/power off (historically “halt”; with systemd often powers off)
`-H`	Halt (may leave power on — hardware dependent)
`-P`	Power off
`-c`	Cancel a pending shutdown
`-k`	Send wall messages only (**no** real shutdown)

Option	Description
`--no-wall`	Skip wall broadcast (where supported)
`-f` / `-F`	Legacy fsck flags — often ignored under systemd

systemd / loginctl cousins

Command	Effect
`systemctl reboot`	Reboot
`systemctl poweroff`	Power off
`systemctl halt`	Halt
`systemctl kexec`	kexec reboot when configured
`systemctl suspend` / `hibernate`	Sleep states
`loginctl reboot` / `poweroff`	Session manager entry points

Safety

- Unsaved work is lost; broadcast intent early (`wall`, chat, tickets).
- Remote reboot without **out-of-band console** (iLO, serial, cloud console) risks lockout if network/boot fails.
- Cloud VMs: provider APIs may force-stop differently from graceful ACPI/systemd shutdown — prefer guest-clean shutdown first.
- Cancelling (`-c`) only works if the schedule is still pending — not after the machine is already going down.
- Avoid `reboot -f` / immediate forced paths unless the system is already wedged.

Key Use Cases

1. Graceful reboot after kernel/package updates
2. Timed maintenance windows with user notification
3. Cancel a mistaken schedule
4. Dry-run messaging with `-k`

Examples with Explanations

Example: power off now

```
sudo shutdown -h now
sudo poweroff
sudo systemctl poweroff
```

Equivalent intents on modern Ubuntu; prefer one style and stay consistent in runbooks.

Example: reboot now

```
sudo shutdown -r now
sudo reboot
sudo systemctl reboot
```

Example: delayed reboot with message

```
sudo shutdown -r +15 "kernel update reboot in 15 minutes - save work"
```

Users get wall notifications; new logins may be blocked as the time approaches (systemd/login semantics).

Example: wall-clock schedule

```
sudo shutdown -r 23:30 "nightly maintenance reboot"
```

Handy for agreed maintenance slots. Confirm timezone with `timedatectl`.

Example: cancel

```
sudo shutdown -c
sudo shutdown -c "reboot cancelled - window postponed"
```

Always cancel explicitly if the change is aborted so users are not left expecting downtime.

Example: message only (drill)

```
sudo shutdown -k +10 "DRILL only - no reboot will occur"
```

Tests communication without risking the host.

Example: systemd reboot with wall message

```
sudo systemctl reboot
# scheduled style often still via shutdown(8):
sudo shutdown -r +5 "applying HA failover test"
```

Example: check last reboots

```
last reboot | head
who -b
journalctl --list-boots | tail
```

Forensics after unexpected restarts.

Example: inhibit shutdown (awareness)

```
systemd-inhibit --list
# apps can delay sleep/shutdown for a reason - check during "why won't it reboot?"
```

Example: forced paths (last resort)

```
sudo systemctl reboot -i          # ignore inhibitors (systemd)
# magic SysRq (if enabled) is an emergency tool - not routine ops
```

Use only when orderly shutdown is impossible and data risk is accepted.

Understanding Output

- `shutdown` typically prints confirmation of the scheduled time.
- Broadcast messages appear on terminals of logged-in users.
- Journal will show shutdown targets activating (`shutdown.target`, `reboot.target`).

Notes & Pitfalls

- On systemd, many binaries are related; `reboot` may be a symlink into systemctl machinery — behavior is unified.
- Containers: `shutdown` inside a container usually does **not** mean what you want for the host; use the orchestrator.
- Virt guests: clean shutdown lets the hypervisor mark the VM stopped cleanly.

- UPS scripts and cloud metadata “maintenance events” may call these tools for you — know who owns the action.
- `shutdown -h` vs `-P` vs `poweroff` nuances matter on bare metal with odd firmware; test in the lab once.

Related Commands

- `systemctl` — reboot/poweroff targets and inhibitors
- `reboot` / `poweroff` / `halt` — short forms
- `wall` — manual broadcast
- `timedatectl` — confirm clock before `hh:mm` schedules
- `last` / `journalctl` — reboot history
- `loginctl` — session-layer power commands

Additional Resources

- `man shutdown`
- `man systemd-shutdownd.service` / `man logind.conf`

reboot

Overview

`reboot` restarts the system. On modern systemd systems it is typically a symlink or wrapper around **`systemctl reboot`**. Prefer explicit `systemctl` forms in scripts for clarity and options (wall messages, force flags). Always ensure workloads can stop cleanly.

Syntax

```
reboot [options]
systemctl reboot
systemctl reboot --message="..."
```

Common Options

Option / form	Description
(none)	Normal reboot via init/systemd
`--help`	Help
`systemctl reboot -i`	Ignore inhibitors (careful)
`systemctl reboot --force`	Forceful (see man)
`systemctl reboot --firmware-setup`	Reboot to firmware setup (UEFI) when supported
`shutdown -r now`	Equivalent classic form
`shutdown -r +15 "msg"`	Scheduled reboot

Examples with Explanations

Normal reboot

```
sudo reboot
sudo systemctl reboot
sudo shutdown -r now
```

Scheduled

```
sudo shutdown -r +30 "Reboot for kernel update"
sudo shutdown -c                      # cancel
```

With broadcast message

```
sudo systemctl reboot --message="Kernel update; back in 5 minutes"
```

Inhibitors

```
systemd-inhibit --list
sudo systemctl reboot -i              # ignore inhibitors if stuck
```

Firmware setup (UEFI)

```
sudo systemctl reboot --firmware-setup
```

After kernel install

```
# apply new kernel
sudo reboot
uname -r                              # after boot
```

Containers / hosts

```
# inside a container, reboot may be blocked or reboot the host depending on privileges -
# almost always wrong. Restart the container instead:
podman restart myctr
systemctl restart myapp.service
```

Notes / Pitfalls

- Unsaved work dies; notify users (`wall`, tickets) on multi-user systems.
- Force flags can skip clean unmounts — risk filesystem recovery work.
- Cloud VMs: reboot vs stop/start may change ephemeral networking or public IPs depending on provider.
- Serial/console access ready before remote reboot of network-critical hosts.

- `reboot` during package transactions can leave broken state — finish `apt`/`dnf` first.

2026-relevant notes

- Live kernel patching reduces some reboot urgency; still needed for many updates.
- Immutable OS / image-based systems reboot into new deployments more often — treat as normal.
- Prefer orchestration drain (k8s) before node reboot.

Related Commands

- `shutdown` / `poweroff` / `halt`
- `systemctl reboot` / `poweroff`
- `timedatectl` / `hostnamectl` — post-boot identity
- `journalctl -b` — logs for current boot
- `who` / `w` — who is logged in before reboot

Additional Resources

- `man reboot`, `man systemctl`, `man shutdown`

podman

Overview

`podman` is a **daemonless** OCI container engine with a CLI largely compatible with Docker. It excels at **rootless** containers and integrates cleanly with systemd via `podman generate systemd` / Quadlet. Prefer it when you want containers without a long-lived root daemon.

```
sudo apt install podman    # or distro equivalent
sudo dnf install podman
```

Syntax

```
podman [global options] command [options]
```

Common Commands

Command	Description
`run`	Create + start container
`ps` / `ps -a`	List containers
`images`	List images
`pull` / `push`	Registry ops
`build`	Build from Containerfile/Dockerfile
`exec`	Exec into running container
`stop` / `start` / `rm` / `rmi`	Lifecycle
`logs` / `inspect`	Debug
`pod`	Pod grouping (K8s-like)
`generate systemd`	Unit files
`compose`	Compose support (podman compose / podman-compose—check install)
`login`	Registry auth

Key Use Cases

1. Rootless development containers

2. Drop-in `docker` CLI workflows (`alias docker=podman` with caveats)
3. systemd-managed long-running containers
4. Building images without dockerd

Examples with Explanations

Run

```
podman run -it --rm alpine:3.20 sh
podman run -d --name web -p 8080:80 docker.io/library/nginx:alpine
podman ps
podman logs -f web
```

Build

```
podman build -t myapp:dev .
podman images
```

Rootless tips

```
id
podman info
# subuid/subgid must be configured for the user on many distros
grep ^$USER: /etc/subuid /etc/subgid
```

systemd

```
podman create --name web -p 8080:80 nginx:alpine
podman generate systemd --new --name web > ~/.config/systemd/user/container-web.service
systemctl --user daemon-reload
systemctl --user enable --now container-web.service
```

Quadlet units (`.container` files) are the newer preferred approach on current Fedora/RHEL—check current Podman docs.

One-liner recipes

```
# Cleanup
podman container prune
podman image prune -a   # careful

# Docker socket compatibility is optional; prefer native podman
podman run --rm quay.io/podman/hello
```

Notes & Pitfalls

- **Docker Compose v2** (`docker compose`) is Docker's plugin; Podman users often use `podman compose` or `podman-compose`—behavior is not 100% identical. Verify with current docs.
- Rootless port binding under 1024 needs config/workarounds.
- Volume SELinux labels (`:Z`, `:z`) matter on enforcing systems.
- `alias docker=podman` breaks tools that expect Docker API quirks—test.

2026-relevant notes

- Podman 4/5.x generations emphasize Quadlet and improved compose compatibility—**check your distro's version** rather than assuming flags.
- Kubernetes remains the multi-node orchestrator; Podman is local/engine level.
- Prefer short image names only if registries.conf short-name mode is understood—fully qualify `docker.io/library/...` when debugging pulls.

Related Commands

- `docker` — daemon-based engine
- `nerdctl` — containerd CLI
- `buildah` — image build focus
- `skopeo` — copy/inspect images
- `systemctl` — service management

Additional Resources

- `man podman`
- docs.podman.io (verify current)

systemd-run

Overview

`systemd-run` creates **transient** units on the fly: one-shot commands, background services, or temporary timers without writing unit files. Ideal for resource-limited jobs, ad-hoc schedules, and testing cgroup settings.

Syntax

```
systemd-run [options] command [args...]
systemd-run --user [options] command...
```

Common Options

Option	Description
`--unit=name`	Name the transient unit
`--property=…` / `-p`	Set unit properties (`CPUQuota=`, `MemoryMax=`, …)
`--timer-property=…`	For `--on-calendar` / timers
`--on-calendar=`	Schedule like a timer
`--on-active=`	Run after a delay
`-t` / `--pty`	PTY for interactive
`-P` / `--pipe`	Connect stdin/stdout
`--wait`	Wait until finish
`--collect`	Garbage-collect unit after exit
`--uid=` / `--gid=`	Run as user/group
`--working-directory=`	cwd

Safety

- Transient timers/services disappear after reboot (unless you convert to real units).
- Resource properties can OOM-kill aggressively — test with `--user` when possible.
- Privileged properties need root.

Examples with Explanations

One-shot with wait

```
sudo systemd-run --wait --unit=scratch-job \
  /usr/local/bin/backup.sh
systemctl status scratch-job
```

CPU/memory limits

```
sudo systemd-run -p CPUQuota=50% -p MemoryMax=512M --unit=heavy \
  --wait /usr/local/bin/encode.sh
```

Transient timer

```
sudo systemd-run --on-calendar='*-*-* 03:00:00' --unit=nightly-backup \
  /usr/local/bin/backup.sh
systemctl list-timers | grep nightly
```

User scope

```
systemd-run --user --on-active=10min --notify-me \
  notify-send 'Timer fired'
```

Interactive shell in a scope

```
systemd-run --pty --same-dir --wait --collect bash
```

Related Commands

- `systemctl` — manage the resulting unit
- `systemd-analyze` — calendar expression help
- `cron` / `at` — classic scheduling
- `nice` / `ionice` / `systemd-cgtop` — resource control neighbors

Additional Resources

- `man systemd-run`
- `man systemd.timer`
- `man systemd.resource-control`

systemd-analyze

Overview

`systemd-analyze` diagnoses **boot performance** and unit graphs: how long boot took, which units delayed it, whether unit files are valid, and whether calendar/timer expressions parse. First tool when "the box boots slowly".

Syntax

```
systemd-analyze [options] [subcommand]
```

Common Subcommands

Subcommand	Description
`time`	Firmware/loader/kernel/userspace timings
`blame`	Units sorted by time to start
`critical-chain`	Critical path to default target
`plot > boot.svg`	SVG timeline
`verify UNIT…`	Static unit file checks
`calendar EXPR`	Explain calendar schedule
`security UNIT`	Sandbox exposure score (newer)
`dot`	Dependency graph (graphviz)

Examples with Explanations

Boot timings

```
systemd-analyze time
systemd-analyze blame | head
systemd-analyze critical-chain
```

Plot

```
systemd-analyze plot > /tmp/boot.svg
# open in a browser
```

Verify units after editing

```
systemd-analyze verify /etc/systemd/system/myapp.service
```

Calendar expressions (timers)

```
systemd-analyze calendar 'Mon..Fri *-*-* 09:00:00'
systemd-analyze calendar 'weekly'
```

Security profile (where available)

```
systemd-analyze security ssh.service | head
```

Notes & Pitfalls

- First boot after kernel update can skew blame times.
- Network mounts and cloud-init often dominate critical-chain.
- Containers without systemd won't produce meaningful host boot data.

Related Commands

- `systemctl`
- `journalctl -b`
- `systemd-run`
- `bootctl` — EFI bootloader (where applicable)

Additional Resources

- `man systemd-analyze`

loginctl

Overview

`loginctl` manages **systemd-logind** seats, sessions, and users: who is logged in, idle hints, kill sessions, and **lingering** (keep user services running after logout). Essential for systemd user instances and multi-seat systems.

Syntax

```
loginctl [options] command [args...]
```

Common Commands

Command	Description
`list-sessions`	Active sessions
`session-status ID`	Detail
`user-status USER`	User's sessions / linger
`list-users`	Users with sessions
`enable-linger` / `disable-linger`	User services without login
`terminate-session` / `kill-session`	End session
`lock-session`	Lock session (where a lock agent exists)
`show-session`	Low-level properties

Examples with Explanations

Who is logged in

```
loginctl list-sessions
loginctl list-users
who
```

Session details

```
loginctl session-status
loginctl show-session "$XDG_SESSION_ID" -p Remote -p Type -p State
```

Enable lingering for user services

```
loginctl enable-linger "$USER"
# or for another account:
sudo loginctl enable-linger deploy
systemctl --user status
```

Required for rootless Podman/user timers to survive logout.

Terminate a stale session

```
loginctl list-sessions
sudo loginctl terminate-session 42
```

Notes & Pitfalls

- Killing sessions drops that session's processes (including SSH shells attached to it).
- Linger creates `/var/lib/systemd/linger/USER`.
- In containers, logind may be absent.

Related Commands

- `systemctl --user`
- `who` / `w` / `last`
- `machinectl`

Additional Resources

- `man loginctl`
- `man logind.conf`

modprobe

Overview

`modprobe` loads and unloads **kernel modules**, resolving dependencies via modules.dep — preferred over raw `insmod`/`rmmod` for daily admin. Pair with `lsmod` to see what is loaded and `dmesg` for driver messages.

Syntax

```
sudo modprobe [options] module [module_options]
sudo modprobe -r module
```

Common Options

Option	Description
`-r` / `--remove`	Unload module (+ unused deps)
`-v`	Verbose
`-n`	Dry-run
`-c`	Show config
`--show-depends`	Print dependency set
`-f`	Force (dangerous)

Safety

- Removing modules in use can fail or disrupt devices (`rmmod` may force more aggressively — still dangerous).
- Blacklist broken modules via `/etc/modprobe.d/` rather than fighting boot.
- Out-of-tree modules must match the running kernel.

Examples with Explanations

Load / unload

```
lsmod | head
sudo modprobe overlay
sudo modprobe -r overlay
```

Dependencies

```
modprobe --show-depends xfs
```

Module parameters

```
# example only - parameters are module-specific
sudo modprobe nfs nfsvers=4.2
modinfo nfs | less
```

Blacklist (persist)

```
echo 'blacklist pcspkr' | sudo tee /etc/modprobe.d/blacklist-pcspkr.conf
```

List available modules

```
find /lib/modules/"$(uname -r)" -name '*.ko*' | head
```

Related Commands

- `lsmod` / `modinfo`
- `rmmod` / `insmod` — low-level
- `dmesg` — load errors
- `uname -r` — running kernel

Additional Resources

- `man modprobe`
- `modules.dep(5)`

systemd-creds

Overview

`systemd-creds` lists, shows, **encrypts**, and **decrypts** system and service credentials — small secrets and parameters passed into units as files under `$CREDENTIALS_DIRECTORY`, not as inherited environment variables. Encrypted credentials use AES256-GCM with a key derived from the **TPM2**, a host secret under `/var/lib/systemd/credential.secret`, or both (default when TPM and persistent `/var` exist).

Companion unit settings (see `systemd.exec(5)`): `LoadCredential=`, `LoadCredentialEncrypted=`, `SetCredential=`, `SetCredentialEncrypted=`, `ImportCredential=`.

Syntax

```
systemd-creds [OPTIONS] list
systemd-creds [OPTIONS] cat NAME
systemd-creds encrypt [OPTIONS] INPUT|- OUTPUT|-
systemd-creds decrypt [OPTIONS] INPUT|- OUTPUT|-
systemd-creds setup   # ensure host credential secret exists (as needed)
```

Common options:

Option	Meaning
`--system`	Operate on **system** credentials (PID1), not the current service context
`--name=NAME`	Embed/override credential name in the encrypted blob
`-p` / `--pretty`	With `encrypt` and stdout: print a `SetCredentialEncrypted=` line for unit files
`--with-key=host|tpm2|auto|…`	Select encryption key backend (see man page; `auto` is typical)
`-H` / `-T`	Shortcuts affecting host/TPM key use (see `systemd-creds(1)`)

Safety

- Plaintext files used as `encrypt` input should be `shred -u` (or equivalent) after producing ciphertext.
- `SetCredential=` in unit files is **world-readable** — never put raw secrets there; use `SetCredentialEncrypted=` or `LoadCredentialEncrypted=`.
- Decryption needs the **same machine** key material (TPM +/or `/var` secret). Ciphertext is not portable across hosts by default.
- Credentials are limited (~1 MiB per service aggregate). Not a general secret store for huge blobs.
- Prefer `PrivateMounts=` / sandboxing so `$CREDENTIALS_DIRECTORY` is not visible to other services.

Examples with Explanations

Encrypt a file and load it in a transient service

```
echo -n 'supersecret' > /tmp/db.pass
systemd-creds encrypt --name=dbpass /tmp/db.pass /etc/credstore.encrypted/dbpass.cred
shred -u /tmp/db.pass

sudo systemd-run -P --wait \
  -p LoadCredentialEncrypted=dbpass:/etc/credstore.encrypted/dbpass.cred \
  systemd-creds cat dbpass
# prints: supersecret
```

`LoadCredentialEncrypted=` decrypts at activation and exposes plaintext only as `$CREDENTIALS_DIRECTORY/dbpas` inside the service.

Emit `SetCredentialEncrypted=` for a unit file

```
systemd-ask-password -n | systemd-creds encrypt --name=mysql-password -p - -
# Paste the printed SetCredentialEncrypted=mysql-password: \ … block into the unit [Service]
```

Relative names and credstore search paths

With a **relative** source, systemd searches system credentials, then `/etc/credstore/`, `/run/credstore/`, `/usr/lib/credstore/` (and `*.encrypted` variants for encrypted load/import):

```
sudo mkdir -p /etc/credstore.encrypted
sudo systemd-creds encrypt --name=api-token - /etc/credstore.encrypted/api-token.cred <<'EOF
tok_desk_example
EOF
```

```
# Unit snippet:
# [Service]
# LoadCredentialEncrypted=api-token
# ExecStart=/usr/local/bin/app --token-file=${CREDENTIALS_DIRECTORY}/api-token
```

List / show in context

```
# From an interactive shell (usually empty):
systemd-creds list

# System credentials passed into the machine:
systemd-creds --system list
systemd-creds --system cat some-name
```

Application read pattern

```
# Inside the service (ExecStart script):
#   test -n "$CREDENTIALS_DIRECTORY"
#   install -m 0400 "$CREDENTIALS_DIRECTORY/dbpass" /run/app/db.pass
# Or point the daemon at %d/dbpass via Environment= or ExecStart=
```

In unit files, `%d` expands to the credential directory — useful for `Environment=TOKEN_FILE=%d/api-token` without hardcoding `/run/credentials/…`.

TPM / host key notes

```
# Default encrypt binds to local TPM2 + host secret when available
systemd-creds encrypt --name=backup ./plaintext ./plaintext.cred

# Initrd/generator consumers may need TPM-only wrapping so /var is not required yet:
systemd-creds encrypt --with-key=auto-initrd --name=early ./plaintext ./plaintext.cred
```

Exact `--with-key=` values evolve with systemd releases — check `systemd-creds encrypt --help` on the target host.

Related Commands

Command / setting	Role
`LoadCredential=`	Load plaintext credential from path / AF_UNIX / system cred
`LoadCredentialEncrypted=`	Load ciphertext; decrypt at activation
`SetCredentialEncrypted=`	Embed ciphertext in the unit file
`ImportCredential=`	Pull from credstore / system credentials by name/glob
`systemd-run -p LoadCredentialEncrypted=`	Experiment without writing a unit
`systemd-ask-password`	Prompt; pipe into `systemd-creds encrypt -p`

Try this

1. Encrypt a one-line secret to `/tmp/demo.cred`, run `systemd-run` with `LoadCredentialEncrypted=`, and `cat` it via `systemd-creds`.
2. Generate a `SetCredentialEncrypted=` line with `-p` and drop it into a throwaway unit under `/etc/systemd/system/`.
3. Compare `systemd-creds encrypt --help` key backends on a machine with and without TPM2.
4. Confirm a non-root service user can read `$CREDENTIALS_DIRECTORY/NAME` but another user cannot.

Sources

- systemd — System and Service Credentials
- systemd-creds(1)
- systemd.exec(5) — credential settings

unshare / nsenter

Overview

Linux **namespaces** give a process its own private view of one kernel resource: hostnames, network stacks, mount tables, PID numbers, user IDs, IPC objects, cgroup roots, or clocks. Containers are mostly namespaces plus cgroups. You do not need a container engine to use them, though. Two util-linux commands drive namespaces directly:

- **`unshare`** runs a program in **new** namespaces.
- **`nsenter`** runs a program inside the **existing** namespaces of another process (a container, a sandboxed service, a test rig).
- **`lsns`** lists the namespaces that exist and which processes hold them.

Typical operator jobs: run a build or test with **no network**; give a script a throwaway **private `/tmp` or mount**; debug a container's sockets with the **host's `ss`**, **`ip`**, and **`tcpdump`**; create a **network lab** of namespaces wired with veth pairs; or check what "root inside a container" really means.

```
# Debian/Ubuntu: util-linux is installed by default (unshare, nsenter, lsns)
unshare --version
```

```
unshare from util-linux 2.41.5
```

Every namespace is visible as a symlink under `/proc/<pid>/ns/`. Two processes share a namespace exactly when those links point at the same inode:

```
readlink /proc/self/ns/net
unshare -r -n readlink /proc/self/ns/net
```

```
net:[4026531840]
net:[4026532326]
```

Syntax

```
unshare [options] [program [args...]]          # default program: $SHELL
nsenter [options] [program [args...]]
lsns    [options] [namespace-inode]
```

Namespace	`unshare` / `nsenter` flag	Isolates
mount	`-m` / `--mount`	Mount table (what `mount`, `findmnt` see)
UTS	`-u` / `--uts`	Hostname and NIS domain name
IPC	`-i` / `--ipc`	System V IPC, POSIX message queues
network	`-n` / `--net`	Interfaces, routes, firewall, sockets, ports
PID	`-p` / `--pid`	Process ID numbering (new PID 1)
user	`-U` / `--user`	UID/GID mappings and capabilities
cgroup	`-C` / `--cgroup`	View of the cgroup hierarchy
time	`-T` / `--time`	`CLOCK_MONOTONIC` / `CLOCK_BOOTTIME` offsets

Key `unshare` options:

Option	Meaning
`-r`, `--map-root-user`	New user namespace; map your UID/GID to 0 inside (**rootless** sandboxes)
`-c`, `--map-current-user`	New user namespace; keep your own UID inside
`--map-auto` / `--map-users` / `--map-groups`	Map ranges from `/etc/subuid` / `/etc/subgid` (needs `newuidmap`/`newgidmap`)
`-f`, `--fork`	Fork before running the program (required with `--pid`)
`--mount-proc[=dir]`	Mount a fresh `/proc` (implies `--mount`); needed for `ps` in a new PID namespace
`--kill-child[=signal]`	If `unshare` dies, kill the forked child (implies `--fork`)
`--propagation private|slave|shared|unchanged`	Mount propagation in the new mount namespace (default: `private`)
`--net=FILE` (any `--TYPE=FILE`)	Make the namespace **persistent** by bind-mounting it onto `FILE`
`-R DIR` / `-w DIR`	Set root directory / working directory
`--boottime S` / `--monotonic S`	Clock offsets for a new time namespace

Key `nsenter` options:

Option	Meaning
`-t PID`, `--target PID`	Take namespaces from this process
`-m -u -i -n -p -U -C -T`	Enter that namespace of the target (or `--net=FILE` for a bind-mounted one)
`-a`, `--all`	Enter all of the target's namespaces
`--preserve-credentials`	Do not change UID/GID/groups after entering a user namespace
`-S UID` / `-G GID`	Set UID/GID inside
`-r[DIR]` / `-w[DIR]`	Use the target's root / working directory (or the given one)
`-e`, `--env`	Inherit the target's environment variables
`-c`, `--join-cgroup`	Also join the target's cgroup

Safety

- **"Root" inside a user namespace is not host root.** `unshare -r` gives UID 0 and a full capability set *over resources owned by that namespace* only. Files owned by real root stay protected (see example 1). That is what makes rootless sandboxes possible.
- **`sudo unshare` / `sudo nsenter` are real root.** Entering a container's mount namespace as host root can edit its filesystem; entering its network namespace can change its firewall. Prefer entering only the namespaces you need (`-n` for network debugging), and read before you write.
- **Unprivileged user namespaces can be restricted by policy.** Ubuntu 24.04+ ships `kernel.apparmor_restrict_unprivileged_userns=1`. `unshare -r` then fails with `write failed /proc/self/uid_map: Operation not permitted` unless an AppArmor profile grants `userns`. Some hardened kernels set `user.max_user_namespaces=0`. Prefer a per-program AppArmor profile over disabling the restriction globally.
- **Persistent namespaces outlive their processes.** A namespace bind-mounted with `--net=FILE` or `ip netns add` keeps its interfaces, routes, and firewall rules until you unmount or delete it.
- **Namespaces are isolation, not a security boundary on their own.** Container runtimes add seccomp, dropped capabilities, cgroups, and LSM profiles. A bare `unshare` sandbox has none of those.

Examples with Explanations

1. Rootless "root": a user namespace

```
id -u
unshare --user --map-root-user id
unshare -r cat /proc/self/uid_map
```

```
1000
uid=0(root) gid=0(root) groups=0(root),65534(nogroup)
         0       1000          1
```

`uid_map` reads "inside 0 outside 1000, range 1". Only that one UID is mapped. Anything else, including real root, appears as the overflow ID `65534` (`nobody`):

```
echo "root secret" | sudo tee root-only.txt >/dev/null && sudo chmod 600 root-only.txt
touch mine.txt
unshare -r stat -c '%u:%g %A %n' mine.txt root-only.txt
unshare -r cat root-only.txt
```

```
0:0 -rw-r--r-- mine.txt
65534:65534 -rw------- root-only.txt
cat: root-only.txt: Permission denied
```

Your own file looks root-owned inside; real root's file stays out of reach. `-r` is the building block for every other rootless example below. Unprivileged users can create the other namespace types only when they also create a user namespace that owns them.

2. Run a command with no network

```
unshare -r -n curl -sS --max-time 3 https://example.com/
echo "exit=$?"
unshare -r -n ip -br link
```

```
curl: (6) Could not resolve host: example.com
exit=6
lo               DOWN           00:00:00:00:00:00 <LOOPBACK>
```

A new network namespace contains only a **down** loopback interface. This is a cheap way to prove that a test suite, build, or installer is hermetic. The command fails instead of silently downloading. Bring `lo` up if the program talks to itself over `127.0.0.1`:

```
unshare -r -n sh -c 'ip link set lo up; ip -br addr'
```

```
lo               UNKNOWN        127.0.0.1/8 ::1/128
```

3. Private hostname (UTS)

```
unshare -r -u sh -c 'hostname lab-ns; hostname'
hostname
```

```
lab-ns
<your-hostname>
```

The change exists only inside the namespace. This is handy for software that keys configuration or certificates off `hostname`.

4. New PID namespace: be PID 1

```
unshare -r -p -f --mount-proc sh -c 'sleep 30 & ps -o pid,ppid,user,cmd'
```

```
    PID    PPID USER     CMD
      1       0 root     sh -c sleep 30 & ps -o pid,ppid,user,cmd
      2       1 root     sleep 30
      3       1 root     ps -o pid,ppid,user,cmd
```

Both flags matter:

- `-f`: `unshare()` with `CLONE_NEWPID` affects the caller's **children**, not the caller. Without `--fork`, the program's *first child* becomes PID 1, and when that child exits the namespace is dead:

  ```
  unshare -r -p sh -c '/bin/true; /bin/true; echo after'
  ```

  ```
  sh: 1: Cannot fork
  ```

- `--mount-proc`: `ps` reads `/proc`. Without a fresh `/proc` for the new PID namespace, it sees the host's and gets confused:

  ```
  unshare -r -p -f sh -c 'ps -o pid,cmd'
  ```

  ```
  fatal library error, lookup self
  ```

Remember that PID 1 has special duties: it reaps orphans, and it ignores signals it has no handler for. For long-lived sandboxes, run a small init (`tini`, `dumb-init`) as PID 1.

5. Private mounts: a throwaway tmpfs

```
unshare -r -m sh -c 'mount -t tmpfs scratch /mnt && echo inside > /mnt/note && cat /mnt/note
findmnt /mnt || echo "host: nothing on /mnt"
```

```
inside
/mnt scratch tmpfs
host: nothing on /mnt
```

`unshare` sets mount propagation to **private** by default, so mounts made inside never leak back to the host. When the last process exits, the tmpfs and its contents vanish. This pattern gives scripts a private `/tmp` without touching anyone else's.

6. Time namespace: fake uptime

```
cut -d' ' -f1 /proc/uptime
unshare -r -T --boottime 86400 cut -d' ' -f1 /proc/uptime
```

```
7268.18
93668.19
```

The process inside sees `CLOCK_BOOTTIME` (and `/proc/uptime`) one day ahead. This is useful for testing uptime-based logic and checkpoint/restore. The wall clock (`CLOCK_REALTIME`) is **not** namespaced.

7. Enter a running sandbox with `nsenter`

Start a long-running rootless sandbox, find its PID, compare namespaces, and enter it:

```
#!/usr/bin/env bash
# lsns-demo.sh - start a sandbox, inspect it, enter it, stop it
unshare -r -n -u --fork --kill-child sh -c 'hostname sandbox; exec sleep 600' &
UPID=$!
sleep 1
PID=$(pgrep -P "$UPID")
echo "sandbox PID: $PID"
for ns in user net uts mnt; do
  printf '%-5s host=%-18s sandbox=%s\n' "$ns" "$(readlink /proc/$$/ns/$ns)" "$(readlink /pro
done
lsns -t net -o NS,TYPE,NPROCS,PID,COMMAND | awk 'NR==1 || /sleep 600/'
nsenter -t "$PID" -U -n -u --preserve-credentials sh -c 'echo "inside: $(hostname)"; ip -br
kill "$PID"
wait 2>/dev/null
```

```
bash lsns-demo.sh
```

```
sandbox PID: 70874
user  host=user:[4026531837]  sandbox=user:[4026532584]
net   host=net:[4026531840]   sandbox=net:[4026532586]
uts   host=uts:[4026532318]   sandbox=uts:[4026532585]
mnt   host=mnt:[4026532317]   sandbox=mnt:[4026532317]
        NS TYPE NPROCS   PID COMMAND
4026532586 net       2 70872 unshare -r -n -u --fork --kill-child sh -c hostname sandbox; ex
```

```
inside: sandbox
lo               DOWN           00:00:00:00:00:00 <LOOPBACK>
```

Read it like this:

- `user`, `net`, and `uts` differ; `mnt` is shared because we did not ask for `-m`.
- `lsns` reports 2 processes in the new net namespace. `unshare` itself joined the namespaces before forking (PID namespaces are the exception), so the lowest PID shown is the `unshare` process.
- As an unprivileged user you may enter a namespace owned by **your** user namespace. You must enter the user namespace too (`-U`). Use `--preserve-credentials`, because `nsenter` otherwise calls `setgroups()`, which `unshare -r` disables (`/proc/<pid>/setgroups` reads `deny`):

  ```
  nsenter -t "$PID" -U -n -u sh -c id
  ```

  ```
  nsenter: setgroups failed: Operation not permitted
  ```

- `kill "$PID"` targets the child. With `--fork`, the `unshare` parent ignores `SIGINT` and `SIGTERM` while it waits, so `kill $UPID` alone leaves both processes running. `--kill-child` covers the case where the parent dies by `SIGKILL`.

Host root can enter just the namespaces it needs, without the user namespace:

```
sudo nsenter -t "$PID" -n -u sh -c 'id -u; hostname; ip -br link'
```

```
0
sandbox
lo               DOWN           00:00:00:00:00:00 <LOOPBACK>
```

8. Debug a container or sandboxed service with host tools

Enter only the **network** namespace, so the container's sockets are visible to the host's `ss`, `ip`, `tcpdump`, and `curl`. Those tools need not exist in the image:

```
# Podman / Docker container (illustrative)
PID=$(podman inspect -f '{{.State.Pid}}' web)        # or: docker inspect -f '{{.State.Pid}}
sudo nsenter -t "$PID" -n ss -ltnp
sudo nsenter -t "$PID" -n tcpdump -ni any port 8080

# systemd service with PrivateNetwork=yes / PrivateTmp=yes (illustrative)
PID=$(systemctl show -p MainPID --value myapp.service)
sudo nsenter -t "$PID" -n ip -br addr
sudo nsenter -t "$PID" -m ls -la /tmp                 # the service's private /tmp
```

These two snippets are illustrative. They were not run on the test machine, which had no systemd or container engine. The `nsenter` mechanics are the same as in examples 7 and 9.

Because the mount namespace was **not** entered with `-n` alone, the binaries come from the host. Adding `-m` switches to the container's filesystem, and therefore to the container's binaries.

9. Persistent namespaces and `ip netns`

Bind-mounting a namespace onto a file keeps it alive with no process inside:

```
sudo touch /run/lab-net
sudo unshare --net=/run/lab-net true                # process exits; namespace survives
findmnt -n -o TARGET,SOURCE,FSTYPE /run/lab-net
sudo nsenter --net=/run/lab-net sh -c 'ip link set lo up'
sudo nsenter --net=/run/lab-net ip -br addr         # state persisted between entries
sudo umount /run/lab-net && sudo rm /run/lab-net
```

```
/run/lab-net nsfs[net:[4026532227]] nsfs
lo               UNKNOWN        127.0.0.1/8 ::1/128
```

`ip netns` is the same mechanism with a convention: named network namespaces are bind mounts under `/run/netns/`. A two-endpoint lab connected by a veth pair, serving HTTP from inside the namespace:

```
sudo ip netns add lab
sudo ip link add veth-host type veth peer name veth-lab
sudo ip link set veth-lab netns lab
sudo ip addr add 10.200.0.1/24 dev veth-host && sudo ip link set veth-host up
sudo ip netns exec lab sh -c 'ip addr add 10.200.0.2/24 dev veth-lab; ip link set veth-lab u

sudo nsenter --net=/run/netns/lab ip -br addr       # nsenter and ip netns are interchangea

mkdir -p www && echo "hello from netns lab" > www/index.html
sudo ip netns exec lab python3 -m http.server 8080 --bind 10.200.0.2 -d "$PWD/www" >/dev/nul
sleep 1
curl -s http://10.200.0.2:8080/
sudo ss -N lab -ltn

# clean up: stop processes inside, then delete (deleting the netns destroys veth-lab and its
sudo ip netns pids lab | xargs -r sudo kill
sudo ip netns del lab
```

```
lo               UNKNOWN        127.0.0.1/8 ::1/128
veth-lab@if5     UP             10.200.0.2/24 fe80::b433:a7ff:fe20:e061/64
hello from netns lab
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
```

```
LISTEN 0      5          10.200.0.2:8080      0.0.0.0:*
```

`ss -N NAME` (`--net=NAME`) is a shortcut for "run `ss` inside `/run/netns/NAME`". The `fe80::` link-local address and the `@ifN` index will differ on your machine.

10. List namespaces

```
lsns                                        # all namespaces visible to you, one line each
lsns -t net                                 # only network namespaces
lsns -p "$PID"                              # every namespace of one process
lsns -T                                     # tree view (by owning user namespace)
sudo lsns -t net -o NS,NPROCS,PID,NETNSID,NSFS,COMMAND   # include ip-netns bind mounts
```

Run `lsns` as root to see every process; unprivileged, it shows only what `/proc` lets you read. The `NSFS` column shows bind-mount paths such as `/run/netns/lab`.

Notes & Pitfalls

Pitfall	Symptom / fix
`--pid` without `--fork`	`Cannot fork` after the first child exits. Always use `-p -f`
New PID namespace without `--mount-proc`	`ps`/`top` show host processes or fail (`lookup self`)
`nsenter -U` into an `unshare -r` sandbox without `--preserve-credentials`	`setgroups failed: Operation not permitted`
`kill <unshare-pid>` does nothing	With `--fork` the parent ignores `SIGINT`/`SIGTERM`. Signal the child, or use `--kill-child` and `SIGKILL`
`unshare -r` → `write failed /proc/self/uid_map`	Ubuntu 24.04+ AppArmor userns restriction, or userns disabled by sysctl
`--map-auto` → `failed to execute newuidmap`	Install the `uidmap` package; the user needs `/etc/subuid` and `/etc/subgid` entries
`-n` alone in a container debug session	Binaries and config come from the host (usually what you want). Add `-m` for the container's view
Forgotten `ip netns` / `--net=FILE` mounts	Namespaces and their interfaces linger. Check `ip netns list`, `findmnt -t nsfs`
`CLOCK_REALTIME` in a time namespace	Not namespaced. Only monotonic and boottime offsets exist

Related Commands

Command	Role
`lsns`	List namespaces and their processes
`ip netns`	Named network namespaces under `/run/netns` (`add`, `exec`, `pids`, `del`)
`ss -N NAME`	Sockets inside a named netns
`setpriv`	Drop capabilities / set no-new-privs before running a sandboxed program
`systemd-run -p PrivateNetwork=yes -p PrivateTmp=yes`	The same namespaces, managed as a transient unit
`podman`	Full rootless containers built on user namespaces
`chroot`	Change root directory only, with no namespace isolation

Try this

1. Prove a build is hermetic: run your project's test command under `unshare -r -n`, and fix every test that tries to reach the network.
2. Give a script a private `/tmp`: `unshare -r -m sh -c 'mount -t tmpfs tmpfs /tmp && ./script.sh'`. Confirm that nothing it writes appears in the host `/tmp`.
3. Start `unshare -r -p -f --mount-proc bash`, run `sleep 1000 &` and `ps`, then find the same `sleep` from the host with `pgrep sleep`. Compare the two PIDs.
4. Build the veth lab from example 9, then add a second namespace and a Linux bridge so that two namespaces can reach each other through the host.
5. On an Ubuntu 24.04+ machine, check `sysctl kernel.apparmor_restrict_unprivileged_userns`, reproduce the `uid_map` error, and write a minimal AppArmor profile that allows `userns` for one script.

Sources

- unshare(1), nsenter(1), lsns(8) — util-linux manual pages
- namespaces(7), user_namespaces(7), pid_namespaces(7), time_namespaces(7)
- ip-netns(8) — iproute2
- AppArmor wiki — unprivileged user namespace restrictions (Ubuntu 23.10/24.04 behaviour)

systemd-sysext

Overview

Thesis: immutable and image-based Linux hosts still need *optional* tools — `strace`, a newer debug agent, a lab-only CLI — without baking every binary into the base OS or punching holes in the read-only root. **`systemd-sysext`** activates **system extension images** that overlay `/usr/` and `/opt/` via OverlayFS at runtime. Drop a versioned image (directory, `.raw` disk image, or GPT image) under the extension search path, `merge` (or enable `systemd-sysext.service`), and the files appear as if they had always been part of the OS. `unmerge` takes them away again. The companion **`systemd-confext`** does the same for `/etc/`.

Typical operator jobs:

- Add debug/compiler toolchains to MicroOS / Flatcar / custom immutable images without a reboot-heavy transactional update.
- Temporarily override one `/usr` component with a locally built tree (`DESTDIR=… make install && systemd-sysext refresh`).
- Deliver optional additive software as versioned artefacts (including OCI-published `.raw` images) that track the base OS ID/VERSION.

```
systemd-sysext --version | head -1
systemd-sysext status
systemd-sysext list
```

```
systemd 257 (257.13-1~deb13u1)
HIERARCHY EXTENSIONS SINCE
/opt      none       -
/usr      none       -
No OS extensions found.
```

Outputs below come from **systemd 257** on Debian 13 (kernel 6.12). Commands match `systemd-sysext(8)` as of systemd 262~devel man pages; mutability modes arrived in 256, `--always-refresh=` in 260.

Mental model

```
/var/lib/extensions/<name>/     or  <name>.raw
/run/extensions/…              or  /etc/extensions/… (often symlinks)


      extension-release.NAME     ← must match image name; ID=/VERSION_ID=/SYSEXT_LEVEL=


 systemd-sysext merge|refresh    OverlayFS over host /usr and /opt


      binaries appear under /usr and /opt as if shipped in the base image
      (files outside /usr and /opt in the image are ignored)
```

Fact	Consequence
Merge is **OverlayFS**, not a package install	No dependency solver. Carry what you need, or link only against libraries the base OS already has
Only `/usr` and `/opt` from the image are merged	Dropping files in the image's `/etc` or `/var` does **nothing** for sysext (use **confext** for `/etc`)
Matching uses **`extension-release.NAME`**	`ID=` must match host `os-release` (or be `_any`); then `SYSEXT_LEVEL=` or `VERSION_ID=` must match unless forced
Installed activated (no enable bit)	Every image found at boot is merged if `systemd-sysext.service` runs. Mask with an empty `/etc/extensions/NAME/` directory
sysext portable services	Portable services are sandboxed units with their own root. Sysext is *additive host files* with **no** isolation

Syntax

```
systemd-sysext [OPTIONS...] status|merge|unmerge|refresh|list
systemd-confext [OPTIONS...] status|merge|unmerge|refresh|list   # /etc only
```

Command	Meaning
`status`	Show whether `/usr` and `/opt` (or `/etc` for confext) are currently merged
`list`	Brief list of installed extension images
`merge`	Overlay all currently installed images (fails if already merged)
`unmerge`	Tear down the overlay
`refresh`	Unmerge + merge (after installing/removing an image). Brief window where extension files disappear

Common options:

Option	Meaning
`--root=PATH`	Operate relative to PATH (image builds, chroots)
`--force`	Ignore version / ID mismatches
`--mutable=…`	`no` (default), `auto`, `yes`, `import`, `ephemeral`, `ephemeral-import` — see Mutability
`--image-policy=…`	Disk-image dissection policy (`systemd.image-policy(7)`)
`--no-reload`	Skip manager reload / unit restart hooks from `extension-release`

Search paths (sysext): `/etc/extensions/`, `/run/extensions/`, `/var/lib/extensions/` (primary for large images), plus `/.extra/sysext/` in the initrd.

Safety

- **Sysext is not a security boundary.** Merged binaries run as part of the host OS with the same trust as `/usr`. Prefer portable services or containers when you need isolation.
- **`refresh` briefly unmounts.** Anything that must not disappear mid-flight (in-use DSOs, running binaries from the extension) can break across refresh; prefer a maintenance window or restart units listed in `EXTENSION_RESTART_UNITS=`.
- **Default merge makes `/usr` and `/opt` read-only** on mutable hosts. That surprises admins who then cannot `apt install` into `/usr` until they `unmerge` or enable a mutability mode.
- **`--force` skips compatibility checks.** Easy way to load an extension built for another OS release and get subtle ABI breakage.
- Merging needs privileges and a working mount namespace (OverlayFS). Confined CI containers often can `list`/`status` but fail `merge`.

Examples with Explanations

1. Inventory and status

```
systemd-sysext status
systemd-sysext list
systemctl is-enabled systemd-sysext.service 2>/dev/null || true
```

```
HIERARCHY EXTENSIONS SINCE
/opt      none       -
/usr      none       -
No OS extensions found.
```

On a stock Debian workstation you will usually see an empty list. On MicroOS / image-based fleets, `list` is the first question after "why is `/usr/bin/strace` missing?"

2. Build a directory-based extension

A directory tree that looks like a mini OS, plus a matching `extension-release` file:

```
# layout.sh - create a toy sysext that ships one script
ROOT=/var/lib/extensions/mytools   # needs root to install system-wide
# For labs, build under a staging dir first:
STAGE=/tmp/mytools
rm -rf "$STAGE"
mkdir -p "$STAGE/usr/bin" \
         "$STAGE/usr/lib/extension-release.d"
cat > "$STAGE/usr/bin/hello-sysext" <<'EOF'
#!/bin/sh
echo "hello from sysext"
EOF
chmod 755 "$STAGE/usr/bin/hello-sysext"
cat > "$STAGE/usr/lib/extension-release.d/extension-release.mytools" <<'EOF'
ID=_any
# VERSION_ID= / SYSEXT_LEVEL= omitted: ID=_any skips host version match
EOF
find "$STAGE" -type f | sort
```

```
/tmp/mytools/usr/bin/hello-sysext
/tmp/mytools/usr/lib/extension-release.d/extension-release.mytools
```

Install by copying/symlinking into a search path, then merge:

```
sudo mkdir -p /var/lib/extensions
sudo cp -a /tmp/mytools /var/lib/extensions/mytools
sudo systemd-sysext merge          # or: refresh, if already merged
systemd-sysext status
command -v hello-sysext && hello-sysext
sudo systemd-sysext unmerge
```

Expected after a successful merge on a real host:

```
HIERARCHY EXTENSIONS SINCE
/usr      mytools    …
hello from sysext
```

`--root=` is useful in image pipelines. `systemd-sysext --root=/path/to/rootfs list` enumerates extensions under that root without touching the live `/usr`. Full `merge` still needs a capable mount namespace; confined environments may return `Failed to merge hierarchies`.

3. `extension-release` matching

Field	Role
`ID=`	Must equal host `ID=` unless `_any`
`SYSEXT_LEVEL=`	If set, must match host `SYSEXT_LEVEL=`
`VERSION_ID=`	Used when `SYSEXT_LEVEL` is absent
`ARCHITECTURE=`	Optional; `_any` or match `uname` arch identifiers
`EXTENSION_RELOAD_MANAGER=1`	Reload PID 1 after merge/refresh
`EXTENSION_RESTART_UNITS=`	Units to restart after merge/unmerge (binary replaced)
`EXTENSION_RELOAD_OR_RESTART_UNITS=`	Prefer reload when the unit supports it

Name rule: file must be `extension-release.<image-name>` matching the directory or the `.raw` basename. Sysext images should **not** ship `/usr/lib/os-release` (it would overlay the host's identity).

4. Mutability (systemd 256+)

By default, merging on a writable host **locks** `/usr` and `/opt` to read-only. Modes:

Mode	Behaviour
`no` (default)	Force immutable overlay
`auto`	Mutable only if write-routing dirs exist under `/var/lib/extensions.mutable/`
`yes`	Force mutable; create routing dirs as needed
`import`	Immutable, but merge contents of routing dirs into the host view
`ephemeral`	Mutable via temp dirs; changes discarded on unmerge
`ephemeral-import`	Like ephemeral, but also import routing-dir contents

```
sudo systemd-sysext --mutable=ephemeral merge
# experiment under /usr …
sudo systemd-sysext unmerge   # experiments gone
```

5. OCI / image delivery pattern (openSUSE-style)

Upstream `systemd-sysext` itself accepts directories and disk images, **not** an `oci:` URL. Distros layer delivery on top:

- **openSUSE MicroOS `sysextmgrcli` / `sysextmgrd`** (2026): Varlink daemon downloads verified images into `/var/lib/sysext-store`, symlinks them into `/etc/extensions/` (so they ride Btrfs snapshots / rollbacks), then you still **`systemd-sysext merge`** to activate. Default catalogue includes `debug`, `gcc`, and `git` sysexts — e.g. `sysextmgrcli install git && systemd-sysext merge`.

- **OCI proxy / sysupdate experiments**: publish a `.raw` (or single-layer scratch image wrapping it) to a registry; a helper pulls into the sysext search path. Kairos and community `sysext-oci-proxy` document `oci:` URIs as *alpha* delivery — activation remains `systemd-sysext`.

Boring rule for fleets: treat OCI/HTTP as **transport**. The trust boundary is still `extension-release` matching + image policy (verity/signatures when you use `.raw` GPT images), and activation is always merge/refresh.

```
# conceptual MicroOS flow (package names from openSUSE 2026 docs)
sysextmgrcli list
sysextmgrcli install debug          # store + symlink
systemd-sysext merge                # actually overlay /usr
# …
systemd-sysext unmerge
```

6. confext vs sysext

```
systemd-confext status    # overlays /etc only; nosuid (+ noexec by default)
```

Use **sysext** for binaries and libraries under `/usr`/`/opt`. Use **confext** when you want to swap drop-in config without shipping code. Do not expect a sysext's `/etc` files to appear on the host.

The trap

1. **Expecting package-manager semantics.** No deps, no scripts, no triggers — just files.
2. **Putting config in a sysext.** Ignored. Ship a confext or bake config into the base image.
3. **Forgetting `extension-release` name match.** Silent skip / version error; `--force` hides the lesson.
4. **Leaving sysext merged while trying to mutate `/usr` with `apt`/`dnf`.** Remount/read-only surprises; `unmerge` first or use `--mutable=`.
5. **Confusing sysext with portable services or Flatpak.** Different isolation and lifecycle models.

The boring rule

For optional additive software on an immutable base: build a sysext (directory or `.raw`) with a correct `extension-release`, install it under `/var/lib/extensions/`, enable `systemd-sysext.service` (or `merge` by hand), and keep delivery (HTTP/OCI/sysextmgr) separate from activation. Prefer `_any` + careful library choices, or pin `VERSION_ID`/`SYSEXT_LEVEL` to the base image you tested.

Try this

1. `systemd-sysext list && systemd-sysext status` on your hosts; enable the service only where you intend extensions to apply at boot.
2. Stage a directory extension under `/tmp`, install it into `/var/lib/extensions/`, `merge`, run the binary, `unmerge`.
3. On MicroOS, install the `debug` sysext via `sysextmgrcli`, merge, run `strace --version`, unmerge, and confirm the binary disappears.

Sources

- `systemd-sysext(8)`, `sysext.conf(5)`, `os-release(5)` — systemd project man pages (verified against systemd 257 locally; man text through 262~devel)
- Managing System Extensions with sysextmgrcli — openSUSE MicroOS, 2026-04-23
- Inspiration: X discussion of openSUSE OCI/sysext delivery (post `2099088197942935737`); chapter verified against systemd man + MicroOS docs, not the post

systemd early-boot units

Overview

Thesis: most unit files can ignore boot ordering — `DefaultDependencies=yes` pulls in the ordinary "start me after basic system setup" edges. The interesting (and dangerous) case is a unit that opts **out** of those defaults and places itself **before** `sysinit.target`. That is how legitimate early helpers run (filesystem fsck helpers, module loaders, some cryptsetup paths). It is also how malware authors try to load code before security agents attach probes. Understanding the knobs matters for both writing safe early units and for spotting hostile ones.

This chapter teaches the **operator** reading of early-boot units: `DefaultDependencies=no`, `Before=sysinit.target`, explicit `After=` / `Requires=` / `Conflicts=`, and what `sysinit.target` actually means. No packages are installed; examples are unit text plus read-only `systemctl` / `systemd-analyze` probes on **systemd 257**.

```
systemctl --version | head -1
```

```
systemd 257 (257.13-1~deb13u1)
```

Mental model

```
  cold boot

  local-fs-pre / swap / early mounts

    units with DefaultDependencies=no
       and Before=sysinit.target     ← "very early userspace"

  sysinit.target   ← sync point: "essential system initialization done"

  basic.target → multi-user.target / graphical.target

  ordinary services (EDR agents, sshd, docker, …)
```

Fact	Consequence
`DefaultDependencies=yes` (default)	Service units get implicit conflicts/after edges toward shutdown and basic ordering — you rarely think about `sysinit.target`
`DefaultDependencies=no`	**You** must declare every dependency you need. Forget `After=local-fs.target` and you may run before `/var` exists
`Before=sysinit.target`	Pulls the unit into the early-boot window *if* something also pulls it in (`WantedBy=sysinit.target`, a `Requires=` from an early target, etc.)
`sysinit.target` is a synchronization point	It is not "all security tooling". Many modules and generators already ran
Early invisible	Journal, `systemd-analyze blame/critical-chain`, and on-disk unit files still exist for forensics

Syntax

```
# /etc/systemd/system/early-helper.service  - shape only; do not install blindly
[Unit]
Description=Early helper (example shape)
DefaultDependencies=no
# Run after local filesystems exist, still before sysinit completes:
After=local-fs.target
Before=sysinit.target shutdown.target
Conflicts=shutdown.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/libexec/early-helper

[Install]
WantedBy=sysinit.target
```

Directive	Role in early boot
`DefaultDependencies=`	`no` disables the type's standard dependency set (`systemd.unit(5)`, "Default Dependencies")
`Before=sysinit.target`	Orders this unit earlier than the sysinit sync point
`After=`	Name **every** prerequisite (`local-fs.target`, `systemd-modules-load.service`, …)
`Conflicts=shutdown.target` + `Before=shutdown.target`	Classic oneshot pattern so shutdown tears you down cleanly
`WantedBy=sysinit.target`	Makes `systemctl enable` create the symlink that actually pulls the unit in

`systemd.special(7)` describes `sysinit.target` as the target that pulls in early services for **system initialization** — mounts, swap, modules, etc. — before `basic.target`.

Safety

- **Do not put business daemons here.** Early boot has fewer mounts, less networking, and less logging infrastructure than you think.
- **Malware pattern to recognise:** `DefaultDependencies=no` + `Before=sysinit.target` + `WantedBy=sysinit.target` (or similar) shipping an `insmod` of an out-of-tree module. The Talos write-up discussed on X used that shape; comments correctly noted that `After=systemd-modules-load.service` means *some* modules already loaded, so "before any security tooling" is too absolute.
- **Detection angle:** inventory units with `DefaultDependencies=no` and any `Before=` referencing `sysinit.target` / `basic.target`. Read `ExecStart=` — unsigned `insmod` paths are a red flag.
- **Lab constraint:** creating or enabling system units needs root. This chapter **does not** install units on the scout host. Validate with `systemd-analyze verify` on a copy when you have a disposable VM.

Examples with Explanations

1. Inventory: who opts out of default dependencies?

```
# List unit files that mention DefaultDependencies=no (read-only scan of common paths)
grep -Rsl '^DefaultDependencies=no' /lib/systemd/system /usr/lib/systemd/system 2>/dev/null
```

Representative idea of what you will see on a Debian systemd 257 system: generators and early helpers (`systemd-remount-fs.service`, pieces of `cryptsetup`, `systemd-sysctl.service`, …). Your own additions under `/etc/systemd/system` should be rare.

2. Confirm sysinit sits before basic

```
systemctl list-dependencies sysinit.target 2>/dev/null | head -20
systemctl list-dependencies basic.target 2>/dev/null | head -15
systemd-analyze critical-chain sysinit.target 2>/dev/null | head -25
```

```
# critical-chain shape (times vary; paste is representative of systemd 257):
sysinit.target @NNNms
  ... early services / mounts ...
```

Exact timing lines differ per boot; the structural point is that `basic.target` depends on `sysinit.target`, not the reverse.

3. `systemd-analyze verify` on a draft unit (no install)

```
cat > /tmp/early-helper.service <<'EOF'
[Unit]
Description=Early helper (draft)
DefaultDependencies=no
After=local-fs.target
Before=sysinit.target shutdown.target
Conflicts=shutdown.target

[Service]
Type=oneshot
ExecStart=/bin/true

[Install]
WantedBy=sysinit.target
EOF
systemd-analyze verify /tmp/early-helper.service 2>&1 || true
```

```
# Success is silence, or warnings about missing Install context when verifying
# a loose file. Fix any hard errors before copying into /etc/systemd/system.
```

(`/bin/true` is a stand-in. Production oneshots must declare real ordering for every path they touch.)

4. Reading a suspicious unit

When triage finds an unexpected early service:

```
systemctl cat suspicious.service 2>/dev/null
systemctl show suspicious.service -p After -p Before -p WantedBy -p Requires -p DefaultDepen
```

Checklist:

1. Is `DefaultDependencies` off?
2. Is there `Before=sysinit.target` (or `Before=basic.target` with other early edges)?
3. What does `ExecStart=` run — script in `/tmp`, `insmod`, curl-pipe?
4. Is the unit file owned by root with mode `0644` under `/etc` or did something drop it in `/run`?

5. What early boot is *not*

```
Before=sysinit.target
       "runs before the kernel"
       "runs before every module"
       "invisible to the journal"
```

```
        "bypasses Secure Boot"
```

It means: in the **userspace** dependency graph, this unit is ordered earlier than the sysinit sync point. Kernel lockdown, signatures, and modules loaded via `systemd-modules-load.service` are separate stories — declare `After=` if you depend on them, and never assume absence.

The trap

- Enabling `DefaultDependencies=no` and forgetting `After=local-fs.target` → mysterious failures when `/var` or `/usr` is not ready.
- Treating "before sysinit" as a security boundary for your own agents (EDR should not *rely* on starting after sysinit if the threat model includes early units).
- Copying a malware unit "for research" onto a production host.
- Using Mermaid for topology — this book prefers plain text / dual-theme cards; keep diagrams in ASCII as above.

The boring rule

Ordinary services: leave `DefaultDependencies` alone. Early oneshots: `DefaultDependencies=no`, explicit `After=`/`Before=`/`Conflicts=`, `WantedBy=` a real early target, `systemd-analyze verify` before enable, and code review any unit that asks to run before `sysinit.target`.

Try this

1. On a disposable VM, `grep -Rsl DefaultDependencies=no /lib/systemd/system | wc -l` and sample five files.
2. Draft a oneshot that only writes a stamp file under `/run`, verify, enable, reboot, confirm stamp and journal order relative to `sysinit.target`.
3. Practice triage: given a unit with `Before=sysinit.target` and `ExecStart=/sbin/insmod …`, list three detection points that still apply.

Sources

- `systemd.unit(5)` — `DefaultDependencies=`, dependency model
- `systemd.special(7)` — `sysinit.target`, `basic.target`
- freedesktop systemd documentation (same content as the man pages)
- Inspiration: malmoeb on X (early-boot rootkit unit discussion / Talos Spectre write-up) — chapter teaches the **legit** mechanism and how to recognise abuse; not a how-to for implants

Reading cgroup v2 CPU and Memory Limits

Reading cgroup v2 CPU and Memory Limits

Thesis

On unified **cgroup v2**, the files under your process's cgroup directory tell you the **CPU quota** and **memory cap** the container/orchestrator applied. Reading them beats guessing why Go's `GOMAXPROCS` or a JVM heap looks "wrong" inside a pod.

Mental model

```
/proc/self/cgroup          →  0::/path/inside/mount
/sys/fs/cgroup/<path>/
     cpu.max               →  "<quota> <period>" or "max <period>"
     memory.max            →  bytes or "max"
     memory.current        →  current usage (bytes)
```

`cpu.max` of `max 100000` means unlimited quota with a 100 ms period. A quota of `50000 100000` is half a CPU.

Worked example

Verified on this box (cgroup v2, controllers available):

```
cut -d: -f3 /proc/self/cgroup
# → /agent   (example)

cat /sys/fs/cgroup/cgroup.controllers
cat /sys/fs/cgroup/agent/cpu.max
cat /sys/fs/cgroup/agent/memory.max
cat /sys/fs/cgroup/agent/memory.current
```

Observed:

```
controllers: cpuset cpu io memory hugetlb pids
self_cgroup: 0::/agent
cpu.max: max 100000
memory.max: max
memory.current: 6229708800
```

So this agent cgroup currently has **unlimited** CPU quota and memory max, with several GB already accounted in `memory.current`. In Kubernetes you would instead see numeric `memory.max` and a fractional `cpu.max` under the pod cgroup.

Trap

Reading `/sys/fs/cgroup/cpu.max` at the root when your process is in a nested cgroup. Always resolve `/proc/self/cgroup` first. Also: `memory.high` vs `memory.max` — hitting `high` throttles; hitting `max` OOMs.

Boring rule

Script the triple: self cgroup path → `cpu.max` → `memory.max` (+ `memory.current`). Feed CPU quota into runtime knobs only after you understand period units (microseconds).

Try this

1. Inside a `docker run --cpus=1.5 --memory=512m` (on a host where you may), re-read the three files.
2. Compare with `sysconf(_SC_NPROCESSORS_ONLN)` — it often still shows host CPUs.
3. Read `cpu.stat` / `memory.events` after a throttle or OOM.

Sources

- Kernel cgroup-v2 docs
- `man 7 cgroups`

Containers from Scratch: unshare, mount and pivot_root

Containers from Scratch: unshare, mount and pivot_root

After reading this chapter, you will be able to assemble Linux kernel namespaces (`user`, `mount`, `pid`, `uts`, `ipc`), mount tables, and `pivot_root` into a small rootless "container" script, with no container runtime and no root privileges, and explain what each step is for.

Thesis

A container is not a virtual machine. A runtime such as `runc` builds one from ordinary kernel features: it creates namespaces, makes mount propagation private, prepares a root filesystem, and uses `pivot_root` to make that filesystem the process's `/`. Do those steps yourself with `unshare`, `mount` and `pivot_root` and the sequence stops being magic.

Environment verification

Everything on this page ran with these versions:

```
uname -r; unshare --version; bash --version | head -n 1
```

```
unshare from util-linux 2.41.5
6.12.94+
GNU bash, version 5.2.37(1)-release (x86_64-pc-linux-gnu)
```

> **Note**
>
> **Not run here.** Some distributions restrict unprivileged user namespaces, for example through AppArmor on Ubuntu 24.04 and later. If `unshare --user --map-root-user true` fails with "Operation not permitted", this lab cannot run as an ordinary user on that host. Changing that policy needs root and is outside this page. User namespaces were allowed on the machine used here.

The plan

The script does five things, in this order:

```
Host Environment


1. Create Namespaces (unshare: user, mount, pid, uts, ipc)


2. Private Mount Propagation (mount --make-rprivate /)


3. Construct Root Directory (tmpfs root + ro /usr bind mount)


4. Set Hostname & Swap Root (pivot_root . old-root)


5. Mount Fresh /proc & Detach Host Root (umount -l /old-root)


Execute /bin/sh (PID 1 inside isolated root)
```

1. **Unshare Namespaces**: Isolate user IDs, mount tables, process IDs, hostnames, and IPC primitives.
2. **Mount Propagation**: Make every mount private, so mount events don't propagate back to the host and `pivot_root`'s no-shared-mounts rule is met.
3. **Assemble Rootfs**: Prepare a temporary mount point, bind host binaries read-only, and provide essential configuration files.
4. **Swap Root**: Make the tmpfs the namespace's root mount, then detach the old root so host paths are no longer reachable.
5. **Re-mount /proc**: Mount a clean `/proc` instance so `ps` and PID checks reflect only containerized processes.

The container script

Here is the whole script. It is a teaching toy, not a runtime:

```
#!/usr/bin/env bash
# tiny-container.sh - a rootless "container" built from unshare, mount and pivot_root.
# Run as an ordinary user: ./tiny-container.sh
set -euo pipefail

ROOT=$(mktemp -d /tmp/tiny-root.XXXXXX)

# Everything below runs inside new user, mount, PID, UTS and IPC namespaces.
# --map-root-user makes us "root" inside, mapped to our own UID outside.
unshare --user --map-root-user --mount --pid --uts --ipc --fork \
```

```
  bash -euo pipefail -c '
ROOT=$1
export PATH=/usr/sbin:/usr/bin:/sbin:/bin

# 1. Stop mount events leaking back to the host.
mount --make-rprivate /

# 2. A root filesystem: a tmpfs with the host /usr bind-mounted read-only.
mount -t tmpfs -o size=16m tiny "$ROOT"
mkdir -p "$ROOT"/{usr,proc,tmp,old-root,etc}
mount --rbind /usr "$ROOT/usr"
mount -o remount,bind,ro "$ROOT/usr"
for d in bin sbin lib lib64; do ln -s "usr/$d" "$ROOT/$d"; done
echo "tiny:x:0:0:root:/:/bin/sh" > "$ROOT/etc/passwd"

# 3. Own hostname (UTS namespace).
hostname tiny

# 4. Swap the root: pivot_root needs the new root to be a mount point.
cd "$ROOT"
pivot_root . old-root
cd /

# 5. A /proc that only shows our PID namespace, then drop the host root.
mount -t proc proc /proc
umount -l /old-root
rmdir /old-root

exec /bin/sh -c "
echo \"hostname: \$(hostname)\"
echo \"whoami:   \$(id -un) (uid \$(id -u))\"
echo \"pid of sh: \$\$\"
echo \"processes:\"; ps -o pid,user,args
echo \"root dir:\"; ls /
echo \"uid_map:  \$(cat /proc/self/uid_map)\"
touch /usr/should-fail 2>&1 || echo \"/usr is read-only\"
"
' tiny-container "$ROOT"

# Back on the host: the tmpfs vanished with the mount namespace; remove the empty directory.
rmdir "$ROOT"
```

Running the container

Run it as an ordinary user:

```
./tiny-container.sh
```

```
hostname: tiny
whoami:   tiny (uid 0)
pid of sh: 1
processes:
    PID USER     COMMAND
      1 tiny     /bin/sh -c  echo "hostname: $(hostname)" echo "whoami:   $(id -un) (uid $(i
     19 tiny     ps -o pid,user,args
root dir:
bin
etc
lib
lib64
proc
sbin
tmp
usr
uid_map:           0       1000          1
touch: cannot touch '/usr/should-fail': Read-only file system
/usr is read-only
exit status: 0
```

What to look for in the output: - The hostname changes to `tiny`. - The shell runs as user `tiny` with UID `0` inside the namespace. - The process ID (`pid of sh`) is `1`. - `ps` shows only two processes: the shell (PID 1) and `ps` itself. - `ls /` shows only what the script built: `bin`, `etc`, `lib`, `lib64`, `proc`, `sbin`, `tmp`, `usr`. - `uid_map` reads `0 1000 1`: UID 0 inside is UID 1000 outside, a range of one ID. - Writing to `/usr` fails with a read-only filesystem error.

What the host sees

Run the script again from a small check script that compares the host before and after:

```
before=$(hostname)
./tiny-container.sh > /dev/null
[ "$(hostname)" = "$before" ] && echo "host hostname is still the host's own: yes"
echo "tiny mounts on host: $(grep -c ' /tmp/tiny-root' /proc/self/mounts)"
echo "leftover dirs: $(ls -d /tmp/tiny-root.* 2>/dev/null | wc -l)"
```

```
host hostname is still the host's own: yes
tiny mounts on host: 0
leftover dirs: 0
```

What to look for in the output: - The host's original hostname remains untouched. - No `tiny-root` mount is visible on the host. The tmpfs only ever existed in the container's mount namespace, and it disappeared with that namespace. The script's last line removed the empty directory.

Why pivot_root instead of chroot?

`chroot(2)` only changes the root directory that one process uses to look up paths. The old root and all its mounts are still in the mount namespace.

`pivot_root(2)` changes the namespace itself: it makes `new_root` the root mount and moves the old root to `put_old`. Because the old root is now an ordinary mount, it can be unmounted, which the script does with `umount -l /old-root`.

Requirements for pivot_root(2)

According to `pivot_root(2)`, the following conditions must be met: - `new_root` and `put_old` must be directories. - `new_root` and `put_old` must not be on the same mount as the current root. - `put_old` must be at or underneath `new_root`. - `new_root` must be a mount point, but not `/` (a directory that is not a mount point can be made one by bind-mounting it onto itself). - The parent mounts of `new_root` and of the current root must not have shared propagation, and neither may `put_old` if it is a mount point. - The current root must be a mount point.

Break the mount-point rule and the kernel refuses. Here `pivot_root` runs in a plain temporary directory that is not a mount point:

```
unshare --user --map-root-user --mount bash -c '
  export PATH=/usr/sbin:/usr/bin:/sbin:/bin
  mount --make-rprivate /
  d=$(mktemp -d) && mkdir "$d/old" && cd "$d"
  pivot_root . old; echo "exit status: $?"
  cd / && rm -r "$d"'
```

```
pivot_root: failed to change root from `.' to `old': Invalid argument
exit status: 1
```

`EINVAL` ("Invalid argument") is how the kernel reports a broken precondition. In `tiny-container.sh`, mounting the tmpfs at `$ROOT` is what makes `$ROOT` a mount point.

The PATH gotcha

`pivot_root` lives in `/usr/sbin`, and an ordinary user's `PATH` often leaves that out. The shell inside `unshare` inherits your `PATH`, so the first version of this script failed with exit status 127:

```
pivot_root: command not found
```

The script sets `export PATH=/usr/sbin:/usr/bin:/sbin:/bin` near the top. Inside the new root, `/bin` and `/sbin` are symlinks into the bind-mounted `/usr`, so the same `PATH` still works after the pivot.

What a real container runtime adds

`tiny-container.sh` covers the core of isolation. A real runtime such as `runc` or `crun` reads an OCI bundle (a `config.json` plus a root filesystem) and does much more. A rough comparison:

Feature	`tiny-container.sh`	Production Runtime (`runc`)
Devices	No `/dev` at all	A minimal `/dev` with the device nodes the config asks for
Resource Limits	Whatever cgroup you started in	Its own cgroup with the limits from the config (for example `memory.max`)
Syscall Filtering	None	A seccomp filter from the config
Capabilities	Full capability set inside the user namespace	Only the capabilities listed in the config
Networking	Shares the host's network namespace	Joins or creates the network namespace the config names; wiring it up (veth, bridges) is usually done by the engine above the runtime
Image Layers	One tmpfs plus a bind mount	Uses the root filesystem it is given, which engines usually assemble from overlayfs layers
Configuration	Hardcoded shell commands	Reads standardized OCI `config.json` bundle specifications

Common mistakes

- **Forgetting `--fork` with `--pid`**: a new PID namespace applies only to children created after the `unshare` call. Without `--fork`, `unshare` execs your command directly, so the command stays in the old PID namespace and never becomes PID 1. Use `--fork` so a child becomes PID 1.
- **Mounting `/proc` from the wrong namespace**: a `/proc` mount shows the PID namespace of the process that mounted it. Mount it from inside the new PID namespace (after `--fork`), or `ps` lists host processes.
- **Skipping private propagation**: many distributions that use systemd mount `/` as shared, and `pivot_root` returns `EINVAL` while shared mounts are involved. `unshare(1)` already sets propagation to private by default when it creates a mount namespace (its `--propagation` option). The script's explicit `mount --make-rprivate /` states the requirement and still holds if you swap `unshare(1)` for the raw `unshare(2)` call or another tool.
- **`pivot_root` on a directory that is not a mount point**: `EINVAL`, as shown above. Mount something there first (here, the tmpfs).

- **Pointing the symlinks at the wrong place**: on a usr-merged host, `bin -> usr/bin` resolves inside whatever root the process has, so it keeps working after the pivot. Keep these links relative to the new root, and check that every path the container needs lands inside it.

Exercises

1. **Network Isolation**: Pass `--net` to the `unshare` command inside `tiny-container.sh`. Add `ip link` to the commands the inner shell runs, and confirm that only the loopback interface `lo` exists and that it is down.
2. **Expose Device Nodes**: Before the pivot, create `$ROOT/dev/null` as an empty file and bind-mount the host `/dev/null` onto it. A user namespace can't create device nodes, but it can bind-mount them. Check that `echo hi > /dev/null` works inside the container.
3. **Single-directory pivot_root**: Replace `pivot_root . old-root` with `pivot_root . .` followed by `umount -l .`. This trick from `pivot_root(2)` needs no `put_old` directory.

Summary

Container runtimes assemble kernel features into an execution sandbox. Using `unshare`, you instantiate isolated user, mount, PID, UTS, and IPC namespaces. By establishing `MS_PRIVATE` mount propagation, creating a custom `tmpfs` root, performing `pivot_root`, and mounting a fresh `/proc`, you give a process tree its own view of the system using nothing but standard tools. Devices, cgroups, seccomp, capabilities and networking are what a real runtime adds on top.

Sources

- pivot_root(2) manual page
- unshare(1) manual page
- mount_namespaces(7) manual page
- user_namespaces(7) manual page
- OCI Runtime Specification: Filesystem Bundle

Shell Commands

Intro

Intro

Everyday shell built-ins and small utilities for output, history, name resolution, environment, timing, and the directory stack. Deeper programming belongs in **Scripting → Bash** (this book).

Commands in this part

Command	Role
`echo`	echo writes its arguments to standard output, separated by spaces, usually followed by a newline.
`printf`	printf formats and prints arguments using a format string, like C's printf.
`history`	history shows the shell's command history list (bash).
type / command	type explains how the shell will resolve a name — alias, keyword, function, builtin, or external file.
`export`	export marks shell variables for the environment so child processes inherit them.
`time`	time runs a command and reports how long it took.
pushd / popd / dirs	pushd, popd, and dirs maintain a directory stack in bash.

Suggested starting points

1. Print: `echo`, `printf`.
2. Recall: `history`.
3. What is this name?: `type` / `command`.
4. Environment: `export` (see also `env` in System information).
5. Measure: `time`; navigate: `pushd/popd/dirs`.

Related parts

- Help and documentation tools — `help` for builtins

- Text and pipes — consume what you print
- Scripting → Bash (this book) — functions, arrays, strict mode

Continue with the individual command pages in this part.

echo

Overview

`echo` writes its arguments to standard output, separated by spaces, usually followed by a newline. It is the everyday way to print status lines in the shell. Behavior of escapes (`-e`) and options differs across shells and `/bin/echo` — for portable formatting prefer **`printf`**.

This page stays **operator-light**. Shell programming depth lives in the Shell scripting parts of this book.

Syntax

```
echo [options] [string...]
```

Common Options

Option	Description
`-n`	No trailing newline
`-e`	Enable backslash escapes (`\n`, `\t`, `\\`, …) — **bash builtin** / some `echo`s
`-E`	Disable escapes (bash default)

BusyBox and non-bash `echo` may ignore or treat flags differently. When it matters, use `printf`.

Examples with Explanations

Print text

```
echo 'Hello'
echo Hello world
echo "User: $USER"
```

Single quotes preserve literals; double quotes expand variables.

No newline

```
echo -n 'Prompt: '
# useful before read in interactive snippets
```

Escapes (bash)

```
echo -e 'line1\nline2\tcolumn'
```

Prefer `printf 'line1\nline2\tcolumn\n'` for scripts you share.

Redirect / append

```
echo 'enabled=1' > /tmp/flags.conf
echo 'note' >> /tmp/flags.conf
```

Pipe into other commands

```
echo 'example.com' | dig +short
echo "$TOKEN" | wc -c
```

Avoid `echo $TOKEN` unquoted (word-splitting/globbing).

Show a path or command you are about to run

```
echo "Installing into $PREFIX"
echo "+ apt install -y curl"
```

Common in install scripts for operator visibility.

Notes

- `echo` is often a **shell builtin** (`type echo`); `/bin/echo` may differ.
- Arguments starting with `-` can be misread as options — use `printf '%s\n' "$arg"` for arbitrary data.
- `echo *` expands globs; quote when you mean a literal `*`.
- Don't use `echo` to print binary-safe data; use `printf` or `cat`.

Related Commands

- `printf` — reliable formatting
- `cat` — write multi-line or file contents
- `tee` — print and log
- `yes` — repeat a line

Additional Resources

- `help echo` (bash)
- `man echo`

printf

Overview

`printf` formats and prints arguments using a format string, like C's `printf`. Prefer it over `echo` when you need reliable escapes, fixed columns, no unexpected options parsing, or portable scripts. Available as a bash builtin and `/usr/bin/printf` (usually identical enough for ops use).

Keep complex format logic in **Scripting → Bash** in this book; here are the operator essentials.

Syntax

```
printf FORMAT [ARGUMENT...]
```

Common Format Pieces

Sequence	Meaning
`%s`	String
`%d` / `%i`	Decimal integer
`%x` / `%X`	Hex integer
`%f`	Floating point
`%b`	String with backslash escapes interpreted
`%%`	Literal `%`
`\n \t \\`	Newline, tab, backslash (in FORMAT)

Width/precision: `%-20s`, `%08d`, `%.2f`.

Examples with Explanations

Safe newline print

```
printf '%s\n' "$var"
printf '%s\n' '---' line1 line2
```

Repeats the format as needed for leftover arguments (POSIX).

No trailing newline

```
printf 'Status: %s' ok
```

Columns

```
printf '%-12s %8s\n' NAME SIZE
printf '%-12s %8s\n' root 120G
printf '%-12s %8s\n' home 800G
```

Handy for quick operator tables in scripts.

Zero-pad numbers

```
printf 'ticket-%04d\n' 7
# ticket-0007
```

Interpret escapes in data

```
printf '%b\n' 'line1\nline2'
```

`%b` vs putting `\n` only in FORMAT — know which side holds the escapes.

Hex / bytes (light)

```
printf '%02x\n' 255
printf '\\x%x' 42; echo
```

For full hex dumps use `xxd`.

Build a file

```
printf 'user=%s\nrole=%s\n' "$USER" admin > /tmp/who.ini
```

Arbitrary strings that look like options

```
printf '%s\n' -n
# correctly prints -n  (echo -n would swallow it as a flag)
```

Notes

- Exit status is non-zero on write errors; format errors are shell-dependent.
- Floating formats and locales: decimal comma/point can surprise — set `LC_ALL=C` for machine output.
- Prefer `printf '%s\n' "$x"` over `echo "$x"` in production shell snippets.
- Builtin `printf` vs `/usr/bin/printf`: rare differences; stick to common formats.

Related Commands

- `echo` — simple print
- `sprintf` — not a shell command; use command substitution with `printf`
- `column` — align tables from text
- `xxd` — binary/hex dumps

Additional Resources

- `help printf` (bash)
- `man 1 printf`

history

Overview

`history` shows the shell's command history list (bash). Operators use it to recall prior commands, audit what was run in a session, and re-run lines with event designators (`!!`, `!n`). Configuration lives in `HISTFILE`, `HISTSIZE`, `HISTFILESIZE`, and related bash variables — deep customization belongs in shell-config docs.

Syntax

```
history [n]
history -c
history -d offset
history -a|-n|-r|-w [filename]
```

Without options, lists history with numbers.

Common Options

Option	Description
`n`	Show last `n` entries
`-c`	Clear history list
`-d offset`	Delete entry at offset
`-a`	Append new lines to `$HISTFILE`
`-n`	Read unseen lines from `$HISTFILE`
`-r`	Read file and replace current history
`-w`	Write current history to file
`-p`	Expand history designators (advanced)
`-s`	Append args as a single history entry

Examples with Explanations

Show recent commands

```
history | tail -20
history 30
```

Search history (interactive bash)

```
# Ctrl-R  reverse incremental search (readline)
history | grep ssh
```

Re-run by number / last command

```
!552          # re-run event 552
!!            # re-run last command
sudo !!       # common: repeat last with sudo
```

Be careful with destructive lines — review before firing `!!`.

Append now (multi-session)

```
history -a
```

Useful so another terminal can `history -n` and see your latest commands.

Clear session history

```
history -c
history -w    # optional: persist the empty/cleared state carefully
```

Clearing does not erase other users' audits (journal, `auth.log`, bash_history on disk may still exist until overwritten).

Delete one awkward line

```
history | tail
history -d 1042
```

Still may exist in `$HISTFILE` until rewritten — know privacy limits of shell history.

Where history is stored

```
echo "$HISTFILE"          # typically ~/.bash_history
echo "$HISTSIZE" "$HISTFILESIZE"
```

Notes

- Commands prefixed with space may be omitted if `HISTCONTROL=ignorespace` (common).
- `HISTCONTROL=ignoredups` / `erasedups` change what gets stored.
- History is **not** a security log — rootkits and other shells won't appear; privileged ops need proper auditing.
- zsh uses a different history system (`fc`, `HISTFILE`); this page is bash-oriented as Ubuntu default login shell for many admins.
- Avoid putting secrets on the command line; they land in history and process tables.

Related Commands

- `type` / `command` — resolve what will run
- `alias` — expansions that also show up as typed
- `fc` — fix/re-run history ranges (bash)
- `script` — full terminal typescript recording

Additional Resources

- `help history`
- `man bash` (HISTORY section)

type / command

Overview

`type` explains **how the shell will resolve a name** — alias, keyword, function, builtin, or external file. `command` runs a program **bypassing** functions/aliases of the same name (and can query paths). Together they answer: "Why did *that* run?" when `which` alone misleads (it ignores builtins/aliases depending on setup).

Syntax

```
type [-afptP] name [name...]
command [-pVv] command [arg...]
```

Common Options

type (bash)

Option	Description
`-a`	All locations / matches
`-t`	Single word: `alias`, `keyword`, `function`, `builtin`, `file`
`-p`	Path to disk executable if `file`
`-P`	Force path search even if non-file
`-f`	Suppress function lookup

command

Option	Description
`-v` / `-V`	Describe command (similar spirit to `type`)
`-p`	Use default PATH search
(default)	Execute, skipping shell functions/aliases

Examples with Explanations

What is this name?

```
type ls
type type
type echo cd time
type -a python3
```

Typical: `ls is aliased to …`, `echo is a shell builtin`, `python3 is /usr/bin/python3`.

Machine-readable kind

```
type -t ls
# alias
type -t mkdir
# file
```

Useful in scripts choosing behavior.

Path only

```
type -p sha256sum
# /usr/bin/sha256sum
```

Empty if pure builtin/alias without a file.

Run without alias/function

```
command ls            # unaliased ls
command -v ls
command -p ls
```

Compare to which / command -v

```
which ls
type ls
command -v ls
```

`which` is an external program and can disagree with the shell's true resolution — prefer `type`/`command -v` inside bash.

Debug “wrong binary”

```
type -a node
echo "$PATH"
```

Multiple `file` hits show PATH order; the first is used for external commands.

Notes

- Keywords (`if`, `for`) are not executables — `type` reports `keyword`.
- Hashed paths: bash caches external locations (`hash`); after moves, `hash -r` refreshes.
- `command` is also the safe way to call externals from functions named the same.
- Dash/`sh` may have fewer `type` features — check `help type` in your shell.

Related Commands

- `which` / `whereis` — external path helpers
- `alias` — define aliases
- `hash` — bash command path cache
- `export` / `env` — PATH and environment

Additional Resources

- `help type`, `help command`
- `man bash`

export

Overview

`export` marks shell variables for the **environment** so child processes inherit them. Without export, a variable is shell-local. Operators use it for `PATH`, `EDITOR`, proxy variables, language settings, and app config in a session. Permanent setup belongs in shell rc files or systemd units — not ad-hoc forever in one terminal.

Syntax

```
export [-fn] [name[=value] ...]
export -p
```

Common Options

Option	Description
`name=value`	Set and export in one step
`-p`	Print all exported variables (declare style)
`-n`	Remove export property (variable may remain)
`-f`	Export functions (bash; use sparingly)

Examples with Explanations

Set for children

```
export EDITOR=vim
export LANG=en_US.UTF-8
export PATH="$HOME/.local/bin:$PATH"
```

New processes from this shell see these values.

One-shot without permanent export

```
EDITOR=vim git commit
env NODE_ENV=production node app.js
```

Prefix assignment affects one command; `export` is for the rest of the session (or until changed).

Show exports

```
export -p | head
export -p | grep -E 'PATH|HOME|EDITOR'
env | sort
printenv PATH
```

Unexport / unset

```
export -n MYTEMP          # still set, not in environment
unset MYTEMP              # remove entirely
```

Common operator variables

```
export http_proxy=http://proxy.example:3128
export https_proxy=$http_proxy
export no_proxy=localhost,127.0.0.1,.example.com
export DEBIAN_FRONTEND=noninteractive   # apt in scripts (careful)
```

Persist for your user (pointer)

```
# add to ~/.bashrc or ~/.profile as appropriate - then open a new shell
echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc
```

System services should use unit `Environment=` / `EnvironmentFile=`, not your interactive exports.

Notes

- Child shells inherit exports; already-running processes do not update retroactively.
- `export` is a **builtin** — see `type export`.
- Secrets in the environment are visible to other processes with same privileges (`/proc/pid/environ`) — prefer files with mode `600` or secret managers for sensitive material.
- Arrays and complex bash structures don't export portably; stick to strings for environment.

- Typo'd `PATH=` without preserving old value can break the session (`command not found`).

Related Commands

- `env` — run with modified environment / print env
- `printenv` — print variables
- `set` — shell options and variables (broader)
- `declare -x` — bash equivalent flavor
- `systemctl` — service environment

Additional Resources

- `help export`
- `man bash` (PARAMETERS / ENVIRONMENT)

time

Overview

`time` runs a command and reports how long it took. Bash provides a **builtin** `time` (pipeline-aware) and Ubuntu also ships **`/usr/bin/time`** (GNU time) with more format options. Use it for quick performance checks of scripts, builds, and one-liners. For deep profiling use `perf`, language profilers, or application metrics.

Syntax

```
time command [args...]          # usually bash builtin
/usr/bin/time [options] command [args...]
```

Common Options

Bash builtin

Controlled by `TIMEFORMAT` (see `help time` / bash manual). No traditional flags like GNU time.

GNU `/usr/bin/time` (package `time`)

Option	Description
`-v`	Verbose resource stats
`-p`	Portable output (real/user/sys)
`-f FORMAT`	Custom format string
`-o file`	Write timing to file
`-a`	Append to file

If `type time` shows builtin, call `/usr/bin/time` explicitly for GNU features (`sudo apt install time` if missing).

Examples with Explanations

Basic timing

```
time sleep 2
time tar -czf /tmp/t.tgz /var/log
```

Reports **real** (wall clock), **user** (CPU userspace), **sys** (CPU kernel).

Time a pipeline (bash builtin)

```
time du -sh /var/* 2>/dev/null | sort -h | tail
```

Builtin `time` can attribute the pipeline as a whole depending on shell rules.

GNU verbose

```
/usr/bin/time -v sha256sum large.iso
```

Shows max RSS, page faults, voluntary context switches, exit status, etc.

Portable three lines

```
/usr/bin/time -p make -j"$(nproc)"
```

Custom format

```
/usr/bin/time -f 'real %e sec | maxrss %M KB' ./build.sh
```

Compare two approaches

```
time gzip -k bigfile
time zstd -k bigfile
```

Wall clock is what operators feel; user+sys helps spot CPU-bound vs I/O-bound work.

Bash TIMEFORMAT

```
TIMEFORMAT='real %3R s; user %3U s; sys %3S s'
time ls -R /usr/share/doc >/dev/null
```

Notes

- `real` can exceed `user+sys` when waiting on disk/network/CPU contention.
- `user+sys` can exceed `real` on multi-threaded commands (CPU time summed across cores).
- Output of `time` often goes to **stderr**, separate from the command's stdout.
- Alias/functions: `time` times the resolved command; use `command time` / full path if shadowed.
- Micro-benchmarks need warm caches and repeats; single runs mislead.

Related Commands

- `perf stat` — hardware performance counters
- `strace -c` — syscall time summary
- `hyperfine` — benchmarking tool (extra install)
- `date +%s.%N` — manual timestamps around blocks
- `timeout` — limit runtime, not measure it

Additional Resources

- `help time`
- `man time` (GNU)
- `man bash` (`TIMEFORMAT`)

pushd / popd / dirs

Overview

`pushd`, `popd`, and `dirs` maintain a **directory stack** in bash. `pushd` changes directory and pushes the previous location; `popd` returns; `dirs` lists the stack. Handy when hopping between build trees, config dirs, and logs without losing your place. For a single previous directory, `cd -` is enough.

Light operator page — stack gymnastics in large scripts belong in **Scripting → Bash** in this book.

Syntax

```
pushd [dir]
pushd +N | -N
popd
popd +N | -N
dirs [-clpv] [+N | -N]
```

Common Options

dirs

Option	Description
`-l`	Long paths (no ~ abbreviation)
`-p`	One entry per line
`-v`	Numbered list
`-c`	Clear stack (bash)

pushd / popd

Form	Description
`pushd dir`	Push current, `cd` to `dir`
`pushd`	Swap top two entries
`pushd +N` / `-N`	Rotate stack

Form	Description
`popd`	Pop top and `cd` there
`popd +N`	Delete Nth entry without rotating to it (see help)

`set -o` / `shopt` can affect `cd` behavior (`cdspell`, `autocd`); stack commands still need valid paths.

Examples with Explanations

Jump and return

```
pwd
pushd /var/log
# ... inspect logs ...
popd
pwd
```

Nested hops

```
pushd /etc/nginx
pushd /etc/ssl/certs
dirs -v
popd
popd
```

List the stack

```
dirs
dirs -v
dirs -l -p
```

Swap two project roots

```
cd ~/src/app
pushd ~/src/infra
pushd              # swap top two
```

Numbered rotation

```
dirs -v
pushd +2            # bring entry 2 to top and cd
```

Clear

```
dirs -c
```

Notes

- Stack is **per shell session** — not shared across terminals.
- Failed `pushd` to a bad path usually leaves the stack unchanged (bash) — still verify `pwd`.
- Prefer absolute paths in muscle memory when trees are deep and symlinked.
- `CDPATH` can make `pushd some-name` resolve in unexpected directories — know if you set it.
- Fish/zsh have similar but not identical stack commands.

Related Commands

- `cd` / `pwd` — basic navigation
- `cd -` — previous directory only
- `realpath` / `readlink -f` — resolve paths
- `tree` / `ls` — inspect destinations

Additional Resources

- `help pushd`, `help popd`, `help dirs`
- `man bash` (DIRECTORY STACK)

Security in Shell Scripts

Intro

Intro

SELinux mode and file labels (Fedora/RHEL-family), Linux file capabilities, OpenPGP (`gpg`) for encrypt/sign/verify, modern file encryption with `age`, io_uring/seccomp policy, and unprivileged Landlock sandboxes from `setpriv`. On Ubuntu, AppArmor is more common for MAC — still useful to recognize SELinux tools on mixed fleets. Use `age` for operator file encryption (small keys, pipes, multi-recipient); use `gpg` when you need OpenPGP signatures or verifying signed checksum lists before trusting downloads. Use Landlock when a process should impose an extra filesystem (and, from ABI 4, TCP) allow-list on itself.

Commands in this part

Command	Role
`getenforce`	Prints the current SELinux mode: Enforcing, Permissive, or Disabled.
`setenforce`	Switches SELinux between Enforcing (1) and Permissive (0) until reboot (or until changed again).
`restorecon`	Restores the default SELinux file contexts for paths based on policy file-context rules.
`getcap` / `setcap`	Linux file capabilities grant subsets of root privilege to executables.
`gpg`	OpenPGP encrypt, decrypt, sign, and verify — keys, armored files, and signed checksum workflows.
`age`	Simple modern file encryption — native or SSH recipients, armor, passphrase, tar pipes.
`setpriv --landlock-*`	Unprivileged Landlock sandbox: filesystem allow-lists from the command line; TCP via the Landlock syscalls (ABI 4+).

Suggested starting points

1. Mode: `getenforce` / `setenforce` (temporary permissive for triage only).
2. Labels after copy/mv: `restorecon`.

3. Selective privilege on binaries: `getcap` / `setcap`.
4. Download authenticity: `gpg --verify` on signed `SHA256SUMS`, then `sha256sum -c`.
5. Encrypt a backup tarball: `age-keygen`, then `tar … \| age -r …`.
6. Unprivileged filesystem sandbox: `setpriv --landlock-access fs` plus `path-beneath` rules (see the Landlock chapter).

Related parts

- Users and groups — `sudo` and accounts
- Files and paths — DAC permissions, ACLs, and `sha256sum`
- Networking — host firewalls; `openssl` for TLS cert triage

Continue with the individual command pages in this part.

getenforce

Overview

`getenforce` prints the current **SELinux** mode: `Enforcing`, `Permissive`, or `Disabled`. Standard on Fedora/RHEL/CentOS; on Ubuntu SELinux is usually inactive in favor of AppArmor (`aa-status`). Use this before debugging "AVC denied" issues.

```
# RHEL family
sudo dnf install policycoreutils
```

Syntax

```
getenforce
```

Examples with Explanations

Check mode

```
getenforce
# Enforcing | Permissive | Disabled
```

Related status

```
sestatus
getsebool -a | head
```

Notes & Pitfalls

- `Disabled` requires a reboot to re-enable on traditional setups (prefer permissive for troubleshooting).
- Containers may show different SELinux visibility than the host.

Related Commands

- `setenforce` — toggle enforcing/permissive
- `sestatus` — richer status
- `restorecon` / `chcon` — file contexts
- `ausearch` / `audit2allow` — AVC triage
- `aa-status` — AppArmor on Ubuntu

Additional Resources

- `man getenforce`
- RHEL SELinux documentation

setenforce

Overview

`setenforce` switches SELinux between **Enforcing** (1) and **Permissive** (0) until reboot (or until changed again). Permissive logs denials without blocking — useful for triage, not a permanent hardening strategy.

Syntax

```
sudo setenforce 1    # Enforcing
sudo setenforce 0    # Permissive
getenforce
```

Safety

- Permissive mode reduces security — use briefly, then return to enforcing.
- Does not enable SELinux if it is fully **Disabled** in the kernel/config; that needs config + reboot.
- Production changes should go through change control; prefer boolean/`semanage` fixes over long-term permissive.

Examples with Explanations

Temporary permissive for debugging

```
getenforce
sudo setenforce 0
# reproduce the app failure; watch audit log
sudo ausearch -m avc -ts recent | tail
sudo setenforce 1
```

Persist mode (config — not setenforce)

```
# /etc/selinux/config
# SELINUX=enforcing|permissive|disabled
```

Related Commands

- `getenforce` / `sestatus`
- `restorecon` — fix labels
- `semanage` / `setsebool` — lasting policy tweaks
- `audit2allow` — generate local policy modules carefully

Additional Resources

- `man setenforce`

restorecon

Overview

`restorecon` restores the default **SELinux file contexts** for paths based on policy file-context rules. First fix when a moved/copied file is denied because it carries the wrong label (`httpd` cannot read a file labeled `user_home_t`, etc.).

Syntax

```
sudo restorecon [options] path...
```

Common Options

Option	Description
`-v`	Verbose (show changes)
`-R`	Recursive
`-F`	Force reset
`-n`	Dry-run (no change)
`-e dir`	Exclude directory

Examples with Explanations

Fix a web tree after rsync/mv

```
sudo restorecon -Rv /var/www/html
ls -Z /var/www/html | head
```

Dry-run

```
sudo restorecon -Rvn /srv/data
```

Single file after manual copy

```
sudo cp ~/index.html /var/www/html/
sudo restorecon -v /var/www/html/index.html
```

Notes & Pitfalls

- `cp -a` preserves contexts; `mv` within a filesystem keeps labels that may be wrong for the destination type.
- Custom paths may need `semanage fcontext` **before** restorecon will apply the right type.
- On Ubuntu/AppArmor systems this tool may be absent or irrelevant.

Related Commands

- `chcon` — temporary/manual context (prefer restorecon + semanage)
- `ls -Z` / `id -Z` — show contexts
- `setenforce` / `getenforce`
- `semanage fcontext`

Additional Resources

- `man restorecon`
- `man semanage-fcontext`

getcap / setcap

Overview

Linux **file capabilities** grant subsets of root privilege to executables (e.g. bind low ports without full uid 0). `getcap` lists them; `setcap` assigns them. Prefer capabilities over setuid-root when you must elevate a single binary.

```
sudo apt install libcap2-bin
```

Syntax

```
getcap [-r] path...
sudo setcap cap_spec path
sudo setcap -r path          # remove
```

Examples with Explanations

Inspect

```
getcap /usr/bin/ping
getcap -r /usr/bin 2>/dev/null | head
```

Allow binding privileged ports (example)

```
# illustration - understand security impact first
sudo setcap 'cap_net_bind_service=+ep' /usr/local/bin/myapp
getcap /usr/local/bin/myapp
```

Remove capabilities

```
sudo setcap -r /usr/local/bin/myapp
```

Safety

- Capabilities are powerful; `+ep` (effective+permitted) can be equivalent to selective root.
- Package updates may overwrite custom capabilities.
- Prefer systemd `AmbientCapabilities=` / `CapabilityBoundingSet=` for services.

Notes & Pitfalls

- Filesystems must support extended attributes (`xattr`).
- Copy tools may drop capabilities unless preserved.
- Combine with seccomp/namespaces for real confinement.

Related Commands

- `capsh` — capability-aware shell helpers
- `systemctl edit` — service capabilities
- `sudo` — broader elevation
- `getfacl` — ACLs (different mechanism)

Additional Resources

- `man getcap` / `man setcap`
- `man capabilities`

gpg

Overview

`gpg` (GnuPG) is the OpenPGP tool for **encrypting**, **decrypting**, **signing**, and **verifying** files and data. Operators use it to protect backups and secrets off-box, to exchange confidential artifacts with colleagues, and — most often on Ubuntu servers — to **verify signed checksum lists** before trusting an ISO or release tarball. Prefer GnuPG for human/operator file crypto and signature checks; use `openssl` for TLS cert triage and ACME clients for certificate issuance.

On Ubuntu 22.04/24.04 Server the package is `gnupg` (GnuPG 2.x; `gpg` is the CLI). Minimal installs may already include it; if not, install once and keep the keyring under `~/.gnupg` backed up and mode-restricted.

```
sudo apt update
sudo apt install gnupg
gpg --version
```

Syntax

```
gpg [options] command [args]
gpg -c file                    # symmetric encrypt
gpg -e -r recipient file       # public-key encrypt
gpg -d file.gpg                # decrypt
gpg -b --armor file            # detached ASCII signature
gpg --verify sig data          # verify signature
```

Common Options

Option	Description
`-c`, `--symmetric`	Encrypt with a passphrase (AES-256 default on modern GnuPG)
`-e`, `--encrypt`	Encrypt to one or more public keys
`-d`, `--decrypt`	Decrypt to stdout (or `-o`); also verifies an inner signature
`-s`, `--sign`	Sign with your private key (combinable with encrypt)
`-b`, `--detach-sign`	Create a detached signature (preferred for releases)
`-a`, `--armor`	ASCII-armor output (`.asc` instead of binary `.gpg`/`.sig`)

Option	Description
`-r`, `--recipient`	Recipient user-id / email / fingerprint for `-e`
`-u`, `--local-user`	Select which private key signs
`-o`, `--output`	Output path (avoid overwriting sources by accident)
`-k`, `--list-keys`	List public keys
`-K`, `--list-secret-keys`	List secret keys
`--fingerprint`	List keys with fingerprints (compare out-of-band)
`--import` / `--export`	Import or export keys
`--full-generate-key`	Interactive key generation (all options)
`--batch`	Non-interactive / script-friendly mode
`--yes`	Assume yes on prompts (use carefully)

Safety

- **Private keys and passphrases** never belong in tickets, chat, or world-readable paths. Keep `~/.gnupg` at `700` and secret key material at `600`.
- Encrypting to the wrong recipient is silent until decrypt fails elsewhere — always confirm **fingerprint** over a second channel before trusting an imported key.
- `gpg --verify` reporting "Good signature" with trust `[unknown]` only means the crypto matched **some** key in your keyring — not that you have identified the signer. Compare fingerprints, then assign ownertrust if you rely on that key regularly.
- Symmetric encryption (`-c`) caches the passphrase in `gpg-agent` briefly; use `--no-symkey-cache` when sharing a console.
- Never email or paste `--export-secret-keys` output. Paper or offline encrypted backups only.
- For unattended verify-only workflows, prefer `gpgv` with an explicit `--keyring` of keys you already trust.

Key Use Cases

1. Symmetric encrypt a config or backup tarball for transport.
2. Generate or import a key, list fingerprints, exchange public keys.
3. Encrypt a file to a colleague's public key; decrypt inbound mail attachments.
4. Detached-sign a release artifact; verify a vendor's signature.
5. Verify Ubuntu/ISO `SHA256SUMS` + `.gpg` before installing from a mirror.

Examples with Explanations

Example: install and confirm the toolchain

```
sudo apt update
sudo apt install -y gnupg
gpg --version
# gpg (GnuPG) 2.4.x on Ubuntu 24.04 (noble); 2.2.x on 22.04 (jammy)
```

Ubuntu's `gnupg` meta-package pulls in `gpg`, `gpg-agent`, and related helpers. Re-check after major upgrades if scripts assume specific default ciphers.

Example: generate a key pair (interactive)

```
gpg --full-generate-key
# Choose: (1) RSA and RSA, 3072+ bits - or the offered ECC default on newer GnuPG
# Real name / email / comment → passphrase → wait for entropy
```

This walks through algorithm, size/curve, expiry, and user-id. A revocation certificate is written under `~/.gnupg/openpgp-revocs.d/` — back that up offline. For fully scripted labs, prefer `gpg --batch --passphrase … --quick-generate-key` (see Notes).

Example: list keys and show fingerprints

```
gpg --list-secret-keys --keyid-format=long
gpg --list-keys --keyid-format=long
gpg --fingerprint alice@example.com
```

Fingerprints are the 40-hex identity you compare on a call or printed card before trusting a key. Prefer fingerprint (or full fingerprint) over short key IDs — short IDs are forgeable.

Example: export and import a public key

```
gpg --armor --export alice@example.com > alice-public.asc
# hand alice-public.asc to the peer over any channel; authenticity comes from fingerprint ch

gpg --import bob-public.asc
gpg --fingerprint bob@example.com
# Compare every hex group with Bob over a second channel before encrypting to him
```

`--armor` (`-a`) produces mail-safe ASCII. Import alone does **not** mean "trusted" — only that the key is available for encrypt/verify once you decide it is the right one.

Example: symmetric encrypt and decrypt a file

```
gpg --symmetric --armor secrets.env
# prompts for passphrase twice → writes secrets.env.asc

gpg --decrypt --output secrets.env secrets.env.asc
# or: gpg -d -o secrets.env secrets.env.asc
chmod 600 secrets.env
```

Symmetric mode needs no key pair — good for "encrypt this tarball for myself on another host." Prefer `--armor` when the ciphertext must survive email or paste; omit it for large binaries (smaller `.gpg`). Always write decrypt output with `-o` to a controlled path rather than relying on embedded filenames.

Example: public-key encrypt and decrypt

```
# Encrypt for Bob (his public key must already be imported)
gpg --encrypt --recipient bob@example.com --armor report.tar.gz
# → report.tar.gz.asc

# Bob decrypts with his private key
gpg --decrypt --output report.tar.gz report.tar.gz.asc
```

Anyone with Bob's public key can encrypt; only Bob's private key decrypts. Combine with signing when you also need authenticity: `gpg -s -e -r bob@example.com file`.

Example: detached sign and verify (ASCII)

```
gpg --detach-sign --armor release.tar.gz
# → release.tar.gz.asc   (signature only; original file unchanged)

gpg --verify release.tar.gz.asc release.tar.gz
# gpg: Good signature from "Alice <alice@example.com>"
```

Detached signatures are the release-engineering default: ship `artifact` + `artifact.asc` (or `.sig`). Always pass **both** the signature and the data to `--verify`; do not rely on suffix guessing. Cleartext (`--clear-sign`) is for human-readable messages, not binary releases.

Example: verify a signed checksum list (download integrity)

```
# Typical Ubuntu Server ISO layout next to the image:
#   SHA256SUMS          - published digests
#   SHA256SUMS.gpg      - detached signature over that list

gpg --keyid-format long --verify SHA256SUMS.gpg SHA256SUMS
# If the signing key is missing, fetch from a trusted keyserver or keyring, then:
#   gpg --keyserver hkps://keyserver.ubuntu.com --recv-keys <FINGERPRINT>
# Re-run verify, then confirm the fingerprint matches Ubuntu's published Image Signing key.

sha256sum -c --ignore-missing SHA256SUMS
```

Workflow: (1) verify the signature on `SHA256SUMS`, (2) only then run `sha256sum -c`. A matching hash without a verified signature only proves integrity against the list you downloaded — not that Canonical (or the vendor) published that list. On a stock Ubuntu system, `ubuntu-keyring` / archive keyrings often already hold the CD Image signing keys; use `gpgv --keyring /usr/share/keyrings/…` when you want verify-only against a pinned keyring.

Example: armor / dearmor a keyring blob for apt-style keyrings

```
# Convert ASCII public key material to a binary keyring file (common for /usr/share/keyrings
curl -fsSL https://example.com/project.asc | sudo gpg --dearmor -o /usr/share/keyrings/proje
sudo chmod 644 /usr/share/keyrings/project.gpg
```

`gpg --dearmor` / `--enarmor` pack or unpack OpenPGP ASCII armor. Modern `apt` sources often point at a `.gpg` keyring under `/usr/share/keyrings/` rather than the legacy trusted.gpg keyring — keep permissions non-secret (`644`) for public keys.

Understanding Output

Message / field	Meaning
`Good signature from "…"`	Cryptographic match to a key you have; check trust/fingerprint next
`BAD signature`	Data or signature was altered, or wrong key — do not trust the file
`Can't check signature: No public key`	Import the signer's key (after fingerprint check) and retry
`[unknown]` trust	Key is present but you have not assigned ownertrust / certified it

Message / field	Meaning
`sec` / `ssb` vs `pub` / `sub`	Secret primary/subkey vs public primary/subkey in `--list-*` output
`#` after `sec`/`ssb`	Secret key not currently usable (e.g. offline / stub)

Exit status is non-zero on failed verify/decrypt — usable in scripts with `set -e` or `gpg --verify … \|\| exit 1`. For machine parsing of key lists, use `--with-colons` (human `--list-keys` format is not stable).

Notes & Pitfalls

- **Short key IDs are unsafe** for identification; always use full fingerprints.
- `gpg` and `gpg2` on older docs usually mean the same GnuPG 2.x binary named `gpg` on current Ubuntu.
- Default symmetric cipher is AES-256 on modern GnuPG; do not weaken it with casual `--cipher-algo` experiments.
- Agent caching: decrypt/sign may not re-prompt immediately — expected with `gpg-agent`.
- Batch/CI keygen needs `--batch`, explicit algorithm, and a controlled passphrase source; interactive `--full-generate-key` is for humans.
- macOS/Homebrew and BusyBox environments differ; this page assumes GNU userland on Ubuntu Server.
- Encrypting a directory: archive first (`tar czf - dir | gpg -c -o dir.tgz.gpg`), then decrypt/extract in reverse — `gpg` operates on files/streams, not trees.

Related Commands

- `gpgv` — verify-only tool against a trusted keyring (good for scripts)
- `sha256sum` — check digests after the signature on the checksum file is verified
- `openssl` — TLS/cert inspection; different problem space than OpenPGP file crypto
- `ssh-keygen` — SSH identities (not interchangeable with OpenPGP keys)
- `apt-key` — **deprecated**; use `signed-by=` + `/usr/share/keyrings/*.gpg` instead

Additional Resources

- `man gpg` (Ubuntu jammy/noble)
- GnuPG operational commands
- Ubuntu image integrity verification
- Ubuntu: set up and manage PGP keys

age

Overview

`age` is a **simple, modern file encryption** tool: small explicit keys, no config file, and UNIX-style pipes. Operators use it to encrypt backups, secrets, and tarballs for transport without the ceremony of OpenPGP. Prefer **`age`** when you want "encrypt this file to these recipients and decrypt later with an identity file." Prefer **`gpg`** when you need signatures, web-of-trust workflows, or verifying vendor-signed checksum lists (see Related). Prefer **`openssl`** for TLS cert triage — different problem space.

On Ubuntu 22.04/24.04 Server, install from universe:

```
sudo apt update
sudo apt install age
age --version
age-keygen --version
```

Jammy ships age **1.0.x**; Noble ships **1.1.x**. Both cover the classic X25519 workflow below. Newer upstream builds also offer post-quantum hybrid keys (`age-keygen -pq`); check `age-keygen --help` before relying on `-pq` on an older package.

Syntax

```
age-keygen [-o OUTPUT]                  # generate identity (+ print recipient)
age-keygen -y [-o OUTPUT] [IDENTITY]    # identity → recipient(s)

age [-e] (-r RECIPIENT | -R PATH)... [-a] [-o OUTPUT] [INPUT]
age [-e] -p [-a] [-o OUTPUT] [INPUT]    # passphrase (interactive)
age -d [-i IDENTITY]... [-o OUTPUT] [INPUT]
```

Common Options

Option	Description
`-r`, `--recipient`	Encrypt to one recipient string (`age1…`, or `ssh-ed25519` / `ssh-rsa …`)

Option	Description
`-R`, `--recipients-file`	Encrypt to recipients listed in a file (one per line; `#` comments OK)
`-i`, `--identity`	Decrypt with an identity file (native age secret, passphrase-wrapped identity, or SSH private key)
`-d`, `--decrypt`	Decrypt mode
`-p`, `--passphrase`	Encrypt with an interactive passphrase (not combinable with `-r`/`-R`)
`-a`, `--armor`	ASCII-armor ciphertext (PEM-like `AGE ENCRYPTED FILE`)
`-o`, `--output`	Output path (overwrites if present)
`-e`, `--encrypt`	Explicit encrypt mode (required when encrypting with `-i`)
`-y` (keygen)	Convert an identity file to its public recipient string(s)
`-pq` (keygen)	Generate a post-quantum hybrid ML-KEM-768 + X25519 identity (newer age only)

Safety

- **Identity files are private keys.** Mode `600`, directory not world-readable, never commit to git or paste into tickets.
- Encrypting to the **wrong recipient** succeeds locally — only the matching identity can decrypt. Confirm the `age1…` string (or SSH fingerprint) out-of-band before shipping secrets.
- `-o` **overwrites** an existing output path without prompting.
- Passphrase mode (`-p`) is for "encrypt for myself with a memorable secret." Do not combine mental models: a passphrase-encrypted file is not the same as a passphrase-**protected identity** file.
- SSH recipients are a **convenience**. Prefer native age keys for long-term encryption; SSH keys are often rotated for auth and may live on hardware tokens / `ssh-agent` (neither is supported for decrypt).
- Do **not** mix classic (`age1…`) and post-quantum (`age1pq1…`) recipients on one ciphertext — that would defeat the PQ guarantee. Newer age rejects that combination.
- Partial decrypt failures never release unauthenticated plaintext, but a failed run can still leave a truncated `-o` file you created — verify exit status before trusting outputs.

Key Use Cases

1. Generate a key pair; encrypt a file or stream to your public recipient; decrypt with the identity.
2. Encrypt one artifact to several teammates via `-r` / `-R`.
3. Armor ciphertext for email or paste (`-a`).
4. Reuse an existing Ed25519/RSA SSH public key as a recipient when the peer has no age key yet.
5. Pipe `tar` into `age` for encrypted directory archives.

Examples with Explanations

Example: install and confirm the toolchain

```
sudo apt update
sudo apt install -y age
age --version
age-keygen --version
# Ubuntu 22.04 (jammy): typically age 1.0.x from universe
# Ubuntu 24.04 (noble): typically age 1.1.x from universe
```

If `apt` cannot find the package, enable the **universe** component and retry. For a newer upstream binary than the distro package, use the official install options — this page assumes the Ubuntu package unless noted.

Example: generate a native identity

```
umask 077
age-keygen -o ~/age-key.txt
# Public key: age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p
chmod 600 ~/age-key.txt
```

`age-keygen` writes the **identity** (secret) to the file and prints the **recipient** (public) to stderr when not writing to a TTY. The identity file looks like:

```
# created: 2024-01-02T15:30:45+01:00
# public key: age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p
AGE-SECRET-KEY-1...
```

Back up the identity offline. Share only the `age1…` public string.

On age builds that support it, prefer post-quantum hybrid keys for new long-lived identities:

```
age-keygen -pq -o ~/age-key-pq.txt
# Public key: age1pq1…   (much longer than classic age1…)
# Identity line: AGE-SECRET-KEY-PQ-1…
```

Example: identity → recipient string

```
age-keygen -y ~/age-key.txt
# age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p
```

Use this when you have the identity file and need the public string for a recipients list or a colleague's `-r` flag — without opening the file and copying by hand.

Example: encrypt and decrypt a file (native recipient)

```
RECIPIENT="$(age-keygen -y ~/age-key.txt)"
age -r "$RECIPIENT" -o secrets.env.age secrets.env
# or armor for mail/paste:
age -a -r "$RECIPIENT" -o secrets.env.age secrets.env

age -d -i ~/age-key.txt -o secrets.env secrets.env.age
chmod 600 secrets.env
```

Anyone with the recipient can encrypt; only the matching identity decrypts. Armor (`-a`) is detected automatically on decrypt — you do not pass `-a` to `-d`.

Example: multiple recipients and a recipients file

```
age -o report.tar.gz.age \
  -r age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p \
  -r age1zggyaq84g3u5l9hc1amf5eenc9h66ysyzx9h4j9un9au0z9wl34jsj5r3t \
  report.tar.gz

cat > recipients.txt <<'EOF'
# Alice - ops laptop
age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p
# Bob - backup host
age1zggyaq84g3u5l9hc1amf5eenc9h66ysyzx9h4j9un9au0z9wl34jsj5r3t
EOF
age -R recipients.txt -o report.tar.gz.age report.tar.gz
```

Every listed recipient can decrypt independently. Keep `recipients.txt` as ordinary public data; never put `AGE-SECRET-KEY-…` lines there.

Example: passphrase encrypt (no key file)

```
age -p -o notes.txt.age notes.txt
# Enter passphrase (leave empty to autogenerate a secure one):

age -d -o notes.txt notes.txt.age
# Enter passphrase:
```

Good for ad-hoc “encrypt this for myself on another machine” without managing identity files. For automation and multi-recipient sharing, use native keys instead.

Example: passphrase-protected identity file

```
# Wrap a newly generated identity with a passphrase
age-keygen | age -p -o ~/age-key.age
# note the Public key: age1… line from stderr - store that as the recipient

age -r age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p \
  -o secrets.env.age secrets.env

age -d -i ~/age-key.age -o secrets.env secrets.env.age
# Enter passphrase for identity file "…/age-key.age":
```

The identity file itself is an age-encrypted blob of `AGE-SECRET-KEY-…` lines. Decrypting data prompts for that passphrase only when this identity is needed. For most single-user laptops, a plain `600` identity file is enough; wrap it when the identity might sit on shared or less-trusted storage.

Example: encrypt to an SSH public key

```
age -R ~/.ssh/id_ed25519.pub -o example.jpg.age example.jpg
age -d -i ~/.ssh/id_ed25519 -o example.jpg example.jpg.age
```

Supported SSH types: **Ed25519** and **RSA 2048**. age ignores unsupported keys in a `-R` file with a warning (handy with `authorized_keys` or `https://github.com/<user>.keys`). Decrypt needs the **private key file** on disk — not `ssh-agent`, not a YubiKey-held SSH key.

Example: compose with tar (directory archive)

```
RECIPIENT="$(age-keygen -y ~/age-key.txt)"
tar czf - /var/backups/app | age -r "$RECIPIENT" > app-backup.tar.gz.age

age -d -i ~/age-key.txt app-backup.tar.gz.age | tar tzf - | head
age -d -i ~/age-key.txt -o app-backup.tar.gz app-backup.tar.gz.age
tar xzf app-backup.tar.gz -C /restore/path
```

`age` encrypts a **single stream or file**, not a directory tree. Always archive first (`tar`/`cpio`), then encrypt; reverse the order on restore. Binary ciphertext through a pipe needs no `-a`.

Example: encrypt using an identity file's matching recipient

```
# Explicit -e required when -i is used for encryption
age -e -i ~/age-key.txt -o self.age secrets.env
age -d -i ~/age-key.txt -o secrets.env self.age
```

Equivalent to converting the identity with `age-keygen -y` and passing `-R`. Useful in scripts that only keep the identity path on hand.

Understanding Output

Item	Meaning
`Public key: age1…` / `age1pq1…`	Recipient string — safe to share / put in `-r` / recipients files
`AGE-SECRET-KEY-1…` / `AGE-SECRET-KEY-PQ-1…`	Identity — secret; stays in the key file
`.age` ciphertext	Binary by default; small overhead per recipient plus streaming AEAD
Armored blob	PEM-like with type `AGE ENCRYPTED FILE`; decrypt auto-detects
Exit status `0`	Full encrypt/decrypt succeeded; non-zero → do not trust partial `-o`

There is no "Good signature" story — age is encryption (and AEAD integrity of the ciphertext), not a signing tool. For authenticity of downloads, use `gpg --verify` on vendor signatures.

Notes & Pitfalls

- **age vs gpg:** age wins on simple file encryption UX and scripting. gpg wins on signatures, keyservers, and Ubuntu ISO / `SHA256SUMS.gpg` verification workflows.
- Ubuntu package versions lag upstream; confirm `age-keygen --help` for `-pq` before documenting PQ keys in your runbooks.
- Binary age output to a TTY is refused unless you force `-o -` — prefer a file or a pipe.
- `-R -` reads recipients from stdin; then you **must** pass an INPUT filename (stdin is already used).
- Do not put identity material in `recipients.txt`. Comments (`#`) and blank lines are fine in recipient lists.
- Plugins (for example YubiKey) use `age1…plugin…` recipients and matching `AGE-PLUGIN-…` identities; install the plugin binary on `PATH` when you decrypt.
- macOS Homebrew and distro packages are fine; this page's install path is Ubuntu `apt`.

Related Commands

- `gpg` — OpenPGP encrypt/sign/verify; use for signed checksums and existing PGP workflows
- `openssl` — TLS/cert inspection; not a replacement for age file encryption
- `ssh-keygen` — create SSH keys you may reuse as age recipients
- `tar` / `gzip` — archive before encrypting directories
- `sha256sum` — integrity of plaintext or of published digests (after signature verify with gpg)

Additional Resources

- `man age` / `man age-keygen` (Ubuntu jammy/noble)
- age README and install matrix
- age format specification
- Debian/Ubuntu age(1) man page

io_uring and seccomp: filters per opcode

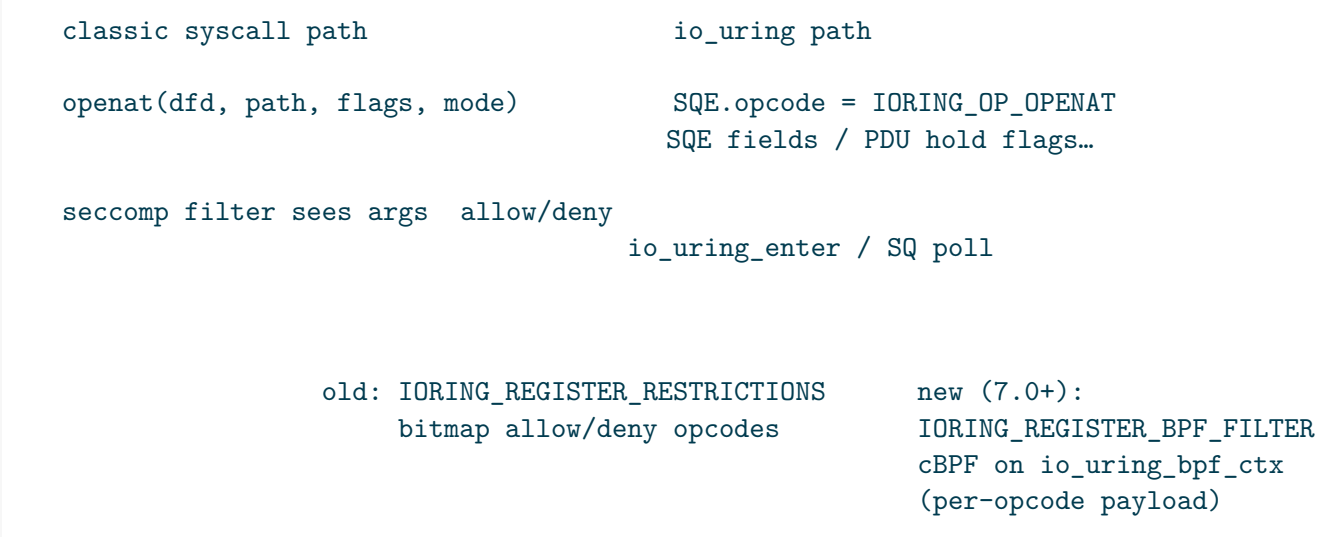
io_uring and seccomp: filters per opcode

Overview

Thesis: classic **seccomp** sees system-call arguments on the syscall ABI. **io_uring** puts the interesting arguments in the **submission-queue entry (SQE)** shared with the kernel. A single `io_uring_enter(2)` (or a registered ring) can perform open, socket, connect, read, write, … without those operations appearing as separate syscalls. Containers that only block `io_uring_setup` therefore face a blunt choice — allow the whole ring or deny it. Linux **7.0+** adds **classic BPF (cBPF) filters per io_uring opcode**, so a policy can inspect opcode-specific fields (open flags, socket family, …) and deny with `-EACCES`. This chapter teaches that model for operators and sandbox authors. It is not an exploit guide and does not build custom kernels.

Companion chapter: `11-storage-and-filesystems/25-io-uring.md` covers rings, CQEs, and `kernel.io_uring_disabled`. Here the focus is **policy**.

Mental model



Mechanism	Granularity	Sees open flags / socket family?	Kernel
seccomp on `io_uring_setup` / `io_uring_enter`	Whole ring / enter	No (args are in the SQ, not in enter's registers)	long-standing

Mechanism	Granularity	Sees open flags / socket family?	Kernel
`IORING_REGISTER_RESTRICTIONS`	Opcode / register-op allowlist	No — opcode only	5.10+
`IORING_REGISTER_BPF_FILTER`	Per-opcode BPF	Yes — via `io_uring_bpf_ctx` PDU	**7.0+**
`kernel.io_uring_disabled` sysctl	Host-wide kill switch	N/A	6.6+

Return convention for BPF filters: **non-zero allow**, **zero deny** → completion `-EACCES`. Multiple filters for one opcode must **all** allow. Task-scoped filters inherit to children and can only be tightened.

Syntax

Coarse allowlist (widely available): restrictions

```
io_uring_queue_init(entries, &ring, IORING_SETUP_R_DISABLED);
io_uring_register_restrictions(&ring, restrictions, nr);
io_uring_enable_rings(&ring);
```

`IORING_RESTRICTION_SQE_OP` entries name permitted opcodes. After enable, other opcodes fail with permission errors.

Fine-grained (Linux 7.0+ / liburing helpers)

```
#include <liburing.h>
#include <liburing/io_uring/bpf_filter.h>
#include <linux/filter.h>
#include <sys/prctl.h>

prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);   /* or hold CAP_SYS_ADMIN */

struct io_uring_bpf bpf = {
    .cmd_type = IO_URING_BPF_CMD_FILTER,
    .filter = {
        .opcode     = IORING_OP_SOCKET,   /* or OPENAT, NOP, … */
        .filter_len = N,
        .filter_ptr = (unsigned long) prog,  /* struct sock_filter[N] */
        .pdu_size   = 12,                 /* SOCKET: family/type/protocol */
        /* .flags = IO_URING_BPF_FILTER_DENY_REST | IO_URING_BPF_FILTER_SZ_STRICT */
    },
};
io_uring_register_bpf_filter(&ring, &bpf);       /* this ring only */
```

```
/* or */ io_uring_register_bpf_filter_task(&bpf); /* future rings + children */
```

Context the filter loads with `BPF_LD|BPF_W|BPF_ABS` (layout from `io_uring_register_bpf_filter(3)`):

Offset	Field	Notes
0	`user_data`	From the SQE
8	`opcode` / `sqe_flags` / `pdu_size`	Packed bytes
16+	PDU union	`socket.{family,type,protocol}` (12 bytes) or `open.{flags,mode,resolve}` (24 bytes)

`pdu_size` must match (or be) the kernel's expectation unless you set `IO_URING_BPF_FILTER_SZ_STRICT`. On `-EMSGSIZE` the kernel writes back the expected size so you can retry.

Safety

- **Teaching only.** Do not use these APIs to probe or bypass production sandboxes you do not own. Registering filters requires `CAP_SYS_ADMIN` or `no_new_privs` for the same reason seccomp does: stop an unprivileged process from installing a filter and then execing a setuid helper.
- Task-level filters **cannot be removed**; children cannot loosen a parent's policy. Prefer allowlists (`IO_URING_BPF_FILTER_DENY_REST`) over "deny one bad opcode and hope."
- Restrictions and BPF filters are **complementary** to `kernel.io_uring_disabled` and LSM/container profiles — not replacements for patching the kernel attack surface.
- This chapter never presents a privilege-escalation path, a container breakout recipe, or a kernel build.

Examples with Explanations

Tested on **Linux 6.12.94+** with **liburing 2.9** and GCC. That kernel **does not** implement `IORING_REGISTER_BPF_FILTER` (7.0+). Example 1 therefore runs for real. Examples 2–3 are **documentation listings** from the liburing manual, for hosts that have the feature.

1. Opcode allowlist with `IORING_REGISTER_RESTRICTIONS` (runs here)

```
/* uring_restrict_demo.c - only IORING_OP_NOP is allowed; READ returns -EACCES */
#include <liburing.h>
#include <linux/io_uring.h>
#include <stdio.h>
#include <string.h>
```

```c
#include <unistd.h>

int main(void) {
    struct io_uring ring;
    int ret = io_uring_queue_init(8, &ring, IORING_SETUP_R_DISABLED);
    if (ret < 0) {
        fprintf(stderr, "queue_init: %s\n", strerror(-ret));
        return 1;
    }

    struct io_uring_restriction res[1];
    memset(res, 0, sizeof(res));
    res[0].opcode = IORING_RESTRICTION_SQE_OP;
    res[0].sqe_op = IORING_OP_NOP;

    ret = io_uring_register_restrictions(&ring, res, 1);
    if (ret < 0) {
        fprintf(stderr, "register_restrictions: %s (%d)\n", strerror(-ret), ret);
        return 1;
    }
    ret = io_uring_enable_rings(&ring);
    if (ret < 0) {
        fprintf(stderr, "enable_rings: %s (%d)\n", strerror(-ret), ret);
        return 1;
    }
    printf("restrictions locked: only IORING_OP_NOP allowed\n");

    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
    io_uring_prep_nop(sqe);
    io_uring_sqe_set_data64(sqe, 1);
    printf("submit+wait NOP: ret=%d\n", io_uring_submit_and_wait(&ring, 1));

    struct io_uring_cqe *cqe;
    io_uring_wait_cqe(&ring, &cqe);
    printf("NOP cqe res=%d user_data=%llu\n", cqe->res,
           (unsigned long long)cqe->user_data);
    io_uring_cqe_seen(&ring, cqe);

    char buf[16];
    sqe = io_uring_get_sqe(&ring);
    io_uring_prep_read(sqe, STDIN_FILENO, buf, sizeof(buf), 0);
    io_uring_sqe_set_data64(sqe, 2);
    printf("submit+wait READ: ret=%d\n", io_uring_submit_and_wait(&ring, 1));
    io_uring_wait_cqe(&ring, &cqe);
    printf("READ cqe res=%d (%s) user_data=%llu\n", cqe->res,
           cqe->res < 0 ? strerror(-cqe->res) : "ok",
           (unsigned long long)cqe->user_data);
```

```
    io_uring_cqe_seen(&ring, cqe);

    io_uring_queue_exit(&ring);
    return 0;
}
```

```
gcc -O2 -o uring_restrict_demo uring_restrict_demo.c -luring
./uring_restrict_demo
```

```
restrictions locked: only IORING_OP_NOP allowed
submit+wait NOP: ret=1
NOP cqe res=0 user_data=1
submit+wait READ: ret=1
READ cqe res=-13 (Permission denied) user_data=2
```

`-13` is `-EACCES`: the ring accepted the submit, but the restricted opcode was denied. That is the coarse ancestor of per-opcode BPF filters — still blind to *which* file a hypothetical `OPENAT` would open.

Confirm the host policy knob while you are here:

```
uname -r
cat /proc/sys/kernel/io_uring_disabled
```

```
6.12.94+
0
```

`0` means rings may be created. `2` would refuse new rings entirely (existing rings keep working).

2. Allow only `AF_INET` sockets (docs listing — needs Linux 7.0+)

From `io_uring_register_bpf_filter(3)` — illustrative; not executed on 6.12:

```
#define CTX_OFF_SOCKET_FAMILY  16

struct sock_filter inet_only_filter[] = {
    BPF_STMT(BPF_LD | BPF_W | BPF_ABS, CTX_OFF_SOCKET_FAMILY),
    BPF_JUMP(BPF_JMP | BPF_JEQ | BPF_K, AF_INET, 0, 1),
    BPF_STMT(BPF_RET | BPF_K, 1),  /* allow */
    BPF_STMT(BPF_RET | BPF_K, 0),  /* deny  */
};

struct io_uring_bpf bpf = {
    .cmd_type = IO_URING_BPF_CMD_FILTER,
    .filter = {
        .opcode = IORING_OP_SOCKET,
```

```
        .filter_len = 4,
        .filter_ptr = (unsigned long) inet_only_filter,
        .pdu_size = 12,
    },
};
prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0);
io_uring_register_bpf_filter_task(&bpf);
```

An `IORING_OP_SOCKET` with another family completes with `-EACCES`. Open filters use `pdu_size = 24` and load `flags` / `resolve` at offsets 16 and 32.

3. Allowlist pattern: allow NOP, deny everything else

```
struct sock_filter allow_filter[] = {
    BPF_STMT(BPF_RET | BPF_K, 1),
};
struct io_uring_bpf bpf = {
    .cmd_type = IO_URING_BPF_CMD_FILTER,
    .filter = {
        .opcode = IORING_OP_NOP,
        .flags = IO_URING_BPF_FILTER_DENY_REST,
        .filter_len = 1,
        .filter_ptr = (unsigned long) allow_filter,
    },
};
```

`DENY_REST` fills every other opcode slot with a deny stub — the BPF analogue of Example 1's single-opcode restrictions list.

The trap

Trap: "seccomp on `io_uring_enter` can see open flags." Enter's arguments are the ring fd and flags for the enter call. Open flags live in the SQE. Without restrictions or BPF filters you are filtering the wrong layer.

Trap: "block `io_uring_setup` and you are done forever." That is a valid hard disable (and so is `io_uring_disabled=2`), but it also kills legitimate high-performance I/O. Prefer opcode allowlists when you need io_uring at all.

Trap: "run the BPF examples on any recent kernel." Check `uname -r` and the presence of `IORING_REGISTER_BPF_FILTER` in your `linux/io_uring.h`. On 6.12 this box, registration is unavailable; use restrictions or upgrade.

The boring rule

If arguments matter for policy, filter **where the arguments live**. For io_uring that means opcode restrictions (coarse) or per-opcode cBPF on `io_uring_bpf_ctx` (fine, 7.0+), plus `no_new_privs` / capabilities, not a hope that seccomp saw the SQE fields on `enter`.

Try this

1. Run Example 1; then remove the restrictions registration and confirm READ succeeds (Ctrl-D for EOF on stdin).
2. On a 7.0+ lab kernel with matching liburing, run the man-page `AF_INET`-only filter against `IORING_OP_SOCKET` and confirm non-INET families get `-EACCES`.
3. Read your container runtime's seccomp profile: does it kill `io_uring_setup`, allow it fully, or (on new stacks) expect BPF filters?

Verification

- Example 1 compiled and run on this box: Linux **6.12.94+**, liburing **2.9**, `io_uring_disabled=0`. Output pasted above.
- BPF filter API described from `io_uring_register_bpf_filter(3)` / `io_uring_register_bpf_filter_tas` (liburing 2.14 docs) and the upstream `include/uapi/linux/io_uring/bpf_filter.h` (merged for 7.0).
- **No** kernel builds, **no** package installs, **no** exploit PoCs. BPF examples are documentation-only on this host.

Sources

- man7 — `io_uring_register_bpf_filter(3)`, `io_uring_register_bpf_filter_task(3)`
- man7 — `io_uring_register(2)`, `seccomp(2)`
- Kernel docs — Seccomp BPF
- Upstream UAPI — `io_uring/bpf_filter.h` (7.0+)
- Jens Axboe — io_uring BPF filter patch series (lore, Jan 2026)

Landlock from the command line: setpriv

Landlock from the command line: setpriv

Overview

Thesis: Landlock is a stackable Linux Security Module that lets an ordinary process impose an extra **allow-list** on itself and its future children: filesystem actions on path hierarchies, and (from ABI 4) TCP bind/connect on port numbers. **`setpriv --landlock-access / --landlock-rule`** (util-linux **2.41**) is the command-line front end for the filesystem half. Handled rights are denied by default; rules grant selected rights back. The domain is **best-effort across ABI versions**, and it **only subtracts** from what DAC, capabilities, and other LSMs already allow.

This chapter is the operator view: probe the ABI, sandbox a command with `setpriv`, and drive TCP policy through the same three syscalls `setpriv` wraps. It is not a container runtime, and it does not replace user namespaces (`unshare`).

Companion pages in this book: `15-services-and-runtime/12-unshare.md` (namespaces), `07-io-uring-seccomp.md` (seccomp / io_uring policy), `04-getcap.md` (file capabilities).

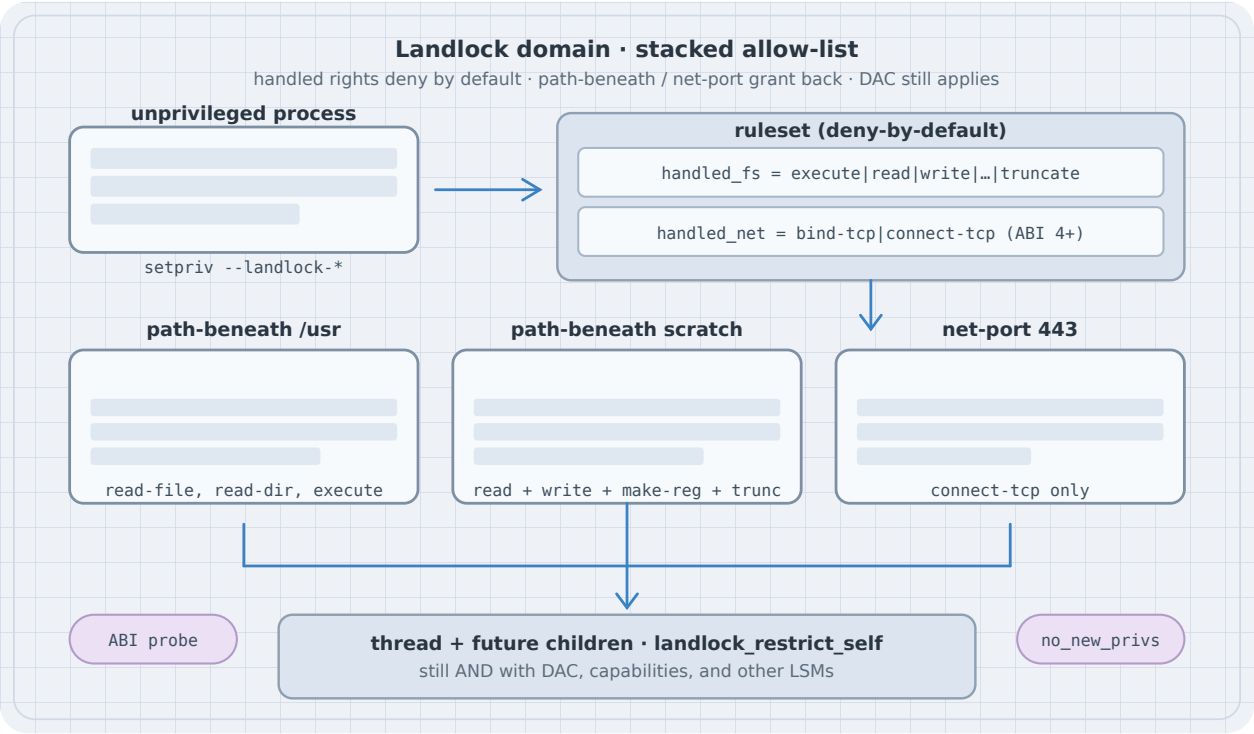


Figure 1: Landlock domain as a stacked allow-list

Mental model

```
unprivileged process

            1. landlock_create_ruleset(handled_access_*)
                 handled bits = "these actions are now deny-by-default"
            2. landlock_add_rule()  (optional, repeatable)
                 path-beneath /usr      → read-file, read-dir, execute
                 path-beneath scratch   → read + write + make-reg + truncate
                 net-port 443           → connect-tcp          (ABI 4+)
            3. prctl(PR_SET_NO_NEW_PRIVS)     (required without CAP_SYS_ADMIN)
            4. landlock_restrict_self()
                 policy sticks on this thread and every future child

access is allowed only if EVERY layer agrees:
     Landlock domain     DAC (mode/owner)     other LSMs     capabilities
```

A **ruleset** names the actions it *handles*. Anything handled and not granted by a rule is denied (`EACCES` for filesystem and TCP; `EPERM` for scoped IPC). Anything *not* handled is left to the rest of the system — that is the compatibility contract: a newer kernel will not suddenly start denying actions the program never listed.

`setpriv` does steps 1–4 then `execve`s the program. util-linux **2.41.5** implements filesystem rules (`Access: fs`, `path-beneath`, rights through `truncate`). TCP, `ioctl_dev`, and IPC scopes exist in the kernel on this host (ABI 6) and are driven below with the raw syscalls. Check `setpriv --help` on your build: later util-linux documents `net:connect-tcp` and `net-port:connect-tcp:PORT` in the same flag pair.

ABI	Mainline kernel (orientation)	What it added
1	5.13	Filesystem rights: execute, read/write file, read dir, remove, make-*
2	5.19	`LANDLOCK_ACCESS_FS_REFER` (cross-directory rename/link)
3	6.2	`LANDLOCK_ACCESS_FS_TRUNCATE`
4	6.7	`LANDLOCK_ACCESS_NET_BIND_TCP`, `CONNECT_TCP`
5	6.10	`LANDLOCK_ACCESS_FS_IOCTL_DEV`
6	6.12	`LANDLOCK_SCOPE_ABSTRACT_UNIX_SOCKET`, `LANDLOCK_SCOPE_SIGNAL`
7	6.15	Audit / `LANDLOCK_RESTRICT_SELF_LOG_*` (observability)
8	7.0	`LANDLOCK_RESTRICT_SELF_TSYNC` (process-wide enforce)
9	7.1	`LANDLOCK_ACCESS_FS_RESOLVE_UNIX`

ABI	Mainline kernel (orientation)	What it added
10	7.2	UDP bind/connect-send; per-object quiet/audit flags

Kernel versions are orientation only. Distros backport. **Always probe the ABI.** This box is Linux **6.12.94+** and reports **ABI 6**: filesystem through `ioctl_dev`, TCP, and IPC scopes. Audit flags, TSYNC, pathname UNIX resolve, and UDP need a newer ABI than this kernel.

Syntax

```
setpriv --landlock-access fs[:right,right,...] \
        --landlock-rule path-beneath:right,right,...:/path \
        [further --landlock-rule ...] \
        program [args...]
```

Piece	Meaning
`--landlock-access fs`	Handle every filesystem right this `setpriv` was built with (2.41.5: through `truncate`). Denied unless a rule grants it.
`--landlock-access fs:truncate,write-file`	Handle only those rights. Unlisted actions stay unrestricted by this ruleset.
`--landlock-rule path-beneath:ACCESS:/PATH`	Allow `ACCESS` on the hierarchy under `PATH` (opened `O_PATH`). Repeatable.
`--no-new-privs`	`setpriv` already sets `no_new_privs` when it applies Landlock. Useful on its own for other drops.

`setpriv --help` prints the rights and rule types the binary actually implements:

```
Landlock accesses:
 Access: fs
 Rule types: path-beneath
 Rules: execute,write-file,read-file,read-dir,remove-dir,remove-file,make-char,make-dir,make
```

Kernel syscalls (what `setpriv` wraps; also the TCP path on 2.41):

```
landlock_create_ruleset(attr, size, flags)   /* flags: VERSION=1, ERRATA=2 */
landlock_add_rule(fd, PATH_BENEATH|NET_PORT, attr, 0)
landlock_restrict_self(fd, flags)
```

Safety

- Landlock is **self-restriction**. It cannot confer a right DAC withholds. A mode-000 file stays unreadable after you "allow" the directory.
- Once `landlock_restrict_self` succeeds, the policy **cannot be removed**, only stacked tighter. Children inherit it.
- Unprivileged enforce requires **`no_new_privs`**. Skipping that step returns `EPERM`. `CAP_SYS_ADMIN` can skip `no_new_privs`; leaving it unset then lets a later setuid/file-cap exec become a confused deputy inside the domain.
- Files (and sockets) **already open** when the ruleset is enforced keep their previous Landlock rights. Close what you do not want the sandbox to keep.
- `setpriv --landlock-access fs` with **no rules** denies execute of the program itself (`exit 126`).
- This page does not present breakout recipes, kernel builds, or LSM bypasses.

Examples with Explanations

Tested unprivileged on **Linux 6.12.94+**, Landlock **ABI 6**, **util-linux 2.41.5**, Python **3.13.5**, GCC **14.2.0**. `/sys/kernel/security/lsm` is `capability,landlock,selinux`. `dmesg` contains `landlock: Up and running.`

1. Probe the ABI (runs here)

```
# landlock_abi_probe.py - landlock_create_ruleset(NULL, 0, VERSION|ERRATA)
import ctypes, os, errno

libc = ctypes.CDLL(None, use_errno=True)
libc.syscall.restype = ctypes.c_long
NR_CREATE = 444
VERSION, ERRATA = 1 << 0, 1 << 1

def create(flags):
    ctypes.set_errno(0)
    r = libc.syscall(ctypes.c_long(NR_CREATE), None, ctypes.c_size_t(0),
                     ctypes.c_uint(flags))
    return r, ctypes.get_errno()

abi, e = create(VERSION)
print(f"landlock_create_ruleset(NULL, 0, VERSION) = {abi}"
      + (f" errno={e} ({errno.errorcode.get(e)} {os.strerror(e)})" if abi < 0 else ""))

errata, e = create(ERRATA)
if errata < 0:
    print(f"ERRATA = {errata} errno={e} ({errno.errorcode.get(e)} {os.strerror(e)})")
```

```
else:
    bits = [i + 1 for i in range(64) if errata & (1 << i)]
    print(f"landlock_create_ruleset(NULL, 0, ERRATA) = {errata} "
          f"(fixed errata numbers: {bits or 'none'})")

print(f"uname: {os.uname().release}")
print(f"lsm: {open('/sys/kernel/security/lsm').read().strip()}")
```

```
python3 landlock_abi_probe.py
```

```
landlock_create_ruleset(NULL, 0, VERSION) = 6
landlock_create_ruleset(NULL, 0, ERRATA) = 7 (fixed errata numbers: [1, 2, 3])
uname: 6.12.94+
lsm: capability,landlock,selinux
```

`6` is ABI 6: `ioctl_dev` and IPC scopes are present; audit flags (ABI 7) are not. `ENOSYS` would mean the kernel was built without Landlock; `EOPNOTSUPP` would mean it is compiled in but disabled at boot (`lsm=` without `landlock`).

The errata bitmask is optional. Kernel docs recommend that almost every program **ignore** it and keep best-effort sandboxing. Bits 1–3 set means this 6.12 kernel includes the TCP-socket-identification, scoped-signal, and disconnected-directory fixes.

Confirm the LSM list and the boot banner:

```
cat /sys/kernel/security/lsm
dmesg | grep -i landlock
setpriv --version
```

```
capability,landlock,selinux
[    0.169094] LSM: initializing lsm=capability,landlock,selinux
[    0.169094] landlock: Up and running.
setpriv from util-linux 2.41.5
```

2. Read-only `/usr`, writable scratch (runs here)

Merged-usr layout: `/bin`, `/lib`, and `/lib64` are symlinks into `/usr`, so one `path-beneath` on `/usr` covers the dynamic linker and `dash`/`cat`/`python3`. `/etc` is a different hierarchy.

```
# landlock_usr_scratch.sh - RO /usr + RW /tmp/ll-scratch
mkdir -p /tmp/ll-scratch
echo hello-scratch > /tmp/ll-scratch/note.txt

RO=read-file,read-dir,execute
RW=read-file,read-dir,write-file,remove-file,make-reg,truncate,make-dir,remove-dir
```

```
setpriv --landlock-access fs \
  --landlock-rule "path-beneath:${RO}:/usr" \
  --landlock-rule "path-beneath:${RW}:/tmp/ll-scratch" \
  /usr/bin/sh -c '
echo "-- read /usr --"
/usr/bin/head -n 2 /usr/lib/os-release
echo "-- read scratch --"
/usr/bin/cat /tmp/ll-scratch/note.txt
echo landlocked-write > /tmp/ll-scratch/out.txt
echo "-- wrote scratch --"
/usr/bin/cat /tmp/ll-scratch/out.txt
echo "-- read /etc/passwd --"
/usr/bin/cat /etc/passwd
echo "-- write /tmp --"
echo nope > /tmp/ll-should-fail
'
```

```
-- read /usr --
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
-- read scratch --
hello-scratch
-- wrote scratch --
landlocked-write
-- read /etc/passwd --
cat: /etc/passwd: Permission denied
-- write /tmp --
/usr/bin/sh: 12: cannot create /tmp/ll-should-fail: Permission denied
```

`Permission denied` here is **`EACCES` (13)**. The same sandbox under Python:

```
setpriv --landlock-access fs \
  --landlock-rule "path-beneath:${RO}:/usr" \
  --landlock-rule "path-beneath:${RW}:/tmp/ll-scratch" \
  /usr/bin/python3.13 -c 'import errno
for p, mode in [("/etc/passwd", "r"), ("/usr/bin/sh", "rb"),
                ("/tmp/ll-scratch/out.txt", "a")]:
    try:
        open(p, mode)
        print(f"open({p!r}, {mode!r}): ok")
    except OSError as e:
        print(f"open({p!r}, {mode!r}): errno={e.errno} ({errno.errorcode[e.errno]}) {e.strer
'
```

```
open('/etc/passwd', 'r'): errno=13 (EACCES) Permission denied
open('/usr/bin/sh', 'rb'): ok
open('/tmp/ll-scratch/out.txt', 'a'): ok
```

With **no** rules, execute of the program is itself denied:

```
setpriv --landlock-access fs /usr/bin/true; echo exit:$?
```

```
setpriv: failed to execute /usr/bin/true: Permission denied
exit:126
```

`setpriv` sets `no_new_privs` as part of applying Landlock. A nested `setpriv --dump` (with `/proc` also allowed so the SELinux label can be read) shows `no_new_privs: 1`.

3. Landlock does not outrank DAC (runs here)

```
echo secret-data > /tmp/ll-scratch/secret-dac
chmod 000 /tmp/ll-scratch/secret-dac
python3 -c 'import errno
try:
    open("/tmp/ll-scratch/secret-dac")
except OSError as e:
    print(f"no-landlock: errno={e.errno} ({errno.errorcode[e.errno]}) {e.strerror}")
'
setpriv --landlock-access fs \
  --landlock-rule path-beneath:read-file,read-dir,execute:/usr \
  --landlock-rule path-beneath:read-file,read-dir,write-file,remove-file,make-reg,truncate,m
  /usr/bin/python3.13 -c 'import errno
try:
    open("/tmp/ll-scratch/secret-dac")
except OSError as e:
    print(f"landlock-RW: errno={e.errno} ({errno.errorcode[e.errno]}) {e.strerror}")
'
```

```
no-landlock: errno=13 (EACCES) Permission denied
landlock-RW: errno=13 (EACCES) Permission denied
```

The ruleset grants `read-file` on that hierarchy. Mode `000` still wins. Landlock is an extra deny layer.

4. Block TCP connect and bind (runs here)

util-linux **2.41.5** does not implement `net:` in `--landlock-access` (see The trap). The kernel on this host does. Same three syscalls, ABI 4+ `handled_access_net`:

```
# landlock_tcp_deny.py - handle bind-tcp|connect-tcp, add no port rules
import ctypes, os, errno, socket

libc = ctypes.CDLL(None, use_errno=True)
libc.syscall.restype = ctypes.c_long
libc.prctl.restype = ctypes.c_int
NR_CREATE, NR_ADD, NR_RESTRICT = 444, 445, 446
PR_SET_NO_NEW_PRIVS = 38
LANDLOCK_RULE_NET_PORT = 2
NET_BIND_TCP, NET_CONNECT_TCP = 1 << 0, 1 << 1

class RulesetAttr(ctypes.Structure):
    _fields_ = [("handled_access_fs", ctypes.c_uint64),
                ("handled_access_net", ctypes.c_uint64),
                ("scoped", ctypes.c_uint64)]

class NetPortAttr(ctypes.Structure):
    _fields_ = [("allowed_access", ctypes.c_uint64),
                ("port", ctypes.c_uint64)]

def must(r, what):
    if r < 0:
        e = ctypes.get_errno()
        raise OSError(e, os.strerror(e), what)
    return r

attr = RulesetAttr(0, NET_BIND_TCP | NET_CONNECT_TCP, 0)
ctypes.set_errno(0)
fd = must(libc.syscall(ctypes.c_long(NR_CREATE), ctypes.byref(attr),
                       ctypes.c_size_t(ctypes.sizeof(attr)), ctypes.c_uint(0)),
          "create_ruleset")
print(f"ruleset fd={fd} handled_access_net=bind-tcp|connect-tcp (no port allow-rules)")
ctypes.set_errno(0)
must(libc.prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0), "nnp")
ctypes.set_errno(0)
must(libc.syscall(ctypes.c_long(NR_RESTRICT), ctypes.c_int(fd), ctypes.c_uint(0)),
     "restrict")
os.close(fd)
print("restricted self")

def try_connect(host, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(3)
    try:
        s.connect((host, port))
        print(f"connect({host},{port}): ok peer={s.getpeername()}")
    except OSError as e:
```

```
        print(f"connect({host},{port}): errno={e.errno} ({errno.errorcode.get(e.errno)}) {e.
    finally:
        s.close()

def try_bind(port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        s.bind(("127.0.0.1", port))
        print(f"bind(127.0.0.1,{port}): ok {s.getsockname()}")
    except OSError as e:
        print(f"bind(127.0.0.1,{port}): errno={e.errno} ({errno.errorcode.get(e.errno)}) {e.
    finally:
        s.close()

try_connect("1.1.1.1", 443)
try_connect("127.0.0.1", 9)
try_bind(0)
try_bind(18080)
```

```
python3 landlock_tcp_deny.py
```

```
ruleset fd=5 handled_access_net=bind-tcp|connect-tcp (no port allow-rules)
restricted self
connect(1.1.1.1,443): errno=13 (EACCES) Permission denied
connect(127.0.0.1,9): errno=13 (EACCES) Permission denied
bind(127.0.0.1,0): errno=13 (EACCES) Permission denied
bind(127.0.0.1,18080): errno=13 (EACCES) Permission denied
```

UDP, UNIX sockets, and already-open descriptors are outside this ruleset. Port `0` on bind means "the ephemeral range"; handling `bind-tcp` without a port-0 rule denies that too.

5. Allow TCP connect to 443 only (runs here)

```
# landlock_tcp_allow443.py - connect-tcp allowed only to port 443
import ctypes, os, errno, socket

libc = ctypes.CDLL(None, use_errno=True)
libc.syscall.restype = ctypes.c_long
libc.prctl.restype = ctypes.c_int
NR_CREATE, NR_ADD, NR_RESTRICT = 444, 445, 446
PR_SET_NO_NEW_PRIVS = 38
LANDLOCK_RULE_NET_PORT = 2
NET_BIND_TCP, NET_CONNECT_TCP = 1 << 0, 1 << 1
```

```
class RulesetAttr(ctypes.Structure):
    _fields_ = [("handled_access_fs", ctypes.c_uint64),
                ("handled_access_net", ctypes.c_uint64),
                ("scoped", ctypes.c_uint64)]

class NetPortAttr(ctypes.Structure):
    _fields_ = [("allowed_access", ctypes.c_uint64),
                ("port", ctypes.c_uint64)]

def must(r, what):
    if r < 0:
        e = ctypes.get_errno()
        raise OSError(e, os.strerror(e), what)
    return r

attr = RulesetAttr(0, NET_BIND_TCP | NET_CONNECT_TCP, 0)
ctypes.set_errno(0)
fd = must(libc.syscall(ctypes.c_long(NR_CREATE), ctypes.byref(attr),
                       ctypes.c_size_t(ctypes.sizeof(attr)), ctypes.c_uint(0)),
          "create_ruleset")
rule = NetPortAttr(NET_CONNECT_TCP, 443)
ctypes.set_errno(0)
must(libc.syscall(ctypes.c_long(NR_ADD), ctypes.c_int(fd),
                  ctypes.c_int(LANDLOCK_RULE_NET_PORT),
                  ctypes.byref(rule), ctypes.c_uint(0)), "add_rule")
must(libc.prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0), "nnp")
ctypes.set_errno(0)
must(libc.syscall(ctypes.c_long(NR_RESTRICT), ctypes.c_int(fd), ctypes.c_uint(0)),
     "restrict")
os.close(fd)
print("restricted: connect-tcp allowed only to port 443")

def try_connect(host, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(3)
    try:
        s.connect((host, port))
        print(f"connect({host},{port}): ok peer={s.getpeername()}")
    except OSError as e:
        print(f"connect({host},{port}): errno={e.errno} ({errno.errorcode.get(e.errno)}) {e.
    finally:
        s.close()

try_connect("1.1.1.1", 443)
try_connect("1.1.1.1", 80)
try_connect("1.1.1.1", 53)
```

```
python3 landlock_tcp_allow443.py
```

```
restricted: connect-tcp allowed only to port 443
connect(1.1.1.1,443): ok peer=('1.1.1.1', 443)
connect(1.1.1.1,80): errno=13 (EACCES) Permission denied
connect(1.1.1.1,53): errno=13 (EACCES) Permission denied
```

Later util-linux expresses the same policy as `setpriv --landlock-access net:connect-tcp --landlock-rule net-port:connect-tcp:443 …` once `setpriv --help` lists `Access: net`. This 2.41.5 binary does not.

6. `EPERM`: missing `no_new_privs`, and scoped signals (runs here)

Filesystem and TCP denials return **`EACCES`**. Two common **`EPERM`** cases:

```
# landlock_eperm.py - restrict_self without nnp; scoped signal to parent
import ctypes, os, errno, signal

libc = ctypes.CDLL(None, use_errno=True)
libc.syscall.restype = ctypes.c_long
libc.prctl.restype = ctypes.c_int
NR_CREATE, NR_RESTRICT = 444, 446
PR_SET_NO_NEW_PRIVS = 38
SCOPE_SIGNAL = 1 << 1
FS_READ_FILE = 1 << 2

class RulesetAttr(ctypes.Structure):
    _fields_ = [("handled_access_fs", ctypes.c_uint64),
                ("handled_access_net", ctypes.c_uint64),
                ("scoped", ctypes.c_uint64)]

attr = RulesetAttr(FS_READ_FILE, 0, 0)
ctypes.set_errno(0)
fd = libc.syscall(ctypes.c_long(NR_CREATE), ctypes.byref(attr),
                  ctypes.c_size_t(24), ctypes.c_uint(0))
ctypes.set_errno(0)
r = libc.syscall(ctypes.c_long(NR_RESTRICT), ctypes.c_int(fd), ctypes.c_uint(0))
e = ctypes.get_errno()
print(f"restrict_self without nnp: ret={r} errno={e} ({errno.errorcode.get(e)}) {os.strerror
      flush=True)
os.close(fd)

parent = os.getpid()
pid = os.fork()
if pid == 0:
    attr = RulesetAttr(0, 0, SCOPE_SIGNAL)
```

```
    ctypes.set_errno(0)
    fd = libc.syscall(ctypes.c_long(NR_CREATE), ctypes.byref(attr),
                      ctypes.c_size_t(24), ctypes.c_uint(0))
    libc.prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0)
    libc.syscall(ctypes.c_long(NR_RESTRICT), ctypes.c_int(fd), ctypes.c_uint(0))
    try:
        os.kill(parent, 0)
        print("kill(parent, 0): ok", flush=True)
    except OSError as e:
        print(f"kill(parent, 0): errno={e.errno} ({errno.errorcode.get(e.errno)}) {e.strerro
              flush=True)
    os._exit(0)
signal.signal(signal.SIGUSR1, lambda *a: None)
os.waitpid(pid, 0)
```

```
python3 landlock_eperm.py
```

```
restrict_self without nnp: ret=-1 errno=1 (EPERM) Operation not permitted
kill(parent, 0): errno=1 (EPERM) Operation not permitted
```

ABI 6 `LANDLOCK_SCOPE_SIGNAL` confines outgoing signals to the same (or a nested) Landlock domain. The parent is outside, so `kill` returns `EPERM`. Abstract UNIX sockets use the sibling scope flag and also deny with `EPERM`.

The trap

Trap: "`--landlock-access net:connect-tcp` sandboxes TCP on util-linux 2.41." On **2.41.5**, parse_landlock_access only honours fs and fs:…. Any other prefix is a silent no-op: setpriv –landlock-access net:connect-tcp still connected to 1.1.1.1:443 here. setpriv –landlock-rule net-port:connect-tcp:443 errors (invalid landlock rule). Read the Landlock accesses: block from setpriv –help' before copying flags from a newer man page.

Trap: "a `path-beneath` rule grants access." It grants Landlock's slice of the right, on that inode hierarchy. Mode bits, ownership, mount flags, and SELinux still have to agree. Example 3 is the short form: `chmod 000` plus a RW ruleset is still `EACCES`.

Trap: "handled rights are an allow-list of what the program may do." They are a deny-by-default *set of actions this ruleset cares about*. Omitting `truncate` from `handled_access_fs` leaves truncation unrestricted by Landlock (ABI 3+). `setpriv --landlock-access fs` handles everything it was compiled with, including `refer` and `truncate`; that is **stricter**, and on an ABI 1 kernel those extra bits make `landlock_create_ruleset` fail with `EINVAL`. The kernel docs' best-effort pattern is: probe ABI, then *mask off* unknown rights. `setpriv` 2.41 does not do that mask.

Trap: "ABI 6 has audit logs." Observability (`LANDLOCK_RESTRICT_SELF_LOG_*`, `AUDIT_LANDLOCK_ACCESS`) is ABI **7** / Linux **6.15**. This 6.12 kernel stops at scopes. Process-wide `TSYNC` is ABI 8; UDP is ABI 10. Probe; do not guess from `uname`.

The boring rule

Name every action you intend to deny in `handled_access_*` / `--landlock-access`, then grant the hierarchies and ports the program actually needs with rules. Probe the ABI and degrade. Keep `no_new_privs`. Treat Landlock as one more AND-clause next to DAC.

Try this

1. Re-run Example 2 and add `--landlock-rule path-beneath:read-file,read-dir:/etc`. Confirm `cat /etc/passwd` works and `echo x > /etc/hostname` still fails.
2. Handle only `fs:truncate` with a rule on the scratch dir, then `os.truncate` a scratch file (allowed) and `/etc/hostname` (`EACCES`). Then handle `write-file,truncate` but grant only `write-file` on scratch: append works, `truncate` returns `EACCES`. (Both were run here.)
3. On a 2.42+/devel `setpriv` whose `--help` lists `Access: net`, express Example 5 as `--landlock-access net:connect-tcp --landlock-rule net-port:connect-tcp:443`.
4. On Linux 6.15+ (ABI 7+), read Landlock audit and watch one denied `connect` as `AUDIT_LANDLOCK_ACCESS`.

Verification

- All numbered examples compiled/run on this box: Linux **6.12.94+**, Landlock **ABI 6**, util-linux **setpriv 2.41.5**, Python **3.13.5**, user `uid=1000`. Output pasted above.
- ABI probe used `landlock_create_ruleset` via `ctypes` (`__NR_landlock_create_ruleset` = 444) with `LANDLOCK_CREATE_RULESET_VERSION` / `_ERRATA` from `/usr/include/linux/landlock.h`.
- TCP examples used the same UAPI (`handled_access_net`, `LANDLOCK_RULE_NET_PORT`). `setpriv --landlock-access net:…` was tried and is a no-op on 2.41.5; that trial is in The trap, not sold as a working sandbox.
- `ioctl-dev` is in the kernel UAPI here (`LANDLOCK_ACCESS_FS_IOCTL_DEV`) and is **not** accepted by this `setpriv` (`could not parse landlock fs access: ioctl-dev`).
- **No** package installs, **no** root, **no** kernel builds. `dmesg` was readable unprivileged on this host. Audit examples (ABI 7) are documentation-only.

Sources

- Kernel userspace API — Landlock: unprivileged access control (ABI table, best-effort mask, TCP/UDP, scopes, audit flags; dated August 2026)
- Kernel admin guide — Landlock LSM (audit record types)
- man7 — `landlock(7)`, `landlock_create_ruleset(2)`, `landlock_add_rule(2)`, `landlock_restrict_self(`
- man7 — `setpriv(1)` (util-linux; `--landlock-access` / `--landlock-rule`)
- util-linux 2.41.5 source — `sys-utils/setpriv-landlock.c` (filesystem-only parser)
- UAPI on this host — `/usr/include/linux/landlock.h`
- Kernel sample — `samples/landlock/sandboxer.c`
- Landlock project — news / ABI timeline

Unprivileged User Namespaces: What “root” Inside One Can Reach

Unprivileged User Namespaces: What "root" Inside One Can Reach

Overview

Thesis: When an ordinary user runs `unshare -r`, they get uid 0 and a **full capability set**. Those capabilities are valid only over objects owned by the new user namespace, so host files, the clock and kernel modules stay out of reach. But they also unlock kernel code that used to be reachable only by real root: mounting filesystems, rtnetlink, netfilter, cgroup and IPC setup. Unprivileged user namespaces are therefore a good **isolation** tool, because rootless containers, browser sandboxes and Flatpak all depend on them. They are also a real **attack surface**: a bug in any of that newly reachable code becomes a local privilege escalation. Whether to allow them depends on the host. This chapter measures both sides on one kernel and shows the knobs.

Version note: every probe below ran as uid 1000 on Linux **6.12.94** (x86_64, a minimal cloud kernel) with util-linux `unshare` and Python 3.13. No sudo was used. The distribution knobs (Ubuntu's AppArmor restriction, Debian's sysctl) are not present on this kernel. They are cited from their documentation and marked as not run.

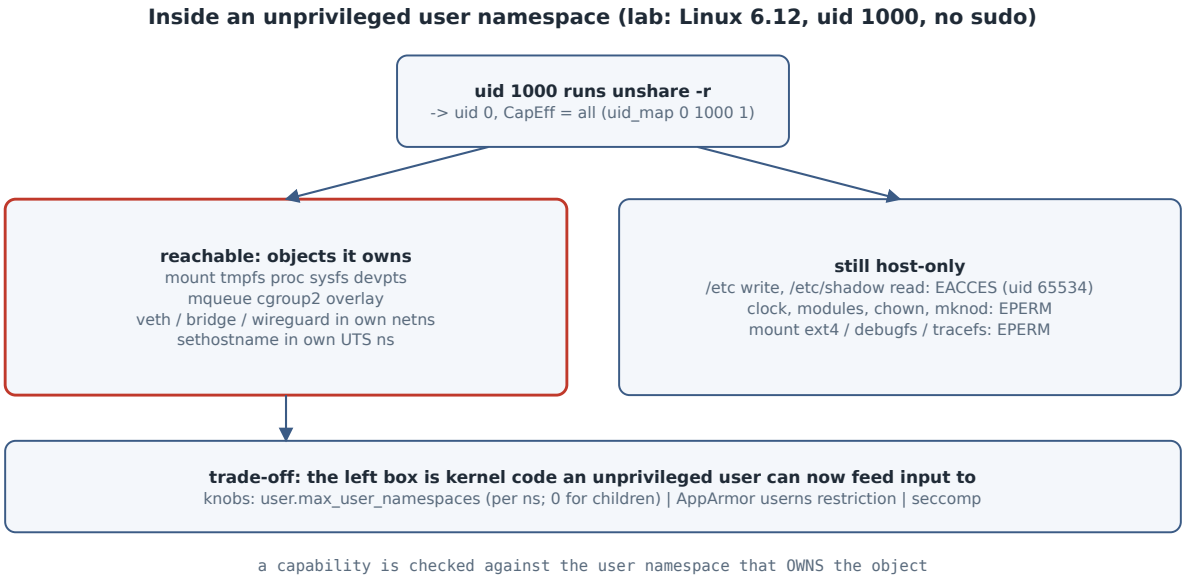


Figure 1: Inside an unprivileged user namespace

Mental model

```
host (initial user namespace)              new user namespace  (unshare -r)
  uid 1000, CapEff = 0                        uid 0, CapEff = all
                          uid_map: "0 1000 1"
  owns: /, /etc, devices, clock, modules     owns: the mount/net/uts/ipc/pid/cgroup
                                                   namespaces created with it
```

A capability is checked **against the user namespace that owns the object**. Inside the new namespace, `CAP_SYS_ADMIN` lets you mount a tmpfs into a mount namespace you own. It does nothing to the host's `/etc`. Host files belong to uids that are not in `uid_map`, so they appear as the overflow uid 65534 and the ordinary permission bits apply to you as "other" (user_namespaces(7)).

The catch is that "you may mount a tmpfs" or "you may configure a network namespace" means the kernel now runs its filesystem-parameter, netlink and netfilter code **on input from an unprivileged user**. That code was written and hardened at a time when only root could reach it.

Examples with Explanations

1. What namespace-root can and cannot do (runs here)

```
"""Run as uid 0 inside a fresh user namespace (unshare -rmnpf --mount-proc). Probe what that
import ctypes, os, errno, socket, time
libc = ctypes.CDLL(None, use_errno=True)
def mount(src, tgt, fstype, data=None):
    os.makedirs(tgt, exist_ok=True)
    r = libc.mount(src.encode(), tgt.encode(), fstype.encode(), 0, data.encode() if data els
    return "ok" if r == 0 else errno.errorcode[ctypes.get_errno()]
def attempt(f):
    try: f(); return "ok"
    except OSError as e: return errno.errorcode[e.errno]

st = dict(l.split(":\t") for l in open("/proc/self/status").read().splitlines() if ":\t" in
print("uid/gid inside      :", os.getuid(), os.getgid(), "| uid_map:", open("/proc/self/uid_
print("CapEff              :", st["CapEff"].strip())
print("owner of / and /etc/shadow:", os.stat("/").st_uid, os.stat("/etc/shadow").st_uid, "(6

print("\n-- filesystems: mount(2) as namespace root")
for fs in ["tmpfs", "ramfs", "proc", "sysfs", "mqueue", "devpts", "cgroup2", "bpf", "overlay
    data = {"overlay": "lowerdir=/usr,upperdir=/tmp/up/u,workdir=/tmp/up/w", "devpts": "newi
    if fs == "overlay":
        os.makedirs("/tmp/up", exist_ok=True); mount("none", "/tmp/up", "tmpfs"); os.makedir
    if fs == "fuse": continue
    print(f"  {fs:10s} {mount('none' if fs != 'ext4' else '/dev/null', '/tmp/m-' + fs, fs, d
```

```
print("\n-- things that stay host-only")
print("  write /etc/hostname     :", attempt(lambda: open("/etc/hostname", "a")))
print("  read /etc/shadow        :", attempt(lambda: open("/etc/shadow").read()))
print("  sethostname (own UTS ns):", attempt(lambda: socket.sethostname("lab")), "->", socke
print("  clock_settime           :", attempt(lambda: time.clock_settime(time.CLOCK_REALTIME,
print("  init_module/finit_module:", "EPERM" if libc.syscall(313, -1, b"", 0) != 0 and ctype
print("  chown host file to 0    :", attempt(lambda: os.chown("/etc/hostname", 0, 0)))
print("  mknod char device       :", attempt(lambda: os.mknod("/tmp/null2", 0o020666, os.mak
```

```
unshare -rmnpfiuC --mount-proc python3 probe.py   # user+mount+net+pid+ipc+uts+cgroup namesp
```

```
uid/gid inside      : 0 0 | uid_map: ['0', '1000', '1']
CapEff              : 000001fffffffffff
owner of / and /etc/shadow: 65534 65534 (65534 = unmapped host uid)

-- filesystems: mount(2) as namespace root
  tmpfs      ok
  ramfs      ok
  proc       ok
  sysfs      ok
  mqueue     ok
  devpts     ok
  cgroup2    ok
  bpf        EPERM
  overlay    ok
  ext4       EPERM
  debugfs    EPERM
  tracefs    EPERM
  securityfs EPERM

-- things that stay host-only
  write /etc/hostname     : EACCES
  read /etc/shadow        : EACCES
  sethostname (own UTS ns): ok -> lab
  clock_settime           : EPERM
  init_module/finit_module: EPERM
  chown host file to 0    : EPERM
  mknod char device       : EPERM
```

Reading it:

- **`CapEff 000001fffffffffff`** has all 41 capability bits set. That is "root" as far as the capability checks are concerned. **`uid_map`** is `0 1000 1`, so uid 0 here is uid 1000 on the host.
- **Mounts.** tmpfs, ramfs, proc, sysfs, devpts, mqueue, cgroup2 and overlay all work. These are the filesystem types that user_namespaces(7) lists as mountable with `CAP_SYS_ADMIN` in the namespace that owns the mount namespace. mqueue and cgroup2 need their own IPC and

cgroup namespaces; without `-i` and `-C`, the first run of this probe got `EPERM` for both. ext4, debugfs, tracefs and securityfs are refused, because block-based filesystems and the kernel's debug interfaces require `CAP_SYS_ADMIN` in the *initial* namespace. The man page also lists bpf, but on this 6.12 kernel a direct `mount(2)` of bpffs from inside the namespace got `EPERM`. Newer kernels gate a bpffs owned by a user namespace through the BPF token setup, which needs a privileged helper.

- **Host-only.** Writing `/etc/hostname` and reading `/etc/shadow` fail with `EACCES`: these are ordinary permission checks against an unmapped owner. The clock, `finit_module`, `chown` to a host uid and `mknod` of a character device fail with `EPERM`: these capabilities are only honoured in the initial namespace. `sethostname` works, but it changes only this namespace's UTS name.

2. The network stack is now yours to configure (runs here)

```
"""Inside the user+net namespace: which rtnetlink link kinds can namespace-root create?"""
from pyroute2 import IPRoute
from pyroute2.netlink.exceptions import NetlinkError
import errno
ip = IPRoute()
ip.link("add", ifname="v0", kind="veth", peer="v0p"); base = ip.link_lookup(ifname="v0")[0]
kinds = {"dummy": {}, "bridge": {}, "veth": {"peer": "t-vethp"}, "bond": {}, "vrf": {"vrf_ta
         "vlan": {"link": base, "vlan_id": 7}, "macvlan": {"link": base, "macvlan_mode": "br
         "ipvlan": {"link": base}, "vxlan": {"vxlan_id": 42, "vxlan_port": 4789},
         "geneve": {"geneve_id": 7, "geneve_remote": "192.0.2.1"}, "gre": {"gre_remote": "19
         "ipip": {"ipip_remote": "192.0.2.1"}, "sit": {"sit_remote": "192.0.2.1"},
         "wireguard": {}, "ifb": {}, "nlmon": {}, "team": {}, "macsec": {"link": base}}
ok, no = [], []
for k, extra in kinds.items():
    try:
        ip.link("add", ifname="t-" + k, kind=k, **extra); ok.append(k)
    except NetlinkError as e:
        no.append(f"{k}({errno.errorcode.get(e.code, e.code)})")
print("created  :", " ".join(ok))
print("refused  :", " ".join(no))
```

```
unshare -rn /workspace/venv-qos/bin/python links.py     # pyroute2 0.9.7 in a venv
/workspace/venv-qos/bin/python links.py                 # same script, no namespace
```

```
created  : bridge veth wireguard
refused  : dummy(ENOTSUP) bond(ENOTSUP) vrf(ENOTSUP) vlan(ENOTSUP) macvlan(ENOTSUP) ipvlan(E
```

Without a user namespace:

```
pyroute2.netlink.exceptions.NetlinkError: (1, 'Operation not permitted')
```

Inside the namespace, uid 1000 created bridge, veth and WireGuard interfaces, which are kernel drivers it could not touch a second earlier. Without the namespace, the very first request gets `EPERM`. The `ENOTSUP` list is this minimal kernel saying "not built". On a distribution kernel, most of those link types are modules, and rtnetlink can load a module on request for a link kind. So the code an unprivileged user can reach is whatever the host kernel has built in **or can load**. The same is true of netfilter (nf_tables), traffic control and the other netlink families that check `CAP_NET_ADMIN` against the namespace.

That is the attack-surface argument in concrete form. Two well-known examples:

- CVE-2022-0185, a heap overflow in the filesystem-context parameter parser, needed `CAP_SYS_ADMIN`.
- CVE-2023-32233, a use-after-free in nf_tables, needed `CAP_NET_ADMIN`.

On hosts that allow unprivileged user namespaces, the capability each one needed is exactly what `unshare -r` hands out.

3. Limits are per namespace, and a sandbox can deny nesting (runs here)

```
#!/bin/sh
# nest.sh - run inside `unshare -r`: user.max_user_namespaces is per user namespace
echo "limit seen in this ns: $(cat /proc/sys/user/max_user_namespaces)"
unshare -r sh -c 'echo "nested unshare -r    : ok, uid $(id -u)"'
echo 0 > /proc/sys/user/max_user_namespaces && echo "set limit in this ns : $(cat /proc/sys/
unshare -r true 2>&1 | sed 's/^/nested unshare -r    : /'
```

```
unshare -r ./nest.sh; echo "host value: $(cat /proc/sys/user/max_user_namespaces)"
```

```
limit seen in this ns: 2147483647
nested unshare -r    : ok, uid 0
set limit in this ns : 0
nested unshare -r    : unshare: unshare failed: No space left on device
host value: 63991
```

`/proc/sys/user/max_user_namespaces` is a **per-user-namespace** limit (sysctl/user.rst). The host value is 63991 here, and the new namespace starts with its own value of 2147483647. Creating a namespace is charged against the limits of every ancestor as well, so raising your own value does not let you escape the host's. Lowering it to 0 inside a sandbox makes `unshare` fail with `ENOSPC` for everything that runs inside, while the host is unaffected. This is how a sandbox stops its payload from making further user namespaces, and with it the extra reach shown in Example 2.

Choosing a policy (not run here)

Knob	Scope	Effect	Breaks
`sysctl user.max_user_namespaces=0`	whole host	no new user namespaces for anyone	rootless Podman/Docker, Chromium/Firefox sandboxes, Flatpak/bubblewrap
Ubuntu 23.10+/24.04 `kernel.apparmor_restrict_unprivileged_userns=1`	per program	unconfined programs get a namespace but no capabilities in it; AppArmor profiles with `userns,` are exempt	programs without a profile that need capabilities in the namespace
Debian `kernel.unprivileged_userns_clone=0` (Debian kernel patch)	whole host	unprivileged `CLONE_NEWUSER` refused	same as the first row; the file does not exist on mainline kernels like this one
A sandbox's own `max_user_namespaces=0` (Example 3)	one sandbox	its children cannot nest	nothing outside the sandbox
A custom container seccomp profile	one container	can deny `clone`/`unshare` with `CLONE_NEWUSER`; note that Docker's *default* profile gates namespace flags behind `CAP_SYS_ADMIN` **except** `CLONE_NEWUSER` / `unshare --user`	rootless tools *inside* the container

The trap

The trap is thinking "it's only root inside a namespace, so it's harmless". The capability checks hold: Example 1 shows the host's files, clock and modules really are out of reach. But every `ok` in Examples 1 and 2 is a kernel subsystem that parses input from an unprivileged user. The trap in the other direction is setting `user.max_user_namespaces=0` on a developer laptop or a CI runner and then wondering why browsers lose their sandbox and rootless containers stop starting.

The boring rule

- **Shared multi-user hosts and servers that run no rootless containers:** restrict. Use Ubuntu's AppArmor restriction with profiles for the tools that need user namespaces, or set the sysctl to 0 if nothing needs them.

- **Desktops, CI runners and container hosts:** allow, keep the kernel patched quickly, and have sandboxes set their own `max_user_namespaces=0` for untrusted payloads.

Either way, check what your host actually does: `cat /proc/sys/user/max_user_namespaces`, and `unshare -r id`.

Try this

1. Run `unshare -r grep Cap /proc/self/status` and decode `CapEff` with `capsh --decode=` where it is available.
2. Re-run Example 1 without `-i` and `-C`, and watch mqueue and cgroup2 flip to `EPERM`.
3. On Ubuntu 24.04 (not run here): `sysctl kernel.apparmor_restrict_unprivileged_userns`, then `unshare -r -n ip link add br0 type bridge`. Compare the result with and without an AppArmor profile that grants `userns,`.

Sources

- user_namespaces(7): https://man7.org/linux/man-pages/man7/user_namespaces.7.html
- capabilities(7): https://man7.org/linux/man-pages/man7/capabilities.7.html
- unshare(1): https://man7.org/linux/man-pages/man1/unshare.1.html
- Linux kernel, Documentation for /proc/sys/user: https://docs.kernel.org/admin-guide/sysctl/user.html
- AppArmor wiki, unprivileged user namespace restrictions: https://gitlab.com/apparmor/apparmor/-/wikis/unprivileged_userns_restriction
- Docker docs, seccomp security profiles: https://docs.docker.com/engine/security/seccomp/
- NVD, CVE-2022-0185: https://nvd.nist.gov/vuln/detail/CVE-2022-0185; CVE-2023-32233: https://nvd.nist.gov/vuln/detail/CVE-2023-32233
- pyroute2: https://docs.pyroute2.org/

OpenSSH on musl (sshole): detect, mitigate, upgrade

OpenSSH on musl (sshole): detect, mitigate, upgrade

After reading this chapter, you will be able to determine if a Linux host or container image is vulnerable to the November 2026 OpenSSH/musl password authentication bypass (nicknamed "sshole"), apply immediate SSH daemon configuration mitigations without disrupting public-key users, and verify upgrade remediation to OpenSSH 10.6p2 or later.

Overview

Thesis: The **sshole** vulnerability allows an unauthenticated remote attacker to bypass password authentication on systems running OpenSSH paired with **musl libc** when Pluggable Authentication Modules (**PAM**) support is disabled in `sshd`. The flaw stems from an edge-case interaction between OpenSSH password evaluation and musl libc's `crypt(3)` function: when an input password exceeds 257 characters, musl's `crypt(3)` returns `"*"` when given an invalid hash string matching the single-character `"*"` lock marker in `/etc/shadow`. OpenSSH portable release **10.6p2** fixes this check in `sshd(8)` by establishing a minimum password hash length requirement. Systems running glibc or systems using PAM authentication are outside the scope of this vulnerability.

```
Client authentication request (>257-byte password)


        sshd (PAM disabled, musl libc)

      shadow lookup returns password hash "*"


      musl libc crypt(password, "*") -> "*"


      sshd compares computed "*" == stored "*"    AUTHENTICATION BYPASS
```

Component	Vulnerable State	Remediated / Safe State
C Library	`musl libc` (any version)	`glibc` (unaffected), or `musl` with patched OpenSSH
OpenSSH Version	`< 10.6p2`	`10.6p2` or later
PAM Support	Disabled (`sshd` built/run without PAM)	Enabled (`UsePAM yes`), or disabled with `PasswordAuthentication no`

Component	Vulnerable State	Remediated / Safe State
Shadow Accounts	Account locked with hash `"*"`	Upgraded OpenSSH package, active password, or disabled password auth

Vulnerability Mechanism

The vulnerability exists at the boundary between OpenSSH authentication logic and musl libc's `crypt(3)` implementation.

musl `crypt(3)` behavior

In standard POSIX implementations, calling `crypt(key, setting)` hashes the string `key` using the hashing algorithm identified by `setting` (or traditional DES if no prefix is given). When musl libc's `crypt(3)` encounters an unrecognized or invalid hash setting, or an unhashable input condition such as a password string longer than 257 characters, it returns a static fallback string: `"*"` (a single asterisk).

Shadow record locking conventions

On standard Linux systems, locked accounts or accounts without an active password store a short marker in the password field of `/etc/shadow`. Unix tools commonly set this field to `"*"` or `"!"` to prevent login matching.

```
# Example /etc/shadow entry for a disabled service account
builduser:*:19680:0:99999:7:::
```

OpenSSH comparison flaw

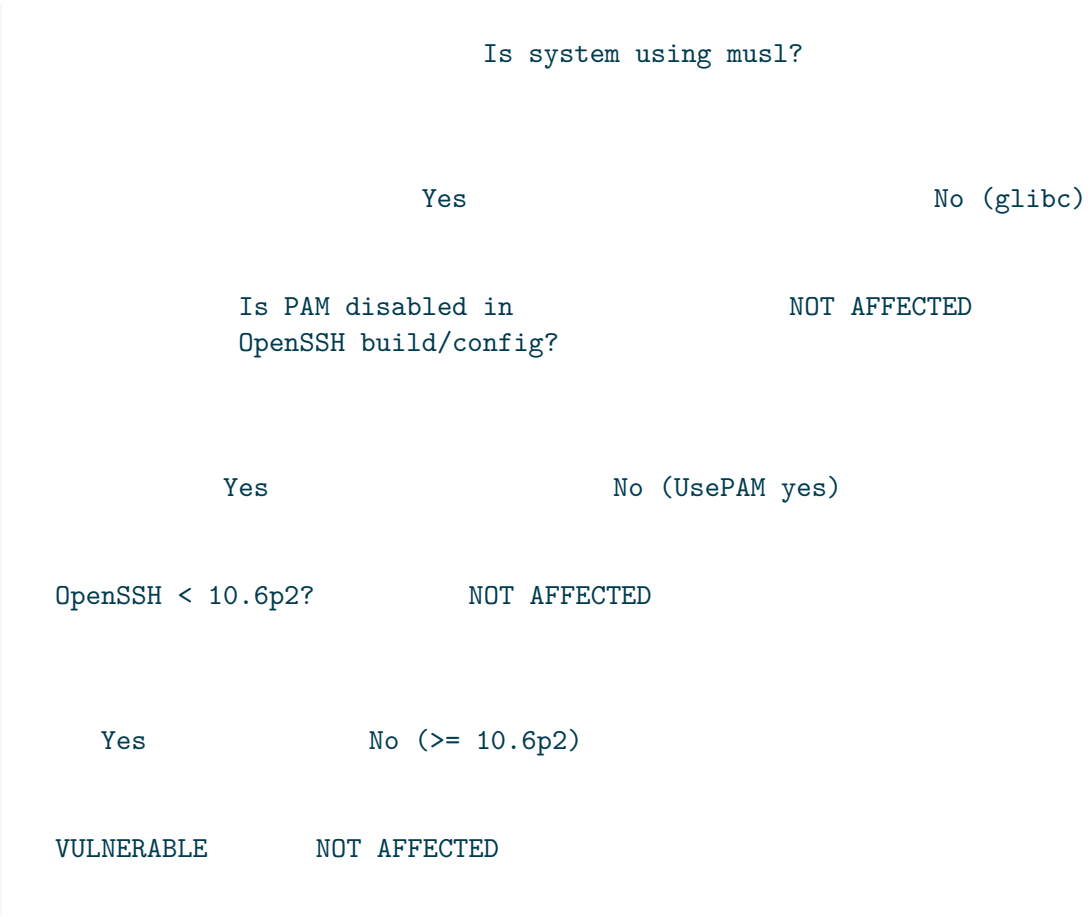
When `sshd(8)` is compiled without PAM support, it performs direct Unix password validation by fetching the target user's shadow entry and calling `crypt(password, shadow_hash)`.

1. The target account has a shadow hash set to `"*"` (indicating password logins are disabled).
2. The remote client submits an authentication request with a password string exceeding 257 characters.
3. OpenSSH passes the submitted password and the stored shadow string `"*"` to `crypt(3)`.
4. musl libc's `crypt(3)` returns `"*"` due to the input length constraint.
5. OpenSSH compares the returned hash (`"*"`) against the stored hash (`"*"`). Because the strings match exactly, OpenSSH authorizes the login session.

OpenSSH 10.6p2 addresses this issue by enforcing `MIN_PASSWORD_HASH_LEN` inside `sshd(8)`, explicitly ignoring hash returns and shadow strings that fall below valid algorithm hash lengths prior to string comparison.

Mental model & Scope

Determining host susceptibility requires assessing three orthogonal factors: standard C library implementation, SSH daemon configuration, and account locking status.



Distributions widely affected include lightweight or container-focused Linux operating systems built on musl libc, including: - **Alpine Linux** (default installation uses musl libc and builds OpenSSH without PAM). - **Buildroot / Yocto / OpenWrt** embedded Linux targets compiled against musl libc. - **Container base images** derived from Alpine or musl-based minimal runtime images containing `openssh-server`.

GNU C library (glibc) Linux distributions (such as Debian, Ubuntu, RHEL, Fedora, Arch Linux) use glibc's `crypt(3)` implementation or `libxcrypt`, which do not yield `"*"` for inputs over 257 bytes in this manner. They are completely outside the scope of this vulnerability.

Safety

> **Note**
>
> **Not run here.** The local environment is a Debian glibc host. The detection and mitigation commands presented in this section are meant to be run directly on target musl systems (such as Alpine Linux nodes or container instances).

- **No attack reproduction.** Do not attempt to synthesize long-payload authentication attempts against remote hosts. Host auditing can be conducted entirely through version checks, configuration inspection, and package status checks.
- **Service disruption awareness.** Changing `PasswordAuthentication` to `no` in `sshd_config` instantly blocks password-based logins. Ensure all administrative users have configured operational public-key (`AuthorizedKeysFile`) access before reloading `sshd`.
- **Container image rebuilds.** Simply restarting a container running an vulnerable base image will not patch the library or daemon. Containerized workloads require updating base image references or rebuilding images with updated package indices.

Auditing and Detection

Checking C library and OpenSSH version

Execute the following shell commands on the target host to determine if musl libc is present and which OpenSSH version is currently active:

```
ldd --version
sshd -V
```

```
<<OUTPUT:audit_ver>>
```

On a musl system, `ldd --version` outputs a line containing `musl libc`. On OpenSSH builds prior to `10.6p2` (e.g., `OpenSSH_9.8p1`), the daemon is vulnerable if PAM is not in use.

Auditing SSH daemon configuration

Check whether `PasswordAuthentication` is enabled and verify if PAM support is compiled in and activated:

```
sshd -T | grep -iE 'passwordauthentication|usepam'
```

```
<<OUTPUT:audit_config>>
```

If the output displays `passwordauthentication yes` and `usepam no` (or if PAM support was omitted during compilation), the daemon permits standard password authentication routines handled directly by musl `crypt(3)`.

Auditing locked shadow accounts

Inspect `/etc/shadow` for non-system accounts that contain `"*"` in the password hash field:

```
awk -F: '$2 == "*" { print $1 }' /etc/shadow
```

```
<<OUTPUT:audit_shadow>>
```

Accounts returned by this command represent locked entries that would evaluate identically under the musl `crypt(3)` return condition if password authentication is permitted by `sshd`.

Mitigation and Remediation

Immediate Mitigation: Disabling Password Authentication

If upgrading the OpenSSH package immediately is not feasible, disable password-based authentication globally in the SSH daemon configuration. This bypasses the direct shadow `crypt(3)` check entirely.

Apply the setting using a drop-in configuration file under `/etc/ssh/sshd_config.d/`:

```
# /etc/ssh/sshd_config.d/99-disable-passwords.conf
# Mitigate sshole (OpenSSH+musl bypass) by forcing publickey auth
PasswordAuthentication no
KbdInteractiveAuthentication no
```

Validate the daemon configuration file syntax before applying changes:

```
sshd -t
rc-service sshd reload || systemctl reload sshd
```

```
<<OUTPUT:reload_sshd>>
```

Verify that the runtime operational parameters reflect the updated configuration:

```
sshd -T | grep -i passwordauthentication
```

```
<<OUTPUT:verify_mitigation>>
```

Remediation: Upgrading OpenSSH on Alpine Linux

Package maintainers across musl-based distributions released patched builds incorporating the OpenSSH 10.6p2 fix. On Alpine Linux environments, update package repositories and upgrade `openssh-server` and `openssh-client`:

```
apk update
apk upgrade openssh
```

```
<<OUTPUT:apk_upgrade>>
```

After updating, verify that `sshd` reports release version `10.6p2` or a distribution-patched equivalent revision:

```
sshd -V
```

```
<<OUTPUT:post_upgrade_ver>>
```

Restart the SSH service to ensure the new binary handles incoming connections:

```
rc-service sshd restart
```

```
<<OUTPUT:service_restart>>
```

Automated Auditing Script

The following POSIX-compliant shell script automates system auditing for the vulnerability state. It checks C library type, SSH daemon options, and installed OpenSSH package versions without performing any authentication attempts.

```
#!/bin/sh
# check_sshole_status.sh - Audit local host for sshole exposure conditions

set -eu

echo "=== System C Library Audit ==="
if ldd /bin/sh 2>&1 | grep -qi 'musl'; then
    IS_MUSL=1
    echo "[!] C library: musl libc detected."
else
    IS_MUSL=0
    echo "[ok] C library: non-musl system (glibc or other)."
fi

echo ""
```

```
echo "=== OpenSSH Configuration Audit ==="
if command -v sshd >/dev/null 2>&1; then
    SSH_VER=$(sshd -V 2>&1 | head -n1)
    echo "Installed version: ${SSH_VER}"

    # Evaluate active runtime configuration parameters
    PASS_AUTH=$(sshd -T 2>/dev/null | grep '^passwordauthentication ' | awk '{print $2}')
    USE_PAM=$(sshd -T 2>/dev/null | grep '^usepam ' | awk '{print $2}' || echo "no")

    echo "PasswordAuthentication: ${PASS_AUTH:-unknown}"
    echo "UsePAM: ${USE_PAM:-no}"

    if [ "${IS_MUSL}" -eq 1 ] && [ "${PASS_AUTH}" = "yes" ] && [ "${USE_PAM}" = "no" ]; then
        echo ""
        echo "[WARNING] Host configuration matches sshole criteria:"
        echo "          - Operating on musl libc"
        echo "          - PasswordAuthentication is enabled"
        echo "          - PAM is disabled or unsupported"
        echo "Action required: Set 'PasswordAuthentication no' or upgrade OpenSSH to >= 10.6
    else
        echo ""
        echo "[ok] Host does not expose the vulnerable authentication path."
    fi
else
    echo "sshd command not found in PATH."
fi
```

Make the script executable and execute it locally:

```
chmod +x check_sshole_status.sh
./check_sshole_status.sh
```

```
<<OUTPUT:run_script>>
```

Common Mistakes

Mistake	Consequence	Corrective Action
Assuming glibc systems need emergency SSH patching for sshole	Wasted maintenance windows and unnecessary updates on unaffected systems	Verify C library implementation using `ldd --version` before initiating emergency patching procedures.
Relying on account locking (`usermod -L` or `passwd -l`) to block SSH access	Account locking places `*` or `!` in `/etc/shadow`, which forms the specific trigger condition under musl `crypt(3)`	Set `PasswordAuthentication no` in `sshd_config` or set the login shell to `/sbin/nologin` or `/bin/false`.

Mistake	Consequence	Corrective Action
Modifying `/etc/ssh/sshd_config` without checking drop-in directories	Settings in `/etc/ssh/sshd_config.d/*.conf` may override global configuration directives	Check effective configuration using `sshd -T` rather than reading raw file lines.
Updating host binaries without updating container base images	Running containers remain vulnerable even if the underlying container host OS is updated	Rebuild container images using updated distribution base tags (e.g. `alpine:latest` after release fixes).

Summary

The sshole vulnerability highlights how C library implementation details can impact application-level security assumptions. By returning `"*"` on inputs over 257 bytes when presented with invalid settings, musl libc's `crypt(3)` creates an authentication bypass condition in non-PAM builds of OpenSSH prior to version 10.6p2. Operators can immediately mitigate vulnerable hosts by setting `PasswordAuthentication no` in `sshd_config` and fully remediate the issue by upgrading packages to OpenSSH `10.6p2` or later.

Sources

- OpenSSH 10.6p2 Release Notes
- OpenSSH 10.6p2 Release Announcement Text
- oss-security: "sshole": password authentication bypass: OpenSSH + musl libc
- sshhole.com Technical Vulnerability Summary

Appendices

Intro

Intro

Long-form extras and cheatsheets that do not fit the one-command page model. Use these as quick reference; prefer the command chapters for explained examples.

Commands in this part

Command	Role
Ubuntu Cheatsheet	Quick Ubuntu-oriented CLI reference

Suggested starting points

1. Ubuntu-oriented command cheatsheet for rapid lookup.

Related parts

- All command parts — detailed pages with safety notes
- Book introduction — learning paths

Continue with the individual command pages in this part.

Ubuntu Cheatsheet

Ubuntu Cheatsheet

System

System Information

- `uname -a` - Displays all system information
- `hostnamectl` - Shows current hostname and related details
- `lscpu` - Lists CPU architecture information
- `timedatectl status` - Shows system time

System Monitoring and Management

- `top` - Displays real-time system processes
- `htop` - An interactive process viewer (needs installation)
- `df -h` - Shows disk usage in a human-readable format
- `free -m` - Displays free and used memory in MB
- `kill <process id>` - Terminates a process

Running Commands

- `[command] &` - Runs command in the background
- `jobs` - Displays background commands
- `fg <command number>` - Brings command to the foreground

Service Management

- `sudo systemctl start <service>` - Starts a service
- `sudo systemctl stop <service>` - Stops a service
- `sudo systemctl status <service>` - Checks the status of a service
- `sudo systemctl reload <service>` - Reloads a service's configuration without interrupting its operation
- `journalctl -f` - Follows the journal, showing new log messages in real time
- `journalctl -u <unit_name>` - Displays logs for a specific systemd unit

Cron Jobs and Scheduling

- `crontab -e` - Edits cron jobs for the current user
- `crontab -l` - Lists cron jobs for the current user

Files

File Management

- `ls` - Lists files and directories
- `touch <filename>` - Creates an empty file or updates the last accessed date
- `cp <source> <destination>` - Copies files from source to destination
- `mv <source> <destination>` - Moves files or renames them
- `rm <filename>` - Deletes a file

Directory Navigation

- `pwd` - Displays the current directory path
- `cd <directory>` - Changes the current directory
- `mkdir <dirname>` - Creates a new directory

File Permissions and Ownership

- `chmod [who][+/-][permissions] <file>` - Changes file permissions
- `chmod u+x <file>` - Makes a file executable by its owner
- `chown [user]:[group] <file>` - Changes file owner and group

Searching and Finding

- `find [directory] -name <search_pattern>` - Finds files and directories
- `grep <search_pattern> <file>` - Searches for a pattern in files

Archiving and Compression

- `tar -czvf <name.tar.gz> [files]` - Compresses files into a tar.gz archive
- `tar -xvf <name.tar.[gz|bz|xz]> [destination]` - Extracts a compressed tar archive

Text Editing and Processing

- `nano [file]` - Opens a file in the Nano text editor
- `cat <file>` - Displays the contents of a file
- `less <file>` - Displays the paginated content of a file
- `head <file>` - Shows the first few lines of a file
- `tail <file>` - Shows the last few lines of a file
- `awk '{print}' [file]` - Prints every line in a file

Packages

Package Management (APT)

- `sudo apt install <package>` - Installs a package
- `sudo apt install -f --reinstall <package>` - Reinstalls a broken package
- `apt search <package>` - Searches for APT packages
- `apt-cache policy <package>` - Lists available package versions
- `sudo apt update` - Updates package lists
- `sudo apt upgrade` - Upgrades all upgradable packages
- `sudo apt remove <package>` - Removes a package
- `sudo apt purge <package>` - Removes a package and all its configuration files

Package Management (Snap)

- `snap find <package>` - Search for Snap packages
- `sudo snap install <snap_name>` - Installs a Snap package
- `sudo snap remove <snap_name>` - Removes a Snap package
- `sudo snap refresh` - Updates all installed Snap packages
- `snap list` - Lists all installed Snap packages
- `snap info <snap_name>` - Displays information about a Snap package

Users & Groups

User Management

- `w` - Shows which users are logged in
- `sudo adduser <username>` - Creates a new user
- `sudo deluser <username>` - Deletes a user
- `sudo passwd <username>` - Sets or changes the password for a user
- `su <username>` - Switches user
- `sudo passwd -l <username>` - Locks a user account
- `sudo passwd -u <username>` - Unlocks a user password
- `sudo chage <username>` - Sets user password expiration date

Group Management

- `id [username]` - Displays user and group IDs
- `groups [username]` - Shows the groups a user belongs to
- `sudo addgroup <groupname>` - Creates a new group
- `sudo delgroup <groupname>` - Deletes a group

Networking

Network Configuration

- `ip addr show` - Displays network interfaces and IP addresses
- `ip -s link` - Shows network statistics
- `ss -l` - Shows listening sockets
- `ping <host>` - Pings a host and outputs results

Netplan Configuration

- `cat /etc/netplan/*.yaml` - Displays the current Netplan configuration
- `sudo netplan try` - Tests a new configuration for a set period of time
- `sudo netplan apply` - Applies the current Netplan configuration

Firewall Management

- `sudo ufw status` - Displays the status of the firewall
- `sudo ufw enable` - Enables the firewall
- `sudo ufw disable` - Disables the firewall
- `sudo ufw allow <port/service>` - Allows traffic on a specific port or service
- `sudo ufw deny <port/service>` - Denies traffic on a specific port or service
- `sudo ufw delete allow/deny <port/service>` - Deletes an existing rule

SSH and Remote Access

- `ssh <user@host>` - Connects to a remote host via SSH
- `scp <source> <user@host>:<destination>` - Securely copies files between hosts

LXD

LXD is a modern, secure and powerful tool that provides a unified experience for running and managing containers or virtual machines. Visit https://canonical.com/lxd for more information.

Basic Setup

- `lxd init` - Initializes LXD before first use

Creating Instances

- `lxc init ubuntu:22.04 <container name>` - Creates a lxc system container (without starting it)
- `lxc launch ubuntu:24.04 <container name>` - Creates and starts a lxc system container
- `lxc launch ubuntu:22.04 <vm name> --vm` - Creates and starts a virtual machine

Managing Instances

- `lxc list` - Lists instances
- `lxc info <instance>` - Shows status information about an instance
- `lxc start <instance>` - Starts an instance
- `lxc stop <instance> [--force]` - Stops an instance
- `lxc delete <instance> [--force|--interactive]` - Deletes an instance

Accessing Instances

- `lxc exec <instance> -- <command>` - Runs a command inside an instance
- `lxc exec <instance> -- bash` - Gets shell access to an instance (if bash is installed)
- `lxc console <instance> [flags]` - Gets console access to an instance
- `lxc file pull <instance>/<instance_filepath> <local_filepath>` - Pulls a file from an instance
- `lxc file push <local_filepath> <instance>/<instance_filepath>` - Pushes a file to an instance

Using Projects

- `lxc project create <project> [--config <option>]` - Creates a project
- `lxc project set <project> <option>` - Configures a project
- `lxc project switch <project>` - Switches to a project

Ubuntu Pro

Ubuntu Pro delivers 10 years of expanded security coverage on top of Ubuntu's Long Term Support (LTS) commitment in addition to management and compliance tooling. Visit https://ubuntu.com/pro to register for free on up to five machines.

Activating Ubuntu Pro

- `sudo pro attach <token>` - Attaches your machine to Ubuntu Pro using a specific token

Managing Services

- `sudo pro status` - Displays the status of all Ubuntu Pro services
- `sudo pro enable <service>` - Enables a specific Ubuntu Pro service
- `sudo pro disable <service>` - Disables a specific Ubuntu Pro service

Extended Security Maintenance (ESM)

- `sudo pro enable esm-infra` - Activates Extended Security Maintenance for infrastructure packages
- `sudo pro enable esm-apps` - Activates ESM for applications

Livepatch Service

- `sudo pro enable livepatch` - Enables the Livepatch service for kernel updates without rebooting

FIPS Mode

- `sudo pro enable fips` - Enables FIPS mode for strict cryptographic standards

Updating Configuration

- `sudo pro refresh` - Refreshes the Ubuntu Pro state
- `sudo pro detach` - Detaches the machine from Ubuntu Pro, disabling all services

Editors

Intro

Overview

Overview: Editing on Linux

Linux gives you more than one way to change a file. Some editors exist to survive a broken package manager. Others are full IDEs in the terminal. Multiplexers keep processes alive after SSH drops. Agent runtimes such as **Herdr** treat AI coding agents as first-class citizens of that same terminal world.

This chapter orients you so the rest of the book is a map, not a maze.

The three layers

Think in layers rather than "best editor":

Layer	Job	Examples
Buffer editor	Change text	vi, Vim, Neovim, Helix, nano, Emacs, Zed
Workspace / session	Keep panes and processes alive	tmux, screen, Zellij, Herdr
Companion CLI	Search, navigate, review	rg, fd, fzf, bat, delta, lazygit

You almost always combine one item from each layer.

```
Herdr / tmux / Zellij  (session layer)

  nvim / hx        shell / tests / AI
  (editor)         agent



        fd | fzf | rg | bat | git delta
```

Why terminal editors still matter

- **SSH and servers**: No display server; your editor must be TUI or remote-aware
- **Rescue and initramfs**: Often only `vi` or `nano` exist
- **Latency**: Modal editors stay efficient on high-latency links
- **Scriptability**: `$EDITOR` hooks for git, kubectl, crontab, sudoedit
- **Agents**: Coding agents live in terminals; workspaces must outlive laptop sleep

Editor families (quick map)

Family	Feel	Best when
vi / Vim / Neovim	Modal, action-oriented	Everywhere, plugins, muscle memory
LazyVim	Neovim distribution	Fast IDE-like Neovim without building from zero
Helix / Kakoune	Selection-first modal	Modern defaults, multi-cursor, less config
nano / Micro	Modeless, discoverable	Quick edits, newcomers
Emacs	Self-contained environment	Deep customization, org, mail, magit
Zed	Fast GUI (native)	Local projects, multiplayer, GPU-accelerated UI

Deep chapters later cover each in turn.

Workspace tools (quick map)

Tool	Strength	Weakness
tmux	Ubiquitous, scriptable, SSH staple	Defaults feel dated; config is a project
GNU screen	Ancient availability	Weaker ergonomics than tmux
Zellij	Friendly defaults, layouts	Less universal on bare servers
Herdr	Agent state, detachable agent herds, API	Newer; focused on agent workflows

If you only learn one classic multiplexer, learn **tmux**. If you run multiple AI coding agents, add **Herdr**.

A sane default stack (2026)

For day-to-day Linux development:

```
# Editor (pick one)
export EDITOR=nvim          # or hx, micro, code --wait, zed --wait

# Search / browse
# ripgrep, fd, fzf, bat, eza, delta, zoxide

# Session
# tmux for long SSH + builds
```

```
# herdr when juggling Claude Code / Codex / opencode / etc.
```

Example daily flow:

1. `herdr` or `tmux new -s work`
2. Left pane: editor (`nvim`, `hx`, or Zed on the host)
3. Right pane: shell, tests, or an agent
4. Detach (`Ctrl-b d` in tmux, or Herdr detach) without killing jobs

Learning order inside this book

1. **Choosing an editor** — decision criteria
2. **Terminal workspaces** — mental model shared by tmux/Herdr/Zellij
3. **One classic editor** — vi or nano so you are never stuck
4. **One daily driver** — Neovim/LazyVim, Helix, or Micro
5. **tmux** — sessions, windows, panes, copy-mode
6. **Herdr** — agent-aware workspaces
7. **Zed** — if you want a modern GUI on Linux desktops
8. **Modern tools** — upgrade the shell around the editor

Practice goals

After the foundations:

- ☐ Open, edit, save, and exit with **both** nano and vi
- ☐ Create a tmux session, split panes, detach, reattach over SSH
- ☐ Set `$EDITOR` and use `git commit` so it opens your editor
- ☐ Search a repo with `rg` and open a hit in your editor
- ☐ (Optional) Run two coding agents under Herdr and jump to a blocked pane

What “deepen” means here

Each editor chapter aims for **production usefulness**, not marketing blurbs:

- Install on common distros
- Modal/modeless model explained clearly
- Navigation, edit, search, buffers
- Config file locations and a sensible starter config
- Integration with git, LSP (where relevant), and multiplexers
- Migration notes from adjacent tools

Start with Choosing an Editor or jump straight to the part that matches your path.

Choosing an Editor

Choosing an Editor

There is no universal best editor. There is a best fit for **constraints** (SSH? rescue? GPU GUI?), **tasks** (one-line config vs multi-package refactor), and **time** you will invest in config and muscle memory.

Decision matrix

Constraint	Prefer
Broken system, single-user mode	`vi` (often busybox/nvi) or `nano`
Fresh server, no packages yet	whatever is installed: usually `vi`/`vim`/`nano`
Long SSH session, many panes	Editor + **tmux** or **Herdr**
Want IDE features with little config	**LazyVim**, **Helix**, **Zed**, or VSCodium
Love customization / Lisp	**Emacs**
Hate modes	**nano**, **Micro**, Zed/VS Code
Maximum portability of skills	**vi** family
Multiple AI agents	**Herdr** + any editor
Low latency typing feel	modal TUI (Vim/Helix) or native GUI (**Zed**)

Time investment vs power

```
Low setup                          High setup
nano → Micro → Helix → LazyVim → Neovim → Emacs
         ↑               ↑
       Zed          Vim configs
```

- **Low setup**: useful in an afternoon
- **Medium**: productive in a week
- **High**: multi-month craft (full Neovim/Emacs ecosystems)

Helix and LazyVim intentionally sit in the "powerful but opinionated defaults" band. Classic Vim and Emacs reward long investment.

Modal vs modeless

Modeless (nano, Micro, most GUIs): typing always inserts text; commands use chords (`Ctrl+…`) or menus.

Modal (vi, Vim, Neovim, Helix, Kakoune): normal mode is for verbs and motions; insert mode is for typing. Modes feel strange for a day and then become a force multiplier for structured edits.

Selection-first (Helix, Kakoune): select the region, then act. Vim-style is often **action + motion** (`dw` = delete word). Both are modal; the order differs.

GUI vs TUI vs remote

Kind	Examples	Notes
TUI	nvim, hx, nano, emacs -nw	Best over SSH; pair with tmux/Herdr
Native GUI	Zed, gvim, Emacs GUI	Best local UX; use remote mounts/SSHFS carefully
Electron GUI	VS Code / Codium	Huge ecosystem; heavier RAM
Remote bridge	code-server, SSH remote	GUI feel on remote machine

On a headless VPS, prioritize TUI. On a Linux workstation, Zed or a terminal editor inside a nice terminal (Kitty, WezTerm, Ghostty, Alacritty) is common.

Compatibility with the rest of your stack

Ask:

1. Does `$EDITOR` work for git, crontab, kubectl edit?
2. Does it speak **LSP** for your languages?
3. Can it run inside **tmux** without keybinding wars?
4. Can you open it from **fzf** / file managers?
5. For agents: does the tool leave a real PTY (Herdr/tmux care about this)?

Example git wiring:

```
git config --global core.editor "nvim"
# or: "hx"
# or: "zed --wait"
# or: "code --wait"
```

Recommendation cheat sheet

You are…	Start with	Graduate to
New to Linux	nano	Micro or Helix
Sysadmin	vi + nano	Vim + tmux
App developer (terminal)	LazyVim or Helix	custom Neovim
App developer (desktop)	Zed or VSCodium	same + tmux for servers
Polyglot power user	Neovim or Emacs	your own distro
Running AI coding agents	any editor + **Herdr**	scripted Herdr workspaces

Anti-patterns

- Switching editors every week before learning motions
- Copy-pasting a 2 000-line Neovim config you do not understand
- Using only a GUI and freezing when SSH drops you into `vi`
- Nested multiplexers without a plan (tmux inside Herdr inside tmux)
- Fighting keybindings: leave **prefix** keys alone until you need them

Pick one for thirty days

1. Choose **one** daily editor from this book
2. Choose **one** session tool (tmux for general, Herdr for agents)
3. Install **rg**, **fd**, **fzf**, **bat**
4. Write a half-page cheatsheet of *your* ten commands
5. Only then evaluate a second editor

Next: Terminal Workspaces for the session layer, then dive into a concrete editor part.

Terminal Workspaces

Terminal Workspaces

Editors change buffers. **Workspaces** change how long processes live, how many shells you juggle, and whether closing a laptop kills a four-hour test or an AI agent.

This chapter is the shared mental model for **tmux**, **screen**, **Zellij**, and **Herdr**. Dedicated chapters go deeper; modern-tools covers Zellij in the CLI toolkit chapter.

Problems workspaces solve

Problem	Without workspace	With workspace
SSH disconnect	Shells die	Detach; processes continue
Many tasks	Terminal tabs on the client only	Server-side panes/windows
Pairing / reattach	Lost context	Named sessions
AI agents	One agent per terminal window; easy to lose	Herdr tracks agent state across panes
Layout	Rebuild every login	Scripted layouts

Core concepts (shared vocabulary)

Session

A **session** is a named container for state that outlives a single client connection.

```
# tmux-shaped idea
tmux new -s api
tmux attach -t api
tmux ls
```

Herdr uses a background **server** plus attachable clients; named sessions group workspaces similarly.

Window / tab

A **window** (tmux) or **tab** (Zellij/Herdr) is a full-screen layout unit. You switch windows to change "desktops" inside the session.

Pane

A **pane** is a split region inside a window: editor left, tests right, logs below.

```
    editor       shell

                 agent
```

Prefix key

Most multiplexers use a **prefix** chord so keybindings do not steal from Vim/Emacs.

Tool	Default prefix (typical)
tmux	`Ctrl-b` then key
screen	`Ctrl-a` then key
Zellij	mode-based (`Ctrl-p`, `Ctrl-t`, …)
Herdr	`Ctrl-b` familiar for tmux users; also mouse-first

Rule: if keys "do nothing" in Vim, check you are not inside a multiplexer waiting for a second key after prefix.

Detach / attach

- **Detach**: client disconnects; session keeps running
- **Attach**: reconnect from another terminal or host

```
# tmux
Ctrl-b d                  # detach
tmux attach -t api        # reattach

# Herdr (conceptually)
# detach leaves the server + agents running; run herdr again to attach
```

Choosing among multiplexers

```
Need maximum portability on random servers?
    tmux (or screen if tmux missing)

Want nicer defaults and layouts without much config?
    Zellij
```

```
Running Claude Code / Codex / opencode / other agent CLIs?
    Herdr (can still live inside or beside tmux)

Only local GUI terminal tabs?
    Fine for simple work; still learn tmux for remote
```

When not to nest

Avoid:

```
tmux → herdr → tmux → nvim
```

Prefer one outer session manager:

```
herdr
    nvim
    agent
    shell

# or

tmux
    nvim
    shell
    optional: herdr only if you need agent UI features
```

Herdr docs note it can run **inside** tmux as the outer terminal environment, but agent detection does not inspect tmux sessions *launched inside* a Herdr pane. Keep the layering intentional.

Editor integration patterns

Pattern A — editor full window

```
Window 1: nvim
Window 2: shell / tests
Window 3: docs / top
```

Simple; great with Vim tabs/buffers for files.

Pattern B — editor + shell split

```
Pane left: hx or nvim
Pane right: cargo test / npm run dev
```

Classic pair-programming-with-yourself layout.

Pattern C — agent herd (Herdr)

```
Workspace "api"
  Tab agents: Claude Code | Codex | opencode
  Tab runtime: dev server | logs
```

Herdr surfaces **working** / **blocked** / **idle** so you jump to the agent that needs input.

Pattern D — remote box

```
ssh build-box
tmux attach -t build || tmux new -s build
# or
herdr --remote build-box
```

Copy mode and scrollback

Terminal multiplexers own **scrollback**. Mouse scrolling may enter copy-mode.

tmux essentials:

```
Ctrl-b [          # copy mode
# move with vi keys if mode-keys vi
Space / Enter     # start/end selection (emacs mode defaults vary)
Ctrl-b ]          # paste buffer
```

Configure vi keys:

```
# ~/.tmux.conf
set -g mode-keys vi
set -g status-keys vi
```

Persistence expectations

Event	tmux/screen/Zellij	Herdr
Close terminal emulator	Session survives if server running	Server keeps panes
SSH drop	Session survives	Session survives
Reboot	Dead unless you script restore	Layout restore / agent resume features (see Herdr docs)
`kill-server` / stop	Everything dies	Everything dies

Do not assume reboot survival without explicit tooling (systemd user services, tmux-resurrect, Herdr session state features).

Minimal practice lab

```
# 1. Session
tmux new -s lab

# 2. Split
Ctrl-b %          # vertical split
Ctrl-b "          # horizontal split
Ctrl-b o          # next pane

# 3. Run something long
ping -c 1000 1.1.1.1

# 4. Detach and kill the terminal window entirely
Ctrl-b d

# 5. Prove it lived
tmux attach -t lab
```

Then repeat a similar flow after reading the **tmux** and **Herdr** parts.

Keybindings to keep conflict-free

If you use Vim/Neovim:

- Prefer tmux prefix `Ctrl-b` (default) or `Ctrl-a` only if you rebind screen habits carefully
- In Neovim, avoid binding the same chords tmux uses without `passthrough`
- For Helix/Zed, conflicts are fewer; still test copy-mode and pane zoom

Next chapters

- **tmux** — the portable standard for SSH and long jobs
- **Herdr** — agent-native workspaces and CLI/API
- **Workflows** — concrete editor + multiplexer recipes
- **Modern tools** — Zellij, prompts, file jumpers, git pagers

The workspace layer is not optional once your work outlives a single shell. Learn detach/attach early; everything else is efficiency.

Vi

Vi: The Original Visual Editor

Vi (Visual Interface) is the original screen-oriented text editor for Unix systems. Created by Bill Joy in 1976, it remains one of the most fundamental and ubiquitous editors in the Unix/Linux world.

Why Learn Vi?

- **Universal availability**: Found on every Linux distribution and Unix system
- **Minimal resource usage**: Works even on minimal Linux installations and embedded systems
- **Foundation knowledge**: Understanding vi helps with vim, neovim, and other vi-like editors
- **Emergency situations**: Often the only editor available in Linux rescue modes and single-user mode
- **System administration**: Essential for Linux server management and configuration
- **POSIX compliance**: Standardized behavior across Linux distributions

Vi Modes

Vi operates in different modes, which is its defining characteristic:

1. Command Mode (Normal Mode)

- Default mode when vi starts
- Used for navigation and text manipulation commands
- Press `Esc` to return to this mode

2. Insert Mode

- Used for typing text
- Enter with `i`, `a`, `o`, or other insert commands
- Exit with `Esc`

3. Command-Line Mode

- Used for file operations and advanced commands
- Enter with `:` from command mode
- Execute with `Enter`

Basic Operations

Starting Vi

```
# Open new file
vi filename.txt

# Open existing file
vi /path/to/file.txt

# Open file at specific line
vi +25 filename.txt

# Open file at first occurrence of pattern
vi +/pattern filename.txt

# Open multiple files
vi file1.txt file2.txt file3.txt
```

Exiting Vi

```
# Save and quit
:wq

# Quit without saving
:q!

# Save file
:w

# Save as new filename
:w newfilename.txt

# Quit (only if no changes)
:q
```

Navigation Commands

Basic Movement

```
# Character movement
h    # Move left
j    # Move down
k    # Move up
l    # Move right

# Word movement
w    # Move to beginning of next word
b    # Move to beginning of previous word
e    # Move to end of current word

# Line movement
0    # Move to beginning of line
^    # Move to first non-blank character
$    # Move to end of line

# Screen movement
H    # Move to top of screen
M    # Move to middle of screen
L    # Move to bottom of screen
```

Advanced Navigation

```
# Page movement
Ctrl+f   # Page forward
Ctrl+b   # Page backward
Ctrl+d   # Half page down
Ctrl+u   # Half page up

# Line jumping
G        # Go to last line
1G       # Go to first line
25G      # Go to line 25
gg       # Go to first line (vim extension)

# Pattern searching
/pattern    # Search forward for pattern
?pattern    # Search backward for pattern
n           # Next search result
N           # Previous search result
```

```
# Character finding
fx       # Find next 'x' on current line
Fx       # Find previous 'x' on current line
tx       # Move to character before next 'x'
Tx       # Move to character after previous 'x'
;        # Repeat last f, F, t, or T command
,        # Repeat last f, F, t, or T command in reverse
```

Text Editing Commands

Inserting Text

```
i    # Insert before cursor
a    # Insert after cursor
I    # Insert at beginning of line
A    # Insert at end of line
o    # Open new line below current line
O    # Open new line above current line
```

Deleting Text

```
x    # Delete character under cursor
X    # Delete character before cursor
dd   # Delete entire line
D    # Delete from cursor to end of line
dw   # Delete word
db   # Delete word backward
d$   # Delete to end of line
d0   # Delete to beginning of line
```

Changing Text

```
r    # Replace single character
R    # Replace mode (overwrite)
cw   # Change word
cc   # Change entire line
C    # Change from cursor to end of line
s    # Substitute character
S    # Substitute line
```

Copying and Pasting

```
yy   # Yank (copy) entire line
Y    # Yank entire line
yw   # Yank word
y$   # Yank to end of line
p    # Paste after cursor
P    # Paste before cursor
```

Working with Multiple Files

```
# Open next file
:n

# Open previous file
:prev

# List all files
:args

# Edit new file
:e filename.txt

# Switch between files
:b filename
```

Search and Replace

Basic Search

```
# Search forward
/search_term

# Search backward
?search_term

# Case-sensitive search
/\Csearch_term

# Whole word search
/\<word\>
```

Search and Replace

```
# Replace first occurrence on current line
:s/old/new/

# Replace all occurrences on current line
:s/old/new/g

# Replace all occurrences in file
:%s/old/new/g

# Replace with confirmation
:%s/old/new/gc

# Replace in specific line range
:1,10s/old/new/g
```

Advanced Features

Marks and Bookmarks

```
# Set mark 'a' at current position
ma

# Go to mark 'a'
'a

# Go to exact position of mark 'a'
`a

# List all marks
:marks
```

Registers

```
# Yank to register 'a'
"ayy

# Paste from register 'a'
"ap

# View all registers
:reg
```

Macros

```
# Start recording macro in register 'a'
qa

# Stop recording
q

# Execute macro 'a'
@a

# Execute macro 'a' 10 times
10@a
```

Configuration

Vi Configuration File (.exrc)

```
# ~/.exrc
set number          # Show line numbers
set autoindent      # Auto-indent new lines
set tabstop=4       # Set tab width
set shiftwidth=4    # Set indent width
set showmatch       # Show matching brackets
set ignorecase      # Ignore case in searches
set wrapscan        # Wrap searches around file
```

Environment Variables

```
# Set default editor
export EDITOR=vi
export VISUAL=vi

# Set vi options
export EXINIT='set number autoindent tabstop=4'
```

Real-World Scenarios

Scenario 1: Emergency Linux System Recovery

```
# Linux system in single-user mode, only vi available
# Edit critical configuration file
vi /etc/fstab

# Navigate to problematic line causing boot failure
/UUID=bad-uuid

# Delete the problematic line
dd

# Save and exit
:wq

# Alternative: Fix systemd service file
vi /etc/systemd/system/problematic.service

# Comment out problematic line
I# <Esc>

# Save and exit
:wq

# Reload systemd and reboot
:!systemctl daemon-reload && reboot
```

Scenario 2: Linux Log File Analysis

```
# Open systemd journal or log file
vi /var/log/syslog
# or
vi /var/log/systemd/journal.log

# Go to end of file to see recent entries
G

# Search backward for critical errors
?CRITICAL
?ERROR
?Failed

# Navigate through error occurrences
```

```
N    # Previous error
n    # Next error

# Copy error line for analysis
yy

# Open new file to save errors
:e /tmp/error_analysis.txt

# Paste error
p

# Add timestamp and system info
O
# Type analysis header
System: $(hostname) - $(date)
Error Analysis:
<Esc>

# Save and return to log
:w
:b syslog
```

Scenario 3: Linux System Configuration

```
# Open multiple Linux configuration files
vi /etc/hosts /etc/resolv.conf /etc/hostname

# Edit first file (/etc/hosts)
# Add new host entry for local network
o
192.168.1.100    newserver.local
192.168.1.101    database.local
192.168.1.102    webserver.local

# Save and move to next file
:w
:n

# Edit resolv.conf for DNS configuration
# Change nameserver to Google DNS
/nameserver
cw
nameserver 8.8.8.8<Esc>
o
```

```
nameserver 8.8.4.4

# Save and move to next file
:w
:n

# Edit hostname for system identification
# Replace entire content
:%d
i
production-server

# Save all and exit
:wq

# Verify changes took effect
:!hostnamectl status
```

Vi vs Modern Editors

Advantages of Vi

- **Ubiquity**: Available everywhere
- **Speed**: Fast startup and operation
- **Keyboard-driven**: No mouse required
- **Powerful**: Complex operations with few keystrokes
- **Stable**: Unchanged interface for decades

Limitations

- **Learning curve**: Modal editing is initially confusing
- **Limited features**: No syntax highlighting, plugins, etc.
- **No GUI**: Text-only interface
- **Basic functionality**: Lacks modern IDE features

Tips and Best Practices

Essential Tips

1. **Master the modes**: Understanding when you're in which mode is crucial
2. **Use Esc liberally**: When in doubt, press Esc to return to command mode
3. **Learn movement commands**: Efficient navigation is key to vi mastery
4. **Practice regularly**: Vi skills deteriorate without use
5. **Use :help**: Built-in help system is comprehensive

Common Mistakes

1. **Typing in command mode**: Results in unintended commands
2. **Forgetting to save**: Always :w before :q
3. **Case sensitivity**: Commands are case-sensitive
4. **Not using counts**: Numbers multiply commands (5dd deletes 5 lines)

Productivity Shortcuts

```
# Repeat last command
.

# Undo last change
u

# Join lines
J

# Change case
~

# Increment number
Ctrl+a

# Decrement number
Ctrl+x
```

Troubleshooting

Common Issues

1. **Can't exit vi**

   ```
   # Press Esc first, then
   :q!
   ```

2. **Accidentally in replace mode**

   ```
   # Press Esc to return to command mode
   Esc
   ```

3. **Lost changes**

   ```
   # Check for swap file
   :recover
   ```

4. **File is read-only**

```
# Force write
:w!
# Or save as new file
:w newfilename
```

Recovery Options

```
# Recover from swap file
vi -r filename.txt

# List available swap files
vi -r

# Start vi in recovery mode
vi -r
```

Vi remains an essential skill for any Linux user or system administrator. While modern editors offer more features, vi's simplicity, universality, and efficiency make it invaluable for quick edits, system recovery, and situations where other editors aren't available. ## Next steps

- Vim — everyday improved vi
- tmux — keep vi sessions alive over SSH
- Choosing an Editor — when to graduate beyond vi

Vim

Vim: Vi Improved

Vim (Vi Improved) is an enhanced version of the vi editor, created by Bram Moolenaar in 1991. It extends vi with numerous features while maintaining backward compatibility, making it one of the most popular and powerful text editors for developers.

Why Choose Vim?

- **Powerful and extensible**: Thousands of plugins and customizations
- **Efficient editing**: Modal editing with powerful commands
- **Native Linux integration**: Seamlessly integrates with Linux workflows
- **Active community**: Large ecosystem of plugins and configurations
- **Programming-focused**: Excellent support for Linux development workflows
- **Highly customizable**: Extensive configuration options
- **System administration**: Perfect for Linux server management and configuration

Installation

Linux Distributions

```
# Ubuntu/Debian
sudo apt update
sudo apt install vim

# CentOS/RHEL/Fedora
sudo yum install vim
# or
sudo dnf install vim

# Arch Linux
sudo pacman -S vim

# Alpine Linux
sudo apk add vim
```

Building from Source

```
# Clone Vim repository
git clone https://github.com/vim/vim.git
cd vim

# Configure build
./configure --with-features=huge \
            --enable-multibyte \
            --enable-python3interp \
            --enable-gui=gtk3 \
            --enable-cscope

# Compile and install
make
sudo make install
```

Vim Modes

Vim extends vi's modal concept with additional modes:

1. Normal Mode

- Default mode for navigation and commands
- Press `Esc` to enter from any other mode

2. Insert Mode

- For typing text
- Enter with `i`, `a`, `o`, `I`, `A`, `O`

3. Visual Mode

- For selecting text
- `v` for character-wise selection
- `V` for line-wise selection
- `Ctrl+v` for block-wise selection

4. Command-Line Mode

- For ex commands
- Enter with `:`, `/`, `?`

5. Replace Mode

- For overwriting text
- Enter with `R`

Enhanced Navigation

Advanced Movement

```
# Word movement (enhanced)
w    # Next word start
W    # Next WORD start (whitespace-separated)
b    # Previous word start
B    # Previous WORD start
e    # End of word
E    # End of WORD
ge   # End of previous word

# Paragraph movement
{    # Previous paragraph
}    # Next paragraph

# Sentence movement
(    # Previous sentence
)    # Next sentence

# Function movement (programming)
[[   # Previous function
]]   # Next function
[{   # Previous unmatched {
]}   # Next unmatched }

# Jump list navigation
Ctrl+o   # Jump to previous location
Ctrl+i   # Jump to next location
```

Window and Tab Management

```
# Window splitting
:split filename    # Horizontal split
:vsplit filename   # Vertical split
Ctrl+w s          # Split current window horizontally
Ctrl+w v          # Split current window vertically
```

```
# Window navigation
Ctrl+w h          # Move to left window
Ctrl+w j          # Move to bottom window
Ctrl+w k          # Move to top window
Ctrl+w l          # Move to right window
Ctrl+w w          # Cycle through windows

# Window resizing
Ctrl+w +          # Increase height
Ctrl+w -          # Decrease height
Ctrl+w >          # Increase width
Ctrl+w <          # Decrease width
Ctrl+w =          # Equalize window sizes

# Tab management
:tabnew filename  # Open new tab
:tabclose         # Close current tab
gt                # Next tab
gT                # Previous tab
:tabs             # List all tabs
```

Advanced Text Editing

Visual Mode Operations

```
# Select text and perform operations
v                 # Start visual selection
V                 # Select entire lines
Ctrl+v           # Block selection

# Operations on selected text
d                # Delete selection
y                # Yank (copy) selection
c                # Change selection
>                # Indent selection
<                # Unindent selection
=                # Auto-indent selection
```

Advanced Copy/Paste

```
# Named registers
"ayy              # Yank line to register 'a'
"ap               # Paste from register 'a'
"Ayy              # Append line to register 'a'

# Special registers
"+y               # Yank to system clipboard
"+p               # Paste from system clipboard
"*y               # Yank to selection clipboard
"*p               # Paste from selection clipboard
""p               # Paste from unnamed register
".p               # Paste last inserted text
```

Text Objects

```
# Inner text objects
iw                # Inner word
is                # Inner sentence
ip                # Inner paragraph
it                # Inner tag
i"                # Inner quotes
i(                # Inner parentheses
i{                # Inner braces
i[                # Inner brackets

# Around text objects
aw                # Around word (includes spaces)
as                # Around sentence
ap                # Around paragraph
at                # Around tag
a"                # Around quotes
a(                # Around parentheses
a{                # Around braces
a[                # Around brackets

# Usage examples
diw               # Delete inner word
ci"               # Change inner quotes
yap               # Yank around paragraph
```

Configuration and Customization

Vimrc Configuration

```
" ~/.vimrc - Basic configuration

" General settings
set nocompatible              " Disable vi compatibility
set number                    " Show line numbers
set relativenumber           " Show relative line numbers
set ruler                     " Show cursor position
set showcmd                   " Show command in status line
set showmatch                " Show matching brackets
set hlsearch                 " Highlight search results
set incsearch                " Incremental search
set ignorecase               " Ignore case in search
set smartcase                " Smart case sensitivity
set autoindent               " Auto-indent new lines
set smartindent              " Smart indentation
set expandtab                " Use spaces instead of tabs
set tabstop=4                " Tab width
set shiftwidth=4             " Indent width
set softtabstop=4            " Soft tab width
set wrap                     " Wrap long lines
set linebreak                " Break lines at word boundaries
set scrolloff=5              " Keep 5 lines visible around cursor
set sidescrolloff=5          " Keep 5 columns visible around cursor
set backspace=indent,eol,start " Allow backspace over everything
set wildmenu                 " Enhanced command completion
set wildmode=longest:full,full " Command completion mode
set laststatus=2             " Always show status line
set encoding=utf-8           " Use UTF-8 encoding
set fileencoding=utf-8       " File encoding
set termencoding=utf-8       " Terminal encoding

" Visual settings
syntax enable                " Enable syntax highlighting
set background=dark          " Dark background
colorscheme desert           " Color scheme
set cursorline              " Highlight current line
set colorcolumn=80          " Show column at 80 characters

" Search settings
set path+=**                " Search in subdirectories
set wildignore+=*.o,*.obj,*.pyc,*.class " Ignore compiled files
```

```
" Backup and swap settings
set backup                  " Enable backups
set backupdir=~/.vim/backup " Backup directory
set directory=~/.vim/swap   " Swap file directory
set undofile                " Persistent undo
set undodir=~/.vim/undo     " Undo directory

" Create directories if they don't exist
if !isdirectory($HOME."/.vim/backup")
    call mkdir($HOME."/.vim/backup", "p")
endif
if !isdirectory($HOME."/.vim/swap")
    call mkdir($HOME."/.vim/swap", "p")
endif
if !isdirectory($HOME."/.vim/undo")
    call mkdir($HOME."/.vim/undo", "p")
endif

" Key mappings
let mapleader = ","         " Set leader key
nnoremap <leader>w :w<CR>   " Quick save
nnoremap <leader>q :q<CR>   " Quick quit
nnoremap <leader>x :x<CR>   " Quick save and quit
nnoremap <leader>h :noh<CR> " Clear search highlighting
nnoremap <C-h> <C-w>h       " Window navigation
nnoremap <C-j> <C-w>j
nnoremap <C-k> <C-w>k
nnoremap <C-l> <C-w>l

" Function key mappings
nnoremap <F2> :set number!<CR>     " Toggle line numbers
nnoremap <F3> :set paste!<CR>      " Toggle paste mode
nnoremap <F4> :set wrap!<CR>       " Toggle line wrapping
nnoremap <F5> :source ~/.vimrc<CR> " Reload vimrc

" Auto commands
autocmd BufWritePre * :%s/\s\+$//e " Remove trailing whitespace on save
autocmd BufRead,BufNewFile *.py set filetype=python
autocmd BufRead,BufNewFile *.js set filetype=javascript
autocmd BufRead,BufNewFile *.html set filetype=html
```

Advanced Vimrc Features

```
" Advanced ~/.vimrc additions

" Status line customization
set statusline=%f         " File name
set statusline+=%m        " Modified flag
set statusline+=%r        " Read-only flag
set statusline+=%h        " Help flag
set statusline+=%w        " Preview flag
set statusline+=%=        " Right align
set statusline+=%y        " File type
set statusline+=[%{&ff}]  " File format
set statusline+=[%{&fenc}] " File encoding
set statusline+=\ %l/%L   " Line number/total lines
set statusline+=\ %c      " Column number
set statusline+=\ %P      " Percentage through file

" Custom functions
function! ToggleBackground()
    if &background == 'dark'
        set background=light
    else
        set background=dark
    endif
endfunction
nnoremap <F6> :call ToggleBackground()<CR>

" Word count function
function! WordCount()
    let s:old_status = v:statusmsg
    let position = getpos(".")
    exe ":silent normal g\<c-g>"
    let stat = v:statusmsg
    let s:word_count = 0
    if stat != '--No lines in buffer--'
        let s:word_count = str2nr(split(v:statusmsg)[11])
        let v:statusmsg = s:old_status
    end
    call setpos('.', position)
    return s:word_count
endfunction

" File type specific settings
autocmd FileType python setlocal expandtab shiftwidth=4 softtabstop=4
autocmd FileType javascript setlocal expandtab shiftwidth=2 softtabstop=2
autocmd FileType html setlocal expandtab shiftwidth=2 softtabstop=2
```

```
autocmd FileType css setlocal expandtab shiftwidth=2 softtabstop=2
autocmd FileType yaml setlocal expandtab shiftwidth=2 softtabstop=2
autocmd FileType markdown setlocal wrap linebreak nolist textwidth=0 wrapmargin=0

" Abbreviations
iabbrev teh the
iabbrev adn and
iabbrev seperate separate
iabbrev recieve receive
```

Plugin Management

Using vim-plug

```
# Install vim-plug
curl -fLo ~/.vim/autoload/plug.vim --create-dirs \
    https://raw.githubusercontent.com/junegunn/vim-plug/master/plug.vim
```

```
" Add to ~/.vimrc
call plug#begin('~/.vim/plugged')

" Essential plugins
Plug 'tpope/vim-sensible'           " Sensible defaults
Plug 'tpope/vim-surround'           " Surround text objects
Plug 'tpope/vim-commentary'         " Comment/uncomment
Plug 'tpope/vim-fugitive'           " Git integration
Plug 'scrooloose/nerdtree'          " File explorer
Plug 'ctrlpvim/ctrlp.vim'           " Fuzzy file finder
Plug 'vim-airline/vim-airline'      " Status line
Plug 'vim-airline/vim-airline-themes' " Airline themes
Plug 'airblade/vim-gitgutter'       " Git diff in gutter
Plug 'dense-analysis/ale'           " Linting and fixing
Plug 'ycm-core/YouCompleteMe'       " Code completion
Plug 'preservim/tagbar'             " Tag browser
Plug 'junegunn/fzf', { 'do': { -> fzf#install() } }
Plug 'junegunn/fzf.vim'             " FZF integration

" Language-specific plugins
Plug 'vim-python/python-syntax'     " Python syntax
Plug 'pangloss/vim-javascript'      " JavaScript syntax
Plug 'mxw/vim-jsx'                  " JSX syntax
Plug 'rust-lang/rust.vim'          " Rust support
Plug 'fatih/vim-go'                 " Go support
```

```
" Color schemes
Plug 'morhetz/gruvbox'               " Gruvbox theme
Plug 'dracula/vim', { 'as': 'dracula' } " Dracula theme
Plug 'joshdick/onedark.vim'          " One Dark theme

call plug#end()

" Plugin configurations
let g:airline_theme='gruvbox'
let g:gruvbox_contrast_dark='hard'
colorscheme gruvbox

" NERDTree configuration
nnoremap <F7> :NERDTreeToggle<CR>
let NERDTreeShowHidden=1

" CtrlP configuration
let g:ctrlp_map = '<c-p>'
let g:ctrlp_cmd = 'CtrlP'
let g:ctrlp_working_path_mode = 'ra'

" ALE configuration
let g:ale_linters = {
\   'python': ['flake8', 'pylint'],
\   'javascript': ['eslint'],
\}
let g:ale_fixers = {
\   'python': ['autopep8', 'black'],
\   'javascript': ['prettier'],
\}
```

Installing Plugins

```
" In Vim command mode
:PlugInstall    " Install plugins
:PlugUpdate     " Update plugins
:PlugClean      " Remove unused plugins
:PlugStatus     " Check plugin status
```

Programming with Vim

Code Navigation

```
# Tag navigation (requires ctags)
Ctrl+]          # Jump to tag definition
Ctrl+t          # Jump back from tag
:tag function   # Jump to specific tag
:tags           # Show tag stack

# Code folding
zf              # Create fold
zo              # Open fold
zc              # Close fold
za              # Toggle fold
zR              # Open all folds
zM              # Close all folds
```

Code Completion

```
# Built-in completion
Ctrl+n          # Next completion
Ctrl+p          # Previous completion
Ctrl+x Ctrl+f   # File name completion
Ctrl+x Ctrl+l   # Line completion
Ctrl+x Ctrl+k   # Dictionary completion
Ctrl+x Ctrl+o   # Omni completion
```

Debugging Integration

```
" GDB integration
packadd termdebug
:Termdebug program_name

" Debugging commands
:Break          " Set breakpoint
:Clear          " Clear breakpoint
:Step           " Step into
:Over           # Step over
:Continue       " Continue execution
```

Real-World Development Scenarios

Scenario 1: Python Development Setup

```
" Python-specific vimrc additions
autocmd FileType python setlocal expandtab shiftwidth=4 softtabstop=4
autocmd FileType python setlocal textwidth=79
autocmd FileType python setlocal colorcolumn=80
autocmd FileType python nnoremap <buffer> <F9> :exec '!python' shellescape(@%, 1)<cr>

" Python debugging
autocmd FileType python nnoremap <buffer> <F10> :exec '!python -m pdb' shellescape(@%, 1)<cr

" Virtual environment detection
py3 << EOF
import os
import sys
if 'VIRTUAL_ENV' in os.environ:
    project_base_dir = os.environ['VIRTUAL_ENV']
    activate_this = os.path.join(project_base_dir, 'bin/activate_this.py')
    exec(open(activate_this).read(), dict(__file__=activate_this))
EOF
```

Scenario 2: Web Development Workflow

```
" Web development configuration
autocmd FileType html,css,javascript setlocal expandtab shiftwidth=2 softtabstop=2

" Live reload setup
autocmd BufWritePost *.html,*.css,*.js silent !browser-sync reload

" Emmet integration
let g:user_emmet_leader_key=','

" Auto-close tags
let g:closetag_filenames = '*.html,*.xhtml,*.phtml,*.jsx'

" Prettier formatting
autocmd BufWritePre *.js,*.jsx,*.css,*.html PrettierAsync
```

Scenario 3: Linux System Administration

```
" Linux configuration file editing
autocmd BufRead,BufNewFile /etc/* setlocal nobackup nowritebackup
autocmd BufRead,BufNewFile /etc/systemd/system/* setlocal filetype=systemd
autocmd BufRead,BufNewFile *.conf setlocal syntax=conf
autocmd BufRead,BufNewFile /etc/nginx/* setlocal filetype=nginx
autocmd BufRead,BufNewFile /etc/apache2/* setlocal filetype=apache

" Linux log file viewing
autocmd BufRead,BufNewFile /var/log/* setlocal readonly
autocmd BufRead,BufNewFile /var/log/* normal G
autocmd BufRead,BufNewFile /var/log/syslog setlocal filetype=messages
autocmd BufRead,BufNewFile /var/log/auth.log setlocal filetype=messages

" Sudo save function for system files
command! W w !sudo tee % > /dev/null
command! Wsudo w !sudo tee % > /dev/null

" Quick Linux config editing
nnoremap <leader>ev :vsplit $MYVIMRC<cr>
nnoremap <leader>sv :source $MYVIMRC<cr>
nnoremap <leader>eh :vsplit /etc/hosts<cr>
nnoremap <leader>es :vsplit /etc/ssh/sshd_config<cr>
nnoremap <leader>en :vsplit /etc/nginx/nginx.conf<cr>

" Systemd service file templates
autocmd BufNewFile *.service 0r ~/.vim/templates/systemd.service
autocmd BufNewFile *.timer 0r ~/.vim/templates/systemd.timer

" Linux-specific commands
command! Systemctl !sudo systemctl
command! Journalctl !journalctl
command! Dmesg !dmesg | tail -20
```

Advanced Vim Techniques

Macros and Automation

```
# Recording macros
qa              # Start recording macro 'a'
# ... perform actions ...
q               # Stop recording
```

```
# Playing macros
@a              # Execute macro 'a'
@@              # Repeat last macro
10@a            # Execute macro 'a' 10 times

# Editing macros
:let @a='...'   # Edit macro 'a' directly
```

Regular Expressions

```
# Search patterns
/\v<word>       # Very magic mode - exact word
/\c<Word>       # Case insensitive search
/\C<Word>       # Case sensitive search

# Substitution patterns
:%s/\v(\w+)\s+(\w+)/\2 \1/g    # Swap words
:%s/\v^(.{80}).*/\1/g          # Truncate lines to 80 chars
:%s/\v\s+$//g                  # Remove trailing whitespace
```

Custom Commands

```
" Custom commands in vimrc
command! -nargs=1 Search execute 'vimgrep /<args>/gj **/*.py' | copen
command! Todo execute 'vimgrep /TODO\|FIXME\|XXX/gj **/*' | copen
command! RemoveTrailingSpaces %s/\s\+$//e
command! -range=% FormatJSON <line1>,<line2>!python -m json.tool

" Usage
:Search function_name
:Todo
:RemoveTrailingSpaces
:FormatJSON
```

Performance Optimization

Vim Performance Tips

```
" Performance optimizations
set lazyredraw              " Don't redraw during macros
set ttyfast                 " Fast terminal connection
set synmaxcol=200          " Limit syntax highlighting column
```

```
set regexpengine=1         " Use old regex engine for speed

" Disable unused features
set noshowmatch            " Don't show matching brackets
let loaded_matchparen = 1  " Disable matchparen plugin
```

Large File Handling

```
" Large file handling
autocmd BufReadPre * if getfsize(expand("%")) > 10000000 | syntax off | endif

" Function for large files
function! LargeFile()
    syntax off
    set eventignore+=FileType
    setlocal bufhidden=unload
    setlocal buftype=nowrite
    setlocal undolevels=-1
endfunction
autocmd BufReadPre * if getfsize(expand("%")) > 50000000 | call LargeFile() | endif
```

Troubleshooting Common Issues

Performance Problems

```
# Profile Vim startup
vim --startuptime startup.log

# Check what's slowing down Vim
:profile start profile.log
:profile func *
:profile file *
# ... use Vim normally ...
:profile pause
:noautocmd qall!
```

Plugin Conflicts

```
# Start Vim without plugins
vim -u NONE -N
```

```
# Start with minimal config
vim -u ~/.vimrc.minimal

# Debug plugin loading
vim -V9myVim.log
```

Recovery and Backup

```
# Recover from swap file
vim -r filename.txt

# List swap files
vim -r

# Disable swap files (not recommended)
set noswapfile
```

Vim's power lies in its extensibility and efficiency. While it has a steep learning curve, mastering Vim can significantly improve your editing speed and workflow, especially for programming and system administration tasks.

Neovim

Neovim: The Future of Vim

Neovim is a modern fork of Vim that focuses on extensibility, usability, and performance. Created in 2014, it maintains Vim compatibility while adding powerful new features like built-in LSP support, Lua scripting, and better plugin architecture.

Why Choose Neovim?

- **Modern architecture**: Asynchronous job control and better plugin API
- **Built-in LSP**: Native Language Server Protocol support for Linux development
- **Lua scripting**: More powerful and faster than Vimscript
- **Better defaults**: Sensible configuration out of the box
- **Active development**: Regular updates and new features
- **Tree-sitter**: Advanced syntax highlighting and code parsing
- **Embedded terminal**: Built-in terminal emulator perfect for Linux workflows
- **Linux-first design**: Optimized for Linux development environments

Installation

Package Managers

```
# Ubuntu/Debian (latest stable)
sudo apt update
sudo apt install neovim

# Ubuntu/Debian (latest unstable)
sudo add-apt-repository ppa:neovim-ppa/unstable
sudo apt update
sudo apt install neovim

# CentOS/RHEL/Fedora
sudo dnf install neovim

# Arch Linux
sudo pacman -S neovim

# SUSE/openSUSE
sudo zypper install neovim
```

```
# Gentoo
sudo emerge app-editors/neovim
```

From Source

```
# Install dependencies (Ubuntu/Debian)
sudo apt install ninja-build gettext libtool libtool-bin autoconf automake cmake g++ pkg-con

# Clone and build
git clone https://github.com/neovim/neovim
cd neovim
make CMAKE_BUILD_TYPE=RelWithDebInfo
sudo make install
```

AppImage (Portable)

```
# Download AppImage
curl -LO https://github.com/neovim/neovim/releases/latest/download/nvim.appimage
chmod u+x nvim.appimage
sudo mv nvim.appimage /usr/local/bin/nvim
```

Configuration Structure

Neovim uses a different configuration structure than Vim:

```
# Configuration directory
~/.config/nvim/

# Main configuration file
~/.config/nvim/init.lua    # Lua configuration (recommended)
# or
~/.config/nvim/init.vim    # Vimscript configuration

# Plugin directory
~/.local/share/nvim/site/pack/
```

Basic Lua Configuration

init.lua Structure

```lua
-- ~/.config/nvim/init.lua

-- Basic settings
vim.opt.number = true
vim.opt.relativenumber = true
vim.opt.expandtab = true
vim.opt.shiftwidth = 4
vim.opt.tabstop = 4
vim.opt.softtabstop = 4
vim.opt.smartindent = true
vim.opt.wrap = false
vim.opt.swapfile = false
vim.opt.backup = false
vim.opt.undodir = os.getenv("HOME") .. "/.vim/undodir"
vim.opt.undofile = true
vim.opt.hlsearch = false
vim.opt.incsearch = true
vim.opt.termguicolors = true
vim.opt.scrolloff = 8
vim.opt.signcolumn = "yes"
vim.opt.isfname:append("@-@")
vim.opt.updatetime = 50
vim.opt.colorcolumn = "80"

-- Leader key
vim.g.mapleader = " "

-- Key mappings
vim.keymap.set("n", "<leader>pv", vim.cmd.Ex)
vim.keymap.set("v", "J", ":m '>+1<CR>gv=gv")
vim.keymap.set("v", "K", ":m '<-2<CR>gv=gv")
vim.keymap.set("n", "J", "mzJ`z")
vim.keymap.set("n", "<C-d>", "<C-d>zz")
vim.keymap.set("n", "<C-u>", "<C-u>zz")
vim.keymap.set("n", "n", "nzzzv")
vim.keymap.set("n", "N", "Nzzzv")

-- System clipboard
vim.keymap.set({"n", "v"}, "<leader>y", [["+y]])
vim.keymap.set("n", "<leader>Y", [["+Y]])
vim.keymap.set({"n", "v"}, "<leader>d", [["_d]])
```

```lua
-- Replace word under cursor
vim.keymap.set("n", "<leader>s", [[:%s/\<<C-r><C-w>\>/<C-r><C-w>/gI<Left><Left><Left>]])

-- Make file executable
vim.keymap.set("n", "<leader>x", "<cmd>!chmod +x %<CR>", { silent = true })

-- Auto commands
local augroup = vim.api.nvim_create_augroup
local autocmd = vim.api.nvim_create_autocmd

local yank_group = augroup('HighlightYank', {})
autocmd('TextYankPost', {
    group = yank_group,
    pattern = '*',
    callback = function()
        vim.highlight.on_yank({
            higroup = 'IncSearch',
            timeout = 40,
        })
    end,
})

-- Remove trailing whitespace on save
local trim_group = augroup('TrimWhitespace', {})
autocmd('BufWritePre', {
    group = trim_group,
    pattern = '*',
    command = '%s/\\s\\+$//e',
})
```

Plugin Management with Packer

Installing Packer

```
git clone --depth 1 https://github.com/wbthomason/packer.nvim\
 ~/.local/share/nvim/site/pack/packer/start/packer.nvim
```

Packer Configuration

```lua
-- ~/.config/nvim/lua/plugins.lua

-- This file can be loaded by calling `lua require('plugins')` from your init.lua
```

```
-- Only required if you have packer configured as `opt`
vim.cmd [[packadd packer.nvim]]

return require('packer').startup(function(use)
  -- Packer can manage itself
  use 'wbthomason/packer.nvim'

  -- Fuzzy finder
  use {
    'nvim-telescope/telescope.nvim', tag = '0.1.4',
    requires = { {'nvim-lua/plenary.nvim'} }
  }

  -- Color schemes
  use({
    'rose-pine/neovim',
    as = 'rose-pine',
    config = function()
      vim.cmd('colorscheme rose-pine')
    end
  })

  use('nvim-treesitter/nvim-treesitter', {run = ':TSUpdate'})
  use('nvim-treesitter/playground')
  use('theprimeagen/harpoon')
  use('mbbill/undotree')
  use('tpope/vim-fugitive')

  -- LSP Support
  use {
    'VonHeikemen/lsp-zero.nvim',
    branch = 'v3.x',
    requires = {
      --- Uncomment these if you want to manage LSP servers from neovim
      {'williamboman/mason.nvim'},
      {'williamboman/mason-lspconfig.nvim'},

      -- LSP Support
      {'neovim/nvim-lspconfig'},
      -- Autocompletion
      {'hrsh7th/nvim-cmp'},
      {'hrsh7th/cmp-nvim-lsp'},
      {'L3MON4D3/LuaSnip'},
    }
  }

  -- File explorer
```

```
use {
  'nvim-tree/nvim-tree.lua',
  requires = {
    'nvim-tree/nvim-web-devicons', -- optional, for file icons
  },
  tag = 'nightly' -- optional, updated every week
}

-- Status line
use {
  'nvim-lualine/lualine.nvim',
  requires = { 'nvim-tree/nvim-web-devicons', opt = true }
}

-- Git integration
use {
  'lewis6991/gitsigns.nvim',
  config = function()
    require('gitsigns').setup()
  end
}

-- Auto pairs
use {
  "windwp/nvim-autopairs",
  config = function() require("nvim-autopairs").setup {} end
}

-- Comments
use {
  'numToStr/Comment.nvim',
  config = function()
    require('Comment').setup()
  end
}

-- Terminal integration
use {"akinsho/toggleterm.nvim", tag = '*', config = function()
  require("toggleterm").setup()
end}

-- Which key
use {
  "folke/which-key.nvim",
  config = function()
    vim.o.timeout = true
    vim.o.timeoutlen = 300
```

```
      require("which-key").setup {}
    end
  }

  -- Indent guides
  use "lukas-reineke/indent-blankline.nvim"

  -- Buffer line
  use {'akinsho/bufferline.nvim', tag = "*", requires = 'nvim-tree/nvim-web-devicons'}

end)
```

Loading Plugins in init.lua

```
-- ~/.config/nvim/init.lua
require('plugins')

-- Plugin configurations
require('telescope').setup{}
require('nvim-tree').setup{}
require('lualine').setup{}
require('bufferline').setup{}
```

Language Server Protocol (LSP) Setup

LSP Configuration

```
-- ~/.config/nvim/lua/lsp-config.lua

local lsp_zero = require('lsp-zero')

lsp_zero.on_attach(function(client, bufnr)
  local opts = {buffer = bufnr, remap = false}

  vim.keymap.set("n", "gd", function() vim.lsp.buf.definition() end, opts)
  vim.keymap.set("n", "K", function() vim.lsp.buf.hover() end, opts)
  vim.keymap.set("n", "<leader>vws", function() vim.lsp.buf.workspace_symbol() end, opts)
  vim.keymap.set("n", "<leader>vd", function() vim.diagnostic.open_float() end, opts)
  vim.keymap.set("n", "[d", function() vim.diagnostic.goto_next() end, opts)
  vim.keymap.set("n", "]d", function() vim.diagnostic.goto_prev() end, opts)
  vim.keymap.set("n", "<leader>vca", function() vim.lsp.buf.code_action() end, opts)
  vim.keymap.set("n", "<leader>vrr", function() vim.lsp.buf.references() end, opts)
  vim.keymap.set("n", "<leader>vrn", function() vim.lsp.buf.rename() end, opts)
```

```
  vim.keymap.set("i", "<C-h>", function() vim.lsp.buf.signature_help() end, opts)
end)

-- Mason setup for LSP server management
require('mason').setup({})
require('mason-lspconfig').setup({
  ensure_installed = {
    'tsserver',
    'rust_analyzer',
    'gopls',
    'pyright',
    'lua_ls',
    'clangd',
  },
  handlers = {
    lsp_zero.default_setup,
    lua_ls = function()
      local lua_opts = lsp_zero.nvim_lua_ls()
      require('lspconfig').lua_ls.setup(lua_opts)
    end,
  }
})

-- Completion setup
local cmp = require('cmp')
local cmp_select = {behavior = cmp.SelectBehavior.Select}

cmp.setup({
  sources = {
    {name = 'path'},
    {name = 'nvim_lsp'},
    {name = 'nvim_lua'},
    {name = 'luasnip', keyword_length = 2},
    {name = 'buffer', keyword_length = 3},
  },
  formatting = lsp_zero.cmp_format(),
  mapping = cmp.mapping.preset.insert({
    ['<C-p>'] = cmp.mapping.select_prev_item(cmp_select),
    ['<C-n>'] = cmp.mapping.select_next_item(cmp_select),
    ['<C-y>'] = cmp.mapping.confirm({ select = true }),
    ['<C-Space>'] = cmp.mapping.complete(),
  }),
})
```

Installing Language Servers

```
# Using Mason (recommended)
:Mason
# Then install servers interactively

# Or install manually
npm install -g typescript-language-server
pip install python-lsp-server
go install golang.org/x/tools/gopls@latest
```

Tree-sitter Configuration

Tree-sitter Setup

```
-- ~/.config/nvim/lua/treesitter-config.lua

require'nvim-treesitter.configs'.setup {
  -- A list of parser names, or "all"
  ensure_installed = {
    "c", "lua", "vim", "vimdoc", "query", "python", "javascript",
    "typescript", "rust", "go", "html", "css", "json", "yaml", "markdown"
  },

  -- Install parsers synchronously (only applied to `ensure_installed`)
  sync_install = false,

  -- Automatically install missing parsers when entering buffer
  auto_install = true,

  highlight = {
    enable = true,
    -- Setting this to true will run `:h syntax` and tree-sitter at the same time.
    additional_vim_regex_highlighting = false,
  },

  indent = {
    enable = true
  },

  incremental_selection = {
    enable = true,
    keymaps = {
      init_selection = "gnn",
```

```
      node_incremental = "grn",
      scope_incremental = "grc",
      node_decremental = "grm",
    },
  },

  textobjects = {
    select = {
      enable = true,
      lookahead = true,
      keymaps = {
        ["af"] = "@function.outer",
        ["if"] = "@function.inner",
        ["ac"] = "@class.outer",
        ["ic"] = "@class.inner",
      },
    },
  },
}
```

Advanced Neovim Features

Telescope Configuration

```
-- ~/.config/nvim/lua/telescope-config.lua

local builtin = require('telescope.builtin')

vim.keymap.set('n', '<leader>pf', builtin.find_files, {})
vim.keymap.set('n', '<C-p>', builtin.git_files, {})
vim.keymap.set('n', '<leader>ps', function()
    builtin.grep_string({ search = vim.fn.input("Grep > ") })
end)
vim.keymap.set('n', '<leader>vh', builtin.help_tags, {})

require('telescope').setup{
  defaults = {
    file_ignore_patterns = {
      "node_modules",
      ".git",
      "dist",
      "build"
    },
    mappings = {
      i = {
```

```
        ["<C-h>"] = "which_key"
      }
    }
  },
  pickers = {
    find_files = {
      theme = "dropdown",
    }
  },
  extensions = {
    fzf = {
      fuzzy = true,
      override_generic_sorter = true,
      override_file_sorter = true,
      case_mode = "smart_case",
    }
  }
}

-- Load extensions
require('telescope').load_extension('fzf')
```

Terminal Integration

```
-- ~/.config/nvim/lua/terminal-config.lua

require("toggleterm").setup{
  size = 20,
  open_mapping = [[<c-\>]],
  hide_numbers = true,
  shade_filetypes = {},
  shade_terminals = true,
  shading_factor = 2,
  start_in_insert = true,
  insert_mappings = true,
  persist_size = true,
  direction = "float",
  close_on_exit = true,
  shell = vim.o.shell,
  float_opts = {
    border = "curved",
    winblend = 0,
    highlights = {
      border = "Normal",
      background = "Normal",
```

```
    },
  },
}

-- Terminal keymaps
function _G.set_terminal_keymaps()
  local opts = {buffer = 0}
  vim.keymap.set('t', '<esc>', [[<C-\><C-n>]], opts)
  vim.keymap.set('t', 'jk', [[<C-\><C-n>]], opts)
  vim.keymap.set('t', '<C-h>', [[<Cmd>wincmd h<CR>]], opts)
  vim.keymap.set('t', '<C-j>', [[<Cmd>wincmd j<CR>]], opts)
  vim.keymap.set('t', '<C-k>', [[<Cmd>wincmd k<CR>]], opts)
  vim.keymap.set('t', '<C-l>', [[<Cmd>wincmd l<CR>]], opts)
end

vim.cmd('autocmd! TermOpen term://* lua set_terminal_keymaps()')
```

Real-World Development Setups

Python Development

```
-- ~/.config/nvim/lua/python-config.lua

-- Python-specific settings
vim.api.nvim_create_autocmd("FileType", {
  pattern = "python",
  callback = function()
    vim.opt_local.expandtab = true
    vim.opt_local.shiftwidth = 4
    vim.opt_local.tabstop = 4
    vim.opt_local.softtabstop = 4
    vim.opt_local.colorcolumn = "88"
  end,
})

-- Python debugging with DAP
local dap = require('dap')
dap.adapters.python = {
  type = 'executable',
  command = 'python',
  args = { '-m', 'debugpy.adapter' },
}

dap.configurations.python = {
```

```
  {
    type = 'python',
    request = 'launch',
    name = "Launch file",
    program = "${file}",
    pythonPath = function()
      return '/usr/bin/python'
    end,
  },
}

-- Python testing
vim.keymap.set('n', '<leader>pt', ':!python -m pytest %<CR>')
vim.keymap.set('n', '<leader>pr', ':!python %<CR>')
```

JavaScript/TypeScript Development

```
-- ~/.config/nvim/lua/js-config.lua

-- JavaScript/TypeScript settings
vim.api.nvim_create_autocmd("FileType", {
  pattern = {"javascript", "typescript", "javascriptreact", "typescriptreact"},
  callback = function()
    vim.opt_local.expandtab = true
    vim.opt_local.shiftwidth = 2
    vim.opt_local.tabstop = 2
    vim.opt_local.softtabstop = 2
  end,
})

-- ESLint and Prettier integration
local null_ls = require("null-ls")
null_ls.setup({
  sources = {
    null_ls.builtins.formatting.prettier,
    null_ls.builtins.diagnostics.eslint,
    null_ls.builtins.code_actions.eslint,
  },
})

-- Auto-format on save
vim.api.nvim_create_autocmd("BufWritePre", {
  pattern = {"*.js", "*.jsx", "*.ts", "*.tsx"},
  callback = function()
    vim.lsp.buf.format()
```

```
  end,
})
```

Go Development

```
-- ~/.config/nvim/lua/go-config.lua

-- Go settings
vim.api.nvim_create_autocmd("FileType", {
  pattern = "go",
  callback = function()
    vim.opt_local.expandtab = false
    vim.opt_local.tabstop = 4
    vim.opt_local.shiftwidth = 4
  end,
})

-- Go-specific keymaps
vim.api.nvim_create_autocmd("FileType", {
  pattern = "go",
  callback = function()
    local opts = { buffer = true }
    vim.keymap.set('n', '<leader>gr', ':!go run %<CR>', opts)
    vim.keymap.set('n', '<leader>gt', ':!go test<CR>', opts)
    vim.keymap.set('n', '<leader>gb', ':!go build<CR>', opts)
    vim.keymap.set('n', '<leader>gf', ':!gofmt -w %<CR>', opts)
  end,
})
```

Debugging with DAP

DAP Configuration

```
-- ~/.config/nvim/lua/dap-config.lua

local dap = require('dap')
local dapui = require('dapui')

-- DAP UI setup
dapui.setup()

-- Auto-open/close DAP UI
dap.listeners.after.event_initialized["dapui_config"] = function()
```

```
  dapui.open()
end
dap.listeners.before.event_terminated["dapui_config"] = function()
  dapui.close()
end
dap.listeners.before.event_exited["dapui_config"] = function()
  dapui.close()
end

-- Keymaps
vim.keymap.set('n', '<F5>', dap.continue)
vim.keymap.set('n', '<F10>', dap.step_over)
vim.keymap.set('n', '<F11>', dap.step_into)
vim.keymap.set('n', '<F12>', dap.step_out)
vim.keymap.set('n', '<leader>b', dap.toggle_breakpoint)
vim.keymap.set('n', '<leader>B', function()
  dap.set_breakpoint(vim.fn.input('Breakpoint condition: '))
end)
vim.keymap.set('n', '<leader>lp', function()
  dap.set_breakpoint(nil, nil, vim.fn.input('Log point message: '))
end)
vim.keymap.set('n', '<leader>dr', dap.repl.open)
vim.keymap.set('n', '<leader>dl', dap.run_last)
```

Performance Optimization

Startup Optimization

```
-- ~/.config/nvim/lua/performance.lua

-- Disable unused providers
vim.g.loaded_python_provider = 0
vim.g.loaded_ruby_provider = 0
vim.g.loaded_perl_provider = 0
vim.g.loaded_node_provider = 0

-- Lazy load plugins
vim.opt.runtimepath:prepend(vim.fn.stdpath("data") .. "/lazy/lazy.nvim")

-- Optimize for large files
vim.api.nvim_create_autocmd("BufReadPre", {
  callback = function()
    local file_size = vim.fn.getfsize(vim.fn.expand("%"))
    if file_size > 1024 * 1024 then -- 1MB
```

```
      vim.opt_local.syntax = "off"
      vim.opt_local.filetype = ""
      vim.opt_local.undolevels = -1
      vim.opt_local.swapfile = false
      vim.opt_local.loadplugins = false
    end
  end,
})
```

Migration from Vim

Compatibility Layer

```
-- ~/.config/nvim/lua/vim-compat.lua

-- Vim compatibility settings
vim.opt.compatible = false
vim.opt.encoding = 'utf-8'
vim.opt.fileencoding = 'utf-8'

-- Source existing vimrc if needed
local vimrc = vim.fn.expand("~/.vimrc")
if vim.fn.filereadable(vimrc) == 1 then
  vim.cmd("source " .. vimrc)
end

-- Convert common vim settings to lua
local function convert_vim_settings()
  -- Add your vim-to-lua conversions here
  if vim.g.loaded_vimrc then
    -- Convert specific vim settings
    vim.opt.number = vim.g.vim_number or false
    vim.opt.relativenumber = vim.g.vim_relativenumber or false
  end
end

convert_vim_settings()
```

Troubleshooting

Common Issues

```
-- ~/.config/nvim/lua/troubleshooting.lua

-- Health check
vim.keymap.set('n', '<leader>h', ':checkhealth<CR>')

-- Plugin debugging
vim.keymap.set('n', '<leader>pi', ':PackerInstall<CR>')
vim.keymap.set('n', '<leader>ps', ':PackerSync<CR>')
vim.keymap.set('n', '<leader>pc', ':PackerClean<CR>')

-- LSP debugging
vim.keymap.set('n', '<leader>li', ':LspInfo<CR>')
vim.keymap.set('n', '<leader>lr', ':LspRestart<CR>')

-- Log files
-- ~/.local/share/nvim/lsp.log
-- ~/.cache/nvim/packer.nvim.log
```

Neovim represents the future of modal editing, combining Vim's efficiency with modern development tools and practices. Its built-in LSP support, Lua scripting, and active plugin ecosystem make it an excellent choice for developers seeking a powerful, customizable editor. ## Next steps

- LazyVim — batteries-included Neovim
- tmux · Herdr — session layers
- Workflows — compose a daily stack

Nano

Nano: The User-Friendly Terminal Editor

Nano is a simple, user-friendly text editor for Unix and Linux systems. Created as a free replacement for the Pico editor, nano provides an intuitive interface with on-screen help, making it ideal for beginners and quick edits.

Why Choose Nano?

- **Beginner-friendly**: Easy to learn with on-screen help
- **No modes**: Direct typing without modal complexity
- **Lightweight**: Minimal resource usage, perfect for Linux servers
- **Universal**: Available on all Linux distributions by default
- **Quick edits**: Perfect for Linux configuration files and system administration
- **Syntax highlighting**: Built-in support for many file types
- **System administration**: Ideal for editing Linux config files and scripts

Installation

Most Linux Distributions (Pre-installed)

```
# Check if nano is installed
which nano
nano --version

# If not installed:
# Ubuntu/Debian
sudo apt update
sudo apt install nano

# CentOS/RHEL/Fedora
sudo yum install nano
# or
sudo dnf install nano

# Arch Linux
sudo pacman -S nano

# Alpine Linux
```

```
sudo apk add nano
```

Additional Linux Distributions

```
# SUSE/openSUSE
sudo zypper install nano

# Gentoo
sudo emerge app-editors/nano

# Void Linux
sudo xbps-install nano
```

Basic Usage

Starting Nano

```
# Create new file
nano filename.txt

# Edit existing file
nano /path/to/file.txt

# Open file at specific line
nano +25 filename.txt

# Open file as read-only
nano -v filename.txt

# Open with line numbers
nano -l filename.txt

# Open with syntax highlighting disabled
nano -Y no filename.txt
```

Interface Overview

```
  GNU nano 6.2                    filename.txt                    Modified

This is the content of your file.
You can type directly here.
No modes to worry about!
```

```
^G Get Help    ^O Write Out   ^W Where Is    ^K Cut Text    ^J Justify
^X Exit        ^R Read File   ^\ Replace     ^U Paste Text  ^T To Spell
```

The bottom shows available commands where ^ means Ctrl key.

Essential Commands

File Operations

```
# Save file
Ctrl+O (Write Out)
# Then press Enter to confirm filename

# Save as different name
Ctrl+O
# Type new filename and press Enter

# Exit nano
Ctrl+X
# If unsaved changes, nano will ask to save

# Read/Insert another file
Ctrl+R
# Type filename to insert at cursor position

# New buffer (new file)
Alt+F (if multiple buffers enabled)
```

Navigation

```
# Basic movement
Arrow keys        # Move cursor
Ctrl+A           # Beginning of line
Ctrl+E           # End of line
Ctrl+Y           # Previous page
Ctrl+V           # Next page

# Advanced navigation
Ctrl+_           # Go to line number
Ctrl+W           # Search (Where Is)
Alt+W            # Search next occurrence
Ctrl+\           # Search and replace
```

```
# Word movement
Ctrl+Space       # Move forward one word
Alt+Space        # Move backward one word

# Beginning/End of file
Alt+\            # Go to beginning of file
Alt+/            # Go to end of file
```

Text Editing

```
# Cut and paste
Ctrl+K           # Cut current line
Ctrl+U           # Paste (uncut)
Alt+6            # Copy current line
Ctrl+6           # Mark text (start selection)
                 # Move cursor to select text
Ctrl+K           # Cut selected text

# Delete operations
Backspace        # Delete character before cursor
Delete           # Delete character at cursor
Alt+Backspace    # Delete word to the left
Ctrl+Delete      # Delete word to the right

# Undo/Redo
Alt+U            # Undo
Alt+E            # Redo
```

Search and Replace

```
# Search
Ctrl+W           # Open search prompt
# Type search term and press Enter
# Use Ctrl+W again to search for next occurrence

# Search and replace
Ctrl+\           # Open replace prompt
# Enter search term, press Enter
# Enter replacement term, press Enter
# Choose: Y (yes), N (no), A (all), Ctrl+C (cancel)

# Case-sensitive search
Alt+C            # Toggle case sensitivity during search
```

```
# Regular expression search
Alt+R            # Toggle regex mode during search
```

Configuration

Global Configuration

```
# System-wide configuration
/etc/nanorc

# User configuration
~/.nanorc
```

Sample .nanorc Configuration

```
# ~/.nanorc

## Use auto-indentation
set autoindent

## Use bold text instead of reverse video text
set boldtext

## Set the characters treated as closing brackets
set brackets "\"')>]}}"

## Do case-sensitive searches by default
set casesensitive

## Constantly display the cursor position in the statusbar
set constantshow

## Use cut-from-cursor-to-end-of-line by default
set cutfromcursor

## Set the line length for wrapping text and justifying paragraphs
set fill 72

## Remember the used search/replace strings for the next session
set historylog

## Display line numbers to the left of the text
set linenumbers
```

```
## Enable vim-style lock-files
set locking

## Don't display the helpful shortcut lists at the bottom of the screen
# set nohelp

## Don't automatically add a newline when a file doesn't end with one
set nonewlines

## Set operating directory (chroot of sorts)
# set operatingdir "/tmp"

## Remember the cursor position in each file for the next editing session
set positionlog

## Do extended regular expression searches by default
set regexp

## Make the Home key smarter
set smarthome

## Use smooth scrolling as the default
set smooth

## Use this spelling checker instead of the internal one
set speller "aspell -x -c"

## Allow nano to be suspended
set suspend

## Use this tab size instead of the default; it must be greater than 0
set tabsize 4

## Convert typed tabs to spaces
set tabstospaces

## Save automatically on exit; don't prompt
# set tempfile

## Disallow file modification; why would you want this in an rc file?
# set view

## The two single-column characters used to display the first characters
## of tabs and spaces
set whitespace "»·"
```

```
## Detect word boundaries more accurately by treating punctuation
## characters as parts of words
set wordbounds

## Color syntax highlighting
include "/usr/share/nano/*.nanorc"
```

Syntax Highlighting

```
# Enable syntax highlighting for specific file types
# Add to ~/.nanorc:

## Python
include "/usr/share/nano/python.nanorc"

## HTML
include "/usr/share/nano/html.nanorc"

## CSS
include "/usr/share/nano/css.nanorc"

## JavaScript
include "/usr/share/nano/javascript.nanorc"

## JSON
include "/usr/share/nano/json.nanorc"

## Markdown
include "/usr/share/nano/markdown.nanorc"

## Shell scripts
include "/usr/share/nano/sh.nanorc"

## Configuration files
include "/usr/share/nano/conf.nanorc"

## Or include all available syntax files
include "/usr/share/nano/*.nanorc"
```

Custom Syntax Highlighting

```
# ~/.config/nano/custom.nanorc

## Custom syntax for log files
syntax "log" "\.log$"
color red "ERROR.*"
color yellow "WARNING.*"
color green "INFO.*"
color blue "DEBUG.*"
color magenta "\[[0-9-]+ [0-9:]+\]"

## Custom syntax for configuration files
syntax "config" "\.(conf|config|cfg)$"
color green "^[[:space:]]*[^=]*="
color red "^[[:space:]]*#.*"
color yellow ""[^"]*""
color cyan "[0-9]+"
```

Advanced Features

Multiple Buffers

```
# Enable multiple buffers in ~/.nanorc
set multibuffer

# Commands for multiple buffers
Alt+F            # Switch to next buffer
Alt+B            # Switch to previous buffer
Ctrl+R           # Read file into new buffer
```

Spell Checking

```
# Install spell checker
sudo apt install aspell aspell-en

# Use spell check in nano
Ctrl+T           # Start spell check
# Navigate through misspelled words
# Choose replacement or skip
```

Backup Files

```
# Enable backup files in ~/.nanorc
set backup

# Set backup directory
set backupdir "/home/user/.nano/backups"

# Create backup directory
mkdir -p ~/.nano/backups
```

Mouse Support

```
# Enable mouse support in ~/.nanorc
set mouse

# Mouse functions:
# - Click to position cursor
# - Double-click to select word
# - Triple-click to select line
# - Scroll wheel to scroll text
```

Real-World Usage Scenarios

Scenario 1: Quick Configuration File Edit

```
# Edit SSH configuration
sudo nano /etc/ssh/sshd_config

# Navigate to specific setting
Ctrl+W
# Search for "PasswordAuthentication"

# Change the value
# Move cursor to "yes" and change to "no"

# Save and exit
Ctrl+O
Enter
Ctrl+X

# Restart SSH service
sudo systemctl restart sshd
```

Scenario 2: System Log Analysis

```
# View system log
sudo nano -v /var/log/syslog

# Search for errors
Ctrl+W
# Type "error" and press Enter

# Continue searching
Ctrl+W
Enter (to search for same term)

# Go to specific time
Ctrl+W
# Type timestamp like "2023-12-01 10:"

# Exit without changes
Ctrl+X
```

Scenario 3: Script Development

```
# Create new script
nano backup_script.sh

# Add shebang and content
#!/bin/bash
# Backup script
DATE=$(date +%Y%m%d)
tar -czf backup_$DATE.tar.gz /home/user/documents

# Enable line numbers for debugging
Alt+N

# Save and make executable
Ctrl+O
Enter
Ctrl+X
chmod +x backup_script.sh
```

Scenario 4: Crontab Editing

```
# Edit user crontab
crontab -e
# If nano is default editor, it opens in nano

# Add scheduled job
# 0 2 * * * /home/user/backup_script.sh

# Save crontab
Ctrl+O
Enter
Ctrl+X
```

Command Line Options

Useful Nano Options

```
# Open with line numbers
nano -l filename.txt

# Open in restricted mode (no file operations)
nano -R filename.txt

# Set tab width
nano -T 8 filename.txt

# Enable soft wrapping
nano -S filename.txt

# Disable wrapping
nano -w filename.txt

# Show cursor position
nano -c filename.txt

# Enable mouse support
nano -m filename.txt

# Ignore nanorc files
nano -I filename.txt

# Set operating directory
nano -o /tmp filename.txt
```

```
# Enable suspension
nano -z filename.txt
```

Environment Variables

```
# Set nano as default editor
export EDITOR=nano
export VISUAL=nano

# Add to ~/.bashrc or ~/.zshrc
echo 'export EDITOR=nano' >> ~/.bashrc
echo 'export VISUAL=nano' >> ~/.bashrc
```

Tips and Best Practices

Productivity Tips

1. **Learn the shortcuts**: Memorize Ctrl+O (save), Ctrl+X (exit), Ctrl+W (search)
2. **Use line numbers**: Enable with -l option or set linenumbers in .nanorc
3. **Configure syntax highlighting**: Include language-specific nanorc files
4. **Use search effectively**: Ctrl+W for quick navigation in large files
5. **Enable mouse support**: Makes selection and positioning easier

Common Shortcuts Summary

```
# File operations
Ctrl+O          # Save (Write Out)
Ctrl+X          # Exit
Ctrl+R          # Read/Insert file

# Navigation
Ctrl+A          # Beginning of line
Ctrl+E          # End of line
Ctrl+Y          # Previous page
Ctrl+V          # Next page
Ctrl+_          # Go to line number

# Editing
Ctrl+K          # Cut line
Ctrl+U          # Paste (Uncut)
Ctrl+6          # Mark text
Alt+U           # Undo
Alt+E           # Redo
```

```
# Search
Ctrl+W          # Search
Ctrl+\          # Replace
Alt+W           # Find next

# Help
Ctrl+G          # Get help
```

Customization for Different Use Cases

For Programming

```
# ~/.nanorc for programming
set autoindent
set linenumbers
set tabsize 4
set tabstospaces
set mouse
set smooth
set constantshow
include "/usr/share/nano/*.nanorc"
```

For System Administration

```
# ~/.nanorc for sysadmin
set backup
set backupdir "/home/user/.nano/backups"
set linenumbers
set constantshow
set mouse
set locking
include "/usr/share/nano/*.nanorc"
```

For Writing

```
# ~/.nanorc for writing
set fill 80
set justify
set smooth
set mouse
set speller "aspell -x -c"
set tempfile
```

Troubleshooting

Common Issues

1. **Nano not found**

   ```
   # Install nano
   sudo apt install nano
   ```

2. **No syntax highlighting**

   ```
   # Check if nanorc files exist
   ls /usr/share/nano/
   # Add to ~/.nanorc
   include "/usr/share/nano/*.nanorc"
   ```

3. **Can't save file (permission denied)**

   ```
   # Use sudo
   sudo nano /etc/hosts
   # Or change file permissions
   sudo chmod 644 filename.txt
   ```

4. **Accidental changes**

   ```
   # Exit without saving
   Ctrl+X
   # Choose 'N' when asked to save
   ```

5. **Lost in large file**

   ```
   # Go to specific line
   Ctrl+_
   # Enter line number
   ```

Recovery Options

```
# If nano crashes, look for backup files
ls ~/.nano/backups/
# or
ls /tmp/

# Recover from backup
nano ~/.nano/backups/filename.txt~
```

Nano excels as a straightforward, accessible text editor that doesn't require learning complex commands or modes. It's perfect for quick edits, system administration tasks, and users who prefer a simple, intuitive interface over the power and complexity of vim or emacs.

Emacs

Emacs: The Extensible Editor

GNU Emacs is one of the most powerful and extensible text editors available. Created by Richard Stallman in 1976, Emacs is more than just an editor—it's a complete computing environment that can be customized and extended using Emacs Lisp.

Why Choose Emacs?

- **Extreme extensibility**: Customize everything with Emacs Lisp
- **Integrated environment**: Email, calendar, file manager, shell, and more
- **Powerful editing**: Advanced text manipulation and programming features
- **Linux integration**: Deep integration with Linux development workflows
- **Active community**: Thousands of packages and configurations
- **Self-documenting**: Comprehensive built-in help system
- **Org-mode**: Powerful organization and note-taking system
- **System administration**: Excellent for managing Linux servers and configurations

Installation

Linux Distributions

```
# Ubuntu/Debian
sudo apt update
sudo apt install emacs

# For GUI version
sudo apt install emacs-gtk

# CentOS/RHEL/Fedora
sudo yum install emacs
# or
sudo dnf install emacs

# Arch Linux
sudo pacman -S emacs

# Alpine Linux
sudo apk add emacs
```

Additional Linux Distributions

```
# SUSE/openSUSE
sudo zypper install emacs

# Gentoo
sudo emerge app-editors/emacs

# Void Linux
sudo xbps-install emacs

# For GUI version on Linux
sudo apt install emacs-gtk    # Ubuntu/Debian with GTK
sudo pacman -S emacs          # Arch Linux (includes GUI)
```

Building from Source

```
# Install dependencies (Ubuntu/Debian)
sudo apt install build-essential texinfo libx11-dev libxpm-dev \
    libjpeg-dev libpng-dev libgif-dev libtiff-dev libgtk-3-dev \
    libncurses-dev libxpm-dev automake autoconf

# Download and build
wget https://ftp.gnu.org/gnu/emacs/emacs-29.1.tar.xz
tar -xf emacs-29.1.tar.xz
cd emacs-29.1
./configure --with-native-compilation
make -j$(nproc)
sudo make install
```

Basic Concepts

Key Notation

- `C-x` means Ctrl+x
- `M-x` means Alt+x (Meta key)
- `C-x C-f` means Ctrl+x followed by Ctrl+f
- `RET` means Enter key
- `SPC` means Space key

Buffers, Windows, and Frames

- **Buffer**: A file or content in memory

- **Window**: A view of a buffer (can have multiple windows)
- **Frame**: The entire Emacs window (can have multiple frames)

Major and Minor Modes

- **Major Mode**: Defines the primary editing behavior (e.g., Python mode, HTML mode)
- **Minor Mode**: Additional features that can be enabled/disabled (e.g., line numbers, auto-complete)

Essential Commands

Starting and Exiting

```
# Start Emacs
emacs

# Start in terminal mode
emacs -nw

# Open specific file
emacs filename.txt

# Exit Emacs
C-x C-c
```

File Operations

```
# Open file
C-x C-f
# Type filename and press RET

# Save file
C-x C-s

# Save as (write file)
C-x C-w

# Save all buffers
C-x s

# Insert file contents
C-x i
```

```
# Recent files
C-x C-r
```

Buffer Management

```
# Switch buffer
C-x b
# Type buffer name or use TAB completion

# List all buffers
C-x C-b

# Kill (close) buffer
C-x k

# Switch to next buffer
C-x →

# Switch to previous buffer
C-x ←
```

Window Management

```
# Split window horizontally
C-x 2

# Split window vertically
C-x 3

# Switch to other window
C-x o

# Delete current window
C-x 0

# Delete other windows
C-x 1

# Resize windows
C-x ^    # Make taller
C-x }    # Make wider
C-x {    # Make narrower
```

Navigation and Editing

Basic Movement

```
# Character movement
C-f      # Forward character
C-b      # Backward character
C-n      # Next line
C-p      # Previous line

# Word movement
M-f      # Forward word
M-b      # Backward word

# Line movement
C-a      # Beginning of line
C-e      # End of line

# Sentence movement
M-a      # Beginning of sentence
M-e      # End of sentence

# Paragraph movement
M-{      # Beginning of paragraph
M-}      # End of paragraph

# Page movement
C-v      # Page down
M-v      # Page up

# Buffer movement
M-<      # Beginning of buffer
M->      # End of buffer
```

Advanced Navigation

```
# Go to line
M-g g    # Go to line number
M-g M-g  # Alternative

# Go to character
M-g c    # Go to character position

# Jump to definition
M-.      # Find definition
```

```
M-,      # Return from definition

# Bookmarks
C-x r m  # Set bookmark
C-x r b  # Jump to bookmark
C-x r l  # List bookmarks
```

Text Editing

```
# Delete operations
C-d      # Delete character forward
DEL      # Delete character backward
M-d      # Delete word forward
M-DEL    # Delete word backward
C-k      # Kill to end of line
M-k      # Kill to end of sentence

# Kill and yank (cut and paste)
C-w      # Kill region
M-w      # Copy region
C-y      # Yank (paste)
M-y      # Cycle through kill ring

# Undo
C-/      # Undo
C-x u    # Undo
C-g      # Quit/cancel command
```

Selection (Regions)

```
# Set mark
C-SPC    # Set mark at cursor
C-x C-x  # Exchange point and mark

# Select all
C-x h    # Select entire buffer

# Rectangle selection
C-x r r  # Copy rectangle
C-x r k  # Kill rectangle
C-x r y  # Yank rectangle
C-x r t  # Replace rectangle with text
```

Search and Replace

Basic Search

```
# Incremental search forward
C-s
# Type search term, C-s for next occurrence

# Incremental search backward
C-r

# Word search
C-s C-w  # Search for word at cursor

# Regular expression search
C-M-s    # Forward regex search
C-M-r    # Backward regex search
```

Replace Operations

```
# Query replace
M-%
# Enter search term, then replacement
# y (yes), n (no), ! (all), q (quit)

# Regular expression replace
C-M-%

# Replace in region
# Select region first, then M-%
```

Advanced Search

```
# Occur (show all lines matching pattern)
M-s o

# Multi-occur (search in multiple buffers)
M-s M-o

# Grep in files
M-x grep
M-x rgrep    # Recursive grep
```

Configuration

Init File

Emacs configuration is stored in `~/.emacs.d/init.el` or `~/.emacs`:

```
;; ~/.emacs.d/init.el - Basic configuration

;; Package management
(require 'package)
(setq package-archives '(("melpa" . "https://melpa.org/packages/")
                         ("org" . "https://orgmode.org/elpa/")
                         ("elpa" . "https://elpa.gnu.org/packages/")))
(package-initialize)

;; Bootstrap use-package
(unless (package-installed-p 'use-package)
  (package-refresh-contents)
  (package-install 'use-package))

(eval-when-compile
  (require 'use-package))

;; Basic settings
(setq inhibit-startup-message t)        ; Disable startup screen
(setq visible-bell t)                   ; Visual bell instead of beep
(setq make-backup-files nil)            ; Disable backup files
(setq auto-save-default nil)            ; Disable auto-save

;; UI improvements
(tool-bar-mode -1)                      ; Disable toolbar
(menu-bar-mode -1)                      ; Disable menu bar
(scroll-bar-mode -1)                    ; Disable scroll bar
(global-display-line-numbers-mode 1)    ; Show line numbers
(column-number-mode 1)                  ; Show column numbers
(show-paren-mode 1)                     ; Highlight matching parentheses

;; Indentation
(setq-default indent-tabs-mode nil)     ; Use spaces instead of tabs
(setq-default tab-width 4)              ; Tab width
(setq indent-line-function 'insert-tab) ; Tab behavior

;; Font and theme
(set-face-attribute 'default nil :font "Fira Code-12")
(load-theme 'wombat t)

;; Key bindings
```

```
(global-set-key (kbd "C-x C-b") 'ibuffer)
(global-set-key (kbd "M-/") 'hippie-expand)
(global-set-key (kbd "C-c c") 'comment-or-uncomment-region)

;; Auto-completion
(use-package company
  :ensure t
  :init
  (global-company-mode 1))

;; Syntax checking
(use-package flycheck
  :ensure t
  :init
  (global-flycheck-mode))

;; Git integration
(use-package magit
  :ensure t
  :bind ("C-x g" . magit-status))

;; Project management
(use-package projectile
  :ensure t
  :init
  (projectile-mode +1)
  :bind-keymap
  ("C-c p" . projectile-command-map))

;; File tree
(use-package neotree
  :ensure t
  :bind ("F8" . neotree-toggle))

;; Better search
(use-package ivy
  :ensure t
  :init
  (ivy-mode 1)
  :config
  (setq ivy-use-virtual-buffers t)
  (setq enable-recursive-minibuffers t))

(use-package counsel
  :ensure t
  :bind (("M-x" . counsel-M-x)
         ("C-x C-f" . counsel-find-file)
```

```
         ("C-h f" . counsel-describe-function)
         ("C-h v" . counsel-describe-variable)))

(use-package swiper
  :ensure t
  :bind ("C-s" . swiper))
```

Package Management

```
;; Install packages manually
M-x package-install RET package-name RET

;; List packages
M-x list-packages

;; Update packages
M-x package-list-packages
# Press 'U' then 'x' to update all
```

Popular Packages

```
;; Essential packages configuration

;; Helm (alternative to ivy/counsel)
(use-package helm
  :ensure t
  :bind (("M-x" . helm-M-x)
         ("C-x C-f" . helm-find-files)
         ("C-x b" . helm-buffers-list))
  :config
  (helm-mode 1))

;; Which-key (show key bindings)
(use-package which-key
  :ensure t
  :config
  (which-key-mode))

;; Multiple cursors
(use-package multiple-cursors
  :ensure t
  :bind (("C-S-c C-S-c" . mc/edit-lines)
         ("C->" . mc/mark-next-like-this)
```

```
         ("C-<" . mc/mark-previous-like-this)))

;; Expand region
(use-package expand-region
  :ensure t
  :bind ("C-=" . er/expand-region))

;; Yasnippet (snippets)
(use-package yasnippet
  :ensure t
  :config
  (yas-global-mode 1))

;; Language Server Protocol
(use-package lsp-mode
  :ensure t
  :hook ((python-mode . lsp)
         (js-mode . lsp)
         (typescript-mode . lsp))
  :commands lsp)

(use-package lsp-ui
  :ensure t
  :commands lsp-ui-mode)

;; Company LSP integration
(use-package company-lsp
  :ensure t
  :commands company-lsp)
```

Programming with Emacs

Language-Specific Modes

```
;; Python development
(use-package python-mode
  :ensure t
  :mode "\\.py\\'"
  :config
  (setq python-indent-offset 4))

;; JavaScript/TypeScript
(use-package js2-mode
  :ensure t
```

```
  :mode "\\.js\\'"
  :config
  (setq js2-basic-offset 2))

(use-package typescript-mode
  :ensure t
  :mode "\\.ts\\'")

;; Web development
(use-package web-mode
  :ensure t
  :mode ("\\.html\\'" "\\.css\\'" "\\.jsx\\'")
  :config
  (setq web-mode-markup-indent-offset 2)
  (setq web-mode-css-indent-offset 2)
  (setq web-mode-code-indent-offset 2))

;; Markdown
(use-package markdown-mode
  :ensure t
  :mode "\\.md\\'")

;; YAML
(use-package yaml-mode
  :ensure t
  :mode "\\.ya?ml\\'")

;; JSON
(use-package json-mode
  :ensure t
  :mode "\\.json\\'")
```

Code Navigation

```
# Tags (requires etags or ctags)
M-.      # Find definition
M-,      # Return from definition
M-*      # Pop tag mark

# Imenu (function/class navigation)
M-x imenu

# Occur (find all occurrences)
M-s o
```

```
# Grep
M-x grep
M-x rgrep
```

Debugging

```
;; GDB integration
M-x gdb

;; Python debugging with pdb
M-x pdb

;; Edebug (Emacs Lisp debugger)
C-u C-M-x   ; Instrument function for debugging
```

Org Mode

Org-mode is one of Emacs' most powerful features for organization and note-taking:

Basic Org Syntax

```
* Top Level Heading
** Second Level Heading
*** Third Level Heading

- Unordered list item
- Another item
  - Sub-item

1. Ordered list item
2. Another ordered item

*bold* /italic/ _underlined_ =code= ~verbatim~

[[https://example.com][Link text]]

#+BEGIN_SRC python
def hello_world():
    print("Hello, World!")
#+END_SRC

| Name | Age | City |
```

```
|------+-----+------|
| John |  25 | NYC  |
| Jane |  30 | LA   |
```

Org Mode Commands

```
# Structure editing
TAB      # Cycle visibility
S-TAB    # Global visibility cycling
M-RET    # Insert new heading
M-←/→    # Promote/demote heading
M-↑/↓    # Move heading up/down

# TODO items
C-c C-t  # Cycle TODO states
C-c C-s  # Schedule item
C-c C-d  # Set deadline

# Tables
C-c |    # Create table
TAB      # Move to next field
S-TAB    # Move to previous field
C-c C-c  # Align table

# Links
C-c C-l  # Insert link
C-c C-o  # Open link

# Export
C-c C-e  # Export menu
```

Org Configuration

```
;; Org-mode configuration
(use-package org
  :ensure t
  :config
  (setq org-todo-keywords
        '((sequence "TODO" "IN-PROGRESS" "WAITING" "|" "DONE" "CANCELLED")))
  (setq org-agenda-files '("~/org/"))
  (setq org-default-notes-file "~/org/notes.org")
  :bind (("C-c l" . org-store-link)
         ("C-c a" . org-agenda)
         ("C-c c" . org-capture)))
```

```
;; Org capture templates
(setq org-capture-templates
      '(("t" "Todo" entry (file+headline "~/org/tasks.org" "Tasks")
         "* TODO %?\n  %i\n  %a")
        ("n" "Note" entry (file+headline "~/org/notes.org" "Notes")
         "* %?\nEntered on %U\n  %i\n  %a")))
```

Real-World Development Scenarios

Scenario 1: Python Development Setup

```
;; Python development configuration
(use-package python-mode
  :ensure t
  :config
  (setq python-indent-offset 4)
  (add-hook 'python-mode-hook
            (lambda ()
              (setq indent-tabs-mode nil)
              (setq tab-width 4)
              (setq python-indent-offset 4))))

;; Virtual environment support
(use-package pyvenv
  :ensure t
  :config
  (pyvenv-mode 1))

;; Python testing
(use-package python-pytest
  :ensure t
  :bind (:map python-mode-map
              ("C-c t" . python-pytest)))

;; Jupyter notebooks
(use-package ein
  :ensure t)
```

Scenario 2: Web Development Workflow

```
;; Web development setup
(use-package web-mode
  :ensure t
  :mode ("\\.html\\'" "\\.css\\'" "\\.js\\'" "\\.jsx\\'" "\\.tsx\\'")
  :config
  (setq web-mode-markup-indent-offset 2)
  (setq web-mode-css-indent-offset 2)
  (setq web-mode-code-indent-offset 2)
  (setq web-mode-enable-auto-pairing t)
  (setq web-mode-enable-css-colorization t))

;; Emmet for HTML expansion
(use-package emmet-mode
  :ensure t
  :hook (web-mode . emmet-mode))

;; Live browser reload
(use-package impatient-mode
  :ensure t)

;; REST client
(use-package restclient
  :ensure t
  :mode ("\\.http\\'" . restclient-mode))
```

Scenario 3: System Administration

```
;; System administration tools
(use-package tramp
  :config
  (setq tramp-default-method "ssh"))

;; Docker integration
(use-package docker
  :ensure t
  :bind ("C-c d" . docker))

(use-package dockerfile-mode
  :ensure t
  :mode "Dockerfile\\'")

;; Ansible
(use-package ansible
```

```
  :ensure t)

;; Log file viewing
(use-package logview
  :ensure t
  :mode ("\\.log\\'" . logview-mode))
```

Advanced Emacs Features

Macros

```
# Record macro
C-x (    # Start recording
# ... perform actions ...
C-x )    # Stop recording

# Execute macro
C-x e    # Execute once
C-u 10 C-x e  # Execute 10 times

# Name and save macro
M-x name-last-kbd-macro
M-x insert-kbd-macro
```

Registers

```
# Save to register
C-x r s a    # Save region to register 'a'
C-x r i a    # Insert from register 'a'

# Save position
C-x r SPC a  # Save position to register 'a'
C-x r j a    # Jump to register 'a'

# Save window configuration
C-x r w a    # Save window config to register 'a'
C-x r j a    # Restore window config from register 'a'
```

Dired (Directory Editor)

```
# Open dired
C-x d

# Dired commands
RET      # Open file/directory
q        # Quit dired
g        # Refresh
m        # Mark file
u        # Unmark file
d        # Mark for deletion
x        # Execute deletions
C        # Copy file
R        # Rename/move file
+        # Create directory
```

Shell Integration

```
# Shell commands
M-!      # Execute shell command
M-|      # Shell command on region
C-u M-!  # Insert command output

# Shell mode
M-x shell
M-x eshell   # Emacs shell
M-x term     # Terminal emulator
```

Customization and Themes

Custom Themes

```
;; Popular themes
(use-package doom-themes
  :ensure t
  :config
  (load-theme 'doom-one t))

(use-package solarized-theme
  :ensure t)

(use-package zenburn-theme
```

```
  :ensure t)

;; Theme switching function
(defun switch-theme (theme)
  "Disable current theme and load THEME."
  (interactive
   (list
    (intern (completing-read "Load custom theme: "
                             (mapcar 'symbol-name
                                     (custom-available-themes))))))
  (let ((enabled-themes custom-enabled-themes))
    (mapc #'disable-theme custom-enabled-themes)
    (load-theme theme t)))
```

Custom Functions

```
;; Useful custom functions
(defun duplicate-line ()
  "Duplicate current line."
  (interactive)
  (move-beginning-of-line 1)
  (kill-line)
  (yank)
  (open-line 1)
  (next-line 1)
  (yank))

(global-set-key (kbd "C-c d") 'duplicate-line)

(defun comment-or-uncomment-region-or-line ()
  "Comments or uncomments the region or the current line if there's no active region."
  (interactive)
  (let (beg end)
    (if (region-active-p)
        (setq beg (region-beginning) end (region-end))
      (setq beg (line-beginning-position) end (line-end-position)))
    (comment-or-uncomment-region beg end)))

(global-set-key (kbd "C-/") 'comment-or-uncomment-region-or-line)
```

Performance Optimization

Startup Optimization

```
;; Startup optimization
(setq gc-cons-threshold (* 50 1000 1000))  ; Increase GC threshold
(setq read-process-output-max (* 1024 1024)) ; Increase read buffer

;; Restore GC threshold after startup
(add-hook 'emacs-startup-hook
          (lambda ()
            (setq gc-cons-threshold (* 2 1000 1000))))

;; Lazy loading
(use-package package-name
  :ensure t
  :defer t  ; Lazy load
  :hook (mode . package-name))
```

Large File Handling

```
;; Large file handling
(defun my-find-file-check-make-large-file-read-only-hook ()
  "If a file is over a given size, make the buffer read only."
  (when (> (buffer-size) (* 1024 1024))
    (setq buffer-read-only t)
    (buffer-disable-undo)
    (fundamental-mode)))

(add-hook 'find-file-hook 'my-find-file-check-make-large-file-read-only-hook)
```

Troubleshooting

Common Issues

```
# Debug init file
emacs --debug-init

# Start with minimal config
emacs -Q

# Profile startup time
M-x emacs-init-time
```

```
# Check package issues
M-x package-list-packages
# Look for packages with issues

# Reset package system
# Delete ~/.emacs.d/elpa/ and restart
```

Recovery

```
# Auto-save recovery
M-x recover-file

# Session recovery
M-x desktop-save-mode  ; Enable session saving
M-x desktop-read       ; Restore session
```

Emacs is incredibly powerful and customizable, making it suitable for everything from simple text editing to complex development workflows. While it has a steep learning curve, the investment pays off with a highly personalized and efficient editing environment.

LazyVim

LazyVim: Modern Neovim Configuration

LazyVim is a popular, pre-configured Neovim setup that provides a modern IDE-like experience out of the box. Built on top of lazy.nvim plugin manager, it offers sensible defaults, beautiful UI, and powerful features for developers.

What is LazyVim?

LazyVim is a Neovim configuration framework that includes: - **Pre-configured plugins**: Carefully selected and configured plugins - **Lazy loading**: Fast startup times with lazy.nvim - **Modern UI**: Beautiful interface with proper theming - **LSP integration**: Built-in Language Server Protocol support - **Extensible**: Easy to customize and extend - **Well-documented**: Comprehensive documentation and examples

Why Choose LazyVim?

- **Zero configuration**: Works great out of the box
- **Fast startup**: Optimized for performance
- **Modern features**: Latest Neovim capabilities
- **Active development**: Regular updates and improvements
- **Great defaults**: Sensible keybindings and settings
- **Easy customization**: Simple configuration structure

Installation

Prerequisites

```
# Neovim 0.9.0 or higher
nvim --version

# Install Neovim if needed
# Ubuntu/Debian
sudo apt install neovim

# Arch Linux
sudo pacman -S neovim
```

```
# SUSE/openSUSE
sudo zypper install neovim

# Gentoo
sudo emerge app-editors/neovim
```

Required Dependencies

```
# Git (required)
git --version

# Node.js (for LSP servers)
node --version
npm --version

# Python (for some plugins)
python3 --version
pip3 --version

# Ripgrep (for telescope)
rg --version

# fd (for telescope)
fd --version

# Install missing dependencies
# Ubuntu/Debian
sudo apt install git nodejs npm python3 python3-pip ripgrep fd-find

# Arch Linux
sudo pacman -S git nodejs npm python python-pip ripgrep fd

# SUSE/openSUSE
sudo zypper install git nodejs npm python3 python3-pip ripgrep fd

# Gentoo
sudo emerge dev-vcs/git net-libs/nodejs dev-lang/python sys-apps/ripgrep sys-apps/fd
```

Installation Steps

```
# Backup existing Neovim configuration
mv ~/.config/nvim ~/.config/nvim.bak
mv ~/.local/share/nvim ~/.local/share/nvim.bak
```

```
mv ~/.local/state/nvim ~/.local/state/nvim.bak
mv ~/.cache/nvim ~/.cache/nvim.bak

# Clone LazyVim starter template
git clone https://github.com/LazyVim/starter ~/.config/nvim

# Remove .git folder to make it your own
rm -rf ~/.config/nvim/.git

# Start Neovim (plugins will install automatically)
nvim
```

Initial Setup and Configuration

First Launch

When you first start LazyVim:

1. **Plugin Installation**: Plugins install automatically
2. **Mason Setup**: Language servers install automatically
3. **Treesitter**: Syntax parsers download automatically
4. **Health Check**: Run `:checkhealth` to verify setup

Basic Configuration Structure

```
~/.config/nvim/
  init.lua                  # Entry point
  lua/
      config/
          autocmds.lua     # Auto commands
          keymaps.lua      # Key mappings
          lazy.lua         # Plugin manager setup
          options.lua      # Neovim options
      plugins/
           example.lua      # Example plugin config
           ...              # Your plugin configurations
  stylua.toml               # Lua formatter config
```

Basic Options Configuration

```
-- ~/.config/nvim/lua/config/options.lua

-- LazyVim auto format
vim.g.autoformat = true
```

```lua
-- LazyVim picker to use
-- Can be one of: telescope, fzf
vim.g.lazyvim_picker = "telescope"

-- LazyVim root dir detection
-- Each entry can be:
-- * the name of a detector function like `lsp` or `cwd`
-- * a pattern or array of patterns like `.git` or `lua`.
-- * a function with signature `function(buf) -> string|string[]`
vim.g.root_spec = { "lsp", { ".git", "lua" }, "cwd" }

-- LazyVim automatically configures lazygit:
--  * theme, based on the active colorscheme.
--  * editorPreset to nvim-remote
--  * enables nerd font icons
-- Set to false to disable.
vim.g.lazygit_config = true

-- Options are automatically loaded before lazy.nvim startup
-- Default options that are always set: https://github.com/LazyVim/LazyVim/blob/main/lua/laz
-- Add any additional options here

-- Line numbers
vim.opt.relativenumber = true -- Relative line numbers
vim.opt.number = true -- Print line number

-- Tabs / Indentation
vim.opt.tabstop = 2 -- 2 spaces for tabs (prettier default)
vim.opt.shiftwidth = 2 -- 2 spaces for indent width
vim.opt.expandtab = true -- expand tab to spaces
vim.opt.autoindent = true -- copy indent from current line when starting new one

-- Line wrapping
vim.opt.wrap = false -- disable line wrapping

-- Search settings
vim.opt.ignorecase = true -- ignore case when searching
vim.opt.smartcase = true -- if you include mixed case in your search, assumes you want case-

-- Cursor line
vim.opt.cursorline = true -- highlight the current cursor line

-- Appearance
vim.opt.termguicolors = true
vim.opt.background = "dark" -- colorschemes that can be light or dark will be made dark
vim.opt.signcolumn = "yes" -- show sign column so that text doesn't shift
```

```
-- Backspace
vim.opt.backspace = "indent,eol,start" -- allow backspace on indent, end of line or insert m

-- Clipboard
vim.opt.clipboard:append("unnamedplus") -- use system clipboard as default register

-- Split windows
vim.opt.splitright = true -- split vertical window to the right
vim.opt.splitbelow = true -- split horizontal window to the bottom

-- Turn off swapfile
vim.opt.swapfile = false
```

Custom Keymaps

```
-- ~/.config/nvim/lua/config/keymaps.lua

-- Keymaps are automatically loaded on the VeryLazy event
-- Default keymaps that are always set: https://github.com/LazyVim/LazyVim/blob/main/lua/laz
-- Add any additional keymaps here

local keymap = vim.keymap -- for conciseness

-- General keymaps
keymap.set("i", "jk", "<ESC>", { desc = "Exit insert mode with jk" })

keymap.set("n", "<leader>nh", ":nohl<CR>", { desc = "Clear search highlights" })

-- increment/decrement numbers
keymap.set("n", "<leader>+", "<C-a>", { desc = "Increment number" }) -- increment
keymap.set("n", "<leader>-", "<C-x>", { desc = "Decrement number" }) -- decrement

-- window management
keymap.set("n", "<leader>sv", "<C-w>v", { desc = "Split window vertically" }) -- split windo
keymap.set("n", "<leader>sh", "<C-w>s", { desc = "Split window horizontally" }) -- split win
keymap.set("n", "<leader>se", "<C-w>=", { desc = "Make splits equal size" }) -- make split w
keymap.set("n", "<leader>sx", "<cmd>close<CR>", { desc = "Close current split" }) -- close c

keymap.set("n", "<leader>to", "<cmd>tabnew<CR>", { desc = "Open new tab" }) -- open new tab
keymap.set("n", "<leader>tx", "<cmd>tabclose<CR>", { desc = "Close current tab" }) -- close
keymap.set("n", "<leader>tn", "<cmd>tabn<CR>", { desc = "Go to next tab" }) --  go to next t
keymap.set("n", "<leader>tp", "<cmd>tabp<CR>", { desc = "Go to previous tab" }) --  go to pr
keymap.set("n", "<leader>tf", "<cmd>tabnew %<CR>", { desc = "Open current buffer in new tab"

-- Move text up and down
```

```lua
keymap.set("v", "J", ":m '>+1<CR>gv=gv", { desc = "Move text down" })
keymap.set("v", "K", ":m '<-2<CR>gv=gv", { desc = "Move text up" })

-- Stay in indent mode
keymap.set("v", "<", "<gv", { desc = "Indent left" })
keymap.set("v", ">", ">gv", { desc = "Indent right" })

-- Keep last yanked when pasting
keymap.set("v", "p", '"_dP', { desc = "Paste without yanking" })
```

Auto Commands

```lua
-- ~/.config/nvim/lua/config/autocmds.lua

-- Autocmds are automatically loaded on the VeryLazy event
-- Default autocmds that are always set: https://github.com/LazyVim/LazyVim/blob/main/lua/la
-- Add any additional autocmds here

local function augroup(name)
  return vim.api.nvim_create_augroup("lazyvim_" .. name, { clear = true })
end

-- Highlight on yank
vim.api.nvim_create_autocmd("TextYankPost", {
  group = augroup("highlight_yank"),
  callback = function()
    vim.highlight.on_yank()
  end,
})

-- resize splits if window got resized
vim.api.nvim_create_autocmd({ "VimResized" }, {
  group = augroup("resize_splits"),
  callback = function()
    local current_tab = vim.fn.tabpagenr()
    vim.cmd("tabdo wincmd =")
    vim.cmd("tabnext " .. current_tab)
  end,
})

-- go to last loc when opening a buffer
vim.api.nvim_create_autocmd("BufReadPost", {
  group = augroup("last_loc"),
  callback = function(event)
    local exclude = { "gitcommit" }
```

```
    local buf = event.buf
    if vim.tbl_contains(exclude, vim.bo[buf].filetype) or vim.b[buf].lazyvim_last_loc then
      return
    end
    vim.b[buf].lazyvim_last_loc = true
    local mark = vim.api.nvim_buf_get_mark(buf, '"')
    local lcount = vim.api.nvim_buf_line_count(buf)
    if mark[1] > 0 and mark[1] <= lcount then
      pcall(vim.api.nvim_win_set_cursor, 0, mark)
    end
  end,
})

-- close some filetypes with <q>
vim.api.nvim_create_autocmd("FileType", {
  group = augroup("close_with_q"),
  pattern = {
    "PlenaryTestPopup",
    "help",
    "lspinfo",
    "man",
    "notify",
    "qf",
    "query",
    "spectre_panel",
    "startuptime",
    "tsplayground",
    "neotest-output",
    "checkhealth",
    "neotest-summary",
    "neotest-output-panel",
  },
  callback = function(event)
    vim.bo[event.buf].buflisted = false
    vim.keymap.set("n", "q", "<cmd>close<cr>", { buffer = event.buf, silent = true })
  end,
})

-- wrap and check for spell in text filetypes
vim.api.nvim_create_autocmd("FileType", {
  group = augroup("wrap_spell"),
  pattern = { "gitcommit", "markdown" },
  callback = function()
    vim.opt_local.wrap = true
    vim.opt_local.spell = true
  end,
})
```

```lua
-- Auto create dir when saving a file, in case some intermediate directory does not exist
vim.api.nvim_create_autocmd({ "BufWritePre" }, {
  group = augroup("auto_create_dir"),
  callback = function(event)
    if event.match:match("^%w%w+://") then
      return
    end
    local file = vim.loop.fs_realpath(event.match) or event.match
    vim.fn.mkdir(vim.fn.fnamemodify(file, ":p:h"), "p")
  end,
})
```

Plugin Configuration

Adding Custom Plugins

```lua
-- ~/.config/nvim/lua/plugins/example.lua

return {
  -- Add your custom plugins here
  {
    "folke/flash.nvim",
    event = "VeryLazy",
    opts = {},
    keys = {
      {
        "s",
        mode = { "n", "x", "o" },
        function()
          require("flash").jump()
        end,
        desc = "Flash",
      },
      {
        "S",
        mode = { "n", "o", "x" },
        function()
          require("flash").treesitter()
        end,
        desc = "Flash Treesitter",
      },
    },
  },
```

```
  -- Disable default plugins
  { "folke/flash.nvim", enabled = false },

  -- Override plugin configuration
  {
    "nvim-telescope/telescope.nvim",
    opts = {
      defaults = {
        layout_strategy = "horizontal",
        layout_config = { prompt_position = "top" },
        sorting_strategy = "ascending",
        winblend = 0,
      },
    },
  },
}
```

Language-Specific Configurations

```
-- ~/.config/nvim/lua/plugins/languages.lua

return {
  -- Python
  {
    "nvim-treesitter/nvim-treesitter",
    opts = function(_, opts)
      if type(opts.ensure_installed) == "table" then
        vim.list_extend(opts.ensure_installed, { "python" })
      end
    end,
  },
  {
    "neovim/nvim-lspconfig",
    opts = {
      servers = {
        pyright = {},
        ruff_lsp = {},
      },
    },
  },

  -- JavaScript/TypeScript
  {
    "nvim-treesitter/nvim-treesitter",
    opts = function(_, opts)
```

```
    if type(opts.ensure_installed) == "table" then
      vim.list_extend(opts.ensure_installed, { "javascript", "typescript", "tsx" })
    end
  end,
},
{
  "neovim/nvim-lspconfig",
  opts = {
    servers = {
      tsserver = {},
      eslint = {},
    },
  },
},

-- Go
{
  "nvim-treesitter/nvim-treesitter",
  opts = function(_, opts)
    if type(opts.ensure_installed) == "table" then
      vim.list_extend(opts.ensure_installed, { "go", "gomod", "gowork", "gosum" })
    end
  end,
},
{
  "neovim/nvim-lspconfig",
  opts = {
    servers = {
      gopls = {},
    },
  },
},

-- Rust
{
  "nvim-treesitter/nvim-treesitter",
  opts = function(_, opts)
    if type(opts.ensure_installed) == "table" then
      vim.list_extend(opts.ensure_installed, { "rust", "toml" })
    end
  end,
},
{
  "neovim/nvim-lspconfig",
  opts = {
    servers = {
```

```
        rust_analyzer = {},
      },
    },
  },
}
```

Essential LazyVim Features

File Explorer (Neo-tree)

```
# Default keymaps
<leader>fe    # Open file explorer
<leader>fE    # Open file explorer (root dir)
<leader>e     # Toggle file explorer
<leader>E     # Toggle file explorer (root dir)

# Neo-tree navigation
j/k           # Move up/down
h/l           # Collapse/expand or enter
<CR>          # Open file
o             # Open file
s             # Open in split
v             # Open in vsplit
t             # Open in tab
a             # Add file/directory
d             # Delete
r             # Rename
c             # Copy
x             # Cut
p             # Paste
q             # Close
```

Fuzzy Finding (Telescope)

```
# File finding
<leader>ff    # Find files
<leader>fr    # Recent files
<leader>fg    # Find files (git)
<leader>fF    # Find files (cwd)

# Text searching
<leader>sg    # Grep search
<leader>sw    # Search word under cursor
```

```
<leader>sG    # Grep search (cwd)
<leader>ss    # Search symbols
<leader>sS    # Search symbols (workspace)

# Buffer management
<leader>fb    # Find buffers
<leader>,     # Switch buffers

# Git integration
<leader>gc    # Git commits
<leader>gs    # Git status

# Help and commands
<leader>sh    # Help pages
<leader>sk    # Key maps
<leader>sc    # Commands
<leader>sC    # Command history
```

LSP Features

```
# Navigation
gd            # Go to definition
gr            # Go to references
gI            # Go to implementation
gy            # Go to type definition
gD            # Go to declaration

# Information
K             # Hover information
gK            # Signature help
<c-k>         # Signature help (insert mode)

# Code actions
<leader>ca    # Code actions
<leader>cA    # Source actions
<leader>cr    # Rename
<leader>cf    # Format
<leader>cF    # Format (injected langs)

# Diagnostics
<leader>cd    # Line diagnostics
<leader>cD    # Workspace diagnostics
]d            # Next diagnostic
[d            # Previous diagnostic
]e            # Next error
```

```
[e            # Previous error
]w            # Next warning
[w            # Previous warning
```

Git Integration (Lazygit)

```
# Lazygit
<leader>gg    # Open Lazygit
<leader>gG    # Open Lazygit (root dir)

# Git signs (gitsigns)
]h            # Next hunk
[h            # Previous hunk
<leader>ghs   # Stage hunk
<leader>ghr   # Reset hunk
<leader>ghS   # Stage buffer
<leader>ghu   # Undo stage hunk
<leader>ghR   # Reset buffer
<leader>ghp   # Preview hunk
<leader>ghb   # Blame line
<leader>ghl   # Blame line (full)
<leader>ghd   # Diff this
<leader>ghD   # Diff this ~
```

Terminal Integration

```
# Terminal
<leader>ft    # Terminal (root dir)
<leader>fT    # Terminal (cwd)
<c-/>         # Terminal (root dir)
<c-_>         # Terminal (root dir) (which-key)

# Terminal navigation (in terminal mode)
<esc><esc>    # Enter normal mode
<C-h>         # Go to left window
<C-j>         # Go to lower window
<C-k>         # Go to upper window
<C-l>         # Go to right window
```

Real-World Development Setups

Python Development

```
-- ~/.config/nvim/lua/plugins/python.lua

return {
  -- Python-specific plugins
  {
    "linux-cultist/venv-selector.nvim",
    dependencies = { "neovim/nvim-lspconfig", "nvim-telescope/telescope.nvim", "mfussenegger
    opts = {
      -- Your options go here
      name = "venv",
      auto_refresh = false,
    },
    event = "VeryLazy", -- Optional: needed only if you want to type `:VenvSelect` without a
    keys = {
      -- Keymap to open VenvSelector to pick a venv.
      { "<leader>vs", "<cmd>VenvSelect<cr>" },
      -- Keymap to retrieve the venv from a cache (the one previously used for the same proj
      { "<leader>vc", "<cmd>VenvSelectCached<cr>" },
    },
  },

  -- Python debugging
  {
    "mfussenegger/nvim-dap-python",
    ft = "python",
    dependencies = {
      "mfussenegger/nvim-dap",
      "rcarriga/nvim-dap-ui",
    },
    config = function(_, opts)
      local path = "~/.local/share/nvim/mason/packages/debugpy/venv/bin/python"
      require("dap-python").setup(path)
    end,
  },

  -- Python testing
  {
    "nvim-neotest/neotest",
    optional = true,
    dependencies = {
      "nvim-neotest/neotest-python",
    },
```

```
  opts = {
    adapters = {
      ["neotest-python"] = {
        -- Here you can specify the settings for the adapter, i.e.
        runner = "pytest",
        python = ".venv/bin/python",
      },
    },
  },
},

-- Enhanced Python syntax
{
  "nvim-treesitter/nvim-treesitter",
  opts = function(_, opts)
    if type(opts.ensure_installed) == "table" then
      vim.list_extend(opts.ensure_installed, { "ninja", "python", "rst", "toml" })
    end
  end,
},

-- Python LSP configuration
{
  "neovim/nvim-lspconfig",
  opts = {
    servers = {
      pyright = {
        settings = {
          python = {
            analysis = {
              typeCheckingMode = "basic",
              autoSearchPaths = true,
              useLibraryCodeForTypes = true,
            },
          },
        },
      },
      ruff_lsp = {
        init_options = {
          settings = {
            args = {},
          },
        },
      },
    },
  },
},
```

```
}
```

Web Development

```
-- ~/.config/nvim/lua/plugins/web.lua

return {
  -- TypeScript/JavaScript
  {
    "neovim/nvim-lspconfig",
    opts = {
      servers = {
        tsserver = {
          settings = {
            typescript = {
              inlayHints = {
                includeInlayParameterNameHints = "literal",
                includeInlayParameterNameHintsWhenArgumentMatchesName = false,
                includeInlayFunctionParameterTypeHints = true,
                includeInlayVariableTypeHints = false,
                includeInlayPropertyDeclarationTypeHints = true,
                includeInlayFunctionLikeReturnTypeHints = true,
                includeInlayEnumMemberValueHints = true,
              },
            },
            javascript = {
              inlayHints = {
                includeInlayParameterNameHints = "all",
                includeInlayParameterNameHintsWhenArgumentMatchesName = false,
                includeInlayFunctionParameterTypeHints = true,
                includeInlayVariableTypeHints = true,
                includeInlayPropertyDeclarationTypeHints = true,
                includeInlayFunctionLikeReturnTypeHints = true,
                includeInlayEnumMemberValueHints = true,
              },
            },
          },
        },
        eslint = {},
        html = {},
        cssls = {},
        tailwindcss = {},
      },
    },
  },
```

```lua
-- Emmet for HTML/CSS
{
  "mattn/emmet-vim",
  ft = { "html", "css", "javascript", "typescript", "javascriptreact", "typescriptreact" }
},

-- Auto close tags
{
  "windwp/nvim-ts-autotag",
  ft = { "html", "javascript", "typescript", "javascriptreact", "typescriptreact", "svelte
  config = function()
    require("nvim-ts-autotag").setup()
  end,
},

-- Package.json management
{
  "vuki656/package-info.nvim",
  dependencies = "MunifTanjim/nui.nvim",
  ft = "json",
  config = function()
    require("package-info").setup()
  end,
},

-- REST client
{
  "rest-nvim/rest.nvim",
  dependencies = { "nvim-lua/plenary.nvim" },
  ft = "http",
  config = function()
    require("rest-nvim").setup({
      result_split_horizontal = false,
      result_split_in_place = false,
      skip_ssl_verification = false,
      encode_url = true,
      highlight = {
        enabled = true,
        timeout = 150,
      },
      result = {
        show_url = true,
        show_curl_command = false,
        show_http_info = true,
        show_headers = true,
        formatters = {
```

```
            json = "jq",
            html = function(body)
              return vim.fn.system({ "tidy", "-i", "-q", "-" }, body)
            end,
          },
        },
      })
    end,
  },
}
```

Go Development

```
-- ~/.config/nvim/lua/plugins/go.lua

return {
  -- Go plugin
  {
    "ray-x/go.nvim",
    dependencies = { -- optional packages
      "ray-x/guihua.lua",
      "neovim/nvim-lspconfig",
      "nvim-treesitter/nvim-treesitter",
    },
    config = function()
      require("go").setup()
    end,
    event = { "CmdlineEnter" },
    ft = { "go", "gomod" },
    build = ':lua require("go.install").update_all_sync()', -- if you need to install/update
  },

  -- Go testing
  {
    "nvim-neotest/neotest",
    optional = true,
    dependencies = {
      "nvim-neotest/neotest-go",
    },
    opts = {
      adapters = {
        ["neotest-go"] = {
          -- Here you can specify the settings for the adapter, i.e.
          args = { "-count=1", "-timeout=60s" },
        },
```

```
    },
  },
},

-- Go LSP configuration
{
  "neovim/nvim-lspconfig",
  opts = {
    servers = {
      gopls = {
        settings = {
          gopls = {
            gofumpt = true,
            codelenses = {
              gc_details = false,
              generate = true,
              regenerate_cgo = true,
              run_govulncheck = true,
              test = true,
              tidy = true,
              upgrade_dependency = true,
              vendor = true,
            },
            hints = {
              assignVariableTypes = true,
              compositeLiteralFields = true,
              compositeLiteralTypes = true,
              constantValues = true,
              functionTypeParameters = true,
              parameterNames = true,
              rangeVariableTypes = true,
            },
            analyses = {
              fieldalignment = true,
              nilness = true,
              unusedparams = true,
              unusedwrite = true,
              useany = true,
            },
            usePlaceholders = true,
            completeUnimported = true,
            staticcheck = true,
            directoryFilters = { "-.git", "-.vscode", "-.idea", "-.vscode-test", "-node_mo
            semanticTokens = true,
          },
        },
      },
```

```
      },
    },
  },

  -- Go debugging
  {
    "leoluz/nvim-dap-go",
    ft = "go",
    dependencies = "mfussenegger/nvim-dap",
    config = function(_, opts)
      require("dap-go").setup(opts)
    end,
  },
}
```

Customization Examples

Custom Colorscheme

```
-- ~/.config/nvim/lua/plugins/colorscheme.lua

return {
  -- Catppuccin theme
  {
    "catppuccin/nvim",
    name = "catppuccin",
    priority = 1000,
    opts = {
      flavour = "mocha", -- latte, frappe, macchiato, mocha
      background = { -- :h background
        light = "latte",
        dark = "mocha",
      },
      transparent_background = false,
      show_end_of_buffer = false,
      term_colors = false,
      dim_inactive = {
        enabled = false,
        shade = "dark",
        percentage = 0.15,
      },
      no_italic = false,
      no_bold = false,
      no_underline = false,
      styles = {
```

```
        comments = { "italic" },
        conditionals = { "italic" },
        loops = {},
        functions = {},
        keywords = {},
        strings = {},
        variables = {},
        numbers = {},
        booleans = {},
        properties = {},
        types = {},
        operators = {},
      },
      color_overrides = {},
      custom_highlights = {},
      integrations = {
        cmp = true,
        gitsigns = true,
        nvimtree = true,
        telescope = true,
        notify = false,
        mini = false,
      },
    },
  },

  -- Configure LazyVim to load catppuccin
  {
    "LazyVim/LazyVim",
    opts = {
      colorscheme = "catppuccin",
    },
  },
}
```

Custom Dashboard

```
-- ~/.config/nvim/lua/plugins/dashboard.lua

return {
  {
    "nvimdev/dashboard-nvim",
    event = "VimEnter",
    opts = function()
      local logo = [[
```

```
                                                Z
                                              Z
                                            z
                                          z


]]

logo = string.rep("\n", 8) .. logo .. "\n\n"

local opts = {
  theme = "doom",
  hide = {
    statusline = false,
  },
  config = {
    header = vim.split(logo, "\n"),
    center = {
      { action = "Telescope find_files", desc = " Find file", icon = " ", key = "f" },
      { action = "ene | startinsert", desc = " New file", icon = " ", key = "n" },
      { action = "Telescope oldfiles", desc = " Recent files", icon = " ", key = "r" }
      { action = "Telescope live_grep", desc = " Find text", icon = " ", key = "g" },
      { action = [[lua require("lazyvim.util").telescope.config_files()()]], desc = "
      { action = 'lua require("persistence").load()', desc = " Restore Session", icon
      { action = "LazyExtras", desc = " Lazy Extras", icon = " ", key = "x" },
      { action = "Lazy", desc = " Lazy", icon = "  ", key = "l" },
      { action = "qa", desc = " Quit", icon = " ", key = "q" },
    },
    footer = function()
      local stats = require("lazy").stats()
      local ms = (math.floor(stats.startuptime * 100 + 0.5) / 100)
      return { "  Neovim loaded " .. stats.loaded .. "/" .. stats.count .. " plugins in
    end,
  },
}

for _, button in ipairs(opts.config.center) do
  button.desc = button.desc .. string.rep(" ", 43 - #button.desc)
  button.key_format = "  %s"
end

-- close Lazy and re-open when the dashboard is ready
if vim.o.filetype == "lazy" then
  vim.cmd.close()
  vim.api.nvim_create_autocmd("User", {
    pattern = "DashboardLoaded",
    callback = function()
```

```
            require("lazy").show()
          end,
        })
      end

      return opts
    end,
  },
}
```

Performance Optimization

Startup Optimization

```
-- ~/.config/nvim/lua/config/lazy.lua

-- Bootstrap lazy.nvim
local lazypath = vim.fn.stdpath("data") .. "/lazy/lazy.nvim"
if not vim.loop.fs_stat(lazypath) then
  vim.fn.system({
    "git",
    "clone",
    "--filter=blob:none",
    "https://github.com/folke/lazy.nvim.git",
    "--branch=stable",
    lazypath,
  })
end
vim.opt.rtp:prepend(lazypath)

require("lazy").setup({
  spec = {
    -- add LazyVim and import its plugins
    { "LazyVim/LazyVim", import = "lazyvim.plugins" },
    -- import any extras modules here
    { import = "lazyvim.plugins.extras.lang.typescript" },
    { import = "lazyvim.plugins.extras.lang.json" },
    { import = "lazyvim.plugins.extras.ui.mini-animate" },
    -- import/override with your plugins
    { import = "plugins" },
  },
  defaults = {
    lazy = false, -- should plugins be lazy-loaded?
    version = false, -- always use the latest git commit
```

```
  },
  install = { colorscheme = { "tokyonight", "habamax" } },
  checker = { enabled = true }, -- automatically check for plugin updates
  performance = {
    rtp = {
      -- disable some rtp plugins
      disabled_plugins = {
        "gzip",
        "matchit",
        "matchparen",
        "netrwPlugin",
        "tarPlugin",
        "tohtml",
        "tutor",
        "zipPlugin",
      },
    },
  },
})
```

Memory Optimization

```
-- ~/.config/nvim/lua/plugins/performance.lua

return {
  -- Optimize treesitter
  {
    "nvim-treesitter/nvim-treesitter",
    opts = {
      highlight = { enable = true },
      indent = { enable = true },
      ensure_installed = {
        "bash",
        "c",
        "html",
        "javascript",
        "json",
        "lua",
        "luadoc",
        "luap",
        "markdown",
        "markdown_inline",
        "python",
        "query",
        "regex",
```

```
        "tsx",
        "typescript",
        "vim",
        "vimdoc",
        "yaml",
      },
      incremental_selection = {
        enable = true,
        keymaps = {
          init_selection = "<C-space>",
          node_incremental = "<C-space>",
          scope_incremental = false,
          node_decremental = "<bs>",
        },
      },
    },
  },

  -- Optimize LSP
  {
    "neovim/nvim-lspconfig",
    opts = {
      diagnostics = {
        underline = true,
        update_in_insert = false,
        virtual_text = {
          spacing = 4,
          source = "if_many",
          prefix = " ",
        },
        severity_sort = true,
      },
      inlay_hints = {
        enabled = true,
      },
      capabilities = {},
      format = {
        formatting_options = nil,
        timeout_ms = nil,
      },
      servers = {},
      setup = {},
    },
  },
}
```

Troubleshooting

Common Issues

```
# Check LazyVim health
:checkhealth

# Check specific components
:checkhealth lazy
:checkhealth mason
:checkhealth telescope
:checkhealth treesitter

# Plugin management
:Lazy                  # Open Lazy plugin manager
:Lazy sync            # Update all plugins
:Lazy clean           # Remove unused plugins
:Lazy profile         # Profile startup time

# LSP troubleshooting
:LspInfo              # Show LSP information
:LspRestart           # Restart LSP servers
:Mason                # Open Mason (LSP installer)

# Clear cache
:lua vim.fn.delete(vim.fn.stdpath("cache"), "rf")
```

Reset Configuration

```
# Backup current config
mv ~/.config/nvim ~/.config/nvim.backup
mv ~/.local/share/nvim ~/.local/share/nvim.backup
mv ~/.local/state/nvim ~/.local/state/nvim.backup
mv ~/.cache/nvim ~/.cache/nvim.backup

# Fresh install
git clone https://github.com/LazyVim/starter ~/.config/nvim
rm -rf ~/.config/nvim/.git
```

Performance Issues

```
-- Profile startup time
-- Add to ~/.config/nvim/lua/config/options.lua
vim.g.loaded_python3_provider = 0
vim.g.loaded_ruby_provider = 0
vim.g.loaded_perl_provider = 0
vim.g.loaded_node_provider = 0

-- Disable unused features
vim.g.loaded_gzip = 1
vim.g.loaded_zip = 1
vim.g.loaded_zipPlugin = 1
vim.g.loaded_tar = 1
vim.g.loaded_tarPlugin = 1
vim.g.loaded_getscript = 1
vim.g.loaded_getscriptPlugin = 1
vim.g.loaded_vimball = 1
vim.g.loaded_vimballPlugin = 1
vim.g.loaded_2html_plugin = 1
vim.g.loaded_logiPat = 1
vim.g.loaded_rrhelper = 1
vim.g.loaded_netrw = 1
vim.g.loaded_netrwPlugin = 1
vim.g.loaded_netrwSettings = 1
vim.g.loaded_netrwFileHandlers = 1
```

LazyVim provides an excellent foundation for a modern Neovim setup, combining the power of Neovim with sensible defaults and a beautiful interface. It's perfect for developers who want a powerful IDE-like experience without the complexity of building a configuration from scratch.

Helix

Helix: A Modern Modal Editor

Helix is a modern, terminal-based text editor written in Rust. It takes inspiration from Kakoune and Vim, featuring multiple selections, built-in LSP support, and a focus on simplicity and performance.

What is Helix?

Helix is designed with modern editing principles: - **Selection-first editing**: Select text first, then act on it - **Multiple selections**: Edit multiple locations simultaneously - **Built-in LSP**: Language Server Protocol support out of the box - **Tree-sitter**: Advanced syntax highlighting and text objects - **No configuration required**: Works great with zero configuration - **Fast and lightweight**: Written in Rust for performance

Why Choose Helix?

- **Modern design**: Built from the ground up with modern editing concepts
- **Zero configuration**: Sensible defaults that work immediately
- **Multiple selections**: Powerful multi-cursor editing
- **Built-in features**: LSP, tree-sitter, and fuzzy finder included
- **Consistent keybindings**: Logical and memorable key combinations
- **Linux-optimized**: Designed with Linux development workflows in mind
- **Active development**: Regular updates and improvements
- **Rust performance**: Fast and memory-efficient on Linux systems

Installation

Package Managers

```
# Ubuntu/Debian (from source)
sudo apt update
sudo apt install git rustc cargo
git clone https://github.com/helix-editor/helix
cd helix
cargo install --path helix-term --locked

# Arch Linux
```

```
sudo pacman -S helix

# Fedora
sudo dnf install helix

# SUSE/openSUSE
sudo zypper install helix

# Gentoo
sudo emerge app-editors/helix

# Void Linux
sudo xbps-install helix

# Nix
nix-env -iA nixpkgs.helix

# Cargo (from source)
cargo install helix-term --locked
```

Building from Source

```
# Clone repository
git clone https://github.com/helix-editor/helix
cd helix

# Build and install
cargo install --path helix-term --locked

# Install runtime files
mkdir -p ~/.config/helix
cp -r runtime ~/.config/helix/

# Or use system-wide installation
sudo mkdir -p /usr/local/lib/helix
sudo cp -r runtime /usr/local/lib/helix/
```

Post-Installation Setup

```
# Install language servers (optional but recommended)
# JavaScript/TypeScript
npm install -g typescript-language-server typescript
```

```
# Python
pip install python-lsp-server

# Rust (if not already installed)
rustup component add rust-analyzer

# Go
go install golang.org/x/tools/gopls@latest

# Check health and see what's available
hx --health
```

Basic Concepts

Selection-First Editing

Unlike Vim's action-first approach, Helix uses selection-first:

```
# Vim way: action + motion
dw    # Delete word (action first)

# Helix way: selection + action
w     # Select word
d     # Delete selection
```

Multiple Selections

Helix excels at multiple selections:

```
# Select all occurrences of current selection
%     # Select all matches

# Add selection
C     # Duplicate cursor/selection
Alt-C # Remove primary selection

# Split selections
S     # Split selection on regex
Alt-s # Split selection on lines
```

Modes

Helix has several modes: - **Normal mode**: Default mode for navigation and commands - **Insert mode**: For typing text - **Select mode**: For making selections - **Command mode**: For ex-style

Essential Commands

Starting Helix

```
# Open file
hx filename.txt

# Open multiple files
hx file1.txt file2.txt

# Open directory (file picker)
hx .

# Open with specific line/column
hx filename.txt:25:10

# Health check
hx --health
```

Basic Navigation

```
# Character movement
h     # Left
j     # Down
k     # Up
l     # Right

# Word movement
w     # Next word start
b     # Previous word start
e     # Next word end

# Line movement
0     # Line start
^     # First non-whitespace
$     # Line end
g h   # Line start (alternative)
g l   # Line end (alternative)

# Document movement
g g   # Document start
g e   # Document end
```

```
Ctrl+u # Page up
Ctrl+d # Page down

# Paragraph movement
[      # Previous paragraph
]      # Next paragraph
```

Text Selection

```
# Basic selection
v      # Enter select mode
V      # Select line
Ctrl+v # Select block (not implemented yet)

# Word/object selection
w      # Select word
W      # Select WORD (whitespace-separated)
p      # Select paragraph
f<char> # Select to character
t<char> # Select until character

# Extend selection
;      # Flip selection direction
Alt+; # Ensure forward selection direction
%      # Select matching bracket
m      # Select matching pair (quotes, brackets, etc.)

# Multiple selections
C      # Duplicate cursor
Alt+C # Remove primary cursor
s      # Select regex matches within selection
S      # Split selection on regex
Alt-s # Split selection on newlines
&      # Align selections
_      # Trim whitespace from selections
```

Editing Commands

```
# Insert modes
i      # Insert before selection
a      # Insert after selection
I      # Insert at line start
A      # Insert at line end
o      # Insert new line below
```

```
O     # Insert new line above

# Delete/Change
d     # Delete selection
c     # Change selection (delete and enter insert mode)
Alt+d # Delete line
Alt+c # Change line

# Copy/Paste
y     # Yank (copy) selection
p     # Paste after selection
P     # Paste before selection
Alt+p # Paste every yanked selection
R     # Replace selection with yanked text

# Undo/Redo
u     # Undo
U     # Redo
Alt+u # Earlier in time
Alt+U # Later in time
```

Search and Replace

```
# Search
/     # Search forward
?     # Search backward
n     # Next search result
N     # Previous search result
*     # Search word under cursor

# Replace
r<char> # Replace character
~     # Switch case
`     # Set case to lowercase
Alt+` # Set case to uppercase

# Global replace (in command mode)
:s/old/new/g        # Replace in current selection
:%s/old/new/g       # Replace in entire file
:s/old/new/gc       # Replace with confirmation
```

File and Buffer Management

File Operations

```
# File commands (in command mode)
:o filename   # Open file
:w            # Write file
:w filename   # Write as filename
:q            # Quit
:q!           # Quit without saving
:wq           # Write and quit

# Buffer navigation
Space b       # Buffer picker
gn            # Next buffer
gp            # Previous buffer
Space x       # Close buffer
```

File Picker and Fuzzy Finding

```
# File operations
Space f       # File picker
Space F       # File picker (from cwd)
Space b       # Buffer picker
Space s       # Symbol picker (LSP)
Space S       # Workspace symbol picker
Space g       # Global search (grep)
Space /       # Global search in current file
Space k       # Show documentation
Space r       # Rename symbol (LSP)
Space a       # Code action (LSP)
```

Configuration

Configuration File

Helix uses TOML for configuration:

```
# ~/.config/helix/config.toml

theme = "onedark"

[editor]
```

```
line-number = "relative"
mouse = true
middle-click-paste = true
scroll-lines = 3
shell = ["sh", "-c"]
text-width = 80
completion-trigger-len = 2
auto-completion = true
auto-format = true
auto-save = false
idle-timeout = 400
preview-completion-insert = true
completion-replace = false
auto-info = true
true-color = false
rulers = [80, 120]
bufferline = "always"
color-modes = false

[editor.statusline]
left = ["mode", "spinner"]
center = ["file-name"]
right = ["diagnostics", "selections", "position", "file-encoding", "file-line-ending", "file
separator = " "
mode.normal = "NORMAL"
mode.insert = "INSERT"
mode.select = "SELECT"

[editor.lsp]
display-messages = true
auto-signature-help = true
display-inlay-hints = true
display-signature-help-docs = true
snippets = true
goto-reference-include-declaration = true

[editor.cursor-shape]
insert = "bar"
normal = "block"
select = "underline"

[editor.file-picker]
hidden = true
follow-symlinks = true
deduplicate-links = true
parents = true
ignore = true
```

```
git-ignore = true
git-global = true
git-exclude = true
max-depth = 25

[editor.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
"'" = "'"
"`" = "`"
'<' = '>'

[editor.search]
smart-case = true
wrap-around = true

[editor.whitespace]
render = "all"
characters = { space = "·", nbsp = " ", tab = "→", newline = " ", tabpad = "·" }

[editor.indent-guides]
render = true
character = " "
skip-levels = 1

[editor.gutters]
layout = ["diff", "diagnostics", "line-numbers", "spacer"]

[editor.soft-wrap]
enable = false
max-wrap = 25
max-indent-retain = 0
wrap-indicator = ""

[keys.normal]
# Custom keybindings
C-s = ":w"
C-o = "file_picker"
C-p = "file_picker"
esc = ["collapse_selection", "keep_primary_selection"]

[keys.insert]
# Insert mode keybindings
C-s = ["normal_mode", ":w", "insert_mode"]
j = { k = "normal_mode" } # jk to exit insert mode
```

```
[keys.select]
# Select mode keybindings
esc = ["collapse_selection", "keep_primary_selection", "normal_mode"]
```

Language Configuration

```
# ~/.config/helix/languages.toml

# Override language server configurations
[[language]]
name = "python"
language-server = { command = "pylsp" }
auto-format = true

[language.config.pylsp.plugins]
autopep8 = { enabled = false }
flake8 = { enabled = true }
mccabe = { enabled = false }
pycodestyle = { enabled = false }
pyflakes = { enabled = false }
pylint = { enabled = true }
yapf = { enabled = true }

[[language]]
name = "javascript"
language-server = { command = "typescript-language-server", args = ["--stdio"] }
auto-format = true

[[language]]
name = "typescript"
language-server = { command = "typescript-language-server", args = ["--stdio"] }
auto-format = true

[[language]]
name = "rust"
language-server = { command = "rust-analyzer" }
auto-format = true

[[language]]
name = "go"
language-server = { command = "gopls" }
auto-format = true

[[language]]
name = "html"
```

```
language-server = { command = "vscode-html-language-server", args = ["--stdio"] }
auto-format = true

[[language]]
name = "css"
language-server = { command = "vscode-css-language-server", args = ["--stdio"] }
auto-format = true

[[language]]
name = "json"
language-server = { command = "vscode-json-language-server", args = ["--stdio"] }
auto-format = true

# Custom language definitions
[[language]]
name = "log"
scope = "text.log"
file-types = ["log"]
comment-token = "#"
indent = { tab-width = 2, unit = "  " }

[[grammar]]
name = "log"
source = { git = "https://github.com/Tudyx/tree-sitter-log", rev = "main" }
```

Themes

Helix comes with many built-in themes:

```
# List available themes
:theme

# Change theme temporarily
:theme dracula

# Popular themes
:theme onedark
:theme gruvbox
:theme catppuccin_mocha
:theme tokyo-night
:theme solarized_dark
:theme nord
```

Custom Theme

```
# ~/.config/helix/themes/mytheme.toml

"ui.background" = { bg = "#1e1e1e" }
"ui.text" = "#d4d4d4"
"ui.text.focus" = "#ffffff"
"ui.selection" = { bg = "#264f78" }
"ui.cursor" = { fg = "#1e1e1e", bg = "#d4d4d4" }
"ui.cursor.match" = { bg = "#3a3d41" }
"ui.linenr" = "#858585"
"ui.linenr.selected" = "#c6c6c6"
"ui.statusline" = { fg = "#d4d4d4", bg = "#007acc" }
"ui.statusline.inactive" = { fg = "#d4d4d4", bg = "#3c3c3c" }
"ui.popup" = { bg = "#252526" }
"ui.window" = "#3c3c3c"
"ui.help" = { bg = "#252526" }

"error" = "#f44747"
"warning" = "#ff8c00"
"info" = "#0080ff"
"hint" = "#969696"

"diagnostic.error" = { underline = { color = "#f44747", style = "curl" } }
"diagnostic.warning" = { underline = { color = "#ff8c00", style = "curl" } }
"diagnostic.info" = { underline = { color = "#0080ff", style = "curl" } }
"diagnostic.hint" = { underline = { color = "#969696", style = "curl" } }

"syntax.comment" = "#6a9955"
"syntax.keyword" = "#569cd6"
"syntax.string" = "#ce9178"
"syntax.number" = "#b5cea8"
"syntax.boolean" = "#569cd6"
"syntax.type" = "#4ec9b0"
"syntax.function" = "#dcdcaa"
"syntax.variable" = "#9cdcfe"
"syntax.constant" = "#4fc1ff"
"syntax.operator" = "#d4d4d4"
"syntax.punctuation" = "#d4d4d4"
```

Real-World Usage Scenarios

Scenario 1: Python Development

```
# Open Python project
hx main.py

# Navigate to function definition
gd    # Go to definition (LSP)
gr    # Go to references (LSP)

# Multiple cursor editing
# Select function name
w     # Select word
%     # Select all occurrences
c     # Change all occurrences
new_function_name<Esc>

# Code formatting
:format   # Format current file
# Or enable auto-format in config

# Symbol search
Space s   # Search symbols in file
Space S   # Search symbols in workspace

# Diagnostics
]d        # Next diagnostic
[d        # Previous diagnostic
Space k   # Show hover information
Space a   # Code actions
```

Scenario 2: Web Development

```
# Open HTML file
hx index.html

# Select HTML tag
mt    # Select matching tag pair
c     # Change tag
div<Esc>

# Multiple selections for attributes
# Select class attribute value
f"    # Find quote
```

```
mi    # Select inside quotes
%     # Select all similar
c     # Change all
new-class<Esc>

# Navigate between files
Space f   # File picker
# Type filename and Enter

# Global search and replace
Space g   # Global search
# Type search term
# Navigate results with j/k
# Press Enter to go to location
```

Scenario 3: Configuration File Editing

```
# Open config file
hx ~/.config/helix/config.toml

# Navigate to specific section
/editor   # Search for "editor"
n         # Next occurrence

# Select and modify values
f=        # Find equals sign
l         # Move right
w         # Select value
c         # Change value
new_value<Esc>

# Comment/uncomment lines
Alt+;     # Toggle line comment
# Or select multiple lines and comment
V         # Select line
jjj       # Extend selection
Alt+;     # Comment selected lines
```

Scenario 4: Log File Analysis

```
# Open log file
hx /var/log/application.log

# Search for errors
```

```
/ERROR    # Search for ERROR
n         # Next occurrence
N         # Previous occurrence

# Select all error lines
*         # Search word under cursor (ERROR)
%         # Select all matches
y         # Yank all error lines

# Open new buffer for analysis
:new
p         # Paste error lines

# Split selections by timestamp
s^\d{4}-\d{2}-\d{2}  # Split on date pattern
```

Advanced Features

Multiple Selections Mastery

```
# Complex selection scenarios
# Select all function calls
/\w+\(   # Search for function calls
%        # Select all matches

# Select inside parentheses for all matches
mi       # Select inside matching pairs

# Add numbers to each selection
C        # Duplicate cursor to each selection
I        # Insert at beginning
1. <Esc> # Add numbering (manual)

# Or use regex replacement
:s/^/1. /  # Add numbering with regex
```

Text Objects

```
# Built-in text objects
mw    # Select word
mp    # Select paragraph
m(    # Select inside parentheses
m)    # Select around parentheses
```

```
m[    # Select inside brackets
m]    # Select around brackets
m{    # Select inside braces
m}    # Select around braces
m"    # Select inside quotes
m'    # Select inside single quotes
m`    # Select inside backticks
mt    # Select inside HTML/XML tags
mf    # Select function
mc    # Select class
ma    # Select argument
```

Tree-sitter Navigation

```
# Navigate syntax tree
Alt+p # Parent node
Alt+c # Child node
Alt+n # Next sibling
Alt+o # Previous sibling

# Select syntax nodes
mf    # Select function
mc    # Select class
ma    # Select argument/parameter
```

LSP Features

```
# Code navigation
gd    # Go to definition
gy    # Go to type definition
gr    # Go to references
gi    # Go to implementation

# Code information
Space k   # Hover information
Space K   # Signature help

# Code modification
Space r   # Rename symbol
Space a   # Code actions
:format   # Format document

# Diagnostics
]d        # Next diagnostic
```

```
[d        # Previous diagnostic
]D        # Next error
[D        # Previous error
Space d   # Show diagnostics
```

Customization and Scripting

Custom Commands

```
# ~/.config/helix/config.toml

[keys.normal]
# Custom command sequences
C-g = [":new", ":insert-output echo 'Hello World'"]
C-t = [":sh date"]

# Macro-like sequences
F1 = ["select_all", "yank", ":new", "paste_after"]
F2 = ["goto_line_start", "insert_mode", "// ", "normal_mode"]
```

Integration with External Tools

```
# Pipe selection to external command
|sort     # Sort selected lines
|uniq     # Remove duplicate lines
|wc -l    # Count lines
|jq       # Format JSON
|xmllint --format -  # Format XML

# Insert command output
:insert-output date
:insert-output ls -la
:insert-output git log --oneline -10
```

Shell Integration

```
# Set Helix as default editor
export EDITOR=hx
export VISUAL=hx

# Git integration
```

```
git config --global core.editor hx

# Shell aliases
alias h='hx'
alias hx.='hx .'
alias hxc='hx ~/.config/helix/config.toml'
```

Performance and Optimization

Large File Handling

Helix handles large files efficiently:

```
# Open large files
hx large_file.log

# Navigate efficiently
gg    # Go to start
ge    # Go to end
Ctrl+u # Page up
Ctrl+d # Page down

# Search in large files
/pattern  # Incremental search
n         # Next match
N         # Previous match
```

Memory Usage

```
# Check memory usage
:debug-info

# Optimize for memory
# Disable unnecessary features in config
[editor]
auto-completion = false
auto-info = false
```

Troubleshooting

Common Issues

```
# Check health
hx --health

# Language server issues
:lsp-workspace-command
:lsp-restart

# Theme issues
:theme default
:reload-config

# Key binding conflicts
# Check config.toml for conflicting bindings
```

Debug Mode

```
# Start with debug logging
RUST_LOG=debug hx

# Check logs
tail -f ~/.cache/helix/helix.log
```

Reset Configuration

```
# Backup current config
mv ~/.config/helix ~/.config/helix.backup

# Start fresh
hx  # Will create default config
```

Migration from Other Editors

From Vim/Neovim

Key differences to remember: - Selection-first editing - Different keybindings for some operations - Built-in LSP and tree-sitter - No plugin system (yet)

From VSCode

Helix provides many VSCode-like features: - Built-in LSP support - Fuzzy file finding - Multiple selections - Integrated terminal (external)

From Kakoune

Helix is inspired by Kakoune: - Similar selection-first philosophy - Multiple selections - Different keybindings and features

Helix offers a modern, efficient editing experience with powerful features built-in. Its selection-first approach and multiple selections make it particularly effective for complex text manipulation tasks, while the zero-configuration philosophy means you can be productive immediately. ## Next steps

- tmux for splits and SSH detach
- Herdr when running AI agent CLIs beside Helix
- Modern Tools — rg, fd, fzf, bat
- Workflows

Micro

Micro: A Modern Terminal Editor

Micro is a modern, intuitive terminal-based text editor that aims to be easy to use and easy to remember. It provides a familiar interface similar to GUI editors while running in the terminal, making it perfect for users transitioning from graphical editors.

What is Micro?

Micro is designed with simplicity and usability in mind: - **Intuitive keybindings**: Familiar Ctrl+S, Ctrl+C, Ctrl+V shortcuts - **Mouse support**: Click to position cursor, select text, scroll - **Syntax highlighting**: Built-in support for 100+ languages - **Plugin system**: Extensible with Lua plugins - **Multiple cursors**: Edit multiple locations simultaneously - **Linux-optimized**: Excellent performance on Linux systems - **No configuration required**: Works great out of the box

Why Choose Micro?

- **Easy to learn**: Familiar keybindings from GUI editors
- **Modern features**: Multiple cursors, syntax highlighting, plugins
- **Lightweight**: Single binary with no dependencies, perfect for Linux servers
- **Customizable**: Extensive configuration options
- **Active development**: Regular updates and improvements
- **Terminal-based**: Works over SSH and in minimal Linux environments
- **Unicode support**: Full Unicode and emoji support
- **System administration**: Great for Linux configuration file editing

Installation

Package Managers

```
# Ubuntu/Debian
sudo apt update
sudo apt install micro

# Fedora
sudo dnf install micro

# Arch Linux
```

```
sudo pacman -S micro

# SUSE/openSUSE
sudo zypper install micro

# Gentoo
sudo emerge app-editors/micro

# Void Linux
sudo xbps-install micro

# Snap (Universal Linux)
sudo snap install micro --classic

# Flatpak
flatpak install flathub io.github.micro-editor.micro
```

Direct Download

```
# Download latest release
curl https://getmic.ro | bash

# Or manually download
wget https://github.com/zyedidia/micro/releases/download/v2.0.11/micro-2.0.11-linux64.tar.gz
tar -xzf micro-2.0.11-linux64.tar.gz
sudo mv micro-2.0.11/micro /usr/local/bin/

# Make executable
chmod +x /usr/local/bin/micro
```

From Source

```
# Install Go if not already installed
# Then build micro
git clone https://github.com/zyedidia/micro
cd micro
make build
sudo mv micro /usr/local/bin/
```

Basic Usage

Starting Micro

```
# Open new file
micro

# Open existing file
micro filename.txt

# Open multiple files
micro file1.txt file2.txt

# Open directory (file browser)
micro .

# Open with specific settings
micro -colorscheme monokai filename.txt
micro -syntax python script.py
```

Interface Overview

```
  filename.txt
This is the content of your file.
You can type directly here.
Mouse clicks work for positioning cursor.









Ctrl+Q Quit  Ctrl+S Save  Ctrl+O Open  Ctrl+F Find  Ctrl+Z Undo  Ctrl+Y Redo
```

Essential Keybindings

File Operations

```
# File management
Ctrl+O          # Open file
Ctrl+S          # Save file
Ctrl+W          # Close current buffer
Ctrl+Q          # Quit micro
Ctrl+E          # Command mode
Ctrl+G          # Open help

# Buffer navigation
Ctrl+Tab        # Next buffer
Ctrl+Shift+Tab  # Previous buffer
Alt+,           # Previous buffer
Alt+.           # Next buffer
```

Navigation

```
# Basic movement
Arrow keys      # Move cursor
Ctrl+Left/Right # Move by word
Ctrl+Up/Down    # Move by paragraph
Home/End        # Beginning/end of line
Ctrl+Home/End   # Beginning/end of file
Page Up/Down    # Page navigation

# Advanced navigation
Ctrl+G          # Go to line
Ctrl+F          # Find
Ctrl+N          # Find next
Ctrl+P          # Find previous
```

Text Editing

```
# Basic editing
Ctrl+Z          # Undo
Ctrl+Y          # Redo
Ctrl+C          # Copy
Ctrl+X          # Cut
Ctrl+V          # Paste
Ctrl+A          # Select all
```

```
Ctrl+D          # Duplicate line
Ctrl+K          # Delete line

# Advanced editing
Ctrl+L          # Select line
Ctrl+Shift+Up   # Move line up
Ctrl+Shift+Down # Move line down
Alt+Shift+Left  # Select word left
Alt+Shift+Right # Select word right
```

Multiple Cursors

```
# Multiple cursor operations
Ctrl+MouseClick # Add cursor at click position
Alt+N           # Create cursor at next word occurrence
Alt+P           # Create cursor at previous word occurrence
Alt+C           # Create cursor at all word occurrences
Alt+M           # Remove all cursors except primary
Escape          # Remove all extra cursors
```

Search and Replace

```
# Search operations
Ctrl+F          # Find
Ctrl+N          # Find next
Ctrl+P          # Find previous
Ctrl+R          # Find and replace

# In find/replace dialog
Tab             # Switch between find and replace fields
Enter           # Execute search/replace
Escape          # Cancel dialog
```

Configuration

Settings File

Micro stores settings in `~/.config/micro/settings.json`:

```
{
    "autoclose": true,
    "autoindent": true,
```

```
    "autosave": 0,
    "autosu": false,
    "backup": true,
    "backupdir": "",
    "basename": false,
    "colorcolumn": 0,
    "colorscheme": "default",
    "cursorline": true,
    "diffgutter": false,
    "divchars": "|-",
    "divreverse": true,
    "encoding": "utf-8",
    "eofnewline": true,
    "fastdirty": false,
    "fileformat": "unix",
    "filetype": "unknown",
    "hlsearch": false,
    "ignorecase": false,
    "indentchar": " ",
    "infobar": true,
    "keepautoindent": false,
    "keymenu": false,
    "matchbrace": true,
    "matchbraceleft": false,
    "mouse": true,
    "parsecursor": false,
    "paste": false,
    "pluginchannels": [
        "https://raw.githubusercontent.com/micro-editor/plugin-channel/master/channel.json"
    ],
    "pluginrepos": [],
    "readonly": false,
    "rmtrailingws": false,
    "ruler": true,
    "savecursor": false,
    "savehistory": true,
    "saveundo": false,
    "scrollbar": false,
    "scrollmargin": 3,
    "scrollspeed": 2,
    "smartpaste": true,
    "softwrap": false,
    "splitbottom": true,
    "splitright": true,
    "statusformatl": "$(filename) $(modified)($(line),$(col)) $(status.paste)| ft:$(opt:file
    "statusformatr": "$(bind:ToggleKeyMenu): bindings, $(bind:ToggleHelp): help",
    "statusline": true,
```

```
    "sucmd": "sudo",
    "syntax": true,
    "tabmovement": false,
    "tabsize": 4,
    "tabstospaces": false,
    "termtitle": false,
    "useprimary": true,
    "wordwrap": false,
    "xterm": false
}
```

Common Configuration Changes

```
# Set tab size to 2
micro -option tabsize 2

# Enable line numbers
micro -option ruler true

# Set color scheme
micro -option colorscheme monokai

# Enable soft wrapping
micro -option softwrap true

# Auto-save every 10 seconds
micro -option autosave 10

# Remove trailing whitespace on save
micro -option rmtrailingws true

# Make changes permanent
# These settings are automatically saved to settings.json
```

Keybinding Customization

Create `~/.config/micro/bindings.json`:

```
{
    "Alt+/": "lua:comment.comment",
    "Ctrl+/": "lua:comment.comment",
    "Ctrl+Shift+D": "DuplicateLine",
    "Ctrl+Shift+K": "DeleteLine",
    "Ctrl+Shift+Up": "MoveLinesUp",
```

```
    "Ctrl+Shift+Down": "MoveLinesDown",
    "Ctrl+J": "JumpLine",
    "Ctrl+B": "ToggleBookmark",
    "F3": "FindNext",
    "Shift+F3": "FindPrevious",
    "F4": "Quit",
    "Ctrl+T": "NewTab",
    "Ctrl+Shift+T": "ReopenLastClosedTab"
}
```

Color Schemes

Micro comes with many built-in color schemes:

```
# List available color schemes
micro -option colorscheme help

# Popular color schemes
micro -option colorscheme monokai
micro -option colorscheme solarized
micro -option colorscheme dracula
micro -option colorscheme gruvbox
micro -option colorscheme atom-dark
micro -option colorscheme github
micro -option colorscheme zenburn
```

Custom Color Scheme

Create `~/.config/micro/colorschemes/mytheme.micro`:

```
color-link default "#d4d4d4,#1e1e1e"
color-link comment "#6a9955"
color-link identifier "#9cdcfe"
color-link constant "#4fc1ff"
color-link statement "#569cd6"
color-link symbol "#d4d4d4"
color-link preproc "#c586c0"
color-link type "#4ec9b0"
color-link special "#dcdcaa"
color-link underlined "#d4d4d4"
color-link error "#f44747"
color-link todo "#ffcc00"
color-link hlsearch "#264f78"
color-link statusline "#d4d4d4,#007acc"
color-link tabbar "#d4d4d4,#2d2d30"
```

```
color-link indent-char "#3c3c3c"
color-link line-number "#858585"
color-link current-line-number "#c6c6c6"
color-link diff-added "#28a745"
color-link diff-modified "#ffd33d"
color-link diff-deleted "#d73a49"
color-link gutter-error "#f44747"
color-link gutter-warning "#ff8c00"
color-link cursor-line "#2a2d2e"
color-link color-column "#3c3c3c"
```

Plugin System

Plugin Management

```
# List installed plugins
micro -plugin list

# Install plugin
micro -plugin install pluginname

# Remove plugin
micro -plugin remove pluginname

# Update plugins
micro -plugin update

# Search for plugins
micro -plugin search keyword

# Plugin information
micro -plugin info pluginname
```

Popular Plugins

```
# Essential plugins
micro -plugin install filemanager    # File browser
micro -plugin install linter         # Code linting
micro -plugin install comment        # Toggle comments
micro -plugin install jump           # Quick navigation
micro -plugin install manipulator    # Text manipulation
micro -plugin install quoter         # Quote manipulation
micro -plugin install snippets       # Code snippets
micro -plugin install autoclose      # Auto-close brackets
```

```
micro -plugin install bookmark       # Bookmarks
micro -plugin install diff           # Diff viewer

# Language-specific plugins
micro -plugin install go             # Go language support
micro -plugin install python         # Python support
micro -plugin install html           # HTML support
micro -plugin install css            # CSS support
micro -plugin install javascript     # JavaScript support
```

Plugin Configuration

Plugins can be configured in `~/.config/micro/settings.json`:

```
{
    "linter": true,
    "linter.ignoredMessages": [],
    "comment.leader": "# ",
    "filemanager.showdotfiles": true,
    "filemanager.showignored": false,
    "snippets.enabled": true,
    "autoclose.enabled": true
}
```

Real-World Usage Scenarios

Scenario 1: Linux System Configuration

```
# Edit SSH daemon configuration
sudo micro /etc/ssh/sshd_config

# Navigate to specific setting
Ctrl+F
# Type "PasswordAuthentication"
Enter

# Change the value
# Use mouse to click and position cursor
# Change "yes" to "no"

# Save and restart SSH service
Ctrl+S
Ctrl+Q
```

```
sudo systemctl restart sshd

# Alternative: Edit systemd service file
sudo micro /etc/systemd/system/myapp.service

# Add service configuration
[Unit]
Description=My Application
After=network.target

[Service]
Type=simple
User=myapp
ExecStart=/usr/local/bin/myapp
Restart=always

[Install]
WantedBy=multi-user.target

# Save and enable service
Ctrl+S
Ctrl+Q
sudo systemctl daemon-reload
sudo systemctl enable myapp
sudo systemctl start myapp
```

Scenario 2: Python Development

```
# Open Python file
micro script.py

# Enable Python-specific settings
# (Automatically detected by file extension)

# Multiple cursor editing for variable renaming
# Double-click on variable name
Alt+C  # Select all occurrences
# Type new variable name

# Comment/uncomment code blocks
# Select lines with mouse or Shift+arrows
Ctrl+/  # Toggle comments

# Duplicate lines for testing
Ctrl+D  # Duplicate current line
```

```
# Save file
Ctrl+S
```

Scenario 3: Log File Analysis

```
# Open log file
micro /var/log/application.log

# Search for errors
Ctrl+F
# Type "ERROR"
Enter

# Navigate through errors
Ctrl+N  # Next occurrence
Ctrl+P  # Previous occurrence

# Select error lines for copying
# Click and drag to select
Ctrl+C  # Copy

# Open new buffer for analysis
Ctrl+E
# Type "tab new"
Enter

# Paste errors
Ctrl+V

# Save analysis
Ctrl+S
# Type filename
Enter
```

Scenario 4: Web Development

```
# Open HTML file
micro index.html

# Use multiple cursors for tag editing
# Select opening tag
Alt+C  # Select all similar tags
# Edit all tags simultaneously
```

```
# Navigate between files
Ctrl+O
# Type filename or use arrow keys
Enter

# Split view for CSS editing
Ctrl+E
# Type "hsplit styles.css"
Enter

# Switch between splits
Ctrl+W  # Cycle through splits

# Format code (if formatter plugin installed)
Ctrl+E
# Type "format"
Enter
```

Advanced Features

Command Mode

Access command mode with `Ctrl+E`:

```
# File operations
open filename.txt       # Open file
save                    # Save current file
save filename.txt       # Save as
quit                    # Quit current buffer
exit                    # Exit micro

# Buffer operations
tab new                 # New tab
tab close               # Close current tab
tab switch N            # Switch to tab N
buffer N                # Switch to buffer N

# View operations
hsplit filename.txt     # Horizontal split
vsplit filename.txt     # Vertical split
unsplit                 # Close current split

# Search and replace
find pattern            # Find pattern
replace old new         # Replace old with new
```

```
replaceall old new    # Replace all occurrences

# Settings
set option value      # Set option
show option           # Show option value
reset option          # Reset option to default

# Plugin operations
plugin install name   # Install plugin
plugin remove name    # Remove plugin
plugin list           # List plugins
plugin update         # Update plugins
```

Macros

```
# Record macro
Ctrl+U  # Start recording
# Perform actions
Ctrl+U  # Stop recording

# Play macro
Ctrl+J  # Execute recorded macro
```

Bookmarks

```
# Set bookmark
Ctrl+B  # Toggle bookmark on current line

# Navigate bookmarks
Alt+Up    # Previous bookmark
Alt+Down  # Next bookmark

# List bookmarks
Ctrl+E
# Type "bookmark list"
Enter
```

Splits and Tabs

```
# Split operations
Ctrl+E hsplit filename  # Horizontal split
Ctrl+E vsplit filename  # Vertical split
Ctrl+W                  # Cycle through splits
```

```
Ctrl+Q                  # Close current split

# Tab operations
Ctrl+T                  # New tab
Ctrl+W                  # Close tab
Ctrl+Tab                # Next tab
Ctrl+Shift+Tab          # Previous tab
```

Integration with Development Tools

Git Integration

```
# View git diff
micro -option diffgutter true filename.txt

# Git commands through shell
Ctrl+E
# Type "term git status"
Enter

# Or use external git tools
# Configure git to use micro as editor
git config --global core.editor micro
```

Language Server Integration

While micro doesn't have built-in LSP support, you can use external tools:

```
# Install language servers
npm install -g typescript-language-server
pip install python-lsp-server

# Use with external LSP client
# Or use plugins that provide LSP integration
micro -plugin install lsp
```

Build System Integration

```
# Run build commands
Ctrl+E
# Type "term make"
Enter
```

```
# Or create custom commands
# In settings.json, add:
{
    "build": "make",
    "test": "npm test",
    "run": "python main.py"
}

# Then use:
Ctrl+E
# Type "build" or "test" or "run"
Enter
```

Customization Examples

Custom Status Line

```
{
    "statusformatl": "$(filename) [$(opt:filetype)] $(modified) | Line $(line)/$(lines) Col
    "statusformatr": "$(opt:encoding) | $(opt:fileformat) | Micro v$(version)"
}
```

Custom Key Bindings for Productivity

```
{
    "Ctrl+Shift+P": "lua:comment.comment",
    "Ctrl+D": "DuplicateLine",
    "Ctrl+Shift+K": "DeleteLine",
    "Ctrl+L": "SelectLine",
    "Ctrl+Shift+L": "SelectToEndOfLine",
    "F2": "Rename",
    "F5": "Reload",
    "F12": "ToggleHelp",
    "Ctrl+`": "ToggleTerminal"
}
```

File Type Specific Settings

Create `~/.config/micro/settings.json`:

```
{
    "*.py": {
        "tabsize": 4,
        "tabstospaces": true,
        "rmtrailingws": true
    },
    "*.js": {
        "tabsize": 2,
        "tabstospaces": true
    },
    "*.go": {
        "tabsize": 4,
        "tabstospaces": false
    },
    "*.md": {
        "softwrap": true,
        "wordwrap": true
    }
}
```

Performance and Optimization

Large File Handling

```
# Open large files efficiently
micro -option syntax false largefile.txt

# Disable features for better performance
micro -option hlsearch false
micro -option matchbrace false
micro -option autoindent false
```

Memory Usage

```
# Check memory usage
ps aux | grep micro

# Optimize for low memory
micro -option backup false
micro -option savehistory false
micro -option saveundo false
```

Troubleshooting

Common Issues

```
# Terminal compatibility issues
export TERM=xterm-256color
micro filename.txt

# Mouse not working
micro -option mouse true filename.txt

# Colors not displaying correctly
micro -option colorscheme default filename.txt

# Plugin issues
micro -plugin remove problematic-plugin
micro -plugin update
```

Debug Mode

```
# Enable debug logging
micro -debug filename.txt

# Check log file
tail -f ~/.config/micro/log.txt
```

Reset Configuration

```
# Backup current config
mv ~/.config/micro ~/.config/micro.backup

# Start fresh (micro will create new config)
micro
```

Migration from Other Editors

From Nano

Micro provides similar simplicity with more features: - Familiar Ctrl+S, Ctrl+O shortcuts - Mouse support - Better syntax highlighting - Plugin system

From GUI Editors

Micro bridges the gap between terminal and GUI editors: - Familiar keybindings (Ctrl+C, Ctrl+V, etc.) - Mouse support - Multiple cursors - Modern interface

From Vim

For Vim users wanting something simpler: - No modal editing - Intuitive keybindings - Built-in help system - Less configuration required

Micro excels as a user-friendly terminal editor that doesn't sacrifice power for simplicity. Its familiar interface, modern features, and extensibility make it an excellent choice for developers who want a capable editor without a steep learning curve.

Kakoune

Kakoune: The Selection-Based Editor

Kakoune is a modal editor that emphasizes selection-based editing and multiple selections. Inspired by Vim but with a different philosophy, Kakoune provides powerful text manipulation capabilities through its unique approach to editing.

What is Kakoune?

Kakoune (kak) is built around several key principles: - **Selection-first editing**: Select text first, then act on it - **Multiple selections**: Edit multiple locations simultaneously - **Orthogonal design**: Commands compose naturally - **Client-server architecture**: Multiple clients can connect to one session - **Powerful text objects**: Rich set of selection primitives - **Minimal configuration**: Works well with minimal setup

Why Choose Kakoune?

- **Intuitive workflow**: Selection-first approach feels natural
- **Multiple selections**: Powerful multi-cursor editing
- **Composable commands**: Commands work together logically
- **Fast and lightweight**: Efficient C++ implementation optimized for Linux
- **Extensible**: Scriptable with shell commands and Linux utilities
- **Unique approach**: Different from Vim/Emacs paradigms
- **Active community**: Growing ecosystem of plugins and tools
- **Linux-native**: Designed with Linux development workflows in mind

Installation

Package Managers

```
# Ubuntu/Debian
sudo apt update
sudo apt install kakoune

# Fedora
sudo dnf install kakoune

# Arch Linux
```

```
sudo pacman -S kakoune

# SUSE/openSUSE
sudo zypper install kakoune

# Gentoo
sudo emerge app-editors/kakoune

# Void Linux
sudo xbps-install kakoune

# FreeBSD
pkg install kakoune

# OpenBSD
pkg_add kakoune
```

Building from Source

```
# Install dependencies
# Ubuntu/Debian
sudo apt install build-essential libncurses5-dev libncursesw5-dev

# Clone and build
git clone https://github.com/mawww/kakoune.git
cd kakoune
make
sudo make install

# Or install to custom location
make install PREFIX=$HOME/.local
```

Development Version

```
# For latest features
git clone https://github.com/mawww/kakoune.git
cd kakoune
git checkout master
make
sudo make install
```

Basic Concepts

Selection-First Philosophy

Unlike Vim's action-first approach, Kakoune uses selection-first:

```
# Vim way: action + motion
dw    # Delete word (action first)

# Kakoune way: selection + action
w     # Select word
d     # Delete selection
```

Multiple Selections

Kakoune excels at multiple selections:

```
# Select all occurrences
%     # Select all matches of current selection
Alt+% # Select all matches in buffer

# Split selections
s     # Split selection on regex
Alt+s # Split selection on lines
```

Modes

Kakoune has several modes: - **Normal mode**: Default mode for navigation and commands - **Insert mode**: For typing text - **Prompt mode**: For entering commands and searches - **Menu mode**: For selecting from options

Essential Commands

Starting Kakoune

```
# Open file
kak filename.txt

# Open multiple files
kak file1.txt file2.txt

# Start server session
kak -s session_name filename.txt
```

```
# Connect to existing session
kak -c session_name

# List sessions
kak -l

# Kill session
kak -c session_name -e 'kill'
```

Basic Navigation

```
# Character movement
h     # Left
j     # Down
k     # Up
l     # Right

# Word movement
w     # Next word start
b     # Previous word start
e     # Next word end
Alt+w # Next WORD start
Alt+b # Previous WORD start
Alt+e # Next WORD end

# Line movement
0     # Line start
^     # First non-blank
$     # Line end
Alt+h # Line start (alternative)
Alt+l # Line end (alternative)

# Document movement
g g   # Document start
g e   # Document end
g k   # Buffer top
g j   # Buffer bottom

# Page movement
Ctrl+b # Page up
Ctrl+f # Page down
Ctrl+u # Half page up
Ctrl+d # Half page down
```

Text Selection

```
# Basic selection
x     # Select line
Alt+x # Extend selection to line
s     # Select regex matches
Alt+s # Split selection on lines

# Word selection
w     # Select word
W     # Select WORD
Alt+w # Extend to word
Alt+W # Extend to WORD

# Character selection
f<char> # Select to character
t<char> # Select until character
F<char> # Select to character (backward)
T<char> # Select until character (backward)

# Extend selection
Shift+<motion> # Extend selection with motion
;     # Reduce selection to cursor
Alt+; # Flip selection direction
```

Multiple Selections

```
# Create multiple selections
C     # Copy selection to new line
Alt+C # Copy selection to previous line
%     # Select all matches of current selection
Alt+% # Select all matches in buffer

# Manipulate selections
Alt+k # Keep selections matching regex
Alt+K # Keep selections not matching regex
$     # Pipe selections through command
|     # Pipe and replace selections
!     # Insert command output

# Selection iteration
)     # Rotate main selection forward
(     # Rotate main selection backward
Alt+) # Rotate selections forward
Alt+( # Rotate selections backward
```

Editing Commands

```
# Insert modes
i     # Insert before selection
a     # Insert after selection
I     # Insert at line start
A     # Insert at line end
o     # Insert new line below
O     # Insert new line above

# Delete/Change
d     # Delete selection
Alt+d # Delete to end of line
c     # Change selection (delete and insert)
Alt+c # Change to end of line

# Copy/Paste
y     # Yank (copy) selection
p     # Paste after selection
P     # Paste before selection
Alt+p # Paste all selections
R     # Replace selection with yanked text

# Case manipulation
~     # Switch case
`     # Convert to lowercase
Alt+` # Convert to uppercase

# Indentation
>     # Indent
<     # Unindent
Alt+> # Indent including empty lines
Alt+< # Unindent including empty lines
```

Search and Replace

```
# Search
/     # Search forward
?     # Search backward
n     # Next search result
N     # Previous search result (Alt+n)
*     # Search word under cursor

# Replace
r<char> # Replace character
```

```
s     # Select matches for replacement
c     # Change selection
|     # Pipe through sed for replacement

# Global replace example
%     # Select all
s pattern # Select all pattern matches
c replacement<Esc> # Replace all
```

File and Buffer Management

Buffer Operations

```
# Buffer navigation
g a   # Go to alternate buffer
g f   # Go to file under cursor
g F   # Go to file under cursor (new buffer)

# Buffer commands (in command mode)
:buffer <name>    # Switch to buffer
:buffer-next      # Next buffer
:buffer-previous  # Previous buffer
:delete-buffer    # Close current buffer
:write           # Save buffer
:write-all       # Save all buffers
:quit            # Quit
:quit!           # Quit without saving
:write-quit      # Save and quit
```

File Operations

```
# File commands
:edit <file>     # Open file
:new             # New buffer
:write <file>    # Save as file
:cd <dir>        # Change directory
:pwd             # Show current directory
```

Window Management

```
# Window splitting (requires tmux or similar)
# Kakoune doesn't have built-in splits
# Use tmux or terminal multiplexer

# Multiple clients
kak -s main file.txt    # Start session
kak -c main            # Connect new client
```

Configuration

Configuration File

Kakoune configuration is stored in `~/.config/kak/kakrc`:

```
# ~/.config/kak/kakrc

# Basic settings
set-option global tabstop 4
set-option global indentwidth 4
set-option global scrolloff 5
set-option global ui_options ncurses_status_on_top=true
set-option global ui_options ncurses_assistant=cat

# Enable line numbers
add-highlighter global/ number-lines -relative -hlcursor

# Enable matching bracket highlighting
add-highlighter global/ show-matching

# Enable whitespace highlighting
add-highlighter global/ show-whitespaces -tab "→" -tabpad "·" -lf "¬" -spc "·"

# Color scheme
colorscheme default

# Key mappings
map global normal <c-p> ': fzf-mode<ret>'
map global normal '#' ': comment-line<ret>'
map global normal '<a-#>' ': comment-block<ret>'

# User mode for custom commands
declare-user-mode user
map global normal <space> ': enter-user-mode user<ret>'
```

```
map global user f ': fzf-mode<ret>'
map global user g ': grep '
map global user r ': replace-mode<ret>'

# Auto-pairs
hook global InsertChar \( %{ exec '<a-;>)<left>' }
hook global InsertChar \[ %{ exec '<a-;>]<left>' }
hook global InsertChar \{ %{ exec '<a-;>}<left>' }
hook global InsertChar \" %{ exec '<a-;>"<left>' }
hook global InsertChar \' %{ exec '<a-;>''<left>' }

# Auto-indent
hook global InsertChar \n %{ exec -draft \; K <a-&> }
hook global InsertChar \} %{ exec -draft <a-;> <a-S> 1<a-&> }

# Highlight TODO/FIXME
add-highlighter global/ regex \b(TODO|FIXME|XXX|NOTE)\b 0:default+rb

# File type specific settings
hook global BufSetOption filetype=python %{
    set-option buffer indentwidth 4
    set-option buffer tabstop 4
}

hook global BufSetOption filetype=javascript %{
    set-option buffer indentwidth 2
    set-option buffer tabstop 2
}

hook global BufSetOption filetype=html %{
    set-option buffer indentwidth 2
    set-option buffer tabstop 2
}

# Clipboard integration
map global user y '<a-|>xclip -i -selection clipboard<ret>'
map global user p '!xclip -o -selection clipboard<ret>'

# Save and quit shortcuts
map global user w ': write<ret>'
map global user q ': quit<ret>'
map global user x ': write-quit<ret>'
```

Plugin Management

Kakoune doesn't have a built-in plugin manager, but you can use external tools:

```
# Install plug.kak (plugin manager)
mkdir -p ~/.config/kak/plugins
git clone https://github.com/andreyorst/plug.kak.git ~/.config/kak/plugins/plug.kak

# Add to kakrc
source "%val{config}/plugins/plug.kak/rc/plug.kak"
plug "andreyorst/plug.kak" noload

# Install plugins
plug "andreyorst/fzf.kak"
plug "andreyorst/smarttab.kak"
plug "ul/kak-lsp"
plug "alexherbo2/auto-pairs.kak"
plug "alexherbo2/surround.kak"
plug "delapouite/kakoune-buffers"
plug "occivink/kakoune-snippets"
```

Popular Plugins

```
# Essential plugins configuration

# FZF integration
plug "andreyorst/fzf.kak" config %{
    map global normal <c-p> ': fzf-mode<ret>'
    map global normal <c-o> ': fzf-file<ret>'
}

# LSP support
plug "ul/kak-lsp" do %{
    cargo install --locked --force --path .
} config %{
    set global lsp_diagnostic_line_error_sign ' '
    set global lsp_diagnostic_line_warning_sign ' '

    define-command ne -docstring 'go to next error/warning from lsp' %{
        lsp-find-error --include-warnings
    }
    define-command pe -docstring 'go to previous error/warning from lsp' %{
        lsp-find-error --previous --include-warnings
    }
    define-command ee -docstring 'go to current error/warning from lsp' %{
        lsp-find-error --include-warnings --previous
        lsp-find-error --include-warnings
    }
```

```
    map global user l %{: enter-user-mode lsp<ret>} -docstring "LSP mode"
    map global insert <tab> '<a-;>: try lsp-snippets-select-next-placeholders catch %{ execu
    map global object a '<a-semicolon>lsp-object<ret>' -docstring 'LSP any symbol'
    map global object <a-a> '<a-semicolon>lsp-object<ret>' -docstring 'LSP any symbol'
    map global object f '<a-semicolon>lsp-object Function Method<ret>' -docstring 'LSP funct
    map global object t '<a-semicolon>lsp-object Class Interface Struct<ret>' -docstring 'LS

    hook global WinSetOption filetype=(c|cpp|cc|rust|javascript|typescript|python) %{
        lsp-enable-window
    }
}

# Surround text objects
plug "alexherbo2/surround.kak" config %{
    map global user s ': surround<ret>' -docstring 'surround'
    map global user S ': surround _ _ * *<ret>' -docstring 'surround with *'
}

# Auto-pairs
plug "alexherbo2/auto-pairs.kak" config %{
    enable-auto-pairs
}

# Snippets
plug "occivink/kakoune-snippets" config %{
    set-option global snippets_directories "%opt{plug_install_dir}/kakoune-snippets/snippets
}

# Buffer management
plug "delapouite/kakoune-buffers" config %{
    map global user b ': enter-user-mode buffers<ret>' -docstring 'buffers'
    map global user B ': enter-user-mode -lock buffers<ret>' -docstring 'buffers (lock)'
}
```

Real-World Usage Scenarios

Scenario 1: Refactoring Variable Names

```
# Open file
kak script.py

# Select variable name
w     # Select word

# Select all occurrences
```

```
%     # Select all matches

# Change all occurrences
c     # Change selection
new_variable_name<Esc>

# Save file
:w
```

Scenario 2: Multiple Line Editing

```
# Select multiple lines
x     # Select line
j     # Move down
x     # Extend to next line
j     # Move down
x     # Extend to next line

# Or select lines with pattern
%     # Select all
s^def # Select lines starting with "def"

# Add prefix to all selected lines
I     # Insert at beginning
# Type prefix
<Esc>
```

Scenario 3: Complex Text Manipulation

```
# Select paragraph
<a-a>p # Select around paragraph

# Split into sentences
s\. # Split on periods

# Process each sentence
|capitalize_first_word.sh # Pipe through script

# Or use built-in commands
~     # Switch case of first character
```

Scenario 4: Log File Analysis

```
# Open log file
kak /var/log/application.log

# Select all error lines
%     # Select all
s^.*ERROR.*$ # Select lines containing ERROR

# Copy to new buffer
y     # Yank
:new  # New buffer
p     # Paste

# Analyze timestamps
s\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2} # Select timestamps
```

Advanced Features

Hooks and Automation

```
# Auto-save on focus lost
hook global FocusOut .* %{ write }

# Auto-format on save
hook global BufWritePre .*\.py %{ format }

# Highlight current word
hook global NormalIdle .* %{
    eval -draft %{ try %{
        exec <space><a-i>w <a-k>\A\w+\z<ret>
        add-highlighter window/word regex "\b\Q%val{selection}\E\b" 0:+u
    } catch %{
        rmhl window/word
    } }
}

# Auto-reload files changed externally
hook global FocusIn .* %{ edit! }
```

Custom Commands

```
# Define custom commands
define-command -docstring "duplicate line" duplicate-line %{
    exec 'x<a-s>y<a-o>p'
}

define-command -docstring "sort selections" sort-selections %{
    exec '|sort<ret>'
}

define-command -docstring "remove duplicates" remove-duplicates %{
    exec '|sort -u<ret>'
}

define-command -docstring "count words" count-words %{
    exec '<a-s>|wc -w<ret>'
}

# Map to keys
map global user d ': duplicate-line<ret>'
map global user s ': sort-selections<ret>'
map global user u ': remove-duplicates<ret>'
map global user c ': count-words<ret>'
```

Text Objects

```
# Built-in text objects
<a-i>w # Inner word
<a-a>w # Around word
<a-i>s # Inner sentence
<a-a>s # Around sentence
<a-i>p # Inner paragraph
<a-a>p # Around paragraph
<a-i>( # Inner parentheses
<a-a>( # Around parentheses
<a-i>{ # Inner braces
<a-a>{ # Around braces
<a-i>[ # Inner brackets
<a-a>[ # Around brackets
<a-i>" # Inner quotes
<a-a>" # Around quotes

# Custom text objects
define-command -docstring "select function" select-function %{
```

```
    exec '<a-s>s^def\s+\w+.*?(?=^def|\z)<ret>'
}

map global object f ': select-function<ret>'
```

Macros and Repetition

```
# Record macro
q<register> # Start recording to register
# ... perform actions ...
q           # Stop recording

# Play macro
@<register> # Execute macro from register
@@          # Repeat last macro

# Repeat last command
.           # Repeat last normal mode command
<a-.>       # Repeat last insert mode command
```

Language-Specific Configurations

Python Development

```
# Python-specific kakrc additions
hook global BufSetOption filetype=python %{
    set-option buffer indentwidth 4
    set-option buffer tabstop 4

    # Python-specific mappings
    map buffer user r ': python-run<ret>'
    map buffer user t ': python-test<ret>'
    map buffer user f ': python-format<ret>'

    # Auto-format with black
    hook buffer BufWritePre .* %{ exec '|black -<ret>' }

    # Highlight Python keywords
    add-highlighter buffer/python-keywords regex \b(def|class|import|from|if|elif|else|for|w
}

define-command python-run %{
    exec '|python3<ret>'
```

```
}

define-command python-test %{
    exec '!python3 -m pytest<ret>'
}

define-command python-format %{
    exec '%|black -<ret>'
}
```

Web Development

```
# HTML/CSS/JavaScript configuration
hook global BufSetOption filetype=(html|css|javascript) %{
    set-option buffer indentwidth 2
    set-option buffer tabstop 2

    # Auto-close tags for HTML
    hook buffer InsertChar > %{
        exec -draft '<a-;>h<a-k><[^/][^>]*<ret>a</<c-r>"><left>'
    }
}

# JavaScript-specific
hook global BufSetOption filetype=javascript %{
    map buffer user r ': javascript-run<ret>'
    map buffer user f ': javascript-format<ret>'

    # Format with prettier
    hook buffer BufWritePre .* %{ exec '%|prettier --stdin-filepath %val{buffile}<ret>' }
}

define-command javascript-run %{
    exec '|node<ret>'
}

define-command javascript-format %{
    exec '%|prettier --stdin-filepath %val{buffile}<ret>'
}
```

Go Development

```
# Go-specific configuration
hook global BufSetOption filetype=go %{
    set-option buffer indentwidth 4
    set-option buffer tabstop 4

    map buffer user r ': go-run<ret>'
    map buffer user b ': go-build<ret>'
    map buffer user t ': go-test<ret>'
    map buffer user f ': go-format<ret>'

    # Auto-format with gofmt
    hook buffer BufWritePre .* %{ exec '%|gofmt<ret>' }
}

define-command go-run %{
    exec '!go run %val{buffile}<ret>'
}

define-command go-build %{
    exec '!go build<ret>'
}

define-command go-test %{
    exec '!go test<ret>'
}

define-command go-format %{
    exec '%|gofmt<ret>'
}
```

Integration with External Tools

Git Integration

```
# Git commands
define-command git-status %{
    exec '!git status<ret>'
}

define-command git-add %{
    exec '!git add %val{buffile}<ret>'
}
```

```
define-command git-commit %{
    exec '!git commit<ret>'
}

define-command git-diff %{
    exec '!git diff %val{buffile}<ret>'
}

# Map to user mode
map global user g ': enter-user-mode git<ret>' -docstring 'git'
declare-user-mode git
map global git s ': git-status<ret>' -docstring 'status'
map global git a ': git-add<ret>' -docstring 'add'
map global git c ': git-commit<ret>' -docstring 'commit'
map global git d ': git-diff<ret>' -docstring 'diff'
```

Build System Integration

```
# Build system commands
define-command make %{
    exec '!make<ret>'
}

define-command make-clean %{
    exec '!make clean<ret>'
}

define-command make-install %{
    exec '!make install<ret>'
}

# Map to user mode
map global user m ': enter-user-mode make<ret>' -docstring 'make'
declare-user-mode make
map global make m ': make<ret>' -docstring 'make'
map global make c ': make-clean<ret>' -docstring 'clean'
map global make i ': make-install<ret>' -docstring 'install'
```

Terminal Integration

```
# Terminal commands
define-command terminal %{
    exec '!$SHELL<ret>'
}

define-command terminal-here %{
    exec '!cd %sh{dirname "$kak_buffile"} && $SHELL<ret>'
}

map global user t ': terminal<ret>' -docstring 'terminal'
map global user T ': terminal-here<ret>' -docstring 'terminal here'
```

Performance and Optimization

Large File Handling

```
# Optimize for large files
hook global BufCreate .*large.* %{
    set-option buffer readonly true
    rmhl buffer/number-lines
    rmhl buffer/show-matching
}

# Disable expensive features for large files
hook global BufOpenFile .* %{
    eval %sh{
        if [ $(wc -l < "$kak_hook_param") -gt 10000 ]; then
            echo "
                rmhl buffer/number-lines
                rmhl buffer/show-matching
                rmhl buffer/show-whitespaces
            "
        fi
    }
}
```

Memory Optimization

```
# Limit history size
set-option global history_size 100

# Disable undo for large operations
define-command no-undo %{
    set-option buffer history_size 0
}

map global user n ': no-undo<ret>' -docstring 'disable undo'
```

Troubleshooting

Common Issues

```
# Debug mode
kak -d

# Check configuration
:debug info

# Reset to defaults
kak -n  # No user config

# Check key mappings
:map

# Check options
:set-option
```

Session Management

```
# List sessions
kak -l

# Kill session
kak -c session_name -e 'kill'

# Kill all sessions
for session in $(kak -l); do
    kak -c "$session" -e 'kill'
done
```

Configuration Issues

```
# Test configuration
kak -f /dev/null  # Empty config

# Check for errors
kak -e 'echo %val{client_env_TERM}'

# Backup and reset config
mv ~/.config/kak ~/.config/kak.backup
```

Kakoune offers a unique and powerful approach to text editing through its selection-first philosophy and multiple selections. While it has a learning curve, its orthogonal design and composable commands make it highly effective for complex text manipulation tasks.

Modern Tools

Modern Tools

Modern Linux Development Tools

This chapter is the **companion CLI layer** around editors and workspaces: better search, navigation, diffs, file managers, and prompts. Pair it with a daily editor (Neovim, Helix, Micro, …) and a session tool (tmux, Herdr, or Zellij below).

For end-to-end recipes, see Workflows.

Why Modern Tools on Linux?

- **Native Performance**: Tools written in Rust and Go provide excellent performance on Linux
- **System Integration**: Deep integration with Linux package managers and system services
- **Development Focus**: Designed with Linux development workflows in mind
- **Resource Efficiency**: Optimized for Linux server environments and containers
- **Open Source**: Most tools are open source and align with Linux philosophy

Suggested “editor kit” install

```
# Debian/Ubuntu names vary slightly (fd-find, batcat)
sudo apt install -y ripgrep fd-find fzf bat git curl
# Optional polish:
# cargo install eza zoxide git-delta starship just
# cargo install --locked lazygit   # or use distro package
```

Wire defaults:

```
export EDITOR=nvim          # or hx, micro, "zed --wait"
export FZF_DEFAULT_COMMAND='fd --type f --hidden --follow --exclude .git'
export MANPAGER="sh -c 'col -bx | bat -l man -p'"
```

Terminal Multiplexers (map)

Tool	Role in this book
tmux	Full chapter: tmux — portable SSH sessions
Herdr	Full chapter: Herdr — AI agent workspaces
Zellij	Covered below — friendly layouts and defaults

Tool	Role in this book
GNU screen	Brief notes below — legacy availability

GNU screen (legacy multiplexer)

Still appears on minimal systems when tmux is missing.

```
sudo apt install -y screen   # or pacman/dnf equivalent
screen -S work               # named session
# Ctrl-a d                   # detach (prefix is Ctrl-a)
screen -r work               # reattach
screen -ls
```

Prefer learning **tmux** for new muscle memory; use screen when it is all you have.

Terminal-Based Development Tools

Zellij: Modern Terminal Multiplexer

Zellij is a modern alternative to tmux with better defaults and user experience. For agent-heavy workflows prefer Herdr; for maximum portability prefer tmux.

Installation

```
# Cargo
cargo install zellij

# Arch Linux
sudo pacman -S zellij

# Ubuntu/Debian (from GitHub releases)
wget https://github.com/zellij-org/zellij/releases/latest/download/zellij-x86_64-unknown-lin
tar -xzf zellij-x86_64-unknown-linux-musl.tar.gz
sudo mv zellij /usr/local/bin/

# SUSE/openSUSE
sudo zypper install zellij

# Gentoo
sudo emerge app-misc/zellij
```

Basic Usage

```
# Start zellij
zellij

# Key bindings (default)
Ctrl+p n    # New pane
Ctrl+p x    # Close pane
Ctrl+p h/j/k/l  # Navigate panes
Ctrl+p r    # Resize mode
Ctrl+p t    # New tab
Ctrl+p q    # Quit

# Sessions
zellij -s session_name    # Start named session
zellij ls                 # List sessions
zellij a session_name     # Attach to session
```

Configuration

```
// ~/.config/zellij/config.kdl
keybinds {
    normal {
        bind "Alt h" { MoveFocus "Left"; }
        bind "Alt l" { MoveFocus "Right"; }
        bind "Alt j" { MoveFocus "Down"; }
        bind "Alt k" { MoveFocus "Up"; }
        bind "Alt n" { NewPane; }
        bind "Alt x" { CloseFocus; }
        bind "Alt t" { NewTab; }
        bind "Alt 1" { GoToTab 1; }
        bind "Alt 2" { GoToTab 2; }
        bind "Alt 3" { GoToTab 3; }
    }
}

theme "catppuccin-mocha"
default_shell "zsh"
pane_frames false
```

Starship: Cross-Shell Prompt

Starship provides a fast, customizable prompt for any shell.

Installation

```
# Install script
curl -sS https://starship.rs/install.sh | sh

# Cargo
cargo install starship --locked

# Package managers
# Arch Linux
sudo pacman -S starship

# Ubuntu/Debian
sudo apt install starship

# SUSE/openSUSE
sudo zypper install starship

# Gentoo
sudo emerge app-shells/starship
```

Configuration

```
# Add to shell config (~/.bashrc, ~/.zshrc, etc.)
eval "$(starship init bash)"  # For bash
eval "$(starship init zsh)"   # For zsh
```

```
# ~/.config/starship.toml
format = """
[](#9A348E)\
$os\
$username\
[](bg:#DA627D fg:#9A348E)\
$directory\
[](fg:#DA627D bg:#FCA17D)\
$git_branch\
$git_status\
[](fg:#FCA17D bg:#86BBD8)\
$c\
$elixir\
$elm\
$golang\
$gradle\
$haskell\
```

```
$java\
$julia\
$nodejs\
$nim\
$rust\
$scala\
[](fg:#86BBD8 bg:#06969A)\
$docker_context\
[](fg:#06969A bg:#33658A)\
$time\
[ ](fg:#33658A)\
"""

[os]
disabled = false
style = "bg:#9A348E"

[username]
show_always = true
style_user = "bg:#9A348E"
style_root = "bg:#9A348E"
format = '[$user ]($style)'
disabled = false

[directory]
style = "bg:#DA627D"
format = "[ $path ]($style)"
truncation_length = 3
truncation_symbol = "…/"

[git_branch]
symbol = ""
style = "bg:#FCA17D"
format = '[ $symbol $branch ]($style)'

[git_status]
style = "bg:#FCA17D"
format = '[$all_status$ahead_behind ]($style)'

[nodejs]
symbol = ""
style = "bg:#86BBD8"
format = '[ $symbol ($version) ]($style)'

[rust]
symbol = ""
```

```
style = "bg:#86BBD8"
format = '[ $symbol ($version) ]($style)'

[golang]
symbol = ""
style = "bg:#86BBD8"
format = '[ $symbol ($version) ]($style)'

[python]
symbol = ""
style = "bg:#86BBD8"
format = '[ $symbol ($version) ]($style)'

[time]
disabled = false
time_format = "%R"
style = "bg:#33658A"
format = '[  $time ]($style)'
```

Zoxide: Smart Directory Navigation

Zoxide is a smarter cd command that learns your habits.

Installation

```
# Install script
curl -sS https://raw.githubusercontent.com/ajeetdsouza/zoxide/main/install.sh | bash

# Cargo
cargo install zoxide --locked

# Package managers
sudo pacman -S zoxide  # Arch
sudo zypper install zoxide  # SUSE/openSUSE
sudo emerge app-shells/zoxide  # Gentoo
```

Setup

```
# Add to shell config
eval "$(zoxide init bash)"  # For bash
eval "$(zoxide init zsh)"   # For zsh
```

Usage

```
# Navigate to directories
z foo       # cd to highest ranked directory matching foo
z foo bar   # cd to highest ranked directory matching foo and bar
zi foo      # cd with interactive selection

# Add directories
zoxide add /path/to/directory

# Query database
zoxide query foo
zoxide query --list
```

Eza: Modern ls Replacement

Eza is a modern replacement for ls with better defaults and features.

Installation

```
# Cargo
cargo install eza

# Arch Linux
sudo pacman -S eza

# Ubuntu/Debian
sudo apt install eza

# SUSE/openSUSE
sudo zypper install eza

# Gentoo
sudo emerge sys-apps/eza
```

Usage

```
# Basic usage
eza                    # List files
eza -l                 # Long format
eza -la                # Long format with hidden files
eza -T                 # Tree view
eza -T -L 2            # Tree view with depth limit
```

```
eza --git              # Show git status
eza --icons            # Show file type icons

# Useful aliases
alias ls='eza'
alias ll='eza -l'
alias la='eza -la'
alias lt='eza -T'
alias lg='eza -l --git'
```

Bat: Better Cat

Bat is a cat clone with syntax highlighting and Git integration.

Installation

```
# Package managers
sudo apt install bat        # Ubuntu/Debian
sudo pacman -S bat          # Arch Linux
sudo zypper install bat     # SUSE/openSUSE
sudo emerge sys-apps/bat    # Gentoo
cargo install bat           # Cargo
```

Usage

```
# Basic usage
bat file.txt               # View file with syntax highlighting
bat -n file.txt            # Show line numbers
bat -A file.txt            # Show all characters
bat --theme=GitHub file.txt # Use specific theme

# Useful aliases
alias cat='bat'
alias less='bat'

# Integration with other tools
export MANPAGER="sh -c 'col -bx | bat -l man -p'"
```

Ripgrep: Fast Text Search

Ripgrep (rg) is a fast grep alternative with better defaults.

Installation

```
# Package managers
sudo apt install ripgrep    # Ubuntu/Debian
sudo pacman -S ripgrep      # Arch Linux
sudo zypper install ripgrep # SUSE/openSUSE
sudo emerge sys-apps/ripgrep # Gentoo
cargo install ripgrep       # Cargo
```

Usage

```
# Basic search
rg "pattern"               # Search in current directory
rg "pattern" /path         # Search in specific path
rg -i "pattern"            # Case insensitive
rg -w "word"               # Whole word search
rg -v "pattern"            # Invert match

# File type filtering
rg -t py "pattern"         # Search only Python files
rg -T py "pattern"         # Exclude Python files
rg --type-list             # List available file types

# Advanced options
rg -n "pattern"            # Show line numbers
rg -C 3 "pattern"          # Show 3 lines of context
rg -A 3 -B 3 "pattern"     # Show 3 lines after and before
rg --hidden "pattern"      # Search hidden files
rg --no-ignore "pattern"   # Don't respect .gitignore

# Regular expressions
rg "\d{3}-\d{2}-\d{4}"     # Phone number pattern
rg "^function.*\{$"        # Function definitions
```

Fzf: Fuzzy Finder

fzf is the interactive filter that turns huge lists into something you can pick with two keystrokes. It is the glue between `fd`/`rg` and your editor.

Installation

```
sudo apt install fzf          # Debian/Ubuntu
sudo pacman -S fzf            # Arch
sudo dnf install fzf          # Fedora
sudo zypper install fzf       # openSUSE
cargo install fzf             # if needed
```

Shell keybindings (bash):

```
# Debian package often provides:
# /usr/share/doc/fzf/examples/key-bindings.bash
source /usr/share/doc/fzf/examples/key-bindings.bash 2>/dev/null || true
# Ctrl-R → fuzzy history, Ctrl-T → fuzzy files (when configured)
```

Editor integration patterns

```
# Open file in $EDITOR
export FZF_DEFAULT_COMMAND='fd --type f --hidden --follow --exclude .git'
export FZF_DEFAULT_OPTS='--height 40% --layout=reverse --border --preview "bat --color=alway

fe() {
  local file
  file=$(fzf) || return
  ${EDITOR:-nvim} "$file"
}

# Search file contents then open at line (requires rg)
feg() {
  local out file line
  out=$(rg -n --hidden --glob '!.git' "${1:-.}" 2>/dev/null | fzf) || return
  file=${out%%:*}
  line=$(printf '%s\n' "$out" | cut -d: -f2)
  ${EDITOR:-nvim} +"$line" "$file"
}
```

Inside Neovim/LazyVim, Telescope or fzf-lua often wraps the same ideas. Inside Helix, use Space-f pickers; still keep shell `fe` for any directory.

Lazygit: Terminal Git UI

lazygit is a keyboard-driven Git TUI. It pairs well with any editor: stage hunks here, press `e` (or your configured key) to open the file in `$EDITOR`.

```
# Install (one of)
sudo pacman -S lazygit
go install github.com/jesseduffield/lazygit@latest
# binary releases: https://github.com/jesseduffield/lazygit/releases
```

```
cd ~/src/app
lazygit
# Common: space=stage, c=commit, p=pull, P=push, / = search
```

LazyVim users often have `<leader>gg` bound to lazygit. In tmux/Herdr, give lazygit its own pane or window.

Yazi: Terminal File Manager (modern)

Yazi is a fast async terminal file manager (Rust) with previews—useful beside Helix/Neovim when you want browsing without leaving the TUI.

```
cargo install --locked yazi-fm yazi-cli
# or check distro packages / https://yazi-rs.github.io/
```

```
ya   # or yazi
# Open selection in $EDITOR; use as a project springboard
```

Alternatives already covered below: **nnn**, **broot**.

Fd: Better Find

Fd is a simple, fast alternative to find with better defaults.

Installation

```
# Package managers
sudo apt install fd-find   # Ubuntu/Debian (command is fdfind)
sudo pacman -S fd          # Arch Linux
sudo zypper install fd     # SUSE/openSUSE
sudo emerge sys-apps/fd    # Gentoo
cargo install fd-find      # Cargo
```

Usage

```
# Basic usage
fd pattern                  # Find files/directories matching pattern
fd -t f pattern             # Find only files
fd -t d pattern             # Find only directories
fd -e py                    # Find Python files
fd -E node_modules pattern # Exclude node_modules

# Advanced options
fd -H pattern               # Include hidden files
fd -I pattern               # Don't respect .gitignore
fd -x command               # Execute command on results
fd pattern -x ls -la        # Execute ls -la on each result

# Useful aliases
alias find='fd'
```

Delta: Better Git Diff

Delta provides syntax highlighting and better formatting for git diffs.

Installation

```
# Package managers
sudo apt install git-delta # Ubuntu/Debian
sudo pacman -S git-delta   # Arch Linux
sudo zypper install git-delta # SUSE/openSUSE
sudo emerge dev-util/git-delta # Gentoo
cargo install git-delta    # Cargo
```

Configuration

```
# Add to ~/.gitconfig
[core]
    pager = delta

[interactive]
    diffFilter = delta --color-only

[delta]
    navigate = true
    light = false
```

```
    side-by-side = true
    line-numbers = true
    syntax-theme = Dracula

[merge]
    conflictstyle = diff3

[diff]
    colorMoved = default
```

File Management Tools

Broot: Interactive Directory Navigation

Broot provides an interactive way to navigate and manipulate directory trees.

Installation

```
# Cargo
cargo install broot

# Package managers
sudo pacman -S broot       # Arch Linux
sudo zypper install broot  # SUSE/openSUSE
sudo emerge app-misc/broot # Gentoo

# Install shell integration
broot --install
```

Usage

```
# Start broot
br                         # Navigate current directory
br /path/to/directory      # Navigate specific directory

# Key bindings
Ctrl+h                     # Toggle hidden files
Ctrl+f                     # Search files
Ctrl+d                     # Delete file/directory
Ctrl+c                     # Copy path
Enter                      # Open file/enter directory
Alt+Enter                  # Open in external editor
```

Nnn: Terminal File Manager

Nnn is a fast, feature-rich terminal file manager.

Installation

```
# Package managers
sudo apt install nnn       # Ubuntu/Debian
sudo pacman -S nnn         # Arch Linux
sudo zypper install nnn    # SUSE/openSUSE
sudo emerge app-misc/nnn   # Gentoo

# From source
git clone https://github.com/jarun/nnn.git
cd nnn
make
sudo make install
```

Configuration

```
# Environment variables
export NNN_PLUG='f:finder;o:fzopen;p:mocplay;d:diffs;t:nmount;v:imgview'
export NNN_FIFO='/tmp/nnn.fifo'
export NNN_COLORS='2136'
export NNN_FCOLORS='c1e2272e006033f7c6d6abc4'

# Aliases
alias n='nnn -de'
alias N='sudo nnn -dH'
```

Usage

```
# Basic navigation
j/k or ↑/↓                # Navigate files
h/l or ←/→                # Navigate directories
Enter                     # Open file/enter directory
q                         # Quit
?                         # Help

# File operations
d                         # Delete
r                         # Rename
c                         # Copy
v                         # Move
```

```
s                         # Select files
a                         # Select all
A                         # Invert selection

# Advanced features
;                         # Plugin menu
!                         # Shell
e                         # Edit in $EDITOR
p                         # Show file info
```

Development Environment Tools

Direnv: Environment Management

Direnv automatically loads and unloads environment variables based on directory.

Installation

```
# Package managers
sudo apt install direnv    # Ubuntu/Debian
sudo pacman -S direnv      # Arch Linux
sudo zypper install direnv # SUSE/openSUSE
sudo emerge dev-util/direnv # Gentoo

# Add to shell config
eval "$(direnv hook bash)" # For bash
eval "$(direnv hook zsh)"  # For zsh
```

Usage

```
# Create .envrc file in project directory
echo 'export DATABASE_URL="postgres://localhost/mydb"' > .envrc
echo 'export DEBUG=true' >> .envrc

# Allow the .envrc file
direnv allow

# The environment variables are now loaded when you cd into the directory
# and unloaded when you leave
```

Advanced .envrc Examples

```
# Python virtual environment
layout python3
pip install -r requirements.txt

# Node.js version management
use node 18

# Go module
export GOPATH=$PWD/.go
export PATH=$GOPATH/bin:$PATH

# Docker development
export COMPOSE_FILE=docker-compose.dev.yml
export DOCKER_BUILDKIT=1

# Load from .env file
dotenv

# Path manipulation
PATH_add bin
PATH_add scripts
```

Mise: Runtime Version Manager

Mise (formerly rtx) manages multiple runtime versions for different languages.

Installation

```
# Install script
curl https://mise.run | sh

# Cargo
cargo install mise

# Package managers
sudo pacman -S mise        # Arch Linux
sudo zypper install mise   # SUSE/openSUSE
# For Gentoo, install via cargo or from source
```

Setup

```
# Add to shell config
echo 'eval "$(mise activate bash)"' >> ~/.bashrc  # For bash
echo 'eval "$(mise activate zsh)"' >> ~/.zshrc    # For zsh
```

Usage

```
# Install runtimes
mise install node@18       # Install Node.js 18
mise install python@3.11   # Install Python 3.11
mise install go@1.26       # Install a current stable Go (or: mise install go@latest)

# Set global versions
mise global node@18
mise global python@3.11

# Set local versions (creates .mise.toml)
mise local node@16
mise local python@3.9

# List available versions
mise list-remote node
mise list-remote python

# List installed versions
mise list
```

Configuration

```
# .mise.toml
[tools]
node = "18"
python = "3.11"
go = "1.21"

[env]
NODE_ENV = "development"
DEBUG = "true"

[tasks.dev]
run = "npm run dev"
description = "Start development server"
```

```toml
[tasks.test]
run = "npm test"
description = "Run tests"
```

Just: Command Runner

Just is a command runner similar to make but with better syntax.

Installation

```bash
# Cargo
cargo install just

# Package managers
sudo pacman -S just        # Arch Linux
sudo zypper install just   # SUSE/openSUSE
sudo emerge sys-devel/just # Gentoo

# Install script
curl --proto '=https' --tlsv1.2 -sSf https://just.systems/install.sh | bash -s -- --to ~/bin
```

Usage

Create a `justfile`:

```
# Default recipe
default:
    @just --list

# Development server
dev:
    npm run dev

# Run tests
test:
    npm test

# Build project
build:
    npm run build

# Deploy to production
deploy: build
    rsync -av dist/ user@server:/var/www/
```

```
# Clean build artifacts
clean:
    rm -rf dist/
    rm -rf node_modules/

# Install dependencies
install:
    npm install

# Format code
fmt:
    prettier --write .
    eslint --fix .

# Docker commands
docker-build:
    docker build -t myapp .

docker-run:
    docker run -p 3000:3000 myapp

# Database commands
db-migrate:
    npx prisma migrate dev

db-seed:
    npx prisma db seed

# Variables
port := "3000"
env := "development"

# Recipe with parameters
serve port=port:
    NODE_ENV={{env}} npm start -- --port {{port}}

# Conditional recipes
backup:
    #!/usr/bin/env bash
    if [ -f "database.db" ]; then
        cp database.db "database.db.backup.$(date +%Y%m%d_%H%M%S)"
        echo "Database backed up"
    else
        echo "No database file found"
    fi
```

```
# Run recipes
just                        # Run default recipe
just dev                    # Run dev recipe
just serve 8080             # Run serve with custom port
just --list                 # List all recipes
```

Text Processing Tools

Jq: JSON Processor

Jq is a lightweight command-line JSON processor.

Installation

```
# Package managers
sudo apt install jq       # Ubuntu/Debian
sudo pacman -S jq         # Arch Linux
sudo zypper install jq    # SUSE/openSUSE
sudo emerge app-misc/jq   # Gentoo
```

Usage

```
# Basic usage
echo '{"name": "John", "age": 30}' | jq '.'
echo '{"name": "John", "age": 30}' | jq '.name'
echo '[{"name": "John"}, {"name": "Jane"}]' | jq '.[0].name'

# Filter arrays
echo '[1,2,3,4,5]' | jq '.[] | select(. > 3)'

# Transform data
echo '{"users": [{"name": "John", "age": 30}, {"name": "Jane", "age": 25}]}' | \
  jq '.users | map({name, adult: (.age >= 18)})'

# Real-world examples
curl -s https://api.github.com/users/octocat | jq '.name, .public_repos'
docker inspect container_name | jq '.[0].NetworkSettings.IPAddress'
kubectl get pods -o json | jq '.items[].metadata.name'
```

Yq: YAML Processor

Yq is like jq but for YAML files.

Installation

```
# Go version (recommended)
go install github.com/mikefarah/yq/v4@latest

# Package managers
sudo apt install yq       # Ubuntu/Debian
sudo zypper install yq    # SUSE/openSUSE
sudo emerge app-misc/yq   # Gentoo
```

Usage

```
# Basic usage
yq '.name' config.yaml
yq '.database.host' config.yaml
yq '.services[0].name' docker-compose.yml

# Modify YAML
yq '.database.port = 5432' config.yaml
yq '.services[0].image = "nginx:latest"' docker-compose.yml

# Multiple files
yq eval-all '. as $item ireduce ({}; . * $item)' file1.yaml file2.yaml
```

Sd: Sed Alternative

Sd is a more intuitive alternative to sed for find and replace operations.

Installation

```
# Cargo
cargo install sd

# Package managers
sudo pacman -S sd         # Arch Linux
sudo zypper install sd    # SUSE/openSUSE
sudo emerge sys-apps/sd   # Gentoo
```

Usage

```
# Basic find and replace
sd 'old_text' 'new_text' file.txt
sd '\d{4}-\d{2}-\d{2}' 'DATE' file.txt

# In-place editing
sd -i 'old_text' 'new_text' file.txt

# Preview changes
sd 'old_text' 'new_text' file.txt --preview

# Multiple files
sd 'old_text' 'new_text' *.txt
```

Monitoring and System Tools

Htop: Process Viewer

Htop is an interactive process viewer and system monitor.

Installation

```
# Package managers
sudo apt install htop     # Ubuntu/Debian
sudo pacman -S htop       # Arch Linux
sudo zypper install htop  # SUSE/openSUSE
sudo emerge sys-process/htop # Gentoo
```

Usage

```
# Start htop
htop

# Key bindings
F1                        # Help
F2                        # Setup
F3                        # Search
F4                        # Filter
F5                        # Tree view
F6                        # Sort by
F9                        # Kill process
```

```
F10                         # Quit
```

Bottom: System Monitor

Bottom (btm) is a cross-platform system monitor written in Rust.

Installation

```
# Cargo
cargo install bottom

# Package managers
sudo pacman -S bottom     # Arch Linux
sudo zypper install bottom # SUSE/openSUSE
sudo emerge sys-process/bottom # Gentoo

# Debian/Ubuntu
curl -LO https://github.com/ClementTsang/bottom/releases/download/0.9.6/bottom_0.9.6_amd64.d
sudo dpkg -i bottom_0.9.6_amd64.deb
```

Usage

```
# Start bottom
btm

# Key bindings
?                           # Help
q                           # Quit
/                           # Search
Tab                         # Next widget
Shift+Tab                   # Previous widget
+/-                         # Zoom in/out
```

Procs: Process Information

Procs is a modern replacement for ps with better output formatting.

Installation

```
# Cargo
cargo install procs

# Package managers
sudo pacman -S procs      # Arch Linux
sudo zypper install procs # SUSE/openSUSE
sudo emerge sys-process/procs # Gentoo
```

Usage

```
# Basic usage
procs                     # List all processes
procs firefox             # Find processes matching "firefox"
procs --tree              # Tree view
procs --watch             # Watch mode
procs --sortd cpu         # Sort by CPU usage (descending)
```

Linux-Specific Integration

Systemd Integration

Many modern tools can be integrated with systemd for better Linux system management:

```
# Create user service for auto-updating tools
mkdir -p ~/.config/systemd/user

# Auto-update mise tools
cat > ~/.config/systemd/user/mise-update.service << 'EOF'
[Unit]
Description=Update mise tools
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/mise upgrade
User=%i
EOF

cat > ~/.config/systemd/user/mise-update.timer << 'EOF'
[Unit]
Description=Update mise tools daily
```

```
[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
EOF

# Enable the timer
systemctl --user enable mise-update.timer
systemctl --user start mise-update.timer
```

Linux Distribution Package Management

```
# Create a script to install modern tools across different distros
#!/bin/bash
# install-modern-tools.sh

detect_distro() {
    if [ -f /etc/os-release ]; then
        . /etc/os-release
        echo $ID
    elif [ -f /etc/redhat-release ]; then
        echo "rhel"
    elif [ -f /etc/debian_version ]; then
        echo "debian"
    else
        echo "unknown"
    fi
}

install_tools() {
    local distro=$(detect_distro)

    case $distro in
        ubuntu|debian)
            sudo apt update
            sudo apt install -y ripgrep fd-find bat eza zoxide starship direnv
            ;;
        arch|manjaro)
            sudo pacman -S ripgrep fd bat eza zoxide starship direnv
            ;;
        fedora)
            sudo dnf install -y ripgrep fd-find bat eza zoxide starship direnv
            ;;
```

```
        opensuse*|suse)
            sudo zypper install -y ripgrep fd bat eza zoxide starship direnv
            ;;
        gentoo)
            sudo emerge sys-apps/ripgrep sys-apps/fd sys-apps/bat sys-apps/eza \
                        app-shells/zoxide app-shells/starship dev-util/direnv
            ;;
        *)
            echo "Unsupported distribution: $distro"
            echo "Please install tools manually using cargo or from source"
            ;;
    esac
}

install_tools
```

Linux Desktop Integration

```
# Create desktop entries for terminal-based tools
mkdir -p ~/.local/share/applications

# Helix desktop entry
cat > ~/.local/share/applications/helix.desktop << 'EOF'
[Desktop Entry]
Name=Helix
Comment=A post-modern modal text editor
Exec=gnome-terminal -- hx %F
Icon=text-editor
Terminal=false
Type=Application
Categories=Development;TextEditor;
MimeType=text/plain;text/x-chdr;text/x-csrc;text/x-c++hdr;text/x-c++src;text/x-java;text/x-d
EOF

# Micro desktop entry
cat > ~/.local/share/applications/micro.desktop << 'EOF'
[Desktop Entry]
Name=Micro
Comment=A modern and intuitive terminal-based text editor
Exec=gnome-terminal -- micro %F
Icon=text-editor
Terminal=false
Type=Application
Categories=Development;TextEditor;
MimeType=text/plain;text/x-chdr;text/x-csrc;text/x-c++hdr;text/x-c++src;text/x-java;text/x-d
```

```
EOF

# Update desktop database
update-desktop-database ~/.local/share/applications
```

Integration Examples

Complete Development Setup

```
# ~/.zshrc or ~/.bashrc

# Modern tools
eval "$(starship init zsh)"
eval "$(zoxide init zsh)"
eval "$(direnv hook zsh)"
eval "$(mise activate zsh)"

# Aliases
alias ls='eza'
alias ll='eza -l'
alias la='eza -la'
alias lt='eza -T'
alias cat='bat'
alias find='fd'
alias grep='rg'
alias ps='procs'
alias top='btm'

# Functions
function fzf-edit() {
    local file
    file=$(fd --type f | fzf --preview 'bat --color=always {}')
    if [[ -n $file ]]; then
        $EDITOR "$file"
    fi
}

function fzf-cd() {
    local dir
    dir=$(fd --type d | fzf --preview 'eza -T {} | head -50')
    if [[ -n $dir ]]; then
        cd "$dir"
    fi
}
```

```
# Key bindings
bindkey '^F' fzf-edit
bindkey '^G' fzf-cd
```

Project Template with Modern Tools

```
# project-setup.sh
#!/bin/bash

PROJECT_NAME=$1
if [[ -z $PROJECT_NAME ]]; then
    echo "Usage: $0 <project-name>"
    exit 1
fi

# Create project directory
mkdir "$PROJECT_NAME"
cd "$PROJECT_NAME"

# Initialize git
git init

# Create .envrc for direnv
cat > .envrc << 'EOF'
export PROJECT_NAME=$(basename $PWD)
export NODE_ENV=development
export DEBUG=true

# Add project bin to PATH
PATH_add bin
PATH_add scripts

# Load .env if it exists
dotenv_if_exists
EOF

# Create .mise.toml for runtime management
cat > .mise.toml << 'EOF'
[tools]
node = "18"
python = "3.11"

[env]
NODE_ENV = "development"
```

```
[tasks.dev]
run = "npm run dev"
description = "Start development server"

[tasks.test]
run = "npm test"
description = "Run tests"

[tasks.build]
run = "npm run build"
description = "Build project"
EOF

# Create justfile for task running
cat > justfile << 'EOF'
# List available recipes
default:
    @just --list

# Install dependencies
install:
    npm install

# Start development server
dev:
    npm run dev

# Run tests
test:
    npm test

# Build project
build:
    npm run build

# Format code
fmt:
    prettier --write .
    eslint --fix .

# Clean build artifacts
clean:
    rm -rf dist/
    rm -rf node_modules/
EOF
```

```
# Create .gitignore
cat > .gitignore << 'EOF'
node_modules/
dist/
.env
.DS_Store
*.log
EOF

# Allow direnv
direnv allow

echo "Project $PROJECT_NAME created successfully!"
echo "Run 'cd $PROJECT_NAME && just install' to get started"
```

Linux-Specific Troubleshooting

Permission Issues

```
# Fix common permission issues with modern tools
# Ensure tools are executable
chmod +x ~/.local/bin/*

# Fix cargo-installed tools permissions
chmod +x ~/.cargo/bin/*

# Add to PATH if needed
echo 'export PATH="$HOME/.local/bin:$HOME/.cargo/bin:$PATH"' >> ~/.bashrc
```

Package Manager Conflicts

```
# Handle conflicts between package managers
# Remove system package before installing via cargo
sudo apt remove ripgrep  # Remove system version
cargo install ripgrep    # Install latest via cargo

# Or use alternatives command on Debian/Ubuntu
sudo update-alternatives --install /usr/bin/rg rg /usr/bin/rg 10
sudo update-alternatives --install /usr/bin/rg rg ~/.cargo/bin/rg 20
```

Performance Optimization for Linux

```
# Optimize tools for Linux systems
# Set environment variables for better performance
export RIPGREP_CONFIG_PATH="$HOME/.ripgreprc"
export BAT_THEME="Dracula"
export FZF_DEFAULT_COMMAND='fd --type f --hidden --follow --exclude .git'

# Create ripgrep config for Linux development
cat > ~/.ripgreprc << 'EOF'
# Search hidden files but not .git
--hidden
--glob=!.git/*

# Follow symbolic links
--follow

# Smart case search
--smart-case

# Show line numbers
--line-number

# Limit search depth for performance
--max-depth=10
EOF
```

Container Integration

```
# Use modern tools in Docker containers
# Dockerfile example
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y \
    curl \
    git \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Install Rust for cargo-based tools
RUN curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
ENV PATH="/root/.cargo/bin:${PATH}"

# Install modern tools
RUN cargo install ripgrep fd-find bat eza zoxide starship
```

```
# Set up shell with modern tools
RUN echo 'eval "$(starship init bash)"' >> ~/.bashrc
RUN echo 'eval "$(zoxide init bash)"' >> ~/.bashrc
RUN echo 'alias ls=eza' >> ~/.bashrc
RUN echo 'alias cat=bat' >> ~/.bashrc
```

Putting it together with editors

Goal	Tooling
Jump to any file	`fd` + `fzf` + `$EDITOR`
Jump to content	`rg` + `fzf` + `$EDITOR +line`
Review changes	`delta` + `lazygit`
Long SSH session	tmux
Multiple AI agents	Herdr
Pretty shell context	`starship` + `zoxide` + `eza`
Full recipes	Workflows

These modern tools enhance the Linux development experience with better defaults and performance—especially when paired with a deliberate editor and workspace choice from the rest of this book.

Tmux

tmux

tmux: Terminal Multiplexer

tmux is the default long-session tool for Linux servers and SSH. It keeps shells, editors, and builds running after disconnects, and it splits one terminal into windows and panes you can script and reattach from anywhere.

If you learn only one multiplexer, learn this one.

Why tmux

- **Detach / reattach** — close SSH without killing jobs
- **Windows and panes** — layouts for editor + shell + logs
- **Scriptable** — create sessions from shell scripts and automation
- **Ubiquitous** — packages on virtually every distro
- **Pairs with any editor** — nvim, Helix, nano, Emacs `-nw`

Related tools: **GNU screen** (older), **Zellij** (friendlier defaults), **Herdr** (agent-aware). See the intro on Terminal Workspaces and the Herdr chapter.

Installation

```
# Debian / Ubuntu
sudo apt update && sudo apt install -y tmux

# Fedora
sudo dnf install -y tmux

# Arch
sudo pacman -S tmux

# openSUSE
sudo zypper install tmux

# Alpine
sudo apk add tmux

# Nix
nix profile install nixpkgs#tmux
# or in a flake/devShell: pkgs.tmux
```

```
# macOS (if you dual-boot workflows)
brew install tmux
```

Verify:

```
tmux -V
```

Mental model

```
tmux server (background)
    session "work"
         window 0: editor
              pane 0: nvim
              pane 1: shell
         window 1: logs
               pane 0: journalctl -f
```

Object	What it is
Server	Background process owning all sessions
Session	Named group you attach to (`work`, `prod`)
Window	Full-screen tab inside a session
Pane	Split rectangle inside a window

Quick start lab

```
# Create a named session
tmux new -s work

# Inside tmux - prefix is Ctrl-b by default
# Split vertical / horizontal
Ctrl-b %
Ctrl-b "

# Move between panes
Ctrl-b o
Ctrl-b ←/→/↑/↓

# New window, rename, switch
Ctrl-b c
Ctrl-b ,
Ctrl-b n / Ctrl-b p
Ctrl-b 0..9
```

```
# Detach (jobs keep running)
Ctrl-b d

# List and reattach
tmux ls
tmux attach -t work
```

Essential keybindings

Prefix is shown as `C-b` (hold Ctrl, press b, release, then press the command key).

Sessions

Keys	Action
`C-b d`	Detach
`C-b s`	Session picker
`C-b $`	Rename session

Windows

Keys	Action
`C-b c`	New window
`C-b ,`	Rename window
`C-b n` / `C-b p`	Next / previous
`C-b w`	Window list
`C-b &`	Kill window
`C-b f`	Find window by name

Panes

Keys	Action
`C-b %`	Split left/right
`C-b "`	Split top/bottom
`C-b o`	Next pane
`C-b ;`	Last pane
`C-b x`	Kill pane
`C-b z`	Zoom pane (toggle fullscreen)
`C-b {` / `C-b }`	Swap panes
`C-b Space`	Cycle layouts
`C-b C-o`	Rotate panes

Keys	Action
`C-b q`	Show pane numbers (then type number)

Copy mode (scrollback)

Keys	Action
`C-b [`	Enter copy mode
`q`	Quit copy mode
`C-b ]`	Paste tmux buffer

With vi mode (recommended):

```
set -g mode-keys vi
```

Then in copy mode: `v` begin selection, `y` copy (depending on version/config—see config section).

Command-line interface

You can drive tmux without keybindings—ideal for scripts and muscle memory via shell aliases.

```
# Sessions
tmux new -s api -d                  # create detached
tmux new -s api -c ~/src/api        # start in directory
tmux attach -t api
tmux attach -t api -d               # attach and detach others
tmux switch -t api
tmux rename-session -t api backend
tmux kill-session -t api
tmux kill-server                    # nuclear: all sessions

# Windows
tmux new-window -t api -n tests
tmux select-window -t api:tests
tmux kill-window -t api:tests

# Panes
tmux split-window -h -t api
tmux split-window -v -t api
tmux select-pane -t api:0.1
tmux send-keys -t api:0.1 'htop' Enter

# Info
tmux list-sessions
```

```
tmux list-windows -t api
tmux list-panes -t api
tmux display-message -p '#S #I #P'
```

Send keys and automation

```
# Start a dev layout detached
SESSION=dev
tmux has-session -t "$SESSION" 2>/dev/null || {
  tmux new-session -d -s "$SESSION" -n editor -c "$HOME/proj"
  tmux send-keys -t "$SESSION:editor" 'nvim' Enter
  tmux split-window -h -t "$SESSION:editor" -c "$HOME/proj"
  tmux send-keys -t "$SESSION:editor.1" 'git status' Enter
  tmux new-window -t "$SESSION" -n run -c "$HOME/proj"
  tmux send-keys -t "$SESSION:run" 'just dev' Enter
}
tmux attach -t "$SESSION"
```

Configuration (`~/.tmux.conf`)

Reload after edits:

```
tmux source-file ~/.tmux.conf
# or inside tmux: C-b : source-file ~/.tmux.conf
```

Sensible starter config

```
# ~/.tmux.conf - practical defaults

# Terminal & color
set -g default-terminal "tmux-256color"
set -ag terminal-overrides ",xterm-256color:RGB"
set -g history-limit 50000
set -g mouse on
set -g base-index 1
setw -g pane-base-index 1
set -g renumber-windows on
set -g set-clipboard on

# Timing
set -sg escape-time 10
set -g focus-events on
```

```
# Vi keys
set -g mode-keys vi
set -g status-keys vi

# Prefix: keep Ctrl-b or switch to Ctrl-a (uncomment)
# unbind C-b
# set -g prefix C-a
# bind C-a send-prefix

# Splits in current path
bind | split-window -h -c "#{pane_current_path}"
bind - split-window -v -c "#{pane_current_path}"
unbind '"'
unbind %

# Pane move (vim-like)
bind h select-pane -L
bind j select-pane -D
bind k select-pane -U
bind l select-pane -R

# Resize
bind -r H resize-pane -L 5
bind -r J resize-pane -D 5
bind -r K resize-pane -U 5
bind -r L resize-pane -R 5

# Reload config
bind r source-file ~/.tmux.conf \; display-message "reloaded"

# Copy mode (vi)
bind -T copy-mode-vi v send -X begin-selection
bind -T copy-mode-vi y send -X copy-selection-and-cancel

# Status
set -g status-interval 5
set -g status-left-length 40
set -g status-left "#[bold]#S #[fg=colour8]| "
set -g status-right "#{?client_prefix,PREFIX ,}%Y-%m-%d %H:%M"
setw -g window-status-current-format "#[bold]#I:#W*"
setw -g window-status-format "#I:#W"
```

Nested tmux (SSH jump hosts)

When you run tmux locally and on a remote:

```
# Local: prefix C-b
# Remote: send prefix with C-b C-b (default) or rebind remote to C-a
```

Or use different prefixes per machine and document them.

Nested editors and key conflicts

Symptom	Likely cause
`C-b` does nothing useful in Vim	You pressed tmux prefix; press again or wait
Esc feels slow in Vim	Raise `escape-time` was high; set to `10`
Colors wrong in nvim	`default-terminal` / `terminal-overrides`
Mouse selects but cannot paste	Copy-mode vs terminal clipboard; enable `set-clipboard` and OSC52-aware terminal

Neovim tip: avoid binding `C-b` without checking tmux.

Helix/Zed: fewer conflicts; still test zoom (`C-b z`) and copy mode.

SSH workflows

```
# Reattach or create
ssh host 'tmux new -A -s main'

# From local alias
alias tmain='ssh -t host tmux new -A -s main'
```

`new -A` attaches if the session exists, otherwise creates it.

Keepalive

```
# ~/.ssh/config
Host *
  ServerAliveInterval 30
  ServerAliveCountMax 4
```

tmux survives SSH death; keepalive reduces nuisance disconnects.

Plugins (optional)

TPM is the common plugin manager:

```
git clone https://github.com/tmux-plugins/tpm ~/.tmux/plugins/tpm
```

```
# end of ~/.tmux.conf
set -g @plugin 'tmux-plugins/tpm'
set -g @plugin 'tmux-plugins/tmux-sensible'
set -g @plugin 'tmux-plugins/tmux-resurrect'   # save/restore sessions
set -g @plugin 'tmux-plugins/tmux-continuum'  # auto-save
set -g @plugin 'tmux-plugins/tmux-yank'       # clipboard helpers

run '~/.tmux/plugins/tpm/tpm'
```

Install plugins: `C-b I` (capital i).

Resurrect restores windows after reboot *if you save*; it is not magic. Critical production jobs still need systemd or proper process supervision.

Pairing with editors

Neovim / Vim

```
tmux new -s code -c ~/src/app
# pane 0
nvim
# C-b %
# pane 1
cargo test -- --nocapture
```

Or use Neovim's built-in terminal (`:terminal`) and keep tmux for session persistence only—one full-window nvim is a valid style.

Helix

```
hx .
# use tmux panes for cargo/npm; Helix file picker for buffers
```

Emacs

```
emacs -nw
# or detach-friendly: emacsclient -nw after emacs --daemon
```

Nano / Micro

Fine inside panes; rely on tmux for multi-tasking rather than editor splits.

Comparison: tmux vs screen vs Zellij vs Herdr

Feature	tmux	screen	Zellij	Herdr
Portability	Excellent	Excellent	Good	Growing
Defaults UX	OK	Dated	Great	Mouse + agent UI
Scripting	Excellent	OK	Good	CLI + socket API
Agent awareness	No	No	No	**Yes**
Typical use	SSH, servers	Legacy	Dev laptops	AI agent herds

Use **tmux** as the baseline skill. Add **Herdr** when agents are the workload.

Troubleshooting

```
# Server alive?
tmux ls

# Stuck session
tmux kill-session -t name

# Reset server (destroys all)
tmux kill-server

# Config error on start
tmux -f /dev/null new -s clean

# 24-bit color test
tmux new 'curl -s https://raw.githubusercontent.com/JohnMorales/dotfiles/master/colors/24-bi
```

Permission or socket issues after reboot: stale sockets under `/tmp/tmux-$(id -u)/`—remove only when no server should run.

Cheatsheet card

```
C-b d     detach              C-b c     new window
C-b %     vsplit              C-b "     hsplit
C-b z     zoom                C-b x     kill pane
C-b [     copy mode           C-b ]     paste
```

```
C-b n/p   next/prev window    C-b s     sessions
tmux new -A -s name           attach or create
```

Practice checklist

- ☐ Create, detach, reattach a named session over SSH
- ☐ Build a 3-pane layout and zoom the editor
- ☐ Put vi keys in copy mode and yank a stack trace
- ☐ Write a 15-line script that recreates your dev session
- ☐ Survive an intentional SSH disconnect mid-`sleep 300`

Next: Herdr for agent-native workspaces, or Workflows for full recipes with Neovim/Helix/Zed.

Herdr

Herdr

Herdr: Agent-Native Terminal Workspaces

Herdr (pronounced like "herder") is a terminal workspace manager built for **AI coding agents** and long-lived terminal work. It owns real PTY panes, keeps them running in a background server, and tracks whether each agent is **working**, **blocked**, or **idle**—so you jump to the pane that needs you instead of hunting through tabs.

It is not a replacement for Neovim or Helix. It is the **session layer** those editors (and agent CLIs) live inside—similar in spirit to tmux, but agent-aware.

Official site: https://herdr.dev/ · Docs: https://herdr.dev/docs/

Why Herdr exists

Classic multiplexers (tmux, screen, Zellij) are process-agnostic: a pane is a pane. In 2026 many developers run **Claude Code**, **Codex**, **opencode**, **Cursor agent CLIs**, **Grok**, and similar tools that:

- Run for a long time
- Block waiting for approval or input
- Multiply (one agent per task or worktree)

Herdr adds:

Capability	What you get
Background server	Agents keep running when you detach or close the laptop lid (machine still on)
Agent detection	Sidebar/state for working / blocked / idle
Mouse-first UX	Click panes, drag splits, right-click menus—no prefix required to start
tmux-like prefix	`Ctrl-b` family bindings if you already know tmux
CLI + socket API	Agents and scripts split panes, wait on status, read output
Remote attach	Work on a box over SSH-style remote modes

Herdr does **not** wrap or replace your agents. You run the same CLIs; Herdr owns their terminals.

Install

Quick install (Linux / macOS)

```
curl -fsSL https://herdr.dev/install.sh | sh
```

Ensure the install location is on your `PATH`, then:

```
herdr
herdr update          # later, for installer-managed binaries
```

Package managers

```
# Homebrew
brew install herdr

# mise
mise use -g herdr
# fallback if registry is old:
# mise use -g github:herdrdev/herdr

# Nix (pin a release tag in real use)
nix profile install github:herdrdev/herdr/v0.x.y
# or: nix run github:herdrdev/herdr/v0.x.y
```

Manual binary

Download from GitHub releases:

System	Asset
Linux x86_64	`herdr-linux-x86_64`
Linux aarch64	`herdr-linux-aarch64`

```
chmod +x herdr-linux-x86_64
mv herdr-linux-x86_64 ~/.local/bin/herdr
```

Channels

Direct installs default to **stable** on Linux/macOS. Preview builds:

```
herdr channel set preview
herdr channel set stable
```

Homebrew / mise / Nix updates go through those tools, not always `herdr update`.

Concepts

```
herdr-server (background)
    session (default or named)
         workspace "api"
              tab "agents"
                   pane: claude (blocked)
                   pane: codex (working)
              tab "runtime"
                    pane: dev server
                    pane: nvim / hx
         workspace "docs"
```

Term	Meaning
Server	Owns PTYs; survives client disconnect
Client	Renders UI; attach/detach
Workspace	Project-scale unit (often one repo or concern)
Tab	Layout group inside a workspace
Pane	Real terminal (shell, editor, agent)
Agent state	Rolled up from pane → tab → workspace

Quick start

```
# Start (spawns/connects to server, opens client)
herdr

# Inside: open your usual tools in panes
claude          # or codex, opencode, etc.
nvim
just test

# Detach - leave the herd running (see keyboard docs for exact bind)
# Default family: Ctrl-b then q  (detach; leave everything running)
```

Reattach later with `herdr` again. Named sessions:

```
herdr session list
herdr session attach work
herdr session attach side-project
herdr session stop work
herdr session delete side-project
```

Remote-oriented usage (patterns from docs):

```
# SSH to the machine, then run herdr like tmux
ssh you@server
herdr

# Or remote client mode when supported
herdr --remote workbox
herdr --remote workbox --session agents
```

Direct attach by agent label when configured:

```
herdr agent attach reviewer
```

Keyboard and mouse

Herdr is **mouse-native**: click panes, drag borders, use context menus.

If you know tmux, the **prefix** model will feel familiar (`Ctrl-b` by default for many actions). Common ideas from the keyboard guide:

Action	Typical binding
Workspace navigation	prefix+w
Detach (leave running)	prefix+q
New workspace	prefix+Shift+n
Rename workspace	prefix+Shift+w
Close workspace	prefix+Shift+d
Goto picker	prefix+g
Toggle sidebar	prefix+b
Close tab	prefix+Shift+x

Bindings are configurable—see Configuration for the live reference. Prefer the docs over memorizing a fork's defaults.

CLI automation

Herdr exposes a CLI (and socket API) so **you or an agent** can drive structure:

```
# Shape a workspace
herdr workspace create --cwd ~/project --label api
herdr tab create --label logs

# Panes
herdr pane split 1-1 --direction right
herdr pane run 1-2 "just test"

# Wait for agent status, then inspect output
herdr wait agent-status 1-1 --status done
herdr pane read 1-2 --source recent-unwrapped
```

Report custom metadata for the UI:

```
herdr pane report-metadata <pane_id> \
  --source my-agent-hook \
  --token model=opus \
  --token summary="reviewing authentication"
```

Exact subcommands evolve—run `herdr --help` and the CLI / socket API docs for your installed version.

Working with editors

Herdr is the outer frame; pick any editor from this book:

Editor	Pattern
Neovim / LazyVim	Pane for `nvim`, pane(s) for agents, pane for tests
Helix	Same; Helix picker for files, Herdr for agents
Zed	GUI editor on the host; Herdr for remote/agent PTYs (or local agent panes)
Micro / nano	Fine for quick edits in a pane beside an agent

Example layout intent:

```
Workspace: payments-service

  nvim                claude (agent)

                      cargo test -q
```

`$EDITOR` still matters

Agents and git often spawn editors:

```
export EDITOR=nvim
export VISUAL=nvim
# or: export EDITOR="zed --wait"
```

Herdr vs tmux vs Zellij

	tmux	Zellij	Herdr
Primary job	General sessions	Friendly multiplexer	Agent herds + sessions
Agent state	Manual	Manual	First-class
Scripting	Excellent	Good	CLI + socket API
Availability on bare VPS	Highest	Medium	Install binary
Mouse UX	Optional	Good	Core
Nested with tmux	Common	Common	Supported carefully

Practical combo:

- **tmux** on every server as baseline skill
- **Herdr** on machines where you run multiple coding agents
- Avoid deep nesting: do not put *tmux inside Herdr inside tmux* without a reason

Herdr can run **inside** an outer tmux. Conversely, if a shell auto-starts tmux *inside* a Herdr pane, Herdr may see `tmux` as the process instead of the agent—disable auto-tmux in those panes.

Agents and detection

Herdr detects many agent CLIs out of the box (Claude Code, Codex, opencode, Cursor-related tools, Grok, Copilot-style CLIs, and others—see current agents docs).

States roll up:

- **Blocked** — needs you (approval, question)
- **Working** — busy

- **Idle / done** — quiet

Your job becomes: open the sidebar, jump to **blocked**, answer, move on.

Configuration sketch

Configuration covers keybindings, themes, sidebar, notifications, scrollback, and advanced options. Paths and schema are versioned—start from:

```
# After first run, check docs for config file location and examples
# https://herdr.dev/docs/configuration/
```

When customizing:

1. Change one key family at a time
2. Keep a "panic" path: mouse + command palette / goto picker
3. Store config in git with your dotfiles

Plugins

Herdr supports **plugins** (manifest + executable actions/hooks) and a marketplace direction. Plugins can restore layouts, add overlays, or react to events such as worktree creation.

Sketch (illustrative—verify against current plugin docs):

```
[[actions]]
id = "list-workspaces"
title = "List workspaces"
contexts = ["workspace"]
command = ["node", "index.js"]
```

See Plugins and Marketplace.

Session state and persistence

Important mental model (aligned with Herdr docs):

- Detach kill: panes keep running in the server
- Restart/restore behavior depends on session state features and agent resume support
- Protocol-breaking upgrades may require stopping the old server (`herdr server stop` / `herdr session stop <name>`) after `herdr update`

For production data jobs, still use proper supervisors (systemd, containers). Herdr is for **interactive and agent** workspaces, not a substitute for process monitoring of daemons—though many people run dev servers inside panes safely.

Security notes

- Treat a machine running agent CLIs with **repo and cloud credentials** as sensitive
- Remote attach expands the attack surface—use SSH hardening, keys, and least privilege
- Do not paste secrets into agent panes you will share or record
- Review plugin source before installing community plugins

Troubleshooting

Issue	What to try
`herdr: command not found`	Restart shell; check PATH / installer location
Cannot reattach	`herdr session list`; confirm server not stopped
Agent shows as idle/wrong	Pane may be nested tmux/shell wrapper; run agent as direct pane command
After update, client/server mismatch	Stop old server, start fresh client
Keys stolen by outer tmux	Use mouse, or send prefix twice, or simplify nesting

```
herdr update
herdr server stop      # default session server
herdr                  # start clean
```

Practice lab

1. Install Herdr and run `herdr`
2. Create two panes: shell + a long `ping` or `htop`
3. Detach, close the terminal emulator, reattach—confirm processes lived
4. Run two agent CLIs (or two shells pretending to be agents) in one workspace
5. Use the sidebar / goto picker to jump between them
6. Script a workspace with `herdr workspace create` and `herdr pane split` if your version supports those verbs

When to use what

```
SSH to random prod bastion, edit nginx.conf
  → tmux + vi/nano

Local multi-repo app work with one shell
  → tmux or Zellij + nvim/hx/zed

Three coding agents + dev server + editor
```

```
→ Herdr (+ your editor of choice)
```

Further reading

- Herdr docs home
- Install
- Agents
- How to work
- Persistence and remote
- Keyboard
- Agent onboarding prompt from docs: point your agent at `https://herdr.dev/agent-guide.md`

Next: Zed for a modern native GUI editor, or Workflows to combine Herdr with Neovim/Helix/tmux deliberately.

Zed

Zed

Zed: Fast Native Code Editor

Zed is a modern, GPU-accelerated code editor written in Rust. It targets snappy local editing, first-class collaboration, and tight language tooling—without Electron. On Linux it is a strong **GUI** counterpart to terminal editors like Neovim and Helix.

Homepage: https://zed.dev/

Why Zed on Linux

- **Performance** — native UI, responsive large files and multi-buffer work
- **Language awareness** — tree-sitter + LSP-style intelligence
- **Collaboration** — real-time multiplayer editing (Zed's signature feature)
- **Vim mode** (optional) — softer landing from modal editors
- **Agent / AI features** — evolving; often used *beside* terminal agents in tmux/Herdr
- **Default themes** — **One Dark** / **One Light** (the palette many terminals and this book's code theme echo)

Zed is **not** a drop-in for rescue-shell editing. Keep `vi`/`nano` skills. Use Zed on workstations and powerful laptops; use TUI editors over raw SSH.

Install

Channels move quickly—prefer the official install docs if a command fails.

```
# Official script (check zed.dev for current recommendation)
curl -f https://zed.dev/install.sh | sh

# Some distros / community packages (availability varies)
# Arch (AUR often): yay -S zed
# Flatpak (if published for your channel): flatpak install ...
# Nix: search nixpkgs for zed-editor
```

Verify:

```
zed --version
which zed
```

Open a project:

```
zed .
zed path/to/file.rs:42
```

Wait flag for `$EDITOR`

GUI editors must block the caller until the file is closed:

```
export EDITOR="zed --wait"
export VISUAL="$EDITOR"
git config --global core.editor "zed --wait"
```

Without `--wait`, git commits may open an empty editor and continue immediately.

Core UI map

Area	Role
Buffer	File contents
Tab bar	Open files
Project panel	File tree
Outline / symbols	Jump inside file
Terminal panel	Integrated shell (optional; many still use external tmux)
Command palette	Fuzzy all-the-commands

Command palette is the discovery tool: open it and type what you want (`format`, `theme`, `vim`, `terminal`).

Default chord families (approximate—confirm in Command Palette / docs):

- **Command palette** — often `Ctrl+Shift+P` (Linux)
- **File finder** — often `Ctrl+P`
- **Save** — `Ctrl+S`
- **Toggle terminal** — palette: "terminal"

Keymaps are configurable in Zed settings (JSON).

Settings and config

Zed stores settings in its config directory (typically under `~/.config/zed/` on Linux). Use the UI **Settings** or edit JSON.

Illustrative settings (keys evolve—validate against your version):

```
{
  "theme": "One Dark",
  "buffer_font_family": "JetBrains Mono",
  "buffer_font_size": 14,
  "ui_font_size": 14,
  "tab_size": 2,
  "hard_tabs": false,
  "format_on_save": "on",
  "autosave": "on_focus_change",
  "vim_mode": false,
  "relative_line_numbers": false,
  "soft_wrap": "editor_width",
  "preferred_line_length": 100,
  "remove_trailing_whitespace_on_save": true,
  "ensure_final_newline_on_save": true
}
```

Themes

Built-in **One Dark** and **One Light** match Zed's defaults. Extensions add more. Theme overrides can tweak syntax tokens (see Zed docs on `theme_overrides`).

Vim mode

If you migrate from Vim/Neovim/Helix:

```
{
  "vim_mode": true
}
```

Expect:

- Motions and operators covering daily editing
- Occasional gaps vs full Neovim (plugins, exact text objects)
- Escape to normal mode still fundamental

Helix users: selection-first muscle memory will not map 1:1; use palette and multi-cursor features instead of forcing Helix habits.

Language servers and extensions

Zed uses extensions for languages, themes, and tools.

Typical flow:

1. Open a file type (e.g. `main.go`)
2. Install recommended extension when prompted
3. Confirm diagnostics and go-to-definition work

Install language tools on the host as needed (`rust-analyzer`, `gopls`, `pyright`, etc.). Zed coordinates; it does not always bundle every server.

Projects and workspaces

```
# Prefer opening the project root (git root)
cd ~/src/myapp && zed .

# Multi-folder workspaces: use Zed's workspace UI / recent projects
```

Trust the project panel + file finder over deep nested terminal `cd` when working locally.

Pairing with tmux and Herdr

Zed is a **GUI**. Common architectures:

A. Local Zed + terminal multiplexer for agents

```
Desktop: Zed (code)
Terminal: herdr or tmux
    claude / codex
    just dev
    lazygit
```

B. All in terminal (no Zed remote)

When only SSH is available: use Neovim/Helix inside tmux—not Zed.

C. Remote code

Options people use:

- SSHFS / mutual NFS + local Zed (latency sensitive)
- Dev containers / remote hosts with local sync (mutagen, etc.)
- Stay on terminal editors for high-latency links

Do not expect classic "Zed over dumb SSH" to feel like local.

Collaboration

Zed's multiplayer lets multiple people edit a project together. Use when:

- Pairing on a design spike
- Mentoring
- Shared incident editing (with care around secrets)

Turn collaboration off for sensitive codebases unless policy allows.

Git workflow

Zed provides git UI affordances (status, diffs—feature set depends on version). Many Linux power users still prefer:

```
# In an adjacent terminal / Herdr pane
lazygit
git diff | delta
```

Use Zed for editing hunks; use terminal for complex rebase surgery if you already know it.

Comparison: Zed vs neighbors

	Zed	**VSCodium/Code**	**Neovim**	**Helix**
UI	Native GPU	Electron	TUI	TUI
SSH-first	Weak	Remote-SSH ext	Strong	Strong
Config	JSON / UI	JSON + ext	Lua	TOML
Plugins	Extensions	Huge	Huge	Minimal (built-ins)
Startup	Fast	Heavier	Fast	Fast
Collab	Built-in focus	Live Share etc.	Plugins	Limited

Migration tips

From VS Code

- Install language extensions analogous to your VS Code set
- Relearn a smaller command palette surface
- Map keybindings gradually; do not clone 200 VS Code chords day one

From Neovim

- Enable `vim_mode` or keep a terminal nvim for remote
- Accept that Lua plugin ecosystems do not transfer
- Keep tmux/Herdr muscle memory for session persistence

From Helix

- Use multi-cursor and palette
- Rely on extensions for features Helix builds in

Troubleshooting

Problem	Try
`zed` not in PATH	Re-login; add install dir to PATH
Git opens and exits	Use `zed --wait`
Ugly fonts	Set `buffer_font_family` to a Nerd/ mono font you installed
LSP dead	Install extension + server binary; check language logs in UI
Wayland quirks	Update GPU drivers; try X11 session only for isolation tests
Flatpak permissions	Grant filesystem access to project paths

Practice checklist

- ☐ Install Zed and open a real git repo with `zed .`
- ☐ Set `EDITOR="zed --wait"` and make a git commit
- ☐ Toggle One Dark / One Light
- ☐ Enable or deliberately skip Vim mode
- ☐ Run tests in an external tmux/Herdr pane while editing in Zed
- ☐ Use command palette to format document and jump to file

When to choose Zed

Choose Zed when you want a fast local GUI, collaboration, and modern defaults on Linux desktops.

Choose Neovim/Helix when you live in SSH, minimal environments, or keyboard-only modal purity.

Choose both: Zed locally, terminal editor remotely—the norm for many teams.

Next: Workflows for end-to-end stacks that combine Zed, tmux, Herdr, and the editors from earlier parts.

Workflows

Workflows

Workflows: Editors + Workspaces + Tools

Isolated chapters teach tools. This chapter **composes** them into daily Linux workflows you can copy and adapt.

Stack templates

1) Server / sysadmin (minimal)

```
SSH → tmux → vi or nano → rg/sed when needed
```

```
ssh -t host 'tmux new -A -s ops'
# inside:
sudoedit /etc/nginx/nginx.conf    # respects $EDITOR if set
journalctl -u nginx -f            # other pane
```

Skills: Vi, Nano, tmux.

2) Terminal IDE (Neovim)

```
tmux session "dev"
  window editor: nvim (LazyVim optional)
  window shell:  tests / services
  window git:    lazygit
```

```
export EDITOR=nvim
tmux new -A -s dev -c ~/src/app
# C-b c for new windows; or splits with C-b %
```

Skills: Neovim, LazyVim, tmux, modern tools (lazygit, rg).

3) Selection-first modal (Helix)

```
tmux or bare terminal
  hx .
  companion pane: cargo test --watch / npm run dev
```

```
export EDITOR=hx
hx .
```

Skills: Helix, tmux.

4) Friendly modeless (Micro)

```
micro config.yaml
# or panes in tmux for logs
```

Skills: Micro.

5) Desktop GUI + agents (Zed + Herdr)

```
Zed  → project editing, diagnostics, collab
Herdr → agent CLIs, long commands, detachable state
```

```
export EDITOR="zed --wait"
cd ~/src/app && zed .
# other terminal:
herdr
# panes: claude / codex / just dev
```

Skills: Zed, Herdr.

6) Multi-agent feature work (Herdr-centric)

```
Herdr workspace "feature-x"
  tab agents:  agent A (impl) | agent B (tests/review)
  tab runtime: dev server | lazygit | nvim/hx
```

```
herdr
# create panes; run agents; jump to blocked
```

When a pane needs a human editor for a messy conflict:

```
# in a free pane
nvim
# or open GUI if local
zed --wait path/to/conflicted.go
```

7) Emacs as environment

```
emacs -nw  or  emacsclient -nw
  magit, vterm/eshell, org
optional outer tmux for detach only
```

Skills: Emacs.

Cross-cutting recipes

Open search hits in your editor

```
# ripgrep → fzf → editor
rg -n "TODO|FIXME" | fzf | awk -F: '{print $1, $2}' | while read -r f l; do
  $EDITOR +"$l" "$f"
done

# simpler: fzf file open
export FZF_DEFAULT_COMMAND='fd --type f --hidden --follow --exclude .git'
f() { local f; f=$(fzf --preview 'bat --color=always {}') && $EDITOR "$f"; }
```

Git commit with correct blocking editor

```
# TUI
git config --global core.editor "nvim"
# GUI
git config --global core.editor "zed --wait"
# VSCodium
git config --global core.editor "codium --wait"
```

kubectl / sudoedit

```
export EDITOR=nvim
sudoedit /etc/ssh/sshd_config
kubectl edit deploy/api
```

One-shot session script (tmux)

```
#!/usr/bin/env bash
set -euo pipefail
S=app
ROOT=~/src/app
tmux has-session -t "$S" 2>/dev/null && exec tmux attach -t "$S"
tmux new-session -d -s "$S" -n edit -c "$ROOT"
tmux send-keys -t "$S:edit" 'nvim' Enter
tmux split-window -h -t "$S:edit" -c "$ROOT"
tmux send-keys -t "$S:edit.2" 'git status -sb' Enter
tmux new-window -t "$S" -n run -c "$ROOT"
tmux send-keys -t "$S:run" 'just dev' Enter
tmux attach -t "$S"
```

Agent + editor discipline

1. **One writable branch per agent** (or worktree) to avoid clobbering
2. Human owns merge commits and production credentials
3. Keep secrets out of agent prompts and shared panes
4. Detach Herdr/tmux instead of killing terminals mid-run

```
git worktree add ../app-agent-b -b agent/b
# point second agent cwd at ../app-agent-b
```

Keybinding peace treaty

Tool	Reserve
tmux	`Ctrl-b` as prefix
Herdr	prefix + mouse; do not nest blindly
Vim/Neovim	leader key (often `Space`)
Helix	`Space` for pickers
Terminal emulator	do not bind `Ctrl-b` / `Ctrl-a` if you can avoid it

If something "eats" a key, ask: **emulator → multiplexer → editor** in that order.

Dotfiles layout (suggested)

```
~/.config/
  nvim/          or helix/  or micro/
  tmux/tmux.conf or ~/.tmux.conf
  git/config
  zed/settings.json
  herdr/         (per current Herdr docs)
```

Store in a git repo; bootstrap with a single `install` script. Do not share machine-local secrets in that repo.

Choosing under pressure

```
Production on fire, unfamiliar host
  → vi or nano, no plugins

Laptop, feature sprint, two agents
  → Herdr + Zed or nvim

Teaching a newcomer Linux
  → nano or Micro, then show vi exits

Deep C++/Rust refactor alone
  → Helix or LazyVim + tmux tests pane
```

Practice: build your "default"

Write this down and stick to it for a month:

```
Daily editor: _____________
Session tool:  tmux / herdr / both: ________
$EDITOR:      _____________
Search:       rg + fzf + bat
Git UI:       lazygit / magit / zed / plain
```

Then deepen *that* stack using the matching chapters—not every chapter at once.

See also

- Choosing an Editor
- Terminal Workspaces

- tmux
- Herdr
- Zed
- Modern Tools

Scripting

Bash

Intro to Shells

Shell vs Terminal: Understanding the Difference

Shell vs Terminal: Understanding the Difference

Shell vs Terminal: Understanding the Difference

Many people use the terms "shell" and "terminal" interchangeably, but they are actually different components that work together to provide your command-line experience.

What is a Terminal?

A terminal (also called terminal emulator) is a program that provides a window for you to interact with the shell. It handles:

- **Display**: Shows text output from programs
- **Input**: Captures keyboard input and sends it to the shell
- **Window Management**: Provides scrolling, resizing, and visual features
- **Character Encoding**: Handles text rendering and fonts

Common Terminal Emulators

Linux:

- **GNOME Terminal**: Default on GNOME desktop
- **Konsole**: KDE's terminal emulator
- **xterm**: Classic X11 terminal
- **Alacritty**: GPU-accelerated terminal
- **Terminator**: Terminal with split panes

macOS:

- **Terminal.app**: Built-in terminal
- **iTerm2**: Popular third-party option

Windows:

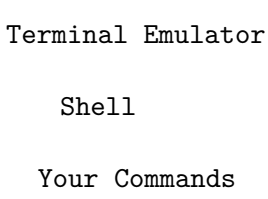
- **Windows Terminal**: Modern terminal for Windows 10/11
- **Command Prompt**: Traditional Windows terminal
- **PowerShell**: Microsoft's advanced shell and terminal

What is a Shell?

A shell is the command interpreter that runs inside the terminal. It:

- **Processes Commands**: Interprets what you type
- **Manages Environment**: Handles variables and settings
- **Executes Programs**: Launches other applications
- **Provides Scripting**: Enables automation through scripts

The Relationship



Practical Examples

Example 1: Multiple Shells in One Terminal

```
# Start in bash
$ echo $SHELL
/bin/bash

# Switch to zsh
$ zsh
$ echo $SHELL
/bin/zsh

# Switch to fish
$ fish
$ echo $SHELL
/usr/bin/fish

# Exit back to previous shells
$ exit  # Back to zsh
$ exit  # Back to bash
```

Example 2: Same Shell in Different Terminals

You can run the same shell (like bash) in multiple terminal windows simultaneously. Each terminal window is independent, but they're all running the same shell program.

Example 3: Remote Shells

```
# Connect to remote server via SSH
$ ssh user@remote-server
# Now you're running a shell on the remote machine
# but displaying it in your local terminal
```

Terminal Features vs Shell Features

Terminal Features:

- **Colors and Themes**: Visual appearance
- **Font Settings**: Text rendering
- **Window Management**: Tabs, splits, transparency
- **Copy/Paste**: Clipboard integration
- **Scrollback**: History of displayed text

Shell Features:

- **Command History**: Previously executed commands
- **Tab Completion**: Auto-complete functionality
- **Variables**: Environment and user-defined variables
- **Scripting**: Programming capabilities
- **Job Control**: Managing background processes

Configuration Files

Terminal Configuration:

- Usually GUI-based settings
- Profiles for different appearances
- Keyboard shortcuts

Shell Configuration:

```
# Bash configuration files
~/.bashrc      # Interactive non-login shells
~/.bash_profile # Login shells
~/.bash_logout  # Logout cleanup

# Zsh configuration files
~/.zshrc       # Main configuration
~/.zprofile    # Login shells

# Fish configuration
~/.config/fish/config.fish
```

Choosing Terminal and Shell Combinations

For Development:

- **Terminal**: iTerm2 (macOS) or GNOME Terminal (Linux)
- **Shell**: bash or zsh with customizations

For System Administration:

- **Terminal**: Any reliable terminal emulator
- **Shell**: bash for compatibility

For Beginners:

- **Terminal**: Default system terminal
- **Shell**: bash (most common) or fish (user-friendly)

Common Misconceptions

Misconception 1: "I'm using Terminal"

Reality: You're using a terminal emulator to access a shell.

Misconception 2: "Bash is a terminal"

Reality: Bash is a shell that runs inside a terminal.

Misconception 3: "All terminals are the same"

Reality: Different terminals have different features and capabilities.

Practical Tips

Check what terminal you're using:

```
$ echo $TERM
xterm-256color

$ ps -o comm= -p "$PPID"
gnome-terminal-
```

Check what shell you're using:

```
$ echo $0
bash

$ ps -p $$
  PID TTY          TIME CMD
 1234 pts/0    00:00:00 bash
```

Launch different shells:

```
# Start a new shell session
$ bash
$ zsh
$ fish

# Run a single command in a different shell
$ zsh -c "echo 'This runs in zsh'"
$ fish -c "echo 'This runs in fish'"
```

Summary

- **Terminal**: The window/interface you see and interact with
- **Shell**: The command interpreter that processes your commands
- **Relationship**: The terminal displays the shell's input/output
- **Independence**: You can use different shells in the same terminal, or the same shell in different terminals

Understanding this distinction helps you troubleshoot issues, customize your environment effectively, and communicate more precisely about command-line tools.

Types of Shells

Types of Shells

Types of Shells

Linux and Unix systems support various types of shells, each with its own features and capabilities. Understanding the different shells helps you choose the right tool for your needs.

Common Shell Types

1. Bourne Shell (sh)

- **Original**: The first Unix shell, developed by Stephen Bourne
- **Features**: Basic scripting capabilities, simple syntax
- **Usage**: Still used for system scripts and compatibility
- **Location**: Usually `/bin/sh`

```
# Check if sh is available
$ which sh
/bin/sh
```

2. Bash (Bourne Again Shell)

- **Most Popular**: Default shell on most Linux distributions
- **Features**: Enhanced version of Bourne shell with many improvements
- **Advantages**:
  - Command history
  - Tab completion
  - Job control
  - Arrays and associative arrays
  - Advanced scripting features

```
# Check Bash version
$ bash --version
GNU bash, version 5.1.16(1)-release
```

3. Zsh (Z Shell)

- **Advanced**: Feature-rich shell with powerful customization
- **Features**:
  - Excellent tab completion
  - Spelling correction
  - Plugin system
  - Themes support
- **Popular Framework**: Oh My Zsh

```
# Install zsh (Ubuntu/Debian)
$ sudo apt install zsh

# Make zsh default shell
$ chsh -s $(which zsh)
```

4. Fish (Friendly Interactive Shell)

- **User-Friendly**: Designed for ease of use
- **Features**:
  - Syntax highlighting
  - Auto-suggestions
  - Web-based configuration
  - No configuration required

```
# Install fish (Ubuntu/Debian)
$ sudo apt install fish

# Start fish shell
$ fish
```

5. Dash (Debian Almquist Shell)

- **Lightweight**: Minimal shell focused on speed
- **Usage**: Often used as `/bin/sh` on Debian-based systems
- **Purpose**: System scripts and boot processes

6. Tcsh (TENEX C Shell)

- **C-like Syntax**: Similar to C programming language
- **Features**: Command history, job control, aliases
- **Usage**: Less common in modern systems

7. Ksh (Korn Shell)

- **Commercial**: Developed by David Korn at Bell Labs
- **Features**: Combines features of Bourne and C shells
- **Variants**: ksh88, ksh93, mksh (MirBSD Korn Shell)

Checking Available Shells

View all available shells:

```
$ cat /etc/shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
/usr/bin/zsh
/usr/bin/fish
```

Check your current shell:

```
$ echo $SHELL
/bin/bash

# Or use ps command
$ ps -p $$
  PID TTY          TIME CMD
 1234 pts/0    00:00:00 bash
```

Check shell being used by a script:

```
$ ps -o pid,ppid,cmd
```

Shell Comparison

Feature	sh	bash	zsh	fish	dash
Speed	Fast	Medium	Medium	Medium	Very Fast
Scripting	Basic	Advanced	Advanced	Good	Basic

Feature	sh	bash	zsh	fish	dash
Interactive	Basic	Good	Excellent	Excellent	Basic
Compatibility	High	High	Good	Low	High
Customization	Limited	Good	Excellent	Good	Limited

Choosing the Right Shell

For Scripting:

- **bash**: Best balance of features and compatibility
- **sh**: Maximum portability across systems
- **dash**: When speed is critical

For Interactive Use:

- **zsh**: Power users who want customization
- **fish**: Beginners who want user-friendly features
- **bash**: Good default choice for most users

For System Administration:

- **bash**: Most common, good documentation
- **sh**: For system scripts that need portability

Switching Between Shells

Temporarily switch:

```
# Start a new shell session
$ zsh
$ fish
$ bash

# Exit back to previous shell
$ exit
```

Change default shell:

```
# Change default shell for current user
$ chsh -s /usr/bin/zsh

# Verify the change
$ grep $USER /etc/passwd
```

Shell Compatibility

When writing scripts, consider compatibility:

```
#!/bin/bash
# This script requires bash-specific features

#!/bin/sh
# This script should work with any POSIX shell

#!/usr/bin/env bash
# Portable way to find bash
```

Understanding the different types of shells available helps you make informed decisions about which shell to use for different tasks and environments.

What is a Shell?

What is a Shell?

What is a Shell?

A shell is a command-line interface that acts as an intermediary between you and the operating system kernel. It's called a "shell" because it wraps around the kernel, providing a user-friendly way to interact with the system.

The Role of a Shell

The shell serves several critical functions:

1. **Command Interpreter**: Reads and executes commands you type
2. **Program Launcher**: Starts other programs and applications
3. **Environment Manager**: Manages environment variables and settings
4. **Scripting Platform**: Allows automation through script files

How Shells Work

When you type a command, here's what happens:

```
$ ls -la
```

1. The shell reads your input
2. Parses the command and arguments
3. Locates the program (`ls`) in the system PATH
4. Executes the program with the specified arguments (`-la`)
5. Displays the output back to you

Interactive vs Non-Interactive Shells

Interactive Shell

- Accepts commands from user input
- Provides a prompt (like `$` or `#`)
- Offers features like command history and tab completion

Non-Interactive Shell

- Executes scripts without user interaction
- Used for automation and batch processing
- No prompt or interactive features

Login vs Non-Login Shells

Login Shell

- Started when you log into the system
- Reads configuration files like `.bash_profile`
- Sets up the initial environment

Non-Login Shell

- Started from within another shell or application
- Reads different configuration files like `.bashrc`
- Inherits environment from parent shell

Shell Capabilities

Modern shells provide powerful features:

- **Command History**: Access previously executed commands
- **Tab Completion**: Auto-complete commands and filenames
- **Wildcards**: Use patterns like `*.txt` to match multiple files
- **Pipes and Redirection**: Chain commands and redirect output
- **Variables**: Store and manipulate data
- **Control Structures**: Conditional statements and loops
- **Functions**: Create reusable code blocks

Example: Basic Shell Interaction

```
# Display current directory
$ pwd
/home/username

# List files
$ ls
Documents  Downloads  Pictures
```

```
# Create a directory
$ mkdir my_scripts

# Navigate to the directory
$ cd my_scripts

# Check current location
$ pwd
/home/username/my_scripts
```

Understanding what a shell is and how it works is fundamental to becoming proficient in shell scripting and system administration.

Scripting Fundamentals

Your First Shell Script

Your First Shell Script

Your First Shell Script

Let's create your first shell script step by step. This hands-on approach will help you understand the basics and get you started with shell scripting.

Step 1: Create Your First Script

Using a Text Editor

Open your favorite text editor and create a new file called `hello.sh`:

```
# Using nano
$ nano hello.sh

# Using vim
$ vim hello.sh

# Using VS Code
$ code hello.sh
```

Write the Script

Type the following content into your file:

```
#!/bin/bash
# My first shell script
# Purpose: Display a greeting message

echo "Hello, World!"
echo "Welcome to shell scripting!"
echo "Today's date is: $(date)"
echo "Current user: $(whoami)"
echo "Current directory: $(pwd)"
```

Save the File

- **nano**: Press `Ctrl+X`, then `Y`, then `Enter`
- **vim**: Press `Esc`, type `:wq`, then `Enter`
- **VS Code**: Press `Ctrl+S`

Step 2: Understanding the Script

Let's break down each part:

Shebang Line

```
#!/bin/bash
```

- **Purpose**: Tells the system to use bash to execute this script
- **Must be**: The very first line of the script
- **Alternative**: `#!/bin/sh` for POSIX compatibility

Comments

```
# My first shell script
# Purpose: Display a greeting message
```

- **Purpose**: Document what the script does
- **Syntax**: Lines starting with `#` (except shebang)
- **Best Practice**: Always comment your scripts

Commands

```
echo "Hello, World!"
```

- **echo**: Command to display text
- **Quotes**: Protect text with spaces and special characters

Command Substitution

```
echo "Today's date is: $(date)"
```

- **$(command)**: Executes command and substitutes its output
- **Alternative**: Backticks `` `date` `` (older syntax)

Step 3: Make the Script Executable

Before you can run the script, you need to make it executable:

```
# Check current permissions
$ ls -l hello.sh
```

```
-rw-r--r-- 1 user user 245 Jan 15 10:30 hello.sh

# Make it executable
$ chmod +x hello.sh

# Verify permissions changed
$ ls -l hello.sh
-rwxr-xr-x 1 user user 245 Jan 15 10:30 hello.sh
```

Understanding Permissions

- **r**: Read permission
- **w**: Write permission
- **x**: Execute permission
- **First rwx**: Owner permissions
- **Second rwx**: Group permissions
- **Third rwx**: Other users permissions

Step 4: Run Your Script

There are several ways to execute your script:

Method 1: Direct Execution

```
$ ./hello.sh
Hello, World!
Welcome to shell scripting!
Today's date is: Mon Jan 15 10:35:22 EST 2024
Current user: john
Current directory: /home/john/scripts
```

Method 2: Using bash Command

```
$ bash hello.sh
# Same output as above
```

Method 3: Using sh Command

```
$ sh hello.sh
# Same output as above
```

Method 4: Full Path

```
$ /home/john/scripts/hello.sh
# Same output as above
```

Step 5: Enhance Your Script

Let's make the script more interactive:

```
#!/bin/bash
# Enhanced greeting script
# Purpose: Interactive greeting with user input

echo "====================================="
echo "    Welcome to Shell Scripting!"
echo "====================================="
echo

# Get user's name
echo "What's your name?"
read name

# Get user's favorite color
echo "What's your favorite color?"
read color

# Display personalized greeting
echo
echo "Hello, $name!"
echo "I see your favorite color is $color."
echo "That's a great choice!"
echo
echo "System Information:"
echo "==================="
echo "Date: $(date)"
echo "User: $(whoami)"
echo "Home Directory: $HOME"
echo "Current Directory: $(pwd)"
```

```
echo "Shell: $SHELL"
echo
echo "Have a great day, $name!"
```

Save this as `enhanced_hello.sh` and run it:

```
$ chmod +x enhanced_hello.sh
$ ./enhanced_hello.sh
```

Step 6: Adding Error Handling

Let's add some basic error handling:

```
#!/bin/bash
# Script with error handling
# Purpose: Demonstrate basic error handling

set -e  # Exit immediately if a command exits with a non-zero status

echo "Starting script..."

# Check if a directory exists
if [ -d "/home" ]; then
    echo "  /home directory exists"
else
    echo "  /home directory not found"
    exit 1
fi

# Try to create a test file
TEST_FILE="test_file.txt"
echo "Creating test file: $TEST_FILE"

if echo "This is a test" > "$TEST_FILE"; then
    echo "  Test file created successfully"

    # Clean up
    rm "$TEST_FILE"
    echo "  Test file cleaned up"
else
    echo "  Failed to create test file"
    exit 1
fi

echo "Script completed successfully!"
```

Step 7: Script with Functions

Let's organize code using functions:

```
#!/bin/bash
# Script with functions
# Purpose: Demonstrate function usage

# Function to display a separator
show_separator() {
    echo "================================="
}

# Function to display system info
show_system_info() {
    echo "System Information:"
    echo "OS: $(uname -s)"
    echo "Kernel: $(uname -r)"
    echo "Architecture: $(uname -m)"
    echo "Hostname: $(hostname)"
}

# Function to display disk usage
show_disk_usage() {
    echo "Disk Usage:"
    df -h | head -5
}

# Main script execution
main() {
    show_separator
    echo "    System Status Report"
    show_separator
    echo

    show_system_info
    echo

    show_disk_usage
    echo

    show_separator
    echo "Report generated on: $(date)"
    show_separator
}

# Call main function
```

```
main
```

Common Beginner Mistakes

1. Forgetting the Shebang

```
# Wrong - no shebang
echo "Hello World"

# Correct
#!/bin/bash
echo "Hello World"
```

2. Not Making Script Executable

```
# This will fail
$ ./myscript.sh
bash: ./myscript.sh: Permission denied

# Fix it
$ chmod +x myscript.sh
$ ./myscript.sh
```

3. Wrong Path in Shebang

```
# Might not work on all systems
#!/usr/bin/bash

# More portable
#!/bin/bash

# Most portable
#!/usr/bin/env bash
```

4. Spaces in Variable Assignment

```
# Wrong - spaces around =
name = "John"

# Correct - no spaces
```

```
name="John"
```

Best Practices for Beginners

1. Always Use Shebang

```
#!/bin/bash
```

2. Add Comments

```
# What the script does
# Author and date
# Usage instructions
```

3. Use Meaningful Names

```
# Good
backup_directory="/backup"
user_name="john"

# Avoid
dir="/backup"
n="john"
```

4. Quote Variables

```
# Good
echo "Hello, $name"
cp "$source_file" "$destination"

# Risky
echo Hello, $name
cp $source_file $destination
```

5. Check for Errors

```
#!/bin/bash
set -e  # Exit on error

# Or check individual commands
if ! command_that_might_fail; then
    echo "Command failed!"
    exit 1
fi
```

Testing Your Scripts

1. Syntax Check

```
$ bash -n myscript.sh
# No output means no syntax errors
```

2. Debug Mode

```
$ bash -x myscript.sh
# Shows each command as it's executed
```

3. Use shellcheck

```
$ shellcheck myscript.sh
# Provides suggestions for improvement
```

Next Steps

Now that you've created your first script, you can:

1. **Experiment**: Modify the examples and see what happens
2. **Practice**: Create scripts for tasks you do regularly
3. **Learn More**: Study variables, conditionals, and loops
4. **Read Others' Scripts**: Look at system scripts in `/etc/init.d/`
5. **Use Version Control**: Start tracking your scripts with git

Congratulations! You've written and executed your first shell script. This is the foundation for all the advanced scripting techniques you'll learn in the following chapters.

Script Execution Methods

Script Execution Methods

Script Execution Methods

Understanding how to execute shell scripts is crucial for effective shell scripting. There are multiple ways to run scripts, each with its own use cases and implications.

Execution Methods Overview

Method	Syntax	New Process	Environment	Use Case
Direct	`./script.sh`	Yes	Inherited	Most common
Interpreter	`bash script.sh`	Yes	Inherited	Testing/debugging
Source	`source script.sh`	No	Modified	Environment setup
Absolute Path	`/path/to/script.sh`	Yes	Inherited	System scripts

Method 1: Direct Execution

This is the most common way to run scripts.

Prerequisites

```
# Script must be executable
$ chmod +x myscript.sh

# Script must have proper shebang
$ head -1 myscript.sh
#!/bin/bash
```

Execution

```
# Run from current directory
$ ./myscript.sh

# Run from anywhere (if in PATH)
```

```
$ myscript.sh
```

Example Script

```
#!/bin/bash
# direct_example.sh
echo "Running in process: $$"
echo "Parent process: $PPID"
echo "Current directory: $(pwd)"
export NEW_VAR="Hello from script"
```

Test It

```
$ chmod +x direct_example.sh
$ ./direct_example.sh
Running in process: 12345
Parent process: 12344
Current directory: /home/user/scripts

# Check if NEW_VAR exists in current shell
$ echo $NEW_VAR
# (empty - variable doesn't exist in parent shell)
```

Method 2: Using Shell Interpreter

Run the script by explicitly calling the shell interpreter.

Bash Interpreter

```
$ bash myscript.sh
```

Other Interpreters

```
$ sh myscript.sh      # POSIX shell
$ zsh myscript.sh     # Z shell
$ dash myscript.sh    # Debian Almquist shell
```

Advantages

- **No execute permission needed**
- **Override shebang**: Force specific interpreter
- **Testing**: Test script with different shells

Example

```
# Create script without execute permission
$ cat > test_interpreter.sh << 'EOF'
#!/bin/bash
echo "Shell: $0"
echo "Bash version: $BASH_VERSION"
EOF

# Run without making executable
$ bash test_interpreter.sh
Shell: bash
Bash version: 5.1.16(1)-release

# Try with different shell
$ sh test_interpreter.sh
Shell: sh
Bash version:
```

Method 3: Source (Dot) Command

Execute script in the current shell environment.

Syntax

```
$ source myscript.sh
# or
$ . myscript.sh
```

Key Characteristics

- **No new process**: Runs in current shell
- **Environment changes persist**: Variables and functions remain
- **No shebang needed**: Uses current shell
- **No execute permission needed**

Example Script

```
# source_example.sh
echo "Setting up environment..."
export PROJECT_ROOT="/home/user/myproject"
export PATH="$PROJECT_ROOT/bin:$PATH"

# Define a function
greet() {
    echo "Hello from sourced function!"
}

echo "Environment setup complete"
echo "PROJECT_ROOT: $PROJECT_ROOT"
```

Test Sourcing

```
# Before sourcing
$ echo $PROJECT_ROOT
# (empty)

# Source the script
$ source source_example.sh
Setting up environment...
Environment setup complete
PROJECT_ROOT: /home/user/myproject

# After sourcing - variables persist
$ echo $PROJECT_ROOT
/home/user/myproject

# Function is available
$ greet
Hello from sourced function!
```

Method 4: Absolute Path Execution

Run script using its full path.

Examples

```
# Full path execution
$ /home/user/scripts/myscript.sh

# Using $HOME variable
$ $HOME/scripts/myscript.sh

# Using tilde expansion
$ ~/scripts/myscript.sh
```

Use Cases

- **System scripts**: `/etc/init.d/apache2 start`
- **Cron jobs**: Full paths prevent PATH issues
- **Remote execution**: SSH with full paths

Method 5: Command Substitution

Execute script and capture output.

Syntax

```
# Modern syntax
result=$(./myscript.sh)

# Legacy syntax (avoid)
result=`./myscript.sh`
```

Example

```
# Script that returns a value
$ cat > get_timestamp.sh << 'EOF'
#!/bin/bash
date +%Y%m%d_%H%M%S
EOF

$ chmod +x get_timestamp.sh

# Capture output
$ timestamp=$(./get_timestamp.sh)
```

```
$ echo "Backup file: backup_$timestamp.tar.gz"
Backup file: backup_20240115_143022.tar.gz
```

Execution Context and Environment

Process Creation

```
#!/bin/bash
# process_info.sh
echo "Script PID: $$"
echo "Parent PID: $PPID"
echo "Shell: $0"
ps -f --pid $$ --pid $PPID
```

Environment Inheritance

```
#!/bin/bash
# env_test.sh
echo "PATH: $PATH"
echo "HOME: $HOME"
echo "USER: $USER"
echo "Custom var: $MY_CUSTOM_VAR"
```

Test Environment

```
# Set a custom variable
$ export MY_CUSTOM_VAR="test value"

# Run script - inherits environment
$ ./env_test.sh
PATH: /usr/local/bin:/usr/bin:/bin
HOME: /home/user
USER: user
Custom var: test value
```

Script Arguments and Parameters

Passing Arguments

```
$ ./myscript.sh arg1 arg2 arg3
```

Accessing Arguments

```
#!/bin/bash
# args_example.sh
echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "All arguments: $@"
echo "Number of arguments: $#"
```

Test Arguments

```
$ ./args_example.sh hello world 123
Script name: ./args_example.sh
First argument: hello
Second argument: world
All arguments: hello world 123
Number of arguments: 3
```

Exit Status and Error Handling

Exit Status

```
#!/bin/bash
# exit_status.sh
echo "Performing operation..."

# Simulate success or failure
if [ "$1" = "fail" ]; then
    echo "Operation failed!"
    exit 1
else
    echo "Operation successful!"
    exit 0
fi
```

Check Exit Status

```
$ ./exit_status.sh success
Operation successful!
$ echo $?
0

$ ./exit_status.sh fail
Operation failed!
$ echo $?
1
```

Advanced Execution Techniques

Background Execution

```
# Run script in background
$ ./long_running_script.sh &
[1] 12345

# Check background jobs
$ jobs
[1]+  Running    ./long_running_script.sh &
```

Conditional Execution

```
# Run second command only if first succeeds
$ ./script1.sh && ./script2.sh

# Run second command only if first fails
$ ./script1.sh || ./script2.sh

# Always run second command
$ ./script1.sh; ./script2.sh
```

Piping Script Output

```
# Pipe script output to another command
$ ./generate_data.sh | sort | uniq

# Redirect output to file
$ ./report_script.sh > report.txt 2>&1
```

Debugging Script Execution

Debug Mode

```
# Show commands as they execute
$ bash -x myscript.sh

# Or add to script
#!/bin/bash -x
```

Verbose Mode

```
# Show input lines as they're read
$ bash -v myscript.sh
```

Syntax Check

```
# Check syntax without executing
$ bash -n myscript.sh
```

Combined Options

```
# Multiple debug options
$ bash -xv myscript.sh
```

Security Considerations

File Permissions

```
# Secure script permissions
$ chmod 755 myscript.sh  # Owner: rwx, Others: r-x
$ chmod 700 myscript.sh  # Owner only: rwx
```

PATH Security

```
#!/bin/bash
# Use absolute paths for security
/bin/ls /home/user
/usr/bin/whoami
```

Input Validation

```
#!/bin/bash
# Validate input before execution
if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

filename="$1"
if [ ! -f "$filename" ]; then
    echo "Error: File '$filename' not found"
    exit 1
fi
```

Best Practices

1. Always Use Shebang

```
#!/bin/bash
# or
#!/usr/bin/env bash  # More portable
```

2. Set Error Handling

```
#!/bin/bash
set -euo pipefail  # Exit on error, undefined vars, pipe failures
```

3. Use Appropriate Execution Method

- **Direct execution**: General purpose scripts
- **Source**: Environment setup scripts
- **Interpreter**: Testing and debugging
- **Background**: Long-running processes

4. Handle Arguments Properly

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <arguments>"
    exit 1
fi
```

5. Provide Clear Output

```
#!/bin/bash
echo "Starting process..."
# ... script logic ...
echo "Process completed successfully"
```

Understanding these execution methods gives you the flexibility to run scripts in the most appropriate way for your specific needs and environment.

What is Shell Scripting?

What is Shell Scripting?

What is Shell Scripting?

Shell scripting is the practice of writing programs (scripts) that are executed by a shell. These scripts automate tasks, combine multiple commands, and create powerful workflows that would be tedious to perform manually.

Definition and Purpose

A shell script is a text file containing a series of commands that the shell can execute. Instead of typing commands one by one, you write them in a file and execute the entire sequence at once.

Key Benefits:

- **Automation**: Eliminate repetitive manual tasks
- **Consistency**: Ensure tasks are performed the same way every time
- **Efficiency**: Save time and reduce human error
- **Scheduling**: Run scripts automatically at specific times
- **Complex Logic**: Implement decision-making and loops

When to Use Shell Scripts

Perfect for:

- **System Administration**: User management, backups, log rotation
- **File Operations**: Batch processing, file organization
- **Development Workflows**: Build processes, deployment scripts
- **Data Processing**: Log analysis, report generation
- **System Monitoring**: Health checks, alert systems

Not Ideal for:

- **Complex Calculations**: Use Python, R, or specialized tools
- **GUI Applications**: Shell scripts are command-line oriented
- **Performance-Critical Tasks**: Compiled languages are faster
- **Cross-Platform Applications**: Shell scripts are OS-specific

Types of Shell Scripts

1. Simple Command Sequences

```
#!/bin/bash
# Backup script
cp /home/user/documents/* /backup/
tar -czf /backup/backup_$(date +%Y%m%d).tar.gz /backup/*.txt
echo "Backup completed at $(date)"
```

2. Interactive Scripts

```
#!/bin/bash
# User input script
echo "What's your name?"
read name
echo "Hello, $name! Welcome to shell scripting."
```

3. System Administration Scripts

```
#!/bin/bash
# System monitoring script
echo "System Status Report - $(date)"
echo "============================"
echo "Disk Usage:"
df -h
echo "Memory Usage:"
free -h
echo "CPU Load:"
uptime
```

4. Data Processing Scripts

```
#!/bin/bash
# Log analysis script
echo "Analyzing web server logs..."
grep "ERROR" /var/log/apache2/error.log | wc -l
echo "Total errors found"
```

Shell Script Components

1. Shebang Line

```
#!/bin/bash
# Tells the system which interpreter to use
```

2. Comments

```
# This is a single-line comment
: '
This is a
multi-line comment
'
```

3. Variables

```
name="John"
age=25
echo "Name: $name, Age: $age"
```

4. Commands

```
ls -la
pwd
date
```

5. Control Structures

```
if [ $age -gt 18 ]; then
    echo "Adult"
else
    echo "Minor"
fi
```

Script Execution Methods

1. Make Executable and Run

```
# Make script executable
$ chmod +x myscript.sh

# Run the script
$ ./myscript.sh
```

2. Run with Shell Interpreter

```
# Run with bash
$ bash myscript.sh

# Run with sh
$ sh myscript.sh
```

3. Source the Script

```
# Execute in current shell (affects current environment)
$ source myscript.sh
# or
$ . myscript.sh
```

Script Structure Best Practices

Basic Template:

```
#!/bin/bash

# Script: example.sh
# Purpose: Demonstrate shell script structure
# Author: Your Name
# Date: $(date)

# Set strict error handling
set -euo pipefail

# Global variables
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
LOG_FILE="/var/log/myscript.log"

# Functions
log_message() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" | tee -a "$LOG_FILE"
}

main() {
    log_message "Script started"

    # Main script logic here
    echo "Hello, World!"

    log_message "Script completed"
}

# Execute main function
main "$@"
```

Common Shell Scripting Patterns

1. Error Handling

```
#!/bin/bash
set -e  # Exit on any error

command_that_might_fail || {
    echo "Command failed!"
    exit 1
}
```

2. Argument Processing

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

filename="$1"
echo "Processing file: $filename"
```

3. Configuration Files

```
#!/bin/bash
# Load configuration
CONFIG_FILE="config.conf"
if [ -f "$CONFIG_FILE" ]; then
    source "$CONFIG_FILE"
else
    echo "Configuration file not found!"
    exit 1
fi
```

Shell Scripting vs Other Languages

Shell Scripts Excel At:

- System integration
- File manipulation
- Process management
- Quick automation tasks
- Gluing together existing tools

Other Languages Better For:

- **Python**: Complex logic, data analysis, web development
- **C/C++**: Performance-critical applications
- **JavaScript**: Web applications, Node.js servers
- **Go**: Network services, system tools

Development Environment

Essential Tools:

- **Text Editor**: vim, nano, VS Code, or any editor
- **Shell**: bash, zsh, or your preferred shell
- **Version Control**: git for tracking changes
- **Testing**: shellcheck for syntax checking

Useful Commands:

```
# Check script syntax
$ bash -n myscript.sh

# Debug script execution
$ bash -x myscript.sh

# Use shellcheck for best practices
$ shellcheck myscript.sh
```

Real-World Example

Here's a practical backup script:

```
#!/bin/bash
# backup.sh - Simple backup script

# Configuration
SOURCE_DIR="/home/user/documents"
BACKUP_DIR="/backup"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_NAME="backup_$DATE.tar.gz"

# Create backup directory if it doesn't exist
mkdir -p "$BACKUP_DIR"

# Create backup
echo "Starting backup of $SOURCE_DIR..."
tar -czf "$BACKUP_DIR/$BACKUP_NAME" "$SOURCE_DIR"

# Check if backup was successful
if [ $? -eq 0 ]; then
    echo "Backup completed successfully: $BACKUP_DIR/$BACKUP_NAME"

    # Remove backups older than 7 days
    find "$BACKUP_DIR" -name "backup_*.tar.gz" -mtime +7 -delete
    echo "Old backups cleaned up"
else
    echo "Backup failed!"
    exit 1
fi
```

Shell scripting is a powerful skill that bridges the gap between simple command-line usage and full programming. It's an essential tool for anyone working with Linux or Unix systems.

Bash Features

Bash Environment

Bash Environment

Bash Environment

The Bash environment consists of variables, settings, and configurations that determine how Bash behaves. Understanding and managing this environment is crucial for effective shell scripting and system administration.

Environment Variables

Environment variables are key-value pairs that affect how programs run. They're inherited by child processes and available to all programs launched from the shell.

Viewing Environment Variables

```
# Display all environment variables
$ env
PATH=/usr/local/bin:/usr/bin:/bin
HOME=/home/username
USER=username
SHELL=/bin/bash

# Display specific variable
$ echo $HOME
/home/username

# Alternative method
$ printenv HOME
/home/username

# List all variables (including shell variables)
$ set
```

Common Environment Variables

System Variables

```
# User information
echo $USER          # Current username
echo $HOME          # User's home directory
echo $UID           # User ID number
echo $GROUPS        # User's group memberships

# System information
echo $HOSTNAME      # System hostname
echo $HOSTTYPE      # Machine type
echo $OSTYPE        # Operating system type
echo $MACHTYPE      # Machine description

# Shell information
echo $SHELL         # Path to current shell
echo $BASH_VERSION  # Bash version
echo $0             # Name of current script/shell
```

Path Variables

```
# Executable search path
echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin:/sbin

# Library search path
echo $LD_LIBRARY_PATH

# Manual page search path
echo $MANPATH
```

Locale Variables

```
# Language and locale settings
echo $LANG          # Default locale
echo $LC_ALL        # Override all locale settings
echo $LC_TIME       # Time format locale
echo $LC_NUMERIC    # Number format locale
```

Setting Environment Variables

Temporary Variables (Current Session)

```
# Set variable for current session
$ export MY_VAR="Hello World"
$ echo $MY_VAR
Hello World

# Alternative syntax
$ MY_VAR="Hello World"
$ export MY_VAR
```

Permanent Variables

```
# Add to ~/.bashrc for user-specific variables
echo 'export MY_APP_PATH="/opt/myapp"' >> ~/.bashrc

# Add to /etc/environment for system-wide variables (Ubuntu/Debian)
echo 'MY_GLOBAL_VAR="value"' | sudo tee -a /etc/environment

# Add to /etc/profile for system-wide variables (all systems)
echo 'export GLOBAL_PATH="/usr/local/custom"' | sudo tee -a /etc/profile
```

Unsetting Variables

```
# Remove environment variable
$ unset MY_VAR
$ echo $MY_VAR
# (empty output)

# Remove from environment but keep as shell variable
$ export -n MY_VAR
```

Shell Variables vs Environment Variables

Shell Variables

Only available in the current shell:

```
# Shell variable (not exported)
$ my_shell_var="local value"
$ echo $my_shell_var
local value

# Start subshell - variable not available
$ bash
$ echo $my_shell_var
# (empty)
$ exit
```

Environment Variables

Available to child processes:

```
# Environment variable (exported)
$ export my_env_var="global value"
$ echo $my_env_var
global value

# Start subshell - variable is available
$ bash
$ echo $my_env_var
global value
$ exit
```

Special Variables

Bash provides several special variables with predefined meanings:

Positional Parameters

```
#!/bin/bash
# script_args.sh
echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "All arguments: $@"
echo "Number of arguments: $#"
echo "All arguments as single string: $*"
```

Process Variables

```
#!/bin/bash
echo "Current process ID: $$"
echo "Parent process ID: $PPID"
echo "Last background process ID: $!"
echo "Exit status of last command: $?"
```

Test Special Variables

```
$ ./script_args.sh hello world test
Script name: ./script_args.sh
First argument: hello
Second argument: world
All arguments: hello world test
Number of arguments: 3
All arguments as single string: hello world test
```

Configuration Files

Bash reads various configuration files at startup, depending on how it's invoked:

Login Shell Files (in order)

1. `/etc/profile` - System-wide login settings
2. `~/.bash_profile` - User login settings
3. `~/.bash_login` - Alternative user login settings
4. `~/.profile` - POSIX-compatible user settings

Non-Login Interactive Shell Files

1. `/etc/bash.bashrc` - System-wide interactive settings
2. `~/.bashrc` - User interactive settings

Non-Interactive Shell Files

- Uses `$BASH_ENV` if set

Logout Files

- `~/.bash_logout` - Executed when login shell exits

Configuration File Examples

~/.bashrc

```
# ~/.bashrc - User interactive shell configuration

# Source global definitions
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi

# User specific aliases and functions
alias ll='ls -la'
alias la='ls -A'
alias l='ls -CF'

# Custom prompt
PS1='\u@\h:\w\$ '

# Environment variables
export EDITOR=vim
export PAGER=less

# Add personal bin to PATH
if [ -d "$HOME/bin" ]; then
    PATH="$HOME/bin:$PATH"
fi

# Custom functions
mkcd() {
    mkdir -p "$1" && cd "$1"
}

# History settings
HISTSIZE=1000
HISTFILESIZE=2000
HISTCONTROL=ignoredups
```

~/.bash_profile

```
# ~/.bash_profile - User login shell configuration

# Get the aliases and functions
if [ -f ~/.bashrc ]; then
    . ~/.bashrc
```

```
fi

# User specific environment and startup programs
export PATH="$HOME/.local/bin:$PATH"

# Start SSH agent
if [ -z "$SSH_AUTH_SOCK" ]; then
    eval $(ssh-agent -s)
fi

# Welcome message
echo "Welcome, $USER!"
echo "Today is $(date)"
```

Prompt Customization

The shell prompt is controlled by the `PS1` variable:

Basic Prompt Variables

```
# Default prompt
PS1='\u@\h:\w\$ '
# Results in: user@hostname:/current/path$

# Prompt escape sequences
\u  # Username
\h  # Hostname (short)
\H  # Hostname (full)
\w  # Current working directory
\W  # Basename of current directory
\d  # Date
\t  # Time (24-hour)
\T  # Time (12-hour)
\$  # $ for regular user, # for root
```

Colorized Prompt

```
# Colors
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
```

```
NC='\033[0m' # No Color

# Colorized prompt
PS1="${GREEN}\u@\h${NC}:${BLUE}\w${NC}\$ "
```

Advanced Prompt with Git Status

```
# Function to show git branch
git_branch() {
    git branch 2>/dev/null | grep '^*' | colrm 1 2
}

# Prompt with git branch
PS1='\u@\h:\w$(git_branch)\$ '
```

PATH Management

The PATH variable determines where the shell looks for executable files:

Viewing PATH

```
# Display PATH
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin:/sbin

# Display PATH in readable format
$ echo $PATH | tr ':' '\n'
/usr/local/bin
/usr/bin
/bin
/usr/local/sbin
/usr/sbin
/sbin
```

Modifying PATH

```
# Add directory to beginning of PATH
export PATH="/new/directory:$PATH"

# Add directory to end of PATH
export PATH="$PATH:/new/directory"
```

```
# Add multiple directories
export PATH="/dir1:/dir2:$PATH:/dir3"
```

PATH Best Practices

```
# Check if directory exists before adding
if [ -d "$HOME/bin" ]; then
    export PATH="$HOME/bin:$PATH"
fi

# Remove duplicate entries
PATH=$(echo "$PATH" | awk -v RS=':' -v ORS=":" '!a[$1]++{if (NR > 1) printf ORS; printf $a[$
```

Shell Options and Settings

Set Options

```
# Display all set options
$ set -o

# Enable options
set -o errexit      # Exit on error
set -o nounset      # Exit on undefined variable
set -o pipefail     # Exit on pipe failure

# Disable options
set +o errexit      # Disable exit on error

# Short form
set -e              # Same as set -o errexit
set -u              # Same as set -o nounset
set -x              # Enable debug mode
```

Shopt Options

```
# Display all shopt options
$ shopt

# Enable options
shopt -s extglob    # Extended globbing
shopt -s globstar   # Recursive globbing (**)
```

```
shopt -s nocaseglob # Case-insensitive globbing

# Disable options
shopt -u extglob    # Disable extended globbing
```

Environment Management Scripts

Environment Setup Script

```
#!/bin/bash
# setup_env.sh - Development environment setup

echo "Setting up development environment..."

# Java environment
export JAVA_HOME="/usr/lib/jvm/java-11-openjdk"
export PATH="$JAVA_HOME/bin:$PATH"

# Node.js environment
export NODE_PATH="/usr/local/lib/node_modules"
export PATH="/usr/local/node/bin:$PATH"

# Python environment
export PYTHONPATH="/usr/local/lib/python3.9/site-packages:$PYTHONPATH"

# Custom application paths
export APP_HOME="/opt/myapp"
export PATH="$APP_HOME/bin:$PATH"

# Development tools
export EDITOR=vim
export BROWSER=firefox

echo "Environment setup complete!"
echo "Java version: $(java -version 2>&1 | head -1)"
echo "Node version: $(node --version 2>/dev/null || echo 'Not installed')"
echo "Python version: $(python3 --version)"
```

Environment Backup and Restore

```
#!/bin/bash
# env_backup.sh - Backup current environment

BACKUP_FILE="env_backup_$(date +%Y%m%d_%H%M%S).sh"

echo "#!/bin/bash" > "$BACKUP_FILE"
echo "# Environment backup created on $(date)" >> "$BACKUP_FILE"
echo "" >> "$BACKUP_FILE"

# Backup important variables
for var in PATH HOME USER SHELL LANG; do
    echo "export $var='${!var}'" >> "$BACKUP_FILE"
done

echo "Environment backed up to $BACKUP_FILE"
```

Debugging Environment Issues

Common Environment Problems

```
# Check if command is found
$ which python3
/usr/bin/python3

$ type python3
python3 is /usr/bin/python3

# Check if variable is set
$ [ -z "$JAVA_HOME" ] && echo "JAVA_HOME not set"

# Debug PATH issues
$ echo $PATH | grep -o '[^:]*' | while read dir; do
    [ -d "$dir" ] || echo "Directory not found: $dir"
done
```

Environment Diagnostic Script

```
#!/bin/bash
# env_diagnostic.sh - Diagnose environment issues

echo "=== Environment Diagnostic ==="
```

```
echo "Date: $(date)"
echo "User: $USER"
echo "Shell: $SHELL"
echo "Bash Version: $BASH_VERSION"
echo

echo "=== PATH Analysis ==="
echo "PATH has $(echo $PATH | tr ':' '\n' | wc -l) directories"
echo $PATH | tr ':' '\n' | while read dir; do
    if [ -d "$dir" ]; then
        echo "  $dir"
    else
        echo "  $dir (not found)"
    fi
done
echo

echo "=== Important Variables ==="
for var in HOME USER SHELL LANG EDITOR PAGER; do
    echo "$var: ${!var:-'(not set)'}"
done
```

Understanding and properly managing the Bash environment is essential for creating robust scripts and maintaining a productive development environment. The environment affects everything from command execution to script behavior, making it a fundamental aspect of shell scripting.

Bash Features

Bash Features

Bash Features

Bash offers a rich set of features that make it powerful for both interactive use and scripting. Understanding these features will help you write more efficient and effective scripts.

Interactive Features

Command History

Bash maintains a history of commands you've executed:

```
# View command history
$ history
1001  ls -la
1002  cd /home
1003  pwd
1004  echo "hello world"
1005  history

# Execute previous command
$ !!

# Execute command by number
$ !1003

# Search history
$ !pw
pwd

# Reverse search (Ctrl+R)
(reverse-i-search)`ls': ls -la
```

History Configuration

```
# In ~/.bashrc
HISTSIZE=1000          # Commands in memory
HISTFILESIZE=2000      # Commands in history file
HISTCONTROL=ignoredups # Ignore duplicate commands
```

```
# Don't save certain commands
HISTIGNORE="ls:pwd:exit:clear"
```

Tab Completion

Bash provides intelligent tab completion:

```
# Complete commands
$ ec<TAB>
echo

# Complete file names
$ ls /usr/b<TAB>
bin/

# Complete variable names
$ echo $HO<TAB>
$HOME

# Complete options
$ ls --<TAB><TAB>
--all                     --escape
--almost-all              --file-type
--author                  --format
```

Custom Completion

```
# Add to ~/.bashrc
complete -W "start stop restart status" service
complete -d cd  # Only directories for cd
```

Command Line Editing

Bash uses readline library for command editing:

```
# Cursor movement
Ctrl+A    # Beginning of line
Ctrl+E    # End of line
Ctrl+B    # Back one character
Ctrl+F    # Forward one character
Alt+B     # Back one word
Alt+F     # Forward one word
```

```
# Editing
Ctrl+D    # Delete character under cursor
Ctrl+H    # Delete character before cursor
Ctrl+K    # Kill from cursor to end of line
Ctrl+U    # Kill from cursor to beginning of line
Ctrl+W    # Kill word before cursor
Alt+D     # Kill word after cursor

# History navigation
Ctrl+P    # Previous command
Ctrl+N    # Next command
Ctrl+R    # Reverse search
```

Job Control

Bash provides sophisticated job control capabilities:

Background Jobs

```
# Run command in background
$ long_running_command &
[1] 12345

# List active jobs
$ jobs
[1]+  Running    long_running_command &

# Bring job to foreground
$ fg %1

# Send job to background
$ bg %1
```

Job Management

```
#!/bin/bash
# job_control_demo.sh

echo "Starting background jobs..."

# Start multiple background jobs
sleep 30 &
```

```
job1=$!

sleep 40 &
job2=$!

echo "Job 1 PID: $job1"
echo "Job 2 PID: $job2"

# Wait for specific job
wait $job1
echo "Job 1 completed"

# Wait for all background jobs
wait
echo "All jobs completed"
```

Process Control

```
# Suspend current process
Ctrl+Z

# Kill current process
Ctrl+C

# Send signals to processes
$ kill -TERM 12345
$ kill -KILL 12345
$ killall process_name
```

Aliases and Functions

Aliases

Simple command shortcuts:

```
# Define aliases
alias ll='ls -la'
alias la='ls -A'
alias l='ls -CF'
alias grep='grep --color=auto'

# Use aliases
$ ll
total 24
```

```
drwxr-xr-x  3 user user 4096 Jan 15 10:30 .
drwxr-xr-x 25 user user 4096 Jan 15 10:25 ..

# List all aliases
$ alias

# Remove alias
$ unalias ll
```

Functions

More complex reusable code:

```
# Simple function
greet() {
    echo "Hello, $1!"
}

# Function with multiple parameters
backup_file() {
    local file="$1"
    local backup_dir="$2"

    if [ -f "$file" ]; then
        cp "$file" "$backup_dir/$(basename "$file").bak"
        echo "Backed up $file"
    else
        echo "File $file not found"
        return 1
    fi
}

# Use functions
$ greet "World"
Hello, World!

$ backup_file "/etc/hosts" "/backup"
Backed up /etc/hosts
```

Variable Features

Variable Types

```
# Regular variables
name="John"
age=25

# Arrays
fruits=("apple" "banana" "orange")
echo ${fruits[0]}  # apple
echo ${fruits[@]}  # all elements

# Associative arrays (Bash 4+)
declare -A colors
colors[red]="#FF0000"
colors[green]="#00FF00"
echo ${colors[red]}  # #FF0000
```

Parameter Expansion

```
filename="document.txt"

# Basic expansion
echo $filename          # document.txt
echo ${filename}        # document.txt

# Length
echo ${#filename}       # 12

# Substring
echo ${filename:0:8}    # document

# Remove from end
echo ${filename%.*}     # document

# Remove from beginning
echo ${filename##*.}    # txt

# Replace
echo ${filename/txt/pdf}  # document.pdf
```

Default Values

```
# Use default if variable is unset
echo ${name:-"Anonymous"}

# Set default if variable is unset
echo ${name:="Default Name"}

# Error if variable is unset
echo ${name:?"Variable name is required"}

# Use alternative if variable is set
echo ${name:+"Name is set"}
```

Pattern Matching and Globbing

Basic Wildcards

```
# Match any characters
$ ls *.txt
file1.txt  file2.txt  document.txt

# Match single character
$ ls file?.txt
file1.txt  file2.txt

# Match character ranges
$ ls file[1-3].txt
file1.txt  file2.txt  file3.txt

# Match character sets
$ ls file[abc].txt
filea.txt  fileb.txt  filec.txt
```

Extended Globbing

```
# Enable extended globbing
shopt -s extglob

# Match one of several patterns
$ ls *.@(txt|pdf|doc)

# Match zero or one occurrence
```

```
$ ls file?(s).txt

# Match zero or more occurrences
$ ls file*(s).txt

# Match one or more occurrences
$ ls file+(s).txt

# Match anything except pattern
$ ls !(*.tmp)
```

Globstar (Bash 4+)

```
# Enable globstar
shopt -s globstar

# Recursive matching
$ ls **/*.txt
dir1/file1.txt
dir1/subdir/file2.txt
dir2/file3.txt
```

Brace Expansion

Sequence Generation

```
# Number sequences
$ echo {1..10}
1 2 3 4 5 6 7 8 9 10

$ echo {01..10}
01 02 03 04 05 06 07 08 09 10

# Letter sequences
$ echo {a..z}
a b c d e f g h i j k l m n o p q r s t u v w x y z

# Step sequences (Bash 4+)
$ echo {1..10..2}
1 3 5 7 9
```

String Expansion

```
# Multiple strings
$ echo {cat,dog,bird}
cat dog bird

# Combinations
$ echo {a,b}{1,2}
a1 a2 b1 b2

# File operations
$ cp file.txt{,.bak}  # Same as: cp file.txt file.txt.bak
$ mkdir -p project/{src,docs,tests}
```

Process Substitution

Input Substitution

```
# Compare output of two commands
$ diff <(ls dir1) <(ls dir2)

# Use command output as input
$ while read line; do
    echo "Processing: $line"
done < <(find /var/log -name "*.log")
```

Output Substitution

```
# Send output to multiple commands
$ echo "data" | tee >(command1) >(command2)

# Log to file and display
$ make 2>&1 | tee >(logger -t build)
```

Command Substitution

Modern Syntax

```
# Preferred method
current_date=$(date)
file_count=$(ls | wc -l)
user_home=$(eval echo ~$USER)

echo "Today is $current_date"
echo "Files in directory: $file_count"
```

Legacy Syntax

```
# Older method (still works)
current_date=`date`
file_count=`ls | wc -l`
```

Nested Substitution

```
# Complex nested commands
backup_name="backup_$(date +%Y%m%d)_$(hostname).tar.gz"
echo $backup_name  # backup_20240115_myserver.tar.gz
```

Arithmetic Operations

Arithmetic Expansion

```
# Basic arithmetic
result=$((5 + 3))
echo $result  # 8

# Variables in arithmetic
a=10
b=5
echo $((a + b))   # 15
echo $((a * b))   # 50
echo $((a / b))   # 2
echo $((a % b))   # 0
```

Arithmetic Commands

```
# let command
let "result = 5 + 3"
echo $result  # 8

# expr command (external)
result=$(expr 5 + 3)
echo $result  # 8
```

Advanced Arithmetic

```
# Increment/decrement
counter=0
((counter++))
echo $counter  # 1

((counter += 5))
echo $counter  # 6

# Comparison in arithmetic
if ((counter > 5)); then
    echo "Counter is greater than 5"
fi
```

Conditional Expressions

Test Command

```
# File tests
if [ -f "file.txt" ]; then
    echo "File exists"
fi

# String tests
if [ "$name" = "John" ]; then
    echo "Hello John"
fi

# Numeric tests
if [ $age -gt 18 ]; then
    echo "Adult"
fi
```

Extended Test

```
# Bash extended test
if [[ $name == "John" ]]; then
    echo "Hello John"
fi

# Pattern matching
if [[ $filename == *.txt ]]; then
    echo "Text file"
fi

# Regular expressions
if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
    echo "Valid email"
fi
```

I/O Redirection

Basic Redirection

```
# Redirect stdout
$ echo "Hello" > file.txt

# Redirect stderr
$ command 2> error.log

# Redirect both
$ command > output.log 2>&1
$ command &> output.log  # Bash shortcut
```

Advanced Redirection

```
# Append to file
$ echo "More data" >> file.txt

# Here documents
$ cat << EOF > config.txt
Setting1=value1
Setting2=value2
EOF

# Here strings
```

```
$ grep "pattern" <<< "$variable"
```

Shell Options

Set Options

```
# Exit on error
set -e

# Exit on undefined variable
set -u

# Exit on pipe failure
set -o pipefail

# Combine options
set -euo pipefail
```

Shopt Options

```
# Enable extended globbing
shopt -s extglob

# Enable case-insensitive globbing
shopt -s nocaseglob

# Enable recursive globbing
shopt -s globstar

# List all options
shopt
```

These features make Bash a powerful and flexible shell for both interactive use and scripting. Understanding and utilizing these features will significantly improve your productivity and the quality of your scripts.

What is Bash?

What is Bash?

What is Bash?

Bash (Bourne Again Shell) is the most widely used shell in Linux and Unix systems. It's an enhanced version of the original Bourne shell (sh) with many additional features that make it powerful for both interactive use and scripting.

History and Background

The Evolution

1. **Bourne Shell (sh)** - 1977: Original Unix shell by Stephen Bourne
2. **C Shell (csh)** - 1978: Added command history and job control
3. **Korn Shell (ksh)** - 1983: Combined features of sh and csh
4. **Bash** - 1989: GNU's "Bourne Again Shell" by Brian Fox

Why "Bourne Again"?

- **Pun**: Play on "born again" (reborn)
- **Compatibility**: Maintains compatibility with original Bourne shell
- **Enhancement**: Adds modern features while preserving classic functionality

Bash Characteristics

POSIX Compliance

Bash is largely POSIX-compliant, meaning scripts written for POSIX shells should work in Bash:

```
#!/bin/sh
# This POSIX script works in bash
echo "Hello, World!"
if [ -f "/etc/passwd" ]; then
    echo "Password file exists"
fi
```

GNU Project

- **Free Software**: Part of the GNU operating system
- **Open Source**: Source code freely available
- **Community Driven**: Developed and maintained by volunteers

Bash Versions

Check Your Bash Version

```
$ bash --version
GNU bash, version 5.1.16(1)-release (x86_64-pc-linux-gnu)

# Or from within bash
$ echo $BASH_VERSION
5.1.16(1)-release

# Short version
$ echo ${BASH_VERSION%%.*}
5
```

Major Version Differences

Bash 3.x (2004-2009)

- Regular expressions in conditional expressions
- Process substitution improvements

Bash 4.x (2009-2020)

- Associative arrays
- Case modification operators
- Globstar (**) pattern matching

Bash 5.x (2019-present)

- Improved performance
- New variable expansion features
- Enhanced debugging capabilities

Version-Specific Features

```
#!/bin/bash

# Check bash version for feature compatibility
if [ "${BASH_VERSION%%.*}" -ge 4 ]; then
    # Bash 4+ features
    declare -A assoc_array
    assoc_array[key]="value"
    echo "Using associative arrays"
else
    # Fallback for older bash
    echo "Using indexed arrays"
fi
```

Bash vs Other Shells

Bash vs Bourne Shell (sh)

```
# Bash-specific features not in sh
echo ${BASH_VERSION}        # Bash version variable
echo ${!var*}               # Variable name expansion
declare -A array            # Associative arrays
[[ $var =~ regex ]]         # Regular expression matching
```

Bash vs Zsh

```
# Bash
$ echo $SHELL
/bin/bash

# Zsh has more advanced features
$ echo $SHELL
/usr/bin/zsh
```

Bash vs Fish

```
# Bash syntax
if [ $status -eq 0 ]; then
    echo "Success"
fi
```

```
# Fish syntax (different)
if test $status -eq 0
    echo "Success"
end
```

Where Bash is Used

Default Shell

Bash is the default shell on most Linux distributions:

```
# Check default shell
$ echo $SHELL
/bin/bash

# Check available shells
$ cat /etc/shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
```

System Scripts

Many system scripts use Bash:

```
# System initialization scripts
$ head -1 /etc/init.d/ssh
#!/bin/bash

# Package management scripts
$ head -1 /usr/bin/apt-get
#!/bin/bash
```

Development and DevOps

```
# Build scripts
#!/bin/bash
make clean
make all
```

```
make install

# Deployment scripts
#!/bin/bash
git pull origin main
npm install
npm run build
systemctl restart myapp
```

Bash Capabilities

Interactive Features

```
# Command history
$ history | tail -5
1001  ls -la
1002  cd /home
1003  pwd
1004  echo "hello"
1005  history | tail -5

# Tab completion
$ ls /usr/b<TAB>
bin/

# Command editing
# Use arrow keys, Ctrl+A (beginning), Ctrl+E (end)
```

Programming Features

```
#!/bin/bash

# Variables
name="John"
age=25

# Arrays
fruits=("apple" "banana" "orange")

# Functions
greet() {
    echo "Hello, $1!"
}
```

```
# Control structures
if [ $age -gt 18 ]; then
    echo "Adult"
fi

# Loops
for fruit in "${fruits[@]}"; do
    echo "Fruit: $fruit"
done
```

Advanced Features

```
#!/bin/bash

# Process substitution
diff <(ls dir1) <(ls dir2)

# Parameter expansion
filename="document.txt"
echo ${filename%.*}     # document
echo ${filename##*.}    # txt

# Regular expressions
if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
    echo "Valid email"
fi
```

Bash Configuration

Configuration Files

```
# System-wide configuration
/etc/bash.bashrc        # All users
/etc/profile           # Login shells

# User-specific configuration
~/.bashrc              # Interactive non-login shells
~/.bash_profile        # Login shells
~/.bash_logout         # Logout cleanup
```

Customization Example

```
# ~/.bashrc
# Custom prompt
PS1='\u@\h:\w\$ '

# Aliases
alias ll='ls -la'
alias grep='grep --color=auto'

# Functions
mkcd() {
    mkdir -p "$1" && cd "$1"
}

# Environment variables
export EDITOR=vim
export PATH="$HOME/bin:$PATH"
```

Bash Built-in Commands

Bash includes many built-in commands for efficiency:

```
# File operations
$ type cd
cd is a shell builtin

$ type echo
echo is a shell builtin

# List all built-ins
$ help
# or
$ compgen -b
```

Common Built-ins

```
# Navigation
cd /path/to/directory
pwd

# Variables
export VAR=value
unset VAR
```

```
# Control
exit 0
return 1

# Information
type command
which command
help command
```

Bash Scripting Advantages

1. Ubiquity

- Available on virtually all Unix-like systems
- Default shell on most Linux distributions
- Consistent behavior across platforms

2. Integration

- Seamless integration with system commands
- Easy to call external programs
- Natural for system administration tasks

3. Learning Curve

- Familiar to anyone who uses the command line
- Gradual learning from simple commands to complex scripts
- Extensive documentation and community support

4. Power and Flexibility

```
#!/bin/bash
# Powerful one-liners
find /var/log -name "*.log" -mtime +7 -exec rm {} \;

# Complex data processing
awk '/ERROR/ {count++} END {print "Errors:", count}' logfile.txt

# System monitoring
ps aux | awk '$3 > 80 {print $2, $11}' | head -10
```

When to Use Bash

Perfect For:

- **System administration**: User management, backups, monitoring
- **DevOps tasks**: Deployment, CI/CD pipelines
- **File processing**: Batch operations, log analysis
- **Glue scripts**: Connecting different tools and programs
- **Quick automation**: Tasks that don't require complex logic

Consider Alternatives For:

- **Complex applications**: Use Python, Go, or other languages
- **Performance-critical tasks**: Compiled languages are faster
- **Cross-platform GUI apps**: Use appropriate frameworks
- **Heavy mathematical computations**: Use specialized tools

Getting Started with Bash

Check if Bash is Available

```
$ which bash
/bin/bash

$ bash --version
GNU bash, version 5.1.16(1)-release
```

Start Using Bash

```
# If not your default shell
$ bash

# Make bash your default shell
$ chsh -s $(which bash)
```

First Bash Script

```
#!/bin/bash
echo "Welcome to Bash scripting!"
echo "Bash version: $BASH_VERSION"
echo "Today is: $(date)"
```

Bash is more than just a command interpreter—it's a powerful programming environment that bridges the gap between simple command-line operations and full-featured programming languages. Its widespread adoption, extensive features, and excellent documentation make it an essential tool for anyone working with Linux or Unix systems.

The Bash Execution Model

How Bash Reads and Parses Input

How Bash Reads and Parses Input

Understanding the Bash execution model begins with lexical analysis and parsing. When Bash receives a command line, it does not execute it immediately. Instead, it processes the text through a series of discrete phases:

1. **Reading and Tokenization:** Bash splits the input stream into tokens (words and operators) based on metacharacters (`space`, `tab`, `newline`, `;`, `&`, `(`, `)`, `|`, `<`, `>`).
2. **Grammar Parsing:** Identifies command structures, pipelines, control flow constructs (`if`, `while`, `for`, `case`), and reserved words.
3. **Compound Expansions:** Executes parameter, command, arithmetic, and pathname expansions in a strict predefined sequence.
4. **Command Execution:** Locates the executable (special built-in, function, alias, regular built-in, or external binary via `$PATH`) and forks/execs.

Builtin vs. Keyword vs. External Binary

Bash treats commands in a strict lookup priority:

```
# 1. Alias (only in interactive shells by default)
alias myls='ls -la'

# 2. Shell Keyword (part of the language grammar)
type if
# if is a shell keyword

# 3. Shell Function (user-defined routines)
type myfunc

# 4. Shell Builtin (executed directly inside the shell process, zero fork cost)
type cd
# cd is a shell builtin

# 5. External Binary (found via PATH lookup in the filesystem)
type grep
# grep is /usr/bin/grep
```

The Expansion Order: A Critical Sequence

The Expansion Order: A Critical Sequence

The most common source of elusive Bash bugs is misunderstanding the **Expansion Order**. Bash evaluates expansions strictly in this sequence:

1. **Brace Expansion:** `{a,b,c}` and `{1..5}`
2. **Tilde Expansion:** `~` and `~user`
3. **Parameter & Variable Expansion:** `$VAR` and `${VAR:-default}`
4. **Arithmetic Expansion:** `$(( 2 + 2 ))`
5. **Command Substitution:** `$(command)`
6. **Process Substitution:** `<(command)` and `>(command)`
7. **Word Splitting:** Splits results of unquoted expansions on `$IFS` characters
8. **Pathname Expansion (Globbing):** `*.txt`, `[a-z]*`
9. **Quote Removal:** Strips unquoted single quotes, double quotes, and backslashes

Why Order Matters: The Variable in Brace Pitfall

```
# This DOES NOT WORK because Brace Expansion happens BEFORE Parameter Expansion:
count=5
echo {1..$count}
# Output: {1..5} (literal text, not 1 2 3 4 5!)

# Solution: Use standard C-style loops or seq:
for ((i=1; i<=count; i++)); do
    echo "$i"
done
```

Word Splitting and Pathname Expansion

Word Splitting and Pathname Expansion

After parameter expansion and command substitution, Bash performs **Word Splitting** on unquoted words using the Internal Field Separator (`$IFS`). By default, `$IFS` is set to space, tab, and newline.

The Dangers of Word Splitting

```
filename="My Important Document.pdf"

# DANGEROUS: Word splitting breaks filename into three arguments: "My", "Important", "Docume
rm $filename
# rm: cannot remove 'My': No such file or directory
# rm: cannot remove 'Important': No such file or directory
# rm: cannot remove 'Document.pdf': No such file or directory

# SAFE: Always double-quote variables
rm "$filename"
```

Controlling Globbing (Pathname Expansion)

Globbing matches filesystem patterns. When no match is found, Bash's default behavior is to return the unexpanded pattern as literal text, which can cause subtle failures:

```
# Enable nullglob: unmatched globs expand to nothing instead of literal string
shopt -s nullglob

# Enable failglob: unmatched globs throw a hard error
shopt -s failglob

# Disable globbing entirely
set -f  # or set -o noglob
```

Subshells and Process Forking

Subshells and Process Forking

A **subshell** is a separate child process spawned by the current shell that inherits environment variables, open file descriptors, and shell options, but cannot modify the parent shell's memory space.

When Subshells Are Created

- Parentheses grouping: `( cd /tmp && rm -rf build )`
- Command substitution: `output=$(grep "ERROR" /var/log/syslog)`
- Pipelines: `cat file.txt | while read -r line; do count=$((count+1)); done`

The Pipeline Subshell Trap

```
count=0
cat << 'EOF' > test.txt
alpha
beta
gamma
EOF

# In standard Bash, every component of a pipeline executes in a subshell!
cat test.txt | while read -r line; do
    count=$((count + 1))
done

echo "Final Count: $count"
# Output: Final Count: 0 (The parent count was never modified!)

# Solution 1: Process Substitution
count=0
while read -r line; do
    count=$((count + 1))
done < <(cat test.txt)
echo "Final Count: $count"  # Output: 3

# Solution 2: Enable lastpipe in non-interactive scripts
shopt -s lastpipe
set +m
cat test.txt | while read -r line; do count=$((count + 1)); done
```

```
echo "Final Count: $count"  # Output: 3
```

Interactive vs Non-Interactive Shells

Interactive vs Non-Interactive Shells

Bash operates differently depending on whether it is running interactively (attached to a terminal prompt) or executing a non-interactive automation script:

Feature	Interactive Shell	Non-Interactive Script
`$-` flag string	Contains `i`	Lacks `i`
Aliases	Enabled by default	Disabled (must `shopt -s expand_aliases`)
Job control (`set -m`)	Enabled	Disabled by default
Config files loaded	`~/.bashrc`	Non-login loads `$BASH_ENV`
History expansion (`!`)	Enabled	Disabled

```
# Check if running interactively
if [[ $- == *i* ]]; then
    echo "Running interactively"
else
    echo "Running non-interactively in automation"
fi
```

Variables, Quoting, and Safe Syntax

The Three Types of Quotes

The Three Types of Quotes

Proper quoting is the single most critical defensive skill in Bash. Bash supports three quoting mechanisms, each with distinct semantic rules:

1. Single Quotes (`'...'`)

Single quotes preserve the literal value of every character within the quotes. No expansions occur—no parameter substitution, no command substitution, no arithmetic expansion, and no escape sequences.

```
# Literal string preservation
name="Alice"
echo 'Hello, $name! Today is $(date).'
# Output: Hello, $name! Today is $(date).
```

Note: You cannot include a single quote inside single quotes, even with a backslash (`'` does not work). To include a literal single quote, close the quote, insert an escaped quote, and reopen: `'Don'''t touch this'`.

2. Double Quotes (`"..."`)

Double quotes suppress word splitting and globbing, but allow: * Parameter expansion (`$VAR`, `${VAR}`) * Command substitution (`$(command)`) * Arithmetic expansion (`$(( ... ))`) * Escaped dollar signs (`\$`), backticks (`` \` ``), double quotes (`"`), and backslashes (`\`).

```
greeting="Hello"
user="Bob"
echo "$greeting, $user! PID is $$"
# Output: Hello, Bob! PID is 1234
```

3. ANSI-C Quoting (`$'...'`)

ANSI-C quoting expands C-style backslash escape sequences into literal binary characters before execution:

```
# Newlines, tabs, and hex characters
newline_string=$'Line 1\nLine 2\tIndented'
echo "$newline_string"

# Carriage return in terminal status lines
printf $'Downloading...\r'
```

Parameter Expansion and Default Values

Parameter Expansion and Default Values

Parameter expansion allows you to inspect, modify, and assign fallback values to variables without invoking external tools like `sed` or `awk`.

Default Values and Fallbacks

```
# Use default if unset or empty: ${VAR:-default}
user_input=""
username="${user_input:-guest}"
echo "$username" # guest

# Assign default if unset or empty: ${VAR:=default}
# (Note: Does not work on positional parameters like $1)
${DATABASE_HOST:="localhost"}

# Throw error and exit if unset or empty: ${VAR:?error_message}
: "${CONFIG_FILE:?Error: CONFIG_FILE environment variable must be set}"

# Use alternate value if set and non-empty: ${VAR:+alternate}
${DEBUG:+"--verbose"}
```

String Slicing and Length

```
text="LinuxDistribution"

# String length: ${#VAR}
echo "${#text}" # 17

# Substring extraction: ${VAR:offset:length}
echo "${text:0:5}"   # Linux
echo "${text:5}"     # Distribution
echo "${text: -4}"   # tion (note the space before negative offset)
```

Pattern Stripping and Substitution

```
filepath="/var/log/nginx/access.log.tar.gz"

# Strip shortest match from front: ${VAR#pattern}
echo "${filepath#*/}"     # var/log/nginx/access.log.tar.gz

# Strip longest match from front: ${VAR##pattern} (equivalent to basename)
echo "${filepath##*/}"    # access.log.tar.gz

# Strip shortest match from back: ${VAR%pattern}
echo "${filepath%.*}"     # /var/log/nginx/access.log.tar

# Strip longest match from back: ${VAR%%pattern}
echo "${filepath%%.*}"    # /var/log/nginx/access

# Replace first match: ${VAR/pattern/replacement}
echo "${filepath/log/archive}"

# Replace all matches: ${VAR//pattern/replacement}
echo "${filepath//\//_}"  # _var_log_nginx_access.log.tar.gz
```

Special Parameters: Positional, IFS, and PATH

Special Parameters: Positional, IFS, and PATH

Bash reserves single-character parameters for tracking runtime state, command-line arguments, and environment boundaries:

Positional Arguments and $@ vs $*

Parameter	Meaning	Quoted Expansion
`$0`	Name of the script or shell	`"$0"`
`$1 .. $9`	Individual positional arguments	`"$1"`
`${10}`	Arguments beyond 9 require braces	`"${10}"`
`$#`	Number of positional arguments	`"$#"`
`$@`	All arguments as individual words	`"$@"` expands to `"$1" "$2" "$3"` (Safe)
`$*`	All arguments concatenated	`"$*"` expands to `"$1 $2 $3"` joined by `$IFS`

```
# ALWAYS use "$@" when iterating over command-line arguments
for arg in "$@"; do
    echo "Processing: $arg"
done
```

Process & Runtime Parameters

- `$?`: Exit status of the most recently executed foreground pipeline (0 = success).
- `$$`: Process ID (PID) of the current shell.
- `$!`: Process ID of the most recently spawned background process.
- `$_`: The last argument of the previous command executed.

Common Quoting Pitfalls and Safe Syntax

Common Quoting Pitfalls and Safe Syntax

Failing to quote variables correctly is the source of CVE vulnerabilities and catastrophic production outages.

Pitfall 1: Unquoted Variables in Commands

```
# Catastrophic error if target_dir is empty or contains spaces
target_dir="/tmp/demo build"
rm -rf $target_dir
# rm -rf /tmp/demo build -> Deletes both /tmp/demo AND current directory 'build'!

# Safe:
rm -rf "$target_dir"
```

Pitfall 2: Unquoted Test Conditions in [ ]

```
# If $user is empty, this evaluates to: [ = "admin" ] -> syntax error!
[ $user = "admin" ]

# Safe POSIX:
[ "$user" = "admin" ]

# Safe Bash:
[[ $user == "admin" ]]
```

Pitfall 3: Storing Commands in Variables

```
# WRONG: Trying to store complex commands with flags or quotes in a variable
cmd="grep -E 'ERROR|FATAL' /var/log/syslog"
$cmd # Fails because quotes inside variables are not parsed after expansion!

# CORRECT: Use a bash function or array
run_search() {
    grep -E 'ERROR|FATAL' /var/log/syslog
```

```
}

# Or array:
cmd=(grep -E "ERROR|FATAL" "/var/log/syslog")
"${cmd[@]}"
```

Variable Basics

Variable Basics

Variable Basics

Variables are fundamental building blocks in Bash scripting. They allow you to store data, make scripts flexible, and create reusable code. Understanding how to work with variables effectively is essential for writing robust shell scripts.

What are Variables?

Variables are named storage locations that hold data. In Bash, variables can store: - Text strings - Numbers (treated as strings) - Command output - File paths - Configuration values

Variable Declaration and Assignment

Basic Syntax

```
# Variable assignment (no spaces around =)
variable_name="value"

# Examples
name="John Doe"
age=25
city="New York"
is_active=true
```

Important Rules

1. **No spaces** around the equals sign
2. **Case sensitive**: `name` and `Name` are different variables
3. **No type declaration** needed
4. **Quote values** with spaces or special characters

Common Mistakes

```
# Wrong - spaces around =
name = "John"        # Error!

# Wrong - unquoted value with spaces
name=John Doe        # Error!

# Correct
name="John Doe"      # Correct
```

Variable Naming Conventions

Valid Variable Names

```
# Good variable names
username="john"
user_name="john"
userName="john"
USER_NAME="john"
file1="document.txt"
_private_var="secret"
```

Invalid Variable Names

```
# Invalid - cannot start with number
1name="john"         # Error!

# Invalid - contains special characters
user-name="john"     # Error!
user@name="john"     # Error!

# Invalid - reserved words
for="value"          # Error!
if="value"           # Error!
```

Best Practices

```
# Use descriptive names
config_file="/etc/myapp.conf"
max_connections=100
database_host="localhost"
```

```
# Use uppercase for constants
readonly PI=3.14159
readonly MAX_RETRIES=3

# Use lowercase for regular variables
current_user="john"
temp_directory="/tmp"
```

Accessing Variable Values

Basic Access

```
name="Alice"

# Access variable value
echo $name           # Alice
echo ${name}         # Alice (preferred for clarity)
```

When to Use Braces

```
name="Alice"

# Without braces - can be ambiguous
echo $names          # Empty (variable 'names' doesn't exist)

# With braces - clear separation
echo ${name}s        # Alices

# Concatenation examples
prefix="file"
echo ${prefix}_backup.txt    # file_backup.txt
echo $prefix_backup.txt      # Empty (looks for variable 'prefix_backup')
```

Variable Types and Behavior

String Variables

```
# Simple strings
greeting="Hello"
message="Welcome to Bash scripting"

# Strings with special characters
path="/home/user/My Documents"
command="ls -la"

# Empty strings
empty_var=""
another_empty=""
```

Numeric Variables

```
# Numbers are stored as strings but can be used in arithmetic
count=10
price=19.99
year=2024

# Arithmetic operations
total=$((count * 2))
echo $total          # 20

# Comparison
if [ $count -gt 5 ]; then
    echo "Count is greater than 5"
fi
```

Boolean-like Variables

```
# Bash doesn't have true boolean types
# Use strings to represent boolean values
is_enabled="true"
debug_mode="false"
has_permission="yes"

# Check boolean-like values
if [ "$is_enabled" = "true" ]; then
    echo "Feature is enabled"
fi
```

Variable Scope

Global Variables

```
#!/bin/bash
# Global variable - accessible everywhere
global_var="I'm global"

function test_function() {
    echo "Inside function: $global_var"
}

echo "Outside function: $global_var"
test_function
```

Local Variables

```
#!/bin/bash
function test_local() {
    local local_var="I'm local"
    global_var="Modified global"

    echo "Local variable: $local_var"
    echo "Global variable: $global_var"
}

global_var="Original global"
echo "Before function: $global_var"

test_local

echo "After function: $global_var"
echo "Local variable outside: $local_var"  # Empty
```

Reading User Input

Basic Input

```
#!/bin/bash
echo "What's your name?"
read name
echo "Hello, $name!"
```

Input with Prompt

```
#!/bin/bash
read -p "Enter your age: " age
echo "You are $age years old"
```

Multiple Variables

```
#!/bin/bash
echo "Enter first and last name:"
read first_name last_name
echo "Hello, $first_name $last_name!"
```

Reading into Array

```
#!/bin/bash
echo "Enter three colors:"
read -a colors
echo "First color: ${colors[0]}"
echo "All colors: ${colors[@]}"
```

Silent Input (Passwords)

```
#!/bin/bash
read -s -p "Enter password: " password
echo  # New line after hidden input
echo "Password length: ${#password}"
```

Variable Assignment Methods

Direct Assignment

```
name="John"
age=25
```

Command Substitution

```
# Modern syntax (preferred)
current_date=$(date)
file_count=$(ls | wc -l)
current_user=$(whoami)

# Legacy syntax (still works)
current_date=`date`
file_count=`ls | wc -l`
```

Reading from Files

```
# Read first line of file
first_line=$(head -n1 /etc/passwd)

# Read entire file content
file_content=$(cat /etc/hostname)

# Read with error handling
if config=$(cat config.txt 2>/dev/null); then
    echo "Config loaded: $config"
else
    echo "Failed to read config file"
fi
```

Arithmetic Assignment

```
# Simple arithmetic
count=5
count=$((count + 1))
echo $count          # 6

# Using let command
let count=count+1
echo $count          # 7

# Increment/decrement
((count++))
echo $count          # 8
```

Variable Manipulation

String Length

```
text="Hello World"
echo ${#text}        # 11
```

Substring Extraction

```
text="Hello World"

# Extract substring: ${variable:start:length}
echo ${text:0:5}     # Hello
echo ${text:6}       # World
echo ${text:6:3}     # Wor
echo ${text: -5}     # World (note the space before -)
```

Case Conversion

```
text="Hello World"

# Convert to uppercase (Bash 4+)
echo ${text^^}       # HELLO WORLD

# Convert to lowercase (Bash 4+)
echo ${text,,}       # hello world

# Convert first character
echo ${text^}        # Hello World
echo ${text,}        # hello World
```

Pattern Replacement

```
filename="document.txt"

# Replace first occurrence
echo ${filename/txt/pdf}     # document.pdf

# Replace all occurrences
echo ${filename//t/T}        # documenT.TxT

# Remove from beginning
```

```
echo ${filename#doc}          # ument.txt

# Remove from end
echo ${filename%.txt}         # document
```

Default Values and Error Handling

Default Values

```
# Use default if variable is unset or empty
name=${name:-"Anonymous"}
echo "Hello, $name"

# Set default if variable is unset
name=${name:="Default Name"}

# Use alternative if variable is set
message=${name:+"Hello, $name"}
```

Error on Unset Variables

```
# Exit with error if variable is unset
required_var=${required_var:?"Variable required_var must be set"}

# Custom error message
database_host=${database_host:?"Error: DATABASE_HOST environment variable is required"}
```

Practical Example

```
#!/bin/bash
# Configuration script with defaults

# Set defaults
config_file=${CONFIG_FILE:-"/etc/myapp.conf"}
log_level=${LOG_LEVEL:-"INFO"}
max_connections=${MAX_CONNECTIONS:-"100"}

echo "Configuration:"
echo "Config file: $config_file"
echo "Log level: $log_level"
echo "Max connections: $max_connections"
```

```
# Validate required variables
database_url=${DATABASE_URL:?"Error: DATABASE_URL must be set"}
api_key=${API_KEY:?"Error: API_KEY must be set"}
```

Variable Arrays (Preview)

Simple Array

```
# Create array
fruits=("apple" "banana" "orange")

# Access elements
echo ${fruits[0]}    # apple
echo ${fruits[1]}    # banana

# All elements
echo ${fruits[@]}    # apple banana orange

# Array length
echo ${#fruits[@]}   # 3
```

Best Practices

1. Always Quote Variables

```
# Good - prevents word splitting
if [ "$name" = "John Doe" ]; then
    echo "Hello, $name"
fi

# Bad - can break with spaces
if [ $name = "John Doe" ]; then
    echo "Hello, $name"
fi
```

2. Use Meaningful Names

```
# Good
database_connection_string="mysql://localhost:3306/mydb"
max_retry_attempts=3
```

```
# Bad
dcs="mysql://localhost:3306/mydb"
mra=3
```

3. Initialize Variables

```
# Good - initialize variables
count=0
total=0
error_message=""

# Process data
for file in *.txt; do
    ((count++))
    # ... processing ...
done
```

4. Use readonly for Constants

```
# Constants should be readonly
readonly PI=3.14159
readonly MAX_USERS=1000
readonly CONFIG_DIR="/etc/myapp"

# This will cause an error
# PI=3.14  # Error: PI is readonly
```

5. Validate Input

```
#!/bin/bash
read -p "Enter a number: " number

# Validate input
if [[ ! $number =~ ^[0-9]+$ ]]; then
    echo "Error: Please enter a valid number"
    exit 1
fi

echo "You entered: $number"
```

Common Variable Patterns

Configuration Variables

```
#!/bin/bash
# Application configuration

# Default configuration
APP_NAME="MyApplication"
APP_VERSION="1.0.0"
CONFIG_FILE="/etc/${APP_NAME,,}.conf"
LOG_FILE="/var/log/${APP_NAME,,}.log"
PID_FILE="/var/run/${APP_NAME,,}.pid"

# Load configuration file if it exists
if [ -f "$CONFIG_FILE" ]; then
    source "$CONFIG_FILE"
fi
```

Temporary Variables

```
#!/bin/bash
# Temporary file handling

temp_dir=$(mktemp -d)
temp_file=$(mktemp)

# Cleanup function
cleanup() {
    rm -rf "$temp_dir"
    rm -f "$temp_file"
}

# Set trap to cleanup on exit
trap cleanup EXIT

# Use temporary files
echo "Processing data..." > "$temp_file"
```

Understanding variable basics is crucial for Bash scripting. Variables make your scripts flexible, maintainable, and powerful. Practice these concepts and patterns to build a solid foundation for more advanced scripting techniques.

Control Flow and Logic

Test Commands: [ ], [[ ]], and test

Test Commands: [ ], [[ ]], and test

Conditional branching in Bash relies on test evaluation. Understanding the difference between `[ ]` (the test builtin) and `[[ ]]` (the Bash compound keyword) is essential.

Single Bracket `[ ]` (POSIX standard)

`[` is literally a command (identical to `test`), requiring closing bracket `]`. Because it is a normal command, arguments are subject to word splitting and pathname expansion:

```
# Must quote all variables to prevent syntax errors on empty strings
if [ "$name" = "root" ]; then
    echo "Superuser"
fi
```

Double Bracket `[[ ]]` (Bash Keyword)

`[[` is a shell keyword with special grammar rules: * No word splitting or pathname expansion on variable expansions inside `[[ ]]`. * Supports regular expression matching with `=~`. * Logical operators `&&` and `||` work natively without `-a` or `-o`. * Pattern matching with `==` and `!=`.

```
# Regex validation
email="dev@kanthi.in"
if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
    echo "Valid email address"
fi
```

Pattern Matching with case Statements

Pattern Matching with case Statements

The `case` statement provides high-performance pattern matching without invoking subshells:

```
case "$os_type" in
    ubuntu|debian)
        pkg_mgr="apt-get"
        ;;
    fedora|rhel|centos)
        pkg_mgr="dnf"
        ;;
    alpine)
        pkg_mgr="apk"
        ;;
    arch)
        pkg_mgr="pacman"
        ;;
    *)
        echo "Unsupported distribution: $os_type" >&2
        exit 1
        ;;
esac
```

Advanced Terminators in Bash

- `;;`: Terminates execution (standard).
- `;&`: Fall-through to the next clause without testing its pattern.
- `;;&`: Tests subsequent patterns and executes if matched.

Break, Continue, and Early Exit Patterns

Break, Continue, and Early Exit Patterns

Controlling loop flow and establishing defensive guard clauses prevents deeply nested indentation.

```
# Guard clause pattern: validate early, exit early
process_file() {
    local file="$1"
    [[ -f "$file" ]] || return 1
    [[ -r "$file" ]] || return 2

    # Actual work continues un-indented
    cat "$file"
}

# Multi-level break
for outer in {1..5}; do
    for inner in {a..e}; do
        if [[ $outer -eq 3 && $inner == "c" ]]; then
            break 2 # Exits BOTH inner and outer loops
        fi
    done
done
```

Loops in Depth (C-Style, Nested, and Range Loops)

Loops in Depth (C-Style, Nested, and Range Loops)

Loops are one of the core building blocks in Bash scripting. They let you run a set of commands repeatedly, which is useful for anything from processing files in a directory to automating system tasks. Among the different types of loops, the for loop is one of the most commonly used. It lets you iterate over a sequence of values, such as numbers, strings, or items in an array.

In this guide, you'll learn how Bash for loops work. We'll go over the standard syntax, the C-style variant, and how to use break and continue to control loop execution. You'll also see how nested loops work, with plenty of examples along the way to show how you can use them in real scripts.

Standard Bash for Loop

The standard Bash for loop has the following syntax:

for variable in list do commands done Here, variable is a temporary variable that takes on each value from the list in turn, and the commands are executed for each iteration. The list can be a list of values separated by spaces, strings separated by spaces, a range of numbers, or even the output of a command.

Loop Over Number Ranges

One common use case for the for loop is to iterate over a range of numbers. This can be achieved using the seq command, which generates a sequence of numbers:

Loop from 1 to 5

for i in $(seq 1 5) do echo $i done Here's how it works:

The loop starts by assigning the first value from the sequence $(seq 1 5) to the variable i. The echo $i command within the loop body is then executed. The loop then moves to the next value in the sequence, assigns it to the variable, and executes the echo $i command again. This process continues until all values in the sequence have been iterated over. You can also specify an increment value for the sequence:

Loop from 1 to 10, incrementing by 2

for i in $(seq 1 2 10) do echo $i done Here is another common alternative syntax for specifying a range of numbers:

```
for i in {1..5}
do
  echo "Iteration $i"
done
```

The above syntax is more common in bash scripts. Starting from Bash 4, it's now possible to specify an increment when using ranges. Here is what it looks like:

```
# start..end..increment
for i in {1..10..2}
do
  echo "Iteration $i"
done
```

Loop Over Strings

The for loop can also iterate over a list of strings. Consider the following example:

Loop over a list of strings

```
for color in red green blue
do
  echo "The color is $color"
done
```

In this case, the loop iterates over the specified color strings, displaying each one in the output.

You can also use the for loop to iterate over characters in a string:

```
mystring="Hello, World!"
for char in $(echo "$mystring" | grep -o .)
do
    echo "$char"
done
```

In the above example:

The string “Hello, World!” is stored in the variable mystring. The grep -o . command matches and prints each character in the string individually. The for loop iterates over each character output by grep, assigning it to the variable char. Within the loop body, the current character is printed using the echo command. Loop Over Arrays

Bash arrays provide a convenient way to store multiple values. Let’s see how to use a for loop to iterate over the elements of an array:

Define an array

colors=(“red” “green” “blue”)

Loop over the array

```
for color in "${colors[@]}"
do
  echo "The color is $color"
done
```

Here we defined an array named colors and iterate over each element of the array echoing its value. Note the use of the [@] syntax to expand the array into a list of values.

If my Linux guides, cheat sheets, or deep dives have helped you learn or solve a problem, consider going paid. It keeps this work going and independent.

Upgrade to paid C-Style Bash for Loop

Bash also supports a C-style for loop, which provides more control over loop variables and conditions. The syntax for the C-style for loop is:

for ((variable assignment; condition; iteration process)) do commands done Here, variable assignment is executed once before the loop starts, condition is the condition that is evaluated before each iteration, and iteration process is executed after each iteration.

The following example prints numbers from 1 to 5.

Loop from 1 to 5

for ((i=1; i<=5; i++)) do echo $i done There are a couple of things you need to know about this syntax:

Variable value assignments can contain spaces. It's possible to omit the dollar sign for the variable in the condition. The equation for the iteration process doesn't use the expr command format to increment the value of the value. Break and Continue Statements

The break and continue statements can be used to control the flow of execution within a for loop, allowing you to skip or terminate iterations based on specific conditions. These statements can be particularly useful when dealing with complex or nested loops, or when you want to implement specific logic within your loop.

Break Statement

The break statement terminates the loop immediately and transfers control to the next statement following the loop. This can be useful in situations where you want to exit the loop early based on a certain condition, such as when you've found the desired value or when a specific condition is met.

Loop through an array and break when a specific value is found

```
fruits=("apple" "banana" "orange" "grape" "pear")

for fruit in "${fruits[@]}"
do
  if [ "$fruit" == "orange" ]
  then
    echo "Found orange!"
    break
  fi
  echo "Current fruit: $fruit"
done
```

In this example, the loop iterates over the fruits array, and when the value “orange” is encountered, the break statement is executed, terminating the loop and printing “Found orange!”.

Continue Statement

The continue statement skips the current iteration of the loop and moves to the next iteration. This can be useful when you want to skip certain iterations based on a specific condition, such as when dealing with invalid or unwanted data.

Loop through a range of numbers and skip multiples of 3

for i in $(seq 1 10) do if [$((i % 3)) -eq 0] then continue fi echo $i done In this example, the loop iterates over the numbers from 1 to 10, but it skips the multiples of 3 using the continue statement.

Both break and continue statements can be used in combination with conditional statements (if, case, etc.) to implement complex logic within your loops. However, it's important to use them judiciously and avoid excessive use, as this can make your code harder to read and maintain.

Nested For Loops

In Bash, it's possible to have one for loop nested inside another for loop. This construct is known as a nested loop, and it allows you to iterate over multidimensional data structures or perform operations that require nested iterations.

The syntax for a nested for loop is as follows:

for outer_variable in outer_list do for inner_variable in inner_list do commands done done Here, the outer loop iterates over theouter_list, and for each iteration of the outer loop, the inner loop iterates over the inner_list. The commands are executed for each combination of the outer and inner loop variables.

Consider the following example:

for i in $(seq 1 2) do for j in $(seq 1 3) do echo "Outer: $i, Inner: $j" done done Let's break down the code:

The outer loop iterates over the numbers 1 and 2, assigning each value to the variable i. For each iteration of the outer loop, the inner loop iterates over the numbers 1, 2, and 3, assigning each value to the variable j. Within the inner loop body, the current values of i and j are printed using the echo command. Thanks for reading!

If you enjoyed this content, don't forget to leave a like and subscribe to get more posts like this every week.

Conditional Statements

Conditional Statements

Conditional Statements

Conditional statements allow your scripts to make decisions and execute different code paths based on conditions. They are essential for creating intelligent, responsive scripts that can handle various scenarios.

The if Statement

The `if` statement is the most basic conditional construct in Bash.

Basic Syntax

```
if [ condition ]; then
    # commands to execute if condition is true
fi
```

Simple Example

```
#!/bin/bash
age=20

if [ $age -ge 18 ]; then
    echo "You are an adult"
fi
```

if-else Statement

Add an alternative path when the condition is false:

```
#!/bin/bash
age=16

if [ $age -ge 18 ]; then
    echo "You are an adult"
else
    echo "You are a minor"
```

```
fi
```

if-elif-else Statement

Handle multiple conditions:

```
#!/bin/bash
score=85

if [ $score -ge 90 ]; then
    echo "Grade: A"
elif [ $score -ge 80 ]; then
    echo "Grade: B"
elif [ $score -ge 70 ]; then
    echo "Grade: C"
elif [ $score -ge 60 ]; then
    echo "Grade: D"
else
    echo "Grade: F"
fi
```

Test Conditions

Numeric Comparisons

```
#!/bin/bash
num1=10
num2=20

# Equal
if [ $num1 -eq $num2 ]; then
    echo "Numbers are equal"
fi

# Not equal
if [ $num1 -ne $num2 ]; then
    echo "Numbers are not equal"
fi

# Greater than
if [ $num1 -gt $num2 ]; then
    echo "$num1 is greater than $num2"
fi
```

```
# Less than
if [ $num1 -lt $num2 ]; then
    echo "$num1 is less than $num2"
fi

# Greater than or equal
if [ $num1 -ge $num2 ]; then
    echo "$num1 is greater than or equal to $num2"
fi

# Less than or equal
if [ $num1 -le $num2 ]; then
    echo "$num1 is less than or equal to $num2"
fi
```

String Comparisons

```
#!/bin/bash
name1="Alice"
name2="Bob"
empty_string=""

# String equality
if [ "$name1" = "$name2" ]; then
    echo "Names are the same"
else
    echo "Names are different"
fi

# String inequality
if [ "$name1" != "$name2" ]; then
    echo "Names are different"
fi

# String length (non-zero)
if [ -n "$name1" ]; then
    echo "Name1 is not empty"
fi

# String length (zero)
if [ -z "$empty_string" ]; then
    echo "String is empty"
fi

# Lexicographic comparison
```

```
if [[ "$name1" < "$name2" ]]; then
    echo "$name1 comes before $name2 alphabetically"
fi
```

File Tests

```
#!/bin/bash
filename="test.txt"
directory="mydir"

# File exists
if [ -e "$filename" ]; then
    echo "File exists"
fi

# Regular file
if [ -f "$filename" ]; then
    echo "It's a regular file"
fi

# Directory
if [ -d "$directory" ]; then
    echo "It's a directory"
fi

# Readable
if [ -r "$filename" ]; then
    echo "File is readable"
fi

# Writable
if [ -w "$filename" ]; then
    echo "File is writable"
fi

# Executable
if [ -x "$filename" ]; then
    echo "File is executable"
fi

# File size greater than zero
if [ -s "$filename" ]; then
    echo "File is not empty"
fi
```

```
# Symbolic link
if [ -L "$filename" ]; then
    echo "It's a symbolic link"
fi
```

Advanced Test Constructs

Double Brackets [[ ]]

The `[[ ]]` construct provides more features than single brackets:

```
#!/bin/bash
name="Alice"
number="123"

# Pattern matching
if [[ $name == A* ]]; then
    echo "Name starts with A"
fi

# Regular expressions
if [[ $number =~ ^[0-9]+$ ]]; then
    echo "It's a number"
fi

# Multiple conditions with && and ||
if [[ $name == "Alice" && $number -gt 100 ]]; then
    echo "Name is Alice and number is greater than 100"
fi

if [[ $name == "Alice" || $name == "Bob" ]]; then
    echo "Name is either Alice or Bob"
fi
```

Arithmetic Evaluation (( ))

For numeric comparisons, you can use arithmetic evaluation:

```
#!/bin/bash
x=10
y=20

if (( x < y )); then
    echo "$x is less than $y"
```

```
fi

if (( x + y == 30 )); then
    echo "Sum is 30"
fi

if (( x % 2 == 0 )); then
    echo "$x is even"
fi
```

Logical Operators

AND Operator (&&)

```
#!/bin/bash
age=25
has_license=true

if [ $age -ge 18 ] && [ "$has_license" = "true" ]; then
    echo "Can drive"
fi

# Alternative syntax
if [[ $age -ge 18 && $has_license == "true" ]]; then
    echo "Can drive"
fi
```

OR Operator (||)

```
#!/bin/bash
day="Saturday"

if [ "$day" = "Saturday" ] || [ "$day" = "Sunday" ]; then
    echo "It's weekend!"
fi

# Alternative syntax
if [[ $day == "Saturday" || $day == "Sunday" ]]; then
    echo "It's weekend!"
fi
```

NOT Operator (!)

```
#!/bin/bash
filename="test.txt"

# Negate condition
if [ ! -f "$filename" ]; then
    echo "File does not exist"
fi

# Alternative
if ! [ -f "$filename" ]; then
    echo "File does not exist"
fi
```

Practical Examples

File Backup Script

```
#!/bin/bash
source_file="$1"
backup_dir="/backup"

# Check if source file is provided
if [ $# -eq 0 ]; then
    echo "Usage: $0 <source_file>"
    exit 1
fi

# Check if source file exists
if [ ! -f "$source_file" ]; then
    echo "Error: Source file '$source_file' does not exist"
    exit 1
fi

# Check if backup directory exists
if [ ! -d "$backup_dir" ]; then
    echo "Creating backup directory..."
    mkdir -p "$backup_dir"
fi

# Create backup
backup_name="$(basename "$source_file").backup.$(date +%Y%m%d_%H%M%S)"
if cp "$source_file" "$backup_dir/$backup_name"; then
```

```
    echo "Backup created: $backup_dir/$backup_name"
else
    echo "Error: Failed to create backup"
    exit 1
fi
```

System Health Check

```
#!/bin/bash
# System health check script

echo "=== System Health Check ==="

# Check disk usage
disk_usage=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
if [ $disk_usage -gt 90 ]; then
    echo "WARNING: Disk usage is ${disk_usage}%"
elif [ $disk_usage -gt 80 ]; then
    echo "CAUTION: Disk usage is ${disk_usage}%"
else
    echo "OK: Disk usage is ${disk_usage}%"
fi

# Check memory usage
memory_usage=$(free | awk 'NR==2{printf "%.0f", $3*100/$2}')
if [ $memory_usage -gt 90 ]; then
    echo "WARNING: Memory usage is ${memory_usage}%"
else
    echo "OK: Memory usage is ${memory_usage}%"
fi

# Check if critical services are running
services=("ssh" "nginx" "mysql")
for service in "${services[@]}"; do
    if systemctl is-active --quiet "$service"; then
        echo "OK: $service is running"
    else
        echo "ERROR: $service is not running"
    fi
done
```

User Input Validation

```
#!/bin/bash
# User registration script with validation

echo "User Registration"
echo "=================="

# Get username
while true; do
    read -p "Enter username (3-20 characters, alphanumeric only): " username

    if [[ -z "$username" ]]; then
        echo "Username cannot be empty"
    elif [[ ${#username} -lt 3 ]]; then
        echo "Username must be at least 3 characters"
    elif [[ ${#username} -gt 20 ]]; then
        echo "Username must be no more than 20 characters"
    elif [[ ! $username =~ ^[a-zA-Z0-9]+$ ]]; then
        echo "Username can only contain letters and numbers"
    else
        break
    fi
done

# Get email
while true; do
    read -p "Enter email address: " email

    if [[ -z "$email" ]]; then
        echo "Email cannot be empty"
    elif [[ ! $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
        echo "Please enter a valid email address"
    else
        break
    fi
done

# Get age
while true; do
    read -p "Enter age (18-120): " age

    if [[ ! $age =~ ^[0-9]+$ ]]; then
        echo "Age must be a number"
    elif [ $age -lt 18 ]; then
        echo "You must be at least 18 years old"
```

```
    elif [ $age -gt 120 ]; then
        echo "Please enter a realistic age"
    else
        break
    fi
done

echo
echo "Registration successful!"
echo "Username: $username"
echo "Email: $email"
echo "Age: $age"
```

Nested Conditionals

You can nest if statements for complex logic:

```
#!/bin/bash
weather="sunny"
temperature=75

if [ "$weather" = "sunny" ]; then
    if [ $temperature -gt 70 ]; then
        echo "Perfect day for outdoor activities!"
    else
        echo "Sunny but a bit cold"
    fi
elif [ "$weather" = "rainy" ]; then
    if [ $temperature -gt 60 ]; then
        echo "Warm rain, good for plants"
    else
        echo "Cold and rainy, stay inside"
    fi
else
    echo "Weather is $weather"
fi
```

Short-Circuit Evaluation

Use `&&` and `||` for concise conditional execution:

```
#!/bin/bash
filename="test.txt"
```

```
# Execute command only if condition is true
[ -f "$filename" ] && echo "File exists"

# Execute command only if condition is false
[ ! -f "$filename" ] || echo "File exists"

# Chain multiple conditions
[ -f "$filename" ] && [ -r "$filename" ] && echo "File exists and is readable"

# Alternative to if-else
[ $# -eq 0 ] && echo "No arguments provided" || echo "Arguments provided"
```

Common Pitfalls and Best Practices

1. Always Quote Variables

```
# Wrong - can break with spaces
if [ $name = "John Doe" ]; then
    echo "Hello John"
fi

# Correct
if [ "$name" = "John Doe" ]; then
    echo "Hello John"
fi
```

2. Use [[ ]] for String Operations

```
# Better for pattern matching and regex
if [[ $filename == *.txt ]]; then
    echo "Text file"
fi

if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]; then
    echo "Valid email"
fi
```

3. Handle Empty Variables

```
# Safe way to handle potentially empty variables
if [ -n "${name:-}" ]; then
    echo "Name is set: $name"
else
    echo "Name is not set"
fi
```

4. Use Meaningful Conditions

```
# Good - readable and clear
if [ $user_age -ge $minimum_age ]; then
    grant_access
fi

# Bad - unclear magic numbers
if [ $age -ge 21 ]; then
    do_something
fi
```

5. Exit Codes for Error Handling

```
#!/bin/bash
if ! command_that_might_fail; then
    echo "Command failed"
    exit 1
fi

# Or check exit code explicitly
command_that_might_fail
if [ $? -ne 0 ]; then
    echo "Command failed with exit code $?"
    exit 1
fi
```

Conditional statements are the foundation of decision-making in shell scripts. Master these concepts to create scripts that can intelligently respond to different situations and handle various scenarios gracefully.

Loops (for, while, until)

Loops (for, while, until)

Loops

Loops allow you to execute a block of code repeatedly, making them essential for automating repetitive tasks. Bash provides several types of loops, each suited for different scenarios.

for Loops

The `for` loop is the most commonly used loop in Bash scripting.

Basic for Loop Syntax

```
for variable in list; do
    # commands
done
```

Simple Examples

```
#!/bin/bash

# Loop through a list of items
for fruit in apple banana orange; do
    echo "I like $fruit"
done

# Loop through numbers
for number in 1 2 3 4 5; do
    echo "Number: $number"
done
```

Looping Through Files

```
#!/bin/bash

# Process all .txt files in current directory
for file in *.txt; do
    if [ -f "$file" ]; then
```

```
        echo "Processing: $file"
        wc -l "$file"
    fi
done

# Process files in a specific directory
for file in /var/log/*.log; do
    if [ -f "$file" ]; then
        echo "Log file: $(basename "$file")"
        echo "Size: $(du -h "$file" | cut -f1)"
    fi
done
```

C-style for Loop

```
#!/bin/bash

# C-style for loop
for ((i=1; i<=10; i++)); do
    echo "Iteration: $i"
done

# Counting backwards
for ((i=10; i>=1; i--)); do
    echo "Countdown: $i"
done

# Step by 2
for ((i=0; i<=20; i+=2)); do
    echo "Even number: $i"
done
```

Using Brace Expansion

```
#!/bin/bash

# Loop through a range of numbers
for i in {1..10}; do
    echo "Number: $i"
done

# Loop with step
for i in {1..20..2}; do
    echo "Odd number: $i"
```

```
done

# Loop through letters
for letter in {a..z}; do
    echo "Letter: $letter"
done
```

Looping Through Arrays

```
#!/bin/bash

# Define an array
colors=("red" "green" "blue" "yellow")

# Loop through array elements
for color in "${colors[@]}"; do
    echo "Color: $color"
done

# Loop through array indices
for i in "${!colors[@]}"; do
    echo "Index $i: ${colors[i]}"
done
```

while Loops

The `while` loop continues executing as long as a condition is true.

Basic while Loop

```
#!/bin/bash

counter=1
while [ $counter -le 5 ]; do
    echo "Counter: $counter"
    ((counter++))
done
```

Reading File Line by Line

```
#!/bin/bash

filename="data.txt"

# Create sample file
cat > "$filename" << EOF
Line 1
Line 2
Line 3
EOF

# Read file line by line
while IFS= read -r line; do
    echo "Processing: $line"
done < "$filename"
```

Menu System with while Loop

```
#!/bin/bash

show_menu() {
    echo "=== Menu ==="
    echo "1. Show date"
    echo "2. Show users"
    echo "3. Show disk usage"
    echo "4. Exit"
    echo -n "Choose option: "
}

while true; do
    show_menu
    read choice

    case $choice in
        1)
            echo "Current date: $(date)"
            ;;
        2)
            echo "Logged in users:"
            who
            ;;
        3)
            echo "Disk usage:"
```

```
            df -h
            ;;
        4)
            echo "Goodbye!"
            break
            ;;
        *)
            echo "Invalid option. Please try again."
            ;;
    esac

    echo
    read -p "Press Enter to continue..."
    echo
done
```

Monitoring with while Loop

```
#!/bin/bash

# Monitor system load
echo "Monitoring system load (Ctrl+C to stop)..."

while true; do
    load=$(uptime | awk -F'load average:' '{print $2}' | awk '{print $1}' | sed 's/,//')
    echo "$(date): Load average: $load"

    # Alert if load is high
    if (( $(echo "$load > 2.0" | bc -l) )); then
        echo "WARNING: High system load!"
    fi

    sleep 5
done
```

until Loops

The `until` loop continues executing until a condition becomes true (opposite of while).

Basic until Loop

```
#!/bin/bash

counter=1
until [ $counter -gt 5 ]; do
    echo "Counter: $counter"
    ((counter++))
done
```

Waiting for a Service

```
#!/bin/bash

service_name="nginx"

echo "Waiting for $service_name to start..."
until systemctl is-active --quiet "$service_name"; do
    echo "Service not ready, waiting..."
    sleep 2
done

echo "$service_name is now running!"
```

Waiting for File

```
#!/bin/bash

target_file="/tmp/ready.flag"

echo "Waiting for file $target_file to appear..."
until [ -f "$target_file" ]; do
    echo "File not found, waiting..."
    sleep 1
done

echo "File found! Continuing..."
```

Loop Control Statements

break Statement

Exits the loop immediately:

```
#!/bin/bash

# Find first .txt file
for file in *; do
    if [[ "$file" == *.txt ]]; then
        echo "Found text file: $file"
        break
    fi
    echo "Checking: $file"
done
```

continue Statement

Skips the rest of the current iteration:

```
#!/bin/bash

# Process only .txt files
for file in *; do
    # Skip non-txt files
    if [[ "$file" != *.txt ]]; then
        continue
    fi

    echo "Processing text file: $file"
    wc -l "$file"
done
```

Nested Loops with break and continue

```
#!/bin/bash

# Nested loops example
for i in {1..3}; do
    echo "Outer loop: $i"

    for j in {1..5}; do
        if [ $j -eq 3 ]; then
```

```
            echo "  Skipping inner loop iteration $j"
            continue
        fi

        if [ $i -eq 2 ] && [ $j -eq 4 ]; then
            echo "  Breaking inner loop at $i,$j"
            break
        fi

        echo "  Inner loop: $j"
    done
done
```

Practical Loop Examples

Batch File Processing

```
#!/bin/bash

# Batch resize images
image_dir="./images"
output_dir="./thumbnails"

# Create output directory
mkdir -p "$output_dir"

for image in "$image_dir"/*.{jpg,jpeg,png}; do
    # Skip if no files match
    [ ! -f "$image" ] && continue

    filename=$(basename "$image")
    name="${filename%.*}"
    extension="${filename##*.}"

    echo "Processing: $filename"

    # Resize image (requires ImageMagick)
    if command -v convert >/dev/null 2>&1; then
        convert "$image" -resize 200x200 "$output_dir/${name}_thumb.$extension"
        echo "Created thumbnail: ${name}_thumb.$extension"
    else
        echo "ImageMagick not installed, skipping resize"
    fi
done
```

Log Analysis

```
#!/bin/bash

log_file="/var/log/apache2/access.log"
declare -A ip_count

# Count IP addresses in log file
while IFS= read -r line; do
    # Extract IP address (first field)
    ip=$(echo "$line" | awk '{print $1}')

    # Skip invalid IPs
    if [[ $ip =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
        ((ip_count[$ip]++))
    fi
done < "$log_file"

# Display top 10 IPs
echo "Top 10 IP addresses:"
for ip in "${!ip_count[@]}"; do
    echo "$ip: ${ip_count[$ip]}"
done | sort -k2 -nr | head -10
```

System Backup Script

```
#!/bin/bash

# Directories to backup
backup_dirs=("/home/user/documents" "/home/user/projects" "/etc")
backup_destination="/backup/$(date +%Y%m%d)"
log_file="/var/log/backup.log"

# Create backup destination
mkdir -p "$backup_destination"

# Function to log messages
log_message() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" | tee -a "$log_file"
}

log_message "Starting backup process"

# Backup each directory
for dir in "${backup_dirs[@]}"; do
```

```
    if [ -d "$dir" ]; then
        dir_name=$(basename "$dir")
        log_message "Backing up $dir"

        if tar -czf "$backup_destination/${dir_name}.tar.gz" "$dir"; then
            log_message "Successfully backed up $dir"
        else
            log_message "ERROR: Failed to backup $dir"
        fi
    else
        log_message "WARNING: Directory $dir does not exist"
    fi
done

log_message "Backup process completed"

# Clean up old backups (keep only last 7 days)
find /backup -name "20*" -type d -mtime +7 -exec rm -rf {} \;
log_message "Cleaned up old backups"
```

User Management Script

```
#!/bin/bash

# Create multiple users from a list
users_file="users.txt"

# Create sample users file
cat > "$users_file" << EOF
john:John Doe:developer
jane:Jane Smith:admin
bob:Bob Johnson:user
EOF

# Process each user
while IFS=':' read -r username fullname role; do
    # Skip empty lines and comments
    [[ -z "$username" || "$username" =~ ^# ]] && continue

    echo "Processing user: $username"

    # Check if user already exists
    if id "$username" >/dev/null 2>&1; then
        echo "User $username already exists, skipping"
        continue
```

```
    fi

    # Create user
    if useradd -m -c "$fullname" "$username"; then
        echo "Created user: $username ($fullname)"

        # Set initial password
        echo "$username:temp123" | chpasswd
        echo "Set temporary password for $username"

        # Add to appropriate group based on role
        case "$role" in
            "admin")
                usermod -aG sudo "$username"
                echo "Added $username to sudo group"
                ;;
            "developer")
                usermod -aG developers "$username" 2>/dev/null || echo "developers group not
                ;;
        esac
    else
        echo "Failed to create user: $username"
    fi

done < "$users_file"
```

Loop Performance Tips

Avoid Unnecessary Command Substitution

```
# Slow - calls external command in each iteration
for i in {1..1000}; do
    date_str=$(date)
    echo "$i: $date_str"
done

# Fast - call command once
date_str=$(date)
for i in {1..1000}; do
    echo "$i: $date_str"
done
```

Use Built-in Commands When Possible

```
# Slow - external command
for file in *.txt; do
    lines=$(wc -l < "$file")
    echo "$file: $lines lines"
done

# Faster - but still external command per file
for file in *.txt; do
    while IFS= read -r line; do
        ((count++))
    done < "$file"
    echo "$file: $count lines"
    count=0
done
```

Process Files Efficiently

```
# Process multiple files with one command
find . -name "*.log" -print0 | while IFS= read -r -d '' file; do
    echo "Processing: $file"
    # Process file
done
```

Common Loop Patterns

Retry Pattern

```
retry_command() {
    local max_attempts=3
    local attempt=1

    while [ $attempt -le $max_attempts ]; do
        echo "Attempt $attempt of $max_attempts"

        if command_that_might_fail; then
            echo "Command succeeded"
            return 0
        fi

        echo "Command failed, retrying..."
        ((attempt++))
```

```
        sleep 2
    done

    echo "Command failed after $max_attempts attempts"
    return 1
}
```

Progress Indicator

```
#!/bin/bash

total_items=100

for ((i=1; i<=total_items; i++)); do
    # Simulate work
    sleep 0.1

    # Calculate progress
    progress=$((i * 100 / total_items))

    # Display progress bar
    printf "\rProgress: [%-50s] %d%%" $(printf "#%.0s" $(seq 1 $((progress/2)))) $progress
done

echo
echo "Complete!"
```

Loops are powerful constructs that enable automation and batch processing in your shell scripts. Understanding when and how to use each type of loop will help you write more efficient and effective scripts.

Functions and Modular Design

Local Variables and Scope

Local Variables and Scope

In Bash, all variables are global by default—even when defined inside a function. Always use `local` to prevent state contamination:

```
counter=100

my_function() {
    local counter=1 # Scoped exclusively to this function and its callees
    echo "Inside: $counter"
}

my_function
echo "Outside: $counter" # 100 (Unchanged)
```

Dynamic Scoping Warning: Bash uses dynamic scoping, not lexical scoping. A `local` variable declared in function `A` is visible to function `B` if `A` calls `B`!

Positional Parameters in Functions

Positional Parameters in Functions

Inside a function, positional parameters (`$1`, `$2`, `$@`, `$#`) reflect the function's arguments, completely isolated from the script's outer command-line arguments:

```
calculate_sum() {
    local sum=0
    for num in "$@"; do
        (( sum += num ))
    done
    echo "$sum"
}

total=$(calculate_sum 10 20 30)
echo "Total: $total" # 60
```

Return Values and Exit Codes

Return Values and Exit Codes

A critical limitation in shell programming: `return` only returns an integer status code between 0 and 255. It cannot return strings or data objects!

Returning Data Strategies

```
# Strategy 1: stdout capture (spawns subshell)
get_hostname() {
    echo "host-$(hostname -s)"
}
host=$(get_hostname)

# Strategy 2: Pass-by-reference with nameref (Bash 4.3+, zero subshell overhead!)
get_system_info() {
    local -n out_ref="$1" # nameref to caller variable
    out_ref="OS=Linux Kernel=$(uname -r)"
}

info=""
get_system_info info
echo "$info"
```

Script Libraries and Sourcing

Script Libraries and Sourcing

Modular scripts reuse libraries using `source` or `.`. Prevent multiple inclusions using header guards:

```
# lib/logger.sh
[[ -n "${_LIB_LOGGER_SH:-}" ]] && return 0
readonly _LIB_LOGGER_SH=1

log_info() {
    printf '[%s] [INFO] %s\n' "$(date +'%Y-%m-%dT%H:%M:%S%z')" "$*" >&2
}
```

Locate libraries reliably relative to the executing script:

```
# Locate directory containing this script
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
source "${SCRIPT_DIR}/../lib/logger.sh"
```

Function Basics

Function Basics

Function Basics

Functions are reusable blocks of code that help organize your scripts, reduce repetition, and make your code more maintainable. They allow you to break complex scripts into smaller, manageable pieces.

What are Functions?

Functions in Bash are named blocks of code that can be called multiple times throughout your script. They help you:

- **Organize code**: Break large scripts into logical pieces
- **Reduce repetition**: Write once, use many times
- **Improve readability**: Make scripts easier to understand
- **Enable testing**: Test individual components
- **Facilitate maintenance**: Update code in one place

Function Syntax

Method 1: Using the `function` keyword

```
function function_name() {
    # function body
    commands
}
```

Method 2: POSIX-compatible syntax (preferred)

```
function_name() {
    # function body
    commands
}
```

Basic Function Examples

Simple Function

```
#!/bin/bash

# Define a function
greet() {
    echo "Hello, World!"
}

# Call the function
greet
```

Function with Commands

```
#!/bin/bash

show_date() {
    echo "Current date and time:"
    date
    echo "Uptime:"
    uptime
}

# Call the function
show_date
```

Function Parameters

Functions can accept parameters (arguments) to make them more flexible:

Accessing Parameters

```
#!/bin/bash

greet_user() {
    echo "Hello, $1!"
    echo "Welcome to $2"
}

# Call function with arguments
```

```
greet_user "Alice" "Bash scripting"
```

Multiple Parameters

```
#!/bin/bash

user_info() {
    local name="$1"
    local age="$2"
    local city="$3"

    echo "Name: $name"
    echo "Age: $age"
    echo "City: $city"
}

# Call with multiple arguments
user_info "John Doe" "30" "New York"
```

Parameter Variables

```
#!/bin/bash

show_params() {
    echo "Function name: $0"
    echo "First parameter: $1"
    echo "Second parameter: $2"
    echo "All parameters: $@"
    echo "Number of parameters: $#"
    echo "All parameters as single string: $*"
}

show_params "arg1" "arg2" "arg3"
```

Return Values

Functions can return values using the `return` statement:

Return Exit Status

```
#!/bin/bash

check_file() {
    local filename="$1"

    if [ -f "$filename" ]; then
        echo "File $filename exists"
        return 0  # Success
    else
        echo "File $filename does not exist"
        return 1  # Failure
    fi
}

# Use function and check return value
if check_file "test.txt"; then
    echo "File check passed"
else
    echo "File check failed"
fi
```

Return Values via Echo

```
#!/bin/bash

get_file_size() {
    local filename="$1"

    if [ -f "$filename" ]; then
        stat -f%z "$filename" 2>/dev/null || stat -c%s "$filename" 2>/dev/null
    else
        echo "0"
    fi
}

# Capture function output
file_size=$(get_file_size "test.txt")
echo "File size: $file_size bytes"
```

Local Variables

Use `local` to create variables that exist only within the function:

Without Local Variables (Global)

```
#!/bin/bash

global_example() {
    name="Alice"  # This modifies global variable
    echo "Inside function: $name"
}

name="Bob"
echo "Before function: $name"
global_example
echo "After function: $name"  # Changed to Alice
```

With Local Variables

```
#!/bin/bash

local_example() {
    local name="Alice"  # This is local to the function
    echo "Inside function: $name"
}

name="Bob"
echo "Before function: $name"
local_example
echo "After function: $name"  # Still Bob
```

Best Practice Example

```
#!/bin/bash

calculate_area() {
    local length="$1"
    local width="$2"
    local area

    area=$((length * width))
    echo "$area"
}

# Use the function
room_area=$(calculate_area 10 12)
```

```
echo "Room area: $room_area square feet"
```

Practical Function Examples

File Operations Function

```
#!/bin/bash

backup_file() {
    local source_file="$1"
    local backup_dir="${2:-./backup}"

    # Validate input
    if [ -z "$source_file" ]; then
        echo "Error: Source file not specified"
        return 1
    fi

    if [ ! -f "$source_file" ]; then
        echo "Error: Source file '$source_file' does not exist"
        return 1
    fi

    # Create backup directory if it doesn't exist
    mkdir -p "$backup_dir"

    # Create backup with timestamp
    local backup_name="$(basename "$source_file").backup.$(date +%Y%m%d_%H%M%S)"

    if cp "$source_file" "$backup_dir/$backup_name"; then
        echo "Backup created: $backup_dir/$backup_name"
        return 0
    else
        echo "Error: Failed to create backup"
        return 1
    fi
}

# Use the function
backup_file "/etc/hosts"
backup_file "important.txt" "/safe/location"
```

System Information Function

```
#!/bin/bash

show_system_info() {
    local info_type="$1"

    case "$info_type" in
        "cpu")
            echo "CPU Information:"
            lscpu | grep "Model name" || echo "CPU info not available"
            ;;
        "memory")
            echo "Memory Information:"
            free -h
            ;;
        "disk")
            echo "Disk Usage:"
            df -h
            ;;
        "all")
            show_system_info "cpu"
            echo
            show_system_info "memory"
            echo
            show_system_info "disk"
            ;;
        *)
            echo "Usage: show_system_info {cpu|memory|disk|all}"
            return 1
            ;;
    esac
}

# Use the function
show_system_info "all"
```

String Processing Function

```
#!/bin/bash

process_string() {
    local input="$1"
    local operation="$2"
```

```
    case "$operation" in
        "upper")
            echo "${input^^}"
            ;;
        "lower")
            echo "${input,,}"
            ;;
        "length")
            echo "${#input}"
            ;;
        "reverse")
            echo "$input" | rev
            ;;
        "words")
            echo "$input" | wc -w
            ;;
        *)
            echo "Usage: process_string <string> {upper|lower|length|reverse|words}"
            return 1
            ;;
    esac
}

# Use the function
text="Hello World"
echo "Original: $text"
echo "Uppercase: $(process_string "$text" "upper")"
echo "Lowercase: $(process_string "$text" "lower")"
echo "Length: $(process_string "$text" "length")"
echo "Reversed: $(process_string "$text" "reverse")"
echo "Word count: $(process_string "$text" "words")"
```

Function Libraries

You can create reusable function libraries:

Create a Library File (utils.sh)

```
#!/bin/bash
# utils.sh - Utility functions library

# Logging function
log() {
    local level="$1"
```

```
    shift
    local message="$*"
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$level] $message"
}

# Check if command exists
command_exists() {
    command -v "$1" >/dev/null 2>&1
}

# Validate email format
is_valid_email() {
    local email="$1"
    [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]
}

# Get file extension
get_extension() {
    local filename="$1"
    echo "${filename##*.}"
}

# Create directory if it doesn't exist
ensure_directory() {
    local dir="$1"
    if [ ! -d "$dir" ]; then
        mkdir -p "$dir"
        log "INFO" "Created directory: $dir"
    fi
}
```

Use the Library in Your Script

```
#!/bin/bash
# main_script.sh

# Source the utility functions
source ./utils.sh

# Use the functions
log "INFO" "Script started"

if command_exists "git"; then
    log "INFO" "Git is available"
else
```

```
    log "ERROR" "Git is not installed"
    exit 1
fi

email="user@example.com"
if is_valid_email "$email"; then
    log "INFO" "Email format is valid"
else
    log "ERROR" "Invalid email format"
fi

filename="document.pdf"
extension=$(get_extension "$filename")
log "INFO" "File extension: $extension"

ensure_directory "/tmp/myapp"
log "INFO" "Script completed"
```

Function Best Practices

1. Use Descriptive Names

```
# Good
calculate_monthly_payment() {
    # function body
}

# Bad
calc() {
    # function body
}
```

2. Validate Input Parameters

```
backup_database() {
    local db_name="$1"
    local backup_path="$2"

    # Validate required parameters
    if [ -z "$db_name" ]; then
        echo "Error: Database name is required"
        return 1
    fi
```

```
    if [ -z "$backup_path" ]; then
        echo "Error: Backup path is required"
        return 1
    fi

    # Function logic here
}
```

3. Use Local Variables

```
process_data() {
    local input_file="$1"
    local output_file="$2"
    local temp_file
    local line_count

    # All variables are local to this function
    temp_file=$(mktemp)
    line_count=$(wc -l < "$input_file")

    # Process data...
}
```

4. Provide Usage Information

```
convert_temperature() {
    local temp="$1"
    local from_unit="$2"
    local to_unit="$3"

    if [ $# -ne 3 ]; then
        echo "Usage: convert_temperature <temperature> <from_unit> <to_unit>"
        echo "Units: C (Celsius), F (Fahrenheit), K (Kelvin)"
        return 1
    fi

    # Conversion logic here
}
```

5. Handle Errors Gracefully

```
download_file() {
    local url="$1"
    local destination="$2"

    if ! command -v curl >/dev/null 2>&1; then
        echo "Error: curl is not installed"
        return 1
    fi

    if ! curl -o "$destination" "$url"; then
        echo "Error: Failed to download $url"
        return 1
    fi

    echo "Successfully downloaded to $destination"
    return 0
}
```

Common Function Patterns

Configuration Function

```
load_config() {
    local config_file="${1:-config.conf}"

    if [ -f "$config_file" ]; then
        source "$config_file"
        echo "Configuration loaded from $config_file"
    else
        echo "Warning: Configuration file $config_file not found"
        echo "Using default settings"
    fi
}
```

Cleanup Function

```
cleanup() {
    local temp_files=("$@")

    echo "Cleaning up temporary files..."
    for file in "${temp_files[@]}"; do
```

```
        if [ -f "$file" ]; then
            rm "$file"
            echo "Removed: $file"
        fi
    done
}

# Set trap to call cleanup on exit
trap 'cleanup "$temp_file1" "$temp_file2"' EXIT
```

Menu Function

```
show_menu() {
    echo "=== Main Menu ==="
    echo "1. Option 1"
    echo "2. Option 2"
    echo "3. Option 3"
    echo "4. Exit"
    echo -n "Choose an option: "
}

handle_choice() {
    local choice="$1"

    case "$choice" in
        1) echo "You chose option 1" ;;
        2) echo "You chose option 2" ;;
        3) echo "You chose option 3" ;;
        4) echo "Goodbye!"; exit 0 ;;
        *) echo "Invalid choice" ;;
    esac
}
```

Functions are powerful tools that make your Bash scripts more organized, maintainable, and reusable. They're essential for writing professional-quality shell scripts that are easy to understand and modify.

Arrays and Associative Arrays

Indexed Arrays: Creation and Manipulation

Indexed Arrays: Creation and Manipulation

Indexed arrays store sequences of values accessible by integer indices:

```
# Creation
declare -a servers=("web01" "web02" "db01")

# Appending elements
servers+=("cache01" "cache02")

# Accessing
echo "${servers[0]}"   # web01
echo "${servers[-1]}"  # cache02 (last element)
echo "Count: ${#servers[@]}" # 5
```

Iterating Over Arrays Safely

Iterating Over Arrays Safely

Never iterate over unquoted arrays or use `${array[*]}`. Always iterate with `"${array[@]}"`:

```
files=("file with spaces.txt" "another.log")

# SAFE: Preserves whitespace boundaries
for file in "${files[@]}"; do
    echo "Processing: $file"
done

# Iterating by index (for parallel or indexed arrays):
for i in "${!files[@]}"; do
    echo "Index $i contains ${files[$i]}"
done
```

Array Operations: Slicing, Searching, Deleting

Array Operations: Slicing, Searching, Deleting

```
nums=(zero one two three four five)

# Slicing: ${array[@]:start:length}
echo "${nums[@]:2:3}" # two three four

# Deleting an element
unset "nums[1]" # Removes 'one' (array becomes sparse!)

# Pattern replacement across all elements
echo "${nums[@]/e/E}" # zEro two thrEe four fivE
```

Associative Arrays for Key-Value Data

Associative Arrays for Key-Value Data

Associative arrays (hashes/maps) map arbitrary string keys to string values (Bash 4.0+):

```
# MUST be explicitly declared with -A
declare -A ports=(
    ["http"]="80"
    ["https"]="443"
    ["ssh"]="22"
)

# Adding or updating
ports["dns"]="53"

# Querying keys and values
echo "SSH port: ${ports["ssh"]}"

# Iterating over all keys
for service in "${!ports[@]}"; do
    echo "Service $service runs on port ${ports[$service]}"
done
```

Portability Considerations for Arrays

Portability Considerations for Arrays

Arrays are not part of the POSIX standard. If a script runs under `/bin/sh` on Debian (dash) or Alpine (ash), arrays will fail with syntax errors.

```
# POSIX workaround 1: Use positional parameters
set -- "arg 1" "arg 2 with spaces" "arg 3"
for item in "$@"; do
    echo "$item"
done

# POSIX workaround 2: Delimited string with IFS
list="item1:item2 with space:item3"
old_ifs=$IFS
IFS=":"
set -f
for item in $list; do
    echo "$item"
done
IFS=$old_ifs
set +f
```

Input/Output, Redirection, and File Descriptors

Standard Streams and File Descriptors

Standard Streams and File Descriptors

Linux processes inherit three default file descriptors: * `0`: `stdin` (Standard Input) * `1`: `stdout` (Standard Output) * `2`: `stderr` (Standard Error)

Custom descriptors (3 to 9) can be allocated for custom logging or teeing:

```
# Allocate FD 3 to write to a log file
exec 3> /var/log/custom-audit.log

# Write directly to FD 3
echo "Audit event at $(date)" >&3

# Close FD 3
exec 3>&-
```

Redirection Operators Explained

Redirection Operators Explained

```
# Overwrite stdout: >
echo "text" > file.txt

# Append stdout: >>
echo "text" >> file.txt

# Redirect stderr to stdout: 2>&1
command > output.log 2>&1

# Bash shorthand redirect both stdout and stderr: &>
command &> output.log

# Read stdin from file: <
grep "root" < /etc/passwd

# Open file for read/write on FD: <>
exec 3<> /tmp/scratch.dat
```

Pipelines and Command Chaining

Pipelines and Command Chaining

Pipelines connect the stdout of one process to the stdin of another:

```
cat /var/log/auth.log | grep "Failed password" | awk '{print $11}' | sort | uniq -c | sort -
```

Conditional Execution: && and ||

```
# Execute second command ONLY if first succeeds:
mkdir -p /tmp/deploy && cp dist.tar /tmp/deploy/

# Execute second command ONLY if first fails:
systemctl status nginx || systemctl restart nginx
```

Here-Documents and Here-Strings

Here-Documents and Here-Strings

Here-documents provide multi-line string input to commands:

```
# Expands variables
cat << EOF
User: $USER
Home: $HOME
EOF

# Quoted EOF: SUPPRESSES variable expansion (great for generating scripts/configs!)
cat << 'EOF' > /tmp/template.sh
#!/bin/bash
echo "Literal $VAR is not expanded during write"
EOF

# Here-Strings (<<<): Pass single strings to stdin without echo pipe
grep "root" <<< "$passwd_content"
```

Process Substitution and Named Pipes

Process Substitution and Named Pipes

Process substitution runs commands and exposes their output as a temporary file descriptor path (e.g. `/dev/fd/63`):

```
# Diff the output of two commands without temporary files on disk
diff -u <(systemctl list-unit-files --state=enabled) <(ssh remote-node systemctl list-unit-f

# Named Pipes (FIFOs):
mkfifo /tmp/myfifo
# Process 1 writes: cat /dev/urandom > /tmp/myfifo &
# Process 2 reads:  head -c 100 < /tmp/myfifo
rm /tmp/myfifo
```

Process and Job Management

Background Jobs and the & Operator

Background Jobs and the & Operator

Appending `&` to a command forks the process into the background, returning immediate control to the script:

```
# Start background worker
/opt/scripts/backup.sh /var/data &
BACKUP_PID=$!
echo "Backup running in background with PID: $BACKUP_PID"

# Built-in job management (interactive or set -m)
jobs -l
```

Waiting for Processes to Complete

Waiting for Processes to Complete

The `wait` builtin pauses execution until specified child processes terminate:

```
# Spawn multiple workers
pids=()
for target in node1 node2 node3; do
    ping -c 3 "$target" &
    pids+=($!)
done

# Wait for all background tasks and collect exit status
failed=0
for pid in "${pids[@]}"; do
    wait "$pid" || (( failed++ ))
done

echo "Failed pings: $failed"
```

Killing and Managing Child Processes

Killing and Managing Child Processes

Always terminate child processes gracefully using `SIGTERM` before resorting to `SIGKILL`:

```
kill -TERM "$PID" 2>/dev/null || true

# Enforce a hard timeout with GNU timeout
timeout --preserve-status --kill-after=5s 30s long_running_task
```

Process Groups and Sessions

Process Groups and Sessions

Killing a parent script does not automatically kill its spawned child processes unless you target the entire **Process Group**:

```
# Kill an entire process group (negative PID):
kill -TERM -- -$$
```

Monitoring Long-Running Operations

Monitoring Long-Running Operations

Creating terminal progress indicators for long-running scripts:

```
spin() {
    local pid=$1
    local chars=('/' '-' '\' '|')
    while kill -0 "$pid" 2>/dev/null; do
        for c in "${chars[@]}"; do
            printf "\r[%s] Working..." "$c"
            sleep 0.1
        done
    done
    printf "\r[ ] Done!         \n"
}

sleep 5 &
spin $!
```

Signals, Traps, and Cleanup

Understanding Unix Signals

Understanding Unix Signals

Signals are asynchronous software interrupts sent to a process by the kernel or other processes:

Signal	Number	Default Action	Description	Catchable?
`SIGHUP`	1	Terminate	Terminal hangup or daemon reload	Yes
`SIGINT`	2	Terminate	Keyboard interrupt (Ctrl+C)	Yes
`SIGQUIT`	3	Core Dump	Keyboard quit (Ctrl+)	Yes
`SIGKILL`	9	Immediate Terminate	Hard kernel uncatchable kill	No
`SIGTERM`	15	Terminate	Polite request to gracefully terminate	Yes
`SIGUSR1/SIGUSR2`	10/12	Terminate	User-defined custom hooks	Yes

The trap Command

The trap Command

`trap` establishes signal handlers inside shell scripts:

```
# Register a cleanup function for termination signals and standard script EXIT:
cleanup() {
    echo "Caught signal or exit, cleaning up..."
}
trap cleanup EXIT INT TERM HUP
```

Handling Common Signals in Scripts

Handling Common Signals in Scripts

```
# Graceful daemon loop responding to SIGHUP configuration reload:
reload_config() {
    echo "Reloading configuration..."
    source /etc/mydaemon.conf
}
trap reload_config HUP
```

Ensuring Cleanup Runs on Exit

Ensuring Cleanup Runs on Exit

The pseudo-signal `EXIT` (or `0`) runs when the shell exits for any reason (normal finish, `exit` call, or `set -e` abort):

```
TEMP_DIR=$(mktemp -d)
trap 'rm -rf "${TEMP_DIR}"' EXIT

# Even if a command fails below with set -e, TEMP_DIR is guaranteed to be deleted!
curl -sS https://example.com/data.tar.gz | tar -xz -C "$TEMP_DIR"
```

Signal Handling in Pipelines and Subshells

Signal Handling in Pipelines and Subshells

Traps defined in parent scripts are reset to default behavior inside subshells and asynchronous background commands unless explicitly re-declared or exported.

Error Handling and Defensive Scripting

Exit Codes and Their Meaning

Exit Codes and Their Meaning

- `0`: Success.
- `1`: General non-specific error.
- `2`: Misuse of shell builtins (syntax error).
- `126`: Command invoked cannot execute (permission problem).
- `127`: Command not found.
- `128+N`: Fatal error with signal `N` (e.g. 130 = 128 + 2 for SIGINT).

The set Command: -e, -u, -o pipefail

The set Command: -e, -u, -o pipefail

The **Unofficial Bash Strict Mode** ensures scripts fail fast on errors rather than continuing silently:

```
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

- `-e` (`set -o errexit`): Exit immediately if a pipeline returns a non-zero status.
- `-u` (`set -o nounset`): Treat unset variables as an error and exit immediately.
- `-o pipefail`: Return the exit status of the LAST command in a pipeline that failed (instead of ignoring upstream pipeline failures).

Validating Input and Preconditions

Validating Input and Preconditions

Always validate preconditions before executing destructive actions:

```
# Validate argument count
[[ $# -ge 2 ]] || { echo "Usage: $0 <src> <dest>" >&2; exit 1; }

# Validate file existence and permissions
[[ -f "$1" ]] || { echo "Error: File $1 does not exist" >&2; exit 2; }
[[ -r "$1" ]] || { echo "Error: File $1 is not readable" >&2; exit 3; }
```

Graceful Failure and Partial Success

Graceful Failure and Partial Success

Handling optional failures under `set -e`:

```
# Allow a command to fail without terminating the script:
status_output=$(systemctl is-active myapp 2>/dev/null) || true

# Try-catch idiom:
if ! run_risky_task; then
    echo "Task failed; executing fallback mitigation..."
    run_fallback_task
fi
```

Logging Errors Effectively

Logging Errors Effectively

Structured logging to standard error:

```
log() {
    local level="$1"; shift
    local ts
    ts=$(date -u +'%Y-%m-%dT%H:%M:%SZ')
    printf '{"time":"%s","level":"%s","message":"%s"}\n' "$ts" "$level" "$*" >&2
}

log INFO "Service initialization started"
log ERROR "Database connection timeout"
```

Debugging and Testing Scripts

Bash Built-In Debugging Options

Bash Built-In Debugging Options

- `bash -n script.sh`: Dry-run syntax check without executing commands.
- `bash -v script.sh`: Verbose mode (prints script lines as read).
- `bash -x script.sh`: Execution tracing (prints commands after expansions).

Using set -x Wisely

Using set -x Wisely

Wrap suspicious code sections selectively:

```
set -x # Enable tracing
risky_function "$param"
set +x # Disable tracing
```

Customize trace prefix for debugging source files and line numbers:

```
export PS4='+ ${BASH_SOURCE}:${LINENO}: ${FUNCNAME[0]:+${FUNCNAME[0]}(): }'
```

Static Analysis Tools for Shell Scripts

Static Analysis Tools for Shell Scripts

ShellCheck is the definitive static analysis tool for shell scripts:

```
shellcheck -s bash -e SC2086 script.sh
```

Embed inline override directives when an unquoted expansion is deliberate:

```
# shellcheck disable=SC2086
echo $intentional_word_split
```

Writing Tests for Shell Scripts

Writing Tests for Shell Scripts

Testing shell scripts using **Bats-core** (Bash Automated Testing System):

```
# test/math.bats
@test "addition calculates correctly" {
    result="$(./calc.sh 2 3)"
    [ "$result" -eq 5 ]
}

@test "missing argument throws exit code 1" {
    run ./calc.sh
    [ "$status" -eq 1 ]
}
```

Continuous Integration for Shell Scripts

Continuous Integration for Shell Scripts

GitHub Actions workflow for automated ShellCheck and Bats test execution:

```
name: Lint & Test Scripts
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run ShellCheck
        uses: ludeeus/action-shellcheck@master
      - name: Run Bats Tests
        run: sudo apt-get install -y bats && bats test/
```

Text Processing Essentials in Scripts

grep and Extended Pattern Matching in Scripts

grep and Extended Pattern Matching in Scripts

When using `grep` in scripts: * Use `grep -q` (quiet) for testing conditions without output: `if grep -q "pattern" file; then ...` * Use `grep -E` for modern ERE regexes. * Use `grep -F` (fixed strings) when searching literal text (avoids regex compilation overhead).

sed for Stream Editing in Scripts

sed for Stream Editing in Scripts

Safely editing configuration files with `sed`:

```
# Pick an alternative delimiter when editing paths to avoid escaping slashes:
sed -i 's|/usr/local/bin|/usr/bin|g' /etc/app.conf
```

awk for Field-Based Processing

1989

awk for Field-Based Processing

Extracting columnar data cleanly:

```
# Print memory usage percentage
free | awk '/Mem:/ {printf "%.1f%%\n", $3/$2 * 100}'
```

Safe File Discovery with find and xargs

Safe File Discovery with find and xargs

Filenames on Linux can contain newlines, spaces, and quotes. Always use null-byte delimiters:

```
find /var/log -type f -name "*.log" -mtime +30 -print0 | xargs -0 -r rm -f
```

Combining Tools in Complex Pipelines

Combining Tools in Complex Pipelines

Choosing between native Bash parameter expansions vs `sed` vs `awk`: * Single string transformations: Native parameter expansion (100x faster, zero fork). * Stream editing across files: `sed`. * Complex field-based tabular manipulation: `awk`. * Nested JSON/YAML: External tools like `jq` or `yq`.

POSIX Compatibility and Portability

1995

The POSIX Shell Standard

1996

The POSIX Shell Standard

The POSIX shell standard (IEEE Std 1003.1) guarantees compatibility across every Unix and Linux distribution. A POSIX script does not use Bash-specific extensions.

Bashisms vs POSIX Shell Features

Bashisms vs POSIX Shell Features

Feature	GNU Bash	POSIX sh (/bin/sh)
Test syntax	`[[ $a == $b ]]`	`[ "$a" = "$b" ]`
Arrays	`arr=(a b)`	None (use `$@` or IFS)
String replacement	`${var//find/replace}`	`echo "$var" | sed 's/find/replace/g'`
Function keyword	`function myfunc() { }`	`myfunc() { }`
Local variables	`local x=1`	Not standard POSIX (widely supported)
Regex match	`[[ $v =~ pattern ]]`	Use `expr` or `grep`

Audit scripts for accidental bashisms using `checkbashisms script.sh`.

Detecting the Running Shell

Detecting the Running Shell

```
if [ -n "$BASH_VERSION" ]; then
    echo "Running GNU Bash version $BASH_VERSION"
elif [ -n "$ZSH_VERSION" ]; then
    echo "Running Zsh version $ZSH_VERSION"
else
    echo "Running standard POSIX /bin/sh"
fi
```

Writing Portable Scripts with Strict Options

2002

Writing Portable Scripts with Strict Options

In `/bin/sh`, `-o pipefail` is not defined in standard POSIX. Use portable strict options:

```
#!/bin/sh
set -eu
```

When to Use #!/bin/sh vs #!/usr/bin/env bash

2004

When to Use #!/bin/sh vs #!/usr/bin/env bash

- `#!/usr/bin/env bash`: Best for scripts relying on Bash features (finds bash in PATH across Linux, macOS, FreeBSD).
- `#!/bin/sh`: Best for minimal, ultra-portable scripts that execute under Debian Dash or Alpine Busybox Ash.

Linux Distribution Differences

2006

Default Shell Versions Across Distributions

2007

Default Shell Versions Across Distributions

- **Debian & Ubuntu:** `/bin/sh` points to **Dash** (Debian Almquist Shell), an ultra-fast, minimal POSIX shell. Any bashism in `#!/bin/sh` will fail!
- **Alpine Linux:** `/bin/sh` and `/bin/bash` (if not installed) point to **BusyBox Ash**.
- **Fedora, RHEL, CentOS, Arch:** `/bin/sh` is a symlink to **GNU Bash** running in POSIX compatibility mode.

Path and Tooling Variations

Path and Tooling Variations

Never assume binaries reside in `/bin` vs `/usr/bin` vs `/sbin`. Always verify executable presence with `command -v`:

```
if ! command -v curl >/dev/null 2>&1; then
    echo "Error: curl is required but not installed." >&2
    exit 1
fi
```

Systemd vs SysVinit vs OpenRC in Scripts

2011

Systemd vs SysVinit vs OpenRC in Scripts

Abstracting service restarts across different distributions:

```
restart_service() {
    local svc="$1"
    if command -v systemctl >/dev/null 2>&1; then
        systemctl restart "$svc"
    elif command -v rc-service >/dev/null 2>&1; then
        rc-service "$svc" restart
    elif [ -x "/etc/init.d/$svc" ]; then
        "/etc/init.d/$svc" restart
    else
        echo "No supported init system found" >&2
        return 1
    fi
}
```

Package Managers in Scripts

2013

Package Managers in Scripts

Always pass non-interactive flags when automating package managers:

```
# Debian / Ubuntu
DEBIAN_FRONTEND=noninteractive apt-get install -y -q nginx

# Fedora / RHEL
dnf install -y -q nginx

# Alpine
apk add --no-cache nginx
```

Alpine Linux and BusyBox Considerations

Alpine Linux and BusyBox Considerations

BusyBox applets omit many GNU long options (e.g. `grep -P`, `sed -i ""` variations, `date -d`). Write scripts using single-letter flags when targeting Alpine container images.

Security in Shell Scripts

2017

Command Injection Vulnerabilities

2018

Command Injection Vulnerabilities

Never pass unsanitized input to `eval` or execute unquoted command substitutions:

```
# DANGEROUS: If user_input is "test; rm -rf /", eval executes both!
eval "echo $user_input"

# SAFE: Never use eval with untrusted input
printf '%s\n' "$user_input"
```

PATH Manipulation and Environment Attacks

2020

PATH Manipulation and Environment Attacks

Reset the PATH to a trusted system default at the start of privileged scripts:

```
export PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
unset IFS
```

Handling User Input Safely

Handling User Input Safely

Validate all user input against strict whitelists before usage:

```
validate_username() {
    local u="$1"
    [[ "$u" =~ ^[a-z_][a-z0-9_-]{0,31}$ ]] || {
        echo "Error: Invalid username format" >&2
        return 1
    }
}
```

Secrets and Credentials in Scripts

2024

Secrets and Credentials in Scripts

Never pass passwords or API keys as command line arguments (they are visible to all users via `ps aux` or `/proc/<pid>/cmdline`). Pass them via environment variables or stdin.

File Permissions, SUID, and Safe Temp Files

2026

File Permissions, SUID, and Safe Temp Files

Always create temporary files using `mktemp` with restrictive umask:

```
TMP_FILE=$(mktemp)
chmod 0600 "$TMP_FILE"
trap 'rm -f "$TMP_FILE"' EXIT
```

Performance Optimization and Concurrency

The Cost of Forking External Commands

The Cost of Forking External Commands

Forking an external command (like `expr` or `date`) inside a loop takes ~1-2ms per invocation. In a loop of 10,000 iterations, forking adds 15+ seconds of overhead!

Using Builtins Over External Tools

Using Builtins Over External Tools

- Use `$(( a + b ))` instead of `$(expr $a + $b)`
- Use `${var#*/}` instead of `$(basename "$var")`
- Use `printf` instead of external formatting tools

Minimizing Subshells and Efficient I/O

Minimizing Subshells and Efficient I/O

Read large files line-by-line without pipes:

```
while IFS= read -r line; do
    # Process line with zero subshell overhead
    :
done < /var/log/large_file.log
```

Parallel Execution with Background Jobs and Limits

Parallel Execution with Background Jobs and Limits

Throttling background jobs to a maximum concurrency pool:

```
MAX_JOBS=4
for url in "${urls[@]}"; do
    curl -sO "$url" &
    # If pool is full, wait for any one job to complete
    if [[ $(jobs -r -p | wc -l) -ge $MAX_JOBS ]]; then
        wait -n
    fi
done
wait
```

Synchronization, Locking, and Atomic Operations

Synchronization, Locking, and Atomic Operations

Prevent concurrent executions using `flock`:

```
exec 200>/var/lock/myscript.lock
flock -n 200 || { echo "Script is already running!" >&2; exit 1; }
```

Automation, DevOps, and CI/CD

Shell Scripts in CI/CD Pipelines

Shell Scripts in CI/CD Pipelines

Best practices for CI runners: * Fail fast with `set -euo pipefail`. * Print group headers for log collapsing: `echo "::group::Build Steps"`. * Always export exit codes faithfully.

Infrastructure as Code and Shell Automation

Infrastructure as Code and Shell Automation

Writing idempotent cloud-init provisioning scripts that can be safely re-run without causing unintended side effects.

Container Build Scripts and Entrypoints

Container Build Scripts and Entrypoints

Always end Docker `entrypoint.sh` scripts with `exec "$@"` to pass PID 1 to the application so it properly receives OS signals:

```
#!/bin/sh
set -e
# Perform migrations or config generation
exec "$@"
```

Configuration Management Integration

Configuration Management Integration

Templating configuration files with `envsubst`:

```
export DB_HOST="db.prod.internal"
envsubst '${DB_HOST}' < config.template > /etc/config.json
```

Monitoring and Alerting Scripts

Monitoring and Alerting Scripts

Automating webhook notifications to Slack / Discord on incident detection:

```
send_alert() {
    local msg="$1"
    curl -s -X POST -H 'Content-type: application/json' \
        --data "{\"text\":\"$msg\"}" "$SLACK_WEBHOOK_URL"
}
```

System Administration Patterns

User and Group Management Scripts

User and Group Management Scripts

Batch provisioning Linux user accounts with SSH public keys and standard secondary groups.

Log Processing and Rotation

Log Processing and Rotation

Archiving and compressing rotated logs with `zstd`:

```
find /var/log/apps -type f -name "*.log" -mtime +7 -exec zstd --rm {} +
```

Backup and Restore Procedures

Backup and Restore Procedures

Automating incremental snapshots using `rsync` with hard links (`--link-dest`).

Service Health Checks with Retries

Service Health Checks with Retries

Exponential backoff health check loop:

```
wait_for_service() {
    local url="$1"
    local retries=5
    local wait_sec=2
    until curl -sSf "$url" >/dev/null || [ $retries -eq 0 ]; do
        echo "Service unavailable, retrying in ${wait_sec}s..."
        sleep "$wait_sec"
        (( retries-- ))
        (( wait_sec *= 2 ))
    done
}
```

Disk Space and Resource Management

Disk Space and Resource Management

Alerting when disk usage exceeds safety thresholds:

```
df -H | grep -vE '^Filesystem|tmpfs|cdrom' | awk '{ print $5 " " $1 }' | while read -r outpu
    usage=$(echo "$output" | awk '{ print $1}' | cut -d'%' -f1)
    partition=$(echo "$output" | awk '{ print $2 }')
    if [ "$usage" -ge 90 ]; then
        echo "CRITICAL: Partition $partition is at ${usage}% capacity!" >&2
    fi
done
```

Workbook

Basic Bash Examples

Basic Bash Examples

Basic Bash Examples

This section contains fundamental Bash scripting examples to help you practice and understand core concepts.

Hello World Variations

Simple Hello World

```
#!/bin/bash
echo "Hello, World!"
```

Hello World with Variables

```
#!/bin/bash
name="World"
echo "Hello, $name!"
```

Interactive Hello World

```
#!/bin/bash
echo "What's your name?"
read name
echo "Hello, $name! Nice to meet you."
```

Hello World with Date

```
#!/bin/bash
echo "Hello, World!"
echo "Today is: $(date)"
echo "Current user: $(whoami)"
```

Variable Examples

Basic Variable Operations

```
#!/bin/bash
# Variable assignment
first_name="John"
last_name="Doe"
age=25

# Variable usage
echo "Name: $first_name $last_name"
echo "Age: $age"

# Variable concatenation
full_name="$first_name $last_name"
echo "Full name: $full_name"
```

Environment Variables

```
#!/bin/bash
echo "Your home directory: $HOME"
echo "Your username: $USER"
echo "Your shell: $SHELL"
echo "Current path: $PATH"
```

Variable Default Values

```
#!/bin/bash
# Set default values
database_host=${DB_HOST:-"localhost"}
database_port=${DB_PORT:-"5432"}
database_name=${DB_NAME:-"myapp"}

echo "Database configuration:"
echo "Host: $database_host"
echo "Port: $database_port"
echo "Database: $database_name"
```

Variable Length and Manipulation

```
#!/bin/bash
text="Hello World"

echo "Original text: $text"
echo "Length: ${#text}"
echo "Uppercase: ${text^^}"
echo "Lowercase: ${text,,}"
echo "First 5 characters: ${text:0:5}"
echo "Last 5 characters: ${text: -5}"
```

Command Line Arguments

Basic Argument Handling

```
#!/bin/bash
echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "All arguments: $@"
echo "Number of arguments: $#"
```

Argument Validation

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <name> [age]"
    exit 1
fi

name="$1"
age="${2:-unknown}"

echo "Name: $name"
echo "Age: $age"
```

Processing Multiple Arguments

```
#!/bin/bash
echo "Processing all arguments:"
for arg in "$@"; do
    echo "Argument: $arg"
done
```

Simple Calculations

Basic Arithmetic

```
#!/bin/bash
a=10
b=5

echo "a = $a, b = $b"
echo "Addition: $((a + b))"
echo "Subtraction: $((a - b))"
echo "Multiplication: $((a * b))"
echo "Division: $((a / b))"
echo "Modulo: $((a % b))"
```

Calculator Script

```
#!/bin/bash
echo "Simple Calculator"
read -p "Enter first number: " num1
read -p "Enter operator (+, -, *, /): " op
read -p "Enter second number: " num2

case $op in
    +) result=$((num1 + num2)) ;;
    -) result=$((num1 - num2)) ;;
    \*) result=$((num1 * num2)) ;;
    /)
        if [ $num2 -eq 0 ]; then
            echo "Error: Division by zero"
            exit 1
        fi
        result=$((num1 / num2))
        ;;
    *) echo "Invalid operator"; exit 1 ;;
```

```
esac

echo "Result: $num1 $op $num2 = $result"
```

Temperature Converter

```
#!/bin/bash
echo "Temperature Converter"
echo "1. Celsius to Fahrenheit"
echo "2. Fahrenheit to Celsius"
read -p "Choose option (1 or 2): " choice

case $choice in
    1)
        read -p "Enter temperature in Celsius: " celsius
        fahrenheit=$(echo "scale=2; $celsius * 9/5 + 32" | bc)
        echo "$celsius°C = $fahrenheit°F"
        ;;
    2)
        read -p "Enter temperature in Fahrenheit: " fahrenheit
        celsius=$(echo "scale=2; ($fahrenheit - 32) * 5/9" | bc)
        echo "$fahrenheit°F = $celsius°C"
        ;;
    *)
        echo "Invalid choice"
        exit 1
        ;;
esac
```

File Operations

File Information

```
#!/bin/bash
filename="$1"

if [ -z "$filename" ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

if [ -f "$filename" ]; then
    echo "File: $filename"
```

```
    echo "Size: $(stat -c%s "$filename") bytes"
    echo "Last modified: $(stat -c%y "$filename")"
    echo "Permissions: $(stat -c%A "$filename")"
    echo "Owner: $(stat -c%U "$filename")"
    echo "Group: $(stat -c%G "$filename")"
else
    echo "File '$filename' does not exist"
fi
```

File Backup

```
#!/bin/bash
source_file="$1"

if [ -z "$source_file" ]; then
    echo "Usage: $0 <source_file>"
    exit 1
fi

if [ ! -f "$source_file" ]; then
    echo "Error: File '$source_file' does not exist"
    exit 1
fi

backup_file="${source_file}.backup.$(date +%Y%m%d_%H%M%S)"

if cp "$source_file" "$backup_file"; then
    echo "Backup created: $backup_file"
else
    echo "Error: Failed to create backup"
    exit 1
fi
```

Directory Listing

```
#!/bin/bash
directory="${1:-.}"

echo "Contents of directory: $directory"
echo "================================="

if [ -d "$directory" ]; then
    for item in "$directory"/*; do
        if [ -f "$item" ]; then
```

```
            echo "FILE: $(basename "$item")"
        elif [ -d "$item" ]; then
            echo "DIR:  $(basename "$item")"
        fi
    done
else
    echo "Error: '$directory' is not a directory"
fi
```

Text Processing

Word Count

```
#!/bin/bash
filename="$1"

if [ -z "$filename" ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

if [ -f "$filename" ]; then
    lines=$(wc -l < "$filename")
    words=$(wc -w < "$filename")
    chars=$(wc -c < "$filename")

    echo "File: $filename"
    echo "Lines: $lines"
    echo "Words: $words"
    echo "Characters: $chars"
else
    echo "File '$filename' does not exist"
fi
```

Text Search

```
#!/bin/bash
if [ $# -ne 2 ]; then
    echo "Usage: $0 <search_term> <filename>"
    exit 1
fi

search_term="$1"
```

```
filename="$2"

if [ ! -f "$filename" ]; then
    echo "File '$filename' does not exist"
    exit 1
fi

echo "Searching for '$search_term' in '$filename':"
grep -n "$search_term" "$filename"

count=$(grep -c "$search_term" "$filename")
echo "Found $count occurrences"
```

Line Numbering

```
#!/bin/bash
filename="$1"

if [ -z "$filename" ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

if [ -f "$filename" ]; then
    line_number=1
    while IFS= read -r line; do
        printf "%3d: %s\n" $line_number "$line"
        ((line_number++))
    done < "$filename"
else
    echo "File '$filename' does not exist"
fi
```

System Information

System Status

```
#!/bin/bash
echo "=== System Information ==="
echo "Hostname: $(hostname)"
echo "Uptime: $(uptime -p)"
echo "Current user: $(whoami)"
echo "Current directory: $(pwd)"
```

```
echo "Date: $(date)"
echo

echo "=== Memory Usage ==="
free -h

echo
echo "=== Disk Usage ==="
df -h

echo
echo "=== CPU Information ==="
lscpu | grep "Model name"
```

Process Monitor

```
#!/bin/bash
echo "Top 10 processes by CPU usage:"
ps aux --sort=-%cpu | head -11

echo
echo "Top 10 processes by memory usage:"
ps aux --sort=-%mem | head -11
```

Network Information

```
#!/bin/bash
echo "=== Network Information ==="
echo "Hostname: $(hostname)"
echo "Domain: $(hostname -d 2>/dev/null || echo "Not set")"

echo
echo "=== Network Interfaces ==="
ip addr show | grep -E "^[0-9]+:|inet "

echo
echo "=== Routing Table ==="
ip route show
```

User Input Examples

Menu System

```
#!/bin/bash
while true; do
    echo
    echo "=== Main Menu ==="
    echo "1. Show date and time"
    echo "2. Show current directory"
    echo "3. List files"
    echo "4. Show disk usage"
    echo "5. Exit"
    echo
    read -p "Choose an option (1-5): " choice

    case $choice in
        1) date ;;
        2) pwd ;;
        3) ls -la ;;
        4) df -h ;;
        5) echo "Goodbye!"; exit 0 ;;
        *) echo "Invalid choice. Please try again." ;;
    esac

    read -p "Press Enter to continue..."
done
```

User Registration

```
#!/bin/bash
echo "User Registration Form"
echo "======================"

read -p "First Name: " first_name
read -p "Last Name: " last_name
read -p "Email: " email
read -p "Age: " age
read -s -p "Password: " password
echo

echo
echo "Registration Summary:"
echo "Name: $first_name $last_name"
echo "Email: $email"
```

```
echo "Age: $age"
echo "Password: [hidden]"

read -p "Is this information correct? (y/n): " confirm
if [ "$confirm" = "y" ] || [ "$confirm" = "Y" ]; then
    echo "Registration completed!"
else
    echo "Registration cancelled."
fi
```

Quiz Game

```
#!/bin/bash
score=0
total_questions=3

echo "Welcome to the Bash Quiz!"
echo "========================"

# Question 1
echo "Question 1: What command shows the current directory?"
echo "a) pwd"
echo "b) cd"
echo "c) ls"
read -p "Your answer: " answer1

if [ "$answer1" = "a" ] || [ "$answer1" = "A" ]; then
    echo "Correct!"
    ((score++))
else
    echo "Wrong! The correct answer is 'a) pwd'"
fi

# Question 2
echo
echo "Question 2: What symbol is used for comments in bash?"
echo "a) //"
echo "b) #"
echo "c) /*"
read -p "Your answer: " answer2

if [ "$answer2" = "b" ] || [ "$answer2" = "B" ]; then
    echo "Correct!"
    ((score++))
else
```

```
    echo "Wrong! The correct answer is 'b) #'"
fi

# Question 3
echo
echo "Question 3: What command lists files and directories?"
echo "a) list"
echo "b) dir"
echo "c) ls"
read -p "Your answer: " answer3

if [ "$answer3" = "c" ] || [ "$answer3" = "C" ]; then
    echo "Correct!"
    ((score++))
else
    echo "Wrong! The correct answer is 'c) ls'"
fi

echo
echo "Quiz completed!"
echo "Your score: $score out of $total_questions"

percentage=$((score * 100 / total_questions))
if [ $percentage -ge 80 ]; then
    echo "Excellent work!"
elif [ $percentage -ge 60 ]; then
    echo "Good job!"
else
    echo "Keep practicing!"
fi
```

String Manipulation

String Operations

```
#!/bin/bash
text="Hello World Programming"

echo "Original: $text"
echo "Length: ${#text}"
echo "Uppercase: ${text^^}"
echo "Lowercase: ${text,,}"
echo "First word: ${text% *}"
echo "Last word: ${text##* }"
echo "Remove 'Hello': ${text/Hello/}"
```

```
echo "Replace 'World' with 'Bash': ${text/World/Bash}"
echo "First 10 chars: ${text:0:10}"
echo "From position 6: ${text:6}"
```

Palindrome Checker

```
#!/bin/bash
read -p "Enter a word to check if it's a palindrome: " word

# Convert to lowercase and remove spaces
clean_word=$(echo "$word" | tr '[:upper:]' '[:lower:]' | tr -d ' ')

# Reverse the string
reversed=$(echo "$clean_word" | rev)

if [ "$clean_word" = "$reversed" ]; then
    echo "'$word' is a palindrome!"
else
    echo "'$word' is not a palindrome."
fi
```

Password Generator

```
#!/bin/bash
read -p "Enter password length (default 12): " length
length=${length:-12}

# Generate password using /dev/urandom
password=$(tr -dc 'A-Za-z0-9!@#$%^&*' < /dev/urandom | head -c "$length")

echo "Generated password: $password"
echo "Password strength:"
echo "Length: $length characters"

# Check password strength
if [[ $password =~ [A-Z] ]]; then
    echo "  Contains uppercase letters"
fi

if [[ $password =~ [a-z] ]]; then
    echo "  Contains lowercase letters"
fi

if [[ $password =~ [0-9] ]]; then
```

```
    echo "  Contains numbers"
fi

if [[ $password =~ [!@#\$%\^&\*] ]]; then
    echo "  Contains special characters"
fi
```

Array Examples

Basic Array Operations

```
#!/bin/bash
# Create array
fruits=("apple" "banana" "orange" "grape")

echo "All fruits: ${fruits[@]}"
echo "Number of fruits: ${#fruits[@]}"
echo "First fruit: ${fruits[0]}"
echo "Last fruit: ${fruits[-1]}"

# Add element
fruits+=("mango")
echo "After adding mango: ${fruits[@]}"

# Loop through array
echo "Fruits list:"
for i in "${!fruits[@]}"; do
    echo "$((i+1)). ${fruits[i]}"
done
```

Shopping List

```
#!/bin/bash
declare -a shopping_list

echo "Shopping List Manager"
echo "====================="

while true; do
    echo
    echo "1. Add item"
    echo "2. Remove item"
    echo "3. Show list"
```

```
echo "4. Clear list"
echo "5. Exit"
read -p "Choose option: " choice

case $choice in
    1)
        read -p "Enter item to add: " item
        shopping_list+=("$item")
        echo "Added '$item' to the list"
        ;;
    2)
        if [ ${#shopping_list[@]} -eq 0 ]; then
            echo "List is empty"
        else
            echo "Current items:"
            for i in "${!shopping_list[@]}"; do
                echo "$((i+1)). ${shopping_list[i]}"
            done
            read -p "Enter item number to remove: " num
            if [ "$num" -ge 1 ] && [ "$num" -le ${#shopping_list[@]} ]; then
                removed_item="${shopping_list[$((num-1))]}"
                unset shopping_list[$((num-1))]
                shopping_list=("${shopping_list[@]}")  # Re-index array
                echo "Removed '$removed_item'"
            else
                echo "Invalid item number"
            fi
        fi
        ;;
    3)
        if [ ${#shopping_list[@]} -eq 0 ]; then
            echo "Shopping list is empty"
        else
            echo "Shopping List:"
            for i in "${!shopping_list[@]}"; do
                echo "$((i+1)). ${shopping_list[i]}"
            done
        fi
        ;;
    4)
        shopping_list=()
        echo "List cleared"
        ;;
    5)
        echo "Goodbye!"
        exit 0
        ;;
```

```
        *)
            echo "Invalid choice"
            ;;
    esac
done
```

Grade Calculator

```
#!/bin/bash
declare -a grades
declare -a subjects

echo "Grade Calculator"
echo "================"

read -p "How many subjects? " num_subjects

for ((i=1; i<=num_subjects; i++)); do
    read -p "Enter subject $i name: " subject
    read -p "Enter grade for $subject: " grade

    subjects+=("$subject")
    grades+=("$grade")
done

# Calculate average
total=0
for grade in "${grades[@]}"; do
    total=$((total + grade))
done

average=$((total / num_subjects))

echo
echo "Grade Report:"
echo "============="
for i in "${!subjects[@]}"; do
    echo "${subjects[i]}: ${grades[i]}"
done

echo
echo "Average: $average"

if [ $average -ge 90 ]; then
    echo "Grade: A (Excellent)"
```

```
elif [ $average -ge 80 ]; then
    echo "Grade: B (Good)"
elif [ $average -ge 70 ]; then
    echo "Grade: C (Average)"
elif [ $average -ge 60 ]; then
    echo "Grade: D (Below Average)"
else
    echo "Grade: F (Fail)"
fi
```

Intermediate Bash Examples

Intermediate Bash Examples

Intermediate Bash Examples

This section contains more advanced Bash scripting examples that demonstrate intermediate concepts and real-world applications.

Function Examples

Library of Utility Functions

```
#!/bin/bash
# utils.sh - Utility functions library

# Logging function with levels
log() {
    local level="$1"
    shift
    local message="$*"
    local timestamp=$(date '+%Y-%m-%d %H:%M:%S')

    case "$level" in
        ERROR) echo "[$timestamp] ERROR: $message" >&2 ;;
        WARN)  echo "[$timestamp] WARN:  $message" ;;
        INFO)  echo "[$timestamp] INFO:  $message" ;;
        DEBUG) [ "$DEBUG" = "1" ] && echo "[$timestamp] DEBUG: $message" ;;
    esac
}

# Check if command exists
command_exists() {
    command -v "$1" >/dev/null 2>&1
}

# Validate email format
validate_email() {
    local email="$1"
    [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]]
}

# Get file extension
```

```
get_extension() {
    local filename="$1"
    echo "${filename##*.}"
}

# Convert bytes to human readable format
bytes_to_human() {
    local bytes="$1"
    local units=("B" "KB" "MB" "GB" "TB")
    local unit=0

    while [ $bytes -gt 1024 ] && [ $unit -lt 4 ]; do
        bytes=$((bytes / 1024))
        ((unit++))
    done

    echo "$bytes ${units[$unit]}"
}

# Create backup with timestamp
backup_file() {
    local file="$1"
    local backup_dir="${2:-./backups}"

    if [ ! -f "$file" ]; then
        log ERROR "File '$file' does not exist"
        return 1
    fi

    mkdir -p "$backup_dir"
    local backup_name="$(basename "$file").backup.$(date +%Y%m%d_%H%M%S)"

    if cp "$file" "$backup_dir/$backup_name"; then
        log INFO "Backup created: $backup_dir/$backup_name"
        echo "$backup_dir/$backup_name"
        return 0
    else
        log ERROR "Failed to create backup"
        return 1
    fi
}

# Example usage
if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then
    # Test the functions
    log INFO "Testing utility functions"
```

```
    if command_exists "git"; then
        log INFO "Git is available"
    else
        log WARN "Git is not installed"
    fi

    if validate_email "user@example.com"; then
        log INFO "Email format is valid"
    else
        log ERROR "Invalid email format"
    fi

    echo "File extension of script.sh: $(get_extension "script.sh")"
    echo "1048576 bytes = $(bytes_to_human 1048576)"
fi
```

Configuration Manager

```
#!/bin/bash
# config_manager.sh - Configuration file manager

CONFIG_FILE="app.conf"

# Default configuration
declare -A config=(
    [app_name]="MyApp"
    [version]="1.0.0"
    [debug]="false"
    [port]="8080"
    [database_host]="localhost"
    [database_port]="5432"
    [log_level]="INFO"
)

# Load configuration from file
load_config() {
    if [ -f "$CONFIG_FILE" ]; then
        while IFS='=' read -r key value; do
            # Skip comments and empty lines
            [[ $key =~ ^[[:space:]]*# ]] && continue
            [[ -z $key ]] && continue

            # Remove leading/trailing whitespace
            key=$(echo "$key" | xargs)
            value=$(echo "$value" | xargs)
```

```
            config["$key"]="$value"
        done < "$CONFIG_FILE"
        echo "Configuration loaded from $CONFIG_FILE"
    else
        echo "Configuration file not found, using defaults"
    fi
}

# Save configuration to file
save_config() {
    echo "# Application Configuration" > "$CONFIG_FILE"
    echo "# Generated on $(date)" >> "$CONFIG_FILE"
    echo >> "$CONFIG_FILE"

    for key in "${!config[@]}"; do
        echo "$key=${config[$key]}" >> "$CONFIG_FILE"
    done

    echo "Configuration saved to $CONFIG_FILE"
}

# Get configuration value
get_config() {
    local key="$1"
    echo "${config[$key]}"
}

# Set configuration value
set_config() {
    local key="$1"
    local value="$2"
    config["$key"]="$value"
    echo "Set $key = $value"
}

# Show all configuration
show_config() {
    echo "Current Configuration:"
    echo "======================"
    for key in "${!config[@]}"; do
        printf "%-20s = %s\n" "$key" "${config[$key]}"
    done
}

# Interactive configuration editor
edit_config() {
```

```
    while true; do
        echo
        echo "Configuration Editor"
        echo "===================="
        echo "1. Show current config"
        echo "2. Edit a value"
        echo "3. Save config"
        echo "4. Load config"
        echo "5. Exit"

        read -p "Choose option: " choice

        case $choice in
            1) show_config ;;
            2)
                read -p "Enter key to edit: " key
                if [[ -n "${config[$key]}" ]]; then
                    echo "Current value: ${config[$key]}"
                    read -p "Enter new value: " value
                    set_config "$key" "$value"
                else
                    echo "Key '$key' not found"
                fi
                ;;
            3) save_config ;;
            4) load_config ;;
            5) break ;;
            *) echo "Invalid choice" ;;
        esac
    done
}

# Main execution
load_config

if [ $# -eq 0 ]; then
    edit_config
else
    case "$1" in
        get) get_config "$2" ;;
        set) set_config "$2" "$3" ;;
        show) show_config ;;
        save) save_config ;;
        load) load_config ;;
        *) echo "Usage: $0 {get|set|show|save|load} [key] [value]" ;;
    esac
fi
```

Database Backup Script

```
#!/bin/bash
# db_backup.sh - Database backup script with rotation

# Configuration
DB_HOST="${DB_HOST:-localhost}"
DB_PORT="${DB_PORT:-5432}"
DB_USER="${DB_USER:-postgres}"
DB_NAME="${DB_NAME:-myapp}"
BACKUP_DIR="${BACKUP_DIR:-/backup/database}"
RETENTION_DAYS="${RETENTION_DAYS:-7}"
COMPRESSION="${COMPRESSION:-gzip}"

# Logging function
log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $*" | tee -a "$BACKUP_DIR/backup.log"
}

# Check dependencies
check_dependencies() {
    local missing_deps=()

    if ! command -v pg_dump >/dev/null 2>&1; then
        missing_deps+=("postgresql-client")
    fi

    if [ "$COMPRESSION" = "gzip" ] && ! command -v gzip >/dev/null 2>&1; then
        missing_deps+=("gzip")
    fi

    if [ ${#missing_deps[@]} -gt 0 ]; then
        log "ERROR: Missing dependencies: ${missing_deps[*]}"
        exit 1
    fi
}

# Create backup directory
setup_backup_dir() {
    if [ ! -d "$BACKUP_DIR" ]; then
        mkdir -p "$BACKUP_DIR"
        log "INFO: Created backup directory: $BACKUP_DIR"
    fi
}

# Perform database backup
```

```
backup_database() {
    local timestamp=$(date +%Y%m%d_%H%M%S)
    local backup_file="$BACKUP_DIR/${DB_NAME}_${timestamp}.sql"

    log "INFO: Starting backup of database '$DB_NAME'"

    # Set password if provided
    if [ -n "$DB_PASSWORD" ]; then
        export PGPASSWORD="$DB_PASSWORD"
    fi

    # Perform backup
    if pg_dump -h "$DB_HOST" -p "$DB_PORT" -U "$DB_USER" -d "$DB_NAME" > "$backup_file"; the
        log "INFO: Database backup completed: $backup_file"

        # Compress if requested
        if [ "$COMPRESSION" = "gzip" ]; then
            if gzip "$backup_file"; then
                backup_file="${backup_file}.gz"
                log "INFO: Backup compressed: $backup_file"
            else
                log "WARN: Compression failed"
            fi
        fi

        # Calculate file size
        local file_size=$(stat -c%s "$backup_file" 2>/dev/null || stat -f%z "$backup_file" 2
        log "INFO: Backup size: $(numfmt --to=iec-i --suffix=B $file_size)"

        echo "$backup_file"
    else
        log "ERROR: Database backup failed"
        return 1
    fi
}

# Clean old backups
cleanup_old_backups() {
    log "INFO: Cleaning up backups older than $RETENTION_DAYS days"

    local deleted_count=0
    while IFS= read -r -d '' file; do
        rm "$file"
        log "INFO: Deleted old backup: $(basename "$file")"
        ((deleted_count++))
    done < <(find "$BACKUP_DIR" -name "${DB_NAME}_*.sql*" -mtime +$RETENTION_DAYS -print0)
```

```
    log "INFO: Deleted $deleted_count old backup(s)"
}

# Verify backup integrity
verify_backup() {
    local backup_file="$1"

    if [ ! -f "$backup_file" ]; then
        log "ERROR: Backup file not found: $backup_file"
        return 1
    fi

    # Check if file is compressed
    if [[ "$backup_file" == *.gz ]]; then
        if gzip -t "$backup_file"; then
            log "INFO: Backup file integrity verified (compressed)"
            return 0
        else
            log "ERROR: Backup file is corrupted (compressed)"
            return 1
        fi
    else
        # Check SQL file for basic structure
        if grep -q "PostgreSQL database dump" "$backup_file"; then
            log "INFO: Backup file integrity verified"
            return 0
        else
            log "ERROR: Backup file appears to be corrupted"
            return 1
        fi
    fi
}

# Send notification (email or webhook)
send_notification() {
    local status="$1"
    local message="$2"

    if [ -n "$NOTIFICATION_EMAIL" ] && command -v mail >/dev/null 2>&1; then
        echo "$message" | mail -s "Database Backup $status" "$NOTIFICATION_EMAIL"
        log "INFO: Notification sent to $NOTIFICATION_EMAIL"
    fi

    if [ -n "$WEBHOOK_URL" ] && command -v curl >/dev/null 2>&1; then
        curl -X POST -H "Content-Type: application/json" \
             -d "{\"text\":\"Database Backup $status: $message\"}" \
             "$WEBHOOK_URL" >/dev/null 2>&1
```

```
        log "INFO: Webhook notification sent"
    fi
}

# Main backup process
main() {
    log "INFO: Starting database backup process"

    check_dependencies
    setup_backup_dir

    if backup_file=$(backup_database); then
        if verify_backup "$backup_file"; then
            cleanup_old_backups
            send_notification "SUCCESS" "Database backup completed successfully: $(basename
            log "INFO: Backup process completed successfully"
        else
            send_notification "FAILED" "Backup verification failed"
            log "ERROR: Backup verification failed"
            exit 1
        fi
    else
        send_notification "FAILED" "Database backup failed"
        log "ERROR: Backup process failed"
        exit 1
    fi
}

# Handle command line arguments
case "${1:-backup}" in
    backup) main ;;
    verify) verify_backup "$2" ;;
    cleanup) cleanup_old_backups ;;
    *)
        echo "Usage: $0 {backup|verify|cleanup}"
        echo "Environment variables:"
        echo "  DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME"
        echo "  BACKUP_DIR, RETENTION_DAYS, COMPRESSION"
        echo "  NOTIFICATION_EMAIL, WEBHOOK_URL"
        exit 1
        ;;
esac
```

File Processing Examples

Log Analyzer

```
#!/bin/bash
# log_analyzer.sh - Analyze web server logs

LOG_FILE="${1:-/var/log/apache2/access.log}"
OUTPUT_DIR="${2:-./log_analysis}"

# Check if log file exists
if [ ! -f "$LOG_FILE" ]; then
    echo "Error: Log file '$LOG_FILE' not found"
    exit 1
fi

# Create output directory
mkdir -p "$OUTPUT_DIR"

echo "Analyzing log file: $LOG_FILE"
echo "Output directory: $OUTPUT_DIR"
echo "================================="

# Basic statistics
echo "Generating basic statistics..."
total_requests=$(wc -l < "$LOG_FILE")
echo "Total requests: $total_requests" > "$OUTPUT_DIR/summary.txt"

# Unique IP addresses
unique_ips=$(awk '{print $1}' "$LOG_FILE" | sort -u | wc -l)
echo "Unique IP addresses: $unique_ips" >> "$OUTPUT_DIR/summary.txt"

# Date range
first_date=$(head -1 "$LOG_FILE" | awk '{print $4}' | tr -d '[')
last_date=$(tail -1 "$LOG_FILE" | awk '{print $4}' | tr -d '[')
echo "Date range: $first_date to $last_date" >> "$OUTPUT_DIR/summary.txt"

# Top 10 IP addresses
echo "Generating top IP addresses..."
awk '{print $1}' "$LOG_FILE" | sort | uniq -c | sort -nr | head -10 > "$OUTPUT_DIR/top_ips.t

# Top 10 requested pages
echo "Generating top requested pages..."
awk '{print $7}' "$LOG_FILE" | sort | uniq -c | sort -nr | head -10 > "$OUTPUT_DIR/top_pages

# HTTP status codes
```

```
echo "Analyzing HTTP status codes..."
awk '{print $9}' "$LOG_FILE" | sort | uniq -c | sort -nr > "$OUTPUT_DIR/status_codes.txt"

# User agents
echo "Analyzing user agents..."
awk -F'"' '{print $6}' "$LOG_FILE" | sort | uniq -c | sort -nr | head -20 > "$OUTPUT_DIR/use

# Hourly traffic distribution
echo "Generating hourly traffic distribution..."
awk '{print $4}' "$LOG_FILE" | cut -d: -f2 | sort | uniq -c > "$OUTPUT_DIR/hourly_traffic.tx

# Error analysis (4xx and 5xx)
echo "Analyzing errors..."
awk '$9 ~ /^[45]/ {print $9, $7}' "$LOG_FILE" | sort | uniq -c | sort -nr > "$OUTPUT_DIR/err

# Bandwidth usage (if log format includes bytes)
echo "Calculating bandwidth usage..."
if awk '{print $10}' "$LOG_FILE" | head -1 | grep -q '^[0-9]*$'; then
    total_bytes=$(awk '{sum += $10} END {print sum}' "$LOG_FILE")
    echo "Total bandwidth: $(numfmt --to=iec-i --suffix=B $total_bytes)" >> "$OUTPUT_DIR/sum
fi

# Generate HTML report
echo "Generating HTML report..."
cat > "$OUTPUT_DIR/report.html" << EOF
<!DOCTYPE html>
<html>
<head>
    <title>Log Analysis Report</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; }
        .section { margin: 20px 0; }
        .data { background: #f5f5f5; padding: 10px; border-radius: 5px; }
        pre { white-space: pre-wrap; }
    </style>
</head>
<body>
    <h1>Log Analysis Report</h1>
    <p>Generated on: $(date)</p>

    <div class="section">
        <h2>Summary</h2>
        <div class="data">
            <pre>$(cat "$OUTPUT_DIR/summary.txt")</pre>
        </div>
    </div>
```

```
    <div class="section">
        <h2>Top 10 IP Addresses</h2>
        <div class="data">
            <pre>$(cat "$OUTPUT_DIR/top_ips.txt")</pre>
        </div>
    </div>

    <div class="section">
        <h2>Top 10 Requested Pages</h2>
        <div class="data">
            <pre>$(cat "$OUTPUT_DIR/top_pages.txt")</pre>
        </div>
    </div>

    <div class="section">
        <h2>HTTP Status Codes</h2>
        <div class="data">
            <pre>$(cat "$OUTPUT_DIR/status_codes.txt")</pre>
        </div>
    </div>
</body>
</html>
EOF

echo "Analysis complete! Results saved in: $OUTPUT_DIR"
echo "View the HTML report: $OUTPUT_DIR/report.html"
```

File Organizer

```
#!/bin/bash
# file_organizer.sh - Organize files by type and date

SOURCE_DIR="${1:-.}"
TARGET_DIR="${2:-./organized}"
DRY_RUN="${DRY_RUN:-false}"

# File type associations
declare -A file_types=(
    ["jpg,jpeg,png,gif,bmp,tiff"]="Images"
    ["mp4,avi,mkv,mov,wmv,flv"]="Videos"
    ["mp3,wav,flac,aac,ogg"]="Audio"
    ["pdf,doc,docx,txt,rtf,odt"]="Documents"
    ["zip,rar,7z,tar,gz,bz2"]="Archives"
    ["exe,msi,deb,rpm,dmg"]="Executables"
    ["html,css,js,php,py,java,cpp,c"]="Code"
```

```
)

# Logging function
log() {
    echo "$(date '+%H:%M:%S') - $*"
}

# Get file category based on extension
get_file_category() {
    local filename="$1"
    local extension="${filename##*.}"
    extension=$(echo "$extension" | tr '[:upper:]' '[:lower:]')

    for types in "${!file_types[@]}"; do
        if [[ ",$types," == *",$extension,"* ]]; then
            echo "${file_types[$types]}"
            return
        fi
    done

    echo "Other"
}

# Create directory structure
create_directory() {
    local dir="$1"
    if [ "$DRY_RUN" = "true" ]; then
        log "DRY RUN: Would create directory: $dir"
    else
        mkdir -p "$dir"
        log "Created directory: $dir"
    fi
}

# Move file
move_file() {
    local source="$1"
    local target="$2"

    if [ "$DRY_RUN" = "true" ]; then
        log "DRY RUN: Would move '$source' to '$target'"
    else
        if mv "$source" "$target"; then
            log "Moved: $(basename "$source") -> $target"
        else
            log "ERROR: Failed to move $(basename "$source")"
        fi
```

```
    fi
}

# Organize files
organize_files() {
    local source_dir="$1"
    local target_dir="$2"
    local file_count=0
    local dir_count=0

    log "Starting file organization..."
    log "Source: $source_dir"
    log "Target: $target_dir"

    # Process all files in source directory
    find "$source_dir" -maxdepth 1 -type f | while read -r file; do
        # Skip hidden files unless specified
        if [[ "$(basename "$file")" == .* ]] && [ "$INCLUDE_HIDDEN" != "true" ]; then
            continue
        fi

        # Get file info
        filename=$(basename "$file")
        category=$(get_file_category "$filename")

        # Get file date
        if command -v stat >/dev/null 2>&1; then
            file_date=$(stat -c %Y "$file" 2>/dev/null || stat -f %m "$file" 2>/dev/null)
            year=$(date -d "@$file_date" +%Y 2>/dev/null || date -r "$file_date" +%Y 2>/dev/
            month=$(date -d "@$file_date" +%m 2>/dev/null || date -r "$file_date" +%m 2>/dev
        else
            year=$(date +%Y)
            month=$(date +%m)
        fi

        # Create target directory structure
        target_path="$target_dir/$category/$year/$month"
        create_directory "$target_path"

        # Move file
        move_file "$file" "$target_path/$filename"
        ((file_count++))
    done

    log "Organization complete. Processed $file_count files."
}
```

```
# Generate report
generate_report() {
    local target_dir="$1"
    local report_file="$target_dir/organization_report.txt"

    log "Generating organization report..."

    {
        echo "File Organization Report"
        echo "========================"
        echo "Generated on: $(date)"
        echo "Target directory: $target_dir"
        echo

        echo "Directory Structure:"
        echo "===================="
        if [ -d "$target_dir" ]; then
            find "$target_dir" -type d | sort
        fi
        echo

        echo "File Count by Category:"
        echo "======================="
        for category in "${file_types[@]}" "Other"; do
            if [ -d "$target_dir/$category" ]; then
                count=$(find "$target_dir/$category" -type f | wc -l)
                printf "%-15s: %d files\n" "$category" "$count"
            fi
        done

    } > "$report_file"

    log "Report saved: $report_file"
}

# Show usage
show_usage() {
    cat << EOF
Usage: $0 [source_dir] [target_dir]

Options:
  source_dir    Source directory to organize (default: current directory)
  target_dir    Target directory for organized files (default: ./organized)

Environment Variables:
  DRY_RUN=true         Show what would be done without actually moving files
  INCLUDE_HIDDEN=true  Include hidden files in organization
```

```
Examples:
  $0                          # Organize current directory
  $0 /home/user/Downloads     # Organize Downloads folder
  DRY_RUN=true $0             # Preview organization without moving files

File Categories:
EOF

    for types in "${!file_types[@]}"; do
        printf "  %-15s: %s\n" "${file_types[$types]}" "$types"
    done
}

# Main execution
main() {
    if [ "$1" = "-h" ] || [ "$1" = "--help" ]; then
        show_usage
        exit 0
    fi

    if [ ! -d "$SOURCE_DIR" ]; then
        log "ERROR: Source directory '$SOURCE_DIR' does not exist"
        exit 1
    fi

    if [ "$DRY_RUN" = "true" ]; then
        log "DRY RUN MODE - No files will be moved"
    fi

    organize_files "$SOURCE_DIR" "$TARGET_DIR"

    if [ "$DRY_RUN" != "true" ]; then
        generate_report "$TARGET_DIR"
    fi
}

main "$@"
```

CSV Processor

```
#!/bin/bash
# csv_processor.sh - Process and analyze CSV files

CSV_FILE="$1"
DELIMITER="${DELIMITER:-,}"
```

```
OUTPUT_FORMAT="${OUTPUT_FORMAT:-table}"

# Check if file exists
if [ ! -f "$CSV_FILE" ]; then
    echo "Usage: $0 <csv_file>"
    echo "Environment variables:"
    echo "  DELIMITER     - Field delimiter (default: ,)"
    echo "  OUTPUT_FORMAT - Output format: table, json, html (default: table)"
    exit 1
fi

# Function to get CSV headers
get_headers() {
    head -1 "$CSV_FILE" | tr "$DELIMITER" '\n' | nl -w2 -s': '
}

# Function to count rows
count_rows() {
    local total_rows=$(wc -l < "$CSV_FILE")
    local data_rows=$((total_rows - 1))
    echo "Total rows: $total_rows (including header)"
    echo "Data rows: $data_rows"
}

# Function to show column statistics
column_stats() {
    local column_num="$1"
    local column_name=$(head -1 "$CSV_FILE" | cut -d"$DELIMITER" -f"$column_num")

    echo "Statistics for column $column_num: $column_name"
    echo "=============================================="

    # Get column data (skip header)
    tail -n +2 "$CSV_FILE" | cut -d"$DELIMITER" -f"$column_num" > /tmp/column_data.tmp

    # Count non-empty values
    local non_empty=$(grep -v '^$' /tmp/column_data.tmp | wc -l)
    local empty=$(grep '^$' /tmp/column_data.tmp | wc -l)

    echo "Non-empty values: $non_empty"
    echo "Empty values: $empty"

    # Check if column contains numbers
    if grep -q '^[0-9]*\.?[0-9]*$' /tmp/column_data.tmp; then
        echo "Data type: Numeric"

        # Calculate numeric statistics
```

```
        local sum=$(awk '{sum += $1} END {print sum}' /tmp/column_data.tmp)
        local count=$(grep -v '^$' /tmp/column_data.tmp | wc -l)
        local avg=$(echo "scale=2; $sum / $count" | bc -l 2>/dev/null || echo "N/A")
        local min=$(sort -n /tmp/column_data.tmp | head -1)
        local max=$(sort -n /tmp/column_data.tmp | tail -1)

        echo "Sum: $sum"
        echo "Average: $avg"
        echo "Min: $min"
        echo "Max: $max"
    else
        echo "Data type: Text"

        # Show unique values count
        local unique_count=$(sort /tmp/column_data.tmp | uniq | wc -l)
        echo "Unique values: $unique_count"

        # Show most common values
        echo "Most common values:"
        sort /tmp/column_data.tmp | uniq -c | sort -nr | head -5
    fi

    rm -f /tmp/column_data.tmp
}

# Function to filter CSV data
filter_data() {
    local column="$1"
    local operator="$2"
    local value="$3"

    echo "Filtering: Column $column $operator '$value'"
    echo "=========================================="

    # Show header
    head -1 "$CSV_FILE"

    # Apply filter based on operator
    case "$operator" in
        "="|"eq")
            tail -n +2 "$CSV_FILE" | awk -F"$DELIMITER" -v col="$column" -v val="$value" '$c
            ;;
        "!="|"ne")
            tail -n +2 "$CSV_FILE" | awk -F"$DELIMITER" -v col="$column" -v val="$value" '$c
            ;;
        ">"|"gt")
            tail -n +2 "$CSV_FILE" | awk -F"$DELIMITER" -v col="$column" -v val="$value" '$c
```

```
            ;;
        "<"|"lt")
            tail -n +2 "$CSV_FILE" | awk -F"$DELIMITER" -v col="$column" -v val="$value" '$c
            ;;
        "contains")
            tail -n +2 "$CSV_FILE" | awk -F"$DELIMITER" -v col="$column" -v val="$value" 'in
            ;;
        *)
            echo "Supported operators: =, !=, >, <, contains"
            return 1
            ;;
    esac
}

# Function to convert to JSON
to_json() {
    local headers=($(head -1 "$CSV_FILE" | tr "$DELIMITER" ' '))

    echo "["
    local first_row=true

    tail -n +2 "$CSV_FILE" | while IFS="$DELIMITER" read -r -a fields; do
        if [ "$first_row" = true ]; then
            first_row=false
        else
            echo -n ","
        fi

        echo -n "  {"
        for i in "${!headers[@]}"; do
            if [ $i -gt 0 ]; then
                echo -n ", "
            fi
            echo -n "\"${headers[i]}\": \"${fields[i]}\""
        done
        echo -n "}"
    done

    echo
    echo "]"
}

# Function to convert to HTML table
to_html() {
    echo "<table border='1'>"
    echo "<thead><tr>"
```

```
    # Headers
    head -1 "$CSV_FILE" | tr "$DELIMITER" '\n' | while read -r header; do
        echo "<th>$header</th>"
    done

    echo "</tr></thead><tbody>"

    # Data rows
    tail -n +2 "$CSV_FILE" | while IFS="$DELIMITER" read -r -a fields; do
        echo "<tr>"
        for field in "${fields[@]}"; do
            echo "<td>$field</td>"
        done
        echo "</tr>"
    done

    echo "</tbody></table>"
}

# Function to show menu
show_menu() {
    echo
    echo "CSV Processor Menu"
    echo "=================="
    echo "1. Show headers"
    echo "2. Count rows"
    echo "3. Column statistics"
    echo "4. Filter data"
    echo "5. Convert to JSON"
    echo "6. Convert to HTML"
    echo "7. Show first 10 rows"
    echo "8. Show last 10 rows"
    echo "9. Exit"
    echo
}

# Interactive mode
interactive_mode() {
    while true; do
        show_menu
        read -p "Choose option: " choice

        case $choice in
            1)
                echo "CSV Headers:"
                get_headers
                ;;
```

```
            2)
                count_rows
                ;;
            3)
                echo "Available columns:"
                get_headers
                read -p "Enter column number: " col_num
                column_stats "$col_num"
                ;;
            4)
                echo "Available columns:"
                get_headers
                read -p "Enter column number: " col_num
                read -p "Enter operator (=, !=, >, <, contains): " operator
                read -p "Enter value: " value
                filter_data "$col_num" "$operator" "$value"
                ;;
            5)
                to_json
                ;;
            6)
                to_html
                ;;
            7)
                echo "First 10 rows:"
                head -11 "$CSV_FILE" | column -t -s"$DELIMITER"
                ;;
            8)
                echo "Last 10 rows:"
                tail -10 "$CSV_FILE" | column -t -s"$DELIMITER"
                ;;
            9)
                echo "Goodbye!"
                exit 0
                ;;
            *)
                echo "Invalid choice"
                ;;
        esac

        echo
        read -p "Press Enter to continue..."
    done
}

# Main execution
```

```
echo "Processing CSV file: $CSV_FILE"
echo "Delimiter: '$DELIMITER'"
echo

# If no additional arguments, start interactive mode
if [ $# -eq 1 ]; then
    interactive_mode
else
    # Command line mode
    case "$2" in
        headers) get_headers ;;
        count) count_rows ;;
        stats) column_stats "$3" ;;
        filter) filter_data "$3" "$4" "$5" ;;
        json) to_json ;;
        html) to_html ;;
        *)
            echo "Usage: $0 <csv_file> [command] [args...]"
            echo "Commands: headers, count, stats <col>, filter <col> <op> <val>, json, html
            ;;
    esac
fi
```

System Administration Examples

System Monitor

```
#!/bin/bash
# system_monitor.sh - Comprehensive system monitoring script

ALERT_CPU_THRESHOLD=${ALERT_CPU_THRESHOLD:-80}
ALERT_MEMORY_THRESHOLD=${ALERT_MEMORY_THRESHOLD:-85}
ALERT_DISK_THRESHOLD=${ALERT_DISK_THRESHOLD:-90}
LOG_FILE=${LOG_FILE:-"/var/log/system_monitor.log"}
EMAIL_ALERT=${EMAIL_ALERT:-""}
WEBHOOK_URL=${WEBHOOK_URL:-""}

# Colors for output
RED='\033[0;31m'
YELLOW='\033[1;33m'
GREEN='\033[0;32m'
NC='\033[0m' # No Color

# Logging function
```

```
log_message() {
    local level="$1"
    shift
    local message="$*"
    local timestamp=$(date '+%Y-%m-%d %H:%M:%S')

    echo "[$timestamp] [$level] $message" >> "$LOG_FILE"

    case "$level" in
        ERROR) echo -e "${RED}[$timestamp] ERROR: $message${NC}" ;;
        WARN)  echo -e "${YELLOW}[$timestamp] WARN:  $message${NC}" ;;
        INFO)  echo -e "${GREEN}[$timestamp] INFO:  $message${NC}" ;;
    esac
}

# Send alert notification
send_alert() {
    local subject="$1"
    local message="$2"

    # Email notification
    if [ -n "$EMAIL_ALERT" ] && command -v mail >/dev/null 2>&1; then
        echo "$message" | mail -s "$subject" "$EMAIL_ALERT"
        log_message INFO "Alert sent via email to $EMAIL_ALERT"
    fi

    # Webhook notification
    if [ -n "$WEBHOOK_URL" ] && command -v curl >/dev/null 2>&1; then
        curl -X POST -H "Content-Type: application/json" \
             -d "{\"text\":\"$subject: $message\"}" \
             "$WEBHOOK_URL" >/dev/null 2>&1
        log_message INFO "Alert sent via webhook"
    fi
}

# Check CPU usage
check_cpu() {
    local cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
    cpu_usage=${cpu_usage%.*}  # Remove decimal part

    echo "CPU Usage: ${cpu_usage}%"

    if [ "$cpu_usage" -gt "$ALERT_CPU_THRESHOLD" ]; then
        local message="High CPU usage detected: ${cpu_usage}% (threshold: ${ALERT_CPU_THRESH
        log_message WARN "$message"
        send_alert "CPU Alert" "$message"
```

```
        # Show top processes
        echo "Top CPU consuming processes:"
        ps aux --sort=-%cpu | head -6
    else
        log_message INFO "CPU usage normal: ${cpu_usage}%"
    fi
}

# Check memory usage
check_memory() {
    local memory_info=$(free | grep Mem)
    local total=$(echo $memory_info | awk '{print $2}')
    local used=$(echo $memory_info | awk '{print $3}')
    local memory_usage=$((used * 100 / total))

    echo "Memory Usage: ${memory_usage}% ($(numfmt --to=iec $((used * 1024))) / $(numfmt --t

    if [ "$memory_usage" -gt "$ALERT_MEMORY_THRESHOLD" ]; then
        local message="High memory usage detected: ${memory_usage}% (threshold: ${ALERT_MEMO
        log_message WARN "$message"
        send_alert "Memory Alert" "$message"

        # Show top memory consuming processes
        echo "Top memory consuming processes:"
        ps aux --sort=-%mem | head -6
    else
        log_message INFO "Memory usage normal: ${memory_usage}%"
    fi
}

# Check disk usage
check_disk() {
    echo "Disk Usage:"
    df -h | grep -vE '^Filesystem|tmpfs|cdrom' | while read output; do
        usage=$(echo $output | awk '{print $5}' | cut -d'%' -f1)
        partition=$(echo $output | awk '{print $1}')
        mount_point=$(echo $output | awk '{print $6}')

        echo "  $mount_point: ${usage}%"

        if [ "$usage" -gt "$ALERT_DISK_THRESHOLD" ]; then
            local message="High disk usage detected on $mount_point: ${usage}% (threshold: $
            log_message WARN "$message"
            send_alert "Disk Alert" "$message"
        else
            log_message INFO "Disk usage normal on $mount_point: ${usage}%"
        fi
```

```
    done
}

# Check system load
check_load() {
    local load_avg=$(uptime | awk -F'load average:' '{print $2}')
    local load_1min=$(echo $load_avg | awk '{print $1}' | tr -d ',')
    local cpu_cores=$(nproc)

    echo "Load Average: $load_avg"
    echo "CPU Cores: $cpu_cores"

    # Check if 1-minute load is higher than number of cores
    if (( $(echo "$load_1min > $cpu_cores" | bc -l) )); then
        local message="High system load detected: $load_1min (cores: $cpu_cores)"
        log_message WARN "$message"
        send_alert "Load Alert" "$message"
    else
        log_message INFO "System load normal: $load_1min"
    fi
}

# Check running services
check_services() {
    local services=("ssh" "nginx" "apache2" "mysql" "postgresql")

    echo "Service Status:"
    for service in "${services[@]}"; do
        if systemctl is-active --quiet "$service" 2>/dev/null; then
            echo "  $service: Running"
            log_message INFO "Service $service is running"
        elif systemctl list-unit-files | grep -q "^$service.service"; then
            echo "  $service: Stopped"
            log_message WARN "Service $service is stopped"
            send_alert "Service Alert" "Service $service is not running"
        fi
    done
}

# Check network connectivity
check_network() {
    local test_hosts=("8.8.8.8" "1.1.1.1" "google.com")

    echo "Network Connectivity:"
    for host in "${test_hosts[@]}"; do
        if ping -c 1 -W 5 "$host" >/dev/null 2>&1; then
            echo "  $host: OK"
```

```
            log_message INFO "Network connectivity to $host OK"
        else
            echo "  $host: FAILED"
            log_message ERROR "Network connectivity to $host failed"
            send_alert "Network Alert" "Cannot reach $host"
        fi
    done
}

# Check log files for errors
check_logs() {
    local log_files=("/var/log/syslog" "/var/log/auth.log" "/var/log/kern.log")
    local time_threshold="1 hour ago"

    echo "Recent Log Errors:"
    for log_file in "${log_files[@]}"; do
        if [ -f "$log_file" ]; then
            local error_count=$(find "$log_file" -newermt "$time_threshold" -exec grep -i "e
            echo "  $(basename "$log_file"): $error_count errors in last hour"

            if [ "$error_count" -gt 10 ]; then
                log_message WARN "High error count in $log_file: $error_count"
                send_alert "Log Alert" "High error count in $log_file: $error_count errors i
            fi
        fi
    done
}

# Generate system report
generate_report() {
    local report_file="/tmp/system_report_$(date +%Y%m%d_%H%M%S).txt"

    {
        echo "System Monitoring Report"
        echo "========================"
        echo "Generated: $(date)"
        echo "Hostname: $(hostname)"
        echo

        echo "System Information:"
        echo "==================="
        uname -a
        echo

        echo "Uptime:"
        echo "======="
        uptime
```

```
        echo

        echo "CPU Information:"
        echo "================"
        lscpu | grep -E "Model name|CPU\(s\)|Thread|Core"
        echo

        echo "Memory Information:"
        echo "==================="
        free -h
        echo

        echo "Disk Usage:"
        echo "==========="
        df -h
        echo

        echo "Network Interfaces:"
        echo "==================="
        ip addr show
        echo

        echo "Running Processes (Top 10 by CPU):"
        echo "==================================="
        ps aux --sort=-%cpu | head -11
        echo

        echo "Running Processes (Top 10 by Memory):"
        echo "======================================"
        ps aux --sort=-%mem | head -11

    } > "$report_file"

    echo "System report generated: $report_file"
    log_message INFO "System report generated: $report_file"
}

# Main monitoring function
run_monitoring() {
    log_message INFO "Starting system monitoring"

    echo "System Monitoring Report - $(date)"
    echo "=================================="
    echo

    check_cpu
    echo
```

```
    check_memory
    echo

    check_disk
    echo

    check_load
    echo

    check_services
    echo

    check_network
    echo

    check_logs
    echo

    log_message INFO "System monitoring completed"
}

# Show usage
show_usage() {
    cat << EOF
Usage: $0 [options]

Options:
  -c, --continuous    Run monitoring continuously
  -i, --interval N    Set monitoring interval in seconds (default: 300)
  -r, --report        Generate detailed system report
  -h, --help          Show this help message

Environment Variables:
  ALERT_CPU_THRESHOLD     CPU usage alert threshold (default: 80)
  ALERT_MEMORY_THRESHOLD  Memory usage alert threshold (default: 85)
  ALERT_DISK_THRESHOLD    Disk usage alert threshold (default: 90)
  LOG_FILE               Log file path (default: /var/log/system_monitor.log)
  EMAIL_ALERT            Email address for alerts
  WEBHOOK_URL            Webhook URL for alerts

Examples:
  $0                      # Run monitoring once
  $0 -c                   # Run continuous monitoring
  $0 -c -i 60            # Run continuous monitoring every 60 seconds
  $0 -r                   # Generate detailed report
EOF
```

```
}

# Parse command line arguments
CONTINUOUS=false
INTERVAL=300

while [[ $# -gt 0 ]]; do
    case $1 in
        -c|--continuous)
            CONTINUOUS=true
            shift
            ;;
        -i|--interval)
            INTERVAL="$2"
            shift 2
            ;;
        -r|--report)
            generate_report
            exit 0
            ;;
        -h|--help)
            show_usage
            exit 0
            ;;
        *)
            echo "Unknown option: $1"
            show_usage
            exit 1
            ;;
    esac
done

# Main execution
if [ "$CONTINUOUS" = true ]; then
    log_message INFO "Starting continuous monitoring (interval: ${INTERVAL}s)"
    while true; do
        run_monitoring
        echo "Sleeping for $INTERVAL seconds..."
        sleep "$INTERVAL"
    done
else
    run_monitoring
fi
```

This workbook provides a comprehensive collection of Bash scripting examples ranging from basic to intermediate level. Each example includes practical, real-world scenarios that demonstrate different aspects of Bash programming. The examples cover:

1. **Basic Examples**: Variables, user input, calculations, file operations
2. **Intermediate Examples**: Functions, configuration management, database operations
3. **File Processing**: Log analysis, file organization, CSV processing
4. **System Administration**: Comprehensive system monitoring

These examples can be used as learning materials, reference implementations, or starting points for your own scripts.

Advanced Bash Examples

Advanced Bash Examples

Advanced Bash Examples

This section contains advanced Bash scripting examples that demonstrate complex concepts, patterns, and real-world applications.

Process Management and IPC

Process Pool Manager

```
#!/bin/bash
# process_pool.sh - Manage a pool of worker processes

MAX_WORKERS=${MAX_WORKERS:-4}
WORK_QUEUE="/tmp/work_queue.$$"
WORKER_PIDS=()
RUNNING=true

# Cleanup function
cleanup() {
    RUNNING=false
    echo "Shutting down workers..."

    for pid in "${WORKER_PIDS[@]}"; do
        if kill -0 "$pid" 2>/dev/null; then
            kill "$pid"
        fi
    done

    rm -f "$WORK_QUEUE"
    exit 0
}

# Set up signal handlers
trap cleanup SIGINT SIGTERM

# Worker function
worker() {
    local worker_id="$1"
    echo "Worker $worker_id started (PID: $$)"
```

```
    while $RUNNING; do
        if [ -p "$WORK_QUEUE" ]; then
            if read -t 1 task < "$WORK_QUEUE"; then
                echo "Worker $worker_id processing: $task"
                # Simulate work
                sleep $((RANDOM % 5 + 1))
                echo "Worker $worker_id completed: $task"
            fi
        fi
    done

    echo "Worker $worker_id shutting down"
}

# Create work queue (named pipe)
mkfifo "$WORK_QUEUE"

# Start worker processes
for ((i=1; i<=MAX_WORKERS; i++)); do
    worker "$i" &
    WORKER_PIDS+=($!)
done

echo "Started $MAX_WORKERS workers"
echo "Send tasks by writing to: $WORK_QUEUE"
echo "Press Ctrl+C to shutdown"

# Add some sample tasks
for task in "task1" "task2" "task3" "task4" "task5"; do
    echo "$task" > "$WORK_QUEUE" &
done

# Wait for workers
wait
```#
Distributed Task Runner
```bash
#!/bin/bash
# distributed_runner.sh - Run tasks across multiple servers

SERVERS_FILE="${SERVERS_FILE:-servers.txt}"
SSH_KEY="${SSH_KEY:-~/.ssh/id_rsa}"
PARALLEL_JOBS="${PARALLEL_JOBS:-3}"
TIMEOUT="${TIMEOUT:-300}"

# Server list format: user@hostname:port
declare -a SERVERS
```

```
# Load servers from file
load_servers() {
    if [ ! -f "$SERVERS_FILE" ]; then
        echo "Error: Servers file '$SERVERS_FILE' not found"
        exit 1
    fi

    while IFS= read -r server; do
        [[ $server =~ ^[[:space:]]*# ]] && continue
        [[ -z $server ]] && continue
        SERVERS+=("$server")
    done < "$SERVERS_FILE"

    echo "Loaded ${#SERVERS[@]} servers"
}

# Execute command on remote server
execute_remote() {
    local server="$1"
    local command="$2"
    local output_file="$3"

    local user_host="${server%:*}"
    local port="${server##*:}"
    [ "$port" = "$server" ] && port="22"

    echo "Executing on $user_host: $command" >> "$output_file"

    timeout "$TIMEOUT" ssh -i "$SSH_KEY" -p "$port" -o ConnectTimeout=10 \
        -o StrictHostKeyChecking=no "$user_host" "$command" >> "$output_file" 2>&1

    local exit_code=$?
    echo "Exit code: $exit_code" >> "$output_file"
    return $exit_code
}

# Run command on all servers in parallel
run_distributed() {
    local command="$1"
    local output_dir="results_$(date +%Y%m%d_%H%M%S)"

    mkdir -p "$output_dir"
    echo "Running command on ${#SERVERS[@]} servers: $command"
    echo "Output directory: $output_dir"

    local pids=()
```

```
    local server_index=0

    for server in "${SERVERS[@]}"; do
        local output_file="$output_dir/server_${server_index}.log"

        execute_remote "$server" "$command" "$output_file" &
        pids+=($!)

        ((server_index++))

        # Limit parallel jobs
        if [ ${#pids[@]} -ge $PARALLEL_JOBS ]; then
            wait "${pids[0]}"
            pids=("${pids[@]:1}")
        fi
    done

    # Wait for remaining jobs
    for pid in "${pids[@]}"; do
        wait "$pid"
    done

    echo "All tasks completed. Results in: $output_dir"

    # Generate summary
    generate_summary "$output_dir"
}

# Generate execution summary
generate_summary() {
    local output_dir="$1"
    local summary_file="$output_dir/summary.txt"

    {
        echo "Distributed Execution Summary"
        echo "============================="
        echo "Executed at: $(date)"
        echo "Total servers: ${#SERVERS[@]}"
        echo

        local success_count=0
        local failure_count=0

        for log_file in "$output_dir"/server_*.log; do
            local server_num=$(basename "$log_file" .log | cut -d_ -f2)
            local server="${SERVERS[$server_num]}"
```

```
            if grep -q "Exit code: 0" "$log_file"; then
                echo "  $server - SUCCESS"
                ((success_count++))
            else
                echo "  $server - FAILED"
                ((failure_count++))
            fi
        done

        echo
        echo "Results: $success_count successful, $failure_count failed"

    } > "$summary_file"

    cat "$summary_file"
}

# Main execution
if [ $# -eq 0 ]; then
    echo "Usage: $0 <command>"
    echo "Environment variables:"
    echo "  SERVERS_FILE    - File containing server list (default: servers.txt)"
    echo "  SSH_KEY         - SSH private key (default: ~/.ssh/id_rsa)"
    echo "  PARALLEL_JOBS   - Number of parallel jobs (default: 3)"
    echo "  TIMEOUT         - Command timeout in seconds (default: 300)"
    exit 1
fi

load_servers
run_distributed "$*"
```

Network Programming

HTTP Server in Bash

```
#!/bin/bash
# http_server.sh - Simple HTTP server in pure Bash

PORT="${PORT:-8080}"
DOCUMENT_ROOT="${DOCUMENT_ROOT:-./www}"
LOG_FILE="${LOG_FILE:-access.log}"

# MIME types
declare -A MIME_TYPES=(
    ["html"]="text/html"
```

```
    ["css"]="text/css"
    ["js"]="application/javascript"
    ["json"]="application/json"
    ["txt"]="text/plain"
    ["png"]="image/png"
    ["jpg"]="image/jpeg"
    ["gif"]="image/gif"
)

# Log HTTP request
log_request() {
    local client_ip="$1"
    local method="$2"
    local path="$3"
    local status="$4"
    local size="$5"

    echo "$(date '+%d/%b/%Y:%H:%M:%S %z') $client_ip \"$method $path HTTP/1.1\" $status $siz
}

# Get MIME type from file extension
get_mime_type() {
    local file="$1"
    local extension="${file##*.}"
    echo "${MIME_TYPES[$extension]:-application/octet-stream}"
}

# Send HTTP response
send_response() {
    local status="$1"
    local content_type="$2"
    local content="$3"
    local content_length="${#content}"

    echo "HTTP/1.1 $status"
    echo "Content-Type: $content_type"
    echo "Content-Length: $content_length"
    echo "Connection: close"
    echo
    echo -n "$content"
}

# Handle HTTP request
handle_request() {
    local client_ip="$1"

    # Read request line
```

```
    read -r method path protocol

    # Read headers (until empty line)
    while read -r header && [ -n "$header" ]; do
        header=$(echo "$header" | tr -d '\r')
        # Process headers if needed
    done

    # Remove query string from path
    path="${path%%\?*}"

    # Security: prevent directory traversal
    path="${path//..\/}"

    # Default to index.html for directory requests
    if [[ "$path" == */ ]]; then
        path="${path}index.html"
    fi

    local file_path="$DOCUMENT_ROOT$path"

    if [ -f "$file_path" ]; then
        local content_type=$(get_mime_type "$file_path")
        local content=$(cat "$file_path")
        send_response "200 OK" "$content_type" "$content"
        log_request "$client_ip" "$method" "$path" "200" "${#content}"
    else
        local error_content="<html><body><h1>404 Not Found</h1><p>The requested file was not
        send_response "404 Not Found" "text/html" "$error_content"
        log_request "$client_ip" "$method" "$path" "404" "${#error_content}"
    fi
}

# Start server
start_server() {
    echo "Starting HTTP server on port $PORT"
    echo "Document root: $DOCUMENT_ROOT"
    echo "Log file: $LOG_FILE"

    # Create document root if it doesn't exist
    mkdir -p "$DOCUMENT_ROOT"

    # Create a simple index.html if it doesn't exist
    if [ ! -f "$DOCUMENT_ROOT/index.html" ]; then
        cat > "$DOCUMENT_ROOT/index.html" << EOF
<!DOCTYPE html>
<html>
```

```
<head>
    <title>Bash HTTP Server</title>
</head>
<body>
    <h1>Welcome to Bash HTTP Server</h1>
    <p>Server started at: $(date)</p>
    <p>This is a simple HTTP server written in Bash.</p>
</body>
</html>
EOF
    fi

    # Start listening
    while true; do
        # Use netcat to listen for connections
        if command -v nc >/dev/null 2>&1; then
            nc -l -p "$PORT" -e "$0" handle_connection
        elif command -v socat >/dev/null 2>&1; then
            socat TCP-LISTEN:$PORT,reuseaddr,fork EXEC:"$0 handle_connection"
        else
            echo "Error: netcat or socat required"
            exit 1
        fi
    done
}

# Handle individual connection
handle_connection() {
    local client_ip="${SOCAT_PEERADDR:-unknown}"
    handle_request "$client_ip"
}

# Main execution
case "${1:-start}" in
    start)
        start_server
        ;;
    handle_connection)
        handle_connection
        ;;
    *)
        echo "Usage: $0 [start]"
        exit 1
        ;;
esac
```

Network Scanner

```
#!/bin/bash
# network_scanner.sh - Network discovery and port scanning

NETWORK="${1:-192.168.1.0/24}"
TIMEOUT="${TIMEOUT:-1}"
THREADS="${THREADS:-50}"

# Common ports to scan
COMMON_PORTS=(22 23 25 53 80 110 143 443 993 995 3389 5432 3306)

# Parse CIDR notation
parse_network() {
    local network="$1"
    local ip="${network%/*}"
    local prefix="${network#*/}"

    if [ "$prefix" = "$network" ]; then
        # No CIDR notation, assume single host
        echo "$ip"
        return
    fi

    # Convert CIDR to IP range
    local IFS='.'
    local ip_parts=($ip)
    local ip_int=$((${ip_parts[0]} << 24 | ${ip_parts[1]} << 16 | ${ip_parts[2]} << 8 | ${ip

    local mask=$((0xFFFFFFFF << (32 - prefix)))
    local network_int=$((ip_int & mask))
    local broadcast_int=$((network_int | (0xFFFFFFFF >> prefix)))

    for ((i=network_int+1; i<broadcast_int; i++)); do
        printf "%d.%d.%d.%d\n" $((i >> 24 & 0xFF)) $((i >> 16 & 0xFF)) $((i >> 8 & 0xFF)) $(
    done
}

# Ping sweep to discover live hosts
ping_sweep() {
    local network="$1"
    local output_file="/tmp/live_hosts.$$"

    echo "Performing ping sweep on $network..."

    parse_network "$network" | while read -r ip; do
        {
```

```
            if ping -c 1 -W "$TIMEOUT" "$ip" >/dev/null 2>&1; then
                echo "$ip" >> "$output_file"
                echo "Host alive: $ip"
            fi
        } &

        # Limit concurrent processes
        (($(jobs -r | wc -l) >= THREADS)) && wait
    done

    wait

    if [ -f "$output_file" ]; then
        sort -V "$output_file"
        rm "$output_file"
    fi
}

# Port scan a single host
port_scan() {
    local host="$1"
    local ports=("${@:2}")

    echo "Scanning ports on $host..."

    for port in "${ports[@]}"; do
        {
            if timeout "$TIMEOUT" bash -c "echo >/dev/tcp/$host/$port" 2>/dev/null; then
                echo "$host:$port - OPEN"

                # Try to identify service
                local service=$(getent services "$port/tcp" 2>/dev/null | awk '{print $1}')
                [ -n "$service" ] && echo "  Service: $service"
            fi
        } &

        # Limit concurrent processes
        (($(jobs -r | wc -l) >= THREADS)) && wait
    done

    wait
}

# Comprehensive network scan
full_scan() {
    local network="$1"
    local output_dir="scan_results_$(date +%Y%m%d_%H%M%S)"
```

```
    mkdir -p "$output_dir"

    echo "Starting comprehensive network scan..."
    echo "Network: $network"
    echo "Output directory: $output_dir"

    # Discover live hosts
    echo "Phase 1: Host discovery"
    local live_hosts=()
    while IFS= read -r host; do
        live_hosts+=("$host")
    done < <(ping_sweep "$network")

    echo "Found ${#live_hosts[@]} live hosts"
    echo "${live_hosts[@]}" > "$output_dir/live_hosts.txt"

    # Port scan each live host
    echo "Phase 2: Port scanning"
    for host in "${live_hosts[@]}"; do
        echo "Scanning $host..."
        port_scan "$host" "${COMMON_PORTS[@]}" > "$output_dir/${host}_ports.txt"
    done

    # Generate summary report
    {
        echo "Network Scan Report"
        echo "==================="
        echo "Scan date: $(date)"
        echo "Network: $network"
        echo "Live hosts: ${#live_hosts[@]}"
        echo

        echo "Host Summary:"
        echo "============"
        for host in "${live_hosts[@]}"; do
            local open_ports=$(grep -c "OPEN" "$output_dir/${host}_ports.txt" 2>/dev/null ||
            echo "$host - $open_ports open ports"
        done

        echo
        echo "Open Ports by Service:"
        echo "====================="
        for port in "${COMMON_PORTS[@]}"; do
            local hosts_with_port=()
            for host in "${live_hosts[@]}"; do
                if grep -q "$host:$port - OPEN" "$output_dir/${host}_ports.txt" 2>/dev/null;
```

```
                    hosts_with_port+=("$host")
                fi
            done

            if [ ${#hosts_with_port[@]} -gt 0 ]; then
                local service=$(getent services "$port/tcp" 2>/dev/null | awk '{print $1}' |
                echo "Port $port ($service): ${hosts_with_port[*]}"
            fi
        done

    } > "$output_dir/summary.txt"

    echo "Scan completed. Results in: $output_dir"
    cat "$output_dir/summary.txt"
}

# Main execution
case "${2:-full}" in
    ping)
        ping_sweep "$NETWORK"
        ;;
    port)
        if [ $# -lt 2 ]; then
            echo "Usage: $0 <host> port [port...]"
            exit 1
        fi
        port_scan "$1" "${COMMON_PORTS[@]}"
        ;;
    full)
        full_scan "$NETWORK"
        ;;
    *)
        echo "Usage: $0 <network> [ping|port|full]"
        echo "Examples:"
        echo "  $0 192.168.1.0/24 ping    # Ping sweep only"
        echo "  $0 192.168.1.1 port       # Port scan single host"
        echo "  $0 192.168.1.0/24 full    # Full network scan"
        exit 1
        ;;
esac
```

Data Processing and Analysis

JSON Processor

```
#!/bin/bash
# json_processor.sh - JSON processing without external dependencies

# Simple JSON parser (basic implementation)
parse_json() {
    local json="$1"
    local key="$2"

    # Remove whitespace
    json=$(echo "$json" | tr -d ' \t\n\r')

    # Extract value for key
    local pattern="\"$key\":\"([^\"]*)\""
    if [[ $json =~ $pattern ]]; then
        echo "${BASH_REMATCH[1]}"
    else
        # Try numeric value
        pattern="\"$key\":([0-9]+)"
        if [[ $json =~ $pattern ]]; then
            echo "${BASH_REMATCH[1]}"
        fi
    fi
}

# Extract array elements
parse_json_array() {
    local json="$1"
    local array_key="$2"

    # Extract array content
    local pattern="\"$array_key\":\[([^\]]*)\]"
    if [[ $json =~ $pattern ]]; then
        local array_content="${BASH_REMATCH[1]}"

        # Split by comma and clean up
        IFS=',' read -ra elements <<< "$array_content"
        for element in "${elements[@]}"; do
            element=$(echo "$element" | sed 's/^[[:space:]]*"//;s/"[[:space:]]*$//')
            echo "$element"
        done
    fi
}
```

```
# Create JSON object
create_json() {
    local -n data_ref=$1
    local json="{"
    local first=true

    for key in "${!data_ref[@]}"; do
        if [ "$first" = true ]; then
            first=false
        else
            json+=","
        fi

        json+="\"$key\":\"${data_ref[$key]}\""
    done

    json+="}"
    echo "$json"
}

# Process JSON file
process_json_file() {
    local file="$1"
    local operation="$2"
    local key="$3"

    if [ ! -f "$file" ]; then
        echo "Error: File '$file' not found"
        return 1
    fi

    local json_content=$(cat "$file")

    case "$operation" in
        get)
            parse_json "$json_content" "$key"
            ;;
        array)
            parse_json_array "$json_content" "$key"
            ;;
        keys)
            # Extract all keys
            echo "$json_content" | grep -o '"[^"]*"[[:space:]]*:' | sed 's/"//g;s/[[:space:]
            ;;
        validate)
            # Basic JSON validation
```

```
            if echo "$json_content" | python3 -m json.tool >/dev/null 2>&1; then
                echo "Valid JSON"
            else
                echo "Invalid JSON"
            fi
            ;;
        pretty)
            # Pretty print JSON
            if command -v python3 >/dev/null 2>&1; then
                echo "$json_content" | python3 -m json.tool
            else
                echo "$json_content"
            fi
            ;;
        *)
            echo "Unknown operation: $operation"
            echo "Available operations: get, array, keys, validate, pretty"
            return 1
            ;;
    esac
}

# Example usage and testing
if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then
    # Create sample JSON file for testing
    cat > sample.json << 'EOF'
{
    "name": "John Doe",
    "age": 30,
    "city": "New York",
    "hobbies": ["reading", "swimming", "coding"],
    "active": true,
    "address": {
        "street": "123 Main St",
        "zip": "10001"
    }
}
EOF

    if [ $# -eq 0 ]; then
        echo "JSON Processor Demo"
        echo "==================="
        echo

        echo "Sample JSON content:"
        cat sample.json
        echo
```

```
        echo "Extracting 'name':"
        process_json_file sample.json get name
        echo

        echo "Extracting 'age':"
        process_json_file sample.json get age
        echo

        echo "Extracting 'hobbies' array:"
        process_json_file sample.json array hobbies
        echo

        echo "All keys:"
        process_json_file sample.json keys
        echo

        echo "Validating JSON:"
        process_json_file sample.json validate

    else
        process_json_file "$@"
    fi
fi
```

Data Aggregator

```
#!/bin/bash
# data_aggregator.sh - Aggregate and analyze data from multiple sources

declare -A metrics
declare -A counters
OUTPUT_FORMAT="${OUTPUT_FORMAT:-table}"

# Add metric
add_metric() {
    local key="$1"
    local value="$2"

    if [[ $value =~ ^[0-9]+(\.[0-9]+)?$ ]]; then
        metrics["$key"]="${metrics["$key"]:-0}"
        metrics["$key"]=$(echo "${metrics["$key"]} + $value" | bc -l)
        counters["$key"]=$((${counters["$key"]:-0} + 1))
    fi
}
```

```
# Process log file
process_log_file() {
    local file="$1"
    local format="$2"

    echo "Processing log file: $file (format: $format)"

    case "$format" in
        apache)
            # Apache access log format
            while IFS= read -r line; do
                # Extract response time if available
                if [[ $line =~ ([0-9]+)$ ]]; then
                    add_metric "response_time" "${BASH_REMATCH[1]}"
                fi

                # Extract status code
                if [[ $line =~ \"[^\"]*\"[[:space:]]+([0-9]{3}) ]]; then
                    add_metric "status_${BASH_REMATCH[1]}" 1
                fi

                # Extract bytes sent
                if [[ $line =~ [[:space:]]([0-9]+)[[:space:]]+\"[^\"]*\"[[:space:]]*$ ]]; th
                    add_metric "bytes_sent" "${BASH_REMATCH[1]}"
                fi
            done < "$file"
            ;;
        nginx)
            # Nginx access log format
            while IFS= read -r line; do
                # Similar processing for nginx logs
                if [[ $line =~ [[:space:]]([0-9]{3})[[:space:]] ]]; then
                    add_metric "status_${BASH_REMATCH[1]}" 1
                fi
            done < "$file"
            ;;
        csv)
            # CSV format - assume first line is header
            local headers=()
            local line_num=0

            while IFS=',' read -ra fields; do
                ((line_num++))

                if [ $line_num -eq 1 ]; then
                    headers=("${fields[@]}")
                else
```

```
                    for i in "${!fields[@]}"; do
                        local header="${headers[i]}"
                        local value="${fields[i]}"

                        if [[ $value =~ ^[0-9]+(\.[0-9]+)?$ ]]; then
                            add_metric "$header" "$value"
                        fi
                    done
                fi
            done < "$file"
            ;;
        *)
            echo "Unknown format: $format"
            return 1
            ;;
    esac
}

# Calculate statistics
calculate_stats() {
    declare -A averages
    declare -A totals

    for key in "${!metrics[@]}"; do
        totals["$key"]="${metrics["$key"]}"
        if [ "${counters["$key"]}" -gt 0 ]; then
            averages["$key"]=$(echo "scale=2; ${metrics["$key"]} / ${counters["$key"]}" | bc
        fi
    done

    case "$OUTPUT_FORMAT" in
        table)
            printf "%-20s %15s %15s %10s\n" "Metric" "Total" "Average" "Count"
            printf "%-20s %15s %15s %10s\n" "------" "-----" "-------" "-----"

            for key in $(printf '%s\n' "${!metrics[@]}" | sort); do
                printf "%-20s %15.2f %15.2f %10d\n" \
                    "$key" "${totals["$key"]}" "${averages["$key"]}" "${counters["$key"]}"
            done
            ;;
        json)
            echo "{"
            local first=true
            for key in "${!metrics[@]}"; do
                if [ "$first" = true ]; then
                    first=false
                else
```

```
                    echo -n ","
                fi
                printf '  "%s": {"total": %.2f, "average": %.2f, "count": %d}' \
                    "$key" "${totals["$key"]}" "${averages["$key"]}" "${counters["$key"]}"
            done
            echo
            echo "}"
            ;;
        csv)
            echo "metric,total,average,count"
            for key in "${!metrics[@]}"; do
                printf "%s,%.2f,%.2f,%d\n" \
                    "$key" "${totals["$key"]}" "${averages["$key"]}" "${counters["$key"]}"
            done
            ;;
    esac
}

# Generate report
generate_report() {
    local output_file="$1"

    {
        echo "Data Aggregation Report"
        echo "======================="
        echo "Generated: $(date)"
        echo "Files processed: ${#processed_files[@]}"
        echo

        calculate_stats

    } > "$output_file"

    echo "Report saved to: $output_file"
}

# Main processing function
main() {
    local files=("$@")
    local processed_files=()

    if [ ${#files[@]} -eq 0 ]; then
        echo "Usage: $0 <file1> [file2] ..."
        echo "Environment variables:"
        echo "  OUTPUT_FORMAT - Output format: table, json, csv (default: table)"
        exit 1
    fi
```

```
    for file in "${files[@]}"; do
        if [ ! -f "$file" ]; then
            echo "Warning: File '$file' not found, skipping"
            continue
        fi

        # Detect file format based on extension or content
        local format="apache"  # default
        case "$file" in
            *.csv) format="csv" ;;
            *nginx*) format="nginx" ;;
            *apache*) format="apache" ;;
        esac

        process_log_file "$file" "$format"
        processed_files+=("$file")
    done

    if [ ${#processed_files[@]} -eq 0 ]; then
        echo "No files were processed"
        exit 1
    fi

    echo
    echo "Aggregation Results:"
    echo "===================="
    calculate_stats

    # Generate detailed report
    local report_file="aggregation_report_$(date +%Y%m%d_%H%M%S).txt"
    generate_report "$report_file"
}

main "$@"
```

Security and Encryption

Password Manager

```
#!/bin/bash
# password_manager.sh - Simple encrypted password manager

VAULT_FILE="${VAULT_FILE:-$HOME/.password_vault.gpg}"
TEMP_FILE="/tmp/vault_temp.$$"
```

```
# Cleanup function
cleanup() {
    rm -f "$TEMP_FILE"
}
trap cleanup EXIT

# Check dependencies
check_dependencies() {
    if ! command -v gpg >/dev/null 2>&1; then
        echo "Error: GPG is required but not installed"
        exit 1
    fi
}

# Initialize vault
init_vault() {
    if [ -f "$VAULT_FILE" ]; then
        echo "Vault already exists at: $VAULT_FILE"
        return 1
    fi

    echo "Initializing new password vault..."
    echo "# Password Vault - Created $(date)" > "$TEMP_FILE"
    echo "# Format: service:username:password:notes" >> "$TEMP_FILE"

    encrypt_vault
    echo "Vault initialized at: $VAULT_FILE"
}

# Encrypt vault
encrypt_vault() {
    if ! gpg --symmetric --cipher-algo AES256 --output "$VAULT_FILE" "$TEMP_FILE"; then
        echo "Error: Failed to encrypt vault"
        exit 1
    fi
}

# Decrypt vault
decrypt_vault() {
    if [ ! -f "$VAULT_FILE" ]; then
        echo "Error: Vault file not found. Run 'init' first."
        exit 1
    fi

    if ! gpg --decrypt --output "$TEMP_FILE" "$VAULT_FILE" 2>/dev/null; then
        echo "Error: Failed to decrypt vault (wrong password?)"
```

```
        exit 1
    fi
}

# Add password entry
add_entry() {
    local service="$1"
    local username="$2"
    local password="$3"
    local notes="$4"

    if [ -z "$service" ] || [ -z "$username" ]; then
        echo "Error: Service and username are required"
        return 1
    fi

    decrypt_vault

    # Check if entry already exists
    if grep -q "^$service:$username:" "$TEMP_FILE"; then
        echo "Entry already exists. Use 'update' to modify."
        return 1
    fi

    # Generate password if not provided
    if [ -z "$password" ]; then
        password=$(generate_password)
        echo "Generated password: $password"
    fi

    echo "$service:$username:$password:$notes" >> "$TEMP_FILE"
    encrypt_vault

    echo "Entry added for $service ($username)"
}

# Generate secure password
generate_password() {
    local length="${1:-16}"
    tr -dc 'A-Za-z0-9!@#$%^&*' < /dev/urandom | head -c "$length"
}

# Search entries
search_entries() {
    local query="$1"

    decrypt_vault
```

```
    echo "Search results for: $query"
    echo "=========================="

    grep -i "$query" "$TEMP_FILE" | grep -v '^#' | while IFS=':' read -r service username pa
        echo "Service: $service"
        echo "Username: $username"
        echo "Password: [hidden - use 'show' to reveal]"
        echo "Notes: $notes"
        echo "---"
    done
}

# Show specific entry
show_entry() {
    local service="$1"
    local username="$2"

    decrypt_vault

    local entry=$(grep "^$service:$username:" "$TEMP_FILE")
    if [ -n "$entry" ]; then
        IFS=':' read -r service username password notes <<< "$entry"
        echo "Service: $service"
        echo "Username: $username"
        echo "Password: $password"
        echo "Notes: $notes"
    else
        echo "Entry not found: $service ($username)"
    fi
}

# List all entries
list_entries() {
    decrypt_vault

    echo "Password Vault Entries"
    echo "======================"

    grep -v '^#' "$TEMP_FILE" | while IFS=':' read -r service username password notes; do
        echo "$service ($username)"
    done
}

# Update entry
update_entry() {
    local service="$1"
```

```
    local username="$2"
    local new_password="$3"
    local new_notes="$4"

    decrypt_vault

    if ! grep -q "^$service:$username:" "$TEMP_FILE"; then
        echo "Entry not found: $service ($username)"
        return 1
    fi

    # Create updated file
    local updated_file="/tmp/vault_updated.$$"

    while IFS=':' read -r s u p n; do
        if [ "$s" = "$service" ] && [ "$u" = "$username" ]; then
            echo "$s:$u:${new_password:-$p}:${new_notes:-$n}"
        else
            echo "$s:$u:$p:$n"
        fi
    done < "$TEMP_FILE" > "$updated_file"

    mv "$updated_file" "$TEMP_FILE"
    encrypt_vault

    echo "Entry updated: $service ($username)"
}

# Delete entry
delete_entry() {
    local service="$1"
    local username="$2"

    decrypt_vault

    if ! grep -q "^$service:$username:" "$TEMP_FILE"; then
        echo "Entry not found: $service ($username)"
        return 1
    fi

    # Create file without the entry
    grep -v "^$service:$username:" "$TEMP_FILE" > "/tmp/vault_filtered.$$"
    mv "/tmp/vault_filtered.$$" "$TEMP_FILE"
    encrypt_vault

    echo "Entry deleted: $service ($username)"
}
```

```
# Show usage
show_usage() {
    cat << EOF
Password Manager - Encrypted password storage

Usage: $0 <command> [arguments]

Commands:
  init                            Initialize new vault
  add <service> <username> [password] [notes]  Add new entry
  show <service> <username>       Show specific entry
  search <query>                  Search entries
  list                            List all entries
  update <service> <username> [password] [notes]  Update entry
  delete <service> <username>     Delete entry
  generate [length]               Generate secure password

Environment Variables:
  VAULT_FILE                      Path to vault file (default: ~/.password_vault.gpg)

Examples:
  $0 init
  $0 add gmail john.doe@gmail.com
  $0 show gmail john.doe@gmail.com
  $0 search gmail
  $0 generate 20
EOF
}

# Main execution
check_dependencies

case "${1:-help}" in
    init)
        init_vault
        ;;
    add)
        add_entry "$2" "$3" "$4" "$5"
        ;;
    show)
        show_entry "$2" "$3"
        ;;
    search)
        search_entries "$2"
        ;;
    list)
```

```
        list_entries
        ;;
    update)
        update_entry "$2" "$3" "$4" "$5"
        ;;
    delete)
        delete_entry "$2" "$3"
        ;;
    generate)
        generate_password "$2"
        ;;
    help|--help|-h)
        show_usage
        ;;
    *)
        echo "Unknown command: $1"
        show_usage
        exit 1
        ;;
esac
```

These advanced examples demonstrate sophisticated Bash scripting techniques including process management, network programming, data processing, and security implementations. Each example showcases real-world applications and advanced programming patterns that can be adapted for various use cases.

Automation Examples

Automation Examples

Automation Examples

This section contains practical automation scripts for common system administration and development tasks.

System Administration Automation

Automated Backup System

```
#!/bin/bash
# automated_backup.sh - Comprehensive backup automation system

# Configuration
BACKUP_CONFIG="${BACKUP_CONFIG:-/etc/backup.conf}"
BACKUP_ROOT="${BACKUP_ROOT:-/backup}"
LOG_FILE="${LOG_FILE:-/var/log/backup.log}"
RETENTION_DAYS="${RETENTION_DAYS:-30}"
COMPRESSION="${COMPRESSION:-gzip}"
ENCRYPTION="${ENCRYPTION:-false}"
GPG_RECIPIENT="${GPG_RECIPIENT:-}"

# Default backup sources
declare -a BACKUP_SOURCES=(
    "/home"
    "/etc"
    "/var/www"
    "/opt"
)

# Logging function
log() {
    local level="$1"
    shift
    local message="$*"
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$level] $message" | tee -a "$LOG_FILE"
}

# Load configuration
load_config() {
```

```
    if [ -f "$BACKUP_CONFIG" ]; then
        source "$BACKUP_CONFIG"
        log INFO "Configuration loaded from $BACKUP_CONFIG"
    else
        log WARN "Configuration file not found, using defaults"
    fi
}

# Create backup directory structure
setup_backup_dirs() {
    local timestamp=$(date +%Y%m%d_%H%M%S)
    BACKUP_DIR="$BACKUP_ROOT/$timestamp"

    mkdir -p "$BACKUP_DIR"
    log INFO "Created backup directory: $BACKUP_DIR"
}

# Backup database
backup_database() {
    local db_type="$1"
    local db_name="$2"
    local db_user="$3"
    local db_host="${4:-localhost}"

    log INFO "Starting database backup: $db_type/$db_name"

    case "$db_type" in
        mysql)
            if command -v mysqldump >/dev/null 2>&1; then
                mysqldump -h "$db_host" -u "$db_user" -p"$DB_PASSWORD" "$db_name" > "$BACKUP
                log INFO "MySQL database backup completed: $db_name"
            else
                log ERROR "mysqldump not found"
                return 1
            fi
            ;;
        postgresql)
            if command -v pg_dump >/dev/null 2>&1; then
                PGPASSWORD="$DB_PASSWORD" pg_dump -h "$db_host" -U "$db_user" "$db_name" > "
                log INFO "PostgreSQL database backup completed: $db_name"
            else
                log ERROR "pg_dump not found"
                return 1
            fi
            ;;
        *)
```

```
            log ERROR "Unsupported database type: $db_type"
            return 1
            ;;
    esac
}

# Backup files and directories
backup_files() {
    local source="$1"
    local exclude_file="/tmp/backup_exclude.$$"

    # Create exclude file
    cat > "$exclude_file" << EOF
*.tmp
*.log
*.cache
*~
.DS_Store
Thumbs.db
EOF

    if [ -d "$source" ]; then
        local backup_name=$(basename "$source")
        local archive_file="$BACKUP_DIR/${backup_name}.tar"

        log INFO "Backing up directory: $source"

        if tar --exclude-from="$exclude_file" -cf "$archive_file" "$source"; then
            # Compress if requested
            if [ "$COMPRESSION" = "gzip" ]; then
                gzip "$archive_file"
                archive_file="${archive_file}.gz"
            elif [ "$COMPRESSION" = "bzip2" ]; then
                bzip2 "$archive_file"
                archive_file="${archive_file}.bz2"
            fi

            # Encrypt if requested
            if [ "$ENCRYPTION" = "true" ] && [ -n "$GPG_RECIPIENT" ]; then
                gpg --trust-model always --encrypt -r "$GPG_RECIPIENT" "$archive_file"
                rm "$archive_file"
                archive_file="${archive_file}.gpg"
            fi

            log INFO "Backup completed: $archive_file"
        else
```

```
            log ERROR "Backup failed for: $source"
        fi
    else
        log WARN "Source not found: $source"
    fi

    rm -f "$exclude_file"
}

# Verify backup integrity
verify_backup() {
    local backup_file="$1"

    log INFO "Verifying backup: $backup_file"

    case "$backup_file" in
        *.tar.gz)
            if gzip -t "$backup_file" && tar -tzf "$backup_file" >/dev/null; then
                log INFO "Backup verification successful: $backup_file"
                return 0
            fi
            ;;
        *.tar.bz2)
            if bzip2 -t "$backup_file" && tar -tjf "$backup_file" >/dev/null; then
                log INFO "Backup verification successful: $backup_file"
                return 0
            fi
            ;;
        *.tar)
            if tar -tf "$backup_file" >/dev/null; then
                log INFO "Backup verification successful: $backup_file"
                return 0
            fi
            ;;
        *.gpg)
            if gpg --list-packets "$backup_file" >/dev/null 2>&1; then
                log INFO "Encrypted backup verification successful: $backup_file"
                return 0
            fi
            ;;
    esac

    log ERROR "Backup verification failed: $backup_file"
    return 1
}

# Clean old backups
```

```
cleanup_old_backups() {
    log INFO "Cleaning up backups older than $RETENTION_DAYS days"

    local deleted_count=0
    find "$BACKUP_ROOT" -maxdepth 1 -type d -name "20*" -mtime +$RETENTION_DAYS | while read
        log INFO "Removing old backup: $old_backup"
        rm -rf "$old_backup"
        ((deleted_count++))
    done

    log INFO "Cleanup completed, removed $deleted_count old backups"
}

# Send notification
send_notification() {
    local status="$1"
    local summary="$2"

    if [ -n "$NOTIFICATION_EMAIL" ] && command -v mail >/dev/null 2>&1; then
        {
            echo "Backup Status: $status"
            echo "Time: $(date)"
            echo "Summary: $summary"
            echo
            echo "Recent log entries:"
            tail -20 "$LOG_FILE"
        } | mail -s "Backup Report - $status" "$NOTIFICATION_EMAIL"

        log INFO "Notification sent to $NOTIFICATION_EMAIL"
    fi

    if [ -n "$SLACK_WEBHOOK" ] && command -v curl >/dev/null 2>&1; then
        local color="good"
        [ "$status" != "SUCCESS" ] && color="danger"

        curl -X POST -H 'Content-type: application/json' \
            --data "{\"attachments\":[{\"color\":\"$color\",\"title\":\"Backup Report\",\"te
            "$SLACK_WEBHOOK" >/dev/null 2>&1

        log INFO "Slack notification sent"
    fi
}

# Generate backup report
generate_report() {
    local report_file="$BACKUP_DIR/backup_report.txt"
    local total_size=0
```

```
    local file_count=0

    {
        echo "Backup Report"
        echo "============="
        echo "Date: $(date)"
        echo "Backup Directory: $BACKUP_DIR"
        echo

        echo "Backed up files:"
        echo "================"

        for file in "$BACKUP_DIR"/*; do
            if [ -f "$file" ] && [[ "$(basename "$file")" != "backup_report.txt" ]]; then
                local size=$(stat -c%s "$file" 2>/dev/null || stat -f%z "$file" 2>/dev/null)
                local human_size=$(numfmt --to=iec-i --suffix=B "$size")

                echo "$(basename "$file"): $human_size"
                total_size=$((total_size + size))
                ((file_count++))
            fi
        done

        echo
        echo "Summary:"
        echo "========"
        echo "Total files: $file_count"
        echo "Total size: $(numfmt --to=iec-i --suffix=B $total_size)"
        echo "Compression: $COMPRESSION"
        echo "Encryption: $ENCRYPTION"

    } > "$report_file"

    log INFO "Backup report generated: $report_file"
}

# Main backup process
main() {
    log INFO "Starting automated backup process"

    load_config
    setup_backup_dirs

    local backup_success=true
    local backed_up_items=0

    # Backup files and directories
```

```
for source in "${BACKUP_SOURCES[@]}"; do
    if backup_files "$source"; then
        ((backed_up_items++))
    else
        backup_success=false
    fi
done

# Backup databases if configured
if [ -n "$DATABASES" ]; then
    IFS=',' read -ra DB_LIST <<< "$DATABASES"
    for db_config in "${DB_LIST[@]}"; do
        IFS=':' read -ra DB_PARTS <<< "$db_config"
        if [ ${#DB_PARTS[@]} -ge 3 ]; then
            if backup_database "${DB_PARTS[0]}" "${DB_PARTS[1]}" "${DB_PARTS[2]}" "${DB_
                ((backed_up_items++))
            else
                backup_success=false
            fi
        fi
    done
fi

# Verify backups
for backup_file in "$BACKUP_DIR"/*; do
    if [ -f "$backup_file" ] && [[ "$(basename "$backup_file")" != "backup_report.txt" ]
        if ! verify_backup "$backup_file"; then
            backup_success=false
        fi
    fi
done

# Generate report
generate_report

# Cleanup old backups
cleanup_old_backups

# Send notification
if [ "$backup_success" = true ]; then
    send_notification "SUCCESS" "Backup completed successfully ($backed_up_items items)"
    log INFO "Backup process completed successfully"
else
    send_notification "FAILED" "Backup completed with errors"
    log ERROR "Backup process completed with errors"
    exit 1
fi
```

```
}

# Handle command line arguments
case "${1:-backup}" in
    backup)
        main
        ;;
    config)
        cat > "$BACKUP_CONFIG" << EOF
# Backup Configuration File
# Generated on $(date)

# Backup sources (space-separated)
BACKUP_SOURCES=(
    "/home"
    "/etc"
    "/var/www"
    "/opt"
)

# Database backups (format: type:name:user:host)
# DATABASES="mysql:myapp:dbuser:localhost,postgresql:webapp:pguser:localhost"

# Backup settings
RETENTION_DAYS=30
COMPRESSION="gzip"  # gzip, bzip2, or none
ENCRYPTION="false"
GPG_RECIPIENT=""

# Notifications
NOTIFICATION_EMAIL=""
SLACK_WEBHOOK=""

# Database credentials
DB_PASSWORD=""
EOF
        echo "Configuration template created: $BACKUP_CONFIG"
        ;;
    restore)
        echo "Restore functionality not implemented yet"
        ;;
    *)
        echo "Usage: $0 {backup|config|restore}"
        echo "Environment variables:"
        echo "  BACKUP_CONFIG    - Configuration file path"
        echo "  BACKUP_ROOT      - Root backup directory"
```

```
        echo "  LOG_FILE         - Log file path"
        exit 1
        ;;
esac
```

System Health Monitor

```
#!/bin/bash
# health_monitor.sh - Comprehensive system health monitoring

MONITOR_CONFIG="${MONITOR_CONFIG:-/etc/health_monitor.conf}"
LOG_FILE="${LOG_FILE:-/var/log/health_monitor.log}"
ALERT_LOG="${ALERT_LOG:-/var/log/health_alerts.log}"
CHECK_INTERVAL="${CHECK_INTERVAL:-300}"  # 5 minutes

# Thresholds
CPU_THRESHOLD="${CPU_THRESHOLD:-80}"
MEMORY_THRESHOLD="${MEMORY_THRESHOLD:-85}"
DISK_THRESHOLD="${DISK_THRESHOLD:-90}"
LOAD_THRESHOLD="${LOAD_THRESHOLD:-2.0}"
INODE_THRESHOLD="${INODE_THRESHOLD:-90}"

# Alert settings
ALERT_EMAIL="${ALERT_EMAIL:-}"
ALERT_WEBHOOK="${ALERT_WEBHOOK:-}"
ALERT_COOLDOWN="${ALERT_COOLDOWN:-3600}"  # 1 hour

# Services to monitor
CRITICAL_SERVICES=("ssh" "nginx" "apache2" "mysql" "postgresql")

# Logging function
log() {
    local level="$1"
    shift
    local message="$*"
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$level] $message" | tee -a "$LOG_FILE"
}

# Alert function
alert() {
    local severity="$1"
    local message="$2"
    local alert_key="$3"

    # Check cooldown
```

```
    local last_alert_file="/tmp/alert_${alert_key}.last"
    if [ -f "$last_alert_file" ]; then
        local last_alert=$(cat "$last_alert_file")
        local current_time=$(date +%s)
        local time_diff=$((current_time - last_alert))

        if [ $time_diff -lt $ALERT_COOLDOWN ]; then
            log DEBUG "Alert suppressed due to cooldown: $alert_key"
            return
        fi
    fi

    # Log alert
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$severity] $message" >> "$ALERT_LOG"
    log WARN "ALERT [$severity]: $message"

    # Send notifications
    send_alert_notification "$severity" "$message"

    # Update cooldown
    date +%s > "$last_alert_file"
}

# Send alert notifications
send_alert_notification() {
    local severity="$1"
    local message="$2"

    # Email notification
    if [ -n "$ALERT_EMAIL" ] && command -v mail >/dev/null 2>&1; then
        {
            echo "System Health Alert"
            echo "==================="
            echo "Severity: $severity"
            echo "Time: $(date)"
            echo "Host: $(hostname)"
            echo
            echo "Message: $message"
            echo
            echo "System Information:"
            echo "==================="
            uname -a
            echo
            echo "Load Average:"
            uptime
            echo
```

```
            echo "Memory Usage:"
            free -h
            echo
            echo "Disk Usage:"
            df -h
        } | mail -s "[$severity] System Health Alert - $(hostname)" "$ALERT_EMAIL"
    fi

    # Webhook notification
    if [ -n "$ALERT_WEBHOOK" ] && command -v curl >/dev/null 2>&1; then
        local color="warning"
        [ "$severity" = "CRITICAL" ] && color="danger"

        curl -X POST -H 'Content-type: application/json' \
            --data "{\"attachments\":[{\"color\":\"$color\",\"title\":\"System Health Alert\
            "$ALERT_WEBHOOK" >/dev/null 2>&1
    fi
}

# Check CPU usage
check_cpu() {
    local cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)
    cpu_usage=${cpu_usage%.*}

    log INFO "CPU usage: ${cpu_usage}%"

    if [ "$cpu_usage" -gt "$CPU_THRESHOLD" ]; then
        alert "WARNING" "High CPU usage: ${cpu_usage}% (threshold: ${CPU_THRESHOLD}%)" "cpu_

        # Get top CPU processes
        local top_processes=$(ps aux --sort=-%cpu | head -6 | tail -5 | awk '{print $11 " ("
        log INFO "Top CPU processes: $top_processes"
    fi

    echo "cpu_usage:$cpu_usage"
}

# Check memory usage
check_memory() {
    local memory_info=$(free | grep Mem)
    local total=$(echo $memory_info | awk '{print $2}')
    local used=$(echo $memory_info | awk '{print $3}')
    local available=$(echo $memory_info | awk '{print $7}')
    local memory_usage=$((used * 100 / total))

    log INFO "Memory usage: ${memory_usage}% (${used}/${total})"
```

```
    if [ "$memory_usage" -gt "$MEMORY_THRESHOLD" ]; then
        alert "WARNING" "High memory usage: ${memory_usage}% (threshold: ${MEMORY_THRESHOLD}

        # Get top memory processes
        local top_processes=$(ps aux --sort=-%mem | head -6 | tail -5 | awk '{print $11 " ("
        log INFO "Top memory processes: $top_processes"
    fi

    echo "memory_usage:$memory_usage"
    echo "memory_available:$available"
}

# Check disk usage
check_disk() {
    local alerts_sent=false

    df -h | grep -vE '^Filesystem|tmpfs|cdrom' | while read output; do
        local usage=$(echo $output | awk '{print $5}' | cut -d'%' -f1)
        local partition=$(echo $output | awk '{print $1}')
        local mount_point=$(echo $output | awk '{print $6}')
        local size=$(echo $output | awk '{print $2}')
        local available=$(echo $output | awk '{print $4}')

        log INFO "Disk usage $mount_point: ${usage}% ($available available)"

        if [ "$usage" -gt "$DISK_THRESHOLD" ]; then
            alert "WARNING" "High disk usage on $mount_point: ${usage}% (threshold: ${DISK_T
            alerts_sent=true
        fi

        echo "disk_usage_${mount_point//\//_}:$usage"
    done

    # Check inode usage
    df -i | grep -vE '^Filesystem|tmpfs|cdrom' | while read output; do
        local inode_usage=$(echo $output | awk '{print $5}' | cut -d'%' -f1)
        local mount_point=$(echo $output | awk '{print $6}')

        if [ "$inode_usage" -gt "$INODE_THRESHOLD" ]; then
            alert "WARNING" "High inode usage on $mount_point: ${inode_usage}% (threshold: $
        fi

        echo "inode_usage_${mount_point//\//_}:$inode_usage"
    done
}
```

```
# Check system load
check_load() {
    local load_1min=$(uptime | awk -F'load average:' '{print $2}' | awk '{print $1}' | tr -d
    local cpu_cores=$(nproc)
    local load_per_core=$(echo "scale=2; $load_1min / $cpu_cores" | bc -l)

    log INFO "Load average: $load_1min (${load_per_core} per core)"

    if (( $(echo "$load_per_core > $LOAD_THRESHOLD" | bc -l) )); then
        alert "WARNING" "High system load: $load_1min (${load_per_core} per core, threshold:
    fi

    echo "load_1min:$load_1min"
    echo "load_per_core:$load_per_core"
}

# Check critical services
check_services() {
    local failed_services=()

    for service in "${CRITICAL_SERVICES[@]}"; do
        if systemctl is-active --quiet "$service" 2>/dev/null; then
            log INFO "Service $service: running"
            echo "service_${service}:1"
        elif systemctl list-unit-files | grep -q "^$service.service"; then
            log WARN "Service $service: stopped"
            failed_services+=("$service")
            echo "service_${service}:0"
        else
            log DEBUG "Service $service: not installed"
            echo "service_${service}:-1"
        fi
    done

    if [ ${#failed_services[@]} -gt 0 ]; then
        alert "CRITICAL" "Critical services not running: ${failed_services[*]}" "services_do
    fi
}

# Check network connectivity
check_network() {
    local test_hosts=("8.8.8.8" "1.1.1.1")
    local failed_hosts=()

    for host in "${test_hosts[@]}"; do
        if ping -c 1 -W 5 "$host" >/dev/null 2>&1; then
            log INFO "Network connectivity to $host: OK"
```

```
            echo "network_${host//./_}:1"
        else
            log WARN "Network connectivity to $host: FAILED"
            failed_hosts+=("$host")
            echo "network_${host//./_}:0"
        fi
    done

    if [ ${#failed_hosts[@]} -gt 0 ]; then
        alert "CRITICAL" "Network connectivity issues: ${failed_hosts[*]}" "network_down"
    fi
}

# Check SSL certificates
check_ssl_certificates() {
    local cert_paths=("/etc/ssl/certs" "/etc/letsencrypt/live")
    local expiring_certs=()

    for cert_path in "${cert_paths[@]}"; do
        if [ -d "$cert_path" ]; then
            find "$cert_path" -name "*.crt" -o -name "cert.pem" | while read -r cert_file; d
                if [ -f "$cert_file" ]; then
                    local expiry_date=$(openssl x509 -in "$cert_file" -noout -enddate 2>/dev
                    if [ -n "$expiry_date" ]; then
                        local expiry_timestamp=$(date -d "$expiry_date" +%s 2>/dev/null)
                        local current_timestamp=$(date +%s)
                        local days_until_expiry=$(( (expiry_timestamp - current_timestamp) /

                        log INFO "SSL certificate $cert_file expires in $days_until_expiry d

                        if [ "$days_until_expiry" -lt 30 ]; then
                            expiring_certs+=("$(basename "$cert_file"):$days_until_expiry")
                        fi

                        echo "ssl_cert_$(basename "$cert_file" .crt | tr '.' '_')_days:$days
                    fi
                fi
            done
        fi
    done

    if [ ${#expiring_certs[@]} -gt 0 ]; then
        alert "WARNING" "SSL certificates expiring soon: ${expiring_certs[*]}" "ssl_expiring
    fi
}
```

```
# Check log files for errors
check_log_errors() {
    local log_files=("/var/log/syslog" "/var/log/auth.log" "/var/log/kern.log")
    local error_threshold=50
    local time_window="1 hour ago"

    for log_file in "${log_files[@]}"; do
        if [ -f "$log_file" ]; then
            local error_count=$(find "$log_file" -newermt "$time_window" -exec grep -i "erro

            log INFO "Errors in $(basename "$log_file"): $error_count (last hour)"

            if [ "$error_count" -gt "$error_threshold" ]; then
                alert "WARNING" "High error count in $log_file: $error_count errors in last
            fi

            echo "log_errors_$(basename "$log_file"):$error_count"
        fi
    done
}

# Generate metrics for monitoring systems
generate_metrics() {
    local metrics_file="/tmp/health_metrics.$$"

    {
        check_cpu
        check_memory
        check_disk
        check_load
        check_services
        check_network
        check_ssl_certificates
        check_log_errors
    } > "$metrics_file"

    # Output in Prometheus format if requested
    if [ "$OUTPUT_FORMAT" = "prometheus" ]; then
        while IFS=':' read -r metric value; do
            echo "system_${metric} ${value}"
        done < "$metrics_file"
    else
        cat "$metrics_file"
    fi

    rm -f "$metrics_file"
}
```

```
# Main monitoring loop
monitor_loop() {
    log INFO "Starting health monitoring loop (interval: ${CHECK_INTERVAL}s)"

    while true; do
        log INFO "Running health checks..."

        generate_metrics > "/tmp/current_metrics.txt"

        log INFO "Health check completed"

        sleep "$CHECK_INTERVAL"
    done
}

# One-time health check
health_check() {
    log INFO "Running one-time health check"
    generate_metrics
}

# Show system status dashboard
show_dashboard() {
    clear
    echo "System Health Dashboard - $(hostname)"
    echo "======================================"
    echo "Last updated: $(date)"
    echo

    # System info
    echo "System Information:"
    echo "=================="
    echo "Uptime: $(uptime -p)"
    echo "Kernel: $(uname -r)"
    echo "Architecture: $(uname -m)"
    echo

    # Current metrics
    echo "Current Metrics:"
    echo "==============="
    generate_metrics | while IFS=':' read -r metric value; do
        printf "%-30s: %s\n" "$metric" "$value"
    done

    echo
    echo "Recent Alerts:"
```

```
    echo "============="
    if [ -f "$ALERT_LOG" ]; then
        tail -5 "$ALERT_LOG"
    else
        echo "No recent alerts"
    fi
}

# Load configuration
load_config() {
    if [ -f "$MONITOR_CONFIG" ]; then
        source "$MONITOR_CONFIG"
        log INFO "Configuration loaded from $MONITOR_CONFIG"
    fi
}

# Main execution
load_config

case "${1:-check}" in
    check)
        health_check
        ;;
    monitor)
        monitor_loop
        ;;
    dashboard)
        show_dashboard
        ;;
    metrics)
        OUTPUT_FORMAT="prometheus" generate_metrics
        ;;
    config)
        cat > "$MONITOR_CONFIG" << EOF
# Health Monitor Configuration
# Generated on $(date)

# Check intervals (seconds)
CHECK_INTERVAL=300

# Thresholds
CPU_THRESHOLD=80
MEMORY_THRESHOLD=85
DISK_THRESHOLD=90
LOAD_THRESHOLD=2.0
INODE_THRESHOLD=90
```

```
# Alert settings
ALERT_EMAIL=""
ALERT_WEBHOOK=""
ALERT_COOLDOWN=3600

# Critical services to monitor
CRITICAL_SERVICES=("ssh" "nginx" "apache2" "mysql" "postgresql")

# Log files
LOG_FILE="/var/log/health_monitor.log"
ALERT_LOG="/var/log/health_alerts.log"
EOF
        echo "Configuration template created: $MONITOR_CONFIG"
        ;;
    *)
        echo "Usage: $0 {check|monitor|dashboard|metrics|config}"
        echo "Commands:"
        echo "  check      - Run one-time health check"
        echo "  monitor    - Start continuous monitoring"
        echo "  dashboard  - Show system status dashboard"
        echo "  metrics    - Output metrics in Prometheus format"
        echo "  config     - Generate configuration template"
        exit 1
        ;;
esac
```##
Development Automation

CI/CD Pipeline Script
```bash
#!/bin/bash
# cicd_pipeline.sh - Continuous Integration/Continuous Deployment pipeline

PROJECT_NAME="${PROJECT_NAME:-myapp}"
GIT_REPO="${GIT_REPO:-}"
BUILD_DIR="${BUILD_DIR:-./build}"
DEPLOY_DIR="${DEPLOY_DIR:-/var/www/$PROJECT_NAME}"
LOG_FILE="${LOG_FILE:-./pipeline.log}"

# Pipeline stages
STAGES=("checkout" "test" "build" "deploy" "notify")

# Configuration
SLACK_WEBHOOK="${SLACK_WEBHOOK:-}"
EMAIL_NOTIFICATIONS="${EMAIL_NOTIFICATIONS:-}"
DOCKER_REGISTRY="${DOCKER_REGISTRY:-}"
KUBERNETES_NAMESPACE="${KUBERNETES_NAMESPACE:-default}"
```

```
# Colors for output
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

# Logging function
log() {
    local level="$1"
    shift
    local message="$*"
    local timestamp=$(date '+%Y-%m-%d %H:%M:%S')

    echo "[$timestamp] [$level] $message" | tee -a "$LOG_FILE"

    case "$level" in
        ERROR) echo -e "${RED}[$timestamp] ERROR: $message${NC}" ;;
        WARN)  echo -e "${YELLOW}[$timestamp] WARN:  $message${NC}" ;;
        INFO)  echo -e "${GREEN}[$timestamp] INFO:  $message${NC}" ;;
        DEBUG) echo -e "${BLUE}[$timestamp] DEBUG: $message${NC}" ;;
    esac
}

# Send notification
send_notification() {
    local status="$1"
    local stage="$2"
    local message="$3"

    # Slack notification
    if [ -n "$SLACK_WEBHOOK" ] && command -v curl >/dev/null 2>&1; then
        local color="good"
        [ "$status" != "SUCCESS" ] && color="danger"

        curl -X POST -H 'Content-type: application/json' \
            --data "{\"attachments\":[{\"color\":\"$color\",\"title\":\"Pipeline $status\",\
            "$SLACK_WEBHOOK" >/dev/null 2>&1
    fi

    # Email notification
    if [ -n "$EMAIL_NOTIFICATIONS" ] && command -v mail >/dev/null 2>&1; then
        {
            echo "Pipeline Status: $status"
            echo "Project: $PROJECT_NAME"
            echo "Stage: $stage"
```

```
            echo "Time: $(date)"
            echo
            echo "Message: $message"
            echo
            echo "Recent log entries:"
            tail -20 "$LOG_FILE"
        } | mail -s "Pipeline $status - $PROJECT_NAME" "$EMAIL_NOTIFICATIONS"
    fi
}

# Checkout stage
stage_checkout() {
    log INFO "Starting checkout stage"

    if [ -z "$GIT_REPO" ]; then
        log ERROR "GIT_REPO not specified"
        return 1
    fi

    # Clean previous build
    rm -rf "$BUILD_DIR"
    mkdir -p "$BUILD_DIR"

    # Clone repository
    if git clone "$GIT_REPO" "$BUILD_DIR"; then
        log INFO "Repository cloned successfully"

        # Get commit information
        cd "$BUILD_DIR"
        local commit_hash=$(git rev-parse HEAD)
        local commit_message=$(git log -1 --pretty=%B)
        local author=$(git log -1 --pretty=%an)

        log INFO "Commit: $commit_hash"
        log INFO "Author: $author"
        log INFO "Message: $commit_message"

        return 0
    else
        log ERROR "Failed to clone repository"
        return 1
    fi
}

# Test stage
stage_test() {
```

```
    log INFO "Starting test stage"

    cd "$BUILD_DIR"

    # Detect project type and run appropriate tests
    if [ -f "package.json" ]; then
        # Node.js project
        log INFO "Detected Node.js project"

        if command -v npm >/dev/null 2>&1; then
            log INFO "Installing dependencies..."
            if npm install; then
                log INFO "Dependencies installed"
            else
                log ERROR "Failed to install dependencies"
                return 1
            fi

            log INFO "Running tests..."
            if npm test; then
                log INFO "Tests passed"
            else
                log ERROR "Tests failed"
                return 1
            fi
        else
            log ERROR "npm not found"
            return 1
        fi

    elif [ -f "requirements.txt" ] || [ -f "setup.py" ]; then
        # Python project
        log INFO "Detected Python project"

        if command -v python3 >/dev/null 2>&1; then
            # Create virtual environment
            python3 -m venv venv
            source venv/bin/activate

            # Install dependencies
            if [ -f "requirements.txt" ]; then
                log INFO "Installing Python dependencies..."
                if pip install -r requirements.txt; then
                    log INFO "Dependencies installed"
                else
                    log ERROR "Failed to install dependencies"
                    return 1
```

```
                fi
            fi

            # Run tests
            if [ -f "pytest.ini" ] || [ -d "tests" ]; then
                log INFO "Running Python tests..."
                if python -m pytest; then
                    log INFO "Tests passed"
                else
                    log ERROR "Tests failed"
                    return 1
                fi
            fi
        else
            log ERROR "python3 not found"
            return 1
        fi

    elif [ -f "Makefile" ]; then
        # Make-based project
        log INFO "Detected Make-based project"

        if make test; then
            log INFO "Tests passed"
        else
            log ERROR "Tests failed"
            return 1
        fi

    else
        log WARN "No recognized test framework found, skipping tests"
    fi

    return 0
}

# Build stage
stage_build() {
    log INFO "Starting build stage"

    cd "$BUILD_DIR"

    # Detect project type and build
    if [ -f "Dockerfile" ]; then
        # Docker build
        log INFO "Building Docker image"
```

```
    local image_tag="$PROJECT_NAME:$(git rev-parse --short HEAD)"

    if docker build -t "$image_tag" .; then
        log INFO "Docker image built: $image_tag"

        # Push to registry if configured
        if [ -n "$DOCKER_REGISTRY" ]; then
            local registry_image="$DOCKER_REGISTRY/$image_tag"

            if docker tag "$image_tag" "$registry_image"; then
                log INFO "Tagged image for registry: $registry_image"

                if docker push "$registry_image"; then
                    log INFO "Image pushed to registry"
                else
                    log ERROR "Failed to push image to registry"
                    return 1
                fi
            fi
        fi
    else
        log ERROR "Docker build failed"
        return 1
    fi

elif [ -f "package.json" ]; then
    # Node.js build
    log INFO "Building Node.js application"

    if npm run build 2>/dev/null || npm run compile 2>/dev/null; then
        log INFO "Build completed"
    else
        log WARN "No build script found or build failed"
    fi

elif [ -f "Makefile" ]; then
    # Make build
    log INFO "Building with Make"

    if make; then
        log INFO "Build completed"
    else
        log ERROR "Build failed"
        return 1
    fi

else
```

```
        log INFO "No build process detected, copying files"

        # Simple file copy
        mkdir -p dist
        cp -r . dist/
        log INFO "Files copied to dist/"
    fi

    return 0
}

# Deploy stage
stage_deploy() {
    log INFO "Starting deploy stage"

    cd "$BUILD_DIR"

    # Backup current deployment
    if [ -d "$DEPLOY_DIR" ]; then
        local backup_dir="${DEPLOY_DIR}.backup.$(date +%Y%m%d_%H%M%S)"
        log INFO "Creating backup: $backup_dir"
        cp -r "$DEPLOY_DIR" "$backup_dir"
    fi

    # Deploy based on project type
    if [ -f "Dockerfile" ] && command -v kubectl >/dev/null 2>&1; then
        # Kubernetes deployment
        log INFO "Deploying to Kubernetes"

        local image_tag="$PROJECT_NAME:$(git rev-parse --short HEAD)"

        # Update deployment
        if kubectl set image deployment/$PROJECT_NAME container=$DOCKER_REGISTRY/$image_tag
            log INFO "Kubernetes deployment updated"

            # Wait for rollout
            if kubectl rollout status deployment/$PROJECT_NAME -n $KUBERNETES_NAMESPACE; the
                log INFO "Deployment rollout completed"
            else
                log ERROR "Deployment rollout failed"
                return 1
            fi
        else
            log ERROR "Failed to update Kubernetes deployment"
            return 1
        fi
```

```
    else
        # File-based deployment
        log INFO "Deploying files to $DEPLOY_DIR"

        # Create deploy directory
        mkdir -p "$DEPLOY_DIR"

        # Copy files
        if [ -d "dist" ]; then
            cp -r dist/* "$DEPLOY_DIR/"
        else
            cp -r . "$DEPLOY_DIR/"
        fi

        # Set permissions
        chown -R www-data:www-data "$DEPLOY_DIR" 2>/dev/null || true
        chmod -R 755 "$DEPLOY_DIR"

        # Restart services if needed
        if systemctl is-active --quiet nginx; then
            systemctl reload nginx
            log INFO "Nginx reloaded"
        fi

        if systemctl is-active --quiet apache2; then
            systemctl reload apache2
            log INFO "Apache reloaded"
        fi

        log INFO "Deployment completed"
    fi

    return 0
}

# Notify stage
stage_notify() {
    log INFO "Starting notify stage"

    local commit_hash=$(cd "$BUILD_DIR" && git rev-parse --short HEAD)
    local commit_message=$(cd "$BUILD_DIR" && git log -1 --pretty=%B | head -1)

    send_notification "SUCCESS" "deploy" "Deployment completed successfully. Commit: $commit

    return 0
}
```

```
# Run pipeline
run_pipeline() {
    local start_time=$(date +%s)
    log INFO "Starting CI/CD pipeline for $PROJECT_NAME"

    for stage in "${STAGES[@]}"; do
        log INFO "Running stage: $stage"

        local stage_start=$(date +%s)

        if "stage_$stage"; then
            local stage_end=$(date +%s)
            local stage_duration=$((stage_end - stage_start))
            log INFO "Stage $stage completed in ${stage_duration}s"
        else
            local stage_end=$(date +%s)
            local stage_duration=$((stage_end - stage_start))
            log ERROR "Stage $stage failed after ${stage_duration}s"

            send_notification "FAILED" "$stage" "Pipeline failed at stage: $stage"
            exit 1
        fi
    done

    local end_time=$(date +%s)
    local total_duration=$((end_time - start_time))

    log INFO "Pipeline completed successfully in ${total_duration}s"
}

# Rollback function
rollback() {
    local backup_dir="$1"

    if [ -z "$backup_dir" ]; then
        # Find latest backup
        backup_dir=$(ls -td "${DEPLOY_DIR}.backup."* 2>/dev/null | head -1)
    fi

    if [ -d "$backup_dir" ]; then
        log INFO "Rolling back to: $backup_dir"

        rm -rf "$DEPLOY_DIR"
        cp -r "$backup_dir" "$DEPLOY_DIR"

        log INFO "Rollback completed"
    else
```

```
        log ERROR "Backup directory not found: $backup_dir"
        exit 1
    fi
}

# Show pipeline status
show_status() {
    echo "CI/CD Pipeline Status"
    echo "====================="
    echo "Project: $PROJECT_NAME"
    echo "Build Directory: $BUILD_DIR"
    echo "Deploy Directory: $DEPLOY_DIR"
    echo

    if [ -f "$LOG_FILE" ]; then
        echo "Recent Activity:"
        echo "================"
        tail -20 "$LOG_FILE"
    fi

    echo
    echo "Available Backups:"
    echo "=================="
    ls -la "${DEPLOY_DIR}.backup."* 2>/dev/null || echo "No backups found"
}

# Main execution
case "${1:-run}" in
    run)
        run_pipeline
        ;;
    checkout)
        stage_checkout
        ;;
    test)
        stage_test
        ;;
    build)
        stage_build
        ;;
    deploy)
        stage_deploy
        ;;
    rollback)
        rollback "$2"
        ;;
    status)
```

```
        show_status
        ;;
    *)
        echo "Usage: $0 {run|checkout|test|build|deploy|rollback|status}"
        echo "Environment variables:"
        echo "  PROJECT_NAME         - Project name"
        echo "  GIT_REPO             - Git repository URL"
        echo "  BUILD_DIR            - Build directory"
        echo "  DEPLOY_DIR           - Deployment directory"
        echo "  SLACK_WEBHOOK        - Slack webhook URL"
        echo "  EMAIL_NOTIFICATIONS - Email for notifications"
        echo "  DOCKER_REGISTRY      - Docker registry URL"
        echo "  KUBERNETES_NAMESPACE - Kubernetes namespace"
        exit 1
        ;;
esac
```

Environment Provisioning Script

```
#!/bin/bash
# provision_environment.sh - Automated environment provisioning

ENVIRONMENT="${1:-development}"
CONFIG_DIR="${CONFIG_DIR:-./config}"
ANSIBLE_PLAYBOOK="${ANSIBLE_PLAYBOOK:-}"
TERRAFORM_DIR="${TERRAFORM_DIR:-./terraform}"

# Environment configurations
declare -A ENV_CONFIGS=(
    ["development"]="dev"
    ["staging"]="stage"
    ["production"]="prod"
)

# Required tools
REQUIRED_TOOLS=("ansible" "terraform" "kubectl" "docker")

# Logging function
log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $*"
}

# Check prerequisites
check_prerequisites() {
    log "Checking prerequisites..."
```

```
    local missing_tools=()

    for tool in "${REQUIRED_TOOLS[@]}"; do
        if ! command -v "$tool" >/dev/null 2>&1; then
            missing_tools+=("$tool")
        fi
    done

    if [ ${#missing_tools[@]} -gt 0 ]; then
        log "ERROR: Missing required tools: ${missing_tools[*]}"
        log "Please install the missing tools and try again."
        exit 1
    fi

    log "All prerequisites satisfied"
}

# Load environment configuration
load_environment_config() {
    local env="$1"
    local config_file="$CONFIG_DIR/${env}.conf"

    if [ -f "$config_file" ]; then
        source "$config_file"
        log "Loaded configuration for environment: $env"
    else
        log "WARNING: Configuration file not found: $config_file"
        log "Using default configuration"
    fi
}

# Provision infrastructure with Terraform
provision_infrastructure() {
    local env="$1"

    if [ ! -d "$TERRAFORM_DIR" ]; then
        log "Terraform directory not found: $TERRAFORM_DIR"
        return 1
    fi

    log "Provisioning infrastructure for environment: $env"

    cd "$TERRAFORM_DIR"

    # Initialize Terraform
    if terraform init; then
```

```
        log "Terraform initialized"
    else
        log "ERROR: Terraform initialization failed"
        return 1
    fi

    # Select workspace
    if terraform workspace select "$env" 2>/dev/null || terraform workspace new "$env"; then
        log "Terraform workspace: $env"
    else
        log "ERROR: Failed to select/create workspace: $env"
        return 1
    fi

    # Plan infrastructure changes
    log "Planning infrastructure changes..."
    if terraform plan -var-file="$CONFIG_DIR/${env}.tfvars" -out="$env.tfplan"; then
        log "Terraform plan completed"
    else
        log "ERROR: Terraform planning failed"
        return 1
    fi

    # Apply infrastructure changes
    log "Applying infrastructure changes..."
    if terraform apply "$env.tfplan"; then
        log "Infrastructure provisioned successfully"

        # Save outputs
        terraform output -json > "$CONFIG_DIR/${env}_outputs.json"
        log "Terraform outputs saved"
    else
        log "ERROR: Terraform apply failed"
        return 1
    fi

    cd - >/dev/null
}

# Configure servers with Ansible
configure_servers() {
    local env="$1"
    local playbook="${ANSIBLE_PLAYBOOK:-site.yml}"

    if [ ! -f "$playbook" ]; then
        log "Ansible playbook not found: $playbook"
        return 1
```

```
    fi

    log "Configuring servers for environment: $env"

    # Run Ansible playbook
    if ansible-playbook -i "inventory/${env}" "$playbook" --extra-vars "environment=$env"; t
        log "Server configuration completed"
    else
        log "ERROR: Ansible playbook execution failed"
        return 1
    fi
}

# Deploy applications
deploy_applications() {
    local env="$1"

    log "Deploying applications for environment: $env"

    # Deploy to Kubernetes if configured
    if [ -n "$KUBERNETES_NAMESPACE" ] && command -v kubectl >/dev/null 2>&1; then
        log "Deploying to Kubernetes namespace: $KUBERNETES_NAMESPACE"

        # Apply Kubernetes manifests
        if [ -d "k8s/$env" ]; then
            kubectl apply -f "k8s/$env/" -n "$KUBERNETES_NAMESPACE"
            log "Kubernetes manifests applied"
        fi

        # Wait for deployments to be ready
        kubectl wait --for=condition=available --timeout=300s deployment --all -n "$KUBERNET
        log "All deployments are ready"
    fi

    # Deploy with Docker Compose if configured
    if [ -f "docker-compose.${env}.yml" ]; then
        log "Deploying with Docker Compose"

        docker-compose -f "docker-compose.${env}.yml" up -d
        log "Docker Compose deployment completed"
    fi
}

# Setup monitoring and logging
setup_monitoring() {
    local env="$1"
```

```
    log "Setting up monitoring for environment: $env"

    # Deploy monitoring stack
    if [ -d "monitoring/$env" ]; then
        kubectl apply -f "monitoring/$env/" -n "monitoring-$env" 2>/dev/null || true
        log "Monitoring stack deployed"
    fi

    # Configure log aggregation
    if [ -f "logging/fluentd-${env}.yml" ]; then
        kubectl apply -f "logging/fluentd-${env}.yml" -n "logging-$env" 2>/dev/null || true
        log "Log aggregation configured"
    fi
}

# Run health checks
run_health_checks() {
    local env="$1"

    log "Running health checks for environment: $env"

    # Load health check endpoints from configuration
    local health_endpoints=()
    if [ -f "$CONFIG_DIR/${env}_health_checks.txt" ]; then
        while IFS= read -r endpoint; do
            health_endpoints+=("$endpoint")
        done < "$CONFIG_DIR/${env}_health_checks.txt"
    fi

    # Default health checks
    if [ ${#health_endpoints[@]} -eq 0 ]; then
        health_endpoints=("http://localhost:8080/health" "http://localhost:3000/health")
    fi

    local failed_checks=0

    for endpoint in "${health_endpoints[@]}"; do
        log "Checking health endpoint: $endpoint"

        if curl -f -s "$endpoint" >/dev/null; then
            log "  Health check passed: $endpoint"
        else
            log "  Health check failed: $endpoint"
            ((failed_checks++))
        fi
    done
```

```
    if [ $failed_checks -eq 0 ]; then
        log "All health checks passed"
        return 0
    else
        log "ERROR: $failed_checks health checks failed"
        return 1
    fi
}

# Generate environment documentation
generate_documentation() {
    local env="$1"
    local doc_file="docs/${env}_environment.md"

    log "Generating documentation for environment: $env"

    mkdir -p docs

    {
        echo "# $env Environment Documentation"
        echo
        echo "Generated on: $(date)"
        echo

        echo "## Infrastructure"
        if [ -f "$CONFIG_DIR/${env}_outputs.json" ]; then
            echo "### Terraform Outputs"
            echo '```json'
            cat "$CONFIG_DIR/${env}_outputs.json"
            echo '````'
        fi

        echo
        echo "## Services"
        if command -v kubectl >/dev/null 2>&1; then
            echo "### Kubernetes Services"
            echo '````'
            kubectl get services -n "$KUBERNETES_NAMESPACE" 2>/dev/null || echo "No Kubernet
            echo '````'
        fi

        echo
        echo "## Configuration"
        if [ -f "$CONFIG_DIR/${env}.conf" ]; then
            echo "### Environment Configuration"
            echo '```bash'
            cat "$CONFIG_DIR/${env}.conf"
```

```
            echo '```'
        fi

    } > "$doc_file"

    log "Documentation generated: $doc_file"
}

# Cleanup environment
cleanup_environment() {
    local env="$1"

    log "Cleaning up environment: $env"

    # Confirm cleanup
    read -p "Are you sure you want to destroy the $env environment? (yes/no): " confirm
    if [ "$confirm" != "yes" ]; then
        log "Cleanup cancelled"
        return 0
    fi

    # Destroy Kubernetes resources
    if [ -d "k8s/$env" ] && command -v kubectl >/dev/null 2>&1; then
        kubectl delete -f "k8s/$env/" -n "$KUBERNETES_NAMESPACE" 2>/dev/null || true
        log "Kubernetes resources deleted"
    fi

    # Destroy Docker Compose services
    if [ -f "docker-compose.${env}.yml" ]; then
        docker-compose -f "docker-compose.${env}.yml" down -v
        log "Docker Compose services stopped"
    fi

    # Destroy Terraform infrastructure
    if [ -d "$TERRAFORM_DIR" ]; then
        cd "$TERRAFORM_DIR"
        terraform workspace select "$env"
        terraform destroy -var-file="$CONFIG_DIR/${env}.tfvars" -auto-approve
        terraform workspace select default
        terraform workspace delete "$env"
        cd - >/dev/null
        log "Infrastructure destroyed"
    fi

    log "Environment cleanup completed"
}
```

```
# Show environment status
show_status() {
    local env="$1"

    echo "Environment Status: $env"
    echo "======================="
    echo

    # Terraform status
    if [ -d "$TERRAFORM_DIR" ]; then
        echo "Infrastructure Status:"
        cd "$TERRAFORM_DIR"
        terraform workspace select "$env" 2>/dev/null && terraform show -json | jq -r '.valu
        cd - >/dev/null
        echo
    fi

    # Kubernetes status
    if command -v kubectl >/dev/null 2>&1; then
        echo "Kubernetes Status:"
        kubectl get all -n "$KUBERNETES_NAMESPACE" 2>/dev/null || echo "No Kubernetes resour
        echo
    fi

    # Docker status
    if [ -f "docker-compose.${env}.yml" ]; then
        echo "Docker Compose Status:"
        docker-compose -f "docker-compose.${env}.yml" ps
    fi
}

# Main provisioning process
provision_environment() {
    local env="$1"

    log "Starting environment provisioning: $env"

    check_prerequisites
    load_environment_config "$env"

    # Provision infrastructure
    if provision_infrastructure "$env"; then
        log "Infrastructure provisioning completed"
    else
        log "ERROR: Infrastructure provisioning failed"
        exit 1
    fi
```

```
    # Configure servers
    if configure_servers "$env"; then
        log "Server configuration completed"
    else
        log "ERROR: Server configuration failed"
        exit 1
    fi

    # Deploy applications
    if deploy_applications "$env"; then
        log "Application deployment completed"
    else
        log "ERROR: Application deployment failed"
        exit 1
    fi

    # Setup monitoring
    setup_monitoring "$env"

    # Run health checks
    if run_health_checks "$env"; then
        log "Health checks passed"
    else
        log "WARNING: Some health checks failed"
    fi

    # Generate documentation
    generate_documentation "$env"

    log "Environment provisioning completed successfully: $env"
}

# Main execution
case "${2:-provision}" in
    provision)
        provision_environment "$ENVIRONMENT"
        ;;
    infrastructure)
        check_prerequisites
        load_environment_config "$ENVIRONMENT"
        provision_infrastructure "$ENVIRONMENT"
        ;;
    configure)
        check_prerequisites
        load_environment_config "$ENVIRONMENT"
        configure_servers "$ENVIRONMENT"
```

```
        ;;
    deploy)
        check_prerequisites
        load_environment_config "$ENVIRONMENT"
        deploy_applications "$ENVIRONMENT"
        ;;
    health)
        load_environment_config "$ENVIRONMENT"
        run_health_checks "$ENVIRONMENT"
        ;;
    status)
        show_status "$ENVIRONMENT"
        ;;
    cleanup)
        cleanup_environment "$ENVIRONMENT"
        ;;
    docs)
        generate_documentation "$ENVIRONMENT"
        ;;
    *)
        echo "Usage: $0 <environment> {provision|infrastructure|configure|deploy|health|stat
        echo
        echo "Environments: ${!ENV_CONFIGS[*]}"
        echo
        echo "Commands:"
        echo "  provision      - Full environment provisioning"
        echo "  infrastructure - Provision infrastructure only"
        echo "  configure      - Configure servers only"
        echo "  deploy         - Deploy applications only"
        echo "  health         - Run health checks"
        echo "  status         - Show environment status"
        echo "  cleanup        - Destroy environment"
        echo "  docs           - Generate documentation"
        exit 1
        ;;
esac
```

Web Scraping and API Automation

Web Scraper

```
#!/bin/bash
# web_scraper.sh - Web scraping and data extraction tool

URL="$1"
```

```
OUTPUT_DIR="${OUTPUT_DIR:-./scraped_data}"
USER_AGENT="${USER_AGENT:-Mozilla/5.0 (Linux; Bash Web Scraper)}"
DELAY="${DELAY:-1}"
MAX_PAGES="${MAX_PAGES:-10}"

# Create output directory
mkdir -p "$OUTPUT_DIR"

# Logging function
log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $*" | tee -a "$OUTPUT_DIR/scraper.log"
}

# Download page with retry logic
download_page() {
    local url="$1"
    local output_file="$2"
    local max_retries=3
    local retry_count=0

    while [ $retry_count -lt $max_retries ]; do
        if curl -s -L -A "$USER_AGENT" \
               -H "Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8"
               -H "Accept-Language: en-US,en;q=0.5" \
               -H "Accept-Encoding: gzip, deflate" \
               -H "Connection: keep-alive" \
               --compressed \
               "$url" > "$output_file"; then
            log "Downloaded: $url"
            return 0
        else
            ((retry_count++))
            log "Retry $retry_count/$max_retries for: $url"
            sleep $((retry_count * 2))
        fi
    done

    log "Failed to download after $max_retries retries: $url"
    return 1
}

# Extract links from HTML
extract_links() {
    local html_file="$1"
    local base_url="$2"

    # Extract all href attributes
```

```
    grep -oP 'href="\K[^"]*' "$html_file" | while read -r link; do
        # Convert relative URLs to absolute
        if [[ $link =~ ^https?:// ]]; then
            echo "$link"
        elif [[ $link =~ ^/ ]]; then
            echo "${base_url%/}$link"
        elif [[ $link =~ ^[^/] ]]; then
            echo "${base_url%/}/$link"
        fi
    done | sort -u
}

# Extract text content
extract_text() {
    local html_file="$1"
    local output_file="$2"

    # Remove HTML tags and extract text
    sed 's/<[^>]*>//g' "$html_file" | \
    sed 's/ / /g; s/&amp;/\&/g; s/&lt;/</g; s/&gt;/>/g; s/&quot;/"/g' | \
    sed '/^[[:space:]]*$/d' > "$output_file"

    log "Extracted text to: $output_file"
}

# Extract specific data using patterns
extract_data() {
    local html_file="$1"
    local pattern="$2"
    local output_file="$3"

    grep -oP "$pattern" "$html_file" > "$output_file"
    local count=$(wc -l < "$output_file")
    log "Extracted $count items matching pattern: $pattern"
}

# Extract emails
extract_emails() {
    local html_file="$1"
    local output_file="$2"

    grep -oE '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}' "$html_file" | sort -u > "$out
    local count=$(wc -l < "$output_file")
    log "Extracted $count unique email addresses"
}

# Extract phone numbers
```

```
extract_phones() {
    local html_file="$1"
    local output_file="$2"

    # Various phone number patterns
    grep -oE '(\+?1[-.\s]?)?\(?[0-9]{3}\)?[-.\s]?[0-9]{3}[-.\s]?[0-9]{4}' "$html_file" | sor
    local count=$(wc -l < "$output_file")
    log "Extracted $count phone numbers"
}

# Extract images
extract_images() {
    local html_file="$1"
    local base_url="$2"
    local output_file="$3"

    grep -oP 'src="\K[^"]*\.(jpg|jpeg|png|gif|webp)' "$html_file" | while read -r img_src; d
        # Convert relative URLs to absolute
        if [[ $img_src =~ ^https?:// ]]; then
            echo "$img_src"
        elif [[ $img_src =~ ^/ ]]; then
            echo "${base_url%/}$img_src"
        elif [[ $img_src =~ ^[^/] ]]; then
            echo "${base_url%/}/$img_src"
        fi
    done | sort -u > "$output_file"

    local count=$(wc -l < "$output_file")
    log "Extracted $count image URLs"
}

# Download images
download_images() {
    local image_list="$1"
    local download_dir="$OUTPUT_DIR/images"

    mkdir -p "$download_dir"

    while IFS= read -r img_url; do
        local filename=$(basename "$img_url" | cut -d'?' -f1)
        local output_path="$download_dir/$filename"

        if [ ! -f "$output_path" ]; then
            if curl -s -L -A "$USER_AGENT" "$img_url" -o "$output_path"; then
                log "Downloaded image: $filename"
            else
```

```
                log "Failed to download image: $img_url"
            fi
            sleep "$DELAY"
        fi
    done < "$image_list"
}

# Generate sitemap
generate_sitemap() {
    local links_file="$1"
    local sitemap_file="$OUTPUT_DIR/sitemap.xml"

    {
        echo '<?xml version="1.0" encoding="UTF-8"?>'
        echo '<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">'

        while IFS= read -r url; do
            echo "  <url>"
            echo "    <loc>$url</loc>"
            echo "    <lastmod>$(date -I)</lastmod>"
            echo "  </url>"
        done < "$links_file"

        echo '</urlset>'
    } > "$sitemap_file"

    log "Generated sitemap: $sitemap_file"
}

# Analyze page structure
analyze_structure() {
    local html_file="$1"
    local analysis_file="$OUTPUT_DIR/structure_analysis.txt"

    {
        echo "Page Structure Analysis"
        echo "======================="
        echo "Generated: $(date)"
        echo

        echo "HTML Tags Count:"
        echo "================"
        grep -o '<[^>]*>' "$html_file" | sed 's/<\([^ >]*\).*/\1/' | sort | uniq -c | sort -

        echo
        echo "Meta Tags:"
        echo "=========="
```

```
        grep -i '<meta' "$html_file" || echo "No meta tags found"

        echo
        echo "Title:"
        echo "======"
        grep -oP '<title>\K[^<]*' "$html_file" || echo "No title found"

        echo
        echo "Headings:"
        echo "========="
        grep -oP '<h[1-6][^>]*>\K[^<]*' "$html_file" || echo "No headings found"

        echo
        echo "Forms:"
        echo "======"
        grep -c '<form' "$html_file" | xargs echo "Form count:"

        echo
        echo "Scripts:"
        echo "========"
        grep -c '<script' "$html_file" | xargs echo "Script count:"

        echo
        echo "Stylesheets:"
        echo "============"
        grep -c '<link.*stylesheet' "$html_file" | xargs echo "Stylesheet count:"

    } > "$analysis_file"

    log "Structure analysis saved: $analysis_file"
}

# Main scraping function
scrape_website() {
    local start_url="$1"
    local base_url=$(echo "$start_url" | sed 's|^\(https\?://[^/]*\).*|\1|')

    log "Starting web scraping: $start_url"
    log "Base URL: $base_url"

    local page_count=0
    local visited_file="$OUTPUT_DIR/visited_urls.txt"
    local queue_file="$OUTPUT_DIR/url_queue.txt"

    # Initialize queue
    echo "$start_url" > "$queue_file"
    touch "$visited_file"
```

```
while [ $page_count -lt $MAX_PAGES ] && [ -s "$queue_file" ]; do
    # Get next URL from queue
    local current_url=$(head -1 "$queue_file")
    sed -i '1d' "$queue_file"

    # Skip if already visited
    if grep -Fxq "$current_url" "$visited_file"; then
        continue
    fi

    # Mark as visited
    echo "$current_url" >> "$visited_file"

    ((page_count++))
    log "Processing page $page_count/$MAX_PAGES: $current_url"

    # Download page
    local html_file="$OUTPUT_DIR/page_${page_count}.html"
    if download_page "$current_url" "$html_file"; then
        # Extract data
        extract_text "$html_file" "$OUTPUT_DIR/page_${page_count}.txt"
        extract_emails "$html_file" "$OUTPUT_DIR/page_${page_count}_emails.txt"
        extract_phones "$html_file" "$OUTPUT_DIR/page_${page_count}_phones.txt"
        extract_images "$html_file" "$base_url" "$OUTPUT_DIR/page_${page_count}_images.t

        # Extract links for further crawling
        local links_file="$OUTPUT_DIR/page_${page_count}_links.txt"
        extract_links "$html_file" "$base_url" > "$links_file"

        # Add new links to queue (same domain only)
        while IFS= read -r link; do
            if [[ $link =~ ^$base_url ]] && ! grep -Fxq "$link" "$visited_file"; then
                echo "$link" >> "$queue_file"
            fi
        done < "$links_file"

        # Analyze page structure for first page
        if [ $page_count -eq 1 ]; then
            analyze_structure "$html_file"
        fi
    fi

    # Respect rate limiting
    sleep "$DELAY"
done
```

```
    log "Scraping completed. Processed $page_count pages."

    # Generate consolidated reports
    generate_reports
}

# Generate consolidated reports
generate_reports() {
    log "Generating consolidated reports..."

    # Consolidate all emails
    cat "$OUTPUT_DIR"/page_*_emails.txt 2>/dev/null | sort -u > "$OUTPUT_DIR/all_emails.txt"
    local email_count=$(wc -l < "$OUTPUT_DIR/all_emails.txt" 2>/dev/null || echo 0)
    log "Total unique emails found: $email_count"

    # Consolidate all phone numbers
    cat "$OUTPUT_DIR"/page_*_phones.txt 2>/dev/null | sort -u > "$OUTPUT_DIR/all_phones.txt"
    local phone_count=$(wc -l < "$OUTPUT_DIR/all_phones.txt" 2>/dev/null || echo 0)
    log "Total unique phone numbers found: $phone_count"

    # Consolidate all images
    cat "$OUTPUT_DIR"/page_*_images.txt 2>/dev/null | sort -u > "$OUTPUT_DIR/all_images.txt"
    local image_count=$(wc -l < "$OUTPUT_DIR/all_images.txt" 2>/dev/null || echo 0)
    log "Total unique images found: $image_count"

    # Consolidate all links
    cat "$OUTPUT_DIR"/page_*_links.txt 2>/dev/null | sort -u > "$OUTPUT_DIR/all_links.txt"
    local link_count=$(wc -l < "$OUTPUT_DIR/all_links.txt" 2>/dev/null || echo 0)
    log "Total unique links found: $link_count"

    # Generate sitemap
    if [ -f "$OUTPUT_DIR/all_links.txt" ]; then
        generate_sitemap "$OUTPUT_DIR/all_links.txt"
    fi

    # Generate summary report
    {
        echo "Web Scraping Summary Report"
        echo "==========================="
        echo "Generated: $(date)"
        echo "Target URL: $URL"
        echo "Pages processed: $(ls "$OUTPUT_DIR"/page_*.html 2>/dev/null | wc -l)"
        echo "Unique emails: $email_count"
        echo "Unique phones: $phone_count"
        echo "Unique images: $image_count"
        echo "Unique links: $link_count"
        echo
```

```
        echo "Output directory: $OUTPUT_DIR"
        echo
        echo "Files generated:"
        ls -la "$OUTPUT_DIR"
    } > "$OUTPUT_DIR/summary_report.txt"

    log "Summary report generated: $OUTPUT_DIR/summary_report.txt"
}

# Show usage
show_usage() {
    cat << EOF
Web Scraper - Extract data from websites

Usage: $0 <url> [options]

Options:
  -d, --delay SECONDS     Delay between requests (default: 1)
  -m, --max-pages NUM     Maximum pages to scrape (default: 10)
  -o, --output DIR        Output directory (default: ./scraped_data)
  -u, --user-agent STR    User agent string
  -i, --images            Download images
  -h, --help              Show this help

Environment Variables:
  OUTPUT_DIR              Output directory
  USER_AGENT              User agent string
  DELAY                   Delay between requests
  MAX_PAGES               Maximum pages to scrape

Examples:
  $0 https://example.com
  $0 https://example.com -d 2 -m 5
  $0 https://example.com --images
EOF
}

# Parse command line arguments
DOWNLOAD_IMAGES=false

while [[ $# -gt 0 ]]; do
    case $1 in
        -d|--delay)
            DELAY="$2"
            shift 2
            ;;
```

```
        -m|--max-pages)
            MAX_PAGES="$2"
            shift 2
            ;;
        -o|--output)
            OUTPUT_DIR="$2"
            shift 2
            ;;
        -u|--user-agent)
            USER_AGENT="$2"
            shift 2
            ;;
        -i|--images)
            DOWNLOAD_IMAGES=true
            shift
            ;;
        -h|--help)
            show_usage
            exit 0
            ;;
        -*)
            echo "Unknown option: $1"
            show_usage
            exit 1
            ;;
        *)
            if [ -z "$URL" ]; then
                URL="$1"
            fi
            shift
            ;;
    esac
done

# Main execution
if [ -z "$URL" ]; then
    echo "Error: URL is required"
    show_usage
    exit 1
fi

# Validate URL
if ! [[ $URL =~ ^https?:// ]]; then
    echo "Error: Invalid URL format. Must start with http:// or https://"
    exit 1
fi
```

```
# Check dependencies
if ! command -v curl >/dev/null 2>&1; then
    echo "Error: curl is required but not installed"
    exit 1
fi

# Start scraping
scrape_website "$URL"

# Download images if requested
if [ "$DOWNLOAD_IMAGES" = true ] && [ -f "$OUTPUT_DIR/all_images.txt" ]; then
    log "Downloading images..."
    download_images "$OUTPUT_DIR/all_images.txt"
fi

log "Web scraping completed successfully"
echo "Results saved in: $OUTPUT_DIR"
```

These automation examples demonstrate comprehensive solutions for common DevOps and system administration tasks. Each script includes error handling, logging, configuration management, and notification capabilities, making them suitable for production environments.

Utility Scripts

Utility Scripts

Utility Scripts

This section contains practical utility scripts for everyday tasks, system maintenance, and productivity enhancement.

File and Directory Utilities

Duplicate File Finder

```
#!/bin/bash
# find_duplicates.sh - Find and manage duplicate files

SEARCH_DIR="${1:-.}"
ACTION="${ACTION:-list}"
MIN_SIZE="${MIN_SIZE:-1}"
HASH_ALGORITHM="${HASH_ALGORITHM:-md5sum}"

# Temporary files
TEMP_DIR="/tmp/duplicate_finder.$$"
mkdir -p "$TEMP_DIR"

# Cleanup function
cleanup() {
    rm -rf "$TEMP_DIR"
}
trap cleanup EXIT

# Logging function
log() {
    echo "$(date '+%H:%M:%S') - $*"
}

# Calculate file hash
calculate_hash() {
    local file="$1"
    case "$HASH_ALGORITHM" in
        md5sum) md5sum "$file" | cut -d' ' -f1 ;;
        sha1sum) sha1sum "$file" | cut -d' ' -f1 ;;
        sha256sum) sha256sum "$file" | cut -d' ' -f1 ;;
        *) md5sum "$file" | cut -d' ' -f1 ;;
```

```
    esac
}

# Find files by size
find_by_size() {
    log "Finding files by size..."

    find "$SEARCH_DIR" -type f -size +${MIN_SIZE}c -exec stat -c "%s %n" {} \; | \
    sort -n > "$TEMP_DIR/files_by_size.txt"

    # Group files by size
    awk '{print $1}' "$TEMP_DIR/files_by_size.txt" | uniq -d > "$TEMP_DIR/duplicate_sizes.tx

    local duplicate_size_count=$(wc -l < "$TEMP_DIR/duplicate_sizes.txt")
    log "Found $duplicate_size_count different file sizes with potential duplicates"
}

# Find duplicates by hash
find_duplicates() {
    log "Calculating file hashes..."

    local processed=0
    local total=$(wc -l < "$TEMP_DIR/duplicate_sizes.txt")

    while IFS= read -r size; do
        ((processed++))
        echo -ne "\rProgress: $processed/$total"

        # Get all files of this size
        grep "^$size " "$TEMP_DIR/files_by_size.txt" | cut -d' ' -f2- | while IFS= read -r f
            if [ -f "$file" ]; then
                local hash=$(calculate_hash "$file")
                echo "$hash $size $file"
            fi
        done >> "$TEMP_DIR/files_with_hash.txt"
    done < "$TEMP_DIR/duplicate_sizes.txt"

    echo

    # Group by hash
    sort "$TEMP_DIR/files_with_hash.txt" > "$TEMP_DIR/sorted_hashes.txt"
    awk '{print $1}' "$TEMP_DIR/sorted_hashes.txt" | uniq -d > "$TEMP_DIR/duplicate_hashes.t

    local duplicate_hash_count=$(wc -l < "$TEMP_DIR/duplicate_hashes.txt")
    log "Found $duplicate_hash_count groups of duplicate files"
}
```

```
# List duplicate files
list_duplicates() {
    log "Listing duplicate files..."

    local group_num=0
    local total_duplicates=0
    local total_wasted_space=0

    while IFS= read -r hash; do
        ((group_num++))
        echo
        echo "Duplicate Group $group_num (Hash: $hash):"
        echo "========================================="

        local files=()
        local file_count=0
        local file_size=0

        grep "^$hash " "$TEMP_DIR/sorted_hashes.txt" | while IFS=' ' read -r h size file; do
            ((file_count++))
            file_size="$size"
            echo "$file_count. $file ($(numfmt --to=iec-i --suffix=B $size))"
            files+=("$file")
        done

        # Calculate wasted space (all but one file)
        local wasted=$((file_size * (file_count - 1)))
        total_wasted_space=$((total_wasted_space + wasted))
        total_duplicates=$((total_duplicates + file_count - 1))

        echo "Files in group: $file_count"
        echo "Wasted space: $(numfmt --to=iec-i --suffix=B $wasted)"

    done < "$TEMP_DIR/duplicate_hashes.txt"

    echo
    echo "Summary:"
    echo "========"
    echo "Duplicate groups: $group_num"
    echo "Total duplicate files: $total_duplicates"
    echo "Total wasted space: $(numfmt --to=iec-i --suffix=B $total_wasted_space)"
}

# Interactive duplicate removal
interactive_remove() {
    log "Starting interactive duplicate removal..."
```

```
    local group_num=0
    local removed_count=0
    local freed_space=0

    while IFS= read -r hash; do
        ((group_num++))
        echo
        echo "Duplicate Group $group_num (Hash: $hash):"
        echo "========================================"

        local files=()
        local file_count=0
        local file_size=0

        # Collect files in this group
        while IFS=' ' read -r h size file; do
            if [ "$h" = "$hash" ]; then
                ((file_count++))
                file_size="$size"
                files+=("$file")
                echo "$file_count. $file"
                echo "   Size: $(numfmt --to=iec-i --suffix=B $size)"
                echo "   Modified: $(stat -c %y "$file")"
            fi
        done < "$TEMP_DIR/sorted_hashes.txt"

        echo
        echo "Choose action:"
        echo "k) Keep all files"
        echo "1-$file_count) Keep only file number X (delete others)"
        echo "a) Auto-keep oldest file"
        echo "s) Skip this group"

        read -p "Your choice: " choice

        case "$choice" in
            k|K)
                echo "Keeping all files in this group"
                ;;
            [1-9]*)
                if [ "$choice" -ge 1 ] && [ "$choice" -le $file_count ]; then
                    local keep_index=$((choice - 1))
                    echo "Keeping: ${files[$keep_index]}"

                    for i in "${!files[@]}"; do
                        if [ $i -ne $keep_index ]; then
                            echo "Removing: ${files[$i]}"
```

```
                        if rm "${files[$i]}"; then
                            ((removed_count++))
                            freed_space=$((freed_space + file_size))
                        fi
                    fi
                done
            else
                echo "Invalid choice, skipping group"
            fi
            ;;
        a|A)
            # Keep oldest file
            local oldest_file=""
            local oldest_time=0

            for file in "${files[@]}"; do
                local mtime=$(stat -c %Y "$file")
                if [ $oldest_time -eq 0 ] || [ $mtime -lt $oldest_time ]; then
                    oldest_time=$mtime
                    oldest_file="$file"
                fi
            done

            echo "Auto-keeping oldest file: $oldest_file"

            for file in "${files[@]}"; do
                if [ "$file" != "$oldest_file" ]; then
                    echo "Removing: $file"
                    if rm "$file"; then
                        ((removed_count++))
                        freed_space=$((freed_space + file_size))
                    fi
                fi
            done
            ;;
        s|S)
            echo "Skipping this group"
            ;;
        *)
            echo "Invalid choice, skipping group"
            ;;
    esac

done < "$TEMP_DIR/duplicate_hashes.txt"

echo
```

```
    echo "Removal Summary:"
    echo "================"
    echo "Files removed: $removed_count"
    echo "Space freed: $(numfmt --to=iec-i --suffix=B $freed_space)"
}

# Auto-remove duplicates (keep oldest)
auto_remove() {
    log "Auto-removing duplicates (keeping oldest files)..."

    local removed_count=0
    local freed_space=0

    while IFS= read -r hash; do
        local files=()
        local file_size=0

        # Collect files in this group
        while IFS=' ' read -r h size file; do
            if [ "$h" = "$hash" ]; then
                files+=("$file")
                file_size="$size"
            fi
        done < "$TEMP_DIR/sorted_hashes.txt"

        # Find oldest file
        local oldest_file=""
        local oldest_time=0

        for file in "${files[@]}"; do
            local mtime=$(stat -c %Y "$file")
            if [ $oldest_time -eq 0 ] || [ $mtime -lt $oldest_time ]; then
                oldest_time=$mtime
                oldest_file="$file"
            fi
        done

        echo "Keeping oldest: $oldest_file"

        # Remove other files
        for file in "${files[@]}"; do
            if [ "$file" != "$oldest_file" ]; then
                echo "Removing: $file"
                if rm "$file"; then
                    ((removed_count++))
                    freed_space=$((freed_space + file_size))
                fi
```

```
            fi
        done

    done < "$TEMP_DIR/duplicate_hashes.txt"

    echo
    echo "Auto-removal Summary:"
    echo "====================="
    echo "Files removed: $removed_count"
    echo "Space freed: $(numfmt --to=iec-i --suffix=B $freed_space)"
}

# Generate report
generate_report() {
    local report_file="duplicate_report_$(date +%Y%m%d_%H%M%S).txt"

    {
        echo "Duplicate Files Report"
        echo "======================"
        echo "Generated: $(date)"
        echo "Search directory: $SEARCH_DIR"
        echo "Minimum file size: $MIN_SIZE bytes"
        echo "Hash algorithm: $HASH_ALGORITHM"
        echo

        list_duplicates

    } > "$report_file"

    log "Report saved: $report_file"
}

# Main execution
main() {
    if [ ! -d "$SEARCH_DIR" ]; then
        echo "Error: Directory '$SEARCH_DIR' does not exist"
        exit 1
    fi

    log "Starting duplicate file search in: $SEARCH_DIR"
    log "Minimum file size: $MIN_SIZE bytes"
    log "Hash algorithm: $HASH_ALGORITHM"

    find_by_size
    find_duplicates

    case "$ACTION" in
```

```
        list)
            list_duplicates
            ;;
        interactive)
            interactive_remove
            ;;
        auto)
            auto_remove
            ;;
        report)
            generate_report
            ;;
        *)
            echo "Unknown action: $ACTION"
            echo "Available actions: list, interactive, auto, report"
            exit 1
            ;;
    esac
}

# Show usage
show_usage() {
    cat << EOF
Duplicate File Finder - Find and manage duplicate files

Usage: $0 [directory] [options]

Options:
  -a, --action ACTION     Action to perform: list, interactive, auto, report
  -s, --min-size SIZE     Minimum file size in bytes (default: 1)
  -h, --hash ALGORITHM    Hash algorithm: md5sum, sha1sum, sha256sum
  --help                  Show this help

Environment Variables:
  ACTION                  Action to perform
  MIN_SIZE                Minimum file size
  HASH_ALGORITHM          Hash algorithm

Actions:
  list                    List duplicate files (default)
  interactive             Interactive duplicate removal
  auto                    Auto-remove duplicates (keep oldest)
  report                  Generate detailed report

Examples:
  $0 /home/user/Documents
  $0 /home/user/Pictures --action interactive
```

```
  ACTION=auto MIN_SIZE=1024 $0 /var/log
EOF
}

# Parse command line arguments
while [[ $# -gt 0 ]]; do
    case $1 in
        -a|--action)
            ACTION="$2"
            shift 2
            ;;
        -s|--min-size)
            MIN_SIZE="$2"
            shift 2
            ;;
        -h|--hash)
            HASH_ALGORITHM="$2"
            shift 2
            ;;
        --help)
            show_usage
            exit 0
            ;;
        -*)
            echo "Unknown option: $1"
            show_usage
            exit 1
            ;;
        *)
            if [ -z "$SEARCH_DIR" ] || [ "$SEARCH_DIR" = "." ]; then
                SEARCH_DIR="$1"
            fi
            shift
            ;;
    esac
done

main
```

Directory Synchronizer

```
#!/bin/bash
# sync_directories.sh - Synchronize directories with various options

SOURCE_DIR="$1"
```

```
TARGET_DIR="$2"
SYNC_MODE="${SYNC_MODE:-mirror}"
DRY_RUN="${DRY_RUN:-false}"
EXCLUDE_FILE="${EXCLUDE_FILE:-}"
LOG_FILE="${LOG_FILE:-sync_$(date +%Y%m%d_%H%M%S).log}"

# Sync statistics
declare -i files_copied=0
declare -i files_updated=0
declare -i files_deleted=0
declare -i dirs_created=0
declare -i bytes_transferred=0

# Logging function
log() {
    local level="$1"
    shift
    local message="$*"
    echo "$(date '+%Y-%m-%d %H:%M:%S') [$level] $message" | tee -a "$LOG_FILE"
}

# Check if file should be excluded
is_excluded() {
    local file="$1"

    if [ -n "$EXCLUDE_FILE" ] && [ -f "$EXCLUDE_FILE" ]; then
        while IFS= read -r pattern; do
            # Skip comments and empty lines
            [[ $pattern =~ ^[[:space:]]*# ]] && continue
            [[ -z $pattern ]] && continue

            if [[ $file =~ $pattern ]]; then
                return 0  # File is excluded
            fi
        done < "$EXCLUDE_FILE"
    fi

    return 1  # File is not excluded
}

# Copy file with verification
copy_file() {
    local src="$1"
    local dst="$2"
    local action="$3"  # "copy" or "update"

    if [ "$DRY_RUN" = "true" ]; then
```

```
        log INFO "DRY RUN: Would $action $src -> $dst"
        return 0
    fi

    # Create destination directory if needed
    local dst_dir=$(dirname "$dst")
    if [ ! -d "$dst_dir" ]; then
        if mkdir -p "$dst_dir"; then
            log INFO "Created directory: $dst_dir"
            ((dirs_created++))
        else
            log ERROR "Failed to create directory: $dst_dir"
            return 1
        fi
    fi

    # Copy file
    if cp -p "$src" "$dst"; then
        local file_size=$(stat -c%s "$src" 2>/dev/null || stat -f%z "$src" 2>/dev/null)
        bytes_transferred=$((bytes_transferred + file_size))

        if [ "$action" = "copy" ]; then
            log INFO "Copied: $src -> $dst ($(numfmt --to=iec-i --suffix=B $file_size))"
            ((files_copied++))
        else
            log INFO "Updated: $src -> $dst ($(numfmt --to=iec-i --suffix=B $file_size))"
            ((files_updated++))
        fi
        return 0
    else
        log ERROR "Failed to $action: $src -> $dst"
        return 1
    fi
}

# Delete file or directory
delete_item() {
    local item="$1"

    if [ "$DRY_RUN" = "true" ]; then
        log INFO "DRY RUN: Would delete $item"
        return 0
    fi

    if rm -rf "$item"; then
        log INFO "Deleted: $item"
        ((files_deleted++))
```

```
        return 0
    else
        log ERROR "Failed to delete: $item"
        return 1
    fi
}

# Compare files
files_different() {
    local src="$1"
    local dst="$2"

    # Check if destination exists
    [ ! -f "$dst" ] && return 0

    # Compare modification times
    local src_mtime=$(stat -c %Y "$src" 2>/dev/null || stat -f %m "$src" 2>/dev/null)
    local dst_mtime=$(stat -c %Y "$dst" 2>/dev/null || stat -f %m "$dst" 2>/dev/null)

    [ "$src_mtime" -gt "$dst_mtime" ] && return 0

    # Compare sizes
    local src_size=$(stat -c %s "$src" 2>/dev/null || stat -f %z "$src" 2>/dev/null)
    local dst_size=$(stat -c %s "$dst" 2>/dev/null || stat -f %z "$dst" 2>/dev/null)

    [ "$src_size" -ne "$dst_size" ] && return 0

    return 1  # Files are the same
}

# Mirror sync (make target identical to source)
sync_mirror() {
    log INFO "Starting mirror synchronization"
    log INFO "Source: $SOURCE_DIR"
    log INFO "Target: $TARGET_DIR"

    # First pass: copy/update files from source to target
    find "$SOURCE_DIR" -type f | while IFS= read -r src_file; do
        # Skip excluded files
        if is_excluded "$src_file"; then
            log DEBUG "Excluded: $src_file"
            continue
        fi

        # Calculate relative path
        local rel_path="${src_file#$SOURCE_DIR/}"
        local dst_file="$TARGET_DIR/$rel_path"
```

```
        if files_different "$src_file" "$dst_file"; then
            if [ -f "$dst_file" ]; then
                copy_file "$src_file" "$dst_file" "update"
            else
                copy_file "$src_file" "$dst_file" "copy"
            fi
        fi
    done

    # Second pass: remove files from target that don't exist in source
    if [ -d "$TARGET_DIR" ]; then
        find "$TARGET_DIR" -type f | while IFS= read -r dst_file; do
            local rel_path="${dst_file#$TARGET_DIR/}"
            local src_file="$SOURCE_DIR/$rel_path"

            if [ ! -f "$src_file" ]; then
                delete_item "$dst_file"
            fi
        done

        # Remove empty directories
        find "$TARGET_DIR" -type d -empty -delete 2>/dev/null || true
    fi
}

# One-way sync (only copy new/updated files)
sync_oneway() {
    log INFO "Starting one-way synchronization"
    log INFO "Source: $SOURCE_DIR"
    log INFO "Target: $TARGET_DIR"

    find "$SOURCE_DIR" -type f | while IFS= read -r src_file; do
        # Skip excluded files
        if is_excluded "$src_file"; then
            log DEBUG "Excluded: $src_file"
            continue
        fi

        # Calculate relative path
        local rel_path="${src_file#$SOURCE_DIR/}"
        local dst_file="$TARGET_DIR/$rel_path"

        if files_different "$src_file" "$dst_file"; then
            if [ -f "$dst_file" ]; then
                copy_file "$src_file" "$dst_file" "update"
            else
```

```
                copy_file "$src_file" "$dst_file" "copy"
            fi
        fi
    done
}

# Bidirectional sync (sync both ways based on modification time)
sync_bidirectional() {
    log INFO "Starting bidirectional synchronization"
    log INFO "Directory A: $SOURCE_DIR"
    log INFO "Directory B: $TARGET_DIR"

    # Create list of all files in both directories
    local all_files_file="/tmp/sync_all_files.$$"

    {
        find "$SOURCE_DIR" -type f | sed "s|^$SOURCE_DIR/||"
        find "$TARGET_DIR" -type f | sed "s|^$TARGET_DIR/||"
    } | sort -u > "$all_files_file"

    while IFS= read -r rel_path; do
        # Skip excluded files
        if is_excluded "$rel_path"; then
            log DEBUG "Excluded: $rel_path"
            continue
        fi

        local file_a="$SOURCE_DIR/$rel_path"
        local file_b="$TARGET_DIR/$rel_path"

        if [ -f "$file_a" ] && [ -f "$file_b" ]; then
            # Both files exist, sync newer to older
            local mtime_a=$(stat -c %Y "$file_a" 2>/dev/null || stat -f %m "$file_a" 2>/dev/
            local mtime_b=$(stat -c %Y "$file_b" 2>/dev/null || stat -f %m "$file_b" 2>/dev/

            if [ "$mtime_a" -gt "$mtime_b" ]; then
                copy_file "$file_a" "$file_b" "update"
            elif [ "$mtime_b" -gt "$mtime_a" ]; then
                copy_file "$file_b" "$file_a" "update"
            fi
        elif [ -f "$file_a" ]; then
            # File only exists in A, copy to B
            copy_file "$file_a" "$file_b" "copy"
        elif [ -f "$file_b" ]; then
            # File only exists in B, copy to A
            copy_file "$file_b" "$file_a" "copy"
        fi
```

```
    done < "$all_files_file"

    rm -f "$all_files_file"
}

# Generate sync report
generate_report() {
    local report_file="sync_report_$(date +%Y%m%d_%H%M%S).txt"

    {
        echo "Directory Synchronization Report"
        echo "================================"
        echo "Generated: $(date)"
        echo "Source: $SOURCE_DIR"
        echo "Target: $TARGET_DIR"
        echo "Sync Mode: $SYNC_MODE"
        echo "Dry Run: $DRY_RUN"
        echo

        echo "Statistics:"
        echo "==========="
        echo "Files copied: $files_copied"
        echo "Files updated: $files_updated"
        echo "Files deleted: $files_deleted"
        echo "Directories created: $dirs_created"
        echo "Bytes transferred: $(numfmt --to=iec-i --suffix=B $bytes_transferred)"
        echo

        echo "Log file: $LOG_FILE"

        if [ -f "$EXCLUDE_FILE" ]; then
            echo
            echo "Exclusion patterns:"
            echo "==================="
            cat "$EXCLUDE_FILE"
        fi

    } > "$report_file"

    log INFO "Report generated: $report_file"
}

# Validate directories
validate_directories() {
    if [ ! -d "$SOURCE_DIR" ]; then
        log ERROR "Source directory does not exist: $SOURCE_DIR"
        exit 1
```

```
    fi

    if [ "$SYNC_MODE" != "bidirectional" ] && [ ! -d "$TARGET_DIR" ]; then
        log INFO "Creating target directory: $TARGET_DIR"
        if ! mkdir -p "$TARGET_DIR"; then
            log ERROR "Failed to create target directory: $TARGET_DIR"
            exit 1
        fi
    fi

    # Check for circular sync (target inside source or vice versa)
    local source_real=$(realpath "$SOURCE_DIR")
    local target_real=$(realpath "$TARGET_DIR" 2>/dev/null || echo "$TARGET_DIR")

    if [[ "$target_real" == "$source_real"* ]] || [[ "$source_real" == "$target_real"* ]]; t
        log ERROR "Circular sync detected: target cannot be inside source or vice versa"
        exit 1
    fi
}

# Show progress
show_progress() {
    if [ "$DRY_RUN" != "true" ]; then
        while true; do
            echo -ne "\rFiles: $files_copied copied, $files_updated updated, $files_deleted
            sleep 1
        done &
        local progress_pid=$!

        # Kill progress display when script exits
        trap "kill $progress_pid 2>/dev/null" EXIT
    fi
}

# Main synchronization function
main() {
    log INFO "Starting directory synchronization"

    validate_directories

    if [ "$DRY_RUN" = "true" ]; then
        log INFO "DRY RUN MODE - No files will be modified"
    fi

    show_progress

    case "$SYNC_MODE" in
```

```
        mirror)
            sync_mirror
            ;;
        oneway)
            sync_oneway
            ;;
        bidirectional)
            sync_bidirectional
            ;;
        *)
            log ERROR "Unknown sync mode: $SYNC_MODE"
            exit 1
            ;;
    esac

    log INFO "Synchronization completed"

    echo
    echo "Synchronization Summary:"
    echo "========================"
    echo "Files copied: $files_copied"
    echo "Files updated: $files_updated"
    echo "Files deleted: $files_deleted"
    echo "Directories created: $dirs_created"
    echo "Bytes transferred: $(numfmt --to=iec-i --suffix=B $bytes_transferred)"

    generate_report
}

# Show usage
show_usage() {
    cat << EOF
Directory Synchronizer - Synchronize directories with various options

Usage: $0 <source_dir> <target_dir> [options]

Options:
  -m, --mode MODE         Sync mode: mirror, oneway, bidirectional
  -n, --dry-run           Show what would be done without making changes
  -e, --exclude FILE      File containing exclusion patterns
  -l, --log FILE          Log file path
  --help                  Show this help

Environment Variables:
  SYNC_MODE               Sync mode
  DRY_RUN                 Dry run mode (true/false)
  EXCLUDE_FILE            Exclusion patterns file
```

```
  LOG_FILE              Log file path

Sync Modes:
  mirror                Make target identical to source (default)
  oneway                Copy new/updated files from source to target
  bidirectional         Sync both ways based on modification time

Examples:
  $0 /home/user/docs /backup/docs
  $0 /home/user/docs /backup/docs --mode oneway --dry-run
  SYNC_MODE=bidirectional $0 /home/user/docs /shared/docs
EOF
}

# Parse command line arguments
while [[ $# -gt 0 ]]; do
    case $1 in
        -m|--mode)
            SYNC_MODE="$2"
            shift 2
            ;;
        -n|--dry-run)
            DRY_RUN="true"
            shift
            ;;
        -e|--exclude)
            EXCLUDE_FILE="$2"
            shift 2
            ;;
        -l|--log)
            LOG_FILE="$2"
            shift 2
            ;;
        --help)
            show_usage
            exit 0
            ;;
        -*)
            echo "Unknown option: $1"
            show_usage
            exit 1
            ;;
        *)
            if [ -z "$SOURCE_DIR" ]; then
                SOURCE_DIR="$1"
            elif [ -z "$TARGET_DIR" ]; then
                TARGET_DIR="$1"
```

```
            fi
            shift
            ;;
    esac
done

# Validate required arguments
if [ -z "$SOURCE_DIR" ] || [ -z "$TARGET_DIR" ]; then
    echo "Error: Both source and target directories are required"
    show_usage
    exit 1
fi

main
```

System Maintenance Utilities

Log Rotator

```
#!/bin/bash
# log_rotator.sh - Advanced log rotation utility

CONFIG_FILE="${CONFIG_FILE:-/etc/logrotate_custom.conf}"
LOG_DIR="${LOG_DIR:-/var/log}"
MAX_AGE_DAYS="${MAX_AGE_DAYS:-30}"
COMPRESS="${COMPRESS:-true}"
COMPRESSION_DELAY="${COMPRESSION_DELAY:-1}"

# Default configuration
declare -A default_config=(
    [rotate]="7"
    [size]="100M"
    [compress]="true"
    [delaycompress]="true"
    [missingok]="true"
    [notifempty]="true"
    [create]="644 root root"
)

# Logging function
log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $*" | tee -a "/var/log/log_rotator.log"
}
```

```
# Parse size string to bytes
parse_size() {
    local size_str="$1"
    local size_num=$(echo "$size_str" | sed 's/[^0-9]*//g')
    local size_unit=$(echo "$size_str" | sed 's/[0-9]*//g' | tr '[:lower:]' '[:upper:]')

    case "$size_unit" in
        K|KB) echo $((size_num * 1024)) ;;
        M|MB) echo $((size_num * 1024 * 1024)) ;;
        G|GB) echo $((size_num * 1024 * 1024 * 1024)) ;;
        *) echo "$size_num" ;;
    esac
}

# Check if file needs rotation
needs_rotation() {
    local log_file="$1"
    local max_size="$2"
    local rotate_count="$3"

    # Check if file exists and is not empty
    if [ ! -f "$log_file" ] || [ ! -s "$log_file" ]; then
        return 1
    fi

    # Check size
    local file_size=$(stat -c%s "$log_file" 2>/dev/null || stat -f%z "$log_file" 2>/dev/null
    local max_bytes=$(parse_size "$max_size")

    if [ "$file_size" -gt "$max_bytes" ]; then
        return 0
    fi

    # Check if we have reached max rotations
    if [ -f "${log_file}.${rotate_count}" ]; then
        return 0
    fi

    return 1
}

# Rotate a single log file
rotate_log() {
    local log_file="$1"
    local config_name="$2"

    # Load configuration for this log
```

```
    local rotate_count="${log_configs[$config_name.rotate]:-${default_config[rotate]}}"
    local max_size="${log_configs[$config_name.size]:-${default_config[size]}}"
    local compress="${log_configs[$config_name.compress]:-${default_config[compress]}}"
    local delaycompress="${log_configs[$config_name.delaycompress]:-${default_config[delayco
    local missingok="${log_configs[$config_name.missingok]:-${default_config[missingok]}}"
    local notifempty="${log_configs[$config_name.notifempty]:-${default_config[notifempty]}}
    local create_mode="${log_configs[$config_name.create]:-${default_config[create]}}"

    # Check if rotation is needed
    if ! needs_rotation "$log_file" "$max_size" "$rotate_count"; then
        log "No rotation needed for: $log_file"
        return 0
    fi

    log "Rotating log file: $log_file"

    # Remove oldest log if it exists
    if [ -f "${log_file}.${rotate_count}" ]; then
        rm "${log_file}.${rotate_count}"
        log "Removed oldest log: ${log_file}.${rotate_count}"
    fi

    # Rotate existing logs
    for ((i=rotate_count-1; i>=1; i--)); do
        local current_log="${log_file}.${i}"
        local next_log="${log_file}.$((i+1))"

        if [ -f "$current_log" ]; then
            mv "$current_log" "$next_log"
            log "Moved: $current_log -> $next_log"
        fi
    done

    # Move current log to .1
    if [ -f "$log_file" ]; then
        mv "$log_file" "${log_file}.1"
        log "Moved: $log_file -> ${log_file}.1"

        # Create new log file with proper permissions
        if [ -n "$create_mode" ]; then
            local mode=$(echo "$create_mode" | awk '{print $1}')
            local owner=$(echo "$create_mode" | awk '{print $2}')
            local group=$(echo "$create_mode" | awk '{print $3}')

            touch "$log_file"
            chmod "$mode" "$log_file" 2>/dev/null || true
```

```
            chown "$owner:$group" "$log_file" 2>/dev/null || true

            log "Created new log file: $log_file ($mode $owner:$group)"
        fi
    fi

    # Compress rotated logs
    if [ "$compress" = "true" ]; then
        local start_index=1
        if [ "$delaycompress" = "true" ]; then
            start_index=2
        fi

        for ((i=start_index; i<=rotate_count; i++)); do
            local log_to_compress="${log_file}.${i}"
            if [ -f "$log_to_compress" ] && [[ ! "$log_to_compress" =~ \.gz$ ]]; then
                if gzip "$log_to_compress"; then
                    log "Compressed: $log_to_compress"
                else
                    log "Failed to compress: $log_to_compress"
                fi
            fi
        done
    fi

    # Execute post-rotation script if configured
    local postrotate_script="${log_configs[$config_name.postrotate]:-}"
    if [ -n "$postrotate_script" ]; then
        log "Executing post-rotation script: $postrotate_script"
        if eval "$postrotate_script"; then
            log "Post-rotation script completed successfully"
        else
            log "Post-rotation script failed"
        fi
    fi
}

# Load configuration file
load_config() {
    declare -gA log_configs

    if [ ! -f "$CONFIG_FILE" ]; then
        log "Configuration file not found: $CONFIG_FILE"
        return 1
    fi

    local current_section=""
```

```
    while IFS= read -r line; do
        # Skip comments and empty lines
        [[ $line =~ ^[[:space:]]*# ]] && continue
        [[ -z $line ]] && continue

        # Check for section header
        if [[ $line =~ ^\[([^\]]+)\] ]]; then
            current_section="${BASH_REMATCH[1]}"
            continue
        fi

        # Parse key=value pairs
        if [[ $line =~ ^[[:space:]]*([^=]+)=(.*)$ ]]; then
            local key="${BASH_REMATCH[1]// /}"
            local value="${BASH_REMATCH[2]}"

            if [ -n "$current_section" ]; then
                log_configs["$current_section.$key"]="$value"
            else
                log_configs["$key"]="$value"
            fi
        fi
    done < "$CONFIG_FILE"

    log "Configuration loaded from: $CONFIG_FILE"
}

# Generate default configuration
generate_config() {
    cat > "$CONFIG_FILE" << EOF
# Log Rotation Configuration
# Generated on $(date)

# Global settings
compress=true
delaycompress=true
missingok=true
notifempty=true

# Apache logs
[apache]
files=/var/log/apache2/*.log
rotate=52
size=100M
create=644 www-data www-data
postrotate=systemctl reload apache2
```

```
# Nginx logs
[nginx]
files=/var/log/nginx/*.log
rotate=52
size=100M
create=644 www-data www-data
postrotate=systemctl reload nginx

# System logs
[syslog]
files=/var/log/syslog
rotate=7
size=50M
create=644 syslog adm

# Application logs
[application]
files=/var/log/myapp/*.log
rotate=30
size=10M
create=644 myapp myapp
postrotate=killall -USR1 myapp
EOF

    log "Configuration template created: $CONFIG_FILE"
}

# Clean old compressed logs
cleanup_old_logs() {
    log "Cleaning up logs older than $MAX_AGE_DAYS days"

    local deleted_count=0
    local freed_space=0

    find "$LOG_DIR" -name "*.gz" -mtime +$MAX_AGE_DAYS -type f | while read -r old_log; do
        local file_size=$(stat -c%s "$old_log" 2>/dev/null || stat -f%z "$old_log" 2>/dev/nu

        if rm "$old_log"; then
            log "Deleted old log: $old_log ($(numfmt --to=iec-i --suffix=B $file_size))"
            ((deleted_count++))
            freed_space=$((freed_space + file_size))
        fi
    done

    log "Cleanup completed: $deleted_count files deleted, $(numfmt --to=iec-i --suffix=B $fr
}
```

```
# Show log statistics
show_statistics() {
    echo "Log File Statistics"
    echo "==================="
    echo "Generated: $(date)"
    echo

    # Count log files by type
    echo "Log Files by Extension:"
    echo "======================="
    find "$LOG_DIR" -type f -name "*.log*" | sed 's/.*\.//' | sort | uniq -c | sort -nr

    echo
    echo "Largest Log Files:"
    echo "==================="
    find "$LOG_DIR" -type f -name "*.log*" -exec ls -lh {} \; | sort -k5 -hr | head -10

    echo
    echo "Disk Usage by Directory:"
    echo "========================"
    du -h "$LOG_DIR"/* 2>/dev/null | sort -hr | head -10

    echo
    echo "Compressed vs Uncompressed:"
    echo "==========================="
    local compressed_count=$(find "$LOG_DIR" -name "*.gz" -type f | wc -l)
    local uncompressed_count=$(find "$LOG_DIR" -name "*.log" -type f | wc -l)
    local compressed_size=$(find "$LOG_DIR" -name "*.gz" -type f -exec stat -c%s {} \; 2>/de
    local uncompressed_size=$(find "$LOG_DIR" -name "*.log" -type f -exec stat -c%s {} \; 2>

    echo "Compressed files: $compressed_count ($(numfmt --to=iec-i --suffix=B $compressed_si
    echo "Uncompressed files: $uncompressed_count ($(numfmt --to=iec-i --suffix=B $uncompres
}

# Main rotation process
main() {
    log "Starting log rotation process"

    if ! load_config; then
        log "Failed to load configuration, using defaults"
    fi

    # Get list of configured log sections
    local sections=()
    for key in "${!log_configs[@]}"; do
        if [[ $key =~ ^([^.]+)\.files$ ]]; then
```

```
            sections+=("${BASH_REMATCH[1]}")
        fi
    done

    # Remove duplicates
    IFS=$'\n' sections=($(sort -u <<<"${sections[*]}"))
    unset IFS

    if [ ${#sections[@]} -eq 0 ]; then
        log "No log sections configured, rotating all .log files in $LOG_DIR"

        # Rotate all .log files with default settings
        find "$LOG_DIR" -name "*.log" -type f | while read -r log_file; do
            rotate_log "$log_file" "default"
        done
    else
        # Rotate configured logs
        for section in "${sections[@]}"; do
            local files_pattern="${log_configs[$section.files]}"

            if [ -n "$files_pattern" ]; then
                log "Processing section: $section ($files_pattern)"

                # Expand file pattern
                for log_file in $files_pattern; do
                    if [ -f "$log_file" ]; then
                        rotate_log "$log_file" "$section"
                    fi
                done
            fi
        done
    fi

    # Cleanup old logs
    cleanup_old_logs

    log "Log rotation process completed"
}

# Show usage
show_usage() {
    cat << EOF
Log Rotator - Advanced log rotation utility

Usage: $0 [command] [options]

Commands:
```

```
  rotate                  Perform log rotation (default)
  config                  Generate configuration template
  stats                   Show log file statistics
  cleanup                 Clean up old compressed logs only

Options:
  -c, --config FILE       Configuration file path
  -d, --log-dir DIR       Log directory path
  -a, --max-age DAYS      Maximum age for log cleanup
  --help                  Show this help

Environment Variables:
  CONFIG_FILE             Configuration file path
  LOG_DIR                Log directory path
  MAX_AGE_DAYS           Maximum age for cleanup

Examples:
  $0                      # Rotate logs using default config
  $0 config               # Generate configuration template
  $0 stats                # Show log statistics
  $0 cleanup --max-age 7 # Clean logs older than 7 days
EOF
}

# Parse command line arguments
COMMAND="rotate"

while [[ $# -gt 0 ]]; do
    case $1 in
        rotate|config|stats|cleanup)
            COMMAND="$1"
            shift
            ;;
        -c|--config)
            CONFIG_FILE="$2"
            shift 2
            ;;
        -d|--log-dir)
            LOG_DIR="$2"
            shift 2
            ;;
        -a|--max-age)
            MAX_AGE_DAYS="$2"
            shift 2
            ;;
        --help)
```

```
            show_usage
            exit 0
            ;;
        -*)
            echo "Unknown option: $1"
            show_usage
            exit 1
            ;;
        *)
            shift
            ;;
    esac
done

# Execute command
case "$COMMAND" in
    rotate)
        main
        ;;
    config)
        generate_config
        ;;
    stats)
        show_statistics
        ;;
    cleanup)
        cleanup_old_logs
        ;;
    *)
        echo "Unknown command: $COMMAND"
        show_usage
        exit 1
        ;;
esac
```

These utility scripts provide comprehensive solutions for common file management and system maintenance tasks. Each script includes extensive configuration options, error handling, and detailed logging to make them suitable for production use.

Appendices

Appendix A: Bash Syntax Cheat Sheet

Appendix A: Bash Syntax Cheat Sheet

Quick reference covering: * Shebang & Strict Mode: `#!/usr/bin/env bash`, `set -euo pipefail` * Quoting: `'literal'`, `"expanded"`, `$'escape'` * Test Operators: `[[ -f file ]]`, `[[ -d dir ]]`, `[[ -z str ]]`, `[[ -n str ]]`, `(( a > b ))` * Parameter Expansions: `${var:-def}`, `${var:=def}`, `${var#prefix}`, `${var%suffix}` * Arrays: `arr=(a b)`, `"${arr[@]}"`, `${#arr[@]}` * Traps: `trap cleanup EXIT`

Appendix B: Common Production Patterns Library

Appendix B: Common Production Patterns Library

Production-tested templates: * **Standard Script Skeleton:** With strict mode, signal trap, CLI arg parsing, logging. * **Safe Atomic File Write:** Writing to `tempfile` and atomically moving `mv "$tempfile" "$dest"`. * **Parallel Worker Pool:** Bounded concurrency using pure bash. * **Service Wait Loop:** Health check with exponential backoff.

Appendix C: POSIX vs. Bash Feature Comparison Matrix

Appendix C: POSIX vs. Bash Feature Comparison Matrix

Exhaustive side-by-side syntax table comparing: * Core Language Features * Quoting and Expansion * Control Flow Constructs * Input/Output Redirection * Functions and Scope * Special Variables * Shell Options (`shopt` vs `set`)